

Thiết kế Ứng dụng Data-Intensive

Tư tưởng lớn đằng sau hệ thống dữ liệu tin cậy, khả mở và bảo trì được

Martin Kleppmann & Chris Riccomini

Mục lục

Lời nói đầu	9
Ai nên đọc cuốn sách này?	10
Có gì mới trong lần xuất bản thứ hai?	10
Tài liệu tham khảo và Đọc thêm	11
Quy ước sử dụng trong cuốn sách này	12
O'Reilly Online Learning.....	12
Cách Liên hệ với Chúng tôi	13
Lời cảm ơn.....	13
Chapter 1. Trade-Offs trong kiến trúc hệ thống dữ liệu	16
Hệ thống vận hành và hệ thống phân tích	19
Đặc trưng hóa Xử lý Giao dịch và Phân tích	20
Data Warehousing.....	22
Systems of Record và Derived Data	26
Cloud so với Self-Hosting	27
Ưu và nhược điểm của Cloud Services.....	28
Kiến trúc Hệ thống Cloud Native	30
Vận hành trong Kỷ nguyên Cloud	33
Hệ thống Phân tán so với Hệ thống Đơn nút	35
Các vấn đề với Distributed Systems	36
Microservices và Serverless	37
Cloud Computing So với Supercomputing	39
Hệ thống dữ liệu, Luật pháp và Xã hội	40
Tóm tắt.....	42
Chapter 2. Xác định các yêu cầu nonfunctional.....	49
Nghiên cứu điển hình: Bảng tin (Home Timelines) của mạng xã hội	50
Biểu diễn người dùng, bài đăng và quan hệ theo dõi	50
Materializing và Cập nhật Timelines	51
Mô tả Hiệu năng.....	53
Latency và Response Time	55
Trung bình, Trung vị và Percentile.....	56
Sử dụng các metric Thời gian Phản hồi	58
Độ tin cậy và Khả năng chịu lỗi	60
Khả năng chịu lỗi	61
Lỗi Phần cứng và Phần mềm.....	62
Con người và độ tin cậy.....	65
Khả năng mở rộng.....	67
Hiểu về Tải (Load)	68
Kiến trúc Shared-Memory, Shared-Disk và Shared-Nothing	69

Các nguyên tắc về khả năng mở rộng.....	70
Maintainability	71
Operability: Giúp công việc của bộ phận vận hành trở nên dễ dàng hơn	72
Sự đơn giản: Quản lý độ phức tạp	73
Evolvability: Giúp việc thay đổi trở nên dễ dàng	75
Tóm tắt.....	75
Chapter 3. Data Models và Query Languages	84
Mô hình Relational so với Mô hình Document.....	85
Sự không tương thích Object-Relational.....	87
Normalization, Denormalization, và Joins	91
Mối quan hệ Many-to-One và Many-to-Many	95
Stars và Snowflakes: Các lược đồ cho Analytics	97
Khi nào nên sử dụng mô hình nào.....	100
Các mô hình dữ liệu dạng graph	104
Property Graphs.....	107
Ngôn ngữ truy vấn Cypher	109
Truy vấn Đồ thị trong SQL.....	111
Triple Stores và SPARQL	114
Datalog: Truy vấn quan hệ đệ quy	118
GraphQL	122
Event Sourcing và CQRS	125
DataFrame, Matrix và Array.....	129
Tóm tắt.....	131
Chapter 4. Lưu trữ và Truy hồi	139
Lưu trữ và Đánh chỉ mục cho OLTP	140
Lưu trữ cấu trúc Log (Log-Structured Storage)	142
B-Trees	149
So sánh B-Trees và LSM-Trees	153
Index đa cột và Secondary Indexes	157
Lưu trữ các giá trị bên trong index.....	158
Lưu trữ mọi thứ trong bộ nhớ.....	158
Lưu trữ dữ liệu cho Analytics.....	160
Cloud Data Warehouses	160
Column-Oriented Storage.....	162
Query Execution: Compilation và Vectorization	168
Materialized Views và Data Cubes	169
Multidimensional và Full-Text Indexes.....	171
Full-Text Search.....	172
Vector Embeddings.....	173
Tóm tắt.....	176
Chapter 5. Encoding và Evolution	186

Các định dạng để encoding dữ liệu	188
Các định dạng đặc thù theo ngôn ngữ.....	189
JSON, XML và các biến thể nhị phân	190
Protocol Buffers	195
Avro.....	197
Ưu điểm của Schema	203
Các chế độ Dataflow	204
Dataflow thông qua các cơ sở dữ liệu.....	205
Luồng dữ liệu qua các dịch vụ: REST và RPC	206
Durable Execution và Workflows	214
Kiến trúc Event-Driven.....	217
Tóm tắt.....	219
Chapter 6. Replication	225
Single-Leader Replication.....	226
Nhấn bản dữ liệu đồng bộ so với bất đồng bộ.....	228
Thiết lập các Follower mới	229
Xử lý sự cố Node	232
Triển khai Replication Logs	235
Các vấn đề với replication lag	238
Giải pháp cho Replication Lag.....	244
Multi-Leader Replication	245
Vận hành phân tán về mặt địa lý.....	246
Sync Engines và Local-First Software.....	251
Giải quyết các xung đột ghi (Conflicting Writes)	253
Leaderless Replication	260
Ghi vào Cơ sở dữ liệu Khi một node Đang ngoại tuyến	261
Hiệu năng của Single-Leader so với Leaderless Replication	267
Vận hành Đa vùng (Multi-Region Operation).....	269
Phát hiện các ghi đồng thời (concurrent writes).....	270
Tóm tắt.....	276
Chapter 7. Sharding.....	283
Ưu và nhược điểm của Sharding	285
Sharding for Multitenancy	287
Sharding dữ liệu Key-Value	288
Sharding theo Key Range	289
Sharding bằng Hash của Key.....	291
Workload bị lệch và giảm thiểu Hot Spots.....	297
Vận hành: Rebalancing Tự động so với Thủ công.....	298
Request Routing	299
Sharding và Secondary Indexes	302
Local Secondary Indexes	302
Global Secondary Indexes.....	304

Tóm tắt.....	305
Chapter 8. Transactions.....	311
Chính xác thì một Transaction là gì?	312
Ý nghĩa của ACID	313
Các thao tác trên một đối tượng và nhiều đối tượng	319
Các Isolation Level yếu	324
Read Committed	325
Snapshot Isolation và Repeatable Read	329
Ngăn chặn lost update.....	335
Write Skew và Phantoms	340
Serializability	345
Actual Serial Execution.....	346
Two-Phase Locking.....	351
Serializable Snapshot Isolation	356
Distributed Transactions.....	362
Two-Phase Commit.....	364
Distributed Transactions giữa các hệ thống khác nhau.....	368
Giao dịch phân tán nội bộ cơ sở dữ liệu	373
Xem xét lại việc xử lý message exactly-once.....	374
Tóm tắt.....	375
Chapter 9. Những rắc rối với distributed systems.....	385
Faults và Partial Failures	386
Network không tin cậy	387
Những hạn chế của TCP.....	389
Lỗi mạng trong thực tế.....	390
Phát hiện lỗi (Fault Detection)	392
Timeout và các sự chậm trễ không giới hạn (Unbounded Delays).....	393
Mạng Đồng bộ so với Mạng Bất đồng bộ	396
Đồng hồ không tin cậy	399
Đồng hồ Monotonic so với Đồng hồ Time-of-Day	400
Đồng bộ hóa và Độ chính xác của Đồng hồ	402
Dựa vào các đồng hồ đã được đồng bộ hóa	404
Process Pauses	409
Kiến thức, Sự thật và Những lời nói dối.....	414
Đa số quyết định.....	415
Distributed Locks và Leases.....	416
Byzantine Faults	421
System Model và Thực tế	424
Formal Methods và Randomized Testing	429
Tóm tắt.....	434
Chapter 10. Consistency và Consensus	445

Linearizability	446
Điều gì làm nên một hệ thống có tính linearizability?	448
Dựa vào Linearizability.....	454
Triển khai các hệ thống linearizable	457
Cái giá của linearizability	459
ID Generators và Logical Clocks.....	463
Logical Clocks.....	466
Các bộ tạo ID có tính linearizability	469
Consensus.....	472
Nhiều khía cạnh của consensus.....	474
Consensus trong thực tế.....	481
Coordination Services.....	487
Tóm tắt.....	490
Chapter 11. Batch Processing	500
Batch Processing với các công cụ Unix	503
Phân tích log Đơn giản	503
Chuỗi lệnh So với Chương trình Tùy chỉnh	505
Sắp xếp so với Tổng hợp trong bộ nhớ (In-Memory Aggregation).....	506
Xử lý Batch trong các Hệ thống Phân tán	506
Filesystem Phân tán	507
Object Stores.....	509
Điều phối công việc phân tán (Distributed Job Orchestration).....	511
Các mô hình Batch Processing.....	516
MapReduce.....	516
Dataflow Engines.....	518
Shuffling dữ liệu.....	520
Joins và Grouping.....	522
Ngôn ngữ truy vấn	524
DataFrames	527
Các trường hợp sử dụng Batch	527
Extract–Transform–Load	528
Analytics.....	529
Machine Learning.....	530
Serving Dữ liệu phái sinh.....	531
Tóm tắt.....	533
Chapter 12. Stream Processing.....	539
Truyền tải Event Streams	540
Messaging Systems.....	541
Các Message Broker dựa trên log	547
Databases và Streams.....	553
Giữ các hệ thống đồng bộ	554
Change Data Capture.....	555

Trạng thái, Streams và Immutability	563
Xử lý các stream	568
Các ứng dụng của Stream Processing	569
Suy luận về Thời gian	575
Stream Joins	580
Khả năng chịu lỗi	584
Tóm tắt	587
Chapter 13. Triết lý về các Streaming Systems	596
Tích hợp dữ liệu	596
Kết hợp các công cụ chuyên dụng bằng cách tạo dữ liệu phái sinh (derived data)	597
Batch và Stream Processing	601
Unbundling Databases	604
Kết hợp các công nghệ lưu trữ dữ liệu	605
Thiết kế Ứng dụng xoay quanh Dataflow	609
Quan sát Trạng thái Phái sinh (Derived State)	614
Hướng tới tính chính xác	620
Lập luận End-to-End cho Database	621
Enforcing Constraints	626
Tính kịp thời và tính toàn vẹn	631
Tin tưởng, nhưng cần Kiểm chứng	636
Tóm tắt	640
Chapter 14. Làm điều đúng đắn	646
Phân tích dự báo	647
Định kiến và Phân biệt đối xử	647
Trách nhiệm và Trách nhiệm giải trình	648
Vòng lặp phản hồi (Feedback Loops)	650
Quyền riêng tư và Theo dõi	650
Giám sát	651
Sự đồng ý và Quyền tự do lựa chọn	653
Quyền riêng tư và việc Sử dụng Dữ liệu	654
Dữ liệu dưới dạng Tài sản và Quyền lực	656
Nhớ về Cách mạng Công nghiệp	657
Luật pháp và Tự điều tiết	658
Tóm tắt	660
Glossary	665

Lời nói đầu

Có hàng trăm cơ sở dữ liệu để lựa chọn. Bạn nên sử dụng loại nào cho ứng dụng của mình? Câu trả lời ngắn gọn là: "Còn tùy". Câu trả lời dài hơn nằm ở... cuốn sách này.

Các công nghệ khác nhau để lưu trữ và xử lý dữ liệu tạo ra những sự đánh đổi (trade-off) khác nhau, và không có một phương pháp nào là tốt nhất cho mọi tình huống. Hệ thống phù hợp hoàn hảo cho ứng dụng này có thể lại không phù hợp với ứng dụng khác. Cuốn sách này là một hướng dẫn về toàn cảnh các hệ thống dữ liệu, không chỉ nhìn vào một sản phẩm đơn lẻ mà còn so sánh điểm mạnh và điểm yếu của nhiều hệ thống.

Mặc dù bối cảnh các công nghệ xử lý và lưu trữ dữ liệu rất đa dạng và thay đổi nhanh chóng, nhưng các nguyên tắc nền tảng vẫn luôn tồn tại. Nếu bạn hiểu được những nguyên tắc đó, bạn sẽ có khả năng nhận biết mỗi công cụ phù hợp ở đâu, cách sử dụng chúng hiệu quả và cách tránh các cạm bẫy của chúng. Cuốn sách này tập trung vào những nguyên tắc đó.

Mặc dù cuốn sách này không phải là một tài liệu hướng dẫn sử dụng một công cụ cụ thể nào, nhưng nó cũng không phải là một cuốn sách giáo khoa đầy rẫy lý thuyết khô khan. Thay vào đó, chúng ta sẽ xem xét nhiều ví dụ về các hệ thống dữ liệu thành công: những công nghệ tạo nên nền tảng cho vô số ứng dụng phổ biến và phải đáp ứng các yêu cầu về khả năng mở rộng, hiệu năng và độ tin cậy trong production mỗi ngày. Chúng ta sẽ đi sâu vào bên trong (internals) của các hệ thống đó, bóc tách các thuật toán then chốt và thảo luận về những sự đánh đổi (trade-off) mà chúng đã thực hiện. Trong hành trình này, chúng ta sẽ cố gắng tìm ra những cách hữu ích để *suy nghĩ về các hệ thống dữ liệu*—không chỉ là *cách* chúng hoạt động, mà còn là *tại sao* chúng lại hoạt động theo cách đó.

Sau khi đọc cuốn sách này, bạn sẽ có vị thế tuyệt vời để xác định loại công nghệ nào phù hợp với mục đích nào và hiểu cách kết hợp các công cụ để tạo nên nền tảng cho một kiến trúc ứng dụng vững chắc. Bạn sẽ phát triển được trực giác mạnh mẽ về những gì hệ thống của bạn đang thực hiện bên trong (under the hood) để có thể suy luận về hành vi của chúng, đưa ra các quyết định thiết kế đúng đắn và truy tìm bất kỳ vấn đề nào có thể phát sinh.

Triết lý dẫn dắt của cuốn sách này là tập hợp một phạm vi rộng lớn các góc nhìn: cả về lý thuyết lẫn thực hành, cả những điều mới mẻ lẫn những điều kinh điển. Ngành công nghiệp máy tính có xu hướng bị thu hút bởi những thứ mới mẻ, hào nhoáng và coi thường những ý tưởng được cho là cũ kỹ hoặc mang tính học thuật. Đó là một sai lầm; nhiều ý tưởng nền tảng và mạnh mẽ trong ngành máy tính nảy sinh từ nghiên cứu, một số mới xuất hiện gần đây, một số đã có từ nhiều thập kỷ trước. Mặt khác, giới học thuật đôi khi thiếu đi ý niệm rõ ràng về những vấn đề quan trọng

trong thực tế. Cuốn sách này kết hợp những gì tốt nhất của cả hai: sự chính xác và chú trọng chi tiết của học thuật với sự tập trung vào tính thực tiễn của công nghiệp.

Ai nên đọc cuốn sách này?

Nếu bất kỳ điều nào sau đây đúng với bạn, bạn sẽ thấy cuốn sách này có giá trị:

- Bạn là một kỹ sư phần mềm, kiến trúc sư phần mềm hoặc quản lý kỹ thuật cần đưa ra các quyết định về kiến trúc của các hệ thống mà bạn đang làm việc—ví dụ, bạn cần chọn các công cụ để giải quyết một vấn đề nhất định và tìm ra cách áp dụng chúng tốt nhất. Điều này đặc biệt áp dụng cho các hệ thống backend.
- Bạn là một kỹ sư dữ liệu muốn hiểu bối cảnh rộng hơn của các hệ thống mà bạn đang làm việc, hoặc một kỹ sư cloud muốn hiểu sâu về nền tảng của các hệ thống mà bạn đang sử dụng. Bạn sẽ thấy rằng mặc dù các hệ thống phân tán hiện đại che giấu rất nhiều sự phức tạp đối với bạn, nhưng việc hiểu các nguyên tắc nền tảng của chúng là cực kỳ hữu ích cho việc tối ưu hóa hiệu năng và debugging.
- Bạn muốn học cách làm cho các hệ thống dữ liệu có khả năng mở rộng (ví dụ: để hỗ trợ các ứng dụng với hàng triệu người dùng), có tính sẵn sàng cao (giảm thiểu thời gian downtime), hoạt động ổn định và để bảo trì hơn trong dài hạn (ngay cả khi chúng phát triển và khi các yêu cầu cũng như công nghệ thay đổi).
- Bạn đang chuẩn bị cho một buổi phỏng vấn công việc về "thiết kế hệ thống" (system design), nơi bạn sẽ được yêu cầu phác thảo kiến trúc cho một ứng dụng, và bạn cần học các nguyên tắc để có được các kiến trúc dữ liệu tốt.
- Bạn tò mò muốn tìm hiểu những gì diễn ra đằng sau hậu trường của các website và dịch vụ trực tuyến lớn, cũng như bên trong các cơ sở dữ liệu và hệ thống xử lý dữ liệu khác nhau—đặc biệt nếu bạn muốn đào sâu hơn thay vì chỉ dừng lại ở các thuật ngữ hào nhoáng để có được sự hiểu biết chính xác về mặt kỹ thuật đối với các công nghệ khác nhau và những đánh đổi (trade-off) của chúng.

Cuốn sách này giả định rằng bạn đã có một số kinh nghiệm xây dựng các ứng dụng dựa trên web và đã quen thuộc với các cơ sở dữ liệu quan hệ cùng SQL. Việc hiểu biết ở mức độ cao về các giao thức mạng phổ biến như TCP và HTTP cũng sẽ rất hữu ích. Lựa chọn ngôn ngữ lập trình hay framework của bạn không gây ảnh hưởng gì đến nội dung cuốn sách này.

Có gì mới trong lần xuất bản thứ hai?

Phiên bản thứ hai này có cùng mục tiêu và phạm vi như phiên bản đầu tiên của *Designing Data-Intensive Applications*, vốn được xuất bản vào năm 2017. Tuy nhiên, chúng tôi đã chỉnh sửa toàn bộ cuốn sách một cách kỹ lưỡng để phản ánh những

thay đổi về công nghệ đã diễn ra trong thập kỷ qua và để cải thiện sự rõ ràng của các lời giải thích.

Những thay đổi kỹ thuật lớn nhất ảnh hưởng đến cuốn sách này kể từ phiên bản đầu tiên là sự bùng nổ sự quan tâm dành cho AI và sự trỗi dậy của các kiến trúc hệ thống dữ liệu cloud native. Mặc dù cuốn sách này không chuyên về AI, chúng tôi đã bổ sung nội dung về các hệ thống dữ liệu hỗ trợ AI và machine learning, bao gồm các vector index (được dùng cho semantic search), DataFrames (được dùng cho các dataset huấn luyện), và các hệ thống batch processing để chuẩn bị lượng lớn dữ liệu huấn luyện. Các ý tưởng về cloud native, chẳng hạn như xây dựng các hệ thống dữ liệu trên nền tảng object stores thay vì các đĩa cục bộ, đã được lồng ghép xuyên suốt cuốn sách.

Chúng tôi cũng đã bổ sung các phần thảo luận về sync engines và phần mềm local-first, workflow engines và durable execution, các phương pháp hình thức (formal methods) và randomized testing, GraphQL, cùng nhiều công nghệ khác đáng để tìm hiểu. Chúng tôi cũng đưa vào một chút bối cảnh pháp lý bằng cách khám phá tác động của Quy định bảo vệ dữ liệu chung (GDPR) của EU và các luật liên quan. Chúng tôi cũng đã lược bỏ một vài thứ—ví dụ, vì MapReduce hiện nay phần lớn đã lỗi thời, chúng tôi đã viết lại chương về batch processing cho phù hợp, và chúng tôi cũng rất tiếc khi quyết định loại bỏ các bản đồ theo phong cách Tolkien.

Một vài phần thảo luận đã được cấu trúc lại, và số thứ tự các chương đã thay đổi. Một số chương chỉ cần chỉnh sửa nhẹ, trong khi những chương khác (chẳng hạn như Chương 10, về consistency và consensus) gần như được viết lại hoàn toàn để trở nên rõ ràng hơn. Nhìn chung, phiên bản thứ hai dài hơn phiên bản đầu khoảng 60 trang.

Tài liệu tham khảo và Đọc thêm

Hầu hết những gì chúng ta thảo luận trong cuốn sách này đã được trình bày ở những nơi khác dưới hình thức này hay hình thức khác—trong các bài thuyết trình tại hội thảo, các bài báo nghiên cứu, các bài viết trên blog, mã nguồn, các trình theo dõi lỗi (bug trackers), danh sách email, hoặc các kinh nghiệm kỹ thuật dân gian. Cuốn sách này tóm tắt những ý tưởng quan trọng nhất từ nhiều nguồn, và nó bao gồm các chỉ dẫn đến các tài liệu gốc xuyên suốt văn bản. Các tài liệu tham khảo ở cuối mỗi chương là nguồn tài nguyên tuyệt vời nếu bạn muốn khám phá sâu hơn một lĩnh vực nào đó, và hầu hết chúng đều có thể truy cập miễn phí trực tuyến.

Trong các phiên bản ebook, chúng tôi đã bao gồm các liên kết dẫn đến toàn văn của các tài nguyên trực tuyến. Vì các liên kết thường xuyên bị hỏng do đặc thù thực của web, chúng tôi cũng đã bao gồm các liên kết lưu trữ (archival links) nếu có thể. Nếu bạn gặp phải một liên kết bị hỏng, hoặc nếu bạn đang đọc bản in của cuốn sách này, bạn có thể sử dụng công cụ tìm kiếm để tra cứu các tài liệu tham khảo. Đối với các

bài báo học thuật, hãy tìm kiếm tiêu đề trên Google Scholar để tìm các tệp PDF truy cập mở (miễn phí). Sách có thể tốn phí, nhưng bạn không nên phải trả tiền cho các bài báo nghiên cứu.

Ngoài ra, bạn có thể tìm thấy tất cả các tài liệu tham khảo tại <https://github.com/ept/ddia2-references>, nơi chúng tôi duy trì các liên kết cập nhật nhất.

Quy ước sử dụng trong cuốn sách này

Các quy ước về trình bày sau đây được sử dụng trong cuốn sách này:

In nghiêng

Biểu thị các thuật ngữ mới, URL, địa chỉ email, tên tệp và phần mở rộng tệp.

Constant width

Được sử dụng cho các listing chương trình, cũng như trong các đoạn văn để tham chiếu đến các thành phần chương trình như tên biến hoặc tên hàm, cơ sở dữ liệu, kiểu dữ liệu, biến môi trường, câu lệnh và từ khóa.

Constant width bold

Hiển thị các lệnh hoặc văn bản khác mà người dùng nên nhập nguyên văn.

Constant width italic

Hiển thị văn bản cần được thay thế bằng các giá trị do người dùng cung cấp hoặc bằng các giá trị được xác định bởi ngữ cảnh.

LƯU Ý

Thành phần này biểu thị một lưu ý chung.

CẢNH BÁO

Thành phần này cho biết một cảnh báo hoặc sự thận trọng.

O'Reilly Online Learning

LƯU Ý

Trong hơn 40 năm qua, O'Reilly Media đã cung cấp việc huấn luyện về công nghệ và kinh doanh, cùng với kiến thức và sự hiểu biết sâu sắc để giúp các công ty thành công.

Mạng lưới độc đáo gồm các chuyên gia và nhà đổi mới của chúng tôi chia sẻ kiến thức và chuyên môn của họ thông qua sách, bài viết và nền tảng học trực tuyến của chúng tôi. Nền tảng học trực tuyến của O'Reilly cho phép bạn truy cập theo yêu cầu vào các khóa học huấn luyện trực tiếp, các lộ trình học tập chuyên sâu, môi trường lập trình tương tác, cùng một bộ sưu tập khổng lồ các văn bản và video từ O'Reilly

và hơn 200 nhà xuất bản khác. Để biết thêm thông tin, hãy truy cập <https://oreilly.com>.

Cách Liên hệ với Chúng tôi

Vui lòng gửi các ý kiến đóng góp và câu hỏi liên quan đến cuốn sách này tới nhà xuất bản:

- O'Reilly Media, Inc.
- 141 Stony Circle, Suite 195
- Santa Rosa, CA 95401
- 800-889-8969 (tại Hoa Kỳ hoặc Canada)
- 707-827-7019 (quốc tế hoặc địa phương)
- 707-829-0104 (fax)
- support@oreilly.com
- <https://oreilly.com/about/contact.html>

Chúng tôi có một trang web dành cho cuốn sách này, nơi chúng tôi liệt kê các lỗi (errata), ví dụ và bất kỳ thông tin bổ sung nào. Bạn có thể truy cập trang này tại <https://oreil.ly/DesigningDataIntensiveApps2>.

Để biết tin tức và thông tin về sách và các khóa học của chúng tôi, hãy truy cập <https://oreilly.com>.

Tìm chúng tôi trên LinkedIn: <https://linkedin.com/company/oreilly>.

Xem chúng tôi trên YouTube: <https://youtube.com/oreilymedia>.

Lời cảm ơn

Cuốn sách này tập hợp và hệ thống hóa kiến thức cũng như ý tưởng từ một số lượng lớn mọi người. Nó chứa khoảng một nghìn tham chiếu đến các bài báo, bài đăng blog, các bài diễn thuyết, tài liệu và nhiều thứ khác, và chúng tôi rất biết ơn các tác giả của những tài liệu này vì đã chia sẻ kiến thức của họ.

Đối với phiên bản thứ hai, Chris Riccomini đã cùng Martin Kleppmann làm đồng tác giả. Chúng tôi muốn cảm ơn tất cả những người đã cung cấp phản hồi và đóng góp giúp cải thiện cuốn sách: đặc biệt là David Booth, Mark Callaghan, Chuck Carman, Aniruddh Chaturvedi, William Dealtry, Arbel Deutsch Peled, Phil Eaton, Joy Gao, Johannes Hauser, Matthew Hertz, Erin R. Hoffman, Matt Housley, Karan Johar, Ling Mao, Pedram Navid, Mohit Palriwal, Alex Petrov, Alex Power, Joe Reis, Jack Vanlightly, Will Wilson, Jacky Zhao, và Zhengliang Zhu. Tất nhiên, chúng tôi chịu mọi trách nhiệm về bất kỳ lỗi nào còn sót lại hoặc những ý kiến không được tán thành trong cuốn sách này.

Trong khi viết phiên bản đầu tiên, Martin Kleppmann đã nhận được sự giúp đỡ từ rất nhiều người đã dành thời gian để thảo luận các ý tưởng hoặc kiên nhẫn giải thích mọi thứ cho ông. Ông muốn cảm ơn Joe Adler, Ross Anderson, Peter Bailis, Márton Balassi, Alastair Beresford, Mark Callaghan, Mat Clayton, Patrick Collison, Sean Cribbs, Shirshanka Das, Niklas Ekström, Stephan Ewen, Alan Fekete, Gyula Fóra, Camille Fournier, Andres Freund, John Garbutt, Seth Gilbert, Tom Haggett, Pat Helland, Joe Hellerstein, Jakob Homan, Heidi Howard, John Hugg, Julian Hyde, Conrad Irwin, Evan Jones, Flavio Junqueira, Jessica Kerr, Kyle Kingsbury, Jay Kreps, Carl Lerche, Nicolas Liochon, Steve Loughran, Lee Mallabone, Nathan Marz, Caitie McCaffrey, Josie McLellan, Christopher Meiklejohn, Ian Meyers, Neha Narkhede, Neha Narula, Cathy O’Neil, Onora O’Neill, Ludovic Orban, Zoran Perkov, Julia Powles, Chris Riccomini, Henry Robinson, David Rosenthal, Jennifer Rullmann, Matthew Sackman, Martin Scholl, Amit Sela, Gwen Shapira, Greg Spurrier, Sam Stokes, Ben Stopford, Tom Stuart, Diana Vasile, Rahul Vohra, Pete Warden, và Brett Wooldridge.

Một số người khác cũng đã đóng góp những phản hồi quý giá cho các bản thảo của lần xuất bản đầu tiên: cụ thể là Raul Agepati, Tyler Akidau, Mattias Andersson, Sasha Baranov, Veena Basavaraj, David Beyer, Jim Brikman, Paul Carey, Raul Castro Fernandez, Joseph Chow, Derek Elkins, Sam Elliott, Alexander Gallego, Mark Grover, Stu Hallows, Heidi Howard, Nicola Kleppmann, Stefan Kruppa, Bjorn Madsen, Sander Mak, Stefan Podkowinski, Phil Potter, Hamid Ramazani, Sam Stokes, và Ben Summers.

Martin Kleppmann cũng bày tỏ lòng biết ơn đối với sự hỗ trợ tài chính mà ông đã nhận được trong quá trình viết cuốn sách này. Khi thực hiện lần xuất bản đầu tiên, ông đã được LinkedIn dành thời gian có trả lương để thực hiện cuốn sách, và sau đó được hỗ trợ bởi một khoản tài trợ từ The Boeing Company. Trong quá trình viết lần xuất bản thứ hai, ông đã được hỗ trợ bởi Freigeist Fellowship từ Volkswagen Foundation, bởi những người ủng hộ qua hình thức gọi vốn cộng đồng bao gồm Ably, Mintter, Prisma, và SoftwareMill, cùng với sự tài trợ từ ScyllaDB.

Chúng tôi muốn gửi lời cảm ơn tới đội ngũ tại O’Reilly đã giúp hiện thực hóa cuốn sách này, cũng như các đồng nghiệp và gia đình vì đã kiên nhẫn với khoảng thời gian khổng lồ mà chúng tôi đã dành cho việc viết lách. Hy vọng bạn sẽ thấy điều này là xứng đáng.

Trade-Offs trong kiến trúc hệ thống dữ liệu

Không có giải pháp nào hoàn hảo; chỉ có những sự đánh đổi (trade-off). [...] Nhưng bạn cố gắng đạt được sự đánh đổi tốt nhất có thể, và đó là tất cả những gì bạn có thể hy vọng.

Thomas Sowell, phỏng vấn với Fred Barnes (2005)

Dữ liệu đóng vai trò trung tâm trong nhiều hoạt động phát triển ứng dụng ngày nay. Với các ứng dụng web và di động, phần mềm dưới dạng dịch vụ (SaaS) và các dịch vụ cloud, việc lưu trữ dữ liệu từ nhiều người dùng khác nhau trong một hạ tầng dữ liệu dùng chung dựa trên máy chủ đã trở nên bình thường. Dữ liệu từ hoạt động của người dùng, các giao dịch kinh doanh, thiết bị và cảm biến cần được lưu trữ và sẵn sàng để phân tích. Khi người dùng tương tác với một ứng dụng, họ vừa đọc dữ liệu đã được lưu trữ, vừa tạo ra thêm nhiều dữ liệu mới.

Lượng dữ liệu nhỏ, có thể được lưu trữ và xử lý trên một máy đơn lẻ, thường khá dễ xử lý. Tuy nhiên, khi khối lượng dữ liệu hoặc tốc độ truy vấn tăng lên, nó cần được phân tán trên nhiều máy, điều này dẫn đến nhiều thách thức. Khi nhu cầu của ứng dụng trở nên phức tạp hơn, việc lưu trữ mọi thứ trong một hệ thống duy nhất không còn đủ nữa, và có thể cần phải kết hợp nhiều hệ thống lưu trữ hoặc xử lý cung cấp các khả năng khác nhau.

Chúng ta gọi một ứng dụng là *data-intensive* (thâm dụng dữ liệu) nếu quản lý dữ liệu là một trong những thách thức chính trong việc phát triển ứng dụng đó [1]. Trong khi ở các hệ thống *compute-intensive* (thâm dụng tính toán), thách thức nằm ở việc song song hóa một phép tính rất lớn, thì trong các ứng dụng *data-intensive*, chúng ta thường lo lắng nhiều hơn về những việc như lưu trữ và xử lý khối lượng dữ liệu lớn, quản lý các thay đổi đối với dữ liệu, đảm bảo tính nhất quán trước các lỗi và tính đồng thời, cũng như đảm bảo các dịch vụ có độ sẵn sàng cao.

Các ứng dụng như vậy thường được xây dựng từ các khối xây dựng tiêu chuẩn cung cấp các tính năng cần thiết phổ biến. Ví dụ, nhiều ứng dụng cần thực hiện các việc sau:

- Lưu trữ dữ liệu để chúng, hoặc một ứng dụng khác, có thể tìm lại được sau đó (*databases*)
- Ghi nhớ kết quả của một thao tác tốn kém để tăng tốc độ đọc (*caches*)

- Cho phép người dùng tìm kiếm dữ liệu bằng từ khóa hoặc lọc dữ liệu theo nhiều cách khác nhau (*search indexes*)
- Xử lý các sự kiện và thay đổi dữ liệu ngay khi chúng vừa xảy ra (*stream processing*)
- Định kỳ xử lý một lượng lớn dữ liệu đã tích lũy (*batch processing*)

Khi xây dựng một ứng dụng, chúng ta thường lấy một vài hệ thống hoặc dịch vụ phần mềm, chẳng hạn như các database hoặc API, và gắn kết chúng lại với nhau bằng mã nguồn ứng dụng. Nếu bạn đang làm chính xác những gì mà các hệ thống dữ liệu được thiết kế để thực hiện, quá trình này có thể khá dễ dàng.

Tuy nhiên, khi ứng dụng của bạn trở nên tham vọng hơn, các thách thức sẽ nảy sinh. Có rất nhiều hệ thống database với các đặc điểm khác nhau, phù hợp cho các mục đích khác nhau—làm thế nào để bạn chọn lựa hệ thống nào để sử dụng? Có nhiều cách tiếp cận khác nhau để *cache*ing, nhiều cách để xây dựng *search indexes*, v.v.—làm thế nào để bạn suy luận về các *trade-off* của chúng? Bạn cần tìm ra công cụ và cách tiếp cận nào là phù hợp nhất cho nhiệm vụ đang thực hiện, và việc kết hợp các công cụ có thể trở nên khó khăn khi bạn cần làm điều gì đó mà một công cụ đơn lẻ không thể tự mình thực hiện được.

Cuốn sách này là một hướng dẫn giúp bạn đưa ra quyết định về việc nên sử dụng những công nghệ nào và cách kết hợp chúng như thế nào. Như bạn sẽ thấy, không có cách tiếp cận nào về cơ bản là tốt hơn những cách khác; mọi thứ đều có ưu và nhược điểm. Với cuốn sách này, bạn sẽ học cách đặt ra những câu hỏi đúng để đánh giá và so sánh các hệ thống dữ liệu, từ đó bạn có thể tìm ra cách tiếp cận phục vụ tốt nhất cho nhu cầu của ứng dụng cụ thể của mình.

Chúng ta sẽ bắt đầu hành trình bằng cách xem xét một số cách mà dữ liệu thường được sử dụng trong các tổ chức ngày nay. Nhiều ý tưởng ở đây có nguồn gốc từ *enterprise software* (tức là các nhu cầu phần mềm và thực hành kỹ thuật của các tổ chức lớn, chẳng hạn như các tập đoàn lớn và chính phủ), vì về mặt lịch sử, chỉ các tổ chức lớn mới có khối lượng dữ liệu khổng lồ đòi hỏi các giải pháp kỹ thuật phức tạp. Nếu khối lượng dữ liệu của bạn đủ nhỏ, bạn có thể chỉ cần lưu trữ nó trong một bảng tính! Tuy nhiên, gần đây việc các công ty nhỏ hơn và các startup quản lý khối lượng dữ liệu lớn và xây dựng các hệ thống data-intensive cũng đã trở nên phổ biến.

Một trong những thách thức chính đối với các hệ thống dữ liệu là những người khác nhau cần thực hiện những công việc rất khác nhau với dữ liệu. Nếu bạn đang làm việc tại một công ty, bạn và nhóm của mình sẽ có một tập hợp các ưu tiên riêng, trong khi một nhóm khác có thể có những mục tiêu hoàn toàn khác biệt, ngay cả khi bạn có thể đang làm việc với cùng một dataset! Hơn nữa, những mục tiêu đó có thể không được diễn đạt một cách rõ ràng, điều này có thể dẫn đến sự hiểu lầm và bất đồng về cách tiếp cận đúng đắn.

Để giúp bạn hiểu rõ các lựa chọn của mình, chương này sẽ so sánh một số khái niệm tương phản và khám phá các trade-off của chúng. Chúng ta sẽ xem xét các chủ đề sau:

- Sự khác biệt giữa các hệ thống operational và analytical (“Operational Versus Analytical Systems”)
- Ưu và nhược điểm của các dịch vụ cloud và các hệ thống tự lưu trữ (“Cloud Versus Self-Hosting”)
- Khi nào nên chuyển từ các hệ thống single-node sang các hệ thống distributed (“Distributed Versus Single-Node Systems”)
- Cân bằng giữa nhu cầu của doanh nghiệp và quyền lợi của người dùng (“Data Systems, Law, and Society”)

Chương này cũng định nghĩa các thuật ngữ mà bạn sẽ cần cho phần còn lại của cuốn sách.

Thuật ngữ: Frontends và Backends

Phần lớn những gì chúng ta sẽ thảo luận trong cuốn sách này liên quan đến *backend development*. Để giải thích thuật ngữ đó: đối với các ứng dụng web, mã phía client (chạy trong trình duyệt web) được gọi là *frontend*, và mã phía server xử lý các yêu cầu của người dùng được gọi là *backend*. Các ứng dụng di động cũng tương tự như frontend ở chỗ chúng cung cấp giao diện người dùng, thường giao tiếp qua internet với một backend phía server. Các frontend đôi khi quản lý dữ liệu cục bộ trên thiết bị của người dùng [2], nhưng những thách thức lớn nhất về hạ tầng dữ liệu thường nằm ở backend: một frontend chỉ cần xử lý dữ liệu của một người dùng, trong khi backend quản lý dữ liệu thay mặt cho *tất cả* người dùng.

Một backend service thường có thể truy cập được thông qua HTTP (hoặc đôi khi là WebSocket); nó thường bao gồm mã ứng dụng thực hiện đọc và ghi dữ liệu trong một hoặc nhiều cơ sở dữ liệu và đôi khi giao tiếp với các hệ thống dữ liệu bổ sung, chẳng hạn như các cache hoặc message queue (mà chúng ta có thể gọi chung là *hạ tầng dữ liệu*). Mã ứng dụng thường là *stateless* (tức là khi nó hoàn tất việc xử lý một HTTP request, nó sẽ quên mọi thứ về request đó), và bất kỳ thông tin nào cần được duy trì từ request này sang request khác đều cần được lưu trữ ở phía client hoặc trong hạ tầng dữ liệu phía server.

Hệ thống vận hành và hệ thống phân tích

Nếu bạn đang làm việc với các hệ thống dữ liệu trong một doanh nghiệp, bạn có khả năng sẽ gặp nhiều loại người khác nhau làm việc với dữ liệu. Loại đầu tiên là các *backend engineer* xây dựng các service xử lý các yêu cầu đọc và cập nhật dữ liệu; các service này thường phục vụ người dùng bên ngoài, trực tiếp hoặc gián tiếp thông qua các service khác (xem “Microservices and Serverless”). Đôi khi các service được dùng cho mục đích nội bộ bởi các bộ phận khác của tổ chức.

Ngoài các nhóm quản lý backend services, hai nhóm người khác thường yêu cầu quyền truy cập vào dữ liệu của một tổ chức: *business analysts* (chuyên viên phân tích kinh doanh), những người tạo ra các báo cáo về hoạt động của tổ chức để giúp ban quản lý đưa ra các quyết định tốt hơn (*business intelligence*, hay BI), và *data scientists* (nhà khoa học dữ liệu), những người tìm kiếm các thông tin chuyên sâu mới mẻ trong dữ liệu hoặc tạo ra các feature sản phẩm hướng tới người dùng được hỗ trợ bởi phân tích dữ liệu và machine learning (ML)/AI (ví dụ: các đề xuất “người mua X cũng đã mua Y ” trên một website thương mại điện tử, phân tích dự đoán như chấm điểm rủi ro hoặc lọc spam, và xếp hạng kết quả tìm kiếm).

Mặc dù các business analysts và data scientists có xu hướng sử dụng các công cụ khác nhau và hoạt động theo những cách khác nhau, họ có một số thực hành chung. Thứ nhất, cả hai đều thực hiện *analytics* (phân tích), nghĩa là họ xem xét dữ liệu mà người dùng và các backend services đã tạo ra. Thứ hai, họ thường không sửa đổi dữ liệu này (ngoại trừ việc có thể sửa các lỗi), mặc dù họ có thể tạo ra các tập dữ liệu phái sinh (derived datasets) mà trong đó dữ liệu gốc đã được xử lý theo một cách nào đó.

Điều này dẫn đến sự phân tách giữa hai loại hệ thống—một sự phân biệt mà chúng ta sẽ sử dụng xuyên suốt cuốn sách này:

- *Operational systems* (hệ thống vận hành) bao gồm các backend services và hạ tầng dữ liệu nơi dữ liệu được tạo ra—ví dụ, bằng cách phục vụ người dùng bên ngoài. Tại đây, mã nguồn ứng dụng thực hiện cả việc đọc và sửa đổi dữ liệu trong các cơ sở dữ liệu của nó, dựa trên các hành động được thực hiện bởi người dùng.
- *Analytical systems* (hệ thống phân tích) phục vụ nhu cầu của các business analysts và data scientists. Chúng chứa một bản sao chỉ đọc của dữ liệu từ các operational systems, và chúng được tối ưu hóa cho các loại xử lý dữ liệu cần thiết cho analytics.

Như chúng ta sẽ thấy trong mục tiếp theo, các operational systems và analytical systems thường được giữ tách biệt vì những lý do chính đáng. Khi các hệ thống này đã trưởng thành, hai vai trò chuyên biệt mới đã xuất hiện: data engineers và analytics engineers. *Data engineers* là những người biết cách tích hợp các operational systems và analytical systems, đồng thời chịu trách nhiệm rộng hơn về hạ tầng dữ liệu của tổ chức [3]. *Analytics engineers* thực hiện mô hình hóa và biến đổi dữ liệu để làm cho

nó trở nên hữu ích hơn cho các business analysts và data scientists trong một tổ chức [4].

Nhiều kỹ sư chuyên về mảng vận hành (operational) hoặc mảng phân tích (analytical). Tuy nhiên, cuốn sách này bao hàm cả hệ thống dữ liệu operational và analytical, vì cả hai đều đóng vai trò quan trọng trong vòng đời dữ liệu trong một tổ chức. Chúng ta sẽ khám phá chuyên sâu về hạ tầng dữ liệu được sử dụng để cung cấp dịch vụ cho cả người dùng nội bộ và bên ngoài để bạn có thể làm việc tốt hơn với các đồng nghiệp ở phía bên kia của sự phân tách này.

Đặc trưng hóa Xử lý Giao dịch và Phân tích

Trong những ngày đầu của việc xử lý dữ liệu kinh doanh, một thao tác ghi vào cơ sở dữ liệu thường tương ứng với một giao dịch thương mại đang diễn ra: thực hiện một vụ bán hàng, đặt hàng với nhà cung cấp, trả lương cho nhân viên, v.v. Khi các cơ sở dữ liệu mở rộng sang các lĩnh vực không liên quan đến việc trao đổi tiền tệ, thuật ngữ *transaction* (giao dịch) vẫn được giữ lại, ám chỉ một nhóm các thao tác đọc và ghi tạo thành một đơn vị logic.

LƯU Ý

Chương 8 khám phá chi tiết những gì chúng ta hiểu về một transaction. Chương này sử dụng thuật ngữ này một cách linh hoạt để chỉ các thao tác đọc và ghi có độ trễ thấp.

Mặc dù các cơ sở dữ liệu bắt đầu được sử dụng cho nhiều loại dữ liệu—các bài đăng trên mạng xã hội, các nước đi trong một trò chơi, danh bạ trong một cuốn sổ địa chỉ, và nhiều, nhiều thứ khác nữa—mẫu truy cập cơ bản vẫn tương tự như việc xử lý các giao dịch kinh doanh. Một operational system thường tra cứu một số lượng nhỏ các bản ghi bằng một khóa (điều này được gọi là *point query*). Các bản ghi được chèn, cập nhật hoặc xóa dựa trên đầu vào của người dùng. Vì các ứng dụng này có tính tương tác, mẫu truy cập này trở nên được gọi là *online transaction processing* (OLTP).

Tuy nhiên, các cơ sở dữ liệu cũng bắt đầu được sử dụng ngày càng nhiều cho phân tích, vốn có các pattern truy cập rất khác so với OLTP. Thông thường, một truy vấn phân tích sẽ quét qua một lượng khổng lồ các bản ghi và tính toán các thống kê tổng hợp (chẳng hạn như count, sum, hoặc average) thay vì trả về các bản ghi riêng lẻ cho người dùng. Ví dụ, một nhà phân tích kinh doanh tại một chuỗi siêu thị có thể muốn trả lời các truy vấn phân tích như sau:

- Tổng doanh thu của mỗi cửa hàng của chúng ta trong tháng Một là bao nhiêu?
- Chúng ta đã bán được nhiều chuối hơn mức bình thường bao nhiêu trong đợt khuyến mãi vừa qua?
- Thương hiệu thức ăn dặm nào thường được mua cùng với tã giấy thương hiệu X nhất?

Các báo cáo có được từ những loại truy vấn này rất quan trọng đối với BI, giúp ban quản lý quyết định những việc cần làm tiếp theo. Để phân biệt pattern sử dụng cơ sở dữ liệu này với xử lý giao dịch (transaction processing), nó được gọi là *online analytical processing* (OLAP) [5]. Sự khác biệt giữa OLTP và phân tích không phải lúc nào cũng rạch ròi, nhưng một số đặc điểm điển hình được liệt kê trong Bảng 1-1.

Table 1-1. So sánh các đặc điểm của hệ thống *operational* và *analytical*

Property	Hệ thống vận hành (OLTP)	Hệ thống phân tích (OLAP)
Mô hình đọc chính	Point queries (truy vấn từng bản ghi riêng lẻ theo key)	Aggregate (tổng hợp) trên số lượng lớn các bản ghi
Mô hình ghi chính	Tạo, cập nhật và xóa các bản ghi riêng lẻ	Bulk import (ETL) hoặc event stream
Ví dụ người dùng là con người	Người dùng cuối của ứng dụng web/mobile	Nhà phân tích nội bộ, để hỗ trợ ra quyết định
Ví dụ sử dụng bởi máy móc	Kiểm tra xem một hành động có được cấp phép hay không	Phát hiện các mô hình gian lận/lạm dụng
Loại truy vấn	Cố định, được định nghĩa trước bởi ứng dụng	Tùy ý, khám phá ad-hoc bởi các nhà phân tích
Khối lượng truy vấn	Nhiều truy vấn nhỏ	Ít truy vấn, mỗi truy vấn đều phức tạp
Dữ liệu đại diện cho	Trạng thái mới nhất của dữ liệu (tại thời điểm hiện tại)	Lịch sử của các sự kiện đã xảy ra theo thời gian
Kích thước dataset	Gigabytes đến terabytes	Terabytes đến petabytes

LƯU Ý

Ý nghĩa của từ *online* trong *OLAP* là không rõ ràng; nó có lẽ ám chỉ rằng các truy vấn không chỉ dành cho các báo cáo đã được định nghĩa trước, mà các nhà phân tích còn sử dụng hệ thống OLAP một cách tương tác để thực hiện các truy vấn khám phá.

Với các hệ thống vận hành (operational systems), người dùng thường không được phép xây dựng các truy vấn SQL tùy chỉnh và chạy chúng trên cơ sở dữ liệu, vì điều đó có khả năng cho phép họ đọc hoặc sửa đổi dữ liệu mà họ không có quyền truy cập. Họ cũng có thể viết các truy vấn tốn kém để thực thi và do đó ảnh hưởng đến hiệu năng cơ sở dữ liệu đối với những người dùng khác. Vì những lý do này, các hệ thống OLTP chủ yếu chạy các tập hợp truy vấn cố định đã được tích hợp sẵn vào mã nguồn ứng dụng, với các truy vấn tùy chỉnh dùng một lần chỉ được sử dụng thỉnh thoảng để bảo trì hoặc xử lý sự cố. Ngược lại, các cơ sở dữ liệu phân tích thường cho phép người dùng tự do viết các truy vấn SQL tùy ý bằng tay, hoặc tạo ra các truy vấn một cách tự động bằng cách sử dụng một công cụ trực quan hóa dữ liệu hoặc dashboard như Tableau, Looker, hoặc Microsoft Power BI.

Một loại hệ thống khác được thiết kế cho các khối lượng công việc phân tích (các truy vấn tổng hợp trên nhiều bản ghi) nhưng được nhúng vào các sản phẩm hướng tới người dùng. Các hệ thống được thiết kế cho loại sử dụng này, được gọi là *product analytics* hoặc *real-time analytics*, bao gồm Pinot, Druid, và ClickHouse [6]. Các hệ thống như vậy nạp dữ liệu trong thời gian thực và được tối ưu hóa để phản hồi truy vấn với độ trễ (latency) thấp. Ngược lại, các hệ thống OLAP truyền thống thường nạp dữ liệu theo batch và được tối ưu hóa để xử lý truy vấn với throughput cao.

Data Warehousing

Ban đầu, cùng một cơ sở dữ liệu được sử dụng cho cả xử lý giao dịch và truy vấn phân tích. SQL hóa ra khá linh hoạt về mặt này; nó hoạt động tốt cho cả hai loại truy vấn. Tuy nhiên, vào cuối những năm 1980 và đầu những năm 1990, một xu hướng đã nảy sinh là các công ty ngừng sử dụng các hệ thống OLTP của họ cho mục đích phân tích và thay vào đó chạy phân tích trên một hệ thống cơ sở dữ liệu riêng biệt. Cơ sở dữ liệu riêng biệt này được gọi là một *data warehouse*.

Một doanh nghiệp lớn có thể có hàng chục, thậm chí hàng trăm hệ thống OLTP: các hệ thống vận hành website hướng tới khách hàng, kiểm soát các hệ thống điểm bán hàng (point-of-sale) tại các cửa hàng vật lý, theo dõi hàng tồn kho trong kho hàng, lập kế hoạch lộ trình cho phương tiện, quản lý nhà cung cấp, quản lý nhân viên và thực hiện nhiều tác vụ khác. Mỗi hệ thống này đều phức tạp và cần một đội ngũ nhân sự để bảo trì, vì vậy chúng cuối cùng hoạt động hầu như độc lập với nhau.

Thông thường, việc các nhà phân tích kinh doanh và nhà khoa học dữ liệu truy vấn trực tiếp các hệ thống OLTP này là không mong muốn, vì một số lý do sau:

- Dữ liệu cần quan tâm có thể bị phân tán trên nhiều hệ thống vận hành, khiến việc kết hợp các dataset đó trong một truy vấn duy nhất trở nên khó khăn (một vấn đề được gọi là *data silos*).
- Các loại schema và bố cục dữ liệu tốt cho OLTP thì ít phù hợp hơn cho phân tích (xem “Stars and Snowflakes: Schemas for Analytics”).
- Các truy vấn phân tích có thể rất tốn kém, và việc chạy chúng trên một cơ sở dữ liệu OLTP sẽ ảnh hưởng đến hiệu năng đối với những người dùng khác.
- Các hệ thống OLTP có thể nằm trong một mạng riêng biệt mà người dùng không được phép truy cập trực tiếp vì lý do bảo mật hoặc tuân thủ.

Ngược lại, một *data warehouse* là một cơ sở dữ liệu riêng biệt mà các nhà phân tích có thể truy vấn thoải mái mà không ảnh hưởng đến các hoạt động OLTP [7]. Như chúng ta sẽ thấy trong Chương 4, các data warehouse thường lưu trữ dữ liệu rất khác so với các cơ sở dữ liệu OLTP, nhằm tối ưu hóa cho các loại truy vấn phổ biến trong phân tích.

Data warehouse chứa một bản sao chỉ đọc của dữ liệu từ tất cả các hệ thống OLTP khác nhau trong công ty. Dữ liệu được trích xuất từ các cơ sở dữ liệu OLTP (sử dụng phương pháp trích xuất dữ liệu định kỳ hoặc một luồng cập nhật liên tục), được chuyển đổi thành một schema thân thiện với việc phân tích, được làm sạch, và sau đó được nạp vào data warehouse. Quy trình đưa dữ liệu vào data warehouse này được gọi là *extract-transform-load* (ETL) và được minh họa trong Hình 1-1. Đôi khi thứ tự của các bước *transform* và *load* được hoán đổi (tức là, việc chuyển đổi được thực hiện trong data warehouse sau khi nạp dữ liệu), dẫn đến *ELT*.

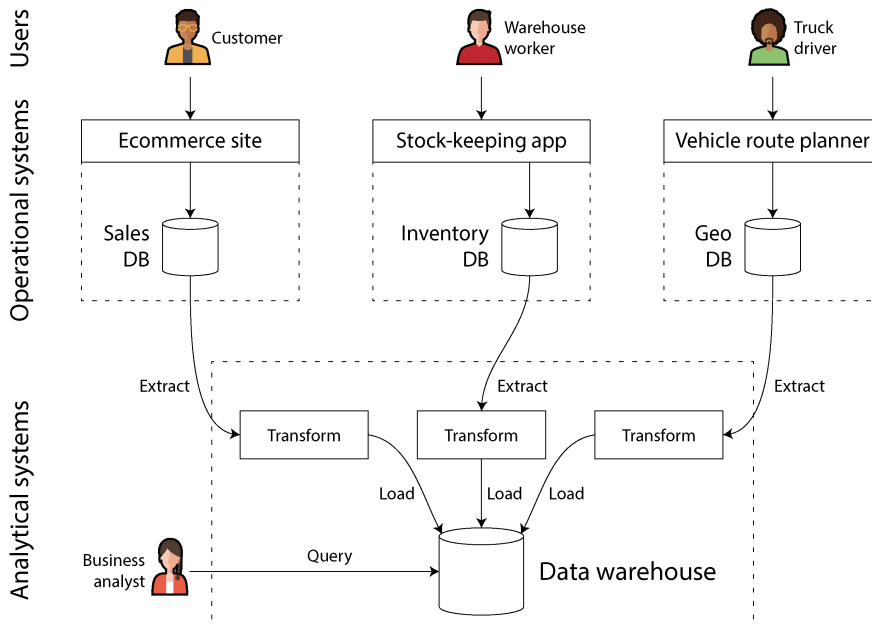


Figure 1-1. *Hình thức đơn giản về ETL vào một data warehouse*

Trong một số trường hợp, các nguồn dữ liệu của các quy trình ETL là các sản phẩm SaaS bên ngoài như quản lý quan hệ khách hàng (CRM), email marketing, hoặc các hệ thống xử lý thẻ tín dụng. Trong những trường hợp đó, bạn không có quyền truy cập trực tiếp vào cơ sở dữ liệu gốc, vì nó chỉ có thể truy cập được thông qua API của nhà cung cấp phần mềm. Việc đưa dữ liệu từ các hệ thống bên ngoài này vào data warehouse của riêng bạn có thể cho phép thực hiện các phân tích mà không thể thực hiện được thông qua SaaS API. ETL cho các SaaS API thường được triển khai bởi các dịch vụ kết nối dữ liệu chuyên dụng như Fivetran, Singer, hoặc Airbyte.

Một số hệ thống cơ sở dữ liệu cung cấp *hybrid transactional/analytical processing* (HTAP - xử lý giao dịch/phân tích hỗn hợp), nhằm mục đích cho phép cả OLTP và phân tích hoạt động trong một hệ thống duy nhất mà không yêu cầu ETL từ hệ thống này sang hệ thống khác [8, 9]. Tuy nhiên, nhiều hệ thống HTAP về mặt nội bộ bao gồm một hệ thống OLTP kết hợp với một hệ thống phân tích riêng biệt,

được ẩn sau một giao diện chung—vì vậy sự phân biệt giữa hai hệ thống này vẫn quan trọng để hiểu cách các hệ thống đó hoạt động.

Hơn nữa, mặc dù HTAP tồn tại, việc tách biệt giữa các hệ thống giao dịch và hệ thống phân tích là điều phổ biến do các mục tiêu và yêu cầu khác nhau của chúng. Cụ thể, việc mỗi hệ thống vận hành có cơ sở dữ liệu riêng được coi là một thực hành tốt (best practice) (xem “Microservices and Serverless”), dẫn đến khả năng có tới hàng trăm cơ sở dữ liệu vận hành riêng biệt; mặt khác, một doanh nghiệp thường chỉ có một data warehouse duy nhất, để các nhà phân tích kinh doanh có thể kết hợp dữ liệu từ nhiều hệ thống vận hành trong một truy vấn duy nhất.

Do đó, HTAP không thay thế các data warehouse. Thay vào đó, nó hữu ích khi cùng một ứng dụng cần vừa thực hiện các truy vấn phân tích quét một số lượng lớn các hàng, vừa đọc và cập nhật các bản ghi riêng lẻ với độ trễ thấp. Ví dụ, việc phát hiện gian lận có thể bao gồm các khối lượng công việc như vậy [10].

Sự tách biệt giữa các hệ thống vận hành và hệ thống phân tích là một phần của một xu hướng rộng lớn hơn. Khi khối lượng công việc trở nên khắt khe hơn, các hệ thống trở nên chuyên biệt hóa hơn và được tối ưu hóa cho các khối lượng công việc cụ thể. Các hệ thống đa năng có thể xử lý các khối lượng dữ liệu nhỏ một cách thoải mái, nhưng quy mô càng lớn, các hệ thống càng có xu hướng trở nên chuyên biệt hơn [11].

Từ data warehouse đến data lake

Một data warehouse thường sử dụng mô hình dữ liệu *relational* (quan hệ) được truy vấn thông qua SQL (xem Chương 3), có thể sử dụng phần mềm BI chuyên dụng. Mô hình này hoạt động tốt cho các loại truy vấn mà các nhà phân tích kinh doanh cần thực hiện, nhưng nó ít phù hợp hơn với nhu cầu của các nhà khoa học dữ liệu khi thực hiện các tác vụ như sau:

- Chuyển đổi dữ liệu sang một dạng phù hợp để huấn luyện một mô hình ML. Điều này thường đòi hỏi việc chuyển các hàng và cột của một bảng trong cơ sở dữ liệu thành một vector hoặc matrix các giá trị số được gọi là các *feature*. Quá trình thực hiện việc chuyển đổi này theo cách tối đa hóa hiệu năng của mô hình đã được huấn luyện được gọi là *feature engineering*, và nó thường yêu cầu mã tùy chỉnh (custom code) vốn khó có thể diễn đạt bằng SQL.
- Sử dụng các kỹ thuật xử lý ngôn ngữ tự nhiên (NLP) trên dữ liệu văn bản (ví dụ: các đánh giá về một sản phẩm) để cố gắng trích xuất thông tin có cấu trúc từ đó (ví dụ: cảm xúc của tác giả, hoặc những chủ đề mà chúng đề cập). Tương tự, các nhà khoa học dữ liệu có thể cần trích xuất thông tin có cấu trúc từ ảnh bằng cách sử dụng các kỹ thuật computer vision.

Mặc dù đã có những nỗ lực nhằm thêm các toán tử ML vào mô hình dữ liệu SQL [12] và xây dựng các hệ thống ML hiệu quả trên nền tảng relational [13], nhiều nhà

khoa học dữ liệu vẫn ưu tiên không làm việc trong một cơ sở dữ liệu relational như một data warehouse. Thay vào đó, nhiều người ưu tiên sử dụng các thư viện phân tích dữ liệu Python như Pandas và scikit-learn, các ngôn ngữ phân tích thống kê như R, và các framework phân tích phân tán như Spark [14]. Chúng ta sẽ thảo luận kỹ hơn về những thứ này trong mục “DataFrames, Matrices, and Arrays”.

Hệ quả là, các tổ chức đối mặt với nhu cầu cung cấp dữ liệu dưới một dạng thức phù hợp để các nhà khoa học dữ liệu có thể sử dụng. Câu trả lời chính là một *data lake* (hồ dữ liệu): một kho lưu trữ dữ liệu tập trung chứa bản sao của bất kỳ dữ liệu nào có thể hữu ích cho việc phân tích, được thu thập từ các hệ thống vận hành thông qua các quy trình ETL. Sự khác biệt so với một data warehouse là một data lake đơn giản chỉ chứa các tệp, mà không áp đặt bất kỳ định dạng tệp, mô hình dữ liệu hay schema nào cụ thể [15]. Các tệp trong một data lake có thể là tập hợp các bản ghi cơ sở dữ liệu, được mã hóa bằng một định dạng tệp như Avro hoặc Parquet (xem Chương 5), nhưng một data lake cũng có thể chứa văn bản, hình ảnh, video, dữ liệu đọc từ cảm biến, sparse matrices, feature vectors, trình tự bộ gen, hoặc bất kỳ loại dữ liệu nào khác [16]. Bên cạnh tính linh hoạt cao hơn, một data lake cũng thường rẻ hơn so với lưu trữ dữ liệu quan hệ, vì nó có thể sử dụng lưu trữ tệp thương mại như các object stores (xem “Kiến trúc hệ thống Cloud Native”).

Các quy trình ETL đã được khái quát hóa thành các *data pipeline* (luồng dữ liệu), và trong một số trường hợp, data lake đã trở thành một điểm dừng trung gian trên con đường từ các hệ thống vận hành đến data warehouse. Data lake chứa dữ liệu dưới dạng “thô” được tạo ra bởi các hệ thống vận hành, mà không qua quá trình chuyển đổi thành schema của một data warehouse quan hệ. Cách tiếp cận này có ưu điểm là mỗi bên tiêu thụ dữ liệu có thể chuyển đổi dữ liệu thô thành dạng thức phù hợp nhất với nhu cầu của họ. Nó đôi khi được gọi là *nguyên tắc sushi*: “dữ liệu thô thì tốt hơn” [17].

Vượt ra ngoài data lake

Khi các thực hành phân tích đã trở nên trưởng thành, các tổ chức ngày càng chú trọng hơn đến việc quản lý và vận hành các hệ thống phân tích cũng như các data pipeline, như đã được ghi lại, ví dụ, trong DataOps Manifesto [18]. Điều này được thúc đẩy một phần bởi các vấn đề về quản trị, quyền riêng tư và tuân thủ các quy định như Quy định chung về bảo vệ dữ liệu (GDPR) và Đạo luật quyền riêng tư của người tiêu dùng California (CCPA), những nội dung mà chúng ta sẽ thảo luận trong mục “Hệ thống dữ liệu, Luật pháp và Xã hội” và trong Chương 14.

Một yếu tố quan trọng khác là dữ liệu phục vụ phân tích ngày càng được cung cấp không chỉ dưới dạng các tệp và bảng quan hệ, mà còn dưới dạng các stream of events (dòng sự kiện) (xem Chương 12). Với phân tích dữ liệu dựa trên tệp, bạn có thể chạy lại phân tích định kỳ (ví dụ: hàng ngày) để phản hồi các thay đổi trong dữ liệu, nhưng stream processing cho phép các hệ thống phân tích phản hồi các sự kiện

nhANH hơn nhiều, ở mức tính bằng giây. Tùy thuộc vào ứng dụng và độ nhạy về thời gian của nó, một cách tiếp cận stream processing có thể rất giá trị, ví dụ, để xác định và ngăn chặn các hoạt động có khả năng gian lận hoặc lạm dụng.

Trong một số trường hợp, đầu ra của các hệ thống phân tích được cung cấp cho các hệ thống vận hành (một quy trình đôi khi được gọi là *reverse ETL* [19]). Ví dụ, một mô hình ML đã được huấn luyện trên dữ liệu trong một hệ thống phân tích có thể được triển khai lên production để nó có thể tạo ra các đề xuất cho người dùng cuối, chẳng hạn như "những người đã mua X cũng đã mua Y ." Các mô hình machine learning có thể được triển khai lên các hệ thống vận hành bằng cách sử dụng các công cụ chuyên dụng như TFX, Kubeflow, hoặc MLflow.

Systems of Record và Derived Data

Liên quan đến sự phân biệt giữa hệ thống vận hành và hệ thống phân tích, cuốn sách này cũng phân biệt giữa *systems of record* (hệ thống bản ghi gốc) và *derived data systems* (hệ thống dữ liệu phái sinh). Các thuật ngữ này hữu ích vì chúng giúp làm rõ luồng dữ liệu đi qua một hệ thống:

Systems of record

Một system of record, còn được gọi là *source of truth* (nguồn sự thật), nắm giữ phiên bản có thẩm quyền hoặc phiên bản *canonical* (chuẩn) của dữ liệu. Khi dữ liệu mới được đưa vào—ví dụ, dưới dạng đầu vào của người dùng—nó sẽ được ghi vào đây đầu tiên. Mỗi fact được đại diện chính xác một lần (sự đại diện này thường là *normalized*; xem “Normalization, Denormalization, and Joins”). Nếu có bất kỳ sự sai lệch nào giữa một hệ thống khác và system of record, thì giá trị trong system of record (theo định nghĩa) là giá trị chính xác.

Derived data systems

Dữ liệu trong một hệ thống phái sinh (derived system) là kết quả của việc lấy dữ liệu hiện có từ một hệ thống khác và biến đổi hoặc xử lý nó theo một cách nào đó. Nếu bạn làm mất dữ liệu phái sinh, bạn có thể tái tạo lại nó từ nguồn gốc ban đầu. Một ví dụ điển hình là cache: dữ liệu có thể được phục vụ từ cache nếu có sẵn, nhưng nếu cache không chứa những gì bạn cần, bạn có thể chuyển sang sử dụng cơ sở dữ liệu bên dưới. Các giá trị đã được denormalization, các index, materialized view, các representation dữ liệu đã được biến đổi, và các mô hình được huấn luyện trên một dataset cũng thuộc danh mục này.

Về mặt kỹ thuật, dữ liệu phái sinh là *dư thừa*, theo nghĩa là nó sao chép các thông tin hiện có. Tuy nhiên, dữ liệu này thường rất thiết yếu để đạt được hiệu năng tốt cho các truy vấn đọc. Bạn có thể phái sinh nhiều dataset từ một nguồn duy nhất, cho phép bạn xem xét dữ liệu từ các góc nhìn khác nhau.

Các hệ thống phân tích thường là các hệ thống dữ liệu phái sinh, bởi vì chúng là bên tiêu thụ dữ liệu được tạo ra từ nơi khác. Các dịch vụ vận hành có thể chứa sự kết hợp giữa các `system of record` và các hệ thống dữ liệu phái sinh. Các `system of record` là các cơ sở dữ liệu chính mà dữ liệu được ghi vào đầu tiên, trong khi các hệ thống dữ liệu phái sinh là các `index` và `cache` giúp tăng tốc các thao tác đọc phổ biến, đặc biệt là đối với các truy vấn mà `system of record` không thể trả lời một cách hiệu quả.

Hầu hết các cơ sở dữ liệu, `storage engine` và ngôn ngữ truy vấn không mặc định là `system of record` hay hệ thống phái sinh. Một cơ sở dữ liệu chỉ là một công cụ; việc bạn sử dụng nó như thế nào là tùy thuộc vào bạn. Sự phân biệt giữa một `system of record` và một hệ thống dữ liệu phái sinh không phụ thuộc vào công cụ, mà phụ thuộc vào cách bạn sử dụng nó trong ứng dụng của mình. Bằng cách làm rõ dữ liệu nào được phái sinh từ dữ liệu nào khác, bạn có thể mang lại sự rõ ràng cho một kiến trúc hệ thống vốn dĩ gây nhầm lẫn.

Khi dữ liệu trong một hệ thống được phái sinh từ dữ liệu trong một hệ thống khác, bạn cần một quy trình để cập nhật dữ liệu phái sinh khi dữ liệu gốc trong `system of record` thay đổi. Thật không may, nhiều cơ sở dữ liệu được thiết kế dựa trên giả định rằng ứng dụng của bạn sẽ luôn chỉ cần sử dụng duy nhất cơ sở dữ liệu đó, và chúng không tạo điều kiện thuận lợi để tích hợp nhiều hệ thống nhằm lan truyền các cập nhật như vậy. Trong Chương 11, chúng ta sẽ thảo luận về các `pipeline` dữ liệu như một cách tiếp cận cho việc *tích hợp dữ liệu* (*data integration*), cho phép chúng ta kết hợp nhiều hệ thống dữ liệu để đạt được những điều mà một hệ thống đơn lẻ không thể làm được.

Điều đó đưa chúng ta đến phần kết thúc của việc so sánh giữa phân tích và xử lý giao dịch. Trong mục tiếp theo, chúng ta sẽ xem xét một `trade-off` khác mà có thể bạn đã thấy tranh luận nhiều lần.

Cloud so với Self-Hosting

Với bất kỳ điều gì mà một tổ chức cần thực hiện, một trong những câu hỏi đầu tiên là liệu việc đó nên được thực hiện nội bộ hay thuê ngoài. Nghĩa là, bạn nên tự xây dựng hay nên mua ngoài?

Cuối cùng, đây là một câu hỏi về các ưu tiên kinh doanh. Một quy tắc ngón tay cái phổ biến là những thứ là năng lực cốt lõi hoặc lợi thế cạnh tranh của tổ chức bạn nên được thực hiện nội bộ, trong khi những thứ không cốt lõi, mang tính định kỳ hoặc phổ biến nên được giao cho một nhà cung cấp [20]. Để đưa ra một ví dụ cực đoan, hầu hết các công ty không tự chế tạo CPU của riêng mình, vì mua chúng từ các nhà sản xuất bán dẫn sẽ rẻ hơn.

Đối với phần mềm, hai quyết định quan trọng cần đưa ra là ai xây dựng phần mềm và ai triển khai nó. Các khả năng có thể xảy ra được minh họa trong Hình 1-2. Ở một cực là phần mềm bespoke (thiết kế riêng) mà bạn tự viết và chạy nội bộ; ở cực còn lại là các dịch vụ cloud hoặc sản phẩm SaaS được sử dụng rộng rãi, được triển khai và vận hành bởi một nhà cung cấp bên ngoài và bạn chỉ truy cập chúng thông qua một giao diện web hoặc API.

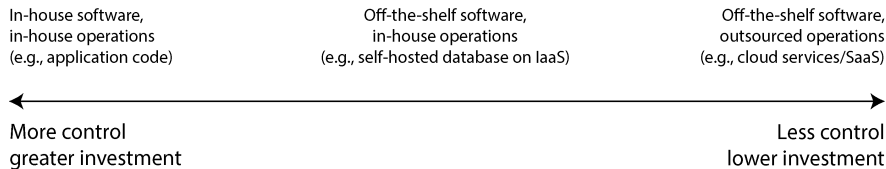


Figure 1-2. Phổ các quyết định về việc tự xây dựng hay mua ngoài phần mềm và các hoạt động vận hành của nó

Điểm trung gian là các phần mềm có sẵn (mã nguồn mở hoặc thương mại) mà bạn *tự host (self-host)*, hoặc tự mình triển khai—ví dụ, nếu bạn tải MySQL về và cài đặt nó trên một máy chủ mà bạn kiểm soát. Việc này có thể thực hiện trên phần cứng riêng của bạn (thường được gọi là *on premises* (tại chỗ), ngay cả khi máy chủ nằm trong một giá rack của trung tâm dữ liệu thuê và không thực sự nằm trong khuôn viên của bạn), hoặc trên một máy ảo (VM) trong cloud (*infrastructure as a service* (hạ tầng dưới dạng dịch vụ), hay IaaS). Có nhiều điểm khác nữa dọc theo phổ này, chẳng hạn như việc lấy phần mềm mã nguồn mở và chạy một phiên bản đã được sửa đổi của nó.

Một câu hỏi liên quan là bạn triển khai các service *như thế nào*, dù là trong cloud hay on premises—ví dụ, liệu bạn có sử dụng một orchestration framework như Kubernetes hay không. Tuy nhiên, việc lựa chọn công cụ triển khai nằm ngoài phạm vi của cuốn sách này, vì các yếu tố khác có ảnh hưởng lớn hơn đến kiến trúc của các hệ thống dữ liệu.

Ưu và nhược điểm của Cloud Services

Việc sử dụng một cloud service, thay vì tự mình chạy các phần mềm tương đương, về cơ bản là thuê ngoài việc vận hành phần mềm đó cho nhà cung cấp cloud. Có những lập luận hợp lý cho cả hai hướng tiếp cận này. Các nhà cung cấp cloud khẳng định rằng việc sử dụng dịch vụ của họ giúp bạn tiết kiệm thời gian, tiền bạc và cho phép bạn di chuyển nhanh hơn so với việc tự thiết lập hạ tầng của riêng mình.

Tuy nhiên, việc sử dụng một cloud service có thực sự rẻ hơn và dễ dàng hơn so với tự host hay không phụ thuộc rất nhiều vào kỹ năng của bạn và khối lượng công việc (workload) trên các hệ thống của bạn. Nếu bạn đã có kinh nghiệm thiết lập và vận hành các hệ thống mình cần, và nếu tải (load) của bạn khá dễ dự đoán (tức là số lượng máy bạn cần không biến động quá mạnh), thì việc tự mua máy móc riêng và tự chạy phần mềm trên đó thường sẽ rẻ hơn [21, 22].

Mặt khác, nếu bạn cần một hệ thống mà bạn chưa biết cách triển khai và vận hành, việc áp dụng một cloud service thường dễ dàng và nhanh chóng hơn là học cách quản lý hệ thống đó. Việc thuê và huấn luyện nhân viên chuyên biệt để bảo trì và vận hành hệ thống có thể rất tốn kém. Bạn vẫn cần một đội ngũ vận hành khi sử dụng cloud (xem “Operations in the Cloud Era”), nhưng việc thuê ngoài các tác vụ quản trị hệ thống cơ bản có thể giải phóng đội ngũ của bạn để tập trung vào các vấn đề ở cấp độ cao hơn.

Việc thuê ngoài vận hành một hệ thống cho một công ty chuyên về chạy hệ thống đó có khả năng mang lại dịch vụ tốt hơn, vì nhà cung cấp có được chuyên môn vận hành từ việc cung cấp dịch vụ cho nhiều khách hàng. Ngược lại, nếu bạn tự chạy dịch vụ, bạn có thể cấu hình và tinh chỉnh nó để đạt hiệu năng tốt trên khối lượng công việc cụ thể của mình. Một cloud service có lẽ sẽ không sẵn lòng thực hiện các tùy chỉnh như vậy thay cho bạn.

Các cloud service đặc biệt có giá trị nếu tải trên hệ thống của bạn thay đổi rất nhiều theo thời gian. Nếu bạn cấp phát (provision) các máy của mình để có thể xử lý tải đỉnh (peak load), nhưng các tài nguyên tính toán đó lại ở trạng thái rảnh rỗi (idle) hầu hết thời gian, thì hệ thống sẽ trở nên kém hiệu quả về chi phí. Trong tình huống này, các cloud service có lợi thế là chúng có thể giúp việc scale up hoặc scale down các tài nguyên tính toán của bạn trở nên dễ dàng hơn để đáp ứng với những thay đổi về nhu cầu.

Ví dụ, các hệ thống phân tích thường có tải cực kỳ biến động. Chạy một truy vấn phân tích lớn một cách nhanh chóng đòi hỏi rất nhiều tài nguyên tính toán chạy song song, nhưng khi truy vấn hoàn tất, các tài nguyên đó sẽ ở trạng thái rảnh rỗi cho đến khi người dùng thực hiện truy vấn tiếp theo. Các truy vấn được định nghĩa trước (ví dụ: cho các báo cáo hàng ngày) có thể được đưa vào hàng đợi và lập lịch để làm mượt tải, nhưng đối với các truy vấn tương tác, bạn muốn chúng hoàn tất càng nhanh thì khối lượng công việc càng trở nên biến động. Nếu dataset của bạn lớn đến mức việc truy vấn nhanh đòi hỏi tài nguyên tính toán đáng kể, việc sử dụng cloud có thể tiết kiệm tiền, vì bạn có thể trả lại các tài nguyên không sử dụng cho nhà cung cấp thay vì để chúng rảnh rỗi. Đối với các dataset nhỏ hơn, sự khác biệt này là không đáng kể.

Nhược điểm lớn nhất của một cloud service là bạn không có quyền kiểm soát nó:

- Nếu nó thiếu một feature mà bạn cần, tất cả những gì bạn có thể làm là lịch sự hỏi nhà cung cấp liệu họ có thêm nó hay không; bạn thường không thể tự mình triển khai nó.
- Nếu dịch vụ bị sập, tất cả những gì bạn có thể làm là chờ đợi nó phục hồi.
- Nếu bạn sử dụng dịch vụ theo cách kích hoạt một lỗi (bug) hoặc gây ra các vấn đề về hiệu năng, việc chẩn đoán vấn đề sẽ rất khó khăn. Với phần mềm mà bạn tự chạy, bạn có thể lấy các chỉ số hiệu năng và thông tin debugging từ hệ điều hành

để giúp bạn hiểu hành vi của nó, đồng thời bạn có thể xem các server logs. Với một dịch vụ được lưu trữ bởi nhà cung cấp, bạn thường không có quyền truy cập vào các thành phần bên trong này.

- Nếu dịch vụ ngừng hoạt động hoặc trở nên đắt đỏ đến mức không thể chấp nhận được, hoặc nếu nhà cung cấp thay đổi sản phẩm của họ theo cách bạn không thích, bạn sẽ hoàn toàn phụ thuộc vào họ; việc tiếp tục chạy một phiên bản cũ của phần mềm thường không phải là một lựa chọn, vì vậy bạn sẽ bị buộc phải di chuyển sang một dịch vụ thay thế [23]. Rủi ro này được giảm thiểu nếu các dịch vụ thay thế cung cấp một API tương thích, nhưng đối với nhiều dịch vụ cloud, không có các API tiêu chuẩn, điều này làm tăng chi phí chuyển đổi và khiến việc bị phụ thuộc vào nhà cung cấp (vendor lock-in) trở thành một vấn đề.
- Nếu nhà cung cấp cloud ở một quốc gia khác và nảy sinh xung đột chính trị giữa quốc gia đó với quốc gia của bạn, bạn có nguy cơ bị chặn quyền truy cập dịch vụ do các lệnh trừng phạt được áp đặt.
- Nhà cung cấp cloud cần được tin cậy để giữ cho dữ liệu được bảo mật, điều này có thể làm phức tạp quá trình tuân thủ các quy định về quyền riêng tư và bảo mật.

Bất chấp tất cả những rủi ro này, việc các tổ chức xây dựng các ứng dụng mới dựa trên các dịch vụ cloud, hoặc áp dụng phương pháp hybrid (lai) trong đó các dịch vụ cloud được sử dụng cho một số khía cạnh của hệ thống, ngày càng trở nên phổ biến. Tuy nhiên, các dịch vụ cloud sẽ không thay thế hoàn toàn tất cả các hệ thống dữ liệu nội bộ (in-house). Nhiều hệ thống cũ có trước thời đại cloud, và đối với bất kỳ dịch vụ nào có các yêu cầu chuyên biệt mà các dịch vụ cloud hiện có không thể đáp ứng, các hệ thống nội bộ vẫn là cần thiết. Ví dụ, các ứng dụng cực kỳ nhạy cảm với độ trễ (latency) như giao dịch tần suất cao yêu cầu quyền kiểm soát hoàn toàn đối với phần cứng.

Kiến trúc Hệ thống Cloud Native

Bên cạnh việc có một mô hình kinh tế khác biệt (đăng ký sử dụng một dịch vụ thay vì mua phần cứng và mua bản quyền phần mềm để chạy trên đó), sự trỗi dậy của cloud cũng có ảnh hưởng sâu sắc đến cách các hệ thống dữ liệu được triển khai ở cấp độ kỹ thuật. Thuật ngữ *cloud native* được dùng để mô tả một kiến trúc được thiết kế để tận dụng các dịch vụ cloud.

Về nguyên tắc, hầu như bất kỳ phần mềm nào mà bạn có thể tự lưu trữ (self-host) cũng có thể được cung cấp dưới dạng một dịch vụ cloud, và thực tế là các dịch vụ được quản lý (managed services) như vậy hiện đã có sẵn cho nhiều hệ thống dữ liệu phổ biến. Tuy nhiên, các hệ thống được thiết kế ngay từ đầu để trở thành cloud native đã cho thấy có một số lợi thế: hiệu năng tốt hơn trên cùng một phần cứng, phục hồi nhanh hơn sau các lỗi, có khả năng scale nhanh chóng các tài nguyên tính

toán để phù hợp với tải, và hỗ trợ các tập dữ liệu lớn hơn [24, 25, 26]. Bảng 1-2 liệt kê một số ví dụ về cả hai loại hệ thống này.

Table 1-2. Các ví dụ về hệ thống cơ sở dữ liệu self-hosted và cloud native

Category	Hệ thống tự lưu trữ (Self-hosted)	Hệ thống Cloud native
Operational/OLTP	MySQL, PostgreSQL, MongoDB	AWS Aurora [24], Azure SQL DB Hyperscale [25], Google Cloud Spanner
Analytical/OLAP	Teradata, ClickHouse, Spark	Snowflake [26], Google BigQuery, Azure Synapse Analytics

Phân lớp các dịch vụ cloud

Nhiều hệ thống dữ liệu tự lưu trữ (self-hosted) có yêu cầu hệ thống đơn giản; chúng chạy trên một hệ điều hành thông thường như Linux hoặc Windows, lưu trữ dữ liệu dưới dạng các tệp trên filesystem, và giao tiếp thông qua các giao thức mạng tiêu chuẩn như TCP/IP. Một vài hệ thống phụ thuộc vào phần cứng đặc biệt như GPU (dành cho ML) hoặc các giao diện mạng remote direct memory access (RDMA), nhưng nhìn chung, phần mềm tự lưu trữ có xu hướng sử dụng các tài nguyên tính toán phổ thông: CPU, RAM, một filesystem và một mạng IP.

Trong một cloud, loại phần mềm này có thể được chạy trong một môi trường IaaS, sử dụng một hoặc nhiều VM (hay còn gọi là *instances*) với một mức phân bổ nhất định về CPU, bộ nhớ, đĩa và băng thông mạng. So với các máy vật lý, các cloud instance có thể được cấp phát nhanh hơn và có sự đa dạng lớn hơn về kích thước, nhưng về cơ bản chúng tương tự như các máy tính truyền thống: bạn có thể chạy bất kỳ phần mềm nào mình muốn trên chúng, nhưng bạn phải tự chịu trách nhiệm quản trị chúng.

Ngược lại, ý tưởng then chốt của các dịch vụ cloud native không chỉ là sử dụng các tài nguyên tính toán do hệ điều hành của bạn quản lý, mà còn là xây dựng dựa trên các dịch vụ cloud cấp thấp hơn để tạo ra các dịch vụ cấp cao hơn. Ví dụ:

- Các dịch vụ object storage như Amazon S3, Azure Blob Storage và Cloudflare R2 dùng để lưu trữ các tệp lớn. Chúng cung cấp các API hạn chế hơn so với một filesystem thông thường (chỉ các thao tác đọc và ghi tệp cơ bản), nhưng chúng có lợi thế là che giấu các máy vật lý bên dưới; dịch vụ sẽ tự động phân phối dữ liệu trên nhiều máy để bạn không phải lo lắng về việc hết dung lượng đĩa trên bất kỳ máy đơn lẻ nào. Ngay cả khi một số máy hoặc đĩa của chúng bị lỗi hoàn toàn, dữ liệu vẫn không bị mất.
- Nhiều dịch vụ khác, đến lượt mình, lại được xây dựng dựa trên object storage và các dịch vụ cloud khác. Chẳng hạn, Snowflake là một cơ sở dữ liệu phân tích dựa

trên cloud (data warehouse) dựa vào S3 để lưu trữ dữ liệu [26], và một số dịch vụ khác lại tiếp tục xây dựng dựa trên Snowflake.

Giống như mọi sự trừu tượng hóa trong tính toán, không có một câu trả lời duy nhất đúng cho việc bạn nên sử dụng cái gì. Theo quy tắc chung, các sự trừu tượng hóa cấp cao hơn có xu hướng hướng tới các trường hợp sử dụng cụ thể hơn. Nếu nhu cầu của bạn khớp với các tình huống mà một hệ thống cấp cao được thiết kế để giải quyết, việc sử dụng hệ thống cấp cao có sẵn có lẽ sẽ đáp ứng nhu cầu của bạn với ít rắc rối hơn nhiều so với việc tự xây dựng nó từ các hệ thống cấp thấp hơn. Mặt khác, nếu không có hệ thống cấp cao nào đáp ứng được nhu cầu của bạn, thì tự xây dựng nó từ các thành phần cấp thấp hơn là lựa chọn duy nhất.

Tách rời lưu trữ và tính toán

Trong tính toán truyền thống, lưu trữ đĩa được coi là có tính bền vững (ta giả định rằng một khi thứ gì đó đã được ghi vào đĩa, nó sẽ không bị mất). Để chịu được lỗi của một ổ cứng riêng lẻ, RAID (redundant array of independent disks) thường được sử dụng để duy trì các bản sao dữ liệu trên nhiều đĩa được gắn vào cùng một máy. RAID có thể được triển khai bằng phần cứng hoặc bằng phần mềm bởi hệ điều hành, và nó hoàn toàn minh bạch đối với các ứng dụng đang truy cập filesystem.

Trong cloud, các compute instance (VM) cũng có thể được gắn các đĩa cục bộ, nhưng các hệ thống cloud native thường coi các đĩa này giống như một ephemeral cache hơn là một bộ lưu trữ dài hạn. Điều này là do đĩa cục bộ sẽ không thể truy cập được nếu instance liên quan bị lỗi, hoặc nếu instance đó được thay thế bằng một cái lớn hơn hoặc nhỏ hơn (trên một máy vật lý khác) để thích ứng với những thay đổi về tải.

Thay vì sử dụng các đĩa cục bộ, các dịch vụ cloud cũng cung cấp lưu trữ đĩa ảo có thể được tách rời khỏi một instance này và gắn vào một instance khác (ví dụ: Amazon EBS, Azure managed disks, và persistent disks trong Google Cloud). Một đĩa ảo như vậy không phải là một đĩa vật lý, mà là một dịch vụ cloud được cung cấp bởi một tập hợp các máy chủ riêng biệt nhằm mô phỏng hành vi của một đĩa (một *block device* (thiết bị khối), nơi mỗi block thường có kích thước 4 KiB). Công nghệ này giúp việc chạy các phần mềm dựa trên đĩa truyền thống trên cloud trở nên khả thi, nhưng việc mô phỏng block device sẽ tạo ra các overhead (chi phí xử lý bổ sung) mà có thể tránh được trong các hệ thống được thiết kế ngay từ đầu cho cloud [24]. Việc sử dụng đĩa ảo cũng khiến ứng dụng trở nên rất nhạy cảm với các lỗi mạng, vì mọi thao tác I/O trên thiết bị block ảo đều là một network call [27].

Để giải quyết vấn đề này, các dịch vụ cloud native thường tránh sử dụng đĩa ảo mà thay vào đó xây dựng dựa trên các dịch vụ lưu trữ chuyên dụng được tối ưu hóa cho các workload cụ thể. Các dịch vụ object storage như S3 được thiết kế để lưu trữ lâu dài các tệp khá lớn, có kích thước từ hàng trăm kilobyte đến vài gigabyte. Các hàng hoặc giá trị riêng lẻ được lưu trữ trong một database thường nhỏ hơn nhiều so với

mức này; do đó, các cloud database thường quản lý các giá trị nhỏ hơn trong một dịch vụ riêng biệt và lưu trữ các khối dữ liệu lớn hơn (chứa nhiều giá trị riêng lẻ) trong một object store [25, 28]. Chúng ta sẽ tìm hiểu các cách thực hiện việc này trong Chương 4.

Trong một kiến trúc hệ thống truyền thống, cùng một máy tính chịu trách nhiệm cho cả lưu trữ (đĩa) và tính toán (CPU và RAM), nhưng trong các hệ thống cloud native, hai trách nhiệm này đã trở nên tách rời (*disaggregated*) ở một mức độ nào đó [9, 26, 29, 30]: ví dụ, S3 chỉ lưu trữ các tệp, và nếu bạn muốn phân tích dữ liệu đó, bạn sẽ phải chạy mã phân tích ở đâu đó bên ngoài S3. Điều này đòi hỏi việc chuyển dữ liệu qua mạng, vấn đề mà chúng ta sẽ thảo luận kỹ hơn trong mục “Distributed Versus Single-Node Systems”.

Hơn nữa, các hệ thống cloud native thường là *multitenant* (đa người dùng), nghĩa là thay vì có một máy chủ riêng biệt cho mỗi khách hàng, dữ liệu và tính toán từ nhiều khách hàng được xử lý trên cùng một phần cứng dùng chung bởi cùng một dịch vụ [31]. Multitenancy có thể cho phép tận dụng phần cứng tốt hơn, khả năng mở rộng dễ dàng hơn và quản lý dễ dàng hơn bởi nhà cung cấp cloud, nhưng nó cũng đòi hỏi kỹ thuật cẩn thận để đảm bảo rằng hoạt động của một khách hàng không ảnh hưởng đến hiệu năng hoặc bảo mật của hệ thống đối với các khách hàng khác [32].

Vận hành trong Kỷ nguyên Cloud

Theo truyền thống, những người quản lý hạ tầng dữ liệu phía server của một tổ chức được gọi là *database administrators* (DBA - quản trị viên cơ sở dữ liệu) hoặc *system administrators* (sysadmin - quản trị viên hệ thống). Gần đây hơn, nhiều tổ chức đã cố gắng tích hợp các vai trò phát triển phần mềm và vận hành vào các nhóm có trách nhiệm chung cho cả backend services và hạ tầng dữ liệu; triết lý *DevOps* đã dẫn dắt xu hướng này. *Site reliability engineers* (SRE - kỹ sư độ tin cậy hệ thống) là cách Google triển khai ý tưởng này [33].

Vai trò của vận hành là đảm bảo các dịch vụ được cung cấp một cách tin cậy đến người dùng (bao gồm cấu hình hạ tầng và triển khai ứng dụng) và đảm bảo một môi trường production ổn định (bao gồm giám sát và chẩn đoán bất kỳ vấn đề nào có thể ảnh hưởng đến độ tin cậy). Đối với các hệ thống tự lưu trữ (self-hosted), vận hành theo truyền thống bao gồm một lượng lớn công việc ở cấp độ các máy chủ riêng lẻ, chẳng hạn như lập kế hoạch năng lực (ví dụ: giám sát dung lượng đĩa trống và thêm nhiều đĩa hơn trước khi bạn hết dung lượng), cấp phát các máy chủ mới, di chuyển các dịch vụ từ máy chủ này sang máy chủ khác, và cài đặt các bản vá hệ điều hành.

Nhiều dịch vụ cloud cung cấp một API giúp che giấu các máy đơn lẻ đang triển khai dịch vụ đó. Ví dụ, lưu trữ cloud thay thế các ổ đĩa có kích thước cố định bằng hình thức *metered billing* (tính phí theo mức sử dụng), nơi bạn có thể lưu trữ dữ liệu mà không cần lập kế hoạch nhu cầu dung lượng trước, và sau đó bạn sẽ bị tính phí dựa

trên không gian đã sử dụng. Hơn nữa, nhiều dịch vụ cloud vẫn duy trì độ sẵn sàng cao, ngay cả khi các máy đơn lẻ gặp lỗi (xem “Reliability and Fault Tolerance”).

Sự chuyển dịch trọng tâm từ các máy đơn lẻ sang các dịch vụ này đi kèm với sự thay đổi trong vai trò của operations. Mục tiêu cấp cao là cung cấp một dịch vụ tin cậy vẫn không đổi, nhưng các quy trình và công cụ đã tiến hóa.

Triết lý DevOps/SRE nhấn mạnh nhiều hơn vào các yếu tố sau:

- Thiết lập automation, ưu tiên các quy trình có khả năng lặp lại thay vì các tác vụ thủ công nhất thời
- Sử dụng các VM và dịch vụ tạm thời (ephemeral) thay vì các máy chủ chạy lâu dài
- Cho phép cập nhật ứng dụng thường xuyên
- Học hỏi từ các sự cố
- Lưu giữ kiến thức của tổ chức về hệ thống, ngay cả khi các cá nhân thay đổi [34]

Với sự trỗi dậy của các dịch vụ cloud, một sự phân tách vai trò đã xảy ra. Các đội ngũ operations tại các công ty hạ tầng chuyên về các chi tiết nhằm cung cấp một dịch vụ tin cậy cho một lượng lớn khách hàng, trong khi khách hàng của dịch vụ đó dành ít thời gian và công sức nhất có thể cho hạ tầng [35].

Khách hàng của các dịch vụ cloud vẫn cần đến operations, nhưng họ tập trung vào các khía cạnh khác nhau, chẳng hạn như chọn dịch vụ phù hợp nhất cho một tác vụ nhất định, tích hợp các dịch vụ với nhau, và di chuyển từ dịch vụ này sang dịch vụ khác. Mặc dù metered billing loại bỏ nhu cầu lập kế hoạch dung lượng theo nghĩa truyền thống, việc biết bạn đang sử dụng những tài nguyên nào cho mục đích gì vẫn rất quan trọng để bạn không lãng phí tiền vào các tài nguyên cloud không cần thiết. Lập kế hoạch dung lượng trở thành lập kế hoạch tài chính, và tối ưu hóa hiệu năng trở thành tối ưu hóa chi phí [36]. Ngoài ra, các dịch vụ cloud cũng có các giới hạn tài nguyên hoặc *quotas* (chẳng hạn như số lượng tối đa các tiến trình bạn có thể chạy đồng thời), điều mà bạn cần biết và lập kế hoạch trước khi chạm tới chúng [37].

Áp dụng một dịch vụ cloud có thể dễ dàng và nhanh chóng hơn so với việc cấp phát và vận hành hạ tầng riêng của bạn, mặc dù bạn vẫn phải học cách sử dụng dịch vụ cloud đó và có lẽ phải tìm cách xử lý các hạn chế của nó. Việc tích hợp giữa các dịch vụ trở thành một thách thức đặc biệt khi ngày càng nhiều nhà cung cấp cung cấp phạm vi dịch vụ cloud ngày càng rộng lớn nhằm hướng tới các trường hợp sử dụng khác nhau [38, 39]. ETL (xem “Data Warehousing”) chỉ là một phần của câu chuyện; các dịch vụ cloud vận hành cũng cần được tích hợp với nhau. Hiện tại, chúng ta đang thiếu các tiêu chuẩn để tạo điều kiện cho loại tích hợp này, vì vậy nó thường đòi hỏi nỗ lực thủ công đáng kể.

Các khía cạnh vận hành khác mà không thể hoàn toàn thuê ngoài cho các dịch vụ cloud bao gồm việc duy trì bảo mật cho một ứng dụng và các thư viện mà nó sử

dụng, quản lý các tương tác giữa các dịch vụ của chính bạn, giám sát tải trên các dịch vụ của bạn, và truy tìm nguyên nhân của các vấn đề như suy giảm hiệu năng hoặc ngừng hoạt động (outages). Mặc dù cloud đang thay đổi vai trò của operations, nhu cầu về operations vẫn lớn như bao giờ hết.

Hệ thống Phân tán so với Hệ thống Đơn nút

Một hệ thống bao gồm nhiều máy giao tiếp với nhau thông qua một mạng được gọi là một *distributed system* (hệ thống phân tán). Mỗi tiến trình tham gia vào một distributed system được gọi là một *node*. Bạn có thể muốn sử dụng loại hệ thống này vì nhiều lý do khác nhau:

Phân tán vốn có

Nếu một ứng dụng bao gồm hai hoặc nhiều người dùng tương tác với nhau, mỗi người sử dụng thiết bị riêng của họ, thì hệ thống đó chắc chắn là phân tán: việc giao tiếp giữa các thiết bị sẽ phải diễn ra thông qua một mạng.

Các yêu cầu giữa các dịch vụ cloud

Nếu dữ liệu được lưu trữ ở một dịch vụ nhưng được xử lý ở một dịch vụ khác, dữ liệu đó phải được chuyển qua mạng từ dịch vụ này sang dịch vụ kia. Do đó, các hệ thống cloud native và microservices (xem “Microservices and Serverless”) là các hệ thống phân tán.

Fault tolerance/high availability (khả năng chịu lỗi/độ sẵn sàng cao)

Nếu ứng dụng của bạn cần tiếp tục hoạt động ngay cả khi một máy (hoặc nhiều máy, hoặc mạng, hoặc toàn bộ datacenter) bị sập, bạn có thể sử dụng nhiều máy để tạo ra tính dự phòng. Khi một máy gặp lỗi, một máy khác có thể tiếp quản. Xem “Reliability and Fault Tolerance” và Chương 6.

Khả năng mở rộng (Scalability)

Nếu khối lượng dữ liệu hoặc yêu cầu tính toán của bạn tăng lên vượt quá khả năng xử lý của một máy đơn lẻ, bạn có thể phân tán tải trọng trên nhiều máy. Xem “Scalability”.

Độ trễ (Latency)

Nếu bạn có người dùng trên khắp thế giới, bạn có thể muốn đặt các máy chủ tại nhiều khu vực khác nhau trên toàn cầu để mỗi người dùng có thể được phục vụ bởi một máy chủ nằm gần họ về mặt địa lý. Điều đó giúp người dùng không phải chờ đợi các gói tin mạng di chuyển nửa vòng trái đất để nhận được phản hồi cho các yêu cầu của họ. Xem “Describing Performance”.

Tính đàn hồi (Elasticity)

Nếu ứng dụng của bạn bận rộn vào những thời điểm này nhưng lại nhàn rỗi vào những thời điểm khác, một deployment trên cloud có thể scale up hoặc scale down để đáp ứng nhu cầu, nhờ đó bạn chỉ phải trả tiền cho những tài nguyên mà

bạn đang thực sự sử dụng. Điều này khó thực hiện hơn trên một máy đơn lẻ, vốn cần được cấp phát để xử lý tải trọng tối đa, ngay cả vào những lúc nó hầu như không được sử dụng.

Phần cứng chuyên dụng (Specialized hardware)

Các thành phần khác nhau của hệ thống có thể tận dụng các loại phần cứng khác nhau để phù hợp với khối lượng công việc của chúng. Ví dụ, một object store có thể sử dụng các máy có nhiều ổ đĩa nhưng ít CPU, trong khi một hệ thống phân tích dữ liệu có thể sử dụng các máy có nhiều CPU và bộ nhớ nhưng không có ổ đĩa, và một hệ thống machine learning có thể sử dụng các máy có GPU (vốn hiệu quả hơn nhiều so với CPU trong việc huấn luyện các deep neural network và các tác vụ ML khác).

Tuân thủ pháp lý (Legal compliance)

Một số quốc gia có luật về lưu trữ dữ liệu (data residency), yêu cầu dữ liệu về con người trong phạm vi quyền hạn của họ phải được lưu trữ và xử lý về mặt địa lý trong quốc gia đó [40]. Phạm vi của các quy định này khác nhau—ví dụ, trong một số trường hợp nó chỉ áp dụng cho dữ liệu y tế hoặc tài chính, trong khi các trường hợp khác thì rộng hơn. Do đó, một dịch vụ có người dùng ở nhiều khu vực pháp lý như vậy sẽ phải phân phối dữ liệu của họ trên các máy chủ tại nhiều địa điểm khác nhau.

Tính bền vững (Sustainability)

Nếu bạn có sự linh hoạt về nơi chốn và thời gian để chạy các job của mình, bạn có thể chạy chúng vào thời điểm và tại nơi có sẵn nhiều điện tái tạo, đồng thời tránh chạy chúng khi lưới điện đang bị quá tải. Điều này có thể giảm lượng khí thải carbon và cho phép bạn tận dụng nguồn điện giá rẻ khi có sẵn [41, 42].

Những lý do này áp dụng cho cả các dịch vụ mà bạn tự viết (mã nguồn ứng dụng) và các dịch vụ bao gồm phần mềm có sẵn (chẳng hạn như các cơ sở dữ liệu).

Các vấn đề với Distributed Systems

Các distributed systems cũng có những mặt hạn chế. Mọi yêu cầu và lời gọi API đi qua mạng đều cần phải đối mặt với khả năng xảy ra lỗi. Mạng có thể bị gián đoạn, hoặc dịch vụ có thể bị quá tải hoặc crash, và do đó bất kỳ yêu cầu nào cũng có thể bị timeout mà không nhận được phản hồi. Trong trường hợp này, chúng ta không biết liệu dịch vụ đã nhận được yêu cầu hay chưa, và việc chỉ đơn giản là thử lại có thể không an toàn. Chúng ta sẽ thảo luận chi tiết về những vấn đề này trong Chương 9.

Mặc dù mạng datacenter rất nhanh, việc thực hiện một lời gọi đến một dịch vụ khác vẫn chậm hơn rất nhiều so với việc gọi một hàm trong cùng một process [43]. Khi vận hành trên khối lượng dữ liệu lớn, thay vì chuyển dữ liệu từ bộ lưu trữ sang một máy riêng biệt để xử lý, việc đưa tính toán đến máy đã có sẵn dữ liệu có thể sẽ nhanh hơn [44]. Thêm nhiều node không phải lúc nào cũng nhanh hơn; trong một số

trường hợp, một chương trình đơn luồng (single-threaded) đơn giản trên một máy tính có thể hoạt động tốt hơn đáng kể so với một cluster với hơn 100 nhân CPU [45].

Xử lý sự cố (troubleshooting) một distributed system thường rất khó khăn—nếu hệ thống phản hồi chậm, làm thế nào bạn có thể xác định vấn đề nằm ở đâu? Các kỹ thuật để chẩn đoán vấn đề trong các distributed system được phát triển dưới tiêu đề *observability* (khả năng quan sát) [46, 47], bao gồm việc thu thập dữ liệu về quá trình thực thi của một hệ thống và cho phép truy vấn dữ liệu đó theo những cách giúp phân tích cả các metrics cấp cao lẫn các sự kiện riêng lẻ. Các công cụ *tracing* (truy vết) như OpenTelemetry, Zipkin và Jaeger cho phép bạn theo dõi client nào đã gọi server nào cho thao tác nào và mỗi lần gọi mất bao lâu [48].

Các database cung cấp nhiều cơ chế khác nhau để đảm bảo tính nhất quán dữ liệu, như chúng ta sẽ thấy trong Chương 6 và Chương 8. Tuy nhiên, khi mỗi service đều có database riêng, việc duy trì tính nhất quán của dữ liệu giữa các service khác nhau đó trở thành vấn đề của ứng dụng. Distributed transactions, thứ mà chúng ta sẽ khám phá trong Chương 8, là một kỹ thuật khả thi để đảm bảo tính nhất quán, nhưng chúng hiếm khi được sử dụng trong ngữ cảnh microservices vì chúng đi ngược lại mục tiêu làm cho các service độc lập với nhau, và nhiều database không hỗ trợ chúng [49].

Vì tất cả những lý do này, thực hiện một tác vụ trên một máy đơn lẻ thường đơn giản và rẻ hơn nhiều so với việc thiết lập một distributed system [22, 45, 50]. CPU, memory và đĩa cứng đã trở nên lớn hơn, nhanh hơn và đáng tin cậy hơn. Khi kết hợp với các database single-node như DuckDB, SQLite và KuduDB, nhiều workload hiện nay có thể chạy trên một node duy nhất. Chúng ta sẽ khám phá sâu hơn về chủ đề này trong Chương 4.

Microservices và Serverless

Cách phổ biến nhất để phân tán một hệ thống trên nhiều máy là chia chúng thành các client và server, rồi để các client gửi các request tới các server. Thông thường nhất, HTTP được sử dụng cho việc giao tiếp này, như chúng ta sẽ thảo luận trong mục “Dataflow Through Services: REST and RPC”. Một quy trình tương tự có thể vừa là một server (xử lý các request đến) vừa là một client (gửi các request đi tới các service khác).

Cách xây dựng ứng dụng này theo truyền thống được gọi là *service-oriented architecture* (SOA - kiến trúc hướng dịch vụ); gần đây hơn, ý tưởng này đã được tinh chỉnh thành *microservices architecture* (kiến trúc microservices) [51, 52]. Trong một microservices architecture, một service có một mục đích xác định rõ ràng (ví dụ, trong trường hợp của S3, đó là lưu trữ tệp); mỗi service cung cấp một API có thể được gọi bởi các client thông qua mạng, và mỗi service có một đội ngũ chịu trách

nhiệm bảo trì nó. Do đó, một ứng dụng phức tạp có thể được phân tách thành nhiều service tương tác với nhau, mỗi service được quản lý bởi một đội ngũ riêng biệt. Các hệ thống cloud native sử dụng mạnh mẽ việc phân tách thành các service, nhưng các hệ thống on-premises cũng có thể sử dụng cách tiếp cận hướng dịch vụ.

Việc chia một phần mềm phức tạp thành nhiều service mang lại một số lợi thế: mỗi service có thể được cập nhật độc lập, giúp giảm nỗ lực điều phối giữa các đội ngũ; mỗi service có thể được cấp phát các tài nguyên phần cứng mà nó cần; và việc che giấu các chi tiết triển khai đằng sau một API có nghĩa là những người chủ sở hữu service có thể tự do thay đổi cách triển khai mà không ảnh hưởng đến các client. Về mặt lưu trữ dữ liệu, việc mỗi service có các database riêng và không chia sẻ database giữa các service là điều phổ biến. Việc chia sẻ một database sẽ khiến toàn bộ cấu trúc database trở thành một phần của API của service đó, và khi đó cấu trúc này sẽ rất khó thay đổi. Các database dùng chung cũng có thể khiến các truy vấn của một service gây ảnh hưởng tiêu cực đến hiệu năng của các service khác.

Mặt khác, việc có nhiều service có thể tự nó gây ra sự phức tạp. Việc testing một service trong quá trình phát triển có thể rất phức tạp, vì bạn cũng cần phải chạy tất cả các service khác mà nó phụ thuộc vào. Hơn nữa, mỗi service đều yêu cầu hạ tầng để triển khai các bản release mới, điều chỉnh tài nguyên phần cứng được cấp phát để phù hợp với tải, thu thập logs, giám sát sức khỏe service, và gửi cảnh báo cho một kỹ sư on-call trong trường hợp xảy ra vấn đề. Các orchestration framework như Kubernetes đã trở thành một cách phổ biến để triển khai các service, vì chúng cung cấp một nền tảng cho hạ tầng này.

Ngoài ra, các microservice API có thể gặp thách thức trong việc tiến hóa. Các client gọi một API mong đợi nó có các field nhất định. Các lập trình viên có thể muốn thêm hoặc xóa các field trong một API khi nhu cầu kinh doanh thay đổi, nhưng làm như vậy có thể khiến các client bị lỗi. Tệ hơn nữa, những lỗi như vậy thường không được phát hiện cho đến giai đoạn muộn trong vòng đời phát triển, khi API của service đã được cập nhật và triển khai lên môi trường staging hoặc production. Các tiêu chuẩn mô tả API như OpenAPI và gRPC giúp quản lý mối quan hệ giữa client API và server API; chúng ta sẽ thảo luận kỹ hơn về những điều này trong Chương 5.

Microservices chủ yếu là một giải pháp kỹ thuật cho một vấn đề về con người: cho phép các nhóm khác nhau có thể tiến triển độc lập mà không cần phải phối hợp với nhau. Điều này rất giá trị trong một công ty lớn, nhưng trong một công ty nhỏ với ít nhóm hơn, việc sử dụng microservices có khả năng là một sự overhead không cần thiết, và việc triển khai ứng dụng theo cách đơn giản nhất có thể sẽ được ưu tiên hơn [51].

Serverless, hay *function as a service* (FaaS), là một cách tiếp cận khác để triển khai các service, trong đó việc quản lý hạ tầng được thuê ngoài cho một nhà cung cấp cloud [32]. Khi sử dụng các VM, bạn phải chọn một cách rõ ràng khi nào cần khởi động

hoặc tắt một instance; ngược lại, với mô hình serverless, nhà cung cấp cloud sẽ tự động cấp phát và giải phóng tài nguyên phần cứng khi cần thiết, dựa trên các request gửi đến service của bạn [53]. Giống như việc cloud storage đã thay thế việc lập kế hoạch dung lượng (quyết định trước xem nên mua bao nhiêu ổ đĩa) bằng một mô hình tính phí theo mức sử dụng, cách tiếp cận serverless đang mang việc tính phí theo mức sử dụng đến với việc thực thi mã nguồn: bạn chỉ trả tiền cho khoảng thời gian mã ứng dụng của bạn đang chạy thay vì phải cấp phát tài nguyên trước.

Để mang lại những lợi ích như vậy, nhiều nhà cung cấp hạ tầng serverless áp đặt giới hạn thời gian cho việc thực thi function và giới hạn các môi trường runtime, và các service có thể gặp phải tình trạng thời gian khởi động chậm khi một function được gọi lần đầu. Thuật ngữ “serverless” cũng có thể gây hiểu lầm; mỗi lần thực thi serverless function vẫn chạy trên một server, nhưng các lần thực thi tiếp theo có thể chạy trên một server khác. Hơn nữa, các dịch vụ hạ tầng như BigQuery và các dịch vụ Kafka khác nhau đã áp dụng thuật ngữ “serverless” để báo hiệu rằng các dịch vụ của chúng có khả năng autoscaling và chúng tính phí theo mức sử dụng thay vì theo các machine instance.

Cloud Computing So với Supercomputing

Cloud computing không phải là cách duy nhất để xây dựng các hệ thống tính toán quy mô lớn; một lựa chọn thay thế là *high-performance computing* (HPC), còn được gọi là *supercomputing*. Mặc dù có những điểm giao thoa, HPC thường có các ưu tiên khác nhau và sử dụng các kỹ thuật khác với cloud computing và các hệ thống datacenter doanh nghiệp. Dưới đây là một số khác biệt chính:

- Các supercomputer thường được sử dụng cho các tác vụ tính toán khoa học đòi hỏi cường độ tính toán cao, chẳng hạn như dự báo thời tiết, mô hình hóa khí hậu, động lực học phân tử (mô phỏng chuyển động của các nguyên tử và phân tử), các bài toán tối ưu hóa phức tạp, và giải các phương trình vi phân riêng phần. Mặt khác, cloud computing có xu hướng được sử dụng cho các dịch vụ trực tuyến, các hệ thống dữ liệu kinh doanh, và các hệ thống tương tự cần phục vụ các request của người dùng với độ tin cậy cao.
- Một siêu máy tính thường chạy các batch job lớn, thực hiện checkpoint trạng thái tính toán của chúng vào đĩa định kỳ. Nếu một node gặp lỗi, một giải pháp phổ biến là chỉ đơn giản dừng toàn bộ workload của cluster, sửa chữa node bị lỗi, sau đó khởi động lại quá trình tính toán từ checkpoint cuối cùng [54, 55]. Với các dịch vụ cloud, việc dừng toàn bộ cluster thường không được mong muốn, vì các dịch vụ cần phục vụ người dùng liên tục với sự gián đoạn tối thiểu.
- Các node của siêu máy tính thường giao tiếp thông qua shared memory và RDMA, những thứ hỗ trợ băng thông cao và latency (độ trễ) thấp nhưng giả định mức độ tin cậy cao giữa những người dùng hệ thống [56]. Trong điện toán

cloud, mạng và các máy chủ thường được chia sẻ bởi các tổ chức không tin tưởng lẫn nhau, đòi hỏi các cơ chế bảo mật mạnh mẽ hơn như resource isolation (tách rời tài nguyên) (ví dụ: virtual machines), mã hóa và xác thực.

- Mạng datacenter cloud thường dựa trên IP và Ethernet, được sắp xếp theo các cấu trúc Clos topology để cung cấp bisection bandwidth cao—một thước đo thường được sử dụng cho hiệu năng tổng thể của một mạng [54, 57]. Các siêu máy tính thường sử dụng các topology mạng chuyên dụng, chẳng hạn như multidimensional meshes và toruses [58], giúp mang lại hiệu năng tốt hơn cho các workload HPC với các mô hình giao tiếp đã biết.
- Điện toán cloud cho phép các node được phân tán trên nhiều vùng địa lý khác nhau, trong khi các siêu máy tính thường giả định rằng tất cả các node của chúng đều nằm gần nhau.

Các hệ thống phân tích quy mô lớn đôi khi chia sẻ một số đặc điểm với siêu máy tính, đó là lý do tại sao việc tìm hiểu về các kỹ thuật này có thể rất hữu ích nếu bạn đang làm việc trong lĩnh vực này. Tuy nhiên, cuốn sách này chủ yếu quan tâm đến các dịch vụ cần phải luôn sẵn sàng, như đã thảo luận trong mục “Reliability and Fault Tolerance”.

Hệ thống dữ liệu, Luật pháp và Xã hội

Như bạn đã thấy trong chương này, kiến trúc của các hệ thống dữ liệu không chỉ bị ảnh hưởng bởi các mục tiêu và yêu cầu kỹ thuật, mà còn bởi nhu cầu của con người trong các tổ chức mà chúng hỗ trợ. Ngày càng có nhiều kỹ sư hệ thống dữ liệu nhận ra rằng việc phục vụ nhu cầu của chính doanh nghiệp mình là chưa đủ; chúng ta còn có trách nhiệm đối với toàn xã hội.

Một mối quan tâm cụ thể là các hệ thống lưu trữ dữ liệu về con người và hành vi của họ. Kể từ năm 2018, GDPR đã trao cho cư dân của nhiều quốc gia châu Âu quyền kiểm soát và quyền pháp lý lớn hơn đối với dữ liệu cá nhân của họ, và các quy định về quyền riêng tư tương tự đã được áp dụng tại nhiều quốc gia và tiểu bang khác trên thế giới (ví dụ như CCPA). Các quy định xung quanh AI, chẳng hạn như EU AI Act, đặt ra thêm các hạn chế về cách dữ liệu cá nhân có thể được sử dụng.

Hơn nữa, ngay cả trong các lĩnh vực không trực tiếp chịu sự điều chỉnh của các quy định, người ta ngày càng nhận thức rõ hơn về những tác động mà các hệ thống máy tính gây ra đối với con người và xã hội. Mạng xã hội đã thay đổi cách các cá nhân tiêu thụ tin tức, điều này ảnh hưởng đến quan điểm chính trị của họ và do đó có thể ảnh hưởng đến kết quả của các cuộc bầu cử. Các hệ thống tự động ngày càng đưa ra nhiều quyết định có hệ quả sâu sắc đối với các cá nhân, chẳng hạn như ai nên được cấp khoản vay hoặc bảo hiểm, ai nên được mời phỏng vấn xin việc, hoặc ai nên bị nghi ngờ phạm tội [59].

Tất cả những người làm việc trên các hệ thống như vậy đều chia sẻ trách nhiệm trong việc xem xét tác động đạo đức từ các quyết định của mình và đảm bảo rằng chúng tuân thủ các luật pháp liên quan. Không phải tất cả mọi người đều cần trở thành chuyên gia về luật pháp và đạo đức, nhưng một nhận thức cơ bản về các nguyên tắc pháp lý và đạo đức cũng quan trọng giống như, chẳng hạn, một số kiến thức nền tảng về distributed systems.

Các cân nhắc về pháp lý đang ảnh hưởng đến chính những nền tảng của việc thiết kế hệ thống dữ liệu [60]. Ví dụ, GDPR trao cho các cá nhân quyền yêu cầu xóa dữ liệu của họ (đôi khi được gọi là *quyền được lãng quên*). Tuy nhiên, như chúng ta sẽ thấy trong cuốn sách này, nhiều hệ thống dữ liệu dựa trên các cấu trúc bất biến như append-only log như một phần trong thiết kế của chúng. Làm thế nào chúng ta có thể đảm bảo việc xóa một số dữ liệu ở giữa một tệp vốn được cho là bất biến? Làm thế nào chúng ta xử lý việc xóa dữ liệu đã được đưa vào các dataset phái sinh (xem “Systems of Record and Derived Data”), chẳng hạn như dữ liệu huấn luyện cho các mô hình ML? Việc trả lời những câu hỏi này tạo ra các thách thức kỹ thuật mới.

Hiện tại, chúng ta chưa có các hướng dẫn rõ ràng về việc những công nghệ hay kiến trúc hệ thống cụ thể nào nên được coi là tuân thủ GDPR. Quy định này cố tình không bắt buộc các công nghệ cụ thể, bởi vì chúng có thể thay đổi nhanh chóng khi công nghệ tiến bộ. Thay vào đó, các văn bản pháp luật đưa ra các nguyên tắc cấp cao vốn tùy thuộc vào cách diễn giải. Do đó, việc làm thế nào để tuân thủ các quy định về quyền riêng tư không có câu trả lời đơn giản, nhưng chúng ta sẽ xem xét một số công nghệ thông qua lăng kính này.

Nhìn chung, chúng ta lưu trữ dữ liệu vì chúng ta nghĩ rằng giá trị của nó lớn hơn chi phí lưu trữ. Tuy nhiên, cần nhớ rằng chi phí lưu trữ không chỉ dừng lại ở hóa đơn bạn trả cho S3 hoặc một dịch vụ khác. Việc tính toán chi phí-lợi ích cũng nên tính đến các rủi ro về trách nhiệm pháp lý và tổn hại danh tiếng nếu dữ liệu bị rò rỉ hoặc bị xâm nhập bởi các đối thủ, cũng như rủi ro về chi phí pháp lý và các khoản tiền phạt nếu việc lưu trữ và xử lý dữ liệu bị phát hiện là không tuân thủ pháp luật [50].

Chính phủ hoặc các lực lượng cảnh sát cũng có thể buộc các công ty phải bàn giao dữ liệu. Khi dữ liệu có thể tiết lộ các hành vi bị hình sự hóa (ví dụ: đồng tính luyến ái ở một số quốc gia Trung Đông và Châu Phi, hoặc việc tìm kiếm dịch vụ phá thai ở một số bang tại Hoa Kỳ), việc lưu trữ dữ liệu đó tạo ra các rủi ro an toàn thực sự cho người dùng. Việc di chuyển đến một phòng khám phá thai, chẳng hạn, có thể dễ dàng bị tiết lộ bởi dữ liệu vị trí, hoặc thậm chí có thể bởi một log về các địa chỉ IP của người dùng theo thời gian (những thứ chỉ ra vị trí xấp xỉ).

Một khi tất cả các rủi ro đã được tính đến, việc quyết định rằng một số dữ liệu đơn giản là không đáng để lưu trữ, và do đó chúng nên được xóa bỏ, có thể là một quyết định hợp lý. Nguyên tắc *giảm thiểu dữ liệu* (*data minimization*) này (đôi khi được biết đến với thuật ngữ tiếng Đức là *Datensparsamkeit*) đi ngược lại triết lý “big data”

về việc lưu trữ thật nhiều dữ liệu một cách mang tính suy đoán để phòng trường hợp chúng trở nên hữu ích trong tương lai [61]. Nhưng việc giảm thiểu dữ liệu phù hợp với GDPR, quy định rằng dữ liệu cá nhân chỉ có thể được thu thập cho một mục đích cụ thể, rõ ràng; không thể được sử dụng cho bất kỳ mục đích nào khác sau đó; và không được giữ lâu hơn mức cần thiết cho các mục đích mà nó đã được thu thập [62].

Các doanh nghiệp cũng đã chú ý đến các mối quan tâm về quyền riêng tư và an toàn. Các công ty thẻ tín dụng yêu cầu các doanh nghiệp xử lý thanh toán phải tuân thủ các tiêu chuẩn Payment Card Industry (PCI) nghiêm ngặt. Các đơn vị xử lý phải trải qua các đợt đánh giá thường xuyên từ các kiểm toán viên độc lập để xác minh việc duy trì tuân thủ. Các nhà cung cấp phần mềm cũng đang phải đối mặt với sự giám sát ngày càng tăng. Nhiều bên mua hiện yêu cầu các nhà cung cấp của họ phải tuân thủ các tiêu chuẩn Service Organization Control (SOC) Type 2. Tương tự như tuân thủ PCI, các nhà cung cấp phải trải qua các cuộc kiểm toán từ bên thứ ba để xác minh việc tuân thủ.

Nhìn chung, điều quan trọng là phải cân bằng giữa nhu cầu của doanh nghiệp bạn với nhu cầu của những người mà dữ liệu bạn đang thu thập và xử lý. Vẫn còn nhiều khía cạnh khác về chủ đề này; trong Chương 14, chúng ta sẽ đi sâu hơn vào đạo đức và tuân thủ pháp lý, bao gồm cả các vấn đề về định kiến và phân biệt đối xử.

Tóm tắt

Chủ đề của chương này là tìm hiểu về các đánh đổi (trade-off)—tức là nhận ra rằng nhiều câu hỏi không có một câu trả lời duy nhất đúng, mà có nhiều khả năng khác nhau, mỗi khả năng đều có ưu và nhược điểm riêng. Chúng ta đã khám phá một số lựa chọn quan trọng nhất ảnh hưởng đến kiến trúc của các hệ thống dữ liệu, và chúng ta cũng đã giới thiệu các thuật ngữ sẽ được sử dụng xuyên suốt phần còn lại của cuốn sách này.

Chúng ta bắt đầu bằng việc phân biệt giữa các hệ thống vận hành (xử lý giao dịch, OLTP) và hệ thống phân tích (OLAP), đồng thời khám phá sự khác biệt giữa chúng, không chỉ trong việc quản lý các loại dữ liệu khác nhau với các mẫu truy cập khác nhau, mà còn trong việc phục vụ các đối tượng người dùng khác nhau. Trong quá trình đó, chúng ta đã gặp các khái niệm về data warehouse và data lake, những nơi nhận các luồng dữ liệu từ các hệ thống vận hành thông qua ETL. Trong Chương 4, chúng ta sẽ thấy rằng các hệ thống vận hành và phân tích thường sử dụng các bố cục dữ liệu nội bộ rất khác nhau do các loại truy vấn khác nhau mà chúng cần phục vụ.

Sau đó, chúng ta đã so sánh các dịch vụ cloud, một sự phát triển tương đối gần đây, với mô hình truyền thống về phần mềm tự lưu trữ (self-hosted) vốn từng thống trị kiến trúc hệ thống dữ liệu trước đây. Việc cách tiếp cận nào trong số này hiệu quả về

chi phí hơn phụ thuộc nhiều vào tình huống cụ thể của bạn, nhưng không thể phủ nhận rằng các cách tiếp cận cloud native đang mang lại những thay đổi lớn cho cách kiến trúc các hệ thống dữ liệu—ví dụ, trong cách chúng tách rời storage và compute.

Các hệ thống cloud về bản chất là phân tán, và chúng ta đã xem xét ngắn gọn một số đánh đổi (trade-off) của các hệ thống phân tán so với việc sử dụng một máy đơn lẻ. Trong một số tình huống, bạn không thể tránh khỏi việc phải chuyển sang phân tán, nhưng lời khuyên là không nên vội vàng biến một hệ thống thành phân tán nếu vẫn có thể duy trì nó trên một máy đơn lẻ. Trong Chương 9, chúng ta sẽ tìm hiểu chi tiết hơn về những thách thức đối với các hệ thống phân tán.

Cuối cùng, chúng ta thấy rằng kiến trúc của một hệ thống dữ liệu không chỉ được quyết định bởi nhu cầu của doanh nghiệp triển khai hệ thống, mà còn bởi các quy định về quyền riêng tư nhằm bảo vệ quyền lợi của những người có dữ liệu đang được xử lý—một khía cạnh mà nhiều kỹ sư thường có xu hướng bỏ qua. Cách chúng ta chuyển đổi các yêu cầu pháp lý thành các triển khai kỹ thuật vẫn chưa được chính thức hóa, nhưng điều quan trọng là phải ghi nhớ câu hỏi này khi chúng ta đi tiếp phần còn lại của cuốn sách.

Footnotes

References

- [1] Richard T. Kouzes, Gordon A. Anderson, Stephen T. Elbert, Ian Gorton, and Deborah K. Gracio. “The Changing Paradigm of Data-Intensive Computing.” *IEEE Computer*, volume 42, issue 1, pages 26–34, January 2009. [doi:10.1109/MC.2009.26](https://doi.org/10.1109/MC.2009.26)
- [2] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. “Local-First Software: You Own Your Data, in Spite of the Cloud.” At *2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software* (Onward!), October 2019. [doi:10.1145/3359591.3359737](https://doi.org/10.1145/3359591.3359737)
- [3] Joe Reis and Matt Housley. *Fundamentals of Data Engineering*. O’Reilly Media, 2022. ISBN: 9781098108304
- [4] Rui Pedro Machado and Helder Russa. *Analytics Engineering with SQL and dbt*. O’Reilly Media, 2023. ISBN: 9781098142384
- [5] Edgar F. Codd, S. B. Codd, and C. T. Salley. “Providing OLAP to User-Analysts: An IT Mandate.” E. F. Codd Associates, 1993. Archived at perma.cc/RKX8-2GEE
- [6] Chinmay Soman and Neha Pawar. “Comparing Three Real-Time OLAP Databases: Apache Pinot, Apache Druid, and ClickHouse.” *startree.ai*, April 2023. Archived at perma.cc/8BZP-VWPA
- [7] Surajit Chaudhuri and Umeshwar Dayal. “An Overview of Data Warehousing and OLAP Technology.” *ACM SIGMOD Record*, volume 26, issue 1, pages 65–74, March 1997. [doi:10.1145/248603.248616](https://doi.org/10.1145/248603.248616)
- [8] Fatma Özcan, Yuanyuan Tian, and Pinar Tözün. “Hybrid Transactional/Analytical Processing: A Survey.” At *ACM International Conference on Management of Data (SIGMOD)*, May 2017. [doi:10.1145/3035918.3054784](https://doi.org/10.1145/3035918.3054784)

- [9] Adam Prout, Szu-Po Wang, Joseph Victor, Zhou Sun, Yongzhu Li, Jack Chen, Evan Bergeron, Eric Hanson, Robert Walzer, Rodrigo Gomes, and Nikita Shamgunov. “Cloud-Native Transactions and Analytics in SingleStore.” At *International Conference on Management of Data (SIGMOD)*, June 2022. [doi:10.1145/3514221.3526055](https://doi.org/10.1145/3514221.3526055)
- [10] Chao Zhang, Guoliang Li, Jintao Zhang, Xinning Zhang, and Jianhua Feng. “HTAP Databases: A Survey.” *IEEE Transactions on Knowledge and Data Engineering*, volume 36, issue 11, pages 6410–6429, April 2024. [doi:10.1109/TKDE.2024.3389693](https://doi.org/10.1109/TKDE.2024.3389693)
- [11] Michael Stonebraker and Uğur Çetintemel. “‘One Size Fits All’: An Idea Whose Time Has Come and Gone.” At *21st International Conference on Data Engineering (ICDE)*, April 2005. [doi:10.1109/ICDE.2005.1](https://doi.org/10.1109/ICDE.2005.1)
- [12] Jeffrey Cohen, Brian Dolan, Mark Dunlap, Joseph M. Hellerstein, and Caleb Welton. “MAD Skills: New Analysis Practices for Big Data.” *Proceedings of the VLDB Endowment*, volume 2, issue 2, pages 1481–1492, August 2009. [doi:10.14778/1687553.1687576](https://doi.org/10.14778/1687553.1687576)
- [13] Dan Olteanu. “The Relational Data Borg Is Learning.” *Proceedings of the VLDB Endowment*, volume 13, issue 12, pages 3502–3515, August 2020. [doi:10.14778/3415478.3415572](https://doi.org/10.14778/3415478.3415572)
- [14] Matt Bornstein, Martin Casado, and Jennifer Li. “Emerging Architectures for Modern Data Infrastructure: 2020.” *future.a16z.com*, October 2020. Archived at perma.cc/LF8W-KDCC
- [15] Rihan Hai, Christos Koutras, Christoph Quix, and Matthias Jarke. “Data Lakes: A Survey of Functions and Systems.” *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, volume 35, issue 12, pages 12571–12590, December 2023. [doi:10.1109/TKDE.2023.3270101](https://doi.org/10.1109/TKDE.2023.3270101)
- [16] Martin Fowler. “Data Lake.” *martinfowler.com*, February 2015. Archived at perma.cc/4WKN-CZUK
- [17] Bobby Johnson and Joseph Adler. “The Sushi Principle: Raw Data Is Better.” At *Strata+Hadoop World*, February 2015.
- [18] DataKitchen, Inc. “The DataOps Manifesto.” *dataopsmanifesto.org*, 2017. Archived at perma.cc/3F5N-FUQ4
- [19] Tejas Manohar. “What Is Reverse ETL: A Definition & Why It’s Taking Off.” *hightouch.io*, November 2021. Archived at perma.cc/A7TN-GLYJ
- [20] Camille Fournier. “Why Is It So Hard to Decide to Buy?” *skamille.medium.com*, July 2021. Archived at perma.cc/6VSG-HQ5X
- [21] David Heinemeier Hansson. “Why We’re Leaving the Cloud.” *world.bey.com*, October 2022. Archived at perma.cc/82E6-UJ65
- [22] Nima Badizadegan. “Use One Big Server.” *specbranch.com*, August 2022. Archived at perma.cc/M8NB-95UK
- [23] Steve Yegge. “Dear Google Cloud: Your Deprecation Policy Is Killing You.” *steve-yegge.medium.com*, August 2020. Archived at perma.cc/KQP9-SPGU
- [24] Alexandre Verbitski, Anurag Gupta, Debanjan Saha, Murali Brahmadesam, Kamal Gupta, Raman Mittal, Sailesh Krishnamurthy, Sandor Maurice, Tengiz Kharatishvili, and Xiaofeng Bao. “Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases.” At *ACM International Conference on Management of Data (SIGMOD)*, May 2017. [doi:10.1145/3035918.3056101](https://doi.org/10.1145/3035918.3056101)
- [25] Panagiotis Antonopoulos, Alex Budovski, Cristian Diaconu, Alejandro Hernandez Saenz, Jack Hu, Hanuma Kodavalla, Donald Kossmann, Sandeep Lingam, Umar Farooq Minhas, Naveen Prakash, Vijendra Purohit, Hugh Qu, Chaitanya Sreenivas Ravella, Krystyna Reisteter, Sheetal Shrotri, Dixin

- Tang, and Vikram Wakade. “Socrates: The New SQL Server in the Cloud.” At *ACM International Conference on Management of Data (SIGMOD)*, June 2019. doi:10.1145/3299869.3314047
- [26] Midhul Vuppapapati, Justin Miron, Rachit Agarwal, Dan Truong, Ashish Motivala, and Thierry Cruanes. “Building an Elastic Query Engine on Disaggregated Storage.” At *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, February 2020.
- [27] Nick Van Wiggeren. “The Real Failure Rate of EBS.” *planetscale.com*, March 2025. Archived at perma.cc/43CR-SAH5
- [28] Colin Breck. “Predicting the Future of Distributed Systems.” *blog.colinbreck.com*, August 2024. Archived at perma.cc/K5FC-4XX2
- [29] Gwen Shapira. “Compute-Storage Separation Explained.” *thenile.dev*, January 2023. Archived at perma.cc/QCV3-XJNZ
- [30] Ravi Murthy and Gurmeet Goindi. “AlloyDB for PostgreSQL Under the Hood: Intelligent, Database-Aware Storage.” *cloud.google.com*, May 2022. Archived at archive.org
- [31] Jack Vanlightly. “The Architecture of Serverless Data Systems.” *jack-vanlightly.com*, November 2023. Archived at perma.cc/UDV4-TNJ5
- [32] Eric Jonas, Johann Schleier-Smith, Vikram Sreekanti, Chia-Che Tsai, Anurag Khandelwal, Qifan Pu, Vaishaal Shankar, Joao Carreira, Karl Krauth, Neeraja Yadwadkar, Joseph E. Gonzalez, Raluca Ada Popa, Ion Stoica, and David A. Patterson. “Cloud Programming Simplified: A Berkeley View on Serverless Computing.” *arXiv:1902.03383*, February 2019.
- [33] Betsy Beyer, Jennifer Petoff, Chris Jones, and Niall Richard Murphy. *Site Reliability Engineering: How Google Runs Production Systems*. O’Reilly Media, 2016. ISBN: 9781491929124
- [34] Thomas Limoncelli. “The Time I Stole \$10,000 from Bell Labs.” *ACM Queue*, volume 18, issue 5, November 2020. doi:10.1145/3434571.3434773
- [35] Charity Majors. “The Future of Ops Jobs.” *acloudguru.com*, August 2020. Archived at perma.cc/GRU2-CZG3
- [36] Boris Cherkasky. “(Over)Pay as You Go for Your Datastore.” *medium.com*, September 2021. Archived at perma.cc/Q8TV-2AM2
- [37] Shlomi Kushchi. “Serverless Doesn’t Mean DevOpsLess or NoOps.” *thenewstack.io*, February 2023. Archived at perma.cc/3NJR-AYYU
- [38] Erik Bernhardsson. “Storm in the Stratosphere: How the Cloud Will Be Reshuffled.” *erikbern.com*, November 2021. Archived at perma.cc/SYB2-99P3
- [39] Benn Stancil. “The Data OS.” *benn.substack.com*, September 2021. Archived at perma.cc/WQ43-FHS6
- [40] Maria Korolov. “Data Residency Laws Pushing Companies Toward Residency as a Service.” *csoonline.com*, January 2022. Archived at perma.cc/CHE4-XZZ2
- [41] Severin Borenstein. “Can Data Centers Flex Their Power Demand?” *energyatbaas.wordpress.com*, April 2025. Archived at perma.cc/MUD3-A6FF
- [42] Bilge Acun, Benjamin Lee, Fiodar Kazhamiaka, Aditya Sundarajan, Kiwan Maeng, Manoj Chakkaravarthy, David Brooks, and Carole-Jean Wu. “Carbon Dependencies in Datacenter Design and Management.” *ACM SIGENERGY Energy Informatics Review*, volume 3, issue 3, pages 21–26, October 2023. doi:10.1145/3630614.3630619
- [43] Kousik Nath. “These Are the Numbers Every Computer Engineer Should Know.” *freecodecamp.org*, September 2019. Archived at perma.cc/RW73-36RL

- [44] Joseph M. Hellerstein, Jose Faleiro, Joseph E. Gonzalez, Johann Schleier-Smith, Vikram Sreekanti, Alexey Tumanov, and Chenggang Wu. “Serverless Computing: One Step Forward, Two Steps Back.” *arXiv:1812.03651*, December 2018.
- [45] Frank McSherry, Michael Isard, and Derek G. Murray. “Scalability! But at What COST?” At *15th USENIX Workshop on Hot Topics in Operating Systems (HotOS)*, May 2015.
- [46] Cindy Sridharan. *Distributed Systems Observability: A Guide to Building Robust Systems*. Report, O’Reilly Media, 2018. Archived at perma.cc/M6JL-XKCM
- [47] Charity Majors. “Observability—A 3-Year Retrospective.” *thenewstack.io*, August 2019. Archived at perma.cc/CG62-TJWL
- [48] Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspán, and Chandan Shanbhag. “Dapper, a Large-Scale Distributed Systems Tracing Infrastructure.” Google Technical Report dapper-2010-1, April 2010. Archived at perma.cc/K7KU-2TMH
- [49] Rodrigo Laigner, Yongluan Zhou, Marcos Antonio Vaz Salles, Yijian Liu, and Marcos Kalinowski. “Data Management in Microservices: State of the Practice, Challenges, and Research Directions.” *Proceedings of the VLDB Endowment*, volume 14, issue 13, pages 3348–3361, September 2021. [doi:10.14778/3484224.3484232](https://doi.org/10.14778/3484224.3484232)
- [50] Jordan Tigani. “Big Data Is Dead.” *motherduck.com*, February 2023. Archived at perma.cc/HT4Q-K77U
- [51] Sam Newman. *Building Microservices*, 2nd edition. O’Reilly Media, 2021. ISBN: 9781492034025
- [52] Chris Richardson. “Microservices: Decomposing Applications for Deployability and Scalability.” *infoq.com*, May 2014. Archived at perma.cc/CKN4-YEQ2
- [53] Mohammad Shahradd, Rodrigo Fonseca, Íñigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. “Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider.” At *USENIX Annual Technical Conference (ATC)*, July 2020.
- [54] Luiz André Barroso, Urs Hölzle, and Parthasarathy Ranganathan. *The Datacenter as a Computer: Designing Warehouse-Scale Machines*, 3rd edition. Springer Nature, 2019. ISBN: 9783031017612
- [55] David Fiala, Frank Mueller, Christian Engelmann, Rolf Riesen, Kurt Ferreira, and Ron Brightwell. “Detection and Correction of Silent Data Corruption for Large-Scale High-Performance Computing.” At *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, November 2012. [doi:10.1109/SC.2012.49](https://doi.org/10.1109/SC.2012.49)
- [56] Anna Kornfeld Simpson, Adriana Szekeres, Jacob Nelson, and Irene Zhang. “Securing RDMA for High-Performance Datacenter Storage Systems.” At *12th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*, July 2020.
- [57] Arjun Singh, Joon Ong, Amit Agarwal, Glen Anderson, Ashby Armistead, Roy Bannon, Seb Boving, Gaurav Desai, Bob Felderman, Paulie Germano, Anand Kanagala, Jeff Provost, Jason Simmons, Eiichi Tanda, Jim Wanderer, Urs Hölzle, Stephen Stuart, and Amin Vahdat. “Jupiter Rising: A Decade of Clos Topologies and Centralized Control in Google’s Datacenter Network.” At *Annual Conference of the ACM Special Interest Group on Data Communication (SIGCOMM)*, August 2015. [doi:10.1145/2785956.2787508](https://doi.org/10.1145/2785956.2787508)
- [58] Glenn K. Lockwood. “Hadoop’s Uncomfortable Fit in HPC.” *glennklockwood.blogspot.co.uk*, May 2014. Archived at perma.cc/S8XX-Y67B

- [59] Cathy O’Neil. *Weapons of Math Destruction: How Big Data Increases Inequality and Threatens Democracy*. Crown Publishing, 2016. ISBN: 9780553418811
- [60] Supreeth Shastri, Vinay Banakar, Melissa Wasserman, Arun Kumar, and Vijay Chidambaram. “Understanding and Benchmarking the Impact of GDPR on Database Systems.” *Proceedings of the VLDB Endowment*, volume 13, issue 7, pages 1064–1077, March 2020. [doi:10.14778/3384345.3384354](https://doi.org/10.14778/3384345.3384354)
- [61] Martin Fowler. “Datensparsamkeit.” *martinfowler.com*, December 2013. Archived at perma.cc/R9QX-CME6
- [62] “Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 (General Data Protection Regulation).” *Official Journal of the European Union* L 119/1, May 2016.

Xác định các yêu cầu nonfunctional

Internet được xây dựng tốt đến mức hầu hết mọi người coi nó như một nguồn tài nguyên thiên nhiên giống như Thái Bình Dương, thay vì một thứ gì đó do con người tạo ra. Lần cuối cùng một công nghệ với quy mô như vậy hoạt động mà không có lỗi là khi nào?

Alan Kay, trong cuộc phỏng vấn với *Dr. Dobbs Journal* (2012)

Nếu bạn đang xây dựng một ứng dụng, bạn sẽ bị dẫn dắt bởi một danh sách các yêu cầu. Đứng đầu danh sách của bạn có khả năng cao nhất là các tính năng mà ứng dụng phải cung cấp: bạn cần những màn hình và nút bấm nào, và mỗi thao tác được kỳ vọng sẽ làm gì để hoàn thành mục đích của phần mềm. Đây là các *functional requirements* (yêu cầu chức năng) của bạn.

Ngoài ra, bạn có thể có các *nonfunctional requirements* (yêu cầu phi chức năng): ví dụ, ứng dụng phải nhanh, tin cậy, bảo mật, tuân thủ pháp luật và dễ bảo trì. Những yêu cầu này có thể không được viết ra một cách rõ ràng vì chúng có vẻ khá hiển nhiên, nhưng chúng cũng quan trọng tương đương với tính năng của ứng dụng; một ứng dụng chậm chạp hoặc không tin cậy đến mức không thể chịu nổi thì cũng coi như không tồn tại.

Nhiều yêu cầu nonfunctional, chẳng hạn như bảo mật, nằm ngoài phạm vi của cuốn sách này. Nhưng chúng ta sẽ xem xét một vài yêu cầu, và chương này sẽ giúp bạn diễn đạt chúng cho các hệ thống của riêng mình. Cụ thể, chúng ta sẽ xem xét các mục sau:

- Xác định và đo lường *performance* (hiệu năng) của một hệ thống
- Ý nghĩa của việc một dịch vụ có độ tin cậy (*reliable*)—cụ thể là tiếp tục hoạt động chính xác ngay cả khi có sự cố xảy ra
- Cho phép một hệ thống có khả năng mở rộng (*scalable*) bằng cách có các phương pháp hiệu quả để tăng cường năng lực tính toán khi tải trên hệ thống tăng lên
- Giúp việc bảo trì một hệ thống trong dài hạn trở nên dễ dàng hơn

Các thuật ngữ được giới thiệu trong chương này cũng sẽ hữu ích trong các chương tiếp theo, khi chúng ta đi sâu vào chi tiết cách các hệ thống data-intensive được triển khai. Tuy nhiên, các định nghĩa trừu tượng có thể khá khô khan; để làm cho các ý tưởng trở nên cụ thể hơn, chúng ta sẽ bắt đầu chương này với một nghiên cứu điển hình về một dịch vụ mạng xã hội, nơi sẽ cung cấp các ví dụ thực tế về performance và khả năng mở rộng.

Nghiên cứu điển hình: Bảng tin (Home Timelines) của mạng xã hội

Hãy tưởng tượng chúng ta được giao nhiệm vụ triển khai một mạng xã hội theo phong cách của X (trước đây là Twitter), nơi người dùng có thể đăng tin nhắn và theo dõi những người dùng khác. Đây sẽ là một sự đơn giản hóa cực lớn so với cách một dịch vụ như vậy thực sự hoạt động [1, 2, 3], nhưng nó sẽ giúp minh họa một số vấn đề phát sinh trong các hệ thống quy mô lớn.

Hãy giả định rằng người dùng thực hiện tổng cộng 500 triệu bài đăng mỗi ngày, tương đương trung bình 5.800 bài đăng mỗi giây. Thỉnh thoảng, tốc độ có thể tăng vọt lên tới 150.000 bài đăng mỗi giây [4]. Chúng ta cũng giả định rằng một người dùng trung bình theo dõi 200 người và có 200 người theo dõi (mặc dù có một phạm vi rất rộng: hầu hết mọi người chỉ có một vài người theo dõi, và một số ít người nổi tiếng, chẳng hạn như Barack Obama, có hơn 100 triệu người theo dõi).

Biểu diễn người dùng, bài đăng và quan hệ theo dõi

Chúng ta lưu trữ tất cả dữ liệu trong một relational database, như được hiển thị trong Hình 2-1. Chúng ta có một bảng cho người dùng, một bảng cho bài đăng và một bảng cho các mối quan hệ theo dõi.

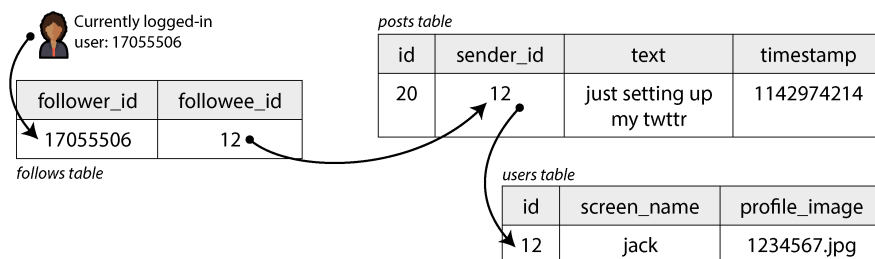


Figure 2-1. Một schema quan hệ đơn giản cho một mạng xã hội, trong đó người dùng có thể follow lẫn nhau

Hãy giả sử thao tác đọc chính mà mạng xã hội của chúng ta cần hỗ trợ là *home timeline* (dòng thời gian chính), nơi hiển thị các bài đăng gần đây của những người mà người dùng đang theo dõi (để đơn giản, chúng ta sẽ bỏ qua quảng cáo, các bài đăng được gợi ý từ những người họ không theo dõi và các phần mở rộng khác). Chúng ta có thể viết truy vấn SQL sau đây để lấy home timeline cho một người dùng cụ thể:

```

SELECT posts.*, users.* FROM posts
JOIN follows ON posts.sender_id = follows.followee_id
JOIN users ON posts.sender_id = users.id
WHERE follows.follower_id = current_user
ORDER BY posts.timestamp DESC
LIMIT 1000

```

Để thực thi truy vấn này, cơ sở dữ liệu sẽ sử dụng bảng `follows` để tìm tất cả những người mà `current_user` đang theo dõi, tra cứu các bài đăng gần đây của những người dùng đó, và sắp xếp chúng theo timestamp để lấy ra 1.000 bài đăng gần nhất từ bất kỳ người dùng nào được theo dõi.

Các bài đăng cần phải có tính kịp thời, vì vậy hãy giả sử rằng sau khi ai đó đăng một bài viết, chúng ta muốn những người theo dõi họ có thể nhìn thấy nó trong vòng năm giây. Một cách tiếp cận là client của người dùng sẽ lặp lại truy vấn trước đó sau mỗi năm giây trong khi người dùng đang trực tuyến (điều này được gọi là *polling*). Nếu chúng ta giả định có 10 triệu người dùng đang trực tuyến và đăng nhập cùng một lúc, điều đó có nghĩa là phải chạy truy vấn 2 triệu lần mỗi giây. Ngay cả khi chúng ta thực hiện polling với tần suất thấp hơn, con số này vẫn là rất lớn.

Truy vấn này cũng khá tốn kém: nếu một người dùng đang theo dõi 200 người, truy vấn cần phải lấy ra một danh sách các bài đăng gần đây của mỗi người trong số 200 người đó và hợp nhất các danh sách này lại. Hai triệu truy vấn timeline mỗi giây nhân với 200 tài khoản được theo dõi sẽ tạo ra 400 triệu lượt tra cứu mỗi giây—một con số khổng lồ. Và đó mới chỉ là trường hợp trung bình. Một số người dùng theo dõi hàng chục nghìn tài khoản; đối với họ, truy vấn này rất tốn kém để thực thi và khó có thể thực hiện nhanh chóng.

Materializing và Cập nhật Timelines

Làm thế nào để chúng ta làm tốt hơn? Thứ nhất, thay vì polling, sẽ tốt hơn nếu server chủ động đẩy các bài đăng mới tới bất kỳ người theo dõi nào hiện đang trực tuyến. Thứ hai, chúng ta nên tính toán trước (precompute) kết quả của truy vấn để yêu cầu của người dùng về home timeline có thể được phục vụ từ một cache.

Hãy tưởng tượng rằng đối với mỗi người dùng, chúng ta lưu trữ một cấu trúc dữ liệu chứa home timeline của họ (tức là các bài đăng gần đây của những người họ đang theo dõi). Mỗi khi một người dùng đăng một bài viết, chúng ta tra cứu tất cả những người theo dõi họ và chèn bài đăng đó vào home timeline của mỗi người theo dõi—giống như việc chuyển một tin nhắn vào hộp thư. Giờ đây khi một người dùng đăng nhập, chúng ta chỉ đơn giản là đưa cho họ home timeline đã được tính toán trước này. Hơn nữa, để nhận được thông báo về bất kỳ bài đăng mới nào trên timeline của mình, client của người dùng chỉ cần đăng ký (subscribe) vào stream các bài đăng đang được thêm vào home timeline của họ.

Nhược điểm của cách tiếp cận này là giờ đây chúng ta cần thực hiện nhiều việc hơn mỗi khi một người dùng đăng một bài viết, bởi vì các home timeline là dữ liệu phái sinh (derived data) cần phải được cập nhật. Quy trình này được minh họa trong Hình 2-2. Khi một yêu cầu ban đầu dẫn đến nhiều yêu cầu downstream được thực hiện, chúng ta sử dụng thuật ngữ *fan-out* để mô tả hệ số mà số lượng yêu cầu tăng lên.

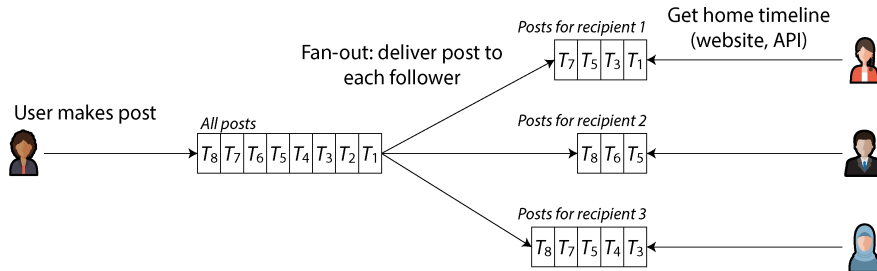


Figure 2-2. **Fan-out:** phân phối các bài đăng mới tới mọi follower của người dùng đã thực hiện bài đăng đó

Với tốc độ 5.800 bài đăng mỗi giây, nếu một bài đăng trung bình tiếp cận được 200 người theo dõi (tức là hệ số fan-out là 200), chúng ta sẽ cần thực hiện hơn 1 triệu lượt ghi vào home timeline mỗi giây. Con số này rất lớn, nhưng nó vẫn là một khoản tiết kiệm đáng kể so với 400 triệu lượt tra cứu bài đăng theo từng người gửi mỗi giây mà nếu không có phương pháp này, chúng ta sẽ phải thực hiện.

Nếu tốc độ bài đăng tăng đột biến do một sự kiện đặc biệt, chúng ta không cần phải thực hiện việc phân phối timeline ngay lập tức—chúng ta có thể đưa chúng vào hàng đợi và chấp nhận rằng các bài đăng sẽ mất thêm một chút thời gian để xuất hiện trên timeline của những người theo dõi. Ngay cả trong những đợt tăng tải như vậy, các timeline vẫn tải nhanh, vì chúng ta chỉ đơn giản là phục vụ chúng từ một cache.

Quá trình tính toán trước và cập nhật kết quả của một truy vấn này được gọi là *materialization* (hiện thực hóa), và timeline cache là một ví dụ về một *materialized view* (view đã được hiện thực hóa) (một khái niệm mà chúng ta sẽ thảo luận kỹ hơn trong các chương sau). Materialized view giúp tăng tốc độ đọc, nhưng đổi lại chúng ta phải thực hiện nhiều công việc hơn cho các lượt ghi. Chi phí ghi đối với hầu hết người dùng là không đáng kể, nhưng một mạng xã hội cũng phải xem xét một số trường hợp hợp cực đoan:

- Nếu một người dùng theo dõi một số lượng tài khoản rất lớn, và các tài khoản đó đăng bài rất nhiều, người dùng đó sẽ có tốc độ ghi cao vào materialized timeline của họ. Tuy nhiên, người dùng đó không có khả năng sẽ đọc tất cả các bài đăng trong timeline của mình, vì vậy việc đơn giản là loại bỏ một số lượt ghi vào timeline của họ và chỉ hiển thị cho người dùng một mẫu các bài đăng từ các tài khoản mà họ đang theo dõi là hoàn toàn ổn [5].

- Khi một tài khoản người nổi tiếng với số lượng người theo dõi rất lớn thực hiện một bài đăng, chúng ta phải thực hiện rất nhiều công việc để chèn bài đăng đó vào home timeline của mỗi người trong số hàng triệu người theo dõi họ. Trong trường hợp này, việc loại bỏ một số lượt ghi đó là không thể chấp nhận được. Một cách để giải quyết vấn đề này là xử lý các bài đăng của người nổi tiếng tách biệt với bài đăng của những người khác: chúng ta có thể tiết kiệm công sức thêm các bài đăng của người nổi tiếng vào hàng triệu timeline bằng cách lưu trữ chúng riêng biệt và hợp nhất chúng với materialized timeline khi nó được đọc. Bất chấp những tối ưu hóa như vậy, việc xử lý người nổi tiếng trên một mạng xã hội có thể đòi hỏi rất nhiều hạ tầng [6].

Mô tả Hiệu năng

Hầu hết các cuộc thảo luận về hiệu năng phần mềm đều xem xét hai loại metric chính:

Response time

Thời gian trôi qua từ thời điểm người dùng thực hiện một yêu cầu cho đến khi họ nhận được câu trả lời được yêu cầu. Đơn vị đo lường là giây (hoặc mili giây, hoặc micro giây).

Throughput

Số lượng yêu cầu mỗi giây, hoặc khối lượng dữ liệu mỗi giây, mà hệ thống đang xử lý. Đối với một sự phân bổ tài nguyên phần cứng nhất định, sẽ có một *maximum throughput* (throughput tối đa) có thể xử lý được. Đơn vị đo lường là "cái gì đó mỗi giây".

Trong nghiên cứu tình huống về mạng xã hội, "số bài đăng mỗi giây" và "số lượt ghi timeline mỗi giây" là các metric về throughput, trong khi "thời gian để tải home timeline" và "thời gian cho đến khi một bài đăng được phân phối tới người theo dõi" là các metric về response time.

Throughput và response time thường có liên quan đến nhau. Một ví dụ về mối quan hệ như vậy đối với một dịch vụ trực tuyến được phác thảo trong Hình 2-3. Dịch vụ có response time thấp khi throughput của yêu cầu thấp, nhưng response time sẽ tăng lên khi tải tăng lên. Điều này là do *queueing* (xếp hàng): khi một yêu cầu đến một hệ thống đang bị tải cao, CPU có khả năng đã đang trong quá trình xử lý một yêu cầu trước đó, và do đó yêu cầu đang đến cần phải chờ cho đến khi yêu cầu trước đó được hoàn tất. Khi throughput tiến gần đến mức tối đa mà phần cứng có thể xử lý, độ trễ xếp hàng sẽ tăng mạnh.

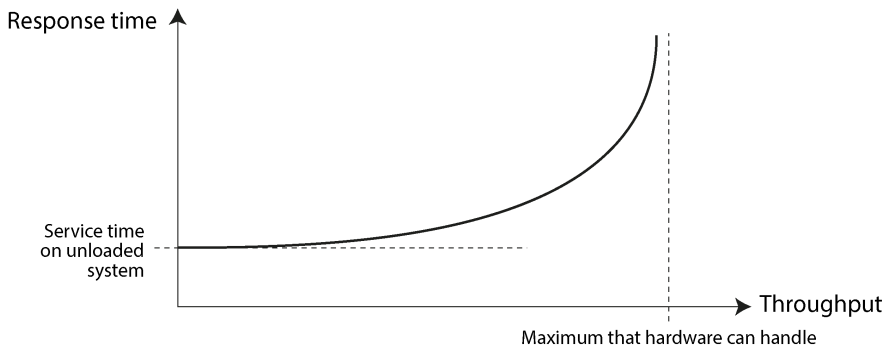


Figure 2-3. *Khi throughput của một service tiến gần đến giới hạn khả năng của nó, response time sẽ tăng lên đột ngột do hiện tượng queuing.*

Khi một hệ thống bị quá tải không thể phục hồi

Nếu một hệ thống đang tiến gần đến trạng thái quá tải, với throughput bị đẩy lên sát giới hạn, đôi khi nó có thể rơi vào một vòng xoáy luẩn quẩn, nơi nó trở nên kém hiệu quả hơn và do đó lại càng bị quá tải hơn. Ví dụ, nếu một hàng đợi request dài đang chờ được xử lý, response time có thể tăng lên nhiều đến mức các client bị timeout và gửi lại các request của chúng. Điều này khiến tốc độ request tăng lên mạnh mẽ hơn nữa, làm cho vấn đề trở nên tồi tệ hơn—gọi là một *retry storm* (cơn bão thử lại). Ngay cả khi tải đã giảm xuống, một hệ thống như vậy có thể vẫn duy trì trạng thái quá tải cho đến khi nó được khởi động lại hoặc được reset bằng cách khác. Hiện tượng này được gọi là *metastable failure* (lỗi siêu ổn định), và nó có thể gây ra các sự cố nghiêm trọng trong các hệ thống production [7, 8, 9].

Để tránh việc các lần thử lại làm quá tải một service, bạn có thể tăng và ngẫu nhiên hóa khoảng thời gian giữa các lần thử lại liên tiếp ở phía client (*exponential backoff* [10, 11]) và tạm thời ngừng gửi các request tới một service vừa trả về lỗi hoặc vừa bị timeout gần đây (bằng cách sử dụng *circuit breaker* [12, 13] hoặc thuật toán *token bucket* [14]). Server cũng có thể phát hiện khi nó đang tiến gần đến ngưỡng quá tải và bắt đầu chủ động từ chối các request (*load shedding* [15]), hoặc gửi lại các phản hồi yêu cầu client giảm tốc độ (*backpressure* [1, 16]). Việc lựa chọn các thuật toán queuing và load balancing cũng có thể tạo ra sự khác biệt [17].

Về các chỉ số hiệu năng, response time thường là điều mà người dùng quan tâm nhất, trong khi throughput quyết định các tài nguyên tính toán cần thiết (ví dụ: bạn cần bao nhiêu server) và do đó quyết định chi phí để phục vụ một workload cụ thể. Nếu throughput có khả năng tăng vượt quá khả năng của phần cứng hiện tại, thì cần phải

mở rộng năng lực; một hệ thống được gọi là *scalable* (có khả năng mở rộng) nếu throughput tối đa của nó có thể được tăng lên đáng kể bằng cách thêm các tài nguyên tính toán.

Trong mục này, chúng ta sẽ tập trung chủ yếu vào response time, và chúng ta sẽ quay lại với throughput và scalability trong mục “Scalability”.

Latency và Response Time

“Latency” và “response time” đôi khi được sử dụng thay thế cho nhau, nhưng trong cuốn sách này, chúng ta sẽ sử dụng chúng cùng một vài thuật ngữ liên quan theo một cách cụ thể (được minh họa trong Hình 2-4):

- *Response time* là những gì client nhìn thấy; nó bao gồm tất cả các độ trễ phát sinh ở bất kỳ đâu trong hệ thống.
- *Service time* là khoảng thời gian mà service đang tích cực xử lý request của client.
- *Queueing delays* có thể xảy ra tại nhiều điểm trong luồng xử lý—ví dụ, sau khi một request được tiếp nhận, nó có thể cần phải chờ cho đến khi một CPU sẵn sàng trước khi có thể được xử lý, hoặc một gói tin phản hồi có thể cần được đệm trước khi nó được gửi qua mạng nếu các tác vụ khác trên cùng một máy đang gửi rất nhiều dữ liệu qua giao diện mạng đầu ra (outbound network interface).
- *Latency* (độ trễ) là một thuật ngữ bao quát dùng để chỉ khoảng thời gian mà một request không được xử lý một cách chủ động—nghĩa là, khoảng thời gian mà nó ở trạng thái *latent* (tiềm tàng). Cụ thể hơn, *network latency* (độ trễ mạng) hoặc *network delay* (trễ mạng) đề cập đến thời gian mà một request và phản hồi mất để di chuyển qua mạng.

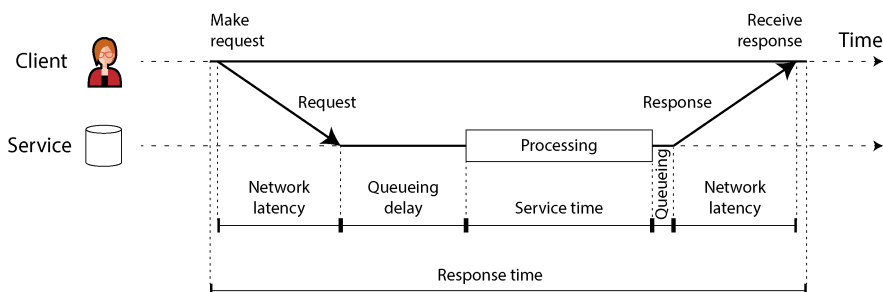


Figure 2-4. *Response time, service time, network latency và queueing delay*

Trong Hình 2-4, thời gian trôi từ trái sang phải; mỗi node giao tiếp được hiển thị dưới dạng một đường ngang, và một thông điệp request hoặc response được hiển thị dưới dạng một mũi tên chéo dày từ node này sang node khác. Bạn sẽ gặp phong cách sơ đồ này thường xuyên trong suốt cuốn sách này.

Response time có thể thay đổi đáng kể giữa các request kế tiếp nhau, ngay cả khi bạn thực hiện cùng một request lặp đi lặp lại nhiều lần. Nhiều yếu tố có thể gây ra các độ trễ ngẫu nhiên—ví dụ, một context switch sang một process chạy ngầm, việc mất một packet mạng và TCP retransmission, một đợt garbage collection pause, một page fault buộc phải đọc từ đĩa, hoặc các rung động cơ học trong server rack [18]. Chúng ta sẽ thảo luận chủ đề này chi tiết hơn trong mục “Timeouts and Unbounded Delays”.

Queueing delay thường chiếm một phần lớn sự biến động trong response time. Vì một server chỉ có thể xử lý một số lượng nhỏ các tác vụ song song (ví dụ, bị giới hạn bởi số lượng CPU core của nó), nên chỉ cần một số ít các request chậm là đủ để làm đình trệ việc xử lý các request tiếp theo—một hiệu ứng được gọi là *head-of-line blocking*. Ngay cả khi các request tiếp theo đó có service time nhanh, client vẫn sẽ thấy một response time tổng thể chậm do thời gian chờ đợi request trước đó hoàn tất. Queueing delay không phải là một phần của service time, và vì lý do này, việc đo lường response time ở phía client là rất quan trọng.

Trung bình, Trung vị và Percentile

Vì response time thay đổi từ request này sang request khác, chúng ta cần coi nó không phải là một con số đơn lẻ, mà là một *distribution* (phân phối) các giá trị mà chúng ta có thể đo lường. Trong Hình 2-5, mỗi thanh màu xám đại diện cho một request tới một service, và chiều cao của nó cho biết request đó mất bao lâu. Hầu hết các request đều khá nhanh, nhưng thỉnh thoảng có những *outlier* (giá trị ngoại lệ) mất nhiều thời gian hơn nhiều. Sự biến động trong độ trễ mạng còn được gọi là *jitter*.

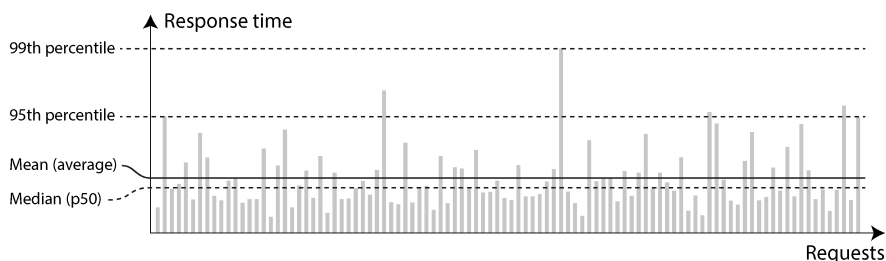


Figure 2-5. Minh họa giá trị trung bình và các phân vị (percentiles); thời gian phản hồi cho một mẫu gồm 100 yêu cầu gửi tới một service

Việc báo cáo thời gian phản hồi *trung bình* của một service là điều phổ biến (về mặt kỹ thuật, đó là *số trung bình cộng*, được tính bằng cách cộng tất cả các thời gian phản hồi rồi chia cho số lượng yêu cầu). Thời gian phản hồi trung bình hữu ích để ước tính các giới hạn về throughput [19]. Tuy nhiên, số trung bình không phải là một

metric tốt nếu bạn muốn biết thời gian phản hồi "điển hình" của mình, bởi vì nó không cho bạn biết thực tế có bao nhiêu người dùng đã trải qua mức độ trễ đó.

Thông thường, sử dụng *percentiles* (phân vị) sẽ tốt hơn. Nếu bạn lấy danh sách các thời gian phản hồi và sắp xếp chúng từ nhanh nhất đến chậm nhất, thì *median* (trung vị) là điểm nằm ở giữa—ví dụ, nếu median của thời gian phản hồi là 200 ms, điều đó có nghĩa là một nửa số yêu cầu của bạn trả về trong ít hơn 200 miligiây (ms), và một nửa số yêu cầu mất nhiều thời gian hơn. Điều này khiến median trở thành một metric tốt nếu bạn muốn biết người dùng thường phải chờ đợi trong bao lâu. Median còn được gọi là *50th percentile*, đôi khi được viết tắt là *p50*.

Để tìm hiểu xem các outlier (giá trị ngoại lệ) của bạn tệ đến mức nào, bạn có thể xem xét các percentile cao hơn: các *95th*, *99th*, và *99.9th percentiles* là những giá trị phổ biến (viết tắt là *p95*, *p99*, và *p999*). Ví dụ, nếu percentile thứ 95 của thời gian phản hồi là 1,5 giây, điều đó có nghĩa là 95 trên 100 yêu cầu mất ít hơn 1,5 giây, và 5 trên 100 yêu cầu mất từ 1,5 giây trở lên. Điều này được minh họa trong Hình 2-5.

Các percentile có thời gian phản hồi cao, còn được gọi là *tail latencies* (độ trễ đuôi), rất quan trọng vì chúng ảnh hưởng trực tiếp đến trải nghiệm của người dùng đối với service. Ví dụ, Amazon mô tả các yêu cầu về thời gian phản hồi cho các service nội bộ dựa trên percentile thứ 99,9, mặc dù điều này chỉ ảnh hưởng đến 1 trong 1.000 yêu cầu. Điều này là do những khách hàng có các yêu cầu chậm nhất thường là những người có nhiều dữ liệu nhất trong tài khoản của họ, vì họ đã thực hiện nhiều giao dịch mua hàng—tức là, họ là những khách hàng giá trị nhất [20]. Việc giữ cho những khách hàng đó hài lòng bằng cách đảm bảo website hoạt động nhanh chóng đối với họ là rất quan trọng.

Việc tối ưu hóa percentile thứ 99,99 (1 trong 10.000 yêu cầu chậm nhất) được coi là quá tốn kém và được thấy là không mang lại đủ lợi ích cho các mục tiêu của Amazon. Việc giảm thời gian phản hồi ở các percentile rất cao là rất khó khăn vì chúng dễ bị ảnh hưởng bởi các sự kiện ngẫu nhiên nằm ngoài tầm kiểm soát của bạn, và lợi ích thu được sẽ giảm dần.

Tác động của Thời gian Phản hồi đối với Người dùng

Có vẻ hiển nhiên rằng một service nhanh sẽ tốt hơn cho người dùng so với một service chậm [21]. Tuy nhiên, việc thu thập dữ liệu tin cậy để định lượng ảnh hưởng của latency đối với hành vi người dùng lại khó khăn một cách đáng ngạc nhiên.

Một số thống kê thường được trích dẫn lại không đáng tin cậy. Ví dụ, vào năm 2006, Google đã báo cáo rằng việc kết quả tìm kiếm bị chậm từ 400 ms xuống 900 ms có liên quan đến việc giảm 20% lưu lượng truy cập và doanh thu [22]. Tuy nhiên, một nghiên cứu khác của Google từ năm 2009 báo cáo rằng việc tăng 400 ms latency chỉ dẫn đến số lượng tìm kiếm mỗi ngày giảm 0,6% [23], và trong cùng năm đó, Bing nhận thấy rằng việc tăng hai giây thời gian tải đã làm giảm 4,3% doanh thu quảng cáo [24]. Các dữ liệu mới hơn từ những công ty này dường như không được công khai.

Một nghiên cứu gần đây hơn của Akamai tuyên bố rằng việc tăng 100 ms thời gian phản hồi đã làm giảm tỷ lệ chuyển đổi của các trang thương mại điện tử lên đến 7% [25]; tuy nhiên, khi xem xét kỹ hơn, chính nghiên cứu đó lại tiết lộ rằng thời gian tải trang rất *nhẹ* cũng tương quan với tỷ lệ chuyển đổi thấp hơn! Kết quả có vẻ nghịch lý này được giải thích bởi thực tế là những trang tải nhanh nhất thường là những trang không có nội dung hữu ích (ví dụ: các trang lỗi 404). Tuy nhiên, vì nghiên cứu không thực hiện nỗ lực nào để tách biệt ảnh hưởng của nội dung trang khỏi ảnh hưởng của thời gian tải, nên kết quả của nó có lẽ không có ý nghĩa.

Một nghiên cứu của Yahoo thực hiện vào năm sau đó đã so sánh tỷ lệ nhấp (click-through rates) giữa các kết quả tìm kiếm tải nhanh so với tải chậm, sau khi đã kiểm soát chất lượng của các kết quả tìm kiếm [26]. Nghiên cứu báo cáo rằng có nhiều lượt nhấp hơn từ 20%–30% đối với các tìm kiếm nhanh khi sự khác biệt giữa phản hồi nhanh và chậm là 1,25 giây hoặc nhiều hơn.

Sử dụng các metric Thời gian Phản hồi

Các percentile cao đặc biệt quan trọng trong các backend service được gọi nhiều lần như một phần của việc phục vụ một yêu cầu duy nhất từ người dùng cuối. Ngay cả khi bạn thực hiện các cuộc gọi song song, yêu cầu vẫn cần phải chờ cuộc gọi chậm nhất trong số các cuộc gọi song song đó hoàn tất. Chỉ cần một cuộc gọi chậm là đủ để khiến toàn bộ yêu cầu của người dùng cuối trở nên chậm, như được minh họa trong Hình 2-6. Ngay cả khi chỉ một tỷ lệ nhỏ các cuộc gọi backend bị chậm, xác suất gặp phải một cuộc gọi chậm sẽ tăng lên nếu một yêu cầu của người dùng cuối đòi hỏi nhiều cuộc gọi backend, do đó một tỷ lệ cao hơn các yêu cầu người dùng cuối như vậy cuối cùng sẽ bị chậm (một hiệu ứng được gọi là *tail latency amplification* [27]).

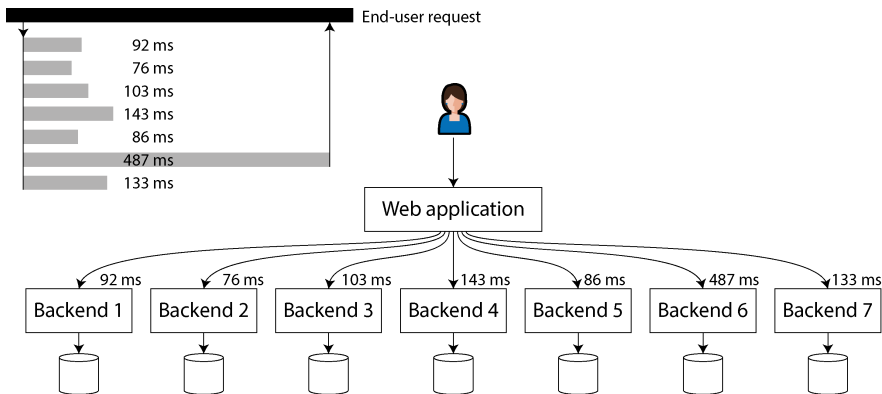


Figure 2-6. Khi cần thực hiện nhiều cuộc gọi backend để phục vụ một yêu cầu, chỉ cần một cuộc gọi chậm duy nhất cũng có thể làm chậm toàn bộ yêu cầu của người dùng cuối.

Percentile (phân vị) thường được sử dụng trong các *service level objectives* (SLO) và *service level agreements* (SLA) như những cách để xác định hiệu năng và độ khả dụng dự kiến của một dịch vụ [28]. Ví dụ, một SLO có thể đặt mục tiêu cho một dịch vụ có thời gian phản hồi trung vị (median) dưới 200 ms và phân vị thứ 99 (99th percentile) dưới 1 giây, cùng với mục tiêu là ít nhất 99,9% các yêu cầu hợp lệ sẽ trả về các phản hồi không có lỗi. Một SLA là một bản hợp đồng quy định điều gì sẽ xảy ra nếu SLO không được đáp ứng (ví dụ, khách hàng có thể được quyền hoàn tiền). Đó là ý tưởng cơ bản; trong thực tế, việc xác định các chỉ số độ khả dụng tốt cho SLO và SLA không hề đơn giản [29, 30].

Tính toán Percentile

Nếu bạn muốn thêm các percentile thời gian phản hồi vào các dashboard giám sát cho các dịch vụ của mình, bạn cần tính toán chúng một cách hiệu quả và liên tục. Ví dụ, bạn có thể muốn duy trì một rolling window (cửa sổ trượt) các thời gian phản hồi cho các yêu cầu trong 10 phút qua. Mỗi phút, bạn tính toán giá trị trung vị và các percentile khác nhau dựa trên các giá trị trong cửa sổ đó rồi vẽ các chỉ số này lên một biểu đồ.

Cách triển khai đơn giản nhất là duy trì một danh sách các thời gian phản hồi cho tất cả các yêu cầu trong cửa sổ thời gian và sắp xếp danh sách đó mỗi phút. Nếu việc đó quá kém hiệu quả đối với bạn, có những thuật toán có thể tính toán một giá trị xấp xỉ tốt cho các percentile với chi phí CPU và bộ nhớ tối thiểu. Các thư viện ước tính percentile mã nguồn mở bao gồm HdrHistogram [31], t-digest [32, 33], OpenHistogram [34], và DDSketch [35].

Hãy lưu ý rằng việc tính trung bình các percentile (ví dụ, để giảm độ phân giải thời gian hoặc để kết hợp dữ liệu từ nhiều máy chủ) là vô nghĩa về mặt toán học. Cách đúng đắn để tổng hợp dữ liệu thời gian phản hồi là cộng các histogram [36].

Độ tin cậy và Khả năng chịu lỗi

Mọi người đều có một ý niệm trực quan về việc một thứ gì đó là tin cậy hay không tin cậy. Đối với phần mềm, các kỳ vọng điển hình bao gồm những điều sau:

- Ứng dụng thực hiện đúng chức năng mà người dùng mong đợi.
- Ứng dụng có thể chịu đựng được việc người dùng mắc lỗi hoặc sử dụng phần mềm theo những cách không mong muốn.
- Hiệu năng của nó đủ tốt cho trường hợp sử dụng yêu cầu, dưới tải trọng và khối lượng dữ liệu dự kiến.
- Hệ thống ngăn chặn mọi truy cập trái phép và sự lạm dụng.

Nếu tất cả những điều đó cùng nhau có nghĩa là "hoạt động chính xác", thì chúng ta có thể hiểu *độ tin cậy* (reliability) có nghĩa là, về cơ bản, "tiếp tục hoạt động chính xác, ngay cả khi có sự cố xảy ra". Để chính xác hơn về việc các sự cố xảy ra, chúng ta sẽ phân biệt giữa fault và failure [37, 38, 39]:

Fault

Một fault xảy ra khi một *phần* cụ thể của hệ thống ngừng hoạt động chính xác—ví dụ, nếu một ổ cứng duy nhất bị trục trặc, hoặc một máy chủ duy nhất bị crash, hoặc một dịch vụ bên ngoài (mà hệ thống phụ thuộc vào) bị gián đoạn.

Failure

Một failure xảy ra khi *toàn bộ* hệ thống ngừng cung cấp dịch vụ yêu cầu cho người dùng—nói cách khác, khi nó không đáp ứng được SLO.

Sự phân biệt giữa fault và failure có thể gây nhầm lẫn vì chúng là cùng một thứ, chỉ khác nhau ở các cấp độ. Ví dụ, nếu một ổ cứng ngừng hoạt động, chúng ta nói rằng ổ cứng đó đã bị failure; nếu hệ thống chỉ bao gồm duy nhất ổ cứng đó, thì nó đã ngừng cung cấp dịch vụ yêu cầu và do đó cũng bị failure. Tuy nhiên, nếu hệ thống bao gồm nhiều ổ cứng, sự failure của một ổ cứng duy nhất chỉ là một fault từ góc nhìn của hệ thống lớn hơn, và hệ thống lớn hơn có thể chịu đựng được fault đó bằng cách có một bản sao dữ liệu trên một ổ cứng khác.

Khả năng chịu lỗi

Chúng ta gọi một hệ thống là *fault-tolerant* (có khả năng chịu lỗi) nếu nó tiếp tục cung cấp dịch vụ yêu cầu cho người dùng bất chấp một số fault nhất định xảy ra. Nếu một hệ thống không thể chịu đựng được việc một phần cụ thể trở nên bị lỗi, chúng ta gọi phần đó là *single point of failure* (SPOF - điểm yếu duy nhất), bởi vì một fault ở phần đó sẽ leo thang gây ra failure cho toàn bộ hệ thống.

Ví dụ, trong nghiên cứu tình huống về mạng xã hội, một lỗi có thể xảy ra là trong quá trình fan-out, một máy chủ tham gia vào việc cập nhật các *materialized timelines* bị crash hoặc trở nên không khả dụng. Để làm cho quá trình này có khả năng chịu lỗi (fault-tolerant), chúng ta cần đảm bảo rằng một máy chủ khác có thể tiếp quản nhiệm vụ này mà không bỏ lỡ bất kỳ bài đăng nào đáng lẽ phải được chuyển đến, và không làm trùng lặp bất kỳ bài đăng nào. (Ý tưởng này được gọi là *exactly-once semantics* (ngữ nghĩa đúng một lần), và chúng ta sẽ xem xét chi tiết nó trong Chương 12.)

Khả năng chịu lỗi luôn bị giới hạn ở một số lượng nhất định các loại lỗi cụ thể. Ví dụ, một hệ thống có thể có khả năng chịu đựng tối đa hai ổ cứng bị lỗi cùng một lúc, hoặc tối đa một trong ba node bị crash. Việc chịu đựng bất kỳ số lượng lỗi nào là điều không hợp lý; nếu tất cả các node đều crash, thì không thể làm gì được nữa. Nếu toàn bộ hành tinh Trái Đất (và tất cả các máy chủ trên đó) bị một lỗ đen nuốt chửng, thì việc chịu đựng lỗi đó sẽ đòi hỏi phải đặt web hosting trong không gian—chúc bạn may mắn khi xin được phê duyệt khoản ngân sách đó.

Ngược với trực giác, trong các hệ thống chịu lỗi như vậy, việc *tăng* tỷ lệ lỗi bằng cách cố tình kích hoạt chúng có thể là một điều hợp lý—ví dụ, bằng cách ngẫu nhiên kill các process riêng lẻ mà không báo trước. Điều này được gọi là *fault injection* (tiêm lỗi). Nhiều lỗi (bug) nghiêm trọng thực chất là do xử lý lỗi kém [40]; bằng cách cố tình gây ra lỗi, bạn đảm bảo rằng bộ máy chịu lỗi liên tục được vận hành và kiểm tra, điều này có thể làm tăng sự tin cậy của bạn rằng các lỗi sẽ được xử lý chính xác khi

chúng xảy ra một cách tự nhiên. *Chaos engineering* là một kỹ thuật nhằm cải thiện độ tin cậy vào các cơ chế chịu lỗi thông qua các thử nghiệm như cố tình tiêm lỗi [41].

Mặc dù chúng ta thường ưu tiên việc chịu đựng lỗi hơn là ngăn ngừa lỗi, nhưng trong một số trường hợp, việc ngăn ngừa sẽ tốt hơn là cứu chữa (ví dụ, vì không có cách cứu chữa nào tồn tại). Đây là trường hợp của các vấn đề bảo mật, chẳng hạn; nếu một kẻ tấn công đã xâm nhập hệ thống và giành được quyền truy cập vào dữ liệu nhạy cảm, sự kiện đó không thể đảo ngược. Tuy nhiên, cuốn sách này chủ yếu giải quyết các loại lỗi có thể cứu chữa được, như được mô tả trong các mục sau.

Lỗi Phần cứng và Phần mềm

Khi chúng ta nghĩ về các nguyên nhân gây ra sự cố hệ thống, các lỗi phần cứng nhanh chóng hiện lên trong tâm trí:

- Khoảng 2%–5% ổ cứng từ tính bị lỗi mỗi năm [42, 43]; do đó, trong một storage cluster với 10.000 đĩa, chúng ta nên kỳ vọng trung bình có một đĩa bị lỗi mỗi ngày. Dữ liệu gần đây cho thấy các đĩa cứng đang trở nên đáng tin cậy hơn, nhưng tỷ lệ lỗi vẫn ở mức đáng kể [44].
- Khoảng 0,5%–1% ổ cứng thể rắn (SSD) bị lỗi mỗi năm [45]. Một số lượng nhỏ các lỗi bit được sửa chữa tự động [46], nhưng các lỗi không thể sửa chữa xảy ra khoảng một lần mỗi năm trên mỗi ổ đĩa, ngay cả đối với những ổ đĩa còn khá mới (tức là, chúng ít bị hao mòn). Tỷ lệ lỗi này cao hơn so với ổ cứng từ tính [47, 48].
- Các thành phần phần cứng khác (chẳng hạn như bộ nguồn, bộ điều khiển RAID và các module bộ nhớ) cũng bị lỗi, mặc dù ít thường xuyên hơn so với ổ cứng [49, 50].
- Khoảng 1 trong 1.000 máy chủ có một CPU core thỉnh thoảng tính toán sai kết quả, có khả năng là do các lỗi sản xuất [51, 52, 53]. Trong một số trường hợp, một phép tính sai dẫn đến crash, nhưng trong các trường hợp khác, nó chỉ khiến một chương trình trả về kết quả sai.
- Dữ liệu trong RAM có thể bị hỏng, do các sự kiện ngẫu nhiên như tia vũ trụ hoặc do các lỗi vật lý vĩnh viễn. Ngay cả khi sử dụng bộ nhớ có mã sửa lỗi (ECC), hơn 1% máy chủ vẫn gặp phải lỗi không thể sửa chữa trong một năm nhất định, điều này thường dẫn đến việc máy chủ bị crash và module bộ nhớ bị ảnh hưởng cần phải được thay thế [54]. Hơn nữa, một số mẫu truy cập bộ nhớ bệnh lý nhất định có thể làm đảo bit với xác suất cao [55].
- Toàn bộ một datacenter có thể trở nên không khả dụng (ví dụ: do mất điện hoặc cấu hình sai network) hoặc thậm chí bị phá hủy vĩnh viễn (ví dụ: do hỏa hoạn, lũ lụt hoặc động đất [56]). Một cơn bão mặt trời, thứ gây ra các dòng điện lớn trong các dây dẫn đường dài khi mặt trời phun ra một khối lớn các hạt tích điện, có thể làm hỏng lưới điện và các cáp network dưới biển [57]. Mặc dù những lỗi quy mô

lớn như vậy hiếm khi xảy ra, nhưng tác động của chúng có thể rất thảm khốc nếu một dịch vụ không thể chịu đựng được việc mất đi một datacenter [58].

Những sự kiện này đủ hiếm để bạn thường không cần lo lắng về chúng khi làm việc với một hệ thống nhỏ, miễn là bạn có thể dễ dàng thay thế các phần cứng bị lỗi. Tuy nhiên, trong một hệ thống quy mô lớn, các lỗi phần cứng xảy ra đủ thường xuyên để chúng trở thành một phần của hoạt động hệ thống bình thường.

Chịu lỗi phần cứng thông qua redundancy

Phản ứng đầu tiên của chúng ta đối với phần cứng không tin cậy thường là thêm redundancy vào các thành phần phần cứng riêng lẻ nhằm giảm tỷ lệ lỗi của hệ thống. Các ổ đĩa có thể được thiết lập trong cấu hình RAID (phân tán dữ liệu trên nhiều ổ đĩa trong cùng một máy để một ổ đĩa bị lỗi không gây mất dữ liệu), các server có thể có bộ nguồn kép và CPU hot-swappable, và các datacenter có thể có pin cùng máy phát điện diesel để dự phòng nguồn điện. Sự redundancy như vậy thường có thể giữ cho một máy chạy liên tục trong nhiều năm.

Redundancy hiệu quả nhất khi các lỗi thành phần là độc lập—tức là khi sự xảy ra của một lỗi không làm thay đổi khả năng xảy ra của một lỗi khác. Tuy nhiên, kinh nghiệm đã chỉ ra những mối tương quan đáng kể giữa các lỗi thành phần [43, 59, 60]. Tình trạng không khả dụng của cả một rack server hoặc cả một datacenter vẫn xảy ra thường xuyên hơn mức chúng ta mong muốn.

Redundancy phần cứng làm tăng thời gian uptime của một máy đơn lẻ; tuy nhiên, như đã thảo luận trong mục “Distributed Versus Single-Node Systems”, việc sử dụng một distributed system có những lợi thế, chẳng hạn như khả năng chịu đựng việc một datacenter bị ngắt kết nối hoàn toàn. Vì lý do này, các cloud system có xu hướng ít tập trung vào độ tin cậy của từng máy riêng lẻ mà thay vào đó hướng tới việc làm cho các dịch vụ có tính sẵn sàng cao bằng cách chịu lỗi các node ở cấp độ phần mềm. Các nhà cung cấp cloud sử dụng *availability zones* để xác định các tài nguyên nào nằm cùng một vị trí vật lý; các tài nguyên ở cùng một nơi có khả năng bị lỗi cùng lúc cao hơn so với các tài nguyên tách biệt về mặt địa lý.

Các kỹ thuật fault-tolerance mà chúng ta thảo luận trong cuốn sách này được thiết kế để chịu đựng việc mất đi toàn bộ máy, rack, hoặc các availability zones. Chúng thường hoạt động bằng cách cho phép một máy ở datacenter này tiếp quản khi một máy ở datacenter khác bị lỗi hoặc không thể kết nối được. Chúng ta sẽ thảo luận về các kỹ thuật fault tolerance như vậy trong các Chương 6, 10 và nhiều điểm khác trong cuốn sách này.

Các hệ thống có thể chịu đựng việc mất đi toàn bộ máy cũng có những lợi thế về vận hành. Một hệ thống single-server yêu cầu thời gian downtime có kế hoạch nếu bạn cần khởi động lại máy (ví dụ: để áp dụng các bản vá bảo mật của hệ điều hành), trong khi một hệ thống multi-node fault-tolerant có thể được vá lỗi bằng cách khởi động

lại từng node một, mà không ảnh hưởng đến dịch vụ đối với người dùng. Điều này được gọi là *rolling upgrade*, và chúng ta sẽ thảo luận thêm về nó trong Chương 5.

Lỗi phần mềm

Mặc dù các lỗi phần cứng có thể tương quan yếu, chúng vẫn chủ yếu là độc lập—ví dụ, nếu một ổ đĩa bị lỗi, các ổ đĩa khác trong cùng một máy có khả năng vẫn ổn, ít nhất là trong một khoảng thời gian. Mặt khác, các lỗi phần mềm thường có tương quan rất cao, bởi vì việc nhiều node chạy cùng một phần mềm và do đó có cùng các bug là điều phổ biến [61, 62]. Những lỗi như vậy khó dự đoán hơn, và chúng có xu hướng gây ra nhiều lỗi hệ thống hơn so với các lỗi phần cứng không tương quan [49]. Các ví dụ bao gồm sau đây:

- Một lỗi phần mềm khiến mọi node đều gặp lỗi cùng một lúc trong những điều kiện cụ thể. Ví dụ, vào ngày 30 tháng 6 năm 2012, một giây nhuận (leap second) đã khiến nhiều ứng dụng Java bị treo đồng thời do một lỗi trong Linux kernel, làm sập nhiều dịch vụ internet [63]. Và do một lỗi firmware, tất cả SSD của một số model nhất định đột ngột bị lỗi sau chính xác 32.768 giờ hoạt động (chưa đầy bốn năm), khiến dữ liệu trên chúng không thể khôi phục được [64].
- Một tiến trình chạy quá mức (runaway process) sử dụng hết một tài nguyên dùng chung và có hạn, chẳng hạn như thời gian CPU, bộ nhớ, dung lượng đĩa, băng thông mạng, hoặc các thread [65]. Ví dụ, một tiến trình tiêu thụ quá nhiều bộ nhớ trong khi xử lý một request lớn có thể bị hệ điều hành kill, hoặc một lỗi trong một client library có thể gây ra lưu lượng request cao hơn nhiều so với dự kiến [66].
- Một service mà hệ thống phụ thuộc vào bị chậm lại, trở nên không phản hồi, hoặc bắt đầu trả về các phản hồi bị lỗi (corrupted).
- Sự tương tác giữa các hệ thống khác nhau dẫn đến hành vi emergent (hành vi mới phát sinh) mà không xảy ra khi mỗi hệ thống được kiểm thử riêng biệt [67].
- Các lỗi cascading (lỗi dây chuyền), nơi một vấn đề trong một thành phần khiến thành phần khác bị quá tải và chậm lại, từ đó lại làm sập một thành phần khác [68, 69].

Các lỗi phần mềm gây ra những loại fault này thường nằm tiềm ẩn trong một thời gian dài cho đến khi chúng bị kích hoạt bởi một tập hợp các điều kiện bất thường. Trong những điều kiện đó, người ta nhận thấy rằng phần mềm đang đưa ra một loại giả định nào đó về môi trường của nó—và mặc dù giả định đó *thông thường* là đúng, nhưng cuối cùng nó lại không còn đúng vì một lý do nào đó [70, 71].

Vấn đề về các lỗi mang tính hệ thống trong phần mềm không có giải pháp nhanh chóng. Rất nhiều việc nhỏ có thể giúp ích: suy nghĩ cẩn thận về các giả định và sự tương tác trong hệ thống; kiểm thử kỹ lưỡng; đảm bảo cô lập tiến trình; cho phép các

tiến trình crash và khởi động lại; tránh các vòng lặp phản hồi như retry storms (xem “Khi một hệ thống quá tải không thể phục hồi”); đo lường, giám sát và phân tích hành vi hệ thống trong production.

Con người và độ tin cậy

Con người thiết kế và xây dựng các hệ thống phần mềm, và những người vận hành để giữ cho các hệ thống hoạt động cũng là con người. Không giống như máy móc, con người không chỉ tuân theo các quy tắc; một trong những thế mạnh của họ là sự sáng tạo và khả năng thích nghi để hoàn thành công việc. Tuy nhiên, đặc điểm này cũng dẫn đến sự khó đoán, và đôi khi dẫn đến những sai lầm có thể gây ra lỗi, bất chấp những ý định tốt nhất. Ví dụ, một nghiên cứu về các dịch vụ internet lớn đã thấy rằng các thay đổi cấu hình bởi những người vận hành là nguyên nhân hàng đầu gây ra các đợt outage, trong khi các lỗi phần cứng (máy chủ hoặc mạng) chỉ đóng vai trò trong 10%–25% các trường hợp [72].

Thật dễ dàng khi dán nhãn những vấn đề như vậy là “lỗi con người” và mong muốn rằng chúng có thể được giải quyết bằng cách kiểm soát hành vi con người tốt hơn thông qua các quy trình chặt chẽ hơn và việc tuân thủ các quy tắc. Tuy nhiên, việc đổ lỗi cho con người về những sai lầm là phản tác dụng. Những gì chúng ta gọi là “lỗi con người” thực chất không phải là nguyên nhân của một sự cố, mà đúng hơn là một triệu chứng của vấn đề trong hệ thống sociotechnical (kỹ thuật - xã hội) nơi con người đang cố gắng hết sức để làm tốt công việc của mình [73]. Thông thường, các hệ thống phức tạp có hành vi emergent, trong đó các tương tác không mong đợi giữa các thành phần cũng có thể dẫn đến lỗi [74].

Nhiều biện pháp kỹ thuật khác nhau có thể giúp giảm thiểu tác động của sai lầm con người, bao gồm kiểm thử kỹ lưỡng (cả kiểm thử viết tay và *property testing* trên nhiều đầu vào ngẫu nhiên) [40], các cơ chế rollback để nhanh chóng hoàn tác các thay đổi cấu hình, triển khai dần dần (gradual rollouts) mã mới, giám sát chi tiết và rõ ràng, các công cụ observability để chẩn đoán các vấn đề trong production (xem “Các vấn đề với hệ thống phân tán”), và các interface được thiết kế tốt nhằm khuyến khích “điều đúng đắn” và ngăn chặn “điều sai trái”.

Tuy nhiên, tất cả những điều này đều đòi hỏi sự đầu tư về thời gian và tiền bạc, và trong thực tế thực dụng của kinh doanh hàng ngày, các tổ chức thường ưu tiên các hoạt động tạo ra doanh thu hơn là các biện pháp nhằm tăng cường độ tin cậy của hệ thống trước các sai sót. Khi phải lựa chọn giữa việc có thêm nhiều tính năng hơn hay thực hiện nhiều kiểm thử hơn, nhiều tổ chức sẽ chọn tính năng, điều này hoàn toàn có thể hiểu được. Sau đó, khi một sai sót có thể ngăn chặn được chắc chắn xảy ra, việc đổ lỗi cho người đã gây ra sai sót đó là vô nghĩa; vấn đề nằm ở các ưu tiên của tổ chức.

Ngày càng có nhiều tổ chức áp dụng văn hóa *blameless postmortems* (hậu kiểm không đổ lỗi): sau một sự cố, những người liên quan được khuyến khích chia sẻ đầy đủ chi tiết về những gì đã xảy ra mà không sợ bị trừng phạt, vì điều này cho phép những người khác trong tổ chức học cách ngăn chặn các vấn đề tương tự trong tương lai [75]. Quá trình này có thể làm lộ ra nhu cầu cần thay đổi các ưu tiên kinh doanh, đầu tư vào các lĩnh vực đã bị bỏ qua, thay đổi các cơ chế khuyến khích cho những người liên quan, hoặc đưa một vấn đề mang tính hệ thống khác lên sự chú ý của ban quản lý.

Như một nguyên tắc chung, khi điều tra một sự cố, bạn nên nghi ngờ những câu trả lời quá đơn giản. Câu nói “Bob lẽ ra nên cẩn thận hơn khi triển khai thay đổi đó” là không hiệu quả, nhưng câu “Chúng ta phải viết lại backend bằng Haskell” cũng vậy. Thay vào đó, ban quản lý nên tận dụng cơ hội này để tìm hiểu chi tiết về cách thức hoạt động của hệ thống sociotechnical (kỹ thuật - xã hội) từ góc nhìn của những người làm việc với nó hàng ngày, và thực hiện các bước để cải thiện nó dựa trên những phản hồi này [73].

Độ tin cậy quan trọng đến mức nào?

Độ tin cậy không chỉ dành cho các nhà máy điện hạt nhân và kiểm soát không lưu; các ứng dụng đời thường hơn cũng được kỳ vọng là phải hoạt động một cách tin cậy. Các lỗi trong các ứng dụng kinh doanh dẫn đến việc mất năng suất (và các rủi ro pháp lý nếu các con số được báo cáo không chính xác), và việc các trang thương mại điện tử bị ngừng hoạt động có thể gây ra chi phí khổng lồ về mặt mất doanh thu và tổn hại danh tiếng.

Trong nhiều ứng dụng, việc ngừng hoạt động tạm thời trong vài phút hoặc thậm chí vài giờ là có thể chấp nhận được [76], nhưng việc mất dữ liệu hoặc làm hỏng dữ liệu vĩnh viễn sẽ là một thảm họa. Hãy xét trường hợp một phụ huynh lưu trữ tất cả ảnh và video về con cái họ trong ứng dụng ảnh của bạn [77]. Họ sẽ cảm thấy thế nào nếu cơ sở dữ liệu đó đột ngột bị hỏng? Liệu họ có biết cách khôi phục bộ sưu tập của mình từ một bản sao lưu không?

Một ví dụ khác về việc phần mềm không đáng tin cậy có thể gây hại cho con người là vụ bê bối Post Office Horizon. Từ năm 1999 đến 2019, hàng trăm người quản lý các chi nhánh của Bưu điện tại Anh đã bị kết án trộm cắp hoặc gian lận vì phần mềm kế toán hiển thị sự thiếu hụt trong tài khoản của họ. Cuối cùng, mọi chuyện trở nên rõ ràng rằng nhiều sự thiếu hụt này là do các lỗi trong phần mềm, dẫn đến việc nhiều bản án này đã bị hủy bỏ [78]. Điều dẫn đến sự việc này, có lẽ là vụ oan sai lớn nhất trong lịch sử nước Anh, là một giả định của luật pháp Anh rằng máy tính hoạt động chính xác (và do đó, bằng chứng do máy tính tạo ra là đáng tin cậy) trừ khi có bằng chứng ngược lại [79]. Các kỹ sư phần mềm có thể cười nhạo ý tưởng rằng phần mềm có thể hoàn toàn không có lỗi, nhưng điều này chẳng mang lại chút an ủi nào cho những người đã bị bỏ tù oan, bị tuyên bố phá sản, hoặc thậm chí đã tự sát do kết quả của một bản án sai lầm từ một hệ thống máy tính không đáng tin cậy.

Trong một số tình huống, chúng ta có thể chọn hy sinh độ tin cậy để giảm chi phí phát triển (ví dụ: khi phát triển một sản phẩm mẫu cho một thị trường chưa được kiểm chứng)—nhưng chúng ta nên hết sức ý thức về việc khi nào mình đang đi tắt và luôn ghi nhớ những hậu quả tiềm tàng.

Khả năng mở rộng

Ngay cả khi một hệ thống đang hoạt động tin cậy vào ngày hôm nay, điều đó không có nghĩa là nó chắc chắn sẽ hoạt động tin cậy trong tương lai. Một lý do phổ biến dẫn đến sự suy giảm là sự gia tăng tải. Có lẽ hệ thống đã tăng trưởng từ 10.000 người dùng đồng thời lên 100.000 người dùng đồng thời, hoặc từ 1 triệu lên 10 triệu. Có lẽ nó đang xử lý khối lượng dữ liệu lớn hơn nhiều so với trước đây.

Scalability (khả năng mở rộng) là thuật ngữ chúng ta dùng để mô tả khả năng của một hệ thống trong việc đối phó với tải tăng lên. Đôi khi, khi thảo luận về scalability, mọi người thường đưa ra những nhận xét kiểu như: "Bạn không phải là Google hay Amazon. Đừng lo lắng về scale nữa mà hãy cứ dùng một relational database đi." Việc châm ngôn này có áp dụng cho bạn hay không tùy thuộc vào loại ứng dụng mà bạn đang xây dựng.

Nếu bạn đang xây dựng một sản phẩm mới hiện chỉ có một số lượng nhỏ người dùng, có lẽ là tại một startup, thì mục tiêu kỹ thuật bao trùm thường là giữ cho hệ thống càng đơn giản và linh hoạt càng tốt để bạn có thể dễ dàng sửa đổi và điều chỉnh các tính năng của sản phẩm khi tìm hiểu thêm về nhu cầu của khách hàng [80]. Trong một môi trường như vậy, việc lo lắng về quy mô giả định có thể cần thiết trong tương lai là phản tác dụng. Trong trường hợp tốt nhất, các khoản đầu tư vào scalability là nỗ lực lãng phí và là sự tối ưu hóa sớm (premature optimization); trong trường hợp xấu nhất, chúng khóa bạn vào một thiết kế thiếu linh hoạt và khiến việc phát triển ứng dụng trở nên khó khăn hơn.

Scalability không phải là một nhãn dán một chiều—sẽ thật vô nghĩa nếu nói "*X* có khả năng mở rộng" hoặc "*Y* không có khả năng scale." Thay vào đó, thảo luận về scalability nghĩa là xem xét các câu hỏi như sau:

- Nếu hệ thống tăng trưởng theo một cách cụ thể, các lựa chọn của chúng ta để đối phó với sự tăng trưởng đó là gì?
- Làm thế nào chúng ta có thể thêm các tài nguyên tính toán để xử lý lượng tải bổ sung?
- Dựa trên các dự báo tăng trưởng hiện tại, khi nào chúng ta sẽ chạm tới giới hạn của kiến trúc hiện tại?

Nếu bạn thành công trong việc làm cho ứng dụng của mình trở nên phổ biến, và do đó phải xử lý lượng tải ngày càng tăng, bạn sẽ biết được các điểm nghẽn hiệu năng nằm ở đâu và cần phải scale theo những chiều nào. Tại thời điểm đó, đã đến lúc bắt đầu lo lắng về các kỹ thuật cho scalability.

Hiểu về Tải (Load)

Trước tiên, bạn cần có sự hiểu biết rõ ràng về tải hiện tại của hệ thống. Chỉ khi đó bạn mới có thể thảo luận về các câu hỏi tăng trưởng ("Điều gì xảy ra nếu tải của chúng ta tăng gấp đôi?"). Thông thường, đây sẽ là một thước đo về throughput—ví dụ: số lượng yêu cầu mỗi giây gửi đến một service, số gigabyte dữ liệu mới đến mỗi ngày, hoặc số lượt thanh toán giỏ hàng mỗi giờ. Đôi khi bạn cần quan tâm đến giá trị đỉnh của một đại lượng biến thiên, chẳng hạn như số lượng người dùng trực tuyến đồng thời trong trường hợp nghiên cứu về mạng xã hội của chúng ta.

Thông thường, các đặc tính thống kê khác của tải cũng ảnh hưởng đến các mô hình truy cập (access patterns) và do đó ảnh hưởng đến các yêu cầu về scalability. Ví dụ, bạn có thể cần biết tỷ lệ giữa số lượt đọc so với số lượt ghi trong một database, tỷ lệ hit của một cache, hoặc số lượng mục dữ liệu trên mỗi người dùng (followers, trong trường hợp nghiên cứu của chúng ta). Có lẽ trường hợp trung bình là điều quan trọng đối với bạn, hoặc có lẽ điểm nghẽn của bạn bị chi phối bởi một số ít các trường hợp cực đoan. Tất cả phụ thuộc vào chi tiết của ứng dụng cụ thể mà bạn đang xây dựng.

Khi đã hiểu về tải trên hệ thống của mình, bạn có thể nghiên cứu điều gì sẽ xảy ra khi tải tăng lên. Bạn có thể xem xét điều này theo hai cách:

- Khi bạn tăng tải theo một cách nhất định và giữ nguyên các tài nguyên hệ thống (CPU, memory, network bandwidth, v.v.), hiệu năng của hệ thống bị ảnh hưởng như thế nào?
- Khi bạn tăng tải theo một cách nhất định, bạn cần tăng bao nhiêu tài nguyên nếu muốn giữ cho hiệu năng không đổi?

Thông thường, mục tiêu là giữ hiệu năng của hệ thống nằm trong các yêu cầu của SLA (xem mục "Sử dụng các chỉ số Response Time") đồng thời giảm thiểu chi phí vận hành hệ thống. Tài nguyên tính toán yêu cầu càng lớn thì chi phí càng cao. Một số loại phần cứng có thể hiệu quả về chi phí hơn những loại khác, và các yếu tố này có thể thay đổi theo thời gian khi các loại phần cứng mới xuất hiện.

Nếu việc gấp đôi tài nguyên cho phép bạn xử lý khối lượng công việc gấp đôi trong khi vẫn giữ nguyên hiệu năng, chúng ta nói rằng bạn có *khả năng mở rộng tuyến tính* (*linear scalability*), và đây được coi là một điều tốt. Đôi khi, ta có thể xử lý khối lượng công việc gấp đôi với ít hơn một nửa số tài nguyên tăng thêm, nhờ vào lợi thế quy mô hoặc việc phân phối tải đỉnh tốt hơn [81, 82]. Tuy nhiên, khả năng cao hơn là chi phí sẽ tăng nhanh hơn mức tuyến tính. Có thể có nhiều lý do dẫn đến sự kém hiệu quả này; ví dụ, nếu bạn có rất nhiều dữ liệu, việc xử lý một yêu cầu ghi duy nhất có thể đòi hỏi nhiều công việc hơn so với khi bạn có ít dữ liệu, ngay cả khi kích thước của yêu cầu là như nhau.

Kiến trúc Shared-Memory, Shared-Disk và Shared-Nothing

Cách đơn giản nhất để tăng tài nguyên phần cứng của một dịch vụ là chuyển nó sang một máy chủ mạnh mẽ hơn. Các lõi CPU riêng lẻ không còn trở nên nhanh hơn đáng kể nữa, nhưng bạn có thể mua một máy chủ (hoặc thuê một cloud instance) với nhiều lõi CPU hơn, nhiều RAM hơn và không gian đĩa lớn hơn. Cách tiếp cận này được gọi là *vertical scaling* hoặc *scaling up*.

Bạn có thể đạt được tính song song trên một máy chủ duy nhất bằng cách sử dụng nhiều process hoặc thread. Tất cả các thread thuộc cùng một process đều có thể truy

cập vào cùng một RAM, và do đó cách tiếp cận này cũng được gọi là *shared-memory architecture*. Vấn đề với cách tiếp cận *shared-memory* là chi phí tăng nhanh hơn mức tuyến tính; một máy chủ cao cấp với tài nguyên phần cứng gấp đôi một máy chủ cấu hình thấp thường có giá cao hơn đáng kể so với mức gấp đôi. Và do các nút thắt cổ chai, máy chủ đó khó có khả năng thực sự xử lý được khối lượng công việc gấp đôi.

Một cách tiếp cận khác là *shared-disk architecture*, sử dụng nhiều máy chủ với CPU và RAM độc lập nhưng lưu trữ dữ liệu trên một mảng các đĩa được chia sẻ giữa các máy chủ, vốn được kết nối thông qua một mạng tốc độ cao: *network-attached storage* (NAS) hoặc một *storage area network* (SAN). Kiến trúc này theo truyền thống đã được sử dụng cho các khối lượng công việc data warehousing tại chỗ (on-premises), nhưng sự tranh chấp (contention) và chi phí quản lý (overhead) của việc locking làm hạn chế khả năng mở rộng của cách tiếp cận *shared-disk* [83].

Ngược lại, *shared-nothing architecture* [84] (còn được gọi là *horizontal scaling* hoặc *scaling out*) bao gồm một hệ thống phân tán với nhiều node, mỗi node đều có CPU, RAM và đĩa riêng. Mọi sự điều phối giữa các node đều được thực hiện ở cấp độ phần mềm, thông qua một mạng thông thường.

Ưu điểm của cách tiếp cận này, vốn đã trở nên phổ biến trong những năm gần đây, là nó có tiềm năng mở rộng tuyến tính, có thể sử dụng bất kỳ phần cứng nào mang lại tỷ lệ hiệu năng/giá thành tốt nhất (đặc biệt là trên cloud), có thể điều chỉnh tài nguyên phần cứng dễ dàng hơn khi tải tăng hoặc giảm, và có thể đạt được khả năng chịu lỗi (fault tolerance) lớn hơn bằng cách phân phối hệ thống qua nhiều datacenters và các vùng (regions). Nhược điểm là nó yêu cầu sharding rõ ràng (xem Chương 7) và kéo theo tất cả sự phức tạp của các hệ thống phân tán (được thảo luận trong Chương 9).

Một số hệ thống cơ sở dữ liệu cloud native sử dụng các dịch vụ riêng biệt cho lưu trữ và thực thi transaction (xem “Separation of storage and compute”), với nhiều compute node cùng chia sẻ quyền truy cập vào cùng một dịch vụ lưu trữ. Mô hình này có một số điểm tương đồng với kiến trúc *shared-disk*, nhưng nó tránh được các vấn đề về khả năng mở rộng của các hệ thống cũ. Thay vì cung cấp một sự trừu tượng hóa filesystem (NAS) hoặc thiết bị khối (SAN), dịch vụ lưu trữ cung cấp một API chuyên dụng được thiết kế cho các nhu cầu cụ thể của cơ sở dữ liệu [85].

Các nguyên tắc về khả năng mở rộng

Kiến trúc của các hệ thống hoạt động ở quy mô lớn thường rất đặc thù cho từng ứng dụng. Không có thứ gọi là một kiến trúc có khả năng mở rộng chung cho mọi trường hợp (thường được gọi một cách không chính thức là *magic scaling sauce*). Ví dụ, một hệ thống được thiết kế để xử lý 100.000 yêu cầu mỗi giây, mỗi yêu cầu có kích thước 1 kB, sẽ trông rất khác so với một hệ thống được thiết kế cho 3 yêu cầu

mỗi phút, mỗi yêu cầu có kích thước 2 GB—ngay cả khi hai hệ thống có cùng throughput dữ liệu (100 MB/giây).

Hơn nữa, một kiến trúc phù hợp với một mức tải nhất định sẽ khó có khả năng đối phó được với mức tải gấp 10 lần. Do đó, nếu bạn đang làm việc với một dịch vụ đang tăng trưởng nhanh, rất có thể bạn sẽ cần phải thiết kế lại kiến trúc của mình sau mỗi lần tải tăng lên một bậc độ lớn (order of magnitude). Vì nhu cầu của ứng dụng thường có xu hướng tiến hóa, việc lập kế hoạch cho các nhu cầu về khả năng mở rộng (scalability) trong tương lai quá xa — vượt quá một bậc độ lớn — thường là không đáng.

Một nguyên tắc chung tốt để đạt được khả năng mở rộng là chia nhỏ hệ thống thành các thành phần nhỏ hơn có thể hoạt động độc lập với nhau. Đây chính là nguyên tắc nền tảng đằng sau microservices (xem “Microservices and Serverless”), sharding (Chương 7), stream processing (Chương 12) và các kiến trúc shared-nothing. Thách thức nằm ở việc biết đâu là ranh giới giữa những thứ nên đi cùng nhau và những thứ nên tách rời. Các hướng dẫn thiết kế cho microservices có thể được tìm thấy trong các cuốn sách khác [86], và chúng ta sẽ thảo luận về sharding của các hệ thống shared-nothing trong Chương 7.

Một nguyên tắc tốt khác là không làm mọi thứ trở nên phức tạp hơn mức cần thiết. Nếu một cơ sở dữ liệu chạy trên một máy đơn lẻ có thể hoàn thành công việc, thì nó có lẽ vẫn ưu việt hơn một thiết lập phân tán phức tạp. Các hệ thống autoscaling (tự động thêm hoặc bớt tài nguyên để đáp ứng nhu cầu) rất tuyệt vời, nhưng nếu tải của bạn khá dễ dự đoán, một hệ thống được scale thủ công có thể ít gặp phải các bất ngờ về vận hành hơn (xem “Operations: Automatic Versus Manual Rebalancing”). Một hệ thống với 5 services sẽ đơn giản hơn một hệ thống với 50 services. Các kiến trúc tốt thường bao gồm sự kết hợp thực dụng giữa nhiều phương pháp tiếp cận.

Maintainability

Phần mềm không bị mòn hay chịu sự mài mòn vật liệu, vì vậy nó không bị hỏng theo những cách giống như các vật thể cơ khí. Tuy nhiên, các yêu cầu của một ứng dụng thường xuyên tiến hóa, môi trường mà phần mềm chạy có sự thay đổi (chẳng hạn như các dependencies và nền tảng hạ tầng bên dưới), và nó có thể có các lỗi cần được sửa.

Người ta công nhận rộng rãi rằng phần lớn chi phí của phần mềm không nằm ở giai đoạn phát triển ban đầu mà nằm ở quá trình bảo trì (maintenance) liên tục — sửa lỗi, giữ cho các hệ thống hoạt động, điều tra các sự cố, thích ứng với các nền tảng mới, sửa đổi để phục vụ các trường hợp sử dụng mới, trả nợ kỹ thuật (technical debt) và thêm các tính năng mới [87, 88].

Bảo trì có thể rất phức tạp, đặc biệt là đối với các hệ thống legacy. Một hệ thống đã chạy thành công trong một thời gian dài có thể đang sử dụng các công nghệ lỗi thời mà không nhiều kỹ sư ngày nay hiểu rõ (chẳng hạn như mainframe và mã COBOL), và kiến thức nội bộ về việc tại sao và làm thế nào hệ thống được thiết kế theo một cách nhất định có thể đã bị mất đi khi nhân sự rời khỏi tổ chức. Việc sửa chữa lỗi của người khác cũng có thể là điều cần thiết. Vì các hệ thống máy tính thường đan xen với các tổ chức con người mà chúng hỗ trợ, nên việc bảo trì các hệ thống như vậy vừa là vấn đề về con người, vừa là vấn đề về kỹ thuật [89].

Mọi hệ thống chúng ta tạo ra ngày nay rồi sẽ có ngày trở thành một hệ thống legacy nếu nó đủ giá trị để tồn tại trong thời gian dài. Để giảm thiểu khó khăn cho các thế hệ tương lai khi họ cần bảo trì phần mềm của chúng ta, chúng ta nên thiết kế nó với sự chú trọng vào việc bảo trì. Mặc dù chúng ta không thể luôn dự đoán được quyết định nào có thể gây ra những rắc rối về bảo trì trong tương lai, nhưng trong cuốn sách này, chúng ta sẽ chú ý đến một số nguyên tắc có khả năng áp dụng rộng rãi:

Operability

Hãy giúp tổ chức vận hành hệ thống một cách trơn tru.

Simplicity

Hãy giúp các kỹ sư mới dễ dàng hiểu được hệ thống bằng cách triển khai nó sử dụng các pattern và cấu trúc nhất quán, dễ hiểu, đồng thời tránh các sự phức tạp không cần thiết.

Evolvability

Hãy giúp các kỹ sư dễ dàng thực hiện các thay đổi đối với hệ thống trong tương lai, thích ứng và mở rộng nó cho các trường hợp sử dụng chưa được dự tính khi các yêu cầu thay đổi.

Operability: Giúp công việc của bộ phận vận hành trở nên dễ dàng hơn

Trước đó, chúng ta đã thảo luận về vai trò của vận hành trong mục “Operations in the Cloud Era”, và chúng ta thấy rằng các quy trình con người cũng quan trọng đối với vận hành tin cậy ít nhất là ngang bằng với các công cụ phần mềm. Trên thực tế, đã có ý kiến cho rằng “vận hành tốt thường có thể xoay sở được các hạn chế của phần mềm kém (hoặc không hoàn thiện), nhưng phần mềm tốt không thể chạy một cách tin cậy với các hoạt động vận hành kém” [62].

Trong các hệ thống quy mô lớn bao gồm hàng nghìn máy, việc bảo trì thủ công sẽ cực kỳ tốn kém một cách vô lý, và tự động hóa là điều thiết yếu. Tuy nhiên, tự động hóa có thể là một con dao hai lưỡi. Sẽ luôn có những trường hợp biên (edge cases) (chẳng hạn như các kịch bản lỗi hiếm gặp) đòi hỏi sự can thiệp thủ công từ đội ngũ vận hành, và vì những trường hợp không thể xử lý tự động thường là những trường

hợp phức tạp nhất, nên việc tự động hóa nhiều hơn đòi hỏi một đội ngũ vận hành có kỹ năng *cao hơn* để có thể giải quyết những vấn đề đó [90].

Ngoài ra, một hệ thống tự động khi gặp lỗi thường khó troubleshoot hơn so với một hệ thống dựa vào người vận hành để thực hiện một số hành động thủ công. Vì lý do đó, tự động hóa nhiều hơn không phải lúc nào cũng tốt hơn cho khả năng vận hành (operability). Tuy nhiên, một lượng tự động hóa nhất định là rất quan trọng—điểm cân bằng (sweet spot) sẽ phụ thuộc vào các đặc thù của ứng dụng và tổ chức cụ thể của bạn.

Khả năng vận hành tốt nghĩa là làm cho các tác vụ định kỳ trở nên dễ dàng, cho phép đội ngũ vận hành tập trung vào các hoạt động có giá trị cao. Các hệ thống dữ liệu có thể hỗ trợ bằng cách thực hiện các việc sau [91]:

- Cho phép các công cụ monitoring kiểm tra các chỉ số chính của hệ thống và hỗ trợ các công cụ observability (xem “Problems with Distributed Systems”) để cung cấp thông tin chi tiết về hành vi runtime của hệ thống. Nhiều công cụ thương mại và mã nguồn mở có thể hỗ trợ việc này [92].
- Tránh phụ thuộc vào các máy riêng lẻ (cho phép các máy có thể được hạ xuống để bảo trì trong khi toàn bộ hệ thống vẫn tiếp tục chạy mà không bị gián đoạn).
- Cung cấp tài liệu hướng dẫn tốt và một mô hình vận hành dễ hiểu (“Nếu tôi làm *X*, thì *Y* sẽ xảy ra”).
- Cung cấp các hành vi mặc định tốt, nhưng cũng cho phép quản trị viên có quyền ghi đè các giá trị mặc định khi cần thiết.
- Khả năng tự phục hồi (self-healing) khi phù hợp, nhưng cũng cho phép quản trị viên kiểm soát thủ công trạng thái hệ thống khi cần thiết.
- Thể hiện hành vi có thể dự đoán được, giảm thiểu các yếu tố gây bất ngờ.

Sự đơn giản: Quản lý độ phức tạp

Các dự án phần mềm nhỏ có thể có mã nguồn đơn giản và giàu tính biểu đạt một cách thú vị, nhưng khi các dự án trở nên lớn hơn, chúng thường trở nên rất phức tạp và khó hiểu. Sự phức tạp này làm chậm tiến độ của tất cả những người cần làm việc trên hệ thống, làm tăng thêm chi phí bảo trì. Một dự án phần mềm bị sa lầy trong sự phức tạp đôi khi được mô tả là một *big ball of mud* [93].

Khi sự phức tạp khiến việc bảo trì trở nên khó khăn, ngân sách và tiến độ thường bị vượt mức. Trong phần mềm phức tạp, cũng có rủi ro lớn hơn về việc tạo ra các lỗi khi thực hiện một thay đổi. Khi hệ thống trở nên khó hiểu và khó suy luận hơn đối với các lập trình viên, các giả định ẩn, các hệ quả không mong muốn và các tương tác bất ngờ sẽ dễ bị bỏ qua hơn [71]. Ngược lại, việc giảm thiểu sự phức tạp sẽ cải thiện

đáng kể khả năng bảo trì của phần mềm, và do đó, sự đơn giản nên là một mục tiêu then chốt cho các hệ thống mà chúng ta xây dựng.

Các hệ thống đơn giản thì dễ hiểu hơn, vì vậy chúng ta nên cố gắng giải quyết một vấn đề nhất định theo cách đơn giản nhất có thể. Thật không may, điều này nói dễ hơn làm. Việc một thứ gì đó có đơn giản hay không thường là vấn đề chủ quan, vì không có tiêu chuẩn khách quan nào cho sự đơn giản [94]. Ví dụ, một hệ thống có thể che giấu một triển khai phức tạp đằng sau một giao diện đơn giản, trong khi một hệ thống khác có thể có một triển khai đơn giản nhưng lại để lộ nhiều chi tiết nội bộ hơn cho người dùng—hệ thống nào đơn giản hơn?

Một cách tiếp cận để suy luận về độ phức tạp là chia nó thành hai loại: thiết yếu (*essential*) và ngẫu nhiên (*accidental*) [95]. Ý tưởng ở đây là độ phức tạp *essential* vốn có trong miền bài toán của ứng dụng, trong khi độ phức tạp *accidental* chỉ phát sinh do những hạn chế của các công cụ mà chúng ta sử dụng. Thật không may, sự phân biệt này cũng có sai sót, bởi vì ranh giới giữa cái thiết yếu và cái ngẫu nhiên sẽ thay đổi khi các công cụ của chúng ta tiến hóa [96].

Một trong những công cụ tốt nhất mà chúng ta có để quản lý độ phức tạp là *abstraction* (sự trừu tượng hóa). Một abstraction tốt có thể che giấu rất nhiều chi tiết triển khai đằng sau một lớp vỏ ngoài sạch sẽ, dễ hiểu. Một abstraction tốt cũng có thể được sử dụng cho nhiều ứng dụng khác nhau. Việc tái sử dụng này không chỉ hiệu quả hơn so với việc triển khai lại cùng một thứ nhiều lần, mà nó còn dẫn đến phần mềm có chất lượng cao hơn, vì những cải tiến về chất lượng trong thành phần đã được abstraction sẽ mang lại lợi ích cho tất cả các ứng dụng sử dụng nó.

Ví dụ, các ngôn ngữ lập trình bậc cao là các abstraction giúp che giấu mã máy (*machine code*), các thanh ghi CPU và các *system call*. SQL là một abstraction che giấu các cấu trúc dữ liệu phức tạp trên đĩa và trong bộ nhớ, các yêu cầu đồng thời từ các client khác, và sự không nhất quán sau khi hệ thống bị crash. Tất nhiên, khi lập trình bằng một ngôn ngữ bậc cao, chúng ta vẫn đang sử dụng mã máy; chúng ta chỉ là không sử dụng nó *trực tiếp*, bởi vì abstraction của ngôn ngữ lập trình giúp chúng ta không phải bận tâm suy nghĩ về nó.

Các abstraction cho mã nguồn ứng dụng nhằm mục đích giảm độ phức tạp có thể được tạo ra bằng cách sử dụng các phương pháp như *design patterns* [97] và *domain-driven design* (DDD) [98]. Cuốn sách này không nói về các abstraction đặc thù cho ứng dụng như vậy, mà thay vào đó là về các abstraction đa dụng mà trên đó bạn có thể xây dựng các ứng dụng của mình, chẳng hạn như các transaction của cơ sở dữ liệu, các index và các event log. Nếu bạn muốn sử dụng các kỹ thuật như DDD, bạn có thể triển khai chúng dựa trên những nền tảng được mô tả trong cuốn sách này.

Evolvability: Giúp việc thay đổi trở nên dễ dàng

Rất khó có khả năng các yêu cầu của hệ thống sẽ giữ nguyên mãi mãi. Chúng có nhiều khả năng sẽ luôn biến động: bạn học được những sự thật mới, các trường hợp sử dụng chưa được dự tính trước xuất hiện, các ưu tiên kinh doanh thay đổi, người dùng yêu cầu các tính năng mới, các nền tảng mới thay thế các nền tảng cũ, các yêu cầu pháp lý hoặc quy định thay đổi, sự tăng trưởng của hệ thống buộc phải thay đổi kiến trúc, v.v.

Xét về các quy trình tổ chức, các mô hình làm việc *Agile* cung cấp một khung làm việc để thích ứng với sự thay đổi. Cộng đồng Agile cũng đã phát triển các công cụ và quy trình kỹ thuật hữu ích khi xây dựng phần mềm trong một môi trường thay đổi thường xuyên, chẳng hạn như test-driven development (TDD) và refactoring. Trong cuốn sách này, chúng ta tìm kiếm các cách để tăng tính linh hoạt ở cấp độ một hệ thống bao gồm nhiều ứng dụng hoặc service với các đặc tính khác nhau.

Sự dễ dàng mà bạn có thể sửa đổi một hệ thống dữ liệu và điều chỉnh nó theo các yêu cầu thay đổi liên quan chặt chẽ đến tính đơn giản và các abstraction của nó. Các hệ thống được tách rời (loosely coupled) và đơn giản thường dễ sửa đổi hơn các hệ thống bị gắn kết chặt chẽ (tightly coupled) và phức tạp. Vì đây là một ý tưởng quan trọng, chúng ta sẽ sử dụng một từ khác để chỉ tính linh hoạt ở cấp độ hệ thống dữ liệu: *evolvability* (khả năng tiến hóa) [99].

Một yếu tố chính khiến việc thay đổi trở nên khó khăn trong các hệ thống lớn là tính không thể đảo ngược (irreversibility) [100]. Ví dụ, giả sử bạn đang di chuyển từ cơ sở dữ liệu này sang cơ sở dữ liệu khác. Nếu bạn không thể chuyển về hệ thống cũ trong trường hợp có vấn đề với hệ thống mới, thì rủi ro sẽ cao hơn nhiều so với việc bạn có thể dễ dàng quay lại. Do đó, các hành động không thể đảo ngược cần được thực hiện rất cẩn thận. Việc giảm thiểu tính không thể đảo ngược sẽ giúp cải thiện tính linh hoạt.

Tóm tắt

Trong chương này, chúng ta đã xem xét một vài ví dụ về các yêu cầu phi chức năng: hiệu năng, độ tin cậy, khả năng mở rộng và khả năng bảo trì. Thông qua các chủ đề này, chúng ta cũng gặp phải các nguyên tắc và thuật ngữ sẽ có liên quan xuyên suốt phần còn lại của cuốn sách.

Chúng ta bắt đầu với một nghiên cứu điển hình về việc triển khai home timeline trong một mạng xã hội, qua đó minh họa một số thách thức nảy sinh khi mở rộng quy mô (at scale). Sau đó, chúng ta thảo luận về cách đo lường hiệu năng (ví dụ: sử dụng các percentile của response time) và tải của một hệ thống (ví dụ: sử dụng các metrics về throughput), cũng như cách các metrics này được sử dụng trong các SLA. Scalability là một khái niệm liên quan chặt chẽ: nó tập trung vào việc đảm bảo rằng

hiệu năng vẫn giữ nguyên khi tải tăng lên. Chúng ta đã thấy một số nguyên tắc chung cho scalability, chẳng hạn như chia nhỏ một tác vụ thành các phần nhỏ hơn có thể hoạt động độc lập, và chúng ta sẽ đi sâu vào chi tiết kỹ thuật hơn về các kỹ thuật scalability trong các chương tiếp theo.

Để đạt được độ tin cậy (reliability), bạn có thể sử dụng các kỹ thuật fault-tolerance, cho phép một hệ thống tiếp tục cung cấp các dịch vụ của nó ngay cả khi một thành phần (ví dụ: một ổ đĩa, một máy chủ, hoặc một service khác) bị lỗi. Chúng ta đã xem xét các ví dụ về lỗi phần cứng có thể xảy ra và phân biệt chúng với các lỗi phần mềm, vốn có thể khó xử lý hơn vì chúng thường có mối tương quan chặt chẽ. Một khía cạnh khác để đạt được độ tin cậy là xây dựng khả năng chống chịu trước những sai sót của con người, và chúng ta đã thấy blameless postmortems như một kỹ thuật để học hỏi từ các sự cố.

Cuối cùng, chúng ta đã xem xét nhiều khía cạnh của maintainability, bao gồm hỗ trợ công việc của các đội ngũ vận hành, quản lý sự phức tạp, và giúp việc phát triển các tính năng của một ứng dụng theo thời gian trở nên dễ dàng. Không có câu trả lời dễ dàng cho việc làm thế nào để đạt được các mục tiêu này, nhưng một cách tiếp cận có thể giúp ích là xây dựng các ứng dụng bằng cách sử dụng các building blocks đã được hiểu rõ, những thứ cung cấp các abstraction hữu ích. Phần còn lại của cuốn sách này sẽ bao gồm một số building blocks đã được chứng minh là có giá trị trong thực hành.

Footnotes

References

- [1] Mike Cvet. “How We Learned to Stop Worrying and Love Fan-in at Twitter.” At *QCon San Francisco*, December 2016.
- [2] Raffi Krikorian. “Timelines at Scale.” At *QCon San Francisco*, November 2012. Archived at perma.cc/V9G5-KLYK
- [3] Twitter. “Twitter’s Recommendation Algorithm.” *blog.x.com*, March 2023. Archived at perma.cc/L5GT-229T
- [4] Raffi Krikorian. “New Tweets per Second Record, and How!” *blog.x.com*, August 2013. Archived at perma.cc/6JZN-XJYN
- [5] Jaz Volpert. “When Imperfect Systems Are Good, Actually: Bluesky’s Lossy Timelines.” *jazco.dev*, February 2025. Archived at perma.cc/2PVE-L2MX
- [6] Samuel Axon. “3% of Twitter’s Servers Dedicated to Justin Bieber.” *mashable.com*, September 2010. Archived at perma.cc/F35N-CGVX
- [7] Nathan Bronson, Abutalib Aghayev, Aleksey Charapko, and Timothy Zhu. “Metastable Failures in Distributed Systems.” At *Workshop on Hot Topics in Operating Systems (HotOS)*, May 2021. [doi: 10.1145/3458336.3465286](https://doi.org/10.1145/3458336.3465286)
- [8] Marc Brooker. “Metastability and Distributed Systems.” *brooker.co.za*, May 2021. Archived at perma.cc/7FGJ-7XRK

- [9] Lexiang Huang, Matthew Magnusson, Abishek Bangalore Muralikrishna, Salman Estyak, Rebecca Isaacs, Abutalib Aghayev, Timothy Zhu, and Aleksey Charapko. “Metastable Failures in the Wild.” At *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, July 2022.
- [10] Marc Brooker. “Exponential Backoff and Jitter.” *aws.amazon.com*, March 2015. Archived at perma.cc/R6MS-AZKH
- [11] Marc Brooker. “What Is Backoff For?” *brooker.co.za*, August 2022. Archived at perma.cc/PW9N-55Q5
- [12] Michael T. Nygard. *Release It!*, 2nd edition. Pragmatic Bookshelf, 2018. ISBN: 9781680502398
- [13] Frank Chen. “Slowing Down to Speed Up—Circuit Breakers for Slack’s CI/CD.” *slack.engineering*, August 2022. Archived at perma.cc/5FGS-ZPH3
- [14] Marc Brooker. “Fixing Retries with Token Buckets and Circuit Breakers.” *brooker.co.za*, February 2022. Archived at perma.cc/MD6N-GW26
- [15] David Yanacek. “Using Load Shedding to Avoid Overload.” Amazon Builders’ Library, *aws.amazon.com*. Archived at perma.cc/9SAW-68MP
- [16] Matthew Sackman. “Pushing Back.” *wellquite.org*, May 2016. Archived at perma.cc/3KCZ-RUFY
- [17] Dmitry Kopytkov and Patrick Lee. “Meet Bandaaid, the Dropbox Service Proxy.” *dropbox.tech*, March 2018. Archived at perma.cc/KUU6-YG4S
- [18] Haryadi S. Gunawi, Riza O. Suminto, Russell Sears, Casey Golliher, Swaminathan Sundararaman, Xing Lin, Tim Emami, Weiguang Sheng, Nematollah Bidokhti, Caitie McCaffrey, Gary Grider, Parks M. Fields, Kevin Harms, Robert B. Ross, Andree Jacobson, Robert Ricci, Kirk Webb, Peter Alvaro, H. Biralí Runesha, Mingzhe Hao, and Huaicheng Li. “Fail-Slow at Scale: Evidence of Hardware Performance Faults in Large Production Systems.” At *16th USENIX Conference on File and Storage Technologies*, February 2018.
- [19] Marc Brooker. “Is the Mean Really Useless?” *brooker.co.za*, December 2017. Archived at perma.cc/USAE-CVEM
- [20] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Voshall, and Werner Vogels. “Dynamo: Amazon’s Highly Available Key-Value Store.” At *21st ACM Symposium on Operating Systems Principles (SOSP)*, October 2007. *doi:10.1145/1294261.1294281*
- [21] Kathryn Whitenon. “The Need for Speed, 23 Years Later.” *nngroup.com*, May 2020. Archived at perma.cc/C4ER-LZYA
- [22] Greg Linden. “Marissa Mayer at Web 2.0.” *glinden.blogspot.com*, November 2005. Archived at perma.cc/V7EA-3VXB
- [23] Jake Brutlag. “Speed Matters for Google Web Search.” *services.google.com*, June 2009. Archived at perma.cc/BK7R-X7M2
- [24] Eric Schurman and Jake Brutlag. “Performance Related Changes and Their User Impact.” Talk at *Velocity 2009*.
- [25] Akamai Technologies, Inc. “The State of Online Retail Performance.” *akamai.com*, April 2017. Archived at perma.cc/UEK2-HYCS
- [26] Xiao Bai, Ioannis Arapakis, B. Barla Cambazoglu, and Ana Freire. “Understanding and Leveraging the Impact of Response Latency on User Behaviour in Web Search.” *ACM Transactions on Information Systems*, volume 36, issue 2, article 21, April 2018. *doi:10.1145/3106372*

- [27] Jeffrey Dean and Luiz André Barroso. “The Tail at Scale.” *Communications of the ACM*, volume 56, issue 2, pages 74–80, February 2013. [doi:10.1145/2408776.2408794](#)
- [28] Alex Hidalgo. *Implementing Service Level Objectives: A Practical Guide to SLIs, SLOs, and Error Budgets*. O’Reilly Media, 2020. ISBN: 9781492076813
- [29] Jeffrey C. Mogul and John Wilkes. “Nines Are Not Enough: Meaningful Metrics for Clouds.” At *17th Workshop on Hot Topics in Operating Systems (HotOS)*, May 2019. [doi:10.1145/3317550.3321432](#)
- [30] Tamás Hauer, Philipp Hoffmann, John Lunney, Dan Ardelean, and Amer Diwan. “Meaningful Availability.” At *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, February 2020.
- [31] Gil Tene. “HdrHistogram: A High Dynamic Range Histogram.” [hdrhistogram.github.io/HdrHistogram](#)
- [32] Ted Dunning. “The *t*-digest: Efficient Estimates of Distributions.” *Software Impacts*, volume 7, article 100049, February 2021. [doi:10.1016/j.simpa.2020.100049](#)
- [33] David Kohn. “How Percentile Approximation Works (and Why It’s More Useful than Averages).” *timescale.com*, September 2021. Archived at [perma.cc/3PDP-NR8B](#)
- [34] Heinrich Hartmann and Theo Schlossnagle. “Circllhist—A Log-Linear Histogram Data Structure for IT Infrastructure Monitoring.” *arXiv:2001.06561*, January 2020.
- [35] Charles Masson, Jee E. Rim, and Homin K. Lee. “DDSketch: A Fast and Fully-Mergeable Quantile Sketch with Relative-Error Guarantees.” *Proceedings of the VLDB Endowment*, volume 12, issue 12, pages 2195–2205, August 2019. [doi:10.14778/3352063.3352135](#)
- [36] Baron Schwartz. “Why Percentiles Don’t Work the Way You Think.” *solarwinds.com*, November 2016. Archived at [perma.cc/469T-6UGB](#)
- [37] Walter L. Heimerdinger and Charles B. Weinstock. “A Conceptual Framework for System Fault Tolerance.” Technical Report CMU/SEI-92-TR-033, Software Engineering Institute, Carnegie Mellon University, October 1992. Archived at [perma.cc/GD2V-DMJW](#)
- [38] Felix C. Gärtner. “Fundamentals of Fault-Tolerant Distributed Computing in Asynchronous Environments.” *ACM Computing Surveys*, volume 31, issue 1, pages 1–26, March 1999. [doi:10.1145/311531.311532](#)
- [39] Algirdas Avizienis, Jean-Claude Laprie, Brian Randell, and Carl Landwehr. “Basic Concepts and Taxonomy of Dependable and Secure Computing.” *IEEE Transactions on Dependable and Secure Computing*, volume 1, issue 1, pages 11–33, January 2004. [doi:10.1109/TDSC.2004.2](#)
- [40] Ding Yuan, Yu Luo, Xin Zhuang, Guilherme Renna Rodrigues, Xu Zhao, Yongle Zhang, Pranay U. Jain, and Michael Stumm. “Simple Testing Can Prevent Most Critical Failures: An Analysis of Production Failures in Distributed Data-Intensive Systems.” At *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, October 2014.
- [41] Casey Rosenthal and Nora Jones. *Chaos Engineering*. O’Reilly Media, 2020. ISBN: 9781492043867
- [42] Eduardo Pinheiro, Wolf-Dietrich Weber, and Luiz Andre Barroso. “Failure Trends in a Large Disk Drive Population.” At *5th USENIX Conference on File and Storage Technologies (FAST)*, February 2007.
- [43] Bianca Schroeder and Garth A. Gibson. “Disk Failures in the Real World: What Does an Mttf of 1,000,000 Hours Mean to You?” At *5th USENIX Conference on File and Storage Technologies (FAST)*, February 2007.
- [44] Andy Klein. “Backblaze Drive Stats for Q2 2021.” *backblaze.com*, August 2021. Archived at [perma.cc/2943-UD5E](#)

- [45] Iyswarya Narayanan, Di Wang, Myeongjae Jeon, Bikash Sharma, Laura Caulfield, Anand Sivasubramaniam, Ben Cutler, Jie Liu, Badriddine Khessib, and Kushagra Vaid. “SSD Failures in Datacenters: What? When? And Why?” At *9th ACM International on Systems and Storage Conference (SYSTOR)*, June 2016. doi:10.1145/2928275.2928278
- [46] Alibaba Cloud Storage Team. “Storage System Design Analysis: Factors Affecting NVMe SSD Performance (1).” *alibabacloud.com*, January 2019. Archived at *archive.org*
- [47] Bianca Schroeder, Raghav Lagisetty, and Arif Merchant. “Flash Reliability in Production: The Expected and the Unexpected.” At *14th USENIX Conference on File and Storage Technologies (FAST)*, February 2016.
- [48] Jacob Alter, Ji Xue, Alma Dimnaku, and Evgenia Smirni. “SSD Failures in the Field: Symptoms, Causes, and Prediction Models.” At *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, November 2019. doi:10.1145/3295500.3356172
- [49] Daniel Ford, François Labelle, Florentina I. Popovici, Murray Stokely, Van-Anh Truong, Luiz Barroso, Carrie Grimes, and Sean Quinlan. “Availability in Globally Distributed Storage Systems.” At *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, October 2010.
- [50] Kashi Venkatesh Vishwanath and Nachiappan Nagappan. “Characterizing Cloud Computing Hardware Reliability.” At *1st ACM Symposium on Cloud Computing (SoCC)*, June 2010. doi:10.1145/1807128.1807161
- [51] Peter H. Hochschild, Paul Turner, Jeffrey C. Mogul, Rama Govindaraju, Parthasarathy Ranganathan, David E. Culler, and Amin Vahdat. “Cores That Don’t Count.” At *Workshop on Hot Topics in Operating Systems (HotOS)*, June 2021. doi:10.1145/3458336.3465297
- [52] Harish Dattatraya Dixit, Sneha Pendharkar, Matt Beadon, Chris Mason, Tejasvi Chakravarthy, Bharath Muthiah, and Sriram Sankar. *Silent Data Corruptions at Scale*. arXiv:2102.11245, February 2021.
- [53] Diogo Behrens, Marco Serafini, Sergei Arnautov, Flavio P. Junqueira, and Christof Fetzer. “Scalable Error Isolation for Distributed Systems.” At *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, May 2015.
- [54] Bianca Schroeder, Eduardo Pinheiro, and Wolf-Dietrich Weber. “DRAM Errors in the Wild: A Large-Scale Field Study.” At *11th International Joint Conference on Measurement and Modeling of Computer Systems (SIGMETRICS)*, June 2009. doi:10.1145/1555349.1555372
- [55] Yoongu Kim, Ross Daly, Jeremie Kim, Chris Fallin, Ji Hye Lee, Donghyuk Lee, Chris Wilkerson, Konrad Lai, and Onur Mutlu. “Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors.” At *41st Annual International Symposium on Computer Architecture (ISCA)*, June 2014. doi:10.5555/2665671.2665726
- [56] Tim Bray. “Worst Case.” *bray.org*, October 2021. Archived at *perma.cc/4QQM-RTHN*
- [57] Sangeetha Abdu Jyothei. “Solar Superstorms: Planning for an Internet Apocalypse.” At *ACM SIGCOMM Conference*, August 2021. doi:10.1145/3452296.3472916
- [58] Adrian Cockcroft. “Failure Modes and Continuous Resilience.” *adrianco.medium.com*, November 2019. Archived at *perma.cc/7SYS-BVJP*
- [59] Shujie Han, Patrick P. C. Lee, Fan Xu, Yi Liu, Cheng He, and Jiongzhou Liu. “An In-Depth Study of Correlated Failures in Production SSD-Based Data Centers.” At *19th USENIX Conference on File and Storage Technologies (FAST)*, February 2021.

- [60] Edmund B. Nightingale, John R. Douceur, and Vince Orgovan. “Cycles, Cells and Platters: An Empirical Analysis of Hardware Failures on a Million Consumer PCs.” At *6th European Conference on Computer Systems* (EuroSys), April 2011. doi:10.1145/1966445.1966477
- [61] Haryadi S. Gunawi, Mingzhe Hao, Tanakorn Leesatapornwongsa, Tiratat Patana-anake, Thanh Do, Jeffry Adityatama, Kurnia J. Eliazar, Agung Laksono, Jeffrey F. Lukman, Vincentius Martin, and Anang D. Satria. “What Bugs Live in the Cloud? A Study of 3000+ Issues in Cloud Systems.” At *5th ACM Symposium on Cloud Computing* (SoCC), November 2014. doi:10.1145/2670979.2670986
- [62] Jay Kreps. “Getting Real About Distributed System Reliability.” *blog.empathybox.com*, March 2012. Archived at perma.cc/9B5Q-AEBW
- [63] Nelson Minar. “Leap Second Crashes Half the Internet.” *somebits.com*, July 2012. Archived at perma.cc/2WB8-D6EU
- [64] Hewlett Packard Enterprise. “Support Alerts—Customer Bulletin a00092491en_us.” *support.hpe.com*, November 2019. Archived at perma.cc/55F6-7ZAC
- [65] Lorin Hochstein. “Awesome Limits.” *github.com*, November 2020. Archived at perma.cc/3R5M-E5Q4
- [66] Caitie McCaffrey. “Clients Are Jerks: AKA How Halo 4 DoSed the Services at Launch & How We Survived.” *caitiem.com*, June 2015. Archived at perma.cc/MXX4-W373
- [67] Lilia Tang, Chaitanya Bhandari, Yongle Zhang, Anna Karanika, Shuyang Ji, Indranil Gupta, and Tianyin Xu. “Fail Through the Cracks: Cross-System Interaction Failures in Modern Cloud Systems.” At *18th European Conference on Computer Systems* (EuroSys), May 2023. doi:10.1145/3552326.3587448
- [68] Mike Ulrich. Addressing Cascading Failures. In Betsy Beyer, Jennifer Petoff, Chris Jones, and Niall Richard Murphy (ed). *Site Reliability Engineering: How Google Runs Production Systems*. O’Reilly Media, 2016. ISBN: 9781491929124
- [69] Harri Faßbender. “Cascading Failures in Large-Scale Distributed Systems.” *blog.mi.bdm-stuttgart.de*, March 2022. Archived at perma.cc/K7VY-YJRX
- [70] Richard I. Cook. “How Complex Systems Fail.” Cognitive Technologies Laboratory, April 2000. Archived at perma.cc/RDS6-2YVA
- [71] David D. Woods. “STELLA: Report from the SNAFUcatchers Workshop on Coping with Complexity.” *snafucatchers.github.io*, March 2017. Archived at archive.org
- [72] David Oppenheimer, Archana Ganapathi, and David A. Patterson. “Why Do Internet Services Fail, and What Can Be Done About It?” At *4th USENIX Symposium on Internet Technologies and Systems* (USITS), March 2003.
- [73] Sidney Dekker. *The Field Guide to Understanding “Human Error”*, 3rd edition. CRC Press, 2017. ISBN: 9781472439055
- [74] Sidney Dekker. *Drift into Failure: From Hunting Broken Components to Understanding Complex Systems*. CRC Press, 2011. ISBN: 9781315257396
- [75] John Allspaw. “Blameless PostMortems and a Just Culture.” *etsy.com*, May 2012. Archived at perma.cc/YMJ7-NTAP
- [76] Itzy Sabo. “Uptime Guarantees—A Pragmatic Perspective.” *world.bey.com*, March 2023. Archived at perma.cc/F7TU-78JB
- [77] Michael Jurewitz. “The Human Impact of Bugs.” *jury.me*, March 2013. Archived at perma.cc/5KQ4-VDYL

- [78] Mark Halper. “How Software Bugs Led to ‘One of the Greatest Miscarriages of Justice’ in British History.” *Communications of the ACM*, volume 68, issue 3, pages 12–14, January 2025. doi: [10.1145/3703779](https://doi.org/10.1145/3703779)
- [79] Nicholas Bohm, James Christie, Peter Bernard Ladkin, Bev Littlewood, Paul Marshall, Stephen Mason, Martin Newby, Steven J. Murdoch, Harold Thimbleby, and Martyn Thomas. “The Legal Rule That Computers Are Presumed to be Operating Correctly—Unforeseen and Unjust Consequences.” Briefing note, *benthams gaze.org*, June 2022. Archived at perma.cc/WQ6X-TMW4
- [80] Dan McKinley. “Choose Boring Technology.” *mcfunley.com*, March 2015. Archived at perma.cc/7QW7-J4YP
- [81] Andy Warfield. “Building and Operating a Pretty Big Storage System Called S3.” *allthingsdistributed.com*, July 2023. Archived at perma.cc/7LPK-TP7V
- [82] Marc Brooker. “Surprising Scalability of Multitenancy.” *brooker.co.za*, March 2023. Archived at perma.cc/ZZD9-VV8T
- [83] Ben Stopford. “Shared Nothing vs. Shared Disk Architectures: An Independent View.” *benstopford.com*, November 2009. Archived at perma.cc/7BXH-EDUR
- [84] Michael Stonebraker. “The Case for Shared Nothing.” *IEEE Database Engineering Bulletin*, volume 9, issue 1, pages 4–9, March 1986. perma.cc/P9YL-C4PS
- [85] Panagiotis Antonopoulos, Alex Budovski, Cristian Diaconu, Alejandro Hernandez Saenz, Jack Hu, Hanuma Kodavalla, Donald Kossmann, Sandeep Lingam, Umar Farooq Minhas, Naveen Prakash, Vijendra Purohit, Hugh Qu, Chaitanya Sreenivas Ravela, Krystyna Reisteter, Sheetal Shrotri, Dixin Tang, and Vikram Wakade. “Socrates: The New SQL Server in the Cloud.” At *ACM International Conference on Management of Data (SIGMOD)*, June 2019. doi: [10.1145/3299869.3314047](https://doi.org/10.1145/3299869.3314047)
- [86] Sam Newman. *Building Microservices*, 2nd edition. O’Reilly Media, 2021. ISBN: 9781492034025
- [87] Nathan Ensmenger. “When Good Software Goes Bad: The Surprising Durability of an Ephemeral Technology.” At *The Maintainers Conference*, April 2016. Archived at perma.cc/ZXT4-HGZB
- [88] Robert L. Glass. *Facts and Fallacies of Software Engineering*. Addison-Wesley Professional, 2002. ISBN: 9780321117427
- [89] Marianne Bellotti. *Kill It with Fire*. No Starch Press, 2021. ISBN: 9781718501188
- [90] Lisanne Bainbridge. “Ironies of Automation.” *Automatica*, volume 19, issue 6, pages 775–779, November 1983. doi: [10.1016/0005-1098\(83\)90046-8](https://doi.org/10.1016/0005-1098(83)90046-8)
- [91] James Hamilton. “On Designing and Deploying Internet-Scale Services.” At *21st Large Installation System Administration Conference (LISA)*, November 2007.
- [92] Dotan Horovits. “Open Source for Better Observability.” *horovits.medium.com*, October 2021. Archived at perma.cc/R2HD-U2ZT
- [93] Brian Foote and Joseph Yoder. “Big Ball of Mud.” At *4th Conference on Pattern Languages of Programs (PLoP)*, September 1997. Archived at perma.cc/4GUP-2PBV
- [94] Marc Brooker. “What Is a Simple System?” *brooker.co.za*, May 2022. Archived at perma.cc/U72T-BFVE
- [95] Frederick P. Brooks. “No Silver Bullet—Essence and Accident in Software Engineering.” In *The Mythical Man-Month*, Anniversary edition, Addison-Wesley, 1995. ISBN: 9780201835953
- [96] Dan Luu. “Against Essential and Accidental Complexity.” *danluu.com*, December 2020. Archived at perma.cc/HSES-69KC

- [97] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994. ISBN: 9780201633610
- [98] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional, 2003. ISBN: 9780321125217
- [99] Hongyu Pei Breivold, Ivica Crnkovic, and Peter J. Eriksson. “Analyzing Software Evolvability.” At *32nd Annual IEEE International Computer Software and Applications Conference (COMPSAC)*, July 2008. doi:10.1109/COMPSAC.2008.50
- [100] Enrico Zaninotto. “From X Programming to the X Organisation.” At *XP Conference*, May 2002. Archived at perma.cc/R9AR-QCKZ

Data Models và Query Languages

Giới hạn của ngôn ngữ cũng chính là giới hạn của thế giới tôi.

Ludwig Wittgenstein, *Tractatus Logico-Philosophicus* (1922)

Data models có lẽ là phần quan trọng nhất trong việc phát triển phần mềm, bởi vì chúng có ảnh hưởng sâu sắc không chỉ đến cách viết phần mềm, mà còn đến cách chúng ta *suy nghĩ về vấn đề* mà chúng ta đang giải quyết.

Hầu hết các ứng dụng được xây dựng bằng cách xếp chồng các data model lên nhau. Đối với mỗi lớp, câu hỏi then chốt là nó được *biểu diễn* như thế nào dựa trên lớp ngay bên dưới nó. Dưới đây là một ví dụ về các lớp ứng dụng từ cấp cao nhất đến cấp thấp nhất:

1. Với tư cách là một lập trình viên ứng dụng, bạn nhìn vào thế giới thực (bao gồm con người, tổ chức, hàng hóa, hành động, dòng tiền, cảm biến, v.v.) và mô hình hóa nó dưới dạng các đối tượng hoặc cấu trúc dữ liệu và các API dùng để thao tác trên các cấu trúc dữ liệu đó, vốn thường đặc thù cho ứng dụng của bạn.
2. Khi bạn muốn lưu trữ các cấu trúc dữ liệu đó, bạn biểu diễn chúng dưới dạng một data model đa dụng, chẳng hạn như các tài liệu JSON hoặc XML, các bảng trong một relational database, hoặc các đỉnh và cạnh trong một graph. Những data model đó chính là chủ đề của chương này.
3. Các kỹ sư xây dựng phần mềm database của bạn đã quyết định cách biểu diễn dữ liệu dạng document, relational, hoặc graph đó dưới dạng các byte trong bộ nhớ, trên đĩa hoặc trên mạng. Cách biểu diễn này có thể cho phép dữ liệu được truy vấn, tìm kiếm, thao tác và xử lý theo nhiều cách khác nhau. Chúng ta sẽ thảo luận về các thiết kế storage engine này trong Chương 4.
4. Ở các cấp thấp hơn nữa, các kỹ sư phần cứng đã tìm ra cách biểu diễn các byte dưới dạng dòng điện, xung ánh sáng, từ trường, và nhiều thứ khác.

Trong một ứng dụng phức tạp, có thể có thêm nhiều cấp trung gian, chẳng hạn như các API được xây dựng dựa trên các API khác, nhưng ý tưởng cơ bản vẫn không đổi: mỗi lớp che giấu sự phức tạp của các lớp bên dưới bằng cách cung cấp một data model sạch sẽ. Những sự trừu tượng hóa này cho phép các nhóm người khác nhau—ví dụ, các kỹ sư tại nhà cung cấp database và các lập trình viên ứng dụng sử dụng database của họ—có thể làm việc với nhau một cách hiệu quả.

Một số data model được sử dụng rộng rãi trong thực tế, thường cho các mục đích khác nhau. Một số loại dữ liệu và một số truy vấn dễ dàng biểu diễn trong mô hình này nhưng lại khó khăn trong mô hình khác. Trong chương này, chúng ta sẽ khám phá những sự đánh đổi (trade-off) đó bằng cách so sánh relational model, document model, các data model dựa trên graph, event sourcing, và DataFrames. Chúng ta cũng sẽ xem xét ngắn gọn các query languages cho phép bạn làm việc với các mô hình này. Sự so sánh này sẽ giúp bạn quyết định khi nào nên sử dụng mô hình nào.

Thuật ngữ: Declarative Query Languages

Nhiều query languages được thảo luận trong chương này (chẳng hạn như SQL, Cypher, SPARQL, và Datalog) có tính *declarative* (khai báo), nghĩa là bạn chỉ định mẫu dữ liệu mà bạn muốn—những điều kiện mà kết quả phải đáp ứng và cách bạn muốn dữ liệu được biến đổi (ví dụ: sắp xếp, nhóm, và tổng hợp)—nhưng không chỉ định *cách thức* để đạt được mục tiêu đó. Bộ query optimizer của hệ thống database có thể quyết định sử dụng các index và thuật toán join nào, cũng như thứ tự thực thi các phần khác nhau của truy vấn.

Ngược lại, với hầu hết các ngôn ngữ lập trình (chẳng hạn như Python và Java), bạn sẽ phải viết một *algorithm* (thuật toán) để chỉ cho máy tính biết cần thực hiện các thao tác nào theo thứ tự nào. Một declarative query language rất hấp dẫn vì nó thường súc tích hơn và dễ viết hơn so với một algorithm tường minh. Quan trọng hơn, nó che giấu các chi tiết triển khai của query engine, điều này giúp hệ thống database có thể đưa ra các cải tiến về hiệu năng mà không yêu cầu bất kỳ thay đổi nào đối với các truy vấn [1, 2].

Ví dụ, một database có thể thực thi một truy vấn declarative song song trên nhiều lõi CPU và máy tính khác nhau mà bạn không cần phải lo lắng về cách triển khai tính song song đó [3]. Với một algorithm được viết bằng tay, việc tự mình triển khai việc thực thi song song như vậy sẽ là một khối lượng công việc rất lớn.

Mô hình Relational so với Mô hình Document

Mô hình dữ liệu nổi tiếng nhất hiện nay có lẽ là mô hình của SQL, dựa trên mô hình relational (quan hệ) do Edgar Codd đề xuất vào năm 1970 [4]. Trong mô hình này, dữ liệu được tổ chức thành các *relation* (được gọi là *table* trong SQL), trong đó mỗi relation là một tập hợp không thứ tự gồm các *tuple* (row trong SQL).

Mô hình relational ban đầu là một đề xuất lý thuyết, và nhiều người vào thời điểm đó đã nghi ngờ liệu nó có thể được triển khai một cách hiệu quả hay không. Tuy nhiên, đến giữa những năm 1980, các hệ quản trị cơ sở dữ liệu relational (RDBMSs) và SQL đã trở thành công cụ được lựa chọn cho hầu hết những người cần lưu trữ và truy vấn dữ liệu có cấu trúc quy tắc nào đó. Nhiều trường hợp sử dụng quản lý dữ liệu—ví dụ, phân tích kinh doanh (xem “Stars and Snowflakes: Schemas for Analytics”)—vẫn bị thống trị bởi dữ liệu relational nhiều thập kỷ sau đó.

Qua nhiều năm, đã có nhiều cách tiếp cận cạnh tranh nhau về lưu trữ và truy vấn dữ liệu. Trong những năm 1970 và đầu những năm 1980, *network model* và *hierarchical model* là những lựa chọn thay thế chính, nhưng mô hình relational đã trở nên thống trị chúng. Các object database (đừng nhầm lẫn với object storage dành cho các tệp lớn, một dịch vụ cloud phổ biến hiện nay) đã xuất hiện rồi biến mất vào cuối những năm 1980 và đầu những năm 1990. Các XML database xuất hiện vào đầu những năm 2000, nhưng chúng chỉ được áp dụng trong các thị trường ngách. Mỗi đối thủ cạnh tranh với mô hình relational đều tạo ra rất nhiều sự kỳ vọng (hype) vào thời điểm của nó, nhưng không có cái nào tồn tại lâu dài [5]. Thay vào đó, SQL đã phát triển để tích hợp các loại dữ liệu khác—ví dụ, thêm hỗ trợ cho XML, JSON và dữ liệu graph [6].

Trong những năm 2010, *NoSQL* là thuật ngữ thời thượng mới nhất cố gắng lật đổ sự thống trị của các cơ sở dữ liệu relational. NoSQL không ám chỉ một công nghệ duy nhất mà là một tập hợp các ý tưởng lỏng lẻo xoay quanh các mô hình dữ liệu mới, tính linh hoạt của schema, khả năng mở rộng, và sự chuyển dịch hướng tới các mô hình cấp phép mã nguồn mở. Một số cơ sở dữ liệu tự gắn nhãn là *NewSQL*, phản ánh mục tiêu cung cấp khả năng mở rộng của các hệ thống NoSQL cùng với mô hình dữ liệu và các đảm bảo về transaction của các cơ sở dữ liệu relational truyền thống. Các ý tưởng NoSQL và NewSQL đã có ảnh hưởng rất lớn trong việc thiết kế các hệ thống dữ liệu, nhưng khi các nguyên tắc này đã được áp dụng rộng rãi, việc sử dụng các thuật ngữ đó đã mờ nhạt dần.

Một hiệu ứng lâu dài của phong trào NoSQL là sự phổ biến của *document model*, vốn thường biểu diễn dữ liệu dưới dạng JSON. Mô hình này ban đầu được phổ biến bởi các document database chuyên dụng như MongoDB và Couchbase, mặc dù hầu hết các cơ sở dữ liệu relational hiện nay cũng đã thêm hỗ trợ JSON. So với các table trong relational, vốn thường được xem là có schema cứng nhắc và thiếu linh hoạt, các tài liệu JSON được cho là linh hoạt hơn.

Ưu và nhược điểm của dữ liệu document và relational đã được tranh luận rộng rãi. Hãy cùng xem xét một số điểm chính của cuộc tranh luận đó.

Sự không tương thích Object-Relational

Phần lớn việc phát triển ứng dụng ngày nay được thực hiện bằng các ngôn ngữ lập trình hướng đối tượng, điều này dẫn đến một lời chỉ trích phổ biến đối với mô hình dữ liệu SQL: nếu dữ liệu được lưu trữ trong các table relational, thì cần một lớp chuyển đổi vùng về giữa các object trong mã nguồn ứng dụng và mô hình database gồm các table, row và column. Sự mất kết nối giữa các mô hình này đôi khi được gọi là *impedance mismatch*.

LƯU Ý

Thuật ngữ *impedance mismatch* được mượn từ lĩnh vực điện tử. Mọi mạch điện đều có một trở kháng (impedance) nhất định (sự kháng lại dòng điện xoay chiều) tại các đầu vào và đầu ra của nó. Khi bạn kết nối đầu ra của một mạch với đầu vào của một mạch khác, việc truyền tải năng lượng qua kết nối sẽ đạt mức tối đa nếu trở kháng đầu ra và đầu vào của hai mạch khớp nhau. Một sự không tương thích trở kháng (impedance mismatch) có thể dẫn đến phản xạ tín hiệu và các vấn đề khác.

Ảnh xạ Object-Relational

Các framework Object-relational mapping (ORM) như ActiveRecord và Hibernate giúp giảm lượng mã boilerplate cần thiết cho lớp chuyển đổi này, nhưng chúng thường bị chỉ trích [7]. Một số vấn đề thường được trích dẫn như sau:

- Các ORM rất phức tạp và không thể giấu hoàn toàn sự khác biệt giữa hai mô hình, vì vậy các lập trình viên cuối cùng vẫn phải suy nghĩ về cả biểu diễn dữ liệu theo kiểu quan hệ (relational) và biểu diễn đối tượng (object).
- Các ORM thường chỉ được sử dụng để phát triển ứng dụng OLTP (xem mục “Đặc điểm của Xử lý giao dịch và Phân tích”); các kỹ sư dữ liệu khi chuẩn bị dữ liệu cho mục đích phân tích cần phải làm việc với biểu diễn quan hệ bên dưới, do đó việc thiết kế schema quan hệ vẫn quan trọng khi sử dụng một ORM.
- Nhiều ORM chỉ hoạt động với các cơ sở dữ liệu quan hệ OLTP. Các tổ chức có hệ thống dữ liệu đa dạng như công cụ tìm kiếm, cơ sở dữ liệu graph và các hệ thống NoSQL có thể thấy rằng sự hỗ trợ của ORM còn thiếu sót.
- Một số ORM tự động tạo ra các schema quan hệ, nhưng chúng có thể gây khó khăn cho những người dùng đang truy cập trực tiếp vào dữ liệu quan hệ, và chúng có thể hoạt động không hiệu quả trên cơ sở dữ liệu bên dưới. Việc tùy chỉnh schema và quá trình tạo truy vấn của ORM có thể rất phức tạp và làm mất đi lợi ích của việc sử dụng ORM ngay từ đầu.
- ORMs khiến việc vô tình viết các truy vấn không hiệu quả trở nên dễ dàng. Một ví dụ về điều này là *vấn đề truy vấn N+1* [8]. Ví dụ, giả sử bạn muốn hiển thị một danh sách các bình luận của người dùng trên một trang, vì vậy bạn thực hiện một truy vấn trả về N bình luận, mỗi bình luận đều chứa ID của tác giả. Để hiển thị

tên của tác giả mỗi bình luận, bạn cần tra cứu ID đó trong bảng `users`. Nếu viết SQL thủ công, bạn có lẽ sẽ thực hiện phép join này ngay trong truy vấn và trả về tên tác giả cùng với mỗi bình luận. Tuy nhiên, với một ORM, bạn có thể kết thúc bằng việc thực hiện một truy vấn riêng biệt trên bảng `users` cho mỗi bình luận trong số N bình luận để tra cứu tác giả, dẫn đến tổng cộng có $N+1$ truy vấn cơ sở dữ liệu, điều này chậm hơn so với việc thực hiện phép join trong cơ sở dữ liệu. Để tránh vấn đề này, bạn có thể cần yêu cầu ORM lấy thông tin tác giả cùng lúc với việc lấy các bình luận.

Tuy nhiên, ORMs cũng có những ưu điểm:

- Đối với dữ liệu phù hợp với mô hình quan hệ, một kiểu chuyển đổi giữa biểu diễn quan hệ bền vững (persistent) và biểu diễn đối tượng trong bộ nhớ (in-memory) là không thể tránh khỏi, và ORMs giúp giảm lượng mã boilerplate cần thiết cho việc chuyển đổi này. Các truy vấn phức tạp có thể vẫn cần được xử lý bên ngoài ORM, nhưng ORM có thể giúp xử lý các trường hợp đơn giản và lặp đi lặp lại.
- Một số ORMs hỗ trợ caching kết quả của các truy vấn cơ sở dữ liệu, điều này có thể giúp giảm tải cho cơ sở dữ liệu.
- ORMs cũng có thể giúp quản lý schema migrations và các hoạt động quản trị khác.

Mô hình dữ liệu tài liệu cho các mối quan hệ một-nhiều

Không phải tất cả dữ liệu đều phù hợp với biểu diễn quan hệ. Hãy cùng xem xét một ví dụ để khám phá một hạn chế của mô hình quan hệ. Hình 3-1 minh họa cách một bản sơ yếu lý lịch (một hồ sơ LinkedIn) có thể được thể hiện trong một schema quan hệ. Toàn bộ hồ sơ có thể được xác định bằng một định danh duy nhất, `user_id`. Các trường như `first_name` và `last_name` xuất hiện chính xác một lần cho mỗi người dùng, vì vậy chúng có thể được mô hình hóa thành các cột trên bảng `users`.

Hầu hết mọi người đều có nhiều hơn một công việc (vị trí) trong sự nghiệp của họ, và mọi người có thể có số lượng các giai đoạn giáo dục khác nhau cũng như bất kỳ số lượng thông tin liên lạc nào. Một cách để biểu diễn các *mối quan hệ một-nhiều* như vậy là đưa các vị trí, giáo dục và thông tin liên lạc vào các bảng riêng biệt, mỗi bảng đều có một tham chiếu khóa ngoại (foreign-key) tới bảng `users`, như trong Hình 3-1.

Một cách khác để biểu diễn cùng một thông tin, điều có lẽ tự nhiên hơn và ánh xạ gần gũi hơn với cấu trúc đối tượng trong mã ứng dụng, là dưới dạng một tài liệu JSON, như được hiển thị trong Ví dụ 3-1.

<https://www.linkedin.com/in/barackobama/>



Barack Obama

Washington, DC, United States

Former President of the United States of America

Experience

President • United States of America
2009 – 2017

US Senator (D-IL) • United States Senate
2005 – 2008

Education

Juris Doctor, Law • Harvard University
1988 – 1991

Bachelor of Arts • Columbia University
1981 – 1983

Contact Info

Website: barackobama.com
X: @barackobama

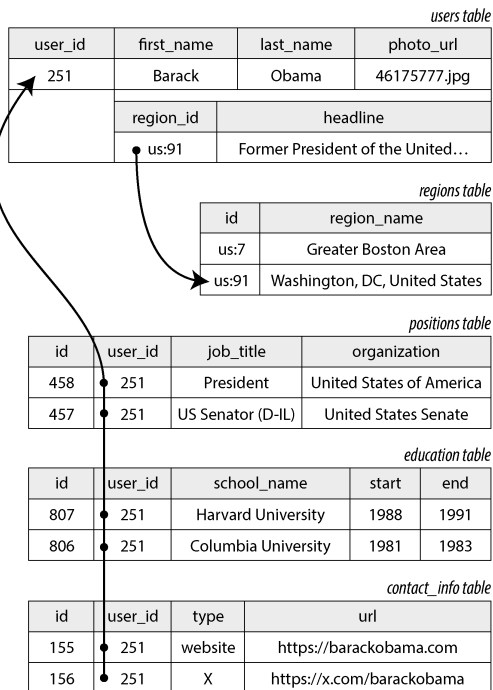


Figure 3-1. Sử dụng một relational schema để biểu diễn một hồ sơ LinkedIn

Example 3-1. Biểu diễn một hồ sơ LinkedIn dưới dạng tài liệu JSON

```
{
  "user_id": 251,
  "first_name": "Barack",
  "last_name": "Obama",
  "headline": "Former President of the United States of America",
  "region_id": "us:91",
  "photo_url": "/p/7/000/253/05b/308dd6e.jpg",
  "positions": [
    {"job_title": "President", "organization": "United States of America"},
    {"job_title": "US Senator (D-IL)", "organization": "United States Senate"}
  ],
  "education": [
    {"school_name": "Harvard University", "start": 1988, "end": 1991},
    {"school_name": "Columbia University", "start": 1981, "end": 1983}
  ],
  "contact_info": {
    "website": "https://barackobama.com",
    "x": "https://x.com/barackobama"
  }
}
```

Một số lập trình viên cảm thấy rằng mô hình JSON giúp giảm thiểu impedance mismatch (sự không tương thích về cấu trúc dữ liệu) giữa mã ứng dụng và lớp lưu trữ. Việc thiếu một schema cũng thường được coi là một lợi thế; chúng ta sẽ thảo luận điều này trong mục “Schema flexibility in the document model”. Tuy nhiên, như chúng ta sẽ thấy trong Chương 5, JSON cũng gặp phải một số vấn đề khi đóng vai trò là một định dạng encoding dữ liệu.

Biểu diễn JSON có tính *locality* (tính cục bộ) tốt hơn so với schema đa bảng trong Hình 3-1 (xem “Data locality for reads and writes”). Nếu bạn muốn lấy một hồ sơ trong ví dụ về mô hình quan hệ, bạn cần phải thực hiện nhiều truy vấn (truy vấn từng bảng bằng `user_id`) hoặc thực hiện một phép join đa hướng phức tạp giữa bảng `users` và các bảng con của nó [9, 10]. Trong biểu diễn JSON, tất cả thông tin liên quan đều nằm ở một nơi, giúp truy vấn vừa nhanh hơn vừa đơn giản hơn.

Các mối quan hệ một-nhiều từ hồ sơ người dùng đến các vị trí công việc, lịch sử giáo dục và thông tin liên hệ của người dùng hàm ý một cấu trúc cây trong dữ liệu, và biểu diễn JSON làm cho cấu trúc cây này trở nên rõ ràng (xem Hình 3-2).

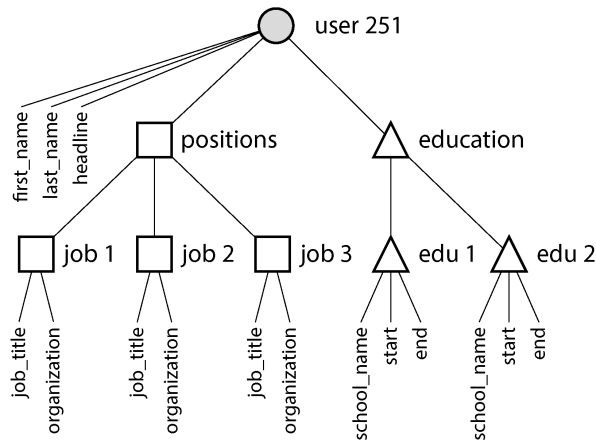


Figure 3-2. Các mối quan hệ một-nhiều tạo thành một cấu trúc cây

LƯU Ý

Một mối quan hệ một-nhiều đôi khi được gọi là *one-to-few* (một-với-vài), vì một bản sơ yếu lý lịch (résumé) thường chỉ có một số lượng nhỏ các vị trí công tác [11, 12]. Nếu bạn có một số lượng thực sự lớn các mục liên quan—chẳng hạn như các bình luận trên một bài đăng mạng xã hội của một người nổi tiếng, vốn có thể lên tới hàng nghìn—việc embedding tất cả chúng vào cùng một tài liệu có thể quá cồng kềnh, vì vậy cách tiếp cận quan hệ (relational) trong Hình 3-1 sẽ được ưu tiên hơn.

Normalization, Denormalization, và Joins

Trong Ví dụ 3-1 ở mục trước, `region_id` được đưa ra dưới dạng một ID, chứ không phải là chuỗi văn bản thuần túy Washington, DC, United States. Tại sao?

Nếu UI có một trường văn bản tự do để nhập vùng miền, việc lưu trữ nó dưới dạng một chuỗi văn bản thuần túy là hợp lý. Tuy nhiên, việc có các danh sách chuẩn hóa cho các vùng địa lý và cho phép người dùng chọn từ một danh sách thả xuống (drop-down list) hoặc tính năng tự động hoàn thành (autocomplete) sẽ mang lại nhiều lợi thế. Chúng bao gồm các điểm sau:

- Phong cách và chính tả nhất quán giữa các hồ sơ
- Tránh sự mơ hồ nếu nhiều địa danh có cùng tên (nếu chuỗi chỉ là Washington, DC, liệu nó sẽ ám chỉ DC hay là một tiểu bang?)
- Dễ dàng cập nhật—tên chỉ được lưu trữ ở một nơi duy nhất, vì vậy sẽ dễ dàng cập nhật trên toàn bộ hệ thống nếu nó cần được thay đổi (ví dụ: thay đổi tên thành phố do các sự kiện chính trị)

- Hỗ trợ bản địa hóa—khi trang web được dịch sang các ngôn ngữ khác, các danh sách chuẩn hóa có thể được bản địa hóa, nhờ đó vùng miền có thể được hiển thị bằng ngôn ngữ của người xem
- Chức năng tìm kiếm tốt hơn (ví dụ: một tìm kiếm về những người ở Bồ Đào Nha Kỳ có thể khớp với hồ sơ này, bởi vì danh sách các vùng có thể mã hóa thực tế rằng Washington nằm ở Bồ Đào Nha—điều mà nếu chỉ dựa vào chuỗi Washington, DC thì không thể thấy được)

Việc bạn lưu trữ một ID hay một chuỗi văn bản là một câu hỏi về *normalization*. Khi bạn sử dụng một ID, dữ liệu của bạn được chuẩn hóa hơn: thông tin có ý nghĩa với con người (chẳng hạn như văn bản *Washington, DC*) chỉ được lưu trữ ở một nơi duy nhất, và mọi thứ tham chiếu đến nó đều sử dụng một ID (thứ chỉ có ý nghĩa trong phạm vi cơ sở dữ liệu). Khi bạn lưu trữ trực tiếp văn bản, bạn đang nhân bản thông tin có ý nghĩa với con người trong mọi bản ghi có sử dụng nó; cách biểu diễn này là *denormalization* (phi chuẩn hóa).

Lợi thế của việc sử dụng ID là vì nó không có ý nghĩa với con người, nên nó không bao giờ cần phải thay đổi: ID có thể giữ nguyên ngay cả khi thông tin mà nó định danh bị thay đổi. Bất cứ thứ gì có ý nghĩa với con người đều có thể cần thay đổi vào một thời điểm nào đó trong tương lai—và nếu thông tin đó bị nhân bản, tất cả các bản sao dư thừa sẽ cần phải được cập nhật. Điều đó đòi hỏi nhiều mã nguồn hơn, nhiều thao tác ghi hơn, tốn nhiều không gian đĩa hơn và có nguy cơ gây ra sự không nhất quán (vì một số bản sao của thông tin được cập nhật nhưng những bản khác thì không).

Nhược điểm của một cách biểu diễn đã được chuẩn hóa là mỗi khi bạn muốn hiển thị một bản ghi chứa một ID, bạn phải thực hiện thêm một thao tác tra cứu để chuyển đổi ID đó thành thứ gì đó mà con người có thể đọc được. Trong một mô hình dữ liệu quan hệ, việc này được thực hiện bằng cách sử dụng một *join*. Ví dụ:

```
SELECT users.*, regions.region_name
FROM users
JOIN regions ON users.region_id = regions.id
WHERE users.id = 251;
```

Các cơ sở dữ liệu document có thể lưu trữ cả dữ liệu đã được chuẩn hóa và dữ liệu phi chuẩn hóa, nhưng chúng thường gắn liền với việc denormalization—một phần vì mô hình dữ liệu JSON giúp việc lưu trữ các trường phi chuẩn hóa bổ sung trở nên dễ dàng, và một phần vì sự hỗ trợ yếu kém cho các phép *join* trong nhiều cơ sở dữ liệu document khiến việc *normalization* trở nên bất tiện. Một số cơ sở dữ liệu document thậm chí không hỗ trợ *join*, vì vậy bạn phải thực hiện chúng trong mã nguồn ứng dụng—tức là, trước tiên bạn lấy một document chứa một ID, sau đó thực hiện truy vấn thứ hai để chuyển đổi ID đó thành một document khác. Trong MongoDB, ta cũng có thể thực hiện một phép *join* bằng cách sử dụng toán tử *\$lookup* trong một *aggregation pipeline*:

```

db.users.aggregate([
  { $match: { _id: 251 } },
  { $lookup: {
    from: "regions",
    localField: "region_id",
    foreignField: "_id",
    as: "region"
  } }
])

```

Đánh đổi (trade-off) của normalization

Trong ví dụ về sơ yếu lý lịch, trong khi trường `region_id` là một tham chiếu đến một tập hợp các vùng đã được chuẩn hóa, thì các trường `organization` (công ty hoặc chính phủ nơi người đó đã làm việc) và `school_name` (nơi họ đã học) chỉ là các chuỗi văn bản. Cách biểu diễn này là denormalized: nhiều người có thể đã làm việc tại cùng một công ty, nhưng không có ID nào liên kết họ lại với nhau.

Chúng ta cũng nên cân nhắc liệu tên tổ chức và trường học có nên là các entity (thực thể) thay thế hay không, và profile nên tham chiếu đến các ID của chúng. Những lập luận tương tự dành cho việc tham chiếu ID của một khu vực cũng được áp dụng ở đây. Ví dụ, giả sử chúng ta muốn bao gồm cả logo của trường học hoặc công ty bên cạnh tên của nó:

- Trong một biểu diễn denormalized (phi chuẩn hóa), chúng ta sẽ bao gồm URL hình ảnh của logo trên mọi profile cá nhân. Điều này giúp tài liệu JSON trở nên độc lập, nhưng nó sẽ gây ra một vấn đề đau đầu nếu chúng ta cần thay đổi logo, vì khi đó chúng ta phải tìm tất cả các vị trí xuất hiện của URL cũ và cập nhật chúng [11].
- Trong một biểu diễn normalized (chuẩn hóa), chúng ta sẽ tạo ra một entity đại diện cho một tổ chức hoặc trường học và lưu trữ tên, URL logo, và có thể là các thuộc tính khác (mô tả, news feed, v.v.) một lần duy nhất như một phần của entity đó. Mọi résumé có đề cập đến tổ chức đó sau đó sẽ chỉ đơn giản là tham chiếu đến ID của nó, và việc cập nhật logo sẽ trở nên dễ dàng.

Theo một nguyên tắc chung, dữ liệu normalized thường ghi nhanh hơn (vì chỉ có một bản sao duy nhất) nhưng truy vấn chậm hơn (vì yêu cầu các phép join); dữ liệu denormalized thường đọc nhanh hơn (ít phép join hơn) nhưng ghi tốn kém hơn (nhiều bản sao cần cập nhật hơn, sử dụng nhiều không gian đĩa hơn). Bạn có thể thấy hữu ích khi xem denormalization như một dạng của derived data (dữ liệu phái sinh) (xem mục “Systems of Record and Derived Data”), vì bạn cần thiết lập một quy trình để cập nhật các bản sao dư thừa của dữ liệu.

Bên cạnh chi phí thực hiện tất cả các cập nhật này, bạn cần xem xét tính nhất quán của cơ sở dữ liệu nếu một quy trình bị crash khi đang thực hiện cập nhật dở dang. Các cơ sở dữ liệu cung cấp atomic transactions (giao dịch nguyên tử) (xem mục

“Atomicity”) giúp việc duy trì tính nhất quán trở nên dễ dàng hơn, nhưng không phải tất cả cơ sở dữ liệu đều cung cấp tính nguyên tử trên nhiều document. Việc đảm bảo tính nhất quán thông qua stream processing cũng là một khả năng, điều mà chúng ta sẽ thảo luận trong Chương 12.

Normalization có xu hướng tốt hơn cho các hệ thống OLTP, nơi cả việc đọc và cập nhật đều cần phải nhanh; các hệ thống phân tích thường hoạt động tốt hơn với dữ liệu denormalized, vì chúng thực hiện cập nhật theo lô và hiệu năng của các truy vấn chỉ đọc là mối quan tâm chính. Trong các hệ thống có quy mô nhỏ đến trung bình, một mô hình dữ liệu normalized thường là tốt nhất vì bạn không phải lo lắng về việc giữ cho nhiều bản sao dữ liệu nhất quán với nhau, và chi phí thực hiện các phép join là có thể chấp nhận được. Tuy nhiên, trong các hệ thống quy mô rất lớn, chi phí của các phép join có thể trở thành vấn đề.

Denormalization trong nghiên cứu tình huống mạng xã hội

Trong “Case Study: Social Network Home Timelines”, chúng ta đã so sánh một biểu diễn normalized (Hình 2-1) và một biểu diễn denormalized (các timeline đã được tính toán trước, được materialized). Ở đây, phép join giữa posts và follows quá tốn kém, và materialized timeline là một cache của kết quả phép join đó. Quy trình fan-out nhằm chèn một bài đăng mới vào timeline của những người theo dõi chính là cách chúng ta giữ cho biểu diễn denormalized được nhất quán.

Tuy nhiên, việc triển khai các materialized timelines tại X (trước đây là Twitter) không lưu trữ văn bản thực tế của mỗi bài đăng. Mỗi mục chỉ lưu trữ ID của bài đăng, ID của người dùng đã đăng bài, và một chút thông tin bổ sung để xác định các bài đăng lại (reposts) và phản hồi (replies) [13]. Nói cách khác, nó là một kết quả được tính toán trước của (xấp xỉ) truy vấn sau đây:

```
SELECT posts.id, posts.sender_id FROM posts
  JOIN follows ON posts.sender_id = follows.followee_id
 WHERE follows.follower_id = current_user
 ORDER BY posts.timestamp DESC
 LIMIT 1000
```

Điều này có nghĩa là bất cứ khi nào timeline được đọc, dịch vụ vẫn cần thực hiện hai phép join: nó tra cứu ID bài đăng để lấy nội dung bài đăng thực tế (cũng như các số liệu thống kê như số lượt thích và phản hồi), và nó tra cứu profile của người gửi bằng ID (để lấy username, ảnh đại diện và các chi tiết khác). Quy trình tra cứu thông tin có thể đọc được bằng người này thông qua ID được gọi là *hydrating* các ID, và về cơ bản nó là một phép join được thực hiện trong mã nguồn ứng dụng [13].

Lý do chúng ta chỉ lưu trữ các ID trong timeline đã được tính toán trước là vì dữ liệu mà chúng tham chiếu đến thay đổi rất nhanh. Số lượng lượt thích và phản hồi có thể thay đổi nhiều lần mỗi giây đối với một bài đăng phổ biến, và một số người dùng thường xuyên thay đổi tên người dùng hoặc ảnh hồ sơ của họ. Vì timeline nên hiển

thị số lượt thích và ảnh hồ sơ mới nhất khi được xem, nên việc denormalization (phi chuẩn hóa) thông tin này vào materialized timeline sẽ không hợp lý. Hơn nữa, chi phí lưu trữ sẽ tăng lên đáng kể nếu thực hiện denormalization như vậy.

Ví dụ này cho thấy việc phải thực hiện các phép join khi đọc dữ liệu không phải là một trở ngại đối với việc tạo ra các dịch vụ có hiệu năng cao và khả năng mở rộng, như đôi khi có người khẳng định. Việc hydrating các ID bài đăng và ID người dùng thực tế là một thao tác khá dễ dàng để scale, vì nó có khả năng song song hóa tốt, và chi phí không phụ thuộc vào số lượng tài khoản mà bạn đang theo dõi hay số lượng người theo dõi mà bạn có.

Nếu bạn cần quyết định xem có nên denormalize thứ gì đó trong ứng dụng của mình hay không, nghiên cứu tình huống về mạng xã hội cho thấy lựa chọn này không hề hiển nhiên; cách tiếp cận có khả năng mở rộng nhất có thể bao gồm việc denormalize một số thứ và để những thứ khác ở dạng normalized. Bạn sẽ phải cân nhắc kỹ lưỡng tần suất thay đổi của thông tin cũng như chi phí đọc và ghi (vốn có thể bị chi phối bởi các outlier, chẳng hạn như những người dùng có rất nhiều lượt theo dõi/người theo dõi trong trường hợp của một mạng xã hội điển hình). Normalization và denormalization không tự thân là tốt hay xấu—chúng đơn giản là đại diện cho các trade-off về mặt hiệu năng của các thao tác đọc, ghi và nỗ lực triển khai.

Mối quan hệ Many-to-One và Many-to-Many

Trong khi các bảng `positions` và `education` trong Hình 3-1 là ví dụ về các mối quan hệ one-to-many hoặc one-to-few (một sơ yếu lý lịch có nhiều vị trí, nhưng mỗi vị trí chỉ thuộc về một sơ yếu lý lịch), thì trường `region_id` là một ví dụ về mối quan hệ *many-to-one* (nhiều người sống trong cùng một khu vực, nhưng chúng ta giả định rằng mỗi người chỉ sống ở một khu vực tại bất kỳ thời điểm nào).

Nếu chúng ta đưa vào các thực thể cho tổ chức và trường học rồi tham chiếu chúng bằng ID từ sơ yếu lý lịch, thì chúng ta cũng có các mối quan hệ *many-to-many* (một người có thể đã làm việc cho nhiều tổ chức, và một tổ chức có nhiều nhân viên cũ hoặc hiện tại). Trong một relational model, một mối quan hệ như vậy thường được biểu diễn dưới dạng một *associative table*, hoặc *join table*, như được hiển thị trong Hình 3-3: mỗi vị trí liên kết một ID người dùng với một ID tổ chức.

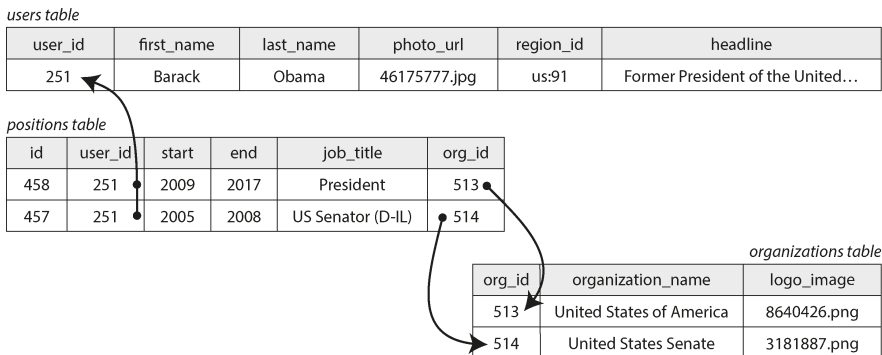


Figure 3-3. **Hình: Các mối quan hệ many-to-many trong relational model**

Các mối quan hệ many-to-one và many-to-many không dễ dàng khớp vào trong một tài liệu JSON độc lập; chúng phù hợp hơn với một cách biểu diễn đã được *normalization* (chuẩn hóa). Trong một *document model*, một cách biểu diễn khả thi được đưa ra trong Ví dụ 3-2 và được minh họa trong Hình 3-4. Dữ liệu bên trong mỗi hình chữ nhật nét đứt có thể được nhóm thành một *document*, nhưng các liên kết tới các tổ chức và trường học thì được biểu diễn tốt nhất dưới dạng các tham chiếu tới các *document* khác.

Example 3-2. Một bản sơ yếu lý lịch tham chiếu các tổ chức bằng ID

```
{
  "user_id":    251,
  "first_name": "Barack",
  "last_name":  "Obama",
  "positions": [
    {"start": 2009, "end": 2017, "job_title": "President",
     "org_id": 513},
    {"start": 2005, "end": 2008, "job_title": "US Senator (D-IL)",
     "org_id": 514}
  ],
  ...
}
```

Các mối quan hệ many-to-many thường cần được truy vấn theo “cả hai hướng”—ví dụ, tìm tất cả các tổ chức mà một người cụ thể đã từng làm việc, và tìm tất cả những người đã từng làm việc tại một tổ chức cụ thể. Một cách để cho phép các truy vấn như vậy là lưu trữ các tham chiếu ID ở cả hai phía, sao cho một bản sơ yếu lý lịch bao gồm ID của mỗi tổ chức mà người đó đã làm việc, và *document* của tổ chức bao gồm các ID của các bản sơ yếu lý lịch có nhắc đến tổ chức đó. Cách biểu diễn này là *denormalized*, vì mỗi quan hệ được lưu trữ ở hai nơi, điều này có thể dẫn đến việc chúng trở nên không nhất quán với nhau.

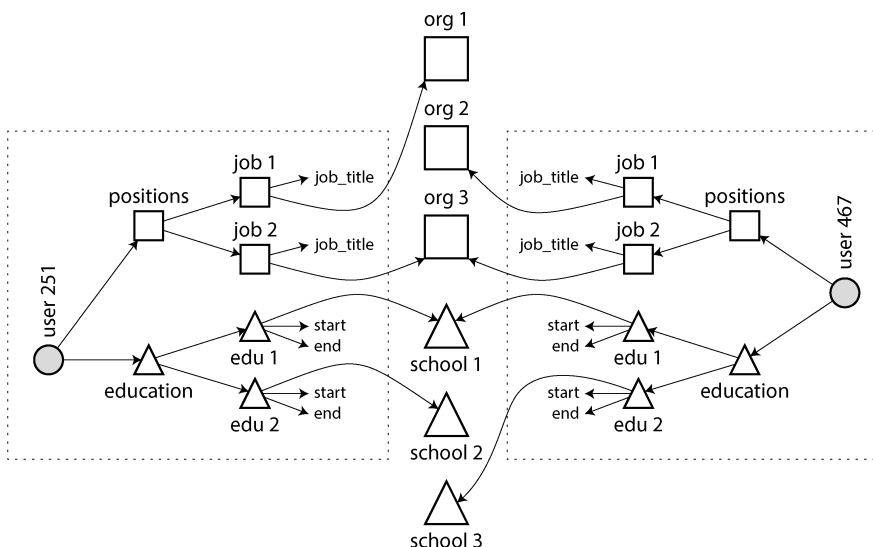


Figure 3-4. Các mối quan hệ many-to-many trong document model—dữ liệu bên trong mỗi hộp nét đứt có thể được nhóm thành một document

Một biểu diễn chuẩn hóa (normalized representation) chỉ lưu trữ mối quan hệ ở duy nhất một nơi và dựa vào các *secondary index* (mà chúng ta sẽ thảo luận trong Chương 4) để cho phép truy vấn mối quan hệ đó một cách hiệu quả theo cả hai hướng. Trong lược đồ quan hệ (relational schema) được hiển thị ở Hình 3-3, chúng ta sẽ yêu cầu cơ sở dữ liệu tạo các index trên cả hai cột `user_id` và `org_id` của bảng `positions`.

Trong document model của Ví dụ 3-2, cơ sở dữ liệu cần đánh index field `org_id` của các đối tượng nằm bên trong mảng `positions`. Nhiều document database và các relational database có hỗ trợ JSON có khả năng tạo các index như vậy trên các giá trị nằm bên trong một document.

Stars và Snowflakes: Các lược đồ cho Analytics

Các data warehouse (xem “Data Warehousing”) thường là relational, và có một vài quy ước được sử dụng rộng rãi cho cấu trúc của các bảng trong một data warehouse, bao gồm star schema, snowflake schema, dimensional modeling [14], và one big table (OBT). Các cấu trúc này được tối ưu hóa cho nhu cầu của các nhà phân tích kinh doanh. Các quy trình ETL chuyển đổi dữ liệu từ các hệ thống vận hành (operational systems) sang lược đồ đã chọn.

Hình 3-5 cho thấy một ví dụ về *star schema* (sơ đồ hình sao) có thể được tìm thấy trong data warehouse của một nhà bán lẻ thực phẩm. Tại trung tâm của schema là cái gọi là *fact table* (bảng sự kiện) (trong ví dụ này, nó được gọi là `fact_sales`). Mỗi hàng của fact table đại diện cho một sự kiện đã xảy ra tại một thời điểm cụ thể (ở đây,

mỗi hàng đại diện cho việc một khách hàng mua một sản phẩm). Nếu chúng ta đang phân tích lưu lượng truy cập website thay vì doanh số bán lẻ, mỗi hàng có thể đại diện cho một lượt xem trang hoặc một cú click của người dùng.

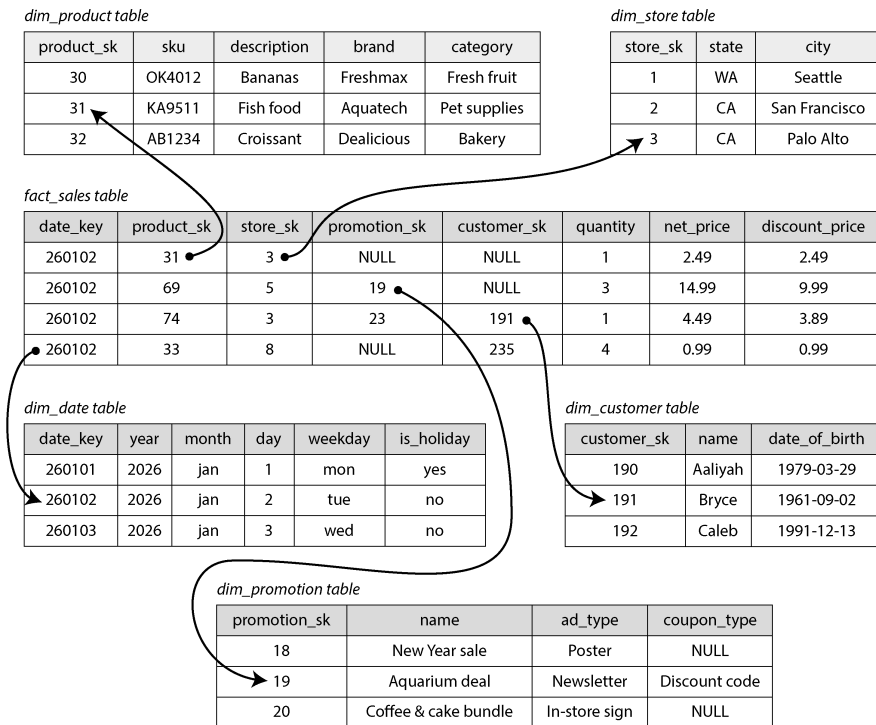


Figure 3-5. Một star schema dùng trong một data warehouse

Thông thường, các fact được ghi lại dưới dạng các sự kiện riêng lẻ, vì điều này cho phép sự linh hoạt tối đa trong việc phân tích sau này. Tuy nhiên, điều này có nghĩa là fact table có thể trở nên cực kỳ lớn. Một doanh nghiệp lớn có thể có hàng petabyte lịch sử giao dịch trong data warehouse của mình, phần lớn được biểu diễn dưới dạng các fact table.

Một số cột trong fact table là các thuộc tính, chẳng hạn như giá bán của sản phẩm và chi phí mua sản phẩm đó từ nhà cung cấp (cho phép tính toán biên lợi nhuận). Các cột khác trong fact table là các tham chiếu foreign key đến các bảng khác, được gọi là *dimension tables* (bảng chiều). Vì mỗi hàng trong fact table đại diện cho một sự kiện, nên các dimension sẽ đại diện cho các yếu tố *ai*, *cái gì*, *ở đâu*, *khi nào*, *như thế nào*, và *tại sao* của sự kiện đó.

Ví dụ, trong Hình 3-5, một trong các dimension là sản phẩm đã được bán. Mỗi hàng trong bảng `dim_product` đại diện cho một loại sản phẩm đang được bán, bao gồm mã đơn vị lưu kho (SKU), mô tả, tên thương hiệu, danh mục, hàm lượng chất béo và kích thước đóng gói. Mỗi hàng trong bảng `fact_sales` sử dụng một foreign key để

chỉ ra sản phẩm nào đã được bán trong giao dịch cụ thể đó. Các truy vấn thường liên quan đến nhiều phép join với nhiều dimension tables.

Ngay cả ngày và giờ cũng thường được biểu diễn bằng cách sử dụng các dimension tables, vì điều này cho phép mã hóa thêm thông tin về ngày tháng (chẳng hạn như các ngày lễ), giúp các truy vấn có thể phân biệt giữa doanh số vào ngày lễ và ngày thường.

Tên gọi *star schema* (sơ đồ hình sao) bắt nguồn từ thực tế là khi các mối quan hệ giữa các bảng được trực quan hóa, fact table nằm ở giữa, được bao quanh bởi các dimension tables của nó (như được hiển thị trong Hình 3-5); các kết nối tới các bảng này giống như các tia của một ngôi sao.

Một biến thể của mẫu này là *snowflake schema* (sơ đồ hình bông tuyết), nơi các dimension được chia nhỏ hơn thành các subdimensions. Ví dụ, có thể có các bảng riêng biệt cho thương hiệu và danh mục sản phẩm, và mỗi hàng trong bảng `dim_product` có thể tham chiếu đến thương hiệu và danh mục dưới dạng foreign keys, thay vì lưu trữ chúng dưới dạng chuỗi trong bảng `dim_product`. Các snowflake schemas có tính normalization cao hơn so với star schemas, nhưng star schemas thường được ưu tiên hơn vì chúng đơn giản hơn cho các nhà phân tích làm việc [14].

Trong một data warehouse điển hình, các bảng thường khá rộng: các fact tables thường có hơn một trăm cột, đôi khi lên tới vài trăm cột. Dimension tables cũng có thể rộng, vì chúng bao gồm tất cả metadata có thể liên quan đến việc phân tích—ví dụ, bảng `dim_store` có thể bao gồm chi tiết về việc các dịch vụ nào được cung cấp tại mỗi cửa hàng, liệu cửa hàng đó có tiệm bánh trong cửa hàng hay không, diện tích, ngày cửa hàng mở cửa lần đầu, khi nào nó được cải tạo lần cuối, và khoảng cách từ đó đến đường cao tốc gần nhất.

Một star hoặc snowflake schema bao gồm chủ yếu là các mối quan hệ many-to-one (ví dụ: nhiều lượt bán xảy ra cho một sản phẩm cụ thể, tại một cửa hàng cụ thể), được biểu diễn bằng việc fact table có các foreign keys trỏ vào các dimension tables, hoặc các dimension trỏ vào các subdimensions. Về nguyên tắc, các loại mối quan hệ khác có thể tồn tại, nhưng chúng thường được denormalized để đơn giản hóa các truy vấn. Ví dụ, nếu một khách hàng mua nhiều sản phẩm khác nhau cùng một lúc, giao dịch nhiều mặt hàng đó không được biểu diễn một cách tường minh; thay vào đó, fact table có một hàng riêng biệt cho mỗi sản phẩm được mua, và các fact đó tình cờ đều có cùng customer ID, store ID và timestamp.

Một số schema của data warehouse còn thực hiện denormalization sâu hơn nữa và loại bỏ hoàn toàn các dimension tables, thay vào đó là gộp thông tin từ các dimension vào các cột đã được denormalized trong fact table (về cơ bản là tính toán trước phép join giữa fact table và các dimension tables). Cách tiếp cận này được gọi là *one big*

table (OBT), và mặc dù nó yêu cầu nhiều không gian lưu trữ hơn, nhưng đôi khi nó cho phép thực hiện các truy vấn nhanh hơn [15].

Trong bối cảnh phân tích (analytics), việc denormalization như vậy không gây ra vấn đề, vì dữ liệu thường đại diện cho một log dữ liệu lịch sử vốn sẽ không thay đổi (ngoại trừ việc thỉnh thoảng có thể hiệu chỉnh một lỗi). Các vấn đề về tính nhất quán dữ liệu và write overheads phát sinh với denormalization trong các hệ thống OLTP không quá cấp bách trong phân tích.

Khi nào nên sử dụng mô hình nào

Các lập luận chính ủng hộ document data model là tính linh hoạt của schema, hiệu năng tốt hơn nhờ tính cục bộ (locality), và đối với một số ứng dụng, nó gần gũi với object model mà ứng dụng sử dụng hơn. Mô hình relational phản biện lại bằng cách cung cấp khả năng hỗ trợ tốt hơn cho các phép join và các mối quan hệ many-to-one hoặc many-to-many. Hãy cùng xem xét các lập luận này một cách chi tiết hơn.

Nếu dữ liệu trong ứng dụng của bạn có cấu trúc giống như document (tức là một cây các mối quan hệ one-to-many, nơi mà thông thường toàn bộ cây được tải lên cùng một lúc), thì việc sử dụng document model có lẽ là một ý tưởng hay. Kỹ thuật *shredding* (chia nhỏ) của relational—chia một cấu trúc giống document thành nhiều bảng (như `positions`, `education`, và `contact_info` trong Hình 3-1)—có thể dẫn đến các schema cồng kềnh và mã nguồn ứng dụng phức tạp không cần thiết.

Document model có những hạn chế. Ví dụ, bạn không thể tham chiếu trực tiếp đến một mục lồng nhau (nested item) bên trong một document; thay vào đó, bạn cần nói điều gì đó như, "mục thứ hai trong danh sách các vị trí của người dùng 251". Nếu bạn cần tham chiếu đến các mục lồng nhau, cách tiếp cận relational sẽ hoạt động tốt hơn, vì bạn có thể tham chiếu trực tiếp đến bất kỳ mục nào thông qua ID của nó.

Một số ứng dụng cho phép người dùng chọn thứ tự của các mục—ví dụ, hãy tưởng tượng một danh sách việc cần làm hoặc trình theo dõi vấn đề (issue tracker) nơi người dùng có thể kéo và thả các tác vụ để thay đổi thứ tự của chúng. Document model hỗ trợ tốt các ứng dụng như vậy, bởi vì các mục (hoặc ID của chúng) đơn giản là có thể được lưu trữ trong một JSON array để xác định thứ tự. Trong các cơ sở dữ liệu relational, không có cách tiêu chuẩn nào để biểu diễn các danh sách có thể thay đổi thứ tự như vậy, và nhiều mẹo khác nhau được sử dụng, chẳng hạn như sắp xếp theo một cột số nguyên (đòi hỏi phải đánh số lại khi bạn chèn vào giữa), duy trì một linked list các ID, hoặc sử dụng fractional indexing [16, 17, 18].

Tính linh hoạt của schema trong document model

Hầu hết các document databases, và cả sự hỗ trợ JSON trong các cơ sở dữ liệu relational, đều không áp đặt bất kỳ schema nào lên dữ liệu trong các document. Sự hỗ trợ XML trong các cơ sở dữ liệu relational thường đi kèm với việc xác thực

schema tùy chọn. Không có schema có nghĩa là các key và value tùy ý có thể được thêm vào một document, và khi đọc, các client không có sự đảm bảo về việc các document có thể chứa những field nào.

Document databases đôi khi được gọi là *schemaless*, nhưng điều đó gây hiểu lầm vì mã nguồn đọc dữ liệu thường giả định một loại cấu trúc nào đó—tức là, có một schema ngầm định, nhưng nó không được áp đặt bởi cơ sở dữ liệu [19]. Một thuật ngữ chính xác hơn là *schema-on-read* (cấu trúc của dữ liệu là ngầm định và chỉ được diễn giải khi dữ liệu được đọc), trái ngược với *schema-on-write* (cách tiếp cận truyền thống của các cơ sở dữ liệu relational, nơi schema là tường minh và cơ sở dữ liệu đảm bảo rằng tất cả dữ liệu đều tuân thủ nó khi dữ liệu được ghi) [20].

Schema-on-read tương tự như việc kiểm tra kiểu động (dynamic type checking) trong các ngôn ngữ lập trình, trong khi schema-on-write tương tự như việc kiểm tra kiểu tĩnh (static type checking). Giống như việc những người ủng hộ kiểm tra kiểu tĩnh và động có những cuộc tranh luận lớn về ưu điểm tương đối của chúng [21], việc áp đặt schema trong các cơ sở dữ liệu là một chủ đề gây tranh cãi, và nhìn chung không có người chiến thắng rõ ràng.

Sự khác biệt giữa các cách tiếp cận này đặc biệt dễ nhận thấy khi một ứng dụng muốn thay đổi định dạng dữ liệu của nó. Ví dụ, giả sử bạn hiện đang lưu trữ tên đầy đủ của mỗi người dùng trong một field, và thay vào đó bạn muốn lưu trữ tên (first name) và họ (last name) riêng biệt [22]. Trong một document database, bạn chỉ cần bắt đầu ghi các document mới với các field mới và có mã nguồn trong ứng dụng để xử lý trường hợp khi các document cũ được đọc. Ví dụ:

```
if (user && user.name && !user.first_name) {  
    // Documents written before Dec 8, 2023 don't have first_name  
    user.first_name = user.name.split(" ")[0];  
}
```

Nhược điểm của cách tiếp cận này là mọi phần của ứng dụng thực hiện đọc từ cơ sở dữ liệu giờ đây đều cần phải xử lý các document ở định dạng cũ, vốn có thể đã được ghi từ rất lâu trong quá khứ. Mặt khác, trong một cơ sở dữ liệu schema-on-write, bạn thường sẽ thực hiện một quá trình *migration* (di trú) theo hướng như sau:

```
ALTER TABLE users ADD COLUMN first_name text DEFAULT NULL;  
UPDATE users SET first_name = split_part(name, ' ', 1);      --  
PostgreSQL  
UPDATE users SET first_name = substring_index(name, ' ', 1);  
-- MySQL
```

Trong hầu hết các cơ sở dữ liệu relational, việc thêm một column với giá trị mặc định là nhanh chóng và không gặp vấn đề gì, ngay cả trên các bảng lớn. Tuy nhiên, việc chạy câu lệnh UPDATE có khả năng sẽ chậm trên một bảng lớn vì mọi hàng đều cần được ghi lại, và các thao tác schema khác (chẳng hạn như thay đổi kiểu dữ liệu của một column) cũng thường yêu cầu phải sao chép toàn bộ bảng.

Có nhiều công cụ khác nhau cho phép thực hiện loại thay đổi schema này ở chế độ chạy nền mà không gây downtime [23, 24, 25, 26], nhưng việc thực hiện các quá trình migration như vậy trên các cơ sở dữ liệu lớn vẫn là một thách thức về mặt vận hành. Có thể tránh được các quá trình migration phức tạp bằng cách thêm column `first_name` với giá trị mặc định là NULL (việc này rất nhanh) và điền dữ liệu vào đó tại thời điểm đọc, giống như cách bạn làm với một document database.

Cách tiếp cận schema-on-read sẽ có lợi nếu các mục trong collection không có cùng cấu trúc (tức là dữ liệu không đồng nhất); ví dụ:

- Có nhiều loại đối tượng khác nhau, và việc đưa mỗi loại đối tượng vào một bảng riêng biệt là không khả thi.
- Cấu trúc của dữ liệu được quyết định bởi các hệ thống bên ngoài mà bạn không có quyền kiểm soát và chúng có thể thay đổi bất cứ lúc nào.

Trong những tình huống như thế này, một schema có thể gây hại nhiều hơn là giúp ích, và các document không có schema (schemaless) có thể là một mô hình dữ liệu tự nhiên hơn nhiều. Nhưng khi tất cả các bản ghi được kỳ vọng là có cùng cấu trúc, các schema là một cơ chế hữu ích để tài liệu hóa và thực thi cấu trúc đó. Chúng ta sẽ thảo luận chi tiết hơn về schema và schema evolution trong Chương 5.

Data locality cho các thao tác đọc và ghi

Một document thường được lưu trữ dưới dạng một chuỗi liên tục duy nhất, được mã hóa dưới dạng JSON, XML, hoặc một biến thể nhị phân của chúng (chẳng hạn như BSON của MongoDB). Nếu ứng dụng của bạn thường xuyên cần truy cập toàn bộ document (ví dụ: để hiển thị nó lên một trang web), thì *storage locality* (tính cục bộ của lưu trữ) này mang lại lợi thế về hiệu năng. Nếu dữ liệu bị chia tách qua nhiều bảng, như trong Hình 3-1, cần phải thực hiện nhiều lần tra cứu index để truy xuất toàn bộ dữ liệu, điều này có thể yêu cầu nhiều lần tìm kiếm đĩa (disk seek) hơn và tốn nhiều thời gian hơn.

Lợi thế về tính cục bộ chỉ áp dụng nếu bạn cần các phần lớn của document tại cùng một thời điểm. Cơ sở dữ liệu thường cần tải toàn bộ document, điều này có thể gây lãng phí nếu bạn chỉ cần truy cập một phần nhỏ của một document lớn. Hơn nữa, khi cập nhật một document, toàn bộ document đó thường cần phải được ghi lại. Vì những lý do này, thông thường bạn nên giữ các document tương đối nhỏ và tránh các cập nhật nhỏ thường xuyên.

Tuy nhiên, việc lưu trữ các dữ liệu liên quan cùng nhau để đảm bảo tính cục bộ không chỉ giới hạn ở mô hình document. Ví dụ, cơ sở dữ liệu Spanner của Google cung cấp các thuộc tính cục bộ tương tự trong một mô hình dữ liệu relational, bằng cách cho phép schema khai báo rằng các hàng của một bảng nên được lồng ghép (interleaved) bên trong một bảng cha [27]. Oracle cũng cho phép điều tương tự bằng cách sử dụng một tính năng gọi là *multi-table index cluster tables* [28]. Mô hình dữ

liệu *wide-column* được phổ biến bởi Bigtable của Google và được sử dụng, ví dụ, trong HBase và Accumulo có các *column families*, vốn có mục đích tương tự trong việc quản lý tính cục bộ [29].

Ngôn ngữ truy vấn cho document

Một sự khác biệt khác giữa cơ sở dữ liệu relational và document là ngôn ngữ hoặc API mà bạn sử dụng để truy vấn nó. Hầu hết các cơ sở dữ liệu relational được truy vấn bằng SQL, nhưng các cơ sở dữ liệu document thì đa dạng hơn. Một số chỉ cho phép truy cập key-value theo primary key, trong khi số khác cũng cung cấp các secondary index để truy vấn các giá trị bên trong document, và một số cung cấp các ngôn ngữ truy vấn phong phú.

Các cơ sở dữ liệu XML thường được truy vấn bằng XQuery và XPath, vốn được thiết kế để cho phép thực hiện các truy vấn phức tạp, bao gồm cả các phép join giữa nhiều tài liệu, và định dạng kết quả của chúng dưới dạng XML [30]. JSON Pointer [31] và JSONPath [32] cung cấp một giải pháp tương đương với XPath dành cho JSON. aggregation pipeline của MongoDB, với toán tử \$lookup dùng để join mà chúng ta đã thấy trong mục “Normalization, Denormalization, and Joins”, là một ví dụ về ngôn ngữ truy vấn cho các tập hợp tài liệu JSON.

Hãy cùng xem xét một ví dụ khác để nắm bắt ngôn ngữ này—lần này là một phép aggregation, thứ đặc biệt cần thiết cho analytics. Hãy tưởng tượng bạn là một nhà sinh vật học biển, và bạn thêm một bản ghi quan sát vào cơ sở dữ liệu mỗi khi bạn nhìn thấy các loài động vật dưới đại dương. Bây giờ, bạn muốn tạo một báo cáo cho biết bạn đã nhìn thấy bao nhiêu con cá mập mỗi tháng. Trong PostgreSQL, bạn có thể biểu diễn truy vấn đó như thế này:

```
SELECT date_trunc('month', observation_timestamp) AS
observation_month, .
      sum(num_animals) AS total_animals
FROM observations
WHERE family = 'Sharks'
GROUP BY observation_month;
```

•

Hàm `date_trunc('month', observation_timestamp)` xác định tháng trong lịch chứa `timestamp` và trả về một `timestamp` khác đại diện cho thời điểm bắt đầu của tháng đó. Nói cách khác, hàm này làm tròn một `timestamp` xuống tháng gần nhất.

Truy vấn này trước tiên lọc các quan sát để chỉ hiển thị các loài thuộc họ Sharks, sau đó nhóm các quan sát theo tháng trong lịch mà chúng xảy ra, và cuối cùng cộng tổng số lượng động vật được nhìn thấy trong tất cả các quan sát của tháng đó. Truy vấn tương tự có thể được biểu diễn bằng aggregation pipeline của MongoDB như sau:

```

db.observations.aggregate([
  { $match: { family: "Sharks" } },
  { $group: {
    _id: {
      year: { $year: "$observationTimestamp" },
      month: { $month: "$observationTimestamp" }
    },
    totalAnimals: { $sum: "$numAnimals" }
  } }
]);

```

Ngôn ngữ aggregation pipeline có khả năng biểu đạt tương đương với một tập hợp con của SQL, nhưng nó sử dụng cú pháp dựa trên JSON thay vì cú pháp kiểu câu tiếng Anh của SQL. Sự khác biệt này có lẽ chỉ là vấn đề về sở thích.

Sự hội tụ của cơ sở dữ liệu document và relational

Cơ sở dữ liệu document và cơ sở dữ liệu relational ban đầu là những cách tiếp cận rất khác nhau đối với việc quản lý dữ liệu, nhưng chúng đã trở nên giống nhau hơn theo thời gian [33]. Cơ sở dữ liệu relational đã bổ sung hỗ trợ cho các kiểu dữ liệu JSON và các toán tử truy vấn, cũng như khả năng đánh index các thuộc tính bên trong các tài liệu. Một số cơ sở dữ liệu document (chẳng hạn như MongoDB, Couchbase và RethinkDB) đã bổ sung hỗ trợ cho các phép join, secondary index và các ngôn ngữ truy vấn declarative.

Sự hội tụ của các mô hình này là tin tốt cho các lập trình viên ứng dụng, bởi vì mô hình relational và mô hình document hoạt động tốt nhất khi bạn có thể kết hợp cả hai trong cùng một cơ sở dữ liệu. Nhiều cơ sở dữ liệu document cần các tham chiếu kiểu relational đến các tài liệu khác, và nhiều cơ sở dữ liệu relational có các phần mà sự linh hoạt của schema mang lại lợi ích. Các hệ thống lai giữa relational và document là một sự kết hợp mạnh mẽ.

LƯU Ý

Mô tả ban đầu của Codd về mô hình relational [4] đã cho phép một thứ tương tự như JSON bên trong một relational schema. Ông gọi đó là *nonsimple domains*. Ý tưởng là một giá trị trong một hàng không nhất thiết phải là một kiểu dữ liệu nguyên thủy như số hoặc chuỗi; nó cũng có thể là một quan hệ lồng nhau (bảng), vì vậy bạn có thể có một cấu trúc cây lồng nhau tùy ý làm giá trị. Cấu trúc này có thể so sánh với sự hỗ trợ JSON và XML đã được thêm vào SQL hơn 30 năm sau đó.

Các mô hình dữ liệu dạng graph

Trước đó chúng ta đã thấy rằng loại quan hệ là một feature quan trọng để phân biệt giữa các mô hình dữ liệu. Nếu ứng dụng của bạn chủ yếu có các quan hệ một-nhiều (dữ liệu cấu trúc dạng cây) và ít các quan hệ khác giữa các bản ghi, thì mô hình document là phù hợp.

Nhưng nếu các quan hệ nhiều-nhiều rất phổ biến trong dữ liệu của bạn thì sao? Mô hình relational có thể xử lý các trường hợp quan hệ nhiều-nhiều đơn giản, nhưng khi các kết nối trong dữ liệu của bạn trở nên phức tạp hơn, việc bắt đầu mô hình hóa dữ liệu đó dưới dạng một graph sẽ trở nên tự nhiên hơn.

Một graph bao gồm hai loại đối tượng: *vertices* (còn được gọi là *nodes* hoặc *entities*) và *edges* (còn được gọi là *relationships* hoặc *arcs*). Nhiều loại dữ liệu có thể được mô hình hóa dưới dạng một graph. Các ví dụ điển hình bao gồm sau đây:

Social graphs

Các vertices là con người, và các edges chỉ ra những người nào biết nhau.

The web graph

Các vertices là các trang web, và các edges chỉ ra các liên kết HTML tới các trang khác.

Mạng lưới đường bộ hoặc đường sắt

Các vertex là các điểm nút, và các edge đại diện cho các con đường hoặc các tuyến đường sắt giữa chúng.

Các thuật toán nổi tiếng có thể hoạt động trên các graph này—ví dụ, các ứng dụng điều hướng bản đồ tìm kiếm đường đi ngắn nhất giữa hai điểm trong một mạng lưới đường bộ, và PageRank có thể được sử dụng trên web graph để xác định mức độ phổ biến của một trang web, từ đó quyết định thứ hạng của nó trong kết quả tìm kiếm [34].

Các graph có thể được biểu diễn theo nhiều cách khác nhau. Trong mô hình *adjacency list* (danh sách kề), mỗi vertex lưu trữ các ID của các vertex lân cận cách nó một edge. Ngoài ra, bạn có thể sử dụng một *adjacency matrix* (ma trận kề), một mảng hai chiều trong đó mỗi hàng và cột tương ứng với một vertex, với giá trị là 0 khi không có edge nào giữa vertex ở hàng và vertex ở cột, và là 1 khi có một edge. Một adjacency list rất tốt cho các thao tác duyệt graph, còn các matrix thì tốt cho machine learning (xem mục “DataFrames, Matrices, and Arrays”).

Trong các ví dụ vừa nêu, tất cả các vertex trong một graph đều đại diện cho cùng một loại đối tượng (tương ứng là con người, trang web, hoặc các nút giao thông). Tuy nhiên, các graph không chỉ giới hạn ở các dữ liệu *homogeneous* (đồng nhất) như vậy. Một cách sử dụng graph mạnh mẽ không kém là cung cấp một phương thức nhất quán để lưu trữ các loại đối tượng hoàn toàn khác nhau trong cùng một cơ sở dữ liệu. Ví dụ:

- Facebook duy trì một graph duy nhất với nhiều loại vertex và edge. Các vertex đại diện cho con người, địa điểm, sự kiện, các lượt check-in, và các bình luận được thực hiện bởi người dùng; các edge chỉ ra những người nào là bạn bè với nhau, lượt check-in nào đã xảy ra tại địa điểm nào, ai đã bình luận vào bài đăng nào, ai đã tham dự sự kiện nào, v.v. [35].

- Các công cụ tìm kiếm sử dụng knowledge graphs để ghi lại các sự thật về các thực thể thường xuất hiện trong các truy vấn tìm kiếm, chẳng hạn như các tổ chức, con người và địa điểm [36]. Thông tin này thu được bằng cách crawling và phân tích văn bản trên các trang web; một số trang web, chẳng hạn như Wikidata, cũng công bố dữ liệu graph dưới dạng có cấu trúc.

Các graph cung cấp một vài cách khác nhau, nhưng có liên quan, để cấu trúc hóa và truy vấn dữ liệu. Trong mục này, chúng ta sẽ thảo luận về mô hình *property graph* (được triển khai bởi Neo4j, Memgraph, KùzuDB [37], và các công cụ khác [38]) và mô hình *triple store* (được triển khai bởi Datomic, AllegroGraph, Blazegraph, và các công cụ khác). Các mô hình này khá tương đồng về những gì chúng có thể biểu diễn, và một số graph databases (chẳng hạn như Amazon Neptune) hỗ trợ cả hai.

Chúng ta cũng sẽ xem xét bốn ngôn ngữ truy vấn cho graph (Cypher, SPARQL, Datalog, và GraphQL), cũng như sự hỗ trợ của SQL để truy vấn các graph. Các ngôn ngữ truy vấn graph khác cũng tồn tại, chẳng hạn như Gremlin [39], nhưng những ngôn ngữ này sẽ cung cấp cho chúng ta một cái nhìn tổng quan mang tính đại diện.

Để minh họa các ngôn ngữ và mô hình này, mục này sử dụng Hình 3-6 làm ví dụ xuyên suốt. Nó có thể được lấy từ một mạng xã hội hoặc một cơ sở dữ liệu phủ hệ; nó hiển thị hai người, Lucy từ Idaho và Alain từ Saint-Lô, Pháp. Họ đã kết hôn và đang sống ở London. Mỗi người và mỗi địa điểm được biểu diễn dưới dạng một vertex, và các mối quan hệ giữa chúng được biểu diễn dưới dạng các edge. Ví dụ này sẽ giúp chứng minh một số truy vấn vốn dễ dàng trong các graph databases nhưng lại khó khăn trong các mô hình dữ liệu khác.

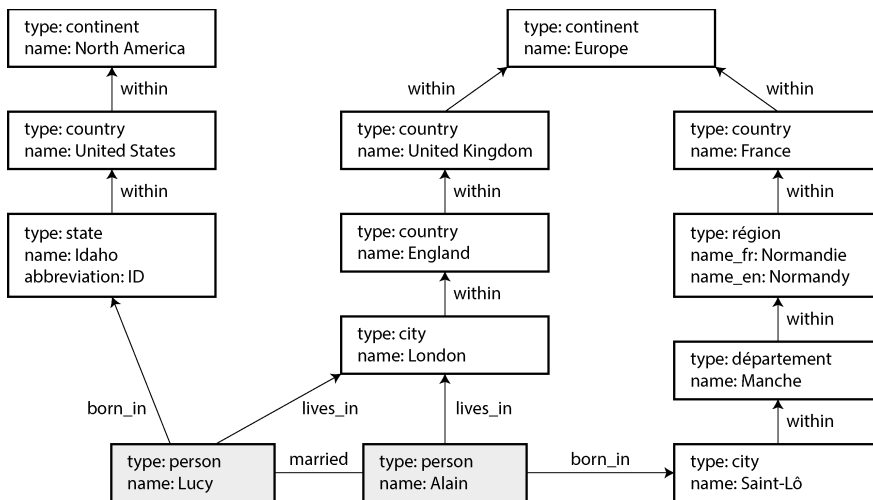


Figure 3-6. *Dữ liệu cấu trúc đồ thị (các hộp đại diện cho vertex, các mũi tên đại diện cho edge)*

Property Graphs

Trong mô hình *property graph* (còn được gọi là *labeled property graph*), mỗi vertex bao gồm các thành phần sau:

- Một định danh duy nhất
- Một label (chuỗi ký tự) để mô tả loại đối tượng mà vertex này đại diện
- Một tập hợp các outgoing edges
- Một tập hợp các incoming edges
- Một tập hợp các properties (cặp key-value)

Mỗi edge bao gồm các thành phần sau:

- Một định danh duy nhất
- Vertex mà tại đó edge bắt đầu (the *tail vertex*)
- Vertex mà tại đó edge kết thúc (the *head vertex*)
- Một label để mô tả loại mối quan hệ giữa hai vertex
- Một tập hợp các properties (cặp key-value)

Bạn có thể coi một graph store bao gồm hai bảng quan hệ, một cho các vertex và một cho các edge, như được trình bày trong Ví dụ 3-3 (schema này sử dụng kiểu dữ liệu PostgreSQL jsonb để lưu trữ các properties của mỗi vertex hoặc edge). Các head vertex và tail vertex được lưu trữ cho mỗi edge; nếu bạn muốn tìm tập hợp các

incoming edges hoặc outgoing edges của một vertex, bạn có thể truy vấn bảng edges bằng `head_vertex` hoặc `tail_vertex`, tương ứng.

Example 3-3. Biểu diễn một property graph bằng một relational schema

```
CREATE TABLE vertices (  
    vertex_id    integer PRIMARY KEY,  
    label        text,  
    properties   jsonb  
);  
  
CREATE TABLE edges (  
    edge_id      integer PRIMARY KEY,  
    tail_vertex  integer REFERENCES vertices (vertex_id),  
    head_vertex  integer REFERENCES vertices (vertex_id),  
    label        text,  
    properties   jsonb  
);  
  
CREATE INDEX edges_tails ON edges (tail_vertex);  
CREATE INDEX edges_heads ON edges (head_vertex);
```

Một số khía cạnh quan trọng của mô hình này như sau:

- Bất kỳ vertex nào cũng có thể có một edge kết nối nó với bất kỳ vertex nào khác. Không có schema nào hạn chế những loại đối tượng nào có thể hoặc không thể được liên kết với nhau.
- Với bất kỳ vertex nào, bạn có thể tìm thấy cả incoming edges và outgoing edges của nó một cách hiệu quả, và từ đó *traverse* (duyệt) đồ thị (tức là đi theo một đường dẫn qua một chuỗi các vertex) theo cả hướng tiến và hướng lùi. (Đó là lý do tại sao Ví dụ 3-3 có các index trên cả các cột `tail_vertex` và `head_vertex`.)
- Bằng cách sử dụng các label khác nhau cho các loại vertex và mối quan hệ khác nhau, bạn có thể lưu trữ nhiều loại thông tin trong một đồ thị duy nhất mà vẫn duy trì được một data model sạch sẽ.

Bảng edges giống như bảng liên kết nhiều-nhiều (many-to-many associative table), hay bảng join mà chúng ta đã thấy trong mục “Mối quan hệ Many-to-One và Many-to-Many”, được tổng quát hóa để cho phép lưu trữ nhiều loại mối quan hệ trong cùng một bảng. Cũng có thể có các index trên các label và properties, cho phép tìm thấy các vertex hoặc edges với các properties nhất định một cách hiệu quả.

LƯU Ý

Một hạn chế của các mô hình đồ thị là một edge chỉ có thể liên kết hai vertex với nhau, trong khi một bảng join quan hệ có thể đại diện cho các mối quan hệ ba bên hoặc thậm chí có bậc cao hơn bằng cách có nhiều tham chiếu foreign-key trên một hàng duy nhất. Các mối quan hệ như vậy có thể được biểu diễn trong một đồ thị bằng cách tạo thêm một vertex tương ứng với mỗi hàng của bảng join và các edge hướng tới/từ vertex đó, hoặc bằng cách sử dụng một *hypergraph*.

Những tính năng đó mang lại cho đồ thị sự linh hoạt rất lớn trong việc modeling dữ liệu, như được minh họa trong Hình 3-6. Hình này cho thấy một vài điều sẽ khó diễn đạt trong một relational schema truyền thống, chẳng hạn như các loại cấu trúc vùng khác nhau ở các quốc gia khác nhau (Pháp có *départements* và *régions*, trong khi Mỹ có *counties* và *states*), những điểm đặc thù của lịch sử như một quốc gia nằm trong một quốc gia (tạm thời bỏ qua sự phức tạp của các quốc gia có chủ quyền và các dân tộc), và độ chi tiết (*granularity*) khác nhau của dữ liệu (nơi cư trú hiện tại của Lucy được chỉ định là một thành phố, trong khi nơi sinh của cô ấy chỉ được chỉ định ở cấp độ một bang).

Bạn có thể tưởng tượng việc mở rộng đồ thị để bao gồm nhiều sự thật khác về Lucy và Alain, hoặc những người khác. Ví dụ, bạn có thể sử dụng đồ thị để chỉ ra bất kỳ tình trạng dị ứng thực phẩm nào mà họ mắc phải (bằng cách giới thiệu một vertex cho mỗi chất gây dị ứng, và một edge giữa một người và một chất gây dị ứng để chỉ ra sự dị ứng), và liên kết các chất gây dị ứng với một tập hợp các vertex cho thấy những loại thực phẩm nào chứa những chất nào. Sau đó, bạn có thể viết một truy vấn để tìm ra loại thực phẩm nào an toàn cho mỗi người. Đồ thị rất tốt cho khả năng tiến hóa (*evolvability*): khi bạn thêm các tính năng vào ứng dụng của mình, một đồ thị có thể dễ dàng được mở rộng để thích ứng với những thay đổi trong các cấu trúc dữ liệu của ứng dụng.

Ngôn ngữ truy vấn Cypher

Cypher là một ngôn ngữ truy vấn dành cho property graph, ban đầu được tạo ra cho cơ sở dữ liệu đồ thị Neo4j và sau đó được phát triển thành một tiêu chuẩn mở là *openCypher* [40]. Bên cạnh Neo4j, Cypher còn được hỗ trợ bởi Memgraph, KuzuDB [37], Amazon Neptune, Apache AGE (với lưu trữ trong PostgreSQL), và các hệ thống khác. Ngôn ngữ này được đặt tên theo một nhân vật trong bộ phim *The Matrix* và không liên quan đến các mã mật mã (*ciphers*) trong mật mã học [41].

Ví dụ 3-4 cho thấy truy vấn Cypher để chèn phần bên trái của Hình 3-6 vào một cơ sở dữ liệu đồ thị. Phần còn lại của đồ thị có thể được thêm vào theo cách tương tự. Mỗi vertex được đặt một cái tên ký hiệu, chẳng hạn như *usa* hoặc *idaho*. Cái tên đó không được lưu trữ trong cơ sở dữ liệu mà chỉ được sử dụng nội bộ trong truy vấn để tạo các edge giữa các vertex, bằng cách sử dụng ký hiệu mũi tên: (*idaho*) –

[:WITHIN] -> (usa) tạo ra một edge được gắn nhãn WITHIN, với idaho là tail node và usa là head node.

Example 3-4. Một tập hợp con của dữ liệu trong Hình 3-6, được biểu diễn dưới dạng một truy vấn Cypher

CREATE

```
(america :Location {name:'North America', type:'continent'}),
(usa :Location {name:'United States', type:'country' }),
(idaho :Location {name:'Idaho', type:'state' }),
(lucy :Person {name:'Lucy' }),
(idaho) -[:WITHIN]-> (usa) -[:WITHIN]-> (america),
(lucy) -[:BORN_IN]-> (idaho)
```

Khi tất cả các vertex và edge của Hình 3-6 được thêm vào cơ sở dữ liệu, chúng ta có thể bắt đầu đặt những câu hỏi thú vị. Ví dụ, giả sử chúng ta muốn tìm tên của tất cả những người đã di cư từ Hoa Kỳ sang Châu Âu. Chúng ta có thể thực hiện việc đó bằng cách tìm tất cả các vertex có một edge BORN_IN tới một địa điểm trong Hoa Kỳ và một edge LIVING_IN tới một địa điểm trong Châu Âu, rồi trả về property name của mỗi vertex đó.

Ví dụ 3-5 cho thấy cách diễn đạt truy vấn đó trong Cypher. Ký hiệu mũi tên tương tự được sử dụng trong một mệnh đề MATCH để tìm các pattern trong đồ thị: (person) -[:BORN_IN]-> () khớp với bất kỳ hai vertex nào có liên quan bởi một edge được gắn nhãn BORN_IN. Tail vertex của edge đó được gán cho biến person, và head vertex được để trống tên.

Example 3-5. Một truy vấn Cypher để tìm những người đã di cư từ Hoa Kỳ sang Châu Âu

MATCH

```
(person) -[:BORN_IN]-> () -[:WITHIN*0..]-> (:Location
{name:'United States'}),
(person) -[:LIVES_IN]-> () -[:WITHIN*0..]-> (:Location
{name:'Europe'})
```

RETURN person.name

Truy vấn có thể được đọc như sau:

Tìm bất kỳ vertex nào (gọi là person) thỏa mãn *cả hai* điều kiện sau:

1. person có một edge BORN_IN hướng ra (outgoing) tới một vertex. Từ vertex đó, bạn có thể đi theo một chuỗi các edge WITHIN hướng ra cho đến khi cuối cùng bạn chạm tới một vertex thuộc kiểu Location, mà có property name bằng với United States.
2. Chính vertex person đó cũng có một edge LIVES_IN hướng ra. Đi theo edge đó, và sau đó là một chuỗi các edge WITHIN hướng

ra, cuối cùng bạn chạm tới một vertex thuộc kiểu `Location`, mà có property name bằng với `Europe`.

Với mỗi vertex `person` như vậy, hãy trả về property name.

Có một vài cách khả thi để thực thi truy vấn. Mô tả được đưa ra ở đây gợi ý rằng bạn bắt đầu bằng việc quét tất cả mọi người trong cơ sở dữ liệu, kiểm tra nơi sinh và nơi cư trú của từng người, và chỉ trả về những người thỏa mãn các tiêu chí.

Nhưng một cách tương đương, bạn có thể bắt đầu với hai vertex `Location` và làm ngược lại. Nếu có một index trên property name, bạn có thể tìm thấy hai vertex đại diện cho Hoa Kỳ và Châu Âu một cách hiệu quả. Sau đó, bạn có thể tiến hành tìm tất cả các địa điểm (các bang, vùng, thành phố, v.v.) tương ứng tại Hoa Kỳ và Châu Âu bằng cách đi theo tất cả các edge `WITHIN` hướng vào (incoming). Cuối cùng, bạn có thể tìm những người có thể được tìm thấy thông qua một edge `BORN_IN` hoặc `LIVES_IN` hướng vào tại một trong các vertex địa điểm đó.

Truy vấn Đồ thị trong SQL

Ví dụ 3-3 đã gợi ý rằng dữ liệu đồ thị có thể được biểu diễn trong một cơ sở dữ liệu quan hệ. Nhưng nếu chúng ta đưa dữ liệu đồ thị vào một cấu trúc quan hệ, liệu chúng ta có thể truy vấn nó bằng SQL không?

Câu trả lời là có, nhưng sẽ gặp một số khó khăn. Mỗi edge mà bạn duyệt qua trong một truy vấn đồ thị về cơ bản là một phép join với bảng edges. Trong một cơ sở dữ liệu quan hệ, bạn thường biết trước những phép join nào cần thiết trong truy vấn của mình. Mặt khác, trong một truy vấn đồ thị, bạn có thể cần phải duyệt qua một số lượng edge biến đổi trước khi tìm thấy vertex mà bạn đang tìm kiếm—tức là, số lượng các phép join không được cố định trước.

Trong ví dụ của chúng ta, điều đó xảy ra theo pattern `() -[:WITHIN*0..]-> ()` trong truy vấn Cypher. Một `LIVES_IN` edge của một người có thể trở đến bất kỳ loại vị trí nào, chẳng hạn như một con đường, một thành phố, một quận, một vùng, hoặc một bang. Một thành phố có thể là `WITHIN` một vùng, một vùng `WITHIN` một bang, một bang `WITHIN` một quốc gia, và cứ tiếp tục như vậy. `LIVES_IN` edge có thể trở trực tiếp đến location vertex mà bạn đang tìm kiếm, hoặc nó có thể nằm cách vài cấp trong phân cấp vị trí (location hierarchy).

Trong Cypher, `:WITHIN*0..` thể hiện sự thật đó một cách rất súc tích: nó có nghĩa là "đi theo một `WITHIN` edge, không hoặc nhiều lần". Nó giống như toán tử `*` trong một biểu thức chính quy (regular expression).

Ý tưởng về các đường dẫn duyệt (traversal paths) có độ dài thay đổi trong một truy vấn có thể được thể hiện bằng cách sử dụng *recursive common table expressions* (cú pháp `WITH RECURSIVE`). Ví dụ 3-6 cho thấy cùng một truy vấn—tìm tên của những người đã di cư từ Mỹ sang Châu Âu—được thể hiện bằng SQL sử dụng kỹ

thuật này (các dòng bắt đầu bằng -- là các chú thích). Như bạn có thể thấy, cú pháp này rất vụng về so với Cypher.

Example 3-6. Cùng một truy vấn như Ví dụ 3-5, được viết bằng SQL sử dụng recursive common table expressions

WITH RECURSIVE

```
-- in_usa is the set of vertex IDs of all locations within the
United States
in_usa(vertex_id) AS (
    SELECT vertex_id FROM vertices
    WHERE label = 'Location' AND properties->>'name' = 'United
States' .
    UNION
    SELECT edges.tail_vertex FROM edges .
    JOIN in_usa ON edges.head_vertex = in_usa.vertex_id
    WHERE edges.label = 'within'
),

-- in_europe is the set of vertex IDs of all locations within
Europe
in_europe(vertex_id) AS (
    SELECT vertex_id FROM vertices
    WHERE label = 'location' AND properties->>'name' = 'Europe' .
    UNION
    SELECT edges.tail_vertex FROM edges
    JOIN in_europe ON edges.head_vertex = in_europe.vertex_id
    WHERE edges.label = 'within'
),

-- born_in_usa is the set of vertex IDs of all people born in the
US
born_in_usa(vertex_id) AS ( .
    SELECT edges.tail_vertex FROM edges
    JOIN in_usa ON edges.head_vertex = in_usa.vertex_id
    WHERE edges.label = 'born_in'
),

-- lives_in_europe is the set of vertex IDs of all people living in
Europe
lives_in_europe(vertex_id) AS ( .
    SELECT edges.tail_vertex FROM edges
    JOIN in_europe ON edges.head_vertex = in_europe.vertex_id
    WHERE edges.label = 'lives_in'
)

SELECT vertices.properties->>'name'
FROM vertices
-- join to find those people who were both born in the US *and* live
in Europe
JOIN born_in_usa      ON vertices.vertex_id = born_in_usa.vertex_id .
JOIN lives_in_europe ON vertices.vertex_id =
lives_in_europe.vertex_id;
```

- Đầu tiên, tìm vertex có thuộc tính name mang giá trị `United States`, và đặt nó làm phần tử đầu tiên của tập hợp các vertex `in_usa`.
- Đi theo tất cả các `within edge` đi vào từ các vertex trong tập hợp `in_usa` và thêm chúng vào cùng một tập hợp đó, cho đến khi tất cả các `within edge` đi vào đã được truy cập.
- Thực hiện tương tự bắt đầu với vertex có thuộc tính name mang giá trị `Europe`, và xây dựng tập hợp các vertex `in_europe`.
- Đối với mỗi vertex trong tập hợp `in_usa`, đi theo các `born_in edge` đi vào để tìm những người đã sinh ra tại một nơi nào đó trong phạm vi nước Mỹ.
- Tương tự, đối với mỗi vertex trong tập hợp `in_europe`, đi theo các `lives_in edge` đi vào để tìm những người đang sống ở Châu Âu.
- Cuối cùng, giao tập hợp những người sinh ra ở Mỹ với tập hợp những người đang sống ở Châu Âu bằng cách join chúng lại.

Việc một truy vấn Cypher chỉ dài 4 dòng lại yêu cầu tới 31 dòng trong SQL cho thấy sự khác biệt lớn mà việc lựa chọn đúng mô hình dữ liệu (data model) và ngôn ngữ truy vấn có thể mang lại. Và đây mới chỉ là sự khởi đầu; còn nhiều chi tiết khác cần xem xét, ví dụ như việc xử lý các chu trình (cycles) và việc lựa chọn giữa duyệt theo chiều rộng (breadth-first traversal) hay duyệt theo chiều sâu (depth-first traversal) [42]. Oracle có một phần mở rộng SQL khác cho các truy vấn đệ quy, mà họ gọi là *hierarchical* [43]. Các ngôn ngữ truy vấn đồ thị khác bao gồm GSQL của TigerGraph [44] và Property Graph Query Language (PGQL) [45].

Tiêu chuẩn ISO của Graph Query Language (GQL), dựa trên Cypher, đã được công bố vào năm 2024 [46, 47, 48]. Mặc dù nó chưa được áp dụng rộng rãi, nhưng hy vọng nó sẽ dẫn đến sự thống nhất lớn hơn giữa các cơ sở dữ liệu đồ thị trong những năm tới.

Triple Stores và SPARQL

triple store model về cơ bản tương đương với *property graph model*, chỉ sử dụng các từ ngữ khác nhau để mô tả cùng một ý tưởng. Tuy nhiên, nó vẫn đáng để thảo luận, vì các công cụ và ngôn ngữ khác nhau dành cho triple stores có thể là những bổ sung giá trị vào bộ công cụ (toolbox) của bạn để xây dựng các ứng dụng.

Trong một triple store, tất cả thông tin được lưu trữ dưới dạng các câu tuyên bố ba phần rất đơn giản: (*subject*, *predicate*, *object*). Ví dụ, trong bộ ba (*Jim*, *likes*, *bananas*), *Jim* là subject, *likes* là predicate (động từ), và *bananas* là object.

LƯU Ý

Để chính xác, các cơ sở dữ liệu cung cấp mô hình dữ liệu dạng triple thường cần lưu trữ thêm metadata trên mỗi tuple. Ví dụ, AWS Neptune sử dụng quads (4-tuples) bằng cách thêm một graph ID vào mỗi triple [49]; Datomic sử dụng 5-tuples, mở rộng mỗi triple với một transaction ID và một giá trị Boolean để chỉ định việc xóa [50]. Vì các cơ sở dữ liệu này vẫn giữ cấu trúc *subject-predicate-object* cơ bản được giải thích ở đây, cuốn sách này vẫn gọi chúng là triple stores.

Subject của một triple tương đương với một vertex trong một đồ thị. Object là một trong hai thứ sau:

- Một giá trị của kiểu dữ liệu nguyên thủy, chẳng hạn như một chuỗi hoặc một số. Trong trường hợp đó, predicate và object của bộ ba (triple) tương đương với key và value của một property trên subject vertex. Sử dụng ví dụ từ Hình 3-6, (*lucy*, *birthYear*, 1989) giống như một vertex *lucy* với các property {"birthYear": 1989}.
- Một vertex khác trong đồ thị. Trong trường hợp đó, predicate là một edge trong đồ thị, subject là tail vertex, và object là head vertex. Ví dụ, trong (*lucy*, *marriedTo*, *alain*), subject và object là *lucy* và *alain* đều là các vertices, và predicate *marriedTo* là nhãn của edge kết nối chúng.

Ví dụ 3-7 hiển thị cùng một dữ liệu như trong Ví dụ 3-4 được viết dưới dạng các bộ ba (triples) trong một định dạng gọi là *Turtle*, một tập con của *Notation3 (N3)* [51].

Example 3-7. Một tập con của dữ liệu trong Hình 3-6, được biểu diễn dưới dạng các Turtle triples

```
@prefix : <urn:example:>.
_:lucy      a          :Person.
_:lucy      :name      "Lucy".
_:lucy      :bornIn    _:idaho.
_:idaho     a          :Location.
_:idaho     :name      "Idaho".
_:idaho     :type      "state".
_:idaho     :within    _:usa.
_:usa       a          :Location.
_:usa       :name      "United States".
_:usa       :type      "country".
_:usa       :within    _:namerica.
_:namerica  a          :Location.
_:namerica  :name      "North America".
_:namerica  :type      "continent".
```

Trong ví dụ này, các vertices của đồ thị được viết dưới dạng `_:someName`. Tên này không có ý nghĩa gì bên ngoài tệp này; nó tồn tại chỉ vì nếu không có nó, chúng ta sẽ không biết bộ ba nào tham chiếu đến cùng một vertex. Khi predicate đại diện cho một edge, object là một vertex, như trong `_:idaho :within _:usa`. Khi predicate là một property, object là một string literal, như trong `_:usa :name "United States"`.

Để có một cách biểu diễn gọn gàng hơn, bạn có thể sử dụng dấu chấm phẩy để nói nhiều điều về cùng một subject, như được hiển thị trong Ví dụ 3-8. Điều này giúp định dạng Turtle trở nên khá dễ đọc.

Example 3-8. Một cách viết dữ liệu trong Ví dụ 3-7 súc tích hơn

```
@prefix : <urn:example:>.
_:lucy    a :Person;    :name "Lucy";           :bornIn _:idaho.
_:idaho   a :Location;  :name "Idaho";           :type
"state";  :within _:usa.
_:usa     a :Location;  :name "United States"; :type
"country";:within _:namerica.
_:namerica a :Location; :name "North America"; :type
"continent".
```

Semantic Web

Một số nỗ lực nghiên cứu và phát triển về triple stores được thúc đẩy bởi *Semantic Web*, một nỗ lực vào đầu những năm 2000 nhằm tạo điều kiện cho việc trao đổi dữ liệu trên toàn internet bằng cách công bố dữ liệu không chỉ dưới dạng các trang web mà con người có thể đọc được, mà còn dưới định dạng chuẩn hóa, máy có thể đọc được. Mặc dù Semantic Web như ý tưởng ban đầu đã không thành công [52, 53], di sản của dự án này vẫn tiếp tục tồn tại, ví dụ như trong các tiêu chuẩn linked data như JSON-LD [54], các ontologies được sử dụng trong khoa học y sinh [55], giao thức Open Graph của Facebook [56] (được sử dụng để link unfurling [57]), các knowledge graphs như Wikidata, và các từ vựng chuẩn hóa cho dữ liệu có cấu trúc được duy trì bởi Schema.org.

Triple stores là một công nghệ Semantic Web khác đã tìm thấy ứng dụng bên ngoài trường hợp sử dụng ban đầu của nó; ngay cả khi bạn không quan tâm đến Semantic Web, triples cũng có thể là một mô hình dữ liệu nội bộ tốt cho các ứng dụng.

Mô hình dữ liệu RDF

Ngôn ngữ Turtle mà chúng ta đã sử dụng trong Ví dụ 3-8 thực chất là một cách để encoding dữ liệu trong *Resource Description Framework* (RDF) [58], một mô hình

dữ liệu được thiết kế cho Semantic Web. Dữ liệu RDF cũng có thể được encoding theo các cách khác, bao gồm (một cách rườm rà hơn) XML, như được hiển thị trong Ví dụ 3-9. Các công cụ như Apache Jena có thể tự động chuyển đổi giữa các RDF encodings khác nhau.

Example 3-9. Dữ liệu từ Ví dụ 3-8 được biểu diễn bằng cú pháp RDF/XML

```
<rdf:RDF xmlns="urn:example:"
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">

  <Location rdf:nodeID="idaho">
    <name>Idaho</name>
    <type>state</type>
    <within>
      <Location rdf:nodeID="usa">
        <name>United States</name>
        <type>country</type>
        <within>
          <Location rdf:nodeID="namerica">
            <name>North America</name>
            <type>continent</type>
          </Location>
        </within>
      </Location>
    </within>
  </Location>

  <Person rdf:nodeID="lucy">
    <name>Lucy</name>
    <bornIn rdf:nodeID="idaho"/>
  </Person>
</rdf:RDF>
```

RDF có một vài điểm kỳ lạ vì nó được thiết kế để trao đổi dữ liệu trên toàn internet. Subject, predicate, và object của một triple thường là các URIs. Ví dụ, một predicate có thể là một URI như <http://my-company.com/namespace#within> hoặc <http://my-company.com/namespace#lives_in>, thay vì chỉ là WITHIN hoặc LIVES_IN. Lý do đằng sau thiết kế này là bạn nên có thể kết hợp dữ liệu của mình với dữ liệu của người khác, và nếu họ gán một ý nghĩa khác với từ within hoặc lives_in, bạn sẽ không gặp phải xung đột vì các predicate của họ thực chất là <http://other.org/foo#within> và <http://other.org/foo#lives_in>.

URL <http://my-company.com/namespace> không nhất thiết phải phân giải thành bất cứ thứ gì—từ góc độ của RDF, nó đơn giản chỉ là một namespace. Để tránh sự nhầm lẫn tiềm ẩn với các URL http://, các ví dụ trong mục này sử dụng các URIs không thể phân giải như urn:example:within. May mắn thay, bạn có thể chỉ định prefix này duy nhất một lần ở đầu tệp và sau đó không cần bận tâm về nó nữa.

Ngôn ngữ truy vấn SPARQL

SPARQL là một ngôn ngữ truy vấn dành cho các triple store sử dụng mô hình dữ liệu RDF [59]. (Tên gọi này là một từ viết tắt đệ quy của *SPARQL Protocol and RDF Query Language*, phát âm là “sparkle”.) Nó có trước Cypher, và vì cơ chế pattern matching của Cypher được mượn từ SPARQL, nên chúng trông khá giống nhau.

Truy vấn tương tự như trước—tìm những người đã chuyển từ Mỹ sang Châu Âu—cũng ngắn gọn trong SPARQL tương tự như trong Cypher (xem Ví dụ 3-10).

Example 3-10. Truy vấn tương tự như Ví dụ 3-5, được thể hiện bằng SPARQL

```
PREFIX : <urn:example:>

SELECT ?personName WHERE {
  ?person :name ?personName.
  ?person :bornIn / :within* / :name "United States".
  ?person :livesIn / :within* / :name "Europe".
}
```

Cấu trúc rất giống nhau. Hai biểu thức sau đây là tương đương (các biến bắt đầu bằng dấu chấm hỏi trong SPARQL):

```
(person) -[:BORN_IN]-> () -[:WITHIN*0..]-> (location)    #
Cypher

?person :bornIn / :within* ?location.                      #
SPARQL
```

Vì RDF không phân biệt giữa properties và edges mà chỉ sử dụng predicates cho cả hai, bạn có thể sử dụng cùng một cú pháp để khớp các properties. Trong biểu thức sau, biến `usa` được liên kết với bất kỳ vertex nào có một property name mà giá trị của nó là chuỗi `United States`:

```
(usa {name:'United States'})    # Cypher

?usa :name "United States".     # SPARQL
```

SPARQL được hỗ trợ bởi Amazon Neptune, AllegroGraph, Blazegraph, OpenLink Virtuoso, Apache Jena và nhiều triple stores khác [38].

Datalog: Truy vấn quan hệ đệ quy

Datalog, một ngôn ngữ lâu đời hơn nhiều so với SPARQL hay Cypher, nảy sinh từ nghiên cứu học thuật vào những năm 1980 [60, 61, 62]. Nó ít được biết đến hơn trong số các lập trình viên phần mềm và không được hỗ trợ rộng rãi trong các cơ sở dữ liệu chính thống, nhưng nó đáng lẽ phải được biết đến nhiều hơn vì là một ngôn ngữ rất giàu tính biểu đạt và đặc biệt mạnh mẽ cho các truy vấn phức tạp. Một số cơ sở dữ liệu gác, bao gồm Datomic, LogicBlox, CozoDB và LIquid của LinkedIn [63], sử dụng Datalog làm ngôn ngữ truy vấn của chúng. Nó dựa trên mô hình dữ

liệu quan hệ chứ không phải đồ thị, nhưng chúng ta thảo luận về nó ở đây vì các truy vấn đề xuất trên đồ thị là một thể mạnh đặc biệt của Datalog.

Nội dung của một cơ sở dữ liệu Datalog được gọi là các *facts*, và mỗi fact tương ứng với một hàng trong một bảng quan hệ. Ví dụ, giả sử chúng ta có một bảng `location` chứa các địa điểm, và nó có ba cột: `ID`, `name`, và `type`. Fact rằng Mỹ là một quốc gia có thể được viết là `location(2, "United States", "country")`, trong đó 2 là ID của Mỹ. Nói chung, câu lệnh `table(val1, val2, ...)` có nghĩa là `table` chứa một hàng mà cột đầu tiên chứa `val1`, cột thứ hai chứa `val2`, và cứ tiếp tục như vậy.

Ví dụ 3-11 cho thấy cách viết dữ liệu từ phía bên trái của Hình 3-6 trong Datalog. Các edges của đồ thị (`within`, `born_in`, và `lives_in`) được biểu diễn dưới dạng các bảng join hai cột. Ví dụ, Lucy có ID là 100 và Idaho có ID là 3, vì vậy mối quan hệ “Lucy sinh ra ở Idaho” được biểu diễn là `born_in(100, 3)`.

Example 3-11. Một tập hợp con của dữ liệu trong Hình 3-6, được biểu diễn dưới dạng các Datalog facts

```
location(1, "North America", "continent").
location(2, "United States", "country").
location(3, "Idaho", "state").

within(2, 1).      /* US is in North America */
within(3, 2).      /* Idaho is in the US      */

person(100, "Lucy").
born_in(100, 3).   /* Lucy was born in Idaho */
```

Bây giờ khi chúng ta đã định nghĩa dữ liệu, chúng ta có thể viết truy vấn tương tự như trước như được trình bày trong Ví dụ 3-12. Nó trông hơi khác so với các phiên bản tương đương trong Cypher hoặc SPARQL, nhưng đừng để điều đó làm bạn nản lòng. Datalog là một tập hợp con của Prolog, một ngôn ngữ lập trình mà bạn có thể đã thấy trước đây nếu bạn đã học khoa machine learning tính.

Example 3-12. Truy vấn tương tự như Ví dụ 3-5, được thể hiện bằng Datalog

```
within_recursive(LocID, PlaceName) :- location(LocID, PlaceName,
_). /* Rule 1 */

within_recursive(LocID, PlaceName) :- within(LocID,
ViaID), /* Rule 2 */
within_recursive(ViaID,
PlaceName).

migrated(PName, BornIn, LivingIn) :- person(PersonID,
PName), /* Rule 3 */
born_in(PersonID, BornID),
within_recursive(BornID,
BornIn),
lives_in(PersonID,
LivingID),
within_recursive(LivingID,
LivingIn).

us_to_europe(Person) :- migrated(Person, "United States",
"Europe"). /* Rule 4 */
/* us_to_europe contains the row "Lucy". */
```

Cypher và SPARQL nhảy vào ngay lập tức với SELECT, nhưng Datalog thực hiện từng bước nhỏ một. Chúng ta định nghĩa các *rules* để dẫn xuất ra các bảng ảo mới từ các facts cơ sở. Các bảng dẫn xuất này giống như các SQL views (ảo): chúng không được lưu trữ trong cơ sở dữ liệu, nhưng bạn có thể truy vấn chúng theo cùng một cách như một bảng chứa các facts đã được lưu trữ.

Trong Ví dụ 3-12, chúng ta định nghĩa ba bảng dẫn xuất: `within_recursive`, `migrated`, và `us_to_europe`. Tên và các cột của các bảng ảo được xác định bởi những gì xuất hiện trước ký hiệu `:-` trong mỗi rule. Ví dụ, `migrated(PName, BornIn, LivingIn)` là một bảng ảo với ba cột: tên của một người, tên của nơi họ sinh ra, và tên của nơi họ đang sống.

Nội dung của một bảng ảo được xác định bởi phần của quy tắc nằm sau ký hiệu `:-`, nơi chúng ta cố gắng tìm các hàng khớp với một pattern nhất định trong các bảng. Ví dụ, `person(PersonID, PName)` khớp với hàng `person(100, "Lucy")`, với biến `PersonID` được gán cho giá trị 100 và biến `PName` được gán cho giá trị "Lucy". Một quy tắc được áp dụng nếu hệ thống có thể tìm thấy sự khớp nhau cho *tất cả* các pattern ở phía bên phải của toán tử `:-`. Khi quy tắc được áp dụng, nó giống như thể phần bên trái của `:-` đã được thêm vào cơ sở dữ liệu (với các biến được thay thế bằng các giá trị mà chúng đã khớp).

Một cách khả thi để áp dụng các quy tắc như vậy là (như được minh họa trong Hình 3-7):

1. `location(1, "North America", "continent")` tồn tại trong cơ sở dữ liệu, vì vậy quy tắc 1 được áp dụng. Nó tạo ra `within_recursive(1, "North America")`.
2. `within(2, 1)` tồn tại trong cơ sở dữ liệu và bước trước đó đã tạo ra `within_recursive(1, "North America")`, vì vậy quy tắc 2 được áp dụng. Nó tạo ra `within_recursive(2, "North America")`.
3. `within(3, 2)` tồn tại trong cơ sở dữ liệu và bước trước đó đã tạo ra `within_recursive(2, "North America")`, vì vậy quy tắc 2 được áp dụng. Nó tạo ra `within_recursive(3, "North America")`.

Bằng cách áp dụng lặp đi lặp lại các quy tắc 1 và 2, bảng ảo `within_recursive` có thể cho chúng ta biết tất cả các vị trí ở Bắc Mỹ (hoặc bất kỳ vị trí nào khác) có trong cơ sở dữ liệu của chúng ta.

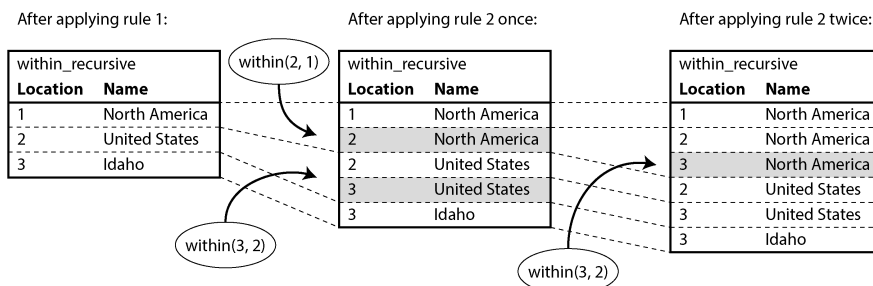


Figure 3-7. Xác định rằng Idaho nằm ở Bắc Mỹ, sử dụng các quy tắc Datalog từ Ví dụ 3-12

Bây giờ quy tắc 3 có thể tìm những người sinh ra tại một địa điểm `BornIn` và sống tại một địa điểm `LivingIn`. Quy tắc 4 gọi quy tắc 3 với `BornIn = 'United States'` và `LivingIn = 'Europe'`, sau đó chỉ trả về tên của những người khớp với tìm kiếm. Bằng cách truy vấn nội dung của bảng ảo `us_to_europe`, hệ thống Datalog cuối cùng cũng nhận được câu trả lời giống như các truy vấn Cypher và SPARQL trước đó.

Cách tiếp cận Datalog đòi hỏi một kiểu tư duy khác so với các ngôn ngữ truy vấn khác được thảo luận trong chương này. Nó cho phép xây dựng các truy vấn phức tạp theo từng quy tắc, với một quy tắc tham chiếu đến các quy tắc khác, tương tự như cách bạn chia nhỏ mã nguồn thành các hàm gọi lẫn nhau. Giống như các hàm có thể có tính đệ quy, các quy tắc Datalog cũng có thể tự gọi chính nó, như quy tắc 2 trong Ví dụ 3-12, điều này cho phép thực hiện duyệt đồ thị trong các truy vấn Datalog.

GraphQL

GraphQL là một ngôn ngữ truy vấn mà theo thiết kế, có tính hạn chế hơn nhiều so với những ngôn ngữ khác mà chúng ta đã thấy trong chương này. Nó được dành cho các truy vấn OLTP; mục đích của nó là cho phép phần mềm client chạy trên thiết bị của người dùng (chẳng hạn như một ứng dụng di động hoặc frontend ứng dụng web JavaScript) yêu cầu một tài liệu JSON với cấu trúc cụ thể, chứa các trường cần thiết để hiển thị UI của nó.

Các interface của GraphQL cho phép lập trình viên thay đổi nhanh chóng các truy vấn trong mã client mà không cần thay đổi các API phía server. Tuy nhiên, sự linh hoạt đó đi kèm với một cái giá phải trả. Các tổ chức áp dụng GraphQL thường cần các công cụ để chuyển đổi các truy vấn thành các yêu cầu tới các dịch vụ nội bộ, vốn thường sử dụng REST hoặc gRPC (xem Chương 5). Các thách thức về authorization, rate limiting và hiệu năng là những mối quan tâm bổ sung [64].

Ngôn ngữ này cũng được giới hạn một cách có ý đồ, bởi vì các truy vấn GraphQL đến từ các nguồn không đáng tin cậy. Nó không cho phép bất cứ điều gì có thể gây tổn kém khi thực thi, vì nếu không, người dùng có thể (có lẽ là vô tình) gây ra tình trạng denial-of-service trên một máy chủ bằng cách chạy rất nhiều truy vấn tốn kém. Cụ thể, GraphQL không cho phép các truy vấn đệ quy (không giống như Cypher, SPARQL, SQL, hoặc Datalog), và nó không cho phép các điều kiện tìm kiếm tùy ý (như yêu cầu "tìm những người sinh ra ở Mỹ và hiện đang sống ở Châu Âu" của chúng ta), trừ khi chủ sở hữu dịch vụ đặc biệt chọn cung cấp chức năng tìm kiếm như vậy.

Tuy nhiên, GraphQL vẫn rất hữu ích. Ví dụ 3-13 cho thấy cách bạn có thể triển khai một ứng dụng chat nhóm như Discord hoặc Slack bằng cách sử dụng GraphQL. Truy vấn yêu cầu tất cả các channel mà người dùng có quyền truy cập, bao gồm tên channel và 50 tin nhắn gần nhất trong mỗi channel. Đối với mỗi tin nhắn, truy vấn yêu cầu timestamp, nội dung tin nhắn, cùng với tên và URL ảnh đại diện của người gửi. Nếu một tin nhắn là phản hồi cho một tin nhắn khác, truy vấn cũng yêu cầu tên của người gửi và nội dung của tin nhắn đó (thứ có thể được hiển thị bằng phong chữ nhỏ hơn phía trên tin nhắn phản hồi, nhằm cung cấp một số ngữ cảnh).

Example 3-13. Một truy vấn GraphQL cho ứng dụng chat nhóm

```
query ChatApp {
  channels {
    name
    recentMessages(latest: 50) {
      timestamp
      content
      sender {
        fullName
        imageUrl
      }
      replyTo {
        content
        sender {
          fullName
        }
      }
    }
  }
}
```

Ví dụ 3-14 cho thấy một phản hồi cho truy vấn trong Ví dụ 3-13 có thể trông như thế nào. Phản hồi là một tài liệu JSON phản chiếu cấu trúc của truy vấn: nó chứa chính xác những thuộc tính đã được yêu cầu, không thừa cũng không thiếu. Cách tiếp cận này có lợi thế là server không cần biết những thuộc tính nào client yêu cầu để hiển thị giao diện người dùng; client chỉ đơn giản là yêu cầu những gì nó cần. Ví dụ, truy vấn này không yêu cầu URL ảnh đại diện cho người gửi của tin nhắn `replyTo`, nhưng nếu UI được thay đổi để bao gồm ảnh đại diện đó, client sẽ dễ dàng thêm thuộc tính `imageUrl` cần thiết vào truy vấn mà không cần thay đổi gì ở phía server.

Example 3-14. Một phản hồi khả thi cho truy vấn trong Ví dụ 3-13

```
{
  "data": {
    "channels": [
      {
        "name": "#general",
        "recentMessages": [
          {
            "timestamp": 1693143014,
            "content": "Hey! How are y'all doing?",
            "sender": {"fullName": "Aaliyah", "imageUrl":
"https://..."},
            "replyTo": null
          },
          {
            "timestamp": 1693143024,
            "content": "Great! And you?",
            "sender": {"fullName": "Caleb", "imageUrl":
"https://..."},
            "replyTo": {
              "content": "Hey! How are y'all doing?",
              "sender": {"fullName": "Aaliyah"}
            }
          },
          ...
        ]
      }
    ]
  }
}
```

Trong ví dụ này, tên và URL hình ảnh của người gửi tin nhắn được nhúng trực tiếp vào trong đối tượng tin nhắn. Nếu cùng một người dùng gửi nhiều tin nhắn, thông tin này sẽ bị lặp lại trong mỗi tin nhắn. Về nguyên tắc, ta hoàn toàn có thể giảm bớt sự trùng lặp này, nhưng GraphQL đã đưa ra lựa chọn thiết kế là chấp nhận kích thước response lớn hơn để việc render UI dựa trên dữ liệu được yêu cầu trở nên đơn giản hơn.

Trường `replyTo` cũng tương tự: trong Ví dụ 3-14, tin nhắn thứ hai là một câu trả lời cho tin nhắn đầu tiên, và nội dung (“Hey...”) cùng tên người gửi (Aaliyah) bị lặp lại dưới `replyTo`. Thay vào đó, ta có thể trả về ID của tin nhắn đang được trả lời, nhưng khi đó client sẽ phải thực hiện thêm một request bổ sung tới server nếu ID đó không nằm trong số 50 tin nhắn gần nhất được trả về. Việc nhân bản nội dung giúp làm việc với dữ liệu trở nên đơn giản hơn nhiều.

Database của server có thể lưu trữ dữ liệu dưới dạng chuẩn hóa hơn và thực hiện các phép join cần thiết để xử lý một query. Ví dụ, server có thể lưu trữ một tin nhắn cùng với user ID của người gửi và ID của tin nhắn mà nó đang trả lời; khi nhận được một query như trong Ví dụ 3-13, server sau đó sẽ resolve các ID đó để tìm các bản ghi mà chúng tham chiếu tới. Tuy nhiên, client chỉ có thể yêu cầu những phép join đã được khai báo rõ ràng trong GraphQL schema.

Mặc dù response cho một GraphQL query trông có vẻ giống với response từ một document database, và mặc dù nó có chữ “graph” trong tên gọi, GraphQL có thể được triển khai trên bất kỳ loại database nào—relational, document, hay graph.

Event Sourcing và CQRS

Trong tất cả các mô hình dữ liệu mà chúng ta đã thảo luận cho đến nay, dữ liệu được truy vấn dưới cùng một dạng như khi nó được ghi—cho dù đó là các JSON document, các hàng trong bảng, hay các đỉnh (vertices) và cạnh (edges) trong một graph. Tuy nhiên, trong các ứng dụng phức tạp, đôi khi rất khó để tìm thấy một representation dữ liệu duy nhất có thể thỏa mãn tất cả các cách mà dữ liệu cần được truy vấn và hiển thị. Trong những tình huống như vậy, việc ghi dữ liệu dưới một dạng rồi sau đó phái sinh ra các representation được tối ưu hóa cho các loại hình đọc khác nhau có thể mang lại lợi ích.

Trước đó chúng ta đã thấy ý tưởng này trong mục “Systems of Record và Derived Data”, và ETL (xem “Data Warehousing”) là một ví dụ về quá trình phái sinh như vậy. Bây giờ chúng ta sẽ tiến xa hơn với ý tưởng này. Nếu dù sao chúng ta cũng sẽ phái sinh một representation dữ liệu này từ một cái khác, chúng ta có thể chọn các representation khác nhau được tối ưu hóa riêng biệt cho việc ghi và việc đọc. Bạn sẽ mô hình hóa dữ liệu của mình như thế nào nếu bạn chỉ muốn tối ưu hóa nó cho việc ghi, và nếu việc truy vấn hiệu quả không phải là vấn đề cần quan tâm?

Có lẽ cách đơn giản nhất, nhanh nhất và có khả năng biểu đạt cao nhất để ghi dữ liệu là một *event log*: mỗi khi bạn muốn ghi một số dữ liệu, bạn encode nó thành một chuỗi tự chứa (có thể là JSON), bao gồm cả một timestamp, và sau đó append nó vào một chuỗi các event. Các event trong log này là *immutable* (bất biến); bạn không bao giờ thay đổi hay xóa chúng, mà chỉ luôn append thêm các event mới vào log (những event này có thể thay thế các event trước đó). Một event có thể chứa các thuộc tính tùy ý.

Hình 3-8 cho thấy một ví dụ có thể được lấy từ một hệ thống quản lý hội nghị. Một hội nghị có thể là một domain nghiệp vụ phức tạp: không chỉ các cá nhân tham dự có thể đăng ký và thanh toán bằng thẻ, mà các công ty cũng có thể đặt chỗ hàng loạt, thanh toán bằng hóa đơn, và sau đó mới chỉ định các chỗ ngồi đó cho từng cá nhân. Một số lượng chỗ ngồi nhất định có thể được dành riêng cho diễn giả, nhà tài trợ, tình nguyện viên hỗ trợ, v.v. Các lượt đặt chỗ cũng có thể bị hủy, và ban tổ chức hội nghị có thể thay đổi sức chứa của sự kiện bằng cách chuyển sang một phòng khác. Với tất cả những việc này diễn ra, việc chỉ đơn thuần tính toán số lượng chỗ ngồi còn trống trở thành một truy vấn đầy thách thức.

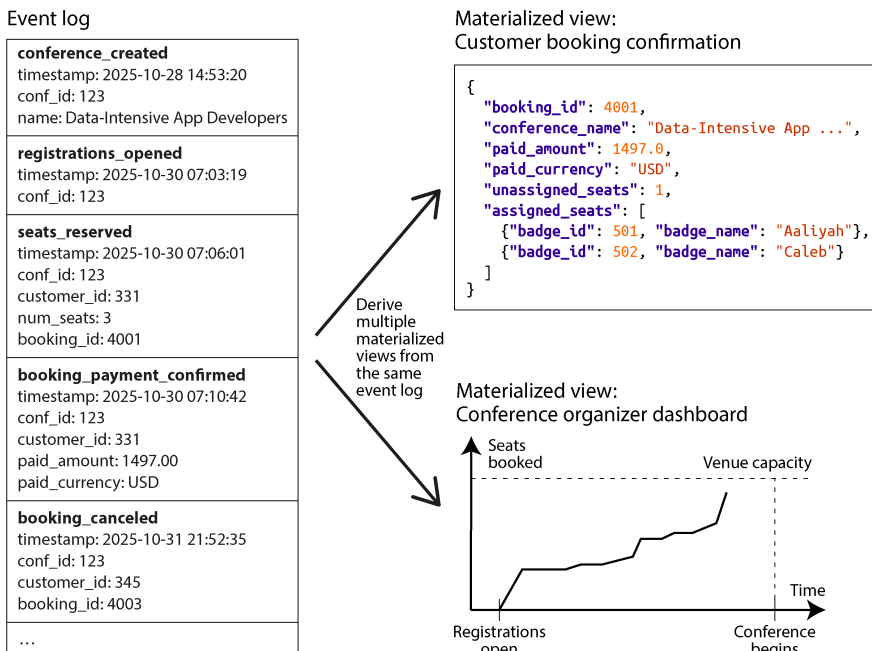


Figure 3-8. Sử dụng một log chứa các event bất biến làm source of truth và tạo ra các materialized view từ đó

Trong Hình 3-8, mọi thay đổi đối với trạng thái của hội nghị (chẳng hạn như việc người tổ chức mở đăng ký, hoặc người tham dự thực hiện và hủy đăng ký) trước tiên đều được lưu trữ dưới dạng một event. Bất cứ khi nào một event được append vào log, một vài *materialized view* (còn được gọi là *projections* hoặc *read models*) cũng sẽ được cập nhật để phản ánh tác động của event đó. Trong ví dụ về hội nghị, có thể có một materialized view thu thập tất cả thông tin liên quan đến trạng thái của mỗi lượt đặt chỗ, một cái khác tính toán các biểu đồ cho dashboard của người tổ chức hội nghị, và cái thứ ba tạo ra các tệp cho máy in để in thẻ tên của người tham dự.

Ý tưởng sử dụng các event làm nguồn sự thật (source of truth) và biểu diễn mọi thay đổi trạng thái dưới dạng một event được gọi là *event sourcing* [65, 66]. Nguyên tắc duy trì các biểu diễn riêng biệt được tối ưu hóa cho việc đọc và dẫn xuất chúng từ biểu diễn được tối ưu hóa cho việc ghi được gọi là *command query responsibility segregation* (CQRS) [67]. Các thuật ngữ này bắt nguồn từ cộng đồng DDD, mặc dù các ý tưởng tương tự đã tồn tại từ lâu—ví dụ, trong state machine replication (xem mục “Using shared logs”).

Khi một yêu cầu từ người dùng gửi đến, nó được gọi là một *command*, và trước tiên nó cần được validate. Một khi command đã được thực thi và được xác định là hợp lệ (ví dụ, có đủ chỗ ngồi trống cho yêu cầu đặt chỗ), nó trở thành một sự thật, và event tương ứng được thêm vào log. Do đó, event log chỉ nên chứa các event hợp lệ, và một

consumer của event log thực hiện xây dựng một materialized view không được phép từ chối bất kỳ event nào.

Khi mô hình hóa dữ liệu của bạn theo phong cách event sourcing, bạn nên đặt tên các event ở thì quá khứ (ví dụ, “the seats were booked” — các chỗ ngồi đã được đặt), bởi vì một event là bản ghi về sự thật rằng một điều gì đó đã xảy ra. Ngay cả khi người dùng sau đó quyết định thay đổi hoặc hủy lượt đặt chỗ của họ, sự thật rằng họ đã từng có một lượt đặt chỗ vẫn không đổi, và việc thay đổi hoặc hủy đó là một event riêng biệt được thêm vào sau đó.

Một điểm tương đồng giữa event sourcing và một fact table trong star schema, đã được thảo luận trong mục “Stars and Snowflakes: Schemas for Analytics”, là cả hai đều là tập hợp các event đã xảy ra trong quá khứ. Tuy nhiên, các hàng trong một fact table đều có cùng một tập hợp các cột, trong khi trong event sourcing có thể có nhiều loại event, mỗi loại có các thuộc tính khác nhau. Ngoài ra, một fact table là một tập hợp không có thứ tự, trong khi trong event sourcing thứ tự của các event là rất quan trọng: nếu một lượt đặt chỗ được thực hiện trước rồi sau đó mới bị hủy, việc xử lý các event đó sai thứ tự sẽ không có ý nghĩa.

Event sourcing và CQRS có một vài ưu điểm:

- Đối với những người phát triển hệ thống, các event truyền đạt tốt hơn ý định về *tại sao* một điều gì đó đã xảy ra. Ví dụ, việc hiểu event “the booking was canceled” (lượt đặt chỗ đã bị hủy) sẽ dễ dàng hơn là “cột active ở hàng 4001 của bảng bookings đã được đặt thành false, ba hàng liên quan đến lượt đặt chỗ đó đã bị xóa khỏi bảng seat_assignments, và một hàng đại diện cho việc hoàn tiền đã được chèn vào bảng payments.” Những thay đổi hàng đó vẫn có thể xảy ra khi một materialized view xử lý event hủy, nhưng khi chúng được dẫn dắt bởi một event, lý do cho các cập nhật đó trở nên rõ ràng hơn nhiều.
- Một nguyên tắc then chốt của event sourcing là các materialized view được dẫn xuất từ event log theo một cách có thể tái lập (reproducible). Bạn luôn phải có khả năng xóa các materialized view và tính toán lại chúng bằng cách xử lý các event tương tự theo cùng một thứ tự, sử dụng cùng một mã nguồn. Nếu có một bug trong mã bảo trì view, bạn chỉ cần xóa view đó và tính toán lại nó với mã nguồn mới. Việc tìm bug cũng dễ dàng hơn vì bạn có thể chạy lại mã bảo trì view bao nhiêu lần tùy thích và kiểm tra hành vi của nó.
- Bạn có thể có nhiều materialized view được tối ưu hóa cho các truy vấn cụ thể mà ứng dụng của bạn yêu cầu. Những view này có thể được lưu trữ trong cùng một cơ sở dữ liệu với các event hoặc trong một cơ sở dữ liệu khác, tùy thuộc vào nhu cầu của bạn. Chúng có thể sử dụng bất kỳ data model nào và có thể được denormalized để đọc nhanh. Bạn thậm chí có thể chỉ giữ một view trong bộ nhớ và tránh việc lưu trữ nó, miễn là việc tính toán lại view từ event log mỗi khi service khởi động lại là chấp nhận được.

- Nếu bạn quyết định muốn trình bày thông tin hiện có theo một cách mới, việc xây dựng một `materialized view` mới từ event log hiện có là rất dễ dàng. Bạn cũng có thể phát triển hệ thống để hỗ trợ các tính năng mới bằng cách thêm các loại event mới hoặc thêm các thuộc tính mới vào các loại event hiện có (bất kỳ event cũ nào cũng vẫn được giữ nguyên không thay đổi). Bạn cũng có thể xâu chuỗi các hành vi mới từ các event hiện có (ví dụ, khi một người tham dự hội nghị hủy đăng ký, chỗ ngồi của họ có thể được dành cho người tiếp theo trong danh sách chờ).
- Nếu một event bị ghi sai, bạn có thể ghi một event xóa tiếp theo để đảo ngược nó. Các view ở hạ nguồn (downstream) sẽ tự động kết hợp việc xóa này, từ đó sửa lại dữ liệu. Ngược lại, trong một cơ sở dữ liệu nơi bạn cập nhật và xóa dữ liệu trực tiếp, một transaction đã được commit thường rất khó để đảo ngược. Do đó, event sourcing có thể giảm số lượng các hành động không thể đảo ngược trong hệ thống, giúp việc thay đổi trở nên dễ dàng hơn (xem mục “Evolvability: Making Change Easy”).
- Event log cũng có thể đóng vai trò như một audit log về những gì đã xảy ra trong hệ thống, điều này rất có giá trị trong các ngành nghề bị kiểm soát chặt chẽ vốn yêu cầu khả năng kiểm toán như vậy.
- Các event log thường có thể xử lý throughput ghi cao hơn các cơ sở dữ liệu nhờ vào các pattern truy cập tuần tự của chúng. Nếu bạn có một đợt bùng phát event tạm thời, log có thể hấp thụ nó, và các hệ thống hạ nguồn duy trì các `materialized view` có thể bắt kịp theo tốc độ riêng của chúng mà không bị quá tải.

Tuy nhiên, event sourcing và CQRS cũng có những nhược điểm:

- Bạn cần cẩn trọng nếu có sự tham gia của thông tin bên ngoài. Ví dụ, giả sử một event chứa một mức giá được đưa ra bằng một loại tiền tệ, và đối với một trong các view, nó cần được chuyển đổi sang một loại tiền tệ khác. Vì tỷ giá hối đoái có thể biến động, việc lấy tỷ giá hối đoái từ một nguồn bên ngoài khi xử lý event sẽ gây ra vấn đề, vì bạn sẽ nhận được một kết quả khác nếu bạn tính toán lại `materialized view` vào một ngày khác. Để làm cho logic xử lý event có tính xác định (deterministic), bạn phải bao gồm tỷ giá hối đoái ngay trong chính event đó hoặc có cách để truy vấn tỷ giá hối đoái lịch sử tại timestamp được chỉ định trong event, đảm bảo rằng truy vấn này luôn trả về cùng một kết quả cho cùng một timestamp.
- Yêu cầu rằng các event phải là bất biến (immutable) sẽ tạo ra vấn đề nếu các event chứa dữ liệu cá nhân của người dùng, vì người dùng có thể thực hiện quyền của họ (ví dụ, theo GDPR) để yêu cầu xóa dữ liệu của họ. Nếu event log được quản lý theo từng người dùng, bạn chỉ cần xóa toàn bộ log của người dùng đó, nhưng điều này không khả thi nếu event log của bạn chứa các event liên quan

đến nhiều người dùng. Bạn có thể thử lưu trữ dữ liệu cá nhân bên ngoài event thực tế, hoặc mã hóa nó bằng một key mà bạn có thể chọn xóa sau đó (một kỹ thuật được gọi là *crypto-shredding* [68]), nhưng điều đó cũng làm cho việc tính toán lại `derived state` khi cần thiết trở nên khó khăn hơn.

- Việc xử lý lại (reprocessing) các event đòi hỏi sự cẩn trọng nếu có các `side effect` có thể nhìn thấy từ bên ngoài—ví dụ, bạn có lẽ không muốn gửi lại email xác nhận mỗi khi bạn xây dựng lại một `materialized view`.

Bạn có thể triển khai `event sourcing` trên bất kỳ cơ sở dữ liệu nào, nhưng một số hệ thống được thiết kế đặc biệt để hỗ trợ pattern này, chẳng hạn như EventStoreDB, MartenDB (dựa trên PostgreSQL), và Axon Framework. Bạn cũng có thể sử dụng các message broker như Apache Kafka để lưu trữ `event log`, và các `stream processor` có thể giữ cho các `materialized view` luôn được cập nhật; chúng ta sẽ quay lại các chủ đề này trong Chương 12.

Yêu cầu quan trọng duy nhất là hệ thống lưu trữ event phải đảm bảo rằng tất cả các `materialized view` xử lý các event theo đúng thứ tự mà chúng xuất hiện trong log. Như chúng ta sẽ thấy trong Chương 10, điều này không phải lúc nào cũng dễ dàng đạt được trong một `distributed system`.

DataFrame, Matrix và Array

Các mô hình dữ liệu mà chúng ta đã thấy cho đến nay trong chương này thường được sử dụng cho cả mục đích xử lý transaction và phân tích (xem “Operational Versus Analytical Systems”). Cũng có một vài mô hình dữ liệu mà bạn có khả năng sẽ gặp trong bối cảnh phân tích hoặc khoa học nhưng hiếm khi xuất hiện trong các hệ thống OLTP, bao gồm DataFrame và các multidimensional array của các con số như matrix.

Mô hình dữ liệu *DataFrame* được hỗ trợ bởi ngôn ngữ R, thư viện Pandas cho Python, Apache Spark, ArcticDB, Dask và các hệ thống khác. DataFrame là một công cụ phổ biến cho các nhà khoa học dữ liệu khi chuẩn bị dữ liệu để huấn luyện các mô hình ML, nhưng chúng cũng được sử dụng rộng rãi để khám phá dữ liệu, phân tích dữ liệu thống kê, trực quan hóa dữ liệu và các mục đích tương tự.

Thoạt nhìn, một DataFrame tương tự như một bảng trong một relational database hoặc một bảng tính (spreadsheet). Một DataFrame hỗ trợ các toán tử giống như quan hệ (relational-like) để thực hiện các thao tác hàng loạt trên nội dung của nó; ví dụ, áp dụng một hàm cho tất cả các hàng, lọc các hàng dựa trên một điều kiện, nhóm các hàng theo một số cột và tổng hợp các cột khác, và join các hàng trong một DataFrame với một DataFrame khác dựa trên một key (thứ mà một relational database gọi là *join* thì thường được gọi là *merge* trên DataFrame).

Thay vì sử dụng một ngôn ngữ truy vấn declarative như SQL, một DataFrame thường được thao tác thông qua một chuỗi các lệnh nhằm sửa đổi cấu trúc và nội dung của nó. Điều này khớp với workflow điển hình của các nhà khoa học dữ liệu, những người thực hiện "wrangle" dữ liệu theo từng bước để đưa chúng về dạng cho phép họ tìm ra câu trả lời cho các câu hỏi mà họ đang đặt ra. Những thao tác này thường diễn ra trên bản sao riêng tư của dataset của nhà khoa học dữ liệu, thường là trên máy cục bộ của họ, mặc dù kết quả cuối cùng có thể được chia sẻ với những người dùng khác.

Các DataFrame API cũng cung cấp rất nhiều thao tác vượt xa những gì các relational database cung cấp, và mô hình dữ liệu này thường được sử dụng theo những cách rất khác so với mô hình hóa dữ liệu quan hệ điển hình [69]. Ví dụ, một cách sử dụng phổ biến của DataFrame là chuyển đổi dữ liệu từ dạng biểu diễn giống quan hệ sang dạng biểu diễn matrix hoặc multidimensional array, vốn là dạng mà nhiều thuật toán ML mong đợi ở đầu vào của chúng.

Một ví dụ đơn giản về sự chuyển đổi như vậy được hiển thị trong Hình 3-9. Bên trái, chúng ta có một bảng quan hệ cho biết xếp hạng của người dùng đối với các bộ phim khác nhau (trên thang điểm từ 1 đến 5), và bên phải, dữ liệu đã được chuyển đổi thành một matrix trong đó mỗi cột là một bộ phim và mỗi hàng là một người dùng (tương tự như một *pivot table* trong bảng tính). Matrix này là *sparse* (thưa), nghĩa là không có dữ liệu cho nhiều tổ hợp người dùng–phim, nhưng điều này không vấn đề gì. Matrix này có thể có hàng ngàn cột và do đó sẽ không phù hợp trong một relational database, nhưng DataFrame và các thư viện cung cấp sparse array (chẳng hạn như NumPy cho Python) có thể xử lý dữ liệu như vậy một cách dễ dàng.

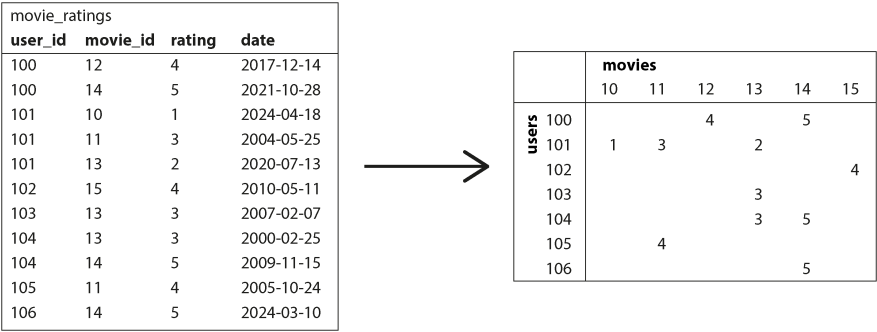


Figure 3-9. Chuyển đổi một cơ sở dữ liệu quan hệ về xếp hạng phim thành một ma trận

Một matrix chỉ có thể chứa các con số, và nhiều kỹ thuật khác nhau được sử dụng để chuyển đổi dữ liệu phi số thành các con số trong matrix. Ví dụ:

- Các ngày tháng (vốn được lược bỏ khỏi ví dụ về matrix trong Hình 3-9) có thể được scale thành các số thực dấu phẩy động (floating-point numbers) trong một phạm vi phù hợp.

- Đối với các cột chỉ có thể nhận một trong một tập hợp nhỏ các giá trị cố định (ví dụ: thể loại của một bộ phim trong một cơ sở dữ liệu phim), *one-hot encoding* thường được sử dụng. Chúng ta tạo một cột cho mỗi giá trị có thể có (“comedy,” “drama,” “horror,” v.v.), và đối với mỗi hàng đại diện cho một bộ phim, chúng ta đặt giá trị 1 vào cột tương ứng với thể loại của bộ phim đó và đặt giá trị 0 vào tất cả các cột còn lại. Cách biểu diễn này cũng dễ dàng mở rộng cho các bộ phim thuộc về nhiều thể loại cùng lúc.

Một khi dữ liệu đã ở dạng *matrix* các con số, nó có thể áp dụng các phép toán đại số tuyến tính, vốn là cơ sở của nhiều thuật toán ML. Ví dụ, dữ liệu trong Hình 3-9 có thể là một phần của hệ thống gợi ý các bộ phim mà người dùng có thể thích. *DataFrames* đủ linh hoạt để cho phép dữ liệu tiến hóa dần dần từ dạng quan hệ sang dạng biểu diễn *matrix*, đồng thời cho phép nhà khoa học dữ liệu kiểm soát cách biểu diễn phù hợp nhất để đạt được các mục tiêu của quá trình phân tích dữ liệu hoặc quá trình huấn luyện mô hình.

Một số cơ sở dữ liệu, chẳng hạn như *TileDB* [70], chuyên về lưu trữ các mảng đa chiều lớn chứa các con số; chúng được gọi là *array databases* và thường được sử dụng nhất cho các *dataset* khoa học như các phép đo địa không gian (dữ liệu raster trên một lưới phân bố đều), hình ảnh y tế, hoặc các quan sát từ kính thiên văn [71]. *DataFrames* cũng được sử dụng trong ngành tài chính để biểu diễn *dữ liệu chuỗi thời gian* (*time-series data*), chẳng hạn như giá của các tài sản và các giao dịch theo thời gian [72]. Do sự phổ biến của chúng đối với các nhà khoa học dữ liệu, *DataFrames* cũng đã được thêm vào các framework xử lý batch như *Spark* và *Flink*; chúng ta sẽ quay lại chủ đề này trong Chương 11.

Tóm tắt

Các mô hình dữ liệu là một chủ đề khổng lồ, và trong chương này, chúng ta đã xem xét nhanh một loạt các mô hình đa dạng. Chúng ta không có đủ không gian để đi sâu vào mọi chi tiết của từng mô hình, nhưng hy vọng rằng phần tổng quan này đã đủ để khơi gợi sự hứng thú của bạn nhằm tìm hiểu thêm về mô hình phù hợp nhất với các yêu cầu ứng dụng của bạn.

Mô hình *relational* (quan hệ), mặc dù đã có tuổi đời hơn nửa thế kỷ, vẫn là một mô hình dữ liệu quan trọng cho nhiều ứng dụng—đặc biệt là trong *data warehousing* và phân tích kinh doanh, nơi các *star schema* hoặc *snowflake schema* dạng quan hệ và các truy vấn SQL xuất hiện ở khắp mọi nơi. Tuy nhiên, một số giải pháp thay thế cho dữ liệu quan hệ đã trở nên phổ biến trong các lĩnh vực khác:

- Mô hình *document* hướng tới các trường hợp sử dụng mà dữ liệu đến dưới dạng các tài liệu JSON độc lập và nơi các mối quan hệ giữa tài liệu này với tài liệu khác là hiếm gặp.

- *Graph data models* đi theo hướng ngược lại, hướng tới các trường hợp sử dụng mà bất cứ thứ gì cũng có khả năng liên quan đến mọi thứ và nơi các truy vấn có khả năng cần phải duyệt qua nhiều bước (hops) để tìm dữ liệu được quan tâm (một nhu cầu có thể được đáp ứng bằng cách sử dụng các truy vấn đệ quy trong Cypher, SPARQL, hoặc Datalog).
- DataFrames tổng quát hóa dữ liệu quan hệ thành một số lượng lớn các cột, tạo ra một cầu nối giữa các cơ sở dữ liệu và các mảng đa chiều vốn là cơ sở của nhiều kỹ thuật ML, phân tích dữ liệu thống kê và tính toán khoa học.

Ở một mức độ nào đó, một mô hình thường có thể được mô phỏng thông qua một mô hình khác—ví dụ, dữ liệu đồ thị có thể được biểu diễn trong một cơ sở dữ liệu quan hệ—nhưng kết quả có thể gây khó khăn, như chúng ta đã thấy với việc hỗ trợ các truy vấn đệ quy trong SQL.

Do đó, nhiều cơ sở dữ liệu chuyên dụng đã được phát triển cho từng data model, cung cấp các ngôn ngữ truy vấn và storage engine được tối ưu hóa cho mô hình cụ thể đó. Tuy nhiên, cũng có một xu hướng là các cơ sở dữ liệu mở rộng sang các ngách lân cận bằng cách thêm hỗ trợ cho các data model khác—ví dụ, các relational database đã thêm hỗ trợ cho document data dưới dạng các cột JSON, các document database đã thêm các phép join giống như relational, và việc hỗ trợ graph data trong SQL cũng đang dần được cải thiện.

Một mô hình khác mà chúng ta đã thảo luận là *event sourcing*, mô hình này biểu diễn dữ liệu dưới dạng một append-only log của các event bất biến và có thể mang lại lợi thế khi mô hình hóa các hoạt động trong các domain kinh doanh phức tạp. Một append-only log rất tốt cho việc ghi dữ liệu (như chúng ta sẽ thấy trong Chương 4); để hỗ trợ các truy vấn hiệu quả, event log được chuyển đổi thành các materialized view được tối ưu hóa cho việc đọc thông qua CQRS.

Một điểm chung của các nonrelational data model là chúng thường không enforce một schema cho dữ liệu mà chúng lưu trữ, điều này có thể giúp việc thích ứng các ứng dụng với các yêu cầu thay đổi trở nên dễ dàng hơn. Tuy nhiên, ứng dụng của bạn rất có thể vẫn giả định rằng dữ liệu có một cấu trúc nhất định; vấn đề chỉ là liệu schema đó là explicit (được enforced khi ghi) hay implicit (được giả định khi đọc).

Mặc dù chúng ta đã đề cập đến rất nhiều nội dung, một số data model vẫn chưa được nhắc tới. Dưới đây là một vài ví dụ ngắn gọn:

- Các nhà nghiên cứu làm việc với dữ liệu genome thường cần thực hiện *sequence similarity searches*, quá trình này lấy một chuỗi rất dài (đại diện cho một phân tử DNA) và khớp nó với một database lớn gồm các chuỗi tương tự nhưng không hoàn toàn giống hệt. Không có cơ sở dữ liệu nào được mô tả ở đây có thể xử lý loại sử dụng này, đó là lý do tại sao các nhà nghiên cứu đã viết các phần mềm database genome chuyên dụng như GenBank [73].

- Nhiều hệ thống tài chính sử dụng *ledgers* với kế toán ghi sổ kép làm data model của chúng. Loại dữ liệu này có thể được biểu diễn trong các relational database, nhưng cũng có những cơ sở dữ liệu (chẳng hạn như TigerBeetle) chuyên về data model này. Cryptocurrencies và blockchains thường dựa trên các distributed ledgers, vốn cũng đã tích hợp việc chuyển giao giá trị vào trong data model của chúng.
- *Full-text search* có thể coi là một loại data model thường được sử dụng song song với các cơ sở dữ liệu. Truy hồi thông tin (information retrieval) là một chủ đề chuyên môn lớn mà chúng ta sẽ không đề cập quá chi tiết trong cuốn sách này, nhưng chúng ta sẽ chạm đến các search index và vector search trong mục “Full-Text Search”.

Chúng ta phải dừng lại ở đây. Trong chương tiếp theo, chúng ta sẽ thảo luận về một số trade-off nảy sinh khi *triển khai* các data model đã được mô tả trong chương này.

Footnotes

References

- [1] Jamie Brandon. “Unexplanations: Query Optimization Works Because SQL Is Declarative.” *scattered-thoughts.net*, February 2024. Archived at perma.cc/P6W2-WMFZ
- [2] Neel Krishnaswami. “What Declarative Languages Are.” *semantic-domain.blogspot.com*, July 2013. Archived at perma.cc/R4LP-T2RV
- [3] Joseph M. Hellerstein. “The Declarative Imperative: Experiences and Conjectures in Distributed Logic.” Tech report UCB/EECS-2010-90, Electrical Engineering and Computer Sciences, University of California at Berkeley, June 2010. Archived at perma.cc/K56R-1VQM
- [4] Edgar F. Codd. “A Relational Model of Data for Large Shared Data Banks.” *Communications of the ACM*, volume 13, issue 6, pages 377–387, June 1970. doi:10.1145/362384.362685
- [5] Michael Stonebraker and Joseph M. Hellerstein. “What Goes Around Comes Around.” In *Readings in Database Systems*, 4th edition, MIT Press, 2005, pages 2–41. ISBN: 9780262693141
- [6] Markus Winand. “Modern SQL: Beyond Relational.” *modern-sql.com*, 2015. Archived at perma.cc/D63V-WAPN
- [7] Martin Fowler. “Orm Hate.” *martinfowler.com*, May 2012. Archived at perma.cc/VCM8-PKNG
- [8] Vlad Mihalcea. “N+1 Query Problem with JPA and Hibernate.” *vladmihalcea.com*, January 2023. Archived at perma.cc/79EV-TZKB
- [9] Jens Schauder. “This Is the Beginning of the End of the N+1 Problem: Introducing Single Query Loading.” *spring.io*, August 2023. Archived at perma.cc/6V96-R333
- [10] Jamie Brandon. “SQL Needed Structure.” *scattered-thoughts.net*, September 2025. Archived at perma.cc/9EVK-HLVR
- [11] William Zola. “6 Rules of Thumb for MongoDB Schema Design.” *mongodb.com*, June 2014. Archived at perma.cc/T2BZ-PPJB
- [12] Sidney Andrews and Christopher McClister. “Data Modeling in Azure Cosmos DB.” *learn.microsoft.com*, February 2023. Archived at archive.org

- [13] Raffi Krikorian. “Timelines at Scale.” At *QCon San Francisco*, November 2012. Archived at perma.cc/V9G5-KLYK
- [14] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd edition. John Wiley & Sons, 2013. ISBN: 9781118530801
- [15] Michael Kaminsky. “Data Warehouse Modeling: Star Schema vs. OBT.” *fiveplan.com*, August 2022. Archived at perma.cc/2PZK-BFFP
- [16] Joe Nelson. “User-defined Order in SQL.” *begriffs.com*, March 2018. Archived at perma.cc/GS3W-F7AD
- [17] Evan Wallace. “Realtime Editing of Ordered Sequences.” *figma.com*, March 2017. Archived at perma.cc/K6ER-CQZW
- [18] David Greenspan. “Implementing Fractional Indexing.” *observablehq.com*, October 2020. Archived at perma.cc/5N4R-MREN
- [19] Martin Fowler. “Schemaless Data Structures.” *martinfowler.com*, January 2013.
- [20] Amr Awadallah. “Schema-on-Read vs. Schema-on-Write.” At *Berkeley EECS RAD Lab Retreat*, May 2009. Archived at perma.cc/DTB2-JCFR
- [21] Martin Odersky. “The Trouble with Types.” At *Strange Loop*, September 2013. Archived at perma.cc/85QE-PVEP
- [22] Conrad Irwin. “MongoDB—Confessions of a PostgreSQL Lover.” At *HTML5DevConf*, October 2013. Archived at perma.cc/C2J6-3AL5
- [23] “Percona Toolkit Documentation: pt-online-schema-change.” *docs.percona.com*, 2023. Archived at perma.cc/9K8R-E5UH
- [24] Shlomi Noach. “gh-ost: GitHub’s Online Schema Migration Tool for MySQL.” *github.blog*, August 2016. Archived at perma.cc/7XAG-XB72
- [25] Shayon Mukherjee. “pg-osc: Zero Downtime Schema Changes in PostgreSQL.” *shayon.dev*, February 2022. Archived at perma.cc/35WN-7WMY
- [26] Carlos Pérez-Arados Herce. “Introducing pgtroll: Zero-Downtime, Reversible, Schema Migrations for Postgres.” *xata.io*, October 2023. Archived at archive.org
- [27] James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, JJ Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, Wilson Hsieh, Sebastian Kanthak, Eugene Kogan, Hongyi Li, Alexander Lloyd, Sergey Melnik, David Mwaura, David Nagle, Sean Quinlan, Rajesh Rao, Lindsay Rolig, Dale Woodford, Yasushi Saito, Christopher Taylor, Michal Szymaniak, and Ruth Wang. “Spanner: Google’s Globally-Distributed Database.” At *10th USENIX Symposium on Operating System Design and Implementation (OSDI)*, October 2012.
- [28] Donald K. Burleson. “Reduce I/O with Oracle Cluster Tables.” *dba-oracle.com*. Archived at perma.cc/7LBJ-9X2C
- [29] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E. Gruber. “Bigtable: A Distributed Storage System for Structured Data.” At *7th USENIX Symposium on Operating System Design and Implementation (OSDI)*, November 2006.
- [30] Priscilla Walmsley. *XQuery*, 2nd edition. O’Reilly Media, 2015. ISBN: 9781491915080
- [31] Paul C. Bryan, Kris Zyp, and Mark Nottingham. “JavaScript Object Notation (JSON) Pointer.” RFC 6901, IETF, April 2013.

- [32] Stefan Gössner, Glyn Normington, and Carsten Bormann. “JSONPath: Query Expressions for JSON.” RFC 9535, IETF, February 2024.
- [33] Michael Stonebraker and Andrew Pavlo. “What Goes Around Comes Around... And Around....” *ACM SIGMOD Record*, volume 53, issue 2, pages 21–37, July 2024. doi:10.1145/3685980.3685984
- [34] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. “The PageRank Citation Ranking: Bringing Order to the Web.” Technical Report 1999-66, Stanford University InfoLab, November 1999. Archived at perma.cc/UML9-UZHW
- [35] Nathan Bronson, Zach Amsden, George Cabrera, Prasad Chakka, Peter Dimov, Hui Ding, Jack Ferris, Anthony Giardullo, Sachin Kulkarni, Harry Li, Mark Marchukov, Dmitri Petrov, Lovro Puzar, Yee Jiun Song, and Venkat Venkataramani. “TAO: Facebook’s Distributed Data Store for the Social Graph.” At *USENIX Annual Technical Conference (ATC)*, June 2013.
- [36] Natasha Noy, Yuqing Gao, Anshu Jain, Anant Narayanan, Alan Patterson, and Jamie Taylor. “Industry-Scale Knowledge Graphs: Lessons and Challenges.” *Communications of the ACM*, volume 62, issue 8, pages 36–43, August 2019. doi:10.1145/3331166
- [37] Xiyang Feng, Guodong Jin, Ziyi Chen, Chang Liu, and Semih Salihoglu. “KÜZU Graph Database Management System.” At *13th Annual Conference on Innovative Data Systems Research (CIDR 2023)*, January 2023. Archived at perma.cc/PS6f-ZBZU
- [38] Maciej Besta, Emanuel Peter, Robert Gerstenberger, Marc Fischer, Michał Podstawski, Claude Barthels, Gustavo Alonso, Torsten Hoefer. “Demystifying Graph Databases: Analysis and Taxonomy of Data Organization, System Designs, and Graph Queries.” *arXiv:1910.09017*, October 2019.
- [39] “Apache TinkerPop. TinkerPop 3.6.3 Documentation.” tinkerpop.apache.org, May 2023. Archived at perma.cc/KM7W-7PAT
- [40] Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andrés Taylor. “Cypher: An Evolving Query Language for Property Graphs.” At *International Conference on Management of Data (SIGMOD)*, May 2018. doi:10.1145/3183713.3190657
- [41] Emil Eifrem. Twitter correspondence, January 2014. Archived at perma.cc/WM4S-BW64
- [42] Francesco Tisot. “Explore the New SEARCH and CYCLE Features in PostgreSQL® 14.” *aiven.io*, December 2021. Archived at perma.cc/j6BT-83UZ
- [43] Gaurav Goel. “Understanding Hierarchies in Oracle.” *towardsdatascience.com*, May 2020. Archived at perma.cc/SZLR-Q7EW
- [44] Alin Deutsch, Yu Xu, and Mingxi Wu. “Seamless Syntactic and Semantic Integration of Query Primitives over Relational and Graph Data in GSQL.” *tigergraph.com*, November 2018. Archived at perma.cc/JG7J-Y35X
- [45] Oskar van Rest, Sungpack Hong, Jinha Kim, Xuming Meng, and Hassan Chafi. “PGQL: A Property Graph Query Language.” At *4th International Workshop on Graph Data Management Experiences and Systems (GRADES)*, June 2016. doi:10.1145/2960414.2960421
- [46] Philip Rathle and Brad Bebee. “GQL: The ISO Standard for Graphs Has Arrived.” *aws.amazon.com*, April 2024. Archived at perma.cc/5TEU-N2Y8
- [47] Alin Deutsch, Nadime Francis, Alastair Green, Keith Hare, Bei Li, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Wim Martens, Jan Michels, Filip Murlak, Stefan Plantikow, Petra Selmer, Oskar van Rest, Hannes Voigt, Domagoj Vrgoč, Mingxi Wu, and Fred Zemke. “Graph Pattern Matching in GQL and SQL/PGQ.” At *International Conference on Management of Data (SIGMOD)*, June 2022. doi:10.1145/3514221.3526057

- [48] Alastair Green. “SQL...And Now GQL.” *opencypher.org*, September 2019. Archived at perma.cc/AFB2-3SY7
- [49] Amazon Web Services. “Neptune Graph Data Model.” Amazon Neptune User Guide, *docs.aws.amazon.com*. Archived at perma.cc/CX3T-EZU9
- [50] Cognitect. “Datomic Data Model.” Datomic Cloud Documentation, *docs.datomic.com*. Archived at perma.cc/LGM9-LEUT
- [51] David Beckett and Tim Berners-Lee. “Turtle—Terse RDF Triple Language.” W3C Team Submission, March 2011.
- [52] Sinclair Target. “Whatever Happened to the Semantic Web?” *twobithistory.org*, May 2018. Archived at perma.cc/M8GL-9KHS
- [53] Gavin Mendel-Gleason. “The Semantic Web Is Dead—Long Live the Semantic Web!” *terminusdb.com*, August 2022. Archived at perma.cc/G2MZ-DSS3
- [54] Manu Sporny. “JSON-LD and Why I Hate the Semantic Web.” *manu.sporny.org*, January 2014. Archived at perma.cc/7PT4-PJKF
- [55] University of Michigan Library. “Biomedical Ontologies and Controlled Vocabularies.” *guides.lib.umich.edu/ontology*. Archived at perma.cc/Q5GA-F2N8
- [56] Facebook. “The Open Graph Protocol.” *ogp.me*. Archived at perma.cc/C49A-GUSY
- [57] Matt Haughey. “Everything You Ever Wanted to Know About Unfurling but Were Afraid to Ask / or/ How to Make Your Site Previews Look Amazing in Slack.” *medium.com*, November 2015. Archived at perma.cc/C7S8-4PZN
- [58] W3C RDF Working Group. “Resource Description Framework (RDF).” *w3.org*, February 2004.
- [59] Steve Harris, Andy Seaborne, and Eric Prud’hommeaux. “SPARQL 1.1 Query Language.” W3C Recommendation, March 2013.
- [60] Todd J. Green, Shan Shan Huang, Boon Thau Loo, and Wenchao Zhou. “Datalog and Recursive Query Processing.” *Foundations and Trends in Databases*, volume 5, issue 2, pages 105–195, November 2013. [doi:10.1561/19000000017](https://doi.org/10.1561/19000000017)
- [61] Stefano Ceri, Georg Gottlob, and Letizia Tanca. “What You Always Wanted to Know About Datalog (And Never Dared to Ask).” *IEEE Transactions on Knowledge and Data Engineering*, volume 1, issue 1, pages 146–166, March 1989. [doi:10.1109/69.43410](https://doi.org/10.1109/69.43410)
- [62] Serge Abiteboul, Richard Hull, and Victor Vianu. *Foundations of Databases*. Addison-Wesley, 1995. ISBN: 9780201537710. Available online at webdam.inria.fr/Alice.
- [63] Scott Meyer, Andrew Carter, and Andrew Rodriguez. “LIquid: The Soul of a New Graph Database, Part 2.” *engineering.linkedin.com*, September 2020. Archived at perma.cc/K9M4-PD6Q
- [64] Matt Bessey. “Why, After 6 Years, I’m over GraphQL.” *bessey.dev*, May 2024. Archived at perma.cc/2PAU-JYRA
- [65] Dominic Betts, Julián Domínguez, Grigori Melnik, Fernando Simonazzi, and Mani Subramanian. *Exploring CQRS and Event Sourcing*. Microsoft Patterns & Practices, 2012. ISBN: 9781621140164. Archived at perma.cc/7A39-3NM8
- [66] Greg Young. “CQRS and Event Sourcing.” At *Code on the Beach*, August 2014.
- [67] Greg Young. “CQRS Documents.” *cqrs.wordpress.com*, November 2010. Archived at perma.cc/X5R6-R47F

- [68] Brent Robinson. “Crypto Shredding: How It Can Solve Modern Data Retention Challenges.” *medium.com*, January 2019. Archived at perma.cc/4LFK-S6XE
- [69] Devin Petersohn, Stephen Macke, Doris Xin, William Ma, Doris Lee, Xiangxi Mo, Joseph E. Gonzalez, Joseph M. Hellerstein, Anthony D. Joseph, and Aditya Parameswaran. “Towards Scalable Dataframe Systems.” *Proceedings of the VLDB Endowment*, volume 13, issue 11, pages 2033–2046, July 2020. [doi:10.14778/3407790.3407807](https://doi.org/10.14778/3407790.3407807)
- [70] Stavros Papadopoulos, Kushal Datta, Samuel Madden, and Timothy Mattson. “The TileDB Array Data Storage Manager.” *Proceedings of the VLDB Endowment*, volume 10, issue 4, pages 349–360, November 2016. [doi:10.14778/3025111.3025117](https://doi.org/10.14778/3025111.3025117)
- [71] Florin Rusu. “Multidimensional Array Data Management.” *Foundations and Trends in Databases*, volume 12, issues 2–3, pages 69–220, February 2023. [doi:10.1561/19000000069](https://doi.org/10.1561/19000000069)
- [72] Ed Targett. “Bloomberg, Man Group Team Up to Develop Open Source ‘ArcticDB’ Database.” *thestack.technology*, March 2023. Archived at perma.cc/M5YD-QQYV
- [73] Dennis A. Benson, Ilene Karsch-Mizrachi, David J. Lipman, James Ostell, and David L. Wheeler. *GenBank*. *Nucleic Acids Research*, volume 36, issue suppl_1, pages D25–D30, January 2008. [doi:10.1093/nar/gkm929](https://doi.org/10.1093/nar/gkm929)

Lưu trữ và Truy hồi

Một trong những nỗi khổ của cuộc đời là mọi người đều đặt tên cho mọi thứ sai một chút. Và điều đó khiến mọi thứ trên thế giới trở nên khó hiểu hơn một chút so với việc nếu chúng được đặt tên khác đi. Một máy tính không chủ yếu tính toán theo nghĩa thực hiện các phép toán số học. [...] Chúng chủ yếu là các hệ thống lưu trữ thô sơ.

Richard Feynman, hội thảo *Idiosyncratic Thinking* (1985)

Ở cấp độ cơ bản nhất, một cơ sở dữ liệu cần thực hiện hai việc: khi bạn đưa cho nó một ít dữ liệu, nó nên lưu trữ dữ liệu đó, và khi bạn yêu cầu lại sau đó, nó nên trả dữ liệu đó lại cho bạn.

Trong Chương 3, chúng ta đã thảo luận về các mô hình dữ liệu và ngôn ngữ truy vấn —tức là định dạng mà bạn đưa dữ liệu cho cơ sở dữ liệu, và giao diện mà qua đó bạn có thể yêu cầu lại dữ liệu đó sau này. Trong chương này, chúng ta sẽ thảo luận cùng một vấn đề nhưng từ góc độ của cơ sở dữ liệu: cách cơ sở dữ liệu có thể lưu trữ dữ liệu mà bạn đưa cho nó, và cách nó có thể tìm lại dữ liệu khi bạn yêu cầu.

Tại sao bạn, với tư cách là một lập trình viên ứng dụng, lại phải quan tâm đến việc cơ sở dữ liệu xử lý lưu trữ và truy hồi bên trong như thế nào? Có lẽ bạn sẽ không tự triển khai một storage engine của riêng mình từ đầu, nhưng bạn *thực sự* cần chọn một storage engine phù hợp cho ứng dụng của mình từ rất nhiều lựa chọn có sẵn. Để cấu hình một storage engine hoạt động tốt với loại workload của bạn, bạn cần có một ý niệm sơ bộ về những gì storage engine đang thực hiện bên trong (under the hood).

Đặc biệt, có một sự khác biệt lớn giữa các storage engine được tối ưu hóa cho các workload giao dịch (OLTP) và những cái được tối ưu hóa cho phân tích (chúng ta đã giới thiệu sự phân biệt này trong mục “Hệ thống Vận hành so với Hệ thống Phân tích”). Chương này bắt đầu bằng việc xem xét hai họ storage engine cho OLTP: các storage engine *log-structured* ghi ra các tệp dữ liệu bất biến, và các storage engine như *B-trees* cập nhật dữ liệu tại chỗ (in place). Các cấu trúc này được sử dụng cho cả lưu trữ key-value và các secondary index.

Trong mục “Lưu trữ dữ liệu cho Phân tích”, chúng ta sẽ thảo luận về một họ các storage engine được tối ưu hóa cho phân tích, và trong mục “Chỉ mục đa chiều và Toàn văn”, chúng ta sẽ xem xét các index cho các truy vấn nâng cao hơn, chẳng hạn như truy hồi văn bản.

Lưu trữ và Đánh chỉ mục cho OLTP

Hãy xét cơ sở dữ liệu đơn giản nhất thế giới, được triển khai dưới dạng hai hàm bash:

```
#!/bin/bash

db_set () {
    echo "$1,$2" >> database
}

db_get () {
    grep "^$1," database | sed -e "s/^$1,//" | tail -n 1
}
```

Hai hàm này triển khai một key-value store. Bạn có thể gọi `db_set key value`, nó sẽ lưu trữ `key` và `value` vào cơ sở dữ liệu. `Key` và `value` có thể là (gần như) bất cứ thứ gì bạn muốn—ví dụ, `value` có thể là một tài liệu JSON. Sau đó, bạn có thể gọi `db_get key`, nó sẽ tìm kiếm `value` gần nhất liên kết với `key` cụ thể đó và trả về nó.

Và nó hoạt động:

```
$ db_set 12 '{"name":"London","attractions":["Big Ben","London Eye"]}'

$ db_set 42 '{"name":"San Francisco","attractions":["Golden Gate Bridge"]}'

$ db_get 42
{"name":"San Francisco","attractions":["Golden Gate Bridge"]}
```

Định dạng lưu trữ rất đơn giản: một tệp văn bản trong đó mỗi dòng chứa một cặp key-value, được phân tách bởi dấu phẩy (gần giống như một tệp CSV, nếu bỏ qua các vấn đề về escaping). Mọi lệnh gọi đến `db_set` đều thực hiện append vào cuối tệp. Nếu bạn cập nhật một key nhiều lần, các phiên bản cũ của `value` sẽ không bị ghi đè—you cần tìm lần xuất hiện cuối cùng của một key trong tệp để tìm giá trị mới nhất (do đó có `tail -n 1` trong `db_get`):

```
$ db_set 42 '{"name":"San Francisco","attractions":["Exploratorium"]}'

$ db_get 42
{"name":"San Francisco","attractions":["Exploratorium"]}

$ cat database
12,{"name":"London","attractions":["Big Ben","London Eye"]}
42,{"name":"San Francisco","attractions":["Golden Gate Bridge"]}
42,{"name":"San Francisco","attractions":["Exploratorium"]}
```

Hàm `db_set` có hiệu năng khá tốt đối với một thứ đơn giản như vậy, bởi vì việc append vào một tệp thường rất hiệu quả. Tương tự như những gì `db_set` thực

hiện, nhiều cơ sở dữ liệu sử dụng một *log* bên trong, vốn là một tệp dữ liệu chỉ cho phép append. Các cơ sở dữ liệu thực tế có nhiều vấn đề hơn phải xử lý (chẳng hạn như xử lý các lệnh ghi đồng thời, thu hồi không gian đĩa để log không tăng lên mãi mãi, và xử lý các bản ghi bị ghi dở dang khi khôi phục sau sự cố crash), nhưng nguyên tắc cơ bản là giống nhau. Các log cực kỳ hữu ích, và chúng ta sẽ gặp lại chúng nhiều lần trong cuốn sách này.

LƯU Ý

Từ *log* thường được dùng để chỉ các application log, nơi một ứng dụng xuất ra văn bản mô tả những gì đang diễn ra. Trong cuốn sách này, *log* được sử dụng theo nghĩa tổng quát hơn: một chuỗi các bản ghi chỉ cho phép ghi thêm (append-only) trên đĩa. Nó không nhất thiết phải ở dạng con người có thể đọc được; nó có thể ở dạng nhị phân và chỉ dành cho việc sử dụng nội bộ bởi hệ thống cơ sở dữ liệu.

Mặt khác, hàm `db_get` có hiệu năng rất kém nếu bạn có một số lượng lớn các bản ghi trong cơ sở dữ liệu của mình. Mỗi khi bạn muốn tra cứu một key, `db_get` phải quét toàn bộ tệp cơ sở dữ liệu từ đầu đến cuối để tìm các vị trí xuất hiện của key đó. Theo thuật ngữ thuật toán, chi phí của một lần tra cứu là $O(n)$: nếu bạn tăng gấp đôi số lượng bản ghi n trong cơ sở dữ liệu, một lần tra cứu sẽ mất thời gian gấp đôi. Điều đó không hề tốt.

Để tìm giá trị cho một key cụ thể trong cơ sở dữ liệu một cách hiệu quả, chúng ta cần một cấu trúc dữ liệu khác: một *index*. Trong chương này, chúng ta sẽ xem xét một loạt các cấu trúc indexing và so sánh chúng. Ý tưởng chung là cấu trúc dữ liệu theo một cách cụ thể (ví dụ: được sắp xếp theo một key) để giúp việc định vị dữ liệu bạn muốn trở nên nhanh hơn. Nếu bạn muốn tìm kiếm cùng một dữ liệu theo nhiều cách khác nhau, bạn có thể cần nhiều index trên các phần khác nhau của dữ liệu.

Một index là một cấu trúc *bổ sung* được phái sinh từ dữ liệu chính. Nhiều cơ sở dữ liệu cho phép bạn thêm và xóa các index, và việc này không ảnh hưởng đến nội dung của cơ sở dữ liệu; nó chỉ ảnh hưởng đến hiệu năng của các truy vấn. Việc duy trì các cấu trúc bổ sung sẽ gây ra overhead, đặc biệt là đối với các thao tác ghi. Đối với các thao tác ghi, rất khó để vượt qua hiệu năng của việc chỉ đơn giản là ghi thêm vào một tệp, vì đó là thao tác ghi đơn giản nhất có thể. Bất kỳ loại index nào cũng thường làm chậm các thao tác ghi, vì index cũng cần được cập nhật mỗi khi dữ liệu được ghi.

Đây là một đánh đổi (trade-off) quan trọng trong các hệ thống lưu trữ: các index được lựa chọn kỹ lưỡng sẽ tăng tốc các truy vấn đọc, nhưng mỗi index đều tiêu tốn thêm không gian đĩa và làm chậm các thao tác ghi, đôi khi là đáng kể [1]. Vì lý do này, các cơ sở dữ liệu thường không index mọi thứ theo mặc định, mà yêu cầu bạn—người viết ứng dụng hoặc quản trị cơ sở dữ liệu—phải tự chọn các index một cách thủ công, bằng cách sử dụng kiến thức của bạn về các mẫu truy vấn điển hình của ứng dụng. Sau đó, bạn có thể chọn các index mang lại lợi ích lớn nhất cho ứng dụng của mình mà không gây ra thêm overhead cho các thao tác ghi quá mức cần thiết.

Lưu trữ cấu trúc Log (Log-Structured Storage)

Để bắt đầu, hãy giả sử rằng bạn muốn tiếp tục lưu trữ dữ liệu trong tệp append-only được ghi bởi `db_set`, và bạn chỉ muốn tăng tốc các thao tác đọc. Một cách bạn có thể thực hiện việc này là duy trì một hash map trong bộ nhớ, ánh xạ mọi key tới byte offset nơi có thể tìm thấy giá trị gần nhất của key đó, như được minh họa trong Hình 4-1.

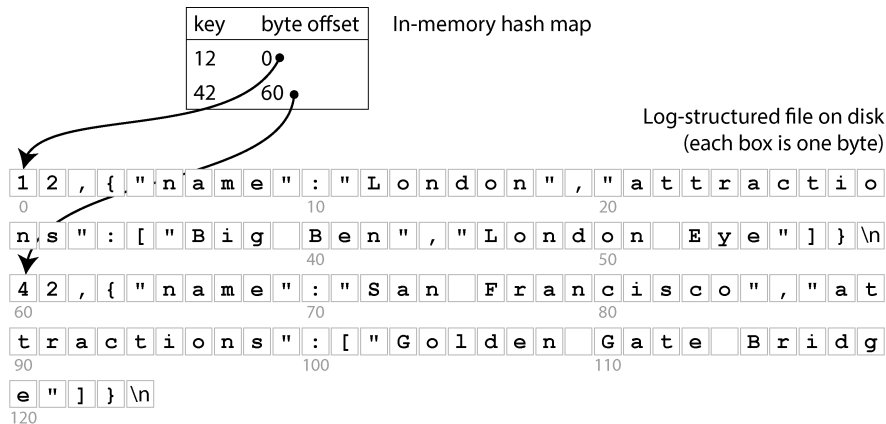


Figure 4-1. Lưu trữ một log gồm các cặp key-value dưới định dạng giống CSV, được đánh chỉ mục bằng một in-memory hash map

Bất cứ khi nào bạn thêm một cặp key-value mới vào tệp, bạn cũng cập nhật hash map để phản ánh offset của dữ liệu mà bạn vừa ghi. Khi bạn muốn tra cứu một giá trị, bạn sử dụng hash map để tìm offset trong log file, seek đến vị trí đó và đọc giá trị. Nếu phần dữ liệu đó của tệp dữ liệu đã nằm trong filesystem cache, một thao tác đọc sẽ không yêu cầu bất kỳ disk I/O nào cả.

Cách tiếp cận này nhanh hơn nhiều, nhưng nó vẫn gặp phải một vài vấn đề:

- Bạn không bao giờ giải phóng được không gian đĩa bị chiếm dụng bởi các log entry cũ đã bị ghi đè; nếu bạn tiếp tục ghi vào database, bạn có thể bị hết dung lượng đĩa.
- Hash map không được persisted, vì vậy bạn phải rebuild lại nó khi khởi động lại database—ví dụ, bằng cách quét toàn bộ log file để tìm byte offset mới nhất cho mỗi key. Điều này khiến việc khởi động lại trở nên chậm chạp nếu bạn có nhiều dữ liệu.
- Hash table phải nằm gọn trong memory. Về nguyên tắc, bạn có thể duy trì một hash table trên đĩa, nhưng đáng tiếc là rất khó để làm cho một hash map trên đĩa hoạt động hiệu quả. Nó đòi hỏi rất nhiều random access I/O, việc mở rộng khi nó bị đầy sẽ rất tốn kém, và các hash collision đòi hỏi logic phức tạp [2].

- Các truy vấn theo phạm vi (range queries) không hiệu quả. Ví dụ, bạn không thể dễ dàng quét qua tất cả các key từ 10000 đến 19999—bạn phải tra cứu từng key một trong hash map.

Định dạng tệp SSTable

Trong thực tế, hash table không được sử dụng quá thường xuyên cho các database index. Thay vào đó, việc giữ dữ liệu trong một cấu trúc được *sắp xếp theo key* [3] phổ biến hơn nhiều. Một ví dụ về cấu trúc như vậy là *Sorted Strings Table*, gọi tắt là *SSTable*, như được hiển thị trong Hình 4-2. Định dạng tệp này cũng lưu trữ các cặp key-value, nhưng nó đảm bảo rằng chúng được sắp xếp theo key, và mỗi key chỉ xuất hiện duy nhất một lần trong tệp.

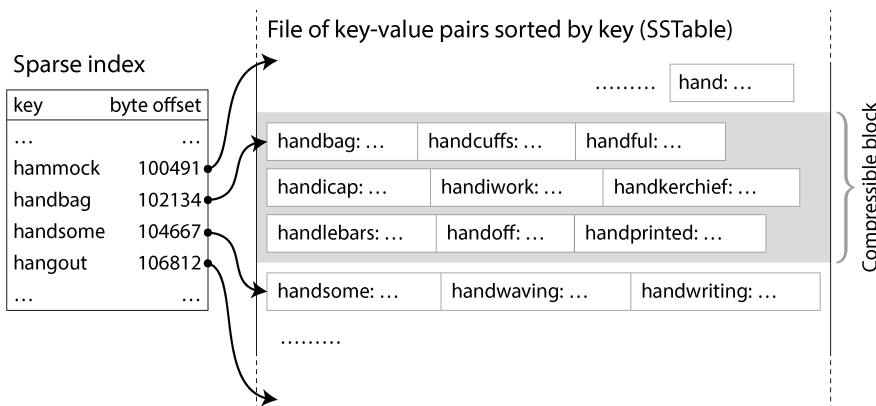


Figure 4-2. Một SSTable với một sparse index (chỉ mục thưa), cho phép các truy vấn nhảy thẳng đến đúng block

Bây giờ, bạn không cần phải giữ tất cả các key trong bộ nhớ. Bạn có thể nhóm các cặp key-value trong một SSTable thành các *block* (khối) vài kilobyte và sau đó lưu trữ key đầu tiên của mỗi block vào index. Loại index này, vốn chỉ lưu trữ một số key nhất định, được gọi là *sparse* (thưa). Index này được lưu trữ trong một phần riêng biệt của SSTable—ví dụ, bằng cách sử dụng một B-tree bất biến, một trie, hoặc một cấu trúc dữ liệu khác cho phép các truy vấn tìm kiếm nhanh một key cụ thể [4].

Ví dụ, trong Hình 4-2, key đầu tiên của một block là handbag, và key đầu tiên của block tiếp theo là handsome. Giả sử bạn đang tìm kiếm key handiwork, key này không xuất hiện trong sparse index. Nhờ vào việc đã được sắp xếp, bạn biết rằng handiwork chắc chắn phải xuất hiện giữa handbag và handsome. Điều này có nghĩa là bạn có thể tìm đến offset của handbag và quét file từ đó cho đến khi tìm thấy handiwork (hoặc không, nếu key không có mặt trong file). Một block vài kilobyte có thể được quét rất nhanh chóng.

Mỗi block chứa các bản ghi cũng có thể được nén (được biểu thị bởi vùng tô bóng trong Hình 4-2). Bên cạnh việc tiết kiệm không gian đĩa, việc nén còn giúp giảm mức sử dụng băng thông I/O, với cái giá là sử dụng thêm một chút thời gian CPU.

Xây dựng và hợp nhất các SSTable

Định dạng file SSTable tốt hơn cho việc đọc so với một append-only log, nhưng nó khiến việc ghi trở nên khó khăn hơn. Chúng ta không thể chỉ đơn giản là append vào cuối, vì khi đó file sẽ không còn được sắp xếp nữa (trừ khi các key tình cờ được ghi theo thứ tự tăng dần). Nếu chúng ta phải ghi lại toàn bộ SSTable mỗi khi một key được chèn vào đầu đó ở giữa, việc ghi sẽ trở nên quá tốn kém.

Chúng ta có thể giải quyết vấn đề này bằng cách tiếp cận *log-structured* (cấu trúc log), một sự kết hợp giữa append-only log và một file đã được sắp xếp:

1. Khi một lệnh ghi đến, hãy thêm nó vào một cấu trúc dữ liệu *ordered map* trong bộ nhớ, chẳng hạn như *red-black tree*, *skip list* [5], hoặc *trie* [6]. Với các cấu trúc dữ liệu này, bạn có thể chèn các key theo bất kỳ thứ tự nào, tìm kiếm chúng một cách hiệu quả và đọc lại chúng theo thứ tự đã sắp xếp. Cấu trúc dữ liệu trong bộ nhớ này được gọi là *memtable*.
2. Khi *memtable* trở nên lớn hơn một ngưỡng nhất định—thường là vài megabyte—hãy ghi nó ra đĩa theo thứ tự đã sắp xếp dưới dạng một file SSTable. Chúng ta gọi file SSTable mới này là *segment* (phân đoạn) mới nhất của cơ sở dữ liệu, và nó được lưu trữ dưới dạng một file riêng biệt bên cạnh các *segment* cũ hơn. Mỗi *segment* có một index riêng cho nội dung của nó. Trong khi *segment* mới đang được ghi ra đĩa, cơ sở dữ liệu có thể tiếp tục ghi vào một instance *memtable* mới, và bộ nhớ của *memtable* cũ sẽ được giải phóng khi việc ghi SSTable hoàn tất.
3. Để đọc giá trị của một key, trước tiên hãy thử tìm key đó trong *memtable* và *segment* trên đĩa mới nhất. Nếu không thấy, hãy tiếp tục tìm trong *segment* cũ hơn tiếp theo cho đến khi bạn tìm thấy key hoặc chạm đến *segment* cũ nhất. Nếu key không xuất hiện trong bất kỳ *segment* nào, nó không tồn tại trong cơ sở dữ liệu.
4. Thỉnh thoảng, hãy chạy một quá trình *merging* và *compaction* ở chế độ *background* để kết hợp các file *segment* và loại bỏ các giá trị đã bị ghi đè hoặc đã bị xóa.

Việc *merging* các *segment* hoạt động tương tự như thuật toán *mergesort* [5]. Quá trình này được minh họa trong Hình 4-3: bắt đầu đọc các file đầu vào song song, xem xét key đầu tiên trong mỗi file, sao chép key thấp nhất (theo thứ tự sắp xếp) vào file đầu ra, và lặp lại. Nếu cùng một key xuất hiện trong nhiều hơn một file đầu vào, chỉ giữ lại giá trị mới nhất. Việc này tạo ra một file *segment* đã được hợp nhất mới, cũng được sắp xếp theo key, với một giá trị cho mỗi key, và nó sử dụng bộ nhớ tối thiểu vì chúng ta có thể lặp qua các SSTable từng key một.

Để đảm bảo rằng dữ liệu trong memtable không bị mất nếu cơ sở dữ liệu bị crash, storage engine sẽ giữ một log riêng trên đĩa mà mọi lệnh ghi đều được append ngay lập tức vào đó. Log này không được sắp xếp theo key, nhưng điều đó không quan trọng, vì mục đích duy nhất của nó là khôi phục memtable sau khi crash. Mỗi khi memtable được ghi ra một SSTable, phần tương ứng của log có thể được loại bỏ.



Figure 4-3. Trộn nhiều phân đoạn SSTable, chỉ giữ lại giá trị mới nhất cho mỗi key

Nếu bạn muốn xóa một key và giá trị đi kèm của nó, bạn phải chèn thêm một bản ghi xóa đặc biệt gọi là *tombstone* vào tập dữ liệu. Khi các log segment được merge, tombstone sẽ báo cho quá trình merge biết rằng cần loại bỏ bất kỳ giá trị nào trước đó của key đã bị xóa. Một khi tombstone đã được merge vào segment cũ nhất, nó có thể được loại bỏ.

Thuật toán được mô tả ở đây về cơ bản là những gì được sử dụng trong RocksDB [7], Cassandra, ScyllaDB, và HBase [8], tất cả đều được truyền cảm hứng từ bài báo Bigtable của Google [9] (nơi giới thiệu các thuật ngữ *SSTable* và *memtable*). Thuật toán này ban đầu được công bố vào năm 1996 với tên gọi *Log-Structured Merge-tree*, hay *LSM-tree* [10], dựa trên các nghiên cứu trước đó về log-structured filesystems [11]. Vì lý do này, các storage engine dựa trên nguyên tắc merge và compaction các tệp đã được sắp xếp thường được gọi là *LSM storage engines*.

Trong các LSM storage engine, một segment file được ghi trong một lượt (bằng cách ghi memtable ra hoặc bằng cách merge một số segment hiện có), và sau đó nó là immutable. Việc merge và compaction các segment có thể được thực hiện trong một background thread. Trong khi quá trình merge đang diễn ra, chúng ta vẫn có thể tiếp

tục phục vụ các yêu cầu read bằng cách sử dụng các input segment của quá trình merge (như trước đây, các yêu cầu read sẽ tìm trong memtable và các segment file gần đây nhất trước). Khi quá trình merging hoàn tất, chúng ta chuyển các yêu cầu read sang sử dụng segment mới đã được merge thay vì các input segment, và sau đó các input segment file có thể được xóa.

Các segment file không nhất thiết phải được lưu trữ trên đĩa cục bộ; chúng cũng rất phù hợp để ghi vào object storage. Ví dụ, SlateDB và Delta Lake [12] áp dụng cách tiếp cận này.

Việc có các segment file immutable cũng giúp đơn giản hóa crash recovery. Nếu một sự cố crash xảy ra trong khi đang ghi memtable hoặc trong khi đang merge các segment, cơ sở dữ liệu chỉ cần xóa SSTable chứa hoàn tất và bắt đầu lại từ đầu. Log lưu trữ các lần ghi vào memtable có thể chứa các bản ghi không hoàn chỉnh nếu có sự cố crash xảy ra giữa chừng khi đang ghi một bản ghi, hoặc nếu đĩa bị đầy; những vấn đề này thường được phát hiện bằng cách đưa các checksum vào log và loại bỏ các entry log bị lỗi hoặc không hoàn chỉnh. Chúng ta sẽ thảo luận thêm về durability và crash recovery trong Chương 8.

Bloom filters

Với LSM storage, việc đọc một key đã được cập nhật lần cuối từ rất lâu, hoặc cố gắng đọc một key không tồn tại, có thể sẽ chậm vì storage engine sẽ cần kiểm tra nhiều segment file. Để tăng tốc các yêu cầu read như vậy, các LSM storage engine thường bao gồm một *Bloom filter* [13] trong mỗi segment, cung cấp một cách nhanh chóng nhưng mang tính xấp xỉ để kiểm tra xem một key cụ thể có xuất hiện trong một SSTable cụ thể hay không.

Hình 4-4 cho thấy một ví dụ về Bloom filter chứa hai key và 16 bit (trong thực tế, nó sẽ chứa nhiều key và nhiều bit hơn). Đối với mỗi key trong SSTable, ta tính toán một hàm hash, tạo ra một tập hợp các số sau đó được diễn giải thành các index trong mảng bit [14]. Ta đặt các bit tương ứng với các index đó thành 1 và để phần còn lại là 0. Ví dụ, key *handbag* hash thành số (2, 9, 4), vì vậy ta đặt bit thứ hai, thứ chín và thứ tư thành 1. Bitmap sau đó được lưu trữ như một phần của SSTable, cùng với sparse index của các key. Việc này tốn thêm một chút không gian, nhưng Bloom filter thường rất nhỏ so với phần còn lại của SSTable.

Khi chúng ta muốn biết liệu một key có xuất hiện trong SSTable hay không, chúng ta tính toán cùng một mã hash của key đó như trước đây và kiểm tra các bit tại các index đó. Ví dụ, trong Hình 4-4, chúng ta đang truy vấn key *handheld*, mã hash của nó là (6, 11, 2). Một trong số các bit đó là 1 (cụ thể là bit số 2), trong khi hai bit còn lại là 0. Những kiểm tra này có thể được thực hiện cực kỳ nhanh chóng bằng cách sử dụng các phép toán bitwise mà tất cả CPU đều hỗ trợ.

Keys that exist in the SSTable:

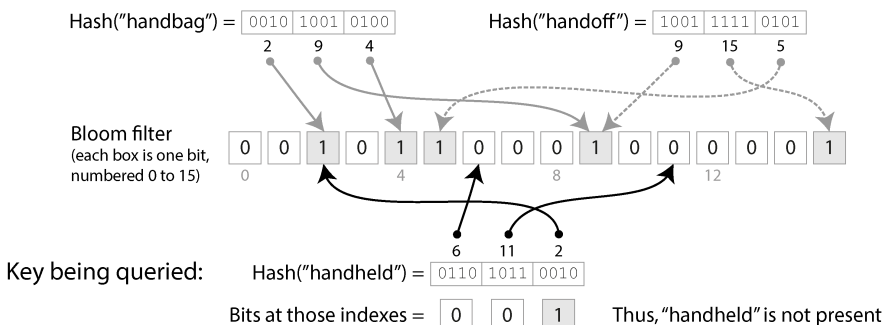


Figure 4-4. Một Bloom filter cung cấp khả năng kiểm tra xác suất nhanh chóng xem một key cụ thể có tồn tại trong một SSTable cụ thể hay không.

Nếu có ít nhất một bit bằng 0, chúng ta biết chắc chắn rằng key đó không xuất hiện trong SSTable. Nếu tất cả các bit trong truy vấn đều là 1, key đó có khả năng nằm trong SSTable, nhưng cũng có thể do trùng hợp mà tất cả các bit đó đều được đặt thành 1 bởi các key khác. Trường hợp này, khi có vẻ như một key đang hiện diện mặc dù thực tế không phải vậy, được gọi là *false positive* (dương tính giả).

Xác suất xảy ra false positive phụ thuộc vào số lượng key, số lượng bit được đặt cho mỗi key, và tổng số bit trong Bloom filter. Bạn có thể sử dụng một công cụ tính toán trực tuyến để tìm ra các tham số phù hợp cho ứng dụng của mình [15]. Theo quy tắc kinh nghiệm, bạn cần phân bổ 10 bit không gian Bloom filter cho mỗi key trong SSTable để đạt được xác suất false positive là 1%, và xác suất này sẽ giảm đi mười lần cho mỗi 5 bit bổ sung mà bạn phân bổ cho mỗi key.

Trong bối cảnh các LSM storage engine, false positive không phải là vấn đề:

- Nếu Bloom filter nói rằng một key *không* hiện diện, chúng ta có thể an tâm bỏ qua SSTable đó, vì chúng ta có thể chắc chắn rằng nó không chứa key này.
- Nếu Bloom filter nói rằng key *có* hiện diện, chúng ta phải tham chiếu đến sparse index và giải mã block chứa các cặp key-value để kiểm tra xem key đó có thực sự ở đó hay không. Nếu đó là một false positive, chúng ta chỉ tốn thêm một chút công sức không cần thiết, nhưng ngoài ra không gây hại gì—chúng ta chỉ cần tiếp tục tìm kiếm với segment cũ kế tiếp.

Các chiến lược compaction

Một chi tiết quan trọng là cách LSM storage chọn thời điểm thực hiện compaction và chọn những SSTable nào để đưa vào một quá trình compaction. Nhiều hệ thống lưu trữ dựa trên LSM cho phép bạn cấu hình chiến lược compaction nào sẽ được sử dụng. Một số lựa chọn phổ biến như sau [16, 17]:

Size-tiered compaction

Các SSTable mới hơn và nhỏ hơn sẽ lần lượt được hợp nhất vào các SSTable cũ hơn và lớn hơn. Ví dụ, bốn SSTable 256 MB có thể được compacted thành một SSTable 898 MB (kết quả không phải là 1,024 MB do các thao tác xóa, ghi đè, hết hạn time-to-live, v.v.). Các SSTable chứa dữ liệu cũ có thể trở nên rất lớn, và việc hợp nhất chúng đòi hỏi rất nhiều không gian đĩa tạm thời. Ưu điểm của chiến lược này là nó có thể xử lý throughput ghi rất cao vì hầu hết dữ liệu chỉ được ghi lại một vài lần trong các đợt hợp nhất tuần tự lớn hơn.

Leveled compaction

Thay vì ghi các SSTable lớn, leveled compaction giữ kích thước SSTable cố định và nhóm chúng thành các "level" (tầng) tăng dần (được gọi là L0, L1, v.v.). L0 chứa dữ liệu được ghi gần đây nhất. Tất cả các level sau L0 đều chứa các SSTable được phân mảnh theo phạm vi key (key range-partitioned). Ví dụ, L1 có thể có hai SSTable: cái đầu tiên với các key a–m và cái thứ hai với n–z. Mỗi level có giới hạn kích thước riêng, và mỗi level đều lớn hơn level đứng trước nó (ví dụ, L2 sẽ lớn hơn L1). Khi các SSTable của một level kết hợp lại vượt quá giới hạn kích thước tối đa, một hoặc nhiều SSTable từ level i sẽ được hợp nhất vào level $i + 1$ và bị xóa khỏi level i . Cách tiếp cận này cho phép compaction diễn ra dần dần hơn và sử dụng ít không gian đĩa hơn so với chiến lược size-tiered. Leveled compaction hiệu quả hơn cho các thao tác đọc so với size-tiered compaction vì storage engine cần đọc ít SSTable hơn để kiểm tra xem chúng có chứa key hay không.

Theo quy tắc kinh nghiệm, size-tiered compaction hoạt động tốt hơn nếu bạn có chủ yếu là các thao tác ghi và ít thao tác đọc, trong khi leveled compaction hoạt động tốt hơn nếu khối lượng công việc của bạn bị chi phối bởi các thao tác đọc. Nếu bạn ghi một số lượng nhỏ các key một cách thường xuyên và một số lượng lớn các key một cách hiếm hoi, thì leveled compaction cũng có thể mang lại lợi thế [18]. May mắn thay, hầu hết các triển khai LSM-tree đều cung cấp nhiều chiến lược compaction khác nhau cho các khối lượng công việc khác nhau.

Mặc dù có nhiều điểm tinh tế, nhưng ý tưởng cơ bản của LSM-trees—duy trì một chuỗi các SSTable được hợp nhất trong nền—là đơn giản và hiệu quả. Chúng ta sẽ thảo luận chi tiết hơn về các đặc tính hiệu năng của chúng trong mục “Comparing B-Trees and LSM-Trees”.

Các Embedded Storage Engines

Nhiều cơ sở dữ liệu chạy như một service chấp nhận các truy vấn qua mạng, nhưng cũng có những cơ sở dữ liệu *embedded* (nhúng) không cung cấp một network API. Thay vào đó, chúng là các thư viện chạy trong cùng một process với mã nguồn ứng dụng của bạn, thường là đọc và ghi các tệp trên đĩa cục bộ, và bạn tương tác với chúng thông qua các lời gọi hàm thông thường. Các ví dụ về các storage engine dạng embedded bao gồm RocksDB, SQLite, LMDB, DuckDB và KùzuDB [19].

Các cơ sở dữ liệu embedded được sử dụng rất phổ biến trong các ứng dụng di động để lưu trữ dữ liệu của người dùng cục bộ. Ở phía backend, chúng có thể là một lựa chọn phù hợp nếu dữ liệu đủ nhỏ để vừa với một máy đơn lẻ và nếu không có quá nhiều transaction đồng thời. Ví dụ, trong một hệ thống multitenant mà mỗi tenant đủ nhỏ và hoàn toàn tách biệt với những tenant khác (tức là, bạn không cần chạy các truy vấn kết hợp dữ liệu từ nhiều tenant), bạn có khả năng có thể sử dụng một instance cơ sở dữ liệu embedded riêng biệt cho mỗi tenant [20].

Các phương pháp lưu trữ và truy hồi mà chúng ta thảo luận trong chương này được sử dụng trong cả cơ sở dữ liệu embedded và cơ sở dữ liệu client/server. Trong Chương 6 và 7, chúng ta sẽ thảo luận về các kỹ thuật để scaling một cơ sở dữ liệu trên nhiều máy.

B-Trees

Cách tiếp cận log-structured rất phổ biến, nhưng nó không phải là hình thức lưu trữ key-value duy nhất. Cấu trúc được sử dụng rộng rãi nhất để đọc và ghi các bản ghi cơ sở dữ liệu theo key là *B-tree*.

Được giới thiệu vào năm 1970 [21] và được gọi là "ubiquitous" (phổ biến khắp nơi) chưa đầy 10 năm sau đó [22], B-trees đã vượt qua thử thách của thời gian một cách rất tốt. Chúng vẫn là implementation index tiêu chuẩn trong hầu hết tất cả các cơ sở dữ liệu relational, và nhiều cơ sở dữ liệu nonrelational cũng sử dụng chúng.

Giống như SSTables, B-trees giữ các cặp key-value được sắp xếp theo key, điều này cho phép thực hiện các truy vấn range và lookup key-value một cách hiệu quả. Nhưng sự tương đồng chỉ dừng lại ở đó; B-trees có một triết lý thiết kế rất khác biệt.

Các index log-structured mà chúng ta đã thấy trước đó chia cơ sở dữ liệu thành các *segments* có kích thước thay đổi, thường có kích thước vài megabyte hoặc lớn hơn, được ghi một lần và sau đó là immutable. Ngược lại, B-trees chia cơ sở dữ liệu thành các *blocks* hoặc *pages* có kích thước cố định và có thể ghi đè một page tại chỗ. Một

page theo truyền thống có kích thước 4 KiB, nhưng PostgreSQL hiện sử dụng 8 KiB và MySQL sử dụng 16 KiB theo mặc định.

Mỗi page có thể được xác định bằng cách sử dụng một số trang (page number), cho phép một page tham chiếu đến một page khác—tương tự như một pointer, nhưng nằm trên đĩa thay vì trong memory. Nếu tất cả các page được lưu trữ trong cùng một tệp, việc nhân số trang với kích thước trang sẽ cho chúng ta byte offset trong tệp nơi page đó nằm. Chúng ta có thể sử dụng các tham chiếu page này để xây dựng một cây các page, như được minh họa trong Hình 4-5.

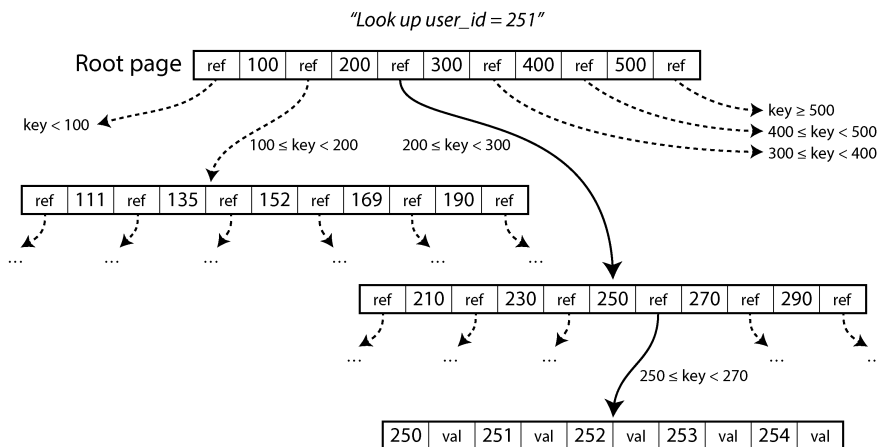


Figure 4-5. Tra cứu key 251 bằng cách sử dụng B-tree index. Từ root page, trước tiên chúng ta đi theo tham chiếu tới page chứa các key từ 200–300, sau đó tới page chứa các key từ 250–270.

Một trang được chỉ định làm *root* (gốc) của B-tree; bất cứ khi nào bạn muốn tra cứu một key trong index, bạn sẽ bắt đầu tại đây. Trang này chứa một vài key và các tham chiếu đến các trang con. Mỗi trang con chịu trách nhiệm cho một phạm vi key liên tục, và các key nằm giữa các tham chiếu sẽ cho biết ranh giới giữa các phạm vi đó nằm ở đâu. (Cấu trúc này đôi khi được gọi là B+ tree, nhưng chúng ta không cần phải phân biệt nó với các biến thể B-tree khác.)

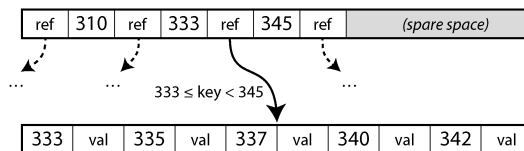
Trong ví dụ ở Hình 4-5, chúng ta đang tìm kiếm key 251, vì vậy chúng ta biết rằng mình cần đi theo tham chiếu trang nằm giữa các ranh giới 200 và 300. Điều đó dẫn chúng ta đến một trang có hình dạng tương tự, nơi chia nhỏ phạm vi 200–300 thành các phạm vi con. Cuối cùng, chúng ta sẽ đi xuống đến một trang chứa các key riêng lẻ (một *leaf page*), trang này sẽ chứa giá trị của mỗi key trực tiếp (inline) hoặc chứa các tham chiếu đến các trang nơi có thể tìm thấy các giá trị đó.

Số lượng tham chiếu đến các trang con trong một trang của B-tree được gọi là *branching factor* (hệ số nhánh). Ví dụ, trong Hình 4-5, *branching factor* là sáu. Trong

thực tế, *branching factor* phụ thuộc vào lượng không gian cần thiết để lưu trữ các tham chiếu trang và các ranh giới phạm vi, nhưng thông thường nó lên đến vài trăm.

Nếu bạn muốn cập nhật giá trị cho một key đã tồn tại trong B-tree, bạn tìm kiếm *leaf page* chứa key đó và ghi đè trang đó trên đĩa bằng một phiên bản chứa giá trị mới. Nếu bạn muốn thêm một key mới, bạn cần tìm trang có phạm vi bao hàm key mới đó và thêm nó vào trang đó. Nếu không có đủ không gian trống trong trang để chứa key mới, trang đó sẽ được chia thành hai trang chứa đầy một nửa, và trang cha sẽ được cập nhật để ghi nhận việc chia nhỏ mới của các phạm vi key, như được hiển thị trong Hình 4-6.

Before:



After adding key 334:

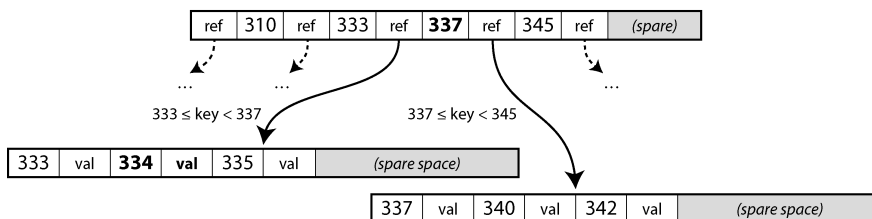


Figure 4-6. **Hình mở rộng một B-tree bằng cách chia tách một page tại boundary key 337. Page cha được cập nhật để tham chiếu đến cả hai child.**

Trong ví dụ này, chúng ta muốn chèn key 334, nhưng trang cho phạm vi 333–345 đã đầy. Do đó, chúng ta chia nó thành một trang cho phạm vi 333–337 (bao gồm cả key mới, 334) và một trang cho 337–345. Chúng ta cũng phải cập nhật trang cha để có các tham chiếu đến cả hai trang con, với một giá trị ranh giới là 337 ở giữa chúng. Nếu trang cha không có đủ không gian cho tham chiếu mới, nó cũng có thể cần được chia tách, và việc chia tách có thể tiếp tục cho đến tận gốc của cây. Khi gốc bị chia tách, chúng ta tạo một gốc mới nằm phía trên nó. Việc xóa các key (có thể yêu cầu các node phải được hợp nhất) thì phức tạp hơn [5].

Thuật toán này đảm bảo rằng cây luôn được *balanced* (cân bằng): một B-tree với n key luôn có độ sâu là $O(\log n)$. Hầu hết các cơ sở dữ liệu có thể nằm gọn trong một B-tree sâu ba hoặc bốn tầng, vì vậy bạn không cần phải đi theo quá nhiều tham chiếu trang để tìm trang mà mình đang tìm kiếm. (Một cây bốn tầng với các trang 4 KiB và hệ số nhánh là 500 có thể lưu trữ tới 250 TB.)

Làm cho B-tree trở nên tin cậy

Thao tác ghi cơ bản bên dưới của một B-tree là ghi đè một trang trên đĩa bằng dữ liệu mới. Giả định rằng việc ghi đè không làm thay đổi vị trí của trang; tất cả các tham chiếu đến trang đó vẫn được giữ nguyên khi trang bị ghi đè. Điều này hoàn toàn trái ngược với các index cấu trúc dạng log như LSM-tree, vốn chỉ thực hiện append vào các tệp (và cuối cùng là xóa các tệp lỗi thời) chứ không bao giờ sửa đổi trực tiếp các tệp tại chỗ.

Ghi đè nhiều trang cùng một lúc, như trong trường hợp chia tách trang, là một thao tác nguy hiểm. Nếu cơ sở dữ liệu bị crash chỉ sau khi một số trang đã được ghi, bạn sẽ kết thúc với một cây bị lỗi (ví dụ, có thể có một trang *orphan* (mồ côi) không phải là con của bất kỳ trang cha nào). Nếu phần cứng không thể ghi nguyên tử (atomically) toàn bộ một trang, bạn cũng có thể gặp tình trạng một trang chỉ được ghi một phần (điều này được gọi là *torn page* [23]).

Để giúp cơ sở dữ liệu có khả năng chống chịu với các lỗi crash, các triển khai B-tree thường bao gồm một cấu trúc dữ liệu bổ sung trên đĩa: một *write-ahead log* (WAL) (nhật ký ghi trước). Đây là một tệp chỉ cho phép append mà mọi sửa đổi B-tree đều phải được ghi vào đó trước khi có thể áp dụng vào chính các trang của cây. Khi cơ sở dữ liệu khởi động lại sau một lỗi crash, nhật ký này được sử dụng để khôi phục B-tree về trạng thái nhất quán [2, 24]. Trong các hệ thống tệp, cơ chế tương đương được gọi là *journaling*.

Để cải thiện hiệu năng, các triển khai B-tree thường không ghi ngay lập tức mọi trang đã sửa đổi xuống đĩa, mà đệm các trang B-tree trong bộ nhớ một thời gian. Sau đó, write-ahead log cũng đảm bảo rằng dữ liệu không bị mất trong trường hợp xảy ra crash. Miễn là dữ liệu đã được ghi vào WAL và được flush xuống đĩa bằng lời gọi hệ thống `fsync`, dữ liệu sẽ có tính bền vững (durable), vì cơ sở dữ liệu sẽ có thể khôi phục nó sau khi crash [25].

Sử dụng các biến thể B-tree

Vì B-tree đã tồn tại từ rất lâu, nhiều biến thể đã được phát triển qua nhiều năm. Chỉ để kể ra một vài ví dụ:

- Thay vì ghi đè các trang và duy trì một WAL để khôi phục sau crash, một số cơ sở dữ liệu (như LMDB) sử dụng cơ chế copy-on-write [26]. Một trang đã sửa đổi sẽ được ghi vào một vị trí khác, và một phiên bản mới của các trang cha trong cây được tạo ra, trỏ đến vị trí mới đó. Cách tiếp cận này cũng hữu ích cho việc kiểm soát đồng thời (concurrency control), như chúng ta sẽ thấy trong mục “Snapshot Isolation và Repeatable Read”.
- Chúng ta có thể tiết kiệm không gian trong các trang bằng cách không lưu trữ toàn bộ key mà chỉ viết tắt nó. Đặc biệt là trong các trang nằm ở phần bên trong của cây, các key chỉ cần cung cấp đủ thông tin để đóng vai trò là ranh giới giữa các

phạm vi key. Việc đóng gói nhiều key hơn vào một trang cho phép cây có hệ số nhánh cao hơn và do đó có ít tầng hơn.

- Để tăng tốc các thao tác quét qua phạm vi key theo thứ tự đã sắp xếp, một số triển khai B-tree cố gắng bố trí cây sao cho các trang lá xuất hiện theo thứ tự tuần tự trên đĩa, giúp giảm số lượng các lần tìm kiếm đĩa (disk seeks). Tuy nhiên, việc duy trì thứ tự đó là rất khó khi cây lớn dần lên.
- Các con trỏ bổ sung đã được thêm vào cây. Ví dụ, mỗi trang lá có thể có các tham chiếu đến các trang anh em ở bên trái và bên phải của nó, cho phép quét các key theo thứ tự mà không cần nhảy ngược lại các trang cha.

So sánh B-Trees và LSM-Trees

Theo quy tắc kinh nghiệm, LSM-trees phù hợp hơn cho các ứng dụng có khối lượng ghi lớn (write-heavy), trong khi B-trees nhanh hơn cho các thao tác đọc [27, 28]. Tuy nhiên, các benchmark thường rất nhạy cảm với các chi tiết của khối lượng công việc (workload). Bạn cần kiểm thử các hệ thống với workload cụ thể của mình để có một sự so sánh hợp lệ. Hơn nữa, đây không phải là sự lựa chọn loại trừ lẫn nhau giữa LSM- và B-trees; các storage engine đôi khi kết hợp các đặc tính của cả hai cách tiếp cận—ví dụ, bằng cách có nhiều B-trees và hợp nhất chúng theo kiểu LSM. Trong mục này, chúng ta sẽ thảo luận ngắn gọn về một vài điều đáng cân nhắc khi đo lường hiệu năng của một storage engine.

Hiệu năng đọc

Trong một B-tree, việc tra cứu một key bao gồm việc đọc một page tại mỗi tầng. Vì số lượng các tầng thường khá nhỏ, các thao tác đọc từ một B-tree nhìn chung nhanh và có hiệu năng có thể dự đoán được. Trong một LSM storage engine, các thao tác đọc thường phải kiểm tra nhiều SSTable ở các giai đoạn compaction khác nhau, nhưng Bloom filter giúp giảm số lượng các thao tác disk I/O cần thiết. Cả hai cách tiếp cận đều có thể hoạt động tốt, và việc cách nào nhanh hơn phụ thuộc vào chi tiết của storage engine và workload.

Các truy vấn phạm vi (range queries) diễn ra đơn giản và nhanh chóng trên B-trees, vì chúng có thể tận dụng cấu trúc đã được sắp xếp của cây. Trên lưu trữ LSM, các range queries cũng có thể tận dụng việc sắp xếp của SSTable, nhưng chúng cần phải quét tất cả các segment song song và kết hợp các kết quả lại. Bloom filter không giúp ích cho các range queries (vì bạn sẽ cần phải tính toán hash của mọi key khả thi trong phạm vi đó, điều này là không thực tế), khiến các range queries trở nên tốn kém hơn so với các truy vấn điểm (point queries) trong cách tiếp cận LSM [29].

Throughput ghi cao có thể gây ra các đợt tăng vọt độ trễ (latency spikes) trong một LSM storage engine nếu memtable bị đầy. Điều này xảy ra nếu dữ liệu

không thể được ghi ra đĩa đủ nhanh, có lẽ vì quá trình *compaction* không thể theo kịp các thao tác ghi đang đến. Nhiều *storage engine*, bao gồm cả RocksDB, áp dụng *backpressure* (áp lực ngược) trong tình huống này: chúng tạm dừng tất cả các thao tác đọc và ghi cho đến khi *memtable* đã được ghi ra đĩa [30, 31].

Về *throughput* đọc, các SSD hiện đại (đặc biệt là các SSD NVMe [Non-Volatile Memory Express] kết nối thông qua bus PCIe nhanh hơn nhiều so với bus SATA) có thể thực hiện nhiều yêu cầu đọc độc lập song song. Cả LSM-trees và B-trees đều có khả năng cung cấp *throughput* đọc cao, nhưng các *storage engine* cần được thiết kế cẩn thận để tận dụng tính song song này [32].

Ghi tuần tự so với ghi ngẫu nhiên

Với một B-tree, nếu ứng dụng ghi các *key* nằm rải rác khắp không gian *key*, các thao tác đĩa kết quả cũng sẽ bị rải rác ngẫu nhiên, vì các *page* mà *storage engine* cần ghi đề có thể nằm ở bất kỳ đâu trên đĩa. Mặt khác, một LSM *storage engine* sẽ ghi toàn bộ các *segment* file cùng một lúc (hoặc là ghi *memtable* ra, hoặc là trong khi đang *compaction* các *segment* hiện có), vốn lớn hơn nhiều so với một *page* trong B-tree.

Mô hình của nhiều thao tác ghi nhỏ, rải rác (như tìm thấy trong B-trees) được gọi là *ghi ngẫu nhiên* (*random writes*), trong khi mô hình của ít thao tác ghi lớn hơn (như tìm thấy trong LSM-trees) được gọi là *ghi tuần tự* (*sequential writes*). Các đĩa nói chung có *throughput* ghi tuần tự cao hơn *throughput* ghi ngẫu nhiên, điều này có nghĩa là một LSM *storage engine* thường có thể xử lý *throughput* ghi cao hơn trên cùng một phần cứng so với một B-tree. Sự khác biệt này đặc biệt lớn trên các ổ cứng đĩa quay; trên các SSD mà hầu hết các cơ sở dữ liệu sử dụng hiện nay, sự khác biệt nhỏ hơn nhưng vẫn có thể nhận thấy được.

Ghi tuần tự so với Ghi ngẫu nhiên trên SSD

Trên các ổ cứng đĩa quay (HDD), ghi tuần tự (sequential writes) nhanh hơn nhiều so với ghi ngẫu nhiên (random writes). Điều này là do một thao tác ghi ngẫu nhiên phải di chuyển đầu đọc đĩa một cách cơ học đến một vị trí mới và chờ phần phù hợp của phiên đĩa đi qua dưới đầu đọc, việc này mất vài mili giây—một khoảng thời gian dài vô tận trong thang đo thời gian của máy tính. Tuy nhiên, các SSD bao gồm cả NVMe (hoặc bộ nhớ flash gắn vào bus PCI Express) hiện đã vượt qua HDD trong nhiều trường hợp sử dụng, và chúng không phải chịu các giới hạn cơ học như vậy.

Tuy nhiên, SSD cũng có throughput cho ghi tuần tự cao hơn so với ghi ngẫu nhiên. Lý do là bộ nhớ flash có thể được đọc hoặc ghi từng page (thường là 4 KiB) mỗi lần, nhưng nó chỉ có thể xóa từng block (thường là 512 KiB) mỗi lần. Một số page trong một block có thể chứa dữ liệu hợp lệ, trong khi những page khác có thể chứa dữ liệu không còn cần thiết nữa. Trước khi xóa một block, bộ điều khiển phải di chuyển các page chứa dữ liệu hợp lệ vào các block khác; quá trình này được gọi là *garbage collection* (GC) (thu gom rác) [33].

Một khối lượng công việc ghi tuần tự sẽ ghi các chunk dữ liệu lớn hơn cùng một lúc, vì vậy có khả năng cả một block 512 KiB thuộc về một file duy nhất. Khi file đó sau này bị xóa, toàn bộ block có thể được xóa mà không cần phải thực hiện bất kỳ thao tác GC nào. Ngược lại, với khối lượng công việc ghi ngẫu nhiên, một block có nhiều khả năng chứa hỗn hợp các page với dữ liệu hợp lệ và không hợp lệ, do đó bộ thu gom rác phải thực hiện nhiều việc hơn trước khi một block có thể được xóa [34, 35, 36]. Bằng thông ghi bị tiêu tốn bởi GC sau đó sẽ không còn sẵn dụng cho ứng dụng. Các thao tác ghi bổ sung được thực hiện do GC cũng góp phần làm hao mòn bộ nhớ flash; do đó, ghi ngẫu nhiên làm mòn ổ đĩa nhanh hơn so với ghi tuần tự.

Write amplification

Với bất kỳ loại storage engine nào, một yêu cầu ghi từ ứng dụng sẽ chuyển thành nhiều thao tác I/O trên đĩa lưu trữ bên dưới. Với LSM-trees, một giá trị trước tiên được ghi vào log để đảm bảo tính durability (độ bền vững), sau đó lại được ghi khi memtable được ghi vào đĩa, và lại được ghi mỗi khi cặp key-value là một phần của một quá trình compaction. (Nếu các giá trị lớn hơn đáng kể so với các key, chi phí này có thể được giảm bớt bằng cách lưu trữ các giá trị tách biệt với các key và chỉ thực hiện compaction trên các SSTables chứa các key và tham chiếu đến các giá trị [37].)

Một index B-tree phải ghi mọi mảnh dữ liệu ít nhất hai lần: một lần vào write-ahead log, và một lần vào chính page của cây. Ngoài ra, việc ghi ra toàn bộ một page đôi khi

là cần thiết, ngay cả khi chỉ có vài byte trong page đó thay đổi, để đảm bảo rằng B-tree có thể được khôi phục chính xác sau khi xảy ra lỗi crash hoặc mất điện [38, 39].

Nếu bạn lấy tổng số byte được ghi vào đĩa trong một khối lượng công việc và chia số đó cho số byte mà bạn phải ghi nếu bạn chỉ đơn giản ghi một log theo kiểu append-only không có index, bạn sẽ có được *write amplification* (khuếch đại ghi). (Đôi khi write amplification được định nghĩa theo các thao tác I/O thay vì theo số byte.) Trong các ứng dụng ghi nặng, nút thắt cổ chai có thể là tốc độ mà cơ sở dữ liệu có thể ghi vào đĩa. Trong trường hợp này, write amplification càng cao, số lượng thao tác ghi mỗi giây mà nó có thể xử lý trong bảng thông đĩa hiện có càng ít.

Write amplification là một vấn đề trong cả LSM-trees và B-trees. Việc cái nào tốt hơn phụ thuộc vào nhiều yếu tố, chẳng hạn như độ dài của các key và value của bạn và tần suất bạn ghi đề các key hiện có so với việc chèn các key mới. Đối với các khối lượng công việc điển hình, LSM-trees có xu hướng có write amplification thấp hơn vì chúng không phải ghi toàn bộ các page và chúng có thể nén các chunk của SSTable [40]. Đây là một yếu tố khác khiến các storage engine kiểu LSM rất phù hợp cho các khối lượng công việc ghi nặng.

Bên cạnh việc ảnh hưởng đến throughput, write amplification cũng liên quan đến sự hao mòn trên SSD. Một storage engine có write amplification thấp hơn sẽ làm mòn SSD chậm hơn.

Khi đo lường throughput ghi của một storage engine, việc chạy thử nghiệm đủ lâu để các tác động của write amplification trở nên rõ ràng là rất quan trọng. Khi ghi vào một LSM-tree trống, chưa có quá trình compaction nào diễn ra, vì vậy toàn bộ bảng thông đĩa đều sẵn sàng cho các lượt ghi mới. Khi cơ sở dữ liệu lớn dần, các lượt ghi mới cần phải chia sẻ bảng thông đĩa với quá trình compaction.

Sử dụng không gian đĩa

B-trees có thể trở nên *bị phân mảnh (fragmented)* theo thời gian; ví dụ, nếu một lượng lớn các key bị xóa, tệp cơ sở dữ liệu có thể chứa nhiều page không còn được B-tree sử dụng nữa. Các lần thêm dữ liệu tiếp theo vào B-tree có thể sử dụng các page trống đó, nhưng chúng không dễ dàng được trả lại cho hệ điều hành vì chúng nằm ở giữa tệp, do đó chúng vẫn chiếm không gian trên filesystem. Vì vậy, các cơ sở dữ liệu cần một tiến trình chạy ngầm để di chuyển các page nhằm sắp xếp chúng tốt hơn, chẳng hạn như tiến trình vacuum trong PostgreSQL [25].

Sự phân mảnh ít là vấn đề trong LSM-trees, vì quá trình compaction dù sao cũng sẽ ghi lại các tệp dữ liệu theo định kỳ, và các SSTables không có các page chứa không gian không được sử dụng. Hơn nữa, các khối cặp key-value có thể được nén tốt hơn trong SSTables, thường dẫn đến các tệp trên đĩa nhỏ hơn so với B-trees. Các key và value đã bị ghi đè vẫn tiếp tục tiêu tốn không gian cho đến khi chúng bị loại bỏ bởi một quá trình compaction, nhưng chi phí này khá thấp khi sử dụng leveled

compaction [40, 41]. Size-tiered compaction (xem “Compaction strategies”) sử dụng nhiều không gian đĩa hơn, đặc biệt là tạm thời trong quá trình compaction.

Việc có nhiều bản sao của một số dữ liệu trên đĩa cũng có thể là một vấn đề khi bạn cần xóa một số dữ liệu và muốn chắc chắn rằng chúng thực sự đã được xóa (có lẽ để tuân thủ các quy định về bảo vệ dữ liệu). Ví dụ, trong hầu hết các LSM storage engine, một bản ghi đã bị xóa vẫn có thể tồn tại ở các tầng cao hơn cho đến khi tombstone đại diện cho việc xóa được lan truyền qua tất cả các tầng compaction, điều này có thể mất một thời gian dài. Các thiết kế storage engine chuyên dụng có thể lan truyền việc xóa nhanh hơn [42].

Mặt khác, tính chất bất biến (immutable) của các tệp phân đoạn SSTable rất hữu ích nếu bạn muốn chụp một snapshot của cơ sở dữ liệu tại một thời điểm nào đó (ví dụ, để sao lưu hoặc để tạo một bản sao của cơ sở dữ liệu để kiểm thử). Bạn có thể ghi memtable ra và ghi lại các tệp phân đoạn nào đã tồn tại tại thời điểm đó. Miễn là bạn không xóa các tệp là một phần của snapshot, bạn không cần phải thực sự sao chép chúng. Trong một B-tree mà các page bị ghi đè, việc chụp một snapshot như vậy một cách hiệu quả sẽ khó khăn hơn.

Index đa cột và Secondary Indexes

Cho đến nay, chúng ta mới chỉ thảo luận về các key-value indexes, thứ tương tự như các *primary-key indexes* trong mô hình quan hệ. Một primary key định danh duy nhất một hàng trong một bảng quan hệ, hoặc một document trong một document database, hoặc một vertex trong một graph database. Các bản ghi khác trong cơ sở dữ liệu có thể tham chiếu đến hàng/document/vertex đó bằng primary key (hoặc ID) của nó, và index được sử dụng để giải quyết các tham chiếu như vậy.

Việc có các *secondary indexes* cũng rất phổ biến. Trong các cơ sở dữ liệu quan hệ, bạn có thể tạo nhiều secondary indexes trên cùng một bảng bằng cách sử dụng lệnh `CREATE INDEX`, cho phép bạn tìm kiếm theo các cột khác ngoài primary key. Ví dụ, trong schema quan hệ được hiển thị ở Hình 3-1, bạn rất có thể sẽ có một secondary index trên các cột `user_id` để bạn có thể tìm thấy tất cả các hàng thuộc về cùng một người dùng trong mỗi bảng.

Một secondary index có thể dễ dàng được xây dựng từ một key-value index. Sự khác biệt chính là trong một secondary index, các giá trị được index không nhất thiết phải là duy nhất; nghĩa là, có thể có nhiều hàng (documents, vertices) nằm dưới cùng một mục index. Điều này có thể được giải quyết theo hai cách: hoặc là biến mỗi giá trị trong index thành một danh sách các định danh hàng khớp (giống như một postings list trong một full-text index), hoặc là làm cho mỗi mục trở nên duy nhất bằng cách đính kèm một định danh hàng vào đó. Các storage engine với cơ chế in-place updates, như B-trees, và log-structured storage đều có thể được sử dụng để triển khai một index.

Lưu trữ các giá trị bên trong index

Key trong một index là thứ mà các truy vấn tìm kiếm dựa vào. Các dữ liệu khác cũng có thể được lưu trữ trong index, bên cạnh các key, tùy thuộc vào loại index:

- Nếu dữ liệu thực tế (row, document, vertex) được lưu trữ trực tiếp bên trong cấu trúc index, nó được gọi là một *clustered index*. Ví dụ, trong storage engine InnoDB của MySQL, primary key của một bảng luôn là một clustered index, và trong SQL Server, bạn có thể chỉ định một clustered index cho mỗi bảng [43].
- Ngoài ra, giá trị có thể là một tham chiếu đến dữ liệu thực tế: hoặc là primary key của row đang được xét (InnoDB thực hiện điều này cho các secondary indexes) hoặc là một tham chiếu trực tiếp đến một vị trí trên đĩa. Trong trường hợp sau, nơi các row được lưu trữ được gọi là một *heap file*, và nó lưu trữ dữ liệu mà không theo một thứ tự cụ thể nào (nó có thể chỉ cho phép append-only, hoặc nó có thể theo dõi các row đã bị xóa để ghi đè chúng bằng dữ liệu mới sau đó). Ví dụ, Postgres sử dụng cách tiếp cận heap file [44].
- Một giải pháp trung gian giữa hai cách trên là *covering index* hoặc *index with included columns*, loại này lưu trữ *một số* cột của bảng bên trong index, bên cạnh việc lưu trữ toàn bộ row trên heap hoặc trong primary-key clustered index [45]. Điều này cho phép một số truy vấn có thể được trả lời chỉ bằng cách sử dụng index, mà không cần phải giải quyết primary key hoặc tìm trong heap file (khi đó, index được gọi là *cover* truy vấn). Việc này có thể giúp một số truy vấn nhanh hơn, nhưng việc trùng lặp dữ liệu đồng nghĩa với việc index sử dụng nhiều không gian đĩa hơn và làm chậm các thao tác ghi.

Các index đã thảo luận cho đến nay chỉ ánh xạ một key duy nhất tới một giá trị. Nếu bạn cần truy vấn nhiều cột của một bảng (hoặc nhiều field trong một document) cùng một lúc, hãy xem mục “Multidimensional and Full-Text Indexes”.

Khi cập nhật một giá trị mà không thay đổi key, cách tiếp cận heap file có thể cho phép bản ghi được ghi đè tại chỗ, với điều kiện giá trị mới không lớn hơn giá trị cũ. Tình huống sẽ phức tạp hơn nếu giá trị mới lớn hơn, vì nó có thể cần được di chuyển đến một vị trí mới trong heap nơi có đủ không gian. Trong trường hợp đó, tất cả các index cần được cập nhật để trỏ tới vị trí heap mới của bản ghi, hoặc một forwarding pointer phải được để lại tại vị trí heap cũ [2].

Lưu trữ mọi thứ trong bộ nhớ

Các cấu trúc dữ liệu đã thảo luận cho đến nay trong chương này đều là những câu trả lời cho các hạn chế của đĩa. So với bộ nhớ chính, đĩa rất khó làm việc cùng. Với cả đĩa từ và SSD, dữ liệu cần được bố trí cẩn thận nếu bạn muốn có hiệu năng tốt cho các thao tác đọc và ghi. Chúng ta chấp nhận sự bất tiện này vì đĩa có hai ưu điểm

đáng kể: chúng có tính bền vững (nội dung của chúng không bị mất nếu mất điện), và chúng có chi phí trên mỗi gigabyte thấp hơn RAM.

Khi RAM trở nên rẻ hơn, lập luận về chi phí trên mỗi gigabyte dần mất đi sức nặng. Nhiều dataset đơn giản là không lớn đến thế, vì vậy việc giữ chúng hoàn toàn trong bộ nhớ là khả thi, thậm chí có thể phân tán trên nhiều máy. Điều này đã dẫn đến sự phát triển của các *in-memory databases*.

Một số key-value store trong bộ nhớ, chẳng hạn như Memcached, chỉ nhằm mục đích sử dụng để caching, nơi mà việc mất dữ liệu là có thể chấp nhận được nếu một máy bị khởi động lại. Nhưng các in-memory databases khác lại hướng tới tính bền vững, điều này có thể đạt được bằng phần cứng đặc biệt (như RAM chạy bằng pin) hoặc, phổ biến hơn, là ghi một log các thay đổi xuống đĩa, bằng cách ghi các snapshots định kỳ xuống đĩa, hoặc replication trạng thái trong bộ nhớ sang các máy khác.

Điều này cho phép database tải lại trạng thái của nó, từ đĩa hoặc qua mạng từ một replica (trừ khi sử dụng phần cứng đặc biệt), khi nó được khởi động lại. Mặc dù có ghi xuống đĩa, các hệ thống này vẫn được coi là in-memory databases vì đĩa chỉ đơn thuần được sử dụng như một append-only log để đảm bảo tính bền vững, và các thao tác đọc được phục vụ hoàn toàn từ bộ nhớ. Việc ghi xuống đĩa cũng mang lại các lợi thế về vận hành: các tệp trên đĩa có thể dễ dàng được sao lưu, kiểm tra và phân tích bởi các tiện ích bên ngoài.

Các sản phẩm như VoltDB, SingleStore và Oracle TimesTen là các cơ sở dữ liệu in-memory với mô hình relational, và các nhà cung cấp tuyên bố rằng chúng có thể mang lại những cải tiến hiệu năng lớn bằng cách loại bỏ tất cả các overhead liên quan đến việc quản lý các cấu trúc dữ liệu trên đĩa [46, 47]. RAMCloud là một key-value store in-memory mã nguồn mở có tính durability (sử dụng phương pháp tiếp cận log-structured cho dữ liệu trong bộ nhớ cũng như dữ liệu trên đĩa) [48]. Redis và Couchbase cung cấp tính durability yếu bằng cách ghi vào đĩa một cách bất đồng bộ.

Ngược với suy đoán thông thường, lợi thế hiệu năng của các cơ sở dữ liệu in-memory không phải do chúng không cần đọc từ đĩa. Ngay cả một storage engine dựa trên đĩa cũng có thể không bao giờ cần đọc từ đĩa nếu bạn có đủ bộ nhớ, bởi vì hệ điều hành dù sao cũng sẽ cache các khối đĩa vừa được sử dụng vào bộ nhớ. Thay vào đó, chúng nhanh hơn vì chúng tránh được các overhead của việc mã hóa các cấu trúc dữ liệu in-memory sang một dạng có thể ghi được vào đĩa [49].

Bên cạnh hiệu năng, một trường hợp sử dụng thú vị khác cho các cơ sở dữ liệu in-memory là cung cấp các mô hình dữ liệu vốn khó triển khai với các index dựa trên đĩa. Ví dụ, Redis cung cấp một giao diện giống như cơ sở dữ liệu cho nhiều cấu trúc dữ liệu khác nhau, chẳng hạn như priority queues và các tập hợp (sets). Vì nó giữ tất cả dữ liệu trong bộ nhớ, việc triển khai nó tương đối đơn giản.

Lưu trữ dữ liệu cho Analytics

Mô hình dữ liệu của một data warehouse thường là relational, vì SQL nhìn chung rất phù hợp cho các truy vấn phân tích. Có nhiều công cụ phân tích dữ liệu đồ họa giúp tạo ra các truy vấn SQL, trực quan hóa kết quả và cho phép các nhà phân tích khám phá dữ liệu (thông qua các thao tác như *drill-down* và *slicing and dicing*).

Về bề ngoài, một data warehouse và một cơ sở dữ liệu OLTP relational trông có vẻ giống nhau, vì cả hai đều có giao diện truy vấn SQL. Tuy nhiên, các thành phần bên trong của các hệ thống này có thể trông rất khác nhau, vì chúng được tối ưu hóa cho các mẫu truy vấn rất khác biệt. Nhiều nhà cung cấp cơ sở dữ liệu hiện nay tập trung vào việc hỗ trợ xử lý transaction hoặc các khối lượng công việc analytics, chứ không phải cả hai.

Một số cơ sở dữ liệu, chẳng hạn như Microsoft SQL Server, SAP HANA và SingleStore, có hỗ trợ xử lý transaction và data warehousing trong cùng một sản phẩm. Tuy nhiên, các cơ sở dữ liệu HTAP (hybrid transactional and analytical processing) này (đã được giới thiệu trong mục “Data Warehousing”) đang ngày càng trở thành hai engine lưu trữ và truy vấn riêng biệt, vốn tình cờ có thể truy cập thông qua một giao diện SQL chung [50, 51, 52, 53].

Cloud Data Warehouses

Các nhà cung cấp data warehouse lâu đời như Teradata, Vertica và SAP HANA cung cấp các triển khai on-premises dưới các giấy phép thương mại cũng như các giải pháp dựa trên cloud. Nhưng khi có nhiều khách hàng chuyển sang cloud hơn, các data warehouse chỉ dành riêng cho cloud mới như BigQuery của Google Cloud, Amazon Redshift và Snowflake cũng đã trở nên được áp dụng rộng rãi. Không giống như các data warehouse truyền thống, các cloud data warehouse có thể tận dụng hạ tầng cloud có khả năng mở rộng như object storage và các nền tảng tính toán serverless.

Các cloud data warehouse có xu hướng tích hợp tốt hơn với các dịch vụ cloud khác. Ví dụ, nhiều cloud warehouse hỗ trợ việc ingestion log tự động và cung cấp khả năng tích hợp dễ dàng với các framework xử lý dữ liệu như Dataflow của Google Cloud hoặc AWS Kinesis. Các warehouse này cũng có tính đàn hồi cao hơn vì chúng decouple việc tính toán truy vấn khỏi lớp lưu trữ [54]. Dữ liệu được lưu trữ bền vững trong object storage thay vì trên các đĩa cục bộ, điều này giúp dễ dàng điều chỉnh dung lượng lưu trữ và tài nguyên tính toán cho các truy vấn một cách độc lập, như chúng ta đã thấy trong mục “Cloud Native System Architecture”.

Các data warehouse mã nguồn mở như Apache Hive, Trino và Apache Spark cũng đã phát triển cùng với cloud. Khi việc lưu trữ dữ liệu cho analytics đã chuyển sang các data lake trên object storage, các warehouse mã nguồn mở đã bắt đầu tách rời nhau [55]. Các thành phần sau đây, vốn trước đây được tích hợp trong một hệ thống

duy nhất như Hive, giờ đây thường được triển khai dưới dạng các thành phần riêng biệt:

Query engine

Các query engine như Trino, Apache DataFusion và Presto thực hiện phân tích các truy vấn SQL, tối ưu hóa chúng thành các execution plan và thực thi chúng trên dữ liệu. Việc thực thi thường yêu cầu các tác vụ xử lý dữ liệu song song và phân tán. Một số query engine cung cấp khả năng thực thi tác vụ tích hợp sẵn, trong khi những engine khác chọn sử dụng các framework thực thi của bên thứ ba như Spark hoặc Flink.

Storage format

Storage format quyết định cách các hàng của một bảng được mã hóa thành các byte trong một tệp, sau đó thường được lưu trữ trong object storage hoặc một distributed filesystem [12]. Dữ liệu này sau đó có thể được truy cập không chỉ bởi query engine mà còn bởi các ứng dụng khác sử dụng data lake. Ví dụ về các storage format như vậy là Parquet, ORC, Lance và Nimble; chúng ta sẽ thảo luận thêm về chúng trong mục tiếp theo.

Table format

Các tệp được ghi dưới định dạng Parquet và các storage format tương tự thường là immutable sau khi đã ghi. Để hỗ trợ chèn và xóa hàng, có thể sử dụng một table format như Apache Iceberg hoặc định dạng Delta của Databricks. Các table format chỉ định một file format để xác định những tệp nào cấu thành nên một bảng cùng với schema của bảng đó. Các định dạng này cũng cung cấp các tính năng nâng cao như time travel (khả năng truy vấn một bảng tại một thời điểm trong quá khứ), GC, và thậm chí cả các transaction.

Data catalog

Giống như cách một table format xác định những tệp nào tạo nên một bảng, một data catalog xác định những bảng nào nằm trong một cơ sở dữ liệu. Các catalog được sử dụng để tạo, đổi tên và xóa các bảng. Khác với storage và table formats, các data catalog như Polaris của Snowflake và Unity Catalog của Databricks thường chạy như một dịch vụ độc lập có thể được truy vấn bằng giao diện REST. Apache Iceberg cũng cung cấp một catalog, có thể chạy bên trong một client hoặc dưới dạng một tiến trình riêng biệt. Các query engine sử dụng thông tin catalog khi đọc và ghi các bảng. Theo truyền thống, các catalog và query engine thường được tích hợp với nhau, nhưng việc tách rời chúng đã cho phép các hệ thống data discovery và data governance (được thảo luận trong mục “Data Systems, Law, and Society”) cũng có thể truy cập vào metadata của một catalog.

Column-Oriented Storage

Như đã thảo luận trong mục “Stars and Snowflakes: Schemas for Analytics”, theo quy ước, các data warehouse thường sử dụng một relational schema với một bảng fact lớn chứa các tham chiếu foreign-key tới các dimension table. Nếu bạn có hàng nghìn tỷ hàng và hàng petabyte dữ liệu trong các bảng fact của mình, việc lưu trữ và truy vấn chúng một cách hiệu quả sẽ trở nên đầy thách thức. Các dimension table thường nhỏ hơn nhiều và dễ quản lý hơn (vài triệu hàng), vì vậy trong mục này, chúng ta sẽ tập trung vào việc lưu trữ các fact.

Mặc dù các bảng fact thường rộng hơn một trăm cột, một truy vấn data warehouse điển hình chỉ truy cập bốn hoặc năm cột cùng một lúc (các truy vấn `SELECT *` hiếm khi cần thiết cho phân tích) [52]. Hãy xét truy vấn trong Example 4-1: nó truy cập một số lượng lớn các hàng (mọi lần một ai đó mua trái cây hoặc kẹo trong năm dương lịch 2024), nhưng nó chỉ cần truy cập ba cột của bảng `fact_sales`: `date_key`, `product_sk`, và `quantity`. Truy vấn này bỏ qua tất cả các cột khác.

Example 4-1. Phân tích xem mọi người có xu hướng mua trái cây tươi hay kẹo nhiều hơn, tùy thuộc vào ngày trong tuần

```
SELECT
    dim_date.weekday, dim_product.category,
    SUM(fact_sales.quantity) AS quantity_sold
FROM fact_sales
JOIN dim_date ON fact_sales.date_key = dim_date.date_key
JOIN dim_product ON fact_sales.product_sk = dim_product.product_sk
WHERE
    dim_date.year = 2024 AND
    dim_product.category IN ('Fresh fruit', 'Candy')
GROUP BY
    dim_date.weekday, dim_product.category;
```

Làm thế nào chúng ta có thể thực thi truy vấn này một cách hiệu quả?

Trong hầu hết các cơ sở dữ liệu OLTP, storage được bố trí theo kiểu *row-oriented* (hướng hàng): tất cả các giá trị từ một hàng của một bảng được lưu trữ cạnh nhau. Các document database cũng tương tự: toàn bộ một document thường được lưu trữ dưới dạng một chuỗi các byte liên tục. Bạn có thể thấy điều này trong ví dụ CSV ở Hình 4-1.

Để xử lý một truy vấn như trong Example 4-1, bạn có thể có các index trên `fact_sales.date_key` và/hoặc `fact_sales.product_sk` để chỉ cho storage engine biết nơi tìm tất cả các giao dịch bán hàng cho một ngày cụ thể hoặc cho một sản phẩm cụ thể. Nhưng khi đó, một storage engine hướng hàng vẫn cần phải tải tất cả các hàng đó (mỗi hàng bao gồm hơn 100 thuộc tính) từ đĩa vào bộ nhớ, phân tích chúng và lọc ra những hàng không đáp ứng các điều kiện yêu cầu. Việc đó có thể mất rất nhiều thời gian.

Ý tưởng đằng sau lưu trữ *column-oriented* (hướng cột) rất đơn giản: thay vì lưu trữ tất cả các giá trị từ một hàng cùng nhau, chúng ta lưu trữ tất cả các giá trị từ mỗi *column* (cột) cùng nhau [56]. Nếu mỗi column được lưu trữ riêng biệt, một truy vấn chỉ cần đọc và phân tích những column được sử dụng trong truy vấn đó, điều này có thể tiết kiệm được rất nhiều công sức. Hình 4-7 minh họa nguyên lý này bằng cách sử dụng một phiên bản mở rộng của fact table từ Hình 3-5.

LƯU Ý

Lưu trữ theo cột là dễ hiểu nhất trong một relational data model, nhưng nó cũng áp dụng tương tự cho dữ liệu nonrelational. Ví dụ, Parquet là một định dạng lưu trữ columnar hỗ trợ document data model [57] dựa trên Dremel của Google [58] bằng cách sử dụng một kỹ thuật được gọi là *shredding* hoặc *striping* [59].

fact_sales table

date_key	product_sk	store_sk	promotion_sk	customer_sk	quantity	net_price	discount_price
260102	69	4	NULL	NULL	1	13.99	13.99
260102	69	5	19	NULL	3	14.99	9.99
260102	69	5	NULL	191	1	14.99	14.99
260102	74	3	23	202	5	0.99	0.89
260103	30	2	NULL	NULL	1	2.49	2.49
260103	30	3	NULL	NULL	3	14.99	9.99
260103	30	3	21	123	1	49.99	39.99
260103	30	8	NULL	233	1	0.99	0.99

Columnar storage layout

date_key: 260102, 260102, 260102, 260102, 260103, 260103, 260103, 260103
product_sk: 69, 69, 69, 74, 30, 30, 30, 30
store_sk: 4, 5, 5, 3, 2, 3, 3, 8
promotion_sk: NULL, 19, NULL, 23, NULL, NULL, 21, NULL
customer_sk: NULL, NULL, 191, 202, NULL, NULL, 123, 233
quantity: 1, 3, 1, 5, 1, 3, 1, 1
net_price: 13.99, 14.99, 14.99, 0.99, 2.49, 14.99, 49.99, 0.99
discount_price: 13.99, 9.99, 14.99, 0.89, 2.49, 9.99, 39.99, 0.99

Figure 4-7. Hình 4.1: Lưu trữ dữ liệu quan hệ theo cột thay vì theo hàng

Bố cục lưu trữ column-oriented dựa trên việc mỗi cột đều lưu trữ các hàng theo cùng một thứ tự. Do đó, nếu bạn cần lắp ráp lại toàn bộ một hàng, bạn có thể lấy mục thứ 23 từ mỗi cột riêng lẻ và kết hợp chúng lại để tạo thành hàng thứ 23 của bảng.

Trong thực tế, các engine lưu trữ columnar không thực sự lưu trữ toàn bộ một cột (có thể chứa hàng nghìn tỷ hàng) trong một lần duy nhất. Thay vào đó, chúng chia bảng thành các block gồm hàng nghìn hoặc hàng triệu hàng, và trong mỗi block, chúng lưu trữ các giá trị từ mỗi cột một cách riêng biệt [60]. Vì nhiều truy vấn bị giới hạn trong một phạm vi ngày cụ thể, nên việc để mỗi block chứa các hàng cho một

phạm vi timestamp nhất định là điều phổ biến. Khi đó, một truy vấn chỉ cần tải các cột mà nó cần trong các block có phần giao với phạm vi ngày được yêu cầu.

Lưu trữ columnar được sử dụng trong hầu hết các analytical databases ngày nay [60], trải dài từ các data warehouse đám mây quy mô lớn như Snowflake [61] đến các embedded databases đơn node như DuckDB [62] và các hệ thống product analytics như Pinot [63] và Druid [64]. Nó được sử dụng trong các định dạng lưu trữ như Parquet, ORC [65, 66], Lance [67], và Nimble [68] cũng như các định dạng in-memory analytics như Apache Arrow [65, 69] và Pandas/NumPy [70]. Một số time-series databases, chẳng hạn như InfluxDB IOx [71] và TimescaleDB [72], cũng dựa trên lưu trữ column-oriented.

Nén cột (Column compression)

Bên cạnh việc chỉ tải từ đĩa những cột cần thiết cho một truy vấn, chúng ta có thể giảm thêm yêu cầu về throughput của đĩa và băng thông mạng bằng cách nén dữ liệu. May mắn thay, lưu trữ column-oriented thường rất phù hợp với việc nén.

Hãy nhìn vào các chuỗi giá trị cho mỗi cột trong Hình 4-7. Có một lượng đáng kể sự lặp lại, đây là một dấu hiệu tốt cho việc nén. Tùy thuộc vào dữ liệu trong cột, các kỹ thuật nén khác nhau có thể được sử dụng [73]. Một kỹ thuật đặc biệt hiệu quả trong các data warehouse là *bitmap encoding*, được minh họa trong Hình 4-8.

Column values:

product_sk:

69	69	69	69	74	30	30	30	30	29	31	31	30	30	30	68	69	69
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

Bitmap for each possible value:

product_sk = 29:

0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

product_sk = 30:

0	0	0	0	0	1	1	1	1	0	0	0	1	1	1	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

product_sk = 31:

0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

product_sk = 68:

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

product_sk = 69:

1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

product_sk = 74:

0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Run-length encoding:

product_sk = 29: 9, 1 (9 zeros, 1 one, rest zeros)

product_sk = 30: 5, 4, 3, 3 (5 zeros, 4 ones, 3 zeros, 3 ones, rest zeros)

product_sk = 31: 10, 2 (10 zeros, 2 ones, rest zeros)

product_sk = 68: 15, 1 (15 zeros, 1 one, rest zeros)

product_sk = 69: 0, 4, 12, 2 (0 zeros, 4 ones, 12 zeros, 2 ones)

product_sk = 74: 4, 1 (4 zeros, 1 one, rest zeros)

Figure 4-8. Lưu trữ một cột duy nhất được nén và đánh chỉ mục bằng bitmap

Thông thường, số lượng các giá trị riêng biệt trong một cột là nhỏ so với số lượng các hàng (ví dụ, một nhà bán lẻ có thể có hàng tỷ giao dịch bán hàng, nhưng chỉ có 100.000 sản phẩm riêng biệt). Bây giờ chúng ta có thể lấy một cột với n giá trị riêng biệt và chuyển nó thành n bitmap riêng biệt: một bitmap cho mỗi giá trị riêng biệt, với một bit cho mỗi hàng. Bit sẽ là 1 nếu hàng đó có giá trị đó, và là 0 nếu không.

Một lựa chọn là lưu trữ các bitmap bằng cách sử dụng một bit cho mỗi hàng. Tuy nhiên, các bitmap này thường chứa rất nhiều số 0 (chúng ta gọi chúng là *sparse* (thưa)). Trong trường hợp đó, các bitmap có thể được *run-length encoded* (mã hóa độ dài chuỗi) bổ sung, bao gồm việc đếm các số 0 hoặc 1 liên tiếp và lưu trữ các số đếm đó, như được hiển thị ở cuối Hình 4-8. Các kỹ thuật như *roaring bitmaps* sẽ chuyển đổi giữa hai cách biểu diễn bitmap, sử dụng cách nào gọn nhất [74]. Điều này có thể làm cho việc mã hóa một cột trở nên cực kỳ hiệu quả.

Các bitmap index như thế này rất phù hợp cho các loại truy vấn thường gặp trong một data warehouse. Ví dụ:

WHERE product_sk IN (31, 68, 69)

Tải ba bitmap cho product_sk = 31, product_sk = 68, và product_sk = 69, rồi tính toán bitwise OR của ba bitmap này, việc này có thể được thực hiện rất hiệu quả.

WHERE product_sk = 30 AND store_sk = 3

Tải các bitmap cho `product_sk = 30` và `store_sk = 3`, rồi tính toán bitwise AND. Điều này hoạt động vì các cột chứa các hàng theo cùng một thứ tự, do đó bit thứ k trong bitmap của một cột tương ứng với cùng một hàng như bit thứ k trong bitmap của cột khác.

Bitmap cũng có thể được sử dụng để trả lời các truy vấn đồ thị, chẳng hạn như tìm tất cả người dùng của một mạng xã hội được theo dõi bởi người dùng X và cũng theo dõi người dùng Y [75].

LƯU Ý

Đừng nhầm lẫn các column-oriented databases với mô hình dữ liệu *wide-column* (còn được gọi là *column-family*), trong đó một hàng có thể có hàng nghìn cột, và không cần tất cả các hàng phải có cùng các cột [9]. Mặc dù có sự tương đồng về tên gọi, các wide-column databases là row-oriented, vì chúng lưu trữ tất cả các giá trị từ một hàng cùng nhau. Google Bigtable, Apache Accumulo, và HBase là những ví dụ về các hệ thống sử dụng mô hình wide-column.

Thứ tự sắp xếp trong lưu trữ cột

Trong một column store, thứ tự mà các hàng được lưu trữ không nhất thiết phải quan trọng. Cách dễ nhất là lưu trữ chúng theo thứ tự mà chúng được chèn vào, vì khi đó việc chèn một hàng mới chỉ đơn giản là nối thêm vào mỗi cột. Tuy nhiên, chúng ta có thể chọn áp đặt một thứ tự, như chúng ta đã làm với các SSTables trước đó, và sử dụng nó như một cơ chế indexing.

Lưu ý rằng việc sắp xếp mỗi cột một cách độc lập sẽ không có ý nghĩa vì khi đó chúng ta sẽ không còn biết các mục trong các cột nào thuộc về cùng một hàng. Chúng ta chỉ có thể tái cấu trúc một hàng vì chúng ta biết rằng mục thứ k trong một cột thuộc về cùng một hàng với mục thứ k trong một cột khác.

Thay vào đó, dữ liệu cần được sắp xếp theo từng hàng một, mặc dù nó được lưu trữ theo cột. Quản trị viên của cơ sở dữ liệu có thể chọn các cột mà bảng nên được sắp xếp dựa trên kiến thức của họ về các truy vấn phổ biến. Ví dụ, nếu các truy vấn thường nhắm vào các phạm vi ngày, chẳng hạn như tháng trước, thì việc chọn `date_key` làm khóa sắp xếp đầu tiên có thể là hợp lý. Khi đó, truy vấn có thể chỉ quét các hàng từ tháng trước, việc này sẽ nhanh hơn nhiều so với việc quét tất cả các hàng.

Một cột thứ hai có thể xác định thứ tự sắp xếp của bất kỳ hàng nào có cùng giá trị ở cột đầu tiên. Ví dụ, nếu `date_key` là khóa sắp xếp đầu tiên trong Hình 4-7, thì việc chọn `product_sk` làm khóa sắp xếp thứ hai có thể là hợp lý để tất cả các giao dịch bán hàng cho cùng một sản phẩm vào cùng một ngày được nhóm lại với nhau trong lưu trữ. Điều đó sẽ giúp ích cho các truy vấn cần nhóm hoặc lọc các giao dịch bán hàng theo sản phẩm trong một phạm vi ngày nhất định.

Một lợi thế khác của thứ tự đã được sắp xếp là nó có thể hỗ trợ nén các column. Nếu column sắp xếp chính không có nhiều giá trị khác biệt, thì sau khi sắp xếp, nó sẽ có các chuỗi dài nơi mà cùng một giá trị được lặp lại nhiều lần liên tiếp. Một phương pháp run-length encoding đơn giản, giống như cách chúng ta đã sử dụng cho các bitmap trong Hình 4-8, có thể nén column đó xuống chỉ còn vài kilobyte—ngay cả khi bảng có hàng tỷ hàng.

Hiệu ứng nén đó mạnh nhất ở khóa sắp xếp đầu tiên. Các khóa sắp xếp thứ hai và thứ ba sẽ bị xáo trộn nhiều hơn và do đó sẽ không có các chuỗi lặp lại giá trị dài như vậy. Các column nằm xa hơn trong thứ tự ưu tiên sắp xếp về cơ bản xuất hiện theo thứ tự ngẫu nhiên, vì vậy chúng có lẽ sẽ không được nén tốt như vậy. Tuy nhiên, việc có được vài column đầu tiên được sắp xếp vẫn là một lợi thế tổng thể.

Ghi vào lưu trữ column-oriented

Chúng ta đã thấy trong mục “Đặc điểm của Transaction Processing và Analytics” rằng các thao tác đọc trong data warehouse có xu hướng bao gồm các phép tổng hợp trên một số lượng lớn các hàng. Lưu trữ column-oriented, nén và sắp xếp đều giúp làm cho các truy vấn đọc đó nhanh hơn.

Các thao tác ghi trong một data warehouse có xu hướng là nhập dữ liệu hàng loạt, thường thông qua một quy trình ETL. Với lưu trữ columnar, việc ghi một hàng riêng lẻ vào đầu đó ở giữa một bảng đã được sắp xếp sẽ rất kém hiệu quả, vì bạn sẽ phải ghi lại tất cả các column đã được nén từ vị trí chèn trở đi. Tuy nhiên, một thao tác ghi hàng loạt nhiều hàng cùng một lúc sẽ giúp phân bổ chi phí ghi lại các column đó, giúp nó trở nên hiệu quả.

Một cách tiếp cận log-structured thường được sử dụng để thực hiện các thao tác ghi theo batch. Tất cả các thao tác ghi trước tiên sẽ đi vào một bộ lưu trữ trong bộ nhớ (in-memory store) đã được sắp xếp và theo dạng row-oriented. Khi đã tích lũy đủ các thao tác ghi, chúng sẽ được hợp nhất với các file đã được mã hóa theo kiểu column trên đĩa và được ghi vào các file mới dưới dạng hàng loạt. Vì các file cũ vẫn giữ tính immutable và các file mới được ghi trong một lần, nên object storage rất phù hợp để lưu trữ các file này.

Các truy vấn cần kiểm tra cả dữ liệu column trên đĩa và các thao tác ghi gần đây trong bộ nhớ, sau đó kết hợp cả hai. Query execution engine sẽ che giấu sự khác biệt này đối với người dùng. Từ góc độ của một nhà phân tích, dữ liệu đã được sửa đổi bằng các thao tác insert, update, hoặc delete sẽ được phản ánh ngay lập tức trong các truy vấn tiếp theo. Snowflake, Vertica, Apache Pinot, Apache Druid, và nhiều database khác thực hiện điều này [61, 63, 64, 76].

Query Execution: Compilation và Vectorization

Một truy vấn SQL phức tạp cho analytics được chia nhỏ thành một *query plan* bao gồm nhiều giai đoạn, được gọi là các *operator*, chúng có thể được phân phối trên nhiều máy để thực thi song song. Các query planner có thể thực hiện rất nhiều tối ưu hóa bằng cách chọn operator nào để sử dụng, thực thi chúng theo thứ tự nào, và chạy từng operator ở đâu.

Trong mỗi operator, query engine có thể cần thực hiện nhiều việc khác nhau với các giá trị trong một column, chẳng hạn như tìm tất cả các hàng mà giá trị nằm trong một tập giá trị cụ thể nào đó (có lẽ là một phần của phép join), hoặc kiểm tra xem giá trị có lớn hơn, ví dụ là 15, hay không. Query engine có lẽ cũng sẽ cần xem xét nhiều column cho cùng một hàng—ví dụ, để tìm tất cả các giao dịch bán hàng mà sản phẩm là “bananas” và cửa hàng là một cửa hàng cụ thể đang được quan tâm.

Đối với các truy vấn data warehouse phải quét hàng triệu hàng, chúng ta không chỉ cần lo lắng về lượng dữ liệu mà chúng phải đọc từ đĩa, mà còn về thời gian CPU cần thiết để thực thi các operator phức tạp. Loại operator đơn giản nhất giống như một trình thông dịch cho một ngôn ngữ lập trình. Trong khi lặp qua từng hàng, nó kiểm tra một cấu trúc dữ liệu đại diện cho truy vấn để tìm ra những phép so sánh hoặc tính toán nào cần thực hiện trên các column nào. Thật không may, điều này quá chậm đối với nhiều mục đích analytics. Hai cách tiếp cận thay thế để thực thi truy vấn hiệu quả đã xuất hiện [77]:

Query compilation

Query engine tiếp nhận truy vấn SQL và tạo ra mã để thực thi nó. Mã này sẽ lặp qua từng dòng một, kiểm tra các giá trị trong các cột được quan tâm, thực hiện bất kỳ phép so sánh hoặc tính toán nào cần thiết, và sao chép các giá trị cần thiết vào một output buffer nếu các điều kiện yêu cầu được thỏa mãn. Sau đó, query engine biên dịch mã vừa tạo thành mã máy (thường sử dụng một compiler có sẵn như LLVM) và chạy nó trên dữ liệu đã được mã hóa theo cột (column-encoded) đã được nạp vào bộ nhớ. Cách tiếp cận tạo mã này tương tự như phương pháp biên dịch just-in-time (JIT) được sử dụng trong Java Virtual Machine (JVM) và các runtime tương tự.

Xử lý vectorized

Truy vấn được thông dịch thay vì được biên dịch, nhưng nó đạt được tốc độ nhanh nhờ việc xử lý nhiều giá trị từ một cột theo một batch thay vì lặp qua từng dòng một. Một tập hợp cố định các toán tử đã được định nghĩa sẵn được tích hợp sẵn vào cơ sở dữ liệu; chúng ta có thể truyền các đối số vào chúng và nhận lại một batch kết quả [50, 73].

Ví dụ, chúng ta có thể truyền cột `product_sk` và ID của một sản phẩm (chẳng hạn, “bananas”) vào một toán tử so sánh bằng, và nhận lại một bitmap (mỗi giá trị trong cột đầu vào tương ứng với một bit, giá trị là 1 nếu khớp với ID đó). Sau đó,

chúng ta có thể truyền cột `store_sk` và ID của cửa hàng được quan tâm vào chính toán tử so sánh bằng đó, và nhận lại một bitmap khác. Cuối cùng, chúng ta có thể truyền hai bitmap này vào một toán tử bitwise AND, như được hiển thị trong Hình 4-9; kết quả sẽ là một bitmap chứa giá trị 1 cho tất cả các lượt bán chuỗi tại một cửa hàng cụ thể.

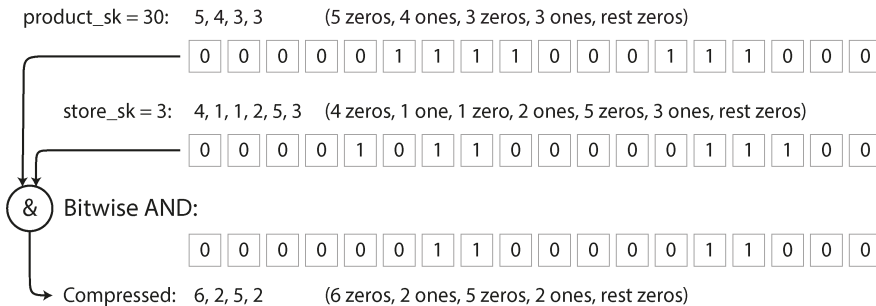


Figure 4-9. *Phép toán bitwise AND giữa hai bitmap rất phù hợp để vectorization (vector hóa).*

Hai cách tiếp cận này rất khác nhau về mặt triển khai, nhưng cả hai đều được sử dụng trong thực tế [77]. Cả hai đều có thể đạt được hiệu năng rất tốt bằng cách tận dụng các đặc điểm của CPU hiện đại:

- Ưu tiên truy cập bộ nhớ tuần tự thay vì truy cập ngẫu nhiên để giảm cache misses [78]
- Thực hiện hầu hết các công việc trong các vòng lặp trong (inner loops) chặt chẽ (tức là với một số lượng nhỏ các lệnh và không có các lời gọi hàm) để giữ cho pipeline xử lý lệnh của CPU luôn bận rộn và tránh các lỗi dự đoán nhánh (branch mispredictions)
- Tận dụng tính song song như nhiều thread và các lệnh single-instruction, multiple data (SIMD) [79, 80]
- Thao tác trực tiếp trên dữ liệu đã nén mà không cần giải nén nó thành một biểu diễn trong bộ nhớ riêng biệt, giúp tiết kiệm chi phí cấp phát và sao chép bộ nhớ

Materialized Views và Data Cubes

Trước đó chúng ta đã gặp *materialized views* trong phần “Materializing and Updating Timelines”: trong một mô hình dữ liệu quan hệ, chúng là các đối tượng dạng bảng mà nội dung là kết quả của một truy vấn. Một materialized view là một bản sao thực sự của kết quả truy vấn được ghi vào đĩa, trong khi một virtual view chỉ là một lối tắt để viết các truy vấn. Khi bạn đọc từ một virtual view, engine SQL sẽ mở rộng nó thành truy vấn cơ sở của view đó ngay lập tức (on the fly) rồi mới xử lý truy vấn đã được mở rộng đó.

Khi dữ liệu cơ sở thay đổi, một materialized view cần được cập nhật tương ứng. Một số cơ sở dữ liệu có thể thực hiện việc đó một cách tự động, và cũng có những hệ thống như Materialize chuyên về bảo trì materialized view [81]. Chúng ta sẽ quay lại chủ đề này trong phần “Maintaining materialized views”. Thực hiện các cập nhật như vậy đồng nghĩa với việc tốn nhiều công việc hơn khi ghi, nhưng các materialized view có thể cải thiện hiệu năng đọc trong các workload cần thực hiện lặp đi lặp lại cùng một truy vấn.

Materialized aggregates là một loại materialized view có thể hữu ích trong các data warehouse. Như đã thảo luận trước đó, các truy vấn trong data warehouse thường liên quan đến một hàm aggregate, chẳng hạn như COUNT, SUM, AVG, MIN, hoặc MAX trong SQL. Nếu cùng một aggregate được sử dụng bởi nhiều truy vấn, việc tính toán qua dữ liệu thô mỗi lần có thể gây lãng phí. Tại sao không cache một số giá trị đếm (counts) hoặc tổng (sums) mà các truy vấn sử dụng thường xuyên nhất? Một *data cube* (còn được gọi là *OLAP cube*) thực hiện việc này bằng cách tạo ra một lưới các aggregate được nhóm theo các dimension khác nhau [82]. Hình 4-10 cho thấy một ví dụ.

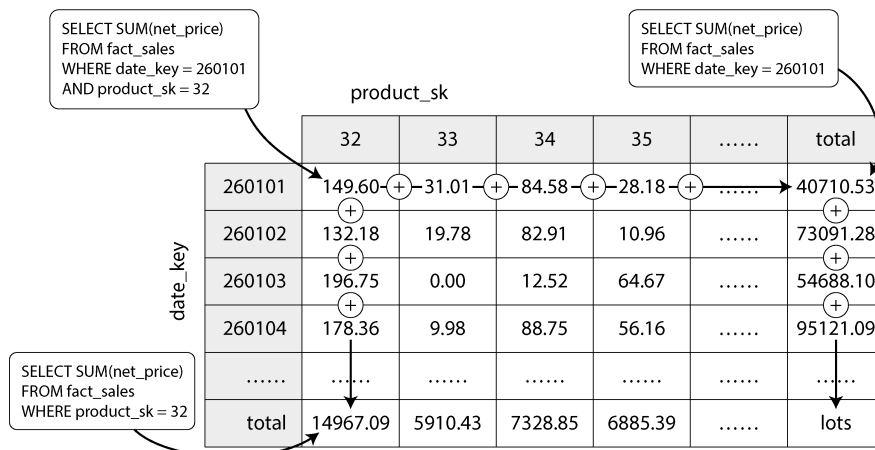


Figure 4-10. Hai chiều của một data cube, tổng hợp dữ liệu bằng cách cộng dồn

Hãy tưởng tượng rằng mỗi fact hiện chỉ có các foreign key trỏ đến hai dimension table; trong Hình 4-10, đó là date_key và product_sk. Bây giờ bạn có thể vẽ một bảng hai chiều, với ngày tháng dọc theo một trục và sản phẩm dọc theo trục còn lại. Mỗi ô chứa giá trị aggregate (ví dụ: SUM) của một attribute (ví dụ: net_price) của tất cả các fact có sự kết hợp ngày-sản phẩm đó. Sau đó, bạn có thể áp dụng cùng một phép aggregate dọc theo mỗi hàng hoặc cột để có được một bản tóm tắt đã được giảm bớt một chiều (ví dụ: doanh số theo sản phẩm bất kể ngày, hoặc doanh số theo ngày bất kể sản phẩm).

Nhìn chung, các fact thường có nhiều hơn hai chiều. Trong Hình 3-5, có năm chiều: ngày, sản phẩm, cửa hàng, khuyến mãi và khách hàng. Sẽ khó hình dung hơn nhiều về việc một hypercube năm chiều trông như thế nào, nhưng nguyên tắc vẫn tương tự: mỗi ô chứa doanh số cho một sự kết hợp cụ thể giữa ngày–sản phẩm–cửa hàng–khuyến mãi–khách hàng. Các giá trị này sau đó có thể được tóm tắt lặp đi lặp lại dọc theo mỗi chiều.

Ưu điểm của một materialized data cube là một số truy vấn nhất định trở nên rất nhanh vì chúng thực tế đã được tính toán trước. Ví dụ, nếu bạn muốn biết tổng doanh số của mỗi cửa hàng vào ngày hôm qua, bạn chỉ cần xem các tổng số dọc theo chiều phù hợp—không cần phải quét hàng triệu dòng.

Nhược điểm là một data cube không có sự linh hoạt như khi truy vấn dữ liệu thô. Ví dụ, không có cách nào để tính toán xem tỷ lệ doanh số đến từ những mặt hàng có giá hơn \$100 là bao nhiêu, bởi vì giá cả không phải là một trong các chiều. Do đó, hầu hết các data warehouse cố gắng giữ lại càng nhiều dữ liệu thô càng tốt và chỉ sử dụng các aggregate như data cubes để tăng cường hiệu năng cho một số truy vấn nhất định.

Multidimensional và Full-Text Indexes

Các B-tree và LSM-tree mà chúng ta đã thấy trong nửa đầu chương này cho phép thực hiện các truy vấn phạm vi (range queries) trên một attribute duy nhất; ví dụ, nếu key là tên người dùng, bạn có thể sử dụng chúng như một index để tìm kiếm hiệu quả tất cả các tên bắt đầu bằng chữ *L*. Nhưng đôi khi, việc tìm kiếm bằng một attribute duy nhất là không đủ.

Loại multicolumn index phổ biến nhất được gọi là *concatenated index*, nó đơn giản là kết hợp nhiều field thành một key bằng cách nối một cột này vào một cột khác (định nghĩa index sẽ chỉ rõ thứ tự các field được nối lại). Điều này giống như một cuốn danh bạ điện thoại bằng giấy kiểu cũ, cung cấp một index từ (*lastname, firstname*) đến số điện thoại. Do thứ tự sắp xếp, index có thể được sử dụng để tìm tất cả những người có một họ cụ thể, hoặc tất cả những người có một sự kết hợp *lastname–firstname* cụ thể. Tuy nhiên, index sẽ trở nên vô dụng nếu bạn muốn tìm tất cả những người có một tên (*firstname*) cụ thể.

Mặt khác, các *multidimensional indexes* cho phép bạn truy vấn nhiều cột cùng một lúc. Điều này đặc biệt quan trọng với dữ liệu địa không gian (geospatial data). Ví dụ, một trang web tìm kiếm nhà hàng có thể có một cơ sở dữ liệu chứa vĩ độ và kinh độ của mỗi nhà hàng. Khi một người dùng đang xem các nhà hàng trên một bản đồ, trang web cần tìm kiếm tất cả các nhà hàng trong khu vực bản đồ hình chữ nhật mà người dùng hiện đang xem. Điều này đòi hỏi một truy vấn phạm vi hai chiều như sau:

```
SELECT * FROM restaurants WHERE latitude > 51.4946 AND latitude
< 51.5079
AND longitude > -0.1162 AND longitude
< -0.1004;
```

Một concatenated index trên các cột `latitude` và `longitude` không thể trả lời loại truy vấn đó một cách hiệu quả. Index có thể cung cấp cho bạn tất cả các nhà hàng trong một phạm vi vĩ độ (nhưng ở bất kỳ kinh độ nào) hoặc tất cả các nhà hàng trong một phạm vi kinh độ (nhưng ở bất kỳ đâu giữa Bắc cực và Nam cực), chứ không thể thực hiện cả hai cùng một lúc.

Một lựa chọn là chuyển đổi một vị trí hai chiều thành một số duy nhất thông qua một đường cong lấp đầy không gian (space-filling curve), sau đó sử dụng một B-tree index thông thường [83]. Phổ biến hơn, các spatial index chuyên dụng như *R-trees* hoặc *Bkd-trees* [84] được sử dụng; chúng chia nhỏ không gian sao cho các điểm dữ liệu gần nhau có xu hướng được nhóm vào cùng một subtree. Ví dụ, PostGIS triển khai các geospatial index dưới dạng R-trees bằng cách sử dụng cơ chế lập chỉ mục Generalized Search Tree của PostgreSQL [85]. Ta cũng có thể sử dụng các lưới hình tam giác, hình vuông hoặc hình lục giác được phân bổ đều [86].

Tuy nhiên, các multidimensional index không chỉ dành cho các vị trí địa lý. Ví dụ, trên một website thương mại điện tử, bạn có thể sử dụng một index ba chiều trên các chiều (*red*, *green*, *blue*) để tìm kiếm các sản phẩm trong một phạm vi màu sắc nhất định, hoặc trong một cơ sở dữ liệu về các quan sát thời tiết, bạn có thể có một index hai chiều trên (*date*, *temperature*) để tìm kiếm hiệu quả tất cả các quan sát trong một năm nhất định mà nhiệt độ nằm trong khoảng từ 25°C đến 30°C. Với một index một chiều, bạn sẽ phải quét qua tất cả các bản ghi của năm đó (bất kể nhiệt độ) rồi mới lọc chúng theo nhiệt độ, hoặc ngược lại. Một index hai chiều có thể thu hẹp kết quả theo cả timestamp và nhiệt độ cùng một lúc [87].

Full-Text Search

Full-text search (tìm kiếm toàn văn) cho phép bạn tìm kiếm một tập hợp các tài liệu văn bản (trang web, mô tả sản phẩm, v.v.) bằng các từ khóa có thể xuất hiện ở bất kỳ đâu trong văn bản [88]. Truy hồi thông tin (information retrieval) là một chủ đề lớn và chuyên biệt, thường liên quan đến việc xử lý đặc thù theo ngôn ngữ; ví dụ, một số ngôn ngữ châu Á được viết mà không có khoảng trắng hoặc dấu câu giữa các từ, và do đó việc chia tách văn bản thành các từ đòi hỏi một mô hình có thể chỉ ra chuỗi ký tự nào cấu thành nên một từ. Full-text search cũng thường liên quan đến việc khớp các từ tương tự nhưng không hoàn toàn giống hệt nhau (tính đến lỗi đánh máy hoặc các dạng ngữ pháp khác nhau của từ) và các từ đồng nghĩa. Những vấn đề đó nằm ngoài phạm vi của cuốn sách này.

Tuy nhiên, về cốt lõi, bạn có thể coi full-text search là một loại truy vấn đa chiều khác. Trong trường hợp này, mỗi từ có thể xuất hiện trong một văn bản (một *term*)

là một chiều. Một tài liệu chứa term x sẽ có giá trị là 1 ở chiều x , và một tài liệu không chứa x sẽ có giá trị là 0. Tìm kiếm các tài liệu đề cập đến “red apples” có nghĩa là một truy vấn tìm kiếm giá trị 1 ở chiều *red* và đồng thời tìm giá trị 1 ở chiều *apples*. Do đó, số lượng các chiều có thể rất lớn.

Cấu trúc dữ liệu mà nhiều công cụ tìm kiếm sử dụng để trả lời các truy vấn như vậy được gọi là một *inverted index*. Đây là một cấu trúc key-value, trong đó key là một term và value là danh sách các ID của tất cả các tài liệu có chứa term đó (gọi là *postings list*). Nếu các ID tài liệu là các số tuần tự, postings list cũng có thể được biểu diễn dưới dạng một sparse bitmap, như trong Hình 4-8; bit thứ n trong bitmap cho term x sẽ là 1 nếu tài liệu có ID n chứa term x [89].

Việc tìm tất cả các tài liệu chứa cả hai term x và y giờ đây tương tự như một truy vấn data warehouse dạng vector hóa nhằm tìm kiếm các hàng khớp với hai điều kiện (Hình 4-9): tải hai bitmap cho các term x và y rồi tính toán phép toán bitwise AND của chúng. Ngay cả khi các bitmap được mã hóa theo kiểu run-length encoding, việc này vẫn có thể được thực hiện rất hiệu quả.

Ví dụ, Lucene, công cụ full-text indexing được sử dụng bởi Elasticsearch và Solr, hoạt động như thế này [90]. Nó lưu trữ ánh xạ từ term sang postings list trong các tệp được sắp xếp dạng SSTable, các tệp này được merge trong background bằng cách sử dụng cùng một phương pháp log-structured mà chúng ta đã thấy ở phần trước trong chương này [91]. Kiểu index GIN của PostgreSQL cũng sử dụng các postings list để hỗ trợ full-text search và lập chỉ mục bên trong các tài liệu JSON [92, 93].

Thay vì chia văn bản thành các từ, một phương án thay thế là tìm tất cả các chuỗi con có độ dài n , được gọi là n -gram. Ví dụ, các trigram ($n = 3$) của chuỗi *hello* là *hel*, *ell*, và *llo*. Nếu chúng ta xây dựng một inverted index của tất cả các trigram, chúng ta có thể tìm kiếm trong các tài liệu các chuỗi con bất kỳ có độ dài ít nhất ba ký tự. Các trigram index thậm chí còn cho phép sử dụng biểu thức chính quy (regular expressions) trong các truy vấn tìm kiếm; nhược điểm là chúng khá lớn [94].

Để đối phó với các lỗi đánh máy trong tài liệu hoặc truy vấn, Lucene có khả năng tìm kiếm văn bản để tìm các từ trong một khoảng cách chỉnh sửa nhất định (một *edit distance* bằng 1 có nghĩa là một chữ cái đã được thêm, xóa, hoặc thay thế) [95]. Nó thực hiện việc này bằng cách lưu trữ tập hợp các term dưới dạng một finite state automaton trên các ký tự trong các key, tương tự như một trie [96], và biến đổi nó thành một *Levenshtein automaton*, hỗ trợ tìm kiếm hiệu quả các từ trong một edit distance cho trước [97].

Vector Embeddings

Semantic search (tìm kiếm ngữ nghĩa) vượt xa các từ đồng nghĩa và lỗi đánh máy để cố gắng hiểu các khái niệm tài liệu và ý định của người dùng. Nó đang trở thành một phần quan trọng của các ứng dụng AI, chẳng hạn như *RAG* (truy hỏi tăng cường

sinh), thứ kết hợp các kết quả tìm kiếm vào đầu ra của một LLM. Ví dụ, nếu các trang trợ giúp của bạn có một trang tiêu đề là “hủy đăng ký của bạn,” người dùng vẫn có thể tìm thấy trang đó khi tìm kiếm “làm thế nào để đóng tài khoản của tôi” hoặc “chấm dứt hợp đồng,” vốn có ý nghĩa gần nhau mặc dù chúng sử dụng các từ hoàn toàn khác nhau.

Để hiểu ngữ nghĩa của một tài liệu—nghĩa của nó—các semantic search index sử dụng các embedding model để dịch một tài liệu văn bản thành một vector gồm các giá trị số thực dấu phẩy động, được gọi là *vector embedding* (embedding). Thông thường việc này được thực hiện bằng cách sử dụng các LLM. Vector đại diện cho một điểm trong không gian đa chiều, và mỗi giá trị dấu phẩy động đại diện cho vị trí của tài liệu dọc theo trục của một chiều. Các embedding model tạo ra các vector embedding nằm gần nhau (trong không gian đa chiều này) khi các tài liệu đầu vào của embedding có sự tương đồng về mặt ngữ nghĩa.

LƯU Ý

Chúng ta đã thấy thuật ngữ *vectorized processing* (xử lý vector hóa) trong mục “Query Execution: Compilation and Vectorization”. Các vector trong semantic search có một ý nghĩa khác. Trong vectorized processing, vector đề cập đến một batch các bit có thể được xử lý bằng mã đã được tối ưu hóa đặc biệt. Trong các embedding model, một vector là một mảng các số thực dấu phẩy động đại diện cho một vị trí trong không gian đa chiều.

Ví dụ, một vector embedding ba chiều cho một trang Wikipedia về nông nghiệp có thể là [0.38, 0.83, 0.41]. Một trang Wikipedia về rau củ sẽ khá gần, có lẽ với một embedding là [0.36, 0.64, 0.67]. Một trang về star schemas có thể có một embedding là [0.85, 0.10, -0.52], nằm ở khoảng cách tương đối xa. Chúng ta có thể nhận biết bằng cách nhìn vào rằng hai vector đầu tiên gần nhau hơn so với vector thứ ba.

Các embedding model sử dụng các vector lớn hơn nhiều (thường là trên 1.000 số), nhưng các nguyên tắc là tương tự nhau. Chúng ta không cố gắng hiểu ý nghĩa của từng con số riêng lẻ; chúng đơn giản là một cách để mô hình chỉ ra một vị trí trong một không gian đa chiều trừu tượng. Các công cụ tìm kiếm sử dụng các hàm khoảng cách như *cosine similarity* (độ tương đồng cosine) hoặc *Euclidean distance* (khoảng cách Euclidean) để đo khoảng cách giữa các vector: cosine similarity đo cosine của góc giữa hai vector để xác định chúng gần nhau như thế nào, trong khi Euclidean distance đo khoảng cách đường thẳng giữa hai điểm trong không gian.

Nhiều embedding model thời kỳ đầu, chẳng hạn như Word2Vec [98], BERT [99], và GPT [100], làm việc với dữ liệu văn bản. Các mô hình như vậy thường được triển khai dưới dạng các neural network. Các nhà nghiên cứu sau đó đã tạo ra các embedding model cho cả video, âm thanh và hình ảnh. Gần đây hơn, kiến trúc mô hình đã trở thành *multimodal* (đa phương thức): một mô hình duy nhất có thể tạo

ra các vector embedding cho nhiều phương thức khác nhau, chẳng hạn như văn bản và hình ảnh.

Các công cụ semantic search sử dụng một embedding model để tạo ra một vector embedding khi người dùng nhập một truy vấn. Truy vấn của người dùng và ngữ cảnh liên quan (chẳng hạn như vị trí của người dùng) được đưa vào embedding model. Sau khi embedding model tạo ra vector embedding của truy vấn, công cụ tìm kiếm phải tìm các tài liệu có vector embedding tương tự bằng cách sử dụng một vector index.

Các vector index lưu trữ các vector embedding của một tập hợp các tài liệu. Để truy vấn index, bạn truyền vào vector embedding của truy vấn, và index sẽ trả về các tài liệu có các vector gần với vector truy vấn nhất. Vì các R-trees mà chúng ta đã thấy trước đó không hoạt động tốt với các vector có nhiều chiều, nên các vector index chuyên dụng được sử dụng, ví dụ như sau:

Flat indexes

Các vector được lưu trữ trong index nguyên bản như chúng vốn có. Một truy vấn phải đọc mọi vector và đo khoảng cách của nó tới vector truy vấn. Flat indexes có độ chính xác cao, nhưng việc đo khoảng cách giữa truy vấn và từng vector thì chậm.

Các index Inverted file (IVF)

Không gian vector được phân cụm thành các phân vùng (gọi là *centroids*) gồm các vector để giảm số lượng vector cần phải so sánh. IVF indexes nhanh hơn flat indexes nhưng chỉ có thể đưa ra các kết quả xấp xỉ; một truy vấn và một tài liệu có thể rơi vào các phân vùng khác nhau, ngay cả khi chúng ở gần nhau. Một truy vấn trên IVF index trước tiên sẽ xác định các *probes*, đơn giản là số lượng phân vùng cần kiểm tra. Các truy vấn sử dụng nhiều probes hơn sẽ chính xác hơn nhưng chậm hơn, vì phải so sánh nhiều vector hơn.

Các index Hierarchical Navigable Small World (HNSW)

Các HNSW indexes duy trì nhiều lớp của không gian vector, như được minh họa trong Hình 4-11. Mỗi lớp được biểu diễn dưới dạng một đồ thị, trong đó các node đại diện cho các vector và các cạnh đại diện cho sự gần kề với các vector lân cận. Một truy vấn bắt đầu bằng cách xác định vector gần nhất ở lớp trên cùng, nơi có số lượng node ít. Sau đó, truy vấn di chuyển đến cùng một node ở lớp bên dưới và đi theo các cạnh trong lớp đó (lớp có kết nối dày đặc hơn) để tìm kiếm một vector gần với vector truy vấn hơn. Quá trình này tiếp tục cho đến khi chạm tới lớp cuối cùng. Giống như IVF indexes, HNSW indexes là các phương pháp xấp xỉ.

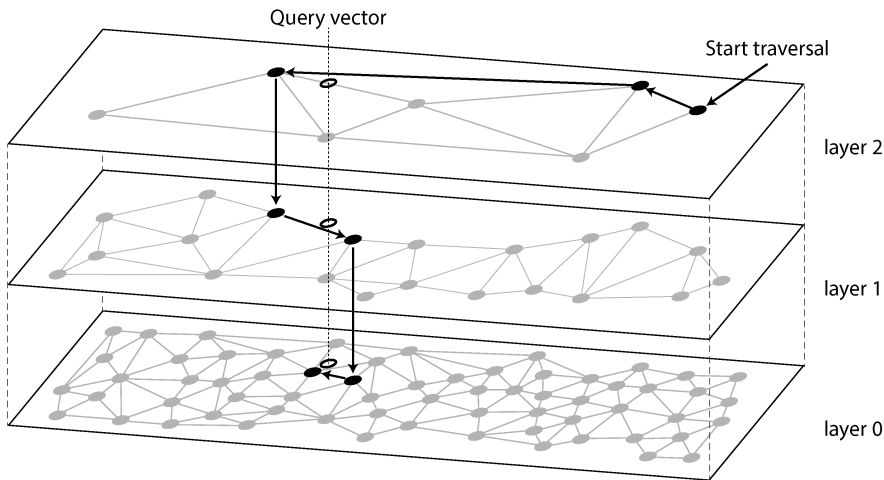


Figure 4-11. Tìm kiếm entry trong cơ sở dữ liệu gần nhất với một vector truy vấn cho trước trong một HNSW index

Nhiều vector database phổ biến triển khai các index IVF và HNSW. Thư viện Faiss của Facebook có vài biến thể của mỗi loại [101], và pgvector của PostgreSQL cũng hỗ trợ cả hai [102]. Chi tiết đầy đủ về các thuật toán IVF và HNSW nằm ngoài phạm vi của cuốn sách này, nhưng các bài báo nghiên cứu về chúng là những tài liệu tham khảo tuyệt vời [103, 104].

Tóm tắt

Trong chương này, chúng ta đã cố gắng tìm hiểu sâu về cách các cơ sở dữ liệu thực hiện lưu trữ và truy hồi. Điều gì xảy ra khi bạn lưu trữ dữ liệu vào một cơ sở dữ liệu, và cơ sở dữ liệu sẽ làm gì khi bạn truy vấn lại dữ liệu đó sau này?

Mục “Operational Versus Analytical Systems” đã giới thiệu sự phân biệt giữa xử lý giao dịch (OLTP) và phân tích (OLAP). Trong chương này, chúng ta thấy rằng các storage engine được tối ưu hóa cho OLTP trông rất khác so với những storage engine được tối ưu hóa cho phân tích:

- Các hệ thống OLTP được tối ưu hóa cho một khối lượng lớn các yêu cầu, mà mỗi yêu cầu đều đọc và ghi một số lượng nhỏ các bản ghi và cần phản hồi nhanh. Các bản ghi thường được truy cập thông qua một primary key hoặc một secondary index, và các index này thường là các ánh xạ có thứ tự từ key đến bản ghi, đồng thời cũng hỗ trợ các truy vấn theo phạm vi (range queries).
- Các data warehouse và các hệ thống phân tích tương tự được tối ưu hóa cho các truy vấn đọc phức tạp thực hiện quét qua một lượng lớn các bản ghi. Chúng thường sử dụng bố cục lưu trữ column-oriented với cơ chế nén giúp giảm

thiểu lượng dữ liệu mà một truy vấn như vậy cần đọc từ đĩa, và sử dụng JIT compilation của các truy vấn hoặc vectorization để giảm thiểu thời gian CPU dành cho việc xử lý dữ liệu.

Về phía OLTP, chúng ta đã thấy các storage engine thuộc về hai trường phái tư duy chính:

- Cách tiếp cận log-structured, cho phép nối thêm vào các tệp và xóa các tệp đã lỗi thời nhưng không bao giờ cập nhật một tệp đã được ghi. Nhìn chung, các storage engine theo kiểu log-structured có xu hướng cung cấp throughput ghi cao. SSTables, LSM-trees, RocksDB, Cassandra, HBase, ScyllaDB, Lucene và các hệ thống khác thuộc nhóm này.
- Cách tiếp cận update-in-place, coi đĩa là một tập hợp các page có kích thước cố định có thể bị ghi đè. B-trees, ví dụ phổ biến nhất của triết lý này, được sử dụng trong tất cả các cơ sở dữ liệu quan hệ OLTP lớn và nhiều cơ sở dữ liệu nonrelational. Theo quy tắc kinh nghiệm, B-trees có xu hướng tốt hơn cho việc đọc, cung cấp throughput đọc cao hơn và thời gian phản hồi thấp hơn so với lưu trữ kiểu log-structured.

Sau đó, chúng ta đã xem xét các index có thể tìm kiếm nhiều điều kiện cùng một lúc: các multidimensional indexes như R-trees có thể tìm kiếm các điểm trên bản đồ theo vĩ độ và kinh độ cùng lúc, và các full-text search indexes có thể tìm kiếm nhiều từ khóa xuất hiện trong cùng một văn bản. Cuối cùng, chúng ta thấy rằng các vector database được sử dụng để semantic search trên các tài liệu văn bản và các phương tiện truyền thông khác; chúng sử dụng các vector với số lượng chiều lớn hơn và tìm các tài liệu tương tự bằng cách so sánh độ tương đồng vector.

Với tư cách là một lập trình viên ứng dụng, việc được trang bị kiến thức này về bên trong (internals) của các storage engine sẽ giúp bạn ở vị thế tốt hơn nhiều để biết công cụ nào phù hợp nhất cho ứng dụng cụ thể của mình. Nếu bạn cần điều chỉnh các tham số tuning của một cơ sở dữ liệu, sự hiểu biết này cho phép bạn hình dung được tác động của một giá trị cao hơn hoặc thấp hơn.

Mặc dù chương này không thể biến bạn thành một chuyên gia trong việc tuning bất kỳ storage engine cụ thể nào, nhưng hy vọng nó đã trang bị cho bạn đủ vốn từ vựng và ý tưởng để bạn có thể hiểu được tài liệu hướng dẫn của cơ sở dữ liệu mà bạn chọn.

Footnotes

References

[1] Nikolay Samokhvalov. “How Partial, Covering, and Multicolumn Indexes May Slow Down UPDATES in PostgreSQL.” *postgres.ai*, October 2021. Archived at perma.cc/PBK3-F4G9

- [2] Goetz Graefe. “Modern B-Tree Techniques.” *Foundations and Trends in Databases*, volume 3, issue 4, pages 203–402, August 2011. [doi:10.1561/19000000028](#)
- [3] Evan Jones. “Why Databases Use Ordered Indexes but Programming Uses Hash Tables.” *evanjones.ca*, December 2019. Archived at [perma.cc/NJX8-3ZZD](#)
- [4] Branimir Lambov. “CEP-25: Trie-Indexed SSTable Format.” *cwiki.apache.org*, November 2022. Archived at [perma.cc/HD7W-PW8U](#) (linked Google Doc archived at [perma.cc/UL6C-AAAE](#))
- [5] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*, 3rd edition. MIT Press, 2009. ISBN: 9780262533058
- [6] Branimir Lambov. “Trie Memtables in Cassandra.” *Proceedings of the VLDB Endowment*, volume 15, issue 12, pages 3359–3371, August 2022. [doi:10.14778/3554821.3554828](#)
- [7] Dhruba Borthakur. “The History of RocksDB.” *rocksdb.blogspot.com*, November 2013. Archived at [perma.cc/Z7C5-JPSP](#)
- [8] Matteo Bertozzi. “Apache HBase I/O—HFile.” *blog.cloudera.com*, June 2012. Archived at [perma.cc/U9XH-L2KL](#)
- [9] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E. Gruber. “Bigtable: A Distributed Storage System for Structured Data.” At *7th USENIX Symposium on Operating System Design and Implementation* (OSDI), November 2006.
- [10] Patrick O’Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O’Neil. “The Log-Structured Merge-Tree (LSM-Tree).” *Acta Informatica*, volume 33, issue 4, pages 351–385, June 1996. [doi:10.1007/s002360050048](#)
- [11] Mendel Rosenblum and John K. Ousterhout. “The Design and Implementation of a Log-Structured File System.” *ACM Transactions on Computer Systems*, volume 10, issue 1, pages 26–52, February 1992. [doi:10.1145/146941.146943](#)
- [12] Michael Armbrust, Tathagata Das, Liwen Sun, Burak Yavuz, Shixiong Zhu, Mukul Murthy, Joseph Torres, Herman van Hovell, Adrian Ionescu, Alicja Łuszczak, Michał Świtakowski, Michał Szafrński, Xiao Li, Takuya Ueshin, Mostafa Mokhtar, Peter Boncz, Ali Ghodsi, Sameer Paranjpye, Pieter Senster, Reynold Xin, and Matei Zaharia. “Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores.” *Proceedings of the VLDB Endowment*, volume 13, issue 12, pages 3411–3424, August 2020. [doi:10.14778/3415478.3415560](#)
- [13] Burton H. Bloom. “Space/Time Trade-offs in Hash Coding with Allowable Errors.” *Communications of the ACM*, volume 13, issue 7, pages 422–426, July 1970. [doi:10.1145/362686.362692](#)
- [14] Adam Kirsch and Michael Mitzenmacher. “Less Hashing, Same Performance: Building a Better Bloom Filter.” *Random Structures & Algorithms*, volume 33, issue 2, pages 187–218, September 2008. [doi:10.1002/rsa.20208](#)
- [15] Thomas Hurst. “Bloom Filter Calculator.” *hur.st*, September 2023. Archived at [perma.cc/L3AV-6VC2](#)
- [16] Chen Luo and Michael J. Carey. “LSM-Based Storage Techniques: a Survey.” *The VLDB Journal*, volume 29, pages 393–418, July 2019. [doi:10.1007/s00778-019-00555-y](#)
- [17] Subhadeep Sarkar and Manos Athanassoulis. “Dissecting, Designing, and Optimizing LSM-Based Data Stores.” Tutorial at *ACM International Conference on Management of Data* (SIGMOD), June 2022. Slides archived at [perma.cc/93B3-E827](#)

- [18] Mark Callaghan. “Name That Compaction Algorithm.” *smalldatum.blogspot.com*, August 2018. Archived at perma.cc/CN4M-82DY
- [19] Prashanth Rao. “Embedded Databases (1): The Harmony of DuckDB, KùzuDB and LanceDB.” *thedataquarry.com*, August 2023. Archived at perma.cc/PA28-2R35
- [20] Hacker News discussion. “Bluesky Migrates to Single-Tenant SQLite.” *news.ycombinator.com*, October 2023. Archived at perma.cc/69LM-5P6X
- [21] Rudolf Bayer and Edward M. McCreight. “Organization and Maintenance of Large Ordered Indices.” Boeing Scientific Research Laboratories, Mathematical and Information Sciences Laboratory, report no. 20, July 1970. [doi:10.1145/1734663.1734671](https://doi.org/10.1145/1734663.1734671)
- [22] Douglas Comer. “The Ubiquitous B-Tree.” *ACM Computing Surveys*, volume 11, issue 2, pages 121–137, June 1979. [doi:10.1145/356770.356776](https://doi.org/10.1145/356770.356776)
- [23] Alex Miller. “Torn Write Detection and Protection.” *transactional.blog*, April 2025. Archived at perma.cc/G7EB-33EW
- [24] C. Mohan and Frank Levine. “ARIES/IM: An Efficient and High Concurrency Index Management Method Using Write-Ahead Logging.” At *ACM International Conference on Management of Data (SIGMOD)*, June 1992. [doi:10.1145/130283.130338](https://doi.org/10.1145/130283.130338)
- [25] Hironobu Suzuki. “The Internals of PostgreSQL.” *interdb.jp*, 2017. Archived at archive.org
- [26] Howard Chu. “LDAP at Lightning Speed.” At *Build Stuff ’14*, November 2014. Archived at perma.cc/GB6Z-P8YH
- [27] Manos Athanassoulis, Michael S. Kester, Lukas M. Maas, Radu Stoica, Stratos Idreos, Anastasia Ailamaki, and Mark Callaghan. “Designing Access Methods: The RUM Conjecture.” At *19th International Conference on Extending Database Technology (EDBT)*, March 2016. [doi:10.5441/002/edbt.2016.42](https://doi.org/10.5441/002/edbt.2016.42)
- [28] Ben Stopford. “Log Structured Merge Trees.” *benstopford.com*, February 2015. Archived at perma.cc/E5BV-KUJ6
- [29] Mark Callaghan. “The Advantages of an LSM vs. a B-Tree.” *smalldatum.blogspot.co.uk*, January 2016. Archived at perma.cc/3TYZ-EFUD
- [30] Oana Balmau, Florin Dinu, Willy Zwaenepoel, Karan Gupta, Ravishankar Chandhiramoorthi, and Diego Didona. “SILK: Preventing Latency Spikes in Log-Structured Merge Key-Value Stores.” At *USENIX Annual Technical Conference*, July 2019.
- [31] Igor Canadi, Siying Dong, Mark Callaghan, et al. “RocksDB Tuning Guide.” *github.com*, 2023. Archived at perma.cc/UNY4-MK6C
- [32] Gabriel Haas and Viktor Leis. “What Modern NVMe Storage Can Do, and How to Exploit It: High-Performance I/O for High-Performance Storage Engines.” *Proceedings of the VLDB Endowment*, volume 16, issue 9, pages 2090–2102. May 2023. [doi:10.14778/3598581.3598584](https://doi.org/10.14778/3598581.3598584)
- [33] Emmanuel Goossaert. “Coding for SSDs.” *codecapsule.com*, February 2014.
- [34] Jack Vanlightly. “Is Sequential IO Dead in the Era of the NVMe Drive?” *jack-vanlightly.com*, May 2023. Archived at perma.cc/7TMZ-TAPU
- [35] Alibaba Cloud Storage Team. “Storage System Design Analysis: Factors Affecting NVMe SSD Performance (2).” *alibabacloud.com*, January 2019. Archived at archive.org
- [36] Xiao-Yu Hu and Robert Haas. “The Fundamental Limit of Flash Random Write Performance: Understanding, Analysis and Performance Modelling.” *dominoweb.draco.res.ibm.com*, March 2010. Archived at perma.cc/8JUL-4ZDS

- [37] Lanyue Lu, Thanumalayan Sankaranarayanan Pillai, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. “WiscKey: Separating Keys from Values in SSD-Conscious Storage.” At *4th USENIX Conference on File and Storage Technologies (FAST)*, February 2016.
- [38] Peter Zaitsev. “InnoDB Double Write.” *percona.com*, August 2006. Archived at perma.cc/NT4S-DK7T
- [39] Tomas Vondra. “On the Impact of Full-Page Writes.” *2ndquadrant.com*, November 2016. Archived at perma.cc/7N6B-CVL3
- [40] Mark Callaghan. “Read, Write & Space Amplification—B-Tree vs. LSM.” *smalldatum.blogspot.com*, November 2015. Archived at perma.cc/S487-WK5P
- [41] Mark Callaghan. “Choosing Between Efficiency and Performance with RocksDB.” At *Code Mesh*, November 2016
- [42] Subhadeep Sarkar, Tarikul Islam Papon, Dimitris Staratzis, Zichen Zhu, and Manos Athanassoulis. “Enabling Timely and Persistent Deletion in LSM-Engines.” *ACM Transactions on Database Systems*, volume 48, issue 3, article no. 8, August 2023. [doi:10.1145/3599724](https://doi.org/10.1145/3599724)
- [43] Lukas Fittl. “Postgres vs. SQL Server: B-Tree Index Differences & the Benefit of Deduplication.” *pganalyze.com*, April 2025. Archived at perma.cc/XY6T-LTPX
- [44] Drew Silcock. “How Postgres Stores Data on Disk—This One’s a Page Turner.” *drew.silcock.dev*, August 2024. Archived at perma.cc/8K7K-7VJ2
- [45] Joe Webb. “Using Covering Indexes to Improve Query Performance.” *simple-talk.com*, September 2008. Archived at perma.cc/6MEZ-R5VR
- [46] Michael Stonebraker, Samuel Madden, Daniel J. Abadi, Stavros Harizopoulos, Nabil Hachem, and Pat Helland. “The End of an Architectural Era (It’s Time for a Complete Rewrite).” At *33rd International Conference on Very Large Data Bases (VLDB)*, September 2007.
- [47] “VoltDB Technical Overview White Paper.” VoltDB, 2017. Archived at perma.cc/B9SF-SK5G
- [48] Stephen M. Rumble, Ankita Kejriwal, and John K. Ousterhout. “Log-Structured Memory for DRAM-Based Storage.” At *12th USENIX Conference on File and Storage Technologies (FAST)*, February 2014.
- [49] Stavros Harizopoulos, Daniel J. Abadi, Samuel Madden, and Michael Stonebraker. “OLTP Through the Looking Glass, and What We Found There.” At *ACM International Conference on Management of Data (SIGMOD)*, June 2008. [doi:10.1145/1376616.1376713](https://doi.org/10.1145/1376616.1376713)
- [50] Per-Åke Larson, Cipri Clinciu, Campbell Fraser, Eric N. Hanson, Mostafa Mokhtar, Michal Nowakiewicz, Vassilis Papadimos, Susan L. Price, Srikumar Rangarajan, Remus Rusanu, and Mayukh Saubhasik. “Enhancements to SQL Server Column Stores.” At *ACM International Conference on Management of Data (SIGMOD)*, June 2013. [doi:10.1145/2463676.2463708](https://doi.org/10.1145/2463676.2463708)
- [51] Franz Färber, Norman May, Wolfgang Lehner, Philipp Große, Ingo Müller, Hannes Rauhe, and Jonathan Dees. “The SAP HANA Database—An Architecture Overview.” *IEEE Data Engineering Bulletin*, volume 35, issue 1, pages 28–33, March 2012. Archived at perma.cc/H2WC-YQZY
- [52] Michael Stonebraker. “The Traditional RDBMS Wisdom Is (Almost Certainly) All Wrong.” Presentation at *EPFL*, May 2013.
- [53] Adam Prout, Szu-Po Wang, Joseph Victor, Zhou Sun, Yongzhu Li, Jack Chen, Evan Bergeron, Eric Hanson, Robert Walzer, Rodrigo Gomes, and Nikita Shamgunov. “Cloud-Native Transactions and Analytics in SingleStore.” At *ACM International Conference on Management of Data (SIGMOD)*, June 2022. [doi:10.1145/3514221.3526055](https://doi.org/10.1145/3514221.3526055)

- [54] Tino Tereshko and Jordan Tigani. “BigQuery Under the Hood.” *cloud.google.com*, January 2016. Archived at perma.cc/WP2Y-FUCF
- [55] Wes McKinney. “The Road to Composable Data Systems: Thoughts on the Last 15 Years and the Future.” *wesmcinney.com*, September 2023. Archived at perma.cc/6L2M-GTJX
- [56] Michael Stonebraker, Daniel J. Abadi, Adam Batkin, Xuedong Chen, Mitch Cherniack, Miguel Ferreira, Edmond Lau, Amerson Lin, Sam Madden, Elizabeth O’Neil, Pat O’Neil, Alex Rasin, Nga Tran, and Stan Zdonik. “C-Store: A Column-Oriented DBMS.” At *31st International Conference on Very Large Data Bases (VLDB)*, September 2005.
- [57] Julien Le Dem. “Dremel Made Simple with Parquet.” *blog.x.com*, September 2013. Archived at archive.org
- [58] Sergey Melnik, Andrey Gubarev, Jing Jing Long, Geoffrey Romer, Shiva Shivakumar, Matt Tolton, and Theo Vassilakis. “Dremel: Interactive Analysis of Web-Scale Datasets.” At *36th International Conference on Very Large Data Bases (VLDB)*, September 2010. [doi:10.14778/1920841.1920886](https://doi.org/10.14778/1920841.1920886)
- [59] Joe Kearney. “Understanding Record Shredding: Storing Nested Data in Columns.” *joekearney.co.uk*, December 2016. Archived at perma.cc/ZD5N-AX5D
- [60] Jamie Brandon. “A Shallow Survey of OLAP and HTAP Query Engines.” *scattered-thoughts.net*, September 2023. Archived at perma.cc/L3KH-J4JF
- [61] Benoit Dageville, Thierry Cruanes, Marcin Zukowski, Vadim Antonov, Artin Avanes, Jon Bock, Jonathan Claybaugh, Daniel Engovatov, Martin Hentschel, Jiansheng Huang, Allison W. Lee, Ashish Motivala, Abdul Q. Munir, Steven Pelley, Peter Povinec, Greg Rahn, Spyridon Triantafyllis, and Philipp Unterbrunner. “The Snowflake Elastic Data Warehouse.” At *ACM International Conference on Management of Data (SIGMOD)*, June 2016. [doi:10.1145/2882903.2903741](https://doi.org/10.1145/2882903.2903741)
- [62] Mark Raasveldt and Hannes Mühleisen. “Data Management for Data Science Towards Embedded Analytics.” At *10th Conference on Innovative Data Systems Research (CIDR)*, January 2020. Archived at perma.cc/65G2-NYDT
- [63] Jean-François Im, Kishore Gopalakrishna, Subbu Subramaniam, Mayank Shrivastava, Adwait Tumbde, Xiaotian Jiang, Jennifer Dai, Seunghyun Lee, Neha Pawar, Jialiang Li, and Ravi Aringunram. “Pinot: Realtime OLAP for 530 Million Users.” At *ACM International Conference on Management of Data (SIGMOD)*, May 2018. [doi:10.1145/3183713.3190661](https://doi.org/10.1145/3183713.3190661)
- [64] Fangjin Yang, Eric Tschetter, Xavier Léauté, Nelson Ray, Gian Merlino, and Deep Ganguli. “Druid: A Real-Time Analytical Data Store.” At *ACM International Conference on Management of Data (SIGMOD)*, June 2014. [doi:10.1145/2588555.2595631](https://doi.org/10.1145/2588555.2595631)
- [65] Chunwei Liu, Anna Pavlenko, Matteo Interlandi, and Brandon Haynes. “Deep Dive into Common Open Formats for Analytical DBMSs.” *Proceedings of the VLDB Endowment*, volume 16, issue 11, pages 3044–3056, July 2023. [doi:10.14778/3611479.3611507](https://doi.org/10.14778/3611479.3611507)
- [66] Xinyu Zeng, Yulong Hui, Jiahong Shen, Andrew Pavlo, Wes McKinney, and Huanchen Zhang. “An Empirical Evaluation of Columnar Storage Formats.” *Proceedings of the VLDB Endowment*, volume 17, issue 2, pages 148–161. [doi:10.14778/3626292.3626298](https://doi.org/10.14778/3626292.3626298)
- [67] Weston Pace. “Lance v2: A Columnar Container Format for Modern Data.” *blog.lancedb.com*, April 2024. Archived at perma.cc/ZK3Q-S9VJ
- [68] Yoav Helfman. “Nimble, A New Columnar File Format.” At *VeloxCon*, April 2024.
- [69] Wes McKinney. “Apache Arrow: High-Performance Columnar Data Framework.” At *CMU Database Group—Vaccination Database Tech Talks*, December 2021.

- [70] Wes McKinney. *Python for Data Analysis*, 3rd edition. O'Reilly Media, 2022. ISBN: 9781098104023
- [71] Paul Dix. “The Design of InfluxDB IOx: An In-Memory Columnar Database Written in Rust with Apache Arrow.” At *CMU Database Group—Vaccination Database Tech Talks*, May 2021.
- [72] Carlota Soto and Mike Freedman. “Building Columnar Compression for Large PostgreSQL Databases.” *timescale.com*, March 2024. Archived at perma.cc/7KTF-V3EH
- [73] Daniel J. Abadi, Peter Boncz, Stavros Harizopoulos, Stratos Idreos, and Samuel Madden. “The Design and Implementation of Modern Column-Oriented Database Systems.” *Foundations and Trends in Databases*, volume 5, issue 3, pages 197–280, December 2013. [doi:10.1561/19000000024](https://doi.org/10.1561/19000000024)
- [74] Daniel Lemire, Gregory Ssi-Yan-Kai, and Owen Kaser. “Consistently Faster and Smaller Compressed Bitmaps with Roaring.” *Software: Practice and Experience*, volume 46, issue 11, pages 1547–1569, November 2016. [doi:10.1002/spe.2402](https://doi.org/10.1002/spe.2402)
- [75] Jaz Volpert. “An Entire Social Network in 1.6GB (GraphD Part 2).” *jazco.dev*, April 2024. Archived at perma.cc/L27Z-QVMG
- [76] Andrew Lamb, Matt Fuller, Ramakrishna Varadarajan, Nga Tran, Ben Vandiver, Lyric Doshi, and Chuck Bear. “The Vertica Analytic Database: C-Store 7 Years Later.” *Proceedings of the VLDB Endowment*, volume 5, issue 12, pages 1790–1801, August 2012. [doi:10.14778/2367502.2367518](https://doi.org/10.14778/2367502.2367518)
- [77] Timo Kersten, Viktor Leis, Alfons Kemper, Thomas Neumann, Andrew Pavlo, and Peter Boncz. “Everything You Always Wanted to Know About Compiled and Vectorized Queries But Were Afraid to Ask.” *Proceedings of the VLDB Endowment*, volume 11, issue 13, pages 2209–2222, September 2018. [doi:10.14778/3275366.3284966](https://doi.org/10.14778/3275366.3284966)
- [78] Forrest Smith. “Memory Bandwidth Napkin Math.” *forrestthewoods.com*, February 2020. Archived at perma.cc/Y8U4-PS7N
- [79] Peter Boncz, Marcin Zukowski, and Niels Nes. “MonetDB/X100: Hyper-Pipelining Query Execution.” At *2nd Biennial Conference on Innovative Data Systems Research (CIDR)*, January 2005. Archived at perma.cc/R4KF-QKHF
- [80] Jingren Zhou and Kenneth A. Ross. “Implementing Database Operations Using SIMD Instructions.” At *ACM International Conference on Management of Data (SIGMOD)*, June 2002. [doi:10.1145/564691.564709](https://doi.org/10.1145/564691.564709)
- [81] Kevin Bartley. “OLTP Queries: Transfer Expensive Workloads to Materialize.” *materialize.com*, August 2024. Archived at perma.cc/4TYM-TYD8
- [82] Jim Gray, Surajit Chaudhuri, Adam Bosworth, Andrew Layman, Don Reichart, Murali Venkatrao, Frank Pellow, and Hamid Pirahesh. “Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals.” *Data Mining and Knowledge Discovery*, volume 1, issue 1, pages 29–53, March 2007. [doi:10.1023/A:1009726021843](https://doi.org/10.1023/A:1009726021843)
- [83] Frank Ramsak, Volker Markl, Robert Fenk, Martin Zirkel, Klaus Elhardt, and Rudolf Bayer. “Integrating the UB-Tree into a Database System Kernel.” At *26th International Conference on Very Large Data Bases (VLDB)*, September 2000.
- [84] Octavian Procopiuc, Pankaj K. Agarwal, Lars Arge, and Jeffrey Scott Vitter. “Bkd-Tree: A Dynamic Scalable kd-Tree.” At *8th International Symposium on Spatial and Temporal Databases (SSTD)*, July 2003. [doi:10.1007/978-3-540-45072-6_4](https://doi.org/10.1007/978-3-540-45072-6_4)
- [85] Joseph M. Hellerstein, Jeffrey F. Naughton, and Avi Pfeffer. “Generalized Search Trees for Database Systems.” At *21st International Conference on Very Large Data Bases (VLDB)*, September 1995.

- [86] Isaac Brodsky. “H3: Uber’s Hexagonal Hierarchical Spatial Index.” *eng.uber.com*, June 2018. Archived at archive.org
- [87] Robert Escriva, Bernard Wong, and Emin Gün Sirer. “HyperDex: A Distributed, Searchable Key-Value Store.” At *ACM SIGCOMM Conference*, August 2012. doi:10.1145/2377677.2377681
- [88] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008. ISBN: 9780521865715. Available online at nlp.stanford.edu/IR-book.
- [89] Jianguo Wang, Chunbin Lin, Yannis Papakonstantinou, and Steven Swanson. “An Experimental Study of Bitmap Compression vs. Inverted List Compression.” At *ACM International Conference on Management of Data (SIGMOD)*, May 2017. doi:10.1145/3035918.3064007
- [90] Adrien Grand. “What Is in a Lucene Index?” At *Lucene/Solr Revolution*, November 2013. Archived at perma.cc/Z7QN-GBYY
- [91] Michael McCandless. “Visualizing Lucene’s Segment Merges.” *blog.mikemccandless.com*, February 2011. Archived at perma.cc/3ZV8-72W6
- [92] Lukas Fittl. “Understanding Postgres GIN Indexes: The Good and the Bad.” *pganalyze.com*, December 2021. Archived at perma.cc/V3MW-26H6
- [93] Jimmy Angelakos. “The State of (Full) Text Search in PostgreSQL 12.” At *FOSDEM*, February 2020. Archived at perma.cc/J6US-3WZS
- [94] Alexander Korotkov. “Index Support for Regular Expression Search.” At *PGConf.EU Prague*, October 2012. Archived at perma.cc/SRFZ-ZKDQ
- [95] Michael McCandless. “Lucene’s FuzzyQuery Is 100 Times Faster in 4.0.” *blog.mikemccandless.com*, March 2011. Archived at perma.cc/E2WC-GHTW
- [96] Steffen Heinz, Justin Zobel, and Hugh E. Williams. “Burst Tries: A Fast, Efficient Data Structure for String Keys.” *ACM Transactions on Information Systems*, volume 20, issue 2, pages 192–223, April 2002. doi:10.1145/506309.506312
- [97] Klaus U. Schulz and Stoyan Mihov. “Fast String Correction with Levenshtein Automata.” *International Journal on Document Analysis and Recognition*, volume 5, issue 1, pages 67–85, November 2002. doi:10.1007/s10032-002-0082-8
- [98] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. “Efficient Estimation of Word Representations in Vector Space.” *arXiv:1301.3781*, September 2013
- [99] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. “BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding.” At *Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, June 2019. doi:10.18653/v1/N19-1423
- [100] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. “Improving Language Understanding by Generative Pre-Training.” *openai.com*, June 2018. Archived at perma.cc/5N3C-DJ4C
- [101] Matthijs Douze, Maria Lomeli, and Lucas Hosseini. “Faiss Indexes.” *github.com*, August 2024. Archived at perma.cc/2EWG-FPBS
- [102] Varik Matevosyan. “Understanding pgvector’s HNSW Index Storage in Postgres.” *lantern.dev*, August 2024. Archived at perma.cc/B2YB-JB59
- [103] Dmitry Baranchuk, Artem Babenko, and Yury Malkov. “Revisiting the Inverted Indices for Billion-Scale Approximate Nearest Neighbors.” At *European Conference on Computer Vision (ECCV)*, September 2018. doi:10.1007/978-3-030-01258-8_13

[104] Yury A. Malkov and Dmitry A. Yashunin. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs.” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, volume 42, issue 4, pages 824–836, April 2020. [doi:10.1109/TPAMI.2018.2889473](https://doi.org/10.1109/TPAMI.2018.2889473)

Encoding và Evolution

Mọi thứ đều thay đổi và không có gì đứng yên.

Heraclitus vùng Ephesus, như Plato đã trích dẫn trong *Cratylus* (360 BCE)

Các ứng dụng chắc chắn sẽ thay đổi theo thời gian. Các tính năng được thêm vào hoặc sửa đổi khi các sản phẩm mới được ra mắt, yêu cầu của người dùng được hiểu rõ hơn, hoặc các điều kiện kinh doanh thay đổi. Trong Chương 2, chúng ta đã giới thiệu khái niệm về *khả năng tiến hóa* (*evolvability*): chúng ta nên hướng tới việc xây dựng các hệ thống giúp việc thích ứng với sự thay đổi trở nên dễ dàng (xem “Evolvability: Making Change Easy”).

Trong hầu hết các trường hợp, một thay đổi đối với các tính năng của ứng dụng cũng đòi hỏi sự thay đổi đối với dữ liệu mà nó lưu trữ. Có lẽ một `field` hoặc loại `record` mới cần được ghi lại, hoặc dữ liệu hiện có cần được trình bày theo một cách mới.

Các mô hình dữ liệu mà chúng ta đã thảo luận trong Chương 3 có những cách khác nhau để đối phó với sự thay đổi như vậy. Các cơ sở dữ liệu relational thường giả định rằng tất cả dữ liệu trong cơ sở dữ liệu đều tuân theo một schema duy nhất. Mặc dù schema đó có thể được thay đổi (thông qua các `schema migrations`; tức là các câu lệnh `ALTER`), nhưng tại bất kỳ thời điểm nào cũng chỉ có duy nhất một schema có hiệu lực. Ngược lại, các cơ sở dữ liệu `schema-on-read` (“`schemaless`”) không áp đặt một schema, vì vậy cơ sở dữ liệu có thể chứa sự pha trộn giữa các định dạng dữ liệu cũ và mới được ghi vào các thời điểm khác nhau (xem “`Schema flexibility in the document model`”).

Khi một định dạng dữ liệu hoặc schema thay đổi, một thay đổi tương ứng đối với mã nguồn ứng dụng thường cần phải diễn ra (ví dụ: bạn thêm một `field` mới vào một `record`, và mã nguồn ứng dụng bắt đầu đọc và ghi `field` đó). Tuy nhiên, trong một ứng dụng lớn, các thay đổi mã nguồn thường không thể diễn ra tức thời vì nhiều lý do khác nhau. Ví dụ:

- Với các ứng dụng chạy phía server, bạn có thể muốn thực hiện một *rolling upgrade* (còn được gọi là *staged rollout*), triển khai phiên bản mới cho một vài node cùng một lúc, giám sát xem nó có chạy mượt mà không, và dần dần thực hiện cho tất cả các node. Điều này cho phép các phiên bản mới được triển khai mà không gây downtime cho dịch vụ, từ đó khuyến khích việc phát hành thường xuyên hơn và khả năng tiến hóa tốt hơn.

- Với các ứng dụng chạy phía client, bạn hoàn toàn phụ thuộc vào người dùng, những người có thể sẽ không cài đặt bản cập nhật trong một khoảng thời gian.

Điều này có nghĩa là các phiên bản cũ và mới của mã nguồn, cũng như các định dạng dữ liệu cũ và mới, có khả năng cùng tồn tại trong hệ thống tại cùng một thời điểm. Để hệ thống tiếp tục chạy mượt mà, bạn cần duy trì tính tương thích theo cả hai hướng:

Backward compatibility

Đảm bảo rằng mã nguồn mới hơn có thể đọc được dữ liệu được ghi bởi mã nguồn cũ hơn

Forward compatibility

Đảm bảo rằng mã nguồn cũ hơn có thể đọc được dữ liệu được ghi bởi mã nguồn mới hơn

Trong ngữ cảnh của các API, nếu bạn muốn một client cũ có thể gọi thành công một service mới hơn, bạn cần backward compatibility đối với request và forward compatibility đối với response. Để một client mới hơn gọi một service cũ hơn, bạn cần forward compatibility đối với request và backward compatibility đối với response.

Backward compatibility thường không khó để đạt được. Với tư cách là tác giả của mã nguồn mới hơn, bạn biết định dạng dữ liệu được ghi bởi mã nguồn cũ hơn, vì vậy bạn có thể xử lý nó một cách rõ ràng (nếu cần, chỉ đơn giản là giữ lại mã nguồn cũ để đọc dữ liệu cũ). Forward compatibility có thể khó khăn hơn, vì nó yêu cầu mã nguồn cũ hơn phải bỏ qua những phần được thêm vào bởi một phiên bản mã nguồn mới hơn.

Một thách thức khác đối với forward compatibility được minh họa trong Hình 5-1. Giả sử bạn thêm một `field` vào một `record` schema, và mã nguồn mới hơn tạo ra một `record` chứa `field` mới đó và lưu nó vào cơ sở dữ liệu. Sau đó, một phiên bản mã nguồn cũ hơn (vốn chưa biết về `field` mới) đọc `record` đó, cập nhật nó và ghi ngược trở lại. Trong tình huống này, hành vi mong muốn thường là mã nguồn cũ giữ nguyên `field` mới đó, mặc dù nó không thể giải mã được. Nhưng nếu `record` được giải mã thành một mô hình object mà không lưu giữ rõ ràng các `field` không xác định, dữ liệu có thể bị mất, như được hiển thị ở đây.

Trong chương này, chúng ta sẽ xem xét một số định dạng để encoding dữ liệu, bao gồm JSON, XML, Protocol Buffers và Avro. Cụ thể, chúng ta sẽ tìm hiểu cách chúng xử lý các thay đổi về schema và cách chúng hỗ trợ các hệ thống cần sự cùng tồn tại của dữ liệu và mã nguồn cũ và mới. Sau đó, chúng ta sẽ thảo luận về cách các định dạng đó được sử dụng để lưu trữ dữ liệu và truyền thông: trong các cơ sở dữ liệu, web services, REST APIs, remote procedure calls (RPCs), workflow engines và các hệ thống event-driven như actors và message queues.

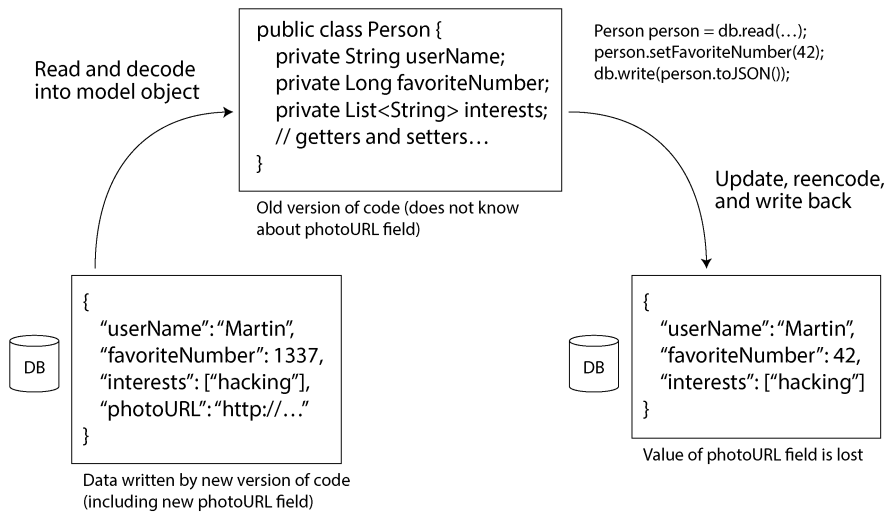


Figure 5-1. *Khi một phiên bản cũ hơn của ứng dụng cập nhật dữ liệu vốn đã được viết bởi một phiên bản mới hơn của ứng dụng, dữ liệu có thể bị mất nếu bạn không cẩn thận.*

Các định dạng để encoding dữ liệu

Các chương trình thường làm việc với dữ liệu dưới (ít nhất) hai dạng biểu diễn:

- Trong bộ nhớ, dữ liệu được lưu giữ trong các object, struct, list, array, hash table, tree, v.v. Các cấu trúc dữ liệu này được tối ưu hóa để CPU có thể truy cập và thao tác một cách hiệu quả (thường sử dụng con trỏ).
- Khi bạn muốn ghi dữ liệu vào một tệp hoặc gửi nó qua mạng, bạn phải encoding nó thành một dạng chuỗi byte độc lập (ví dụ: một tài liệu JSON). Vì một con trỏ sẽ không có ý nghĩa đối với bất kỳ tiến trình nào khác, dạng biểu diễn chuỗi byte này thường trông rất khác so với các cấu trúc dữ liệu thường được sử dụng trong bộ nhớ.

Do đó, chúng ta cần một loại chuyển đổi nào đó giữa hai dạng biểu diễn này. Việc chuyển đổi từ dạng biểu diễn trong bộ nhớ sang một chuỗi byte được gọi là *encoding* (còn được gọi là *serialization* hoặc *marshaling*), và quá trình ngược lại được gọi là *decoding* (hay còn gọi là *parsing*, *deserialization*, hoặc *unmarshaling*).

Xung đột thuật ngữ

Thật không may, thuật ngữ *serialization* cũng được sử dụng trong ngữ cảnh của các *transaction* (xem Chương 8), với một ý nghĩa hoàn toàn khác. Để tránh việc gây nhầm lẫn cho thuật ngữ này, chúng ta sẽ chỉ sử dụng *encoding* trong cuốn sách này, mặc dù *serialization* có lẽ phổ biến hơn.

Đôi khi việc encoding/decoding là không cần thiết—ví dụ, khi một cơ sở dữ liệu hoạt động trực tiếp trên dữ liệu đã được nén được tải từ đĩa, như đã thảo luận trong mục “Query Execution: Compilation and Vectorization”. Cũng có những định dạng dữ liệu *zero-copy* được thiết kế để có thể sử dụng cả khi runtime và trên đĩa/trên mạng mà không cần bước chuyển đổi tường minh, chẳng hạn như Cap’n Proto và FlatBuffers.

Tuy nhiên, hầu hết các hệ thống đều cần chuyển đổi giữa các *object* trong bộ nhớ và các chuỗi byte phẳng. Vì đây là một vấn đề rất phổ biến, nên có vô số thư viện và định dạng encoding để lựa chọn. Hãy cùng điểm qua một lượt ngắn gọn.

Các định dạng đặc thù theo ngôn ngữ

Nhiều ngôn ngữ lập trình đi kèm với sự hỗ trợ tích hợp sẵn để encoding các *object* trong bộ nhớ thành các chuỗi byte. Ví dụ, Java có `java.io.Serializable`, Python có `pickle`, và Ruby có `Marshal`. Cũng có nhiều thư viện bên thứ ba, chẳng hạn như Kryo dành cho Java.

Các thư viện encoding này rất tiện lợi, vì chúng cho phép các *object* trong bộ nhớ được lưu trữ và khôi phục với lượng mã bổ sung tối thiểu. Tuy nhiên, chúng cũng gặp phải một số vấn đề nghiêm trọng:

- Việc encoding thường gắn liền với một ngôn ngữ lập trình cụ thể, và việc đọc dữ liệu bằng một ngôn ngữ khác là rất khó khăn. Nếu bạn lưu trữ hoặc truyền tải dữ liệu bằng cách encoding như vậy, bạn đang tự ràng buộc mình với ngôn ngữ lập trình hiện tại trong một thời gian dài và ngăn cản việc tích hợp hệ thống của bạn với hệ thống của các tổ chức khác (những bên có thể sử dụng các ngôn ngữ khác nhau).
- Để khôi phục dữ liệu về cùng các kiểu *object* đó, quá trình decoding cần có khả năng khởi tạo các *class* bất kỳ. Đây thường là nguồn cơn của các vấn đề bảo mật [1]; nếu một kẻ tấn công có thể khiến ứng dụng của bạn decode một chuỗi byte bất kỳ, chúng có thể khởi tạo các *class* bất kỳ, từ đó thường cho phép chúng thực hiện những hành vi khủng khiếp như thực thi mã tùy ý từ xa [2, 3].
- Việc quản lý phiên bản dữ liệu thường không được chú trọng trong các thư viện này. Vì chúng được thiết kế để encoding dữ liệu nhanh chóng và dễ dàng, chúng

thường bỏ qua các vấn đề bất tiện về tính tương thích xuôi (forward compatibility) và tương thích ngược (backward compatibility) [4].

- Hiệu năng (thời gian CPU dùng để encode hoặc decode, và kích thước của cấu trúc đã được encode) cũng thường không được chú trọng. Ví dụ, tính năng `serialization` tích hợp sẵn của Java nổi tiếng với hiệu năng kém và encoding cồng kềnh [5].

Vì những lý do này, việc sử dụng encoding tích hợp sẵn của ngôn ngữ cho bất kỳ mục đích nào khác ngoài các mục đích tạm thời (transient) thường là một ý tưởng tồi.

JSON, XML và các biến thể nhị phân

Khi chuyển sang các định dạng encoding tiêu chuẩn có thể được viết và đọc bởi nhiều ngôn ngữ lập trình, JSON và XML là những ứng cử viên hiển nhiên: chúng được biết đến rộng rãi và được hỗ trợ rộng rãi. CSV là một định dạng phổ biến khác không phụ thuộc vào ngôn ngữ, nhưng nó chỉ hỗ trợ dữ liệu dạng bảng mà không có cấu trúc phân cấp (nesting).

JSON, XML và CSV là các định dạng văn bản, và do đó chúng ở mức độ nào đó có thể đọc được bởi con người mặc dù cú pháp của chúng là một chủ đề gây tranh cãi phổ biến. Bên cạnh các vấn đề cú pháp bề mặt, chúng còn gặp phải nhiều vấn đề khác:

- XML thường bị chỉ trích vì quá rườm rà và phức tạp một cách không cần thiết [6].
- Có rất nhiều sự mơ hồ xung quanh việc mã hóa các con số. Trong XML và CSV, bạn không thể phân biệt giữa một con số và một chuỗi ký tự tình cờ bao gồm các chữ số (ngoại trừ việc tham chiếu đến một schema bên ngoài). JSON phân biệt được chuỗi và số, nhưng nó không phân biệt giữa số nguyên (`integer`) và số thực dấu phẩy động (`floating-point number`), và nó cũng không chỉ định độ chính xác.

Đây là một vấn đề khi xử lý các số lớn—ví dụ, các số nguyên lớn hơn 2^{53} không thể được biểu diễn chính xác trong một số thực dấu phẩy động độ chính xác kép (`double-precision floating-point number`) theo chuẩn IEEE 754, vì vậy các số như vậy trở nên không chính xác khi được parse trong một ngôn ngữ sử dụng số thực dấu phẩy động, chẳng hạn như JavaScript [7]. Một ví dụ về các số lớn hơn 2^{53} xuất hiện trên X, nơi sử dụng một số 64-bit để định danh mỗi bài đăng. JSON được trả về bởi API bao gồm các ID bài đăng hai lần, một lần dưới dạng số JSON và một lần dưới dạng chuỗi thập phân, để khắc phục việc parse số không chính xác bởi các ứng dụng JavaScript [8].

- JSON và XML hỗ trợ tốt cho các chuỗi ký tự Unicode (tức là văn bản có thể đọc được bởi con người), nhưng chúng không hỗ trợ các chuỗi nhị phân (binary strings — các chuỗi byte không có mã hóa ký tự). Chuỗi nhị phân là một feature hữu ích, vì vậy mọi người thường vượt qua hạn chế này bằng cách mã hóa dữ liệu nhị phân thành văn bản bằng cách sử dụng Base64. Sau đó, schema được sử dụng để chỉ ra rằng giá trị đó nên được diễn giải là dữ liệu đã được mã hóa Base64. Cách này hoạt động, nhưng nó hơi mang tính
- XML Schema và JSON Schema rất mạnh mẽ và do đó khá phức tạp để học và triển khai. Vì việc diễn giải chính xác dữ liệu (chẳng hạn như các con số và chuỗi nhị phân) phụ thuộc vào thông tin trong schema, các ứng dụng không sử dụng XML/JSON Schema có thể cần phải hardcode logic mã hóa/giải mã phù hợp thay thế.
- CSV không có bất kỳ schema nào, vì vậy việc định nghĩa ý nghĩa của mỗi hàng và cột phụ thuộc vào ứng dụng. Nếu một thay đổi trong ứng dụng thêm vào một hàng hoặc cột mới, bạn phải xử lý thay đổi đó một cách thủ công. CSV cũng là một định dạng khá mơ hồ (điều gì sẽ xảy ra nếu một giá trị chứa dấu phẩy hoặc ký tự xuống dòng?). Mặc dù các quy tắc escaping của nó đã được chỉ định chính thức [9], không phải tất cả các parser đều triển khai chúng một cách chính xác.

Bất chấp những thiếu sót này, JSON, XML và CSV vẫn đủ tốt cho nhiều mục đích. Chúng có khả năng sẽ tiếp tục phổ biến, đặc biệt là với vai trò là các định dạng trao đổi dữ liệu (tức là để gửi dữ liệu từ tổ chức này sang tổ chức khác). Trong những tình huống này, miễn là mọi người thống nhất về định dạng, việc nó đẹp hay hiệu quả đến mức nào thường không quan trọng. Khó khăn trong việc khiến các tổ chức khác nhau thống nhất về *bất cứ điều gì* còn lớn hơn hầu hết các mối quan tâm khác.

JSON Schema

JSON Schema đã được áp dụng rộng rãi như một cách để mô hình hóa dữ liệu bất cứ khi nào dữ liệu được trao đổi giữa các hệ thống hoặc được ghi vào storage. Bạn sẽ tìm thấy các JSON Schema trong các web service (xem mục “Web services”) như một phần của đặc tả web service OpenAPI, trong các schema registry như Schema Registry của Confluent và Apicurio Registry của Red Hat, và trong các cơ sở dữ liệu (ví dụ: extension validator pg_jsonschema của PostgreSQL và cú pháp validator \$jsonSchema của MongoDB).

Đặc tả JSON Schema cung cấp một số feature. Các schema bao gồm các kiểu nguyên thủy tiêu chuẩn như string, number, integer, object, array, boolean, và null. Nhưng JSON Schema cũng cung cấp một đặc tả validation riêng biệt cho phép các lập trình viên áp đặt các ràng buộc lên các field. Ví dụ, một field port có thể có giá trị tối thiểu là 1 và tối đa là 65,535.

Các mô hình nội dung mở rất mạnh mẽ, nhưng chúng có thể phức tạp. Ví dụ, giả sử bạn muốn định nghĩa một map từ các số nguyên (chẳng hạn như ID) sang các chuỗi (string). JSON không có kiểu map hoặc dictionary cho phép sử dụng key là số nguyên; các JSON object luôn sử dụng string làm key. Để đáp ứng nhu cầu của bạn, bạn có thể ràng buộc kiểu này bằng JSON Schema sao cho các key chỉ có thể chứa các chữ số và các value chỉ có thể là string bằng cách sử dụng `patternProperties` và `additionalProperties`, như được trình bày trong Ví dụ 5-1.

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "patternProperties": {
    "^[0-9]+$": {
      "type": "string"
    }
  },
  "additionalProperties": false
}
```

Mã hóa nhị phân (Binary encodings)

doctrans.ink

Infoset). Các định dạng này đã được áp dụng trong nhiều lĩnh vực khác nhau, vì chúng gọn nhẹ hơn và đôi khi parse nhanh hơn, nhưng không có định dạng nào được áp dụng rộng rãi như các phiên bản dạng văn bản của JSON và XML [13].

Một số định dạng này mở rộng tập hợp các kiểu dữ liệu (ví dụ: phân biệt giữa số nguyên và số thực dấu phẩy động, hoặc thêm hỗ trợ cho chuỗi nhị phân), nhưng ngoài ra chúng vẫn giữ nguyên mô hình dữ liệu JSON/XML. Đặc biệt, vì chúng không quy định một schema, chúng cần phải bao gồm tất cả tên `field` của `object` bên trong dữ liệu đã được mã hóa. Nghĩa là, trong một mã hóa nhị phân của tài liệu JSON trong Ví dụ 5-2, chúng sẽ cần phải bao gồm các chuỗi `userName`, `favoriteNumber`, và `interests` ở đầu đó.

Example 5-2. Một bản ghi mà chúng ta sẽ mã hóa dưới nhiều định dạng nhị phân trong chương này

```
{  
  "userName": "Martin",  
  "favoriteNumber": 1337,  
  "interests": ["daydreaming", "hacking"]  
}
```

Hãy cùng xem xét một ví dụ về `MessagePack`, một mã hóa nhị phân cho JSON. Hình 5-2 cho thấy chuỗi byte mà bạn nhận được nếu bạn mã hóa tài liệu JSON trong Ví dụ 5-2 bằng `MessagePack`.

MessagePack

Byte sequence (66 bytes):

83	a8	75	73	65	72	4e	61	6d	65	a6	4d	61	72	74	69	6e	ae	66	61
76	6f	72	69	74	65	4e	75	6d	62	65	72	cd	05	39	a9	69	6e	74	65
72	65	73	74	73	92	ab	64	61	79	64	72	65	61	6d	69	6e	67	a7	68
61	63	6b	69	6e	67														

Breakdown:

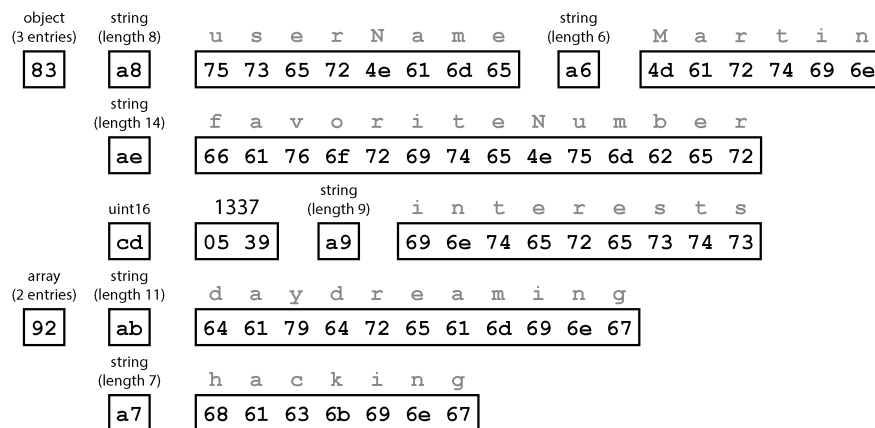


Figure 5-2. Bản ghi của chúng ta (Ví dụ 5-2) được mã hóa bằng MessagePack

Vài byte đầu tiên như sau:

1. Byte đầu tiên, 0x83, cho biết phần tiếp theo là một object (bốn bit có ý nghĩa cao nhất = 0x80) với ba field (bốn bit có ý nghĩa thấp nhất = 0x03). (Trong trường hợp bạn thắc mắc điều gì sẽ xảy ra nếu một object có nhiều hơn 15 field, khiến số lượng field không thể nằm gọn trong bốn bit, thì nó sẽ nhận một chỉ số loại khác, và số lượng field sẽ được mã hóa trong hai hoặc bốn byte.)
2. Byte thứ hai, 0xa8, cho biết phần tiếp theo là một string (bốn bit có ý nghĩa cao nhất = 0xa0) có độ dài tám byte (bốn bit có ý nghĩa thấp nhất = 0x08).
3. Tám byte tiếp theo là tên field `userName` dưới dạng ASCII. Vì độ dài đã được chỉ định trước đó, nên không cần bất kỳ dấu hiệu nào để cho chúng ta biết chuỗi kết thúc ở đâu (hay cần bất kỳ cơ chế escaping nào).
4. Bảy byte tiếp theo mã hóa giá trị chuỗi sáu ký tự `Martin` với tiền tố 0xa6, và cứ tiếp tục như vậy.

Mã hóa nhị phân này dài 66 byte, chỉ ít hơn một chút so với 81 byte mà mã hóa JSON dạng văn bản chiếm dụng (sau khi đã loại bỏ khoảng trắng). Tất cả các mã hóa

nhị phân của JSON đều tương tự nhau về khía cạnh này. Không rõ liệu việc giảm không gian lưu trữ nhỏ như vậy (và có lẽ là tăng tốc độ parsing) có xứng đáng với việc mất đi khả năng đọc hiểu của con người hay không.

Trong các mục tiếp theo, chúng ta sẽ thấy cách chúng ta có thể làm tốt hơn nhiều, bằng cách mã hóa cùng một bản ghi với số lượng byte chỉ bằng một nửa.

Protocol Buffers

Protocol Buffers (protobuf) là một thư viện mã hóa nhị phân được phát triển tại Google. Nó tương tự như Apache Thrift, vốn được phát triển ban đầu bởi Facebook [14]; hầu hết những gì mục này nói về Protocol Buffers cũng áp dụng cho Thrift.

Protocol Buffers yêu cầu một schema cho bất kỳ dữ liệu nào được mã hóa. Để mã hóa dữ liệu trong Ví dụ 5-2, bạn sẽ mô tả schema trong ngôn ngữ định nghĩa giao diện (IDL) của Protocol Buffers như sau:

```
syntax = "proto3";

message Person {
    string user_name = 1;
    int64 favorite_number = 2;
    repeated string interests = 3;
}
```

Protocol Buffers đi kèm với một công cụ tạo mã (code generation) nhận một định nghĩa schema như ví dụ hiển thị ở đây và tạo ra các class thực thi schema đó trong nhiều ngôn ngữ lập trình khác nhau. Mã ứng dụng của bạn có thể gọi mã đã được tạo này để mã hóa hoặc giải mã các bản ghi tuân thủ schema. Ngôn ngữ schema rất đơn giản so với JSON Schema; nó định nghĩa các field của mỗi bản ghi và kiểu dữ liệu của chúng, nhưng không hỗ trợ các ràng buộc khác đối với các giá trị khả thi của các field.

Mã hóa Ví dụ 5-2 bằng bộ mã hóa Protocol Buffers yêu cầu 33 byte, như được hiển thị trong Hình 5-3 [15]. Tương tự như trong Hình 5-2, mỗi field có một chú thích kiểu (để chỉ ra liệu nó là một string, integer, v.v.) và, khi cần thiết, một chỉ số độ dài (chẳng hạn như độ dài của một string). Các string xuất hiện trong dữ liệu (Martin, daydreaming, hacking) được mã hóa bằng ASCII (chính xác hơn là UTF-8), giống như trước đó.

Protocol Buffers

Byte sequence (33 bytes):

0a	06	4d	61	72	74	69	6e	10	b9	0a	1a	0b	64	61	79	64	72	65	61
6d	69	6e	67	1a	07	68	61	63	6b	69	6e	67							

Breakdown:

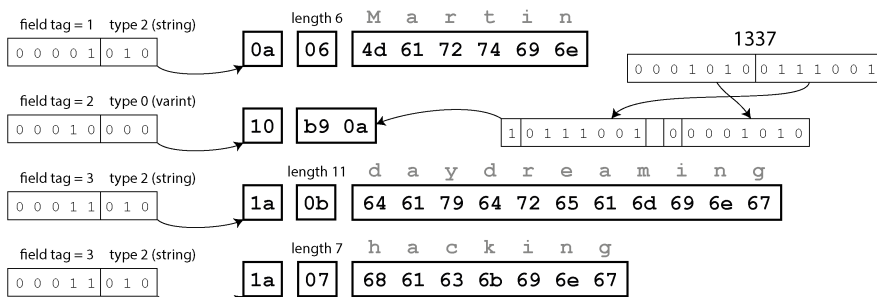


Figure 5-3. Hình: Bản ghi của chúng ta được mã hóa bằng Protocol Buffers

Khác với Hình 5-2, ví dụ này không có tên `field` (`userName`, `favoriteNumber`, `interests`). Thay vào đó, dữ liệu đã được mã hóa chứa các *field tags*, là các con số (1, 2, và 3). Đó chính là những con số xuất hiện trong định nghĩa `schema`. `field tags` giống như các bí danh (alias) cho các `field`—chúng là một cách gọn nhẹ để chỉ ra `field` mà chúng ta đang nói đến mà không cần phải viết đầy đủ tên `field`.

Như bạn có thể thấy, Protocol Buffers tiết kiệm thêm nhiều không gian bằng cách đóng gói kiểu dữ liệu của `field` và số tag vào trong một byte duy nhất. Nó sử dụng các số nguyên có độ dài thay đổi (variable-length integers): số 1337 được mã hóa trong hai byte, với bit cao nhất của mỗi byte được dùng để chỉ ra liệu còn các byte tiếp theo hay không (bảy bit ít quan trọng nhất được lưu trong byte đầu tiên để đơn giản hóa việc tái cấu trúc số nguyên khi các byte được đọc). Điều này có nghĩa là các số từ -64 đến 63 được mã hóa trong một byte, các số từ -8,192 đến 8,191 được mã hóa trong hai byte, v.v. Các số lớn hơn sẽ sử dụng nhiều byte hơn.

Protocol Buffers không có kiểu dữ liệu `list` hoặc `array` tường minh. Thay vào đó, modifier `repeated` trên `field interests` cho biết `field` đó chứa một danh sách các giá trị thay vì một giá trị đơn lẻ. Trong mã hóa nhị phân, các phần tử của `list` được biểu diễn đơn giản bằng việc lặp lại chính `field tag` đó trong cùng một bản ghi.

Field tags và schema evolution

Trước đó chúng ta đã nói rằng các `schema` chắc chắn sẽ cần thay đổi theo thời gian. Chúng ta gọi đây là *schema evolution* (sự tiến hóa schema). Protocol Buffers xử lý các thay đổi `schema` như thế nào trong khi vẫn duy trì tính tương thích ngược (backward compatibility) và tương thích xuôi (forward compatibility)?

Như bạn có thể thấy từ các ví dụ, một bản ghi đã mã hóa chỉ là sự nối tiếp của các `field` đã mã hóa của nó. Mỗi `field` được xác định bởi số tag của nó (các số 1, 2, 3 trong schema mẫu) và được chú thích với một kiểu dữ liệu (ví dụ: `string` hoặc `integer`). Nếu giá trị của một `field` không được thiết lập, nó đơn giản là bị bỏ qua khỏi bản ghi đã mã hóa. Từ đây, bạn có thể thấy rằng `field tags` đóng vai trò quan trọng đối với ý nghĩa của dữ liệu đã mã hóa. Bạn có thể thay đổi tên của một `field` trong schema, vì dữ liệu đã mã hóa không bao giờ tham chiếu đến tên `field`, nhưng bạn không thể thay đổi tag của một `field`, vì điều đó sẽ làm cho tất cả dữ liệu đã mã hóa hiện có trở nên không hợp lệ.

Bạn có thể thêm các `field` mới vào schema, miễn là bạn cấp cho mỗi `field` một số tag mới. Nếu code cũ (vốn không biết về các số tag mới mà bạn đã thêm) cố gắng đọc dữ liệu được viết bởi code mới, bao gồm cả một `field` mới với số tag mà nó không nhận diện được, nó có thể đơn giản là bỏ qua `field` đó. Chú thích kiểu dữ liệu cho phép parser xác định cần phải bỏ qua bao nhiêu byte trong khi vẫn bảo toàn được các `field` chưa biết, nhằm tránh vấn đề trong Hình 5-1. Điều này duy trì tính tương thích xuôi: code cũ có thể đọc được các bản ghi được viết bởi code mới.

Còn về tính tương thích ngược thì sao? Miễn là mỗi `field` có một số tag duy nhất, code mới luôn có thể đọc được dữ liệu cũ, vì các số tag vẫn giữ nguyên ý nghĩa. Nếu một `field` được thêm vào trong schema mới, và bạn đọc dữ liệu cũ chưa chứa `field` đó, nó sẽ được lấp đầy bằng một giá trị mặc định (ví dụ: chuỗi rỗng nếu kiểu của `field` là `string`, hoặc 0 nếu là một con số).

Việc loại bỏ một `field` tương tự như việc thêm một `field`, nhưng các vấn đề về tương thích ngược và tương thích xuôi sẽ bị đảo ngược. Bạn không bao giờ được sử dụng lại cùng một số tag đó nữa, vì bạn có thể vẫn còn dữ liệu được viết ở đâu đó bao gồm số tag cũ, và `field` đó phải được code mới bỏ qua. Các số tag đã sử dụng trong quá khứ có thể được giữ dự phòng (reserved) trong định nghĩa schema để đảm bảo chúng không bị lãng quên.

Còn việc thay đổi kiểu dữ liệu của một `field` thì sao? Điều đó khả thi với một số kiểu dữ liệu nhất định—hãy kiểm tra tài liệu để biết chi tiết—nhưng có rủi ro là các giá trị sẽ bị cắt cụt (truncated). Ví dụ, giả sử bạn thay đổi một số nguyên 32-bit thành một số nguyên 64-bit. Code mới có thể dễ dàng đọc dữ liệu được viết bởi code cũ, vì parser có thể lấp đầy bất kỳ bit thiếu hụt nào bằng các số 0. Tuy nhiên, nếu code cũ đọc dữ liệu được viết bởi code mới, code cũ vẫn đang sử dụng một biến 32-bit để giữ giá trị đó. Nếu giá trị 64-bit sau khi giải mã không vừa với 32 bit, nó sẽ bị cắt cụt.

Avro

Apache Avro là một định dạng mã hóa nhị phân khác, với một số khác biệt thú vị so với Protocol Buffers. Nó được bắt đầu vào năm 2009 như một dự án con của

Hadoop, kết quả từ việc Protocol Buffers không phù hợp với các trường hợp sử dụng của Hadoop [16].

Avro cũng sử dụng một schema để chỉ định cấu trúc của dữ liệu đang được mã hóa. Nó có hai ngôn ngữ schema: một loại (Avro IDL) dành cho con người chỉnh sửa, và một loại (dựa trên JSON) dễ dàng để máy đọc hơn. Tương tự như Protocol Buffers, các ngôn ngữ schema chỉ chỉ định các field và kiểu dữ liệu của chúng chứ không hỗ trợ các quy tắc kiểm định phức tạp như trong JSON Schema.

Được viết bằng Avro IDL, ví dụ về schema của chúng ta có thể trông như thế này:

```
record Person {
  string          userName;
  union { null, long } favoriteNumber = null;
  array<string>    interests;
}
```

Biểu diễn JSON tương đương của schema đó như sau:

```
{
  "type": "record",
  "name": "Person",
  "fields": [
    {"name": "userName",      "type": "string"},
    {"name": "favoriteNumber", "type": ["null", "long"],
    "default": null},
    {"name": "interests",     "type": {"type": "array",
    "items": "string"}}
  ]
}
```

Lưu ý rằng schema không có các số tag. Nếu chúng ta mã hóa bản ghi của mình (Ví dụ 5-2) bằng schema này, mã hóa nhị phân Avro chỉ dài 32 bytes—đây là định dạng nén nhất trong tất cả các định dạng mã hóa mà chúng ta đã thấy. Sự phân tách của chuỗi byte đã mã hóa được hiển thị trong Hình 5-4.

Nếu bạn kiểm tra chuỗi byte, bạn có thể thấy rằng không có gì định danh các field hay kiểu dữ liệu của chúng. Việc mã hóa đơn giản chỉ bao gồm các giá trị được nối lại với nhau. Một string chỉ là một tiền tố độ dài theo sau là các byte UTF-8, nhưng không có gì trong dữ liệu đã mã hóa cho bạn biết đó là một string. Nó hoàn toàn có thể là một số nguyên hoặc một thứ gì đó hoàn toàn khác. Một số nguyên được mã hóa bằng cách sử dụng mã hóa độ dài biến đổi.

Để parse dữ liệu nhị phân, bạn đi qua các field theo thứ tự mà chúng xuất hiện trong schema và sử dụng schema đó để xác định kiểu dữ liệu của từng field. Điều này có nghĩa là dữ liệu nhị phân chỉ có thể được giải mã chính xác nếu mã đọc dữ liệu đang sử dụng *đúng chính xác schema* đó với mã đã ghi dữ liệu. Bất kỳ sự không khớp nào về schema giữa bên đọc và bên ghi sẽ dẫn đến việc dữ liệu bị giải mã sai.

Avro

Byte sequence (32 bytes):

0c	4d	61	72	74	69	6e	02	f2	14	04	16	64	61	79	64	72	65	61	6d
69	6e	67	0e	68	61	63	6b	69	6e	67	00								

Breakdown:

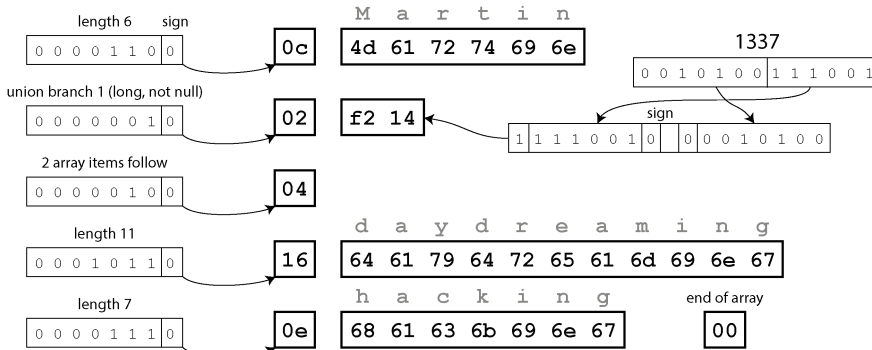


Figure 5-4. Hình: Bản ghi của chúng ta được mã hóa bằng Avro

Vậy, Avro hỗ trợ schema evolution (tiến hóa schema) như thế nào?

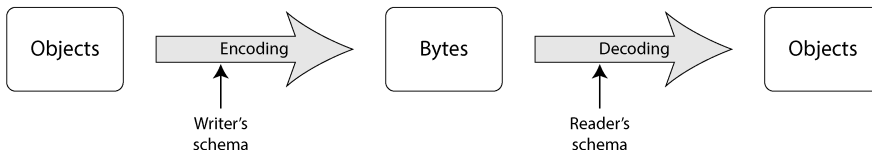
Writer's schema và reader's schema

Khi một ứng dụng muốn encode một số dữ liệu (để ghi vào một file hoặc database, gửi qua mạng, v.v.), ứng dụng đó sẽ sử dụng bất kỳ phiên bản schema nào mà nó biết —ví dụ, một schema đã được biên dịch trực tiếp vào ứng dụng. Điều này được gọi là *writer's schema*.

Để decode một số dữ liệu (đọc từ một file hoặc database, nhận từ mạng, v.v.), một ứng dụng sử dụng hai schema: *writer's schema*, cái vốn giống hệt với schema được dùng để encode, và *reader's schema*, cái có thể khác biệt. Điều này được minh họa trong Hình 5-5. *Reader's schema* định nghĩa các field của mỗi record mà mã nguồn ứng dụng đang mong đợi, cùng với kiểu dữ liệu của chúng.

Nếu reader's schema và writer's schema giống nhau, việc decoding sẽ rất dễ dàng. Nếu chúng khác nhau, Avro sẽ giải quyết các khác biệt bằng cách so sánh cả hai và chuyển đổi dữ liệu từ writer's schema sang reader's schema.

Protocol Buffers



Avro

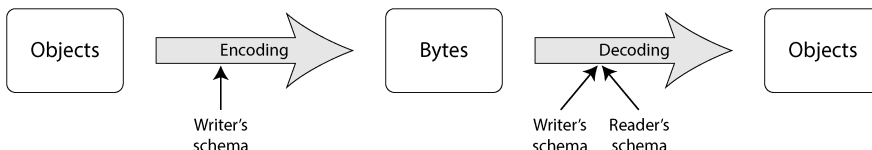


Figure 5-5. Trong Protocol Buffers, encoding và decoding có thể sử dụng các phiên bản khác nhau của một schema. Trong Avro, decoding sử dụng hai schema: schema của người viết (writer's schema) phải giống hệt với schema được dùng để encoding, nhưng schema của người đọc (reader's schema) có thể là một phiên bản cũ hơn hoặc mới hơn.

Đặc tả Avro [17, 18] định nghĩa chính xác cách thức hoạt động của quá trình schema resolution (giải quyết schema) này. Như được minh họa trong Hình 5-6, sẽ không có vấn đề gì nếu schema của writer và schema của reader có các field theo thứ tự khác nhau, bởi vì schema resolution sẽ khớp các field dựa trên tên của chúng. Nếu mã nguồn đọc dữ liệu gặp một field xuất hiện trong schema của writer nhưng không có trong schema của reader, nó sẽ bị bỏ qua. Nếu mã nguồn đọc dữ liệu mong đợi một field nhất định nhưng schema của writer không chứa field có tên đó, nó sẽ được điền bằng một giá trị mặc định đã được khai báo trong schema của reader.

Writer's schema for Person record

Datatype	Field name
string	userName
union {null, long}	favoriteNumber
array<string>	interests
string	photoURL

Reader's schema for Person record

Datatype	Field name
long	userID
union {null, int}	favoriteNumber
string	userName
array<string>	interests

Figure 5-6. Một Avro reader đang giải quyết các khác biệt giữa schema của writer và schema của reader

Các quy tắc schema evolution

Với Avro, forward compatibility (tính tương thích xuôi) có nghĩa là writer có thể sử dụng một phiên bản schema mới hơn so với reader. Ngược lại, backward

compatibility (tính tương thích ngược) có nghĩa là writer có thể sử dụng một phiên bản schema cũ hơn so với reader.

Để duy trì tính tương thích, bạn chỉ có thể thêm hoặc xóa một field có giá trị mặc định (chẳng hạn như field `favoriteNumber` trong schema Avro của chúng ta). Ví dụ, giả sử bạn thêm một field với giá trị mặc định, do đó field mới này tồn tại trong schema mới nhưng không có trong schema cũ. Khi một reader sử dụng schema mới đọc một record được viết bằng schema cũ, giá trị mặc định sẽ được điền vào cho field còn thiếu.

Nếu bạn thêm một field không có giá trị mặc định, các reader mới sẽ không thể đọc được dữ liệu được viết bởi các writer cũ, vì vậy bạn sẽ làm mất tính backward compatibility. Nếu bạn xóa một field không có giá trị mặc định, các reader cũ sẽ không thể đọc được dữ liệu được viết bởi các writer mới, vì vậy bạn sẽ làm mất tính forward compatibility.

Trong một số ngôn ngữ lập trình, `null` là một giá trị mặc định có thể chấp nhận được cho bất kỳ biến nào, nhưng trong Avro thì không phải vậy: nếu bạn muốn cho phép một field có giá trị là `null`, bạn phải sử dụng một *union type*. Ví dụ, `union { null, long, string } field;` chỉ ra rằng `field` có thể là một số, một chuỗi, hoặc `null`. Bạn chỉ có thể sử dụng `null` làm giá trị mặc định nếu nó là nhánh đầu tiên của union. Cách này rườm rà hơn một chút so với việc để mọi thứ mặc định là nullable, nhưng nó giúp ngăn ngừa lỗi bằng cách tường minh về những gì có thể và không thể là `null` [19].

Việc thay đổi datatype của một field là khả thi, miễn là Avro có thể chuyển đổi kiểu dữ liệu đó. Thay đổi tên của một field cũng khả thi nhưng hơi phức tạp. Schema của reader có thể chứa các alias cho tên field, vì vậy nó có thể khớp các tên field trong schema của writer cũ với các alias này. Điều này có nghĩa là việc thay đổi tên field có tính backward compatibility nhưng không có tính forward compatibility. Tương tự, việc thêm một nhánh vào một union type có tính backward compatibility nhưng không có tính forward compatibility.

Nhưng schema của writer là gì?

Chúng ta đã lướt qua một câu hỏi quan trọng: làm thế nào reader biết được schema đã được sử dụng để mã hóa một phần dữ liệu cụ thể? Chúng ta không thể chỉ đính kèm toàn bộ schema vào mỗi record, vì schema có khả năng sẽ lớn hơn nhiều so với dữ liệu đã được mã hóa, làm mất đi tất cả những lợi ích về tiết kiệm không gian từ việc mã hóa nhị phân.

Câu trả lời phụ thuộc vào ngữ cảnh mà Avro đang được sử dụng. Hãy xem một vài ví dụ:

Tập lớn với nhiều record

Một cách dùng phổ biến của Avro là lưu trữ một tệp lớn chứa hàng triệu record, tất cả đều được mã hóa với cùng một schema. (Chúng ta sẽ thảo luận về loại tình huống này trong Chương 11.) Trong trường hợp này, writer của tệp đó chỉ cần đính kèm schema một lần duy nhất ở đầu tệp. Avro quy định một định dạng tệp (object container files) để thực hiện việc này.

Cơ sở dữ liệu với các record được viết riêng lẻ

Trong một cơ sở dữ liệu, các record khác nhau có thể được viết tại các thời điểm khác nhau bằng các schema khác nhau—bạn không thể giả định rằng tất cả các record sẽ có cùng một schema. Giải pháp đơn giản nhất trong trường hợp này là đính kèm một số phiên bản ở đầu mỗi record đã được mã hóa và giữ một danh sách các phiên bản schema trong cơ sở dữ liệu của bạn. Một reader có thể lấy một record, trích xuất số phiên bản, sau đó lấy schema của writer tương ứng với số phiên bản đó từ cơ sở dữ liệu. Sau đó, nó có thể giải mã phần còn lại của record bằng cách sử dụng schema đó. Ví dụ, schema registry của Confluent dành cho Apache Kafka [20] và Espresso của LinkedIn [21] hoạt động theo cách này.

Gửi các record qua một kết nối mạng

Khi hai tiến trình giao tiếp qua một kết nối mạng hai chiều, chúng có thể thương lượng phiên bản schema khi thiết lập kết nối và sau đó sử dụng schema đó trong suốt vòng đời của kết nối. Giao thức Avro RPC (xem “Dataflow Through Services: REST and RPC”) hoạt động như thế này.

Việc sở hữu một database chứa các phiên bản schema là điều hữu ích trong mọi trường hợp, vì nó đóng vai trò như một tài liệu hướng dẫn và cho bạn cơ hội để kiểm tra tính tương thích của schema [22]. Bạn có thể sử dụng một số nguyên tắc đơn giản hoặc một mã hash của schema để làm số phiên bản.

Các schema được tạo động

Một lợi thế trong cách tiếp cận của Avro so với Protocol Buffers là schema không chứa bất kỳ tag number nào. Nhưng tại sao điều này lại quan trọng? Việc giữ một vài con số trong schema thì có vấn đề gì?

Sự khác biệt nằm ở chỗ Avro thân thiện hơn với các schema được tạo động (*dynamically generated*). Ví dụ, giả sử bạn có một relational database mà bạn muốn dump nội dung của nó vào một tệp, và bạn muốn sử dụng một định dạng nhị phân để tránh các vấn đề đã đề cập trước đó với các định dạng văn bản (JSON, CSV, XML). Nếu bạn sử dụng Avro, bạn có thể tạo một Avro schema (dưới dạng biểu diễn JSON mà chúng ta đã thấy trước đó) từ relational schema một cách khá dễ dàng và mã hóa nội dung database bằng schema đó, sau đó dump tất cả vào một tệp Avro object container [23]. Bạn có thể tạo một record schema cho mỗi bảng trong database, và mỗi cột sẽ trở thành một field trong record đó. Tên cột trong database sẽ ánh xạ tới tên field trong Avro.

Bây giờ, nếu schema của database thay đổi (ví dụ: nếu một bảng được thêm một cột và xóa một cột), bạn chỉ cần tạo một Avro schema mới từ database schema đã được cập nhật và xuất dữ liệu theo Avro schema mới. Quá trình xuất dữ liệu không cần phải bận tâm đến sự thay đổi schema—nó chỉ đơn giản là thực hiện chuyển đổi schema mỗi khi chạy. Bất kỳ ai đọc các tệp dữ liệu mới sẽ thấy rằng các field của record đã thay đổi, nhưng vì các field được định danh bằng tên, nên schema của writer đã cập nhật vẫn có thể khớp với schema của reader cũ.

Ngược lại, nếu bạn sử dụng Protocol Buffers cho mục đích này, các field tag có lẽ sẽ phải được gán bằng tay. Mỗi khi database schema thay đổi, một quản trị viên sẽ phải cập nhật thủ công việc ánh xạ từ tên cột database sang các field tag. (Có thể tự động hóa việc này, nhưng bộ tạo schema sẽ phải cực kỳ cẩn thận để không gán các field tag đã được sử dụng trước đó.) Loại schema được tạo động này đơn giản không phải là mục tiêu thiết kế của Protocol Buffers, trong khi đó lại là mục tiêu của Avro.

Ưu điểm của Schema

Như chúng ta đã thấy, cả Protocol Buffers và Avro đều sử dụng một schema để mô tả một định dạng mã hóa nhị phân. Ngôn ngữ schema của chúng đơn giản hơn nhiều so với XML Schema hoặc JSON Schema, vốn hỗ trợ các quy tắc validation chi tiết hơn (ví dụ: "giá trị chuỗi của field này phải khớp với biểu thức chính quy này" hoặc "giá trị số nguyên của field này phải nằm trong khoảng từ 0 đến 100"). Vì Protocol Buffers và Avro dễ triển khai và sử dụng hơn, chúng đã nhận được sự hỗ trợ từ một phạm vi khá rộng các ngôn ngữ lập trình.

Các ý tưởng làm cơ sở cho những phương pháp mã hóa này hoàn toàn không mới. Ví dụ, chúng có nhiều điểm chung với ASN.1, một ngôn ngữ định nghĩa schema lần đầu tiên được tiêu chuẩn hóa vào năm 1984 [24, 25]. Nó được sử dụng để định nghĩa các giao thức mạng khác nhau, và mã hóa nhị phân (DER) của nó vẫn được sử dụng để mã hóa các chứng chỉ SSL (X.509), chẳng hạn [26]. ASN.1 hỗ trợ schema evolution bằng cách sử dụng các tag number, tương tự như Protocol Buffers [27]. Tuy nhiên, nó cũng rất phức tạp và tài liệu hướng dẫn kém, vì vậy ASN.1 có lẽ không phải là một lựa chọn tốt cho các ứng dụng mới.

Nhiều hệ thống dữ liệu cũng triển khai một loại mã hóa nhị phân độc quyền nào đó cho dữ liệu của chúng. Ví dụ, hầu hết các relational database đều có một giao thức mạng mà qua đó bạn có thể gửi các truy vấn tới database và nhận lại các phản hồi. Những giao thức đó thường đặc thù cho một database cụ thể, và nhà cung cấp database sẽ cung cấp một driver (ví dụ: sử dụng các API ODBC hoặc JDBC) để giải mã các phản hồi từ giao thức mạng của database thành các cấu trúc dữ liệu trong bộ nhớ.

Vì vậy, chúng ta có thể thấy rằng mặc dù các định dạng dữ liệu văn bản như JSON, XML và CSV rất phổ biến, nhưng các mã hóa nhị phân dựa trên schema cũng là một lựa chọn khả thi. Chúng có một số đặc tính tốt sau đây:

- Chúng có thể gọn hơn nhiều so với các biến thể "binary JSON" khác nhau, vì chúng có thể lược bỏ tên các field khỏi dữ liệu đã được mã hóa.
- Schema là một dạng tài liệu có giá trị, và vì schema là bắt buộc để giải mã, bạn có thể chắc chắn rằng nó luôn được cập nhật (trong khi tài liệu được duy trì thủ công có thể dễ dàng bị sai lệch so với thực tế).
- Việc duy trì một cơ sở dữ liệu các schema cho phép bạn kiểm tra tính tương thích tiến (forward compatibility) và tương thích lùi (backward compatibility) của các thay đổi schema trước khi bất cứ thứ gì được triển khai.
- Đối với người dùng các ngôn ngữ lập trình có kiểu tĩnh (statically typed), khả năng tạo code từ schema là rất hữu ích, vì nó cho phép kiểm tra kiểu (type checking) tại thời điểm biên dịch.

Tóm lại, schema evolution cho phép sự linh hoạt tương tự như các cơ sở dữ liệu JSON schemaless/schema-on-read cung cấp (xem mục "Schema flexibility in the document model"), đồng thời cũng cung cấp các đảm bảo tốt hơn về dữ liệu và các công cụ tốt hơn. Tuy nhiên, bạn nên giữ số lượng các định dạng schema đồng thời ở mức tối thiểu để giữ cho các hoạt động được đơn giản.

Các chế độ Dataflow

Ở đầu chương này, chúng ta đã nói rằng bất cứ khi nào bạn muốn gửi một số dữ liệu tới một process khác mà bạn không chia sẻ bộ nhớ—ví dụ, khi bạn muốn gửi dữ liệu qua mạng hoặc ghi nó vào một tệp—bạn cần mã hóa nó thành một chuỗi các byte. Sau đó, chúng ta đã thảo luận về nhiều loại mã hóa khác nhau để thực hiện việc này.

Chúng ta đã nói về tính tương thích tiến và tương thích lùi, những yếu tố quan trọng đối với khả năng tiến hóa (evolvability) (giúp việc thay đổi trở nên dễ dàng bằng cách cho phép bạn nâng cấp các phần của hệ thống một cách độc lập thay vì phải thay đổi mọi thứ cùng một lúc). Tính tương thích là mối quan hệ giữa một process thực hiện mã hóa dữ liệu và một process khác thực hiện giải mã nó.

Đó là một ý tưởng khá trừu tượng vì dữ liệu có thể chảy từ process này sang process khác theo nhiều cách. Ai là người mã hóa dữ liệu, và ai là người giải mã nó? Trong phần còn lại của chương này, chúng ta sẽ khám phá một số cách phổ biến nhất mà dữ liệu chảy giữa các process thông qua các cơ sở dữ liệu, các lời gọi service, các workflow engine và các tin nhắn bất đồng bộ.

Dataflow thông qua các cơ sở dữ liệu

Trong một cơ sở dữ liệu, process thực hiện việc ghi sẽ mã hóa dữ liệu, và process thực hiện việc đọc sẽ giải mã nó. Chỉ có một process truy cập vào cơ sở dữ liệu, trong trường hợp đó người đọc đơn giản là một phiên bản sau của chính process đó; trong kịch bản như vậy, bạn có thể coi việc lưu trữ thứ gì đó vào cơ sở dữ liệu giống như việc *gửi một tin nhắn cho chính bản thân bạn trong tương lai*. Tính tương thích lùi rõ ràng là cần thiết ở đây, nếu không bản thân bạn trong tương lai sẽ không thể giải mã những gì bạn đã viết trước đó.

Tuy nhiên, nhìn chung, việc nhiều process cùng truy cập vào một cơ sở dữ liệu tại cùng một thời điểm là rất phổ biến. Các process đó có thể là các ứng dụng hoặc service khác nhau, hoặc chúng có thể chỉ đơn giản là nhiều instance của cùng một service (chạy song song để đảm bảo khả năng mở rộng hoặc độ tin cậy). Dù theo cách nào, trong một môi trường như vậy, có khả năng một số process truy cập cơ sở dữ liệu sẽ chạy code mới hơn và một số sẽ chạy code cũ hơn—ví dụ, vì một phiên bản mới hiện đang được triển khai trong một đợt rolling upgrade, nên một số instance đã được cập nhật trong khi những instance khác thì chưa.

Điều này có nghĩa là một giá trị trong cơ sở dữ liệu có thể được viết bởi một phiên bản code *mới hơn* và sau đó được đọc bởi một phiên bản code *cũ hơn* vẫn đang chạy. Do đó, tính tương thích tiến cũng thường được yêu cầu đối với các cơ sở dữ liệu.

Các giá trị khác nhau được ghi tại các thời điểm khác nhau

Một cơ sở dữ liệu thường cho phép bất kỳ giá trị nào được cập nhật vào bất kỳ lúc nào. Trong cùng một cơ sở dữ liệu, bạn có thể có một số giá trị đã được ghi cách đây năm mili giây và những giá trị khác đã được ghi từ năm năm trước.

Khi bạn triển khai một phiên bản mới của ứng dụng (ít nhất là đối với một ứng dụng phía server), bạn có thể thay thế hoàn toàn phiên bản cũ bằng phiên bản mới chỉ trong vài phút. Điều tương tự không đúng với nội dung cơ sở dữ liệu; dữ liệu từ năm năm trước vẫn sẽ ở đó, với định dạng mã hóa ban đầu, trừ khi bạn đã thực hiện ghi lại nó một cách rõ ràng kể từ đó. Quan sát này đôi khi được tóm gọn là *dữ liệu tồn tại lâu hơn code*.

Mặc dù việc viết lại (*migrating*) dữ liệu sang một schema mới chắc chắn là khả thi, nhưng nó rất tốn kém đối với một tập dữ liệu lớn. Do đó, hầu hết các cơ sở dữ liệu đều trì hoãn thao tác này, thực hiện nó một cách bất đồng bộ và dựa trên nỗ lực tốt nhất (best-effort). Ví dụ, các engine lưu trữ LSM-tree (xem “Log-Structured Storage”) sẽ viết lại dữ liệu bằng định dạng mới nhất trong quá trình compaction. Hầu hết các cơ sở dữ liệu quan hệ cũng cho phép thay đổi schema đơn giản, chẳng hạn như thêm một cột mới với giá trị mặc định là null, mà không cần viết lại dữ liệu hiện có. Khi một hàng cũ được đọc, cơ sở dữ liệu sẽ điền các giá trị null cho bất kỳ cột nào còn thiếu trong dữ liệu đã được mã hóa trên đĩa. Do đó, schema evolution

cho phép toàn bộ cơ sở dữ liệu xuất hiện như thể nó được mã hóa bằng một schema duy nhất, ngay cả khi phần lưu trữ bên dưới có thể chứa các bản ghi được mã hóa với nhiều phiên bản schema lịch sử khác nhau.

Những thay đổi schema phức tạp hơn—ví dụ, thay đổi một thuộc tính đơn trị thành đa trị, hoặc di chuyển một số dữ liệu vào một bảng riêng biệt—vẫn yêu cầu dữ liệu phải được viết lại, thường là ở cấp độ ứng dụng [28]. Việc duy trì tính tương thích tiến (forward compatibility) và tương thích lùi (backward compatibility) xuyên suốt các quá trình di chuyển như vậy vẫn là một vấn đề nghiên cứu [29].

Lưu trữ lưu trữ (Archival storage)

Có lẽ bạn thực hiện một bản snapshot cơ sở dữ liệu của mình theo từng thời điểm—chẳng hạn, vì mục đích sao lưu hoặc để tải vào một data warehouse (xem “Data Warehousing”). Trong trường hợp này, bản dump dữ liệu thường sẽ được mã hóa bằng schema mới nhất, ngay cả khi việc mã hóa ban đầu trong cơ sở dữ liệu nguồn chứa sự pha trộn giữa các phiên bản schema từ các thời kỳ khác nhau. Vì dù sao bạn cũng đang sao chép dữ liệu, nên bạn cũng có thể mã hóa bản sao dữ liệu đó một cách nhất quán.

Vì bản dump dữ liệu được ghi trong một lần và sau đó là bất biến (immutable), các định dạng như tệp Avro object container là một sự lựa chọn phù hợp. Đây cũng là cơ hội tốt để mã hóa dữ liệu theo định dạng column-oriented thân thiện với phân tích như Parquet (xem “Column compression”).

Trong Chương 11, chúng ta sẽ thảo luận thêm về việc sử dụng dữ liệu trong lưu trữ lưu trữ (archival storage).

Luồng dữ liệu qua các dịch vụ: REST và RPC

Khi bạn có các tiến trình cần giao tiếp qua một mạng, bạn có thể sắp xếp việc giao tiếp đó theo một vài cách. Cách sắp xếp phổ biến nhất là có hai vai trò: client và server. Các *server* cung cấp một API qua mạng, và các *client* có thể kết nối tới các server để gửi các yêu cầu tới API đó. API được cung cấp bởi server được gọi là một *service*.

Web hoạt động theo cách này: các client (trình duyệt web) gửi các yêu cầu tới các web server, thực hiện các yêu cầu GET để tải HTML, CSS, JavaScript, hình ảnh, v.v. và thực hiện các yêu cầu POST để gửi dữ liệu tới server. API bao gồm một tập hợp các giao thức và định dạng dữ liệu chuẩn hóa (HTTP, URL, SSL/TLS, HTML, v.v.). Vì các trình duyệt web, web server và tác giả website hầu hết đều đồng ý với các tiêu chuẩn này, bạn có thể sử dụng bất kỳ trình duyệt web nào để truy cập bất kỳ website nào (ít nhất là về mặt lý thuyết!).

Trình duyệt web không phải là loại client duy nhất. Ví dụ, các ứng dụng native chạy trên thiết bị di động và máy tính để bàn thường nói chuyện với các server, và các ứng

dụng JavaScript phía client chạy bên trong trình duyệt web cũng có thể thực hiện các yêu cầu HTTP. Trong trường hợp này, phản hồi của server thường không phải là HTML để hiển thị cho con người, mà là dữ liệu dưới một định dạng mã hóa thuận tiện cho việc xử lý tiếp theo bởi mã ứng dụng phía client (thường xuyên nhất là JSON). Mặc dù HTTP có thể được sử dụng làm giao thức vận chuyển, nhưng API được triển khai bên trên là đặc thù cho ứng dụng, và client cũng như server cần thống nhất về các chi tiết của API đó.

Theo một số cách, các service tương tự như các cơ sở dữ liệu: chúng thường cho phép các client gửi và truy vấn dữ liệu. Tuy nhiên, trong khi các cơ sở dữ liệu cho phép các truy vấn tùy ý bằng cách sử dụng các ngôn ngữ truy vấn mà chúng ta đã thảo luận trong Chương 3, thì các service cung cấp một API đặc thù cho ứng dụng, chỉ cho phép các đầu vào và đầu ra đã được xác định trước bởi logic nghiệp vụ (mã ứng dụng) của service đó [30]. Sự hạn chế này cung cấp một mức độ đóng gói (encapsulation): các service có thể áp đặt các hạn chế chi tiết về những gì client có thể và không thể làm.

Một mục tiêu thiết kế then chốt của kiến trúc hướng dịch vụ (service-oriented) hoặc microservices là giúp ứng dụng dễ dàng thay đổi và bảo trì hơn bằng cách làm cho các service có thể triển khai và phát triển độc lập. Một nguyên tắc phổ biến là mỗi service nên thuộc quyền sở hữu của một nhóm, và nhóm đó phải có khả năng phát hành các phiên bản mới của service một cách thường xuyên mà không cần phải phối hợp với các nhóm khác. Do đó, chúng ta nên kỳ vọng rằng các phiên bản cũ và mới của server cũng như client sẽ chạy cùng một lúc, vì vậy việc mã hóa dữ liệu (data encoding) được sử dụng bởi các server và client phải tương thích giữa các phiên bản của service API. Miễn là các API vẫn duy trì tính tương thích, các nhóm có thể tự do sửa đổi hệ thống của họ theo bất kỳ cách nào họ muốn; đặc tính này giúp các lập trình viên thực hiện việc di chuyển nội bộ (internal migrations) các dữ liệu, service, hoặc thậm chí là toàn bộ hệ thống một cách dễ dàng hơn nhiều.

Web services

Khi HTTP được sử dụng làm giao thức nền tảng để giao tiếp với service, nó được gọi là một *web service*. Web services thường được sử dụng khi xây dựng kiến trúc hướng dịch vụ hoặc microservices (đã thảo luận trước đó trong mục “Microservices and Serverless”). Thuật ngữ này có lẽ là một cách gọi chưa hoàn toàn chính xác, bởi vì web services không chỉ được sử dụng trên web mà còn trong nhiều ngữ cảnh khác. Ví dụ:

- Một ứng dụng client chạy trên thiết bị của người dùng (ví dụ: một ứng dụng native trên thiết bị di động, hoặc một ứng dụng web JavaScript trong trình duyệt) thực hiện các yêu cầu tới một service thông qua HTTP. Các yêu cầu này thường đi qua internet công cộng.

- Một service thực hiện các yêu cầu tới một service khác thuộc sở hữu của cùng một tổ chức, thường nằm trong cùng một mạng nội bộ, như một phần của kiến trúc hướng dịch vụ/microservices.
- Một service thực hiện các yêu cầu tới một service thuộc sở hữu của một tổ chức khác, thường là thông qua internet. Điều này được sử dụng để trao đổi dữ liệu giữa các hệ thống backend của các tổ chức. Danh mục này bao gồm các public API được cung cấp bởi các dịch vụ trực tuyến, chẳng hạn như các hệ thống xử lý thẻ tín dụng, hoặc OAuth để truy cập dùng chung vào dữ liệu người dùng.

Triết lý thiết kế service phổ biến nhất là REST, dựa trên các nguyên tắc của HTTP [31, 32]. REST nhấn mạnh vào các định dạng dữ liệu đơn giản, sử dụng URL để định danh các tài nguyên và sử dụng các tính năng của HTTP để kiểm soát cache, authentication, và thương lượng kiểu nội dung (content type negotiation). Một API được thiết kế theo các nguyên tắc của REST được gọi là *RESTful*.

Code cần gọi một web service API phải biết được HTTP endpoint nào cần truy vấn, cũng như định dạng dữ liệu nào cần gửi đi và định dạng dữ liệu mong đợi trong phản hồi. Ngay cả khi một service áp dụng các nguyên tắc thiết kế RESTful, các client vẫn cần tìm ra những chi tiết này bằng cách nào đó. Các nhà phát triển service thường sử dụng một IDL để định nghĩa và tài liệu hóa các API endpoint và data model của service, đồng thời để tiến hóa chúng theo thời gian. Sau đó, các nhà phát triển khác có thể sử dụng định nghĩa service để xác định cách truy vấn service đó. Hai IDL cho service phổ biến nhất là OpenAPI (còn được gọi là Swagger [33]), được sử dụng cho các web service gửi và nhận JSON, và Protocol Buffers, được sử dụng cho các gRPC service.

Các lập trình viên thường viết các định nghĩa service OpenAPI bằng JSON hoặc YAML (xem Ví dụ 5-3). Định nghĩa service cho phép các lập trình viên định nghĩa các endpoint của service, tài liệu, các phiên bản, data model, và nhiều thứ khác nữa. Các định nghĩa service của Protocol Buffers sử dụng IDL mà chúng ta đã thấy trong mục “Protocol Buffers”.

Example 5-3. Một định nghĩa service OpenAPI trong YAML

```
openapi: 3.0.0
info:
  title: Ping, Pong
  version: 1.0.0
servers:
  - url: http://localhost:8080
paths:
  /ping:
    get:
      summary: Given a ping, returns a pong message
      responses:
        '200':
          description: A pong
          content:
            application/json:
              schema:
                type: object
                properties:
                  message:
                    type: string
                  example: Pong!
```

Ngay cả khi đã áp dụng một triết lý thiết kế và IDL, các lập trình viên vẫn phải viết mã để thực thi các lời gọi API của service của họ. Một *service framework* (khung dịch vụ), chẳng hạn như Spring Boot, FastAPI, hoặc gRPC, thường được áp dụng để đơn giản hóa nỗ lực này. Các service framework cho phép lập trình viên tập trung vào việc viết business logic cho mỗi API endpoint, trong khi mã của framework sẽ xử lý routing, metrics, caching, authentication, và các thành phần khác. Ví dụ 5-4 trình bày một bản thực thi bằng Python của service đã được định nghĩa trong Ví dụ 5-3.

Example 5-4. Một FastAPI service thực thi định nghĩa từ Ví dụ 5-3

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="Ping, Pong", version="1.0.0")

class PongResponse(BaseModel):
    message: str = "Pong!"

@app.get("/ping", response_model=PongResponse,
         summary="Given a ping, returns a pong message")
async def ping():
    return PongResponse()
```

Nhiều framework kết hợp các định nghĩa service và mã nguồn server lại với nhau. Trong một số trường hợp, chẳng hạn như với framework Python FastAPI phổ biến, các server được viết bằng mã nguồn và một IDL sẽ được tạo ra tự động. Trong các trường hợp khác, chẳng hạn như với gRPC, định nghĩa service được viết trước và

khung mã nguồn (scaffolding) của server sẽ được tạo ra. Cả hai cách tiếp cận đều cho phép các lập trình viên tạo ra các client library và SDK bằng nhiều ngôn ngữ khác nhau từ định nghĩa service. Ngoài việc tạo mã (code generation), các công cụ IDL như của Swagger có thể tạo tài liệu, kiểm tra tính tương thích khi thay đổi schema, và cung cấp một giao diện người dùng đồ họa (GUI) để các lập trình viên truy vấn và kiểm thử các service.

Các vấn đề với remote procedure calls

Các web service chỉ là phiên bản mới nhất của một chuỗi dài các công nghệ dùng để thực hiện các yêu cầu API qua mạng, nhiều công nghệ trong số đó đã được quảng bá rầm rộ nhưng lại gặp phải những vấn đề nghiêm trọng. Enterprise JavaBeans (EJB) và Remote Method Invocation (RMI) của Java bị giới hạn trong Java. Distributed Component Object Model (DCOM) bị giới hạn trong các nền tảng Microsoft. Common Object Request Broker Architecture (CORBA) thì cực kỳ phức tạp và không cung cấp tính tương thích ngược (backward compatibility) hay tương thích xuôi (forward compatibility) [34]. SOAP và framework web service WS-* nhằm mục đích cung cấp khả năng tương tác giữa các nhà cung cấp khác nhau nhưng cũng gặp phải các vấn đề về độ phức tạp và tính tương thích [35, 36, 37].

Tất cả những thứ này đều dựa trên ý tưởng về *remote procedure calls* (RPC), được giới thiệu từ những năm 1970 [38]. Mô hình RPC cố gắng làm cho một yêu cầu tới một network service từ xa trông giống như việc gọi một hàm hoặc một method trong cùng một process (sự trừu tượng này được gọi là *location transparency* (tính trong suốt về vị trí)). Mặc dù điều này có vẻ tiện lợi lúc đầu, nhưng cách tiếp cận này về cơ bản là có lỗi [39, 40]. Một yêu cầu mạng rất khác so với một lời gọi hàm cục bộ vì nhiều lý do khác nhau:

- Một lời gọi hàm cục bộ có thể dự đoán được và sẽ thành công hoặc thất bại tùy thuộc vào các tham số nằm trong tầm kiểm soát của bạn. Một yêu cầu mạng thì không thể dự đoán được, vì những lý do hoàn toàn nằm ngoài tầm kiểm soát của bạn. Yêu cầu hoặc phản hồi có thể bị mất do sự cố mạng, ví dụ vậy, hoặc máy từ xa có thể bị chậm hoặc không khả dụng. Các vấn đề về mạng rất phổ biến, vì vậy các ứng dụng phải lường trước chúng (ví dụ, bằng cách thử lại các yêu cầu bị thất bại).
- Một lời gọi hàm cục bộ sẽ trả về một kết quả, ném ra một exception, hoặc không bao giờ trả về (vì nó rơi vào một vòng lặp vô tận hoặc process bị crash). Một yêu cầu mạng có một kết quả khả thi khác: nó có thể trả về mà không có kết quả, do bị *timeout*. Trong trường hợp đó, bạn đơn giản là không biết chuyện gì đã xảy ra; nếu bạn không nhận được phản hồi từ service từ xa, bạn không có cách nào để biết liệu yêu cầu đã được gửi đi thành công hay chưa. (Chúng ta sẽ thảo luận vấn đề này chi tiết hơn trong Chương 9.)

- Nếu bạn thử lại một yêu cầu mạng đã thất bại, có thể xảy ra trường hợp yêu cầu trước đó thực tế đã được thực hiện thành công, và chỉ có phản hồi là bị mất. Trong trường hợp đó, việc thử lại sẽ khiến hành động bị thực hiện nhiều lần, trừ khi bạn xây dựng một cơ chế để loại bỏ trùng lặp (*idempotence*) vào trong protocol [41]. Các lời gọi hàm cục bộ không gặp phải vấn đề này. (Chúng ta sẽ thảo luận chi tiết hơn về idempotence trong Chương 12.)
- Mỗi khi bạn gọi một hàm cục bộ, nó thường mất khoảng cùng một lượng thời gian để thực thi. Một yêu cầu mạng chậm hơn nhiều so với một lời gọi hàm, và latency (độ trễ) của nó cũng biến động rất lớn: vào những thời điểm tốt, nó có thể hoàn tất trong chưa đầy một mili giây, nhưng khi mạng bị tắc nghẽn hoặc service từ xa bị quá tải, nó có thể mất nhiều giây để thực hiện chính xác cùng một việc đó.
- Khi bạn gọi một hàm cục bộ, bạn có thể truyền các tham chiếu (con trỏ) tới các đối tượng trong bộ nhớ cục bộ một cách hiệu quả. Khi bạn thực hiện một yêu cầu mạng, tất cả các tham số đó cần được mã hóa thành một chuỗi các byte để có thể gửi qua mạng. Điều đó ổn nếu các tham số là các kiểu dữ liệu nguyên thủy (primitive) không thể thay đổi (immutable) như số hoặc các chuỗi ngắn, nhưng nó sẽ nhanh chóng trở thành vấn đề với lượng dữ liệu lớn hơn và các đối tượng có thể thay đổi (mutable).
- Client và service có thể được triển khai bằng các ngôn ngữ lập trình khác nhau, vì vậy framework RPC phải chuyển đổi các kiểu dữ liệu từ ngôn ngữ này sang ngôn ngữ khác. Việc này có thể trở nên rắc rối, vì không phải tất cả các ngôn ngữ đều có các kiểu dữ liệu giống nhau—ví dụ, hãy nhớ lại các vấn đề của JavaScript với các số lớn hơn 2^{53} (xem “JSON, XML, and Binary Variants”). Vấn đề này không tồn tại trong một tiến trình đơn lẻ được viết bằng một ngôn ngữ duy nhất.

Tất cả các yếu tố này có nghĩa là không cần thiết phải cố gắng làm cho một remote service trông giống một local object trong ngôn ngữ lập trình của bạn, bởi vì chúng là những thứ khác biệt về mặt bản chất. Một phần sức hấp dẫn của REST là nó coi việc chuyển đổi trạng thái (state transfer) qua mạng như một quá trình tách biệt với một lời gọi hàm (function call).

Load balancers, service discovery, và service meshes

Tất cả các service đều giao tiếp qua mạng. Vì lý do này, một client phải biết địa chỉ của service mà nó đang kết nối—một vấn đề được gọi là *service discovery* (khám phá dịch vụ). Cách tiếp cận đơn giản nhất là cấu hình một client để kết nối tới địa chỉ IP và port nơi service đang chạy. Cấu hình này sẽ hoạt động, nhưng nếu server ngoại tuyến, được chuyển sang một máy mới, hoặc bị quá tải, client sẽ phải được cấu hình lại một cách thủ công.

Để cung cấp độ tin cậy và khả năng mở rộng cao hơn, nhiều instance của một service thường chạy trên nhiều máy khác nhau, bất kỳ máy nào cũng có thể xử lý một yêu cầu đến. Việc phân phối các yêu cầu qua các instance này được gọi là *load balancing* (cân bằng tải) [42]. Có rất nhiều giải pháp load balancing và service discovery hiện có:

Hardware load balancers

Những thiết bị chuyên dụng này được lắp đặt trong các datacenter. Chúng cho phép các client kết nối tới một host và port duy nhất, và các kết nối đến sẽ được định tuyến tới một trong các server đang chạy service. Các load balancer như vậy sẽ phát hiện các lỗi mạng khi kết nối tới một server hạ nguồn và chuyển lưu lượng (traffic) sang các server khác.

Software load balancers (chẳng hạn như NGINX và HAProxy)

Chúng hoạt động theo cách gần như tương tự với hardware load balancers, nhưng thay vì yêu cầu một thiết bị chuyên dụng, chúng là các ứng dụng có thể được cài đặt trên một máy tiêu chuẩn.

Dịch vụ Tên miền (*Domain Name Service* - DNS)

Đây là cách các tên miền được phân giải trên internet khi bạn mở một trang web. Nó hỗ trợ load balancing bằng cách cho phép nhiều địa chỉ IP được liên kết với một tên miền duy nhất. Sau đó, các client có thể được cấu hình để kết nối tới một service thông qua một tên miền thay vì một địa chỉ IP, và lớp mạng của client sẽ chọn địa chỉ IP nào để sử dụng khi thực hiện kết nối. Một nhược điểm của cách tiếp cận này là DNS được thiết kế để lan truyền các thay đổi trong khoảng thời gian dài hơn và để cache các mục nhập DNS. Nếu các server được khởi động, dừng lại, hoặc di chuyển thường xuyên, các client có thể thấy các địa chỉ IP cũ (stale) mà không còn server nào đang chạy trên đó nữa.

Các hệ thống Service discovery

Các hệ thống này sử dụng một registry tập trung như etcd hoặc Apache ZooKeeper thay vì DNS để theo dõi các service endpoint nào đang khả dụng (chúng ta sẽ quay lại các hệ thống này trong mục “Coordination Services”). Khi một service instance mới khởi động, nó tự đăng ký với hệ thống service discovery bằng cách khai báo host và port mà nó đang lắng nghe, cùng với các metadata liên quan như thông tin sở hữu shard (xem Chương 7), vị trí datacenter, và nhiều thông tin khác. Sau đó, service định kỳ gửi một tín hiệu heartbeat tới hệ thống discovery để báo hiệu rằng service vẫn đang khả dụng.

Khi một client muốn kết nối tới một service, trước tiên nó truy vấn hệ thống discovery để lấy danh sách các endpoint khả dụng, sau đó kết nối trực tiếp tới endpoint đó. So với DNS, service discovery hỗ trợ một môi trường năng động hơn nhiều, nơi các service instance thay đổi thường xuyên. Các hệ thống discovery cũng cung cấp cho client nhiều metadata hơn về service mà chúng đang

kết nối, điều này cho phép các client đưa ra các quyết định load-balancing thông minh hơn.

Service meshes

Hình thức load balancing tinh vi này kết hợp các software load balancer và service discovery. Không giống như các software load balancer truyền thống vốn chạy trên một máy riêng biệt, một service mesh load balancer thường được triển khai dưới dạng một client library trong tiến trình (in-process) hoặc dưới dạng một process hoặc "sidecar" container trên cả client và server. Các ứng dụng client kết nối với load balancer dịch vụ cục bộ của chính chúng, sau đó chúng sẽ kết nối với load balancer của server. Từ đó, kết nối được định tuyến đến process server cục bộ.

Mặc dù phức tạp, nhưng cấu trúc topology này mang lại nhiều lợi thế. Vì các client và server được định tuyến hoàn toàn thông qua các kết nối cục bộ, việc mã hóa kết nối có thể được xử lý hoàn toàn ở cấp độ load balancer. Điều này giúp bảo vệ client và server khỏi việc phải xử lý sự phức tạp của các chứng chỉ SSL và TLS. Các hệ thống mesh cũng cung cấp khả năng observability tinh vi. Chúng có thể theo dõi các service nào đang gọi lẫn nhau trong thời gian thực, phát hiện lỗi, theo dõi tải traffic, và nhiều hơn thế nữa.

Giải pháp nào phù hợp sẽ tùy thuộc vào nhu cầu của một tổ chức. Những đơn vị đang vận hành trong một môi trường service rất năng động với một orchestrator như Kubernetes thường chọn chạy một service mesh như Istio hoặc Linkerd. Các hạ tầng chuyên dụng như cơ sở dữ liệu hoặc hệ thống messaging có thể yêu cầu các load balancer được xây dựng riêng cho mục đích đó. Các deployment đơn giản hơn thì sẽ được phục vụ tốt nhất bởi các software load balancer.

Mã hóa dữ liệu và sự tiến hóa cho RPC

Để có khả năng tiến hóa (evolvability), điều quan trọng là các RPC client và server có thể được thay đổi và triển khai một cách độc lập. So với việc dữ liệu chảy qua các cơ sở dữ liệu (như đã mô tả trong mục "Dataflow Through Databases"), chúng ta có thể đưa ra một giả định đơn giản hóa trong trường hợp dữ liệu chảy qua các service: việc giả định rằng tất cả các server sẽ được cập nhật trước và tất cả các client sẽ được cập nhật sau là hoàn toàn hợp lý. Do đó, bạn chỉ cần backward compatibility cho các yêu cầu (requests) và forward compatibility cho các phản hồi (responses).

Các đặc tính backward compatibility và forward compatibility của một scheme RPC được kế thừa từ bất kỳ phương thức encoding nào mà nó sử dụng:

- gRPC (Protocol Buffers) và Avro RPC có thể được tiến hóa theo các quy tắc tương thích của định dạng encoding tương ứng.
- Các RESTful API thường sử dụng JSON cho các phản hồi và sử dụng JSON hoặc các tham số yêu cầu được mã hóa theo kiểu URI-encoded/form-encoded

cho các yêu cầu. Việc thêm các tham số yêu cầu tùy chọn và thêm các field mới vào các đối tượng phản hồi thường được coi là những thay đổi giúp duy trì tính tương thích.

Khả năng tương thích của Service trở nên khó khăn hơn bởi thực tế là RPC thường được sử dụng để giao tiếp xuyên qua các ranh giới tổ chức, vì vậy bên cung cấp một service thường không có quyền kiểm soát đối với các client của nó và không thể ép buộc chúng phải nâng cấp. Do đó, tính tương thích cần được duy trì trong một thời gian dài, thậm chí có thể là vô thời hạn. Nếu cần một thay đổi gây phá vỡ tính tương thích, bên cung cấp service thường kết thúc bằng việc phải duy trì nhiều phiên bản của service API song song với nhau.

Hiện chưa có sự thống nhất về cách thức hoạt động của việc versioning API (tức là cách một client có thể chỉ định phiên bản API mà nó muốn sử dụng [43]). Đối với các RESTful API, các cách tiếp cận phổ biến là sử dụng một số phiên bản trong URL hoặc trong header HTTP Accept. Đối với các service sử dụng API keys để định danh một client cụ thể, một lựa chọn khác là lưu trữ phiên bản API mà client yêu cầu trên server và cho phép việc lựa chọn phiên bản này được cập nhật thông qua một giao diện quản trị riêng biệt [44].

Durable Execution và Workflows

Theo định nghĩa, các kiến trúc dựa trên service có nhiều service mà tất cả đều chịu trách nhiệm cho các phần khác nhau của một ứng dụng. Hãy xét một ứng dụng xử lý thanh toán thực hiện trừ tiền thẻ tín dụng và gửi tiền vào một tài khoản ngân hàng. Hệ thống này có khả năng sẽ có các service khác nhau chịu trách nhiệm về phát hiện gian lận, tích hợp thẻ tín dụng, tích hợp ngân hàng, v.v.

Việc xử lý một khoản thanh toán duy nhất trong ví dụ của chúng ta đòi hỏi nhiều lượt gọi service. Một service xử lý thanh toán có thể gọi service phát hiện gian lận để kiểm tra gian lận, gọi service thẻ tín dụng để trừ tiền thẻ, và gọi service ngân hàng để gửi số tiền đã trừ vào tài khoản, như được hiển thị trong Hình 5-7. Chúng ta gọi chuỗi các bước này là một *workflow* (luồng công việc), và mỗi bước là một *task* (nhiệm vụ). Các workflow thường được định nghĩa dưới dạng một đồ thị của các task. Các định nghĩa workflow có thể được viết bằng một ngôn ngữ lập trình đa mục đích, một ngôn ngữ chuyên biệt cho miền (DSL), hoặc một ngôn ngữ đánh dấu như Business Process Execution Language (BPEL) [45].

Tasks, activities, và functions

Các workflow engine khác nhau sử dụng các tên gọi khác nhau cho các task. Ví dụ, Temporal sử dụng thuật ngữ *activity*. Những hệ thống khác gọi các task là *durable functions*. Mặc dù tên gọi khác nhau, nhưng các khái niệm là tương đương.

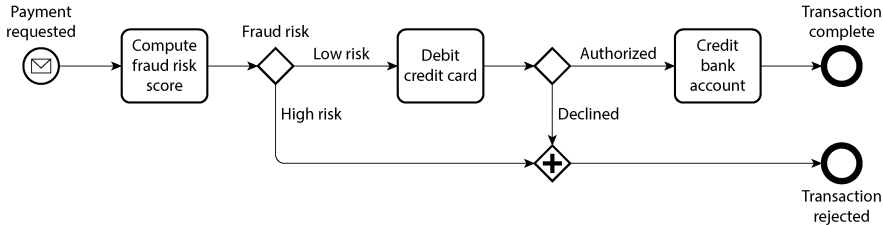


Figure 5-7. Một workflow được thể hiện bằng Business Process Model and Notation (BPMN), một ký hiệu đồ họa

Các workflow được chạy, hoặc thực thi, bởi một *workflow engine*. Các workflow engine quyết định khi nào và trên máy nào để chạy mỗi task, phải làm gì nếu một task thất bại (ví dụ: nếu máy bị crash trong khi task đang chạy), bao nhiêu task được phép thực thi song song, và nhiều hơn nữa.

Các workflow engine thường bao gồm một orchestrator và một executor: *orchestrator* chịu trách nhiệm lập lịch cho các task cần được thực thi, và *executor* chịu trách nhiệm thực thi các task đó. Việc thực thi bắt đầu khi một workflow được kích hoạt. Orchestrator sẽ tự kích hoạt workflow nếu người dùng định nghĩa một lịch trình dựa trên thời gian, chẳng hạn như thực thi hàng giờ. Các nguồn bên ngoài như một web service hoặc thậm chí là con người cũng có thể kích hoạt việc thực thi workflow. Sau khi được kích hoạt, các executor sẽ được gọi để chạy các task.

Có nhiều loại workflow engine khác nhau nhằm giải quyết các tập hợp use case đa dạng. Một số loại, như Airflow, Dagster và Prefect, tích hợp với các hệ thống dữ liệu và điều phối các task ETL. Những loại khác, như Camunda và Orkes, cung cấp một ký hiệu đồ họa cho các workflow (chẳng hạn như BPMN, được sử dụng trong Hình 5-7) để những người không phải kỹ sư có thể định nghĩa và thực thi workflow dễ dàng hơn. Một số loại khác, như Temporal và Restate, cung cấp khả năng *durable execution* (thực thi bền vững).

Các framework durable execution đã trở thành một cách phổ biến để xây dựng các kiến trúc dựa trên service yêu cầu tính transaction (giao dịch). Trong ví dụ về thanh toán của chúng ta, chúng ta muốn xử lý mỗi khoản thanh toán đúng một lần duy nhất. Một lỗi xảy ra trong khi workflow đang thực thi có thể dẫn đến việc thẻ tín dụng bị tính phí nhưng không có khoản tiền tương ứng nào được gửi vào tài khoản ngân hàng. Trong một kiến trúc dựa trên service, chúng ta không thể chỉ đơn giản là

bao bọc hai task đó trong một database transaction. Hơn nữa, chúng ta có thể đang tương tác với các cổng thanh toán của bên thứ ba mà chúng ta có quyền kiểm soát hạn chế.

Các framework durable execution là một cách để cung cấp *exactly-once semantics* (ngữ nghĩa đúng một lần) cho các workflow. Nếu một task thất bại, framework sẽ thực thi lại task đó, nhưng sẽ bỏ qua bất kỳ lời gọi RPC hoặc thay đổi trạng thái nào mà task đã thực hiện thành công trước khi thất bại. Nó sẽ giả vờ thực hiện lời gọi đó, nhưng thay vào đó sẽ trả về kết quả từ lời gọi trước đó. Điều này khả thi vì các framework durable execution ghi lại tất cả các RPC và thay đổi trạng thái vào bộ lưu trữ bền vững như một write-ahead log [46, 47]. Ví dụ 5-5 cho thấy một định nghĩa workflow hỗ trợ durable execution sử dụng Temporal.

Example 5-5. Một đoạn định nghĩa workflow Temporal cho workflow thanh toán trong Hình 5-7

```
@workflow.defn
class PaymentWorkflow:
    @workflow.run
    async def run(self, payment: PaymentRequest) -> PaymentResult:
        is_fraud = await workflow.execute_activity(
            check_fraud,
            payment,
            start_to_close_timeout=timedelta(seconds=15),
        )
        if is_fraud:
            return PaymentResultFraudulent
        credit_card_response = await workflow.execute_activity(
            debit_credit_card,
            payment,
            start_to_close_timeout=timedelta(seconds=15),
        )
        # ...
```

Các framework như Temporal không phải là không có những thách thức. Các dịch vụ bên ngoài, chẳng hạn như cổng thanh toán của bên thứ ba trong ví dụ của chúng ta, vẫn phải cung cấp một API idempotent. Các lập trình viên phải nhớ sử dụng các ID duy nhất cho các API này để ngăn chặn việc thực thi trùng lặp [48]. Và vì các framework durable execution ghi lại từng lời gọi RPC theo thứ tự, chúng kỳ vọng các lần thực thi tiếp theo sẽ thực hiện các lời gọi RPC tương tự theo cùng một thứ tự. Điều này làm cho việc thay đổi code trở nên mong manh; bạn có thể gây ra hành vi không xác định chỉ bằng cách thay đổi thứ tự các lời gọi hàm [49]. Thay vì sửa đổi code của một workflow hiện có, sẽ an toàn hơn nếu triển khai một phiên bản code mới một cách riêng biệt, sao cho việc thực thi lại các lần gọi workflow hiện tại tiếp tục sử dụng phiên bản cũ, và chỉ các lần gọi mới mới sử dụng code mới [50].

Tương tự, vì các framework durable execution kỳ vọng sẽ replay tất cả code một cách deterministic (xác định) (cùng một đầu vào tạo ra cùng một đầu ra), nên các code

nondeterministic như gọi các bộ tạo số ngẫu nhiên hoặc đồng hồ hệ thống sẽ gây ra vấn đề [49]. Các framework thường cung cấp các implementation deterministic của riêng chúng cho các hàm thư viện như vậy, nhưng bạn phải nhớ sử dụng chúng. Một số framework cũng cung cấp các công cụ phân tích tĩnh, như Workflow Check của Temporal, để xác định xem liệu có hành vi nondeterministic nào đã được đưa vào hay không.

LƯU Ý

Việc làm cho code trở nên deterministic là một ý tưởng mạnh mẽ nhưng rất khó để thực hiện một cách robust. Chúng ta sẽ quay lại chủ đề này trong Chương 9.

Kiến trúc Event-Driven

Trong phần cuối cùng này, chúng ta sẽ xem xét ngắn gọn về *event-driven architectures* (kiến trúc hướng sự kiện), một cách khác để dữ liệu đã được mã hóa có thể chảy từ tiến trình này sang tiến trình khác. Trong ngữ cảnh này, một yêu cầu được gọi là một *event* (sự kiện) hoặc *message* (thông điệp). Không giống như với RPC, bên gửi thường không đợi bên nhận xử lý sự kiện đó. Thêm vào đó, các sự kiện thường không được gửi đến bên nhận thông qua một kết nối mạng trực tiếp, mà đi qua một trung gian gọi là *message broker* (bộ môi giới thông điệp) (còn được gọi là *event broker*, *message queue*, hoặc *message-oriented middleware*), nơi lưu trữ thông điệp tạm thời [51].

Việc sử dụng một message broker mang lại nhiều lợi thế so với RPC trực tiếp:

- Nó có thể đóng vai trò như một bộ đệm nếu bên nhận không khả dụng hoặc bị quá tải, giúp cải thiện độ tin cậy của hệ thống.
- Nó có thể tự động gửi lại các thông điệp cho một tiến trình đã bị crash, giúp ngăn chặn việc mất thông điệp.
- Nó tránh được nhu cầu về service discovery, vì bên gửi không cần kết nối trực tiếp đến địa chỉ IP của bên nhận.
- Nó cho phép cùng một thông điệp được gửi đến nhiều bên nhận.
- Nó tách rời (decouple) về mặt logic giữa bên gửi và bên nhận (bên gửi chỉ cần publish các thông điệp và không cần quan tâm ai là người consume chúng).

Việc giao tiếp thông qua một message broker là *asynchronous* (bất đồng bộ): bên gửi không đợi thông điệp được chuyển đi, mà chỉ đơn giản là gửi nó rồi thôi. Tuy nhiên, ta hoàn toàn có thể triển khai một mô hình giống như RPC đồng bộ bằng cách để bên gửi đợi phản hồi trên một kênh riêng biệt.

Message brokers

Trong quá khứ, bối cảnh của các message broker bị thống trị bởi các phần mềm doanh nghiệp thương mại từ các công ty như TIBCO, IBM WebSphere và webMethods, trước khi các triển khai mã nguồn mở như RabbitMQ, ActiveMQ, HornetQ, NATS, Redpanda và Apache Kafka trở nên phổ biến. Gần đây hơn, các dịch vụ cloud như Amazon Kinesis, Azure Service Bus và Google Cloud Pub/Sub đã được áp dụng rộng rãi. Chúng ta sẽ so sánh chúng chi tiết hơn trong Chương 12.

Các ngữ nghĩa chuyển giao (delivery semantics) chi tiết sẽ khác nhau tùy theo cách triển khai và cấu hình, nhưng nhìn chung, có hai pattern phân phối thông điệp thường được sử dụng nhất:

- Một tiến trình thêm một thông điệp vào một *queue* (hàng đợi) đã được đặt tên, và một *consumer* (người tiêu thụ) của hàng đợi đó sẽ nhận được thông điệp. Nếu có nhiều consumer, một trong số chúng sẽ nhận được thông điệp.
- Một tiến trình publish một thông điệp vào một *topic* (chủ đề) đã được đặt tên, và broker sẽ chuyển thông điệp đó đến tất cả các *subscribers* (người đăng ký) của topic đó. Nếu có nhiều subscriber, tất cả chúng đều sẽ nhận được thông điệp.

Các message broker thường không áp đặt bất kỳ mô hình dữ liệu cụ thể nào. Một thông điệp chỉ là một chuỗi các byte cùng với một số metadata, vì vậy bạn có thể sử dụng bất kỳ định dạng mã hóa nào. Một cách tiếp cận phổ biến là sử dụng Protocol Buffers, Avro, hoặc JSON, và triển khai một schema registry cùng với message broker để lưu trữ tất cả các phiên bản schema hợp lệ và kiểm tra tính tương thích của chúng [20, 22]. AsyncAPI, một phiên bản tương đương với OpenAPI dựa trên messaging, cũng có thể được sử dụng để chỉ định schema của các thông điệp.

Các message broker khác nhau về tính bền vững (durability) của thông điệp. Nhiều broker ghi thông điệp vào đĩa để chúng không bị mất nếu message broker bị crash hoặc cần được khởi động lại. Không giống như các cơ sở dữ liệu, nhiều message broker tự động xóa thông điệp sau khi chúng đã được consume. Tuy nhiên, một số broker có thể được cấu hình để lưu trữ thông điệp vô thời hạn, điều mà bạn sẽ cần nếu muốn sử dụng event sourcing (xem mục “Event Sourcing and CQRS”).

Nếu một consumer republish các thông điệp sang một topic khác, bạn có thể cần cẩn thận để bảo toàn các trường (fields) không xác định, nhằm ngăn chặn vấn đề đã được mô tả trước đó trong ngữ cảnh của các cơ sở dữ liệu (Hình 5-1).

Các framework actor phân tán

actor model (mô hình actor) là một mô hình lập trình dành cho tính đồng thời trong một process duy nhất. Thay vì xử lý trực tiếp các thread (và các vấn đề liên quan như race condition, locking, và deadlock), logic được đóng gói trong các *actor*. Mỗi actor thường đại diện cho một client hoặc một thực thể. Nó có thể có một số trạng thái

cục bộ (không được chia sẻ với bất kỳ actor nào khác), và nó giao tiếp với các actor khác bằng cách gửi và nhận các message bất đồng bộ. Việc truyền message không được đảm bảo; trong một số kịch bản lỗi nhất định, các message sẽ bị mất. Vì mỗi actor chỉ xử lý một message tại một thời điểm, nó không cần lo lắng về các thread, và mỗi actor có thể được lập lịch độc lập bởi framework.

Trong các *distributed actor frameworks* như Akka, Orleans [52], và Erlang/OTP, mô hình lập trình này được sử dụng để scale một ứng dụng trên nhiều node. Cơ chế truyền message tương tự nhau được sử dụng, bất kể bên gửi và bên nhận nằm trên cùng một node hay các node khác nhau. Nếu chúng nằm trên các node khác nhau, message sẽ được mã hóa một cách minh bạch thành một chuỗi byte, được gửi qua mạng, và được giải mã ở phía bên kia.

Tính minh bạch về vị trí (location transparency) hoạt động tốt hơn trong actor model so với RPC, bởi vì actor model đã mặc định rằng các message có thể bị mất, ngay cả trong cùng một process. Mặc dù latency qua mạng có khả năng cao hơn so với trong cùng một process, nhưng sự sai lệch (mismatch) cơ bản giữa giao tiếp cục bộ và giao tiếp từ xa là ít hơn khi sử dụng actor model.

Một distributed actor framework về cơ bản tích hợp một message broker và mô hình lập trình actor vào trong một framework duy nhất. Tuy nhiên, nếu bạn muốn thực hiện rolling upgrades cho ứng dụng dựa trên actor của mình, bạn vẫn phải lo lắng về tính tương thích xuôi (forward compatibility) và tương thích ngược (backward compatibility), vì các message có thể được gửi từ một node đang chạy phiên bản mới đến một node đang chạy phiên bản cũ, và ngược lại. Điều này có thể đạt được bằng cách sử dụng một trong các phương pháp mã hóa đã được thảo luận trong chương này.

Tóm tắt

Trong chương này, chúng ta đã xem xét một số cách để chuyển đổi các cấu trúc dữ liệu thành các byte trên mạng hoặc trên đĩa. Chúng ta đã thấy chi tiết của các phương pháp mã hóa này ảnh hưởng không chỉ đến hiệu năng của chúng, mà quan trọng hơn là đến kiến trúc của các ứng dụng và các lựa chọn của bạn để tiến hóa chúng.

Cụ thể, nhiều service cần hỗ trợ rolling upgrades, nơi một phiên bản mới của service được triển khai dần dần tới một vài node mỗi lần thay vì tới tất cả các node cùng một lúc. Rolling upgrades cho phép các phiên bản mới của một service được phát hành mà không gây downtime (do đó khuyến khích các đợt phát hành nhỏ thường xuyên thay vì các đợt phát hành lớn hiếm hoi) và giúp việc deployment ít rủi ro hơn (cho phép phát hiện và roll back các bản phát hành lỗi trước khi chúng ảnh hưởng đến một lượng lớn người dùng). Những đặc tính này cực kỳ có lợi cho *evolvability* (khả năng tiến hóa), tức là sự dễ dàng trong việc thực hiện các thay đổi đối với một ứng dụng.

Trong quá trình rolling upgrades, hoặc vì các lý do khác, chúng ta phải giả định rằng các node khác nhau đang chạy các phiên bản khác nhau của mã nguồn ứng dụng. Do đó, điều quan trọng là tất cả dữ liệu lưu thông trong hệ thống phải được mã hóa theo cách cung cấp tính tương thích ngược (mã mới có thể đọc dữ liệu cũ) và tương thích xuôi (mã cũ có thể đọc dữ liệu mới).

Chúng ta đã thảo luận về một số định dạng mã hóa dữ liệu và các đặc tính tương thích của chúng:

- Các phương pháp mã hóa đặc thù cho ngôn ngữ lập trình bị giới hạn trong một ngôn ngữ lập trình duy nhất và thường không cung cấp được tính tương thích xuôi và tương thích ngược.
- Các định dạng văn bản như JSON, XML, và CSV rất phổ biến, và tính tương thích của chúng phụ thuộc vào cách bạn sử dụng chúng. Chúng có các ngôn ngữ schema tùy chọn, đôi khi hữu ích nhưng cũng có lúc là một trở ngại. Các định dạng này hơi mơ hồ về các kiểu dữ liệu, vì vậy bạn phải cẩn thận với những thứ như số và chuỗi nhị phân.
- Các định dạng dựa trên schema nhị phân như Protocol Buffers và Avro cho phép mã hóa nhỏ gọn, hiệu quả với các ngữ nghĩa về khả năng tương thích forward và backward được định nghĩa rõ ràng. Các schema này có thể hữu ích cho việc làm tài liệu và tạo mã trong các ngôn ngữ có kiểu tĩnh (statically typed languages). Tuy nhiên, các định dạng này có nhược điểm là dữ liệu cần phải được giải mã trước khi con người có thể đọc được.

Chúng ta cũng đã thảo luận về một số chế độ dataflow, minh họa các kịch bản khác nhau mà trong đó việc mã hóa dữ liệu đóng vai trò quan trọng:

Cơ sở dữ liệu

Tiến trình ghi vào cơ sở dữ liệu mã hóa dữ liệu và tiến trình đọc từ cơ sở dữ liệu giải mã nó

RPC và REST APIs

Client mã hóa một request, server giải mã request đó và mã hóa một response, và cuối cùng client giải mã response

Kiến trúc event-driven (sử dụng message broker hoặc actor)

Các node giao tiếp bằng cách gửi cho nhau các message đã được mã hóa bởi bên gửi và giải mã bởi bên nhận

Chúng ta có thể kết luận rằng với một chút cẩn trọng, khả năng tương thích backward/forward và rolling upgrade là hoàn toàn khả thi. Chúc cho sự tiến hóa ứng dụng của bạn diễn ra nhanh chóng và các deployment của bạn diễn ra thường xuyên.

Footnotes

References

- [1] “CWE-502: Deserialization of Untrusted Data.” Common Weakness Enumeration, *cwe.mitre.org*, July 2006. Archived at perma.cc/26EU-UK9Y
- [2] Steve Breen. “What Do WebLogic, WebSphere, JBoss, Jenkins, OpenNMS, and Your Application Have in Common? This Vulnerability.” *foxglovesecurity.com*, November 2015. Archived at perma.cc/9U97-UVVD
- [3] Patrick McKenzie. “What the Rails Security Issue Means for Your Startup.” *kalzumemus.com*, January 2013. Archived at perma.cc/2MBJ-7PZ6
- [4] Brian Goetz. “Towards Better Serialization.” *openjdk.org*, June 2019. Archived at perma.cc/UK6U-GQDE
- [5] Eishay Smith. “jvm-serializers Wiki.” *github.com*, October 2023. Archived at perma.cc/PJP7-WCNG
- [6] “XML Is a Poor Copy of S-Expressions.” *wiki.c2.com*, May 2013. Archived at perma.cc/7FAN-YBKL
- [7] Julia Evans. “Examples of Floating Point Problems.” *jvns.ca*, January 2023. Archived at perma.cc/M57L-QKKW
- [8] Matt Harris. “Snowflake: An Update and Some Very Important Information.” Email to *Twitter Development Talk* mailing list, October 2010. Archived at perma.cc/8UBV-MZ3D
- [9] Yakov Shafranovich. “RFC 4180: Common Format and MIME Type for Comma-Separated Values (CSV) Files.” IETF, October 2005.
- [10] Andy Coates. “Evolving JSON Schemas—Part I.” *creekservice.org*, January 2024. Archived at perma.cc/MZW3-UA54
- [11] Andy Coates. “Evolving JSON Schemas—Part II.” *creekservice.org*, January 2024. Archived at perma.cc/GT5H-WKZ5
- [12] Pierre Genevès, Nabil Layaïda, and Vincent Quint. “Ensuring Query Compatibility with Evolving XML Schemas.” INRIA Technical Report 6711, November 2008. Archived at *arxiv.org*
- [13] Tim Bray. “Bits on the Wire.” *tbray.org*, November 2019. Archived at perma.cc/3BT3-BQU3
- [14] Mark Slee, Aditya Agarwal, and Marc Kwiatkowski. “Thrift: Scalable Cross-Language Services Implementation.” Facebook Technical Report, April 2007. Archived at perma.cc/22BS-TUFB
- [15] Martin Kleppmann. “Schema Evolution in Avro, Protocol Buffers and Thrift.” *martin.kleppmann.com*, December 2012. Archived at perma.cc/E4R2-9RJT
- [16] Doug Cutting et al. “[PROPOSAL] New Subproject: Avro.” Email thread on *hadoop-general* mailing list, *lists.apache.org*, April 2009. Archived at perma.cc/4A79-BMEB
- [17] Apache Software Foundation. “Apache Avro 1.12.0 Specification.” *avro.apache.org*, August 2024. Archived at perma.cc/C36P-5EBQ
- [18] Apache Software Foundation. “Avro Schemas as LL(1) CFG Definitions.” *avro.apache.org*, August 2024. Archived at perma.cc/JB44-EM9Q
- [19] Tony Hoare. “Null References: The Billion Dollar Mistake.” At *QCon London*, March 2009.
- [20] Confluent, Inc. “Schema Registry Overview.” *docs.confluent.io*, 2024. Archived at perma.cc/92C3-A9JA
- [21] Aditya Auradkar and Tom Quiggle. “Introducing Espresso—LinkedIn’s Hot New Distributed Document Store.” *engineering.linkedin.com*, January 2015. Archived at perma.cc/FX4P-VW9T

- [22] Jay Kreps. “Putting Apache Kafka to Use: A Practical Guide to Building a Stream Data Platform (Part 2).” *confluent.io*, February 2015. Archived at perma.cc/8UA4-ZS5S
- [23] Gwen Shapira. “The Problem of Managing Schemas.” *oreilly.com*, November 2014. Archived at perma.cc/BY8Q-RYV3
- [24] John Larmouth. *ASN.1 Complete*. Morgan Kaufmann, 1999. ISBN: 9780122334351. Archived at perma.cc/GB7Y-XSXQ
- [25] Burton S. Kaliski Jr. “A Layman’s Guide to a Subset of ASN.1, BER, and DER.” Technical Note, RSA Data Security, Inc., November 1993. Archived at perma.cc/2LMN-W9U8
- [26] Jacob Hoffman-Andrews. “A Warm Welcome to ASN.1 and DER.” *letsencrypt.org*, April 2020. Archived at perma.cc/CYT2-GPQ8
- [27] Lev Walkin. “Question: Extensibility and Dropping Fields.” *lionet.info*, September 2010. Archived at perma.cc/VX8E-NLH3
- [28] Jacqueline Xu. “Online Migrations at Scale.” *stripe.com*, February 2017. Archived at perma.cc/X59W-DK7Y
- [29] Geoffrey Litt, Peter van Hardenberg, and Orion Henry. “Project Cambria: Translate Your Data with Lenses.” Technical Report, October 2020. Archived at perma.cc/WA4V-VKDB
- [30] Pat Helland. “Data on the Outside Versus Data on the Inside.” At *2nd Biennial Conference on Innovative Data Systems Research* (CIDR), January 2005. Archived at perma.cc/GH56-WYZS
- [31] Roy Thomas Fielding. “Architectural Styles and the Design of Network-Based Software Architectures.” PhD thesis, University of California, Irvine, 2000. Archived at perma.cc/LWY9-7BPE
- [32] Roy Thomas Fielding. “REST APIs Must Be Hypertext-Driven.” *roy.gbiv.com*, October 2008. Archived at perma.cc/M2ZW-8ATG
- [33] “OpenAPI Specification Version 3.1.0.” *swagger.io*, February 2021. Archived at perma.cc/3S6S-K5M4
- [34] Michi Henning. “The Rise and Fall of CORBA.” *Communications of the ACM*, volume 51, issue 8, pages 52–57, August 2008. doi:10.1145/1378704.1378718
- [35] Pete Lacey. “The S Stands for Simple.” *harmful.cat-v.org*, November 2006. Archived at perma.cc/4PMK-Z9X7
- [36] Stefan Tilkov. “Interview: Pete Lacey Criticizes Web Services.” *infoq.com*, December 2006. Archived at perma.cc/JWF4-XY3P
- [37] Tim Bray. “The Loyal WS-Opposition.” *tbray.org*, September 2004. Archived at perma.cc/J5Q8-69Q2
- [38] Andrew D. Birrell and Bruce Jay Nelson. “Implementing Remote Procedure Calls.” *ACM Transactions on Computer Systems* (TOCS), volume 2, issue 1, pages 39–59, February 1984. doi:10.1145/2080.357392
- [39] Jim Waldo, Geoff Wyant, Ann Wollrath, and Sam Kendall. “A Note on Distributed Computing.” Sun Microsystems Laboratories, Inc., Technical Report TR-94-29, November 1994. Archived at perma.cc/8LRZ-BSZR
- [40] Steve Vinoski. “Convenience over Correctness.” *IEEE Internet Computing*, volume 12, issue 4, pages 89–92, July 2008. doi:10.1109/MIC.2008.75
- [41] Brandur Leach. “Designing Robust and Predictable APIs with Idempotency.” *stripe.com*, February 2017. Archived at perma.cc/JD22-XZQT

- [42] Sam Rose. “Load Balancing.” *samwho.dev*, April 2023. Archived at perma.cc/Q7BA-9AE2
- [43] Troy Hunt. “Your API Versioning Is Wrong, Which Is Why I Decided to Do It 3 Different Wrong Ways.” *troyhunt.com*, February 2014. Archived at perma.cc/9DSW-DGR5
- [44] Brandur Leach. “APIs As Infrastructure: Future-Proofing Stripe with Versioning.” *stripe.com*, August 2017. Archived at perma.cc/L63K-USFW
- [45] AOASIS Web Services Business Process Execution Language (WSBPEL) Technical Committee. “Web Services Business Process Execution Language Version 2.0.” *docs.oasis-open.org*, April 2007.
- [46] “Temporal. Temporal Service.” *docs.temporal.io*, 2024. Archived at perma.cc/32P3-CJ9V
- [47] Stephan Ewen. “Why We Built Restate.” *restate.dev*, August 2023. Archived at perma.cc/BJJ2-X75K
- [48] Keith Tenzer and Joshua Smith. “Understanding Idempotency in Distributed Systems.” *temporal.io*, February 2024. Archived at perma.cc/TY4U-EH3W
- [49] “Temporal. Temporal Workflow.” *docs.temporal.io*, 2024. Archived at perma.cc/B5C5-Y396
- [50] Jack Kleeman. “Solving Durable Execution’s Immutability Problem.” *restate.dev*, February 2024. Archived at perma.cc/G55L-EYH5
- [51] Srinath Perera. “Exploring Event-Driven Architecture: A Beginner’s Guide for Cloud Native Developers.” *wso2.com*, August 2023. Archived at archive.org
- [52] Philip A. Bernstein, Sergey Bykov, Alan Geller, Gabriel Kliot, and Jorgen Thelin. “Orleans: Distributed Virtual Actors for Programmability and Scalability.” Microsoft Research Technical Report MSR-TR-2014-41, March 2014. Archived at perma.cc/PD3U-WDMF

Replication

Sự khác biệt chính giữa một thứ có thể xảy ra lỗi và một thứ không thể nào có thể xảy ra lỗi là khi một thứ không thể nào có thể xảy ra lỗi mà lại xảy ra lỗi, thì nó thường trở nên không thể tiếp cận hay sửa chữa được.

Douglas Adams, *Mostly Harmless* (1992)

Replication (nhân bản dữ liệu) có nghĩa là giữ một bản sao của cùng một dữ liệu trên nhiều máy tính được kết nối thông qua một mạng. Như đã thảo luận trong mục “Distributed Versus Single-Node Systems”, có một vài lý do khiến bạn có thể muốn replication dữ liệu, bao gồm:

- Để giữ dữ liệu gần về mặt địa lý với người dùng của bạn (và do đó giảm latency (độ trễ))
- Để cho phép hệ thống tiếp tục hoạt động ngay cả khi một số thành phần của nó đã gặp lỗi (và do đó tăng availability và durability)
- Để scale out số lượng máy tính có thể phục vụ các truy vấn đọc (và do đó tăng read throughput (throughput đọc))

Trong chương này, chúng ta sẽ giả định rằng dataset của bạn đủ nhỏ để mỗi máy tính có thể giữ một bản sao của toàn bộ dataset. Trong Chương 7, chúng ta sẽ nói lỏng giả định đó và thảo luận về *sharding* (*partitioning*) các dataset quá lớn đối với một máy tính duy nhất. Trong các chương sau, chúng ta sẽ thảo luận về các loại fault khác nhau có thể xảy ra trong một hệ thống dữ liệu được replication và cách xử lý chúng.

Nếu dữ liệu mà bạn đang replication không thay đổi theo thời gian, việc replication sẽ rất dễ dàng; bạn chỉ cần sao chép dữ liệu tới mọi node một lần là xong. Mọi khó khăn trong replication nằm ở việc xử lý các *thay đổi* đối với dữ liệu được replication, và đó chính là nội dung của chương này. Chúng ta sẽ thảo luận về ba họ thuật toán để replication các thay đổi giữa các node: *single-leader*, *multi-leader*, và *leaderless* replication. Hầu hết tất cả các distributed database đều sử dụng một trong ba cách tiếp cận này. Mỗi cách đều có ưu và nhược điểm mà chúng ta sẽ xem xét chi tiết.

Có nhiều trade-off (đánh đổi) cần xem xét với replication—ví dụ, liệu nên sử dụng synchronous replication hay asynchronous replication, và cách xử lý các replica bị lỗi. Đó thường là các tùy chọn cấu hình trong các database, và mặc dù các chi tiết khác nhau tùy theo từng database, các nguyên tắc chung vẫn tương tự nhau giữa nhiều triển khai khác nhau. Chúng ta sẽ thảo luận về hệ quả của những lựa chọn như vậy trong chương này.

Replication của các database là một chủ đề cũ. Các nguyên tắc không thay đổi nhiều kể từ khi chúng được nghiên cứu vào những năm 1970 [1] vì các ràng buộc cơ bản của mạng vẫn giữ nguyên. Tuy nhiên, các khái niệm như *eventual consistency* vẫn gây ra sự nhầm lẫn. Trong mục “Problems with Replication Lag”, chúng ta sẽ làm rõ hơn về *eventual consistency* và thảo luận về các đảm bảo như *read-your-writes* và *monotonic reads*.

Backups và Replication

Bạn có thể đang thắc mắc liệu mình có còn cần backups nếu đã có replication hay không. Câu trả lời là có, bởi vì chúng có các mục đích khác nhau: các replica phản ánh nhanh chóng các thao tác ghi từ một node lên các node khác, nhưng backups lưu trữ các snapshot cũ của dữ liệu để bạn có thể quay ngược thời gian. Nếu bạn vô tình xóa mất một số dữ liệu, replication sẽ không giúp ích gì vì việc xóa đó cũng đã được lan truyền tới các replica; bạn cần một backup nếu muốn khôi phục dữ liệu đã bị xóa.

Trên thực tế, replication và backups thường hỗ trợ cho nhau. Backups đôi khi là một phần của quá trình thiết lập replication, như chúng ta sẽ thấy trong mục “Setting Up New Followers”. Ngược lại, việc lưu trữ (archiving) các replication logs có thể là một phần của quá trình backup.

Một số database duy trì các snapshot bất biến (immutable) của các trạng thái trong quá khứ một cách nội bộ, đóng vai trò như một loại backup nội bộ. Tuy nhiên, điều này đồng nghĩa với việc phải giữ các phiên bản cũ của dữ liệu trên cùng một phương tiện lưu trữ với trạng thái hiện tại. Nếu bạn có một lượng lớn dữ liệu, việc giữ các backups của dữ liệu cũ trong một object store được tối ưu hóa cho dữ liệu ít được truy cập sẽ rẻ hơn, và chỉ lưu trữ trạng thái hiện tại của database trong bộ lưu trữ chính (primary storage).

Single-Leader Replication

Mỗi node lưu trữ một bản sao của database được gọi là một *replica*. Với nhiều replica, một câu hỏi tất yếu nảy sinh: làm thế nào để chúng ta đảm bảo rằng tất cả dữ liệu cuối cùng đều nằm trên tất cả các replica?

Mọi thao tác ghi vào cơ sở dữ liệu đều cần được xử lý bởi mọi replica; nếu không, các replica sẽ không còn chứa cùng một dữ liệu. Giải pháp phổ biến nhất được gọi là

replication *leader-based*, *primary-backup*, hoặc *active/passive*. Nó hoạt động như sau (xem Hình 6-1):

1. Một trong các replica được chỉ định làm *leader* (còn được gọi là *primary* hoặc *source* [2]). Khi các client muốn ghi vào cơ sở dữ liệu, chúng phải gửi yêu cầu của mình tới leader, nơi trước tiên sẽ ghi dữ liệu mới vào bộ lưu trữ cục bộ của nó.
2. Các replica còn lại được gọi là *followers* (hoặc *read replicas*, *secondaries*, hoặc *hot standbys*). Bất cứ khi nào leader ghi dữ liệu mới vào bộ lưu trữ cục bộ, nó cũng gửi thay đổi dữ liệu đó tới tất cả các follower của mình như một phần của *replication log* hoặc *change stream*. Mỗi follower nhận log từ leader và cập nhật bản sao cục bộ của cơ sở dữ liệu một cách tương ứng, bằng cách áp dụng tất cả các thao tác ghi theo cùng một thứ tự như chúng đã được xử lý trên leader.
3. Khi một client muốn đọc từ cơ sở dữ liệu, nó có thể truy vấn leader hoặc bất kỳ follower nào. Tuy nhiên, các thao tác ghi chỉ được chấp nhận bởi leader (từ góc độ của client, các follower là chỉ đọc).

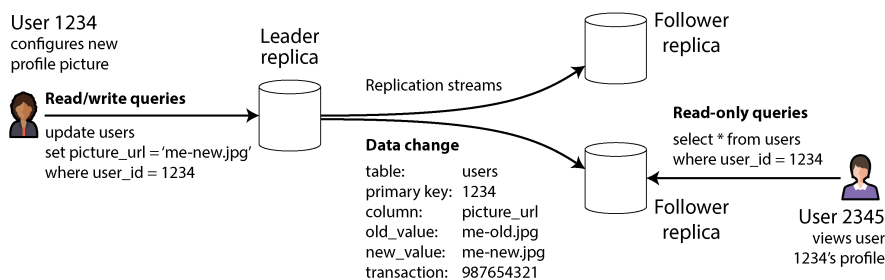


Figure 6-1. *Single-leader replication điều hướng tất cả các thao tác ghi tới một leader được chỉ định, sau đó leader này sẽ gửi một luồng các thay đổi tới các follower replica.*

Nếu cơ sở dữ liệu được sharding (xem Chương 7), mỗi shard sẽ có một leader. Các shard khác nhau có thể có các leader nằm trên các node khác nhau, nhưng mỗi shard vẫn phải có một node leader duy nhất. Trong mục “Multi-Leader Replication”, chúng ta sẽ thảo luận về một mô hình thay thế, trong đó một hệ thống có thể có nhiều leader cho cùng một shard tại cùng một thời điểm.

Single-leader replication được sử dụng rất rộng rãi. Nó là một tính năng tích hợp sẵn của nhiều cơ sở dữ liệu quan hệ, chẳng hạn như PostgreSQL, MySQL, Oracle Data Guard [3], và các availability groups Always On của SQL Server [4]. Nó cũng được sử dụng trong một số document databases (như MongoDB và DynamoDB [5]), các message broker như Kafka, các thiết bị khối được replication như DRBD, và một số network filesystems. Nhiều thuật toán consensus—chẳng hạn như Raft, được sử dụng để replication trong CockroachDB [6], TiDB [7], etcd, và các quorum queues của RabbitMQ (cùng một số loại khác)—cũng dựa trên một single leader và sẽ tự

động bầu chọn một leader mới nếu leader cũ gặp lỗi (chúng ta sẽ thảo luận chi tiết hơn về consensus trong Chương 10).

LƯU Ý

Trong các tài liệu cũ, bạn có thể thấy thuật ngữ *master-slave replication*. Nó có nghĩa tương tự như leader-based replication, nhưng thuật ngữ này nên được tránh vì nó bị coi là gây xúc phạm [8].

Nhân bản dữ liệu đồng bộ so với bất đồng bộ

Một chi tiết quan trọng của một hệ thống được replication là liệu việc replication diễn ra *synchronously* hay *asynchronously*. (Trong các cơ sở dữ liệu quan hệ, đây thường là một tùy chọn có thể cấu hình; các hệ thống khác thường được hardcoded để là một trong hai loại này.)

Hãy xét những gì xảy ra trong Hình 6-1, nơi người dùng của một website cập nhật ảnh hồ sơ của họ. Tại một thời điểm nào đó, client gửi yêu cầu cập nhật tới leader; ngay sau đó, yêu cầu được leader tiếp nhận. Sau đó, leader chuyển tiếp thay đổi dữ liệu tới các follower và thông báo cho client rằng việc cập nhật đã thành công. Hình 6-2 cho thấy một cách khả thi mà các mốc thời gian có thể diễn ra.

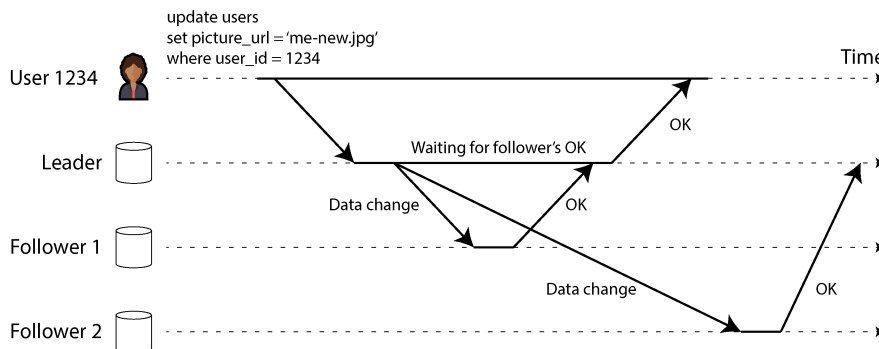


Figure 6-2. *Leader-based replication với một follower synchronous và một follower asynchronous*

Trong ví dụ này, việc replication tới follower 1 là *synchronous* (đồng bộ): leader sẽ đợi cho đến khi follower 1 xác nhận rằng nó đã nhận được lệnh write trước khi báo thành công cho người dùng và trước khi làm cho lệnh write đó hiển thị với các client khác. Việc replication tới follower 2 là *asynchronous* (bất đồng bộ) (hoặc *nonblocking*): leader gửi thông điệp đi nhưng không đợi phản hồi từ follower.

Hình vẽ cho thấy một sự chậm trễ đáng kể trước khi follower 2 xử lý thông điệp. Thông thường, replication diễn ra khá nhanh; hầu hết các hệ thống cơ sở dữ liệu áp dụng các thay đổi cho các follower trong chưa đầy một giây. Tuy nhiên, không có gì đảm bảo việc này sẽ mất bao lâu. Trong một số trường hợp, các follower có thể bị tụt

lại so với leader tới vài phút hoặc hơn—ví dụ, nếu một follower đang phục hồi sau một lỗi, nếu hệ thống đang hoạt động gần mức công suất tối đa, hoặc nếu có các vấn đề về mạng giữa các node.

Ưu điểm của synchronous replication là follower được đảm bảo có một bản sao dữ liệu cập nhật nhất và nhất quán với dữ liệu của leader. Nếu leader đột ngột gặp lỗi, chúng ta có thể chắc chắn rằng dữ liệu vẫn khả dụng trên follower. Nhược điểm là nếu follower đang chạy synchronous không phản hồi (do nó bị crash, hoặc do có lỗi mạng, hoặc vì bất kỳ lý do nào khác), lệnh write sẽ không thể được xử lý. Leader phải chặn tất cả các lệnh write và đợi cho đến khi replica synchronous đó khả dụng trở lại.

Vì lý do đó, việc tất cả các follower đều là synchronous là không khả thi; bất kỳ một sự cố node nào cũng sẽ khiến toàn bộ hệ thống bị đình trệ. Trong thực tế, nếu một cơ sở dữ liệu cung cấp synchronous replication, điều đó thường có nghĩa là *một* trong số các follower là synchronous và các follower còn lại là asynchronous. Nếu follower synchronous trở nên không khả dụng hoặc bị chậm, một trong các follower asynchronous sẽ được chuyển sang chế độ synchronous. Điều này đảm bảo rằng bạn có một bản sao dữ liệu cập nhật nhất trên ít nhất hai node: leader và một follower synchronous. Cấu hình này đôi khi còn được gọi là *semisynchronous*.

Trong một số hệ thống, một *majority* (đa số) các replica (ví dụ: ba trong số năm, bao gồm cả leader) được cập nhật một cách synchronous, và số thiếu số còn lại là asynchronous. Đây là một ví dụ về *quorum* (quorum), điều mà chúng ta sẽ thảo luận kỹ hơn trong mục “Using quorums for reading and writing”. Các majority quorum thường được sử dụng trong các hệ thống eventual consistency hoặc các hệ thống sử dụng một consensus protocol để tự động bầu chọn leader. Chúng ta sẽ quay lại các hệ thống này trong Chương 10.

Đôi khi replication dựa trên leader được cấu hình để hoàn toàn là asynchronous. Trong trường hợp này, nếu leader gặp lỗi và không thể phục hồi, bất kỳ lệnh write nào chưa kịp replication tới các follower sẽ bị mất. Điều này có nghĩa là một lệnh write không được đảm bảo tính durability, ngay cả khi nó đã được xác nhận với client. Tuy nhiên, cấu hình hoàn toàn asynchronous có ưu điểm là leader có thể tiếp tục xử lý các lệnh write, ngay cả khi tất cả các follower của nó đã bị tụt lại phía sau.

Việc làm yếu đi tính durability nghe có vẻ là một sự đánh đổi (trade-off) tồi, nhưng asynchronous replication vẫn được sử dụng rộng rãi, đặc biệt nếu có nhiều follower hoặc nếu chúng được phân bố về mặt địa lý [9]. Chúng ta sẽ quay lại vấn đề này trong mục “Problems with Replication Lag”.

Thiết lập các Follower mới

Thỉnh thoảng, bạn cần thiết lập các follower mới—có lẽ để tăng số lượng replica hoặc để thay thế các node bị lỗi. Làm thế nào để bạn đảm bảo rằng follower mới có một bản sao chính xác dữ liệu của leader?

Việc chỉ đơn giản là sao chép các tệp dữ liệu từ node này sang node khác thường là không đủ. Các client liên tục ghi vào cơ sở dữ liệu, và dữ liệu luôn trong trạng thái biến động, vì vậy một thao tác sao chép tệp tiêu chuẩn sẽ thấy các phần khác nhau của cơ sở dữ liệu tại các thời điểm khác nhau. Kết quả có thể sẽ không hợp lý.

Bạn có thể làm cho các tệp trên đĩa trở nên nhất quán bằng cách khóa cơ sở dữ liệu (khiến nó không khả dụng cho các lệnh write), nhưng điều đó sẽ đi ngược lại mục tiêu về tính availability cao của chúng ta. May mắn thay, việc thiết lập một follower thường có thể được thực hiện mà không cần downtime. Về mặt khái niệm, quy trình trông như thế này:

1. Hãy thực hiện một snapshot nhất quán của cơ sở dữ liệu leader tại một thời điểm nhất định—nếu có thể, hãy thực hiện mà không cần khóa toàn bộ cơ sở dữ liệu. Hầu hết các cơ sở dữ liệu đều có tính năng này, vì nó cũng được yêu cầu cho việc sao lưu. Trong một số trường hợp, cần đến các công cụ của bên thứ ba, chẳng hạn như Percona XtraBackup cho MySQL.
2. Sao chép snapshot đó sang node follower mới.
3. Follower kết nối với leader và yêu cầu tất cả các thay đổi dữ liệu đã xảy ra kể từ khi snapshot được thực hiện. Điều này đòi hỏi snapshot phải được liên kết với một vị trí chính xác trong replication log của leader. Vị trí đó có nhiều tên gọi khác nhau—ví dụ, PostgreSQL gọi nó là *log sequence number*; MySQL có hai cơ chế là *binlog coordinates* và *global transaction identifiers* (GTIDs).
4. Khi follower đã xử lý xong lượng thay đổi dữ liệu tồn đọng kể từ khi snapshot, chúng ta nói rằng nó đã *caught up* (bắt kịp). Giờ đây, nó có thể tiếp tục xử lý các thay đổi dữ liệu từ leader ngay khi chúng xảy ra.

Các bước thực hành để thiết lập một follower khác biệt đáng kể tùy theo từng cơ sở dữ liệu. Trong một số hệ thống, quy trình này được tự động hóa hoàn toàn, trong khi ở những hệ thống khác, nó có thể là một workflow gồm nhiều bước khá phức tạp mà quản trị viên cần phải thực hiện thủ công.

Bạn cũng có thể lưu trữ replication log vào một object store cùng với các snapshot định kỳ của toàn bộ cơ sở dữ liệu. Đây là một cách tốt để triển khai việc sao lưu cơ sở dữ liệu và phục hồi sau thảm họa, và bạn có thể thực hiện các bước 1 và 2 của việc thiết lập một follower mới bằng cách tải các tệp đó từ object store. Ví dụ, WAL-G thực hiện việc này cho PostgreSQL, MySQL và SQL Server, còn Litestream thực hiện điều tương tự cho SQLite.

Cơ sở dữ liệu được hỗ trợ bởi Object Storage

Object storage có thể được sử dụng cho nhiều mục đích hơn là chỉ lưu trữ dữ liệu. Nhiều cơ sở dữ liệu đang bắt đầu sử dụng các object store như Amazon S3, Google Cloud Storage và Azure Blob Storage để phục vụ dữ liệu cho các truy vấn trực tiếp (live queries). Việc lưu trữ dữ liệu cơ sở dữ liệu trong object storage mang lại nhiều lợi ích:

- Object storage có chi phí thấp so với các tùy chọn lưu trữ đám mây khác. Điều này cho phép các cơ sở dữ liệu đám mây lưu trữ dữ liệu ít được truy vấn thường xuyên hơn trên các bộ lưu trữ rẻ hơn, có latency cao hơn, trong khi vẫn phục vụ working set từ bộ nhớ, SSD và NVMe.
- Các object store cung cấp khả năng replication đa zone, dual-region hoặc multi-region với các đảm bảo về độ tin cậy rất cao. Điều này cũng cho phép các cơ sở dữ liệu bỏ qua các chi phí mạng giữa các zone (inter-zone network fees).
- Các cơ sở dữ liệu có thể sử dụng tính năng *conditional write* của một object store—về bản chất là một thao tác *compare-and-set* (CAS)—để triển khai các transaction và bầu chọn leader [10, 11].
- Việc lưu trữ dữ liệu từ nhiều cơ sở dữ liệu trong cùng một object store có thể đơn giản hóa việc tích hợp dữ liệu (xem mục “Cloud Data Warehouses”), đặc biệt khi sử dụng các định dạng mở như Parquet và Iceberg.

Những lợi ích này giúp đơn giản hóa đáng kể kiến trúc cơ sở dữ liệu bằng cách chuyển trách nhiệm của các transaction, bầu chọn leader và replication sang object storage.

Tuy nhiên, các hệ thống áp dụng object storage để replication phải đối mặt với những đánh đổi (trade-off). Đáng chú ý là các object store có latency đọc và ghi cao hơn nhiều so với đĩa cục bộ hoặc các thiết bị block ảo như Amazon EBS. Nhiều nhà cung cấp đám mây cũng tính phí trên mỗi lượt gọi API, điều này buộc các hệ thống phải batch các thao tác đọc và ghi để giảm chi phí. Việc batching như vậy lại làm tăng thêm latency. Các object thường cũng là bất biến (immutable), điều này khiến các thao tác ghi ngẫu nhiên (random writes) trong một object lớn trở thành một thao tác cực kỳ tốn kém tài nguyên. Cuối cùng, nhiều object store không cung cấp các giao diện filesystem tiêu chuẩn, điều này ngăn cản các hệ thống thiếu khả năng tích hợp object storage tận dụng được nó. Các giao diện như *filesystem in userspace* (FUSE) cho phép các quản trị viên mount các bucket của object store thành các filesystem mà ứng dụng có thể sử dụng mà không cần biết dữ liệu của chúng được lưu trữ trên object storage. Tuy nhiên, nhiều giao diện FUSE tới các object store vẫn thiếu

các tính năng POSIX như ghi không tuần tự (nonsequential writes) hoặc symlinks mà các hệ thống có thể phụ thuộc vào.

Các hệ thống khác nhau xử lý những đánh đổi (trade-off) này theo nhiều cách khác nhau. Một số hệ thống giới thiệu kiến trúc *tiered storage* (lưu trữ phân tầng), trong đó đặt dữ liệu ít được truy cập thường xuyên hơn vào object storage, trong khi dữ liệu mới hoặc dữ liệu được truy cập thường xuyên được giữ trên các thiết bị lưu trữ nhanh hơn như SSD hoặc NVMe, hoặc thậm chí là trong bộ nhớ. Các hệ thống khác sử dụng object storage làm tầng lưu trữ chính nhưng lại dùng một hệ thống lưu trữ độ trễ thấp riêng biệt (chẳng hạn như Amazon EBS hoặc Safekeepers của Neon [12]) để lưu trữ WAL của chúng. Gần đây, một số hệ thống thậm chí còn tiến xa hơn bằng cách áp dụng *zero-disk architecture* (ZDA) (kiến trúc không đĩa). Các hệ thống dựa trên ZDA lưu trữ tất cả dữ liệu vào object storage và chỉ sử dụng đĩa cũng như bộ nhớ thuần túy cho việc caching. Điều này cho phép các node không cần có trạng thái persistent, giúp đơn giản hóa đáng kể các hoạt động vận hành. WarpStream, Confluent Freight, Bufstream của Buf, và Redpanda Serverless đều là các hệ thống tương thích với Kafka được xây dựng bằng kiến trúc zero-disk. Hầu hết mọi cloud data warehouse hiện đại cũng áp dụng kiến trúc như vậy, tương tự như Turbopuffer (một vector search engine) và SlateDB (một LSM storage engine cloud native).

Xử lý sự cố Node

Bất kỳ node nào trong hệ thống cũng có thể bị sập, có thể là do một fault bất ngờ, nhưng cũng có thể do bảo trì (maintenance) theo kế hoạch (ví dụ: khởi động lại một máy chủ để cài đặt bản vá bảo mật kernel). Việc có thể khởi động lại các node riêng lẻ mà không gây downtime là một lợi thế lớn cho việc vận hành và bảo trì. Do đó, mục tiêu của chúng ta là giữ cho toàn bộ hệ thống tiếp tục chạy bất chấp các lỗi node riêng lẻ, và giữ cho tác động của một sự cố node ở mức nhỏ nhất có thể.

Làm thế nào để bạn đạt được tính sẵn sàng cao (high availability) với leader-based replication?

Lỗi follower: Phục hồi catch-up

Trên đĩa cục bộ của mình, mỗi follower giữ một log về các thay đổi dữ liệu mà nó đã nhận được từ leader. Nếu một follower bị crash và được khởi động lại, hoặc nếu kết nối mạng giữa leader và follower bị gián đoạn tạm thời, follower đó có thể phục hồi khá dễ dàng: từ log của mình, nó biết được transaction cuối cùng đã được xử lý trước khi fault xảy ra. Do đó, follower có thể kết nối với leader và yêu cầu tất cả các thay đổi dữ liệu đã xảy ra trong thời gian nó bị mất kết nối. Khi đã áp dụng các thay

đổi này, nó đã bắt kịp (catch up) với leader và có thể tiếp tục nhận luồng thay đổi dữ liệu như trước.

Mặc dù việc phục hồi follower về mặt khái niệm là đơn giản, nhưng nó có thể đầy thách thức về mặt hiệu năng. Nếu cơ sở dữ liệu có throughput ghi cao hoặc nếu follower đã ngoại tuyến trong một thời gian dài, có thể có rất nhiều lượt ghi cần phải bắt kịp. Sẽ có tải cao lên cả follower đang phục hồi và leader (nơi cần gửi lượng ghi tồn đọng cho follower) trong khi quá trình catch-up này đang diễn ra.

Leader có thể xóa log ghi của mình sau khi tất cả các follower đã xác nhận rằng chúng đã xử lý xong, nhưng nếu một follower không khả dụng trong một thời gian dài, leader phải đối mặt với một lựa chọn: giữ lại log cho đến khi follower phục hồi và bắt kịp (với rủi ro hết dung lượng đĩa trên leader), hoặc xóa log mà follower không khả dụng đó chưa xác nhận (trong trường hợp này, follower sẽ không thể phục hồi từ log và sẽ phải được khôi phục từ một bản backup khi nó hoạt động trở lại).

Lỗi leader: Failover

Xử lý lỗi của leader thì phức tạp hơn. Một trong các follower cần được thăng cấp để trở thành leader mới, các client cần được cấu hình lại để gửi các lượt ghi của chúng tới leader mới, và các follower khác cần bắt đầu tiêu thụ các thay đổi dữ liệu từ leader mới. Quá trình này được gọi là *failover*.

Failover có thể xảy ra thủ công (một quản trị viên được thông báo rằng leader đã lỗi và thực hiện các bước cần thiết để chỉ định leader mới) hoặc tự động. Một quá trình failover tự động thường bao gồm các bước sau:

1. *Xác định rằng leader đã gặp lỗi.* Nhiều thứ có khả năng xảy ra sai sót: crash, mất điện, sự cố mạng, và nhiều hơn nữa. Không có cách nào đảm bảo tuyệt đối để phát hiện điều gì đã xảy ra, vì vậy hầu hết các hệ thống chỉ đơn giản sử dụng một timeout; các node thường xuyên gửi qua lại các thông điệp với nhau, và nếu một node không phản hồi trong một khoảng thời gian nhất định—ví dụ, 30 giây—nó sẽ được coi là đã chết. (Nếu leader được chủ động hạ xuống để bảo trì theo kế hoạch, điều này không áp dụng vì leader có thể kích hoạt một handoff an toàn trước khi tắt máy.)
2. *Chọn một leader mới.* Việc này có thể được thực hiện thông qua một quy trình bầu cử (nơi leader được chọn bởi đa số các replica còn lại), hoặc một leader mới có thể được chỉ định bởi một *controller node* [13] đã được thiết lập trước đó. Ứng cử viên tốt nhất cho vị trí lãnh đạo thường là replica có các thay đổi dữ liệu mới nhất từ leader cũ (để giảm thiểu bất kỳ sự mất mát dữ liệu nào). Việc khiến tất cả các node đồng ý về một leader mới là một bài toán consensus, được thảo luận chi tiết trong Chương 10.
3. *Cấu hình lại hệ thống để sử dụng leader mới.* Các client giờ đây cần gửi các write request của chúng tới leader mới (chúng ta sẽ thảo luận điều này trong mục

“Request Routing”). Nếu leader cũ quay trở lại, nó có thể vẫn tin rằng mình là leader, mà không nhận ra rằng các replica khác đã buộc nó phải từ chức. Hệ thống cần đảm bảo rằng leader cũ trở thành một follower và công nhận leader mới.

Failover chứa đựng nhiều thứ có thể đi chệch hướng:

- Nếu sử dụng asynchronous replication, leader mới có thể chưa nhận được tất cả các write từ leader cũ trước khi nó gặp lỗi. Nếu leader trước đó gia nhập lại cluster sau khi một leader mới đã được chọn, điều gì sẽ xảy ra với những write đó? Trong thời gian chờ, leader mới có thể đã nhận được các write xung đột. Giải pháp phổ biến nhất là các write chưa được replication của leader cũ sẽ đơn giản bị loại bỏ, điều này có nghĩa là những write mà bạn tin rằng đã được commit rất cuộc lại không có tính bền vững (durability).
- Việc loại bỏ các write đặc biệt nguy hiểm nếu các hệ thống lưu trữ khác bên ngoài cơ sở dữ liệu cần được phối hợp với nội dung của cơ sở dữ liệu. Ví dụ, trong một sự cố tại GitHub [14], một MySQL follower bị lỗi thời đã được thăng cấp lên làm leader. Cơ sở dữ liệu đã sử dụng một bộ đếm tự tăng để gán primary key cho các hàng mới, nhưng vì bộ đếm của leader mới bị chậm hơn so với leader cũ, nó đã tái sử dụng một số primary key vốn đã được gán bởi leader cũ. Những primary key này cũng được sử dụng trong một Redis store, vì vậy việc tái sử dụng primary key đã dẫn đến sự không nhất quán giữa MySQL và Redis, khiến một số dữ liệu riêng tư bị tiết lộ cho sai người dùng.
- Trong một số kịch bản lỗi nhất định (xem Chương 9), cả hai node đều có thể tin rằng chúng là leader. Tình huống này, được gọi là *split brain*, rất nguy hiểm; nếu cả hai leader đều chấp nhận các write, và không có quy trình nào để giải quyết xung đột (xem “Multi-Leader Replication”), dữ liệu có khả năng bị mất hoặc bị hỏng. Như một chốt chặn an toàn, một số hệ thống có cơ chế để tắt một node nếu phát hiện hai leader. Tuy nhiên, nếu cơ chế này không được thiết kế cẩn thận, bạn có thể kết thúc với việc cả hai node đều bị tắt [15]. Hơn nữa, có rủi ro là vào thời điểm split brain được phát hiện và node cũ bị tắt, mọi thứ đã quá muộn và dữ liệu đã bị hỏng.
- Việc quyết định timeout phù hợp trước khi leader được tuyên bố đã chết có thể rất khó khăn. Một timeout dài hơn đồng nghĩa với thời gian phục hồi lâu hơn trong trường hợp leader gặp lỗi. Tuy nhiên, nếu timeout quá ngắn, các failover không cần thiết có thể xảy ra. Ví dụ, một đợt tăng tải (load spike) tạm thời có thể khiến response time của một node tăng lên vượt quá timeout, hoặc một sự cố mạng có thể gây ra các gói tin bị trễ. Nếu hệ thống vốn đã đang gặp khó khăn với tải cao hoặc các vấn đề mạng, một failover không cần thiết có khả năng sẽ làm tình hình tồi tệ hơn chứ không phải tốt hơn.

Việc ngăn chặn split brain bằng cách giới hạn hoặc tắt các leader cũ được gọi là fencing; chúng ta sẽ thảo luận chi tiết hơn về vấn đề này trong mục “Distributed Locks and Leases”. Tuy nhiên, những vấn đề này không có giải pháp dễ dàng. Vì lý do này, một số đội ngũ vận hành ưu tiên thực hiện failover một cách thủ công, ngay cả khi phần mềm có hỗ trợ automatic failover.

Điều quan trọng nhất khi failover là chọn một follower có dữ liệu cập nhật nhất để làm leader mới. Nếu sử dụng synchronous replication hoặc semisynchronous replication, đây sẽ là follower mà leader cũ đã chờ đợi trước khi xác nhận các lệnh ghi. Với asynchronous replication, bạn có thể chọn follower có số thứ tự log (log sequence number) cao nhất. Điều này giúp giảm thiểu lượng dữ liệu bị mất trong quá trình failover; việc mất một phần nhỏ dữ liệu ghi trong vòng chưa đầy một giây có thể chấp nhận được, nhưng việc chọn một follower bị tụt lại phía sau vài ngày có thể dẫn đến thảm họa.

Những vấn đề này—lỗi node, mạng không tin cậy, và các đánh đổi (trade-off) xoay quanh độ tin cậy của replica, durability, availability và latency—thực tế là những vấn đề cơ bản trong các distributed systems. Trong Chương 9 và 10, chúng ta sẽ thảo luận sâu hơn về chúng.

Triển khai Replication Logs

Cơ chế của leader-based replication hoạt động như thế nào bên trong (under the hood)? Một số phương pháp replication được sử dụng trong thực tế. Hãy cùng xem qua từng phương pháp một cách ngắn gọn.

Nhân bản dựa trên tuyên bố

Trong trường hợp đơn giản nhất, leader ghi lại mọi yêu cầu ghi (*statement*) mà nó thực thi và gửi log statement đó tới các follower. Đối với một cơ sở dữ liệu quan hệ, điều này có nghĩa là mọi statement INSERT, UPDATE, hoặc DELETE đều được chuyển tiếp tới các follower, và mỗi follower sẽ phân tích (parse) cũng như thực thi statement SQL đó như thể nó được nhận từ một client.

Mặc dù cách tiếp cận replication này nghe có vẻ hợp lý, nó có thể gặp lỗi theo nhiều cách khác nhau:

- Bất kỳ statement nào gọi một hàm không xác định (nondeterministic function), chẳng hạn như NOW để lấy ngày giờ hiện tại hoặc RAND để lấy một số ngẫu nhiên, đều có khả năng tạo ra một giá trị khác nhau trên mỗi replica.
- Nếu các statement sử dụng một cột tự động tăng (autoincrementing column), hoặc nếu chúng phụ thuộc vào dữ liệu hiện có trong cơ sở dữ liệu (ví dụ: UPDATE ... WHERE <some condition>), chúng phải được thực thi theo đúng thứ tự trên mỗi replica, nếu không chúng có thể gây ra các hiệu ứng khác nhau. Điều này có thể gây hạn chế khi có nhiều transaction đang thực thi đồng thời.

- Các statement có side effects (ví dụ: triggers, stored procedures, user-defined functions) có thể dẫn đến các side effects khác nhau xảy ra trên mỗi replica, trừ khi các side effects đó hoàn toàn mang tính xác định (deterministic).

Có thể khắc phục những vấn đề đó—ví dụ, leader có thể thay thế bất kỳ lời gọi hàm nondeterministic nào bằng một giá trị trả về cố định khi statement được ghi log để tất cả các follower đều nhận được cùng một giá trị. Ý tưởng thực thi các statement mang tính xác định theo một thứ tự cố định tương tự như mô hình event sourcing mà chúng ta đã thảo luận trước đó trong mục “Event Sourcing and CQRS”. Cách tiếp cận này còn được gọi là *state machine replication*, và chúng ta sẽ thảo luận về lý thuyết đằng sau nó trong mục “Using shared logs”.

Statement-based replication đã được sử dụng trong MySQL trước phiên bản 5.1. Nó vẫn đôi khi được sử dụng ngày nay vì khá gọn nhẹ, nhưng theo mặc định, MySQL hiện đã chuyển sang row-based replication (sẽ được thảo luận ngay sau đây) nếu có bất kỳ tính không xác định nào trong một statement. VoltDB sử dụng statement-based replication và làm cho nó an toàn bằng cách yêu cầu các transaction phải mang tính xác định [16]. Tuy nhiên, tính xác định có thể khó đảm bảo trong thực tế, vì vậy nhiều cơ sở dữ liệu ưu tiên các phương pháp replication khác.

Write-ahead log shipping

Trong Chương 4, chúng ta đã thấy rằng cần có một *write-ahead log* để giúp các storage engine B-tree trở nên mạnh mẽ; mọi thay đổi đều được ghi vào WAL trước để cây có thể được khôi phục về trạng thái nhất quán sau khi gặp sự cố crash. Vì WAL chứa tất cả thông tin cần thiết để khôi phục các index và heap về trạng thái nhất quán, chúng ta có thể sử dụng chính log đó để xây dựng một replica trên một node khác; bên cạnh việc ghi log vào đĩa, leader cũng gửi nó qua mạng tới các follower. Khi follower xử lý log này, nó sẽ xây dựng một bản sao của các tệp chính xác giống như những gì tìm thấy trên leader.

Phương pháp replication này được sử dụng trong PostgreSQL và Oracle, cùng với một số hệ thống khác [17, 18]. Nhược điểm chính là log mô tả dữ liệu ở mức rất thấp—một WAL chứa các chi tiết về việc những byte nào đã bị thay đổi trong các disk block nào. Điều này khiến việc replication bị gắn kết chặt chẽ với storage engine. Nếu cơ sở dữ liệu thay đổi định dạng lưu trữ từ phiên bản này sang phiên bản khác, thông thường sẽ không thể chạy các phiên bản phần mềm cơ sở dữ liệu khác nhau trên leader và các follower.

Điều đó có vẻ như là một chi tiết triển khai nhỏ, nhưng nó có thể gây ra tác động vận hành lớn. Nếu giao thức replication cho phép follower sử dụng phiên bản phần mềm mới hơn leader, bạn có thể thực hiện nâng cấp phần mềm cơ sở dữ liệu không gây gián đoạn (zero-downtime upgrade) bằng cách nâng cấp các follower trước, sau đó thực hiện failover để biến một trong các node đã được nâng cấp thành leader mới. Nếu giao thức replication không cho phép sự sai

khác phiên bản này, như thường thấy trong trường hợp WAL shipping, thì các đợt nâng cấp như vậy sẽ yêu cầu downtime.

Replication log dựa trên dòng (logical log replication)

Một giải pháp thay thế là sử dụng các định dạng log khác nhau cho replication và cho storage engine, điều này cho phép replication log được tách rời khỏi các thành phần bên trong của storage engine. Loại replication log này được gọi là *logical log*, để phân biệt nó với biểu diễn dữ liệu (vật lý) của storage engine.

Một logical log cho cơ sở dữ liệu quan hệ thường là một chuỗi các bản ghi mô tả các thao tác ghi vào các bảng cơ sở dữ liệu ở mức độ chi tiết của một dòng:

- Đối với một dòng được chèn (*inserted row*), log chứa các giá trị mới của tất cả các cột.
- Đối với một dòng bị xóa (*deleted row*), log chứa đủ thông tin để xác định duy nhất dòng đã bị xóa. Thông thường đây sẽ là *primary key*, nhưng nếu bảng không có *primary key*, các giá trị cũ của tất cả các cột cần phải được ghi vào log.
- Đối với một dòng được cập nhật (*updated row*), log chứa đủ thông tin để xác định duy nhất dòng đã được cập nhật, và các giá trị mới của tất cả các cột (hoặc ít nhất là tất cả các cột có giá trị đã thay đổi).

Một transaction sửa đổi nhiều dòng sẽ tạo ra nhiều bản ghi log như vậy, theo sau là một bản ghi cho biết transaction đã được commit. Khi được cấu hình để sử dụng *row-based replication*, MySQL duy trì một replication log logic riêng biệt, gọi là *binlog*, bên cạnh WAL. PostgreSQL thực hiện logical replication bằng cách giải mã (decoding) WAL vật lý thành các sự kiện chèn/cập nhật/xóa dòng [19].

Vì một logical log được tách rời khỏi các thành phần bên trong của storage engine, nó có thể dễ dàng duy trì tính tương thích ngược (*backward compatibility*), cho phép leader và follower chạy các phiên bản phần mềm cơ sở dữ liệu khác nhau. Điều này giúp việc nâng cấp lên phiên bản mới với downtime tối thiểu trở nên khả thi [20].

Một định dạng logical log cũng dễ dàng hơn để các ứng dụng bên ngoài phân tích (*parse*). Khía cạnh này hữu ích nếu bạn muốn gửi nội dung của một cơ sở dữ liệu tới một hệ thống bên ngoài, chẳng hạn như một data warehouse để phân tích ngoại tuyến, hoặc một hệ thống chuyên dụng để xây dựng các index và cache tùy chỉnh [21]. Kỹ thuật này được gọi là *change data capture* (CDC), và chúng ta sẽ quay lại nội dung này trong Chương 12.

Các vấn đề với replication lag

Khả năng chịu đựng lỗi node chỉ là một lý do để muốn có replication. Như đã đề cập trong mục “Distributed Versus Single-Node Systems”, các lý do khác bao gồm khả năng mở rộng (*scalability*) (xử lý nhiều yêu cầu hơn mức một máy đơn lẻ có thể xử lý) và *latency* (độ trễ) (đặt các replica ở vị trí gần người dùng về mặt địa lý hơn).

Replication dựa trên leader yêu cầu tất cả các thao tác ghi phải đi qua một node duy nhất, nhưng các truy vấn chỉ đọc có thể gửi tới bất kỳ replica nào. Đối với các khối lượng công việc chủ yếu bao gồm các thao tác đọc với chỉ một tỷ lệ nhỏ các thao tác ghi (điều thường thấy ở các dịch vụ trực tuyến), có một lựa chọn hấp dẫn: tạo ra nhiều follower và phân phối các yêu cầu đọc tới các follower đó. Điều này giúp giảm tải cho leader và cho phép các yêu cầu đọc được phục vụ bởi các replica ở gần.

Trong kiến trúc *mở rộng khả năng đọc (read-scaling)* này, bạn có thể tăng khả năng phục vụ các yêu cầu chỉ đọc một cách đơn giản bằng cách thêm nhiều follower hơn. Tuy nhiên, cách tiếp cận này trên thực tế chỉ hoạt động với asynchronous replication. Nếu bạn cố gắng thực hiện synchronous replication tới tất cả các follower, một lỗi node duy nhất hoặc sự cố mạng sẽ khiến toàn bộ hệ thống không thể thực hiện ghi. Và bạn càng có nhiều node, khả năng một node bị lỗi càng cao, vì vậy một cấu hình hoàn toàn synchronous sẽ rất thiếu độ tin cậy.

Thật không may, một ứng dụng đang đọc từ một follower *asynchronous* có thể thấy thông tin đã cũ nếu follower đó bị tụt lại phía sau. Điều này dẫn đến sự không nhất quán rõ rệt trong cơ sở dữ liệu; nếu bạn chạy cùng một truy vấn trên leader và một follower tại cùng một thời điểm, bạn có thể nhận được các kết quả khác nhau, vì không phải tất cả các thao tác ghi đều đã được phản ánh trên follower. Sự không nhất quán này là một trạng thái tạm thời—nếu bạn ngừng ghi vào cơ sở dữ liệu và đợi một lát, các follower cuối cùng sẽ bắt kịp và trở nên nhất quán với leader. Vì lý do đó, hiệu ứng này được gọi là *eventual consistency* (nhất quán cuối cùng) [22].

LƯU Ý

Thuật ngữ *eventual consistency* (nhất quán cuối cùng) được đặt ra bởi Douglas Terry và các cộng sự [23] và được phổ biến bởi Werner Vogels [24], và nó đã trở thành khẩu hiệu của nhiều dự án NoSQL. Tuy nhiên, không chỉ các cơ sở dữ liệu NoSQL mới có tính eventual consistency; các follower trong một cơ sở dữ liệu quan hệ sử dụng asynchronous replication cũng có các đặc điểm tương tự.

Thuật ngữ “eventually” được cố ý dùng một cách mơ hồ; nhìn chung, không có giới hạn cho việc một replica có thể bị tụt lại phía sau bao xa. Trong hoạt động bình thường, độ trễ giữa một thao tác ghi xảy ra trên leader và việc nó được phản ánh trên một follower—gọi là *replication lag*—có thể chỉ là một phần nhỏ của giây và không đáng kể trong thực tế. Tuy nhiên, nếu hệ thống đang hoạt động gần mức tối đa công

suất hoặc nếu có vấn đề xảy ra trong mạng, độ trễ có thể dễ dàng tăng lên vài giây hoặc thậm chí vài phút.

Khi độ trễ lớn như vậy, sự không nhất quán mà nó gây ra không chỉ là một vấn đề lý thuyết mà là một vấn đề thực tế đối với các ứng dụng. Trong mục này, chúng ta sẽ nêu bật ba ví dụ về các vấn đề có khả năng xảy ra với replication lag. Chúng ta cũng sẽ phác thảo một số cách tiếp cận để giải quyết chúng.

Đọc lại chính các thao tác ghi của bạn

Nhiều ứng dụng cho phép người dùng gửi một số dữ liệu và sau đó xem lại những gì họ đã gửi. Đây có thể là một bản ghi trong cơ sở dữ liệu khách hàng, hoặc một bình luận trong một luồng thảo luận, hoặc thứ gì đó tương tự. Khi dữ liệu mới được gửi, nó phải được gửi tới leader, nhưng khi người dùng xem dữ liệu, nó có thể được đọc từ một follower. Điều này đặc biệt phù hợp nếu dữ liệu được xem thường xuyên nhưng chỉ thỉnh thoảng mới được ghi.

Với asynchronous replication, một vấn đề nảy sinh như được minh họa trong Hình 6-3: nếu người dùng xem dữ liệu ngay sau khi thực hiện một thao tác ghi, dữ liệu mới có thể vẫn chưa đến được replica. Đối với người dùng, điều này trông giống như dữ liệu họ đã gửi đã bị mất, vì vậy họ sẽ cảm thấy không hài lòng là điều dễ hiểu.

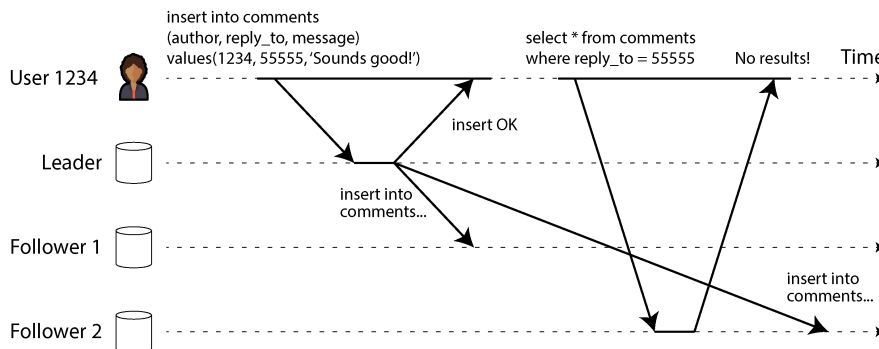


Figure 6-3. Sự không nhất quán có thể phát sinh khi người dùng thực hiện một thao tác ghi, sau đó thực hiện một thao tác đọc từ một replica cũ (stale replica).

Trong tình huống này, chúng ta cần *read-after-write consistency* (tính nhất quán sau khi ghi), hay còn gọi là *read-your-writes consistency* [23]. Đây là một sự đảm bảo rằng nếu người dùng tải lại trang, họ sẽ luôn thấy bất kỳ cập nhật nào mà chính họ đã gửi. Nó không đưa ra lời hứa nào đối với những người dùng khác; các cập nhật của người dùng khác có thể không hiển thị cho đến một thời điểm muộn hơn. Tuy nhiên, nó trấn an người dùng rằng dữ liệu đầu vào của chính họ đã được lưu lại chính xác.

Làm thế nào chúng ta có thể triển khai *read-after-write consistency* trong một hệ thống sử dụng *leader-based replication*? Có nhiều kỹ thuật khả thi khác nhau. Hãy điểm qua một vài kỹ thuật:

- Khi đọc một thứ gì đó mà người dùng có thể đã sửa đổi, hãy đọc nó từ leader hoặc một follower được cập nhật đồng bộ (*synchronously updated*); nếu không, hãy đọc nó từ một follower được cập nhật bất đồng bộ (*asynchronously updated*). Điều này đòi hỏi bạn phải có cách nào đó để biết liệu một thứ gì đó có thể đã bị sửa đổi hay không mà không cần phải truy vấn nó. Ví dụ, thông tin hồ sơ người dùng trên một mạng xã hội thường chỉ có thể được chỉnh sửa bởi chủ sở hữu hồ sơ đó, chứ không phải bởi bất kỳ ai khác. Do đó, một quy tắc đơn giản là: luôn đọc hồ sơ của chính người dùng từ leader, và bất kỳ hồ sơ nào của người dùng khác từ một follower.
- Nếu hầu hết mọi thứ trong ứng dụng đều có khả năng được người dùng sửa đổi, thì cách tiếp cận đó sẽ không hiệu quả, vì hầu hết mọi thứ sẽ phải được đọc từ leader (làm mất đi lợi ích của việc mở rộng khả năng đọc (*read scaling*)). Trong trường hợp đó, các tiêu chí khác có thể được sử dụng để quyết định xem có nên đọc từ leader hay không. Ví dụ, bạn có thể theo dõi thời gian của lần cập nhật cuối cùng và, trong vòng một phút sau lần cập nhật cuối cùng đó, hãy thực hiện tất cả các lượt đọc từ leader [25]. Bạn cũng có thể giám sát replication lag trên các follower và ngăn chặn các truy vấn trên bất kỳ follower nào đang bị chậm hơn leader quá một phút.
- Client có thể ghi nhớ timestamp của lần ghi gần nhất, và hệ thống có thể đảm bảo rằng replica phục vụ bất kỳ lượt đọc nào cho người dùng đó phản ánh các cập nhật ít nhất là cho đến timestamp đó. Nếu một replica không đủ cập nhật, lượt đọc có thể được xử lý bởi một replica khác hoặc truy vấn có thể chờ cho đến khi replica đó bắt kịp [26]. timestamp có thể là một *logical timestamp* (thứ chỉ ra thứ tự của các lần ghi, chẳng hạn như số thứ tự log — *log sequence number*) hoặc đồng hồ hệ thống thực tế (trong trường hợp này, việc đồng bộ hóa đồng hồ trở nên cực kỳ quan trọng; xem mục “Unreliable Clocks”).
- Nếu các replica của bạn được phân bố qua các region (để đảm bảo khoảng cách địa lý gần với người dùng, vì tính sẵn sàng, hoặc vì độ bền vững), sẽ có thêm sự phức tạp. Bất kỳ yêu cầu nào cần được phục vụ bởi leader đều phải được định tuyến đến region chứa leader đó.

Một sự phức tạp khác phát sinh khi cùng một người dùng truy cập dịch vụ của bạn từ nhiều thiết bị, chẳng hạn như trình duyệt web trên máy tính để bàn và một ứng dụng di động. Trong trường hợp này, bạn có thể muốn cung cấp *cross-device read-after-write consistency*: nếu người dùng nhập một số thông tin trên một thiết bị và sau đó xem nó trên một thiết bị khác, họ phải thấy được thông tin mà họ vừa nhập.

Có một số vấn đề bổ sung cần xem xét ở đây:

- Các cách tiếp cận yêu cầu ghi nhớ *timestamp* của lần cập nhật cuối cùng của người dùng sẽ trở nên khó khăn hơn, bởi vì mã chạy trên một thiết bị không biết những cập nhật nào đã xảy ra trên thiết bị kia. *metadata* này sẽ cần được tập trung hóa.
- Nếu các *replica* của bạn được phân bố qua nhiều *region*, không có gì đảm bảo rằng các kết nối từ các thiết bị khác nhau sẽ được định tuyến đến cùng một *region*. (Ví dụ, nếu máy tính để bàn của người dùng sử dụng kết nối băng thông rộng tại nhà và thiết bị di động của họ sử dụng mạng dữ liệu di động, các tuyến mạng của các thiết bị có thể hoàn toàn khác nhau.) Nếu cách tiếp cận của bạn yêu cầu đọc từ *leader*, trước tiên bạn có thể cần định tuyến các yêu cầu từ tất cả các thiết bị của một người dùng đến cùng một *region*.

Regions và Availability Zones

Chúng ta sử dụng thuật ngữ *region* (vùng) để chỉ một hoặc nhiều datacenter tại một vị trí địa lý duy nhất. Các nhà cung cấp cloud đặt nhiều datacenter trong cùng một *region* địa lý. Mỗi datacenter được gọi là một *availability zone* hoặc gọi đơn giản là *zone*. Do đó, một cloud region duy nhất được cấu thành từ nhiều *zone*. Mỗi *zone* là một datacenter riêng biệt nằm trong các cơ sở vật lý khác nhau với hệ thống điện, làm mát và các thành phần khác riêng biệt.

Các *zone* trong cùng một *region* được kết nối với nhau bằng các kết nối mạng tốc độ rất cao. *Latency* (độ trễ) đủ thấp để hầu hết các *distributed systems* có thể chạy với các *node* trải rộng trên nhiều *zone* trong cùng một *region* như thể chúng nằm trong cùng một *zone*. Các cấu hình multi-zone cho phép *distributed systems* sống sót qua các sự cố lỗi *zone* (*zonal outages*) khi một *zone* bị ngoại tuyến, nhưng chúng không bảo vệ được trước các sự cố lỗi vùng (*regional outages*) khi tất cả các *zone* trong một *region* đều không khả dụng. Để sống sót qua một *regional outage*, một *distributed system* phải được *deployment* trên nhiều *region*, điều này có thể dẫn đến *latency* cao hơn, *throughput* (throughput) thấp hơn và tăng chi phí mạng cloud. Chúng ta sẽ thảo luận kỹ hơn về những *trade-off* (đánh đổi) này trong mục “Multi-leader replication topologies”. Hiện tại, bạn chỉ cần biết rằng khi chúng ta nói đến *region*, ý chúng ta là một tập hợp các *zone*/datacenter tại một vị trí địa lý duy nhất.

Monotonic reads

Ví dụ thứ hai của chúng ta về một *anomaly* có thể xảy ra khi đọc từ các *asynchronous followers* là việc một người dùng có thể thấy mọi thứ *di chuyển ngược về quá khứ*.

Điều này có thể xảy ra nếu một người dùng thực hiện nhiều lần đọc từ các *replica* khác nhau. Ví dụ, Hình 6-4 cho thấy người dùng 2345 thực hiện cùng một truy vấn hai lần, lần đầu tới một *follower* có độ trễ thấp, sau đó tới một *follower* có độ trễ lớn hơn. (Kịch bản này rất dễ xảy ra nếu người dùng làm mới một trang web và mỗi *request* được định tuyến tới một *server* ngẫu nhiên.) Truy vấn đầu tiên trả về một bình luận vừa được người dùng 1234 thêm vào, nhưng truy vấn thứ hai không trả về gì cả vì *follower* đang bị trễ vẫn chưa nhận được thao tác *write* đó. Về cơ bản, truy vấn thứ hai quan sát trạng thái hệ thống tại một thời điểm sớm hơn so với truy vấn đầu tiên. Điều này sẽ không quá tệ nếu truy vấn đầu tiên không trả về gì, vì người dùng 2345 có lẽ sẽ không biết rằng người dùng 1234 vừa mới thêm một bình luận. Tuy nhiên, sẽ rất khó hiểu cho người dùng 2345 nếu họ đầu tiên thấy bình luận của người dùng 1234 xuất hiện, rồi sau đó lại thấy nó biến mất.

Monotonic reads [22] cung cấp một sự đảm bảo rằng loại *anomaly* này sẽ không xảy ra. Đây là một sự đảm bảo thấp hơn so với *strong consistency*, nhưng mạnh hơn so với *eventual consistency*. Khi bạn đọc dữ liệu, bạn có thể thấy một giá trị cũ; *monotonic reads* chỉ có nghĩa là nếu một người dùng thực hiện nhiều lần đọc liên tiếp, chúng sẽ không thấy thời gian trôi ngược lại (tức là, họ sẽ không đọc dữ liệu cũ hơn sau khi đã đọc dữ liệu mới hơn trước đó).

Một cách để đạt được *monotonic reads* là đảm bảo rằng mỗi người dùng luôn thực hiện các lần đọc của họ từ cùng một *replica* (các người dùng khác nhau có thể đọc từ các *replica* khác nhau). Ví dụ, *replica* có thể được chọn dựa trên một *hash* của user ID thay vì chọn ngẫu nhiên. Tuy nhiên, nếu *replica* đó gặp lỗi, các truy vấn của người dùng sẽ cần được định tuyến lại tới một *replica* khác.

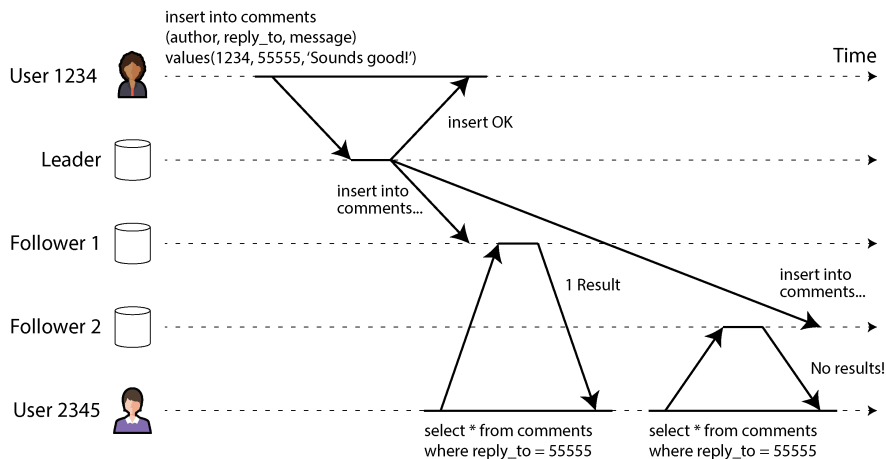


Figure 6-4. *Khi một người dùng lần đầu đọc từ một replica mới, sau đó đọc từ một replica cũ (stale replica), thời gian dường như trôi ngược lại.*

Các lượt đọc prefix nhất quán

Ví dụ thứ ba của chúng ta về các bất thường do replication lag liên quan đến việc vi phạm causality (quan hệ nhân quả). Hãy tưởng tượng đoạn hội thoại ngắn sau đây giữa ông Poons và bà Cake:

Ông Poons: Bà Cake ơi, bà có thể nhìn thấy tương lai xa đến mức nào?

Bà Cake: Thường là khoảng 10 giây, thưa ông Poons.

Có một sự phụ thuộc nhân quả giữa hai câu nói này: bà Cake đã nghe câu hỏi của ông Poons và trả lời nó.

Bây giờ, hãy tưởng tượng một người thứ ba đang lắng nghe cuộc trò chuyện này thông qua các follower. Những gì bà Cake nói đi qua một follower với replication lag rất nhỏ, nhưng những gì ông Poons nói lại có replication lag dài hơn (xem Hình 6-5). Người quan sát này sẽ nghe thấy như sau:

Bà Cake: Thường là khoảng 10 giây, thưa ông Poons.

Ông Poons: Bà Cake ơi, bà có thể nhìn thấy tương lai xa đến mức nào?

Đối với người quan sát, điều này nghe như thể bà Cake đang trả lời câu hỏi ngay cả trước khi ông Poons kịp hỏi. Những năng lực ngoại cảm như vậy thật ấn tượng nhưng cũng rất khó hiểu [27].

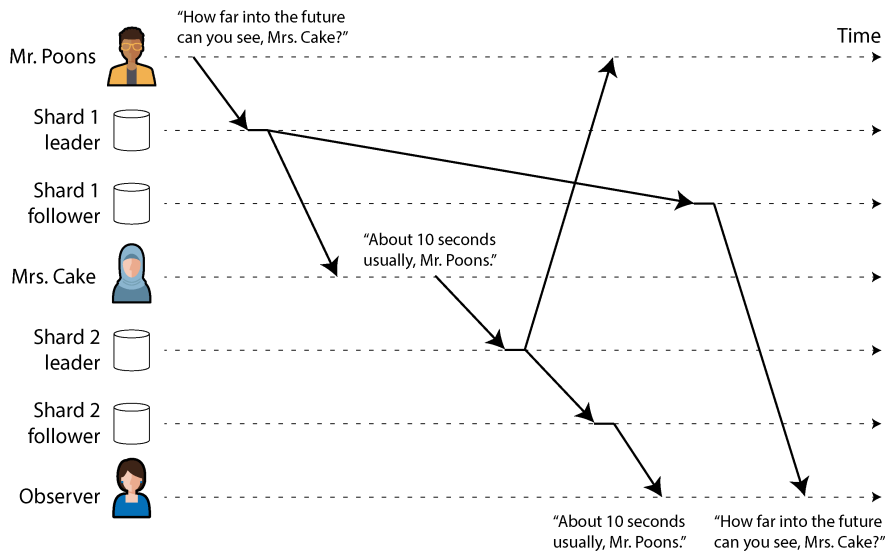


Figure 6-5. Nếu một số shard được replication chậm hơn những shard khác, một người quan sát có thể thấy câu trả lời trước khi thấy câu hỏi.

Để ngăn chặn loại bất thường này, ta cần một loại đảm bảo khác: *consistent prefix reads* (đọc tiền tố nhất quán) [22]. Đảm bảo này nói rằng nếu một chuỗi các thao tác ghi diễn ra theo một thứ tự nhất định, thì bất kỳ ai đọc các thao tác ghi đó cũng sẽ thấy chúng xuất hiện theo cùng thứ tự đó.

Đây là một vấn đề đặc thù trong các cơ sở dữ liệu được shard ینگ (phân mảnh dữ liệu), điều mà chúng ta sẽ thảo luận trong Chương 7. Nếu cơ sở dữ liệu luôn áp dụng các thao tác ghi theo cùng một thứ tự, các thao tác đọc sẽ luôn thấy một tiền tố nhất quán, do đó sự bất thường này không thể xảy ra. Tuy nhiên, trong nhiều cơ sở dữ liệu phân tán, các shard khác nhau hoạt động độc lập, vì vậy không có thứ tự ghi toàn cục. Khi một người dùng đọc từ cơ sở dữ liệu, họ có thể thấy một số phần của cơ sở dữ liệu ở trạng thái cũ và một số phần khác ở trạng thái mới hơn.

Một giải pháp là đảm bảo rằng bất kỳ thao tác ghi nào có quan hệ nhân quả với nhau đều được ghi vào cùng một shard—nhưng trong một số ứng dụng, điều này không thể thực hiện một cách hiệu quả. Một số thuật toán theo dõi rõ ràng các phụ thuộc nhân quả, một chủ đề mà chúng ta sẽ quay lại trong mục “Mối quan hệ happens-before và concurrency (tính đồng thời)”.

Giải pháp cho Replication Lag

Khi làm việc với một hệ thống *eventual consistency*, việc suy nghĩ về cách ứng dụng hoạt động nếu replication lag tăng lên đến vài phút hoặc thậm chí vài giờ là rất đáng giá. Nếu câu trả lời là “không vấn đề gì”, thì thật tuyệt vời. Tuy nhiên, nếu kết quả dẫn đến trải nghiệm tồi tệ cho người dùng, việc thiết kế hệ thống

để cung cấp một đảm bảo mạnh hơn, chẳng hạn như `read-after-write`, là rất quan trọng. Việc giả vờ rằng `replication` là `synchronous` trong khi thực tế nó là `asynchronous` là một công thức dẫn đến các vấn đề về sau.

Như đã thảo luận trước đó, có những cách để một ứng dụng cung cấp một đảm bảo mạnh hơn so với cơ sở dữ liệu nền tảng—ví dụ, bằng cách thực hiện một số loại đọc nhất định trên `leader` hoặc một `follower` được cập nhật đồng bộ (`synchronous`). Tuy nhiên, việc xử lý các vấn đề này trong mã nguồn ứng dụng rất phức tạp và dễ dẫn đến sai sót.

Mô hình lập trình đơn giản nhất cho các lập trình viên ứng dụng là chọn một cơ sở dữ liệu cung cấp đảm bảo `strong consistency` cho các `replica`, chẳng hạn như `linearizability` (xem Chương 10), và hỗ trợ các `transaction ACID` (xem Chương 8). Điều này cho phép bạn hầu như bỏ qua các thách thức phát sinh từ `replication` và đối xử với cơ sở dữ liệu như thể nó chỉ có một `node` duy nhất. Vào đầu những năm 2010, phong trào `NoSQL` đã thúc đẩy quan điểm rằng các tính năng này làm hạn chế khả năng mở rộng và các hệ thống quy mô lớn sẽ phải chấp nhận `eventual consistency`.

Tuy nhiên, kể từ đó, một số cơ sở dữ liệu đã bắt đầu cung cấp `strong consistency` và hỗ trợ `transaction` trong khi vẫn mang lại các lợi thế về `fault tolerance`, tính sẵn sàng cao và khả năng mở rộng của một cơ sở dữ liệu phân tán. Như đã đề cập trong mục “`Relational Versus Document Models`”, xu hướng này được gọi là *NewSQL* để đối lập với `NoSQL` (mặc dù nó ít nói về `SQL` cụ thể mà thiên về các cách tiếp cận mới đối với việc quản lý `transaction` có khả năng mở rộng).

Mặc dù các cơ sở dữ liệu phân tán có `strong consistency` và khả năng mở rộng hiện đã có sẵn, vẫn có những lý do chính đáng khiến một số ứng dụng chọn sử dụng các dạng `replication` khác nhau nhằm cung cấp các đảm bảo `consistency` yếu hơn. Đáng chú ý là chúng có thể cung cấp khả năng phục hồi mạnh mẽ hơn trước các gián đoạn mạng và có chi phí `overhead` thấp hơn so với các hệ thống `transactional`. Chúng ta sẽ khám phá các cách tiếp cận như vậy trong phần còn lại của chương này.

Multi-Leader Replication

Cho đến nay trong chương này, chúng ta mới chỉ xem xét các kiến trúc `replication` sử dụng một `single leader`. Mặc dù đó là một cách tiếp cận phổ biến, nhưng vẫn có những lựa chọn thay thế thú vị khác.

`Single-leader replication` có một nhược điểm lớn: tất cả các thao tác ghi phải đi qua một `leader` duy nhất. Nếu bạn không thể kết nối với `leader` vì bất kỳ lý do gì—ví dụ, do một sự gián đoạn mạng giữa bạn và `leader`—bạn không thể ghi vào cơ sở dữ liệu.

Một phần mở rộng tự nhiên của mô hình replication single-leader là cho phép nhiều hơn một node chấp nhận các lệnh ghi. Replication vẫn diễn ra theo cùng một cách: mỗi node xử lý một lệnh ghi phải chuyển tiếp thay đổi dữ liệu đó tới tất cả các node khác. Chúng ta gọi đây là cấu hình *multi-leader* (còn được gọi là replication *active/active* hoặc *bidirectional*). Trong thiết lập này, mỗi leader đồng thời đóng vai trò là một follower đối với các leader khác.

Tương tự như replication single-leader, bạn có sự lựa chọn giữa việc thực hiện nó một cách synchronous hoặc asynchronous. Hãy giả sử bạn có hai leader, A và B, và bạn đang cố gắng ghi vào A. Nếu các lệnh ghi được replication một cách synchronous từ A tới B, và kết nối mạng giữa hai node bị gián đoạn, bạn không thể ghi vào A cho đến khi kết nối được khôi phục. Do đó, replication multi-leader synchronous mang lại cho bạn một mô hình rất giống với replication single-leader, ví dụ như khi bạn chọn B làm leader và A chỉ đơn giản chuyển tiếp bất kỳ yêu cầu ghi nào tới B để thực thi.

Vì lý do đó, chúng ta sẽ không đi sâu hơn vào replication multi-leader synchronous và sẽ chỉ coi nó tương đương với replication single-leader. Phần còn lại của mục này tập trung vào replication multi-leader asynchronous, trong đó bất kỳ leader nào cũng có thể xử lý các lệnh ghi, ngay cả khi kết nối của nó với các leader khác bị gián đoạn.

Vận hành phân tán về mặt địa lý

Việc sử dụng thiết lập multi-leader trong cùng một region hiếm khi hợp lý, bởi vì lợi ích mang lại hiếm khi vượt quá sự phức tạp tăng thêm. Tuy nhiên, trong một số tình huống, cấu hình này là hợp lý.

Hãy tưởng tượng bạn có một cơ sở dữ liệu với các replica nằm ở nhiều region khác nhau (có lẽ để bạn có thể chịu lỗi khi toàn bộ một region gặp sự cố, hoặc có lẽ để gần gũi hơn với người dùng của mình). Thiết lập này được gọi là thiết lập *geographically distributed*, *geo-distributed*, hoặc *geo-replicated*. Với replication single-leader, leader phải nằm ở *một* trong các region, và tất cả các lệnh ghi phải đi qua region đó.

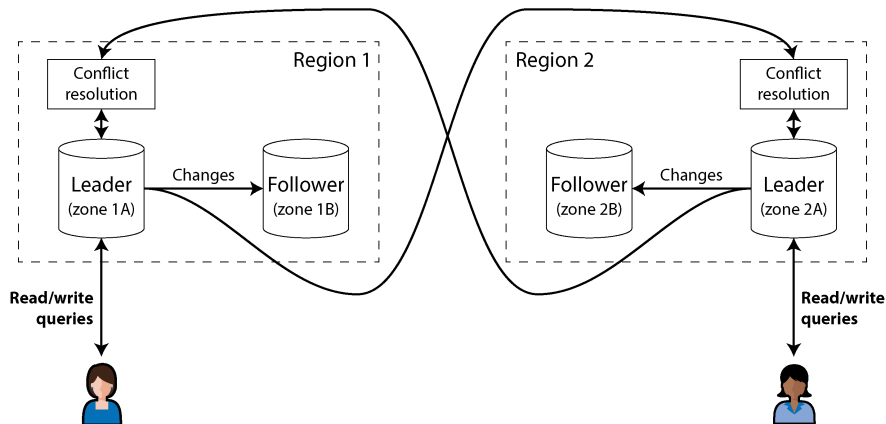


Figure 6-6. *Multi-leader replication trên nhiều region*

Trong cấu hình multi-leader, bạn có thể có một leader ở *mỗi* region. Hình 6-6 cho thấy kiến trúc này có thể trông như thế nào. Bên trong mỗi region, cơ chế replication leader–follower thông thường được sử dụng (với các follower có thể nằm ở một availability zone khác với leader); giữa các region, leader của mỗi region sẽ replication các thay đổi của nó tới các leader ở các region khác. Hãy cùng so sánh cách các cấu hình single-leader và multi-leader hoạt động trong một deployment đa region:

Hiệu năng

Trong cấu hình single-leader, mọi thao tác write đều phải đi qua internet tới region có leader. Điều này có thể làm tăng latency đáng kể cho các thao tác write và có thể làm mất đi mục đích ban đầu của việc có nhiều region. Trong cấu hình multi-leader, mọi thao tác write đều có thể được xử lý tại region cục bộ và sau đó được replication bất đồng bộ tới các region khác. Do đó, độ trễ mạng giữa các region được che giấu khỏi người dùng, nghĩa là hiệu năng cảm nhận được có thể tốt hơn.

Khả năng chịu lỗi trước các sự cố outage của region

Trong cấu hình single-leader, nếu region có leader trở nên không khả dụng, failover có thể thăng cấp một follower ở region khác lên làm leader. Trong cấu hình multi-leader, mỗi region có thể tiếp tục hoạt động độc lập với các region khác, và replication sẽ bắt kịp khi region đang offline quay trở lại trực tuyến.

Khả năng chịu lỗi trước các vấn đề mạng

Ngay cả với các kết nối chuyên dụng, lưu lượng giữa các region có thể kém tin cậy hơn so với lưu lượng giữa các zone trong cùng một region hoặc trong cùng một zone. Cấu hình single-leader rất nhạy cảm với các vấn đề ở liên kết liên region này, bởi vì khi một client ở một region muốn thực hiện write tới một leader ở region khác, nó phải gửi yêu cầu của mình qua liên kết đó và chờ phản hồi trước khi có thể hoàn tất.

Một cấu hình multi-leader với replication bất đồng bộ có thể chịu đựng các vấn đề mạng tốt hơn; trong quá trình gián đoạn mạng tạm thời, leader của mỗi region có thể tiếp tục xử lý các thao tác write một cách độc lập.

Tính nhất quán

Một hệ thống single-leader có thể cung cấp các đảm bảo về strong consistency, chẳng hạn như các transaction có tính serializable, điều mà chúng ta sẽ thảo luận trong Chương 8. Nhược điểm lớn nhất của các hệ thống multi-leader là tính nhất quán mà chúng có thể đạt được yếu hơn nhiều. Ví dụ, bạn không thể đảm bảo rằng một tài khoản ngân hàng sẽ không bị âm hoặc một username là duy nhất; luôn có khả năng các leader khác nhau xử lý các thao tác write mà xét riêng lẻ thì đều ổn (như chi một phần tiền trong tài khoản, đăng ký một username cụ thể) nhưng lại vi phạm ràng buộc khi xét cùng với một thao tác write khác trên một leader khác.

Đây đơn giản là một giới hạn cơ bản của các distributed systems [28]. Nếu bạn cần thực thi các ràng buộc như vậy, bạn nên sử dụng hệ thống single-leader. Tuy nhiên, như chúng ta sẽ thấy trong mục “Dealing with Conflicting Writes”, các hệ thống multi-leader vẫn có thể đạt được các tính chất nhất quán hữu ích trong nhiều ứng dụng không cần đến các ràng buộc như vậy.

Multi-leader replication ít phổ biến hơn single-leader replication, nhưng nó vẫn được hỗ trợ bởi nhiều cơ sở dữ liệu, bao gồm MySQL, Oracle, SQL Server và YugabyteDB. Trong một số trường hợp, nó là một tính năng bổ sung bên ngoài—ví dụ, trong Redis Enterprise, EDB Postgres Distributed và pglogical [29].

Vì multi-leader replication là một tính năng được bổ sung sau (retrofitted) trong nhiều cơ sở dữ liệu, nên thường có những cạm bẫy cấu hình tinh vi và các tương tác gây ngạc nhiên với các tính năng khác của cơ sở dữ liệu. Ví dụ, các key tự tăng (autoincrementing keys), triggers và các ràng buộc toàn vẹn có thể gây ra vấn đề. Vì lý do này, multi-leader replication thường được coi là vùng nguy hiểm cần tránh nếu có thể [30].

Các topology của multi-leader replication

Một *replication topology* (cấu trúc nhân bản) mô tả các đường truyền thông mà qua đó các lệnh ghi được lan truyền từ node này sang node khác. Nếu bạn có hai leader, như trong Hình 6-6, chỉ có một topology khả thi: leader 1 phải gửi tất cả các lệnh ghi của nó tới leader 2, và ngược lại. Với nhiều hơn hai leader, nhiều topology khác nhau có thể xảy ra. Một số ví dụ được minh họa trong Hình 6-7.

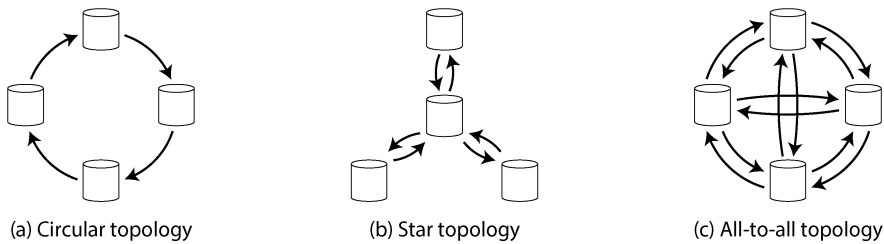


Figure 6-7. **Hình 6.1: Ba ví dụ về topology cho multi-leader replication**

Cấu trúc topology tổng quát nhất là *all-to-all*, được hiển thị trong Hình 6-7(c), trong đó mọi leader đều gửi các lệnh write của mình tới tất cả các leader khác. Tuy nhiên, các topology hạn chế hơn cũng được sử dụng. Ví dụ, trong *circular topology*, được hiển thị trong Hình 6-7(a), mỗi node nhận các lệnh write từ một node khác và chuyển tiếp các lệnh write đó (cùng với bất kỳ lệnh write nào của chính nó) tới một node khác nữa. Topology dạng *star*, được hiển thị trong Hình 6-7(b), cũng rất phổ biến; ở đây, một root node được chỉ định sẽ chuyển tiếp các lệnh write tới tất cả các node còn lại. Topology dạng star có thể được tổng quát hóa thành một cấu trúc tree.

LƯU Ý

Một topology mạng dạng hình sao (star-shaped) không liên quan gì đến *star schema* (xem mục “Stars and Snowflakes: Schemas for Analytics”), vốn mô tả cấu trúc của một mô hình dữ liệu.

Trong các topology dạng circular và star, một lệnh write có thể cần phải đi qua nhiều node trước khi nó tiếp cận được tất cả các replica. Do đó, các node cần phải chuyển tiếp các thay đổi dữ liệu mà chúng nhận được từ các node khác. Để ngăn chặn các vòng lặp replication vô hạn, mỗi node được cấp một định danh duy nhất, và trong replication log, mỗi lệnh write được gắn thẻ với định danh của tất cả các node mà nó đã đi qua [31]. Khi một node nhận được một thay đổi dữ liệu đã được gắn thẻ bằng chính định danh của nó, thay đổi dữ liệu đó sẽ bị bỏ qua, vì node đó biết rằng nó đã được xử lý rồi.

Các vấn đề với các topology khác nhau

Một vấn đề với các topology dạng circular và star là nếu chỉ một node gặp lỗi, nó có thể làm gián đoạn luồng các thông điệp replication giữa các node khác, khiến chúng không thể giao tiếp cho đến khi node đó được sửa lỗi. Topology có thể được cấu hình lại để hoạt động xung quanh node bị lỗi, nhưng trong hầu hết các deployment, việc cấu hình lại như vậy sẽ phải thực hiện thủ công. Khả năng chịu lỗi (fault tolerance) của một topology có kết nối dày đặc hơn (chẳng hạn như all-to-all) sẽ tốt hơn vì nó cho phép các thông điệp di chuyển theo các đường dẫn khác nhau, tránh được một điểm lỗi duy nhất (single point of failure).

Tuy nhiên, các topology all-to-all cũng có thể gặp vấn đề. Cụ thể, một số liên kết mạng có thể nhanh hơn những liên kết khác (ví dụ: do tắc nghẽn mạng), dẫn đến kết quả là một số thông điệp replication có thể "vượt mặt" các thông điệp khác, như được minh họa trong Hình 6-8.

Trong Hình 6-8, client A chèn một dòng vào một bảng trên leader 1, và client B cập nhật dòng đó trên leader 3. Tuy nhiên, leader 2 có thể nhận các lệnh write theo một thứ tự khác. Nó có thể nhận lệnh update trước (mà từ góc nhìn của nó, đó là một lệnh update cho một dòng không tồn tại trong cơ sở dữ liệu) và chỉ nhận được lệnh insert tương ứng sau đó (lệnh đáng lẽ phải diễn ra trước lệnh update).

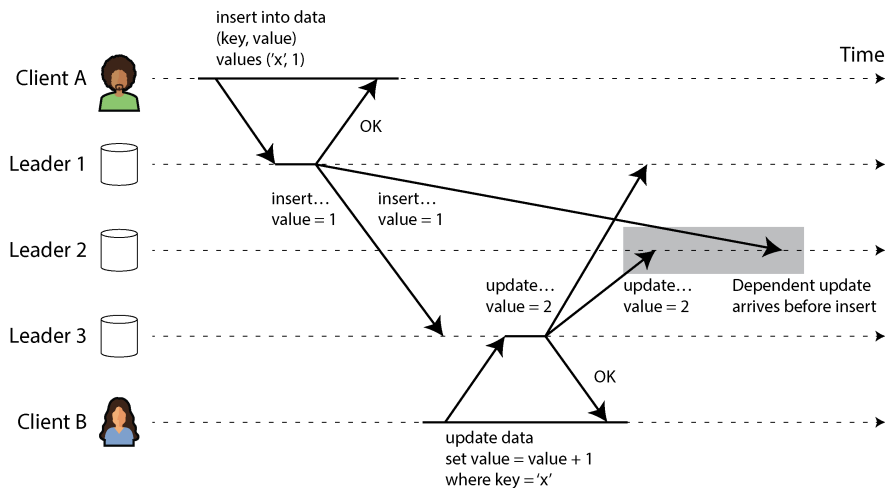


Figure 6-8. Với multi-leader replication, các lệnh ghi có thể đến sai thứ tự tại một số replica.

Đây là một vấn đề về causality (quan hệ nhân quả), tương tự như vấn đề mà chúng ta đã thấy trong mục “Consistent prefix reads”. Việc cập nhật phụ thuộc vào lệnh chèn trước đó, vì vậy chúng ta cần đảm bảo rằng tất cả các node đều xử lý lệnh chèn trước, sau đó mới đến lệnh cập nhật. Chỉ đơn thuần đính kèm một timestamp vào mỗi lần ghi là không đủ, bởi vì không thể tin cậy rằng các đồng hồ có đủ đồng bộ để sắp xếp chính xác các sự kiện này tại leader 2 (xem Chương 9).

Để sắp xếp các sự kiện này một cách chính xác, ta có thể sử dụng một kỹ thuật gọi là *version vectors*, điều mà chúng ta sẽ thảo luận trong mục “Detecting Concurrent Writes”. Tuy nhiên, nhiều hệ thống multi-leader replication không sử dụng các kỹ thuật tốt để sắp xếp các bản cập nhật, khiến chúng dễ bị tổn thương trước các vấn đề như trong Hình 6-8. Nếu bạn đang sử dụng multi-leader replication, bạn nên lưu ý về các vấn đề này, đọc kỹ tài liệu hướng dẫn và kiểm thử kỹ lưỡng cơ sở dữ liệu của mình để đảm bảo rằng nó thực sự cung cấp các đảm bảo mà bạn tin tưởng.

Sync Engines và Local-First Software

Multi-leader replication cũng phù hợp nếu bạn có một ứng dụng cần tiếp tục hoạt động khi nó bị ngắt kết nối internet. Ví dụ, hãy xét các ứng dụng lịch trên điện thoại di động, máy tính xách tay và các thiết bị khác của bạn. Bạn cần có khả năng xem các cuộc họp (thực hiện các read request) và nhập các cuộc họp mới (thực hiện các write request) vào bất kỳ lúc nào, bất kể thiết bị của bạn hiện có kết nối internet hay không. Nếu bạn thực hiện bất kỳ thay đổi nào khi đang ngoại tuyến, chúng cần được đồng bộ với một máy chủ và các thiết bị khác của bạn khi thiết bị đó kết nối internet trở lại.

Trong trường hợp này, mỗi thiết bị có một local database replica đóng vai trò là một leader (nó chấp nhận các write request), và có một quá trình multi-leader replication bất đồng bộ (sync) giữa các replica của lịch trên tất cả các thiết bị của bạn. Replication lag có thể kéo dài hàng giờ hoặc thậm chí hàng ngày, tùy thuộc vào thời điểm bạn có thể truy cập internet.

Từ góc độ kiến trúc, thiết lập này rất giống với multi-leader replication giữa các vùng (regions), nhưng được đẩy đến mức cực đoan. Mỗi thiết bị là một “vùng”, và kết nối mạng giữa chúng cực kỳ không tin cậy.

Real-time collaboration, offline-first, và các ứng dụng local-first

Nhiều ứng dụng web hiện đại cung cấp các tính năng *real-time collaboration* (cộng tác thời gian thực), chẳng hạn như Google Docs và Sheets cho các tài liệu văn bản và bảng tính, Figma cho đồ họa, và Linear cho quản lý dự án. Điều làm cho các ứng dụng này phản hồi nhanh là đầu vào của người dùng được phản ánh ngay lập tức trên giao diện người dùng, mà không cần chờ đợi một vòng truyền tải mạng (network round-trip) tới máy chủ, và các chỉnh sửa của một người dùng được hiển thị cho những người cộng tác của họ với latency (độ trễ) thấp [32, 33, 34].

Điều này một lần nữa dẫn đến kiến trúc multi-leader: mỗi tab trình duyệt web đã mở tệp dùng chung là một replica, và bất kỳ bản cập nhật nào bạn thực hiện đối với tệp đều được replication bất đồng bộ tới thiết bị của những người dùng khác cũng đã mở cùng tệp đó. Ngay cả khi ứng dụng không cho phép bạn tiếp tục chỉnh sửa tệp khi ngoại tuyến, thì việc nhiều người dùng có thể thực hiện chỉnh sửa mà không cần chờ phản hồi từ máy chủ đã khiến nó trở thành kiến trúc multi-leader.

Cả việc chỉnh sửa ngoại tuyến và cộng tác thời gian thực đều yêu cầu một hạ tầng replication tương tự. Ứng dụng cần ghi lại bất kỳ thay đổi nào mà người dùng thực hiện đối với một tệp và gửi chúng tới những người cộng tác ngay lập tức (nếu đang trực tuyến) hoặc lưu trữ chúng cục bộ để gửi sau (nếu đang ngoại tuyến). Ngoài ra, ứng dụng cần nhận các thay đổi từ những người cộng tác, merge chúng vào bản sao cục bộ của tệp trên thiết bị người dùng, và cập nhật UI để phản ánh phiên bản mới

nhất. Nếu nhiều người dùng thay đổi tệp đồng thời, có thể cần đến logic conflict resolution để merge các thay đổi đó.

Một thư viện phần mềm hỗ trợ quá trình này được gọi là một *sync engine* (công cụ đồng bộ). Mặc dù ý tưởng này đã tồn tại từ lâu, nhưng thuật ngữ này gần đây đã nhận được nhiều sự chú ý [35, 36, 37]. Một ứng dụng cho phép người dùng tiếp tục chỉnh sửa một tệp khi ngoại tuyến (điều này có thể được triển khai bằng cách sử dụng một sync engine) được gọi là *offline-first* (ưu tiên ngoại tuyến) [38]. Thuật ngữ *local-first software* (phần mềm ưu tiên cục bộ) đề cập đến các ứng dụng cộng tác không chỉ là offline-first mà còn được thiết kế để tiếp tục hoạt động ngay cả khi lập trình viên tạo ra phần mềm đó đóng cửa tất cả các dịch vụ trực tuyến của họ [39]. Điều này có thể đạt được bằng cách sử dụng một sync engine với một giao thức đồng bộ tiêu chuẩn mở mà nhiều nhà cung cấp dịch vụ có thể đáp ứng [40]. Ví dụ, Git là một hệ thống cộng tác local-first (mặc dù nó không hỗ trợ cộng tác thời gian thực), vì bạn có thể đồng bộ qua GitHub, GitLab, hoặc bất kỳ dịch vụ lưu trữ repository nào khác.

Ưu và nhược điểm của sync engine

Cách chủ đạo để xây dựng các ứng dụng web ngày nay là giữ rất ít trạng thái bền vững (persistent state) trên client và dựa vào việc gửi các yêu cầu tới một server bất cứ khi nào cần hiển thị một phần dữ liệu mới hoặc cần cập nhật một số dữ liệu nào đó. Ngược lại, khi sử dụng một sync engine, bạn có trạng thái bền vững trên client, và việc giao tiếp với server được chuyển thành một tiến trình chạy ngầm. Cách tiếp cận sync engine có một số lợi thế:

- Việc có dữ liệu cục bộ có nghĩa là UI có thể phản hồi nhanh hơn nhiều so với việc phải chờ một lời gọi dịch vụ để lấy dữ liệu. Một số ứng dụng hướng tới việc phản hồi đầu vào của người dùng ngay trong *frame tiếp theo* của hệ thống đồ họa, nghĩa là kết xuất (rendering) trong vòng 16 ms trên một màn hình có tốc độ làm tươi 60 Hz.
- Cho phép người dùng tiếp tục làm việc khi ngoại tuyến là rất giá trị, đặc biệt trên các thiết bị di động có kết nối chậm chạp. Với một sync engine, một ứng dụng không cần một chế độ ngoại tuyến riêng biệt: trạng thái ngoại tuyến cũng giống như việc có một độ trễ mạng rất lớn.
- Một sync engine đơn giản hóa mô hình lập trình cho các ứng dụng frontend so với việc thực hiện các lời gọi dịch vụ tường minh trong mã nguồn ứng dụng. Mọi lời gọi dịch vụ đều yêu cầu xử lý lỗi, như đã thảo luận trong mục “Các vấn đề với remote procedure calls”; ví dụ, nếu một yêu cầu cập nhật dữ liệu trên server thất bại, giao diện người dùng cần phải phản ánh lỗi đó theo cách nào đó. Một sync engine cho phép ứng dụng thực hiện các thao tác đọc và ghi trên dữ liệu cục bộ; các thao tác này hầu như không bao giờ thất bại, dẫn đến một phong cách lập trình khai báo (declarative) hơn [41].

- Để hiển thị các chỉnh sửa từ những người dùng khác trong thời gian thực, bạn cần nhận được thông báo về các chỉnh sửa đó và cập nhật UI một cách hiệu quả tương ứng. Một sync engine kết hợp với mô hình *reactive programming* (lập trình phản ứng) là một cách tốt để triển khai điều này [42].

Sync engines hoạt động tốt nhất khi tất cả dữ liệu mà người dùng có thể cần được tải xuống trước và lưu trữ bền vững trên client. Điều này có nghĩa là dữ liệu luôn sẵn sàng để truy cập ngoại tuyến khi cần, nhưng nó cũng có nghĩa là các sync engines không phù hợp nếu người dùng có quyền truy cập vào một lượng dữ liệu cực lớn. Ví dụ, tải xuống tất cả các tệp mà người dùng đã tạo có lẽ là ổn (một người dùng thường không tạo ra nhiều dữ liệu đến thế), nhưng tải xuống toàn bộ danh mục của một website thương mại điện tử thì có lẽ không hợp lý.

Sync engine đã được tiên phong bởi Lotus Notes vào những năm 1980 [43] (mà không sử dụng thuật ngữ đó), và việc đồng bộ cho các ứng dụng cụ thể, chẳng hạn như lịch, cũng đã tồn tại từ lâu. Ngày nay, chúng ta có vô số các sync engine đa mục đích. Một số sử dụng dịch vụ backend độc quyền (ví dụ: Google Firestore, Realm, hoặc Ditto), và số khác có backend mã nguồn mở, giúp chúng phù hợp để tạo ra local-first software (ví dụ: PouchDB/CouchDB, Automerge, và Yjs).

Các trò chơi video multiplayer có nhu cầu tương tự là phải phản hồi ngay lập tức các hành động cục bộ của người dùng và điều hòa chúng với các hành động của những người chơi khác được nhận một cách bất đồng bộ qua mạng. Trong thuật ngữ chuyên ngành phát triển game, thuật ngữ tương đương với một sync engine được gọi là *netcode*. Các kỹ thuật được sử dụng trong netcode khá đặc thù cho các yêu cầu của trò chơi [44] và không được áp dụng trực tiếp cho các loại phần mềm khác, vì vậy chúng ta sẽ không xem xét chúng thêm trong cuốn sách này.

Giải quyết các xung đột ghi (Conflicting Writes)

Vấn đề lớn nhất với multi-leader replication—cả trong một cơ sở dữ liệu phía server phân tán theo địa lý lẫn một sync engine local-first trên các thiết bị của người dùng cuối—là các thao tác ghi đồng thời trên các leader khác nhau có thể dẫn đến các xung đột cần phải được giải quyết.

Ví dụ, hãy xét một trang wiki đang được chỉnh sửa đồng thời bởi hai người dùng, như được hiển thị trong Hình 6-9. Người dùng 1 thay đổi tiêu đề của trang từ A thành B, và người dùng 2 độc lập thay đổi tiêu đề từ A thành C. Thay đổi của mỗi người dùng đều được áp dụng thành công vào leader cục bộ của họ. Tuy nhiên, khi các thay đổi được replication một cách bất đồng bộ, một xung đột sẽ bị phát hiện. Vấn đề này không xảy ra trong một cơ sở dữ liệu single-leader.

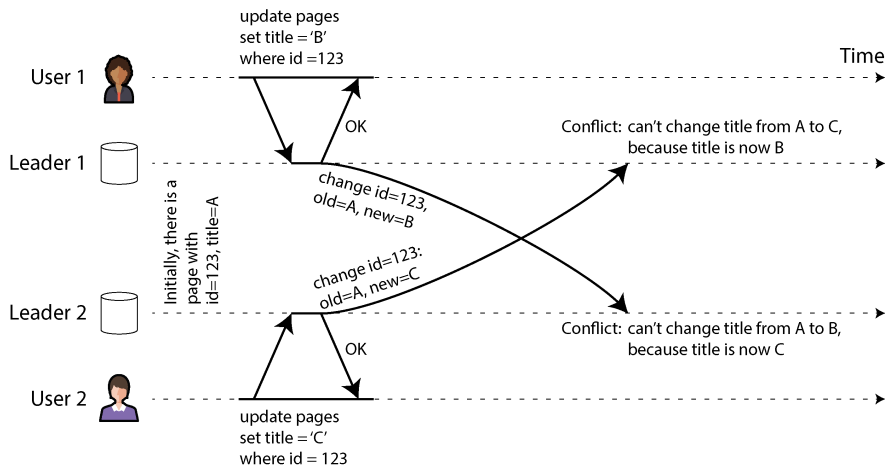


Figure 6-9. Một xung đột ghi (write conflict) gây ra bởi hai leader cùng lúc cập nhật một bản ghi giống nhau

LƯU Ý

Ta nói rằng hai thao tác ghi trong Hình 6-9 là *concurrent* (đồng thời) vì tại thời điểm thao tác ghi ban đầu được thực hiện, không thao tác nào "biết" về thao tác kia. Việc các thao tác ghi có thực sự xảy ra cùng một lúc hay không không quan trọng; thực tế, nếu các thao tác ghi được thực hiện khi đang ngoại tuyến, chúng có thể xảy ra cách nhau một khoảng thời gian. Điều quan trọng là liệu một thao tác ghi có xảy ra trong trạng thái mà thao tác ghi kia đã có hiệu lực hay chưa.

Trong mục "Phát hiện các thao tác ghi đồng thời", chúng ta sẽ giải quyết câu hỏi làm thế nào một cơ sở dữ liệu có thể xác định liệu hai thao tác ghi có phải là *concurrent* hay không. Hiện tại, chúng ta sẽ giả định rằng mình có thể phát hiện các xung đột và muốn tìm ra cách tốt nhất để giải quyết chúng.

Tránh xung đột

Một chiến lược để xử lý các xung đột là ngăn chặn chúng ngay từ đầu. Ví dụ, nếu ứng dụng có thể đảm bảo rằng tất cả các thao tác ghi cho một bản ghi cụ thể đều đi qua cùng một leader, thì xung đột sẽ không thể xảy ra, ngay cả khi toàn bộ cơ sở dữ liệu là multi-leader. Cách tiếp cận này không khả thi đối với một client của công cụ đồng bộ hóa (sync engine) đang được cập nhật khi ngoại tuyến, nhưng đôi khi nó có thể thực hiện được trong các hệ thống máy chủ geo-replicated [30].

Ví dụ, trong một ứng dụng mà người dùng chỉ có thể chỉnh sửa dữ liệu của chính họ, bạn có thể đảm bảo rằng các yêu cầu từ một người dùng cụ thể luôn được định tuyến đến cùng một vùng (region) và sử dụng leader ở vùng đó để đọc và ghi. Các người dùng khác nhau có thể có các vùng "nhà" (home regions) khác nhau (có lẽ

được chọn dựa trên khoảng cách địa lý tới người dùng), nhưng từ góc độ của bất kỳ người dùng nào, cấu hình về cơ bản là *single-leader*.

Tuy nhiên, đôi khi bạn có thể muốn thay đổi leader được chỉ định cho một bản ghi—có lẽ vì một vùng nào đó không khả dụng và bạn cần định tuyến lại lưu lượng đến một vùng khác, hoặc có lẽ vì một người dùng đã chuyển đến một vị trí khác và hiện đang ở gần một vùng khác hơn. Lúc này sẽ có rủi ro là người dùng thực hiện một thao tác ghi trong khi việc thay đổi leader được chỉ định đang diễn ra, dẫn đến một xung đột mà sẽ phải được giải quyết bằng một trong các phương pháp sau. Do đó, việc tránh xung đột sẽ thất bại nếu bạn cho phép thay đổi leader.

Một ví dụ khác về tránh xung đột, hãy tưởng tượng bạn muốn chen các bản ghi mới và tạo các ID duy nhất cho chúng dựa trên một bộ đếm tự động tăng (*autoincrementing counter*). Nếu bạn có hai leader, bạn có thể thiết lập sao cho một leader chỉ tạo ra các số lẻ và leader kia chỉ tạo ra các số chẵn. Bằng cách đó, bạn có thể chắc chắn rằng hai leader sẽ không đồng thời gán cùng một ID cho các bản ghi khác nhau. Chúng ta sẽ thảo luận về các sơ đồ gán ID khác trong mục "ID Generators và Logical Clocks".

Last write wins (loại bỏ các thao tác ghi đồng thời)

Nếu không thể tránh được xung đột, cách đơn giản nhất để giải quyết chúng là gán một timestamp vào mỗi thao tác ghi và luôn sử dụng giá trị có timestamp mới nhất (lớn nhất). Ví dụ, trong Hình 6-9, giả sử timestamp của thao tác ghi của người dùng 1 lớn hơn timestamp của thao tác ghi của người dùng 2. Trong trường hợp đó, cả hai leader sẽ xác định rằng tiêu đề mới của trang phải là B, và chúng sẽ loại bỏ thao tác ghi thiết lập nó thành C. Nếu các thao tác ghi tình cờ có cùng timestamp, người chiến thắng có thể được chọn bằng cách so sánh các giá trị (ví dụ: đối với chuỗi ký tự, lấy chuỗi đứng trước trong bảng chữ cái).

Cách tiếp cận này được gọi là *last write wins* (LWW) vì thao tác ghi có timestamp lớn nhất có thể được coi là thao tác "cuối cùng". Tuy nhiên, thuật ngữ này gây hiểu lầm, bởi vì khi hai thao tác ghi là *concurrent* (như trong Hình 6-9), việc thao tác nào là mới nhất là không xác định, vì vậy thứ tự timestamp của các thao tác ghi đồng thời về cơ bản là ngẫu nhiên.

Do đó, ý nghĩa thực sự của LWW là: khi cùng một bản ghi được ghi đồng thời trên các leader khác nhau, một trong số các thao tác ghi đó sẽ được chọn ngẫu nhiên để trở thành người chiến thắng và các thao tác ghi khác sẽ bị loại bỏ một cách âm thầm, ngay cả khi chúng đã được xử lý thành công bởi các leader tương ứng. Điều này đặt được mục tiêu là cuối cùng tất cả các *replica* sẽ ở trong một trạng thái nhất quán, nhưng phải trả giá bằng việc mất dữ liệu.

Nếu bạn có thể tránh được các xung đột—ví dụ, bằng cách chỉ chen các bản ghi với một khóa duy nhất và không bao giờ cập nhật chúng—thì LWW sẽ không thành vấn

đề. Nhưng nếu bạn cập nhật các bản ghi hiện có, hoặc nếu các leader khác nhau có thể chen các bản ghi với cùng một khóa, thì bạn phải quyết định xem liệu lost updates có phải là vấn đề đối với ứng dụng của mình hay không. Nếu lost updates là không thể chấp nhận được, bạn cần sử dụng một trong các phương pháp conflict resolution được mô tả tiếp theo.

Một vấn đề khác với LWW là nếu một đồng hồ thời gian thực (ví dụ: một Unix timestamp) được sử dụng làm timestamp cho các thao tác ghi, hệ thống sẽ trở nên rất nhạy cảm với việc đồng bộ hóa đồng hồ. Nếu một node có đồng hồ chạy nhanh hơn các node khác, và bạn cố gắng ghi đè một giá trị đã được ghi bởi node đó, thao tác ghi của bạn có thể bị bỏ qua vì nó có timestamp thấp hơn, mặc dù rõ ràng nó xảy ra muộn hơn. Vấn đề này có thể được giải quyết bằng cách sử dụng một *logical clock* (đồng hồ logic), điều mà chúng ta sẽ thảo luận trong mục “ID Generators and Logical Clocks”.

Giải quyết xung đột thủ công

Nếu việc loại bỏ ngẫu nhiên một số thao tác ghi của bạn là không mong muốn, lựa chọn tiếp theo là giải quyết xung đột một cách thủ công. Bạn có thể đã quen thuộc với việc giải quyết xung đột thủ công từ Git và các hệ thống quản lý phiên bản khác: nếu các commit trên hai branch chỉnh sửa cùng các dòng của cùng một tệp, và bạn cố gắng merge các branch đó, bạn sẽ gặp một merge conflict cần phải được giải quyết trước khi quá trình merge hoàn tất.

Trong một cơ sở dữ liệu, việc một xung đột làm dừng toàn bộ quá trình replication là không thực tế cho đến khi một con người giải quyết nó. Thay vào đó, các cơ sở dữ liệu thường lưu trữ tất cả các giá trị được ghi đồng thời cho một bản ghi nhất định—ví dụ, cả B và C trong Hình 6-9. Những giá trị này đôi khi được gọi là *siblings*. Lần tiếp theo bạn truy vấn bản ghi đó, cơ sở dữ liệu sẽ trả về *tất cả* các giá trị đó thay vì chỉ trả về giá trị mới nhất. Sau đó, bạn có thể giải quyết các giá trị đó theo bất kỳ cách nào bạn muốn, hoặc là tự động trong mã nguồn ứng dụng (ví dụ: bạn có thể nối B và C thành B/C) hoặc bằng cách hỏi người dùng. Sau đó, bạn ghi một giá trị mới trở lại cơ sở dữ liệu để giải quyết xung đột.

Cách tiếp cận giải quyết xung đột này được sử dụng trong một số hệ thống, chẳng hạn như CouchDB. Tuy nhiên, nó cũng gặp phải những vấn đề sau:

- API của cơ sở dữ liệu thay đổi—ví dụ, trong khi trước đây tiêu đề của trang wiki chỉ là một chuỗi (string), thì giờ đây nó trở thành một tập hợp các chuỗi thường chứa một phần tử, nhưng đôi khi có thể chứa nhiều phần tử nếu có xung đột. Điều này có thể khiến dữ liệu trở nên khó làm việc trong mã nguồn ứng dụng.
- Việc yêu cầu người dùng dùng hợp nhất các siblings một cách thủ công là một khối lượng công việc lớn, cho cả lập trình viên ứng dụng (người cần xây dựng UI để giải quyết xung đột) và cho người dùng (người có thể bị bối rối về những gì họ

được yêu cầu làm và tại sao). Trong nhiều trường hợp, việc hợp nhất tự động sẽ tốt hơn là làm phiền người dùng.

- Việc hợp nhất các siblings một cách tự động có thể dẫn đến các hành vi gây ngạc nhiên nếu nó không được thực hiện cẩn thận. Ví dụ, giỏ hàng trên Amazon từng cho phép các cập nhật đồng thời, sau đó được hợp nhất bằng cách giữ lại tất cả các mặt hàng trong giỏ hàng xuất hiện trong bất kỳ sibling nào (tức là lấy hợp của các tập hợp giỏ hàng). Điều này có nghĩa là nếu khách hàng đã xóa một mặt hàng khỏi giỏ hàng của họ trong một sibling, nhưng một sibling khác vẫn còn chứa mặt hàng cũ đó, thì mặt hàng đã bị xóa sẽ bất ngờ xuất hiện trở lại trong giỏ hàng của khách hàng [45]. Trong Hình 6-10, thiết bị 1 xóa Book khỏi giỏ hàng và đồng thời thiết bị 2 xóa DVD, nhưng sau khi hợp nhất các siblings, cả hai mặt hàng đều xuất hiện trở lại.
- Nếu nhiều node cùng quan sát thấy xung đột và đồng thời giải quyết nó, bản thân quá trình giải quyết xung đột có thể tạo ra một xung đột mới. Những giải quyết đó thậm chí có thể không nhất quán—ví dụ, một node có thể hợp nhất B và C thành B/C trong khi một node khác có thể hợp nhất chúng thành C/B nếu bạn không cẩn thận sắp xếp chúng một cách nhất quán. Khi xung đột giữa B/C và C/B được hợp nhất, nó có thể dẫn đến kết quả B/C/C/B hoặc một thứ gì đó gây ngạc nhiên tương tự.

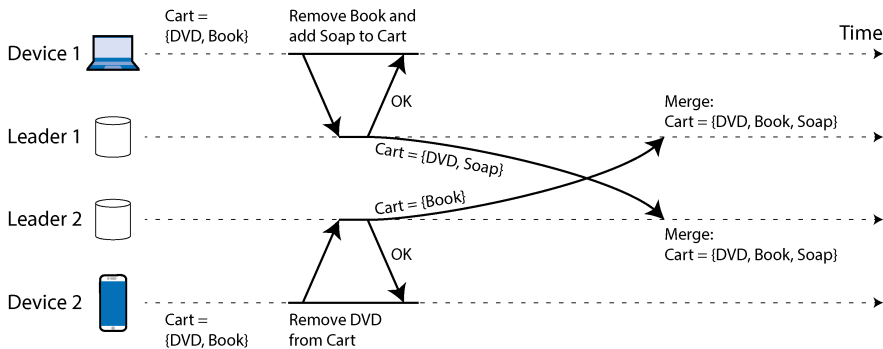


Figure 6-10. Một ví dụ về sự bất thường trong giỏ hàng của Amazon: nếu các xung đột được merge bằng cách lấy hợp (union) của các tập hợp, các mục đã bị xóa có thể xuất hiện trở lại

Tự động giải quyết xung đột

Đối với nhiều ứng dụng, cách tốt nhất để xử lý xung đột là sử dụng một thuật toán tự động hợp nhất các thao tác ghi đồng thời thành một trạng thái nhất quán. Tự động giải quyết xung đột đảm bảo rằng tất cả các replica đều *hội tụ* (converge) về cùng một trạng thái—nghĩa là, tất cả các replica đã xử lý cùng một tập hợp các thao tác ghi đều có cùng một trạng thái, bất kể thứ tự mà các thao tác ghi đó đến. Việc kết hợp

eventual consistency với một sự đảm bảo về hội tụ được gọi là *strong eventual consistency* [46].

LWW là một ví dụ đơn giản về thuật toán giải quyết xung đột. Các thuật toán hợp nhất tinh vi hơn đã được phát triển cho các loại dữ liệu khác nhau, với mục tiêu bảo tồn tối đa hiệu ứng dự kiến của tất cả các cập nhật và từ đó tránh mất dữ liệu:

- Nếu dữ liệu là văn bản (ví dụ: tiêu đề hoặc nội dung của một trang wiki), chúng ta có thể phát hiện những ký tự nào đã được chèn vào hoặc bị xóa đi từ phiên bản này sang phiên bản kế tiếp. Kết quả hợp nhất sau đó sẽ bảo tồn tất cả các thao tác chèn và xóa đã thực hiện trong bất kỳ sibling nào. Nếu người dùng đồng thời chèn văn bản tại cùng một vị trí, nó có thể được sắp xếp một cách xác định để tất cả các node đều nhận được cùng một kết quả hợp nhất.
- Nếu dữ liệu là một tập hợp các mục (được sắp xếp như một danh sách việc cần làm, hoặc không được sắp xếp như một giỏ hàng), chúng ta có thể hợp nhất nó tương tự như văn bản bằng cách theo dõi các thao tác chèn và xóa. Để tránh vấn đề giỏ hàng trong Hình 6-10, các thuật toán theo dõi thực tế rằng Book và DVD đã bị xóa, vì vậy kết quả hợp nhất là $Cart = \{Soap\}$.
- Nếu dữ liệu là một số nguyên đại diện cho một bộ đếm có thể tăng hoặc giảm (ví dụ: số lượt thích trên một bài đăng mạng xã hội), thuật toán hợp nhất có thể cho biết có bao nhiêu lần tăng và giảm đã xảy ra trên mỗi sibling và cộng chúng lại với nhau một cách chính xác để kết quả không bị đếm trùng và không bị mất các cập nhật.
- Nếu dữ liệu là một ánh xạ key-value, chúng ta có thể hợp nhất các cập nhật cho cùng một key bằng cách áp dụng một trong các thuật toán giải quyết xung đột khác cho các value dưới key đó. Các cập nhật cho các key khác nhau có thể được xử lý độc lập với nhau.

Có những giới hạn về những gì có thể thực hiện được với việc giải quyết xung đột. Ví dụ, nếu bạn muốn bắt buộc một danh sách không chứa quá năm mục, và nhiều người dùng đồng thời thêm các mục vào danh sách khiến tổng số mục lớn hơn năm, lựa chọn duy nhất của bạn là loại bỏ một số mục. Tuy nhiên, tự động giải quyết xung đột là đủ để xây dựng nhiều ứng dụng hữu ích. Và nếu bạn bắt đầu từ yêu cầu muốn xây dựng một ứng dụng cộng tác theo kiểu offline-first hoặc local-first, thì việc giải quyết xung đột là không thể tránh khỏi, và tự động hóa nó thường là cách tiếp cận tốt nhất.

Conflict-free replicated datatypes và operational transformation

Hai họ thuật toán thường được sử dụng để triển khai giải quyết xung đột tự động: *conflict-free replicated datatypes* (CRDTs) [46] và *operational transformation* (OT) [47]. Chúng có các triết lý thiết kế và đặc tính hiệu năng khác nhau, nhưng cả hai

đều có khả năng thực hiện hợp nhất tự động cho tất cả các loại dữ liệu đã đề cập ở trên.

Hình 6-11 cho thấy một ví dụ về cách OT và một CRDT hợp nhất các cập nhật đồng thời cho một văn bản. Giả sử bạn có hai replica đều bắt đầu với văn bản *ice*. Một replica chèn thêm chữ *n* vào đầu để tạo thành *nice*, trong khi đó replica còn lại chèn thêm dấu chấm than vào cuối để tạo thành *ice!* một cách đồng thời.

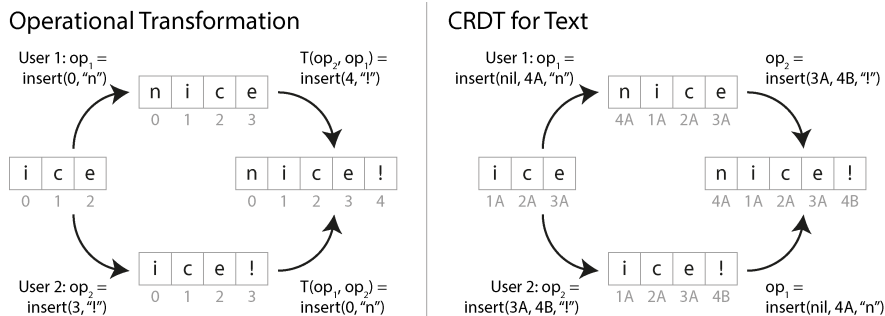


Figure 6-11. Hình: Cách hai thao tác chèn đồng thời vào một chuỗi được hợp nhất bởi OT và một CRDT, tương ứng

Kết quả hợp nhất *nice!* được đạt được theo những cách khác nhau bởi hai loại thuật toán:

OT

Chúng ta ghi lại index mà tại đó các ký tự được chèn vào hoặc bị xóa: *n* được chèn vào index 0 và *!* vào index 3. Tiếp theo, các replica trao đổi các thao tác của chúng. Việc chèn *n* tại index 0 có thể được áp dụng nguyên trạng, nhưng nếu việc chèn *!* tại index 3 được áp dụng vào trạng thái *nice*, chúng ta sẽ nhận được *nice!*, vốn là kết quả không chính xác. Do đó, chúng ta cần biến đổi index của mỗi thao tác để tính đến các thao tác đồng thời đã được áp dụng trước đó. Trong trường hợp này, việc chèn *!* được biến đổi thành index 4 để tính đến việc chèn *n* tại một index sớm hơn.

CRDT

Hầu hết các CRDT gán cho mỗi ký tự một ID duy nhất, không thể thay đổi và sử dụng chúng để xác định vị trí của các thao tác chèn/xóa, thay vì sử dụng index. Ví dụ, trong Hình 6-11, chúng ta gán ID 1A cho *i*, ID 2A cho *c*, v.v. Khi chèn dấu chấm than, chúng ta tạo ra một thao tác chứa ID của ký tự mới (4B) và ID của ký tự hiện có mà sau đó chúng ta muốn chèn nó vào (3A). Để chèn vào đầu chuỗi, chúng ta đưa *nil* làm ID của ký tự đứng trước. Các thao tác chèn đồng thời tại cùng một vị trí được sắp xếp theo ID của các ký tự. Điều này đảm bảo các replica hội tụ mà không cần thực hiện bất kỳ sự biến đổi nào.

Nhiều thuật toán dựa trên các biến thể của những ý tưởng này. Các list và array có thể được hỗ trợ tương tự bằng cách sử dụng các phần tử list thay vì ký tự, và các kiểu dữ liệu khác, chẳng hạn như key-value maps, có thể được thêm vào khá dễ dàng. OT và CRDT có một số đánh đổi (trade-off) về hiệu năng và chức năng, nhưng chúng ta có thể kết hợp các ưu điểm của cả hai vào trong một thuật toán [48].

OT thường được sử dụng nhất cho việc chỉnh sửa văn bản cộng tác thời gian thực, chẳng hạn như trong Google Docs [32], trong khi CRDT có thể được tìm thấy trong các cơ sở dữ liệu phân tán như Redis Enterprise, Riak và Azure Cosmos DB [49]. Các engine đồng bộ cho dữ liệu JSON có thể được triển khai bằng cả CRDT (ví dụ: Automerge hoặc Yjs) và OT (ví dụ: ShareDB).

Các loại xung đột

Một số loại xung đột rất rõ ràng. Trong ví dụ ở Hình 6-9, hai thao tác ghi đã sửa đổi đồng thời cùng một field trong cùng một record, thiết lập nó thành hai giá trị khác nhau. Không còn nghi ngờ gì nữa, đây là một xung đột.

Các loại xung đột khác có thể khó phát hiện hơn. Ví dụ, hãy xét một hệ thống đặt phòng hợp: hệ thống này theo dõi phòng nào được đặt bởi nhóm người nào vào thời gian nào. Thay vì cập nhật một field cụ thể khi đặt hợp, hệ thống này chèn một record mới vào cơ sở dữ liệu cho mỗi lần đặt. Ứng dụng cần đảm bảo rằng mỗi phòng chỉ được đặt bởi một nhóm người tại bất kỳ thời điểm nào (tức là không được có bất kỳ việc đặt phòng nào bị chồng lấn cho cùng một phòng). Trong trường hợp này, một xung đột có thể phát sinh nếu hai lượt đặt phòng được tạo cho cùng một phòng tại cùng một thời điểm. Ngay cả khi ứng dụng kiểm tra tính khả dụng trước khi cho phép người dùng đặt phòng, một xung đột vẫn có thể phát sinh nếu hai lượt đặt được thực hiện đủ gần nhau đến mức cả hai đều thấy phòng đó chưa được đặt trước khi chèn record mới của chúng.

Không có câu trả lời sẵn có nào nhanh chóng, nhưng trong các chương tiếp theo, chúng ta sẽ đi theo một lộ trình hướng tới sự hiểu biết đầy đủ về vấn đề này. Chúng ta sẽ thấy thêm nhiều ví dụ về xung đột trong Chương 8, và trong Chương 13, chúng ta sẽ thảo luận về các phương pháp có khả năng mở rộng để phát hiện và giải quyết xung đột trong một hệ thống được nhân bản.

Leaderless Replication

Các phương pháp replication mà chúng ta đã thảo luận cho đến nay trong chương này—single-leader và multi-leader replication—dựa trên ý tưởng rằng một client gửi một yêu cầu ghi đến một node (leader), và hệ thống cơ sở dữ liệu sẽ lo việc sao chép lệnh ghi đó sang các replica khác. Một leader quyết định thứ tự mà các lệnh ghi nên được xử lý, và các follower áp dụng các lệnh ghi của leader theo cùng thứ tự đó.

Một số hệ thống lưu trữ dữ liệu áp dụng cách tiếp cận khác, từ bỏ khái niệm leader và cho phép bất kỳ replica nào cũng có thể trực tiếp chấp nhận các lệnh ghi từ client. Một số hệ thống dữ liệu được replication sớm nhất là loại leaderless [1, 50], nhưng ý tưởng này hầu như đã bị lãng quên trong kỷ nguyên thống trị của các cơ sở dữ liệu quan hệ. Nó đã trở lại thành một kiến trúc thời thượng cho các cơ sở dữ liệu sau khi Amazon sử dụng nó cho hệ thống Dynamo nội bộ vào năm 2007 [45]. Riak, Cassandra và ScyllaDB là các datastore mã nguồn mở với mô hình leaderless replication được truyền cảm hứng từ Dynamo, vì vậy loại cơ sở dữ liệu này còn được gọi là *Dynamo-style*.

LƯU Ý

Kiến trúc hệ thống Dynamo nguyên bản đã được mô tả trong một bài báo [45] nhưng chưa bao giờ được phát hành ra bên ngoài Amazon. DynamoDB có tên gọi tương tự, một cơ sở dữ liệu đám mây gần đây hơn từ Amazon, lại có kiến trúc hoàn toàn khác: nó sử dụng single-leader replication dựa trên thuật toán consensus Multi-Paxos [5, 51].

Trong một số triển khai leaderless, client gửi trực tiếp các lệnh ghi của mình tới vài replica, trong khi ở những triển khai khác, một coordinator node sẽ thực hiện việc này thay mặt cho client. Tuy nhiên, không giống như một cơ sở dữ liệu có leader, coordinator đó không thực thi một thứ tự ghi cụ thể nào. Như chúng ta sẽ thấy, sự khác biệt trong thiết kế này dẫn đến những hệ quả sâu sắc đối với cách thức sử dụng cơ sở dữ liệu.

Ghi vào Cơ sở dữ liệu Khi một node Đang ngoại tuyến

Hãy tưởng tượng bạn có một cơ sở dữ liệu với ba replica, và một trong các replica đó hiện không khả dụng—có lẽ nó đang được khởi động lại để cài đặt bản cập nhật hệ thống. Trong cấu hình single-leader, nếu bạn muốn tiếp tục xử lý các lệnh ghi, bạn có thể cần thực hiện một thao tác failover (xem mục “Handling Node Outages”).

Mặt khác, trong cấu hình leaderless, không có khái niệm failover, vì tất cả các replica đều bình đẳng và không có leader. Hình 6-12 cho thấy điều gì sẽ xảy ra.

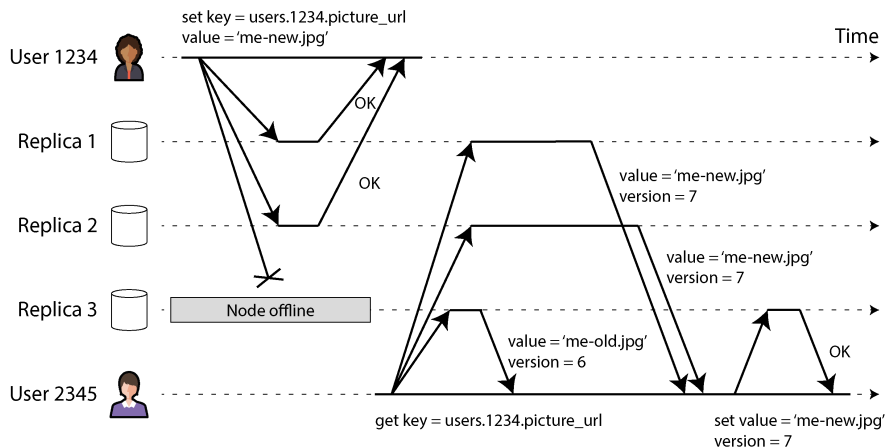


Figure 6-12. Ghi vào đa số các replica, đọc từ đa số các replica, và chuyển giá trị mới nhất tới một replica đã không khả dụng trong quá trình ghi

Client (người dùng 1234) gửi yêu cầu ghi tới cả ba replica song song, và hai replica đang hoạt động chấp nhận yêu cầu ghi, nhưng replica không khả dụng đã bỏ lỡ nó. Hãy giả sử rằng chỉ cần hai trong số ba replica xác nhận yêu cầu ghi là đủ. Sau khi người dùng 1234 nhận được hai phản hồi OK, chúng ta coi yêu cầu ghi đó đã thành công. Client đơn giản là bỏ qua việc một trong các replica đã bỏ lỡ yêu cầu ghi.

Bây giờ hãy tưởng tượng rằng node không khả dụng đó quay trở lại trực tuyến, và các client bắt đầu đọc từ nó. Bất kỳ yêu cầu ghi nào xảy ra trong khi node đó bị ngắt kết nối đều bị thiếu trên nó. Do đó, nếu bạn đọc từ node đó, bạn có thể nhận được các giá trị *stale* (lỗi thời) làm phản hồi.

Để giải quyết vấn đề đó, khi một client đọc từ cơ sở dữ liệu, nó không chỉ gửi yêu cầu tới một replica: *các yêu cầu đọc cũng được gửi tới nhiều node song song*. Client có thể nhận được các phản hồi khác nhau từ các node khác nhau; ví dụ, nhận được giá trị mới nhất từ một node và một giá trị *stale* từ một node khác.

Để client có thể xác định phản hồi nào là mới nhất và phản hồi nào đã lỗi thời, mọi giá trị được ghi cần được gắn với một số phiên bản hoặc *timestamp*, tương tự như những gì chúng ta đã thấy trong mục “Last write wins (discarding concurrent writes)”. Khi một client nhận được nhiều giá trị để phản hồi cho một yêu cầu đọc, nó sẽ sử dụng giá trị có *timestamp* lớn nhất (ngay cả khi giá trị đó chỉ được trả về bởi một replica, và nhiều replica khác trả về các giá trị cũ hơn). Xem mục “Detecting Concurrent Writes” để biết thêm chi tiết.

Cập nhật các yêu cầu ghi bị bỏ lỡ

Hệ thống replication phải đảm bảo rằng cuối cùng tất cả dữ liệu sẽ được sao chép tới mọi replica. Sau khi một node không khả dụng quay trở lại trực tuyến,

làm thế nào để nó cập nhật các yêu cầu ghi mà nó đã bỏ lỡ? Một số cơ chế được sử dụng trong các datastore kiểu Dynamo:

Read repair

Khi một client thực hiện đọc từ nhiều node song song, nó có thể phát hiện bất kỳ phân hồi *stale* nào. Ví dụ, trong Hình 6-12, người dùng 2345 nhận được giá trị phiên bản 6 từ replica 3 và giá trị phiên bản 7 từ các replica 1 và 2. Client thấy rằng replica 3 có giá trị *stale* và ghi lại giá trị mới hơn vào replica đó. Cách tiếp cận này hoạt động tốt đối với các giá trị được đọc thường xuyên.

Hinted handoff

Nếu một replica không khả dụng, một replica khác có thể lưu trữ các yêu cầu ghi thay cho nó dưới dạng các *hints*. Khi replica đáng lẽ phải nhận các yêu cầu ghi đó quay trở lại, replica đang lưu trữ các hints sẽ gửi chúng tới replica vừa được khôi phục và sau đó xóa các hints đi. Quá trình *handoff* này giúp đưa các replica về trạng thái cập nhật mới nhất, ngay cả đối với các giá trị không bao giờ được đọc và do đó không được xử lý bởi read repair.

Anti-entropy

Ngoài ra, một tiến trình chạy ngầm sẽ định kỳ tìm kiếm sự khác biệt về dữ liệu giữa các replica và sau đó sao chép bất kỳ dữ liệu thiếu hụt nào từ một replica này sang replica khác. Không giống như replication log trong leader-based replication, *tiến trình anti-entropy* này không sao chép các yêu cầu ghi theo một thứ tự cụ thể nào, và có thể có một sự chậm trễ đáng kể trước khi dữ liệu được sao chép.

Sử dụng quorum để đọc và ghi

Trong Hình 6-12, chúng ta đã coi yêu cầu ghi là thành công ngay cả khi nó chỉ được xử lý trên hai trong số ba replica. Điều gì sẽ xảy ra nếu chỉ có một trong ba replica chấp nhận yêu cầu ghi? Chúng ta có thể đẩy giới hạn này đi xa đến mức nào?

Nếu chúng ta biết rằng mọi yêu cầu ghi thành công đều được đảm bảo hiện diện trên ít nhất hai trong số ba replica, điều đó có nghĩa là tối đa chỉ có một replica có thể bị *stale*. Do đó, nếu chúng ta đọc từ ít nhất hai replica, chúng ta có thể chắc chắn rằng ít nhất một trong hai replica đó là mới nhất. Nếu replica thứ ba bị ngắt kết nối hoặc phản hồi chậm, các yêu cầu đọc vẫn có thể tiếp tục trả về một giá trị mới nhất.

Tổng quát hơn, nếu có n replica, mọi thao tác ghi phải được xác nhận bởi w node để được coi là thành công, và chúng ta phải truy vấn ít nhất r node cho mỗi thao tác đọc. (Trong ví dụ của chúng ta, $n = 3$, $w = 2$, $r = 2$.) Miễn là $w + r > n$, chúng ta kỳ vọng sẽ nhận được giá trị cập nhật nhất khi đọc, bởi vì ít nhất một trong số r node mà chúng ta đang đọc phải là node đã được cập nhật. Các thao tác đọc và ghi tuân

thủ các giá trị r và w này được gọi là *quorum* reads và writes [50]. Bạn có thể coi r và w là số phiếu bầu tối thiểu cần thiết để thao tác đọc hoặc ghi có hiệu lực.

Trong các cơ sở dữ liệu kiểu Dynamo, các tham số n , w , và r thường có thể cấu hình được. Một lựa chọn phổ biến là đặt n là một số lẻ (thường là 3 hoặc 5) và thiết lập $w = r = (n + 1) / 2$ (làm tròn lên). Tuy nhiên, bạn có thể thay đổi các con số này tùy ý. Ví dụ, một khối lượng công việc có ít thao tác ghi và nhiều thao tác đọc có thể hưởng lợi từ việc thiết lập $w = n$ và $r = 1$. Điều này giúp các thao tác đọc nhanh hơn nhưng có nhược điểm là chỉ cần một node bị lỗi sẽ khiến tất cả các thao tác ghi vào cơ sở dữ liệu đều thất bại.

LƯU Ý

Có thể có nhiều hơn n node trong cluster, nhưng bất kỳ giá trị cụ thể nào cũng chỉ được lưu trữ trên n node. Điều này cho phép dataset được sharding, hỗ trợ các dataset lớn hơn mức mà bạn có thể chứa trên một node duy nhất. Chúng ta sẽ quay lại với sharding trong Chương 7.

Điều kiện *quorum*, $w + r > n$, cho phép hệ thống chịu lỗi đối với các node không khả dụng như sau:

- Nếu $w < n$, chúng ta vẫn có thể xử lý các thao tác ghi ngay cả khi một node không khả dụng.
- Nếu $r < n$, chúng ta vẫn có thể xử lý các thao tác đọc ngay cả khi một node không khả dụng.
- Với $n = 3$, $w = 2$, $r = 2$, chúng ta có thể chịu lỗi một node không khả dụng, như trong Hình 6-12.
- Với $n = 5$, $w = 3$, $r = 3$, chúng ta có thể chịu lỗi hai node không khả dụng. Trường hợp này được minh họa trong Hình 6-13.

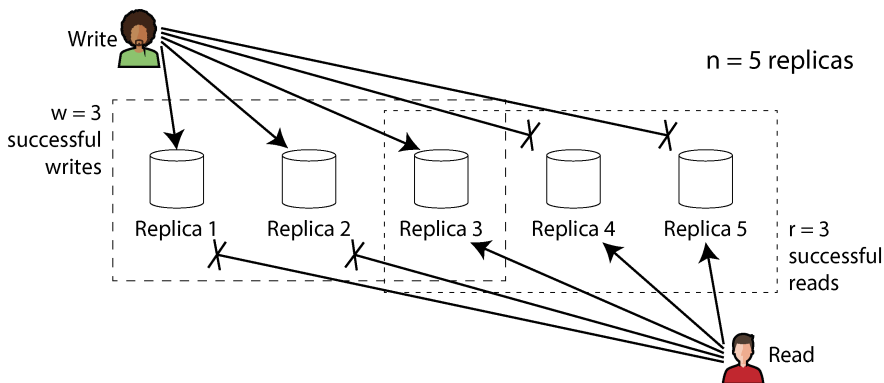


Figure 6-13. Nếu $w + r > n$, ít nhất một trong số r replica mà bạn đọc được phải là replica đã chứng kiến lần write thành công gần nhất.

Thông thường, các thao tác đọc và ghi luôn được gửi đến tất cả n replica một cách song song. Các tham số w và r sẽ quyết định số lượng node mà chúng ta phải chờ—tức là có bao nhiêu node trong số n node cần báo cáo thành công trước khi chúng ta coi thao tác đọc hoặc ghi đó là thành công.

Nếu số lượng node khả dụng ít hơn mức w hoặc r yêu cầu, các thao tác ghi hoặc đọc sẽ trả về lỗi. Một node có thể không khả dụng vì nhiều lý do: node bị sập (ví dụ: bị crash, mất nguồn), xảy ra lỗi trong khi thực thi thao tác (ví dụ: không thể ghi vì đĩa đầy), xảy ra gián đoạn mạng giữa client và node, hoặc bất kỳ lý do nào khác. Chúng ta chỉ quan tâm liệu node đó có trả về phản hồi thành công hay không và không cần phân biệt giữa các loại fault khác nhau.

Hiểu về các giới hạn của quorum consistency

Nếu bạn có n replica, và bạn chọn w và r sao cho $w + r > n$, bạn thường có thể kỳ vọng mọi thao tác đọc đều trả về giá trị mới nhất đã được ghi cho một key. Điều này xảy ra vì tập hợp các node mà bạn đã ghi dữ liệu và tập hợp các node mà bạn đọc dữ liệu phải có phần giao nhau. Tức là, trong số các node mà bạn đọc, phải có ít nhất một node giữ giá trị mới nhất (như được minh họa trong Hình 6-13).

Thông thường, r và w được chọn là đa số (nhiều hơn $n / 2$) các node, vì điều đó đảm bảo $w + r > n$ trong khi vẫn có thể chịu được tới $n / 2$ (làm tròn xuống) lỗi node. Nhưng quorum không nhất thiết phải là đa số—điều quan trọng duy nhất là tập hợp các node được sử dụng bởi các thao tác đọc và ghi phải giao nhau ở ít nhất một node. Các cách gán quorum khác cũng khả thi, cho phép một sự linh hoạt nhất định trong việc thiết kế các thuật toán phân tán [52].

Bạn cũng có thể thiết lập w và r thành các số nhỏ hơn, sao cho $w + r \leq n$ (tức là điều kiện quorum không được thỏa mãn). Trong trường hợp này, các thao tác đọc và ghi vẫn sẽ được gửi đến n node, nhưng chỉ cần một số lượng phản hồi thành công ít hơn để thao tác đó thành công.

Với w và r nhỏ hơn, bạn sẽ có nhiều khả năng đọc phải các giá trị cũ (stale values), vì khả năng cao là thao tác đọc của bạn sẽ không bao gồm node giữ giá trị mới nhất. Về mặt tích cực, cấu hình này cho phép latency thấp hơn, điều này đặc biệt có lợi với cơ chế replication *synchronous (blocking)*. Thiết lập này cũng có tính sẵn sàng cao hơn; nếu có sự gián đoạn mạng và nhiều replica trở nên không thể kết nối, sẽ có cơ hội cao hơn là bạn có thể tiếp tục xử lý các thao tác đọc và ghi. Chỉ sau khi số lượng replica có thể kết nối giảm xuống dưới w hoặc r , cơ sở dữ liệu mới lần lượt trở nên không khả dụng cho việc ghi hoặc đọc.

Tuy nhiên, ngay cả khi $w + r > n$, các đặc tính consistency vẫn có thể gây nhầm lẫn trong một số trường hợp biên (edge cases) nhất định. Một số kịch bản bao gồm sau đây:

- Nếu một node đang giữ giá trị mới bị lỗi, và dữ liệu của nó được khôi phục từ một replica đang giữ giá trị cũ, số lượng replica lưu trữ giá trị mới có thể giảm xuống dưới w , làm phá vỡ điều kiện quorum.
- Trong khi quá trình rebalancing đang diễn ra, nơi một số dữ liệu được di chuyển từ node này sang node khác (xem Chương 7), các node có thể có các góc nhìn không nhất quán về việc những node nào nên giữ n replica cho một giá trị cụ thể. Điều này có thể dẫn đến việc các quorum đọc và ghi không còn giao nhau nữa.
- Nếu một thao tác đọc diễn ra đồng thời với một thao tác ghi, thao tác đọc có thể thấy hoặc không thấy giá trị được ghi đồng thời đó. Cụ thể, có khả năng một thao tác đọc thấy giá trị mới và một thao tác đọc tiếp theo lại thấy giá trị cũ, như chúng ta sẽ thấy trong mục “Implementing Linearizable Systems”.
- Nếu một thao tác ghi thành công trên một số replica nhưng thất bại trên các replica khác (ví dụ: vì đĩa trên một số node đã đầy), và xét về tổng thể nó thành công trên ít hơn w replica, thì nó sẽ không được rollback trên các replica mà nó đã thành công. Điều này có nghĩa là nếu một thao tác ghi được báo cáo là thất bại, các thao tác đọc tiếp theo có thể thấy hoặc không thấy giá trị từ lần ghi đó [53].
- Nếu cơ sở dữ liệu sử dụng timestamp từ một đồng hồ thời gian thực để xác định lần ghi nào là mới hơn (ví dụ như cách Cassandra và ScyllaDB thực hiện), các lệnh ghi có thể bị loại bỏ một cách âm thầm nếu một node khác có đồng hồ chạy nhanh hơn đã ghi vào cùng một key—một vấn đề mà chúng ta đã thấy trước đó trong mục “Last write wins (discarding concurrent writes)”. Chúng ta sẽ thảo luận chi tiết hơn về vấn đề này trong mục “Relying on Synchronized Clocks”.
- Nếu hai lệnh ghi xảy ra đồng thời, một trong hai có thể được xử lý trước trên một replica, và lệnh kia có thể được xử lý trước trên một replica khác. Điều này dẫn đến một xung đột, tương tự như những gì chúng ta đã thấy đối với multi-leader replication (xem mục “Dealing with Conflicting Writes”). Chúng ta sẽ quay lại chủ đề này trong mục “Detecting Concurrent Writes”.

Do đó, mặc dù các quorum có vẻ như đảm bảo rằng một lệnh đọc sẽ trả về giá trị được ghi mới nhất, nhưng trong thực tế thì không đơn giản như vậy. Các cơ sở dữ liệu kiểu Dynamo thường được tối ưu hóa cho các trường hợp sử dụng có thể chấp nhận eventual consistency. Các tham số w và r cho phép bạn điều chỉnh xác suất các giá trị cũ (stale values) bị đọc [54], nhưng tốt nhất là không nên coi chúng là những sự đảm bảo tuyệt đối.

Giám sát độ cũ (staleness)

Từ góc độ vận hành, việc giám sát xem các cơ sở dữ liệu của bạn có đang trả về các kết quả cập nhật nhất hay không là rất quan trọng. Ngay cả khi ứng dụng của bạn có thể chấp nhận các lần đọc dữ liệu cũ (stale reads), bạn vẫn cần phải biết về tình trạng sức khỏe của quá trình replication. Nếu nó bị tụt lại đáng kể, nó nên cảnh báo bạn để bạn có thể điều tra nguyên nhân (ví dụ: một vấn đề trong mạng hoặc một node đang bị quá tải).

Đối với leader-based replication, cơ sở dữ liệu thường cung cấp các metrics cho replication lag, thứ mà bạn có thể đưa vào một hệ thống giám sát. Điều này khả thi vì các lệnh ghi được áp dụng cho leader và các follower theo cùng một thứ tự, và mỗi node đều có một vị trí trong replication log (số lượng lệnh ghi mà nó đã áp dụng cục bộ). Bằng cách lấy vị trí hiện tại của một follower trừ đi vị trí hiện tại của leader, bạn có thể đo lường mức độ replication lag.

Tuy nhiên, trong các hệ thống với leaderless replication, không có thứ tự cố định để các lệnh ghi được áp dụng, điều này khiến việc giám sát trở nên khó khăn hơn. Số lượng các hint mà một replica lưu trữ để handoff có thể là một thước đo cho sức khỏe hệ thống, nhưng rất khó để diễn giải nó một cách hữu ích [55]. Eventual consistency là một sự đảm bảo mơ hồ một cách có chủ đích, nhưng để có thể vận hành được, việc định lượng được sự "eventual" là rất quan trọng.

Hiệu năng của Single-Leader so với Leaderless Replication

Một hệ thống replication dựa trên single leader có thể cung cấp các đảm bảo về strong consistency mà một hệ thống leaderless khó có thể hoặc không thể đạt được. Tuy nhiên, như chúng ta đã thấy trong mục “Problems with Replication Lag”, các lệnh đọc trong một hệ thống leader-based replication cũng có thể trả về các giá trị cũ nếu bạn thực hiện chúng trên một follower được cập nhật bất đồng bộ.

Đọc từ leader đảm bảo các phản hồi được cập nhật nhất, nhưng nó gặp phải các vấn đề về hiệu năng:

- Read throughput bị giới hạn bởi khả năng xử lý các yêu cầu của leader (ngược lại với read scaling, vốn phân phối các lệnh đọc trên các replica được cập nhật bất đồng bộ mà có thể trả về các giá trị cũ).

- Nếu leader gặp lỗi, bạn phải đợi cho đến khi lỗi được phát hiện và quá trình failover hoàn tất trước khi có thể tiếp tục xử lý các yêu cầu. Ngay cả khi quá trình failover diễn ra rất nhanh, người dùng cũng sẽ nhận thấy điều đó do thời gian phản hồi tăng lên tạm thời; nếu failover mất nhiều thời gian, hệ thống sẽ không khả dụng trong suốt khoảng thời gian đó.
- Hệ thống rất nhạy cảm với các vấn đề hiệu năng trên leader. Nếu leader phản hồi chậm (ví dụ: do quá tải hoặc tranh chấp tài nguyên), thời gian phản hồi tăng lên sẽ ngay lập tức ảnh hưởng đến người dùng.

Một lợi thế lớn của kiến trúc leaderless là nó có khả năng chống chịu tốt hơn trước các vấn đề như vậy. Vì không có failover, và các yêu cầu dù sao cũng được gửi đến nhiều replica một cách song song, nên việc một replica trở nên chậm hoặc không khả dụng sẽ có rất ít tác động đến thời gian phản hồi; client chỉ đơn giản là sử dụng các phản hồi từ những replica khác có tốc độ phản hồi nhanh hơn. Việc sử dụng các phản hồi nhanh nhất được gọi là *request hedging*, và nó có thể giảm đáng kể tail latency [56].

Về cốt lõi, khả năng chống chịu của một hệ thống leaderless đến từ việc nó không phân biệt giữa trường hợp bình thường và trường hợp xảy ra lỗi. Điều này đặc biệt hữu ích khi xử lý các *gray failures* (lỗi xám), trong đó một node không bị sập hoàn toàn nhưng đang chạy trong trạng thái suy giảm khiến việc xử lý các yêu cầu trở nên chậm một cách bất thường [57], hoặc khi một node đơn giản là bị quá tải (ví dụ: nếu một node đã ngoại tuyến trong một khoảng thời gian, việc phục hồi thông qua hinted handoff có thể gây ra rất nhiều tải bổ sung). Một hệ thống dựa trên leader phải quyết định xem tình huống đó có đủ tệ để cần thực hiện một cuộc failover hay không (việc này có thể tự gây ra thêm sự gián đoạn), trong khi ở một hệ thống leaderless, câu hỏi đó thậm chí còn không nảy sinh.

Mặc dù vậy, các hệ thống leaderless cũng có thể gặp phải các vấn đề về hiệu năng:

- Mặc dù hệ thống không cần thực hiện failover, một replica vẫn cần phát hiện khi một replica khác không khả dụng để nó có thể lưu trữ các gợi ý (hints) về các thao tác ghi mà replica không khả dụng đó đã bỏ lỡ. Khi replica không khả dụng quay trở lại, quá trình handoff cần gửi các gợi ý đó cho nó. Điều này tạo thêm tải cho các replica vào thời điểm mà hệ thống vốn đã đang chịu áp lực [55].
- Bạn càng có nhiều replica, kích thước của các quorum càng lớn và bạn càng phải chờ đợi nhiều phản hồi hơn trước khi một yêu cầu có thể hoàn tất. Ngay cả khi bạn chỉ chờ r hoặc w replica nhanh nhất phản hồi, và ngay cả khi bạn thực hiện các yêu cầu song song, một giá trị r hoặc w lớn hơn sẽ làm tăng khả năng bạn chạm phải một replica chậm, làm tăng tổng thời gian phản hồi (xem “Sử dụng các chỉ số thời gian phản hồi”). Trong thực tế, các quorum hiếm khi vượt quá bốn trên bảy node hoặc năm trên chín node.

- Một sự gián đoạn mạng quy mô lớn làm ngắt kết nối client khỏi một số lượng lớn các replica có thể khiến việc hình thành một quorum trở nên bất khả thi. Một số cơ sở dữ liệu leaderless cung cấp một tùy chọn cấu hình cho phép bất kỳ replica nào có thể kết nối được đều chấp nhận các thao tác ghi, ngay cả khi nó không phải là một trong những replica thông thường cho key đó (Riak và Dynamo gọi đây là *sloppy quorum* [45]; Cassandra và ScyllaDB gọi là *consistency level ANY*). Không có gì đảm bảo rằng các thao tác đọc tiếp theo sẽ thấy được giá trị đã ghi, nhưng tùy thuộc vào ứng dụng, nó vẫn có thể tốt hơn là để thao tác ghi bị thất bại.

Multi-leader replication có thể cung cấp khả năng chống chịu đối với các gián đoạn mạng thậm chí còn lớn hơn so với leaderless replication, vì các thao tác đọc và ghi chỉ yêu cầu giao tiếp với một leader duy nhất, vốn có thể được đặt cùng vị trí với client. Tuy nhiên, vì một thao tác ghi trên một leader được lan truyền bất đồng bộ đến các leader khác, các thao tác đọc có thể bị lỗi thời một cách tùy ý. Quorum reads và writes cung cấp một sự đánh đổi (trade-off): khả năng chịu lỗi tốt và xác suất cao về việc đọc được dữ liệu cập nhật nhất.

Vận hành Đa vùng (Multi-Region Operation)

Trước đó chúng ta đã thảo luận về cross-region replication như một trường hợp sử dụng cho multi-leader replication (xem “Multi-Leader Replication”). Leaderless replication cũng phù hợp cho vận hành đa vùng, vì nó được thiết kế để chịu đựng được các thao tác ghi đồng thời gây xung đột, các gián đoạn mạng và các đợt tăng đột biến về latency.

Trong Cassandra và ScyllaDB, một client muốn thực hiện một thao tác ghi đa vùng trước tiên sẽ chọn một node trong vùng cục bộ của nó, được gọi là *coordinator node*, và gửi thao tác ghi của nó đến node đó. Coordinator node sẽ chuyển tiếp thao tác ghi tới tất cả các replica trong vùng của chính nó và tới một replica ở mỗi vùng khác, sau đó replica này sẽ chuyển tiếp tới các replica khác trong vùng đó. Sự tối ưu hóa này giúp tránh việc phải thực hiện yêu cầu cross-region nhiều lần.

Bạn có thể chọn từ nhiều mức độ consistency khác nhau để quyết định số lượng phản hồi cần thiết để một yêu cầu được coi là thành công. Ví dụ, bạn có thể yêu cầu một quorum trên tất cả các replica ở mọi region, một quorum riêng biệt trong mỗi region, hoặc chỉ một quorum trong region cục bộ của client. Một local quorum giúp tránh việc phải chờ đợi các yêu cầu chậm chạp tới các region khác, nhưng nó cũng có khả năng cao sẽ trả về các kết quả cũ (stale results).

Riak giữ cho mọi giao tiếp giữa các client và các database node nằm cục bộ trong một region, vì vậy n mô tả số lượng replica trong một region. Việc replication xuyên region giữa các database cluster diễn ra bất đồng bộ ở chế độ chạy ngầm, theo một phong cách tương tự như multi-leader replication.

Phát hiện các ghi đồng thời (concurrent writes)

Tương tự như multi-leader replication, các database leaderless cho phép các ghi đồng thời vào cùng một key, dẫn đến các xung đột cần được giải quyết. Những xung đột như vậy có thể được phát hiện ngay khi các thao tác ghi diễn ra, nhưng không phải lúc nào cũng vậy: chúng cũng có thể được phát hiện muộn hơn, trong quá trình read repair, hinted handoff, hoặc anti-entropy.

Vấn đề là các sự kiện có thể đến theo các thứ tự khác nhau tại các node khác nhau, do độ trễ mạng biến thiên và các lỗi cục bộ (partial failures). Ví dụ, Hình 6-14 cho thấy hai client, A và B, đồng thời thực hiện ghi vào một key *X* trong một datastore gồm ba node:

- Node 1 nhận được thao tác ghi từ A, nhưng không bao giờ nhận được thao tác ghi từ B do một sự cố tạm thời (transient outage).
- Node 2 đầu tiên nhận được thao tác ghi từ A, sau đó là thao tác ghi từ B.
- Node 3 đầu tiên nhận được thao tác ghi từ B, sau đó là thao tác ghi từ A.

Nếu mỗi node chỉ đơn giản ghi đè giá trị của một key bất cứ khi nào nó nhận được một yêu cầu ghi từ một client, các node sẽ trở nên mất nhất quán (inconsistent) vĩnh viễn, như được thể hiện bởi yêu cầu *get* cuối cùng trong Hình 6-14: node 2 nghĩ rằng giá trị cuối cùng của *X* là B, trong khi các node khác nghĩ rằng giá trị đó là A.

Để đạt được eventual consistency, các replica nên hội tụ về cùng một giá trị. Để làm điều này, chúng ta có thể sử dụng bất kỳ cơ chế giải quyết xung đột nào mà ta đã thảo luận trước đó trong mục “Dealing with Conflicting Writes”, chẳng hạn như LWW (được sử dụng bởi Cassandra và ScyllaDB), giải quyết thủ công (manual resolution), hoặc CRDTs (được sử dụng bởi Riak).

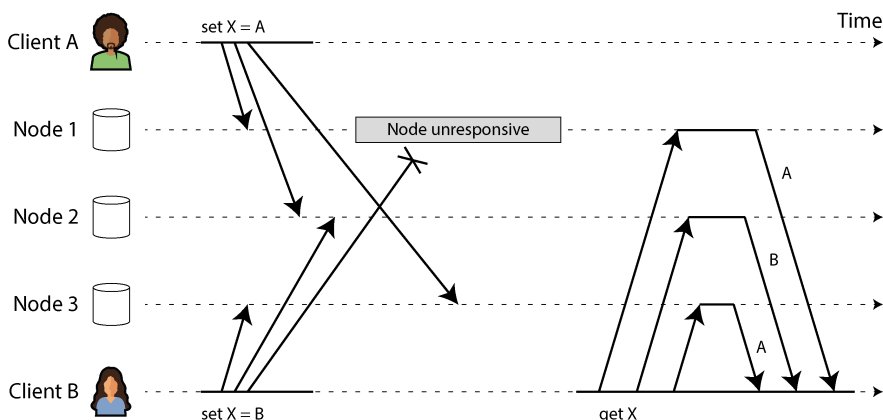


Figure 6-14. Các thao tác ghi đồng thời trong một datastore kiểu Dynamo: không có thứ tự xác định rõ ràng

LWW rất dễ triển khai. Mỗi lần ghi được gắn với một timestamp, và một giá trị có timestamp cao hơn sẽ luôn ghi đè lên giá trị có timestamp thấp hơn. Tuy nhiên, một timestamp không cho bạn biết liệu hai giá trị có thực sự xung đột hay không (tức là chúng được ghi đồng thời) hay không (chúng được ghi nối tiếp nhau). Nếu bạn muốn giải quyết các xung đột một cách rõ ràng, hệ thống cần chú trọng hơn vào việc phát hiện các lần ghi đồng thời.

Quan hệ happens-before và tính đồng thời

Làm thế nào để chúng ta quyết định liệu hai thao tác có đồng thời hay không? Để xây dựng trực giác, hãy cùng xem xét một số ví dụ:

- Trong Hình 6-8, hai lần ghi không đồng thời: thao tác insert của A *happens before* thao tác increment của B, bởi vì giá trị được B tăng lên chính là giá trị đã được A chèn vào. Nói cách khác, thao tác của B được xây dựng dựa trên thao tác của A, vì vậy thao tác của B phải xảy ra sau. Chúng ta cũng nói rằng B *causally dependent* (phụ thuộc nhân quả) vào A.
- Mặt khác, hai lần ghi trong Hình 6-14 là đồng thời: khi mỗi client bắt đầu thao tác, nó không biết rằng một client khác cũng đang thực hiện thao tác trên cùng một key. Do đó, không có sự phụ thuộc nhân quả nào giữa các thao tác.

Một thao tác A *happens before* một thao tác B khác nếu B biết về A, hoặc phụ thuộc vào A, hoặc xây dựng dựa trên A theo một cách nào đó. Việc một thao tác xảy ra trước một thao tác khác là chìa khóa để định nghĩa tính đồng thời nghĩa là gì. Trên thực tế, chúng ta có thể đơn giản nói rằng hai thao tác là đồng thời nếu không có thao tác nào xảy ra trước thao tác kia [58].

Vì vậy, bất cứ khi nào bạn có hai thao tác A và B, sẽ có ba khả năng: hoặc A xảy ra trước B, hoặc B xảy ra trước A, hoặc A và B đồng thời. Điều chúng ta cần là một

thuật toán để cho biết liệu hai thao tác có đồng thời hay không. Nếu một thao tác xảy ra trước một thao tác khác, thao tác xảy ra sau nên ghi đè lên thao tác trước đó, nhưng nếu các thao tác là đồng thời, chúng ta sẽ có một xung đột cần được giải quyết.

Tính đồng thời, Thời gian và Thuyết tương đối

Có vẻ như hai thao tác nên được gọi là đồng thời nếu chúng xảy ra "cùng một lúc"—nhưng trên thực tế, việc chúng có thực sự chồng lấn về mặt thời gian hay không không quan trọng. Do các vấn đề với đồng hồ trong các hệ thống phân tán, thực tế rất khó để biết liệu hai sự việc có xảy ra chính xác cùng một lúc hay không—một vấn đề mà chúng ta sẽ thảo luận chi tiết hơn trong Chương 9.

Để định nghĩa tính đồng thời, thời gian chính xác không quan trọng. Chúng ta đơn giản gọi hai thao tác là đồng thời nếu cả hai đều không biết về nhau, bất kể thời gian vật lý mà chúng xảy ra là khi nào. Mọi người đôi khi tạo ra một mối liên hệ giữa nguyên tắc này và thuyết tương đối đặc biệt trong vật lý [58], vốn đưa ra ý tưởng rằng thông tin không thể truyền nhanh hơn tốc độ ánh sáng. Do đó, hai sự kiện xảy ra cách nhau một khoảng cách nhất định không thể ảnh hưởng đến nhau nếu khoảng thời gian giữa các sự kiện ngắn hơn thời gian ánh sáng cần để di chuyển quãng đường giữa chúng.

Trong các hệ thống máy tính, hai thao tác có thể đồng thời ngay cả khi về nguyên tắc, tốc độ ánh sáng đã cho phép một thao tác ảnh hưởng đến thao tác kia. Ví dụ, nếu mạng bị chậm hoặc bị gián đoạn tại thời điểm đó, hai thao tác có thể xảy ra cách nhau một khoảng thời gian mà vẫn là đồng thời, bởi vì các vấn đề về mạng đã ngăn cản một thao tác có thể biết về thao tác kia.

Nắm bắt quan hệ happens-before

Hãy cùng xem xét một thuật toán xác định liệu hai thao tác là đồng thời hay một thao tác xảy ra trước thao tác kia. Để giữ mọi thứ đơn giản, hãy bắt đầu với một cơ sở dữ liệu chỉ có một replica. Sau khi chúng ta đã tìm ra cách thực hiện điều này trên một replica duy nhất, chúng ta có thể tổng quát hóa cách tiếp cận cho một cơ sở dữ liệu leaderless với nhiều replica. Thuật toán hoạt động như sau:

- Server duy trì một số phiên bản cho mỗi key, tăng số phiên bản mỗi khi key đó được ghi, và lưu trữ số phiên bản mới cùng với giá trị được ghi.
- Khi một client đọc một key, server sẽ trả về tất cả các siblings—tất cả các giá trị chưa bị ghi đè—cũng như số phiên bản mới nhất. Một client phải đọc một key trước khi thực hiện ghi.

- Khi một client ghi một key, nó phải bao gồm số phiên bản từ lần đọc trước đó, và nó phải hợp nhất tất cả các giá trị mà nó đã nhận được trong lần đọc trước (ví dụ: sử dụng một CRDT với đầu vào từ người dùng). Phản hồi từ một yêu cầu ghi cũng trả về tất cả các siblings, điều này cho phép chúng ta thực hiện chuỗi nhiều lần ghi (như trong ví dụ về giỏ hàng đã thảo luận trong mục “Dealing with Conflicting Writes”).
- Khi server nhận được một lệnh ghi với một số phiên bản cụ thể, nó có thể ghi đè tất cả các giá trị có số phiên bản đó hoặc thấp hơn (vì nó biết rằng chúng đã được hợp nhất vào giá trị mới), nhưng nó phải giữ lại tất cả các giá trị có số phiên bản cao hơn (vì những giá trị đó diễn ra đồng thời với lệnh ghi đang đến).

Lưu ý rằng server có thể xác định liệu hai thao tác có diễn ra đồng thời hay không bằng cách nhìn vào các số phiên bản. Server không cần phải giải thích chính giá trị đó, vì vậy giá trị có thể là bất kỳ cấu trúc dữ liệu nào.

Khi một lệnh ghi bao gồm số phiên bản từ lần đọc trước đó, điều đó cho chúng ta biết lệnh ghi này dựa trên trạng thái trước đó nào. Nếu bạn thực hiện một lệnh ghi mà không bao gồm số phiên bản, nó sẽ diễn ra đồng thời với tất cả các lệnh ghi khác, vì vậy nó sẽ không ghi đè bất cứ thứ gì—nó sẽ chỉ được trả về như một trong các giá trị trong các lần đọc tiếp theo. Hình 6-15 cho thấy thuật toán này đang hoạt động.

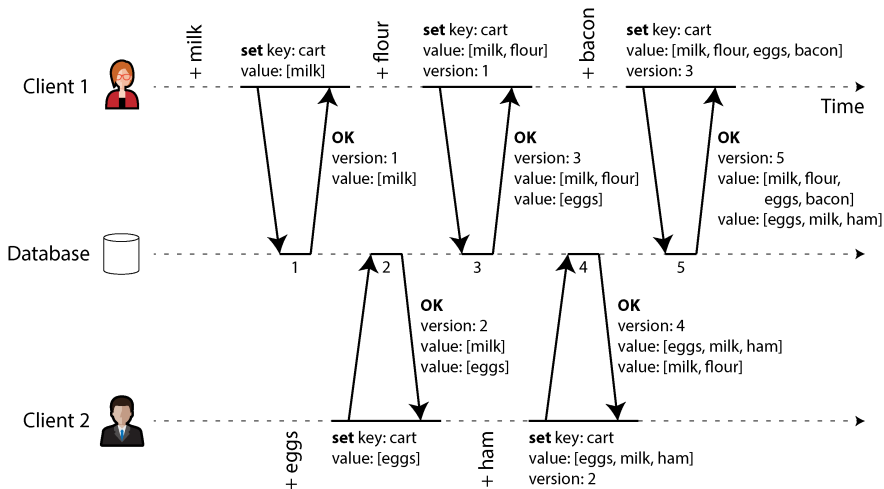


Figure 6-15. Ghi lại các phụ thuộc nhân quả (causal dependencies) giữa hai client đang cùng chỉnh sửa một giỏ hàng

Trong ví dụ này, hai client đang đồng thời thêm các mục vào cùng một giỏ hàng. (Nếu ví dụ này có vẻ quá ngớ ngẩn, hãy tưởng tượng thay vào đó là hai kiểm soát viên không lưu đang đồng thời thêm các máy bay vào khu vực mà chúng đang theo

đối.) Ban đầu, giỏ hàng đang trống. Giữa chúng, các client thực hiện năm lần ghi vào database:

1. Client 1 thêm milk vào giỏ hàng. Đây là lần ghi đầu tiên cho key đó, vì vậy server lưu trữ thành công và gán cho nó version 1. Server cũng phản hồi lại giá trị đó cho client, cùng với số version.
2. Client 2 thêm eggs vào giỏ hàng mà không biết rằng client 1 đã đồng thời thêm milk (client 2 nghĩ rằng eggs của nó là mục duy nhất trong giỏ hàng). Server gán version 2 cho lần ghi này và lưu trữ eggs và milk thành hai giá trị riêng biệt (siblings). Sau đó, nó trả về *cả hai* giá trị cho client, cùng với số version là 2.
3. Client 1, do không biết về lần ghi của client 2, muốn thêm flour vào giỏ hàng, sau đó nó giả định rằng nội dung giỏ hàng sẽ là [milk, flour]. Nó gửi giá trị này tới server, cùng với số version mà server đã cấp cho nó trước đó (1). Server có thể nhận biết từ số version rằng lần ghi [milk, flour] thay thế giá trị trước đó là [milk] nhưng lại đồng thời với [eggs]. Do đó, server gán version 3 cho [milk, flour], ghi đè lên giá trị version 1 là [milk], nhưng vẫn giữ lại giá trị version 2 là [eggs] và trả về cả hai giá trị còn lại cho client.
4. Trong khi đó, client 2 muốn thêm ham vào giỏ hàng mà không biết rằng client 1 vừa mới thêm flour. Client 2 đã nhận được hai giá trị [milk] và [eggs] từ server trong phản hồi trước đó, vì vậy giờ đây client thực hiện merge các giá trị đó và thêm ham để tạo thành một giá trị mới, [eggs, milk, ham]. Nó gửi giá trị đó tới server, cùng với số version trước đó (2). Server phát hiện ra rằng version 2 ghi đè lên [eggs] nhưng lại đồng thời với [milk, flour], vì vậy hai giá trị còn lại là [milk, flour] với version 3 và [eggs, milk, ham] với version 4.
5. Cuối cùng, client 1 muốn thêm bacon. Trước đó nó đã nhận được [milk, flour] và [eggs] từ server ở version 3, vì vậy nó merge chúng lại, thêm bacon, và gửi giá trị cuối cùng là [milk, flour, eggs, bacon] tới server, cùng với số version 3. Lần này ghi đè lên [milk, flour] (lưu ý rằng [eggs] đã bị ghi đè ở bước trước) nhưng lại đồng thời với [eggs, milk, ham], nên server giữ lại cả hai giá trị đồng thời đó.

Dataflow giữa các thao tác trong Hình 6-15 được minh họa bằng đồ thị trong Hình 6-16. Các mũi tên chỉ ra thao tác nào *xảy ra trước* thao tác nào khác, theo nghĩa là thao tác sau *đã biết về* hoặc *phụ thuộc vào* thao tác trước đó. Trong ví dụ này, các client không bao giờ cập nhật hoàn toàn dữ liệu trên server, vì luôn có một thao tác khác đang diễn ra đồng thời. Nhưng các version cũ của giá trị cuối cùng cũng sẽ bị ghi đè, và không có lần ghi nào bị mất.

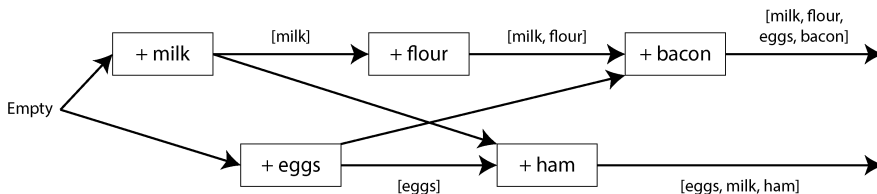


Figure 6-16. Đồ thị về các phụ thuộc nhân quả trong Hình 6-15

Version vectors

Ví dụ trong Hình 6-15 chỉ sử dụng một replica duy nhất. Thuật toán sẽ thay đổi như thế nào khi có nhiều replica nhưng không có leader?

Hình 6-15 sử dụng một số phiên bản duy nhất để ghi lại các phụ thuộc giữa các thao tác, nhưng điều đó là không đủ khi có nhiều replica chấp nhận các thao tác ghi đồng thời. Thay vào đó, chúng ta cần sử dụng một số phiên bản *cho mỗi replica* cũng như cho mỗi key. Mỗi replica sẽ tăng số phiên bản của chính nó khi xử lý một thao tác ghi, đồng thời theo dõi các số phiên bản mà nó đã thấy từ mỗi replica khác. Thông tin này cho biết giá trị nào cần được ghi đè và giá trị nào cần được giữ lại dưới dạng các sibling.

Việc thu thập các số phiên bản từ tất cả các replica được gọi là một *version vector* [59]. Có một vài biến thể của ý tưởng này đang được sử dụng, nhưng thú vị nhất có lẽ là *dotted version vector* [60, 61], thứ được dùng trong Riak 2.0 [62, 63]. Chúng ta sẽ không đi sâu vào chi tiết, nhưng cách nó hoạt động khá giống với những gì chúng ta đã thấy trong ví dụ về giỏ hàng.

Giống như các số phiên bản trong Hình 6-15, các version vector được gửi từ các database replica đến các client khi các giá trị được đọc, và chúng cần được gửi ngược lại database khi một giá trị được ghi sau đó. (Riak mã hóa version vector thành một chuỗi mà nó gọi là *causal context*.) Version vector cho phép database phân biệt giữa việc ghi đè (overwrite) và các lần ghi đồng thời (concurrent writes).

Version vector cũng đảm bảo rằng việc đọc từ một replica và sau đó ghi ngược lại vào một replica khác là an toàn. Làm như vậy có thể dẫn đến việc tạo ra các siblings, nhưng không có dữ liệu nào bị mất miễn là các siblings được merge chính xác.

Version vectors và vector clocks

Một *version vector* đôi khi cũng được gọi là một *vector clock*, mặc dù chúng không hoàn toàn giống nhau. Sự khác biệt là rất tinh tế [61, 64, 65]. Hãy xem các tài liệu tham khảo để biết chi tiết; tóm lại, khi so sánh trạng thái của các replica, version vectors là cấu trúc dữ liệu phù hợp để sử dụng.

Tóm tắt

Trong chương này, chúng ta đã xem xét vấn đề replication. Replication có thể phục vụ một số mục đích:

High availability (tính sẵn sàng cao)

Giữ cho hệ thống tiếp tục chạy, ngay cả khi một máy (hoặc nhiều máy, một zone, hoặc thậm chí cả một region) bị sập

Durability (độ bền dữ liệu)

Đảm bảo bạn không bị mất dữ liệu, ngay cả khi toàn bộ một máy (hoặc thậm chí cả một region) bị lỗi vĩnh viễn

Disconnected operation (vận hành ngoại tuyến)

Cho phép một ứng dụng tiếp tục hoạt động bất chấp sự gián đoạn mạng

Latency (độ trễ)

Đặt dữ liệu gần về mặt địa lý với người dùng để người dùng có thể tương tác với nó nhanh hơn

Scalability (khả năng mở rộng)

Có khả năng xử lý khối lượng đọc cao hơn mức một máy đơn lẻ có thể xử lý, bằng cách thực hiện các lệnh đọc trên các replica

Mặc dù khái niệm này rất đơn giản—giữ một bản sao của cùng một dữ liệu trên nhiều máy—nhưng replication hóa ra lại là một vấn đề cực kỳ hóc búa. Nó đòi hỏi phải suy nghĩ kỹ về tính đồng thời (concurrency), tất cả những thứ có thể xảy ra sai sót, và cách xử lý các hệ quả của những lỗi đó. Ít nhất, chúng ta cần xử lý các node không khả dụng và sự gián đoạn mạng (và đó thậm chí còn chưa tính đến các loại lỗi thâm hiểm hơn, chẳng hạn như lỗi dữ liệu âm thầm do lỗi phần mềm hoặc lỗi phần cứng).

Chúng ta đã thảo luận về ba cách tiếp cận chính cho replication:

Single-leader replication (nhân bản dữ liệu đơn leader)

Các client gửi tất cả các lệnh ghi đến một node duy nhất (leader), node này sẽ gửi một luồng các sự kiện thay đổi dữ liệu đến các replica khác (follower). Các lệnh đọc có thể được thực hiện trên bất kỳ replica nào, nhưng các lệnh đọc từ follower có thể bị cũ (stale).

Multi-leader replication (nhân bản đa leader)

Các client gửi mỗi lệnh ghi đến một trong số nhiều node leader, bất kỳ node nào trong số đó cũng có thể chấp nhận lệnh ghi. Các leader gửi các luồng sự kiện thay đổi dữ liệu cho nhau và cho bất kỳ node follower nào.

Leaderless replication

Các client gửi mỗi lệnh ghi đến nhiều node và đọc từ nhiều node song song để phát hiện và sửa các node có dữ liệu cũ.

Mỗi cách tiếp cận đều có ưu và nhược điểm. Single-leader replication phổ biến vì nó khá dễ hiểu và cung cấp tính nhất quán mạnh (strong consistency). Multi-leader và leaderless replication có thể mạnh mẽ hơn khi có sự hiện diện của các node bị lỗi, gián đoạn mạng và các đợt tăng vọt latency, nhưng phải đánh đổi bằng việc yêu cầu giải quyết xung đột (conflict resolution) và cung cấp các đảm bảo về tính nhất quán yếu hơn.

Replication có thể là synchronous hoặc asynchronous, điều này có ảnh hưởng sâu sắc đến hành vi của hệ thống khi có lỗi xảy ra. Mặc dù asynchronous replication có thể nhanh khi hệ thống đang chạy trơn tru, nhưng việc tìm hiểu điều gì sẽ xảy ra khi replication lag tăng lên và các máy chủ bị lỗi là rất quan trọng. Nếu một leader bị lỗi và bạn thăng cấp một follower được cập nhật bất đồng bộ (asynchronously) lên làm leader mới, các dữ liệu vừa được commit gần đây có thể bị mất.

Chúng ta đã xem xét một số hiệu ứng lậ có thể gây ra bởi replication lag, và chúng ta cũng đã thảo luận về một vài mô hình consistency giúp ích cho việc quyết định một ứng dụng nên hoạt động như thế nào trong điều kiện replication lag:

Read-after-write consistency (độ tin cậy read-after-write)

Người dùng luôn phải thấy được dữ liệu mà chính họ đã gửi lên.

Monotonic reads

Sau khi người dùng đã thấy dữ liệu tại một thời điểm, họ không nên thấy dữ liệu từ một thời điểm sớm hơn sau đó.

Consistent prefix reads

Người dùng nên thấy dữ liệu ở một trạng thái hợp lý về mặt causality—ví dụ, thấy một câu hỏi và câu trả lời của nó theo đúng thứ tự.

Cuối cùng, chúng ta đã thảo luận cách thức mà multi-leader và leaderless replication đảm bảo rằng tất cả các replica cuối cùng sẽ hội tụ về một trạng thái nhất quán: bằng cách sử dụng một version vector hoặc thuật toán tương tự để phát hiện các write nào là concurrent, và bằng cách sử dụng một thuật toán conflict resolution như CRDT để hợp nhất các giá trị được viết đồng thời. LWW và conflict resolution thủ công cũng là những phương án khả thi.

Chương này đã giả định rằng mỗi replica lưu trữ một bản sao đầy đủ của toàn bộ cơ sở dữ liệu, điều này là không thực tế đối với các dataset lớn. Trong chương tiếp theo, chúng ta sẽ xem xét về *sharding*, cho phép mỗi máy chỉ lưu trữ một tập hợp con của dữ liệu.

Footnotes

References

[1] B. G. Lindsay, P. G. Selinger, C. Galtieri, J. N. Gray, R. A. Lorie, T. G. Price, F. Putzolu, I. L. Traiger, and B. W. Wade. “Notes on Distributed Databases.” IBM Research, Research Report RJ2571(33471), July 1979. Archived at perma.cc/EPZ3-MHDD

- [2] Kenny Gryp. “MySQL Terminology Updates.” *dev.mysql.com*, July 2020. Archived at perma.cc/S62G-6RJ2
- [3] Oracle Corporation. “Oracle (Active) Data Guard 19c: Real-Time Data Protection and Availability.” White Paper, *oracle.com*, March 2019. Archived at perma.cc/P5ST-RPKE
- [4] Microsoft. “What Is an Always On Availability Group?” *learn.microsoft.com*, September 2024. Archived at perma.cc/ABH6-3MXF
- [5] Mostafa Elhemali, Niall Gallagher, Nicholas Gordon, Joseph Idziorek, Richard Krog, Colin Lazier, Erben Mo, Akhilesh Mritunjai, Somu Perianayagam, Tim Rath, Swami Sivasubramanian, James Christopher Sorenson III, Sroaj Sosothikul, Doug Terry, and Akshat Vig. “Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service.” At *USENIX Annual Technical Conference (ATC)*, July 2022.
- [6] Rebecca Taft, Irfan Sharif, Andrei Matei, Nathan VanBenschoten, Jordan Lewis, Tobias Grieger, Kai Niemi, Andy Woods, Anne Birzin, Raphael Poss, Paul Bardea, Amruta Ranade, Ben Darnell, Bram Gruneir, Justin Jaffray, Lucy Zhang, and Peter Mattis. “CockroachDB: The Resilient Geo-Distributed SQL Database.” At *ACM SIGMOD International Conference on Management of Data (SIGMOD)*, June 2020. [doi:10.1145/3318464.3386134](https://doi.org/10.1145/3318464.3386134)
- [7] Dongxu Huang, Qi Liu, Qiu Cui, Zhuhe Fang, Xiaoyu Ma, Fei Xu, Li Shen, Liu Tang, Yuxing Zhou, Menglong Huang, Wan Wei, Cong Liu, Jian Zhang, Jianjun Li, Xuelian Wu, Lingyu Song, Ruoxi Sun, Shuaipeng Yu, Lei Zhao, Nicholas Cameron, Liquan Pei, and Xin Tang. “TiDB: A Raft-Based HTAP Database.” *Proceedings of the VLDB Endowment*, volume 13, issue 12, pages 3072–3084, August 2020. [doi:10.14778/3415478.3415535](https://doi.org/10.14778/3415478.3415535)
- [8] Mallory Knodel and Niels ten Oever. “Terminology, Power, and Inclusive Language in Internet-Drafts and RFCs.” *IETF Internet-Draft*, August 2023. Archived at perma.cc/5ZY9-725E
- [9] Buck Hodges. “Postmortem: VSTS 4 September 2018.” *devblogs.microsoft.com*, September 2018. Archived at perma.cc/ZF5R-DYZS
- [10] Gunnar Morling. “Leader Election with S3 Conditional Writes.” *www.morling.dev*, August 2024. Archived at perma.cc/7V2N-J78Y
- [11] Vignesh Chandramohan, Rohan Desai, and Chris Riccomini. “SlateDB Manifest Design.” *github.com*, May 2024. Archived at perma.cc/8EUY-P32Z
- [12] Stas Kelvich. “Why Does Neon Use Paxos Instead of Raft, and What’s the Difference?” *neon.tech*, August 2022. Archived at perma.cc/SEZ4-2GXU
- [13] Dimitri Fontaine. “An Introduction to the pg_auto_failover Project.” *tapoueh.org*, November 2021. Archived at perma.cc/3WH5-6BAF
- [14] Jesse Newland. “GitHub Availability This Week.” *github.blog*, September 2012. Archived at perma.cc/3YRF-FTFJ
- [15] Mark Imbriaco. “Downtime Last Saturday.” *github.blog*, December 2012. Archived at perma.cc/M7X5-E8SQ
- [16] John Hugg. “All In’ with Determinism for Performance and Testing in Distributed Systems.” At *Strange Loop*, September 2015.
- [17] Hironobu Suzuki. “The Internals of PostgreSQL.” *interdb.jp*, 2017. Archived at archive.org
- [18] Amit Kapila. “WAL Internals of PostgreSQL.” At *PostgreSQL Conference (PGCon)*, May 2012. Archived at perma.cc/6225-3SUX
- [19] Amit Kapila. “Evolution of Logical Replication.” *amitkapila16.blogspot.com*, September 2023. Archived at perma.cc/F9VX-JLER

- [20] Aru Petchimuthu. “Upgrade Your Amazon RDS for PostgreSQL or Amazon Aurora PostgreSQL Database, Part 2: Using the pglogical Extension.” *aws.amazon.com*, August 2021. Archived at perma.cc/RXT8-FS2T
- [21] Yogeshwer Sharma, Philippe Ajoux, Petchean Ang, David Callies, Abhishek Choudhary, Laurent Demailly, Thomas Fersch, Liat Atsmon Guz, Andrzej Kotulski, Sachin Kulkarni, Sanjeev Kumar, Harry Li, Jun Li, Evgeniy Makeev, Kowshik Prakasam, Robbert van Renesse, Sabyasachi Roy, Pratyush Seth, Yee Jiun Song, Benjamin Wester, Kaushik Veeraraghavan, and Peter Xie. “Wormhole: Reliable Pub-Sub to Support Geo-Replicated Internet Services.” At *12th USENIX Symposium on Networked Systems Design and Implementation* (NSDI), May 2015.
- [22] Douglas B. Terry. “Replicated Data Consistency Explained Through Baseball.” Microsoft Research, Technical Report MSR-TR-2011-137, October 2011. Archived at perma.cc/F4KZ-AR38
- [23] Douglas B. Terry, Alan J. Demers, Karin Petersen, Mike J. Spreitzer, Marvin M. Theher, and Brent B. Welch. “Session Guarantees for Weakly Consistent Replicated Data.” At *3rd International Conference on Parallel and Distributed Information Systems* (PDIS), September 1994. doi:10.1109/PDIS.1994.331722
- [24] Werner Vogels. “Eventually Consistent.” *ACM Queue*, volume 6, issue 6, pages 14–19, October 2008. doi:10.1145/1466443.1466448
- [25] Simon Willison. Reply to: “My thoughts about Fly.io (so far) and other newish technology I’m getting into”. *news.ycombinator.com*, May 2022.
- [26] Nithin Tharakan. “Scaling Bitbucket’s Database.” *atlassian.com*, October 2020. Archived at perma.cc/JAB7-9FGX
- [27] Terry Pratchett. *Reaper Man: A Discworld Novel*. Victor Gollancz, 1991. ISBN: 9780575049796
- [28] Peter Bailis, Alan Fekete, Michael J. Franklin, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. “Coordination Avoidance in Database Systems.” *Proceedings of the VLDB Endowment*, volume 8, issue 3, pages 185–196, November 2014. doi:10.14778/2735508.2735509
- [29] Yaser Raja and Peter Celentano. “PostgreSQL Bi-Directional Replication Using pglogical.” *aws.amazon.com*, January 2022. Archived at perma.cc/BUQ2-5QWN
- [30] Robert Hodges. “If You *Must* Deploy Multi-Master Replication, Read This First.” *scale-out-blog.blogspot.com*, April 2012. Archived at perma.cc/C2JN-F6Y8
- [31] Lars Hofhansl. “HBASE-7709: Infinite Loop Possible in Master/Master Replication.” *issues.apache.org*, January 2013. Archived at perma.cc/24G2-8NLC
- [32] John Day-Richter. “What’s Different About the New Google Docs: Making Collaboration Fast.” *drive.googleblog.com*, September 2010. Archived at perma.cc/5TL8-TSJ2
- [33] Evan Wallace. “How Figma’s Multiplayer Technology Works.” *figma.com*, October 2019. Archived at perma.cc/L49H-LY4D
- [34] Tuomas Artman. “Scaling the Linear Sync Engine.” *linear.app*, June 2023.
- [35] Amr Saafan. “Why Sync Engines Might Be the Future of Web Applications.” *nilebits.com*, September 2024. Archived at perma.cc/5N73-5M3V
- [36] Isaac Hagoel. “Are Sync Engines the Future of Web Applications?” *dev.to*, July 2024. Archived at perma.cc/R9HF-BKKL
- [37] Sujay Jayakar. “A Map of Sync.” *stack.convex.dev*, October 2024. Archived at perma.cc/82R3-H42A
- [38] Alex Feyerke. “Designing Offline-First Web Apps.” *alistapart.com*, December 2013. Archived at perma.cc/WH7R-S2DS

- [39] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. “Local-First Software: You Own Your Data, in Spite of the Cloud.” At *ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software* (Onward!), October 2019. [doi:10.1145/3359591.3359737](https://doi.org/10.1145/3359591.3359737)
- [40] Martin Kleppmann. “The Past, Present, and Future of Local-First.” At *Local-First Conference*, May 2024.
- [41] Conrad Hofmeyr. “API Calling Is to Sync Engines as jQuery Is to React.” *powersync.com*, November 2024. Archived at perma.cc/2FP9-7WJJ
- [42] Peter van Hardenberg and Martin Kleppmann. “PushPin: Towards Production-Quality Peer-to-Peer Collaboration.” At *7th Workshop on Principles and Practice of Consistency for Distributed Data* (PaPoC), April 2020. [doi:10.1145/3380787.3393683](https://doi.org/10.1145/3380787.3393683)
- [43] Leonard Kawell, Jr., Steven Beckhardt, Timothy Halvorsen, Raymond Ozzie, and Irene Greif. “Replicated Document Management in a Group Communication System.” At *ACM Conference on Computer-Supported Cooperative Work* (CSCW), September 1988. [doi:10.1145/62266.1024798](https://doi.org/10.1145/62266.1024798)
- [44] Ricky Pusch. “Explaining How Fighting Games Use Delay-Based and Rollback Netcode.” *words.infil.net* and *arstechnica.com*, October 2019. Archived at perma.cc/DE7W-RDJ8
- [45] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Voss, and Werner Vogels. “Dynamo: Amazon’s Highly Available Key-Value Store.” At *21st ACM Symposium on Operating Systems Principles* (SOSP), October 2007. [doi:10.1145/1323293.1294281](https://doi.org/10.1145/1323293.1294281)
- [46] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. “Conflict-Free Replicated Data Types.” At *13th International Symposium on Stabilization, Safety, and Security of Distributed Systems* (SSS), October 2011. [doi:10.1007/978-3-642-24550-3_29](https://doi.org/10.1007/978-3-642-24550-3_29)
- [47] Chengzheng Sun and Clarence Ellis. “Operational Transformation in Real-Time Group Editors: Issues, Algorithms, and Achievements.” At *ACM Conference on Computer Supported Cooperative Work* (CSCW), November 1998. [doi:10.1145/289444.289469](https://doi.org/10.1145/289444.289469)
- [48] Joseph Gentle and Martin Kleppmann. “Collaborative Text Editing with Eg-walker: Better, Faster, Smaller.” At *20th European Conference on Computer Systems* (EuroSys), March 2025. [doi:10.1145/3689031.3696076](https://doi.org/10.1145/3689031.3696076)
- [49] Dharma Shukla. “Azure Cosmos DB: Pushing the Frontier of Globally Distributed Databases.” *azure.microsoft.com*, September 2018. Archived at perma.cc/UT3B-HH6R
- [50] David K. Gifford. “Weighted Voting for Replicated Data.” At *7th ACM Symposium on Operating Systems Principles* (SOSP), December 1979. [doi:10.1145/800215.806583](https://doi.org/10.1145/800215.806583)
- [51] Marc Brooker. “Dynamo, DynamoDB, and Aurora DSQL.” *brooker.co.za*, August 2025. Archived at perma.cc/XG3C-ALDQ
- [52] Heidi Howard, Dahlia Malkhi, and Alexander Spiegelman. “Flexible Paxos: Quorum Intersection Revisited.” At *20th International Conference on Principles of Distributed Systems* (OPODIS), December 2016. [doi:10.4230/LIPIcs.OPODIS.2016.25](https://doi.org/10.4230/LIPIcs.OPODIS.2016.25)
- [53] Joseph Blomstedt. “Bringing Consistency to Riak.” At *RICON West*, October 2012. Archived at archive.org
- [54] Peter Bailis, Shivaram Venkataraman, Michael J. Franklin, Joseph M. Hellerstein, and Ion Stoica. “Quantifying Eventual Consistency with PBS.” *The VLDB Journal*, volume 23, issue 2, pages 279–302, April 2014. [doi:10.1007/s00778-013-0330-1](https://doi.org/10.1007/s00778-013-0330-1)

- [55] Colin Breck. “Shared-Nothing Architectures for Server Replication and Synchronization.” *blog.colinbreck.com*, December 2019. Archived at perma.cc/48P3-J6CJ
- [56] Jeffrey Dean and Luiz André Barroso. “The Tail at Scale.” *Communications of the ACM*, volume 56, issue 2, pages 74–80, February 2013. [doi:10.1145/2408776.2408794](https://doi.org/10.1145/2408776.2408794)
- [57] Peng Huang, Chuanxiong Guo, Lidong Zhou, Jacob R. Lorch, Yingnong Dang, Murali Chintalapati, and Randolph Yao. “Gray Failure: The Achilles’ Heel of Cloud-Scale Systems.” At *16th Workshop on Hot Topics in Operating Systems (HotOS)*, May 2017. [doi:10.1145/3102980.3103005](https://doi.org/10.1145/3102980.3103005)
- [58] Leslie Lamport. “Time, Clocks, and the Ordering of Events in a Distributed System.” *Communications of the ACM*, volume 21, issue 7, pages 558–565, July 1978. [doi:10.1145/359545.359563](https://doi.org/10.1145/359545.359563)
- [59] D. Stott Parker Jr., Gerald J. Popek, Gerard Rudisin, Allen Stoughton, Bruce J. Walker, Evelyn Walton, Johanna M. Chow, David Edwards, Stephen Kiser, and Charles Kline. “Detection of Mutual Inconsistency in Distributed Systems.” *IEEE Transactions on Software Engineering*, volume SE-9, issue 3, pages 240–247, May 1983. [doi:10.1109/TSE.1983.236733](https://doi.org/10.1109/TSE.1983.236733)
- [60] Nuno Preguiça, Carlos Baquero, Paulo Sérgio Almeida, Victor Fonte, and Ricardo Gonçalves. “Dotted Version Vectors: Logical Clocks for Optimistic Replication.” *arXiv:1011.5808*, November 2010.
- [61] Giridhar Manepalli. “Clocks and Causality—Ordering Events in Distributed Systems.” *exhypothesi.com*, November 2022. Archived at perma.cc/8REU-KVLQ
- [62] Sean Cribbs. “A Brief History of Time in Riak.” At *RICON*, October 2014. Archived at perma.cc/7U9P-6JFX
- [63] Russell Brown. “Vector Clocks Revisited Part 2: Dotted Version Vectors.” *riak.com*, November 2015. Archived at perma.cc/96QP-W98R
- [64] Carlos Baquero. “Version Vectors Are Not Vector Clocks.” *baslab.wordpress.com*, July 2011. Archived at perma.cc/7PNU-4AMG
- [65] Reinhard Schwarz and Friedemann Mattern. “Detecting Causal Relationships in Distributed Computations: In Search of the Holy Grail.” *Distributed Computing*, volume 7, issue 3, pages 149–174, March 1994. [doi:10.1007/BF02277859](https://doi.org/10.1007/BF02277859)

Sharding

Rõ ràng, chúng ta phải thoát khỏi sự tuần tự và không giới hạn các máy tính. Chúng ta phải đưa ra các định nghĩa, cung cấp các ưu tiên và mô tả về dữ liệu. Chúng ta phải xác định các mối quan hệ, chứ không phải các quy trình.

Grace Murray Hopper, *Management and the Computer of the Future* (1962)

Một cơ sở dữ liệu phân tán thường phân phối dữ liệu trên các node theo hai cách:

- Nó lưu trữ một bản sao của cùng một dữ liệu trên nhiều node. Đây là *replication* (nhân bản dữ liệu), điều mà chúng ta đã thảo luận ở Chương 6.
- Nếu có quá nhiều dữ liệu hoặc throughput ghi cao đến mức một node duy nhất không thể xử lý được, nó sẽ chia dữ liệu thành các *shard* hoặc *partition* (phần mảnh dữ liệu) nhỏ hơn, và lưu trữ các shard khác nhau trên các node khác nhau. Chúng ta sẽ thảo luận về sharding trong chương này.

Thông thường, các shard được định nghĩa sao cho mỗi phần dữ liệu (mỗi bản ghi, hàng, hoặc document) thuộc về chính xác một shard. Có nhiều cách khác nhau để đạt được điều này, điều mà chúng ta sẽ thảo luận chuyên sâu trong chương này. Về cơ bản, mỗi shard là một cơ sở dữ liệu nhỏ của riêng nó, mặc dù một số hệ thống cơ sở dữ liệu hỗ trợ các thao tác tác động lên nhiều shard cùng một lúc.

Sharding thường được kết hợp với replication, sao cho các bản sao của mỗi shard được lưu trữ trên nhiều node. Điều này có nghĩa là mặc dù mỗi bản ghi thuộc về chính xác một shard, nó vẫn có thể được lưu trữ trên vài node khác nhau để đảm bảo fault tolerance (khả năng chịu lỗi).

Một node có thể lưu trữ nhiều hơn một shard. Nếu sử dụng mô hình single-leader replication, sự kết hợp giữa sharding và replication có thể trông giống như Hình 7-1, ví dụ. Leader của mỗi shard được chỉ định cho một node, và các follower của nó được chỉ định cho các node khác. Mỗi node có thể là leader cho một số shard và là follower cho các shard khác, nhưng mỗi shard vẫn chỉ có duy nhất một leader.

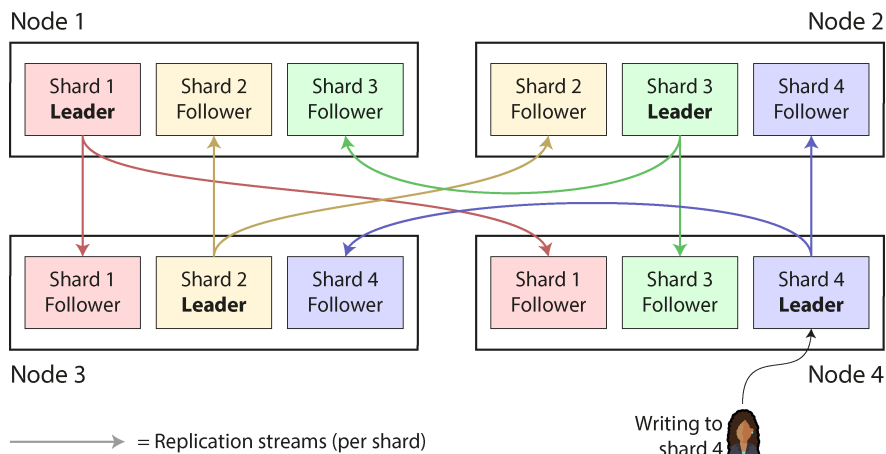


Figure 7-1. Kết hợp replication và sharding: mỗi node đóng vai trò là leader cho một số shard và là follower cho các shard khác

Sharding và Partitioning

Những gì chúng ta gọi là một *shard* trong chương này có nhiều tên gọi khác nhau tùy thuộc vào phần mềm bạn đang sử dụng. Nó được gọi là một *partition* trong Kafka, một *range* trong CockroachDB, một *region* trong HBase và TiDB, một *vBucket* trong Couchbase, một *vnode* trong Riak, một *token-range* trong Cassandra, và một *tablet* trong Bigtable, YugabyteDB, và ScyllaDB, chỉ là một vài ví dụ.

Một số cơ sở dữ liệu coi *partition* và *shard* là hai khái niệm riêng biệt. Ví dụ, trong PostgreSQL, *partitioning* là một cách để chia một bảng lớn thành nhiều tệp được lưu trữ trên cùng một máy (điều này mang lại nhiều lợi thế, chẳng hạn như giúp việc xóa toàn bộ một *partition* trở nên rất nhanh), trong khi *sharding* chia tách một dataset trên nhiều máy khác nhau [1, 2]. Trong nhiều hệ thống khác, *partitioning* chỉ là một cách gọi khác của *sharding*.

Trong khi *partitioning* khá dễ hiểu, thì thuật ngữ *sharding* có lẽ gây ngạc nhiên. Theo một giả thuyết, thuật ngữ này bắt nguồn từ trò chơi nhập vai trực tuyến *Ultima Online*, trong đó một viên pha lê ma thuật bị vỡ thành nhiều mảnh, và mỗi mảnh (*shard*) khúc xạ một bản sao của thế giới trò chơi [3]. Do đó, thuật ngữ *shard* dần có nghĩa là một trong số các máy chủ trò chơi song song, và sau đó nó được chuyển sang lĩnh vực cơ sở dữ liệu. Một giả thuyết khác cho rằng ban đầu nó là tên viết tắt của *System for Highly Available Replicated Data*—được cho là một cơ sở dữ liệu từ những năm 1980 mà chi tiết về nó đã bị thất lạc trong lịch sử.

Nhân tiện, *partitioning* không liên quan gì đến *network partitions* (netsplits), một loại lỗi trong mạng giữa các node. Chúng ta sẽ thảo luận về các lỗi như vậy trong Chương 9.

Mọi thứ về replication của các cơ sở dữ liệu trong Chương 6 đều áp dụng tương tự cho replication của các *shard*. Vì việc lựa chọn *sharding scheme* hầu như độc lập với việc lựa chọn replication scheme, chúng ta sẽ bỏ qua replication trong chương này để cho đơn giản.

Ưu và nhược điểm của Sharding

Lý do chính để *sharding* một cơ sở dữ liệu là *khả năng mở rộng* (*scalability*). *Sharding* là một giải pháp nếu khối lượng dữ liệu hoặc throughput ghi đã trở nên quá lớn để một node duy nhất có thể xử lý, vì nó cho phép bạn phân tán dữ liệu và các thao tác ghi đó trên nhiều node. (Nếu throughput đọc là vấn đề, bạn không nhất thiết cần *sharding*—bạn có thể sử dụng *read scaling*, như đã thảo luận trong Chương 6.)

Trên thực tế, sharding là một trong những công cụ chính mà chúng ta có để đạt được *khả năng mở rộng ngang (horizontal scaling)* (một kiến trúc *scale-out*), như đã thảo luận trong mục “Kiến trúc Shared-Memory, Shared-Disk, và Shared-Nothing”—nghĩa là, cho phép một hệ thống tăng cường khả năng của nó không phải bằng cách chuyển sang một máy lớn hơn, mà bằng cách thêm nhiều máy (nhỏ hơn) vào. Nếu bạn có thể chia nhỏ khối lượng công việc sao cho mỗi shard xử lý một phần tương đương nhau, bạn có thể gán các shard đó vào các máy khác nhau để xử lý dữ liệu và các truy vấn của chúng một cách song song.

Trong khi replication hữu ích ở cả quy mô nhỏ và lớn vì nó cho phép fault tolerance và hoạt động offline, thì sharding là một giải pháp nặng nề chủ yếu chỉ phù hợp ở quy mô lớn. Nếu khối lượng dữ liệu và throughput ghi của bạn ở mức mà một máy duy nhất có thể xử lý (và một máy ngày nay có thể làm được rất nhiều việc!), thì thường tốt hơn là nên tránh sharding và trung thành với một cơ sở dữ liệu single-shard.

Lý do cho khuyến nghị này là sharding làm tăng độ phức tạp. Bạn thường phải quyết định bản ghi nào sẽ nằm trong shard nào bằng cách chọn một *partition key*; tất cả các bản ghi có cùng partition key sẽ được đặt trong cùng một shard [4]. Lựa chọn này rất quan trọng vì việc truy cập một bản ghi sẽ nhanh nếu bạn biết nó nằm ở shard nào, nhưng nếu không, bạn sẽ phải thực hiện một cuộc tìm kiếm không hiệu quả trên tất cả các shard. Sharding scheme cũng rất khó để thay đổi.

Sharding thường hoạt động tốt với dữ liệu key-value, nơi bạn có thể dễ dàng sharding theo key, nhưng sẽ khó hơn với dữ liệu quan hệ (relational data), nơi bạn có thể muốn tìm kiếm bằng một secondary index hoặc join các bản ghi có thể bị phân tán trên các shard khác nhau. Chúng ta sẽ thảo luận kỹ hơn về vấn đề này trong mục “Sharding và Secondary Indexes”.

Một vấn đề khác với sharding là một thao tác ghi có thể cần cập nhật các bản ghi liên quan trong nhiều shard. Trong khi các transaction trên một node đơn lẻ là khá phổ biến, việc đảm bảo tính nhất quán giữa nhiều shard đòi hỏi một *distributed transaction* (giao dịch phân tán). Như chúng ta sẽ thấy trong Chương 8, distributed transaction có sẵn trong một số cơ sở dữ liệu, nhưng chúng thường chậm hơn nhiều so với các transaction trên một node đơn lẻ và có thể trở thành nút thắt cổ chai cho toàn bộ hệ thống.

Một số hệ thống sử dụng sharding ngay cả trên một máy đơn lẻ, thường chạy một tiến trình đơn luồng (single-threaded) cho mỗi lõi CPU để tận dụng tính song song trong CPU hoặc để khai thác kiến trúc *nonuniform memory access* (NUMA - truy cập bộ nhớ không đồng nhất), trong đó một số bank bộ nhớ nằm gần một CPU này hơn so với các CPU khác [5]. Ví dụ, Redis, VoltDB và FoundationDB sử dụng một tiến trình cho mỗi lõi và dựa vào sharding để phân tán tải qua các lõi CPU trong cùng một máy [6].

Sharding for Multitenancy

Các sản phẩm Software as a service (SaaS) và dịch vụ đám mây thường là *multitenant* (đa người thuê), trong đó mỗi tenant là một khách hàng. Nhiều người dùng có thể có tài khoản đăng nhập trên cùng một tenant, nhưng mỗi tenant có một dataset độc lập và tách biệt với dataset của các tenant khác. Ví dụ, trong một dịch vụ email marketing, mỗi doanh nghiệp đăng ký thường là một tenant riêng biệt, vì các lượt đăng ký bản tin, dữ liệu gửi thư, v.v. của một doanh nghiệp là tách biệt với các doanh nghiệp khác.

Đôi khi sharding được sử dụng để triển khai các hệ thống multitenant. Có thể mỗi tenant được cấp một shard riêng biệt, hoặc nhiều tenant nhỏ có thể được nhóm lại với nhau thành một shard lớn hơn. Các shard này có thể là các cơ sở dữ liệu tách biệt về mặt vật lý (điều mà chúng ta đã đề cập trước đó trong mục “Embedded Storage Engines”) hoặc là các phần có thể quản lý riêng biệt của một cơ sở dữ liệu logic lớn hơn [7]. Sử dụng sharding cho multitenancy mang lại một số lợi thế sau:

Cô lập tài nguyên

Nếu một tenant thực hiện một thao tác tốn kém về tính toán, khả năng cao là hiệu năng của các tenant khác sẽ ít bị ảnh hưởng hơn nếu chúng đang chạy trên các shard khác nhau.

Cô lập quyền truy cập

Nếu có lỗi trong logic kiểm soát truy cập của bạn, khả năng bạn vô tình cấp quyền cho một tenant truy cập vào dữ liệu của tenant khác sẽ thấp hơn nếu dataset của các tenant đó được lưu trữ tách biệt về mặt vật lý.

Kiến trúc dựa trên cell (Cell-based architecture)

Bạn có thể áp dụng sharding không chỉ ở cấp độ lưu trữ dữ liệu, mà còn cho các service đang chạy mã nguồn ứng dụng của bạn. Trong một *cell-based architecture*, các service và lưu trữ cho một tập hợp tenant cụ thể được nhóm thành một *cell* độc lập, và các cell khác nhau được thiết lập sao cho chúng có thể chạy gần như độc lập với nhau. Cách tiếp cận này cung cấp khả năng *fault isolation* (cô lập lỗi): một lỗi trong một cell sẽ chỉ giới hạn trong cell đó, và các tenant trong các cell khác sẽ không bị ảnh hưởng [8].

Sao lưu và khôi phục theo từng tenant

Việc sao lưu shard của từng tenant một cách riêng biệt giúp có thể khôi phục trạng thái của một tenant từ bản sao lưu mà không ảnh hưởng đến các tenant khác, điều này có thể hữu ích nếu tenant đó vô tình xóa hoặc ghi đè dữ liệu quan trọng [9].

Tuân thủ quy định

Các quy định về quyền riêng tư dữ liệu như GDPR và CCPA cho phép các cá nhân có quyền truy cập và yêu cầu xóa thông tin cá nhân mà các doanh nghiệp

lưu trữ về họ. Nếu dữ liệu của mỗi người được lưu trữ trong một shard riêng biệt, điều này sẽ chuyển hóa thành các thao tác xuất và xóa dữ liệu đơn giản trên shard của họ [10].

Lưu trữ dữ liệu (Data residence)

Nếu dữ liệu của một tenant cụ thể cần được lưu trữ tại một khu vực pháp lý nhất định để tuân thủ luật lưu trữ dữ liệu, một cơ sở dữ liệu có khả năng nhận biết khu vực (region-aware) có thể cho phép bạn chỉ định shard của tenant đó vào một khu vực cụ thể.

Triển khai schema dần dần

Các đợt di chuyển schema (đã thảo luận trước đó trong mục “Schema flexibility in the document model”) có thể được triển khai dần dần, từng tenant một. Điều này giúp giảm thiểu rủi ro, vì bạn có thể phát hiện các vấn đề trước khi chúng ảnh hưởng đến tất cả các tenant, nhưng việc thực hiện dưới dạng transaction có thể sẽ khó khăn [11].

Các thách thức chính xoay quanh việc sử dụng sharding cho multitenancy như sau:

- Nó giả định rằng mỗi tenant riêng lẻ đều đủ nhỏ để nằm gọn trên một node duy nhất. Nếu không phải như vậy, và bạn có một tenant quá lớn so với một máy, bạn sẽ cần thực hiện thêm việc sharding bên trong tenant đó, điều này đưa chúng ta quay trở lại chủ đề sharding để đạt được khả năng mở rộng [12].
- Nếu bạn có nhiều tenant nhỏ, việc tạo một shard riêng biệt cho mỗi tenant có thể gây ra quá nhiều overhead. Bạn có thể nhóm nhiều tenant nhỏ lại với nhau thành một shard lớn hơn, nhưng khi đó bạn sẽ gặp vấn đề về cách di chuyển các tenant từ shard này sang shard khác khi chúng phát triển.
- Nếu bạn cần hỗ trợ các tính năng kết nối dữ liệu giữa nhiều tenant, chúng sẽ trở nên khó triển khai hơn nếu bạn cần join dữ liệu qua nhiều shard.

Sharding dữ liệu Key-Value

Giả sử bạn có một lượng lớn dữ liệu và muốn sharding nó. Làm thế nào để bạn quyết định bản ghi nào sẽ được lưu trữ trên những node nào?

Mục tiêu của sharding là phân tán dữ liệu và tải truy vấn một cách đồng đều giữa các node. Nếu mỗi node nhận một phần chia công bằng, thì—về lý thuyết—10 node sẽ có thể xử lý lượng dữ liệu gấp 10 lần và throughput đọc/ghi gấp 10 lần so với một node duy nhất (nếu bỏ qua replication). Nếu bạn thêm hoặc xóa một node, bạn cũng muốn có thể *rebalance* tải trọng để nó được phân phối đều giữa số lượng node mới.

Nếu việc sharding không công bằng, khiến một số shard có nhiều dữ liệu hoặc truy vấn hơn những shard khác, chúng ta gọi đó là *skewed* (lệch). Sự hiện diện của skew khiến việc sharding kém hiệu quả hơn nhiều. Trong trường hợp cực đoan, toàn bộ

tải trọng có thể dồn vào một shard duy nhất, khiến 9 trên 10 node ở trạng thái rảnh rỗi, và nút thắt cổ chai của bạn chính là node bận rộn duy nhất đó. Một shard có tải trọng cao bất thường được gọi là *hot shard* hoặc *hot spot*. Nếu một key có tải trọng đặc biệt cao (ví dụ: một người nổi tiếng trong mạng xã hội), chúng ta gọi đó là một *hot key*.

Để chia dataset thành các shard, chúng ta cần một thuật toán nhận đầu vào là partition key của một bản ghi và cho biết shard nào chứa bản ghi đó. Trong một key-value store, partition key thường là key hoặc phần đầu của key. Trong một relational model, partition key có thể là một cột của một bảng (không nhất thiết phải là primary key của nó). Thuật toán đó cần phải có khả năng hỗ trợ rebalancing để giảm bớt các hot spot.

Sharding theo Key Range

Một cách sharding là gán một phạm vi liên tục của các partition key (từ giá trị tối thiểu đến tối đa) cho mỗi shard, giống như các tập của một bộ bách khoa toàn thư bằng giấy, như được minh họa trong Hình 7-2. Trong ví dụ này, partition key của một mục là tiêu đề của nó. Nếu bạn muốn tra cứu mục cho một tiêu đề cụ thể, bạn có thể dễ dàng xác định shard nào chứa mục đó, và từ đó chọn đúng cuốn sách trên kệ, bằng cách tìm tập có phạm vi key chứa tiêu đề mà bạn đang tìm kiếm.

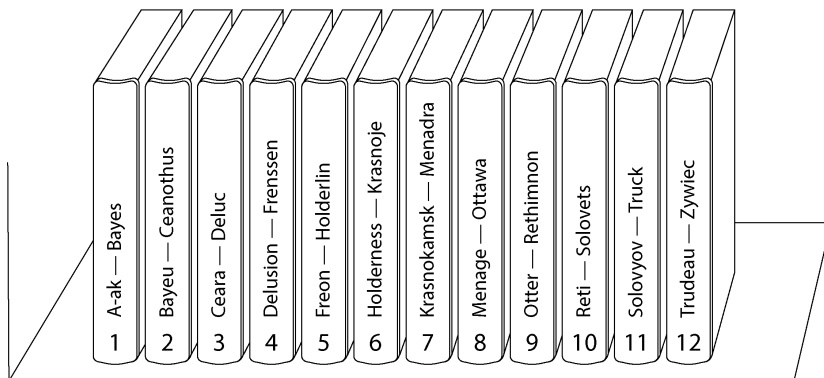


Figure 7-2. Một bộ bách khoa toàn thư in được sharding theo phạm vi key.

Các phạm vi của key không nhất thiết phải cách đều nhau, bởi vì dữ liệu của bạn có thể không được phân phối đều. Ví dụ, trong Hình 7-2, volume 1 chứa các từ bắt đầu bằng chữ *A* và *B*, nhưng volume 12 lại chứa các từ bắt đầu bằng *T*, *U*, *V*, *W*, *X*, *Y*, và *Z*. Nếu chỉ đơn giản phân bổ mỗi volume cho hai chữ cái trong bảng chữ cái thì sẽ dẫn đến tình trạng một số volume lớn hơn nhiều so với các volume khác. Để phân phối dữ liệu đều nhau, các ranh giới shard cần phải thích ứng với dữ liệu.

Các ranh giới shard có thể được chọn thủ công bởi một quản trị viên, hoặc cơ sở dữ liệu có thể chọn chúng một cách tự động. Ví dụ, Vitess (một lớp sharding cho

MySQL) sử dụng sharding theo phạm vi key thủ công; biến thể tự động được sử dụng bởi Bigtable và giải pháp mã nguồn mở tương đương là HBase, tùy chọn sharding dựa trên phạm vi trong MongoDB, cũng như CockroachDB, RethinkDB, và FoundationDB [6]. YugabyteDB cung cấp cả việc chia tách tablet (tablet splitting) thủ công và tự động.

Trong mỗi shard, các key được lưu trữ theo thứ tự đã sắp xếp (ví dụ: trong một B-tree hoặc SSTables, như đã thảo luận ở Chương 4). Điều này có lợi thế là các thao tác quét phạm vi (range scan) trở nên dễ dàng, và bạn có thể coi key như một index được nối chuỗi để truy xuất nhiều bản ghi liên quan trong một truy vấn duy nhất (xem “Multidimensional and Full-Text Indexes”). Ví dụ, hãy xét một ứng dụng lưu trữ dữ liệu từ một mạng lưới các cảm biến, trong đó key là timestamp của phép đo. Các thao tác quét phạm vi rất hữu ích trong trường hợp này, vì chúng cho phép bạn dễ dàng truy xuất, chẳng hạn như, tất cả các giá trị đọc từ một tháng cụ thể.

Một nhược điểm của key-range sharding là bạn có thể dễ dàng gặp phải một hot shard nếu có rất nhiều thao tác ghi vào các key gần nhau. Ví dụ, nếu key là một timestamp, thì các shard sẽ tương ứng với các phạm vi thời gian—chẳng hạn, mỗi shard cho một tháng. Nếu bạn ghi dữ liệu từ các cảm biến vào cơ sở dữ liệu ngay khi các phép đo diễn ra, tất cả các thao tác ghi sẽ tập trung vào cùng một shard (shard của tháng hiện tại), do đó shard đó sẽ bị quá tải các thao tác ghi trong khi các shard khác lại ở trạng thái rảnh rỗi [13].

Để tránh vấn đề này trong cơ sở dữ liệu cảm biến, bạn cần sử dụng một thứ gì đó khác với timestamp làm thành phần đầu tiên của key. Ví dụ, bạn có thể thêm tiền tố cho mỗi timestamp bằng sensor ID để thứ tự của key sẽ ưu tiên theo sensor ID trước, sau đó mới đến timestamp. Giả sử bạn có nhiều cảm biến hoạt động cùng một lúc, tải ghi cuối cùng sẽ được phân phối đều hơn giữa các shard. Nhược điểm là khi bạn muốn truy xuất giá trị của nhiều cảm biến trong một phạm vi thời gian, giờ đây bạn cần thực hiện một truy vấn phạm vi riêng biệt cho từng cảm biến.

Rebalancing dữ liệu được sharding theo phạm vi key

Khi bạn mới thiết lập cơ sở dữ liệu, chưa có các phạm vi key nào để chia thành các shard. Một số cơ sở dữ liệu, chẳng hạn như HBase và MongoDB, cho phép bạn cấu hình một tập hợp các shard ban đầu trên một cơ sở dữ liệu trống, quá trình này được gọi là *pre-splitting*. Điều này đòi hỏi bạn đã có ý niệm về việc phân phối key sẽ trông như thế nào, để bạn có thể chọn các ranh giới phạm vi key phù hợp [14].

Sau đó, khi khối lượng dữ liệu và throughput ghi tăng lên, một hệ thống với key-range sharding sẽ phát triển bằng cách chia một shard hiện có thành hai hoặc nhiều shard nhỏ hơn, mỗi shard nắm giữ một phạm vi con liên tục của phạm vi key ban đầu. Các shard nhỏ hơn thu được sau đó có thể được phân phối trên nhiều node. Nếu một lượng lớn dữ liệu bị xóa, bạn cũng có thể cần hợp nhất (merge) nhiều shard

liền kề đã trở nên nhỏ thành một shard lớn hơn. Quá trình này tương tự như những gì xảy ra ở cấp cao nhất của một B-tree (xem “B-Trees”).

Với các cơ sở dữ liệu quản lý ranh giới shard một cách tự động, một thao tác chia shard (shard split) thường được kích hoạt khi shard đạt đến một kích thước đã cấu hình (ví dụ: trên HBase, mặc định là 10 GB) hoặc, trong một số hệ thống, khi throughput ghi liên tục cao hơn một ngưỡng nhất định. Do đó, một hot shard có thể bị chia tách ngay cả khi nó không lưu trữ nhiều dữ liệu, để tải ghi của nó có thể được phân phối đồng đều hơn.

Thật không may, số lượng shard sẽ thích ứng theo khối lượng dữ liệu. Nếu chỉ có một lượng nhỏ dữ liệu, một số lượng shard nhỏ là đủ, do đó chi phí overhead là thấp; nếu có một lượng dữ liệu khổng lồ, kích thước của mỗi shard riêng lẻ sẽ bị giới hạn ở một mức tối đa có thể cấu hình được [15].

Thật không may, việc chia tách một shard là một thao tác tốn kém, vì nó yêu cầu tất cả dữ liệu của shard đó phải được ghi lại vào các tệp mới, tương tự như quá trình compaction trong một log-structured storage engine. Một shard cần chia tách thường cũng là một shard đang chịu tải cao, và chi phí chia tách có thể làm trầm trọng thêm tải trọng đó, dẫn đến nguy cơ nó bị quá tải.

Sharding bằng Hash của Key

Key-range sharding sẽ hữu ích nếu bạn muốn các bản ghi có partition key gần nhau (nhưng khác nhau) được nhóm vào cùng một shard—ví dụ, điều này có thể xảy ra với các timestamp. Nếu bạn không quan tâm liệu các partition key có nằm gần nhau hay không (ví dụ, nếu chúng là tenant ID trong một ứng dụng multitenant), một cách tiếp cận phổ biến là thực hiện hash partition key trước khi ánh xạ nó vào một shard.

Một hàm hash tốt sẽ lấy dữ liệu bị skew và làm cho chúng được phân phối đồng nhất. Giả sử bạn có một hàm hash 32-bit nhận đầu vào là một chuỗi. Bất cứ khi nào bạn đưa cho nó một chuỗi mới, nó sẽ trả về một con số có vẻ ngẫu nhiên từ 0 đến $2^{32} - 1$. Ngay cả khi các chuỗi đầu vào rất giống nhau, các giá trị hash của chúng vẫn được phân phối đều trong phạm vi các con số đó (nhưng cùng một đầu vào luôn tạo ra cùng một đầu ra).

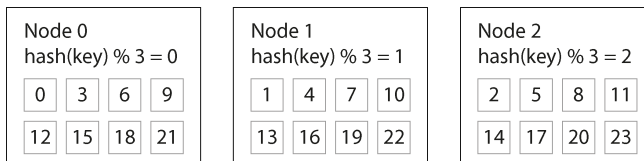
Vì mục đích sharding, hàm hash không cần phải mạnh về mặt mã hóa: ví dụ, MongoDB sử dụng MD5, trong khi Cassandra và ScyllaDB sử dụng Murmur3. Nhiều ngôn ngữ lập trình có sẵn các hàm hash đơn giản (vì chúng được sử dụng cho hash tables), nhưng chúng có thể không phù hợp để sharding: ví dụ, trong `Object.hashCode()` của Java và `Object#hash` của Ruby, cùng một key có thể có giá trị hash khác nhau trong các process khác nhau, khiến chúng không phù hợp để sharding [16].

Hash modulo số lượng node

Sau khi bạn đã hash key, làm thế nào để chọn shard nào để lưu trữ nó? Suy nghĩ đầu tiên của bạn có thể là lấy giá trị hash *modulo* cho số lượng node trong hệ thống (sử dụng toán tử % trong nhiều ngôn ngữ lập trình). Ví dụ, $\text{hash}(\text{key}) \% 10$ sẽ trả về một số từ 0 đến 9 (nếu chúng ta viết giá trị hash dưới dạng số thập phân, $\text{hash} \% 10$ sẽ là chữ số cuối cùng). Nếu chúng ta có 10 node, được đánh số từ 0 đến 9, thì đây có vẻ là một cách dễ dàng để gán mỗi key vào một node.

Vấn đề với cách tiếp cận $\text{mod } N$ là nếu số lượng node N thay đổi, hầu hết các key sẽ phải được di chuyển từ node này sang node khác. Hình 7-3 cho thấy điều gì xảy ra khi bạn có ba node và thêm node thứ tư. Trước khi rebalancing, node 0 lưu trữ các key có giá trị hash là 0, 3, 6, 9, v.v. Sau khi thêm node thứ tư, key có hash là 3 đã di chuyển sang node 3, key có hash là 6 đã di chuyển sang node 2, key có hash là 9 đã di chuyển sang node 1, và cứ tiếp tục như vậy.

Before rebalancing (3 nodes):



After rebalancing (4 nodes):

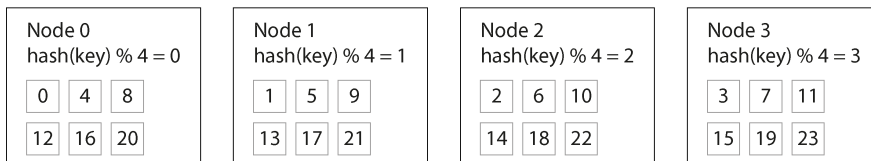


Figure 7-3. Gán các key cho các node bằng cách băm (hashing) key và lấy kết quả modulo cho số lượng node. Khi thay đổi số lượng node, nhiều key sẽ bị di chuyển từ node này sang node khác.

Hàm $\text{mod } N$ rất dễ tính toán, nhưng nó dẫn đến việc rebalancing cực kỳ kém hiệu quả vì có quá nhiều sự di chuyển dữ liệu không cần thiết của các bản ghi từ node này sang node khác. Chúng ta cần một phương pháp di chuyển càng ít dữ liệu càng tốt.

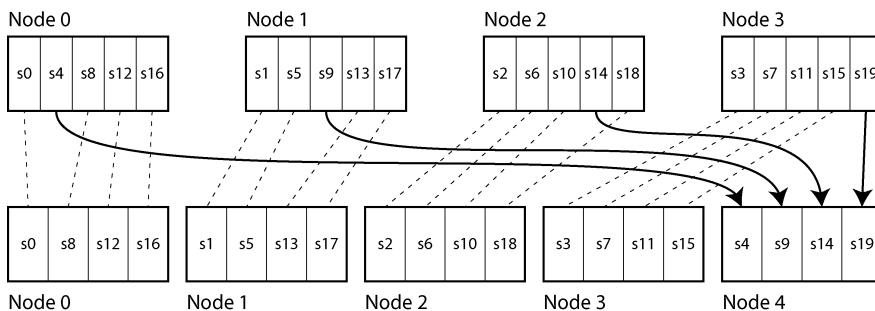
Số lượng shard cố định

Một giải pháp đơn giản nhưng được sử dụng rộng rãi là tạo ra nhiều shard hơn so với số lượng node hiện có và gán nhiều shard cho mỗi node. Ví dụ, một cơ sở dữ liệu chạy trên một cluster gồm 10 node có thể được chia thành 1.000 shard ngay từ đầu, sao cho mỗi node được gán 100 shard. Một key sau đó được lưu trữ trong shard số $\text{hash}(\text{key}) \% 1.000$, và hệ thống sẽ theo dõi riêng biệt xem shard nào được lưu trữ trên node nào.

Bây giờ, nếu một node được thêm vào cluster, hệ thống có thể gán lại một số shard từ các node hiện có sang node mới cho đến khi chúng được phân phối đều một lần nữa. Quá trình này được minh họa trong Hình 7-4. Nếu một node bị loại bỏ khỏi cluster, điều tương tự sẽ xảy ra theo chiều ngược lại.

Trong mô hình này, chỉ có toàn bộ các shard được di chuyển giữa các node, việc này rẻ hơn so với việc chia nhỏ các shard. Số lượng shard không thay đổi, và việc gán các key vào các shard cũng không đổi. Điều duy nhất thay đổi là việc gán các shard vào các node. Việc gán lại này không diễn ra ngay lập tức—mất một khoảng thời gian để chuyển một lượng lớn dữ liệu qua mạng—vì vậy việc gán shard cũ vẫn được sử dụng cho bất kỳ hoạt động đọc và ghi nào diễn ra trong khi quá trình chuyển dữ liệu đang tiến hành.

Before rebalancing (4 nodes in cluster)



After rebalancing (5 nodes in cluster)

Legend:

- Shard remains on the same node
- > Shard migrates to another node

Figure 7-4. Hình 7.x: Thêm một node mới vào một database cluster với nhiều shard trên mỗi node

Việc chọn số lượng shard có thể chia hết cho nhiều thừa số là điều phổ biến, để dataset có thể được chia đều qua các số lượng node khác nhau—ví dụ như không yêu cầu số lượng node phải là lũy thừa của 2 [4]. Bạn thậm chí có thể tính đến sự không đồng nhất về phần cứng trong cluster của mình: bằng cách gán nhiều shard hơn cho các node mạnh hơn, bạn có thể khiến các node đó đảm nhận phần tải lớn hơn.

Cách tiếp cận sharding này được sử dụng trong Citus (một sharding layer cho PostgreSQL), Riak, Elasticsearch, và Couchbase, cùng nhiều hệ thống khác. Nó hoạt động tốt miễn là bạn có một ước tính tốt về số lượng shard sẽ cần khi lần đầu tạo cơ sở dữ liệu. Sau đó, bạn có thể thêm hoặc xóa các node một cách dễ dàng, với giới hạn là bạn không thể có nhiều node hơn số lượng shard mà bạn có.

Nếu bạn thấy số lượng shard được cấu hình ban đầu là sai—ví dụ, nếu bạn đã đạt đến một quy mô mà bạn cần nhiều node hơn số lượng shard hiện có—thì một thao tác *resharding* tốn kém sẽ là bắt buộc. Nó cần phải chia nhỏ mỗi shard và ghi chúng ra các tệp mới, sử dụng rất nhiều dung lượng đĩa bổ sung trong quá trình này. Một số hệ thống không cho phép *resharding* trong khi đang thực hiện ghi vào cơ sở dữ liệu đồng thời, điều này khiến việc thay đổi số lượng shard mà không gây *downtime* trở nên khó khăn.

Việc chọn số lượng shard phù hợp sẽ rất khó khăn nếu tổng kích thước của *dataset* biến động mạnh (ví dụ, nếu nó bắt đầu nhỏ nhưng có thể lớn hơn nhiều theo thời gian). Vì mỗi shard chứa một phần cố định của tổng dữ liệu, kích thước của mỗi shard sẽ tăng tỉ lệ thuận với tổng lượng dữ liệu trong *cluster*. Nếu các shard quá lớn, việc *rebalancing* và phục hồi sau lỗi node sẽ trở nên tốn kém. Nhưng nếu các shard quá nhỏ, chúng sẽ gây ra quá nhiều *overhead*. Hiệu năng tốt nhất đạt được khi kích thước của các shard ở mức "vừa đủ", không quá lớn cũng không quá nhỏ, điều này có thể khó thực hiện nếu số lượng shard là cố định trong khi kích thước *dataset* thay đổi.

Sharding theo hash range

Nếu không thể dự đoán trước số lượng shard cần thiết, tốt hơn là nên sử dụng một lược đồ mà trong đó số lượng shard có thể thích ứng dễ dàng với khối lượng công việc. Lược đồ *key-range sharding* đã đề cập ở trên có đặc tính này, nhưng nó có rủi ro gặp *hot spot* khi có nhiều thao tác ghi vào các key gần nhau. Một giải pháp là kết hợp *key-range sharding* với một hàm hash để mỗi shard chứa một phạm vi các *hash values* thay vì một phạm vi các *keys*.

Hình 7-5 cho thấy một ví dụ sử dụng hàm hash 16-bit trả về một số từ 0 đến $2^{16} - 1$ (trong thực tế, hash thường là 32 bits hoặc nhiều hơn). Ngay cả khi các key đầu vào rất giống nhau (ví dụ: các timestamp liên tiếp), các hash của chúng vẫn được phân phối đồng nhất trong phạm vi đó. Sau đó, chúng ta có thể gán một phạm vi giá trị hash cho mỗi shard—ví dụ, các giá trị từ 0 đến 16.383 cho shard 0, các giá trị từ 16.384 đến 32.767 cho shard 1, và cứ tiếp tục như vậy.

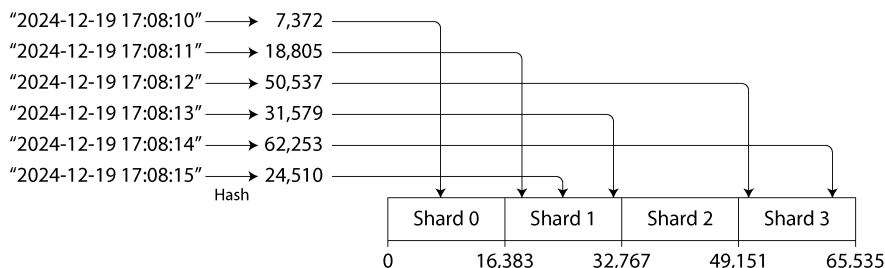


Figure 7-5. Hình 7.x: Gán một phạm vi liên tục các giá trị hash cho mỗi shard

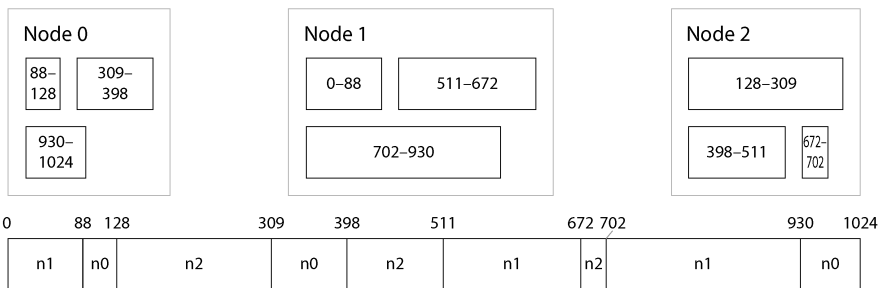
Tương tự như key-range sharding, trong hash-range sharding, một shard có thể được chia tách khi nó trở nên quá lớn hoặc bị tải quá nặng. Đây vẫn là một thao tác tốn kém, nhưng nó có thể diễn ra khi cần thiết, nhờ đó số lượng shard sẽ thích ứng với khối lượng dữ liệu thay vì bị cố định trước.

Nhược điểm so với key-range sharding là các range query trên partition key không hiệu quả, vì các key trong phạm vi đó giờ đây bị phân tán khắp tất cả các shard. Tuy nhiên, nếu các key bao gồm hai hoặc nhiều cột và partition key chỉ là cột đầu tiên trong số này, bạn vẫn có thể thực hiện các range query hiệu quả trên cột thứ hai và các cột sau đó. Miễn là tất cả các bản ghi trong range query đều có cùng partition key, chúng sẽ nằm trong cùng một shard.

Partitioning và Range Queries trong Data Warehouse

Các data warehouse như BigQuery, Snowflake và Delta Lake hỗ trợ một phương pháp lập chỉ mục tương tự, mặc dù thuật ngữ có sự khác biệt. Ví dụ, trong BigQuery, partition key quyết định bản ghi nằm ở partition nào, trong khi các "cluster columns" quyết định cách các bản ghi được sắp xếp trong partition đó. Snowflake gán các bản ghi vào các "micro-partitions" một cách tự động nhưng cho phép người dùng định nghĩa các cluster keys cho một bảng. Delta Lake hỗ trợ cả việc gán partition thủ công và tự động, đồng thời hỗ trợ các cluster keys. Việc clustering dữ liệu không chỉ cải thiện hiệu năng range scan, mà còn có thể cải thiện hiệu năng nén và lọc dữ liệu.

YugabyteDB và DynamoDB [17] sử dụng hash-range sharding, và đây cũng là một tùy chọn trong MongoDB. Cassandra và ScyllaDB sử dụng một biến thể của phương pháp này được minh họa trong Hình 7-6.



After adding Node 3 (with hash range boundaries 60, 276, and 551):

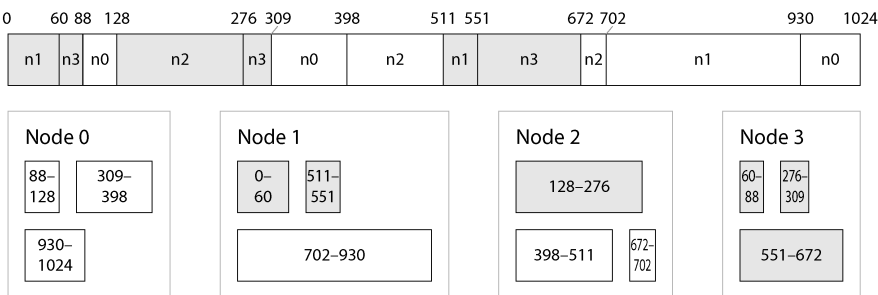


Figure 7-6. *Cassandra và ScyllaDB chia dải các giá trị hash có thể có (ở đây là 0–1024) thành các dải liên tiếp với các ranh giới ngẫu nhiên và gán nhiều dải cho mỗi node.*

Không gian của các giá trị hash được chia thành một số phạm vi tỷ lệ thuận với số lượng node (hình minh họa cho thấy 3 phạm vi cho mỗi node, nhưng con số thực tế mặc định trong Cassandra là 16 mỗi node, và 256 mỗi node trong ScyllaDB), với các ranh giới ngẫu nhiên giữa các phạm vi đó. Điều này có nghĩa là một số phạm vi sẽ lớn hơn những phạm vi khác, nhưng bằng cách có nhiều phạm vi cho mỗi node, sự mất cân bằng đó có xu hướng được san bằng [15].

Khi các node được thêm vào hoặc bị loại bỏ, các ranh giới phạm vi sẽ được điều chỉnh và các shard được chia tách hoặc hợp nhất tương ứng. Trong Hình 7-6, khi node 3 được thêm vào, node 1 chuyển một phần của hai phạm vi của nó sang node 3, và node 2 chuyển một phần của một phạm vi của nó sang node 3. Việc này có tác dụng giúp node mới nhận được một phần dữ liệu tương đối công bằng, mà không cần chuyển quá nhiều dữ liệu không cần thiết từ node này sang node khác.

Consistent hashing

Một thuật toán *consistent hashing* (băm nhất quán) là một hàm hash ánh xạ các key tới một số lượng shard xác định theo cách thỏa mãn hai tính chất:

- Số lượng key được ánh xạ tới mỗi shard xấp xỉ bằng nhau.
- Khi số lượng shard thay đổi, có càng ít key bị di chuyển từ shard này sang shard khác càng tốt.

Lưu ý rằng từ *consistent* ở đây không liên quan gì đến tính nhất quán của replica (xem Chương 6) hay tính nhất quán ACID (xem Chương 8), mà đúng hơn là mô tả xu hướng của một key sẽ ở lại trong cùng một shard nếu có thể.

Thuật toán sharding được sử dụng bởi Cassandra và ScyllaDB tương tự như định nghĩa ban đầu của consistent hashing [18], nhưng một số thuật toán consistent hashing khác cũng đã được đề xuất [19], chẳng hạn như *highest random weight*, còn được gọi là *rendezvous hashing* [20], và *jump consistent hashing* [21]. Với các phương pháp này, thay vì một số lượng nhỏ các shard hiện có bị chia tách thành các phạm vi con để tạo ra các shard mới cho một node được thêm vào, thì node mới thay vào đó sẽ được gán các key riêng lẻ vốn trước đó nằm rải rác trên tất cả các node khác. Việc phương pháp nào ưu việt hơn tùy thuộc vào ứng dụng.

Workload bị lệch và giảm thiểu Hot Spots

Consistent hashing đảm bảo rằng các key được phân phối đồng đều trên các node, nhưng điều đó không có nghĩa là tải thực tế được phân phối đồng đều. Nếu workload bị lệch (skewed) cao—nghĩa là có nhiều dữ liệu dưới một số partition key hơn những key khác, hoặc tốc độ yêu cầu tới một số key cao hơn nhiều so với các key khác—bạn vẫn có thể gặp tình trạng một số máy chủ bị quá tải trong khi những máy chủ khác gần như không hoạt động.

Ví dụ, trên một trang mạng xã hội, một bài đăng của một người dùng nổi tiếng với hàng triệu người theo dõi có thể gây ra một cơn bão hoạt động [22]. Sự kiện này có thể dẫn đến một lượng lớn các thao tác đọc và ghi tới cùng một key (nơi partition key có thể là user ID của người nổi tiếng, hoặc ID của hành động mà mọi người đang bình luận).

Trong những tình huống như vậy, cần một chính sách sharding linh hoạt hơn [23, 24]. Một hệ thống định nghĩa các shard dựa trên các phạm vi của key (hoặc các phạm vi của hash) cho phép đặt một hot key riêng biệt vào một shard duy nhất, thậm chí có thể gán cho nó một máy chủ chuyên dụng [25].

Cũng có thể bù đắp cho sự lệch (skew) ở cấp độ ứng dụng. Ví dụ, nếu một key được biết là rất hot, một kỹ thuật đơn giản là thêm một số ngẫu nhiên vào đầu hoặc cuối của key đó. Chỉ cần thêm hai chữ số ngẫu nhiên sẽ chia đều các thao tác ghi tới key đó thành 100 key, cho phép các key này được phân phối tới các shard khác nhau.

Tuy nhiên, sau khi đã chia tách các thao tác ghi qua nhiều key, bất kỳ thao tác đọc nào giờ đây cũng phải thực hiện thêm công việc bổ sung, vì chúng phải đọc dữ liệu từ cả 100 key và kết hợp chúng lại. Khối lượng đọc tới mỗi shard của hot key không hề giảm đi; chỉ có tải ghi là được chia tách. Kỹ thuật này cũng yêu cầu thêm việc quản lý (bookkeeping): việc thêm số ngẫu nhiên chỉ nên áp dụng cho một số ít các hot key; đối với đại đa số các key có throughput ghi thấp, điều này sẽ gây ra overhead không cần thiết. Do đó, bạn cũng cần một cách nào đó để theo dõi những key nào đang

được chia tách, và một quy trình để chuyển đổi một key thông thường thành một hot key được quản lý đặc biệt.

Vấn đề này càng trở nên phức tạp hơn do sự thay đổi của tải theo thời gian: ví dụ, một bài đăng cụ thể trên mạng xã hội khi trở nên viral có thể chịu tải cao trong vài ngày, nhưng sau đó có khả năng sẽ lắng xuống. Ngoài ra, một số key có thể bị nóng đối với các thao tác ghi, trong khi những key khác lại nóng đối với các thao tác đọc, đòi hỏi các chiến lược khác nhau để xử lý chúng.

Một số hệ thống (đặc biệt là các dịch vụ cloud được thiết kế cho quy mô lớn) có các phương pháp tự động để xử lý các hot shard. Ví dụ, Amazon gọi đó là *beat management* [26] hoặc *adaptive capacity* [17]. Chi tiết về cách các hệ thống này hoạt động nằm ngoài phạm vi của cuốn sách này.

Vận hành: Rebalancing Tự động so với Thủ công

Chúng ta đã lướt qua một câu hỏi quan trọng liên quan đến rebalancing: việc chia tách các shard và rebalancing diễn ra một cách tự động hay thủ công?

Một số hệ thống tự động quyết định khi nào cần chia tách các shard và khi nào cần di chuyển chúng từ node này sang node khác mà không cần bất kỳ sự tương tác nào từ con người, trong khi những hệ thống khác để việc sharding cho quản trị viên cấu hình một cách rõ ràng. Cũng có một giải pháp trung gian—ví dụ, Couchbase và Riak tự động tạo ra một đề xuất phân bổ shard nhưng yêu cầu quản trị viên phải commit trước khi nó có hiệu lực.

Rebalancing hoàn toàn tự động có thể rất tiện lợi, vì có ít công việc vận hành hơn cho việc bảo trì thông thường, và các hệ thống như vậy thậm chí có thể autoscale để thích ứng với những thay đổi trong khối lượng công việc. Các cơ sở dữ liệu cloud như DynamoDB được quảng bá là có khả năng tự động thêm và xóa các shard để thích ứng với những đợt tăng hoặc giảm tải lớn chỉ trong vòng vài phút [17, 27].

Tuy nhiên, việc quản lý shard tự động cũng có thể không thể dự đoán trước được. Rebalancing là một thao tác tốn kém, vì nó yêu cầu định tuyến lại các request và di chuyển một lượng lớn dữ liệu từ node này sang node khác. Nếu quá trình này không được thực hiện cẩn thận, nó có thể làm quá tải mạng hoặc các node, và có thể gây hại đến hiệu năng của các request khác. Hệ thống phải tiếp tục xử lý các thao tác ghi trong khi quá trình rebalancing đang diễn ra; nếu một hệ thống đang ở gần mức throughput ghi tối đa, quá trình chia tách shard thậm chí có thể không theo kịp tốc độ của các thao tác ghi đang đến [27].

Sự tự động hóa như vậy có thể gây nguy hiểm khi kết hợp với việc phát hiện lỗi tự động. Ví dụ, giả sử một node bị quá tải và tạm thời phản hồi các request chậm. Các node khác kết luận rằng node bị quá tải đã chết, và tự động rebalance cluster để chuyển tải ra khỏi nó. Điều này gây thêm tải lên các node khác và mạng, làm cho tình

hình trở nên tồi tệ hơn. Có rủi ro gây ra lỗi dây chuyền (cascading failure), nơi các node khác trở nên quá tải và cũng bị nghi ngờ sai lệch là đã ngừng hoạt động.

Vì lý do đó, việc có một human-in-the-loop cho quá trình rebalancing có thể là một lựa chọn tốt. Nó chậm hơn so với một quy trình hoàn toàn tự động, nhưng nó có thể giúp ngăn chặn những bất ngờ trong vận hành. Rebalancing thủ công cũng hữu ích để thực hiện rebalancing đón đầu nếu dự báo có một sự gia tăng lưu lượng truy cập do một sự kiện đã biết, chẳng hạn như các đợt giảm giá ngày Cyber Monday hoặc việc bán vé cho một sự kiện thể thao phổ biến như World Cup.

Request Routing

Chúng ta đã thảo luận về cách sharding một dataset trên nhiều node, và cách rebalance các shard đó khi các node được thêm vào hoặc xóa đi. Bây giờ hãy chuyển sang một câu hỏi khác: nếu bạn muốn đọc hoặc ghi một key cụ thể, làm thế nào bạn biết được mình cần kết nối tới node nào—tức là địa chỉ IP và số cổng (port) nào?

Chúng ta gọi vấn đề này là *request routing*, và nó rất giống với *service discovery*, thứ mà chúng ta đã thảo luận trước đó trong mục “Load balancers, service discovery, and service meshes”. Sự khác biệt lớn nhất giữa hai khái niệm này là với các service chạy mã ứng dụng, mỗi instance thường là stateless, và một load balancer có thể gửi một request tới bất kỳ instance nào. Với các cơ sở dữ liệu sharded, một request cho một key chỉ có thể được xử lý bởi một node đóng vai trò là replica cho shard chứa key đó.

Điều này có nghĩa là việc routing yêu cầu phải nhận biết được sự phân bổ từ các key sang các shard và từ các shard sang các node. Ở mức độ tổng quát, có một vài cách tiếp cận cho vấn đề này (được minh họa trong Hình 7-7):

1. Cho phép các client liên hệ với bất kỳ node nào (ví dụ, thông qua một load balancer round-robin). Nếu node đó tình cờ sở hữu shard mà yêu cầu hướng tới, node đó có thể xử lý yêu cầu trực tiếp; nếu không, nó sẽ chuyển tiếp yêu cầu đến node phù hợp, nhận phản hồi, rồi chuyển phản hồi đó tới client.
2. Gửi tất cả các yêu cầu từ client đến một routing tier trước, nơi sẽ xác định node nào nên xử lý từng yêu cầu và chuyển tiếp yêu cầu đó một cách tương ứng. Bản thân routing tier này không xử lý bất kỳ yêu cầu nào; nó chỉ đóng vai trò như một load balancer có nhận biết shard.
3. Yêu cầu các client phải nhận biết được việc sharding và sự phân bổ các shard cho các node. Trong trường hợp này, một client có thể kết nối trực tiếp tới node phù hợp mà không cần bất kỳ trung gian nào.

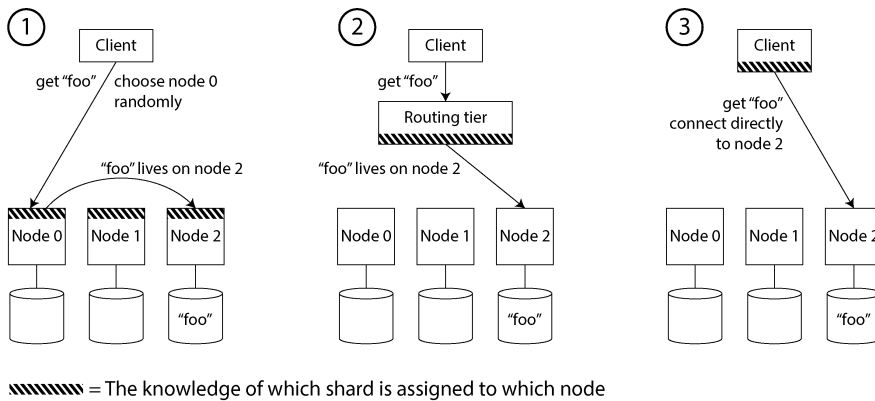
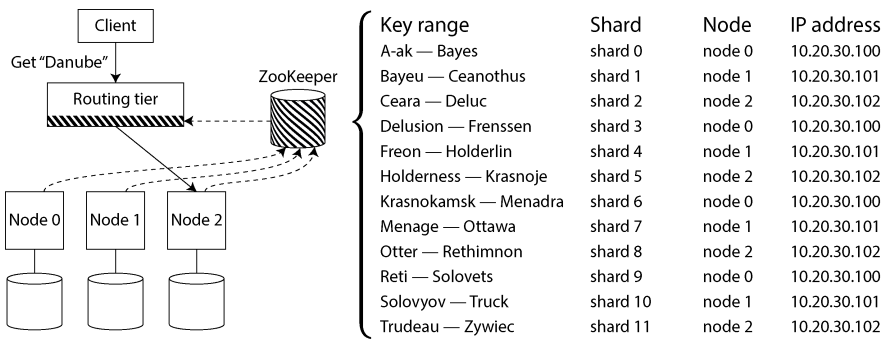


Figure 7-7. **Hình 7.1: Ba cách để routing một request đến đúng node**

Mỗi trường hợp đều gặp phải một số vấn đề then chốt:

- Ai sẽ quyết định shard nào nên nằm trên node nào? Cách đơn giản nhất là có một coordinator duy nhất đưa ra quyết định đó, nhưng trong trường hợp đó, làm thế nào để bạn giúp nó có khả năng chịu lỗi (fault-tolerant) nếu node đang chạy coordinator bị sập? Và nếu vai trò coordinator có thể fail over sang một node khác, làm thế nào để bạn ngăn chặn tình trạng split-brain (xem mục “Handling Node Outages”) khi hai coordinator khác nhau đưa ra các phân bổ shard mâu thuẫn nhau?
- Làm thế nào để thành phần thực hiện routing (có thể là một trong các node, hoặc routing tier, hoặc client) biết được những thay đổi trong việc phân bổ các shard tới các node?
- Trong khi một shard đang được di chuyển từ node này sang node khác, sẽ có một giai đoạn cutover mà trong đó node mới đã tiếp quản, nhưng các request gửi tới node cũ có thể vẫn đang in-flight. Bạn xử lý chúng như thế nào?

Nhiều hệ thống dữ liệu phân tán dựa vào một dịch vụ coordination riêng biệt như ZooKeeper hoặc etcd để theo dõi việc phân bổ shard, như được minh họa trong Hình 7-8. Chúng sử dụng các thuật toán consensus (xem Chương 10) để cung cấp khả năng chịu lỗi và bảo vệ chống lại split-brain. Mỗi node tự đăng ký chính nó trong ZooKeeper, và ZooKeeper duy trì bản đồ ánh xạ có thẩm quyền của các shard tới các node. Các actor khác, chẳng hạn như routing tier hoặc sharding-aware client, có thể đăng ký nhận thông tin này từ ZooKeeper. Bất cứ khi nào một shard thay đổi quyền sở hữu, hoặc một node được thêm vào hoặc bị loại bỏ, ZooKeeper sẽ thông báo cho routing tier để nó có thể cập nhật thông tin routing của mình.



//// = The knowledge of which shard is assigned to which node

Figure 7-8. Sử dụng ZooKeeper để theo dõi việc phân bổ các shard cho các node

Ví dụ, HBase và SolrCloud sử dụng ZooKeeper để quản lý việc gán shard, còn Kubernetes sử dụng etcd để theo dõi xem instance của service đang chạy ở đâu. MongoDB có kiến trúc tương tự, nhưng nó dựa vào việc triển khai *config server* riêng và các daemon *mongos* để làm routing tier. Kafka, YugabyteDB, TiDB, và ScyllaDB [28] sử dụng các triển khai tích hợp sẵn của giao thức consensus Raft để thực hiện chức năng điều phối này.

Riak thực hiện một cách tiếp cận khác: nó sử dụng một *gossip protocol* giữa các node để lan truyền bất kỳ thay đổi nào trong trạng thái cluster. Điều này cung cấp tính nhất quán yếu hơn nhiều so với một giao thức consensus; có khả năng xảy ra split brain, trong đó các phần khác nhau của cluster có các gán shard khác nhau cho cùng một shard. Các cơ sở dữ liệu leaderless có thể chịu đựng được điều này vì chúng thường đưa ra các đảm bảo về tính nhất quán yếu (xem “Understanding the limitations of quorum consistency”).

Khi sử dụng một routing tier hoặc khi gửi các yêu cầu đến một node ngẫu nhiên, các client vẫn cần tìm địa chỉ IP để kết nối. Những địa chỉ này không thay đổi nhanh như việc gán các shard cho các node, vì vậy việc sử dụng DNS cho mục đích này thường là đủ.

Phản thảo luận về request routing này tập trung vào việc tìm shard cho một key riêng lẻ, điều này phù hợp nhất với các cơ sở dữ liệu OLTP được sharding. Các cơ sở dữ liệu phân tích cũng thường sử dụng sharding, nhưng chúng thường có kiểu thực thi truy vấn rất khác: thay vì thực thi trong một shard duy nhất, một truy vấn thường cần aggregate và join dữ liệu từ nhiều shard song song. Chúng ta sẽ thảo luận về các kỹ thuật cho việc thực thi truy vấn song song như vậy trong Chương 11.

Sharding và Secondary Indexes

Các lược đồ sharding mà chúng ta đã thảo luận cho đến nay dựa trên việc client biết partition key cho bất kỳ bản ghi nào mà nó muốn truy cập. Điều này dễ dàng đạt được nhất trong mô hình dữ liệu key-value, nơi partition key là phần đầu tiên của primary key (hoặc toàn bộ primary key), vì vậy chúng ta có thể sử dụng partition key để xác định shard và từ đó điều hướng các thao tác đọc và ghi đến node chịu trách nhiệm cho key đó.

Tình huống trở nên phức tạp hơn nếu có sự tham gia của các secondary indexes (xem “Multicolumn and Secondary Indexes”). Một secondary index thường không xác định duy nhất một bản ghi mà là một cách để tìm kiếm các lần xuất hiện của một giá trị cụ thể: tìm tất cả các hành động của user 123, tìm tất cả các bài báo chứa từ hogwash, tìm tất cả các xe có màu là red, v.v.

Các key-value store thường không có secondary indexes, nhưng chúng là một tính năng tiêu chuẩn của các cơ sở dữ liệu quan hệ và phổ biến trong các cơ sở dữ liệu document. Loại indexing này cũng là *raison d'être* (lý do tồn tại) của các công cụ full-text search như Solr và Elasticsearch. Vấn đề với secondary indexes là chúng không ánh xạ gọn gàng vào các shard. Có hai cách tiếp cận chính để sharding một cơ sở dữ liệu có secondary indexes: local và global.

Local Secondary Indexes

Trong cách tiếp cận indexing đầu tiên, mỗi shard độc lập duy trì các secondary indexes riêng của nó, chỉ bao phủ các bản ghi trong shard đó. Nó không quan tâm dữ liệu nào được lưu trữ trong các shard khác. Bất cứ khi nào bạn ghi vào cơ sở dữ liệu—để thêm, xóa, hoặc cập nhật một bản ghi—bạn chỉ cần xử lý shard chứa bản ghi mà bạn đang ghi. Vì lý do đó, loại secondary index này được gọi là một *local index*. Trong ngữ cảnh truy hồi thông tin, nó còn được gọi là một *document-partitioned index* [29].

Ví dụ, hãy tưởng tượng bạn đang vận hành một website bán xe ô tô đã qua sử dụng. Mỗi listing có một ID duy nhất, và bạn sử dụng ID đó làm partition key để thực hiện sharding, như được minh họa trong Hình 7-9 (các ID từ 0 đến 499 nằm trong shard 0, các ID từ 500 đến 999 nằm trong shard 1, v.v.). Nếu bạn muốn cho phép người dùng tìm kiếm xe, cho phép họ lọc theo màu sắc và theo hãng xe, bạn cần các secondary index trên color và make (trong một document database, chúng sẽ là các field; trong một relational database, chúng sẽ là các column). Nếu bạn đã khai báo index, database có thể thực hiện việc lập chỉ mục một cách tự động. Ví dụ, bất cứ khi nào một chiếc xe màu đỏ được thêm vào database, database shard sẽ tự động thêm ID của nó vào danh sách các ID cho mục index color:red. Như đã thảo luận trong Chương 4, danh sách các ID đó còn được gọi là một *postings list*.

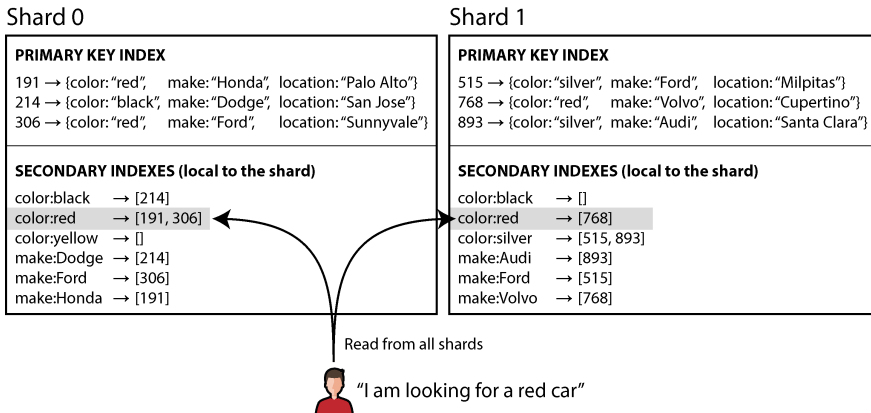


Figure 7-9. Với các **secondary index cục bộ**, mỗi **shard** chỉ lập chỉ mục các bản ghi mà nó chứa.

CẢNH BÁO

Nếu cơ sở dữ liệu của bạn chỉ hỗ trợ model key-value, bạn có thể sẽ bị cám dỗ bởi việc tự triển khai một **secondary index** bằng cách tạo ra một bản đồ ánh xạ từ các giá trị sang ID ngay trong mã nguồn ứng dụng. Nếu đi theo hướng này, bạn cần hết sức cẩn thận để đảm bảo rằng các **index** của mình luôn nhất quán với dữ liệu nền tảng. Các **race condition** và lỗi ghi không liên tục (nơi mà một số thay đổi đã được lưu nhưng những thay đổi khác thì không) có thể rất dễ khiến dữ liệu bị mất đồng bộ—hãy xem mục “Nhu cầu về multi-object transactions”.

Khi đọc từ một **local secondary index**, nếu bạn đã biết **partition key** của bản ghi mà mình đang tìm kiếm, bạn chỉ cần thực hiện tìm kiếm trên **shard** phù hợp. Hơn nữa, nếu bạn chỉ muốn *một số* kết quả và không cần tất cả, bạn có thể gửi yêu cầu tới bất kỳ **shard** nào. Tuy nhiên, nếu bạn muốn lấy tất cả kết quả và không biết trước **partition key** của chúng, bạn sẽ cần phải gửi truy vấn tới tất cả các **shard** và kết hợp các kết quả nhận được, bởi vì các bản ghi khớp có thể nằm rải rác trên tất cả các **shard**. Ví dụ, trong Hình 7-9, các xe màu đỏ xuất hiện ở cả **shard 0** và **shard 1**.

Cách tiếp cận này khi truy vấn một cơ sở dữ liệu đã được **sharding** có thể khiến các truy vấn đọc trên **secondary index** trở nên khá đắt đỏ. Ngay cả khi bạn truy vấn các **shard** song song, nó vẫn dễ gặp phải tình trạng khuếch đại **tail latency** (xem mục “Sử dụng các chỉ số Response Time”). Nó cũng giới hạn khả năng mở rộng của ứng dụng: việc thêm nhiều **shard** hơn cho phép bạn lưu trữ nhiều dữ liệu hơn, nhưng nó không làm tăng **throughput** truy vấn nếu mọi **shard** đều phải xử lý mọi truy vấn.

Tuy nhiên, các `local secondary index` được sử dụng rộng rãi [30]—ví dụ, MongoDB, Riak, Cassandra [31], Elasticsearch [32], SolrCloud, và VoltDB [33] đều sử dụng `local secondary index`.

Global Secondary Indexes

Thay vì mỗi shard có `local secondary index` riêng, chúng ta có thể xây dựng một *global index* bao phủ dữ liệu trong tất cả các shard. Tuy nhiên, chúng ta không thể chỉ lưu trữ index đó trên một node, vì nó có khả năng trở thành một nút thắt cổ chai và làm mất đi mục đích của việc sharding. Một `global index` cũng phải được sharding, nhưng nó có thể được sharding theo cách khác với `primary-key index`.

Hình 7-10 minh họa điều này trông sẽ như thế nào. Các ID của những chiếc xe màu đỏ từ tất cả các shard xuất hiện dưới `color:red` trong index, nhưng index được sharding sao cho các màu bắt đầu bằng các chữ cái từ *a* đến *r* xuất hiện trong shard 0 và các màu bắt đầu bằng *s* đến *z* xuất hiện trong shard 1. Index về hãng xe cũng được partitioning tương tự (với ranh giới shard nằm giữa *f* và *h*).

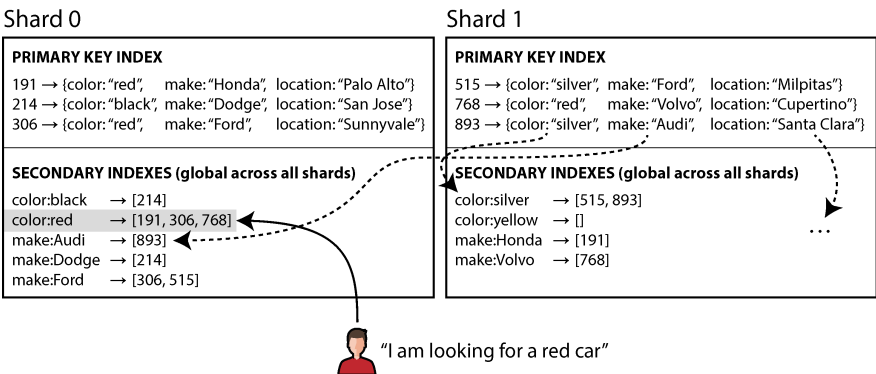


Figure 7-10. Một *secondary index* toàn cục phản ánh dữ liệu từ tất cả các shard và bản thân nó cũng được sharding theo giá trị được index

Loại index này còn được gọi là *term-partitioned* (phân mảnh theo term) [29]. Hãy nhớ lại từ phần “Full-Text Search” rằng trong tìm kiếm toàn văn, một *term* là một từ khóa trong văn bản mà bạn có thể tìm kiếm. Ở đây, chúng ta tổng quát hóa nó để chỉ bất kỳ giá trị nào mà bạn có thể tìm kiếm trong secondary index.

Global index sử dụng term làm partition key, nhờ đó khi bạn đang tìm kiếm một term hoặc giá trị cụ thể, bạn có thể xác định được shard nào cần được truy vấn. Một lần nữa, một shard có thể chứa một phạm vi các term liên tiếp (như trong Hình 7-10), hoặc bạn có thể gán các term vào các shard dựa trên một hàm hash của term đó.

Global index có lợi thế là một truy vấn với một điều kiện duy nhất (chẳng hạn như `color = red`) chỉ cần đọc từ một shard duy nhất để lấy postings list. Tuy nhiên, nếu bạn muốn lấy các bản ghi chứ không chỉ là các ID, bạn vẫn phải đọc từ tất cả các shard chịu trách nhiệm cho các ID đó.

Nếu bạn có nhiều điều kiện hoặc term tìm kiếm (ví dụ: tìm kiếm xe hơi có màu sắc và hãng xe nhất định, hoặc tìm kiếm nhiều từ xuất hiện trong cùng một văn bản), các term đó có khả năng sẽ được gán vào các shard khác nhau. Để tính toán phép toán AND logic của hai điều kiện, hệ thống cần tìm tất cả các ID xuất hiện trong cả hai postings list. Điều đó không thành vấn đề nếu các postings list ngắn, nhưng nếu chúng dài, việc gửi chúng qua mạng để tính toán giao (intersection) của chúng có thể sẽ chậm [29].

Một thách thức khác đối với global secondary index là các thao tác ghi phức tạp hơn so với local index, bởi vì việc ghi một bản ghi duy nhất có thể ảnh hưởng đến nhiều shard của index (mọi term trong tài liệu đều có thể nằm trên các shard khác nhau). Điều này làm cho việc giữ cho secondary index đồng bộ với dữ liệu bên dưới trở nên khó khăn hơn. Một lựa chọn là sử dụng một distributed transaction để cập nhật nguyên tử các shard lưu trữ bản ghi chính và các secondary index của nó (xem Chương 8).

Global secondary index được sử dụng bởi CockroachDB, TiDB, và YugabyteDB; DynamoDB hỗ trợ cả local và global secondary index. Trong trường hợp của DynamoDB, các thao tác ghi được phản ánh bất đồng bộ vào các global index, vì vậy việc đọc từ một global index có thể bị stale (điều này tương tự với tình huống đã thảo luận trong phần “Problems with Replication Lag”). Tuy nhiên, global index vẫn hữu ích nếu throughput đọc cao hơn throughput ghi, và nếu các postings list không quá dài.

Tóm tắt

Trong chương này, chúng ta đã khám phá các cách khác nhau để sharding một dataset lớn thành các tập hợp con nhỏ hơn. Sharding là cần thiết khi bạn có quá nhiều dữ liệu đến mức việc lưu trữ và xử lý chúng trên một máy đơn lẻ không còn khả thi.

Mục tiêu của sharding là phân tán dữ liệu và tải truy vấn một cách đồng đều trên nhiều máy, tránh các hot spot (các node có tải cao bất thường). Điều này đòi hỏi việc lựa chọn một lược đồ sharding phù hợp với dữ liệu của bạn, và thực hiện rebalancing các shard khi các node được thêm vào hoặc bị loại bỏ khỏi cluster.

Chúng ta đã thảo luận về hai cách tiếp cận chính để sharding:

Key range sharding

Các key được sắp xếp, và một shard sở hữu tất cả các key từ giá trị tối thiểu đến giá trị tối đa. Việc sắp xếp có lợi thế là cho phép thực hiện các truy vấn phạm vi (range queries) hiệu quả, nhưng có rủi ro xảy ra hot spots nếu ứng dụng thường xuyên truy cập các key nằm gần nhau trong thứ tự sắp xếp.

Trong cách tiếp cận này, các shard thường được rebalancing bằng cách chia phạm vi thành hai phạm vi con khi một shard trở nên quá lớn.

Hash sharding

Một hàm hash được áp dụng cho mỗi key, và một shard sở hữu một phạm vi các giá trị hash (hoặc một thuật toán consistent hashing khác có thể được sử dụng để ánh xạ các hash vào các shard). Phương pháp này phá vỡ thứ tự của các key, khiến các truy vấn phạm vi trở nên không hiệu quả, nhưng nó có thể phân phối tải đồng đều hơn.

Khi sharding bằng hash, việc tạo trước một số lượng shard cố định, gán nhiều shard cho mỗi node, và di chuyển toàn bộ shard từ node này sang node khác khi các node được thêm vào hoặc bị loại bỏ là điều phổ biến. Việc chia nhỏ các shard, tương tự như với key ranges, cũng là điều khả thi.

Việc sử dụng phần đầu tiên của key làm partition key (tức là để xác định shard) và sắp xếp các bản ghi trong shard đó theo phần còn lại của key là điều phổ biến. Bằng cách đó, bạn vẫn có thể thực hiện các truy vấn phạm vi (range queries) hiệu quả giữa các bản ghi có cùng partition key.

Chúng ta cũng đã thảo luận về các kỹ thuật để điều hướng (routing) các truy vấn tới shard phù hợp, và đã xem xét cách một coordination service thường được sử dụng để theo dõi việc phân bổ các shard cho các node.

Cuối cùng, chúng ta đã xem xét sự tương tác giữa sharding và secondary index. Một secondary index cũng cần phải được sharding. Có hai phương pháp cho việc này:

Local secondary index

Các secondary index được lưu trữ trong cùng một shard với primary key và value. Chỉ có một shard duy nhất cần được cập nhật khi ghi, nhưng việc tra cứu secondary index lại yêu cầu phải đọc từ tất cả các shards.

Global secondary index

Các secondary index được sharding riêng biệt dựa trên các giá trị được index. Một mục trong secondary index có thể tham chiếu tới các bản ghi từ tất cả các shards của primary key. Khi một bản ghi được ghi, có thể cần phải cập nhật nhiều shard của secondary index; tuy nhiên, việc đọc postings list có thể được phục vụ từ một shard duy nhất (việc lấy các bản ghi thực tế vẫn yêu cầu đọc từ nhiều shards).

Theo thiết kế, mỗi shard hoạt động hầu như độc lập—đó là điều cho phép một cơ sở dữ liệu đã được sharding có khả năng mở rộng lên nhiều máy. Tuy nhiên, các thao tác cần ghi vào nhiều shard có thể gây ra vấn đề—ví dụ, điều gì sẽ xảy ra nếu việc ghi

vào một shard thành công, nhưng một shard khác lại thất bại? Chúng ta sẽ giải quyết câu hỏi đó trong các chương tiếp theo.

Footnotes

References

- [1] Claire Giordano. “Understanding Partitioning and Sharding in Postgres and Citus.” *citusdata.com*, August 2023. Archived at perma.cc/8BTK-8959
- [2] Brandur Leach. “Partitioning in Postgres, 2022 Edition.” *brandur.org*, October 2022. Archived at perma.cc/Z5LE-6AKX
- [3] Raph Koster. “Database ‘Sharding’ Came from UO?” *raphkoster.com*, January 2009. Archived at perma.cc/4N9U-5KYF
- [4] Garrett Fidalgo. “Herding Elephants: Lessons Learned from Sharding Postgres at Notion.” *notion.com*, October 2021. Archived at perma.cc/5J5V-W2VX
- [5] Ulrich Drepper. “What Every Programmer Should Know About Memory.” *akkadia.org*, November 2007. Archived at perma.cc/NU6Q-DRXZ
- [6] Jingyu Zhou, Meng Xu, Alexander Shraer, Bala Namasivayam, Alex Miller, Evan Tschannen, Steve Atherton, Andrew J. Beamon, Rusty Sears, John Leach, Dave Rosenthal, Xin Dong, Will Wilson, Ben Collins, David Scherer, Alec Grieser, Young Liu, Alvin Moore, Bhaskar Muppana, Xiaoge Su, and Vishesh Yadav. “FoundationDB: A Distributed Unbundled Transactional Key Value Store.” At *ACM International Conference on Management of Data (SIGMOD)*, June 2021. doi: [10.1145/3448016.3457559](https://doi.org/10.1145/3448016.3457559)
- [7] Marco Slot. “Citus 12: Schema-Based Sharding for PostgreSQL.” *citusdata.com*, July 2023. Archived at perma.cc/R874-EC9W
- [8] Robisson Oliveira. “Reducing the Scope of Impact with Cell-Based Architecture.” AWS Well-Architected White Paper, Amazon Web Services, September 2023. Archived at perma.cc/4KWW-47NR
- [9] Gwen Shapira. “Things DBs Don’t Do—But Should.” *thenile.dev*, February 2023. Archived at perma.cc/C3J4-JSFW
- [10] Malte Schwarzkopf, Eddie Kohler, M. Frans Kaashoek, and Robert Morris. “Position: GDPR Compliance by Construction.” At *Towards Polystores That Manage Multiple Databases, Privacy, Security and/or Policy Issues for Heterogenous Data (Poly)*, August 2019. doi: [10.1007/978-3-030-33752-0_3](https://doi.org/10.1007/978-3-030-33752-0_3)
- [11] Gwen Shapira. “Introducing pg_karnak: Transactional Schema Migration Across Tenant Databases.” *thenile.dev*, November 2024. Archived at perma.cc/R5RD-8HR9
- [12] Arka Ganguli, Guido Iaquinti, Maggie Zhou, and Rafael Chacón. “Scaling Datastores at Slack with Vitess.” *slack.engineering*, December 2020. Archived at perma.cc/UW8F-ALJK
- [13] Ikai Lan. “App Engine Datastore Tip: Monotonically Increasing Values Are Bad.” *ikaisays.com*, January 2011. Archived at perma.cc/BPX8-RPJ8
- [14] Enis Soztutar. “Apache HBase Region Splitting and Merging.” *cloudera.com*, February 2013. Archived at perma.cc/S9HS-2X2C
- [15] Eric Evans. “Rethinking Topology in Cassandra.” At *Cassandra Summit*, June 2013. Archived at perma.cc/2DKM-F438
- [16] Martin Kleppmann. “Java’s hashCode Is Not Safe for Distributed Systems.” *martin.kleppmann.com*, June 2012. Archived at perma.cc/LK5U-VZSN

- [17] Mostafa Elhemi, Niall Gallagher, Nicholas Gordon, Joseph Idziorek, Richard Krog, Colin Lazier, Erben Mo, Akhilesh Mritunjai, Somu Perianayagam, Tim Rath, Swami Sivasubramanian, James Christopher Sorenson III, Sroaj Sosothikul, Doug Terry, and Akshat Vig. “Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service.” At *USENIX Annual Technical Conference (ATC)*, July 2022.
- [18] David Karger, Eric Lehman, Tom Leighton, Rina Panigrahy, Matthew Levine, and Daniel Lewin. “Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web.” At *29th Annual ACM Symposium on Theory of Computing (STOC)*, May 1997. doi:10.1145/258533.258660
- [19] Damian Gryski. “Consistent Hashing: Algorithmic Tradeoffs.” *dgryski.medium.com*, April 2018. Archived at perma.cc/B2WF-TYQ8
- [20] David G. Thaler and China V. Ravishankar. “Using Name-Based Mappings to Increase Hit Rates.” *IEEE/ACM Transactions on Networking*, volume 6, issue 1, pages 1–14, February 1998. doi:10.1109/90.663936
- [21] John Lamping and Eric Veach. “A Fast, Minimal Memory, Consistent Hash Algorithm.” *arXiv:1406.2294*, June 2014.
- [22] Samuel Axon. “3% of Twitter’s Servers Dedicated to Justin Bieber.” *mashable.com*, September 2010. Archived at perma.cc/F55N-CGVX
- [23] Gerald Guo and Thawan Kooburat. “Scaling Services with Shard Manager.” *engineering.fb.com*, August 2020. Archived at perma.cc/EFS3-XQYT
- [24] Sangmin Lee, Zhenhua Guo, Omer Sunercan, Jun Ying, Thawan Kooburat, Suryadeep Biswal, Jun Chen, Kun Huang, Yatpang Cheung, Yiding Zhou, Kaushik Veeraraghavan, Biren Damani, Pol Mauri Ruiz, Vikas Mehta, and Chunqiang Tang. “Shard Manager: A Generic Shard Management Framework for Geo-Distributed Applications.” At *28th ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, October 2021. doi:10.1145/3477132.3483546
- [25] Scott Lystig Fritchie. “A Critique of Resizable Hash Tables: Riak Core & Random Slicing.” *infoq.com*, August 2018. Archived at perma.cc/RPX7-7BLN
- [26] Andy Warfield. “Building and Operating a Pretty Big Storage System Called S3.” *allthingsdistributed.com*, July 2023. Archived at perma.cc/6S7P-GLM4
- [27] Rich Houlihan. “DynamoDB Adaptive Capacity: Smooth Performance for Chaotic Workloads (DAT327).” At *AWS re:Invent*, November 2017.
- [28] Kostja Osipov. “ScyllaDB’s Safe Topology and Schema Changes on Raft.” *scylladb.com*, June 2024. Archived at perma.cc/4S82-M277
- [29] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008. ISBN: 9780521865715. Available online at nlp.stanford.edu/IR-book.
- [30] Michael Busch, Krishna Gade, Brian Larson, Patrick Lok, Samuel Luckenbill, and Jimmy Lin. “Earlybird: Real-Time Search at Twitter.” At *28th IEEE International Conference on Data Engineering (ICDE)*, April 2012. doi:10.1109/ICDE.2012.149
- [31] Nadav Har’El. “Indexing in Cassandra 3.” *github.com*, April 2017. Archived at perma.cc/3ENV-8T9P
- [32] Zachary Tong. “Customizing Your Document Routing.” *elastic.co*, June 2013. Archived at perma.cc/97VM-MREN

[33] Andrew Pavlo. “H-Store Documentation: Frequently Asked Questions.” *hstore.cs.brown.edu*, October 2013. Archived at perma.cc/X3ZA-DW6Z

Transactions

Một số tác giả cho rằng cơ chế two-phase commit tổng quát là quá tốn kém để hỗ trợ, do những vấn đề về hiệu năng hoặc availability mà nó mang lại. Chúng tôi tin rằng sẽ tốt hơn nếu để các lập trình viên ứng dụng xử lý các vấn đề hiệu năng phát sinh do việc lạm dụng transaction gây ra các điểm nghẽn, thay vì luôn phải viết mã để né tránh việc thiếu hụt transaction.

James Corbett và các cộng sự, “Spanner: Google’s Globally-Distributed Database” (2012)

Trong thực tế khắc nghiệt của các hệ thống dữ liệu, nhiều thứ có thể xảy ra sai sót:

- Phần mềm hoặc phần cứng cơ sở dữ liệu có thể gặp lỗi bất cứ lúc nào (bao gồm cả khi đang ở giữa một thao tác ghi).
- Ứng dụng có thể bị crash bất cứ lúc nào (bao gồm cả khi đang thực hiện dở dang một chuỗi các thao tác).
- Sự gián đoạn trong mạng có thể cắt đứt kết nối giữa ứng dụng với cơ sở dữ liệu, hoặc giữa một node cơ sở dữ liệu này với một node khác một cách bất ngờ.
- Nhiều client có thể ghi vào cơ sở dữ liệu cùng một lúc, ghi đè lên các thay đổi của nhau.
- Một client có thể đọc dữ liệu không hợp lý vì nó mới chỉ được cập nhật một phần.
- Các race condition giữa các client có thể gây ra những bug bất ngờ.

Để đạt được độ tin cậy, một hệ thống phải xử lý được tất cả các loại fault này và đảm bảo rằng chúng không gây ra những thất bại thảm khốc. Tuy nhiên, việc triển khai các cơ chế fault-tolerance là một khối lượng công việc khổng lồ. Nó đòi hỏi sự suy xét kỹ lưỡng về tất cả những thứ có thể sai sót và việc kiểm thử nghiêm ngặt để đảm bảo rằng các giải pháp được triển khai thực sự hoạt động hiệu quả.

Trong nhiều thập kỷ, transaction đã là cơ chế được lựa chọn để đơn giản hóa các vấn đề này. Một *transaction* là cách để một ứng dụng nhóm nhiều thao tác đọc và ghi lại với nhau thành một đơn vị logic. Về mặt khái niệm, tất cả các thao tác đọc và ghi trong một transaction được thực thi như một thao tác duy nhất; hoặc là toàn bộ transaction thành công, dẫn đến một lệnh *commit*, hoặc nó thất bại, dẫn đến một lệnh *abort* hoặc *rollback*. Nếu nó thất bại, ứng dụng có thể thử lại một cách an toàn. Với transaction, việc xử lý lỗi trở nên đơn giản hơn nhiều cho một ứng dụng, bởi vì

nó không cần phải lo lắng về các lỗi xảy ra một phần (nơi mà, vì bất kỳ lý do gì, một số thao tác thành công trong khi một số khác lại thất bại).

Nếu bạn đã quen làm việc với transaction, chúng có vẻ hiển nhiên, nhưng chúng ta không nên coi chúng là điều mặc nhiên. Transaction không phải là một quy luật tự nhiên; chúng được tạo ra với một mục đích—cụ thể là để *đơn giản hóa mô hình lập trình* cho các ứng dụng truy cập vào cơ sở dữ liệu. Sử dụng transaction cho phép ứng dụng bỏ qua một số kịch bản lỗi tiềm ẩn và các vấn đề về concurrency, vì cơ sở dữ liệu sẽ đảm nhận thay (chúng ta gọi đây là các *safety guarantees*).

Không phải ứng dụng nào cũng cần transaction, và đôi khi việc nói lỏng các đảm bảo về transaction hoặc từ bỏ hoàn toàn chúng lại mang lại lợi thế (ví dụ: để đạt được hiệu năng tốt hơn hoặc availability cao hơn). Một số tính chất an toàn có thể đạt được mà không cần transaction. Mặt khác, transaction có thể ngăn chặn rất nhiều rắc rối; ví dụ, nguyên nhân kỹ thuật đằng sau vụ bê bối Post Office Horizon (xem mục “How Important Is Reliability?”) có lẽ là do sự thiếu hụt các transaction ACID trong hệ thống kế toán nền tảng [1].

Làm thế nào để bạn xác định liệu mình có cần transaction hay không? Để trả lời câu hỏi đó, trước tiên chúng ta cần hiểu chính xác các đảm bảo an toàn mà transaction có thể cung cấp và các chi phí đi kèm với chúng. Mặc dù thoạt nhìn transaction có vẻ đơn giản, nhưng có nhiều chi tiết tinh vi nhưng quan trọng sẽ tham gia vào quá trình này.

Kiểm soát concurrency (concurrency control) có liên quan đến cả cơ sở dữ liệu đơn node và cơ sở dữ liệu phân tán. Chúng ta sẽ xem xét kỹ chủ đề này trong chương này, thảo luận về các loại race condition khác nhau có thể xảy ra và cách các cơ sở dữ liệu triển khai các isolation level như read-committed, snapshot isolation, và serializability. Chúng ta cũng sẽ xem xét giao thức two-phase commit và thách thức trong việc đạt được tính atomicity trong một distributed transaction.

Chính xác thì một Transaction là gì?

Hầu hết tất cả các cơ sở dữ liệu relational ngày nay, và một số cơ sở dữ liệu nonrelational, đều hỗ trợ transaction. Đa số chúng tuân theo phong cách được giới thiệu vào năm 1975 bởi IBM System R, cơ sở dữ liệu SQL đầu tiên [2, 3, 4]. Mặc dù một số chi tiết triển khai đã thay đổi, nhưng ý tưởng chung về cơ bản vẫn giữ nguyên trong suốt 50 năm qua: sự hỗ trợ transaction trong MySQL, PostgreSQL, Oracle, SQL Server, v.v., tương đồng một cách kỳ lạ với System R.

Vào cuối những năm 2000, các cơ sở dữ liệu nonrelational (NoSQL) bắt đầu trở nên phổ biến. Chúng nhằm mục đích cải thiện trạng thái hiện tại của các cơ sở dữ liệu relational bằng cách cung cấp các lựa chọn về mô hình dữ liệu mới (xem Chương 3) và mặc định bao gồm cả replication và sharding (được thảo luận trong Chương 6 và

7). Transaction là nạn nhân chính của phong trào này: nhiều cơ sở dữ liệu thuộc thể hệ này đã từ bỏ hoàn toàn transaction, hoặc định nghĩa lại thuật ngữ này để mô tả một tập hợp các đảm bảo yếu hơn nhiều so với những gì đã được hiểu trước đó.

Sự thổi phồng xung quanh các cơ sở dữ liệu phân tán NoSQL đã dẫn đến một niềm tin phổ biến rằng transaction về cơ bản là không có khả năng mở rộng và bất kỳ hệ thống quy mô lớn nào cũng sẽ phải từ bỏ chúng để duy trì hiệu năng tốt và độ tin cậy cao. Gần đây hơn, niềm tin đó đã hóa ra là sai lầm. Các cơ sở dữ liệu được gọi là "NewSQL" như CockroachDB [5], TiDB [6], Spanner [7], FoundationDB [8], và YugabyteDB đã chứng minh rằng các hệ thống transactional có thể mở rộng lên khối lượng dữ liệu lớn và throughput cao. Các hệ thống này kết hợp sharding với các giao thức consensus, thứ mà chúng ta sẽ khám phá trong Chương 10, để cung cấp các đảm bảo ACID mạnh mẽ ở quy mô lớn.

Tuy nhiên, điều đó không có nghĩa là mọi hệ thống đều phải có tính transactional; giống như mọi lựa chọn thiết kế kỹ thuật khác, transaction có những ưu điểm và hạn chế riêng. Để hiểu được những đánh đổi (trade-off) đó, trong chương này, chúng ta sẽ khám phá chi tiết về các đảm bảo mà transaction có thể cung cấp, cả trong vận hành bình thường và trong các tình huống cực đoan khác nhau (nhưng thực tế).

Ý nghĩa của ACID

Các đảm bảo an toàn được cung cấp bởi transaction thường được mô tả bằng từ viết tắt nổi tiếng *ACID*, viết tắt của *atomicity*, *consistency*, *isolation*, và *durability*. Thuật ngữ này được đặt ra vào năm 1983 bởi Theo Härder và Andreas Reuter [9], nhằm nỗ lực thiết lập thuật ngữ chính xác cho các cơ chế fault-tolerance trong cơ sở dữ liệu.

Tuy nhiên, trong thực tế, việc triển khai ACID của một cơ sở dữ liệu này không tương đương với cơ sở dữ liệu khác. Ví dụ, như chúng ta sẽ thấy, có rất nhiều sự mơ hồ xung quanh ý nghĩa của *isolation* [10]. Ý tưởng ở cấp độ cao là hợp lý, nhưng vấn đề nằm ở các chi tiết. Ngày nay, khi một hệ thống tuyên bố tuân thủ "ACID", thật không rõ bạn thực sự có thể kỳ vọng vào những đảm bảo nào. Thật không may, "ACID" phần lớn đã trở thành một thuật ngữ tiếp thị.

LƯU Ý

Các hệ thống không đáp ứng được các tiêu chí ACID đôi khi được gọi là *BASE*, viết tắt của *basically available*, *soft state*, và *eventual consistency* [11]. Điều này thậm chí còn mơ hồ hơn cả định nghĩa về ACID. Có vẻ như định nghĩa hợp lý duy nhất về BASE là "không phải ACID" (tức là, nó có thể có nghĩa là bất cứ điều gì bạn muốn).

Hãy cùng đi sâu vào các định nghĩa về atomicity, consistency, isolation, và durability, vì điều này sẽ cho phép chúng ta tinh chỉnh ý tưởng về transaction của mình.

Atomicity

Nói chung, *atomic* dùng để chỉ một thứ gì đó không thể bị chia nhỏ thành các phần nhỏ hơn. Từ này mang ý nghĩa tương tự nhưng có những khác biệt tinh tế trong các nhánh khác nhau của ngành máy tính. Ví dụ, trong lập trình đa luồng (multithreaded), nếu một thread thực thi một thao tác atomic, điều đó có nghĩa là không có cách nào để một thread khác có thể nhìn thấy kết quả đang thực hiện dở dang của thao tác đó. Hệ thống chỉ có thể ở trạng thái trước khi thực hiện thao tác hoặc sau khi thực hiện thao tác, chứ không phải một trạng thái nào đó ở giữa.

Ngược lại, trong ngữ cảnh của ACID, atomicity *không* nói về tính đồng thời (concurrency). Nó không mô tả những gì xảy ra nếu nhiều tiến trình cố gắng truy cập cùng một dữ liệu tại cùng một thời điểm, vì điều đó đã được bao hàm trong chữ cái *I*, tức là *isolation* (xem mục "Isolation").

Thay vào đó, tính atomicity trong ACID mô tả những gì xảy ra nếu một client muốn thực hiện nhiều lệnh ghi, nhưng một fault xảy ra sau khi một số lệnh ghi đã được xử lý—ví dụ, một process bị crash, một kết nối mạng bị gián đoạn, đĩa bị đầy, hoặc một ràng buộc tính toán vện bị vi phạm. Nếu các lệnh ghi được nhóm lại thành một transaction nguyên tử, và transaction đó không thể hoàn tất (commit) do một fault, thì transaction sẽ bị abort và cơ sở dữ liệu phải hủy bỏ hoặc hoàn tác bất kỳ lệnh ghi nào mà nó đã thực hiện cho đến thời điểm đó trong transaction đó.

Nếu không có tính atomicity, khi một lỗi xảy ra giữa chừng trong quá trình thực hiện nhiều thay đổi, sẽ rất khó để biết những thay đổi nào đã có hiệu lực và những thay đổi nào chưa. Ứng dụng có thể thử lại, nhưng điều đó dẫn đến rủi ro thực hiện một số thay đổi hai lần, gây ra dữ liệu bị trùng lặp hoặc không chính xác. Tính atomicity đơn giản hóa vấn đề này: nếu một transaction đã bị abort, ứng dụng có thể chắc chắn rằng nó chưa thay đổi bất cứ điều gì, vì vậy nó có thể được thử lại một cách an toàn.

Khả năng abort một transaction khi có lỗi và hủy bỏ tất cả các lệnh ghi từ transaction đó chính là đặc điểm định nghĩa của tính atomicity trong ACID. Có lẽ *abortability* (khả năng hủy bỏ) sẽ là một thuật ngữ tốt hơn thay vì *atomicity*, nhưng chúng ta sẽ giữ nguyên *atomicity* vì đó là thuật ngữ thông dụng.

Consistency

Từ *consistency* (tính nhất quán) bị dùng quá nhiều nghĩa:

- Trong Chương 6, chúng ta đã thảo luận về *replica consistency* (tính nhất quán của bản sao) và vấn đề *eventual consistency* (tính nhất quán cuối cùng) phát sinh trong các hệ thống replication bất đồng bộ (xem mục "Problems with Replication Lag").
- Một *consistent snapshot* (ảnh chụp nhất quán) của một cơ sở dữ liệu, chẳng hạn như để sao lưu, là một snapshot của toàn bộ cơ sở dữ liệu tại một thời điểm nhất

định. Chính xác hơn, một consistent snapshot là nhất quán với quan hệ happens-before (xem mục “The happens-before relation and concurrency”): nếu snapshot chứa một giá trị đã được ghi tại một thời điểm cụ thể, thì snapshot đó cũng phản ánh tất cả các lệnh ghi đã xảy ra trước khi giá trị đó được ghi.

- *Consistent hashing* là một phương pháp sharding mà một số hệ thống sử dụng để rebalancing (xem mục “Consistent hashing”).
- Trong định lý CAP (được thảo luận trong Chương 10), từ *consistency* được dùng để chỉ *linearizability* (linearizability) (xem mục “Linearizability”).
- Trong ngữ cảnh của ACID, *consistency* đề cập đến một khái niệm đặc thù của ứng dụng về việc cơ sở dữ liệu đang ở trong một “trạng thái tốt”.

Thật không may khi cùng một từ lại có ít nhất năm ý nghĩa khác nhau.

Ý tưởng về tính consistency trong ACID là bạn có những tuyên bố nhất định về dữ liệu của mình (*invariants* - các bất biến) mà chúng phải luôn luôn đúng—ví dụ, trong một hệ thống kế toán, các khoản ghi có và ghi nợ trên tất cả các tài khoản phải luôn luôn cân bằng. Nếu một transaction bắt đầu với một cơ sở dữ liệu hợp lệ theo các invariants này, và bất kỳ lệnh ghi nào trong quá trình transaction vẫn bảo toàn được tính hợp lệ, thì bạn có thể chắc chắn rằng các invariants luôn được thỏa mãn. (Một invariant có thể bị vi phạm tạm thời trong quá trình thực thi transaction, nhưng nó phải được thỏa mãn trở lại khi transaction commit.)

Nếu bạn muốn cơ sở dữ liệu thực thi các invariants của mình, bạn cần khai báo chúng dưới dạng các *constraints* (ràng buộc) như một phần của schema. Ví dụ, các ràng buộc foreign-key, ràng buộc uniqueness, và các ràng buộc check (nhằm giới hạn các giá trị có thể xuất hiện trong một hàng riêng lẻ) thường được sử dụng để mô hình hóa các loại invariants cụ thể. Các yêu cầu về consistency phức tạp hơn đôi khi có thể được mô hình hóa bằng cách sử dụng triggers hoặc materialized views [12].

Tuy nhiên, các invariants phức tạp có thể khó hoặc không thể mô hình hóa bằng các constraints mà cơ sở dữ liệu thường cung cấp. Trong trường hợp đó, trách nhiệm của ứng dụng là định nghĩa các transaction của nó một cách chính xác để chúng bảo toàn tính consistency. Nếu bạn ghi dữ liệu sai vi phạm các invariants của mình nhưng bạn chưa khai báo các invariants đó, cơ sở dữ liệu không thể ngăn bạn lại. Do đó, chữ *C* trong *ACID* thường phụ thuộc vào cách ứng dụng sử dụng cơ sở dữ liệu và không phải là đặc tính của riêng cơ sở dữ liệu.

Isolation

Hầu hết các cơ sở dữ liệu đều được truy cập bởi nhiều client cùng một lúc. Điều đó không thành vấn đề nếu chúng đang đọc và ghi vào các phần khác nhau của cơ sở dữ liệu, nhưng nếu chúng truy cập vào cùng một bản ghi dữ liệu, bạn có thể gặp phải các vấn đề về concurrency (race condition - tình trạng tranh chấp).

Hình 8-1 là một ví dụ đơn giản về loại vấn đề này. Giả sử bạn có hai client đồng thời tăng một counter được lưu trữ trong một database. Mỗi client cần đọc giá trị hiện tại, cộng thêm 1, và ghi giá trị mới trở lại (giả định rằng không có thao tác increment nào được tích hợp sẵn trong database). Trong Hình 8-1, counter đáng lẽ phải tăng từ 42 lên 44, vì đã có hai lần tăng xảy ra, nhưng thực tế nó chỉ lên 43 do race condition.

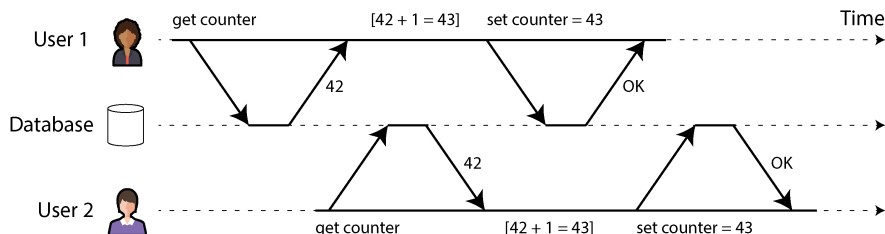


Figure 8-1. Một race condition giữa hai client đang đồng thời tăng một counter

Isolation theo nghĩa của ACID có nghĩa là các transaction thực thi đồng thời được cô lập với nhau; chúng không thể can thiệp vào công việc của nhau. Các sách giáo khoa về cơ sở dữ liệu kinh điển chính thức hóa isolation thành *serializability* (serializability hoá), nghĩa là mỗi transaction có thể giả định rằng nó là transaction duy nhất đang chạy trên toàn bộ cơ sở dữ liệu. Cơ sở dữ liệu đảm bảo rằng khi các transaction đã commit, kết quả thu được sẽ giống như thể chúng đã chạy *serially* (lần lượt từng cái một), ngay cả khi trong thực tế chúng có thể đã chạy đồng thời [13].

Tuy nhiên, serializability có một cái giá về hiệu năng. Trong thực tế, nhiều cơ sở dữ liệu sử dụng các dạng isolation yếu hơn serializability—tức là chúng cho phép các transaction đồng thời can thiệp lẫn nhau theo những cách hạn chế. Một số cơ sở dữ liệu phổ biến, chẳng hạn như Oracle, thậm chí còn không triển khai nó (Oracle có một isolation level gọi là “serializable”, nhưng thực tế nó triển khai *snapshot isolation*, một sự đảm bảo yếu hơn so với serializability [10, 14]). Điều này có nghĩa là một số loại race condition vẫn có thể xảy ra. Chúng ta sẽ khám phá snapshot isolation và các dạng isolation khác trong mục “Weak Isolation Levels”.

Durability

Mục đích của một hệ thống cơ sở dữ liệu là cung cấp một nơi an toàn để dữ liệu có thể được lưu trữ mà không lo bị mất. *Durability* là lời hứa rằng sau khi một transaction đã commit thành công, bất kỳ dữ liệu nào nó đã ghi sẽ không bị lãng quên, ngay cả khi có lỗi phần cứng hoặc cơ sở dữ liệu bị crash.

Trong một cơ sở dữ liệu single-node, durability thường có nghĩa là dữ liệu đã được ghi vào bộ lưu trữ không biến đổi (nonvolatile storage) như ổ cứng hoặc SSD. Các thao tác ghi file thông thường thường được đệm trong bộ nhớ trước khi được gửi đến đĩa vào một thời điểm nào đó sau đó, điều này có nghĩa là chúng có thể bị mất nếu xảy ra mất điện đột ngột; do đó, nhiều cơ sở dữ liệu sử dụng lời gọi hệ thống

fsync để đảm bảo rằng dữ liệu thực sự đã được ghi xuống đĩa. Các cơ sở dữ liệu thường cũng có một write-ahead log hoặc tính năng tương tự (xem “Making B-trees reliable”), cho phép chúng khôi phục trong trường hợp xảy ra crash khi đang thực hiện ghi dở dang. Nhiều cơ sở dữ liệu (chẳng hạn như MySQL, MongoDB và PostgreSQL) lưu trữ dữ liệu của chúng với một checksum, cho phép chúng phát hiện các mục log bị hỏng hoặc không đầy đủ, từ đó giúp khôi phục cơ sở dữ liệu về một snapshot nhất quán sau khi crash.

Trong một cơ sở dữ liệu được replication, durability có thể có nghĩa là dữ liệu đã được sao chép thành công tới một số lượng node nhất định. Để cung cấp một đảm bảo về durability, một cơ sở dữ liệu phải đợi cho đến khi các thao tác ghi hoặc replication này hoàn tất trước khi báo cáo một transaction đã commit thành công. Tuy nhiên, như đã thảo luận trong “Reliability and Fault Tolerance”, durability hoàn hảo không tồn tại; nếu tất cả ổ cứng và tất cả các bản sao lưu của bạn đều bị hủy hoại cùng một lúc, rõ ràng là không có gì cơ sở dữ liệu có thể làm để cứu bạn.

Replication và Durability

Về mặt lịch sử, durability có nghĩa là ghi vào một băng lưu trữ (archive tape). Sau đó, nó được hiểu là ghi vào đĩa hoặc SSD. Gần đây hơn, nó đã được điều chỉnh để có nghĩa là replication. Việc triển khai nào tốt hơn?

Sự thật là, không có gì là hoàn hảo:

- Nếu bạn ghi vào đĩa và máy tính bị hỏng, mặc dù dữ liệu của bạn không bị mất, nhưng nó không thể truy cập được cho đến khi bạn sửa máy hoặc chuyển đĩa sang một máy khác. Các hệ thống được replication có thể duy trì tính khả dụng (availability).
- Một lỗi tương quan (correlated fault)—chẳng hạn như mất điện, hoặc một lỗi gây crash mọi node với một đầu vào cụ thể—có thể đánh sập tất cả các replica cùng một lúc (xem “Reliability and Fault Tolerance”), khiến bất kỳ dữ liệu nào chỉ nằm trong bộ nhớ bị mất sạch. Do đó, việc ghi vào đĩa vẫn rất quan trọng đối với các cơ sở dữ liệu được replication.
- Trong một hệ thống asynchronously replicated, các thao tác ghi gần đây có thể bị mất khi leader không còn khả dụng (xem “Handling Node Outages”).
- Khi nguồn điện đột ngột bị ngắt, các SSD được chứng minh là đôi khi vi phạm các cam kết mà chúng đáng lẽ phải cung cấp; ngay cả fsync cũng không được đảm bảo là sẽ hoạt động chính xác [15]. Firmware của đĩa có thể có lỗi, giống như bất kỳ loại phần mềm nào khác [16, 17]—ví dụ, khiến các ổ đĩa bị lỗi sau đúng 32.768 giờ hoạt động [18]. Và fsync rất khó sử dụng; ngay cả PostgreSQL cũng đã sử dụng nó không chính xác trong hơn 20 năm [19, 20, 21].
- Sự tương tác tinh vi giữa storage engine và việc triển khai filesystem có thể dẫn đến các lỗi khó truy vết và có thể khiến các tệp trên đĩa bị hỏng sau một sự cố crash [22, 23]. Các lỗi filesystem trên một replica đôi khi cũng có thể lan sang các replica khác [24].
- Dữ liệu trên đĩa có thể dần dần bị hỏng mà không bị phát hiện [25, 26]. Nếu dữ liệu đã bị hỏng trong một khoảng thời gian, các replica và các bản sao lưu gần đây cũng có thể bị hỏng. Trong trường hợp này, bạn sẽ cần cố gắng khôi phục dữ liệu từ một bản sao lưu lịch sử.
- Một nghiên cứu về SSD đã phát hiện ra rằng 30% đến 80% các ổ đĩa gặp ít nhất một bad block trong bốn năm hoạt động đầu tiên, và chỉ một số ít trong số này có thể được sửa chữa bởi firmware [27]. Ổ cứng từ (magnetic hard drives) có tỷ lệ bad sector thấp hơn nhưng tỷ lệ lỗi hoàn toàn cao hơn so với SSD.

- Khi một SSD đã bị mòn (đã trải qua nhiều chu kỳ ghi/xóa) bị ngắt nguồn, nó có thể bắt đầu mất dữ liệu trong khoảng thời gian từ vài tuần đến vài tháng, tùy thuộc vào nhiệt độ [28]. Điều này ít trở thành vấn đề hơn đối với các ổ đĩa có mức độ mòn thấp hơn [29].

Trong thực tế, không có kỹ thuật đơn lẻ nào có thể cung cấp các cam kết tuyệt đối. Chỉ có các kỹ thuật giảm thiểu rủi ro khác nhau—bao gồm ghi vào đĩa, nhân bản sang các máy từ xa, và sao lưu—và chúng có thể cũng như nên được sử dụng cùng nhau. Như thường lệ, hãy thận trọng khi tin vào bất kỳ "cam kết" lý thuyết nào.

Các thao tác trên một đối tượng và nhiều đối tượng

Để tóm tắt, trong ACID, *atomicity* và *isolation* mô tả những gì cơ sở dữ liệu nên làm nếu một client thực hiện nhiều thao tác ghi trong cùng một *transaction*:

Atomicity

Nếu một lỗi xảy ra giữa chừng trong một chuỗi các thao tác ghi, *transaction* đó phải bị hủy bỏ, và các thao tác ghi đã thực hiện cho đến thời điểm đó phải bị loại bỏ. Nói cách khác, cơ sở dữ liệu giúp bạn không phải lo lắng về lỗi một phần bằng cách đưa ra một cam kết kiểu "tất cả hoặc không có gì".

Isolation

Các *transaction* chạy đồng thời không nên can thiệp lẫn nhau. Ví dụ, nếu một *transaction* thực hiện nhiều thao tác ghi, thì một *transaction* khác phải thấy được toàn bộ hoặc không thấy gì trong số các thao tác ghi đó, chứ không phải chỉ một tập hợp con.

Các định nghĩa này giả định rằng bạn muốn sửa đổi nhiều đối tượng (hàng, document, bản ghi) cùng một lúc. Các *multi-object transaction* như vậy thường cần thiết nếu nhiều phần dữ liệu cần được giữ đồng bộ. Hình 8-2 cho thấy một ví dụ từ một ứng dụng email. Để hiển thị số lượng tin nhắn chưa đọc của một người dùng, bạn có thể truy vấn một thứ gì đó như thế này:

```
SELECT COUNT(*) FROM emails WHERE recipient_id = 2 AND unread_flag = true
```

Tuy nhiên, bạn có thể thấy truy vấn này quá chậm nếu có nhiều email và quyết định lưu trữ số lượng tin nhắn chưa đọc trong một *field* riêng biệt (một dạng *denormalization*, mà chúng ta sẽ thảo luận trong mục “Normalization, Denormalization, and Joins”). Bây giờ, bất cứ khi nào có tin nhắn mới đến, bạn cũng phải tăng bộ đếm chưa đọc lên, và bất cứ khi nào một tin nhắn được đánh dấu là đã đọc, bạn cũng phải giảm bộ đếm chưa đọc xuống.

Trong Hình 8-2, người dùng 2 gặp phải một sự bất thường (anomaly): danh sách hộp thư hiển thị một tin nhắn chưa đọc, nhưng bộ đếm lại hiển thị số tin nhắn chưa đọc bằng không vì việc tăng bộ đếm vẫn chưa diễn ra. (Nếu một bộ đếm sai trong ứng dụng email có vẻ quá nhỏ nhặt, hãy nghĩ về số dư tài khoản khách hàng thay vì bộ đếm chưa đọc, và một transaction thanh toán thay vì một email.) Isolation sẽ ngăn chặn vấn đề này bằng cách đảm bảo rằng người dùng 2 thấy được cả email được chèn vào và bộ đếm đã cập nhật, hoặc không thấy cả hai, chứ không phải một điểm giữa không nhất quán.

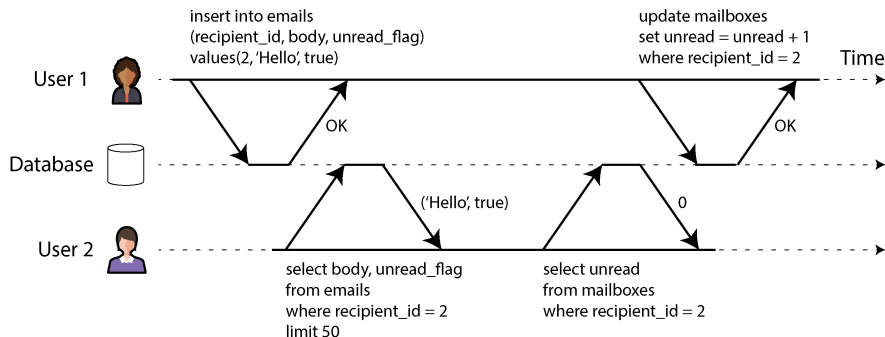


Figure 8-2. *Vi phạm isolation: một transaction đọc các write chưa được commit của một transaction khác (một trường hợp "dirty read")*

Hình 8-3 minh họa nhu cầu về tính atomicity (tính nguyên tử): nếu một lỗi xảy ra tại một thời điểm nào đó trong quá trình thực hiện transaction, nội dung của hộp thư và bộ đếm thư chưa đọc có thể bị mất đồng bộ. Trong một transaction có tính atomic, nếu việc cập nhật bộ đếm thất bại, transaction sẽ bị hủy bỏ và việc chèn email sẽ được rollback.

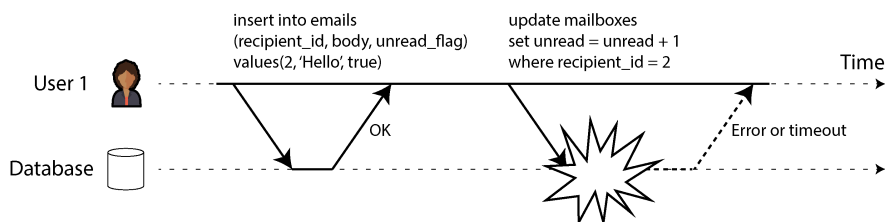


Figure 8-3. *Atomicity đảm bảo rằng nếu có lỗi xảy ra, bất kỳ thao tác ghi nào trước đó từ transaction đó đều được hoàn tác, nhằm tránh một trạng thái không nhất quán.*

Các transaction đa đối tượng yêu cầu một phương thức nào đó để xác định các thao tác đọc và ghi nào thuộc về cùng một transaction. Trong các cơ sở dữ liệu quan hệ, điều đó thường được thực hiện dựa trên kết nối TCP của client tới máy chủ cơ sở dữ liệu. Trên bất kỳ kết nối cụ thể nào, mọi thứ nằm giữa một câu lệnh BEGIN

TRANSACTION và một câu lệnh COMMIT đều được coi là một phần của cùng một transaction. Nếu kết nối TCP bị gián đoạn, transaction đó phải bị hủy bỏ.

Mặt khác, nhiều cơ sở dữ liệu phi quan hệ không có phương thức như vậy để nhóm các thao tác lại với nhau. Ngay cả khi có một multi-object API (ví dụ: một key-value store có thể có thao tác *multi-put* để cập nhật nhiều key trong một thao tác), điều đó không nhất thiết có nghĩa là nó có ngữ nghĩa transaction: lệnh có thể thành công với một số key nhưng lại thất bại với những key khác, khiến cơ sở dữ liệu rơi vào trạng thái đã được cập nhật một phần.

Ghi đơn đối tượng

Tính atomicity và isolation cũng được áp dụng khi một đối tượng duy nhất đang bị thay đổi. Ví dụ, hãy tưởng tượng bạn đang ghi một tài liệu JSON dung lượng 20 kB vào một cơ sở dữ liệu:

- Nếu kết nối mạng bị gián đoạn sau khi 10 kB đầu tiên đã được gửi đi, liệu cơ sở dữ liệu có lưu trữ mảnh JSON 10 kB không thể phân tích đó không?
- Nếu mất điện trong khi cơ sở dữ liệu đang ở giữa quá trình ghi đề giá trị trước đó trên đĩa, liệu bạn có nhận được kết quả là các giá trị cũ và mới bị chấp vá lại với nhau không?
- Nếu một client khác đọc tài liệu đó trong khi quá trình ghi đang diễn ra, liệu nó có thấy một giá trị đã được cập nhật một phần không?

Mỗi kết quả đó đều sẽ cực kỳ gây bối rối, vì vậy các storage engine hầu như luôn hướng tới việc cung cấp tính atomicity và isolation ở cấp độ một đối tượng duy nhất (chẳng hạn như một cặp key-value) trên một node. Tính atomicity có thể được triển khai bằng cách sử dụng một log để phục hồi sau sự cố (xem “Making B-trees reliable”), và tính isolation có thể được triển khai bằng cách sử dụng một lock trên mỗi đối tượng (chỉ cho phép một thread truy cập vào một đối tượng tại bất kỳ thời điểm nào).

Một số cơ sở dữ liệu cũng cung cấp các thao tác nguyên tử phức tạp hơn, chẳng hạn như thao tác increment, giúp loại bỏ nhu cầu về một chu trình read-modify-write như trong Hình 8-1. Một thao tác cũng phổ biến không kém là *conditional write*, cho phép một thao tác ghi chỉ xảy ra nếu giá trị đó chưa bị thay đổi đồng thời bởi người khác (xem “Conditional writes (compare-and-set)”), tương tự như một thao tác compare-and-set hoặc compare-and-swap (CAS) trong concurrency bộ nhớ dùng chung (shared-memory concurrency).

LƯU Ý

Nói một cách chính xác, thuật ngữ *atomic increment* sử dụng từ *atomic* theo nghĩa của lập trình đa luồng (multithreaded programming). Trong ngữ cảnh của ACID, nó nên được gọi là một thao tác increment *isolated* hoặc *serializable*, nhưng đó không phải là thuật ngữ thông thường.

Các thao tác đơn đối tượng này rất hữu ích, vì chúng có thể ngăn chặn các lost updates khi nhiều client cố gắng ghi vào cùng một đối tượng một cách đồng thời (xem “Preventing Lost Updates”). Tuy nhiên, chúng không phải là các transaction theo nghĩa thông thường của từ này. Ví dụ, chế độ “strong consistency” của Aerospike và tính năng “lightweight transactions” của Cassandra và ScyllaDB cung cấp các lượt đọc linearizable (xem “Linearizability”) và các conditional writes trên một đối tượng duy nhất, nhưng không có sự đảm bảo trên nhiều đối tượng.

Nhu cầu về các transaction đa đối tượng

Liệu chúng ta có thực sự cần các transaction đa đối tượng không? Liệu có khả thi để triển khai bất kỳ ứng dụng nào chỉ với mô hình dữ liệu key-value và các thao tác đơn đối tượng?

Trong một số trường hợp sử dụng, các thao tác insert, update và delete đơn đối tượng là đủ. Tuy nhiên, trong nhiều trường hợp khác, việc ghi vào nhiều đối tượng cần phải được điều phối:

- Trong một mô hình dữ liệu quan hệ, một hàng trong một bảng thường có một tham chiếu foreign-key tới một hàng trong bảng khác. Tương tự, trong một mô hình dữ liệu dạng đồ thị (graph-like), một vertex có các edge tới các vertex khác. Các transaction đa đối tượng cho phép bạn đảm bảo rằng các tham chiếu này vẫn hợp lệ; khi chèn nhiều bản ghi tham chiếu lẫn nhau, các foreign keys phải chính xác và được cập nhật, nếu không dữ liệu sẽ trở nên vô nghĩa.
- Trong một document model, các field cần được cập nhật cùng nhau thường nằm trong cùng một document, vốn được xử lý như một đối tượng duy nhất; không cần đến các transaction đa đối tượng khi cập nhật một document đơn lẻ. Tuy nhiên, các database thiếu chức năng join cũng khuyến khích việc denormalization (xem mục “When to Use Which Model”). Khi thông tin đã denormalized cần được cập nhật, như trong ví dụ ở Hình 8-2, bạn cần cập nhật nhiều document cùng một lúc. Các transaction rất hữu ích trong tình huống này để ngăn chặn dữ liệu denormalized bị mất đồng bộ.
- Trong các database có secondary index (gần như mọi thứ ngoại trừ các pure key-value stores), các index cũng cần được cập nhật mỗi khi bạn thay đổi một giá trị. Xét từ góc độ transaction, các index này là những database object khác nhau—ví dụ, nếu không có transaction isolation, một bản ghi có thể xuất hiện trong index

này nhưng lại không có trong index kia vì việc cập nhật cho index thứ hai chưa diễn ra (xem mục “Sharding and Secondary Indexes”).

Những ứng dụng như vậy vẫn có thể được triển khai mà không cần transaction. Tuy nhiên, việc xử lý lỗi sẽ trở nên phức tạp hơn nhiều nếu không có atomicity, và việc thiếu isolation có thể gây ra các vấn đề về concurrency. Chúng ta sẽ thảo luận về những vấn đề đó trong mục “Weak Isolation Levels” và khám phá các phương pháp thay thế trong Chương 13.

Xử lý lỗi và abort

Một tính năng then chốt của một transaction là nó có thể bị abort và được thử lại một cách an toàn nếu có lỗi xảy ra. Các ACID database dựa trên triết lý này: nếu database có nguy cơ vi phạm các đảm bảo về atomicity, isolation, hoặc durability, nó thà từ bỏ hoàn toàn transaction đó còn hơn là để nó ở trạng thái chưa hoàn tất.

Tuy nhiên, không phải tất cả các hệ thống đều tuân theo triết lý đó. Cụ thể, các datastore với leaderless replication (xem mục “Leaderless Replication”) hoạt động theo kiểu “best effort” (nỗ lực tối đa), có thể tóm tắt là “database sẽ làm nhiều nhất có thể, và nếu nó gặp lỗi, nó sẽ không hoàn tác những gì nó đã thực hiện”—vì vậy, trách nhiệm khôi phục sau lỗi thuộc về ứng dụng.

Lỗi chắc chắn sẽ xảy ra, nhưng nhiều lập trình viên phần mềm thích chỉ nghĩ về happy path thay vì những sự phức tạp của việc xử lý lỗi. Ví dụ, các framework ORM (object-relational mapping) phổ biến như Rails ActiveRecord và Django không thử lại các transaction đã bị abort—lỗi thường dẫn đến một exception bị đẩy lên stack, vì vậy bất kỳ đầu vào nào của người dùng cũng bị vứt bỏ, và người dùng nhận được một thông báo lỗi. Điều này thật đáng tiếc, vì mục đích chính của việc rollback các transaction là để cho phép thực hiện các lần thử lại an toàn.

Mặc dù thử lại một transaction đã bị abort là một cơ chế xử lý lỗi đơn giản và hiệu quả, nhưng nó không hoàn hảo:

- Nếu transaction thực sự đã thành công, nhưng mạng bị gián đoạn trong khi server đang cố gắng xác nhận commit thành công tới client (dẫn đến việc nó bị timeout từ góc nhìn của client), thì việc thử lại transaction sẽ khiến nó bị thực hiện hai lần, trừ khi bạn có một cơ chế deduplication ở cấp độ ứng dụng.
- Nếu lỗi là do quá tải hoặc do sự tranh chấp (contention) cao giữa các transaction đồng thời, việc thử lại transaction sẽ làm vấn đề trở nên tồi tệ hơn chứ không tốt lên. Để tránh các vòng lặp phản hồi như vậy, bạn có thể giới hạn số lần thử lại, sử dụng exponential backoff, và xử lý các lỗi liên quan đến quá tải khác với các lỗi khác (xem mục “When an Overloaded System Won’t Recover”).

- Chỉ nên thử lại sau các lỗi tạm thời (ví dụ: do deadlock, vi phạm isolation, gián đoạn mạng tạm thời, hoặc failover). Sau một lỗi vĩnh viễn (ví dụ: vi phạm constraint), việc thử lại sẽ trở nên vô nghĩa.
- Nếu transaction cũng có các side effect bên ngoài cơ sở dữ liệu, những side effect đó có thể xảy ra ngay cả khi transaction bị abort. Ví dụ, nếu bạn đang gửi một email, bạn sẽ không muốn gửi lại email đó mỗi khi bạn retry transaction. Nếu bạn muốn đảm bảo rằng nhiều hệ thống cùng commit hoặc abort cùng nhau, two-phase commit có thể giúp ích (chúng ta sẽ thảo luận về vấn đề này trong mục “Two-Phase Commit”).
- Nếu tiến trình client bị crash trong khi đang retry, bất kỳ dữ liệu nào mà nó đang cố gắng ghi vào cơ sở dữ liệu đều sẽ bị mất.

Các Isolation Level yếu

Nếu hai transaction không truy cập cùng một dữ liệu, hoặc nếu cả hai đều là read-only, chúng có thể được chạy song song một cách an toàn vì không bên nào phụ thuộc vào bên nào. Các vấn đề về concurrency (race conditions) chỉ nảy sinh khi một transaction đọc dữ liệu đang bị sửa đổi đồng thời bởi một transaction khác, hoặc khi hai transaction cố gắng sửa đổi cùng một dữ liệu.

Các lỗi concurrency rất khó tìm thấy thông qua kiểm thử, bởi vì những lỗi như vậy chỉ bị kích hoạt khi bạn không may gặp phải vấn đề về timing. Những vấn đề về timing như vậy có thể hiếm khi xảy ra và thường rất khó để tái hiện. Concurrency cũng rất khó để suy luận, đặc biệt là trong một ứng dụng lớn nơi bạn không nhất thiết biết được những phần mã nguồn khác nào đang truy cập vào cơ sở dữ liệu. Phát triển ứng dụng đã đủ khó nếu bạn chỉ có một người dùng tại một thời điểm; việc có nhiều người dùng đồng thời khiến nó còn khó khăn hơn nhiều, vì bất kỳ phần dữ liệu nào cũng có thể thay đổi bất ngờ vào bất kỳ lúc nào.

Vì lý do đó, các cơ sở dữ liệu từ lâu đã cố gắng che giấu các vấn đề concurrency khỏi các lập trình viên ứng dụng bằng cách cung cấp *transaction isolation*. Về lý thuyết, isolation giúp cuộc sống của bạn dễ dàng hơn bằng cách cho phép bạn giả định rằng không có concurrency nào đang diễn ra; isolation kiểu *serializable* có nghĩa là cơ sở dữ liệu đảm bảo rằng các transaction có hiệu ứng tương tự như thể chúng được chạy *serially* (tức là, thực hiện từng cái một, không có bất kỳ sự concurrency nào).

Trong thực tế, đáng tiếc là isolation không đơn giản đến thế. Isolation kiểu serializable có một cái giá về hiệu năng, và nhiều cơ sở dữ liệu không muốn trả cái giá đó [10]. Do đó, các hệ thống thường sử dụng các mức isolation yếu hơn, vốn chỉ bảo vệ chống lại *một số* vấn đề concurrency chứ không phải tất cả. Những mức isolation đó khó hiểu hơn nhiều và có thể dẫn đến các lỗi tinh vi, nhưng chúng vẫn được sử dụng trong thực tế [30].

Các lỗi concurrency gây ra bởi transaction isolation yếu và race conditions không chỉ là một vấn đề lý thuyết. Chúng đã gây ra tổn thất tiền bạc đáng kể, bao gồm cả việc làm phá sản một sàn giao dịch Bitcoin [31, 32, 33, 34], dẫn đến các cuộc điều tra bởi các kiểm toán viên tài chính [35], và khiến dữ liệu khách hàng bị hư hỏng [36]. Một bình luận phổ biến về những tiết lộ của các vấn đề như vậy là “Hãy sử dụng một cơ sở dữ liệu ACID nếu bạn đang xử lý dữ liệu tài chính!”—nhưng điều đó đã đi chệch trọng tâm. Ngay cả nhiều hệ thống cơ sở dữ liệu quan hệ phổ biến (vốn thường được coi là ACID) cũng sử dụng isolation yếu, vì vậy chúng không nhất thiết sẽ ngăn chặn được các lỗi này xảy ra.

LƯU Ý

Tình cờ thay, phần lớn hệ thống ngân hàng dựa vào các tệp văn bản được trao đổi qua secure FTP [37]. Trong bối cảnh này, việc có một audit trail và một số biện pháp ngăn ngừa gian lận ở cấp độ con người thực tế còn quan trọng hơn các thuộc tính ACID.

Những ví dụ đó cũng làm nổi bật một điểm quan trọng: ngay cả khi các vấn đề concurrency hiếm khi xảy ra trong vận hành bình thường, bạn vẫn phải xem xét khả năng một kẻ tấn công có thể cố tình gửi một loạt các yêu cầu có tính concurrency cao đến API của bạn nhằm nỗ lực khai thác các lỗi concurrency [32]. Do đó, để xây dựng các ứng dụng có độ tin cậy và bảo mật, bạn phải đảm bảo rằng các lỗi như vậy được ngăn chặn một cách có hệ thống.

Trong mục này, chúng ta sẽ xem xét một số isolation level yếu (không có tính serializability) thường được sử dụng trong thực tế và thảo luận chi tiết về các loại race condition có thể hoặc không thể xảy ra với từng mức, để bạn có thể quyết định mức nào phù hợp với ứng dụng của mình. Sau khi hoàn tất, chúng ta sẽ thảo luận chi tiết về serializability (xem mục “Serializability”). Phần thảo luận về các isolation level của chúng ta sẽ mang tính chất không chính thức và sử dụng các ví dụ. Nếu bạn muốn có các định nghĩa và phân tích chặt chẽ về các đặc tính của chúng, bạn có thể tìm thấy trong các tài liệu học thuật [38, 39, 40, 41].

Read Committed

Mức độ isolation của transaction cơ bản nhất là *read committed*, và nó đưa ra hai đảm bảo:

- Khi đọc từ cơ sở dữ liệu, bạn sẽ chỉ thấy dữ liệu đã được commit (không có dirty read).
- Khi ghi vào cơ sở dữ liệu, bạn sẽ chỉ ghi đè lên dữ liệu đã được commit (không có dirty write).

Hãy thảo luận chi tiết hơn về hai đảm bảo này.

Không có dirty read

Hãy tưởng tượng một transaction đã ghi một số dữ liệu vào cơ sở dữ liệu, nhưng transaction đó vẫn chưa commit hoặc abort. Liệu một transaction khác có thể nhìn thấy dữ liệu chưa được commit đó không? Nếu có, đó được gọi là một *dirty read* [3].

Các transaction chạy ở isolation level read-committed phải ngăn chặn được dirty read. Điều này có nghĩa là bất kỳ thao tác ghi nào của một transaction chỉ trở nên hiển thị với các transaction khác khi transaction đó commit (và khi đó tất cả các thao tác ghi của nó sẽ hiển thị cùng một lúc). Điều này được minh họa trong Hình 8-4, nơi người dùng 1 đã thiết lập $x = 3$, nhưng lệnh `get x` của người dùng 2 vẫn trả về giá trị cũ là 2, trong khi người dùng 1 vẫn chưa commit.

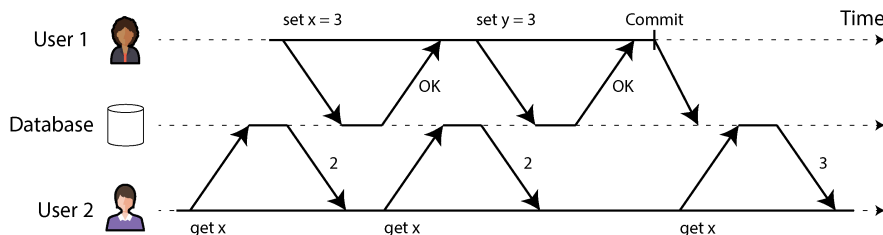


Figure 8-4. **Không có dirty read:** người dùng 2 chỉ thấy giá trị mới của x sau khi transaction của người dùng 1 đã commit

Ngăn chặn dirty read hữu ích vì một vài lý do:

- Nếu một transaction cần cập nhật nhiều hàng, dirty read có nghĩa là một transaction khác có thể thấy một số thay đổi nhưng lại không thấy những thay đổi khác. Ví dụ, trong Hình 8-2, người dùng thấy email mới chưa đọc nhưng không thấy bộ đếm đã được cập nhật. Đây là một dirty read của email. Việc nhìn thấy cơ sở dữ liệu ở trạng thái chỉ mới được cập nhật một phần sẽ gây nhầm lẫn cho người dùng và có thể khiến các transaction khác đưa ra các quyết định sai lầm.
- Nếu một transaction bị hủy (abort), bất kỳ thao tác ghi nào mà nó đã thực hiện đều cần được rollback (như trong Hình 8-3). Nếu cơ sở dữ liệu cho phép dirty reads, một transaction có thể thấy dữ liệu mà sau đó bị rollback—tức là dữ liệu chưa bao giờ thực sự được commit vào cơ sở dữ liệu. Bất kỳ transaction nào đã đọc dữ liệu chưa được commit cũng sẽ cần phải bị hủy, dẫn đến một vấn đề gọi là *cascading aborts* (hủy bỏ dây chuyền).

Không có dirty write

Điều gì sẽ xảy ra nếu hai transaction đồng thời cố gắng cập nhật cùng một hàng trong một cơ sở dữ liệu? Chúng ta không biết các thao tác ghi sẽ xảy ra theo thứ tự nào, nhưng thông thường ta giả định rằng thao tác ghi sau sẽ ghi đè lên thao tác ghi trước.

Tuy nhiên, điều gì sẽ xảy ra nếu thao tác ghi trước là một phần của một transaction chưa được commit, dẫn đến việc thao tác ghi sau ghi đè lên một giá trị chưa được commit? Điều này được gọi là *dirty write* [38]. Các transaction chạy ở isolation level read-committed phải ngăn chặn dirty writes, thường bằng cách trì hoãn thao tác ghi thứ hai cho đến khi transaction của thao tác ghi thứ nhất đã được commit hoặc bị hủy.

Bằng cách ngăn chặn dirty writes, isolation level này tránh được một số loại vấn đề về concurrency:

- Nếu các transaction cập nhật nhiều hàng, dirty writes có thể dẫn đến một kết quả tồi tệ. Ví dụ, hãy xét Hình 8-5, minh họa một trang web bán xe cũ nơi hai người, Aaliyah và Bryce, đang đồng thời cố gắng mua cùng một chiếc xe. Việc mua một chiếc xe yêu cầu hai thao tác ghi vào cơ sở dữ liệu: thông tin đăng bán trên trang web cần được cập nhật để phản ánh người mua, và hóa đơn bán hàng cần được gửi cho người mua. Trong trường hợp của Hình 8-5, việc bán xe được trao cho Bryce (vì anh ấy thực hiện thao tác ghi chiến thắng vào bảng `listings`), nhưng hóa đơn lại được gửi cho Aaliyah (vì cô ấy thực hiện thao tác ghi chiến thắng vào bảng `invoices`). Isolation level read-committed ngăn chặn những sai sót như vậy.
- Tuy nhiên, isolation level read-committed không ngăn chặn race condition giữa hai lần tăng bộ đếm trong Hình 8-1. Trong trường hợp này, thao tác ghi thứ hai xảy ra sau khi transaction thứ nhất đã được commit, vì vậy nó không phải là một dirty write. Nó vẫn sai, nhưng vì một lý do khác; trong mục “Ngăn chặn Lost Updates”, chúng ta sẽ thảo luận cách để thực hiện các lần tăng bộ đếm như vậy một cách an toàn.

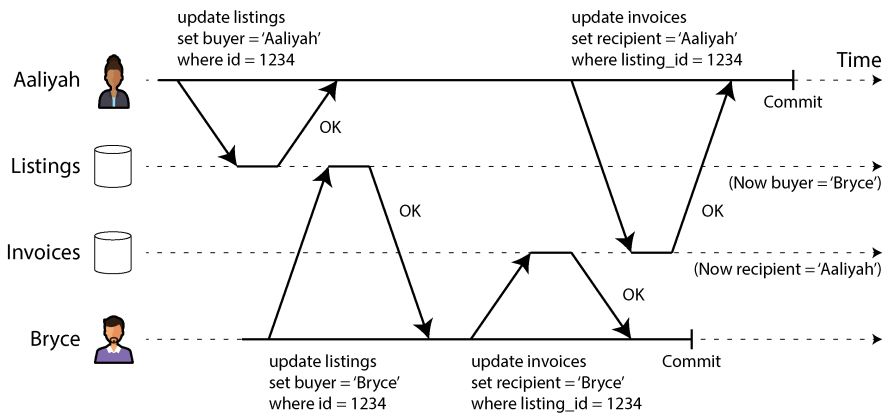


Figure 8-5. Với **dirty write**, các lệnh ghi xung đột từ các **transaction** khác nhau có thể bị trộn lẫn với nhau.

Triển khai read-committed

read-committed là một isolation level rất phổ biến. Nó là thiết lập mặc định trong Oracle Database, PostgreSQL, SQL Server và nhiều cơ sở dữ liệu khác [10].

Thông thường nhất, các cơ sở dữ liệu ngăn chặn dirty writes bằng cách sử dụng row-level locks. Khi một transaction muốn sửa đổi một hàng cụ thể (hoặc document hoặc một đối tượng nào đó), trước tiên nó phải giành được một lock trên hàng đó. Sau đó, nó phải giữ lock đó cho đến khi transaction được commit hoặc bị abort. Chỉ có duy nhất một transaction có thể giữ lock cho bất kỳ hàng nào; nếu một transaction khác muốn ghi vào cùng một hàng, nó phải đợi cho đến khi transaction đầu tiên được commit hoặc bị abort trước khi có thể giành được lock và tiếp tục. Việc locking này được các cơ sở dữ liệu thực hiện tự động trong chế độ read-committed (hoặc ở các isolation level mạnh hơn).

Làm thế nào để chúng ta ngăn chặn dirty reads? Một lựa chọn là sử dụng cùng một lock và yêu cầu bất kỳ transaction nào muốn đọc một hàng phải giành được lock trong thời gian ngắn, sau đó giải phóng nó ngay lập tức sau khi đọc xong. Điều này sẽ đảm bảo rằng việc đọc không thể xảy ra trong khi một hàng đang có giá trị dirty chưa được commit (vì trong thời gian đó, lock sẽ bị giữ bởi transaction đang thực hiện việc ghi).

Tuy nhiên, cách tiếp cận yêu cầu read locks không hoạt động tốt trong thực tế, bởi vì một write transaction chạy lâu có thể buộc nhiều transaction khác phải chờ cho đến khi transaction chạy lâu đó hoàn tất, ngay cả khi các transaction kia chỉ đọc mà không ghi bất cứ thứ gì vào cơ sở dữ liệu. Điều này gây hại cho response time của các transaction chỉ đọc và gây ảnh hưởng xấu đến khả năng vận hành: sự chậm trễ ở một phần của ứng dụng có thể gây ra hiệu ứng dây chuyền đến một phần hoàn toàn khác của ứng dụng do phải chờ đợi các lock.

Mặc dù vậy, các lock vẫn được sử dụng để ngăn chặn dirty reads trong một số cơ sở dữ liệu, chẳng hạn như IBM Db2 và Microsoft SQL Server với thiết lập `read_committed_snapshot=off` [30].

Một cách tiếp cận phổ biến hơn để ngăn chặn dirty reads được minh họa trong Hình 8-4. Đối với mỗi hàng được ghi, cơ sở dữ liệu ghi nhớ cả giá trị đã được commit cũ và giá trị mới được thiết lập bởi transaction hiện đang giữ write lock. Trong khi transaction đang diễn ra, bất kỳ transaction nào khác đọc hàng đó đơn giản là sẽ được cung cấp giá trị cũ. Chỉ khi giá trị mới được commit, các transaction mới chuyển sang đọc giá trị mới (xem mục “Multiversion concurrency control” để biết thêm chi tiết).

Một số cơ sở dữ liệu hỗ trợ một isolation level thậm chí còn yếu hơn gọi là *read uncommitted*. Nó ngăn chặn dirty writes nhưng không ngăn chặn dirty reads. Nói cách khác, nó trả về ngay lập tức giá trị được ghi mới nhất, ngay cả khi transaction đang ghi vẫn chưa được commit. Điều này có thể mang lại hiệu năng tốt hơn, vì cơ sở dữ liệu không cần phải lưu trữ hai phiên bản của hàng đó. Nó cũng có thể làm giảm xác suất xảy ra (nhưng không ngăn chặn hoàn toàn) lost updates, điều mà chúng ta sẽ thảo luận trong mục “Preventing Lost Updates”.

Snapshot Isolation và Repeatable Read

Nếu bạn nhìn lướt qua isolation của read-committed, bạn có thể lầm tưởng rằng nó thực hiện mọi thứ mà một transaction cần làm: nó cho phép abort (cần thiết cho tính atomicity), nó ngăn chặn việc đọc các kết quả chưa hoàn tất của các transaction, và nó ngăn chặn các write đồng thời bị trộn lẫn vào nhau. Thật vậy, đó là những tính năng hữu ích, và chúng là những đảm bảo mạnh mẽ hơn nhiều so với những gì bạn có thể nhận được từ một hệ thống không hỗ trợ transaction.

Tuy nhiên, vẫn còn rất nhiều cách để gặp phải các lỗi concurrency khi sử dụng isolation level này. Ví dụ, Hình 8-6 minh họa một vấn đề có thể xảy ra với isolation của read-committed.

Giả sử Aaliyah có \$1.000 khoản tiết kiệm tại một ngân hàng, được chia đều vào hai tài khoản, mỗi tài khoản có \$500. Một transaction chuyển \$100 từ một trong hai tài khoản của cô sang tài khoản còn lại. Nếu cô không may nhìn vào danh sách số dư tài khoản ngay tại thời điểm transaction đó đang được xử lý, cô có thể thấy số dư của một tài khoản trước khi khoản thanh toán đến được ghi nhận (vẫn là \$500) và tài khoản kia sau khi lệnh chuyển tiền đi đã được thực hiện (số dư mới là \$400). Đối với Aaliyah, hiện tại có vẻ như cô chỉ còn tổng cộng \$900 trong các tài khoản của mình — có vẻ như \$100 đã tan biến vào hư không.

Sự bất thường này được gọi là *read skew*, và nó là một ví dụ về *nonrepeatable read*: nếu Aaliyah đọc lại số dư của tài khoản 1 vào cuối transaction, cô sẽ thấy một giá trị khác (\$600) so với giá trị cô đã thấy trong truy vấn trước đó. Read skew được coi là

có thể chấp nhận được dưới isolation level `read committed`: các số dư tài khoản mà Aaliyah đã thấy thực sự đã được commit tại thời điểm cô đọc chúng.

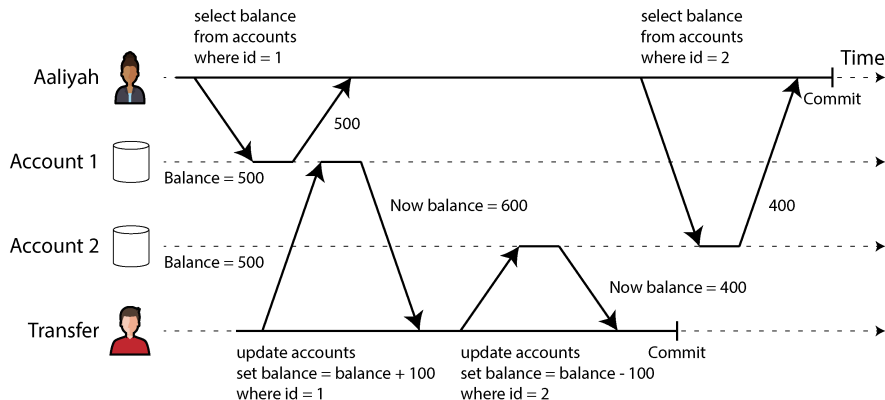


Figure 8-6. *Read skew: Aaliyah quan sát thấy cơ sở dữ liệu đang ở trạng thái không nhất quán*

LƯU Ý

Thuật ngữ *skew* thật không may là bị dùng quá nhiều nghĩa. Trước đó, ta đã sử dụng nó với nghĩa là *khối lượng công việc không cân bằng với các hot spot* (xem “Skewed Workloads and Relieving Hot Spots”), trong khi ở đây nó có nghĩa là một *sự bất thường về thời điểm (timing anomaly)*.

Trong trường hợp của Aaliyah, đây không phải là một vấn đề kéo dài, bởi vì cô ấy rất có thể sẽ thấy số dư tài khoản nhất quán nếu cô ấy tải lại trang web ngân hàng trực tuyến sau đó vài giây. Tuy nhiên, sự không nhất quán tạm thời như vậy là không thể chấp nhận được trong các trường hợp sau, ví dụ:

Backup

Việc thực hiện backup yêu cầu tạo một bản sao của toàn bộ cơ sở dữ liệu, điều này có thể mất nhiều giờ đối với một cơ sở dữ liệu lớn. Trong thời gian quá trình backup đang chạy, các thao tác ghi vẫn sẽ tiếp tục được thực hiện vào cơ sở dữ liệu. Do đó, bạn có thể kết thúc với việc một số phần của bản backup chứa phiên bản dữ liệu cũ hơn và các phần khác chứa phiên bản mới hơn. Nếu bạn cần khôi phục từ một bản backup như vậy, các sự không nhất quán (chẳng hạn như tiền bị biến mất) sẽ trở thành vĩnh viễn.

Các truy vấn phân tích và kiểm tra tính toàn vẹn

Đôi khi bạn có thể muốn chạy một truy vấn quét qua các phần lớn của cơ sở dữ liệu. Những truy vấn như vậy rất phổ biến trong phân tích (xem “Operational Versus Analytical Systems”), hoặc chúng có thể là một phần của quá trình kiểm tra tính toàn vẹn định kỳ để đảm bảo mọi thứ đều ổn thỏa (giám sát việc hư hỏng

dữ liệu). Những truy vấn này có khả năng trả về các kết quả vô nghĩa nếu chúng quan sát các phần của cơ sở dữ liệu tại các thời điểm khác nhau.

Snapshot isolation [38] là giải pháp phổ biến nhất cho vấn đề này. Ý tưởng là mỗi transaction sẽ đọc từ một *snapshot nhất quán* của cơ sở dữ liệu—nghĩa là, nó thấy tất cả dữ liệu đã được commit trong cơ sở dữ liệu tại thời điểm bắt đầu transaction đó. Ngay cả khi dữ liệu sau đó bị thay đổi bởi một transaction khác, mỗi transaction vẫn chỉ thấy dữ liệu cũ tại thời điểm cụ thể đó.

Snapshot isolation là một trợ thủ đắc lực cho các truy vấn chỉ đọc (read-only) chạy lâu như backup và phân tích. Rất khó để suy luận về ý nghĩa của một truy vấn nếu dữ liệu mà nó hoạt động đang thay đổi cùng lúc với lúc truy vấn đang thực thi. Khi một transaction có thể thấy một snapshot nhất quán của cơ sở dữ liệu, được đóng băng tại một thời điểm cụ thể, việc hiểu nó sẽ dễ dàng hơn nhiều.

Snapshot isolation là một tính năng phổ biến: các biến thể của nó được hỗ trợ bởi PostgreSQL, MySQL với storage engine InnoDB, Oracle, SQL Server và các hệ thống khác, mặc dù hành vi chi tiết sẽ khác nhau giữa các hệ thống [30, 42, 43]. Một số cơ sở dữ liệu, chẳng hạn như Oracle, TiDB và Aurora DSQL, thậm chí chọn snapshot isolation làm isolation level cao nhất của chúng. Các data warehouse trên đám mây như BigQuery cũng thường xuyên sử dụng snapshot isolation, vì nó cung cấp một cái nhìn tại một thời điểm (point-in-time view) của cơ sở dữ liệu cho các truy vấn phân tích.

MVCC

Tương tự như với read-committed isolation, các triển khai của snapshot isolation thường sử dụng write lock để ngăn chặn dirty writes (xem “Implementing read-committed”), điều này có nghĩa là một transaction thực hiện ghi có thể chặn tiến trình của một transaction khác đang ghi vào cùng một hàng. Tuy nhiên, các thao tác đọc không yêu cầu bất kỳ lock nào. Từ góc độ hiệu năng, một nguyên tắc then chốt của snapshot isolation là *người đọc không bao giờ chặn người ghi, và người ghi không bao giờ chặn người đọc*. Điều này cho phép một cơ sở dữ liệu xử lý các truy vấn đọc chạy lâu trên một snapshot nhất quán đồng thời với việc xử lý các thao tác ghi một cách bình thường, mà không có bất kỳ sự tranh chấp lock nào giữa cả hai.

Để triển khai snapshot isolation, các cơ sở dữ liệu sử dụng một sự tổng quát hóa của cơ chế mà chúng ta đã thấy để ngăn chặn dirty reads trong Hình 8-4. Thay vì chỉ có hai phiên bản của mỗi hàng (phiên bản đã commit và phiên bản đã bị ghi đè nhưng chưa được commit), cơ sở dữ liệu có khả năng phải giữ lại nhiều phiên bản đã commit của một hàng, bởi vì các transaction đang diễn ra khác nhau có thể cần thấy trạng thái của cơ sở dữ liệu tại các thời điểm khác nhau. Vì nó duy trì nhiều phiên bản của một hàng song song với nhau, kỹ thuật này được gọi là *multiversion concurrency control* (MVCC).

Hình 8-7 minh họa cách thức snapshot isolation dựa trên MVCC được triển khai trong PostgreSQL [42, 44, 45] (các cách triển khai khác cũng tương tự). Khi một transaction được bắt đầu, nó được cấp một transaction ID duy nhất và luôn tăng dần (txid). Bất cứ khi nào một transaction ghi bất kỳ thứ gì vào cơ sở dữ liệu, dữ liệu mà nó ghi sẽ được gắn thẻ với transaction ID của bên thực hiện ghi. (Để chính xác hơn, các transaction ID trong PostgreSQL là các số nguyên 32-bit, vì vậy chúng sẽ bị tràn sau khoảng 4 tỷ transaction. Tiến trình vacuum thực hiện dọn dẹp để đảm bảo rằng việc tràn này không ảnh hưởng đến dữ liệu.)

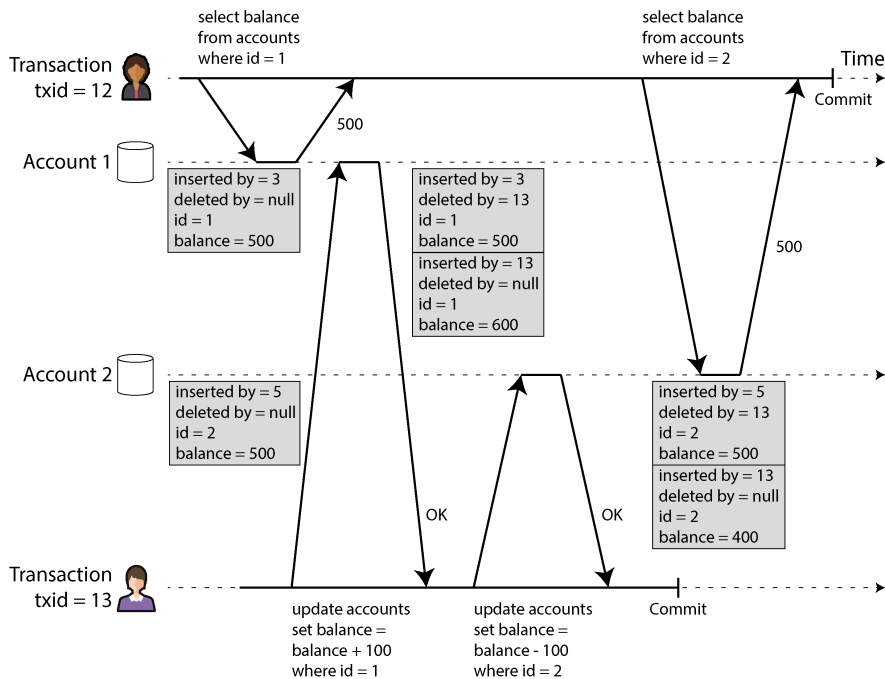


Figure 8-7. Hình 8.x: Triển khai snapshot isolation sử dụng MVCC

Mỗi hàng trong một bảng có một trường `inserted_by`, chứa ID của transaction đã chèn hàng đó vào bảng. Mỗi hàng cũng có một trường `deleted_by`, ban đầu sẽ trống. Nếu một transaction xóa một hàng, hàng đó không bị loại bỏ khỏi cơ sở dữ liệu mà thay vào đó được đánh dấu để xóa bằng cách đặt trường `deleted_by` thành ID của transaction đã yêu cầu việc xóa. Tại một thời điểm sau đó, khi chắc chắn rằng không còn transaction nào có thể truy cập vào dữ liệu đã bị xóa hoặc bị ghi đè, một quá trình `garbage collection` (GC) trong cơ sở dữ liệu sẽ loại bỏ bất kỳ hàng nào đã được đánh dấu để xóa và giải phóng không gian của chúng.

Một thao tác cập nhật về mặt nội bộ được chuyển đổi thành một thao tác xóa và một thao tác chèn [46]. Ví dụ, trong Hình 8-7, transaction 13 khấu trừ 100\$ từ tài

khoản 2, thay đổi số dư từ 500\$ thành 400\$. Bảng accounts hiện chứa hai hàng cho tài khoản 2: một hàng với số dư 500\$ đã được đánh dấu là bị xóa bởi transaction 13, và một hàng với số dư 400\$ đã được chèn bởi transaction 13.

Tất cả các phiên bản của một hàng đều được lưu trữ trong cùng một database heap (xem mục “Storing Values Within the Index”), bất kể các transaction đã ghi chúng đã commit hay chưa. Các phiên bản của cùng một hàng tạo thành một linked list, đi từ phiên bản mới nhất đến cũ nhất hoặc ngược lại, để các truy vấn có thể lập qua tất cả các phiên bản của một hàng về mặt nội bộ [47, 48].

Các quy tắc hiển thị để quan sát một snapshot nhất quán

Khi một transaction đọc từ cơ sở dữ liệu, các transaction ID được sử dụng để quyết định phiên bản hàng nào nó có thể thấy và phiên bản nào là không thể thấy. Bằng cách định nghĩa cẩn thận các quy tắc hiển thị, cơ sở dữ liệu có thể trình bày một snapshot nhất quán về nội dung của nó cho ứng dụng. Quá trình này hoạt động đại khái như sau [45]:

1. Tại thời điểm bắt đầu mỗi transaction, cơ sở dữ liệu lập một danh sách tất cả các transaction khác đang trong quá trình thực hiện (chưa commit hoặc chưa abort) tại thời điểm đó. Bất kỳ thao tác ghi nào mà các transaction đó đã thực hiện đều bị bỏ qua, ngay cả khi các transaction đó sau đó commit. Điều này đảm bảo rằng ứng dụng thấy một snapshot nhất quán mà không bị ảnh hưởng bởi việc một transaction khác commit.
2. Bất kỳ thao tác ghi nào được thực hiện bởi các transaction có transaction ID muộn hơn (tức là, bắt đầu sau khi transaction hiện tại bắt đầu, và do đó không được bao gồm trong danh sách các transaction đang thực hiện) đều bị bỏ qua, bất kể các transaction đó đã commit hay chưa.
3. Bất kỳ thao tác ghi nào được thực hiện bởi các transaction đã bị abort đều bị bỏ qua, bất kể việc abort xảy ra khi nào. Điều này có lợi thế là khi một transaction bị abort, chúng ta không cần phải loại bỏ ngay lập tức các hàng mà nó đã ghi khỏi bộ lưu trữ, vì quy tắc hiển thị sẽ lọc chúng ra. Quá trình GC có thể loại bỏ chúng sau đó.
4. Tất cả các thao tác ghi khác đều hiển thị đối với các truy vấn của ứng dụng.

Các quy tắc này áp dụng cho cả việc chèn và xóa các hàng. Trong Hình 8-7, khi transaction 12 đọc từ tài khoản 2, nó thấy số dư là 500\$ vì việc xóa số dư 500\$ được thực hiện bởi transaction 13 (theo quy tắc 2, transaction 12 không thể thấy một thao tác xóa được thực hiện bởi transaction 13), và việc chèn số dư 400\$ vẫn chưa hiển thị (theo cùng một quy tắc).

Nói cách khác, một hàng được hiển thị nếu cả hai điều kiện sau đây đều đúng:

- Tại thời điểm transaction của người đọc bắt đầu, transaction đã chèn hàng đó đã commit.
- Hàng đó không bị đánh dấu để xóa, hoặc nếu có, transaction yêu cầu xóa vẫn chưa commit tại thời điểm transaction của người đọc bắt đầu.

Một long-running transaction có thể tiếp tục sử dụng một snapshot trong một thời gian dài, tiếp tục đọc các giá trị mà (từ góc nhìn của các transaction khác) đã bị ghi đè hoặc bị xóa từ lâu. Bằng cách không bao giờ cập nhật giá trị tại chỗ (in place) mà thay vào đó chèn một phiên bản mới mỗi khi một giá trị thay đổi, cơ sở dữ liệu có thể cung cấp một snapshot nhất quán trong khi chỉ chịu một lượng overhead nhỏ.

Indexes và snapshot isolation

Các index hoạt động như thế nào trong một cơ sở dữ liệu multiversion? Cách tiếp cận phổ biến nhất là mỗi entry trong index sẽ trỏ tới một trong các version của một hàng khớp với entry đó (có thể là version cũ nhất hoặc mới nhất). Mỗi version của hàng có thể chứa một tham chiếu tới version cũ hơn tiếp theo hoặc mới hơn tiếp theo. Một truy vấn sử dụng index sau đó phải duyệt qua các hàng để tìm ra hàng khả kiến (visible) và có giá trị khớp với những gì truy vấn đang tìm kiếm. Khi GC loại bỏ các version hàng cũ không còn khả kiến với bất kỳ transaction nào, các entry index tương ứng cũng có thể được loại bỏ.

Nhiều chi tiết triển khai ảnh hưởng đến hiệu năng của MVCC (multi-version concurrency control) [47, 48]. Ví dụ, PostgreSQL có các tối ưu hóa để tránh cập nhật index nếu các version khác nhau của cùng một hàng có thể nằm trên cùng một page [42]. Một số cơ sở dữ liệu khác tránh lưu trữ các bản sao đầy đủ của các hàng đã bị sửa đổi mà chỉ lưu trữ các phần khác biệt giữa các version để tiết kiệm không gian.

Một cách tiếp cận khác được sử dụng trong CouchDB, Datomic và LMDB. Mặc dù chúng cũng sử dụng B-trees (xem mục “B-Trees”), nhưng chúng sử dụng một biến thể *immutable* (copy-on-write) không ghi đè lên các page của cây khi chúng được cập nhật, mà thay vào đó tạo ra một bản sao mới cho mỗi page bị sửa đổi. Các page cha, cho đến tận root của cây, sẽ được sao chép và cập nhật để trỏ tới các version mới của các page con của chúng. Bất kỳ page nào không bị ảnh hưởng bởi một thao tác ghi đều không cần phải sao chép và có thể được chia sẻ với cây mới [49].

Với các B-tree immutable, mỗi write transaction (hoặc một batch các transaction) sẽ tạo ra một root B-tree mới, và một root cụ thể là một snapshot nhất quán của cơ sở dữ liệu tại thời điểm nó được tạo ra. Không cần phải lọc các hàng dựa trên transaction ID vì các thao tác ghi tiếp theo không thể sửa đổi một B-tree hiện có; chúng chỉ có thể tạo ra các root cây mới. Cách tiếp cận này cũng yêu cầu một tiến trình chạy ngầm để thực hiện compaction và GC.

Snapshot isolation, repeatable read, và sự nhầm lẫn về tên gọi

MVCC là một kỹ thuật triển khai được sử dụng phổ biến cho các cơ sở dữ liệu, và thường được dùng để triển khai snapshot isolation. Tuy nhiên, các cơ sở dữ liệu khác nhau đôi khi sử dụng các thuật ngữ khác nhau để chỉ cùng một thứ—ví dụ, snapshot isolation được gọi là “repeatable read” trong PostgreSQL và “serializable” trong Oracle [30]. Ngoài ra, đôi khi các hệ thống khác nhau sử dụng cùng một thuật ngữ nhưng với ý nghĩa khác nhau—ví dụ, trong khi ở PostgreSQL “repeatable read” có nghĩa là snapshot isolation, thì ở MySQL nó có nghĩa là một triển khai MVCC với độ tin cậy yếu hơn snapshot isolation [43], và IBM Db2 sử dụng “repeatable read” để chỉ serializability [10].

Lý do cho sự nhầm lẫn về tên gọi này là vì tiêu chuẩn SQL không có khái niệm snapshot isolation, bởi vì tiêu chuẩn này dựa trên định nghĩa về các isolation level của System R năm 1975 [3] và lúc đó snapshot isolation vẫn chưa được phát minh. Thay vào đó, nó định nghĩa isolation level repeatable read, thứ trông có vẻ tương đồng về mặt bề ngoài với snapshot isolation. PostgreSQL gọi isolation level của nó là repeatable read vì nó đáp ứng các yêu cầu của tiêu chuẩn và do đó có thể tuyên bố tuân thủ tiêu chuẩn.

Thật không may, định nghĩa của tiêu chuẩn SQL về các isolation level là có lỗi—nó mơ hồ, thiếu chính xác và không độc lập với triển khai như một tiêu chuẩn nên có [38]. Mặc dù một số cơ sở dữ liệu triển khai isolation level repeatable read, nhưng có những sự khác biệt lớn trong các đảm bảo mà chúng cung cấp, mặc dù các đảm bảo đó bề ngoài có vẻ đã được tiêu chuẩn hóa [30]. Isolation level này đã được định nghĩa chính thức trong các tài liệu nghiên cứu [39, 40], nhưng hầu hết các triển khai đều không thỏa mãn định nghĩa chính thức đó. Kết quả là, không ai thực sự biết repeatable read isolation có nghĩa là gì.

Ngăn chặn lost update

Cuộc thảo luận của chúng ta về các mức isolation read-committed và snapshot isolation chủ yếu tập trung vào các đảm bảo về những gì một transaction chỉ đọc (read-only) có thể nhìn thấy khi có các thao tác ghi đồng thời. Chúng ta hầu như đã bỏ qua vấn đề về hai transaction thực hiện ghi đồng thời—chúng ta mới chỉ thảo luận về dirty writes (xem “No dirty writes”), một loại xung đột write-write cụ thể có thể xảy ra.

Một vài loại xung đột thú vị khác có thể xảy ra giữa các transaction thực hiện ghi đồng thời. Loại phổ biến nhất trong số này là vấn đề *lost update* (cập nhật bị mất), được minh họa trong Hình 8-1 với ví dụ về hai thao tác tăng biến đếm đồng thời.

Vấn đề *lost update* có thể xảy ra nếu một ứng dụng đọc một giá trị từ cơ sở dữ liệu, sửa đổi nó, và ghi lại giá trị đã sửa đổi (chu kỳ read-modify-write đã đề cập trước đó). Nếu hai transaction thực hiện việc này đồng thời, một trong các thay đổi có

thể bị mất, bởi vì lần ghi thứ hai không bao gồm thay đổi của lần thứ nhất. (Đôi khi chúng ta nói rằng lần ghi sau *clobbers* (ghi đè hoàn toàn) lần ghi trước.) Mô hình này xảy ra trong nhiều kịch bản khác nhau, chẳng hạn như:

- Tăng một biến đếm hoặc cập nhật số dư tài khoản (đòi hỏi phải đọc giá trị hiện tại, tính toán giá trị mới, và ghi lại giá trị đã cập nhật)
- Thực hiện thay đổi cục bộ đối với một giá trị phức tạp—ví dụ, thêm một phần tử vào một danh sách bên trong một tài liệu JSON (đòi hỏi phải parse tài liệu, thực hiện thay đổi, và ghi lại tài liệu đã sửa đổi)
- Hai người dùng cùng chỉnh sửa một trang wiki tại một thời điểm, trong đó mỗi người dùng lưu các thay đổi của họ bằng cách gửi toàn bộ nội dung trang tới máy chủ, ghi đè lên bất cứ thứ gì hiện đang có trong cơ sở dữ liệu

Vì đây là một vấn đề rất phổ biến, nhiều giải pháp khác nhau đã được phát triển [50]. Chúng ta sẽ xem xét những giải pháp phổ biến nhất ở đây.

Các thao tác ghi nguyên tử (Atomic write operations)

Nhiều cơ sở dữ liệu cung cấp các thao tác cập nhật nguyên tử (atomic update operations), giúp loại bỏ nhu cầu phải triển khai các chu kỳ read-modify-write trong mã nguồn ứng dụng. Chúng thường là giải pháp tốt nhất nếu mã của bạn có thể được biểu diễn dưới dạng các thao tác đó. Ví dụ, chỉ lệnh sau đây là an toàn về mặt đồng thời (concurrency-safe) trong hầu hết các cơ sở dữ liệu quan hệ:

```
UPDATE counters SET value = value + 1 WHERE key = 'foo';
```

Tương tự, các cơ sở dữ liệu tài liệu như MongoDB cung cấp các thao tác nguyên tử để thực hiện các thay đổi cục bộ đối với một phần của tài liệu JSON, và Redis cung cấp các thao tác nguyên tử để sửa đổi các cấu trúc dữ liệu như `priority queue`. Không phải tất cả các thao tác ghi đều có thể dễ dàng biểu diễn dưới dạng các thao tác nguyên tử—ví dụ, việc cập nhật một trang wiki bao gồm việc chỉnh sửa văn bản tùy ý, thứ có thể được xử lý bằng các thuật toán được thảo luận trong “Conflict-free replicated datatypes and operational transformation”—nhưng trong những tình huống có thể sử dụng các thao tác này, chúng thường là lựa chọn tốt nhất.

Các thao tác nguyên tử thường được triển khai bằng cách khóa độc quyền (exclusively locking) đối tượng khi nó được đọc để không có transaction nào khác có thể đọc nó cho đến khi việc cập nhật được áp dụng. Một lựa chọn khác là chỉ đơn giản ép buộc tất cả các thao tác nguyên tử phải được thực thi trên một `single thread`.

Thật không may, các framework ORM khiến việc vô tình viết mã thực hiện các chu kỳ read-modify-write không an toàn trở nên dễ dàng thay vì sử dụng các thao tác nguyên tử do cơ sở dữ liệu cung cấp [51, 52, 53]. Điều này có thể là nguồn gốc của những lỗi tinh vi vốn rất khó tìm thấy thông qua kiểm thử.

Khóa tường minh (Explicit locking)

Một lựa chọn khác để ngăn chặn `lost update`, nếu các thao tác nguyên tử tích hợp sẵn của cơ sở dữ liệu không cung cấp chức năng cần thiết, là ứng dụng sẽ thực hiện khóa tường minh các đối tượng sắp được cập nhật. Sau đó, ứng dụng có thể thực hiện một chu kỳ `read-modify-write`, và nếu bất kỳ `transaction` nào khác cố gắng cập nhật hoặc khóa cùng một đối tượng một cách đồng thời, nó sẽ bị buộc phải chờ cho đến khi chu kỳ `read-modify-write` đầu tiên hoàn tất.

Ví dụ, hãy xét một trò chơi nhiều người chơi, trong đó nhiều người chơi có thể di chuyển cùng một quân cờ một cách đồng thời. Trong trường hợp này, một thao tác nguyên tử (atomic operation) có thể là chưa đủ, bởi vì ứng dụng cũng cần đảm bảo rằng nước đi của một người chơi tuân thủ các quy tắc của trò chơi, điều này liên quan đến một số logic mà bạn không thể triển khai một cách hợp lý dưới dạng một truy vấn cơ sở dữ liệu. Thay vào đó, bạn có thể sử dụng một `lock` để ngăn hai người chơi di chuyển cùng một quân cờ một cách đồng thời, như được minh họa trong Ví dụ 8-1.

Example 8-1. Khóa các hàng một cách tường minh để ngăn chặn `lost updates`

```
BEGIN TRANSACTION;

SELECT * FROM figures
  WHERE name = 'robot' AND game_id = 222
  FOR UPDATE;

-- Check whether move is valid, then update the position
-- of the piece that was returned by the previous SELECT.
UPDATE figures SET position = 'c4' WHERE id = 1234;

COMMIT;
```

Mệnh đề `FOR UPDATE` chỉ ra rằng cơ sở dữ liệu nên khóa tất cả các hàng được trả về bởi truy vấn này.

Cách này hoạt động, nhưng để thực hiện đúng, bạn cần suy nghĩ kỹ về logic ứng dụng của mình. Rất dễ quên thêm một `lock` cần thiết ở đâu đó trong mã nguồn và từ đó dẫn đến một `race condition`.

Hơn nữa, việc khóa nhiều đối tượng còn mang theo rủi ro `deadlock`, nơi hai hoặc nhiều `transaction` đang chờ đợi nhau giải phóng `lock` của chúng. Nhiều cơ sở dữ liệu tự động phát hiện `deadlock` và hủy bỏ một trong các `transaction` liên quan để hệ thống có thể tiếp tục tiến triển. Bạn có thể xử lý tình huống này ở cấp độ ứng dụng bằng cách thử lại (retry) `transaction` đã bị hủy.

Tự động phát hiện lost updates

Các thao tác nguyên tử và lock là những cách để ngăn chặn lost updates bằng cách buộc các chu kỳ read-modify-write phải diễn ra một cách tuần tự. Một giải pháp thay thế là cho phép chúng thực thi song song và, nếu trình quản lý transaction phát hiện một lost update, nó sẽ hủy bỏ transaction đang xét và buộc nó phải thực hiện lại chu kỳ read-modify-write.

Một lợi thế của cách tiếp cận này là các cơ sở dữ liệu có thể thực hiện kiểm tra này một cách hiệu quả khi kết hợp với snapshot isolation. Thật vậy, các mức isolation repeatable read của PostgreSQL, serializable của Oracle, và snapshot isolation của SQL Server sẽ tự động phát hiện khi một lost update xảy ra và hủy bỏ transaction vi phạm. Tuy nhiên, mức isolation repeatable read của MySQL/InnoDB không phát hiện lost updates [30, 43]. Một số tác giả [38, 40] lập luận rằng một cơ sở dữ liệu phải ngăn chặn được lost updates để đủ điều kiện cung cấp snapshot isolation, vì vậy MySQL không cung cấp snapshot isolation theo định nghĩa này.

Một lợi thế lớn của việc phát hiện lost update là nó không yêu cầu mã ứng dụng phải sử dụng bất kỳ tính năng đặc biệt nào của cơ sở dữ liệu. Bạn có thể quên sử dụng lock hoặc một thao tác nguyên tử và từ đó tạo ra một lỗi (bug), nhưng việc phát hiện lost update diễn ra tự động và do đó ít dễ gây lỗi hơn. Tuy nhiên, bạn cũng phải thử lại các transaction đã bị hủy ở cấp độ ứng dụng.

Ghi có điều kiện (compare-and-set)

Trong các cơ sở dữ liệu không cung cấp transaction, đôi khi bạn sẽ tìm thấy một thao tác *ghi có điều kiện* (conditional write) có thể ngăn chặn lost updates bằng cách chỉ cho phép một cập nhật diễn ra nếu giá trị đó chưa thay đổi kể từ lần cuối bạn đọc nó (đã được đề cập trước đó trong mục “Ghi trên một đối tượng đơn lẻ”). Nếu giá trị hiện tại không khớp với những gì bạn đã đọc trước đó, việc cập nhật sẽ không có tác dụng, và chu kỳ read-modify-write phải được thực hiện lại. Nó tương đương với lệnh CAS nguyên tử được hỗ trợ bởi nhiều CPU.

Ví dụ, để ngăn hai người dùng cùng lúc cập nhật một trang wiki, bạn có thể thử thực hiện điều gì đó như thế này, với kỳ vọng rằng việc cập nhật sẽ chỉ xảy ra nếu nội dung của trang chưa thay đổi kể từ khi người dùng bắt đầu chỉnh sửa nó:

```
-- This may or may not be safe, depending on the database
implementation
UPDATE wiki_pages SET content = 'new content'
WHERE id = 1234 AND content = 'old content';
```

Nếu nội dung đã thay đổi và không còn khớp với old content, việc cập nhật này sẽ không có tác dụng, vì vậy bạn sẽ cần kiểm tra xem việc cập nhật đã có hiệu lực hay chưa và thử lại nếu cần thiết. Thay vì so sánh toàn bộ nội dung, bạn cũng có thể sử dụng một cột số phiên bản mà bạn sẽ tăng lên sau mỗi lần cập nhật và chỉ áp dụng

cập nhật nếu số phiên bản hiện tại chưa thay đổi. Cách tiếp cận này đôi khi được gọi là *optimistic locking* (khóa lạc quan) [54].

Lưu ý rằng nếu một transaction khác đã sửa đổi content một cách đồng thời, nội dung mới có thể không hiển thị theo các quy tắc hiển thị của MVCC (xem “Quy tắc hiển thị để quan sát một snapshot nhất quán”). Nhiều implementation của MVCC có một ngoại lệ đối với các quy tắc hiển thị cho kịch bản này, nơi các giá trị được ghi bởi các transaction khác có thể hiển thị trong quá trình đánh giá mệnh đề WHERE của các truy vấn UPDATE và DELETE, mặc dù các lần ghi đó không hiển thị trong snapshot theo cách thông thường.

Conflict resolution và replication

Trong các cơ sở dữ liệu có replication (xem Chương 6), việc ngăn chặn lost updates mang một khía cạnh khác. Bởi vì các cơ sở dữ liệu này có các bản sao dữ liệu trên nhiều node, và dữ liệu có khả năng bị sửa đổi đồng thời trên các node khác nhau, nên cần phải thực hiện thêm các bước bổ sung.

Các lock và các thao tác ghi có điều kiện (conditional write) giả định rằng có một bản sao duy nhất và cập nhật nhất của dữ liệu. Tuy nhiên, các cơ sở dữ liệu với multi-leader hoặc leaderless replication thường cho phép nhiều lần ghi xảy ra đồng thời và replication chúng một cách bất đồng bộ, vì vậy chúng không thể đảm bảo một bản sao duy nhất và cập nhật nhất của dữ liệu. Do đó, các kỹ thuật dựa trên lock hoặc conditional writes không áp dụng được trong bối cảnh này. (Chúng ta sẽ xem xét vấn đề này chi tiết hơn trong mục “Linearizability”.)

Thay vào đó, như đã thảo luận trong mục “Giải quyết các lần ghi xung đột (Conflicting Writes)”, một cách tiếp cận phổ biến trong các cơ sở dữ liệu replication như vậy là cho phép các lần ghi đồng thời tạo ra nhiều phiên bản xung đột của một giá trị (còn được gọi là *siblings*) và sử dụng mã ứng dụng hoặc các cấu trúc dữ liệu đặc biệt để giải quyết và hợp nhất các phiên bản này sau đó.

Việc hợp nhất các giá trị xung đột có thể ngăn chặn lost updates nếu các lần cập nhật có tính giao hoán (tức là, bạn có thể áp dụng chúng theo các thứ tự khác nhau trên các replica khác nhau mà vẫn nhận được cùng một kết quả). Ví dụ, việc tăng một bộ đếm và thêm một phần tử vào một tập hợp là các thao tác có tính giao hoán. Đó chính là ý tưởng đằng sau các CRDT, thứ mà chúng ta đã gặp trong mục “Conflict-free replicated datatypes và operational transformation”. Tuy nhiên, một số thao tác, chẳng hạn như conditional writes, không thể làm cho trở nên có tính giao hoán.

Ngoài ra, phương pháp giải quyết xung đột LWW, vốn là mặc định trong nhiều cơ sở dữ liệu replication, rất dễ dẫn đến lost updates, như đã thảo luận trong mục “Last write wins (loại bỏ các lần ghi đồng thời)”.

Write Skew và Phantoms

Trong các mục trước, chúng ta đã xem xét *dirty writes* và *lost updates*, hai loại race condition có thể xảy ra khi các transaction khác nhau đồng thời cố gắng ghi vào cùng một đối tượng. Để tránh làm hỏng dữ liệu, các race condition đó cần phải được ngăn chặn—hoặc tự động bởi cơ sở dữ liệu, hoặc bằng các biện pháp bảo vệ thủ công như sử dụng lock hoặc các thao tác ghi nguyên tử (atomic write).

Tuy nhiên, đó chưa phải là kết thúc của danh sách các race condition tiềm ẩn có thể xảy ra giữa các lần ghi đồng thời. Trong mục này, chúng ta sẽ thấy một số ví dụ tinh vi hơn về các xung đột.

Để bắt đầu, hãy tưởng tượng bạn đang viết một ứng dụng cho các bác sĩ để quản lý các ca trực của họ tại một bệnh viện. Bệnh viện thường cố gắng có vài bác sĩ trực tại bất kỳ thời điểm nào, nhưng tuyệt đối phải có ít nhất một người. Các bác sĩ có thể từ bỏ ca trực của mình (ví dụ, nếu họ bị ốm), miễn là có ít nhất một đồng nghiệp khác vẫn đang trực trong ca đó [55, 56].

Bây giờ hãy tưởng tượng Aaliyah và Bryce là hai bác sĩ trực cho một ca cụ thể. Cả hai đều cảm thấy không khỏe, vì vậy họ đều quyết định yêu cầu nghỉ phép. Thật không may, họ tình cờ nhấn nút ngừng trực vào khoảng cùng một thời điểm. Những gì xảy ra tiếp theo được minh họa trong Hình 8-8.

Trong mỗi transaction, ứng dụng của bạn trước tiên kiểm tra xem có hai hoặc nhiều bác sĩ đang trực hay không; nếu có, nó giả định rằng một bác sĩ ngừng trực là an toàn. Vì cơ sở dữ liệu đang sử dụng snapshot isolation, cả hai lần kiểm tra đều trả về 2, vì vậy cả hai transaction đều tiến tới giai đoạn tiếp theo. Aaliyah cập nhật bản ghi của chính mình để ngừng trực, và Bryce cũng cập nhật bản ghi của mình tương tự. Cả hai transaction đều commit, và giờ đây không còn bác sĩ nào đang trực. Yêu cầu của bạn về việc phải có ít nhất một bác sĩ trực đã bị vi phạm.

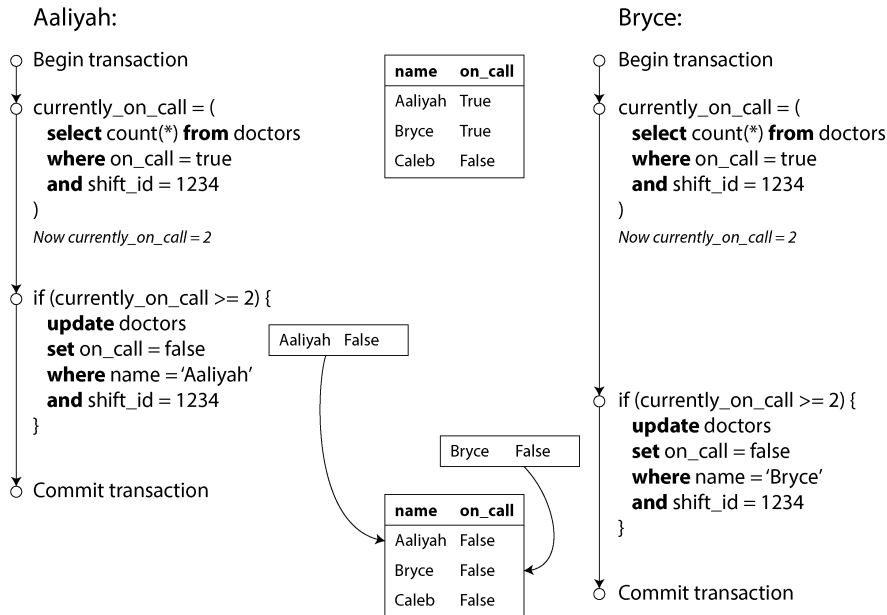


Figure 8-8. Hình: Một lỗi write skew gây ra bug trong ứng dụng

Đặc điểm của write skew

Sự bất thường này được gọi là *write skew* [38]. Nó không phải là dirty write cũng không phải là lost update, bởi vì hai transaction đang cập nhật hai đối tượng khác nhau (lần lượt là hồ sơ trực của Aaliyah và Bryce). Việc xảy ra xung đột ở đây có vẻ ít rõ ràng hơn, nhưng nó chắc chắn là một race condition: nếu hai transaction chạy nối tiếp nhau, bác sĩ thứ hai đã bị ngăn chặn việc thôi trực. Hành vi bất thường này chỉ có thể xảy ra vì các transaction chạy đồng thời.

Bạn có thể coi write skew là một dạng tổng quát hóa của vấn đề lost update. Write skew có thể xảy ra nếu hai transaction cùng đọc các đối tượng giống nhau và sau đó cập nhật một số đối tượng đó (các transaction khác nhau có thể cập nhật các đối tượng khác nhau). Trong trường hợp đặc biệt khi các transaction khác nhau cùng cập nhật một đối tượng, bạn sẽ gặp sự bất thường dirty write hoặc lost update (tùy thuộc vào thời điểm).

Chúng ta đã thấy rằng có nhiều cách khác nhau để ngăn chặn lost update. Với write skew, các lựa chọn của chúng ta bị hạn chế hơn:

- Các thao tác atomic trên một đối tượng duy nhất không giúp ích gì, vì có nhiều đối tượng liên quan.
- Việc tự động phát hiện lost update mà bạn thấy trong một số triển khai snapshot isolation đáng tiếc là cũng không giúp ích gì—write skew không được tự động phát hiện trong mức isolation repeatable read của PostgreSQL, repeatable read

của MySQL/InnoDB, serializable của Oracle, hay snapshot isolation của SQL Server [30]. Để ngăn chặn write skew một cách tự động, bạn cần mức isolation serializable thực thụ (xem mục “Serializability”).

- Một số cơ sở dữ liệu cho phép bạn cấu hình các constraint, sau đó chúng được thực thi bởi cơ sở dữ liệu (ví dụ: tính duy nhất, ràng buộc khóa ngoại, hoặc các hạn chế trên một giá trị cụ thể). Tuy nhiên, để chỉ định rằng phải có ít nhất một bác sĩ đang trực, bạn sẽ cần một constraint liên quan đến nhiều đối tượng. Hầu hết các cơ sở dữ liệu không có hỗ trợ tích hợp cho các constraint như vậy, mặc dù bạn có thể triển khai chúng bằng triggers hoặc materialized views, như đã thảo luận trong mục “Consistency” [12].
- Nếu bạn không thể sử dụng mức isolation serializable, lựa chọn tốt thứ hai trong trường hợp này có lẽ là khóa (lock) một cách rõ ràng các hàng mà transaction phụ thuộc vào. Trong ví dụ về các bác sĩ, bạn có thể viết tương tự như sau:

```
BEGIN TRANSACTION;

SELECT * FROM doctors
  WHERE on_call = true
  AND shift_id = 1234 FOR UPDATE; .

UPDATE doctors
  SET on_call = false
  WHERE name = 'Aaliyah'
  AND shift_id = 1234;

COMMIT;
```

•

Giống như trước, FOR UPDATE yêu cầu cơ sở dữ liệu khóa tất cả các hàng được trả về bởi truy vấn này.

Thêm các ví dụ về write skew

Write skew ban đầu có vẻ như là một vấn đề xa lạ, nhưng một khi bạn đã nhận thức được nó, bạn có thể nhận thấy các tình huống khác mà nó có thể xảy ra. Dưới đây là một số ví dụ khác:

Hệ thống đặt phòng họp

Giả sử bạn muốn đảm bảo rằng không thể có hai lượt đặt phòng cho cùng một phòng họp vào cùng một thời điểm [57]. Khi ai đó muốn đặt phòng, trước tiên bạn kiểm tra xem có bất kỳ lượt đặt nào xung đột hay không (tức là các lượt đặt cho cùng một phòng với khoảng thời gian chồng lấn), và nếu không tìm thấy sự xung đột nào, bạn mới tạo cuộc họp (xem Ví dụ 8-2).

Example 8-2. Một hệ thống đặt phòng hợp cố gắng tránh đặt trùng (không an toàn dưới snapshot isolation)

```
BEGIN TRANSACTION;

-- Check for any existing bookings that overlap with the period of
noon-1pm
SELECT COUNT(*) FROM bookings
WHERE room_id = 123 AND
      end_time > '2025-01-01 12:00' AND start_time < '2025-01-01
13:00';

-- If the previous query returned zero:
INSERT INTO bookings
(room_id, start_time, end_time, user_id)
VALUES (123, '2025-01-01 12:00', '2025-01-01 13:00', 666);

COMMIT;
```

Đáng tiếc là snapshot isolation không ngăn chặn một người dùng khác chen một cuộc họp xung đột một cách đồng thời. Để đảm bảo rằng bạn sẽ không gặp phải xung đột lịch trình, một lần nữa bạn cần mức isolation serializable.

Trò chơi nhiều người chơi

Trong Ví dụ 8-1, chúng ta đã sử dụng một lock để ngăn chặn lost update (đảm bảo rằng hai người chơi không thể di chuyển cùng một quân cờ tại cùng một thời điểm). Tuy nhiên, lock không ngăn chặn người chơi di chuyển hai quân cờ khác nhau đến cùng một vị trí trên bàn cờ hoặc có khả năng thực hiện một nước đi khác vi phạm quy tắc của trò chơi. Tùy thuộc vào loại quy tắc mà bạn đang thực thi, bạn có thể sử dụng một uniqueness constraint, nhưng nếu không, bạn sẽ dễ bị tổn thương bởi write skew.

Đăng ký tên người dùng

Trên một website nơi mỗi người dùng phải có một username duy nhất, hai người dùng có thể cùng lúc cố gắng tạo tài khoản với cùng một username. Bạn có thể sử dụng một transaction để kiểm tra xem tên đó đã có người sử dụng chưa và, nếu chưa, sẽ tạo một tài khoản với tên đó. Tuy nhiên, giống như các ví dụ trước, điều này không an toàn dưới snapshot isolation. May mắn thay, một ràng buộc về tính duy nhất (uniqueness constraint) là một giải pháp đơn giản ở đây (transaction thứ hai cố gắng đăng ký username đó sẽ bị abort do vi phạm ràng buộc).

Ngăn chặn việc chi tiêu gấp đôi

Một dịch vụ cho phép người dùng chi tiêu tiền hoặc điểm cần kiểm tra xem người dùng đó không chi tiêu nhiều hơn số họ đang có. Bạn có thể thực hiện việc này bằng cách chen một mục chi tiêu tạm thời vào tài khoản của người dùng, liệt kê tất cả các mục trong tài khoản, và kiểm tra xem tổng số có dương hay không. Tuy nhiên, với write skew, có thể xảy ra trường hợp hai mục chi tiêu được chen vào

đồng thời khiến số dư trở nên âm, nhưng không có transaction nào nhận ra sự hiện diện của transaction kia.

Phantoms gây ra write skew

Tất cả các ví dụ trước đó đều tuân theo một mô hình tương tự:

1. Một truy vấn `SELECT` kiểm tra xem một yêu cầu có được thỏa mãn hay không bằng cách tìm kiếm các hàng khớp với điều kiện tìm kiếm (ví dụ: có ít nhất hai bác sĩ đang trực, không có yêu cầu đặt phòng nào hiện có cho phòng đó vào thời điểm đó, vị trí trên bảng không có sẵn hình khác, username chưa bị chiếm dụng, tiền vẫn còn trong tài khoản).
2. Tùy thuộc vào kết quả của truy vấn đầu tiên, mã ứng dụng sẽ quyết định cách tiếp tục (có thể là tiến hành thao tác, hoặc có thể là báo lỗi cho người dùng và abort).
3. Nếu ứng dụng quyết định tiến hành, nó sẽ thực hiện một thao tác ghi (`INSERT`, `UPDATE`, hoặc `DELETE`) vào cơ sở dữ liệu và commit transaction.

Hiệu ứng của thao tác ghi này làm thay đổi điều kiện tiên quyết của quyết định ở bước 2. Nói cách khác, nếu bạn lặp lại truy vấn `SELECT` từ bước 1 sau khi đã commit thao tác ghi, bạn sẽ nhận được một kết quả khác, bởi vì thao tác ghi đã thay đổi tập hợp các hàng khớp với điều kiện tìm kiếm (bây giờ có ít hơn một bác sĩ đang trực, phòng họp hiện đã được đặt cho thời gian đó, vị trí trên bảng hiện đã bị chiếm bởi hình vừa được di chuyển, username hiện đã bị chiếm dụng, hoặc hiện có ít tiền hơn trong tài khoản).

Các bước có thể xảy ra theo một thứ tự khác. Ví dụ, bạn có thể thực hiện thao tác ghi trước, sau đó mới thực hiện truy vấn `SELECT`, và cuối cùng mới quyết định abort hay commit dựa trên kết quả của truy vấn.

Trong ví dụ về bác sĩ trực, hàng bị sửa đổi ở bước 3 là một trong những hàng được trả về ở bước 1, vì vậy bạn có thể làm cho transaction trở nên an toàn và tránh write skew bằng cách khóa các hàng ở bước 1 (`SELECT FOR UPDATE`). Tuy nhiên, bốn ví dụ còn lại thì khác: chúng kiểm tra sự *vắng mặt* của các hàng khớp với điều kiện tìm kiếm, và thao tác ghi lại *thêm* một hàng khớp với cùng điều kiện đó. Nếu truy vấn ở bước 1 không trả về bất kỳ hàng nào, `SELECT FOR UPDATE` không thể gắn lock vào bất cứ thứ gì [58].

Hiệu ứng này, nơi một thao tác ghi trong một transaction làm thay đổi kết quả của một truy vấn tìm kiếm trong một transaction khác, được gọi là một *phantom* [4]. Snapshot isolation tránh được phantoms trong các truy vấn chỉ đọc (read-only), nhưng trong các transaction đọc/ghi như các ví dụ chúng ta đã thảo luận, phantoms có thể dẫn đến các trường hợp write skew đặc biệt hóc búa. SQL được tạo ra bởi các ORM cũng dễ gặp phải write skew [52, 53].

Materializing conflicts

Nếu vấn đề của phantoms là không có đối tượng nào để chúng ta có thể gắn các lock vào, thì có lẽ chúng ta có thể đưa một đối tượng lock vào cơ sở dữ liệu một cách nhân tạo?

Ví dụ, trong trường hợp đặt phòng họp, bạn có thể tưởng tượng việc tạo một bảng gồm các khung thời gian và các phòng. Mỗi hàng trong bảng này tương ứng với một phòng cụ thể cho một khoảng thời gian cụ thể (chẳng hạn, 15 phút). Bạn tạo các hàng cho tất cả các tổ hợp có thể có của các phòng và các khoảng thời gian từ trước (ví dụ: cho sáu tháng tới).

Giờ đây, một transaction muốn tạo một lượt đặt chỗ có thể lock (`SELECT FOR UPDATE`) các hàng trong bảng tương ứng với phòng và khoảng thời gian mong muốn. Sau khi giành được các lock, transaction có thể kiểm tra các lượt đặt chỗ bị trùng lặp và chen một lượt đặt chỗ mới như trước đó. Lưu ý rằng bảng bổ sung này không được dùng để lưu trữ thông tin về lượt đặt chỗ—nó thuần túy là một tập hợp các lock được dùng để ngăn chặn việc đặt cùng một phòng và cùng một khoảng thời gian một cách đồng thời.

Cách tiếp cận này được gọi là *materializing conflicts* (hiện thực hóa xung đột), bởi vì nó lấy một phantom và biến nó thành một xung đột lock trên một tập hợp các hàng cụ thể đang tồn tại trong cơ sở dữ liệu [14]. Thật không may, việc tìm ra cách để materializing conflicts có thể rất khó khăn và dễ gây lỗi, đồng thời thật tệ khi để một cơ chế kiểm soát đồng thời (concurrency control) rò rỉ vào mô hình dữ liệu của ứng dụng. Vì những lý do đó, materializing conflicts nên được coi là giải pháp cuối cùng nếu không còn lựa chọn nào khác. Trong hầu hết các trường hợp, một isolation level kiểu serializable sẽ được ưu tiên hơn.

Serializability

Trong chương này, chúng ta đã thấy một vài ví dụ về các transaction dễ gặp phải race conditions. Một số race conditions được ngăn chặn bởi các isolation level read-committed và snapshot isolation, nhưng những lỗi khác thì không. Chúng ta đã gặp phải một số ví dụ đặc biệt hóc búa với write skew và phantoms. Đó là một tình huống đáng buồn:

- Các isolation level rất khó hiểu và được triển khai không nhất quán trong các cơ sở dữ liệu khác nhau (ví dụ, ý nghĩa của “repeatable read” thay đổi đáng kể).
- Có thể rất khó để xác định chỉ bằng cách nhìn vào mã nguồn ứng dụng liệu việc chạy ở một isolation level cụ thể có an toàn hay không—đặc biệt là trong một ứng dụng lớn, nơi bạn có thể không nhận thức được tất cả những thứ đang diễn ra đồng thời.

- Không có công cụ tốt nào giúp chúng ta phát hiện race conditions. Về nguyên tắc, phân tích tĩnh (static analysis) có thể giúp ích [35], nhưng các kỹ thuật nghiên cứu vẫn chưa được đưa vào sử dụng thực tế. Việc kiểm thử các vấn đề về đồng thời rất khó, bởi vì chúng thường không xác định (nondeterministic)—các vấn đề chỉ xảy ra nếu bạn không may mắn với việc căn thời gian (timing).

Đây không phải là một vấn đề mới. Nó đã như vậy kể từ những năm 1970, khi các isolation level yếu được đưa ra lần đầu tiên [3]. Suốt thời gian đó, câu trả lời từ các nhà nghiên cứu luôn đơn giản: hãy sử dụng isolation level *serializable*!

Serializable isolation là isolation level mạnh nhất. Nó đảm bảo rằng mặc dù các transaction có thể thực thi song song, kết quả cuối cùng vẫn giống như thể chúng đã thực thi từng cái một, theo thứ tự *serially* (tuần tự), mà không có bất kỳ sự đồng thời nào. Do đó, cơ sở dữ liệu đảm bảo rằng nếu các transaction hoạt động chính xác khi chạy riêng lẻ, chúng sẽ tiếp tục hoạt động chính xác khi chạy đồng thời—nói cách khác, cơ sở dữ liệu ngăn chặn *tất cả* các race conditions có thể xảy ra.

Nhưng nếu serializable isolation tốt hơn nhiều so với hỗn hợp của các isolation level yếu, tại sao không phải ai cũng sử dụng nó? Để trả lời câu hỏi này, chúng ta cần xem xét các lựa chọn để triển khai serializability và hiệu năng của chúng. Hầu hết các cơ sở dữ liệu cung cấp serializability ngày nay đều sử dụng một trong ba kỹ thuật mà chúng ta sẽ khám phá trong phần còn lại của chương này:

- Thực thi các transaction theo đúng thứ tự tuần tự (xem mục tiếp theo)
- Two-phase locking (xem “Two-Phase Locking”), thứ mà trong nhiều thập kỷ đã là lựa chọn khả thi duy nhất
- Các kỹ thuật kiểm soát đồng thời lạc quan (optimistic concurrency control) chẳng hạn như serializable snapshot isolation (xem “Serializable Snapshot Isolation”)

Actual Serial Execution

Cách đơn giản nhất để tránh các vấn đề về đồng thời là loại bỏ hoàn toàn sự đồng thời: chỉ thực thi một transaction tại một thời điểm, theo thứ tự tuần tự, trên một single thread. Bằng cách làm như vậy, chúng ta hoàn toàn né tránh được vấn đề phát hiện và ngăn chặn xung đột giữa các transaction; sự cô lập (isolation) thu được, theo định nghĩa, chính là serializable.

Mặc dù điều này có vẻ là một ý tưởng hiển nhiên, nhưng chỉ đến những năm 2000, các nhà thiết kế cơ sở dữ liệu mới quyết định rằng một vòng lặp đơn luồng (single-threaded loop) để thực thi các transaction là khả thi [59]. Nếu tính đồng thời đa luồng (multithreaded concurrency) được coi là yếu tố thiết yếu để đạt được hiệu năng tốt trong suốt 30 năm trước đó, thì điều gì đã thay đổi để giúp việc thực thi đơn luồng trở nên khả thi?

Hai sự phát triển đã dẫn đến sự thay đổi tư duy này:

- RAM đã trở nên đủ rẻ để đối với nhiều trường hợp sử dụng, việc giữ toàn bộ dataset đang hoạt động trong bộ nhớ hiện đã khả thi (xem mục “Keeping Everything in Memory”). Khi tất cả dữ liệu mà một transaction cần truy cập đều nằm trong bộ nhớ, các transaction có thể thực thi nhanh hơn nhiều so với việc chúng phải chờ dữ liệu được tải từ đĩa.
- Các nhà thiết kế cơ sở dữ liệu nhận ra rằng các transaction OLTP thường ngắn và chỉ thực hiện một số lượng nhỏ các thao tác đọc và ghi (xem mục “Operational Versus Analytical Systems”). Ngược lại, các truy vấn phân tích chạy lâu (long-running analytical queries) thường chỉ là read-only, vì vậy chúng có thể được chạy trên một snapshot nhất quán (sử dụng snapshot isolation) bên ngoài vòng lặp thực thi tuần tự.

Cách tiếp cận thực thi các transaction tuần tự được triển khai trong VoltDB/H-Store, Redis, và Datomic, ví dụ như vậy [60, 61, 62]. Một hệ thống được thiết kế để thực thi đơn luồng đôi khi có thể hoạt động tốt hơn một hệ thống hỗ trợ tính đồng thời, bởi vì nó có thể tránh được chi phí điều phối (coordination overhead) của việc lock. Tuy nhiên, throughput của nó bị giới hạn ở mức của một lõi CPU duy nhất. Để tận dụng tối đa luồng duy nhất đó, các transaction cần được cấu trúc khác đi so với dạng truyền thống của chúng.

Đóng gói các transaction trong stored procedures

Trong những ngày đầu của cơ sở dữ liệu, ý định là một database transaction có thể bao hàm toàn bộ một luồng hoạt động của người dùng. Ví dụ, đặt vé máy bay là một quá trình gồm nhiều giai đoạn (tìm kiếm các tuyến đường, giá vé và ghế trống; quyết định lịch trình; đặt ghế trên mỗi chuyến bay của lịch trình; nhập chi tiết hành khách; thực hiện thanh toán). Các nhà thiết kế cơ sở dữ liệu nghĩ rằng sẽ thật gọn gàng nếu toàn bộ quá trình đó là một transaction duy nhất để nó có thể được commit một cách nguyên tử (atomically).

Thật không may, con người đưa ra quyết định và phản hồi rất chậm. Nếu một database transaction cần chờ đầu vào từ người dùng, cơ sở dữ liệu sẽ cần hỗ trợ một số lượng khổng lồ các transaction đồng thời, mà hầu hết trong số chúng đều ở trạng thái rảnh rỗi (idle). Hầu hết các cơ sở dữ liệu không thể làm điều đó một cách hiệu quả, vì vậy hầu hết tất cả các ứng dụng OLTP đều giữ các transaction ngắn bằng cách tránh việc chờ đợi tương tác với người dùng trong một transaction. Trên web, điều này có nghĩa là một transaction được commit trong cùng một HTTP request—một transaction không kéo dài qua nhiều request. Một HTTP request mới sẽ bắt đầu một transaction mới.

Mặc dù con người đã được loại bỏ khỏi đường dẫn quan trọng (critical path), các transaction vẫn tiếp tục được thực thi theo kiểu client/server tương tác, thực

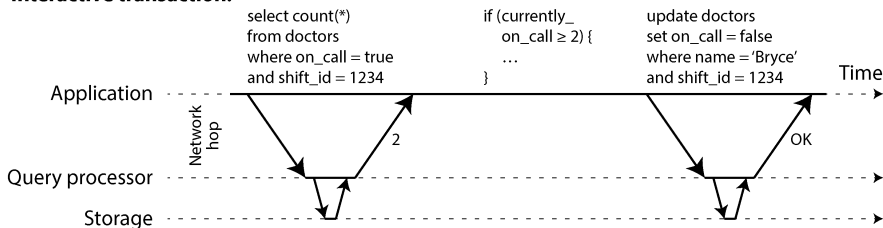
hiện từng câu lệnh một. Một ứng dụng thực hiện một truy vấn, đọc kết quả, có thể thực hiện một truy vấn khác tùy thuộc vào kết quả của truy vấn đầu tiên, và cứ tiếp tục như vậy. Các truy vấn và kết quả được gửi qua lại giữa mã ứng dụng (chạy trên một máy) và database server (trên một máy khác).

Trong kiểu transaction tương tác này, rất nhiều thời gian bị tiêu tốn vào việc giao tiếp mạng giữa ứng dụng và cơ sở dữ liệu. Nếu bạn không cho phép tính đồng thời trong cơ sở dữ liệu và chỉ xử lý một transaction tại một thời điểm, throughput sẽ rất tệ vì cơ sở dữ liệu sẽ dành phần lớn thời gian để chờ ứng dụng đưa ra truy vấn tiếp theo cho transaction hiện tại. Trong loại cơ sở dữ liệu này, việc xử lý nhiều transaction đồng thời là cần thiết để đạt được hiệu năng hợp lý.

Vì lý do này, các hệ thống với xử lý transaction tuần tự đơn luồng không cho phép các transaction tương tác gồm nhiều câu lệnh. Thay vào đó, ứng dụng phải tự giới hạn mình trong các transaction chỉ chứa một câu lệnh duy nhất hoặc gửi toàn bộ mã transaction đến cơ sở dữ liệu trước, dưới dạng một *stored procedure* [63].

Sự khác biệt giữa interactive transaction và stored procedure được minh họa trong Hình 8-9. Với điều kiện là tất cả dữ liệu mà một transaction yêu cầu đều nằm trong bộ nhớ, stored procedure có thể thực thi rất nhanh chóng mà không cần chờ đợi bất kỳ hoạt động I/O mạng hay đĩa nào.

Interactive transaction:



Stored procedure:

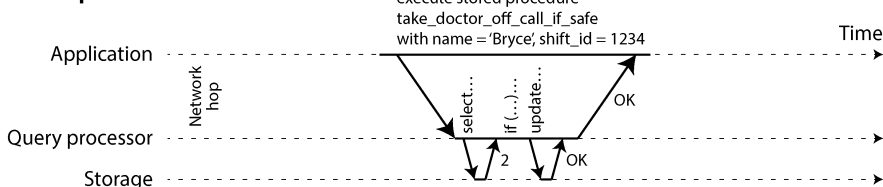


Figure 8-9. Sự khác biệt giữa một transaction tương tác và một stored procedure (sử dụng ví dụ về transaction trong Hình 8-8)

Ưu và nhược điểm của stored procedure

Stored procedure đã tồn tại từ lâu trong các cơ sở dữ liệu quan hệ, và chúng đã là một phần của tiêu chuẩn SQL (SQL/PSM) kể từ năm 1999. Chúng đã phải chịu một danh tiếng không mấy tốt đẹp vì nhiều lý do khác nhau:

- Theo truyền thống, mỗi nhà cung cấp cơ sở dữ liệu đều có ngôn ngữ riêng cho stored procedure (Oracle có PL/SQL, SQL Server có T-SQL, PostgreSQL có PL/pgSQL, v.v.). Những ngôn ngữ này đã không theo kịp sự phát triển của các ngôn ngữ lập trình đa năng, vì vậy chúng trông khá xấu xí và cổ lỗ sĩ dưới góc nhìn ngày nay, đồng thời thiếu hệ sinh thái các thư viện mà bạn thường thấy ở hầu hết các ngôn ngữ lập trình hiện đại.
- Mã chạy trong một cơ sở dữ liệu rất khó quản lý. So với một application server, nó khó debug hơn, khó khăn hơn khi giữ trong version control và deploy, phức tạp hơn khi test, và khó tích hợp với một hệ thống thu thập metrics để giám sát.
- Một cơ sở dữ liệu thường nhạy cảm với hiệu năng hơn nhiều so với một application server, bởi vì một instance cơ sở dữ liệu duy nhất thường được chia sẻ bởi nhiều application server. Một stored procedure được viết kém (ví dụ: sử dụng quá nhiều bộ nhớ hoặc thời gian CPU, hoặc thậm chí gây ra lỗi crash) trong một cơ sở dữ liệu có thể gây ra nhiều rắc rối hơn nhiều so với mã viết kém tương đương trong một application server.
- Trong một hệ thống multitenant cho phép các tenant viết stored procedure của riêng họ, việc thực thi mã không đáng tin cậy trong cùng một process với database kernel là một rủi ro bảo mật [64].

Tuy nhiên, những vấn đề đó có thể được khắc phục. Các triển khai hiện đại của stored procedure đã từ bỏ PL/SQL và thay vào đó sử dụng các ngôn ngữ lập trình đa năng hiện có. VoltDB sử dụng Java hoặc Groovy, Datomic sử dụng Java hoặc Clojure, Redis sử dụng Lua, và MongoDB sử dụng JavaScript.

Stored procedure cũng hữu ích khi logic ứng dụng không dễ dàng được nhúng ở nơi khác. Ví dụ, các ứng dụng sử dụng GraphQL có thể trực tiếp phơi bày cơ sở dữ liệu của chúng thông qua một GraphQL proxy. Nếu proxy không hỗ trợ logic validation phức tạp, bạn có thể nhúng logic đó trực tiếp vào cơ sở dữ liệu bằng cách sử dụng một stored procedure. Nếu cơ sở dữ liệu không hỗ trợ stored procedure, bạn sẽ phải triển khai một validation service giữa proxy và cơ sở dữ liệu để thực hiện việc validation.

Với stored procedure và dữ liệu trong bộ nhớ (in-memory data), việc thực thi tất cả các transaction trên một single thread trở nên khả thi. Khi các stored procedure không cần chờ I/O và tránh được chi phí overhead của các cơ chế concurrency control khác, chúng có thể đạt được throughput khá tốt trên một single thread.

VoltDB cũng sử dụng stored procedure để replication. Thay vì sao chép các thao tác ghi của một transaction từ node này sang node khác, nó thực thi cùng một stored procedure trên mỗi replica. Do đó, VoltDB yêu cầu các stored procedure phải có tính *deterministic* (khi chạy trên các node khác nhau, chúng phải tạo ra cùng một kết quả). Ví dụ, nếu một transaction cần sử dụng ngày và giờ hiện tại, nó phải thực hiện thông qua các API deterministic đặc biệt (xem “Durable Execution and Workflows” để biết thêm chi tiết về các thao tác deterministic). Cách tiếp cận này được gọi là *state machine replication*, và chúng ta sẽ quay lại nội dung này trong Chương 10.

Sharding

Việc thực thi tất cả các transaction một cách tuần tự (serially) giúp việc kiểm soát đồng thời trở nên đơn giản hơn nhiều, nhưng nó giới hạn throughput transaction của cơ sở dữ liệu ở tốc độ của một lõi CPU duy nhất trên một máy duy nhất. Các transaction chỉ đọc (read-only) có thể được thực thi ở nơi khác bằng cách sử dụng snapshot isolation, nhưng đối với các ứng dụng có write throughput cao, bộ xử lý transaction đơn luồng (single-threaded) có thể trở thành một bottleneck nghiêm trọng.

Để scale lên nhiều lõi CPU và nhiều node, bạn có thể sharding dữ liệu của mình (xem Chương 7), điều này được hỗ trợ trong VoltDB. Nếu bạn có thể tìm ra cách sharding dataset sao cho mỗi transaction chỉ cần đọc và ghi dữ liệu trong phạm vi một shard duy nhất, thì mỗi shard có thể có thread xử lý transaction riêng chạy độc lập với các thread khác. Trong trường hợp này, bạn có thể cấp cho mỗi lõi CPU một shard riêng, cho phép throughput transaction của bạn scale tuyến tính theo số lượng lõi CPU [61].

Tuy nhiên, đối với bất kỳ transaction nào cần truy cập vào nhiều shard, cơ sở dữ liệu phải điều phối transaction đó trên tất cả các shard mà nó chạm tới. *stored procedure* cần được thực hiện đồng bộ (in lockstep) trên tất cả các shard để đảm bảo tính serializability trên toàn bộ hệ thống.

Vì các cross-shard transaction có thêm chi phí điều phối (coordination overhead), chúng chậm hơn rất nhiều so với các single-shard transaction. VoltDB báo cáo throughput khoảng 1.000 lượt ghi cross-shard mỗi giây, thấp hơn nhiều bậc so với throughput single-shard của nó và không thể tăng lên bằng cách thêm nhiều máy hơn [63]. Các nghiên cứu gần đây đã khám phá những cách để làm cho các multishard transaction có khả năng mở rộng tốt hơn [65].

Việc các transaction có thể là single-shard hay không phụ thuộc rất nhiều vào cấu trúc dữ liệu được sử dụng bởi ứng dụng. Dữ liệu key-value đơn giản thường có thể được sharding rất dễ dàng, nhưng dữ liệu với nhiều secondary index có khả năng yêu cầu rất nhiều sự điều phối cross-shard (xem “Sharding and Secondary Indexes”).

Tóm tắt về thực thi tuần tự (serial execution)

Thực thi tuần tự các transaction đã trở thành một cách khả thi để đạt được isolation có tính serializability, trong một số ràng buộc nhất định:

- Mọi transaction phải nhỏ và nhanh, bởi vì chỉ cần một transaction chậm là có thể làm đình trệ toàn bộ quá trình xử lý transaction.
- Nó phù hợp nhất khi dataset đang hoạt động có thể nằm gọn trong bộ nhớ. Dữ liệu ít được truy cập có khả năng được chuyển sang đĩa, nhưng nếu nó cần được truy cập trong một transaction đơn luồng (single-threaded), hệ thống sẽ trở nên rất chậm.
- Write throughput phải đủ thấp để có thể xử lý trên một nhân CPU duy nhất, nếu không các transaction cần được sharding mà không yêu cầu điều phối cross-shard.
- Các cross-shard transaction là khả thi, nhưng throughput của chúng rất khó để scale.

Two-Phase Locking

Trong khoảng 30 năm, chỉ có một thuật toán được sử dụng rộng rãi để đạt được tính serializability trong các cơ sở dữ liệu: *two-phase locking* (2PL), đôi khi được gọi là *strong strict two-phase locking* (SS2PL) để phân biệt nó với các biến thể khác của 2PL.

2PL không phải là 2PC

2PL và 2PC là hai thứ rất khác nhau. 2PL cung cấp isolation có tính serializability, trong khi 2PC cung cấp atomic commit trong một cơ sở dữ liệu phân tán (xem “Two-Phase Commit”). Để tránh nhầm lẫn, tốt nhất nên coi chúng là các khái niệm hoàn toàn tách biệt và bỏ qua sự tương đồng đáng tiếc trong tên gọi của chúng.

Chúng ta đã thấy trước đó rằng các lock thường được sử dụng để ngăn chặn dirty writes (xem “No dirty writes”). Nếu hai transaction đồng thời cố gắng ghi vào cùng một đối tượng, lock sẽ đảm bảo rằng người ghi thứ hai phải đợi cho đến khi người thứ nhất hoàn thành transaction của mình (aborted hoặc committed) trước khi có thể tiếp tục.

2PL cũng tương tự, nhưng nó đưa ra các yêu cầu về lock mạnh mẽ hơn nhiều. Một vài transaction được phép đồng thời đọc cùng một đối tượng miễn là không có ai đang ghi vào đó. Nhưng ngay khi bất kỳ ai muốn ghi (sửa đổi hoặc xóa) một đối tượng, việc truy cập độc quyền (exclusive access) là bắt buộc:

- Nếu transaction A đã đọc một đối tượng và transaction B muốn ghi vào đối tượng đó, B phải đợi cho đến khi A commits hoặc aborts trước khi có thể tiếp tục. (Điều này đảm bảo rằng B không thể thay đổi đối tượng một cách bất ngờ sau lưng A.)
- Nếu transaction A đã ghi một đối tượng và transaction B muốn đọc đối tượng đó, B phải đợi cho đến khi A commits hoặc aborts trước khi có thể tiếp tục. (Việc đọc một phiên bản cũ của đối tượng, như trong Hình 8-4, là không thể chấp nhận được dưới cơ chế 2PL.)

Trong 2PL, những người ghi không chỉ chặn các người ghi khác; chúng còn chặn cả những người đọc, và ngược lại. Câu thần chú *readers never block writers, and writers never block readers* (người đọc không bao giờ chặn người ghi, và người ghi không bao giờ chặn người đọc) của snapshot isolation (xem “Multiversion concurrency control”) đã nắm bắt được sự khác biệt then chốt này giữa snapshot isolation và 2PL. Mặt khác, vì 2PL cung cấp tính serializability, nó bảo vệ chống lại tất cả các race condition đã thảo luận trước đó, bao gồm lost updates và write skew.

Triển khai 2PL

2PL được sử dụng bởi isolation level có tính serializable trong MySQL/InnoDB và SQL Server, và bởi isolation level repeatable-read trong Db2 [30].

Việc chặn các reader và writer được thực hiện bằng cách đặt một lock trên mỗi đối tượng trong cơ sở dữ liệu. Lock có thể ở *shared mode* hoặc *exclusive mode* (còn được gọi là lock *multi-reader single-writer*). Nó được sử dụng như sau:

- Nếu một transaction muốn đọc một đối tượng, trước tiên nó phải giành được lock ở *shared mode*. Nhiều transaction được phép cùng giữ lock ở *shared mode* đồng thời, nhưng nếu một transaction khác đã có *exclusive lock* trên đối tượng đó, các transaction này phải chờ.
- Nếu một transaction muốn ghi vào một đối tượng, trước tiên nó phải giành được lock ở *exclusive mode*. Không có transaction nào khác được phép giữ lock tại cùng một thời điểm (dù là ở *shared mode* hay *exclusive mode*), vì vậy nếu đang có bất kỳ lock nào tồn tại trên đối tượng, transaction đó phải chờ.
- Nếu một transaction thực hiện đọc trước rồi mới ghi một đối tượng, nó có thể nâng cấp *shared lock* của mình lên thành *exclusive lock*. Việc nâng cấp này hoạt động tương tự như cách giành được một *exclusive lock* trực tiếp.
- Sau khi một transaction đã giành được lock, nó phải tiếp tục giữ lock đó cho đến khi kết thúc transaction (commit hoặc abort). Đây chính là nguồn gốc của cái tên “two-phase”: giai đoạn đầu (giai đoạn *growing*, khi transaction đang thực thi) là lúc các lock được giành lấy, và giai đoạn thứ hai (giai đoạn *shrinking*, khi kết thúc transaction) là lúc tất cả các lock được giải phóng. Hai giai đoạn này không được chồng lấn lên nhau; một khi lock đã được giải phóng, không có lock mới nào được phép giành lấy trong một transaction.

Vì có quá nhiều lock được sử dụng, việc transaction A bị kẹt trong khi chờ transaction B giải phóng lock của nó, và ngược lại, có thể xảy ra rất dễ dàng. Tình huống này được gọi là *deadlock*. Cơ sở dữ liệu tự động phát hiện deadlock giữa các transaction và abort một trong số chúng để các transaction khác có thể tiếp tục tiến triển. Transaction bị abort cần được ứng dụng thực hiện thử lại (retry).

Hiệu năng của 2PL

Nhược điểm lớn của 2PL, và là lý do tại sao nó không phải là mặc định cho hầu hết các hệ thống kể từ những năm 1970, chính là hiệu năng. Transaction throughput và thời gian phản hồi của các truy vấn kém hơn đáng kể dưới 2PL so với dưới mức isolation yếu.

Điều này một phần là do chi phí (overhead) của việc giành lấy và giải phóng tất cả các lock đó, nhưng quan trọng hơn là do giảm tính đồng thời (concurrency). Theo thiết kế, nếu hai transaction đồng thời cố gắng thực hiện bất kỳ điều gì có thể dẫn đến race condition, một cái phải chờ cái kia hoàn tất.

Ví dụ, nếu bạn có một transaction cần đọc toàn bộ một bảng (ví dụ: một thao tác backup, truy vấn phân tích, hoặc kiểm tra tính toàn vẹn, như đã thảo luận trong mục

“Snapshot Isolation and Repeatable Read”), transaction đó phải lấy một *shared lock* trên toàn bộ bảng. Do đó, transaction đọc trước tiên phải chờ cho đến khi tất cả các transaction đang thực thi việc ghi vào bảng đó hoàn tất; sau đó, trong khi toàn bộ bảng đang được đọc (điều này có thể mất nhiều thời gian đối với một bảng lớn), tất cả các transaction khác muốn ghi vào bảng đó đều bị chặn cho đến khi transaction chỉ đọc lớn đó commit. Về cơ bản, cơ sở dữ liệu trở nên không khả dụng cho các thao tác ghi trong một khoảng thời gian kéo dài.

Vì lý do này, các cơ sở dữ liệu chạy 2PL có thể có latency khá không ổn định, và chúng có thể rất chậm ở các percentile cao (xem mục “Describing Performance”) nếu có sự tranh chấp (contention) trong khối lượng công việc. Chỉ cần một transaction chậm, hoặc một transaction truy cập nhiều dữ liệu và giành lấy nhiều lock, cũng có thể khiến phần còn lại của hệ thống bị đình trệ. Transaction timeout và giám sát truy vấn chậm được sử dụng để phát hiện và giới hạn các truy vấn hoạt động sai lệch.

Mặc dù deadlock có thể xảy ra với mức isolation *read-committed* dựa trên lock, nhưng chúng xảy ra thường xuyên hơn nhiều dưới mức isolation *serializable* của 2PL (tùy thuộc vào pattern truy cập của transaction của bạn). Đây có thể là một vấn đề hiệu năng bổ sung: khi một transaction bị abort do deadlock và được thử lại, nó cần phải thực hiện lại toàn bộ công việc của mình. Nếu deadlock xảy ra thường xuyên, điều này có thể dẫn đến việc lãng phí nỗ lực đáng kể.

Predicate locks

Trong phần mô tả trước đó về các lock, chúng ta đã lướt qua một chi tiết tinh tế nhưng quan trọng. Trong mục “Phantoms gây ra write skew”, chúng ta đã thảo luận về vấn đề *phantoms*—tức là một transaction làm thay đổi kết quả truy vấn tìm kiếm của một transaction khác. Một cơ sở dữ liệu với isolation có tính serializable phải ngăn chặn được phantoms.

Trong ví dụ đặt phòng họp, điều này có nghĩa là nếu một transaction đã tìm kiếm các lượt đặt phòng hiện có cho một phòng trong một khoảng thời gian nhất định (xem Ví dụ 8-2), thì một transaction khác không được phép chen hoặc cập nhật một lượt đặt phòng khác cho cùng phòng và cùng khoảng thời gian đó một cách đồng thời. (Việc chen đồng thời các lượt đặt phòng cho các phòng khác, hoặc cho cùng một phòng vào thời gian khác không ảnh hưởng đến lượt đặt phòng đang được đề xuất, là hoàn toàn hợp lệ.)

Làm thế nào để chúng ta triển khai điều này? Về mặt khái niệm, chúng ta cần một *predicate lock* [4]. Nó hoạt động tương tự như *shared/exclusive lock* đã được mô tả trước đó, nhưng thay vì thuộc về một đối tượng cụ thể (ví dụ: một hàng trong một bảng), nó thuộc về tất cả các đối tượng khớp với một điều kiện tìm kiếm, chẳng hạn như thế này:

```
SELECT * FROM bookings
WHERE room_id = 123 AND
      end_time > '2026-01-01 12:00' AND
      start_time < '2026-01-01 13:00';
```

Một predicate lock hạn chế quyền truy cập như sau:

- Nếu transaction A muốn đọc các đối tượng khớp với một điều kiện, như trong truy vấn SELECT đó, nó phải giành được một predicate lock ở chế độ shared trên các điều kiện của truy vấn. Nếu một transaction B khác hiện đang giữ một exclusive lock trên bất kỳ đối tượng nào khớp với các điều kiện đó, A phải đợi cho đến khi B giải phóng lock của nó trước khi được phép thực hiện truy vấn.
- Nếu transaction A muốn chèn, cập nhật hoặc xóa bất kỳ đối tượng nào, trước tiên nó phải kiểm tra xem giá trị cũ hoặc giá trị mới có khớp với bất kỳ predicate lock hiện có nào không. Nếu một predicate lock khớp đang được giữ bởi transaction B, thì A phải đợi cho đến khi B đã commit hoặc abort trước khi có thể tiếp tục.

Ý tưởng then chốt ở đây là một predicate lock áp dụng ngay cả cho các đối tượng chưa tồn tại trong cơ sở dữ liệu, nhưng có thể được thêm vào trong tương lai (phantoms). Nếu 2PL bao gồm cả predicate locks, cơ sở dữ liệu sẽ ngăn chặn mọi dạng write skew và các race condition khác, và do đó tính isolation của nó sẽ trở thành serializable.

Index-range locks

Thật không may, predicate locks hoạt động không tốt: nếu có nhiều lock bởi các transaction đang hoạt động, việc kiểm tra các lock khớp sẽ trở nên tốn thời gian. Vì lý do đó, hầu hết các cơ sở dữ liệu sử dụng 2PL đều triển khai *index-range locking* (còn được gọi là *next-key locking*), vốn là một sự xấp xỉ đơn giản hóa của predicate locking [56, 66].

Việc đơn giản hóa một predicate bằng cách làm cho nó khớp với một tập hợp đối tượng lớn hơn là an toàn. Ví dụ, nếu bạn có một predicate lock cho các lượt đặt phòng của phòng 123 từ giữa trưa đến 1 giờ chiều, bạn có thể xấp xỉ nó bằng cách lock các lượt đặt phòng của phòng 123 vào bất kỳ thời gian nào, hoặc bạn có thể xấp xỉ nó bằng cách lock tất cả các phòng (không chỉ phòng 123) trong khoảng từ giữa trưa đến 1 giờ chiều. Điều này là an toàn vì bất kỳ thao tác ghi nào khớp với predicate ban đầu chắc chắn cũng sẽ khớp với các bản xấp xỉ này.

Trong cơ sở dữ liệu đặt phòng, bạn có lẽ sẽ có một index trên cột `room_id` và/hoặc các index trên `start_time` và `end_time` (nếu không, truy vấn trước đó sẽ rất chậm trên một cơ sở dữ liệu lớn):

- Giả sử index của bạn nằm trên `room_id`, và cơ sở dữ liệu sử dụng index này để tìm các lượt đặt phòng hiện có cho phòng 123. Bây giờ cơ sở dữ liệu có thể đơn

giản là gắn một shared lock vào entry của index này, cho biết rằng một transaction đã tìm kiếm các lượt đặt phòng của phòng 123.

- Mặt khác, nếu cơ sở dữ liệu sử dụng một index dựa trên thời gian để tìm các lượt đặt phòng hiện có, nó có thể gắn một shared lock vào một phạm vi các giá trị trong index đó, cho biết rằng một transaction đã tìm kiếm các lượt đặt phòng trùng với khoảng thời gian từ giữa trưa đến 1 giờ chiều vào ngày đã chỉ định.

Dù theo cách nào, một sự xấp xỉ của điều kiện tìm kiếm cũng được gắn vào một trong các index. Bây giờ, nếu một transaction khác muốn chen, cập nhật hoặc xóa một lượt đặt phòng cho cùng phòng và/hoặc một khoảng thời gian chồng lấn, nó sẽ phải cập nhật cùng một phần của index đó. Trong quá trình thực hiện, nó sẽ gặp phải shared lock và sẽ buộc phải đợi cho đến khi lock được giải phóng.

Điều này giúp bảo vệ hiệu quả chống lại phantoms và write skew. Index-range locks không chính xác như predicate locks (chúng có thể khóa một phạm vi đối tượng lớn hơn mức thực sự cần thiết để duy trì tính serializability), nhưng vì chúng có chi phí overhead thấp hơn nhiều, nên đây là một sự đánh đổi (trade-off) hợp lý.

Nếu không có index phù hợp để gắn một range lock, cơ sở dữ liệu có thể chuyển sang sử dụng shared lock trên toàn bộ bảng. Điều này sẽ không tốt cho hiệu năng, vì nó sẽ chặn tất cả các transaction khác đang ghi vào bảng, nhưng đó là một phương án dự phòng an toàn.

Serializable Snapshot Isolation

Chương này đã vẽ nên một bức tranh âm ảm về kiểm soát đồng thời (concurrency control) trong các cơ sở dữ liệu. Một mặt, chúng ta có các triển khai của tính serializability hoạt động không tốt (2PL) hoặc không có khả năng mở rộng tốt (serial execution). Mặt khác, chúng ta có các isolation level yếu có hiệu năng tốt nhưng dễ gặp phải các race condition khác nhau (lost updates, write skew, phantoms, v.v.). Liệu tính serializable isolation và hiệu năng tốt có thực sự mâu thuẫn với nhau không?

Có vẻ như là không: một thuật toán gọi là *serializable snapshot isolation* (SSI) cung cấp tính serializability đầy đủ mà chỉ gây ra một tổn thất hiệu năng nhỏ so với snapshot isolation. SSI tương đối mới; nó được mô tả lần đầu vào năm 2008 [55, 67].

Ngày nay, SSI và các thuật toán tương tự được sử dụng trong các cơ sở dữ liệu single-node (isolation level serializable trong PostgreSQL [56], In-Memory OLTP/Hekaton của SQL Server [68], và HyPer [69]), các cơ sở dữ liệu phân tán (CockroachDB [5] và FoundationDB [8]), và các công cụ lưu trữ embedded như BadgerDB.

Kiểm soát đồng thời bi quan so với lạc quan

2PL là một cơ chế kiểm soát đồng thời *bi quan* (*pessimistic*): nó dựa trên nguyên tắc rằng nếu có bất cứ điều gì có khả năng xảy ra sai sót (như được chỉ ra bởi một lock

đang được giữ bởi một transaction khác), thì tốt hơn là nên đợi cho đến khi tình huống trở nên an toàn trở lại trước khi thực hiện bất cứ điều gì. Nó giống như *mutual exclusion* (loại trừ tương hỗ), thứ được sử dụng để bảo vệ các cấu trúc dữ liệu trong lập trình đa luồng (multithreaded).

Serial execution, theo một nghĩa nào đó, là sự bi quan đến cực đoan; về cơ bản nó tương đương với việc mỗi transaction nắm giữ một exclusive lock trên toàn bộ cơ sở dữ liệu (hoặc một shard của cơ sở dữ liệu) trong suốt thời gian diễn ra transaction đó. Chúng ta bù đắp cho sự bi quan này bằng cách làm cho mỗi transaction thực thi cực nhanh, để nó chỉ cần giữ "lock" trong một thời gian ngắn.

Ngược lại, serializable snapshot isolation là một kỹ thuật kiểm soát đồng thời lạc quan. *Lạc quan (optimistic)* trong ngữ cảnh này có nghĩa là thay vì chặn lại nếu có điều gì đó tiềm ẩn nguy hiểm xảy ra, các transaction vẫn tiếp tục thực hiện, với hy vọng rằng mọi thứ sẽ ổn thỏa. Khi một transaction muốn commit, cơ sở dữ liệu sẽ kiểm tra xem có điều gì tồi tệ đã xảy ra hay không (tức là liệu tính isolation có bị vi phạm hay không); nếu có, transaction đó sẽ bị abort và phải thực hiện lại. Chỉ những transaction thực thi theo kiểu serializable mới được phép commit.

Kiểm soát đồng thời lạc quan là một ý tưởng cũ [70], và các ưu điểm cũng như nhược điểm của nó đã được tranh luận từ lâu [71]. Nó hoạt động kém nếu có sự tranh chấp (contention) cao (nhiều transaction cùng cố gắng truy cập vào các đối tượng giống nhau), vì điều này dẫn đến tỷ lệ cao các transaction cần phải abort. Nếu hệ thống đã gần chạm ngưỡng throughput tối đa, tải transaction bổ sung từ các transaction thực hiện lại có thể làm hiệu năng trở nên tồi tệ hơn.

Tuy nhiên, nếu có đủ khả năng dự phòng, và nếu sự tranh chấp giữa các transaction không quá cao, các kỹ thuật kiểm soát đồng thời lạc quan có xu hướng hoạt động tốt hơn các kỹ thuật bi quan. Sự tranh chấp có thể được giảm bớt bằng các thao tác nguyên tử có tính giao hoán (commutative atomic operations): ví dụ, nếu nhiều transaction đồng thời muốn tăng một biến đếm, việc các lệnh tăng được áp dụng theo thứ tự nào không quan trọng (miễn là biến đếm không bị đọc trong cùng một transaction đó), vì vậy các lệnh tăng đồng thời có thể đều được áp dụng mà không gây xung đột.

Như tên gọi, SSI dựa trên snapshot isolation—tức là, tất cả các thao tác đọc trong một transaction đều được thực hiện từ một snapshot nhất quán của cơ sở dữ liệu (xem "Snapshot Isolation and Repeatable Read"). Trên nền tảng snapshot isolation, SSI bổ sung một thuật toán để phát hiện các xung đột serializability giữa các thao tác đọc và ghi, đồng thời xác định xem những transaction nào cần phải abort.

Các quyết định dựa trên một tiền đề lỗi thời

Khi chúng ta thảo luận về write skew trong snapshot isolation trước đây (xem "Write Skew and Phantoms"), chúng ta đã quan sát thấy một mô hình lặp lại: một

transaction đọc dữ liệu từ cơ sở dữ liệu, kiểm tra kết quả của truy vấn, và quyết định thực hiện một hành động (ghi vào cơ sở dữ liệu) dựa trên kết quả mà nó đã thấy. Tuy nhiên, dưới snapshot isolation, kết quả từ truy vấn ban đầu có thể không còn cập nhật tại thời điểm transaction commit, bởi vì dữ liệu có thể đã bị sửa đổi trong khoảng thời gian đó.

Nói cách khác, transaction đang thực hiện một hành động dựa trên một *tiền đề* (một sự thật vốn đúng tại thời điểm bắt đầu transaction, chẳng hạn như “Hiện đang có hai bác sĩ trực”). Sau đó, khi transaction muốn commit, dữ liệu ban đầu có thể đã thay đổi—tiền đề đó có thể không còn đúng nữa.

Khi ứng dụng thực hiện một truy vấn (ví dụ: “Hiện có bao nhiêu bác sĩ đang trực?”), cơ sở dữ liệu không biết logic của ứng dụng sử dụng kết quả của truy vấn đó như thế nào. Để đảm bảo an toàn, cơ sở dữ liệu cần giả định rằng bất kỳ thay đổi nào trong kết quả truy vấn (tiền đề) đều có nghĩa là các thao tác ghi trong transaction đó có thể không còn hợp lệ. Nói cách khác, có thể tồn tại một sự phụ thuộc nhân quả giữa các truy vấn và các thao tác ghi trong transaction. Để cung cấp isolation có tính serializable, cơ sở dữ liệu phải phát hiện được các tình huống mà một transaction có thể đã hành động dựa trên một tiền đề lỗi thời và thực hiện abort transaction đó trong trường hợp đó.

Làm thế nào cơ sở dữ liệu biết được liệu một kết quả truy vấn có thể đã thay đổi? Hãy xét hai trường hợp:

- Phát hiện các thao tác đọc một phiên bản đối tượng MVCC cũ (một thao tác ghi chưa commit đã xảy ra trước khi đọc)
- Phát hiện các thao tác ghi ảnh hưởng đến các thao tác đọc trước đó (thao tác ghi xảy ra sau khi đọc)

Phát hiện các thao tác đọc MVCC cũ

Hãy nhớ rằng snapshot isolation thường được triển khai bằng MVCC (xem “Multiversion concurrency control”). Khi một transaction đọc từ một snapshot nhất quán trong một cơ sở dữ liệu MVCC, nó sẽ bỏ qua các thao tác ghi được thực hiện bởi bất kỳ transaction nào khác chưa commit tại thời điểm snapshot được tạo.

Trong Hình 8-10, transaction 43 thấy Aaliyah có giá trị `on_call = true`, bởi vì transaction 42 (thao tác đã sửa đổi trạng thái trực của Aaliyah) vẫn chưa commit. Tuy nhiên, vào thời điểm transaction 43 muốn commit, transaction 42 đã commit xong. Điều này có nghĩa là thao tác ghi vốn bị bỏ qua khi đọc từ snapshot nhất quán giờ đây đã có hiệu lực, và tiền đề của transaction 43 không còn đúng nữa. Mọi thứ thậm chí còn trở nên phức tạp hơn khi một writer chen dữ liệu chưa từng tồn tại trước đó (xem “Phantoms causing write skew”). Tiếp theo, chúng ta sẽ thảo luận về việc phát hiện các phantom write cho SSI.

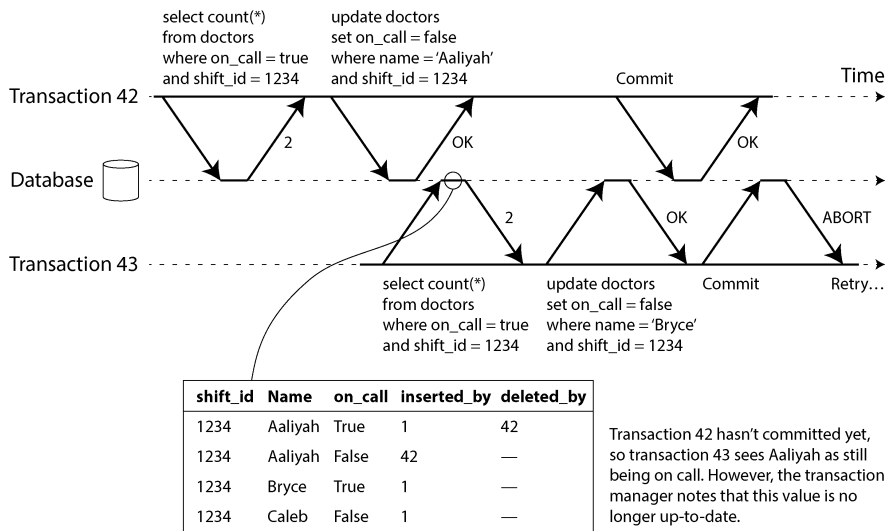


Figure 8-10. *Phát hiện khi một transaction đọc các giá trị đã lỗi thời từ một MVCC snapshot*

Để ngăn chặn sự bất thường này, cơ sở dữ liệu cần theo dõi thời điểm một transaction bỏ qua các lượt ghi của một transaction khác do các quy tắc về khả năng hiển thị (visibility) của MVCC. Khi transaction muốn commit, cơ sở dữ liệu sẽ kiểm tra xem liệu bất kỳ lượt ghi bị bỏ qua nào hiện đã được commit hay chưa. Nếu có, transaction đó phải bị abort.

Tại sao phải đợi đến khi commit? Tại sao không abort transaction 43 ngay lập tức khi phát hiện ra lượt đọc cũ (stale read)? Chà, nếu transaction 43 là một transaction chỉ đọc (read-only), nó sẽ không cần phải bị abort, vì không có rủi ro về write skew. Tại thời điểm transaction 43 thực hiện lượt đọc, cơ sở dữ liệu vẫn chưa biết liệu transaction đó có thực hiện một lượt ghi nào sau đó hay không. Hơn nữa, transaction 42 có thể sẽ bị abort hoặc vẫn chưa được commit tại thời điểm transaction 43 được commit, vì vậy lượt đọc đó cuối cùng có thể không phải là stale. Bằng cách tránh các lệnh abort không cần thiết, SSI bảo toàn khả năng hỗ trợ của snapshot isolation đối với các lượt đọc kéo dài (long-running reads) từ một snapshot nhất quán.

Phát hiện các lượt ghi ảnh hưởng đến các lượt đọc trước đó

Trường hợp thứ hai cần xem xét là một transaction khác sửa đổi dữ liệu sau khi dữ liệu đó đã được đọc. Trường hợp này được minh họa trong Hình 8-11.

Trong ngữ cảnh của 2PL, chúng ta đã thảo luận về index-range locks (xem mục “Index-range locks”), cho phép cơ sở dữ liệu khóa quyền truy cập vào tất cả các hàng khớp với một truy vấn tìm kiếm, chẳng hạn như `WHERE shift_id = 1234`.

Chúng ta có thể sử dụng một kỹ thuật tương tự ở đây, ngoại trừ việc các khóa SSI không chặn các transaction khác.

Trong Hình 8-11, cả transaction 42 và 43 đều tìm kiếm các bác sĩ trực trong ca 1234. Nếu có một index trên `shift_id`, cơ sở dữ liệu có thể sử dụng index entry 1234 để ghi lại thực tế rằng các transaction 42 và 43 đã đọc dữ liệu này. (Nếu không có index, thông tin này có thể được theo dõi ở cấp độ table.) Thông tin này chỉ cần được giữ lại trong một khoảng thời gian; sau khi một transaction đã kết thúc (committed hoặc aborted), và tất cả các transaction đồng thời đã kết thúc, cơ sở dữ liệu có thể quên đi dữ liệu mà nó đã đọc.

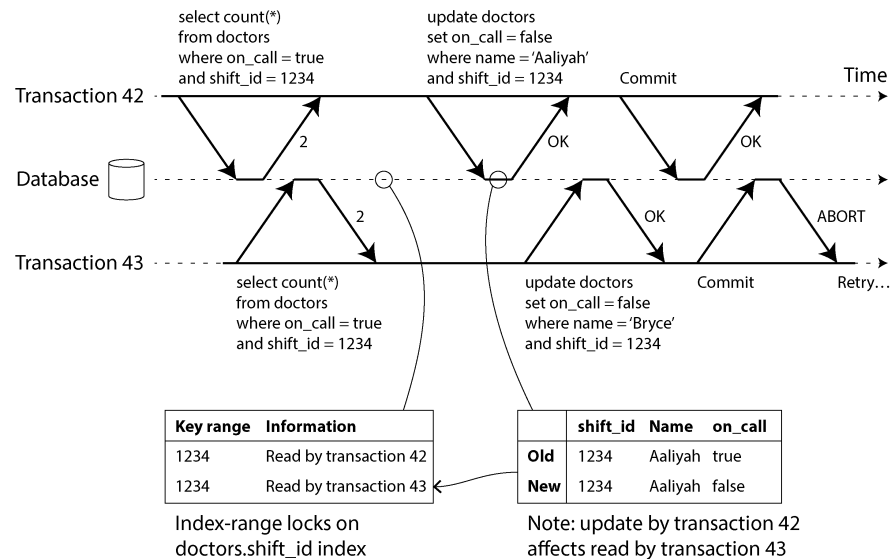


Figure 8-11. Trong serializable snapshot isolation, việc phát hiện khi một transaction sửa đổi các dữ liệu mà một transaction khác đã đọc

Khi một transaction thực hiện ghi vào cơ sở dữ liệu, nó phải kiểm tra trong các index để tìm bất kỳ transaction nào khác đã đọc dữ liệu bị ảnh hưởng gần đây. Quá trình này tương tự như việc giành một write lock trên phạm vi key bị ảnh hưởng, nhưng thay vì chặn cho đến khi các trình đọc đã commit, lock này đóng vai trò như một dây báo (tripwire); nó chỉ đơn giản thông báo cho các transaction rằng dữ liệu mà chúng đã đọc có thể không còn là dữ liệu mới nhất nữa.

Trong Hình 8-11, transaction 43 thông báo cho transaction 42 rằng lần đọc trước đó của nó đã bị lỗi thời, và ngược lại. Transaction 42 là bên commit đầu tiên và nó thành công; mặc dù lệnh ghi của transaction 43 đã ảnh hưởng đến 42, nhưng 43 vẫn chưa commit, nên lệnh ghi đó vẫn chưa có hiệu lực. Tuy nhiên, khi transaction 43 muốn commit, lệnh ghi xung đột từ 42 đã được commit xong, vì vậy 43 phải abort.

Hiệu năng của serializable snapshot isolation

Như thường lệ, nhiều chi tiết kỹ thuật ảnh hưởng đến việc một thuật toán hoạt động tốt như thế nào trong thực tế. Ví dụ, một đánh đổi (trade-off) là độ chi tiết (granularity) mà tại đó các hoạt động đọc và ghi của transaction được theo dõi. Nếu cơ sở dữ liệu theo dõi hoạt động của từng transaction một cách cực kỳ chi tiết, nó có thể xác định chính xác những transaction nào cần phải abort, nhưng chi phí quản lý (bookkeeping overhead) có thể trở nên rất lớn. Việc theo dõi ít chi tiết hơn sẽ nhanh hơn, nhưng nó có thể dẫn đến việc nhiều transaction bị abort hơn mức thực sự cần thiết.

Trong một số trường hợp, việc một transaction đọc thông tin đã bị ghi đè bởi một transaction khác là có thể chấp nhận được. Tùy thuộc vào những gì khác đã xảy ra, đôi khi ta có thể chứng minh được rằng kết quả của việc thực thi vẫn đảm bảo tính serializable. PostgreSQL sử dụng lý thuyết này để giảm số lượng các lệnh abort không cần thiết [14, 56].

So với 2PL, ưu điểm lớn của serializable snapshot isolation là một transaction không cần phải chặn để chờ các lock đang được giữ bởi một transaction khác. Tương tự như snapshot isolation, các trình ghi không chặn các trình đọc, và ngược lại. Nguyên tắc thiết kế này giúp latency của truy vấn trở nên dễ dự đoán hơn nhiều và ít biến động hơn. Đặc biệt, các truy vấn chỉ đọc có thể chạy trên một snapshot nhất quán mà không yêu cầu bất kỳ lock nào, điều này rất hấp dẫn đối với các khối lượng công việc (workload) thiên về đọc.

So với thực thi tuần tự (serial execution), serializable snapshot isolation không bị giới hạn bởi throughput của một lõi CPU duy nhất—ví dụ, FoundationDB phân phối việc phát hiện các xung đột serialization trên nhiều máy, cho phép nó scale lên throughput rất cao. Ngay cả khi dữ liệu có thể được sharding trên nhiều máy, các transaction vẫn có thể đọc và ghi dữ liệu trên nhiều shard trong khi vẫn đảm bảo tính isolation serializable.

So với nonserializable snapshot isolation, nhu cầu kiểm tra các vi phạm tính serializability tạo ra một số chi phí hiệu năng (performance overheads). Việc những chi phí này lớn đến mức nào vẫn là một vấn đề gây tranh cãi: một số người tin rằng việc kiểm tra tính serializability là không đáng [72], trong khi những người khác tin rằng hiệu năng của tính serializability hiện nay đã tốt đến mức không còn cần phải sử dụng snapshot isolation yếu hơn nữa [69].

Tỷ lệ abort ảnh hưởng đáng kể đến hiệu năng tổng thể của SSI. Ví dụ, một transaction thực hiện đọc và ghi dữ liệu trong một khoảng thời gian dài có khả năng cao sẽ gặp xung đột và bị abort, vì vậy SSI yêu cầu các transaction read/write phải tương đối ngắn (các transaction chỉ đọc chạy lâu - long-running read-only transactions - thì vẫn ổn). Tuy nhiên, SSI ít nhạy cảm với các transaction chậm hơn so với 2PL hoặc thực thi tuần tự.

Distributed Transactions

Trong một transaction đơn node (single-node transaction), bạn có một máy duy nhất chịu trách nhiệm thực thi logic của transaction, chẳng hạn như các thuật toán concurrency control để đảm bảo tính isolation của transaction. Nếu cơ sở dữ liệu của bạn sử dụng single-leader replication, việc thực thi transaction chỉ diễn ra trên leader, và các follower chỉ đơn giản là áp dụng log của các lệnh ghi đã được commit bởi các transaction trên leader.

Tuy nhiên, chuyện gì sẽ xảy ra nếu có nhiều node tham gia vào một transaction? Ví dụ, có thể bạn có một transaction cần tác động đến nhiều shard của một cơ sở dữ liệu đã được sharding, hoặc một global secondary index (trong đó entry của index có thể nằm ở một node khác với dữ liệu chính; xem “Sharding and Secondary Indexes”). Điều này được gọi là một *distributed transaction* (giao dịch phân tán).

Các thuật toán để kiểm soát concurrency trong các distributed transaction về cơ bản tương tự như các thuật toán kiểm soát concurrency trên một node đơn lẻ. Trước đó chúng ta đã thảo luận về việc thực thi tuần tự trên các cơ sở dữ liệu sharded; 2PL hoạt động được trong môi trường phân tán, và đối với SSI thì đã có các bộ kiểm tra serializability phân tán [8]. Chúng ta sẽ không đi sâu vào chi tiết hơn về những vấn đề này.

Tuy nhiên, việc đạt được tính atomicity trong một distributed transaction là một thách thức hoàn toàn mới, và đó là trọng tâm của phần còn lại trong chương này.

Đối với các transaction trên một node đơn lẻ, tính atomicity thường được thực hiện bởi storage engine. Khi client yêu cầu database node commit transaction, database sẽ làm cho các thao tác write của transaction trở nên durable (thường là trong một write-ahead log; xem “Making B-trees reliable”) và sau đó append một commit record vào log trên đĩa. Nếu database bị crash giữa quá trình này, transaction sẽ được khôi phục từ log khi node khởi động lại. Nếu commit record đã được ghi thành công xuống đĩa trước khi crash, transaction được coi là đã commit; nếu không, bất kỳ thao tác write nào từ transaction đó đều sẽ được rollback.

Do đó, trên một node đơn lẻ, việc commit transaction phụ thuộc quan trọng vào *thứ tự* mà dữ liệu được ghi bền vững xuống đĩa: đầu tiên là dữ liệu, sau đó mới đến commit record [22]. Thời điểm quyết định then chốt cho việc transaction sẽ commit hay abort xảy ra khi đĩa hoàn tất việc ghi commit record—trước thời điểm đó, việc abort vẫn có thể xảy ra (do crash), nhưng sau thời điểm đó, transaction đã được commit (ngay cả khi database bị crash). Như vậy, chính một thiết bị duy nhất (bộ điều khiển của một ổ đĩa cụ thể, được gắn vào một node cụ thể) là thứ làm cho việc commit trở nên atomic.

Trong một distributed transaction, việc xác định xem một transaction đã commit hay chưa không hề đơn giản như vậy. Ví dụ, khi một transaction muốn commit, việc chỉ

gửi một yêu cầu commit tới tất cả các node và thực hiện commit transaction một cách độc lập trên từng node là không đủ. Việc commit có thể dễ dàng thành công trên một số node nhưng lại thất bại trên các node khác (như được hiển thị trong Hình 8-12) vì nhiều lý do khác nhau:

- Một số node có thể phát hiện ra sự vi phạm ràng buộc hoặc xung đột, khiến việc abort là cần thiết, trong khi các node khác vẫn có thể commit thành công.
- Một số yêu cầu commit có thể bị thất lạc trong mạng, dẫn đến việc cuối cùng phải abort do timeout, trong khi các yêu cầu commit khác vẫn truyền đến được.
- Một số node có thể bị crash trước khi commit record được ghi đầy đủ và thực hiện rollback transaction khi khôi phục, trong khi các node khác lại commit thành công.

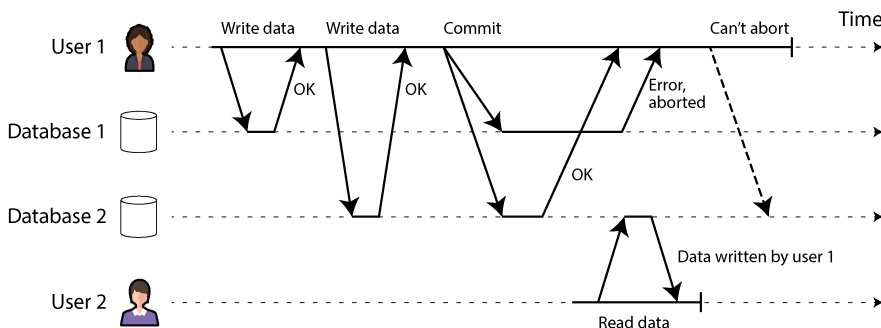


Figure 8-12. Khi một transaction liên quan đến nhiều node cơ sở dữ liệu, nó có thể commit trên một số node nhưng lại thất bại trên các node khác.

Nếu một số node commit transaction nhưng các node khác lại abort, các node sẽ trở nên không nhất quán với nhau. Và một khi một transaction đã được commit trên một node, nó không thể bị rút lại nếu sau đó phát hiện ra rằng nó đã bị abort trên một node khác. Điều này là do một khi dữ liệu đã được commit, nó sẽ trở nên hiển thị đối với các transaction khác dưới mức isolation read-committed hoặc mạnh hơn. Ví dụ, trong Hình 8-12, vào thời điểm người dùng 1 nhận thấy commit của mình đã thất bại trên database 1, người dùng 2 đã đọc dữ liệu từ cùng một transaction đó trên database 2. Nếu transaction của người dùng 1 sau đó bị abort, transaction của người dùng 2 cũng sẽ phải bị hoàn tác, vì nó dựa trên dữ liệu mà về sau lại được tuyên bố là chưa từng tồn tại.

Một cách tiếp cận tốt hơn là đảm bảo rằng các node tham gia vào một transaction hoặc là tất cả cùng commit hoặc là tất cả cùng abort, nhằm ngăn chặn sự lẫn lộn giữa hai trạng thái này. Việc đạt được điều này được gọi là vấn đề *atomic commitment* (cam kết nguyên tử).

Two-Phase Commit

Two-phase commit là một thuật toán để đạt được việc commit transaction nguyên tử trên nhiều node. Đây là một thuật toán kinh điển trong các distributed database [13, 73, 74]. 2PC được sử dụng nội bộ trong một số database và cũng được cung cấp cho các ứng dụng dưới dạng *XA transactions* [75] (ví dụ như được hỗ trợ bởi Java Transaction API) hoặc thông qua WS-AtomicTransaction cho các SOAP web services [76, 77].

Luồng hoạt động cơ bản của 2PC được minh họa trong Hình 8-13. Thay vì chỉ có một yêu cầu commit duy nhất như đối với transaction trên một node, quá trình commit/abort trong 2PC được chia thành hai phase (vì vậy mới có tên gọi này).

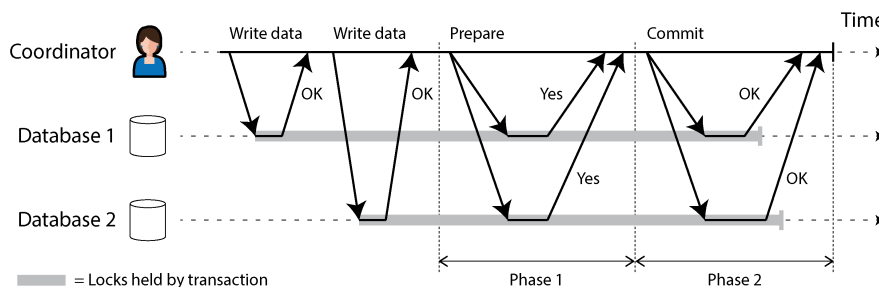


Figure 8-13. Một lần thực thi 2PC thành công

2PC sử dụng một thành phần mới vốn không thường xuất hiện trong các transaction đơn node: một *coordinator* (còn được gọi là *transaction manager*). *coordinator* thường được triển khai dưới dạng một thư viện bên trong chính tiến trình ứng dụng đang yêu cầu transaction (ví dụ: được nhúng trong một Java EE container), nhưng nó cũng có thể là một tiến trình hoặc service riêng biệt. Các ví dụ về các *coordinator* như vậy bao gồm Narayana, JOTM, BTM, và MSDTC.

Khi sử dụng 2PC, một distributed transaction bắt đầu bằng việc ứng dụng đọc và ghi dữ liệu trên nhiều node cơ sở dữ liệu như bình thường. Chúng ta gọi các node cơ sở dữ liệu này là các *participants* trong transaction. Khi ứng dụng đã sẵn sàng để commit, *coordinator* bắt đầu phase 1 bằng cách gửi một yêu cầu *prepare* đến từng node, hỏi xem chúng có khả năng commit hay không. Sau đó, *coordinator* theo dõi các phản hồi từ các *participants*:

- Nếu tất cả các *participants* đều trả lời yes, cho biết chúng đã sẵn sàng để commit, *coordinator* sẽ gửi một yêu cầu *commit* trong phase 2, và việc commit sẽ diễn ra.
- Nếu bất kỳ *participant* nào trả lời no, *coordinator* sẽ gửi một yêu cầu *abort* đến tất cả các node trong phase 2.

Quá trình này hơi giống với nghi lễ đám cưới truyền thống trong văn hóa phương Tây: người chủ hôn hỏi từng người trong cặp đôi xem họ có muốn kết hôn với đối

phương hay không, và thường nhận được câu trả lời "I do" từ cả hai. Sau khi nhận được cả hai sự xác nhận, người chủ hôn tuyên bố cặp đôi đã kết hôn — tức là transaction đã được commit, và sự kiện hạnh phúc này được thông báo đến tất cả những người tham dự. Nếu một trong hai người không nói yes, nghi lễ sẽ bị hủy bỏ [78].

Một hệ thống của những lời hứa

Từ mô tả ngắn gọn này, có thể không rõ tại sao 2PC lại đảm bảo tính atomicity, trong khi việc commit một phase trên nhiều node thì không. Chắc chắn là các yêu cầu *prepare* và *commit* cũng có thể dễ dàng bị thất lạc trong trường hợp hai phase. Điều gì làm cho 2PC trở nên khác biệt?

Để hiểu tại sao nó hoạt động, chúng ta phải phân tích quá trình này chi tiết hơn một chút:

1. Khi ứng dụng muốn bắt đầu một distributed transaction, nó yêu cầu một transaction ID từ *coordinator*. Transaction ID này là duy nhất trên phạm vi toàn cầu.
2. Ứng dụng bắt đầu một transaction đơn node trên mỗi *participant* và đính kèm transaction ID duy nhất toàn cầu đó vào transaction đơn node. Tất cả các thao tác đọc và ghi đều được thực hiện trong một trong các transaction đơn node này. Nếu có bất kỳ điều gì trục trặc ở giai đoạn này (ví dụ: một node bị crash hoặc một yêu cầu bị timeout), *coordinator* hoặc bất kỳ *participant* nào cũng có thể thực hiện *abort*.
3. Khi ứng dụng đã sẵn sàng để commit, *coordinator* gửi một yêu cầu *prepare* đến tất cả các *participants*, kèm theo transaction ID toàn cầu. Nếu bất kỳ yêu cầu nào trong số này thất bại hoặc bị timeout, *coordinator* sẽ gửi một yêu cầu *abort* cho transaction ID đó đến tất cả các *participants*.
4. Khi một *participant* nhận được yêu cầu *prepare*, nó phải đảm bảo rằng nó chắc chắn có thể commit transaction trong mọi tình huống. Điều này bao gồm việc ghi tất cả dữ liệu transaction vào đĩa (một sự cố crash, mất điện, hoặc hết dung lượng đĩa không phải là lý do chấp nhận được để từ chối commit sau đó) và kiểm tra bất kỳ xung đột hoặc vi phạm ràng buộc nào. Bằng cách trả lời yes cho *coordinator*, node đó hứa sẽ commit transaction mà không gặp lỗi nếu được yêu cầu. Nói cách khác, *participant* từ bỏ quyền *abort* transaction, nhưng chưa thực sự commit nó.
5. Khi *coordinator* đã nhận được phản hồi cho tất cả các yêu cầu *prepare*, nó đưa ra một quyết định dứt khoát về việc nên commit hay *abort* transaction (chỉ commit nếu tất cả các *participants* đều bình chọn yes). *coordinator* phải ghi quyết định đó vào transaction log của nó trên đĩa để nó biết mình đã quyết định thế nào trong trường hợp sau đó bị crash. Điều này được gọi là *commit point*.

6. Một khi quyết định của coordinator đã được ghi vào đĩa, yêu cầu commit hoặc abort sẽ được gửi tới tất cả các participant. Nếu yêu cầu này thất bại hoặc bị timeout, coordinator phải thử lại mãi mãi cho đến khi thành công. Không còn đường lui nữa; nếu quyết định là commit, quyết định đó phải được thực thi, bất kể phải thử lại bao nhiêu lần. Nếu một participant bị crash trong thời gian này, transaction sẽ được commit khi nó khôi phục—vì participant đó đã bỏ phiếu yes, nên nó không thể từ chối commit khi khôi phục.

Do đó, giao thức này chứa hai "điểm không thể quay đầu" quan trọng: khi một participant bỏ phiếu yes, nó hứa rằng chắc chắn sẽ có thể commit sau đó (mặc dù coordinator vẫn có thể chọn abort); và một khi coordinator đã quyết định, quyết định đó là không thể đảo ngược. Những lời hứa đó đảm bảo tính atomicity của 2PC. (Việc atomic commit trên một node duy nhất gộp hai sự kiện này làm một: ghi bản ghi commit vào transaction log.)

Quay lại với phép ẩn dụ về hôn nhân, trước khi nói "Tôi đồng ý", bạn và đối tác có quyền abort transaction bằng cách nói "Không đời nào!" (hoặc điều gì đó tương tự). Tuy nhiên, sau khi đã nói "Tôi đồng ý", bạn không thể rút lại tuyên bố đó. Nếu bạn bị ngắt sau khi nói "Tôi đồng ý" và không nghe thấy người chủ hôn tuyên bố hai người đã kết hôn, điều đó cũng không thay đổi sự thật rằng transaction đã được commit. Khi bạn tỉnh lại sau đó, bạn có thể tìm hiểu xem mình đã kết hôn hay chưa bằng cách hỏi người chủ hôn về trạng thái của global transaction ID của mình, hoặc bạn có thể đợi lần thử lại tiếp theo của yêu cầu commit từ người chủ hôn (vì các lần thử lại sẽ tiếp tục diễn ra trong suốt thời gian bạn bất tỉnh).

Lỗi coordinator

Chúng ta đã thảo luận về những gì xảy ra nếu một trong các participant hoặc mạng bị lỗi trong quá trình 2PC: nếu bất kỳ yêu cầu prepare nào thất bại hoặc bị timeout, coordinator sẽ abort transaction; nếu bất kỳ yêu cầu commit hoặc abort nào thất bại, coordinator sẽ thử lại chúng vô thời hạn. Tuy nhiên, việc điều gì xảy ra nếu coordinator bị crash thì ít rõ ràng hơn.

Nếu coordinator gặp lỗi trước khi gửi các yêu cầu prepare, một participant có thể abort transaction một cách an toàn. Nhưng một khi participant đã nhận được yêu cầu prepare và bỏ phiếu yes, nó không còn có thể đơn phương abort nữa—it phải đợi để nghe phản hồi từ coordinator xem transaction đã được commit hay bị abort. Nếu coordinator bị crash hoặc mạng bị lỗi tại thời điểm này, participant không thể làm gì khác ngoài việc chờ đợi. Một transaction của participant trong trạng thái này được gọi là *in doubt* hoặc *uncertain* (không chắc chắn).

Tình huống này được minh họa trong Hình 8-14. Trong ví dụ cụ thể này, coordinator thực sự đã quyết định commit, và database 2 đã nhận được yêu cầu commit. Tuy nhiên, coordinator đã bị crash trước khi kịp gửi yêu cầu commit tới database 1, vì vậy database 1 không biết nên commit hay abort. Ngay cả khi bị

timeout cũng không giúp ích gì ở đây: nếu database 1 đơn phương abort sau khi bị timeout, nó sẽ dẫn đến tình trạng không nhất quán với database 2, vốn đã commit. Tương tự, việc đơn phương commit cũng không an toàn, vì một participant khác có thể đã abort.

Nếu không nhận được phản hồi từ coordinator, một participant không có cách nào để biết nên commit hay abort. Về nguyên tắc, các participant có thể giao tiếp với nhau để tìm hiểu xem mỗi participant đã bỏ phiếu như thế nào và đi đến một thỏa thuận, nhưng đó không phải là một phần của giao thức 2PC.

Cách duy nhất để 2PC có thể hoàn tất là chờ coordinator khôi phục. Đây là lý do tại sao coordinator phải ghi quyết định commit hoặc abort của nó vào một transaction log trên đĩa trước khi gửi các yêu cầu commit hoặc abort tới các participant: khi coordinator khôi phục, nó xác định trạng thái của tất cả các transaction đang ở trạng thái *in-doubt* bằng cách đọc transaction log của nó. Bất kỳ transaction nào không có bản ghi commit trong log của coordinator đều bị abort. Do đó, điểm commit của 2PC quy về một quá trình atomic commit trên một node duy nhất thông thường của coordinator.

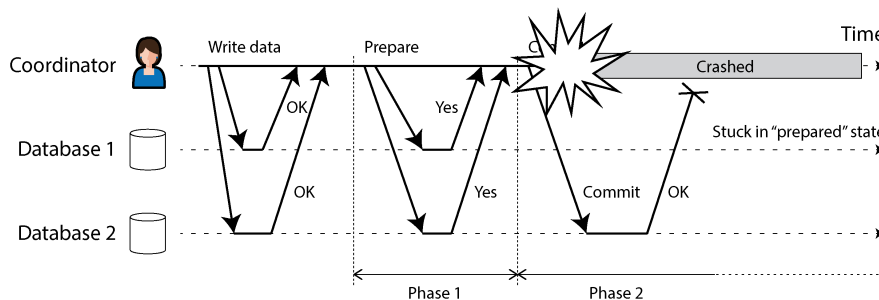


Figure 8-14. Coordinator bị crash sau khi các participant bỏ phiếu yes. Database 1 không biết nên commit hay abort.

Hơn nữa, nếu đĩa của coordinator bị lỗi và log của nó bị mất, hệ thống sẽ không có cách nào để tự động khôi phục. Lựa chọn duy nhất là một quản trị viên phải thực hiện commit hoặc abort các transaction đang ở trạng thái *in-doubt* một cách thủ công. Nếu chỉ có phần gần nhất của transaction log bị mất, coordinator đang khôi phục có thể tin rằng các transaction đã được commit trước đó thực tế vẫn chưa được commit và cố gắng abort chúng, từ đó vi phạm tính atomicity.

Three-phase commit

2PC được gọi là một giao thức atomic commit *blocking* (chặn) vì 2PC có thể bị kẹt trong khi chờ coordinator khôi phục. Việc làm cho một giao thức atomic commit trở thành *nonblocking* (không chặn), để nó không bị kẹt nếu một node bị lỗi, là điều

hoàn toàn có thể. Tuy nhiên, việc thực hiện điều này trong thực tế không hề đơn giản.

Thay thế cho 2PC, một thuật toán được gọi là *three-phase commit* (3PC) đã được đề xuất [13, 79]. Tuy nhiên, 3PC giả định một mạng có độ trễ hữu hạn và các node có thời gian phản hồi hữu hạn; trong hầu hết các hệ thống thực tế với độ trễ mạng không giới hạn và các quá trình tạm dừng (xem Chương 9), 3PC không thể đảm bảo tính atomicity.

Một giải pháp tốt hơn trong thực tế là thay thế coordinator đơn node bằng một giao thức consensus có khả năng chịu lỗi (fault-tolerant). Chúng ta sẽ tìm hiểu cách thực hiện điều này trong Chương 10.

Distributed Transactions giữa các hệ thống khác nhau

Distributed transactions và 2PC có danh tiếng trái chiều. Một mặt, chúng được xem là cung cấp một sự đảm bảo an toàn quan trọng mà nếu không có chúng thì rất khó đạt được; mặt khác, chúng bị chỉ trích vì gây ra các vấn đề vận hành, làm giảm hiệu năng và hứa hẹn nhiều hơn những gì chúng có thể thực hiện [80, 81, 82, 83]. Nhiều dịch vụ cloud chọn không triển khai distributed transactions vì những vấn đề vận hành mà chúng gây ra [84].

Một số cách triển khai distributed transactions phải chịu một hình phạt nặng nề về hiệu năng. Phần lớn chi phí hiệu năng vốn có trong 2PC là do các thao tác *fsync* bổ sung cần thiết để khôi phục sau sự cố (crash recovery) và các lượt truyền tải mạng (network round trips) bổ sung.

Tuy nhiên, thay vì bác bỏ hoàn toàn distributed transactions, chúng ta nên xem xét chúng chi tiết hơn, bởi vì có những bài học quan trọng có thể rút ra từ chúng. Trước hết, chúng ta nên chính xác về ý nghĩa của cụm từ “distributed transactions”. Hai loại distributed transactions khá khác biệt thường bị đánh đồng với nhau:

Distributed transactions nội bộ của database

Một số distributed databases (tức là các cơ sở dữ liệu sử dụng replication và sharding trong cấu hình tiêu chuẩn của chúng) hỗ trợ các transaction nội bộ giữa các node của cơ sở dữ liệu đó. Ví dụ, YugabyteDB, TiDB, FoundationDB, Spanner, VoltDB, Cassandra, và engine lưu trữ NDB của MySQL Cluster có hỗ trợ transaction nội bộ như vậy. Trong trường hợp này, tất cả các node tham gia vào transaction đều đang chạy cùng một phần mềm cơ sở dữ liệu.

Các transaction phân tán không đồng nhất (Heterogeneous distributed transactions)

Trong một transaction *heterogeneous* (không đồng nhất), các bên tham gia là hai hoặc nhiều công nghệ khác nhau—ví dụ, hai cơ sở dữ liệu từ các nhà cung cấp khác nhau, hoặc thậm chí là các hệ thống không phải cơ sở dữ liệu như message

brokers. Một distributed transaction xuyên suốt các hệ thống này phải đảm bảo atomic commit, ngay cả khi các hệ thống có thể hoàn toàn khác nhau về bên trong (under the hood).

Các transaction nội bộ của cơ sở dữ liệu không nhất thiết phải tương thích với bất kỳ hệ thống nào khác, vì vậy chúng có thể sử dụng bất kỳ giao thức nào và áp dụng các tối ưu hóa dành riêng cho công nghệ cụ thể đó. Vì lý do đó, các distributed transactions nội bộ của cơ sở dữ liệu thường có thể hoạt động khá tốt. Mặt khác, các transaction trải dài qua các công nghệ heterogeneous thì thách thức hơn nhiều. Chúng ta sẽ tập trung vào chúng ở đây và thảo luận về các distributed transactions nội bộ của cơ sở dữ liệu trong mục tiếp theo.

Xử lý message exactly-once

Các distributed transaction (giao dịch phân tán) không đồng nhất cho phép tích hợp các hệ thống đa dạng theo những cách mạnh mẽ. Ví dụ, một tin nhắn từ một message queue có thể được xác nhận là đã xử lý nếu và chỉ nếu transaction của cơ sở dữ liệu để xử lý tin nhắn đó được commit thành công. Điều này được thực hiện bằng cách commit nguyên tử việc xác nhận tin nhắn và các thao tác ghi vào cơ sở dữ liệu trong cùng một transaction duy nhất. Với sự hỗ trợ của distributed transaction, điều này khả thi ngay cả khi message broker và cơ sở dữ liệu là hai công nghệ không liên quan chạy trên các máy khác nhau.

Nếu việc chuyển tin nhắn hoặc transaction của cơ sở dữ liệu thất bại, cả hai đều bị abort để message broker có thể gửi lại tin nhắn một cách an toàn sau đó. Do đó, bằng cách commit nguyên tử tin nhắn và các side effect (tác dụng phụ) của việc xử lý nó, chúng ta có thể đảm bảo rằng tin nhắn được xử lý *một cách hiệu quả* đúng một lần (*exactly-once*), ngay cả khi nó cần thực hiện lại vài lần trước khi thành công. Việc abort sẽ loại bỏ bất kỳ side effect nào của transaction đã hoàn thành thành một phần. Điều này được gọi là *exactly-once semantics*.

Tuy nhiên, một distributed transaction như vậy chỉ khả thi nếu tất cả các hệ thống bị ảnh hưởng bởi transaction đó đều có thể sử dụng cùng một atomic commit protocol. Ví dụ, giả sử một side effect của việc xử lý tin nhắn là gửi một email, và email server không hỗ trợ 2PC. Có thể xảy ra tình trạng email bị gửi hai hoặc nhiều lần nếu việc xử lý tin nhắn thất bại và được thử lại. Nhưng nếu tất cả các side effect của việc xử lý tin nhắn đều được rollback khi transaction bị abort, bước xử lý có thể được thử lại một cách an toàn như thể chưa có chuyện gì xảy ra.

Chúng ta sẽ quay lại chủ đề *exactly-once semantics* ở phần sau của chương này. Trước tiên, hãy cùng xem xét atomic commit protocol cho phép thực hiện các distributed transaction không đồng nhất như vậy.

XA transactions

X/Open XA (viết tắt của *eXtended Architecture*) là một tiêu chuẩn để triển khai 2PC giữa các công nghệ không đồng nhất [75]. Nó được giới thiệu vào năm 1991 và đã được triển khai rộng rãi. XA được hỗ trợ bởi nhiều cơ sở dữ liệu quan hệ truyền thống (bao gồm PostgreSQL, MySQL, Db2, SQL Server, và Oracle) và các message broker (bao gồm ActiveMQ, HornetQ, MSMQ, và IBM MQ).

XA không phải là một `network protocol`—nó chỉ đơn thuần là một C API để giao tiếp với một `transaction coordinator`. Các bindings cho API này cũng tồn tại trong các ngôn ngữ khác; ví dụ, trong thế giới của các ứng dụng Java EE, XA transactions được triển khai bằng cách sử dụng Java Transaction API (JTA), và JTA lại được hỗ trợ bởi nhiều driver cho cơ sở dữ liệu sử dụng Java Database Connectivity (JDBC) cũng như các driver cho message broker sử dụng các API của Java Message Service (JMS).

XA giả định rằng ứng dụng của bạn sử dụng một `network driver` hoặc `client library` để giao tiếp với các `participant` cơ sở dữ liệu hoặc các dịch vụ nhắn tin. Nếu driver hỗ trợ XA, điều đó có nghĩa là nó gọi XA API để tìm hiểu xem một thao tác có phải là một phần của `distributed transaction` hay không—và nếu đúng, nó sẽ gửi thông tin cần thiết đến máy chủ cơ sở dữ liệu. Driver cũng cung cấp các callback mà thông qua đó coordinator có thể yêu cầu các `participant` chuẩn bị (`prepare`), `commit`, hoặc `abort`.

`transaction coordinator` triển khai XA API. Tiêu chuẩn này không chỉ định cách nó nên được triển khai, nhưng trong thực tế, coordinator thường chỉ đơn giản là một `library` được nạp vào cùng một `process` với ứng dụng đang thực hiện `transaction` (chứ không phải là một `service` riêng biệt). Nó theo dõi các `participant` trong một `transaction`, thu thập phản hồi của các `participant` sau khi yêu cầu chúng chuẩn bị (thông qua một callback vào driver), và sử dụng một log trên đĩa cục bộ để theo dõi quyết định `commit/abort` cho mỗi `transaction`.

Nếu quá trình ứng dụng bị crash, hoặc máy chủ mà ứng dụng đang chạy bị hỏng, coordinator cũng sẽ mất theo. Bất kỳ `participant` nào có các `transaction` đã chuẩn bị nhưng chưa được `commit` sẽ rơi vào trạng thái nghi vấn (`in-doubt`). Vì log của coordinator nằm trên đĩa cục bộ của `application server`, nên server đó phải được khởi động lại, và thư viện coordinator phải đọc log để khôi phục kết quả `commit/abort` của mỗi `transaction`. Chỉ sau đó, coordinator mới có thể sử dụng các XA callback của database driver để yêu cầu các `participant` `commit` hoặc `abort` một cách phù hợp. Database server không thể liên lạc trực tiếp với coordinator, vì mọi giao tiếp đều phải thông qua thư viện client của nó.

Giữ lock khi đang trong trạng thái nghi vấn

Tại sao chúng ta lại quan tâm nhiều đến việc một transaction bị kẹt trong trạng thái nghi vấn đến vậy? Chẳng lẽ các phần còn lại của hệ thống không thể tiếp tục công việc và bỏ qua transaction đang bị nghi vấn — thứ mà cuối cùng rồi cũng sẽ được dọn dẹp hay sao?

Vấn đề nằm ở việc *locking*. Như đã thảo luận trong mục “Read Committed”, các database transaction thường chiếm các exclusive lock ở cấp độ dòng (row-level) trên bất kỳ dòng nào mà chúng sửa đổi, để ngăn chặn dirty writes. Nếu bạn muốn có isolation level là serializable, một database sử dụng 2PL cũng sẽ phải chiếm các shared lock trên bất kỳ dòng nào được transaction *đọc*.

Database không thể giải phóng các lock đó cho đến khi transaction được commit hoặc abort (được minh họa bằng vùng tô bóng trong Hình 8-13). Do đó, khi sử dụng 2PC, một transaction phải giữ các lock trong suốt thời gian nó ở trạng thái nghi vấn. Nếu coordinator bị crash và mất 20 phút để khởi động lại, các lock đó sẽ bị giữ trong 20 phút. Nếu log của coordinator bị mất hoàn toàn vì lý do nào đó, các lock đó sẽ bị giữ mãi mãi — hoặc ít nhất là cho đến khi tình huống được quản trị viên giải quyết thủ công.

Trong khi các lock đang bị giữ, không có transaction nào khác có thể sửa đổi các dòng đó. Tùy thuộc vào isolation level, các transaction khác thậm chí có thể bị chặn không cho đọc các dòng này. Do đó, các transaction khác không thể đơn giản là tiếp tục công việc của chúng — nếu chúng muốn truy cập vào cùng dữ liệu đó, chúng sẽ bị chặn. Điều này có thể khiến các phần lớn ứng dụng của bạn trở nên không khả dụng cho đến khi transaction đang bị nghi vấn được giải quyết.

Khôi phục sau lỗi của coordinator

Về lý thuyết, nếu coordinator bị crash và được khởi động lại, nó sẽ khôi phục trạng thái của mình một cách sạch sẽ từ log và giải quyết bất kỳ transaction nghi vấn nào. Tuy nhiên, trong thực tế, các transaction nghi vấn *mò côi* (*orphaned*) vẫn xảy ra [85, 86] — tức là các transaction mà coordinator không thể quyết định kết quả vì bất kỳ lý do gì (ví dụ: vì transaction log đã bị mất hoặc bị hỏng do lỗi phần mềm). Những transaction này không thể được giải quyết tự động, vì vậy chúng nằm mãi trong database, giữ các lock và chặn các transaction khác.

Ngay cả việc khởi động lại các database server cũng không giải quyết được vấn đề này, vì một triển khai 2PC chính xác phải bảo toàn các lock của một transaction đang nghi vấn ngay cả sau khi khởi động lại (nếu không, nó sẽ có nguy cơ vi phạm đảm bảo về tính atomicity). Đây là một tình huống rất nan giải.

Cách duy nhất là quản trị viên phải quyết định thủ công xem nên commit hay roll back các transaction. Quản trị viên phải kiểm tra các participant của mỗi transaction nghi vấn, xác định xem có bất kỳ participant nào đã commit hoặc abort rồi hay chưa,

và sau đó áp dụng kết quả tương tự cho các participant khác. Việc giải quyết vấn đề này có khả năng đòi hỏi rất nhiều nỗ lực thủ công, và rất có thể phải thực hiện dưới áp lực lớn về thời gian và căng thẳng trong một sự cố production nghiêm trọng (nếu không, tại sao coordinator lại rơi vào tình trạng tồi tệ như vậy?).

Nhiều triển khai XA có một lối thoát khẩn cấp gọi là *quyết định heuristic* (*heuristic decisions*): cho phép một participant đơn phương quyết định abort hoặc commit một transaction đang nghi vấn mà không cần quyết định dứt khoát từ coordinator [75]. Để nói rõ hơn, *heuristic* ở đây là một cách nói giảm nói tránh cho việc *có khả năng phá vỡ tính atomicity*, vì quyết định heuristic vi phạm hệ thống các cam kết trong 2PC. Do đó, các quyết định heuristic chỉ nhằm mục đích thoát khỏi các tình huống thảm khốc chứ không phải để sử dụng thông thường.

Các vấn đề với giao dịch XA

Một coordinator đơn node là một điểm lỗi duy nhất (single point of failure) cho toàn bộ hệ thống, và việc biến nó thành một phần của application server cũng gây ra nhiều vấn đề vì các log của coordinator trên đĩa cục bộ trở thành một phần quan trọng của trạng thái hệ thống bền vững—quan trọng tương đương với chính các cơ sở dữ liệu.

Về nguyên tắc, coordinator của một giao dịch XA có thể có độ sẵn sàng cao và được replication, giống như những gì chúng ta kỳ vọng ở bất kỳ cơ sở dữ liệu quan trọng nào khác. Thật không may, điều này vẫn không giải quyết được vấn đề cơ bản của XA, đó là nó không cung cấp cách nào để coordinator và các participant của một giao dịch có thể giao tiếp trực tiếp với nhau. Chúng chỉ có thể giao tiếp thông qua mã nguồn ứng dụng đã gọi giao dịch và các database driver mà qua đó nó gọi các participant.

Ngay cả khi coordinator được replication, mã nguồn ứng dụng do đó sẽ trở thành một điểm lỗi duy nhất. Giải quyết vấn đề này đòi hỏi phải thiết kế lại hoàn toàn cách mã nguồn ứng dụng được thực thi để làm cho nó có thể được replication hoặc có thể khởi động lại, điều này có lẽ trông tương tự như *durable execution* (xem “Durable Execution and Workflows”). Tuy nhiên, trên thực tế dường như không có công cụ nào áp dụng cách tiếp cận này.

Một vấn đề khác là vì XA cần phải tương thích với một phạm vi rộng các hệ thống dữ liệu, nên nó tất yếu phải là một giải pháp có tính chất “mẫu số chung thấp nhất” (lowest common denominator). Ví dụ, nó không thể phát hiện deadlock giữa các hệ thống khác nhau (vì điều đó đòi hỏi một giao thức chuẩn hóa để các hệ thống trao đổi thông tin về các lock mà mỗi giao dịch đang chờ đợi), và nó không hoạt động với SSI (xem “Serializable Snapshot Isolation”), vì điều đó đòi hỏi một giao thức để xác định các xung đột giữa các hệ thống khác nhau.

Những vấn đề này phần nào là cố hữu khi thực hiện các giao dịch trên các công nghệ không đồng nhất (heterogeneous). Tuy nhiên, việc giữ cho nhiều hệ thống dữ liệu không đồng nhất nhất quán với nhau vẫn là một vấn đề thực tế và quan trọng, vì vậy chúng ta cần tìm một giải pháp khác. Điều này có thể được thực hiện, như chúng ta sẽ thấy trong mục tiếp theo và trong Chương 12.

Giao dịch phân tán nội bộ cơ sở dữ liệu

Như đã giải thích trước đó, có một sự khác biệt lớn giữa các giao dịch phân tán trải dài trên nhiều công nghệ lưu trữ không đồng nhất và các giao dịch nội bộ trong một hệ thống—nghĩa là, nơi tất cả các node tham gia đều là một phần của cùng một cơ sở dữ liệu chạy cùng một phần mềm. Những giao dịch phân tán nội bộ như vậy là một *feature* đặc trưng của các cơ sở dữ liệu "NewSQL" chẳng hạn như CockroachDB [5], TiDB [6], Spanner [7], FoundationDB [8], và YugabyteDB. Một số *message broker*, chẳng hạn như Kafka, cũng hỗ trợ các giao dịch phân tán nội bộ [87].

Nhiều hệ thống trong số này sử dụng 2PC để đảm bảo *atomicity* của các giao dịch ghi vào nhiều *shard*, nhưng chúng không gặp phải các vấn đề tương tự như các giao dịch XA. Bởi vì các giao dịch phân tán của chúng không cần giao tiếp với bất kỳ công nghệ nào khác, chúng tránh được cái bẫy "mẫu số chung thấp nhất"—các nhà thiết kế của những hệ thống này được tự do sử dụng các giao thức tốt hơn, đáng tin cậy hơn và nhanh hơn.

Các vấn đề lớn nhất với XA có thể được khắc phục bằng các cách sau:

- Replicating coordinator, với khả năng tự động failover sang một node coordinator khác nếu node chính bị crash
- Cho phép coordinator và các data shard giao tiếp trực tiếp mà không cần thông qua mã nguồn ứng dụng trung gian
- Replicating các shard tham gia để giảm thiểu rủi ro phải abort một giao dịch do lỗi ở một trong các shard
- Kết hợp giao thức atomic commitment với một giao thức distributed concurrency control hỗ trợ phát hiện deadlock và các read nhất quán giữa các shard

Các thuật toán consensus thường được sử dụng để replication coordinator và các database shard. Chúng ta sẽ thấy trong Chương 10 cách thực hiện atomic commitment cho các giao dịch phân tán bằng cách sử dụng một thuật toán consensus. Các thuật toán này chịu lỗi bằng cách tự động failover từ node này sang node khác mà không cần sự can thiệp của con người, trong khi vẫn tiếp tục đảm bảo các đặc tính strong consistency.

Các isolation level được cung cấp cho các distributed transaction phụ thuộc vào hệ thống, nhưng snapshot isolation [6] và serializable snapshot isolation [5, 8] đều có thể thực hiện được trên các shard.

Xem xét lại việc xử lý message exactly-once

Chúng ta đã thấy trong mục “Xử lý message exactly-once” rằng một trường hợp sử dụng quan trọng của distributed transaction là để đảm bảo một thao tác có hiệu lực đúng một lần (exactly once), ngay cả khi xảy ra crash trong khi nó đang được xử lý và việc xử lý cần phải được thử lại. Nếu bạn có thể commit một transaction một cách nguyên tử giữa một message broker và một database, bạn có thể gửi xác nhận (acknowledge) message cho broker nếu và chỉ nếu nó đã được xử lý thành công và các lệnh ghi vào database phát sinh từ quá trình đó đã được commit.

Tuy nhiên, bạn thực sự không cần đến distributed transaction để đạt được ngữ nghĩa exactly-once. Một cách tiếp cận thay thế như sau, chỉ yêu cầu các transaction bên trong database:

1. Giả sử mỗi message có một ID duy nhất, và trong database, bạn có một bảng chứa các message ID đã được xử lý. Khi bạn bắt đầu xử lý một message từ broker, bạn bắt đầu một transaction mới trên database và kiểm tra message ID đó. Nếu cùng một message ID đã tồn tại trong database, bạn biết rằng nó đã được xử lý, vì vậy bạn có thể gửi xác nhận message cho broker và loại bỏ nó.
2. Nếu message ID chưa có trong database, bạn thêm nó vào bảng. Sau đó, bạn xử lý message, việc này có thể dẫn đến các lệnh ghi bổ sung vào database trong cùng một transaction. Khi bạn hoàn tất việc xử lý message, bạn commit transaction trên database.
3. Sau khi database transaction được commit thành công, bạn có thể gửi xác nhận message cho broker.
4. Sau khi message đã được gửi xác nhận thành công cho broker, bạn biết rằng nó sẽ không cố gắng xử lý lại cùng một message đó nữa, vì vậy bạn có thể xóa message ID khỏi database (trong một transaction riêng biệt).

Nếu bộ xử lý message bị crash trước khi commit database transaction, transaction đó sẽ bị abort và message broker sẽ thử lại việc xử lý. Nếu nó bị crash sau khi đã commit nhưng trước khi gửi xác nhận message cho broker, nó cũng sẽ thử lại việc xử lý, nhưng lần thử lại sẽ thấy message ID đã có trong database và loại bỏ nó. Nếu nó bị crash sau khi đã gửi xác nhận message nhưng trước khi xóa message ID khỏi database, bạn sẽ có một message ID cũ nằm lại đó, điều này không gây hại gì ngoài việc chiếm một chút không gian lưu trữ. Nếu một lần thử lại xảy ra trước khi database transaction bị abort (điều này có thể xảy ra nếu kết nối giữa bộ xử lý message và database bị gián đoạn), một ràng buộc duy nhất (uniqueness constraint) trên bảng

message ID sẽ ngăn chặn việc cùng một message ID bị chen bởi hai transaction đồng thời.

Do đó, để đạt được việc xử lý exactly-once chỉ yêu cầu các transaction bên trong database—tính nguyên tử (atomicity) giữa database và message broker là không cần thiết cho trường hợp sử dụng này. Việc ghi lại message ID trong database giúp việc xử lý message trở nên *idempotent* (có tính lũy đẳng), sao cho việc xử lý message có thể được thử lại một cách an toàn mà không làm lặp lại các side effect của nó. Một cách tiếp cận tương tự được sử dụng trong các framework stream processing như Kafka Streams để đạt được ngữ nghĩa exactly-once, như chúng ta sẽ thấy trong Chương 12.

Nói như vậy, các distributed transaction nội bộ bên trong database vẫn hữu ích cho khả năng mở rộng của các pattern như thế này; ví dụ, chúng cho phép các message ID được lưu trữ trên một shard và dữ liệu chính được cập nhật bởi việc xử lý message được lưu trữ trên các shard khác, đồng thời đảm bảo tính nguyên tử của việc commit transaction trên các shard đó.

Tóm tắt

Transaction là một lớp trừu tượng cho phép một ứng dụng giả định rằng một số vấn đề về concurrency và một số loại lỗi phần cứng cũng như phần mềm không tồn tại. Một nhóm lớn các lỗi được giảm thiểu thành một lệnh *transaction abort* đơn giản, và ứng dụng chỉ cần thử lại.

Trong chương này, chúng ta đã thấy nhiều ví dụ về các vấn đề mà transaction giúp ngăn chặn. Không phải tất cả các ứng dụng đều dễ bị ảnh hưởng bởi tất cả những vấn đề đó; một ứng dụng có các pattern truy cập rất đơn giản, chẳng hạn như chỉ đọc và ghi một bản ghi duy nhất, có lẽ có thể hoạt động mà không cần transaction. Tuy nhiên, đối với các pattern truy cập phức tạp hơn, transaction có thể giảm thiểu đáng kể số lượng các trường hợp lỗi tiềm ẩn mà bạn cần phải tính đến.

Nếu không có transaction, các kịch bản lỗi khác nhau (process bị crash, gián đoạn mạng, mất điện, đầy đĩa, concurrency bất ngờ, v.v.) có nghĩa là dữ liệu có thể trở nên không nhất quán theo nhiều cách khác nhau. Ví dụ, dữ liệu đã được denormalized có thể dễ dàng bị mất đồng bộ với dữ liệu nguồn. Nếu không có transaction, việc suy luận về các tác động mà các truy cập tương tác phức tạp có thể gây ra đối với cơ sở dữ liệu sẽ trở nên rất khó khăn.

Chúng ta đã đi sâu vào chủ đề concurrency control, thảo luận về một số isolation level được sử dụng rộng rãi: cụ thể là *read-committed*, *snapshot* (đôi khi được gọi là *repeatable read*), và *serializable*. Chúng ta đã đặc tả các isolation level đó bằng cách thảo luận về các ví dụ khác nhau về race condition, được tóm tắt trong Bảng 8-1.

Table 8-1. Bảng tóm tắt các anomalies có thể xảy ra tại các isolation level khác nhau

Isolation level	Dirty reads	Read skew	Phantom reads	Lost updates	Write skew
Read uncommitted	× Có thể xảy ra	× Có thể xảy ra	× Có thể xảy ra	× Có thể xảy ra	× Có thể xảy ra
Read committed	✓ Đã ngăn chặn	× Có thể xảy ra	× Có thể xảy ra	× Có thể xảy ra	× Có thể xảy ra
Snapshot isolation	✓ Đã ngăn chặn	✓ Đã ngăn chặn	✓ Đã ngăn chặn	? Phụ thuộc	× Có thể xảy ra
Serializable	✓ Đã ngăn chặn	✓ Đã ngăn chặn	✓ Đã ngăn chặn	✓ Đã ngăn chặn	✓ Đã ngăn chặn

Dưới đây là phần tóm tắt ngắn gọn:

Dirty reads

Một client đọc các dữ liệu mà client khác đã ghi nhưng chưa được commit. Isolation level read-committed và các cấp độ mạnh hơn sẽ ngăn chặn dirty reads.

Dirty writes

Một client ghi đè lên dữ liệu mà client khác đã ghi nhưng chưa được commit. Hầu hết các triển khai transaction đều ngăn chặn dirty writes (vì vậy, nó không được đưa vào bảng).

Read skew

Một client nhìn thấy các phần khác nhau của cơ sở dữ liệu tại các thời điểm khác nhau. Một số trường hợp read skew còn được gọi là *nonrepeatable reads*. Vấn đề này thường được ngăn chặn nhất bằng snapshot isolation, cho phép một transaction đọc từ một snapshot nhất quán tương ứng với một thời điểm cụ thể. Snapshot isolation thường được triển khai bằng MVCC.

Phantom reads

Một transaction đọc các đối tượng khớp với một điều kiện tìm kiếm. Một client khác thực hiện một thao tác ghi làm ảnh hưởng đến kết quả của tìm kiếm đó. Snapshot isolation ngăn chặn các phantom reads trực tiếp, nhưng các phantom trong ngữ cảnh của write skew yêu cầu xử lý đặc biệt, chẳng hạn như sử dụng index-range locks.

Lost updates

Hai client đồng thời thực hiện một chu kỳ read-modify-write. Một client ghi đè lên thao tác ghi của client kia mà không kết hợp các thay đổi của nó, dẫn đến mất dữ liệu. Một số triển khai snapshot isolation ngăn chặn sự bất thường này một cách tự động, trong khi những triển khai khác yêu cầu một lock thủ công (SELECT FOR UPDATE).

Write skew

Một transaction đọc một thứ gì đó, đưa ra quyết định dựa trên giá trị mà nó đã thấy, và ghi quyết định đó vào cơ sở dữ liệu. Tuy nhiên, vào thời điểm việc ghi được thực hiện, tiền đề của quyết định đó không còn đúng nữa. Chỉ có isolation level serializable mới ngăn chặn được sự bất thường này.

Các isolation level yếu bảo vệ chống lại một số sự bất thường đó nhưng để lại cho bạn, lập trình viên ứng dụng, phải xử lý các lỗi khác một cách thủ công (ví dụ: sử dụng explicit locking). Chỉ có isolation level serializable mới bảo vệ chống lại tất cả các vấn đề này. Chúng ta đã thảo luận về ba cách tiếp cận để triển khai các transaction serializable:

Thực thi các transaction theo đúng thứ tự tuần tự

Nếu bạn có thể làm cho mỗi transaction thực thi rất nhanh (thường là bằng cách sử dụng stored procedures), và throughput của transaction đủ thấp để xử lý trên một nhân CPU duy nhất hoặc có thể được sharding, thì đây là một lựa chọn đơn giản và hiệu quả.

Two-phase locking

Trong nhiều thập kỷ, 2PL đã là cách tiêu chuẩn để triển khai tính serializability, nhưng nhiều ứng dụng tránh sử dụng nó vì hiệu năng kém.

Serializable snapshot isolation

SSI là một thuật toán tương đối mới giúp tránh hầu hết các nhược điểm của các cách tiếp cận trước đó. Nó sử dụng cách tiếp cận lạc quan (optimistic), cho phép các transaction tiếp tục mà không bị chặn. Khi một transaction muốn commit, nó sẽ được kiểm tra, và sẽ bị abort nếu việc thực thi không đảm bảo tính serializable.

Cuối cùng, chúng ta đã xem xét cách đạt được tính atomicity khi một transaction được phân tán trên nhiều node, bằng cách sử dụng 2PC. Nếu các node đó đều chạy cùng một phần mềm cơ sở dữ liệu, các distributed transactions có thể hoạt động khá tốt. Tuy nhiên, khi chạy trên các công nghệ lưu trữ khác nhau (sử dụng XA transactions), 2PC gặp vấn đề; nó rất nhạy cảm với các lỗi trong coordinator và mã nguồn ứng dụng điều khiển transaction, đồng thời nó tương tác kém với các cơ chế concurrency control. May mắn thay, idempotence có thể đảm bảo semantics exactly-once mà không yêu cầu atomic commit giữa các công nghệ lưu trữ khác nhau; chúng ta sẽ tìm hiểu thêm về điều này trong các chương sau.

Các ví dụ trong chương này sử dụng mô hình dữ liệu relational. Tuy nhiên, như đã thảo luận trong mục “The need for multi-object transactions”, transaction là một tính năng giá trị của cơ sở dữ liệu, bất kể mô hình dữ liệu nào được sử dụng.

Footnotes

References

- [1] Steven J. Murdoch. “What Went Wrong with Horizon: Learning from the Post Office Trial.” *benthams gaze.org*, July 2021. Archived at perma.cc/CNMF-553F
- [2] Donald D. Chamberlin, Morton M. Astrahan, Michael W. Blasgen, James N. Gray, W. Frank King, Bruce G. Lindsay, Raymond Lorie, James W. Mehl, Thomas G. Price, Franco Putzolu, Patricia Griffiths Selinger, Mario Schkolnick, Donald R. Slutz, Irving L. Traiger, Bradford W. Wade, and Robert A. Yost. “A History and Evaluation of System R.” *Communications of the ACM*, volume 24, issue 10, pages 632–646, October 1981. [doi:10.1145/358769.358784](https://doi.org/10.1145/358769.358784)
- [3] Jim N. Gray, Raymond A. Lorie, Gianfranco R. Putzolu, and Irving L. Traiger. “Granularity of Locks and Degrees of Consistency in a Shared Data Base.” In *Modelling in Data Base Management Systems: Proceedings of the IFIP Working Conference on Modelling in Data Base Management Systems*, edited by G. M. Nijssen, pages 364–394, Elsevier/North Holland Publishing, 1976. Also in *Readings in Database Systems*, 4th edition, edited by Joseph M. Hellerstein and Michael Stonebraker, MIT Press, 2005. ISBN: 9780262693141
- [4] Kapali P. Eswaran, Jim N. Gray, Raymond A. Lorie, and Irving L. Traiger. “The Notions of Consistency and Predicate Locks in a Database System.” *Communications of the ACM*, volume 19, issue 11, pages 624–633, November 1976. [doi:10.1145/360363.360369](https://doi.org/10.1145/360363.360369)
- [5] Rebecca Taft, Irfan Sharif, Andrei Matei, Nathan VanBenschoten, Jordan Lewis, Tobias Grieger, Kai Niemi, Andy Woods, Anne Birzin, Raphael Poss, Paul Bardea, Amruta Ranade, Ben Darnell, Bram Gruneir, Justin Jaffray, Lucy Zhang, and Peter Mattis. “CockroachDB: The Resilient Geo-Distributed SQL Database.” At *ACM SIGMOD International Conference on Management of Data (SIGMOD)*, June 2020. [doi:10.1145/3318464.3386134](https://doi.org/10.1145/3318464.3386134)
- [6] Dongxu Huang, Qi Liu, Qiu Cui, Zhuhe Fang, Xiaoyu Ma, Fei Xu, Li Shen, Liu Tang, Yuxing Zhou, Menglong Huang, Wan Wei, Cong Liu, Jian Zhang, Jianjun Li, Xuelian Wu, Lingyu Song, Ruoxi Sun, Shuaipeng Yu, Lei Zhao, Nicholas Cameron, Liquan Pei, and Xin Tang. “TiDB: A Raft-Based HTAP Database.” *Proceedings of the VLDB Endowment*, volume 13, issue 12, pages 3072–3084, August 2020. [doi:10.14778/3415478.3415535](https://doi.org/10.14778/3415478.3415535)
- [7] James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, JJ Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, Wilson Hsieh, Sebastian Kanthak, Eugene Kogan, Hongyi Li, Alexander Lloyd, Sergey Melnik, David Mwaura, David Nagle, Sean Quinlan, Rajesh Rao, Lindsay Rolig, Dale Woodford, Yasushi Saito, Christopher Taylor, Michal Szymaniak, and Ruth Wang. “Spanner: Google’s Globally-Distributed Database.” At *10th USENIX Symposium on Operating System Design and Implementation (OSDI)*, October 2012.
- [8] Jingyu Zhou, Meng Xu, Alexander Shraer, Bala Namasivayam, Alex Miller, Evan Tschannen, Steve Atherton, Andrew J. Beamon, Rusty Sears, John Leach, Dave Rosenthal, Xin Dong, Will Wilson, Ben Collins, David Scherer, Alec Grieser, Young Liu, Alvin Moore, Bhaskar Muppana, Xiaoge Su, and Vishesh Yadav. “FoundationDB: A Distributed Unbundled Transactional Key Value Store.” At *ACM International Conference on Management of Data (SIGMOD)*, June 2021. [doi:10.1145/3448016.3457559](https://doi.org/10.1145/3448016.3457559)
- [9] Theo Härder and Andreas Reuter. “Principles of Transaction-Oriented Database Recovery.” *ACM Computing Surveys*, volume 15, issue 4, pages 287–317, December 1983. [doi:10.1145/289.291](https://doi.org/10.1145/289.291)
- [10] Peter Bailis, Alan Fekete, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. “HAT, not CAP: Towards Highly Available Transactions.” At *14th USENIX Workshop on Hot Topics in Operating Systems (HotOS)*, May 2013.

- [11] Armando Fox, Steven D. Gribble, Yatin Chawathe, Eric A. Brewer, and Paul Gauthier. “Cluster-Based Scalable Network Services.” At *16th ACM Symposium on Operating Systems Principles (SOSP)*, October 1997. doi:10.1145/268998.266662
- [12] Tony Andrews. “Enforcing Complex Constraints in Oracle.” *tonyandrews.blogspot.co.uk*, October 2004. Archived at *archive.org*
- [13] Philip A. Bernstein, Vassos Hadzilacos, and Nathan Goodman. *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987. ISBN: 9780201107159. Available online at *microsoft.com*.
- [14] Alan Fekete, Dimitrios Liarokapis, Elizabeth O’Neil, Patrick O’Neil, and Dennis Shasha. “Making Snapshot Isolation Serializable.” *ACM Transactions on Database Systems*, volume 30, issue 2, pages 492–528, June 2005. doi:10.1145/1071610.1071615
- [15] Mai Zheng, Joseph Tucek, Feng Qin, and Mark Lillibridge. “Understanding the Robustness of SSDs Under Power Fault.” At *11th USENIX Conference on File and Storage Technologies (FAST)*, February 2013.
- [16] Laurie Denness. “SSDs: A Gift and a Curse.” *laur.ie*, June 2015. Archived at *perma.cc/6GLP-BX3T*
- [17] Adam Surak. “When Solid State Drives Are Not That Solid.” *blog.algolia.com*, June 2015. Archived at *perma.cc/CBR9-QZEE*
- [18] Hewlett Packard Enterprise. “Bulletin: (Revision) HPE SAS Solid State Drives—Critical Firmware Upgrade Required for Certain HPE SAS Solid State Drive Models to Prevent Drive Failure at 32,768 Hours of Operation.” *support.hpe.com*, November 2019. Archived at *perma.cc/CZR4-AQBS*
- [19] Craig Ringer et al. “PostgreSQL’s Handling of fsync() Errors Is Unsafe and Risks Data Loss at Least on XFS.” Email thread on *pgsql-hackers* mailing list, *postgresql.org*, March 2018. Archived at *perma.cc/SRKU-57FL*
- [20] Anthony Rebello, Yuvraj Patel, Ramnatthan Alagappan, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. “Can Applications Recover from fsync Failures?” At *USENIX Annual Technical Conference (ATC)*, July 2020.
- [21] Thanumalayan Sankaranarayana Pillai, Vijay Chidambaram, Ramnatthan Alagappan, Samer Al-Kiswani, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. “Crash Consistency: Rethinking the Fundamental Abstractions of the File System.” *ACM Queue*, volume 13, issue 7, pages 20–28, July 2015. doi:10.1145/2800695.2801719
- [22] Thanumalayan Sankaranarayana Pillai, Vijay Chidambaram, Ramnatthan Alagappan, Samer Al-Kiswani, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. “All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications.” At *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, October 2014.
- [23] Chris Siebenmann. “Unix’s File Durability Problem.” *utcc.utoronto.ca*, April 2016. Archived at *perma.cc/VSS8-SMC4*
- [24] Aishwarya Ganesan, Ramnatthan Alagappan, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. “Redundancy Does Not Imply Fault Tolerance: Analysis of Distributed Storage Reactions to Single Errors and Corruptions.” At *15th USENIX Conference on File and Storage Technologies (FAST)*, February 2017.
- [25] Lakshmi N. Bairavasundaram, Garth R. Goodson, Bianca Schroeder, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. “An Analysis of Data Corruption in the Storage Stack.” At *6th USENIX Conference on File and Storage Technologies (FAST)*, February 2008.
- [26] Richard van der Hoff. “How We Discovered, and Recovered from, Postgres Corruption on the matrix.org Homeserver.” *matrix.org*, July 2025. Archived at *perma.cc/CDF5-NRBK*

- [27] Bianca Schroeder, Raghav Lagisetty, and Arif Merchant. “Flash Reliability in Production: The Expected and the Unexpected.” At *14th USENIX Conference on File and Storage Technologies (FAST)*, February 2016.
- [28] Don Allison. “SSD Storage—Ignorance of Technology Is No Excuse.” *blog.korelogic.com*, March 2015. Archived at perma.cc/9QN4-9SNJ
- [29] Gordon Mah Ung. “Debunked: Your SSD Won’T Lose Data If Left Unplugged After All.” *pcworld.com*, May 2015. Archived at perma.cc/S46H-JUDU
- [30] Martin Kleppmann. “Hermitage: Testing the ‘I’ in ACID.” *martin.kleppmann.com*, November 2014. Archived at perma.cc/KP2Y-AQ GK
- [31] Vlad Mihalcea. “The Race Condition That Led to Flexcoin Bankruptcy.” *vladmihalcea.com*, February 2025. Archived at perma.cc/RRK5-TFAU
- [32] Todd Warszawski and Peter Bailis. “ACIDRain: Concurrency-Related Attacks on Database-Backed Web Applications.” At *ACM International Conference on Management of Data (SIGMOD)*, May 2017. doi:10.1145/3035918.3064037
- [33] Tristan D’Agosta. “BTC Stolen from Poloniex.” *bitcointalk.org*, March 2014. Archived at perma.cc/YHA6-4C5D
- [34] bitcointhief2. “How I Stole Roughly 100 BTC from an Exchange and How I Could Have Stolen More!” *reddit.com*, February 2014. Archived at archive.org
- [35] Sudhir Jorwekar, Alan Fekete, Krithi Ramamritham, and S. Sudarshan. “Automating the Detection of Snapshot Isolation Anomalies.” At *33rd International Conference on Very Large Data Bases (VLDB)*, September 2007.
- [36] Michael Melanson. “Transactions: The Limits of Isolation.” *michaelmelanson.net*, November 2014. Archived at perma.cc/RG5R-KMYZ
- [37] Edward Kim. “How ACH Works: A Developer Perspective—Part 1.” *engineering.gusto.com*, April 2014. Archived at perma.cc/7B2H-PU94
- [38] Hal Berenson, Philip A. Bernstein, Jim N. Gray, Jim Melton, Elizabeth O’Neil, and Patrick O’Neil. “A Critique of ANSI SQL Isolation Levels.” At *ACM International Conference on Management of Data (SIGMOD)*, May 1995. doi:10.1145/568271.223785
- [39] Atul Adya. “Weak Consistency: A Generalized Theory and Optimistic Implementations for Distributed Transactions.” PhD thesis, Massachusetts Institute of Technology, March 1999. Archived at perma.cc/E97M-HW5Q
- [40] Peter Bailis, Aaron Davidson, Alan Fekete, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. “Highly Available Transactions: Virtues and Limitations.” *Proceedings of the VLDB Endowment*, volume 7, issue 3, pages 181–192, November 2013. doi:10.14778/2732232.2732237.
- [41] Natacha Crooks, Youer Pu, Lorenzo Alvisi, and Allen Clement. “Seeing Is Believing: A Client-Centric Specification of Database Isolation.” At *ACM Symposium on Principles of Distributed Computing (PODC)*, July 2017. doi:10.1145/3087801.3087802
- [42] Bruce Momjian. “MVCC Unmasked.” *momjian.us*, July 2014. Archived at perma.cc/KQ47-9GYB
- [43] Peter Alvaro and Kyle Kingsbury. “MySQL 8.0.34.” *jeepsen.io*, December 2023. Archived at perma.cc/HGE2-Z878
- [44] Egor Rogov. *PostgreSQL 14 Internals*. Postgres Professional, April 2023. Archived at perma.cc/FRK2-D7WB
- [45] Hironobu Suzuki. “The Internals of PostgreSQL.” *interdb.jp*, 2017.

- [46] Rohan Reddy Alleti. “Internals of MVCC in Postgres: Hidden Costs of Updates vs Inserts.” *medium.com*, March 2025. Archived at perma.cc/3ACX-DFXT
- [47] Andy Pavlo and Bohan Zhang. “The Part of PostgreSQL We Hate the Most.” *cs.cmu.edu*, April 2023. Archived at perma.cc/XSP6-3JBN
- [48] Yingjun Wu, Joy Arulraj, Jiexi Lin, Ran Xian, and Andrew Pavlo. “An Empirical Evaluation of In-Memory Multi-Version Concurrency Control.” *Proceedings of the VLDB Endowment*, volume 10, issue 7, pages 781–792, March 2017. doi:10.14778/3067421.3067427
- [49] Nikita Prokopov. “Unofficial Guide to Datomic Internals.” *tonsky.me*, May 2014. Archived at perma.cc/ULM2-T2FW
- [50] Daniil Svetlov. “A Practical Guide to Taming Postgres Isolation Anomalies.” *dansvetlov.me*, March 2025. Archived at perma.cc/L7LE-TDLS
- [51] Nate Wiger. “An Atomic Rant.” *nateware.com*, February 2010. Archived at perma.cc/SZYB-PE44
- [52] James Coglan. “Reading and Writing, Part 3: Web Applications.” *blog.jcoglan.com*, October 2020. Archived at perma.cc/A7EK-PJVS
- [53] Peter Bailis, Alan Fekete, Michael J. Franklin, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. “Feral Concurrency Control: An Empirical Investigation of Modern Application Integrity.” At *ACM International Conference on Management of Data (SIGMOD)*, June 2015. doi:10.1145/2723372.2737784
- [54] Jaana Dogan. “Things I Wished More Developers Knew About Databases.” *rakyll.medium.com*, April 2020. Archived at perma.cc/6EFK-P2TD
- [55] Michael J. Cahill, Uwe Röhm, and Alan Fekete. “Serializable Isolation for Snapshot Databases.” At *ACM International Conference on Management of Data (SIGMOD)*, June 2008. doi:10.1145/1376616.1376690
- [56] Dan R. K. Ports and Kevin Grittnr. “Serializable Snapshot Isolation in PostgreSQL.” *Proceedings of the VLDB Endowment*, volume 5, issue 12, pages 1850–1861, August 2012. doi:10.14778/2367502.2367523
- [57] Douglas B. Terry, Marvin M. Theimer, Karin Petersen, Alan J. Demers, Mike J. Spreitzer and Carl H. Hauser. “Managing Update Conflicts in Bayou, a Weakly Connected Replicated Storage System.” At *15th ACM Symposium on Operating Systems Principles (SOSP)*, December 1995. doi:10.1145/224056.224070
- [58] Hans-Jürgen Schöning. “Constraints over Multiple Rows in PostgreSQL.” *cybertec-postgresql.com*, June 2021. Archived at perma.cc/2TGH-XUPZ
- [59] Michael Stonebraker, Samuel Madden, Daniel J. Abadi, Stavros Harizopoulos, Nabil Hachem, and Pat Helland. “The End of an Architectural Era (It’s Time for a Complete Rewrite).” At *33rd International Conference on Very Large Data Bases (VLDB)*, September 2007.
- [60] John Hugg. “H-Store/VoltDB Architecture vs. CEP Systems and Newer Streaming Architectures.” At *Data @Scale Boston*, November 2014.
- [61] Robert Kallman, Hideaki Kimura, Jonathan Natkins, Andrew Pavlo, Alexander Rasin, Stanley Zdonik, Evan P. C. Jones, Samuel Madden, Michael Stonebraker, Yang Zhang, John Hugg, and Daniel J. Abadi. “H-Store: A High-Performance, Distributed Main Memory Transaction Processing System.” *Proceedings of the VLDB Endowment*, volume 1, issue 2, pages 1496–1499, August 2008. doi:10.14778/1454159.1454211
- [62] Rich Hickey. “The Architecture of Datomic.” *infoq.com*, November 2012. Archived at perma.cc/5YWU-8XJK

- [63] John Hugg. “Debunking Myths About the VoltDB In-Memory Database.” *dzone.com*, May 2014. Archived at perma.cc/2Z9N-HPKF
- [64] Xinjing Zhou, Viktor Leis, Xiangyao Yu, and Michael Stonebraker. “OLTP Through the Looking Glass 16 Years Later: Communication Is the New Bottleneck.” At *15th Annual Conference on Innovative Data Systems Research* (CIDR), January 2025. Archived at perma.cc/Q33D-K9YE
- [65] Xinjing Zhou, Xiangyao Yu, Goetz Graefe, and Michael Stonebraker. “Lotus: Scalable Multi-Partition Transactions On Single-Threaded Partitioned Databases.” *Proceedings of the VLDB Endowment* (PVLDB), volume 15, issue 11, pages 2939–2952, July 2022. doi: [10.14778/3551793.3551843](https://doi.org/10.14778/3551793.3551843)
- [66] Joseph M. Hellerstein, Michael Stonebraker, and James Hamilton. “Architecture of a Database System.” *Foundations and Trends in Databases*, volume 1, issue 2, pages 141–259, November 2007. doi: [10.1561/1900000002](https://doi.org/10.1561/1900000002)
- [67] Michael J. Cahill. “Serializable Isolation for Snapshot Databases.” PhD thesis, University of Sydney, July 2009. Archived at perma.cc/727J-NTMP
- [68] Cristian Diaconu, Craig Freedman, Erik Ismert, Per-Åke Larson, Pravin Mittal, Ryan Stonecipher, Nitin Verma, and Mike Zwilling. “Hekaton: SQL Server’s Memory-Optimized OLTP Engine.” At *ACM SIGMOD International Conference on Management of Data* (SIGMOD), June 2013. doi: [10.1145/2463676.2463710](https://doi.org/10.1145/2463676.2463710)
- [69] Thomas Neumann, Tobias Mühlbauer, and Alfons Kemper. “Fast Serializable Multi-Version Concurrency Control for Main-Memory Database Systems.” At *ACM SIGMOD International Conference on Management of Data* (SIGMOD), May 2015. doi: [10.1145/2723372.2749436](https://doi.org/10.1145/2723372.2749436)
- [70] D. Z. Badal. “Correctness of Concurrency Control and Implications in Distributed Databases.” At *3rd International IEEE Computer Software and Applications Conference* (COMPSAC), November 1979. doi: [10.1109/CMPSAC.1979.762563](https://doi.org/10.1109/CMPSAC.1979.762563)
- [71] Rakesh Agrawal, Michael J. Carey, and Miron Livny. “Concurrency Control Performance Modeling: Alternatives and Implications.” *ACM Transactions on Database Systems* (TODS), volume 12, issue 4, pages 609–654, December 1987. doi: [10.1145/32204.32220](https://doi.org/10.1145/32204.32220)
- [72] Marc Brooker. “Snapshot Isolation vs. Serializability.” *brooker.co.za*, December 2024. Archived at perma.cc/5TRC-CR5G
- [73] B. G. Lindsay, P. G. Selinger, C. Galtieri, J. N. Gray, R. A. Lorie, T. G. Price, F. Putzolu, I. L. Traiger, and B. W. Wade. “Notes on Distributed Databases.” IBM Research, Research Report RJ2571(33471), July 1979. Archived at perma.cc/EPZ3-MHDD
- [74] C. Mohan, Bruce G. Lindsay, and Ron Obermarck. “Transaction Management in the R* Distributed Database Management System.” *ACM Transactions on Database Systems*, volume 11, issue 4, pages 378–396, December 1986. doi: [10.1145/7239.7266](https://doi.org/10.1145/7239.7266)
- [75] X/Open Company Ltd. “Distributed Transaction Processing: The XA Specification.” Technical Standard XO/CAE/91/300, December 1991. ISBN: 9781872630243, archived at perma.cc/Z96H-29JB
- [76] Ivan Silva Neto and Francisco Reverbel. “Lessons Learned from Implementing WS-Coordination and WS-AtomicTransaction.” At *7th IEEE/ACIS International Conference on Computer and Information Science* (ICIS), May 2008. doi: [10.1109/ICIS.2008.75](https://doi.org/10.1109/ICIS.2008.75)
- [77] James E. Johnson, David E. Langworthy, Leslie Lamport, and Friedrich H. Vogt. “Formal Specification of a Web Services Protocol.” At *1st International Workshop on Web Services and Formal Methods* (WS-FM), February 2004. doi: [10.1016/j.entcs.2004.02.022](https://doi.org/10.1016/j.entcs.2004.02.022)

- [78] Jim Gray. “The Transaction Concept: Virtues and Limitations.” At *7th International Conference on Very Large Data Bases* (VLDB), September 1981.
- [79] Dale Skeen. “Nonblocking Commit Protocols.” At *ACM International Conference on Management of Data* (SIGMOD), April 1981. doi:10.1145/582318.582339
- [80] Gregor Hohpe. “Your Coffee Shop Doesn’t Use Two-Phase Commit.” *IEEE Software*, volume 22, issue 2, pages 64–66, March 2005. doi:10.1109/MS.2005.52
- [81] Pat Helland. “Life Beyond Distributed Transactions: An Apostate’s Opinion.” At *3rd Biennial Conference on Innovative Data Systems Research* (CIDR), January 2007. Archived at perma.cc/FC4F-AHGH
- [82] Jonathan Oliver. “My Beef with MSDTC and Two-Phase Commits.” *blog.jonathanoliver.com*, April 2011. Archived at perma.cc/K8HF-Z4EN
- [83] Oren Eini (Ahende Rahien). “The Fallacy of Distributed Transactions.” *ayende.com*, July 2014. Archived at perma.cc/VB87-2JEF
- [84] Clemens Vasters. “Transactions in Windows Azure (with Service Bus)—An Email Discussion.” *learn.microsoft.com*, July 2012. Archived at perma.cc/4EZ9-5SKW
- [85] Ajmer Dhariwal. “Orphaned MSDTC Transactions (-2 spids).” *eraofdata.com*, December 2008. Archived at perma.cc/YG6F-U34C
- [86] Paul Randal. “Real World Story of DBCC PAGE Saving the Day.” *sqlskills.com*, June 2013. Archived at perma.cc/2MJN-A5QH
- [87] Guozhang Wang, Lei Chen, Ayusman Dikshit, Jason Gustafson, Boyang Chen, Matthias J. Sax, John Roesler, Sophie Blee-Goldman, Bruno Cadonna, Apurva Mehta, Varun Madan, and Jun Rao. “Consistency and Completeness: Rethinking Distributed Stream Processing in Apache Kafka.” At *ACM International Conference on Management of Data* (SIGMOD), June 2021. doi:10.1145/3448016.3457556

Những rắc rối với distributed systems

Tại nạn là những thứ thật kỳ lạ. Bạn chẳng bao giờ gặp chúng cho đến khi chúng bắt đầu xảy ra.

A.A. Milne, *The House at Pooh Corner* (1928)

Như đã thảo luận trong mục “Reliability và Fault Tolerance”, việc làm cho một hệ thống trở nên tin cậy có nghĩa là đảm bảo rằng toàn bộ hệ thống tiếp tục hoạt động, ngay cả khi có sự cố xảy ra (tức là khi có một fault). Tuy nhiên, việc dự đoán tất cả các fault có thể xảy ra và xử lý chúng không hề dễ dàng. Với tư cách là một lập trình viên, bạn sẽ rất dễ bị cám dỗ khi chỉ tập trung chủ yếu vào happy path (sau tất cả, hầu hết mọi thứ đều hoạt động tốt!) mà bỏ qua các fault, vì chúng tạo ra rất nhiều edge case.

Nếu bạn muốn hệ thống của mình có độ tin cậy khi có sự hiện diện của các fault, bạn phải thay đổi hoàn toàn tư duy và tập trung vào những gì có thể đi chệch hướng, ngay cả khi điều đó có vẻ khó xảy ra. Việc khả năng xảy ra chỉ là một phần triệu cũng không quan trọng; trong một hệ thống đủ lớn, các sự kiện một phần triệu xảy ra hàng ngày. Những người vận hành hệ thống dày dạn kinh nghiệm sẽ nói với bạn rằng bất cứ thứ gì *có thể* đi chệch hướng *sẽ* đi chệch hướng.

Làm việc với distributed systems cũng khác biệt về mặt cơ bản so với việc viết phần mềm trên một máy tính duy nhất—sự khác biệt chính là mọi thứ có thể đi chệch hướng theo rất nhiều cách mới mẻ và thú vị [1, 2]. Trong chương này, bạn sẽ được trải nghiệm những vấn đề nảy sinh trong thực tế và hiểu được những gì bạn có thể và không thể tin cậy.

Để hiểu những thách thức mà chúng ta đang phải đối mặt, chúng ta sẽ đẩy sự bi quan của mình lên mức tối đa và khám phá nhiều loại vấn đề có thể xảy ra trong một distributed system, bao gồm các vấn đề với network cũng như các vấn đề về clock và timing. Hệ quả của tất cả những vấn đề này rất gây mất phương hướng, vì vậy chúng ta cũng sẽ khám phá cách tư duy về state của một distributed system và cách suy luận về những điều đã xảy ra. Trong Chương 10, chúng ta sẽ xem xét một số ví dụ về cách chúng ta có thể đạt được fault tolerance trước những sự kiện này.

Faults và Partial Failures

Khi bạn viết một chương trình trên một máy tính duy nhất, nó thường hoạt động theo một cách khá dễ đoán: hoặc là nó hoạt động hoặc là không. Phần mềm lỗi có thể tạo cảm giác rằng máy tính đôi khi đang “có một ngày tồi tệ” (một vấn đề thường được khắc phục bằng cách khởi động lại), nhưng đó chủ yếu chỉ là hệ quả của việc viết phần mềm kém.

Không có lý do cơ bản nào khiến phần mềm trên một máy tính duy nhất trở nên flaky. Khi phần cứng hoạt động chính xác, cùng một thao tác luôn tạo ra cùng một kết quả (nó có tính *deterministic*). Nếu có vấn đề về phần cứng (ví dụ: lỗi bộ nhớ hoặc đầu nối bị lỏng), hệ quả thường là sự thất bại hoàn toàn của hệ thống (ví dụ: kernel panic, “màn hình xanh chết chóc”, không thể khởi động). Một máy tính riêng lẻ với phần mềm tốt thường sẽ hoạt động đầy đủ chức năng hoặc bị hỏng hoàn toàn, chứ không nằm ở khoảng giữa.

Đây là một lựa chọn có chủ đích trong thiết kế máy tính. Nếu một fault nội bộ xảy ra, chúng ta thà để máy tính bị crash hoàn toàn còn hơn là trả về một kết quả sai, bởi vì các kết quả sai rất khó và gây bối rối khi xử lý. Do đó, máy tính che giấu thực tế vật lý mờ nhạt mà chúng được triển khai dựa trên đó, và trình diễn một mô hình hệ thống lý tưởng hóa hoạt động với sự hoàn hảo về mặt toán học. Một lệnh CPU luôn thực hiện cùng một việc; nếu bạn ghi một số dữ liệu vào bộ nhớ hoặc đĩa, dữ liệu đó vẫn nguyên vẹn và không bị lỗi ngẫu nhiên. Như đã thảo luận trong mục “Hardware và Software Faults”, điều này thực tế không đúng—trong thực tế, dữ liệu thực sự có thể bị lỗi một cách âm thầm và CPU đôi khi cũng âm thầm trả về kết quả sai—nhưng nó xảy ra đủ hiếm để chúng ta có thể bỏ qua.

Khi bạn viết phần mềm chạy trên nhiều máy tính được kết nối bởi một network, tình huống này khác biệt về mặt cơ bản. Trong distributed systems, các fault xảy ra thường xuyên hơn nhiều, vì vậy chúng ta không còn có thể bỏ qua chúng—chúng ta không còn lựa chọn nào khác ngoài việc đối mặt với thực tế hỗn loạn của thế giới vật lý. Và trong thế giới vật lý, một phạm vi cực kỳ rộng lớn các thứ có thể đi chệch hướng, như được minh họa bởi giai thoại này [3]:

Trong kinh nghiệm hạn hẹp của mình, ta đã từng đối mặt với các đợt network partition kéo dài trong một data center (DC) duy nhất, các lỗi PDU [power distribution unit], lỗi switch, các đợt accidental power cycles của cả một rack, lỗi backbone của toàn bộ DC, lỗi nguồn của toàn bộ DC, và cả một tài xế bị hạ đường huyết đâm chiếc xe bán tải Ford của mình vào hệ thống HVAC [heating, ventilation, and air conditioning] của một DC. Và ta thậm chí còn không phải là một chuyên gia ops.

Coda Hale

Trong một distributed system, có thể có một số phần của hệ thống bị hỏng theo cách không thể dự đoán được, ngay cả khi các phần khác của hệ thống vẫn đang hoạt động bình thường. Điều này được gọi là *partial failure* (lỗi một phần). Khó khăn ở chỗ các partial failure có tính *nondeterministic* (không xác định): nếu bạn cố gắng thực hiện bất kỳ thao tác nào liên quan đến nhiều node và network, đôi khi nó có thể hoạt động và đôi khi lại thất bại một cách khó đoán. Như chúng ta sẽ thấy, thậm chí bạn còn có thể không *biết* được liệu một thứ gì đó đã thành công hay chưa!

Tính chất nondeterminism này và khả năng xảy ra partial failures chính là điều khiến các distributed system trở nên khó làm việc [4]. Mặt khác, nếu một distributed system có thể chịu đựng được các partial failures, điều đó sẽ mở ra những khả năng mạnh mẽ—ví dụ, nó có nghĩa là chúng ta có thể thực hiện một rolling upgrade, khởi động lại từng node một để cài đặt các bản cập nhật phần mềm trong khi toàn bộ hệ thống vẫn tiếp tục hoạt động mà không bị gián đoạn. Do đó, fault tolerance cho phép chúng ta làm cho các distributed system trở nên tin cậy hơn so với các hệ thống đơn node; chúng ta có thể xây dựng một hệ thống tin cậy từ các thành phần không tin cậy.

Nhưng trước khi chúng ta có thể triển khai fault tolerance, chúng ta cần biết nhiều hơn về các lỗi mà chúng ta dự định sẽ chịu đựng. Việc xem xét một phạm vi rộng các lỗi có thể xảy ra—ngay cả những lỗi khá khó xảy ra—và tạo ra các tình huống như vậy một cách nhân tạo trong môi trường testing của bạn để xem điều gì sẽ xảy ra là rất quan trọng. Trong các distributed system, sự nghi ngờ, sự bí quan và sự đa nghi sẽ mang lại kết quả.

Network không tin cậy

Trong quá khứ, các máy tính cũ hơn như mainframe được làm cho trở nên tin cậy bằng cách đảm bảo rằng các thành phần riêng lẻ có tính dư thừa—ví dụ, bằng cách sử dụng RAID để duy trì khi các ổ đĩa riêng lẻ gặp lỗi. Như đã thảo luận trong mục “Shared-Memory, Shared-Disk, and Shared-Nothing Architectures”, các distributed system mà chúng ta tập trung vào trong cuốn sách này chủ yếu là các *shared-nothing systems*: một nhóm các máy tính được kết nối với nhau bằng một network. Thay vì có sự dư thừa của các thành phần bên trong một máy duy nhất, các shared-nothing systems sử dụng replication giữa các máy riêng biệt để tạo sự dư thừa. Network là cách duy nhất để các máy này có thể giao tiếp. Chúng ta giả định rằng mỗi máy có bộ nhớ và đĩa riêng, và một máy không thể truy cập bộ nhớ hoặc đĩa của máy khác (ngoại trừ việc gửi các request tới một service qua network). Ngay cả khi storage được chia sẻ, chẳng hạn như với object storage, các máy vẫn giao tiếp với các shared storage services qua network.

Internet và hầu hết các network nội bộ trong các datacenter (thường là Ethernet) là các *asynchronous packet networks* (mạng gói không đồng bộ). Trong loại network này,

một node có thể gửi một message (một gói tin) tới một node khác, nhưng network không đưa ra bất kỳ đảm bảo nào về việc khi nào nó sẽ đến hoặc liệu nó có đến hay không. Nếu bạn gửi một request và mong đợi một response, nhiều thứ có thể xảy ra sai sót (một số trong đó được minh họa trong Hình 9-1):

- Request của bạn có thể đã bị mất (có lẽ ai đó đã rút cáp network).
- Request của bạn có thể đang chờ trong một queue và sẽ được chuyển tới sau (có lẽ network hoặc bên nhận đang bị quá tải).
- Node từ xa có thể đã bị lỗi (có lẽ nó đã bị crash hoặc đã bị ngắt nguồn).
- Node từ xa có thể đã tạm thời ngừng phản hồi (có lẽ nó đang gặp một đợt GC pause kéo dài; xem mục “Process Pauses”), nhưng nó sẽ bắt đầu phản hồi lại sau đó.
- Node từ xa có thể đã xử lý request của bạn, nhưng response đã bị mất trên network (có lẽ một network switch đã bị cấu hình sai).
- Node từ xa có thể đã xử lý request của bạn, nhưng response đã bị trì hoãn và sẽ được chuyển tới sau (có lẽ network hoặc chính máy của bạn đang bị quá tải).

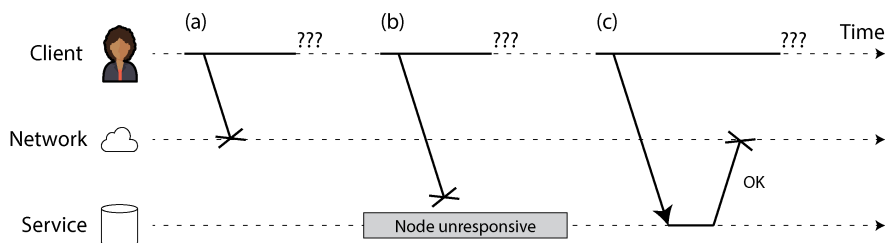


Figure 9-1. Nếu bạn gửi một request mà không nhận được response, bạn không thể phân biệt được liệu (a) request đã bị thất lạc, (b) remote node đã bị sập, hay (c) response đã bị thất lạc.

Người gửi thậm chí không thể biết liệu gói tin đã được chuyển đến hay chưa. Lựa chọn duy nhất là người nhận gửi một thông điệp phản hồi, nhưng thông điệp này cũng có thể bị thất lạc hoặc bị trễ. Những vấn đề này không thể phân biệt được trong một mạng không đồng bộ (asynchronous network); thông tin duy nhất mà bạn có là bạn vẫn chưa nhận được phản hồi. Nếu bạn gửi một yêu cầu đến một node khác mà không nhận được phản hồi, việc xác định lý do tại sao là *không thể*.

Cách thông thường để xử lý vấn đề này là sử dụng một *timeout* (thời gian chờ): sau một khoảng thời gian nhất định, bạn ngừng chờ và giả định rằng phản hồi sẽ không đến. Tuy nhiên, khi một *timeout* xảy ra, bạn vẫn không biết liệu node từ xa có nhận được yêu cầu của bạn hay không (và nếu yêu cầu vẫn đang nằm trong hàng đợi ở đâu đó, nó vẫn có thể được chuyển đến người nhận, ngay cả khi bạn đã từ bỏ khả năng đó).

Những hạn chế của TCP

Các gói tin mạng có kích thước tối đa (thường là vài kilobyte), nhưng nhiều ứng dụng cần gửi các thông điệp (yêu cầu, phản hồi) quá lớn để có thể nằm gọn trong một gói tin. Các ứng dụng này thường sử dụng TCP, tức Transmission Control Protocol, để thiết lập một *connection* (kết nối) giúp chia nhỏ các luồng dữ liệu lớn thành các gói tin riêng lẻ và lắp ghép chúng lại với nhau ở phía bên nhận.

LƯU Ý

Hầu hết những gì chúng ta nói về TCP cũng áp dụng cho giải pháp thay thế gần đây hơn là QUIC, cũng như Stream Control Transmission Protocol (SCTP) được sử dụng trong WebRTC, giao thức BitTorrent uTP và các giao thức truyền tải khác. Để so sánh với UDP, hãy xem mục “TCP Versus UDP”.

TCP thường được mô tả là cung cấp khả năng chuyển phát “tin cậy”, theo nghĩa là nó phát hiện và truyền lại các gói tin bị mất, phát hiện các gói tin bị sai thứ tự và sắp xếp lại chúng đúng trình tự, đồng thời phát hiện gói tin bị lỗi bằng cách sử dụng một checksum đơn giản. Nó cũng tính toán xem có thể gửi dữ liệu nhanh đến mức nào để dữ liệu được chuyển đi nhanh nhất có thể, nhưng không làm quá tải mạng hoặc node nhận; điều này được gọi là *congestion control* (kiểm soát tắc nghẽn), *flow control* (kiểm soát luồng), hoặc *backpressure* (áp lực ngược) [5].

Khi bạn “gửi” dữ liệu bằng cách ghi nó vào một socket, nó không được gửi đi ngay lập tức; nó được đặt vào một *buffer* (bộ đệm) do hệ điều hành của bạn quản lý. Khi thuật toán kiểm soát tắc nghẽn quyết định rằng nó có đủ khả năng để gửi một gói tin, nó sẽ lấy lượng dữ liệu tương ứng với gói tin tiếp theo từ *buffer* đó và chuyển nó đến giao diện mạng. Gói tin đi qua nhiều switch và router, và cuối cùng hệ điều hành của node nhận sẽ đặt dữ liệu của gói tin vào một *receive buffer* (bộ đệm nhận) và gửi một gói tin *acknowledgment* (xác nhận) trở lại cho người gửi. Chỉ sau đó, hệ điều hành bên nhận mới thông báo cho ứng dụng rằng đã có thêm dữ liệu đến [6].

Vì vậy, nếu TCP cung cấp độ tin cậy, liệu điều đó có nghĩa là chúng ta không còn phải lo lắng về việc mạng không đáng tin cậy nữa? Thật không may là không phải vậy. TCP quyết định rằng một gói tin chắc hẳn đã bị mất nếu không có *acknowledgment* nào đến trong một khoảng *timeout* nhất định, nhưng nó không thể biết được chính xác là gói tin gửi đi hay gói tin *acknowledgment* đã bị mất. Mặc dù nó có thể gửi lại gói tin, nhưng nó không thể đảm bảo rằng gói tin mới sẽ đi qua được (ví dụ, nếu cáp mạng bị rút ra, TCP không thể cấm nó lại giúp bạn). Cuối cùng, sau một khoảng *timeout* có thể cấu hình được, nó sẽ từ bỏ và báo lỗi cho ứng dụng. Khả năng loại bỏ trùng lặp và truyền lại của TCP chỉ áp dụng cho một *connection* duy nhất, vì vậy nếu ứng dụng kết nối lại và truyền lại, dữ liệu có thể bị trùng lặp.

Nếu một kết nối TCP bị đóng do lỗi—có lẽ vì node từ xa bị crash hoặc mạng bị gián đoạn—thật không may là bạn không có cách nào để biết được bao nhiêu dữ liệu thực sự đã được xử lý bởi node từ xa [6]. Ngay cả khi bạn nhận được một `acknowledgment` rằng một gói tin đã được chuyển đến, điều này chỉ có nghĩa là `kernel` của hệ điều hành trên node từ xa đã nhận được nó; ứng dụng có thể đã bị crash trước khi nó kịp xử lý dữ liệu đó. Nếu bạn muốn chắc chắn rằng một yêu cầu đã thành công, bạn cần một phản hồi xác nhận từ chính ứng dụng đó [7].

Tuy nhiên, TCP rất hữu ích vì nó cung cấp một cách thuận tiện để gửi và nhận các thông điệp quá lớn so với một `packet`. Một khi kết nối TCP đã được thiết lập, bạn cũng có thể sử dụng nó để gửi nhiều `request` và `response`. Điều này thường được thực hiện bằng cách đầu tiên gửi một `header` cho biết độ dài của thông điệp tiếp theo tính bằng byte, sau đó là thông điệp thực tế. HTTP và nhiều giao thức RPC (xem “Dataflow Through Services: REST and RPC”) hoạt động theo cách này.

Lỗi mạng trong thực tế

Chúng ta đã xây dựng các mạng máy tính trong nhiều thập kỷ—người ta có thể hy vọng rằng đến thời điểm này chúng ta đã tìm ra cách để làm cho chúng trở nên tin cậy. Thật không may, chúng ta vẫn chưa thành công. Một số nghiên cứu hệ thống và nhiều bằng chứng thực tế cho thấy các vấn đề về mạng phổ biến một cách đáng ngạc nhiên, ngay cả trong các môi trường được kiểm soát như một `datacenter` do một công ty vận hành [8]:

- Một nghiên cứu trong một `datacenter` quy mô trung bình đã tìm thấy khoảng 12 lỗi mạng mỗi tháng, trong đó một nửa làm ngắt kết nối một máy đơn lẻ và một nửa làm ngắt kết nối toàn bộ một `rack` [9].
- Một nghiên cứu khác đã đo lường tỷ lệ lỗi của các thành phần như `top-of-rack switches`, `aggregation switches`, và `load balancers` [10]. Nghiên cứu cho thấy việc thêm các thiết bị mạng dự phòng không làm giảm lỗi nhiều như bạn mong đợi, vì nó không ngăn chặn được lỗi do con người (ví dụ: cấu hình sai các `switch`), vốn là nguyên nhân chính gây ra các đợt gián đoạn (outages).
- Sự gián đoạn của các liên kết sợi quang diện rộng từng bị đổ lỗi cho bò [11], hải ly [12], và cá mập [13] (mặc dù các vụ cá mập cắn đã trở nên hiếm hơn nhờ việc bọc bảo vệ tốt hơn cho các cáp dưới biển [14]). Con người cũng thường là nguyên nhân gây lỗi, thông qua việc cấu hình sai vô ý [15], thu gom linh kiện [16], hoặc phá hoại [17].
- Trên khắp các `cloud regions`, thời gian `round-trip` lên tới vài *phút* đã được quan sát thấy ở các `percentile` cao [18, Bảng 3]. Ngay cả trong một `datacenter` duy nhất, độ trễ `packet` hơn một phút có thể xảy ra trong quá trình tái cấu hình `network topology`, được kích hoạt bởi một vấn đề trong

quá trình nâng cấp phần mềm cho một switch [19]. Do đó, chúng ta phải giả định rằng các thông điệp có thể bị trễ một cách tùy ý.

- Đôi khi các hoạt động giao tiếp bị gián đoạn một phần, tùy thuộc vào việc bạn đang nói chuyện với ai—ví dụ, A và B có thể giao tiếp, và B và C có thể giao tiếp, nhưng A và C thì không thể [20, 21]. Các lỗi đáng ngạc nhiên khác bao gồm một network interface đôi khi loại bỏ tất cả các inbound packet nhưng vẫn gửi các outbound packet thành công [22]. Chỉ vì một liên kết mạng hoạt động theo một hướng không đảm bảo rằng nó cũng hoạt động theo hướng ngược lại.
- Ngay cả một sự gián đoạn mạng ngắn ngủi cũng có thể gây ra những hệ lụy kéo dài lâu hơn nhiều so với vấn đề ban đầu [8, 20, 23].

Ngay cả khi các lỗi mạng hiếm khi xảy ra trong môi trường của bạn, thực tế là lỗi *có thể* xảy ra đồng nghĩa với việc phần mềm của bạn cần phải có khả năng xử lý chúng. Bất cứ khi nào có bất kỳ hoạt động giao tiếp nào diễn ra qua mạng, nó có thể thất bại—không có cách nào tránh khỏi điều đó.

Network partitions

Thuật ngữ *network partition*, hay *netsplit*, đôi khi được sử dụng khi một phần của mạng bị tách rời khỏi phần còn lại do một lỗi mạng. Tuy nhiên, điều này về cơ bản không khác gì các loại gián đoạn mạng khác. Network partitions không liên quan đến sharding của một hệ thống lưu trữ, thứ đôi khi cũng được gọi là *partitioning* (xem Chương 7).

Nếu việc xử lý lỗi của các lỗi mạng không được định nghĩa và kiểm thử, những điều tồi tệ một cách tùy ý có thể xảy ra—ví dụ, cluster có thể rơi vào trạng thái deadlock và vĩnh viễn không thể phục vụ các request, ngay cả khi mạng đã khôi phục [24], hoặc nó có khả năng xóa sạch toàn bộ dữ liệu của bạn [25]. Nếu phần mềm bị đưa vào một tình huống không lường trước được, nó có thể thực hiện những hành động bất ngờ một cách tùy ý.

Xử lý các lỗi mạng không nhất thiết có nghĩa là *chịu đựng* chúng. Nếu mạng của bạn bình thường khá tin cậy, một cách tiếp cận hợp lý có thể chỉ đơn giản là hiển thị thông báo lỗi cho người dùng trong khi mạng đang gặp sự cố. Tuy nhiên, bạn cần phải biết phần mềm của mình phản ứng thế nào với các vấn đề mạng và đảm bảo rằng hệ thống có thể phục hồi từ chúng. Việc chủ động kích hoạt các sự cố mạng để kiểm tra phản ứng của hệ thống có thể là một hướng đi hợp lý (xem “Fault injection”).

Phát hiện lỗi (Fault Detection)

Nhiều hệ thống cần tự động phát hiện các node bị lỗi. Ví dụ:

- Một load balancer cần ngừng gửi các request đến một node đã chết (tức là đưa nó ra khỏi vòng xoay).
- Trong một cơ sở dữ liệu phân tán với single-leader replication, nếu leader gặp lỗi, một trong các follower cần được thăng cấp để trở thành leader mới (xem “Handling Node Outages”).

Thật không may, sự bất định về mạng khiến việc xác định một node có đang hoạt động hay không trở nên khó khăn. Trong những trường hợp cụ thể, bạn có thể nhận được phản hồi để biết rõ ràng rằng có thứ gì đó không hoạt động:

- Nếu bạn có thể kết nối tới máy chủ nơi node đang chạy, nhưng không có process nào đang lắng nghe trên port đích (ví dụ: do process đã bị crash), hệ điều hành sẽ hỗ trợ đóng hoặc từ chối các kết nối TCP bằng cách gửi lại một packet RST hoặc FIN để phản hồi.
- Nếu một process của node bị crash (hoặc bị administrator kill) nhưng hệ điều hành của node vẫn đang chạy, một script có thể thông báo cho các node khác về sự cố crash này để một node khác có thể tiếp quản nhanh chóng mà không phải chờ timeout hết hạn. Ví dụ, HBase thực hiện việc này [26].
- Nếu bạn có quyền truy cập vào giao diện quản lý của các network switch trong datacenter, bạn có thể truy vấn chúng để phát hiện các lỗi liên kết ở cấp độ phần cứng (ví dụ: nếu máy chủ từ xa đã bị ngắt nguồn). Lựa chọn này sẽ bị loại trừ nếu bạn đang kết nối qua internet, nếu bạn đang ở trong một datacenter dùng chung mà không có quyền truy cập vào chính các switch, hoặc nếu bạn không thể tiếp cận giao diện quản lý do vấn đề mạng.
- Nếu một router chắc chắn rằng địa chỉ IP mà bạn đang cố gắng kết nối tới không thể truy cập được, nó có thể phản hồi cho bạn bằng một packet ICMP Destination Unreachable. Tuy nhiên, router cũng không có khả năng phát hiện lỗi thần kỳ; nó cũng chịu những hạn chế tương tự như các thành phần khác trong mạng.

Phản hồi nhanh chóng về việc một node từ xa bị ngắt kết nối là rất hữu ích, nhưng bạn không thể hoàn toàn dựa vào nó. Nếu có lỗi xảy ra, bạn có thể nhận được phản hồi lỗi ở một tầng nào đó trong stack, nhưng nhìn chung bạn phải giả định rằng mình sẽ không nhận được bất kỳ phản hồi nào. Bạn có thể thử lại vài lần, chờ timeout trôi qua, và cuối cùng tuyên bố node đó đã chết nếu không nhận được phản hồi trong khoảng thời gian timeout. Vì node đó thực tế có thể vẫn đang sống, bạn cần thực hiện một sự đánh đổi (trade-off) giữa false positive (dương tính giả) và false negative (âm tính giả): timeout quá ngắn sẽ khiến các node đang sống

bị nghi ngờ nhầm là đã chết, còn timeout quá dài sẽ gây ra những sự chậm trễ không cần thiết khi phải chờ đợi các node đã chết.

Timeout và các sự chậm trễ không giới hạn (Unbounded Delays)

Nếu timeout là cách duy nhất chắc chắn để phát hiện lỗi, vậy thời gian timeout nên là bao lâu? Thật không may, không có câu trả lời đơn giản cho vấn đề này.

Một timeout dài đồng nghĩa với việc phải chờ đợi lâu cho đến khi một node được tuyên bố là đã chết (và trong thời gian này, người dùng có thể phải chờ đợi hoặc thấy các thông báo lỗi). Một timeout ngắn giúp phát hiện lỗi nhanh hơn nhưng lại mang đến rủi ro cao hơn về việc tuyên bố nhầm một node đã chết trong khi thực tế nó chỉ vừa gặp tình trạng chậm lại tạm thời (ví dụ: do sự gia tăng tải đột ngột trên node hoặc trên mạng).

Việc tuyên bố một node đã chết quá sớm là một vấn đề lớn. Nếu node đó thực tế vẫn đang sống và đang thực hiện dở một hành động nào đó (ví dụ: gửi một email), và một node khác tiếp quản, hành động đó có thể bị thực hiện hai lần. Chúng ta sẽ thảo luận vấn đề này chi tiết hơn trong mục “Knowledge, Truth, and Lies” và trong các Chương 10 và 12.

Khi một node được tuyên bố là đã chết, các trách nhiệm của nó cần được chuyển giao sang các node khác, điều này gây thêm tải cho các node đó và cả mạng. Nếu hệ thống vốn đã đang gặp khó khăn với tải cao, việc tuyên bố các node chết quá sớm có thể làm vấn đề trở nên tồi tệ hơn. Cụ thể, có thể xảy ra trường hợp node đó thực tế không hề chết mà chỉ phản hồi chậm do quá tải; việc chuyển tải của nó sang các node khác có thể gây ra một lỗi dây chuyền (cascading failure) (trong trường hợp cực đoan, tất cả các node đều tuyên bố các node khác đã chết, và mọi thứ ngừng hoạt động—hãy xem mục “Khi một hệ thống quá tải không thể phục hồi”).

Hãy tưởng tượng một hệ thống giả định với một mạng đảm bảo độ trễ tối đa cho các gói tin. Mỗi gói tin hoặc được chuyển đến trong khoảng thời gian d hoặc bị mất, và việc chuyển giao không bao giờ mất nhiều thời gian hơn d . Hơn nữa, giả sử rằng bạn có thể đảm bảo một node không bị lỗi luôn xử lý một yêu cầu trong khoảng thời gian r . Trong trường hợp này, bạn có thể đảm bảo rằng mọi yêu cầu thành công đều nhận được phản hồi trong khoảng thời gian $2d + r$ —và nếu bạn không nhận được phản hồi trong khoảng thời gian đó, bạn biết rằng hoặc là mạng hoặc là node từ xa đang không hoạt động. Nếu điều này là đúng, $2d + r$ sẽ là một timeout hợp lý để sử dụng.

Thật không may, hầu hết các hệ thống mà chúng ta làm việc cùng đều không có cả hai sự đảm bảo đó. Các mạng bất đồng bộ (asynchronous networks) có *độ trễ không giới hạn* (chúng cố gắng chuyển các gói tin nhanh nhất có thể, nhưng không có giới hạn trên cho thời gian mà một gói tin có thể mất để đến nơi), và hầu hết các triển khai server đều không thể đảm bảo rằng chúng có thể xử lý các yêu cầu trong một

khoảng thời gian tối đa (xem mục “Cung cấp các đảm bảo về thời gian phản hồi”). Để phát hiện lỗi, việc hệ thống hoạt động nhanh trong hầu hết thời gian là chưa đủ: nếu timeout của bạn thấp, chỉ cần một sự gia tăng đột biến tạm thời trong thời gian khứ hồi (round-trip time) cũng đủ để làm hệ thống mất cân bằng.

Ngăn mạng và xếp hàng (queueing)

Khi lái xe ô tô, thời gian di chuyển trên các mạng lưới đường bộ thường thay đổi nhiều nhất do tắc nghẽn giao thông. Tương tự, sự biến động của độ trễ gói tin trên mạng máy tính thường xuyên nhất là do việc xếp hàng (queueing) [27]:

- Nếu nhiều node đồng thời cố gắng gửi các gói tin đến cùng một đích đến, switch mạng phải xếp hàng chúng lại và đưa chúng vào liên kết mạng đích theo thứ tự từng cái một (như được minh họa trong Hình 9-2). Trên một liên kết mạng bận rộn, một gói tin có thể phải chờ một lúc cho đến khi có được một khe cắm (điều này được gọi là *ngăn mạng*). Nếu có quá nhiều dữ liệu đầu vào khiến hàng đợi của switch bị đầy, gói tin sẽ bị loại bỏ (dropped), vì vậy nó cần được gửi lại—ngay cả khi mạng vẫn đang hoạt động bình thường.
- Khi một gói tin đến máy đích, nếu tất cả các lõi CPU hoặc các thread ứng dụng hiện đang bận, yêu cầu đầu vào từ mạng sẽ được hệ điều hành xếp hàng cho đến khi ứng dụng sẵn sàng xử lý nó. Tùy thuộc vào tải trên máy, việc này có thể mất một khoảng thời gian bất kỳ [28].
- Trong các môi trường ảo hóa, một hệ điều hành đang chạy thường bị tạm dừng trong vài chục mili giây trong khi một máy ảo (VM) khác đang sử dụng một lõi CPU. Trong thời gian này, VM không thể tiêu thụ bất kỳ dữ liệu nào từ mạng, vì vậy dữ liệu đầu vào sẽ được VM monitor xếp hàng (buffer) [29], làm tăng thêm sự biến động của độ trễ mạng.
- Như đã đề cập trước đó, để tránh làm quá tải mạng, TCP giới hạn tốc độ mà nó gửi dữ liệu. Điều này có nghĩa là sẽ có thêm việc xếp hàng tại phía người gửi trước khi dữ liệu kịp đi vào mạng.

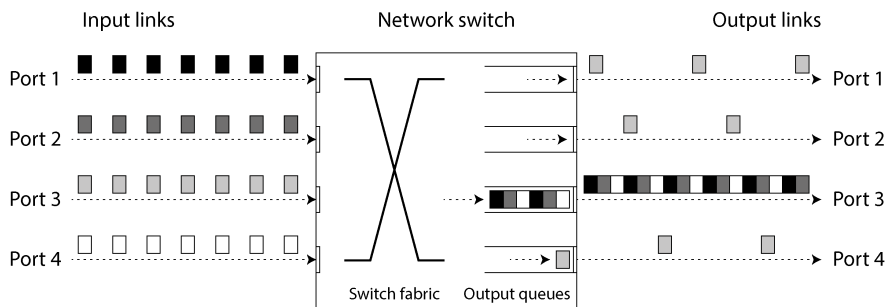


Figure 9-2. Nếu nhiều máy gửi lưu lượng mạng đến cùng một đích, hàng đợi của switch tại đó có thể bị đầy. Ở đây, các port 1, 2 và 4 đều đang cố gắng gửi các packet đến port 3.

Hơn nữa, khi TCP phát hiện và tự động truyền lại một packet bị mất, ứng dụng không nhìn thấy việc mất packet một cách trực tiếp mà sẽ thấy độ trễ (latency) hệ quả (phải chờ cho đến khi hết thời gian timeout, và sau đó chờ packet được truyền lại được xác nhận).

TCP so với UDP

Một số ứng dụng nhạy cảm với latency, chẳng hạn như hội nghị truyền hình và Voice over IP (VoIP), sử dụng UDP thay vì TCP. Lựa chọn này là một sự đánh đổi (trade-off) giữa độ tin cậy và sự biến động của độ trễ: vì UDP không thực hiện kiểm soát luồng (flow control) và không truyền lại các packet bị mất, nó tránh được một số nguyên nhân gây ra độ trễ mạng biến động (mặc dù nó vẫn dễ bị ảnh hưởng bởi hàng đợi switch và độ trễ lập lịch).

UDP là một lựa chọn tốt khi dữ liệu bị trễ không còn giá trị. Ví dụ, trong một cuộc gọi điện thoại VoIP, có lẽ không đủ thời gian để truyền lại một packet bị mất trước khi dữ liệu của nó đến hạn phải được phát qua loa. Trong trường hợp này, việc truyền lại packet là vô nghĩa—thay vào đó, ứng dụng phải lấp đầy khoảng thời gian của packet bị thiếu bằng sự im lặng (gây ra sự gián đoạn âm thanh ngắn) và tiếp tục luồng dữ liệu. Việc thử lại sẽ diễn ra ở tầng con người. ("Bạn có thể nhắc lại được không? Âm thanh vừa bị ngắt một lát.")

Sự biến động của độ trễ mạng

Tất cả các yếu tố này đều góp phần vào sự biến động của độ trễ mạng. Độ trễ hàng đợi (queueing delays) có phạm vi biến động đặc biệt rộng khi một hệ thống tiến gần đến công suất tối đa của nó. Một hệ thống có nhiều công suất dự phòng có thể dễ dàng giải tỏa các hàng đợi, trong khi ở một hệ thống có mức độ sử dụng cao, các hàng đợi dài có thể tích tụ rất nhanh.

Trong các public cloud và datacenter đa người dùng (multitenant), tài nguyên được chia sẻ giữa nhiều khách hàng. Các liên kết mạng và switch, và thậm chí cả giao diện mạng và CPU của mỗi máy chủ (khi chạy trên các VM), đều được chia sẻ. Việc xử lý lượng lớn dữ liệu có thể sử dụng toàn bộ công suất của các liên kết mạng (làm bão hòa chúng). Vì bạn không có quyền kiểm soát hay khả năng quan sát về việc sử dụng tài nguyên dùng chung của các khách hàng khác, độ trễ mạng có thể biến động rất lớn nếu ai đó ở gần bạn (một *noisy neighbor*) đang sử dụng rất nhiều tài nguyên [30, 31].

Trong các môi trường như vậy, bạn chỉ có thể chọn các timeout một cách thực nghiệm: đo lường phân phối của thời gian khứ hồi (round-trip times) mạng trong một khoảng thời gian dài và trên nhiều máy chủ, để xác định sự biến động dự kiến của độ trễ. Sau đó, bằng cách tính đến các đặc tính của ứng dụng, bạn có thể xác định một sự đánh đổi (trade-off) phù hợp giữa độ trễ phát hiện lỗi và rủi ro xảy ra các timeout sớm.

Thậm chí tốt hơn, thay vì sử dụng các timeout hằng số được cấu hình sẵn, các hệ thống có thể liên tục đo lường thời gian phản hồi và sự biến động của chúng (*jitter*) và tự động điều chỉnh timeout theo phân phối thời gian phản hồi quan sát được. Phi Accrual failure detector [32] (thứ được sử dụng, ví dụ, trong Akka và Cassandra [33]), là một cách để thực hiện việc này. Các timeout truyền lại của TCP cũng hoạt động tương tự [5].

Mạng Đồng bộ so với Mạng Bất đồng bộ

Các hệ thống phân tán sẽ đơn giản hơn nhiều nếu chúng ta có thể dựa vào mạng để chuyển phát các packet với một độ trễ tối đa cố định và không làm mất các packet. Tại sao chúng ta không thể giải quyết vấn đề này ở cấp độ phần cứng và làm cho mạng trở nên tin cậy để phần mềm không cần phải lo lắng về nó?

Để trả lời câu hỏi này, thật thú vị khi so sánh mạng datacenter với mạng điện thoại cố định truyền thống (không phải mạng di động, không phải VoIP), vốn cực kỳ tin cậy; các khung âm thanh (audio frames) bị trễ và các cuộc gọi bị rớt là rất hiếm. Một cuộc gọi điện thoại yêu cầu latency đầu cuối (end-to-end latency) thấp liên tục và đủ bằng thông để truyền các mẫu âm thanh giọng nói của bạn. Liệu có tuyệt vời không nếu chúng ta có được độ tin cậy và khả năng dự đoán tương tự trong các mạng máy tính?

Khi bạn thực hiện một cuộc gọi qua mạng điện thoại, nó sẽ thiết lập một *circuit* (mạch chuyển mạch): một lượng băng thông cố định và được đảm bảo sẽ được cấp phát cho cuộc gọi đó, dọc theo toàn bộ lộ trình giữa hai người gọi. *Circuit* này duy trì cho đến khi cuộc gọi kết thúc [34]. Ví dụ, một mạng ISDN chạy với tốc độ cố định là 4.000 khung hình mỗi giây. Khi một cuộc gọi được thiết lập, nó được cấp phát 16 bit không gian trong mỗi khung hình (theo mỗi hướng). Do đó, trong suốt thời gian

diễn ra cuộc gọi, mỗi bên được đảm bảo có thể gửi chính xác 16 bit dữ liệu âm thanh sau mỗi 250 micro giây [35].

Loại mạng này là *synchronous* (đồng bộ): ngay cả khi dữ liệu đi qua nhiều router, nó không phải chịu tình trạng queueing (xếp hàng), bởi vì 16 bit không gian dành cho cuộc gọi đã được dự phòng sẵn tại chặng kế tiếp (next hop) của mạng. Và vì không có queueing, độ trễ (latency) end-to-end tối đa của mạng là cố định. Chúng ta gọi đây là một *bounded delay* (độ trễ có giới hạn).

Liệu chúng ta có thể đơn giản làm cho các độ trễ mạng trở nên có thể dự đoán được không?

Lưu ý rằng một *circuit* trong mạng điện thoại rất khác với một kết nối TCP. Một *circuit* có một lượng băng thông dự phòng cố định mà không ai khác có thể sử dụng chừng nào nó còn được thiết lập, trong khi các packet của một kết nối TCP sử dụng một cách linh hoạt bất kỳ băng thông mạng nào đang có sẵn. Bạn có thể đưa cho TCP một khối dữ liệu có kích thước thay đổi (ví dụ: một email hoặc một trang web), và nó sẽ cố gắng truyền tải khối đó trong thời gian ngắn nhất có thể. Trong khi một kết nối TCP ở trạng thái idle (nhàn rỗi), nó không sử dụng bất kỳ băng thông nào (ngoại trừ có thể là một packet keepalive thỉnh thoảng xuất hiện).

Nếu các mạng datacenter và internet là các mạng chuyển mạch mạch (circuit-switched networks), thì việc thiết lập một thời gian khứ hồi (round-trip time) tối đa được đảm bảo khi một *circuit* được thiết lập là điều khả thi. Tuy nhiên, thực tế không phải vậy. Ethernet và IP là các giao thức chuyển mạch gói (packet-switched protocols), vốn phải chịu tình trạng queueing và do đó dẫn đến các độ trễ không giới hạn trong mạng. Các giao thức này không có khái niệm về *circuit*.

Tại sao các mạng datacenter và internet lại sử dụng chuyển mạch gói? Câu trả lời là vì chúng được tối ưu hóa cho *bursty traffic* (lưu lượng truy cập theo đợt). Một *circuit* rất tốt cho một cuộc gọi âm thanh hoặc video, vốn cần truyền tải một số lượng bit khá ổn định mỗi giây trong suốt thời gian cuộc gọi. Ngược lại, việc yêu cầu một trang web, gửi một email, hoặc truyền một tệp tin không có yêu cầu băng thông cụ thể nào — chúng ta chỉ muốn nó hoàn thành nhanh nhất có thể.

Nếu bạn muốn truyền một tệp tin qua một *circuit*, bạn sẽ phải đoán một mức cấp phát băng thông. Nếu bạn đoán quá thấp, việc truyền tải sẽ chậm một cách không cần thiết, khiến dung lượng mạng không được sử dụng. Nếu bạn đoán quá cao, *circuit* không thể được thiết lập (vì mạng không thể cho phép tạo ra một *circuit* nếu việc cấp phát băng thông của nó không thể được đảm bảo). Ngược lại, TCP thích ứng một cách linh hoạt tốc độ truyền dữ liệu theo dung lượng mạng hiện có.

Latency và Hiệu năng sử dụng tài nguyên

Nói một cách tổng quát hơn, bạn có thể coi các độ trễ thay đổi là hệ quả của việc phân mảnh tài nguyên (resource partitioning) một cách linh hoạt.

Giả sử bạn có một dây dẫn (hoặc cáp quang) giữa hai tổng đài điện thoại có thể tải tới 10.000 cuộc gọi đồng thời. Mỗi *circuit* được chuyển mạch qua dây dẫn này sẽ chiếm một trong các khe (slot) cuộc gọi đó. Do đó, bạn có thể coi dây dẫn là một tài nguyên có thể được chia sẻ bởi tới đa 10.000 người dùng đồng thời. Tài nguyên này được chia theo cách *static* (tĩnh): ngay cả khi bạn là cuộc gọi duy nhất trên dây dẫn lúc này, và tất cả 9.999 khe còn lại đều không được sử dụng, *circuit* của bạn vẫn được cấp phát cùng một lượng băng thông cố định như khi dây dẫn được sử dụng hết công suất.

Ngược lại, internet chia sẻ băng thông mạng một cách *dynamic* (động). Những người gửi đẩy và chen lấn lẫn nhau để đưa các packet của họ qua dây dẫn nhanh nhất có thể, và các switch trong mạng sẽ quyết định packet nào được gửi (tức là việc cấp phát băng thông) từ thời điểm này sang thời điểm khác. Cách tiếp cận này có nhược điểm là gây ra queueing, nhưng ưu điểm là nó tối đa hóa việc sử dụng dây dẫn. Dây dẫn có chi phí cố định, vì vậy nếu bạn sử dụng nó tốt hơn, mỗi byte bạn gửi qua đó sẽ rẻ hơn.

Một tình huống tương tự cũng xảy ra với CPU. Nếu bạn chia sẻ mỗi CPU core một cách linh hoạt giữa nhiều thread, đôi khi một thread phải chờ trong hàng đợi chạy (run queue) của hệ điều hành trong khi một thread khác đang chạy, vì vậy một thread có thể bị tạm dừng trong các khoảng thời gian khác nhau [36]. Tuy nhiên, điều này tận dụng phần cứng tốt hơn so với việc bạn cấp phát một số lượng chu kỳ CPU cố định cho mỗi thread (xem mục “Cung cấp các đảm bảo về thời gian phản hồi”). Việc tận dụng phần cứng tốt hơn cũng là lý do tại sao các nền tảng cloud chạy nhiều VM từ các khách hàng khác nhau trên cùng một máy vật lý.

Các đảm bảo về latency có thể đạt được trong một số môi trường nhất định, nếu các tài nguyên được phân mảnh tĩnh (ví dụ: phần cứng chuyên dụng và các phân bổ băng thông độc quyền). Tuy nhiên, những đảm bảo này đi kèm với cái giá là giảm khả năng tận dụng—nói cách khác, nó đắt hơn. Mặt khác, multitенancy với việc phân mảnh tài nguyên linh hoạt mang lại khả năng tận dụng tốt hơn, vì vậy nó rẻ hơn, nhưng nó có nhược điểm là gây ra các độ trễ biến thiên.

Các độ trễ biến thiên trong mạng không phải là một quy luật tự nhiên mà đơn giản là kết quả của một sự đánh đổi (trade-off) giữa chi phí và lợi ích.

Kết hợp chuyển mạch kênh (circuit switching) và chuyển mạch gói (packet switching)

Đã có một số nỗ lực nhằm xây dựng các mạng hybrid hỗ trợ cả chuyển mạch kênh và chuyển mạch gói. *Asynchronous Transfer Mode* (ATM) từng là đối thủ của Ethernet vào những năm 1980, nhưng nó không được áp dụng rộng rãi bên ngoài các bộ chuyển mạch lõi của mạng điện thoại. InfiniBand có một số điểm tương đồng [37]: nó thực thi kiểm soát luồng đầu-cuối (end-to-end flow control) tại lớp liên kết (link layer), giúp giảm nhu cầu xếp hàng (queueing) trong mạng, mặc dù nó vẫn có thể gặp phải tình trạng trễ do tắc nghẽn liên kết [38].

Với việc sử dụng cẩn thận các cơ chế *quality of service* (QoS - chất lượng dịch vụ) như ưu tiên và lập lịch cho các gói tin cùng với *admission control* (kiểm soát chấp nhận - giới hạn tốc độ của bên gửi), ta có thể mô phỏng chuyển mạch kênh trên các mạng gói, hoặc cung cấp độ trễ được giới hạn về mặt thống kê [27, 34]. Các thuật toán mạng mới như *Low Latency, Low Loss, and Scalable Throughput* (L4S) cố gắng giảm thiểu một số vấn đề về xếp hàng và kiểm soát tắc nghẽn ở cả cấp độ client và router. Bộ điều khiển lưu lượng (TC) của Linux cũng cho phép các ứng dụng thay đổi mức độ ưu tiên của các gói tin vì mục đích QoS.

Tuy nhiên, các cơ chế QoS như vậy hiện không được kích hoạt trong các trung tâm dữ liệu multitenant và public cloud, hoặc khi giao tiếp qua internet. Công nghệ hiện đang được triển khai không cho phép chúng ta đưa ra bất kỳ đảm bảo nào về độ trễ hoặc độ tin cậy của mạng; chúng ta phải giả định rằng tình trạng tắc nghẽn mạng, xếp hàng và các độ trễ không giới hạn sẽ xảy ra. Do đó, không có giá trị "đúng" nào cho các timeout—chúng cần được xác định thông qua thực nghiệm.

Các thỏa thuận peering giữa các nhà cung cấp dịch vụ internet và việc thiết lập các tuyến đường thông qua Border Gateway Protocol (BGP) có sự tương đồng chặt chẽ với chuyển mạch kênh hơn là định tuyến gói IP thông thường. Ở cấp độ này, ta có thể mua băng thông chuyên dụng. Tuy nhiên, định tuyến internet hoạt động ở cấp độ các mạng, không phải ở các kết nối riêng lẻ giữa các host, và trên một thang thời gian dài hơn nhiều.

Đồng hồ không tin cậy

Đồng hồ và thời gian rất quan trọng. Các ứng dụng phụ thuộc vào đồng hồ theo nhiều cách khác nhau để trả lời các câu hỏi như sau:

1. Yêu cầu này đã bị timeout chưa?
2. Thời gian phản hồi ở percentile thứ 99 của dịch vụ này là bao nhiêu?
3. Trung bình dịch vụ này đã xử lý bao nhiêu truy vấn mỗi giây trong năm phút qua?

4. Người dùng đã dành bao lâu trên trang web của chúng ta?
5. Bài viết này được xuất bản khi nào?
6. Email nhắc nhở nên được gửi vào ngày và giờ nào?
7. Khi nào thì mục entry trong cache này hết hạn?
8. Timestamp trên thông báo lỗi này trong file log là gì?

Các câu hỏi từ 1–4 đo lường *khoảng thời gian* (ví dụ: khoảng thời gian giữa lúc một yêu cầu được gửi đi và lúc một phản hồi được nhận), trong khi các câu hỏi từ 5–8 mô tả các *thời điểm* (các sự kiện xảy ra vào một ngày cụ thể, tại một thời điểm cụ thể).

Trong một distributed system, thời gian là một vấn đề phức tạp, bởi vì việc giao tiếp không diễn ra tức thời; cần có thời gian để một thông điệp truyền qua mạng từ máy này sang máy khác. Thời điểm một thông điệp được nhận luôn muộn hơn thời điểm nó được gửi đi, nhưng do các độ trễ biến thiên trong mạng, chúng ta không biết nó muộn hơn bao nhiêu. Thực tế này đôi khi khiến việc xác định thứ tự các sự kiện xảy ra trở nên khó khăn khi có sự tham gia của nhiều máy.

Hơn nữa, mỗi máy trong mạng đều có đồng hồ riêng, vốn là một thiết bị phần cứng —thường là một bộ dao động tinh thể thạch anh. Các thiết bị này không chính xác tuyệt đối, vì vậy mỗi máy có khái niệm riêng về thời gian, có thể nhanh hơn hoặc chậm hơn một chút so với các máy khác. Việc đồng bộ hóa các đồng hồ ở một mức độ nào đó là khả thi; cơ chế được sử dụng phổ biến nhất là Network Time Protocol (NTP), cho phép đồng hồ máy tính được điều chỉnh theo thời gian do một nhóm các máy chủ báo cáo [39]. Ngược lại, các máy chủ này sẽ lấy thời gian từ một nguồn thời gian chính xác hơn, chẳng hạn như bộ thu GPS.

Đồng hồ Monotonic so với Đồng hồ Time-of-Day

Các máy tính hiện đại có ít nhất hai loại đồng hồ: đồng hồ time-of-day và đồng hồ monotonic. Mặc dù cả hai đều đo thời gian, nhưng việc phân biệt giữa chúng là rất quan trọng, vì chúng phục vụ các mục đích khác nhau.

Đồng hồ time-of-day

Một *đồng hồ time-of-day* thực hiện những gì bạn kỳ vọng một cách trực quan ở một chiếc đồng hồ: nó trả về ngày và giờ hiện tại theo lịch (còn được gọi là *wall-clock time*). Ví dụ, `clock_gettime(CLOCK_REALTIME)` trên Linux và `System.currentTimeMillis` trong Java trả về số giây (hoặc mili giây) kể từ *epoch*, được định nghĩa là nửa đêm UTC vào ngày 1 tháng 1 năm 1970 theo lịch Gregorian, không tính các giây nhuận. Một số hệ thống sử dụng các ngày khác làm điểm tham chiếu của chúng. (Mặc dù đồng hồ Linux được gọi là *real-time*, nó không

liên quan gì đến các hệ điều hành real-time, như đã thảo luận trong mục “Cung cấp đảm bảo response time”).

Các đồng hồ time-of-day thường được đồng bộ hóa với NTP, điều này có nghĩa là một timestamp từ một máy (trong điều kiện lý tưởng) sẽ có ý nghĩa tương đương với một timestamp trên một máy khác. Tuy nhiên, các đồng hồ time-of-day cũng có nhiều điểm kỳ lạ khác nhau, như được mô tả trong mục tiếp theo. Cụ thể, nếu đồng hồ cục bộ chạy quá xa so với máy chủ NTP, nó có thể bị buộc phải reset và có vẻ như nhảy ngược về một thời điểm trước đó. Những bước nhảy này, cũng như các bước nhảy tương tự gây ra bởi các giây nhuận, khiến các đồng hồ time-of-day không phù hợp để đo khoảng thời gian đã trôi qua [40].

Các đồng hồ time-of-day cũng có thể gặp phải các bước nhảy do việc bắt đầu và kết thúc của Daylight Saving Time (DST), mặc dù những điều này có thể tránh được bằng cách luôn sử dụng UTC làm múi giờ, vốn không có DST. Trong lịch sử, các đồng hồ này có độ phân giải khá thô (ví dụ, tiến về phía trước theo các bước 10 ms trên các hệ thống Windows cũ [41]). Trên các hệ thống gần đây, đây không còn là vấn đề lớn.

Đồng hồ monotonic

Một *đồng hồ monotonic* phù hợp để đo một khoảng thời gian (duration), chẳng hạn như một timeout hoặc response time của một service; ví dụ, `clock_gettime(CLOCK_MONOTONIC)` hoặc `clock_gettime(CLOCK_BOOTTIME)` trên Linux [42] và `System.nanoTime` trong Java, đo thời gian bằng cách sử dụng một đồng hồ monotonic. Tên gọi này bắt nguồn từ thực tế là loại đồng hồ này được đảm bảo luôn luôn tiến về phía trước (trong khi một đồng hồ time-of-day có thể nhảy ngược về quá khứ).

Bạn có thể kiểm tra giá trị của một đồng hồ monotonic tại một thời điểm, thực hiện một việc gì đó, và sau đó kiểm tra lại đồng hồ vào một thời điểm muộn hơn. *Sự khác biệt* giữa hai giá trị cho bạn biết bao nhiêu thời gian đã trôi qua giữa hai lần kiểm tra —nó giống một chiếc đồng hồ bấm giờ hơn là một chiếc đồng hồ treo tường. Tuy nhiên, giá trị *tuyệt đối* của đồng hồ là vô nghĩa; nó có thể là số nan giây kể từ khi máy tính được khởi động, hoặc một giá trị tùy ý tương tự. Cụ thể, việc so sánh các giá trị đồng hồ monotonic từ hai máy tính là vô nghĩa, vì chúng không có cùng ý nghĩa.

Trên một máy chủ có nhiều socket CPU, có thể có một bộ định thời (timer) riêng biệt cho mỗi CPU, và chúng không nhất thiết phải được đồng bộ hóa với các CPU khác [43]. Các hệ điều hành sẽ bù đắp cho bất kỳ sự sai lệch nào và cố gắng trình diễn một cái nhìn monotonic về đồng hồ cho các thread ứng dụng, ngay cả khi chúng được lập lịch trên nhiều CPU khác nhau. Tuy nhiên, chúng ta nên thận trọng khi tin tưởng hoàn toàn vào sự đảm bảo về tính monotonic này [44].

NTP có thể điều chỉnh tần số mà tại đó đồng hồ monotonic tiến về phía trước (điều này được gọi là *slewing* đồng hồ) nếu nó phát hiện ra rằng thạch anh cục bộ của máy tính đang chạy nhanh hơn hoặc chậm hơn so với NTP server. Theo mặc định, NTP cho phép tốc độ đồng hồ được tăng tốc hoặc giảm tốc lên tới 0,05%, nhưng nó không thể khiến đồng hồ monotonic nhảy vọt về phía trước hoặc lùi về phía sau. Độ phân giải của các đồng hồ monotonic thường khá tốt: trên hầu hết các hệ thống, chúng có thể đo các khoảng thời gian tính bằng micro giây hoặc nhỏ hơn.

Trong một distributed system, việc sử dụng đồng hồ monotonic để đo thời gian đã trôi qua (ví dụ: timeouts) thường là ổn, vì nó không giả định bất kỳ sự đồng bộ hóa nào giữa đồng hồ của các node khác nhau và không nhạy cảm với những sai số nhỏ trong phép đo.

Đồng bộ hóa và Độ chính xác của Đồng hồ

Các đồng hồ monotonic không yêu cầu đồng bộ hóa, nhưng các đồng hồ thời gian trong ngày (time-of-day clocks) cần phải được thiết lập theo một NTP server hoặc nguồn thời gian bên ngoài khác để có thể hữu dụng. Thật không may, các phương pháp của chúng ta để có được một chiếc đồng hồ hiển thị thời gian chính xác không hề đáng tin cậy hay chính xác như bạn kỳ vọng—các đồng hồ phần cứng và NTP có thể là những thực thể rất thất thường. Dưới đây là một vài ví dụ:

- Đồng hồ thạch anh trong một máy tính điển hình không chính xác cho lắm: nó bị *drift* (chạy nhanh hơn hoặc chậm hơn mức cần thiết). Độ lệch đồng hồ (clock drift) thay đổi tùy thuộc vào nhiệt độ của máy. Google giả định độ lệch đồng hồ lên tới 200 ppm (phần triệu) cho các máy chủ của mình [45], tương đương với độ lệch 6 ms đối với một chiếc đồng hồ được tái đồng bộ hóa với một server sau mỗi 30 giây, hoặc độ lệch 17 giây đối với một chiếc đồng hồ được tái đồng bộ hóa mỗi ngày một lần. Độ lệch này giới hạn độ chính xác tốt nhất mà bạn có thể đạt được, ngay cả khi mọi thứ đang hoạt động chính xác.
- Nếu đồng hồ của một máy tính khác biệt quá nhiều so với một NTP server, nó có thể từ chối đồng bộ hóa hoặc bị buộc phải thiết lập lại (reset) [39]. Bất kỳ ứng dụng nào quan sát thời gian trước và sau lần reset này đều có thể thấy thời gian chạy ngược lại hoặc đột ngột nhảy vọt về phía trước.
- Nếu một node vô tình bị chặn bởi firewall khỏi các NTP server, lỗi cấu hình này có thể không bị phát hiện trong một khoảng thời gian, trong thời gian đó độ lệch có thể tích tụ thành những sai lệch lớn giữa nó và đồng hồ của các node khác. Các bằng chứng thực tế cho thấy điều này thực sự xảy ra trong thực tế.
- Việc đồng bộ hóa NTP chỉ tốt tương đương với độ trễ mạng, vì vậy độ chính xác của nó bị giới hạn khi bạn đang ở trên một mạng bị tắc nghẽn với các độ trễ packet biến thiên. Một thí nghiệm đã chỉ ra rằng sai số tối thiểu 35 ms là có thể đạt được khi đồng bộ hóa qua internet [46], mặc dù các đợt tăng vọt độ trễ mạng

thỉnh thoảng có thể dẫn đến sai số khoảng một giây. Tùy thuộc vào cấu hình, độ trễ mạng lớn có thể khiến NTP client từ bỏ hoàn toàn.

- Một số NTP server bị sai hoặc bị cấu hình sai, báo cáo thời gian lệch tới hàng giờ [47, 48]. Các NTP client giảm thiểu những lỗi như vậy bằng cách truy vấn nhiều server và bỏ qua các outlier. Tuy nhiên, việc đặt cược tính chính xác của các hệ thống của bạn vào thời gian mà bạn được thông báo bởi một kẻ lạ mặt trên internet là một điều khá đáng lo ngại.
- Giây nhuận (leap seconds) dẫn đến một phút có độ dài là 59 giây hoặc 61 giây, điều này làm xáo trộn các giả định về thời gian trong các hệ thống không được thiết kế để tính đến giây nhuận [49]. Việc giây nhuận đã làm sập nhiều hệ thống lớn [40, 50] cho thấy các giả định sai lầm về đồng hồ có thể dễ dàng len lỏi vào một hệ thống như thế nào. Cách tốt nhất để xử lý giây nhuận có thể là khiến các NTP server "nói dối", bằng cách thực hiện điều chỉnh giây nhuận một cách dần dần trong suốt một ngày (phương pháp này được gọi là *smearing*) [51, 52], mặc dù hành vi thực tế của NTP server có sự khác biệt trong thực tế [53]. Giây nhuận sẽ không còn được sử dụng kể từ năm 2035, vì vậy thật may mắn là vấn đề này sẽ biến mất.
- Trong các VM, đồng hồ phần cứng được ảo hóa, điều này đặt ra những thách thức bổ sung cho các ứng dụng cần duy trì thời gian chính xác [54]. Khi một lõi CPU được chia sẻ giữa các VM, mỗi VM sẽ bị tạm dừng trong vài chục mili giây trong khi một VM khác đang chạy. Từ góc độ của một ứng dụng, sự tạm dừng này biểu hiện dưới dạng đồng hồ đột ngột nhảy về phía trước khi VM tiếp tục chạy [29]. Một NTP client chạy bên trong VM không biết khi nào sự tạm dừng xảy ra, vì vậy nó có thể báo cáo độ chính xác của đồng hồ một cách sai lệch [55].
- Nếu bạn chạy phần mềm trên các thiết bị mà bạn không hoàn toàn kiểm soát (ví dụ: thiết bị di động hoặc thiết bị nhúng), bạn có lẽ không thể tin tưởng hoàn toàn vào đồng hồ phần cứng của chúng. Một số người dùng cố tình thiết lập đồng hồ phần cứng của thiết bị họ ở một ngày và giờ không chính xác, chẳng hạn như để gian lận trong trò chơi [56]. Kết quả là, đồng hồ có thể bị thiết lập ở một thời điểm sai lệch khủng khiếp.

Việc đạt được độ chính xác của đồng hồ rất tốt là khả thi nếu bạn quan tâm đến nó đủ để đầu tư các nguồn lực đáng kể. Ví dụ, quy định MiFID II của Châu Âu dành cho các tổ chức tài chính yêu cầu tất cả các quỹ giao dịch tần suất cao phải đồng bộ hóa đồng hồ của họ trong phạm vi 100 micro giây so với UTC, nhằm giúp gỡ lỗi các bất thường của thị trường như "flash crashes" (sụt giảm chớp nhoáng) và giúp phát hiện các hành vi thao túng thị trường [57].

Độ chính xác như vậy có thể đạt được bằng các phần cứng đặc biệt (máy thu GPS và/hoặc đồng hồ nguyên tử), giao thức Precision Time Protocol (PTP), cùng với việc triển khai và giám sát cẩn thận [58, 59]. Chỉ dựa vào GPS có thể gặp rủi ro vì tín hiệu

GPS có thể dễ dàng bị gây nhiễu. Ở một số vị trí (ví dụ: gần các cơ sở quân sự), điều này xảy ra thường xuyên [60]. Một số nhà cung cấp cloud đã bắt đầu cung cấp khả năng đồng bộ hóa đồng hồ độ chính xác cao cho các máy ảo của họ [61], nhưng việc đồng bộ hóa đồng hồ vẫn đòi hỏi sự chăm sóc kỹ lưỡng. Nếu NTP daemon của bạn bị cấu hình sai hoặc một firewall đang chặn lưu lượng NTP, lỗi đồng hồ do drift có thể nhanh chóng trở nên lớn.

Dựa vào các đồng hồ đã được đồng bộ hóa

Vấn đề với đồng hồ là mặc dù chúng có vẻ đơn giản và dễ sử dụng, chúng lại ẩn chứa một số cạm bẫy đáng ngạc nhiên. Một ngày có thể không có chính xác 86.400 giây, các đồng hồ hiển thị thời gian trong ngày có thể chạy ngược về quá khứ, và thời gian theo đồng hồ của một node có thể rất khác so với đồng hồ của một node khác.

Trước đó trong chương này, chúng ta đã thảo luận về việc các mạng làm mất và trì hoãn các gói tin một cách tùy ý. Mặc dù các mạng hoạt động ổn định trong hầu hết thời gian, phần mềm phải được thiết kế dựa trên giả định rằng mạng thỉnh thoảng sẽ gặp lỗi, và phần mềm phải xử lý các lỗi đó một cách mượt mà. Điều tương tự cũng đúng với đồng hồ: mặc dù chúng hoạt động khá tốt trong hầu hết thời gian, phần mềm mạnh mẽ cần phải sẵn sàng để đối phó với các đồng hồ không chính xác.

Một phần của vấn đề là các đồng hồ không chính xác rất dễ bị bỏ qua. Nếu CPU của một máy bị lỗi hoặc mạng của nó bị cấu hình sai, rất có thể nó sẽ không hoạt động chút nào, vì vậy vấn đề sẽ nhanh chóng bị phát hiện và khắc phục. Mặt khác, nếu đồng hồ thạch anh của nó bị lỗi hoặc NTP client của nó bị cấu hình sai, hầu hết mọi thứ sẽ có vẻ vẫn hoạt động tốt, mặc dù đồng hồ của nó dần dần drift càng lúc càng xa thực tế. Nếu một phần mềm đang dựa vào một đồng hồ được đồng bộ hóa chính xác, kết quả có nhiều khả năng là sự mất mát dữ liệu âm thầm và tinh vi hơn là một sự cố sập hệ thống đột ngột [62, 63].

Vì vậy, nếu bạn sử dụng phần mềm yêu cầu các đồng hồ được đồng bộ hóa, việc giám sát cẩn thận các độ lệch đồng hồ (clock offsets) giữa tất cả các máy trong cluster của bạn là điều thiết yếu. Bất kỳ node nào có đồng hồ bị drift quá xa so với các node khác đều nên được tuyên bố là đã chết và bị loại bỏ. Việc giám sát như vậy đảm bảo rằng bạn nhận ra các đồng hồ bị lỗi trước khi chúng có thể gây ra quá nhiều thiệt hại.

Timestamp để sắp xếp thứ tự các sự kiện

Hãy xét một tình huống cụ thể mà trong đó việc dựa vào đồng hồ là một sự cám dỗ nhưng lại nguy hiểm: thứ tự của các sự kiện trên nhiều node [64]. Ví dụ, nếu hai client thực hiện ghi vào một distributed database, ai là người đến trước? Lệnh ghi nào là lệnh mới nhất?

Hình 9-3 minh họa một cách sử dụng nguy hiểm các đồng hồ thời gian trong ngày (time-of-day clocks) trong một cơ sở dữ liệu với multi-leader replication (ví dụ này tương tự như Hình 6-8). Client A ghi $x = 1$ lên node 1; thao tác ghi này được replication sang node 3; client B tăng x trên node 3 (lúc này chúng ta có $x = 2$); và cuối cùng, cả hai thao tác ghi đều được replication sang node 2. Như hình này cho thấy, khi một thao tác ghi được replication sang các node khác, nó được gắn một timestamp dựa theo đồng hồ thời gian trong ngày trên chính node nơi thao tác ghi đó bắt nguồn. Sự đồng bộ hóa đồng hồ trong ví dụ này là rất tốt; độ lệch (skew) giữa node 1 và node 3 nhỏ hơn 3 ms, điều mà có lẽ còn tốt hơn cả những gì bạn có thể kỳ vọng trong thực tế.

Vì việc tăng giá trị được xây dựng dựa trên lần ghi $x = 1$ trước đó, chúng ta có thể kỳ vọng rằng lần ghi $x = 2$ sẽ có timestamp lớn hơn trong hai lần. Thật không may, đó không phải là những gì xảy ra trong Hình 9-3. Lần ghi $x = 1$ có timestamp là 42,004 giây, nhưng lần ghi $x = 2$ lại có timestamp là 42,003 giây. Nói cách khác, lần ghi bởi client B xảy ra muộn hơn về mặt nhân quả so với lần ghi bởi client A, nhưng lần ghi của B lại có timestamp sớm hơn.

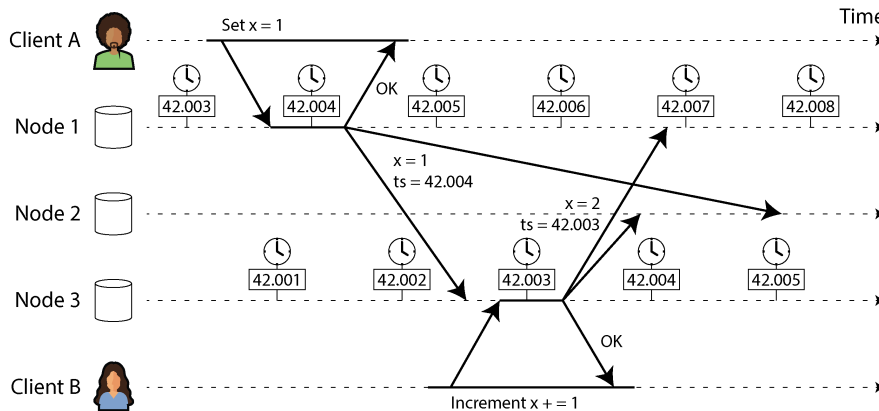


Figure 9-3. Việc dựa vào timestamp để sắp xếp thứ tự các sự kiện có thể gây ra vấn đề khi các đồng hồ theo thời gian trong ngày không được đồng bộ hóa một cách hoàn hảo.

Như đã thảo luận trong mục “Giải quyết các lần ghi xung đột”, một cách để giải quyết các xung đột giữa các giá trị được ghi đồng thời trên các node khác nhau là *last write wins* (LWW), nghĩa là giữ lại lần ghi có timestamp lớn nhất cho một key nhất định và loại bỏ tất cả các lần ghi có timestamp cũ hơn. Trong Hình 9-3, khi node 2 nhận được hai sự kiện này, nó sẽ kết luận sai rằng $x = 1$ là giá trị mới hơn và loại bỏ lần ghi $x = 2$, do đó việc tăng giá trị bị mất.

Vấn đề này có thể được ngăn chặn bằng cách đảm bảo rằng khi một giá trị bị ghi đè, giá trị mới luôn có timestamp cao hơn giá trị bị ghi đè, ngay cả khi timestamp đó vượt quá đồng hồ cục bộ của bên ghi. Tuy nhiên, điều đó làm phát sinh chi phí cho

một lần đọc bổ sung để tìm timestamp lớn nhất hiện có. Một số hệ thống, bao gồm Cassandra và ScyllaDB, được thiết kế để tránh lượt truyền (round trip) bổ sung này và chỉ đơn giản sử dụng timestamp của đồng hồ client cùng với chính sách LWW [62]. Cách tiếp cận này gặp phải một số vấn đề nghiêm trọng:

- Các lần ghi vào database có thể biến mất một cách bí ẩn. Một node có đồng hồ bị trễ sẽ không thể ghi đè các giá trị đã được ghi trước đó bởi một node có đồng hồ chạy nhanh hơn cho đến khi độ lệch đồng hồ (clock skew) giữa các node kết thúc [63, 65]. Kịch bản này có thể khiến một lượng dữ liệu bất kỳ bị loại bỏ một cách âm thầm mà không có bất kỳ lỗi nào được báo cáo về cho ứng dụng.
- LWW không thể phân biệt giữa các lần ghi xảy ra tuần tự liên tiếp (trong Hình 9-3, việc tăng giá trị của client B chắc chắn xảy ra *sau* lần ghi của client A) và các lần ghi thực sự đồng thời (không bên ghi nào biết về bên kia). Cần có các cơ chế theo dõi quan hệ nhân quả (causality) bổ sung, chẳng hạn như version vectors, để ngăn chặn việc vi phạm quan hệ nhân quả (xem “Phát hiện các lần ghi đồng thời”).
- Hai node có thể độc lập tạo ra các lần ghi với cùng một timestamp, đặc biệt là khi đồng hồ chỉ có độ phân giải mili giây. Cần có một giá trị phá vỡ thế bế tắc (tiebreaker) bổ sung (có thể chỉ đơn giản là một số ngẫu nhiên lớn) để giải quyết các xung đột như vậy, nhưng cách tiếp cận này cũng có thể dẫn đến việc vi phạm quan hệ nhân quả [62].

Vì vậy, mặc dù việc giải quyết xung đột bằng cách giữ lại giá trị “mới nhất” và loại bỏ các giá trị khác là rất hấp dẫn, nhưng điều quan trọng là phải nhận thức được rằng định nghĩa về “mới nhất” phụ thuộc vào đồng hồ thời gian thực cục bộ, thứ hoàn toàn có thể không chính xác. Ngay cả với các đồng hồ được đồng bộ hóa chặt chẽ bằng NTP, bạn có thể gửi một gói tin tại timestamp 100 ms (theo đồng hồ của bên gửi) và nó đến nơi tại timestamp 99 ms (theo đồng hồ của bên nhận)—khi đó có vẻ như gói tin đã đến trước khi nó được gửi đi, một điều không thể xảy ra.

Liệu việc đồng bộ hóa NTP có thể đạt được độ chính xác đủ cao để những thứ tự sai lệch như vậy không xảy ra? Có lẽ là không, bởi vì chính độ chính xác đồng bộ hóa của NTP bị giới hạn bởi thời gian truyền vòng (round-trip time) của mạng, bên cạnh các nguồn lỗi khác như độ trôi của tinh thể thạch anh (quartz drift). Để đảm bảo một thứ tự chính xác, bạn sẽ cần lỗi đồng hồ thấp hơn đáng kể so với độ trễ mạng, điều này là không thể.

Các *logical clocks* (đồng hồ logic) [66], dựa trên các bộ đếm tăng dần thay vì một tinh thể thạch anh dao động, là một giải pháp thay thế an toàn hơn để sắp xếp thứ tự các sự kiện (xem “Phát hiện các lần ghi đồng thời”). Logical clocks không đo thời gian trong ngày hay số giây đã trôi qua, mà chỉ đo thứ tự tương đối của các sự kiện (liệu một sự kiện xảy ra trước hay sau một sự kiện khác). Ngược lại, các đồng hồ thời gian thực và đồng hồ monotonic, vốn đo thời gian thực tế đã trôi qua, được gọi là *physical*

clocks (đồng hồ vật lý). Chúng ta sẽ tìm hiểu chi tiết hơn về logical clocks trong mục “ID Generators and Logical Clocks”.

Đọc giá trị đồng hồ với một khoảng tin cậy

Bạn có thể đọc được đồng hồ hiển thị thời gian trong ngày của một máy tính với độ phân giải microsecond hoặc thậm chí là nanosecond. Nhưng ngay cả khi bạn có được phép đo chi tiết đến vậy, điều đó không có nghĩa là giá trị đó thực sự chính xác đến mức độ chính xác như thế. Trên thực tế, rất có thể là không. Như đã đề cập trước đó, độ drift trong một đồng hồ thạch anh không chính xác có thể dễ dàng lên tới vài millisecond, ngay cả khi bạn đồng bộ hóa với một NTP server trên mạng cục bộ mỗi phút một lần. Với một NTP server trên internet công cộng, độ chính xác tốt nhất có thể chỉ đạt mức hàng chục millisecond, và sai số có thể dễ dàng tăng vọt lên trên 100 ms khi mạng bị tắc nghẽn.

Do đó, việc coi một giá trị đọc từ đồng hồ là một thời điểm duy nhất là không hợp lý. Nó giống như một khoảng thời gian nằm trong một khoảng tin cậy (confidence interval) hơn—ví dụ, một hệ thống có thể tin tưởng 95% rằng thời gian hiện tại nằm trong khoảng từ 10,3 đến 10,5 giây sau phút đó, nhưng nó không thể biết chính xác hơn mức đó [67]. Nếu chúng ta chỉ biết thời gian ± 100 ms, thì các chữ số microsecond trong timestamp về cơ bản là vô nghĩa.

Giới hạn bất định có thể được tính toán dựa trên nguồn thời gian của bạn. Nếu bạn có một bộ thu GPS hoặc đồng hồ nguyên tử được kết nối trực tiếp với máy tính, phạm vi sai số dự kiến sẽ được quyết định bởi thiết bị đó và, trong trường hợp của GPS, bởi chất lượng tín hiệu từ các vệ tinh. Nếu bạn lấy thời gian từ một server, độ bất định sẽ dựa trên độ drift thạch anh dự kiến kể từ lần đồng bộ cuối cùng với server, cộng với độ bất định của NTP server, cộng với thời gian khứ hồi (round-trip time) tới server (theo ước tính sơ bộ và giả định rằng bạn tin tưởng server đó).

Thật không may, hầu hết các hệ thống không hiển thị độ bất định này; ví dụ, khi bạn gọi `clock_gettime`, giá trị trả về không cho bạn biết sai số dự kiến của timestamp, vì vậy bạn không biết liệu khoảng tin cậy của nó là năm millisecond hay là năm năm.

Có những ngoại lệ. API *TrueTime* trong Google Spanner [45] và Amazon ClockBound báo cáo rõ ràng khoảng tin cậy của đồng hồ cục bộ. Khi bạn yêu cầu thời gian hiện tại, bạn sẽ nhận lại hai giá trị: `[earliest, latest]`, đó là timestamp *sớm nhất có thể* và *muộn nhất có thể*. Dựa trên các tính toán về độ bất định, đồng hồ biết rằng thời gian hiện tại thực tế nằm đâu đó trong khoảng đó. Độ rộng của khoảng này phụ thuộc vào, cùng với các yếu tố khác, vào việc đã bao lâu kể từ lần cuối cùng đồng hồ thạch anh cục bộ được đồng bộ hóa với một nguồn thời gian chính xác hơn.

Các đồng hồ được đồng bộ hóa cho các snapshot toàn cục

Trong mục “Snapshot Isolation và Repeatable Read”, chúng ta đã thảo luận về *multiversion concurrency control* (MVCC), một tính năng rất hữu ích trong các cơ sở dữ liệu cần hỗ trợ cả các transaction đọc/ghi nhỏ, nhanh và các transaction chỉ đọc lớn, chạy lâu (ví dụ: để sao lưu hoặc phân tích). Nó cho phép các transaction chỉ đọc nhìn thấy một *snapshot* của cơ sở dữ liệu, một trạng thái nhất quán tại một thời điểm cụ thể, mà không cần khóa và gây can thiệp vào các transaction đọc/ghi.

Thông thường, MVCC yêu cầu một transaction ID tăng đơn điệu (monotonically increasing). Nếu một thao tác ghi xảy ra muộn hơn snapshot (tức là thao tác ghi có transaction ID lớn hơn snapshot), thì thao tác ghi đó sẽ không hiển thị đối với transaction snapshot. Trên một cơ sở dữ liệu đơn node, một bộ đếm đơn giản là đủ để tạo ra các transaction ID.

Tuy nhiên, khi một cơ sở dữ liệu được phân tán trên nhiều máy, có khả năng nằm ở nhiều datacenter, việc tạo ra một transaction ID toàn cục, tăng đơn điệu (trên tất cả các shard) là rất khó khăn, vì nó đòi hỏi sự điều phối. Transaction ID phải phản ánh tính nhân quả (causality): nếu transaction B đọc hoặc ghi đè một giá trị đã được ghi trước đó bởi transaction A, thì B phải có transaction ID cao hơn A—nếu không, snapshot sẽ không nhất quán. Với rất nhiều transaction nhỏ và nhanh, việc tạo transaction ID trong một hệ thống phân tán sẽ trở thành một nút thắt cổ chai không thể tránh khỏi. (Chúng ta sẽ thảo luận về các bộ tạo ID như vậy trong mục “ID Generators và Logical Clocks”).

Liệu chúng ta có thể sử dụng các timestamp từ các đồng hồ thời gian thực (time-of-day clocks) đã được đồng bộ hóa để làm transaction ID không? Nếu chúng ta có thể đạt được mức độ đồng bộ hóa đủ tốt, chúng sẽ sở hữu các thuộc tính phù hợp vì các transaction sau sẽ có timestamp cao hơn. Vấn đề nằm ở sự bất định về độ chính xác của đồng hồ.

Spanner triển khai snapshot isolation giữa các datacenter theo cách này [68, 69]. Nó sử dụng khoảng tin cậy (confidence interval) của đồng hồ như được báo cáo bởi TrueTime API và dựa trên quan sát sau: nếu bạn có hai khoảng tin cậy, mỗi khoảng bao gồm một timestamp sớm nhất và muộn nhất có thể ($A = [A_{earliest}, A_{latest}]$ và $B = [B_{earliest}, B_{latest}]$), và hai khoảng đó không chồng lấp lên nhau (tức là, $A_{earliest} < A_{latest} < B_{earliest} < B_{latest}$), thì B chắc chắn đã xảy ra sau A—không thể có sự nghi ngờ nào. Chỉ khi các khoảng này chồng lấp lên nhau, chúng ta mới không chắc chắn về thứ tự xảy ra của A và B.

Để đảm bảo rằng các timestamp của transaction phản ánh tính nhân quả (causality), Spanner chủ động chờ đợi trong khoảng thời gian bằng độ dài của khoảng tin cậy trước khi commit một transaction read/write. Bằng cách này, nó đảm bảo rằng bất kỳ transaction nào có thể đọc dữ liệu đều diễn ra tại một thời điểm đủ muộn để các khoảng tin cậy của chúng không chồng lấp lên nhau. Để giữ thời gian chờ ngắn nhất

có thể, Spanner cần giữ cho sự bất định của đồng hồ ở mức nhỏ nhất có thể; vì mục đích này, Google triển khai một bộ thu tín hiệu GPS hoặc đồng hồ nguyên tử tại mỗi datacenter, cho phép các đồng hồ được đồng bộ hóa trong phạm vi khoảng 7 ms [45].

Các đồng hồ nguyên tử và bộ thu tín hiệu GPS không hoàn toàn bắt buộc trong Spanner. Điều quan trọng là phải có một khoảng tin cậy—các nguồn đồng hồ chính xác chỉ giúp giữ cho khoảng đó nhỏ lại. Các hệ thống khác cũng đang bắt đầu áp dụng các phương pháp tương tự—ví dụ, YugabyteDB có thể tận dụng ClockBound khi chạy trên AWS [70], và một số hệ thống khác hiện cũng dựa vào việc đồng bộ hóa đồng hồ ở các mức độ khác nhau [71, 72].

Process Pauses

Hãy xét một ví dụ khác về việc sử dụng đồng hồ nguy hiểm trong một distributed system. Giả sử bạn có một cơ sở dữ liệu với một single leader cho mỗi shard. Chỉ leader mới được phép chấp nhận các lệnh write. Làm thế nào một node biết rằng nó vẫn là leader (rằng nó chưa bị các node khác tuyên bố là đã chết) và rằng nó có thể an toàn chấp nhận các lệnh write?

Một lựa chọn là leader sẽ nhận được một *lease* từ các node khác, điều này tương tự như một lock có thời gian timeout [73]. Chỉ một node có thể giữ lease tại bất kỳ thời điểm nào. Do đó, khi một node nhận được lease, nó biết rằng nó là leader trong một khoảng thời gian nhất định, cho đến khi lease hết hạn. Để duy trì vị trí leader, node đó phải định kỳ gia hạn lease trước khi nó hết hạn. Nếu node đó gặp lỗi, nó sẽ ngừng gia hạn lease, nhờ đó một node khác có thể tiếp quản khi lease hết hạn.

Bạn có thể hình dung vòng lặp xử lý request trông giống như thế này:

```
while (true) {
    request = getIncomingRequest();

    // Ensure that the lease always has at least 10 seconds remaining
    if (lease.expiryTimeMillis - System.currentTimeMillis() < 10000)
    {
        lease = lease.renew();
    }

    if (lease.isValid()) {
        process(request);
    }
}
```

Có vấn đề gì với đoạn code này? Thứ nhất, nó đang dựa vào các đồng hồ đã được đồng bộ hóa: thời gian hết hạn của lease được thiết lập bởi một máy khác (nơi mà thời gian hết hạn có thể được tính toán bằng thời gian hiện tại cộng thêm 30 giây, vì

dụ), và nó đang được so sánh với đồng hồ hệ thống cục bộ. Nếu các đồng hồ lệch nhau quá vài giây, đoạn code sẽ bắt đầu thực hiện những điều kỳ lạ.

Thứ hai, ngay cả khi chúng ta thay đổi protocol để chỉ sử dụng đồng hồ monotonic cục bộ, vẫn còn một vấn đề khác: đoạn code giả định rằng có rất ít thời gian trôi qua giữa thời điểm nó kiểm tra thời gian (`System.currentTimeMillis()`) và thời điểm request được xử lý (`process(request)`). Thông thường đoạn code này chạy rất nhanh, vì vậy bộ đệm 10 giây là quá đủ để đảm bảo rằng lease không bị hết hạn ngay giữa quá trình xử lý một request.

Tuy nhiên, chuyện gì sẽ xảy ra nếu một sự tạm dừng không mong muốn xảy ra trong quá trình thực thi chương trình? Ví dụ, hãy tưởng tượng một thread bị dừng trong 15 giây quanh dòng lệnh gọi `lease.isValid` trước khi cuối cùng tiếp tục chạy. Trong trường hợp đó, lease có khả năng đã hết hạn vào thời điểm yêu cầu được xử lý, và một node khác có lẽ đã tiếp quản vị trí leader. Tuy nhiên, không có gì thông báo cho thread này rằng nó đã bị tạm dừng quá lâu, vì vậy mã nguồn sẽ không nhận ra lease đã hết hạn cho đến lần lặp tiếp theo của vòng lặp—mà vào thời điểm đó, nó có thể đã thực hiện một hành động không an toàn bằng cách xử lý yêu cầu.

Liệu có hợp lý khi giả định rằng một thread có thể bị tạm dừng trong một khoảng thời gian dài như vậy không? Thật không may, câu trả lời là có. Có nhiều lý do khác nhau dẫn đến điều này:

- Sự tranh chấp (contention) giữa các thread đang truy cập vào một tài nguyên dùng chung, chẳng hạn như một lock hoặc queue, có thể khiến các thread phải dành rất nhiều thời gian để chờ đợi. Những vấn đề như vậy thường tồi tệ hơn trên các máy có nhiều CPU core hơn, và các vấn đề tranh chấp có thể rất khó để chẩn đoán [74].
- Nhiều runtime của ngôn ngữ lập trình (chẳng hạn như JVM) có một garbage collector đôi khi cần phải dừng tất cả các thread đang chạy. Trong quá khứ, những lần tạm dừng “*stop-the-world*” *garbage collection* như vậy đôi khi kéo dài tới vài phút [75]! Với các thuật toán GC hiện đại, vấn đề này đã ít nghiêm trọng hơn, nhưng các lần tạm dừng GC vẫn có thể nhận thấy được (xem mục “Limiting the impact of garbage collection”).
- Trong các môi trường ảo hóa, một VM có thể bị *suspended* (tạm dừng việc thực thi tất cả các process và lưu nội dung của bộ nhớ vào đĩa) và được *resumed* (khôi phục nội dung bộ nhớ và tiếp tục thực thi). Sự tạm dừng này có thể xảy ra bất cứ lúc nào trong quá trình thực thi của một process và có thể kéo dài trong một khoảng thời gian tùy ý. Tính năng này đôi khi được sử dụng để *live migration* các VM từ host này sang host khác mà không cần reboot, trong trường hợp đó độ dài của sự tạm dừng phụ thuộc vào tốc độ mà các process đang ghi vào bộ nhớ [76].

- Trên các thiết bị của người dùng cuối như laptop và điện thoại, việc thực thi cũng có thể bị suspend và resume một cách tùy ý (ví dụ, khi người dùng đóng nắp laptop của họ).
- Khi hệ điều hành thực hiện context-switch sang một thread khác, hoặc khi hypervisor chuyển sang một VM khác (khi đang chạy trong một VM), thread đang chạy hiện tại có thể bị tạm dừng tại bất kỳ điểm tùy ý nào trong mã nguồn. Trong trường hợp của một VM, thời gian CPU dành cho các VM khác được gọi là *steal time*. Nếu máy đang chịu tải nặng—tức là nếu có một hàng dài các thread đang chờ để chạy—có thể sẽ mất một khoảng thời gian trước khi thread bị tạm dừng có thể chạy lại.
- Nếu ứng dụng thực hiện truy cập đĩa đồng bộ, một thread có thể bị tạm dừng trong khi chờ một thao tác disk I/O chậm hoàn tất [77]. Trong nhiều ngôn ngữ, việc truy cập đĩa có thể xảy ra một cách đáng ngạc nhiên, ngay cả khi mã nguồn không đề cập rõ ràng đến việc truy cập tệp—ví dụ, classloader của Java tải các tệp class một cách lười biếng (lazily) khi chúng được sử dụng lần đầu, điều này có thể xảy ra bất cứ lúc nào trong quá trình thực thi chương trình. Các lần tạm dừng I/O và tạm dừng GC thậm chí có thể cùng nhau cộng hưởng để làm tăng độ trễ [78]. Nếu đĩa thực chất là một network filesystem hoặc network block device (chẳng hạn như Amazon EBS), latency của I/O còn chịu ảnh hưởng bởi sự biến động của độ trễ mạng [31].
- Nếu hệ điều hành được cấu hình để cho phép *swapping to disk (paging)*, một thao tác truy cập bộ nhớ đơn giản có thể dẫn đến một page fault, yêu cầu một page từ đĩa phải được nạp vào bộ nhớ. Thread sẽ bị tạm dừng trong khi thao tác I/O chậm này diễn ra. Nếu áp lực bộ nhớ (memory pressure) cao, điều này có thể dẫn đến việc một page khác phải được swap ra đĩa. Trong những tình huống cực đoan, hệ điều hành có thể dành phần lớn thời gian để swap các page ra vào bộ nhớ và thực hiện được rất ít công việc thực tế (điều này được gọi là *thrashing*). Để tránh vấn đề này, paging thường bị vô hiệu hóa trên các máy chủ (nếu bạn thả kill một process để giải phóng bộ nhớ còn hơn là mạo hiểm gặp phải thrashing).
- Một Unix process có thể bị tạm dừng bằng cách gửi cho nó tín hiệu SIGSTOP—ví dụ, bằng cách nhấn Ctrl-Z trong một shell. Tín hiệu này ngay lập tức ngăn process nhận thêm bất kỳ chu kỳ CPU nào cho đến khi nó được tiếp tục với SIGCONT, tại thời điểm đó nó sẽ tiếp tục chạy từ nơi nó vừa tạm dừng. Ngay cả khi môi trường của bạn thường không sử dụng SIGSTOP, nó vẫn có thể bị gửi đi một cách vô ý bởi một kỹ sư vận hành.

Tất cả những trường hợp này có thể *preempt* (ngắt) luồng đang chạy tại bất kỳ thời điểm nào và tiếp tục nó vào một thời điểm muộn hơn mà luồng đó thậm chí không hề nhận ra. Vấn đề này tương tự như việc làm cho mã nguồn đa luồng (multithreaded) trên một máy đơn lẻ trở nên thread-safe; bạn không thể giả định bất

cứ điều gì về thời điểm, bởi vì các lần context switch tùy ý và tính song song có thể xảy ra.

Khi viết mã nguồn đa luồng trên một máy đơn lẻ, chúng ta có các công cụ khá tốt để làm cho nó thread-safe: mutexes, semaphores, atomic counters, lock-free data structures, blocking queues, v.v. Thật không may, những công cụ này không thể chuyển đổi trực tiếp sang các distributed systems, bởi vì một distributed system không có shared memory—chỉ có các message được gửi qua một mạng không đáng tin cậy.

Một node trong một distributed system phải giả định rằng việc thực thi của nó có thể bị tạm dừng trong một khoảng thời gian đáng kể tại bất kỳ thời điểm nào, ngay cả khi đang ở giữa một hàm. Trong quá trình tạm dừng, phần còn lại của thế giới vẫn tiếp tục vận động và thậm chí có thể tuyên bố node đang tạm dừng đã chết vì nó không phản hồi. Cuối cùng, node đang tạm dừng có thể tiếp tục chạy mà thậm chí không nhận ra rằng nó đã ngủ cho đến khi nó kiểm tra đồng hồ của mình vào một lúc nào đó sau đó.

Cung cấp các đảm bảo về thời gian phản hồi (response time)

Như chúng ta vừa thảo luận, trong nhiều ngôn ngữ lập trình và hệ điều hành, các thread và process có thể tạm dừng trong một khoảng thời gian không giới hạn. Tuy nhiên, những lý do gây tạm dừng đó *có thể* bị loại bỏ nếu bạn nỗ lực đủ nhiều.

Một số phần mềm chạy trong các môi trường mà việc không phản hồi trong một khoảng thời gian quy định có thể gây ra thiệt hại nghiêm trọng. Ví dụ, các máy tính điều khiển chuyển động của máy bay, tên lửa, robot, ô tô và các vật thể vật lý khác phải phản hồi nhanh chóng và có thể dự đoán được đối với các đầu vào từ sensor của chúng. Trong các hệ thống được gọi là *real-time* (thời gian thực) nghiêm ngặt này, phần mềm phải phản hồi trước một thời hạn (deadline) quy định; việc không đáp ứng được thời hạn có thể gây ra sự thất bại của toàn bộ hệ thống.

LƯU Ý

Trong các hệ thống nhúng (embedded systems), *real-time* có nghĩa là một hệ thống được thiết kế và kiểm thử cẩn thận để đáp ứng các đảm bảo về thời gian quy định trong mọi tình huống. Ý nghĩa này trái ngược với cách sử dụng thuật ngữ *real-time* mơ hồ hơn trên web, nơi nó mô tả các máy chủ đẩy dữ liệu đến các client và stream processing mà không có các ràng buộc nghiêm ngặt về thời gian phản hồi (xem Chương 12).

Ví dụ, nếu các sensor trên xe của bạn phát hiện rằng bạn đang gặp phải một vụ va chạm, bạn sẽ không muốn việc bung túi khí bị trì hoãn chỉ vì một đợt GC pause không thuận lợi trong hệ thống kích hoạt túi khí.

Việc cung cấp các đảm bảo real-time trong một hệ thống đòi hỏi sự hỗ trợ từ tất cả các tầng của software stack. Cần có một *hệ điều hành thời gian thực* (RTOS) cho

phép các process được lập lịch với sự phân bổ thời gian CPU được đảm bảo trong các khoảng thời gian quy định; các hàm thư viện phải tài liệu hóa thời gian thực thi trong trường hợp xấu nhất của chúng; việc cấp phát bộ nhớ động có thể bị hạn chế hoặc bị cấm hoàn toàn (các garbage collector thời gian thực có tồn tại, nhưng ứng dụng vẫn phải đảm bảo rằng nó không giao quá nhiều việc cho garbage collector); và một lượng lớn việc kiểm thử cũng như đo lường phải được thực hiện để đảm bảo rằng các đảm bảo đang được đáp ứng.

Tất cả những điều này đòi hỏi một lượng lớn công việc bổ sung và hạn chế nghiêm trọng phạm vi các ngôn ngữ lập trình, thư viện và công cụ có thể được sử dụng (vì hầu hết các ngôn ngữ và công cụ không cung cấp các đảm bảo thời gian thực). Vì những lý do này, việc phát triển các hệ thống thời gian thực rất tốn kém, và chúng thường được sử dụng nhất trong các thiết bị nhúng quan trọng về an toàn (safety-critical). Ngoài ra, "thời gian thực" không giống với "hiệu năng cao"—trên thực tế, các hệ thống thời gian thực có thể có throughput thấp hơn, vì chúng phải ưu tiên các phản hồi kịp thời lên trên hết (xem mục "Latency và Resource Utilization").

Đối với hầu hết các hệ thống xử lý dữ liệu phía server, các đảm bảo thời gian thực đơn giản là không kinh tế hoặc không phù hợp. Do đó, các hệ thống này phải chịu đựng các khoảng tạm dừng và sự mất ổn định của đồng hồ do hoạt động trong một môi trường không phải thời gian thực.

Hạn chế tác động của garbage collection

Garbage collection từng là một trong những lý do lớn nhất gây ra các khoảng tạm dừng tiến trình [79], nhưng may mắn thay các thuật toán GC đã cải thiện rất nhiều. Một bộ thu gom (collector) được tinh chỉnh đúng cách giờ đây thường sẽ chỉ tạm dừng các tiến trình trong không quá vài mili giây. Java runtime cung cấp các collector như concurrent mark sweep (CMS), garbage-first (G1), Z garbage collector (ZGC), Epsilon, và Shenandoah. Mỗi loại được tối ưu hóa cho các profile bộ nhớ khác nhau, chẳng hạn như tạo đối tượng với tần suất cao, heap lớn, v.v. Ngược lại, Go cung cấp một garbage collector kiểu concurrent mark and sweep đơn giản hơn, cố gắng tự tối ưu hóa chính nó.

Nếu bạn cần tránh hoàn toàn các khoảng tạm dừng GC, một lựa chọn là sử dụng một ngôn ngữ không có garbage collector. Ví dụ, Swift sử dụng automatic reference counting để xác định khi nào bộ nhớ có thể được giải phóng, trong khi Rust và Mojo theo dõi vòng đời đối tượng thông qua type system để compiler có thể xác định bộ nhớ phải được cấp phát trong bao lâu.

Cũng có thể sử dụng một ngôn ngữ có garbage collection trong khi giảm thiểu tác động của các khoảng tạm dừng. Ví dụ, các đối tượng có thể được lưu trữ và tái sử dụng trong các pool thay vì bị loại bỏ, hoặc dữ liệu có thể được cấp phát off-heap. Một cách tiếp cận cực đoan hơn là coi các khoảng tạm dừng GC như những đợt ngắt quãng ngắn đã được lên kế hoạch của một node và để các node khác xử lý các yêu

cầu từ client trong khi một node đang thu gom rác của nó. Nếu runtime có thể cảnh báo ứng dụng rằng một node sắp cần một khoảng tạm dừng GC, ứng dụng có thể ngừng gửi các yêu cầu mới đến node đó, đợi nó hoàn tất việc xử lý các yêu cầu đang tồn đọng, và sau đó thực hiện GC khi không có yêu cầu nào đang diễn ra. Thủ thuật này che giấu các khoảng tạm dừng GC khỏi các client và giảm các percentile cao của *response time* [80, 81].

Một biến thể của ý tưởng này là chỉ sử dụng garbage collector cho các đối tượng có vòng đời ngắn (vốn được thu gom nhanh) và khởi động lại các tiến trình định kỳ, trước khi chúng tích lũy đủ các đối tượng có vòng đời dài đến mức yêu cầu một đợt full GC cho các đối tượng đó [79, 82]. Mỗi lần có thể khởi động lại một node, và lưu lượng có thể được chuyển hướng khỏi node đó trước khi đợt khởi động lại đã lên kế hoạch diễn ra, giống như trong một rolling upgrade (xem Chương 5).

Các biện pháp này không thể ngăn chặn hoàn toàn các khoảng tạm dừng GC, nhưng chúng có thể giảm thiểu tác động của chúng lên ứng dụng một cách hữu ích.

Kiến thức, Sự thật và Những lời nói dối

Cho đến nay trong chương này, chúng ta đã khám phá những cách mà các hệ thống phân tán khác biệt so với các chương trình chạy trên một máy tính duy nhất. Các hệ thống phân tán không có bộ nhớ dùng chung, chỉ có message passing thông qua một mạng không đáng tin cậy với độ trễ biến đổi, và các hệ thống có thể gặp phải các lỗi cục bộ (partial failures), đồng hồ không đáng tin cậy và các khoảng tạm dừng xử lý.

Hệ quả của những vấn đề này là cực kỳ gây mất phương hướng nếu bạn không quen với các hệ thống phân tán. Một node trong mạng không thể *biết* bất cứ điều gì một cách chắc chắn về các node khác—nó chỉ có thể đưa ra các dự đoán dựa trên các message mà nó nhận được (hoặc không nhận được). Một node chỉ có thể tìm hiểu trạng thái của một node khác (nó đã lưu trữ dữ liệu gì, nó có đang hoạt động chính xác hay không, v.v.) bằng cách trao đổi message với nó. Nếu một node từ xa không phản hồi, sẽ không có cách nào để biết trạng thái của nó, bởi vì các vấn đề trong mạng không thể được phân biệt một cách đáng tin cậy với các vấn đề tại một node.

Các cuộc thảo luận về những hệ thống này tiệm cận với triết học: Chúng ta biết điều gì là đúng hay sai trong hệ thống của mình? Chúng ta có thể chắc chắn đến mức nào về kiến thức đó, nếu các cơ chế nhận thức và đo lường không đáng tin cậy [83]? Liệu các hệ thống phần mềm có nên tuân theo các quy luật mà chúng ta kỳ vọng ở thế giới vật lý, chẳng hạn như quan hệ nhân quả?

May mắn thay, chúng ta không cần phải đi xa đến mức tìm hiểu ý nghĩa của cuộc sống. Trong một distributed system, chúng ta có thể đưa ra các giả định về hành vi (gọi là *system model*) và thiết kế hệ thống thực tế sao cho đáp ứng được các giả định đó. Các thuật toán có thể được chứng minh là hoạt động chính xác trong một *system*

model nhất định. Điều này có nghĩa là hành vi đáng tin cậy là khả thi, ngay cả khi *system model* nền tảng cung cấp rất ít sự đảm bảo.

Tuy nhiên, mặc dù có thể làm cho phần mềm hoạt động tốt trong một *system model* không đáng tin cậy, nhưng việc thực hiện điều đó không hề đơn giản. Trong phần còn lại của chương này, chúng ta sẽ khám phá sâu hơn các khái niệm về kiến thức và sự thật trong distributed systems, điều này sẽ giúp chúng ta suy nghĩ về các loại giả định mà chúng ta có thể đưa ra và các đảm bảo mà chúng ta có thể muốn cung cấp. Trong Chương 10, chúng ta sẽ tiếp tục xem xét một số ví dụ về các thuật toán phân tán cung cấp các đảm bảo cụ thể dưới các giả định cụ thể.

Đa số quyết định

Hãy tưởng tượng một mạng có một fault bất đối xứng: một node có thể nhận được tất cả các message được gửi đến nó, nhưng bất kỳ message gửi đi nào từ node đó đều bị rơi mất hoặc bị trì hoãn [22]. Ngay cả khi node đó đang hoạt động hoàn hảo và đang nhận các request từ các node khác, các node khác vẫn không thể nghe thấy phản hồi của nó. Sau khi hết thời gian timeout, các node khác tuyên bố nó đã chết, vì chúng không nhận được tin tức gì từ node đó. Tình huống diễn ra như một cơn ác mộng—node bị mất kết nối một phần bị kéo vào nghĩa trang, vừa vùng vẫy vừa gào thét "Tôi chưa chết!"—nhưng vì không ai có thể nghe thấy tiếng gào thét của nó, đoàn đưa tang vẫn tiếp tục với sự quyết tâm kiên định.

Trong một kịch bản ít ác mộng hơn một chút, node bị mất kết nối một phần có thể nhận thấy rằng các message mà nó đang gửi đi không được các node khác xác nhận và nhận ra rằng chắc chắn phải có một fault trong mạng. Tuy nhiên, node đó vẫn bị các node khác tuyên bố là đã chết một cách sai lầm, và nó không thể làm gì được.

Với kịch bản thứ ba, hãy tưởng tượng một node tạm dừng thực thi trong một phút. Trong thời gian đó, không có request nào được xử lý và không có phản hồi nào được gửi đi. Các node khác chờ đợi, thử lại, trở nên mất kiên nhẫn, và cuối cùng tuyên bố node đó đã chết rồi đưa nó lên xe tang. Cuối cùng, thời gian tạm dừng kết thúc, và các thread của node tiếp tục hoạt động như thể chưa có chuyện gì xảy ra. Các node khác ngạc nhiên khi node vốn được cho là đã chết đột nhiên ngẩng đầu lên khỏi quan tài, hoàn toàn khỏe mạnh, và bắt đầu trò chuyện vui vẻ với những người đứng xem. Lúc đầu, node bị tạm dừng thậm chí còn không nhận ra rằng cả một phút đã trôi qua và nó đã bị tuyên bố là đã chết—từ góc nhìn của nó, hầu như chưa có thời gian nào trôi qua kể từ lần cuối nó nói chuyện với các node khác.

Bài học rút ra từ những câu chuyện này là một node không nhất thiết phải tin vào phán đoán của chính nó về một tình huống. Một distributed system không thể chỉ dựa duy nhất vào một node đơn lẻ, vì một node có thể gặp lỗi bất cứ lúc nào, có khả năng khiến hệ thống bị kẹt và không thể phục hồi. Thay vào đó, nhiều thuật toán phân tán dựa vào một *quorum* (tức là bỏ phiếu giữa các node; xem mục "Sử dụng

quorums để đọc và ghi”): các quyết định yêu cầu một số lượng phiếu bầu tối thiểu từ nhiều node nhằm giảm sự phụ thuộc vào bất kỳ một node cụ thể nào.

Điều đó bao gồm cả các quyết định về việc tuyên bố các node đã chết. Nếu một quorum các node tuyên bố một node khác đã chết, thì nó phải được coi là đã chết, ngay cả khi node đó vẫn cảm thấy mình còn rất sống khỏe. Node riêng lẻ đó phải tuân theo quyết định của quorum và rút lui.

Thông thường nhất, quorum là đa số tuyệt đối, tức là nhiều hơn một nửa số node (mặc dù các loại quorum khác vẫn có thể tồn tại). Một majority quorum cho phép hệ thống tiếp tục hoạt động nếu một thiểu số các node bị lỗi (với ba node, ta có thể chịu được một node lỗi; với năm node, ta có thể chịu được hai node lỗi). Nó cũng đảm bảo tính an toàn, bởi vì trong hệ thống chỉ có thể có duy nhất một đa số—không thể có hai đa số cùng đưa ra các quyết định xung đột tại cùng một thời điểm. Chúng ta sẽ thảo luận chi tiết hơn về việc sử dụng quorum khi tìm hiểu về các thuật toán consensus trong Chương 10.

Distributed Locks và Leases

Locks và leases trong các ứng dụng phân tán rất dễ bị sử dụng sai cách và là nguồn gốc phổ biến của các lỗi [84]. Hãy cùng xem xét một trường hợp cụ thể về việc chúng có thể dẫn đến sai sót như thế nào.

Trong phần “Process Pauses”, chúng ta đã thấy rằng một lease là một loại lock có thời gian hết hạn (timeout) và có thể được cấp cho một chủ sở hữu mới nếu chủ sở hữu cũ ngừng phản hồi (có lẽ vì nó bị crash, bị tạm dừng quá lâu, hoặc bị ngắt kết nối khỏi mạng). Bạn có thể sử dụng leases khi một hệ thống yêu cầu chỉ có duy nhất một thực thể của một thứ nào đó. Ví dụ:

- Chỉ một node duy nhất được phép làm leader cho một database shard, nhằm tránh split brain (xem mục “Handling Node Outages”).
- Chỉ một transaction hoặc một client duy nhất được phép cập nhật một resource hoặc object cụ thể, để ngăn chặn việc nó bị hỏng do các thao tác ghi đồng thời (concurrent writes).
- Chỉ một node duy nhất nên xử lý một file đầu vào nhất định cho một công việc xử lý lớn, để tránh lãng phí công sức do nhiều node thực hiện cùng một công việc một cách dư thừa.

Cần phải suy nghĩ kỹ về những gì sẽ xảy ra nếu nhiều node đồng thời tin rằng chúng đang nắm giữ lease, có lẽ là do một sự tạm dừng tiến trình (process pause). Trong ví dụ thứ ba, hậu quả chỉ là một số tài nguyên tính toán bị lãng phí, điều này không phải vấn đề lớn. Nhưng trong hai trường hợp đầu tiên, hậu quả có thể là dữ liệu bị mất hoặc bị hỏng, điều này nghiêm trọng hơn nhiều.

Ví dụ, Hình 9-4 cho thấy một lỗi hỏng dữ liệu do việc triển khai locking không chính xác. (Lỗi này không phải là lý thuyết; HBase đã từng gặp vấn đề này [85, 86].) Giả sử bạn muốn đảm bảo rằng một file trong một dịch vụ lưu trữ chỉ có thể được truy cập bởi một client duy nhất tại một thời điểm, vì nếu nhiều client cùng cố gắng ghi vào đó, file sẽ bị hỏng. Bạn cố gắng thực hiện điều này bằng cách yêu cầu một client phải lấy được một lease từ một lock service trước khi truy cập file. Một lock service như vậy thường được triển khai bằng cách sử dụng một thuật toán consensus, sẽ được thảo luận thêm trong Chương 10.

Vấn đề này là một ví dụ về những gì chúng ta đã thảo luận trong phần “Process Pauses”. Nếu client đang nắm giữ lease bị tạm dừng quá lâu, lease của nó sẽ hết hạn. Một client khác sau đó có thể lấy được lease cho cùng một file đó và bắt đầu ghi vào file. Khi client bị tạm dừng quay trở lại, nó tin rằng (một cách sai lầm) rằng nó vẫn còn một lease hợp lệ và tiếp tục ghi vào file. Giờ đây chúng ta có một tình huống split-brain: các thao tác ghi của các client xung đột và làm hỏng file.

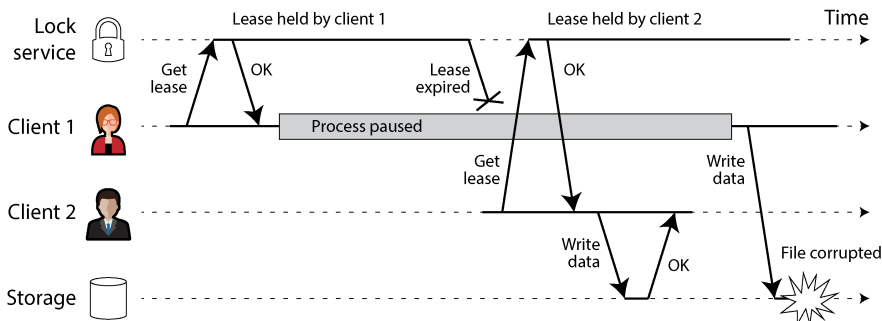


Figure 9-4. Một cách triển khai **distributed lock** không chính xác: client 1 tin rằng nó vẫn còn một lease hợp lệ, mặc dù nó đã hết hạn, và do đó làm hỏng một tệp trong storage

Hình 9-5 cho thấy một vấn đề khác có những hệ quả tương tự. Trong ví dụ này không có sự tạm dừng tiến trình, mà chỉ có một lỗi crash từ client 1. Ngay trước khi client 1 bị crash, nó gửi một yêu cầu write tới storage service, nhưng yêu cầu này bị trì hoãn trong mạng một khoảng thời gian dài. (Hãy nhớ lại từ mục “Network Faults in Practice” rằng các packet đôi khi có thể bị trì hoãn tới một phút hoặc lâu hơn.) Vào thời điểm yêu cầu write đến được storage service, lease của client 1 đã hết hạn, cho phép client 2 giành được nó và thực hiện lệnh write của riêng mình. Kết quả là sự hư hỏng dữ liệu tương tự như trong Hình 9-4.

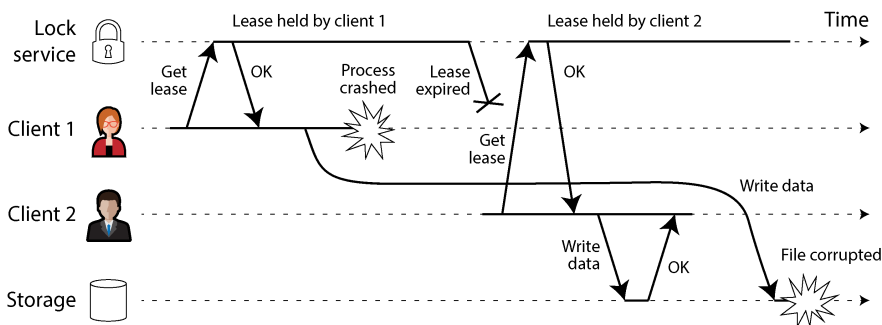


Figure 9-5. Một thông điệp từ một leaseholder cũ có thể bị trì hoãn trong một thời gian dài và đến sau khi một node khác đã tiếp quản lease đó.

Fencing off zombie và các yêu cầu bị trễ

Thuật ngữ *zombie* đôi khi được dùng để mô tả một đối tượng từng nắm giữ lease nhưng vẫn chưa biết rằng mình đã mất lease và vẫn đang hành động như thể nó là người nắm giữ lease hiện tại. Vì chúng ta không thể loại bỏ hoàn toàn các zombie, thay vào đó chúng ta phải đảm bảo rằng chúng không thể gây ra bất kỳ thiệt hại nào dưới dạng split brain. Điều này được gọi là *fencing off* (ngăn chặn) zombie.

Một số hệ thống cố gắng fencing off các zombie bằng cách tắt chúng đi—ví dụ, bằng cách ngắt kết nối chúng khỏi mạng [9], tắt VM thông qua giao diện quản lý của nhà cung cấp cloud, hoặc thậm chí ngắt nguồn vật lý của máy [87]. Cách tiếp cận này đôi khi được gọi là *shoot the other node in the head* (STONITH), mặc dù chúng ta không muốn sử dụng thuật ngữ bạo lực như vậy. Trong mọi trường hợp, nó không đặc biệt hiệu quả: cách tiếp cận này không bảo vệ được hệ thống trước các độ trễ mạng lớn như được mô tả trong Hình 9-5; tất cả các node có thể tắt lẫn nhau [19]; và vào thời điểm một zombie bị phát hiện và tắt đi, có thể đã quá muộn và dữ liệu có thể đã bị hỏng.

Một giải pháp fencing mạnh mẽ hơn, giúp bảo vệ chống lại cả zombie và các yêu cầu bị trễ, được minh họa trong Hình 9-6.

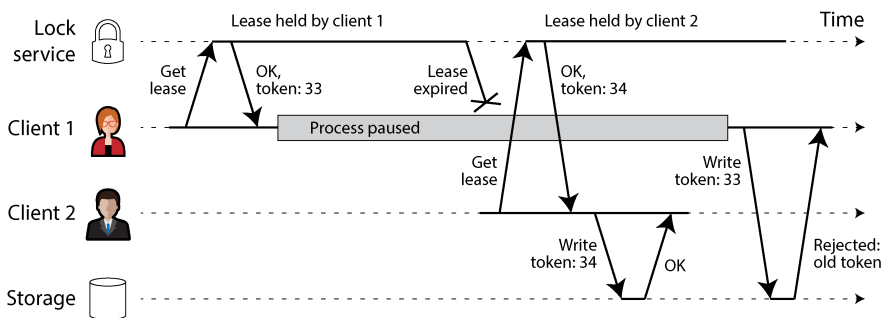


Figure 9-6. Giúp việc truy cập vào storage trở nên an toàn bằng cách chỉ cho phép các thao tác ghi thực hiện theo thứ tự tăng dần của các fencing token.

Hãy giả định rằng mỗi khi dịch vụ lock cấp một lock hoặc lease, nó cũng trả về một *fencing token* (token chốt chặn), là một con số tăng lên mỗi khi một lock được cấp (ví dụ: được tăng bởi dịch vụ lock). Khi đó, chúng ta có thể yêu cầu rằng mỗi khi một client gửi một write request tới dịch vụ storage, nó phải bao gồm fencing token hiện tại của mình.

LƯU Ý

Có một vài tên gọi thay thế cho fencing token. Trong Chubby, dịch vụ lock của Google, chúng được gọi là *sequencers* [88], và trong Kafka chúng được gọi là *epoch numbers*. Trong các thuật toán consensus mà chúng ta sẽ thảo luận trong Chương 10, *ballot number* (Paxos) hoặc *term number* (Raft) cũng phục vụ mục đích tương tự.

Trong Hình 9-6, client 1 giành được lease với token là 33, nhưng sau đó nó rơi vào một khoảng tạm dừng dài và lease hết hạn. Client 2 giành được lease với token là 34 (con số này luôn tăng lên) và gửi write request của nó tới dịch vụ storage, bao gồm cả token. Sau đó, client 1 hoạt động trở lại và gửi lệnh write của nó tới dịch vụ storage, bao gồm giá trị token là 33. Tuy nhiên, dịch vụ storage nhớ rằng nó đã xử lý một lệnh write với số token cao hơn (34), vì vậy nó từ chối yêu cầu với token 33. Một client vừa giành được lease phải thực hiện ngay một lệnh write tới dịch vụ storage, và một khi lệnh write đó hoàn tất, bất kỳ zombie nào cũng sẽ bị fenced off. Các thao tác này tương tự như kỹ thuật optimistic concurrency control (OCC) mà chúng ta đã thấy trong mục “Pessimistic versus optimistic concurrency control”, ngoại trừ việc fencing là vĩnh viễn, trong khi các lỗi concurrency control có thể được thử lại.

Nếu ZooKeeper là dịch vụ lock của bạn, bạn có thể sử dụng transaction ID `zxid` hoặc node version `cversion` làm fencing token [85]. Với etcd, revision number cùng với lease ID phục vụ mục đích tương tự [89]. FencedLock API trong Hazelcast thực hiện tạo fencing token một cách rõ ràng [90].

Cơ chế này yêu cầu dịch vụ storage phải có cách nào đó để kiểm tra xem một lệnh write có dựa trên một token đã lỗi thời hay không. Ngoài ra, chỉ cần dịch vụ hỗ trợ

một lệnh write mà chỉ thành công nếu object đó chưa bị ghi bởi một client khác kể từ lần cuối client hiện tại đọc nó, tương tự như một thao tác atomic CAS. Ví dụ, các dịch vụ object storage đều hỗ trợ kiểu kiểm tra này; Amazon S3 gọi nó là *conditional writes*, Azure Blob Storage gọi là *conditional headers*, và Google Cloud Storage gọi là *request preconditions*.

Fencing với nhiều replica

Nếu các client của bạn chỉ cần ghi vào duy nhất một dịch vụ storage có hỗ trợ conditional writes, thì dịch vụ lock trở nên hơi dư thừa [91, 92], vì việc gán lease có thể đã được triển khai trực tiếp dựa trên chính dịch vụ storage đó [93]. Tuy nhiên, một khi bạn đã có fencing token, bạn có thể sử dụng nó với nhiều dịch vụ hoặc replica và đảm bảo rằng chủ sở hữu lease cũ bị fenced off trên tất cả chúng.

Ví dụ, hãy tưởng tượng dịch vụ storage là một leaderless replicated key-value store với cơ chế giải quyết xung đột LWW (xem “Leaderless Replication”). Trong một hệ thống như vậy, client gửi các lệnh write trực tiếp tới từng replica, và mỗi replica sẽ tự quyết định có chấp nhận lệnh write hay không dựa trên một timestamp được gán bởi client.

Như được minh họa trong Hình 9-7, bạn có thể đặt fencing token của người viết vào các bit hoặc chữ số có ý nghĩa nhất của timestamp. Khi đó, bạn có thể chắc chắn rằng bất kỳ timestamp nào được tạo ra bởi chủ sở hữu lease mới sẽ lớn hơn bất kỳ timestamp nào từ chủ sở hữu lease cũ, ngay cả khi các lệnh write của chủ sở hữu lease cũ xảy ra muộn hơn.

Trong Hình 9-7, client 2 có một fencing token là 34, vì vậy tất cả các timestamp của nó bắt đầu bằng 34... đều lớn hơn bất kỳ timestamp nào bắt đầu bằng 33... được tạo ra bởi client 1. Client 2 thực hiện ghi vào một quorum của các replica, nhưng nó không thể kết nối tới replica 3. Điều này có nghĩa là khi zombie client 1 cố gắng thực hiện ghi sau đó, lệnh ghi của nó có thể thành công tại replica 3 ngay cả khi nó bị các replica 1 và 2 bỏ qua. Đây không phải là một vấn đề, vì một thao tác quorum read tiếp theo sẽ ưu tiên lệnh ghi từ client 2 với timestamp lớn hơn, và read repair hoặc anti-entropy cuối cùng sẽ ghi đè giá trị đã được ghi bởi client 1.

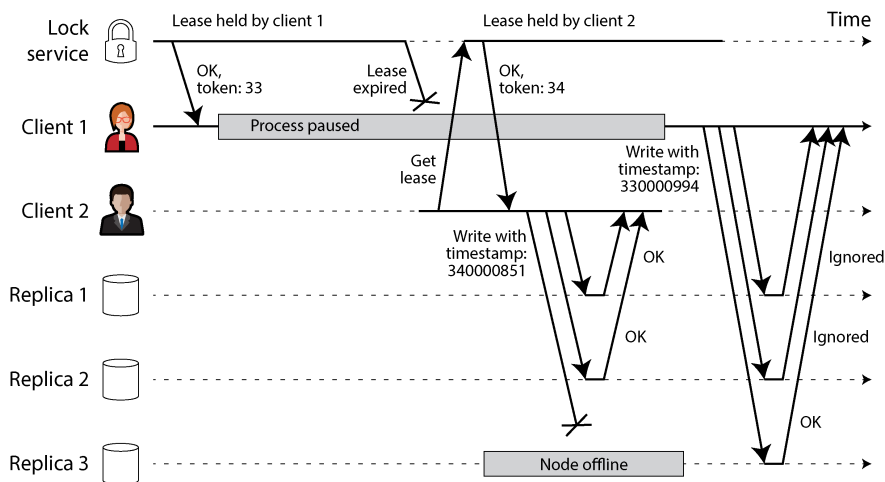


Figure 9-7. Hình 9.x: Sử dụng fencing token để bảo vệ các thao tác ghi vào một cơ sở dữ liệu leaderless replication

Như bạn có thể thấy từ các ví dụ này, việc giả định rằng chỉ có một node đang giữ lease tại bất kỳ thời điểm nào là không an toàn. May mắn thay, với một chút cẩn trọng, bạn có thể sử dụng fencing token để ngăn chặn các zombie và các yêu cầu bị trễ gây ra bất kỳ thiệt hại nào.

Byzantine Faults

Fencing token có thể phát hiện và chặn một node đang *vô tình* hành động sai sót (ví dụ: vì nó vẫn chưa biết rằng lease của mình đã hết hạn). Tuy nhiên, nếu node đó cố tình muốn phá hoại các đảm bảo của hệ thống, nó có thể dễ dàng thực hiện điều đó bằng cách gửi các thông điệp với một fencing token giả.

Trong cuốn sách này, chúng ta giả định rằng các node là không tin cậy nhưng trung thực. Chúng có thể chậm hoặc không bao giờ phản hồi (do một lỗi), và trạng thái của chúng có thể bị lỗi thời (do một đợt GC pause hoặc trễ mạng), nhưng chúng ta giả định rằng nếu một node *có* phản hồi, nó đang nói "sự thật". Trong phạm vi hiểu biết tốt nhất của mình, nó đang tuân thủ các quy tắc của protocol.

Các vấn đề về hệ thống phân tán trở nên khó khăn hơn nhiều nếu có rủi ro rằng các node có thể "nói dối" (gửi các phản hồi lỗi hoặc bị hỏng tùy ý)—ví dụ, một node có thể đưa ra nhiều phiếu bầu mâu thuẫn trong cùng một cuộc bầu chọn. Hành vi như vậy được gọi là *Byzantine fault*, và vấn đề đạt được consensus trong môi trường thiếu tin cậy này được gọi là *Byzantine Generals Problem* [94].

Bài toán các vị tướng Byzantine

Byzantine Generals Problem là một dạng tổng quát hóa của *two generals problem* [95], mô tả hai vị tướng quân cần thống nhất về một kế hoạch tác chiến. Vì họ đã đóng quân tại các địa điểm khác nhau, họ chỉ có thể liên lạc thông qua người đưa tin, và những người đưa tin đôi khi bị trễ hoặc bị thất lạc (giống như các packet trong một mạng). Chúng ta sẽ thảo luận về vấn đề *consensus* này trong Chương 10.

Trong phiên bản Byzantine của vấn đề này, n vị tướng cần phải thống nhất, và nỗ lực của họ bị cản trở bởi những kẻ phản bội ở giữa họ. Hầu hết các vị tướng đều trung thành và do đó gửi các thông điệp trung thực, nhưng những kẻ phản bội có thể cố gắng lừa dối và làm rối loạn những người khác bằng cách gửi các thông điệp giả hoặc không đúng sự thật. Không thể biết trước ai là những kẻ phản bội.

Byzantium là một thành phố Hy Lạp cổ đại mà sau này trở thành Constantinople, tại nơi mà hiện nay là Istanbul ở Thổ Nhĩ Kỳ. Không có bất kỳ bằng chứng lịch sử nào cho thấy các vị tướng của Byzantium có xu hướng tham gia vào các âm mưu và mưu đồ nhiều hơn so với những nơi khác. Thay vào đó, cái tên này bắt nguồn từ *Byzantine* với nghĩa là *cực kỳ phức tạp, quan liêu, xảo quyệt*, vốn đã được sử dụng trong chính trị từ rất lâu trước khi có máy tính [96]. Lamport muốn chọn một quốc tịch không gây xúc phạm đến bất kỳ độ giả nào, và ông đã được khuyên rằng gọi nó là *The Albanian Generals Problem* không phải là một ý kiến hay [97].

Các ứng dụng của Byzantine fault tolerance

Một hệ thống là *Byzantine fault-tolerant* nếu nó tiếp tục hoạt động chính xác ngay cả khi một số node đang gặp trục trặc và không tuân thủ protocol, hoặc nếu các kẻ tấn công độc hại đang can thiệp vào mạng. Mối quan tâm này có liên quan trong một số trường hợp cụ thể. Ví dụ:

- Trong các môi trường hàng không vũ trụ, dữ liệu trong bộ nhớ máy tính hoặc thanh ghi CPU có thể bị hỏng do bức xạ, dẫn đến việc nó phản hồi các node khác theo những cách không thể dự đoán trước một cách tùy ý. Vì một lỗi hệ thống sẽ rất tốn kém (ví dụ: một máy bay rơi và làm thiệt mạng tất cả mọi người trên khoang, hoặc một tên lửa va chạm với Trạm Vũ trụ Quốc tế), các hệ thống điều khiển bay phải có khả năng chịu được Byzantine faults [98, 99].
- Trong một hệ thống với nhiều bên tham gia, một số bên có thể cố gắng gian lận hoặc lừa đảo những bên khác. Trong những trường hợp như vậy, một node không thể chỉ đơn giản tin tưởng thông điệp của một node khác, vì chúng có thể được gửi với ý đồ độc hại. Các cơ chế consensus làm nền tảng cho các loại tiền

điện tử như Bitcoin và các hệ thống dựa trên blockchain khác có thể được coi là một cách để khiến các bên không tin tưởng lẫn nhau đồng ý về việc liệu một transaction đã xảy ra hay chưa, mà không cần dựa vào một cơ quan trung ương [100].

Tuy nhiên, trong các loại hệ thống mà chúng ta thảo luận trong cuốn sách này, chúng ta thường có thể yên tâm giả định rằng không có Byzantine fault. Trong một datacenter, tất cả các node đều được kiểm soát bởi tổ chức của bạn (vì vậy hy vọng rằng chúng có thể được tin cậy), và mức độ bức xạ đủ thấp để việc hỏng hóc bộ nhớ không phải là một vấn đề lớn (mặc dù các datacenter trên quỹ đạo đang được xem xét [101]). Các hệ thống multitenant có các tenant không tin tưởng lẫn nhau, nhưng chúng được cô lập với nhau thông qua firewall, virtualization và các chính sách access control, chứ không sử dụng Byzantine fault tolerance. Các protocol để làm cho hệ thống có khả năng Byzantine fault-tolerant là rất đắt đỏ [102], và các hệ thống embedded fault-tolerant dựa vào sự hỗ trợ từ cấp độ phần cứng [98]. Trong hầu hết các hệ thống dữ liệu phía server, chi phí để triển khai các giải pháp Byzantine fault-tolerant khiến chúng trở nên không khả thi.

Các ứng dụng web thực sự cần dự phòng các hành vi tùy ý và độc hại từ các client nằm dưới sự kiểm soát của người dùng cuối, chẳng hạn như các trình duyệt web. Đây là lý do tại sao việc input validation, sanitization và output escaping lại quan trọng đến vậy: ví dụ, để ngăn chặn SQL injection và cross-site scripting. Tuy nhiên, chúng ta thường không sử dụng các protocol Byzantine fault-tolerant ở đây, mà chỉ đơn giản là biến server thành cơ quan có thẩm quyền để quyết định hành vi nào của client là được phép và không được phép. Trong các mạng peer-to-peer, nơi không có cơ quan trung ương như vậy, Byzantine fault tolerance trở nên phù hợp hơn [103, 104].

Một bug trong phần mềm có thể được coi là một Byzantine fault, nhưng nếu bạn triển khai cùng một phần mềm cho tất cả các node, thì một thuật toán Byzantine fault-tolerant cũng không thể cứu được bạn. Hầu hết các thuật toán Byzantine fault-tolerant yêu cầu một đa số áp đảo gồm hơn hai phần ba số node phải hoạt động chính xác (ví dụ, nếu bạn có bốn node, tối đa chỉ một node có thể gặp trục trặc). Để sử dụng cách tiếp cận này chống lại các bug, bạn sẽ phải có bốn bản triển khai độc lập của cùng một phần mềm và hy vọng rằng một bug nhất định chỉ xuất hiện trong một trong bốn bản triển khai đó.

Tương tự, sẽ rất hấp dẫn nếu một protocol có thể bảo vệ chúng ta khỏi các lỗ hổng, sự xâm phạm bảo mật và các cuộc tấn công độc hại. Thật không may, điều này cũng không thực tế. Trong hầu hết các hệ thống, nếu một kẻ tấn công có thể xâm phạm một node, chúng có thể xâm phạm tất cả các node khác, bởi vì các node có khả năng đang chạy cùng một phần mềm. Do đó, các cơ chế truyền thống (authentication, access control, encryption, firewall, v.v.) tiếp tục là sự bảo vệ chính chống lại những kẻ tấn công.

Các dạng "nói dối" yếu

Mặc dù chúng ta giả định rằng các node nhìn chung là trung thực, nhưng việc thêm các cơ chế vào phần mềm để phòng chống các dạng "nói dối" yếu cũng rất đáng giá—ví dụ, các message không hợp lệ do vấn đề phần cứng, bug phần mềm và misconfiguration. Các cơ chế bảo vệ như vậy không phải là Byzantine fault tolerance đầy đủ, vì chúng sẽ không chống lại được một kẻ thù quyết tâm, nhưng chúng vẫn là những bước đơn giản và thực tế hướng tới độ tin cậy tốt hơn. Ví dụ:

- Các gói tin mạng đôi khi bị hỏng do các vấn đề phần cứng hoặc bug trong hệ điều hành, driver, router, v.v. Thông thường, các gói tin bị hỏng sẽ bị phát hiện bởi các checksum được tích hợp trong TCP và UDP, nhưng đôi khi chúng lọt qua được sự phát hiện [105, 106, 107]. Các biện pháp đơn giản thường là đủ để bảo vệ chống lại sự hỏng hóc như vậy, chẳng hạn như sử dụng checksum trong protocol ở cấp độ ứng dụng. Các kết nối được mã hóa TLS cũng cung cấp sự bảo vệ chống lại sự hỏng hóc.
- Một ứng dụng có thể truy cập công khai phải thực hiện sanitization cẩn thận đối với bất kỳ input nào từ người dùng—ví dụ, bằng cách escaping các ký tự nhất định để ngăn chặn các cuộc tấn công SQL injection, kiểm tra xem một giá trị có nằm trong phạm vi hợp lý hay không, và giới hạn kích thước của các chuỗi để ngăn chặn tấn công denial of service thông qua việc cấp phát bộ nhớ lớn. Một service nội bộ nằm sau firewall có thể thực hiện các bước kiểm tra input ít nghiêm ngặt hơn, nhưng các bước kiểm tra cơ bản trong các protocol parser vẫn là một ý tưởng hay [105].
- Các NTP client có thể được cấu hình với nhiều địa chỉ server. Khi thực hiện đồng bộ hóa, client sẽ liên hệ với tất cả các server đó, ước tính sai số của chúng và kiểm tra xem đa số các server có đồng ý về một khoảng thời gian hay không. Miễn là hầu hết các server đều ổn, một NTP server bị cấu hình sai đang báo cáo thời gian không chính xác sẽ bị phát hiện là một outlier và bị loại khỏi quá trình đồng bộ hóa [39]. Việc sử dụng nhiều server giúp NTP trở nên mạnh mẽ hơn so với việc chỉ sử dụng một server duy nhất.

System Model và Thực tế

Nhiều thuật toán đã được thiết kế để giải quyết các vấn đề của hệ thống phân tán—ví dụ, chúng ta sẽ xem xét các giải pháp cho vấn đề consensus trong Chương 10. Để hữu ích, các thuật toán này cần phải chịu được các loại fault khác nhau của hệ thống phân tán mà chúng ta đã thảo luận trong chương này.

Các thuật toán phải được viết theo cách không phụ thuộc quá nhiều vào các chi tiết về cấu hình phần cứng và phần mềm mà chúng được chạy trên đó. Điều này đòi hỏi chúng ta phải hình thức hóa bằng cách nào đó các loại fault mà chúng ta dự đoán sẽ

xảy ra trong một hệ thống. Chúng ta thực hiện việc này bằng cách định nghĩa một *system model*, vốn là một sự trừu tượng hóa mô tả các giả định của một thuật toán.

Liên quan đến các giả định về thời gian, có ba system model thường được sử dụng:

Synchronous model

Synchronous model giả định rằng độ trễ mạng có giới hạn, các khoảng tạm dừng của process có giới hạn, và lỗi clock có giới hạn. Điều này không có nghĩa là các clock được đồng bộ hóa chính xác hay độ trễ mạng bằng không; nó chỉ có nghĩa là bạn biết rằng độ trễ mạng, các khoảng tạm dừng và clock drift sẽ không bao giờ vượt quá một ngưỡng trên cố định [108]. Synchronous model không phải là một mô hình thực tế cho hầu hết các hệ thống thực tế vì (như đã thảo luận trong chương này) các khoảng trễ và tạm dừng không giới hạn thực sự có xảy ra.

Partially synchronous model (mô hình bán đồng bộ)

Partial synchrony có nghĩa là một hệ thống hoạt động giống như một synchronous system *trong hầu hết thời gian*, nhưng đôi khi nó vượt quá các giới hạn về độ trễ mạng, các khoảng tạm dừng của process và clock drift [108]. Đây là một mô hình thực tế của nhiều hệ thống. Trong hầu hết thời gian, mạng và các process hoạt động khá ổn định—nếu không, chúng ta sẽ không bao giờ có thể hoàn thành bất cứ việc gì—nhưng chúng ta phải tính đến thực tế rằng bất kỳ giả định về thời gian nào cũng có thể thỉnh thoảng bị phá vỡ. Khi điều này xảy ra, độ trễ mạng, các khoảng tạm dừng và lỗi clock có thể trở nên lớn một cách tùy ý.

Asynchronous model

Trong mô hình này, một thuật toán không được phép đưa ra bất kỳ giả định nào về thời gian—thậm chí, nó còn không có một clock (vì vậy nó không thể sử dụng timeout). Một số thuật toán có thể được thiết kế cho asynchronous model, nhưng nó rất hạn chế.

Bên cạnh các vấn đề về thời gian, chúng ta cũng phải xem xét các lỗi node. Một số system model phổ biến cho các node như sau:

Crash-stop faults

Trong mô hình *crash-stop* (hoặc *fail-stop*), một thuật toán có thể giả định rằng một node chỉ có thể lỗi theo một cách duy nhất—đó là bị crash [109]. Node đó có thể đột ngột ngừng phản hồi vào bất kỳ lúc nào, và sau đó node đó sẽ biến mất vĩnh viễn—nó không bao giờ quay trở lại.

Crash-recovery faults

Trong mô hình crash-recovery, chúng ta giả định rằng các node có thể crash vào bất kỳ lúc nào, và có lẽ sẽ bắt đầu phản hồi trở lại sau một khoảng thời gian không xác định. Các node được giả định là có bộ lưu trữ ổn định (tức là bộ lưu trữ đĩa không bay hơi) được bảo toàn qua các lần crash, trong khi trạng thái trong bộ nhớ (in-memory state) được giả định là bị mất.

Hiệu năng suy giảm và chức năng một phần

Ngoài việc bị crash và khởi động lại, các node có thể bị chậm lại. Chúng có thể vẫn có khả năng phản hồi các yêu cầu health check, trong khi lại quá chậm để thực hiện bất kỳ công việc thực tế nào. Ví dụ, một giao diện mạng Gigabit có thể đột ngột giảm throughput xuống còn 1 Kb/s do lỗi driver [110]; một process đang chịu áp lực bộ nhớ có thể dành phần lớn thời gian để thực hiện garbage collection [111]; các ổ SSD bị mòn có thể có hiệu năng thất thường; và phần cứng có thể bị ảnh hưởng bởi nhiệt độ cao, đầu nối lỏng, rung động cơ học, các vấn đề về nguồn điện, lỗi firmware, và nhiều yếu tố khác [112]. Tình huống như vậy được gọi là một *limping node*, *gray failure*, hoặc *fail-slow* [113], và nó thậm chí có thể khó xử lý hơn cả một node bị crash hoàn toàn. Một vấn đề liên quan xảy ra khi một process ngừng thực hiện một số việc mà nó đáng lẽ phải làm trong khi các khía cạnh khác vẫn tiếp tục hoạt động—ví dụ, do một background thread bị crash hoặc bị deadlock [114].

Byzantine (lỗi tùy ý) faults

Các node có thể làm bất cứ điều gì, bao gồm cả việc cố gắng lừa dối và đánh lừa các node khác, như đã được mô tả trong mục trước.

Để mô hình hóa các hệ thống thực tế, mô hình partially synchronous với các lỗi crash-recovery thường là mô hình hữu ích nhất. Nó cho phép độ trễ mạng không giới hạn, các quá trình tạm dừng (process pauses), và các node chậm. Nhưng các thuật toán phân tán đối phó với mô hình đó như thế nào?

Định nghĩa tính đúng đắn của một thuật toán

Để định nghĩa thế nào là một thuật toán *đúng đắn* (correct), chúng ta có thể mô tả các *properties* (tính chất) của nó. Ví dụ, đầu ra của một thuật toán sắp xếp có tính chất là đối với bất kỳ hai phần tử phân biệt nào trong danh sách đầu ra, phần tử nằm xa hơn về phía bên trái sẽ nhỏ hơn phần tử nằm xa hơn về phía bên phải. Đó đơn giản là một cách thức hình thức để định nghĩa thế nào là một danh sách đã được sắp xếp—một invariant của một danh sách đã sắp xếp.

Tương tự, chúng ta có thể viết ra các tính chất mà chúng ta mong muốn ở một thuật toán phân tán để định nghĩa thế nào là đúng đắn. Ví dụ, nếu chúng ta đang tạo các fencing token cho một lock (xem “Fencing off zombies and delayed requests”), chúng ta có thể yêu cầu thuật toán phải có các tính chất sau:

Uniqueness

Không có hai yêu cầu cho một fencing token nào trả về cùng một giá trị.

Monotonic sequence

Nếu yêu cầu x trả về token tx , và yêu cầu y trả về token ty , và x hoàn tất trước khi y bắt đầu, thì $tx < ty$.

Availability

Một node yêu cầu một fencing token và không bị crash cuối cùng sẽ nhận được một phản hồi.

Một thuật toán là đúng đắn trong một mô hình hệ thống nếu nó luôn thỏa mãn các tính chất của nó trong tất cả các tình huống mà chúng ta giả định có thể xảy ra trong mô hình hệ thống đó. Tuy nhiên, nếu tất cả các node đều crash, hoặc tất cả các độ trễ mạng đột ngột trở nên vô hạn, thì không có thuật toán nào có thể thực hiện được bất cứ việc gì. Làm thế nào chúng ta vẫn có thể đưa ra các đảm bảo hữu ích ngay cả trong một mô hình hệ thống cho phép các lỗi hoàn toàn?

Phân biệt giữa safety và liveness

Để làm rõ tình huống này, việc phân biệt giữa hai loại tính chất là rất đáng giá: *safety* và *liveness*. Trong ví dụ vừa nêu, *uniqueness* và *monotonic sequence* là các safety properties, nhưng *availability* là một liveness property.

Điều gì phân biệt hai loại tính chất này? Một dấu hiệu nhận biết là các liveness properties thường bao gồm từ “eventually” (cuối cùng thì) trong định nghĩa của chúng. (Và đúng vậy, bạn đã đoán đúng—*eventual consistency* là một liveness property [115].)

Safety thường được định nghĩa một cách không chính thức là *không có điều gì xấu xảy ra*, và liveness là *một điều gì đó tốt đẹp cuối cùng sẽ xảy ra*. Tuy nhiên, tốt nhất là không nên suy diễn quá nhiều từ những định nghĩa không chính thức đó, bởi vì “tốt” và “xấu” là những phán xét mang tính giá trị và không áp dụng tốt cho các thuật toán. Các định nghĩa thực tế về safety và liveness chính xác hơn [116]:

- Nếu một safety property bị vi phạm, chúng ta có thể chỉ ra thời điểm cụ thể mà nó bị phá vỡ (ví dụ, nếu tính chất uniqueness bị vi phạm, chúng ta có thể xác định thao tác cụ thể mà tại đó một fencing token trùng lặp đã được trả về). Sau khi một safety property đã bị vi phạm, sự vi phạm đó không thể được hoàn tác—thiệt hại đã xảy ra rồi.
- Một tính chất liveness hoạt động theo chiều ngược lại. Nó có thể không được thỏa mãn tại một thời điểm nhất định (ví dụ, một node có thể đã gửi một yêu cầu nhưng vẫn chưa nhận được phản hồi), nhưng luôn có hy vọng rằng nó sẽ được thỏa mãn trong tương lai (cụ thể là bằng việc nhận được một phản hồi).

Một lợi thế của việc phân biệt giữa tính chất safety và liveness là nó giúp chúng ta xử lý các mô hình hệ thống phức tạp. Đối với các thuật toán phân tán, thông thường người ta yêu cầu các tính chất safety phải *luôn luôn* được thỏa mãn trong mọi tình huống có thể xảy ra của một mô hình hệ thống [108]. Ngay cả khi tất cả các node bị crash, hoặc toàn bộ mạng bị lỗi, thuật toán vẫn phải đảm bảo rằng nó không trả về một kết quả sai (tức là các tính chất safety vẫn được thỏa mãn).

Tuy nhiên, với các tính chất liveness, chúng ta được phép đưa ra các lưu ý—ví dụ, chúng ta có thể nói rằng một yêu cầu chỉ cần nhận được phản hồi nếu đa số các node không bị crash, và chỉ khi mạng cuối cùng sẽ phục hồi sau một đợt gián đoạn. Định nghĩa của mô hình partially synchronous yêu cầu rằng cuối cùng hệ thống sẽ trở lại trạng thái synchronous—tức là, bất kỳ giai đoạn gián đoạn mạng nào cũng chỉ kéo dài trong một khoảng thời gian hữu hạn và sau đó sẽ được sửa chữa.

Ánh xạ các mô hình hệ thống vào thế giới thực

Các tính chất safety, liveness và các mô hình hệ thống rất hữu ích để suy luận về tính đúng đắn của một thuật toán phân tán. Tuy nhiên, khi triển khai một thuật toán trong thực tế, những sự thật hỗn loạn của thực tế sẽ quay lại gây khó khăn cho bạn, và rõ ràng là mô hình hệ thống chỉ là một sự trừu tượng hóa đơn giản hóa của thực tế.

Ví dụ, các thuật toán trong mô hình crash-recovery thường giả định rằng dữ liệu trong stable storage sẽ tồn tại sau các đợt crash. Tuy nhiên, điều gì sẽ xảy ra nếu dữ liệu trên đĩa bị hỏng hoặc bị xóa sạch do lỗi phần cứng hoặc cấu hình sai [117]? Điều gì sẽ xảy ra nếu một máy chủ gặp lỗi firmware và không nhận diện được các ổ cứng của nó khi khởi động lại mặc dù các ổ cứng đã được gắn chính xác vào máy chủ [118]?

Các thuật toán quorum (xem “Sử dụng quorum để đọc và ghi”) dựa vào việc một node ghi nhớ dữ liệu mà nó tuyên bố đã lưu trữ. Nếu một node có thể bị mất trí nhớ (amnesia) và quên đi dữ liệu đã lưu trữ trước đó, điều đó sẽ phá vỡ điều kiện quorum và do đó phá vỡ tính đúng đắn của thuật toán. Có lẽ cần một mô hình hệ thống mới, trong đó chúng ta giả định rằng stable storage phần lớn sẽ tồn tại sau các đợt crash nhưng đôi khi có thể bị mất. Nhưng khi đó, mô hình này sẽ trở nên khó suy luận hơn.

Mô tả lý thuyết của một thuật toán có thể tuyên bố rằng một số điều nhất định đơn giản là được giả định là không xảy ra—và trong các hệ thống non-Byzantine, chúng ta thực sự phải đưa ra các giả định về những lỗi có thể và không thể xảy ra. Tuy nhiên, một triển khai thực tế vẫn có thể phải bao gồm mã để xử lý trường hợp một điều gì đó xảy ra mà vốn được giả định là không thể, ngay cả khi việc xử lý đó chỉ gói gọn trong `printf("Sucks to be you")` và `exit(666)`—tức là, để một người vận hành con người dọn dẹp đống hỗn độn [119]. (Đây là một sự khác biệt giữa khoa machine learning tính và kỹ thuật phần mềm.)

Điều đó không có nghĩa là các mô hình hệ thống trừu tượng mang tính lý thuyết là vô giá trị—hoàn toàn ngược lại. Chúng cực kỳ hữu ích trong việc đúc kết sự phức tạp của các hệ thống thực tế thành một tập hợp các lỗi có thể kiểm soát được để chúng ta có thể suy luận, từ đó hiểu được vấn đề và cố gắng giải quyết nó một cách có hệ thống.

Formal Methods và Randomized Testing

Làm thế nào để chúng ta biết một thuật toán thỏa mãn các tính chất yêu cầu? Do tính chất concurrency, các lỗi cục bộ (partial failures) và độ trễ mạng, có một số lượng khổng lồ các trạng thái tiềm năng. Chúng ta cần đảm bảo rằng các tính chất được thỏa mãn trong mọi trạng thái có thể và đảm bảo rằng chúng ta không bỏ sót bất kỳ trường hợp biên (edge case) nào.

Một cách tiếp cận là xác minh chính thức (formally verify) một thuật toán bằng cách mô tả nó dưới dạng toán học và sử dụng các kỹ thuật chứng minh để chỉ ra rằng nó thỏa mãn các thuộc tính yêu cầu trong mọi tình huống mà mô hình hệ thống cho phép. Việc chứng minh một thuật toán là đúng không có nghĩa là *implementation* (triển khai) của nó trên một hệ thống thực tế sẽ luôn hoạt động chính xác. Nhưng đó là một bước đầu tiên rất tốt, bởi vì phân tích lý thuyết có thể phát hiện ra các vấn đề trong một thuật toán vốn có thể bị che khuất trong một thời gian dài trong một hệ thống thực tế và chỉ gây rắc rối cho bạn khi các giả định (ví dụ: về thời gian) bị đánh bại bởi các tình huống bất thường.

Việc kết hợp phân tích lý thuyết với kiểm thử thực nghiệm để xác minh rằng các implementation hoạt động như mong đợi là điều thận trọng. Các kỹ thuật như property-based testing, fuzzing, và deterministic simulation testing sử dụng tính ngẫu nhiên để kiểm thử một hệ thống trong một phạm vi rộng các tình huống. Các tổ chức như Amazon Web Services, FoundationDB, và TigerBeetle đã sử dụng thành công sự kết hợp của các kỹ thuật này trên nhiều sản phẩm của họ [120, 121, 122, 123].

Model checking và các ngôn ngữ đặc tả

Model checkers là các công cụ giúp xác minh rằng một thuật toán hoặc hệ thống hoạt động như mong đợi. Một đặc tả thuật toán được viết bằng một ngôn ngữ chuyên dụng như TLA+, Gallina, hoặc FizzBee. Những ngôn ngữ này giúp việc tập trung vào hành vi của một thuật toán trở nên dễ dàng hơn mà không cần lo lắng về các chi tiết triển khai code. Sau đó, các model checkers sử dụng các mô hình này để xác minh rằng các invariant (bất biến) được duy trì trên tất cả các trạng thái của một thuật toán bằng cách thử nghiệm một cách hệ thống tất cả những gì có thể xảy ra.

Model checking thực tế không thể chứng minh rằng các invariant của một thuật toán được duy trì cho mọi trạng thái có thể, vì hầu hết các thuật toán trong thế giới thực đều có không gian trạng thái vô hạn. Việc xác minh thực sự tất cả các trạng thái sẽ đòi hỏi một chứng minh chính thức, điều này có thể thực hiện được, nhưng thường khó hơn so với việc chạy một model checker. Thay vào đó, các model checkers khuyến khích bạn giảm mô hình của thuật toán xuống một dạng xấp xỉ có thể được xác minh đầy đủ, hoặc giới hạn việc thực thi trong một giới hạn trên (ví dụ: bằng cách thiết lập số lượng tin nhắn tối đa có thể được gửi đi). Bất kỳ bug nào chỉ xảy ra với các lần thực thi dài hơn thì khi đó sẽ không được tìm thấy.

Tuy nhiên, các model checkers tạo ra một sự cân bằng tốt giữa tính dễ sử dụng và khả năng tìm ra các bug không rõ ràng. CockroachDB, TiDB, Kafka, và nhiều hệ thống phân tán khác sử dụng các đặc tả mô hình để tìm và sửa bug [124, 125, 126]. Ví dụ, bằng cách sử dụng TLA+, các nhà nghiên cứu đã có thể chứng minh khả năng mất dữ liệu trong viewstamped replication (VR) gây ra bởi sự mơ hồ trong mô tả bằng văn bản của thuật toán [127].

Theo thiết kế, các model checkers không chạy code thực tế của bạn, mà thay vào đó là một mô hình đơn giản hóa chỉ đặc tả các ý tưởng cốt lõi trong giao thức của bạn. Điều này giúp việc khám phá không gian trạng thái một cách hệ thống trở nên khả thi hơn, nhưng nó cũng tiềm ẩn rủi ro là đặc tả và implementation của bạn sẽ mất đồng bộ với nhau [128]. Có thể kiểm tra xem mô hình và implementation thực tế có hành vi tương đương hay không, nhưng điều này đòi hỏi phải có instrumentation (cài đặt các công cụ đo lường) trong implementation thực tế [129].

Fault injection

Nhiều bug được kích hoạt khi các lỗi máy móc và lỗi mạng xảy ra. *Fault injection* là một kỹ thuật hiệu quả (và đôi khi đáng sợ) giúp xác minh xem implementation của một hệ thống có hoạt động như mong đợi khi có sự cố xảy ra hay không. Ý tưởng rất đơn giản: tiêm các lỗi vào môi trường của một hệ thống đang chạy và xem nó hoạt động như thế nào. Lỗi có thể là lỗi mạng, máy tính bị crash, lỗi hỏng đĩa, các process bị tạm dừng—bất cứ điều gì bạn có thể tưởng tượng là sẽ xảy ra sai sót với một máy tính.

Các bài kiểm thử fault injection thường được chạy trong một môi trường mô phỏng sát với môi trường production nơi hệ thống sẽ vận hành. Một số nơi thậm chí còn tiêm lỗi trực tiếp vào môi trường production của họ. Netflix đã phổ biến cách tiếp cận này với công cụ Chaos Monkey của mình [130]. Việc fault injection trên production thường được gọi là *chaos engineering*, nội dung mà chúng ta đã thảo luận trong mục “Reliability and Fault Tolerance”.

Để chạy các bài kiểm tra fault injection (tiêm lỗi), trước tiên hệ thống cần kiểm tra (system under test) sẽ được triển khai cùng với các fault injection coordinator và các script. Các coordinator chịu trách nhiệm quyết định sẽ thực thi những lỗi nào và thực thi chúng khi nào. Các script cục bộ hoặc từ xa chịu trách nhiệm tiêm các lỗi vào các node hoặc các process riêng lẻ. Các script injection sử dụng nhiều công cụ để kích hoạt lỗi. Một Linux process có thể bị tạm dừng hoặc bị kill bằng lệnh `kill` của Linux, một ổ đĩa có thể bị unmount bằng `umount`, và các kết nối mạng có thể bị gián đoạn thông qua các thiết lập firewall. Bạn có thể kiểm tra hành vi của hệ thống trong và sau khi các lỗi được tiêm để đảm bảo mọi thứ hoạt động như mong đợi.

Sự đa dạng của các công cụ cần thiết để kích hoạt lỗi khiến các bài kiểm tra fault injection trở nên cồng kềnh khi viết. Việc áp dụng một fault injection framework như Jepsen để chạy các bài kiểm tra fault injection nhằm đơn giản hóa quy trình là

điều phổ biến. Các framework như vậy đi kèm với các tích hợp cho nhiều hệ điều hành khác nhau và nhiều fault injector được xây dựng sẵn [131]. Jepsen đã tỏ ra cực kỳ hiệu quả trong việc tìm ra các bug nghiêm trọng trong nhiều hệ thống được sử dụng rộng rãi [132, 133].

Kiểm thử mô phỏng xác định (Deterministic simulation testing)

Một kỹ thuật hình thức hóa khác, vốn đã trở thành một sự bổ sung phổ biến cho model checking và fault injection, là *deterministic simulation testing* (DST) (kiểm thử mô phỏng xác định). Nó sử dụng một quy trình khám phá không gian trạng thái tương tự như một model checker, nhưng nó kiểm thử mã nguồn thực tế của bạn, chứ không phải một mô hình.

Trong DST, một trình mô phỏng sẽ tự động chạy qua một số lượng lớn các lần thực thi ngẫu nhiên của hệ thống. Giao tiếp mạng, I/O, và clock timing trong quá trình mô phỏng đều được thay thế bằng các mock, cho phép trình mô phỏng kiểm soát chính xác thứ tự mà các sự việc xảy ra, bao gồm cả các mốc thời gian và các kịch bản lỗi khác nhau. Điều này cho phép trình mô phỏng khám phá nhiều tình huống hơn so với các bài kiểm tra viết tay hoặc fault injection có thể làm được. Nếu một bài kiểm tra thất bại, nó có thể được chạy lại vì trình mô phỏng biết chính xác thứ tự các thao tác đã kích hoạt lỗi đó—trái ngược với fault injection, vốn không có khả năng kiểm soát tinh vi như vậy đối với hệ thống.

DST yêu cầu trình mô phỏng phải có khả năng kiểm soát tất cả các nguồn gây ra tính không xác định (nondeterminism), chẳng hạn như độ trễ mạng hoặc việc lập lịch thread trong mã nguồn đa luồng (multithreaded). Một trong ba chiến lược thường được áp dụng để làm cho mã nguồn trở nên xác định (deterministic):

Cấp độ ứng dụng (Application-level)

Một số hệ thống được xây dựng ngay từ đầu để giúp việc thực thi mã nguồn một cách xác định trở nên dễ dàng. Ví dụ, FoundationDB, một trong những đơn vị tiên phong trong lĩnh vực DST, được xây dựng bằng cách sử dụng một thư viện giao tiếp bất đồng bộ gọi là Flow. Flow cung cấp một điểm để các lập trình viên tiêm một mô phỏng mạng xác định vào hệ thống [134]. Tương tự, TigerBeetle là một cơ sở dữ liệu OLTP với sự hỗ trợ DST ở mức ưu tiên (first-class). Trạng thái của hệ thống được mô hình hóa dưới dạng một state machine, với tất cả các thay đổi (mutations) đều xảy ra trong một event loop duy nhất. Khi kết hợp với các primitive xác định dạng mock như các đồng hồ, một kiến trúc như vậy có khả năng chạy một cách xác định [135].

Cấp độ runtime (Runtime-level)

Các ngôn ngữ với runtime bất đồng bộ và các thư viện thường dùng cung cấp một điểm chèn để đưa tính xác định vào. Một runtime đơn luồng (single-threaded) được sử dụng để buộc tất cả mã bất đồng bộ phải chạy tuần tự. Ví dụ, FrostDB và runtime của Go để thực thi các goroutine một cách tuần tự [136].

Thư viện MadSim của Rust hoạt động theo cách tương tự. Nó cung cấp các triển khai xác định cho API runtime bất đồng bộ của Tokio, thư viện S3 của Amazon, thư viện Rust của Kafka, và nhiều thư viện khác; các ứng dụng có thể thay thế bằng các thư viện và runtime xác định để có được các lần thực thi kiểm thử xác định mà không cần thay đổi mã nguồn của chúng.

Cấp độ máy (Machine-level)

Thay vì vá mã nguồn tại thời điểm runtime, ta có thể làm cho toàn bộ một máy trở nên deterministic (tính xác định). Đây là một quá trình tinh vi đòi hỏi máy phải phản hồi tất cả các lời gọi vốn dĩ không mang tính nondeterministic bằng các phản hồi mang tính deterministic. Các công cụ như Antithesis thực hiện điều này bằng cách xây dựng một hypervisor tùy chỉnh để thay thế các hoạt động vốn không mang tính nondeterministic bằng các hoạt động mang tính deterministic. Mọi thứ từ đồng hồ đến mạng và lưu trữ đều cần được tính đến. Tuy nhiên, một khi đã hoàn tất, các lập trình viên có thể chạy toàn bộ hệ thống phân tán của họ trong một tập hợp các container bên trong hypervisor và có được một hệ thống phân tán hoàn toàn mang tính deterministic.

DST cung cấp một số lợi thế vượt xa khả năng replay (phát lại). Ví dụ, Antithesis cố gắng khám phá nhiều đường dẫn trong mã nguồn ứng dụng bằng cách phân nhánh một lần thực thi kiểm thử thành nhiều lần thực thi con khi nó phát hiện ra các hành vi ít phổ biến hơn. Và vì các bài kiểm thử mang tính deterministic thường sử dụng các clock và lời gọi mạng đã được mock, các bài kiểm thử như vậy có thể chạy nhanh hơn thời gian thực (wall clock time). Chẳng hạn, sự trừu tượng hóa thời gian của TigerBeetle cho phép các mô phỏng mô phỏng latency (độ trễ) mạng và timeout mà không thực sự mất toàn bộ khoảng thời gian để kích hoạt timeout. Những kỹ thuật như vậy cho phép bộ mô phỏng khám phá nhiều đường dẫn mã nguồn hơn một cách nhanh chóng.

Sức mạnh của tính Determinism

Nondeterminism nằm ở cốt lõi của tất cả các thách thức về hệ thống phân tán mà chúng ta đã thảo luận trong chương này: concurrency (tính đồng thời), độ trễ mạng, việc tạm dừng tiến trình, sự nhảy vọt của đồng hồ, và các lỗi crash đều xảy ra theo những cách không thể dự đoán được, thay đổi từ lần chạy này sang lần chạy khác của một hệ thống. Ngược lại, nếu bạn có thể làm cho một hệ thống trở nên deterministic, điều đó có thể đơn giản hóa mọi thứ một cách đáng kể.

Trên thực tế, việc làm cho mọi thứ trở nên deterministic là một ý tưởng đơn giản nhưng mạnh mẽ, xuất hiện lặp đi lặp lại trong thiết kế hệ thống phân tán. Bên cạnh việc kiểm thử mô phỏng mang tính deterministic, chúng ta đã thấy một số cách sử dụng tính determinism trong các chương trước:

- Một lợi thế then chốt của event sourcing (xem “Event Sourcing và CQRS”) là bạn có thể phát lại một log các sự kiện một cách deterministic để tái cấu trúc các materialized view (view đã được vật chất hóa) phải sinh.
- Các workflow engine (xem “Durable Execution và Workflows”) dựa vào việc các định nghĩa workflow phải mang tính deterministic để cung cấp các ngữ nghĩa thực thi bền vững (durable execution).
- *State machine replication*, thứ mà chúng ta sẽ thảo luận trong mục “Sử dụng shared logs”, nhân bản dữ liệu bằng cách thực thi độc lập cùng một chuỗi các transaction mang tính deterministic trên mỗi replica. Chúng ta đã thấy hai biến thể của ý tưởng đó: statement-based replication (xem “Implementation of Replication Logs”) và thực thi transaction tuần tự bằng cách sử dụng stored procedures (xem “Ưu và nhược điểm của stored procedures”).

Tuy nhiên, việc làm cho mã nguồn hoàn toàn mang tính deterministic đòi hỏi sự cẩn trọng. Ngay cả khi bạn đã loại bỏ tất cả tính đồng thời và thay thế I/O, giao tiếp mạng, đồng hồ, và các bộ tạo số ngẫu nhiên bằng các mô phỏng mang tính deterministic, các yếu tố nondeterminism vẫn có thể tồn tại. Ví dụ, trong một số ngôn ngữ lập trình, thứ tự mà bạn lặp qua các phần tử của một hash table có thể không mang tính deterministic. Việc bạn có gặp phải giới hạn tài nguyên (lỗi cấp phát bộ nhớ, stack overflow) hay không cũng mang tính nondeterministic.

Tóm tắt

Trong chương này, chúng ta đã thảo luận về một loạt các vấn đề có thể xảy ra trong các hệ thống phân tán. Ví dụ:

- Bất cứ khi nào bạn cố gắng gửi một packet qua mạng, nó có thể bị mất hoặc bị trì hoãn một cách tùy ý. Tương tự, phản hồi có thể bị mất hoặc bị trì hoãn, vì vậy nếu bạn không nhận được phản hồi, bạn sẽ không biết liệu thông điệp có truyền đi thành công hay không.
- Đồng hồ của một node có thể bị lệch đáng kể so với các node khác (bất chấp những nỗ lực tốt nhất của bạn để thiết lập NTP), nó có thể đột ngột nhảy về phía trước hoặc lùi lại về thời gian, và việc dựa vào nó là rất nguy hiểm vì rất có thể bạn không có một thước đo tốt về khoảng tin cậy (confidence interval) của đồng hồ của mình.
- Một tiến trình có thể tạm dừng trong một khoảng thời gian đáng kể tại bất kỳ thời điểm nào trong quá trình thực thi, bị các node khác tuyên bố là đã chết, và sau đó sống lại một lần nữa mà không nhận ra rằng nó đã bị tạm dừng.

Việc các *partial failures* (lỗi cục bộ) như vậy có thể xảy ra chính là đặc điểm định nghĩa của các distributed systems. Bất cứ khi nào phần mềm cố gắng thực hiện bất kỳ điều gì liên quan đến các node khác, luôn có khả năng nó có thể thỉnh thoảng gặp lỗi, hoặc đột ngột chạy chậm, hoặc không phản hồi (và cuối cùng là bị time out). Trong các distributed systems, chúng ta cố gắng xây dựng khả năng chịu lỗi đối với các partial failures vào trong phần mềm để toàn bộ hệ thống có thể tiếp tục hoạt động ngay cả khi một số thành phần cấu thành của nó bị hỏng.

Để chịu được các fault, bước đầu tiên là phải *detect* (phát hiện) chúng, nhưng ngay cả việc đó cũng rất khó. Hầu hết các hệ thống không có cơ chế chính xác để phát hiện xem một node đã bị lỗi hay chưa, vì vậy hầu hết các distributed algorithms đều dựa vào timeouts để xác định xem một node từ xa có còn khả dụng hay không. Tuy nhiên, timeouts không thể phân biệt được giữa lỗi mạng và lỗi node, và độ trễ mạng biến đổi đôi khi khiến một node bị nghi ngờ nhầm là đã bị crash. Việc xử lý các limping nodes — những node vẫn đang phản hồi nhưng lại quá chậm để thực hiện bất kỳ việc gì hữu ích — thậm chí còn khó khăn hơn.

Một khi một fault đã được phát hiện, việc làm cho hệ thống chịu đựng được nó cũng không hề dễ dàng; không có biến toàn cục, không có shared memory, không có kiến thức chung hay bất kỳ loại shared state nào khác giữa các máy tính [83]. Các node thậm chí còn không thể thống nhất về việc hiện tại là mấy giờ, nói gì đến những điều sâu sắc hơn. Cách duy nhất để thông tin có thể truyền từ node này sang node khác là gửi nó qua mạng không đáng tin cậy. Các quyết định quan trọng không thể được đưa ra một cách an toàn bởi một node duy nhất, vì vậy chúng ta cần các protocol có

huy động sự trợ giúp từ các node khác và cố gắng đạt được một quorum để thống nhất.

Nếu bạn đã quen với việc viết phần mềm trong sự hoàn hảo toán học lý tưởng của một máy tính đơn lẻ, nơi mà cùng một thao tác luôn trả về cùng một kết quả một cách xác định, thì việc chuyển sang thực tế vật lý hỗn loạn của các distributed systems có thể là một cú sốc nhẹ. Ngược lại, các kỹ sư distributed systems thường coi một vấn đề là tầm thường nếu nó có thể được giải quyết trên một máy tính đơn lẻ [4], và thực tế là một máy tính đơn lẻ ngày nay có thể làm được rất nhiều việc. Nếu bạn có thể tránh mở chiếc hộp Pandora và chỉ đơn giản giữ mọi thứ trên một máy duy nhất — ví dụ, bằng cách sử dụng một embedded storage engine (xem “Embedded Storage Engines”) — thì việc đó thường là rất đáng giá.

Tuy nhiên, như đã thảo luận trong mục “Distributed Versus Single-Node Systems”, khả năng mở rộng không phải là lý do duy nhất để muốn sử dụng một distributed system. Khả năng chịu lỗi và latency (độ trễ) thấp (bằng cách đặt dữ liệu gần về mặt địa lý với người dùng) là những mục tiêu quan trọng không kém, và chúng không thể đạt được chỉ với một node duy nhất. Sức mạnh của các distributed systems là về nguyên tắc, chúng có thể chạy mãi mãi mà không bị gián đoạn ở cấp độ service, bởi vì tất cả các fault và việc bảo trì có thể được xử lý ở cấp độ node. (Trong thực tế, nếu một thay đổi cấu hình lỗi được roll out tới tất cả các node, điều đó vẫn sẽ khiến một distributed system phải gục ngã.)

Trong chương này, chúng ta cũng đã đi chệch hướng một chút để khám phá xem liệu sự không đáng tin cậy của mạng, đồng hồ và các tiến trình có phải là một quy luật tất yếu của tự nhiên hay không. Chúng ta thấy rằng không phải vậy: hoàn toàn có thể đưa ra các đảm bảo phản hồi thời gian thực cứng (hard real-time) và các độ trễ có giới hạn trong mạng, nhưng làm như vậy rất tốn kém và dẫn đến việc sử dụng tài nguyên phần cứng thấp hơn. Hầu hết các hệ thống không thuộc nhóm quan trọng về an toàn (non-safety-critical) đều chọn giải pháp rẻ và không đáng tin cậy thay vì đắt đỏ và đáng tin cậy.

Chương này hoàn toàn nói về các vấn đề, và nó đã trình bày cho chúng ta một triển vọng ảm đạm. Chúng ta thu được rất nhiều lợi ích bằng cách sử dụng các distributed systems đã được kiểm thử kỹ lưỡng và đạt chuẩn production để quản lý các vấn đề này. Trong chương tiếp theo, chúng ta sẽ chuyển sang các giải pháp và thảo luận về một số algorithm mà các hệ thống như vậy sử dụng để đối phó với những vấn đề này.

Footnotes

References

- [1] Mark Cavage. “There’s Just No Getting Around It: You’re Building a Distributed System.” *ACM Queue*, volume 11, issue 4, pages 80–89, April 2013. doi:10.1145/2466486.2482856
- [2] Jay Kreps. “Getting Real About Distributed System Reliability.” *blog.empathybox.com*, March 2012. Archived at perma.cc/9B5Q-AEBW

- [3] Coda Hale. “You Can’t Sacrifice Partition Tolerance.” *codahale.com*, October 2010. Archived at perma.cc/6GJU-X4G5
- [4] Jeff Hodges. “Notes on Distributed Systems for Young Bloods.” *somethingsimilar.com*, January 2013. Archived at perma.cc/B636-62CE
- [5] Van Jacobson. “Congestion Avoidance and Control.” At *ACM Symposium on Communications Architectures and Protocols* (SIGCOMM), August 1988. [doi:10.1145/52324.52356](https://doi.org/10.1145/52324.52356)
- [6] Bert Hubert. “The Ultimate SO_LINGER Page, or: Why Is My TCP Not Reliable.” *blog.netherlabs.nl*, January 2009. Archived at perma.cc/6HDX-L2RR
- [7] Jerome H. Saltzer, David P. Reed, and David D. Clark. “End-To-End Arguments in System Design.” *ACM Transactions on Computer Systems*, volume 2, issue 4, pages 277–288, November 1984. [doi:10.1145/357401.357402](https://doi.org/10.1145/357401.357402)
- [8] Peter Bailis and Kyle Kingsbury. “The Network Is Reliable.” *ACM Queue*, volume 12, issue 7, pages 48–55, July 2014. [doi:10.1145/2639988.2639988](https://doi.org/10.1145/2639988.2639988)
- [9] Joshua B. Leners, Trinabh Gupta, Marcos K. Aguilera, and Michael Walfish. “Taming Uncertainty in Distributed Systems with Help from the Network.” At *10th European Conference on Computer Systems* (EuroSys), April 2015. [doi:10.1145/2741948.2741976](https://doi.org/10.1145/2741948.2741976)
- [10] Phillipa Gill, Navendu Jain, and Nachiappan Nagappan. “Understanding Network Failures in Data Centers: Measurement, Analysis, and Implications.” At *ACM SIGCOMM Conference*, August 2011. [doi:10.1145/2018436.2018477](https://doi.org/10.1145/2018436.2018477)
- [11] Urs Hölzle. “But recently a farmer had started grazing a herd of cows nearby. And whenever they stepped on the fiber link, they bent it enough to cause a blip.” *x.com*, May 2020. Archived at perma.cc/WX8X-ZZA5
- [12] CBC News. “Hundreds Lose Internet Service in Northern B.C. After Beaver Chews Through Cable.” *cbc.ca*, April 2021. Archived at perma.cc/UW8C-H2MY
- [13] Will Oremus. “The Global Internet Is Being Attacked by Sharks, Google Confirms.” *slate.com*, August 2014. Archived at perma.cc/P6F3-C6YG
- [14] Jess Auerbach Jahajeeah. “Down to the Wire: The Ship Fixing Our Internet.” *continent.substack.com*, November 2023. Archived at perma.cc/DP7B-EQ7S
- [15] Santosh Janardhan. “More Details About the October 4 Outage.” *engineering.fb.com*, October 2021. Archived at perma.cc/WW89-VSXX
- [16] Tom Parfitt. “Georgian Woman Cuts off Web Access to Whole of Armenia.” *theguardian.com*, April 2011. Archived at perma.cc/KMC3-N3NZ
- [17] Antonio Voce, Tural Ahmedzade and Ashley Kirk. “‘Shadow Fleets’ and Subaquatic Sabotage: Are Europe’s Undersea Internet Cables Under Attack?” *theguardian.com*, March 2025. Archived at perma.cc/HA7S-ZDBV
- [18] Shengyun Liu, Paolo Viotti, Christian Cachin, Vivien Quéma, and Marko Vukolić. “XFT: Practical Fault Tolerance Beyond Crashes.” At *12th USENIX Symposium on Operating Systems Design and Implementation* (OSDI), November 2016.
- [19] Mark Imbriaco. “Downtime Last Saturday.” *github.blog*, December 2012. Archived at perma.cc/M7X5-E8SQ
- [20] Tom Lianza and Chris Snook. “A Byzantine Failure in the Real World.” *blog.cloudflare.com*, November 2020. Archived at perma.cc/83EZ-ALCY

- [21] Mohammed Alfatafta, Basil Alkhatib, Ahmed Alquraan, and Samer Al-Kiswany. “**Toward a Generic Fault Tolerance Technique for Partial Network Partitioning.**” At *14th USENIX Symposium on Operating Systems Design and Implementation* (OSDI), November 2020.
- [22] Marc A. Donges. “**Re: bnx2 Cards Intermittantly Going Offline.**” Message to Linux *netdev* mailing list, *spinics.net*, September 2012. Archived at perma.cc/TXP6-H8R3
- [23] Troy Toman. “**Inside a CODE RED: Network Edition.**” *signalnoise.com*, September 2020. Archived at perma.cc/BET6-FY25
- [24] Kyle Kingsbury. “**Jepsen: Elasticsearch.**” *aphyr.com*, June 2014. Archived at perma.cc/JK47-S89J
- [25] Salvatore Sanfilippo. “**A Few Arguments About Redis Sentinel Properties and Fail Scenarios.**” *antirez.com*, October 2014. Archived at perma.cc/8XEU-CLM8
- [26] Nicolas Liochon. “**CAP: If All You Have Is a Timeout, Everything Looks Like a Partition.**” *blog.thislongrun.com*, May 2015. Archived at perma.cc/FSS7-V2PZ
- [27] Matthew P. Grosvenor, Malte Schwarzkopf, Ionel Gog, Robert N. M. Watson, Andrew W. Moore, Steven Hand, and Jon Crowcroft. “**Queues Don’t Matter When You Can JUMP Them!**” At *12th USENIX Symposium on Networked Systems Design and Implementation* (NSDI), May 2015.
- [28] Theo Julienne. “**Debugging Network Stalls on Kubernetes.**” *github.blog*, November 2019. Archived at perma.cc/K9M8-XVGL
- [29] Guohui Wang and T. S. Eugene Ng. “**The Impact of Virtualization on Network Performance of Amazon EC2 Data Center.**” At *29th IEEE International Conference on Computer Communications* (INFOCOM), March 2010. [doi:10.1109/INFCOM.2010.5461931](https://doi.org/10.1109/INFCOM.2010.5461931)
- [30] Brandon Philips. “**etcd: Distributed Locking and Service Discovery.**” At *Strange Loop*, September 2014.
- [31] Steve Newman. “**A Systematic Look at EC2 I/O.**” *blog.scalyr.com*, October 2012. Archived at perma.cc/FL4R-H2VE
- [32] Naohiro Hayashibara, Xavier Défago, Rami Yared, and Takuya Katayama. “**The ϕ Accrual Failure Detector.**” Japan Advanced Institute of Science and Technology, School of Information Science, Technical Report IS-RR-2004-010, May 2004. Archived at perma.cc/NSM2-TRYA
- [33] Jeffrey Wang. “**Phi Accrual Failure Detector.**” *ternarysearch.blogspot.co.uk*, August 2013. Archived at perma.cc/L452-AMLV
- [34] Srinivasan Keshav. *An Engineering Approach to Computer Networking: ATM Networks, the Internet, and the Telephone Network*. Addison-Wesley Professional, 1997. ISBN: 9780201634426
- [35] Othmar Kys. *ATM Networks*. International Thomson Publishing, 1995. ISBN: 9781850321286
- [36] Jialin Li, Naveen Kr. Sharma, Dan R. K. Ports, and Steven D. Gribble. “**Tales of the Tail: Hardware, OS, and Application-level Sources of Tail Latency.**” At *ACM Symposium on Cloud Computing* (SOCC), November 2014. [doi:10.1145/2670979.2670988](https://doi.org/10.1145/2670979.2670988)
- [37] Mellanox Technologies. “**InfiniBand FAQ, Rev 1.3.**” *network.nvidia.com*, December 2014. Archived at perma.cc/LQJ4-QZVK
- [38] Jose Renato Santos, Yoshio Turner, and G. (John) Janakiraman. “**End-to-End Congestion Control for InfiniBand.**” At *22nd Annual Joint Conference of the IEEE Computer and Communications Societies* (INFOCOM), April 2003. Also published by HP Laboratories Palo Alto, Tech Report HPL-2002-359. [doi:10.1109/INFCOM.2003.1208949](https://doi.org/10.1109/INFCOM.2003.1208949)
- [39] Ulrich Windl, David Dalton, Marc Martinec, and Dale R. Worley. “**The NTP FAQ and HOWTO.**” *ntp.org*, November 2006. Archived at archive.org

- [40] John Graham-Cumming. “How and Why the Leap Second Affected Cloudflare DNS.” *blog.cloudflare.com*, January 2017. Archived at [archive.org](#)
- [41] David Holmes. “Inside the Hotspot VM: Clocks, Timers and Scheduling Events—Part I—Windows.” *blogs.oracle.com*, October 2006. Archived at [archive.org](#)
- [42] Joran Dirk Greef. “Three Clocks Are Better than One.” *tigerbeetle.com*, August 2021. Archived at [perma.cc/5RXG-EU6B](#)
- [43] Oliver Yang. “Pitfalls of TSC Usage.” *oliveryang.net*, September 2015. Archived at [perma.cc/Z2QY-5FRA](#)
- [44] Steve Loughran. “Time on Multi-Core, Multi-Socket Servers.” *stevelloughran.blogspot.co.uk*, September 2015. Archived at [perma.cc/7M4S-D4U6](#)
- [45] James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, JJ Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, Wilson Hsieh, Sebastian Kanthak, Eugene Kogan, Hongyi Li, Alexander Lloyd, Sergey Melnik, David Mwaura, David Nagle, Sean Quinlan, Rajesh Rao, Lindsay Rolig, Dale Woodford, Yasushi Saito, Christopher Taylor, Michal Szymaniak, and Ruth Wang. “Spanner: Google’s Globally-Distributed Database.” At *10th USENIX Symposium on Operating System Design and Implementation (OSDI)*, October 2012.
- [46] M. Caporalani and R. Ambrosini. “How Closely Can a Personal Computer Clock Track the UTC Timescale Via the Internet?” *European Journal of Physics*, volume 23, issue 4, pages L17–L21, June 2012. [doi:10.1088/0143-0807/23/4/103](#)
- [47] Nelson Minar. “A Survey of the NTP Network.” *alumni.media.mit.edu*, December 1999. Archived at [perma.cc/EV76-7ZV3](#)
- [48] Viliam Holub. “Synchronizing Clocks in a Cassandra Cluster Pt. 1—The Problem.” *blog.rapid7.com*, March 2014. Archived at [perma.cc/N3RV-5LNL](#)
- [49] Poul-Henning Kamp. “The One-Second War (What Time Will You Die?)” *ACM Queue*, volume 9, issue 4, pages 44–48, April 2011. [doi:10.1145/1966989.1967009](#)
- [50] Nelson Minar. “Leap Second Crashes Half the Internet.” *somebits.com*, July 2012. Archived at [perma.cc/2WB8-D6EU](#)
- [51] Christopher Pascoe. “Time, Technology and Leaping Seconds.” *googleblog.blogspot.co.uk*, September 2011. Archived at [perma.cc/U2JL-7E74](#)
- [52] Mingxue Zhao and Jeff Barr. “Look Before You Leap—The Coming Leap Second and AWS.” *aws.amazon.com*, May 2015. Archived at [perma.cc/KPE9-XMFM](#)
- [53] Darryl Veitch and Kanthaiah Vijayalayan. “Network Timing and the 2015 Leap Second.” At *17th International Conference on Passive and Active Measurement (PAM)*, April 2016. [doi:10.1007/978-3-319-30505-9_29](#)
- [54] VMware, Inc. “Timekeeping in VMware Virtual Machines.” *vmware.com*, October 2008. Archived at [perma.cc/HM5R-T5NF](#)
- [55] Victor Yodaiken. “Clock Synchronization in Finance and Beyond.” *yodaiken.com*, November 2017. Archived at [perma.cc/9XZD-8ZZN](#)
- [56] Mustafa Emre Acer, Emily Stark, Adrienne Porter Felt, Sascha Fahl, Radhika Bhargava, Bhanu Dev, Matt Braithwaite, Ryan Sleevi, and Parisa Tabriz. “Where the Wild Warnings Are: Root Causes of Chrome HTTPS Certificate Errors.” At *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, October 2017. [doi:10.1145/3133956.3134007](#)

- [57] European Securities and Markets Authority. “MiFID II / MiFIR: Regulatory Technical and Implementing Standards—Annex I.” *esma.europa.eu*, Report ESMA/2015/1464, September 2015. Archived at perma.cc/ZLX9-FGQ3
- [58] Luke Bigum. “Solving MiFID II Clock Synchronisation with Minimum Spend (Part 1).” *catach.blogspot.com*, November 2015. Archived at perma.cc/4J5W-FNM4
- [59] Oleg Obleukhov and Ahmad Byagowi. “How Precision Time Protocol Is Being Deployed at Meta.” *engineering.fb.com*, November 2022. Archived at perma.cc/29G6-UJNW
- [60] John Wiseman. “GPSJAM: Daily Maps of GPS Interference.” *gpsjam.org*
- [61] Josh Levinson, Julien Ridoux, and Chris Munns. “It’s About Time: Microsecond-Accurate Clocks on Amazon EC2 Instances.” *aws.amazon.com*, November 2023. Archived at perma.cc/56M6-5VMZ
- [62] Kyle Kingsbury. “Jepsen: Cassandra.” *aphyr.com*, September 2013. Archived at perma.cc/4MBR-J96V
- [63] John Daily. “Clocks Are Bad, or, Welcome to the Wonderful World of Distributed Systems.” *riak.com*, November 2013. Archived at perma.cc/4XB5-UCXY
- [64] Marc Brooker. “It’s About Time!” *brooker.co.za*, November 2023. Archived at perma.cc/N6YK-DRPA
- [65] Kyle Kingsbury. “The Trouble with Timestamps.” *aphyr.com*, October 2013. Archived at perma.cc/W3AM-5VAV
- [66] Leslie Lamport. “Time, Clocks, and the Ordering of Events in a Distributed System.” *Communications of the ACM*, volume 21, issue 7, pages 558–565, July 1978. doi: [10.1145/359545.359563](https://doi.org/10.1145/359545.359563)
- [67] Justin Sheehy. “There Is No Now: Problems with Simultaneity in Distributed Systems.” *ACM Queue*, volume 13, issue 3, pages 36–41, March 2015. doi: [10.1145/2733108](https://doi.org/10.1145/2733108)
- [68] Murat Demirbas. “Spanner: Google’s Globally-Distributed Database.” *muratbuffalo.blogspot.co.uk*, July 2013. Archived at perma.cc/6VWR-C9WB
- [69] Dahlia Malkhi and Jean-Philippe Martin. “Spanner’s Concurrency Control.” *ACM SIGACT News*, volume 44, issue 3, pages 73–77, September 2013. doi: [10.1145/2527748.2527767](https://doi.org/10.1145/2527748.2527767)
- [70] Franck Pachot. “Achieving Precise Clock Synchronization on AWS.” *yugabyte.com*, December 2024. Archived at perma.cc/UYM6-RNBS
- [71] Spencer Kimball. “Living Without Atomic Clocks: Where CockroachDB and Spanner Diverge.” *cockroachlabs.com*, January 2022. Archived at perma.cc/AWZ7-RXFT
- [72] Murat Demirbas. “Use of Time in Distributed Databases (Part 4): Synchronized Clocks in Production Databases.” *muratbuffalo.blogspot.com*, January 2025. Archived at perma.cc/9WNX-Q9U3
- [73] Cary G. Gray and David R. Cheriton. “Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency.” At *12th ACM Symposium on Operating Systems Principles* (SOSP), December 1989. doi: [10.1145/74850.74870](https://doi.org/10.1145/74850.74870)
- [74] Daniel Sturman, Scott Delap, Max Ross, et al. “Roblox Return to Service.” *corp.roblox.com*, January 2022. Archived at perma.cc/8ALT-WAS4
- [75] Todd Lipcon. “Avoiding Full GCs with MemStore-Local Allocation Buffers.” *slideshare.net*, February 2011. Archived at perma.cc/CH62-2EWJ
- [76] Christopher Clark, Keir Fraser, Steven Hand, Jacob Gorm Hansen, Eric Jul, Christian Limpach, Ian Pratt, and Andrew Warfield. “Live Migration of Virtual Machines.” At *2nd USENIX Symposium on Symposium on Networked Systems Design & Implementation* (NSDI), May 2005.

- [77] Mike Shaver. “fsyncers and Curveballs.” *shaver.off.net*, May 2008. Archived at [archive.org](#)
- [78] Zhenyun Zhuang and Cuong Tran. “Eliminating Large JVM GC Pauses Caused by Background IO Traffic.” *engineering.linkedin.com*, February 2016. Archived at [perma.cc/ML2M-X9XT](#)
- [79] Martin Thompson. “Java Garbage Collection Distilled.” *mechanical-sympathy.blogspot.co.uk*, July 2013. Archived at [perma.cc/DJT3-NQLQ](#)
- [80] David Terei and Amit Levy. “Blade: A Data Center Garbage Collector.” *arXiv:1504.02578*, April 2015.
- [81] Martin Maas, Tim Harris, Krste Asanović, and John Kubiawicz. “Trash Day: Coordinating Garbage Collection in Distributed Systems.” At *15th USENIX Workshop on Hot Topics in Operating Systems (HotOS)*, May 2015.
- [82] Martin Fowler. “The LMAX Architecture.” *martinfowler.com*, July 2011. Archived at [perma.cc/5AV4-N6RJ](#)
- [83] Joseph Y. Halpern and Yoram Moses. “Knowledge and Common Knowledge in a Distributed Environment.” *Journal of the ACM (JACM)*, volume 37, issue 3, pages 549–587, July 1990. doi: [10.1145/79147.79161](#)
- [84] Chuzhe Tang, Zhaoguo Wang, Xiaodong Zhang, Qianmian Yu, Binyu Zang, Haibing Guan, and Haibo Chen. “Ad Hoc Transactions in Web Applications: The Good, the Bad, and the Ugly.” At *ACM International Conference on Management of Data (SIGMOD)*, June 2022. doi: [10.1145/3514221.3526120](#)
- [85] Flavio P. Junqueira and Benjamin Reed. *ZooKeeper: Distributed Process Coordination*. O’Reilly Media, 2013. ISBN: 9781449361303
- [86] Enis Söztutar. “HBase and HDFS: Understanding Filesystem Usage in HBase.” At *HBaseCon*, June 2013. Archived at [perma.cc/4DXR-9P88](#)
- [87] SUSE LLC. “SUSE Linux Enterprise High Availability 15 SP6 Administration Guide, Section 12: Fencing and STONITH.” *documentation.suse.com*, March 2025. Archived at [perma.cc/8LAR-EL9D](#)
- [88] Mike Burrows. “The Chubby Lock Service for Loosely-Coupled Distributed Systems.” At *7th USENIX Symposium on Operating System Design and Implementation (OSDI)*, November 2006.
- [89] Kyle Kingsbury. “etcd 3.4.3.” *jepsen.io*, January 2020. Archived at [perma.cc/2P3Y-MPWU](#)
- [90] Ensar Basri Kahveci. “Distributed Locks Are Dead; Long Live Distributed Locks!” *hazelcast.com*, April 2019. Archived at [perma.cc/7FS5-LDXE](#)
- [91] Martin Kleppmann. “How to Do Distributed Locking.” *martin.kleppmann.com*, February 2016. Archived at [perma.cc/Y24W-YQ5L](#)
- [92] Salvatore Sanfilippo. “Is Redlock Safe?” *antirez.com*, February 2016. Archived at [perma.cc/B6GA-9Q6A](#)
- [93] Gunnar Morling. “Leader Election with S3 Conditional Writes.” *morling.dev*, August 2024. Archived at [perma.cc/7V2N-J78Y](#)
- [94] Leslie Lamport, Robert Shostak, and Marshall Pease. “The Byzantine Generals Problem.” *ACM Transactions on Programming Languages and Systems (TOPLAS)*, volume 4, issue 3, pages 382–401, July 1982. doi: [10.1145/357172.357176](#)
- [95] Jim N. Gray. “Notes on Data Base Operating Systems.” In *Operating Systems: An Advanced Course*, Lecture Notes in Computer Science, volume 60, edited by R. Bayer, R. M. Graham, and G. Seegmüller, pages 393–481, Springer-Verlag, 1978. ISBN: 9783540087557. Archived at [perma.cc/7S9M-2LZU](#)

- [96] Brian Palmer. “How Complicated Was the Byzantine Empire?” *slate.com*, October 2011. Archived at perma.cc/AN7X-FL3N
- [97] Leslie Lamport. “My Writings.” *lamport.azurewebsites.net*, December 2014. Archived at perma.cc/5NNM-SQGR
- [98] John Rushby. “Bus Architectures for Safety-Critical Embedded Systems.” At *1st International Workshop on Embedded Software* (EMSOFT), October 2001. [doi:10.1007/3-540-45449-7_22](https://doi.org/10.1007/3-540-45449-7_22)
- [99] Jake Edge. “ELC: SpaceX Lessons Learned.” *lwn.net*, March 2013. Archived at perma.cc/AYX8-QP5X
- [100] Shehar Bano, Alberto Sonnino, Mustafa Al-Bassam, Sarah Azouvi, Patrick McCorry, Sarah Meiklejohn, and George Danezis. “SoK: Consensus in the Age of Blockchains.” At *1st ACM Conference on Advances in Financial Technologies* (AFT), October 2019. [doi:10.1145/3318041.3355458](https://doi.org/10.1145/3318041.3355458)
- [101] Ezra Feilden, Adi Oltean, and Philip Johnston. “Why We Should Train AI in Space.” White Paper, *starcloud.com*, September 2024. Archived at perma.cc/7Y3S-8UB6
- [102] James Mickens. “The Saddest Moment.” *USENIX ;login*, May 2013. Archived at perma.cc/T7BZ-XCFR
- [103] Martin Kleppmann and Heidi Howard. “Byzantine Eventual Consistency and the Fundamental Limits of Peer-to-Peer Databases.” *arXiv:2012.00472*, December 2020.
- [104] Martin Kleppmann. “Making CRDTs Byzantine Fault Tolerant.” At *9th Workshop on Principles and Practice of Consistency for Distributed Data* (PaPoC), April 2022. [doi:10.1145/3517209.3524042](https://doi.org/10.1145/3517209.3524042)
- [105] Evan Gilman. “The Discovery of Apache ZooKeeper’s Poison Packet.” *pagerduty.com*, May 2015. Archived at perma.cc/RV6L-Y5CQ
- [106] Jonathan Stone and Craig Partridge. “When the CRC and TCP Checksum Disagree.” At *ACM Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication* (SIGCOMM), August 2000. [doi:10.1145/347059.347561](https://doi.org/10.1145/347059.347561)
- [107] Evan Jones. “How Both TCP and Ethernet Checksums Fail.” *evanjones.ca*, October 2015. Archived at perma.cc/9T5V-B8X5
- [108] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. “Consensus in the Presence of Partial Synchrony.” *Journal of the ACM*, volume 35, issue 2, pages 288–323, April 1988. [doi:10.1145/42282.42283](https://doi.org/10.1145/42282.42283)
- [109] Richard D. Schlichting and Fred B. Schneider. “Fail-Stop Processors: An Approach to Designing Fault-Tolerant Computing Systems.” *ACM Transactions on Computer Systems* (TOCS), volume 1, issue 3, pages 222–238, August 1983. [doi:10.1145/357369.357371](https://doi.org/10.1145/357369.357371)
- [110] Thanh Do, Mingzhe Hao, Tanakorn Leesatapornwongsa, Tiratat Patana-anake, and Haryadi S. Gunawi. “Limplock: Understanding the Impact of Limpware on Scale-out Cloud Systems.” At *4th ACM Symposium on Cloud Computing* (SoCC), October 2013. [doi:10.1145/2523616.2523627](https://doi.org/10.1145/2523616.2523627)
- [111] Josh Snyder and Joseph Lynch. “Garbage Collecting Unhealthy JVMs, a Proactive Approach.” *netflixtechblog.medium.com*, November 2019. Archived at perma.cc/8BTA-N3YB
- [112] Haryadi S. Gunawi, Riza O. Suminto, Russell Sears, Casey Golliher, Swaminathan Sundararaman, Xing Lin, Tim Emami, Weiguang Sheng, Nematollah Bidokhti, Caitie McCaffrey, Gary Grider, Parks M. Fields, Kevin Harms, Robert B. Ross, Andree Jacobson, Robert Ricci, Kirk Webb, Peter Alvaro, H. Biralı Runesha, Mingzhe Hao, and Huaicheng Li. “Fail-Slow at Scale: Evidence of Hardware Performance Faults in Large Production Systems.” At *16th USENIX Conference on File and Storage Technologies*, February 2018.

- [113] Peng Huang, Chuanxiong Guo, Lidong Zhou, Jacob R. Lorch, Yingnong Dang, Murali Chintalapati, and Randolph Yao. “Gray Failure: The Achilles’ Heel of Cloud-Scale Systems.” At *16th Workshop on Hot Topics in Operating Systems* (HotOS), May 2017. doi:10.1145/3102980.3103005
- [114] Chang Lou, Peng Huang, and Scott Smith. “Understanding, Detecting and Localizing Partial Failures in Large System Software.” At *17th USENIX Symposium on Networked Systems Design and Implementation* (NSDI), February 2020.
- [115] Peter Bailis and Ali Ghodsi. “Eventual Consistency Today: Limitations, Extensions, and Beyond.” *ACM Queue*, volume 11, issue 3, pages 55–63, March 2013. doi:10.1145/2460276.2462076
- [116] Bowen Alpern and Fred B. Schneider. “Defining Liveness.” *Information Processing Letters*, volume 21, issue 4, pages 181–185, October 1985. doi:10.1016/0020-0190(85)90056-0
- [117] Flavio P. Junqueira. “Dude, Where’s My Metadata?” *fpi.me*, May 2015. Archived at perma.cc/D2EU-Y9S5
- [118] Scott Sanders. “January 28th Incident Report.” *github.com*, February 2016. Archived at perma.cc/5GZR-88TV
- [119] Jay Kreps. “A Few Notes on Kafka and Jepsen.” *blog.empathybox.com*, September 2013. Archived at perma.cc/XJ5C-F583
- [120] Marc Brooker and Ankush Desai. “Systems Correctness Practices at AWS.” *ACM Queue*, volume 22, issue 6, pages 79–96, November/December 2024. doi:10.1145/3712057
- [121] Andrey Satarin. “Testing Distributed Systems: Curated list of Resources on Testing Distributed Systems.” *asatarin.github.io*. Archived at perma.cc/U5V8-XP24
- [122] Phil Eaton and Joran Dirk Greef. “We Put a Distributed Database in the Browser—And Made a Game of It!” *tigerbeetle.com*, June 2023. Archived at perma.cc/L7M7-X4HD
- [123] Apple, Inc. and the FoundationDB project authors. “FoundationDB—Simulation and Testing.” *apple.github.io*. Archived at perma.cc/4C4L-AUH3
- [124] Jack Vanlightly. “Verifying Kafka Transactions—Diary Entry 2—Writing an Initial TLA+ Spec.” *jack-vanlightly.com*, December 2024. Archived at perma.cc/NSQ8-MQ5N
- [125] Siddon Tang. “From Chaos to Order—Tools and Techniques for Testing TiDB, A Distributed NewSQL Database.” *pingcap.com*, April 2018. Archived at perma.cc/SEJB-R29F
- [126] Nathan VanBenschoten. “Parallel Commits: An Atomic Commit Protocol for Globally Distributed Transactions.” *cockroachlabs.com*, November 2019. Archived at perma.cc/5FZ7-QK6J
- [127] Jack Vanlightly. “Paper: VR Revisited—State Transfer (Part 3).” *jack-vanlightly.com*, December 2022. Archived at perma.cc/KNK3-K6WS
- [128] Hillel Wayne. “What If the Spec Doesn’t Match the Code?” *buttondown.com*, March 2024. Archived at perma.cc/8HEZ-KHER
- [129] Lingzhi Ouyang, Xudong Sun, Ruize Tang, Yu Huang, Madhav Jivrajani, Xiaoxing Ma, Tianyin Xu. “Multi-Grained Specifications for Distributed System Model Checking and Verification.” At *20th European Conference on Computer Systems* (EuroSys), March 2025. doi:10.1145/3689031.3696069
- [130] Yury Izrailevsky and Ariel Tseitlin. “The Netflix Simian Army.” *netflixtechblog.com*, July, 2011. Archived at perma.cc/M3NY-FJW6
- [131] Kyle Kingsbury. “Jepsen: On the Perils of Network Partitions.” *aphyr.com*, May, 2013. Archived at perma.cc/W98G-6HQP
- [132] Kyle Kingsbury. *Analyses*. *jepsen.io*, 2024. Archived at perma.cc/8LDN-D2T8

- [133] Rupak Majumdar and Filip Niksic. “Why Is Random Testing Effective for Partition Tolerance Bugs?” *Proceedings of the ACM on Programming Languages* (PACMPL), volume 2, issue POPL, article no. 46, December 2017. [doi:10.1145/3158134](https://doi.org/10.1145/3158134)
- [134] FoundationDB project authors. “Simulation and Testing.” *apple.github.io*. Archived at perma.cc/NQ3L-PM4C
- [135] Alex Kladov. “Simulation Testing for Liveness.” *tigerbeetle.com*, July 2023. Archived at perma.cc/RKD4-HGCR
- [136] Alfonso Subiotto Marqués. “(Mostly) Deterministic Simulation Testing in Go.” *polarsignals.com*, May 2024. Archived at perma.cc/ULD6-TSA4

Consistency và Consensus

Một câu châm ngôn cổ xưa cảnh báo: “Đừng bao giờ ra khơi với hai đồng hồ bấm giờ; hãy mang theo một hoặc ba.”

Frederick P. Brooks Jr., *The Mythical Man-Month: Essays on Software Engineering* (1995)

Rất nhiều thứ có thể xảy ra sai sót trong các distributed systems, như đã thảo luận ở Chương 9. Nếu chúng ta muốn một service tiếp tục hoạt động chính xác bất chấp những sai sót đó, chúng ta cần tìm cách để chịu lỗi (fault tolerance).

Một trong những công cụ tốt nhất mà chúng ta có để đạt được fault tolerance là *replication* (nhân bản dữ liệu). Tuy nhiên, như chúng ta đã thấy ở Chương 6, việc có nhiều bản sao dữ liệu trên nhiều replica làm tăng nguy cơ không nhất quán. Các thao tác đọc có thể được xử lý bởi một replica không cập nhật dữ liệu mới nhất, dẫn đến kết quả cũ (stale). Nếu nhiều replica có thể chấp nhận các thao tác ghi, chúng ta phải đối mặt với các xung đột tiềm tàng giữa các giá trị được ghi đồng thời trên các replica khác nhau. Ở mức độ cao, chúng ta có hai triết lý đối lập để giải quyết các vấn đề này:

Eventual consistency

Trong triết lý này, thực tế là một hệ thống được replication sẽ được hiển thị rõ ràng cho ứng dụng, và bạn — với tư cách là lập trình viên ứng dụng — được kỳ vọng sẽ xử lý các sự không nhất quán và xung đột có thể phát sinh. Cách tiếp cận này thường được sử dụng trong các hệ thống có multi-leader (xem “Multi-Leader Replication”) và leaderless replication (xem “Leaderless Replication”).

Strong consistency

Triết lý này cho rằng các ứng dụng không phải lo lắng về các chi tiết nội bộ của replication và hệ thống nên hoạt động như thể nó là một node duy nhất. Ưu điểm của cách tiếp cận này là nó đơn giản hơn cho bạn, người lập trình viên ứng dụng. Nhược điểm là strong consistency có cái giá về hiệu năng, và một số loại lỗi mà một hệ thống eventual consistency có thể chịu đựng được lại gây ra tình trạng ngừng hoạt động (outages) trong các hệ thống strong consistency.

Như thường lệ, cách tiếp cận nào tốt hơn tùy thuộc vào ứng dụng của bạn. Nếu ứng dụng của bạn cho phép người dùng thực hiện các thay đổi đối với dữ liệu khi ngoại tuyến (offline), eventual consistency là điều không thể tránh khỏi, như đã thảo luận trong mục “Sync Engines and Local-First Software”. Tuy nhiên, eventual consistency có thể gây khó khăn cho các ứng dụng trong việc xử lý. Nếu các replica của bạn nằm

trong các datacenters có kết nối nhanh và tin cậy, strong consistency thường là lựa chọn phù hợp vì chi phí của nó có thể chấp nhận được.

Trong chương này, chúng ta sẽ đi sâu hơn vào cách tiếp cận strong consistency, tập trung vào ba lĩnh vực:

- Một thách thức là “strong consistency” khá mơ hồ, vì vậy chúng ta sẽ xây dựng một định nghĩa chính xác hơn về những gì chúng ta muốn đạt được: linearizability.
- Chúng ta sẽ xem xét vấn đề tạo ID và timestamp. Điều này nghe có vẻ không liên quan đến consistency, nhưng nó có mối liên hệ chặt chẽ.
- Chúng ta sẽ khám phá cách các distributed systems có thể đạt được linearizability trong khi vẫn duy trì khả năng fault-tolerant — câu trả lời chính là các consensus algorithms.

Trong quá trình đó, chúng ta sẽ thấy rằng có những giới hạn cơ bản về những gì có thể thực hiện được trong một distributed system.

Các chủ đề được thảo luận trong chương này nổi tiếng là khó triển khai một cách chính xác. Rất dễ để xây dựng các hệ thống hoạt động tốt khi không có lỗi, nhưng lại hoàn toàn sụp đổ khi đối mặt với một sự kết hợp không may của các lỗi hoặc thứ tự tin nhắn mà các nhà thiết kế chưa tính đến. Rất nhiều lý thuyết đã được phát triển để giúp chúng ta suy nghĩ thấu đáo về các trường hợp biên (edge cases) đó, cho phép chúng ta xây dựng các hệ thống có thể chịu lỗi một cách mạnh mẽ.

Chương này sẽ chỉ mới chạm đến bề nổi. Chúng ta sẽ bám sát các trực giác không chính thức và tránh đi sâu vào các chi tiết thuật toán phức tạp, các mô hình hình thức và các chứng minh. Để thực hiện các công việc nghiêm túc về các hệ thống consensus và hạ tầng tương tự, bạn sẽ cần đi sâu hơn nhiều vào lý thuyết nếu muốn có bất kỳ cơ hội nào để hệ thống của mình hoạt động mạnh mẽ. Như thường lệ, các tài liệu tham khảo trong chương này sẽ cung cấp những chỉ dẫn ban đầu.

Linearizability

Nếu bạn muốn một cơ sở dữ liệu được replication (nhân bản dữ liệu) sao cho càng đơn giản càng tốt khi sử dụng, bạn nên làm cho nó hoạt động như thể nó là một cơ sở dữ liệu single-node có tính nhất quán. Khi đó, người dùng không phải lo lắng về replication lag, các xung đột và những sự không nhất quán khác; điều này mang lại cho bạn lợi thế về fault tolerance (khả năng chịu lỗi) nhưng không gặp phải sự phức tạp khi phải suy nghĩ về nhiều replica.

Đây chính là ý tưởng đằng sau *linearizability* (linearizability) [1] (còn được gọi là *atomic consistency* [2], *strong consistency*, *immediate consistency*, hoặc *external consistency* [3]). Định nghĩa chính xác về linearizability khá tinh tế, và chúng ta sẽ

khám phá nó trong phần còn lại của mục này. Tuy nhiên, ý tưởng cơ bản là làm cho một hệ thống xuất hiện như thể chỉ có một bản sao duy nhất của dữ liệu, và tất cả các thao tác trên đó đều là atomic. Với sự đảm bảo này, mặc dù trên thực tế có thể có nhiều replica, ứng dụng không cần phải lo lắng về chúng.

Trong một hệ thống linearizable, ngay khi một client hoàn tất thành công một thao tác write, tất cả các client đang đọc từ cơ sở dữ liệu đều phải có thể nhìn thấy giá trị vừa được ghi. Việc duy trì ảo giác về một bản sao duy nhất của dữ liệu có nghĩa là phải đảm bảo rằng giá trị được đọc là giá trị mới nhất, cập nhật nhất và không đến từ một cache hoặc replica đã cũ (stale). Nói cách khác, linearizability là một *recency guarantee* (đảm bảo tính mới nhất). Để làm rõ ý tưởng này, hãy cùng xem xét một ví dụ về một hệ thống không có tính linearizable.

Hình 10-1 cho thấy một website thể thao không có tính linearizable [4]. Aaliyah và Bryce đang ngồi trong cùng một phòng, cả hai đều đang kiểm tra điện thoại để xem kết quả trận đấu của đội bóng yêu thích. Ngay sau khi tỉ số cuối cùng được công bố, Aaliyah làm mới trang, thấy người chiến thắng đã được thông báo, và hào hứng kể với Bryce về điều đó. Bryce không tin vào mắt mình nên đã nhấn tải lại trên điện thoại của mình, nhưng request của anh ấy lại gửi đến một database replica đang bị replication lag, vì vậy điện thoại của anh ấy hiển thị rằng trận đấu vẫn đang tiếp diễn.

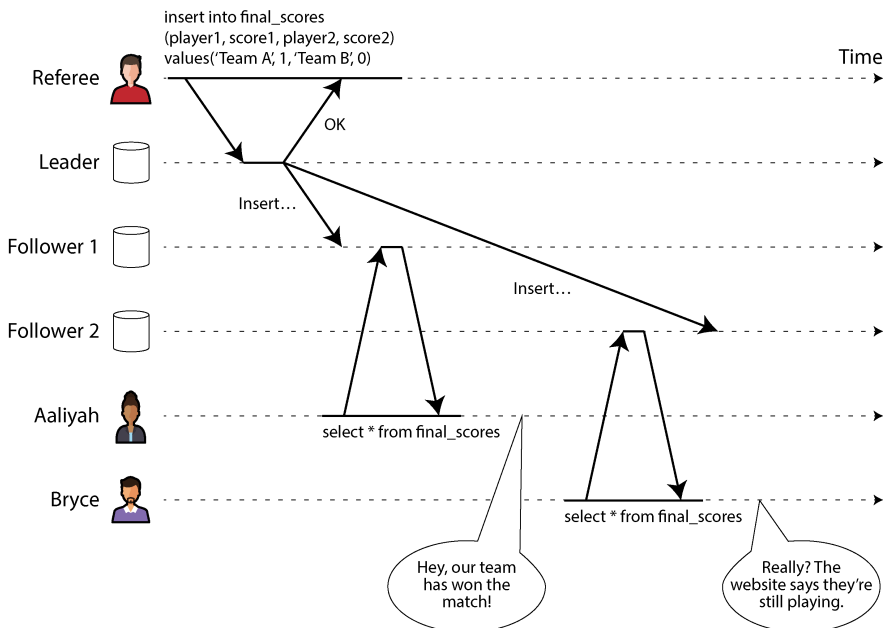


Figure 10-1. Hệ thống này không có tính linearizability, khiến những người hâm mộ thể thao cảm thấy bối rối.

Nếu Aaliyah và Bryce cùng nhấn reload tại một thời điểm, việc họ nhận được các kết quả truy vấn khác nhau sẽ ít gây ngạc nhiên hơn, bởi vì họ sẽ không biết chính xác thời điểm các yêu cầu tương ứng của mình được máy chủ xử lý. Tuy nhiên, Bryce biết rằng anh ấy đã nhấn nút reload (khởi tạo truy vấn của mình) *sau khi* nghe Aaliyah reo lên về điểm số cuối cùng, và do đó anh ấy kỳ vọng kết quả truy vấn của mình phải mới ít nhất là bằng với kết quả của Aaliyah. Việc truy vấn của anh ấy trả về một kết quả cũ (stale) là một sự vi phạm tính linearizability.

Điều gì làm nên một hệ thống có tính linearizability?

Để hiểu rõ hơn về linearizability, hãy cùng xem xét thêm các ví dụ. Hình 10-2 cho thấy ba client đang đồng thời đọc và ghi cùng một đối tượng x trong một cơ sở dữ liệu có tính linearizability. Trong lý thuyết distributed systems, x được gọi là một *register*—trong thực tế, ví dụ như nó có thể là một key trong một key-value store, một hàng trong một relational database, hoặc một document trong một document database.

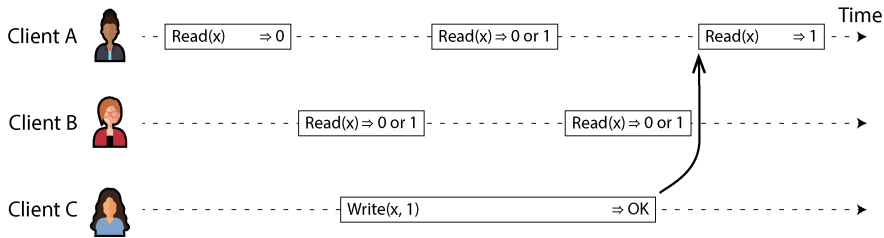


Figure 10-2. Nếu một yêu cầu đọc diễn ra đồng thời với một yêu cầu ghi, nó có thể trả về giá trị cũ hoặc giá trị mới.

Để đơn giản, Hình 10-2 chỉ hiển thị các yêu cầu từ góc nhìn của client, không hiển thị các thành phần bên trong của database. Mỗi thanh biểu diễn một yêu cầu được thực hiện bởi một client. Điểm bắt đầu của thanh là thời điểm yêu cầu được gửi đi, và điểm kết thúc của thanh là khi client nhận được phản hồi. Do độ trễ mạng thay đổi, client không biết chính xác thời điểm database xử lý yêu cầu của nó. Nó chỉ biết rằng việc xử lý chắc chắn đã xảy ra vào một thời điểm nào đó giữa lúc client gửi yêu cầu và lúc nhận được phản hồi.

Trong ví dụ này, register có hai loại thao tác:

- $Read(x) \Rightarrow v$ có nghĩa là client đã yêu cầu đọc giá trị của register x , và database đã trả về giá trị v .
- $Write(x, v) \Rightarrow r$ có nghĩa là client đã yêu cầu thiết lập register x thành giá trị v , và database đã trả về phản hồi r (có thể là OK hoặc Error).

Trong Hình 10-2, giá trị của x ban đầu là 0, và client C thực hiện một yêu cầu write để thiết lập nó thành 1. Trong khi việc này đang diễn ra, các client A và B liên tục polling database để đọc giá trị mới nhất. Những phản hồi khả thi mà A và B có thể nhận được cho các yêu cầu read của họ là gì?

Hãy cùng phân tích:

- Thao tác read đầu tiên của client A hoàn tất trước khi việc write bắt đầu, vì vậy nó phải trả về giá trị cũ là 0.
- Lần read cuối cùng của client A bắt đầu sau khi việc write đã hoàn tất, vì vậy nếu database có tính linearizability, nó phải trả về giá trị mới là 1, bởi vì thao tác read phải được xử lý sau thao tác write.
- Bất kỳ thao tác read nào chồng lấn về mặt thời gian với thao tác write đều có thể trả về 0 hoặc 1, vì chúng ta không biết liệu việc write đã có hiệu lực tại thời điểm thao tác read được xử lý hay chưa. Các thao tác này diễn ra *concurrent* (đồng thời) với thao tác write.

Tuy nhiên, điều này vẫn chưa đủ để mô tả đầy đủ tính linearizability. Nếu các thao tác read diễn ra concurrent với một thao tác write có thể trả về cả giá trị cũ hoặc giá trị

mới, thì người đọc có thể thấy giá trị bị đảo qua đảo lại giữa giá trị cũ và giá trị mới nhiều lần trong khi một thao tác write đang diễn ra. Đó không phải là những gì chúng ta mong đợi ở một hệ thống mô phỏng "một bản sao duy nhất của dữ liệu."

Để làm cho hệ thống có tính linearizability, chúng ta cần thêm một ràng buộc khác, được minh họa trong Hình 10-3.

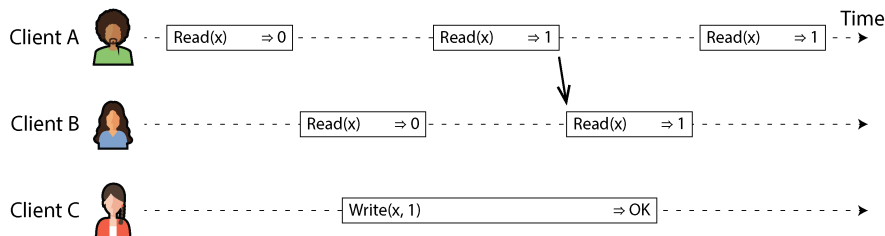


Figure 10-3. Sau khi bất kỳ một lượt đọc nào trả về giá trị mới, tất cả các lượt đọc tiếp theo (trên cùng một client hoặc các client khác) cũng phải trả về giá trị mới đó.

Trong một hệ thống có tính linearizability, ta hình dung rằng phải có một thời điểm nào đó (nằm giữa lúc bắt đầu và kết thúc của thao tác write) mà tại đó giá trị của x chuyển đổi nguyên tử từ 0 sang 1. Do đó, nếu thao tác read của một client trả về giá trị mới là 1, thì tất cả các thao tác read tiếp theo cũng phải trả về giá trị mới này, ngay cả khi thao tác write vẫn chưa hoàn tất.

Sự phụ thuộc vào thời điểm này được minh họa bằng một mũi tên trong Hình 10-3. Client A là người đầu tiên đọc được giá trị mới là 1. Ngay sau khi thao tác read của A trả về kết quả, B bắt đầu một thao tác read mới. Vì thao tác read của B xảy ra nghiêm ngặt sau thao tác read của A, nó cũng phải trả về 1, mặc dù thao tác write của C vẫn đang diễn ra. (Tình huống này tương tự như Aaliyah và Bryce trong Hình 10-1: sau khi Aaliyah đã đọc được giá trị mới, Bryce cũng kỳ vọng sẽ đọc được giá trị mới đó.)

Chúng ta có thể tinh chỉnh thêm biểu đồ thời gian này để trực quan hóa việc mỗi thao tác có hiệu lực một cách nguyên tử tại một thời điểm nào đó [5], giống như trong ví dụ phức tạp hơn được trình bày ở Hình 10-4. Trong ví dụ này, chúng ta thêm vào loại thao tác thứ ba bên cạnh *read* và *write*:

$CAS(x, v_{old}, v_{new}) \rightarrow r$ có nghĩa là client đã yêu cầu một thao tác CAS nguyên tử (xem mục “Conditional writes (compare-and-set)”). Nếu giá trị hiện tại của register x bằng v_{old} , nó sẽ được thiết lập nguyên tử thành v_{new} . Nếu giá trị của x khác với v_{old} , thì thao tác đó sẽ giữ nguyên register và trả về một lỗi. r là phản hồi của database (OK hoặc Error).

Mỗi thao tác trong Hình 10-4 được đánh dấu bằng một đường thẳng đứng (nằm bên trong thanh của mỗi thao tác) tại thời điểm mà chúng ta cho rằng thao tác đó đã

được thực thi. Những dấu mốc đó được nối lại với nhau theo thứ tự tuần tự, và kết quả phải là một chuỗi các thao tác read và write hợp lệ cho một register (mọi thao tác read phải trả về giá trị đã được thiết lập bởi thao tác write gần nhất).

Yêu cầu của linearizability là các đường nối các dấu mốc thao tác luôn luôn tiến về phía trước theo thời gian (từ trái sang phải), không bao giờ lùi lại. Yêu cầu này đảm bảo tính đảm bảo về độ mới (recency guarantee) mà chúng ta đã thảo luận trước đó: một khi một giá trị mới đã được write hoặc read, tất cả các thao tác read tiếp theo sẽ thấy giá trị đã được write đó, cho đến khi nó bị ghi đè một lần nữa.

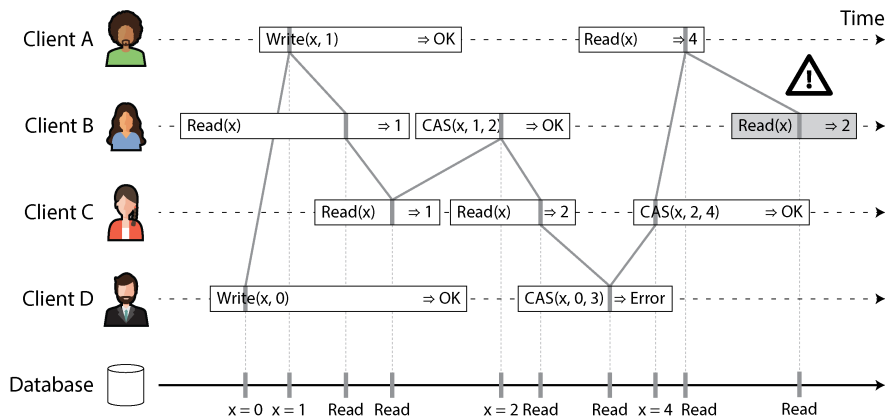


Figure 10-4. *Trực quan hóa các thời điểm mà các thao tác read và write có vẻ như đã có hiệu lực—lần read cuối cùng của B không có tính linearizable*

Có một vài chi tiết thú vị cần lưu ý trong Hình 10-4:

- Đầu tiên, client B gửi một yêu cầu để đọc x , sau đó client D gửi một yêu cầu để thiết lập x thành 0, và sau đó client A gửi một yêu cầu để thiết lập x thành 1. Tuy nhiên, giá trị trả về cho lượt đọc của B là 1 (giá trị được ghi bởi A). Điều này là OK: nó có nghĩa là cơ sở dữ liệu đã xử lý lượt ghi của D trước, sau đó đến lượt ghi của A, và cuối cùng là lượt đọc của B. Mặc dù đây không phải là thứ tự mà các yêu cầu đã được gửi đi, nhưng đó là một thứ tự có thể chấp nhận được, vì ba yêu cầu này diễn ra đồng thời (concurrent). Có lẽ yêu cầu đọc của B đã bị trễ một chút trong mạng, nên nó chỉ đến được cơ sở dữ liệu sau khi hai lượt ghi đã hoàn tất.
- Lượt đọc của client B trả về 1 trước khi client A nhận được phản hồi từ cơ sở dữ liệu nói rằng việc ghi giá trị 1 đã thành công. Điều này cũng OK, vì nó chỉ có nghĩa là phản hồi OK từ cơ sở dữ liệu gửi tới client A đã bị trễ một chút trong mạng.
- Mô hình này không giả định bất kỳ isolation level nào cho transaction; một client khác có thể thay đổi một giá trị vào bất kỳ lúc nào. Ví dụ, C đầu tiên đọc 1 và sau

đó đọc 2, vì giá trị đã bị thay đổi bởi B giữa hai lượt đọc. Một thao tác CAS nguyên tử có thể được sử dụng để kiểm tra xem giá trị đó có bị thay đổi đồng thời bởi một client khác hay không: các yêu cầu CAS của B và C thành công, nhưng yêu cầu CAS của D thất bại (vào thời điểm cơ sở dữ liệu xử lý nó, giá trị của x không còn là 0 nữa).

- Lượt đọc cuối cùng của client B (trong thanh có bóng mờ) không có tính linearizability. Thao tác này diễn ra đồng thời với lượt ghi CAS của C, vốn cập nhật x từ 2 thành 4. Nếu không có các yêu cầu khác, việc lượt đọc của B trả về 2 sẽ là OK. Tuy nhiên, client A đã đọc giá trị mới (4) trước khi lượt đọc của B bắt đầu, vì vậy B không được phép đọc một giá trị cũ hơn giá trị mà A đã đọc. Một lần nữa, đây là tình huống tương tự như với Aaliyah và Bryce trong Hình 10-1.

Đó chính là trực giác đằng sau linearizability; định nghĩa chính thức [1] mô tả nó một cách chính xác hơn. Có thể (mặc dù tốn kém về mặt tính toán) kiểm tra xem hành vi của một hệ thống có linearizable hay không bằng cách ghi lại thời điểm của tất cả các yêu cầu và phản hồi, sau đó kiểm tra xem liệu chúng có thể được sắp xếp thành một thứ tự tuần tự hợp lệ hay không [6, 7].

Giống như việc có nhiều isolation level yếu hơn cho các transaction bên cạnh serializability (xem mục “Weak Isolation Levels”), cũng có nhiều mô hình consistency yếu hơn cho các hệ thống được nhân bản (replicated) bên cạnh linearizability [8]. Các đảm bảo về read-after-write consistency, monotonic reads, và consistent prefix reads mà chúng ta đã thấy trong mục “Problems with Replication Lag” là những ví dụ về các mô hình này. Linearizability bao gồm tất cả các đảm bảo này và nhiều hơn thế nữa; nó là mô hình consistency mạnh nhất được sử dụng phổ biến.

Linearizability so với Serializability

Linearizability rất dễ bị nhầm lẫn với serializability (xem mục “Serializability”), vì cả hai từ đều có vẻ mang nghĩa kiểu như “có thể được sắp xếp theo một thứ tự tuần tự.” Tuy nhiên, chúng là những đảm bảo hoàn toàn khác nhau, và việc phân biệt giữa chúng là rất quan trọng:

Serializability

Serializability là một isolation level của các transaction, trong đó mỗi transaction có thể đọc và ghi *hiều đối tượng* (các hàng, tài liệu, bản ghi). Nó đảm bảo rằng các transaction hoạt động giống như thể chúng đã được thực thi theo *một* thứ tự tuần tự nào đó—nghĩa là, như thể bạn thực hiện tất cả các thao tác của một transaction trước, sau đó mới đến tất cả các thao tác của một transaction khác, và cứ tiếp tục như vậy, mà không xen kẽ chúng. Việc thứ tự tuần tự đó khác với thứ tự mà các transaction thực tế đã chạy là hoàn toàn OK [9].

Linearizability

Linearizability là một sự đảm bảo cho các thao tác đọc và ghi của một register (một *đối tượng riêng lẻ*). Nó không nhóm các thao tác lại với nhau thành các transaction, vì vậy nó không ngăn chặn được các vấn đề như write skew liên quan đến nhiều đối tượng (xem mục “Write Skew and Phantoms”). Tuy nhiên, linearizability là một sự đảm bảo về *tính mới* (*recency*): nó yêu cầu rằng nếu một thao tác kết thúc trước khi một thao tác khác bắt đầu, thì thao tác sau đó phải quan sát được một trạng thái ít nhất là mới bằng trạng thái của thao tác trước đó. Serializability không có yêu cầu đó—ví dụ, các stale reads được cho phép bởi serializability [10].

Sequential consistency lại là một thứ khác [8], nhưng chúng ta sẽ không thảo luận về nó ở đây.

Một cơ sở dữ liệu có thể cung cấp cả serializability và linearizability; sự kết hợp này được gọi là *strict serializability* hoặc *strong one-copy serializability* (*strong-ISR*) [11, 12]. Các cơ sở dữ liệu single-node thường có tính linearizable. Với các cơ sở dữ liệu phân tán sử dụng các phương pháp optimistic như SSI (xem “Serializable Snapshot Isolation”), tình huống này phức tạp hơn. Ví dụ, CockroachDB cung cấp serializability và một số đảm bảo về tính mới cho các thao tác đọc, nhưng không phải là strict serializability [13], vì điều này sẽ yêu cầu sự coordination tốn kém giữa các transaction [14]. Ngược lại, Spanner và FoundationDB cung cấp strict serializability [15, 16].

Cũng có thể kết hợp một isolation level yếu hơn với linearizability, hoặc một mô hình consistency yếu hơn với serializability; trên thực tế, mô hình

consistency và isolation level có thể được lựa chọn gần như độc lập với nhau [17, 18].

Dựa vào Linearizability

Trong những trường hợp nào thì linearizability hữu ích? Việc xem điểm số cuối cùng của một trận đấu thể thao có lẽ là một ví dụ phù hợp; một kết quả bị lỗi thời vài giây khó có thể gây ra bất kỳ tác hại thực sự nào trong tình huống này. Tuy nhiên, trong một vài lĩnh vực, linearizability là một yêu cầu quan trọng để làm cho một hệ thống hoạt động chính xác.

Locking và leader election

Một hệ thống sử dụng single-leader replication cần đảm bảo rằng thực sự chỉ có duy nhất một leader, chứ không phải nhiều leader (split brain). Một cách để bầu chọn một leader là sử dụng một lease. Mỗi node khi khởi động đều cố gắng giành được lease, và node nào thành công sẽ trở thành leader [19]. Bất kể cơ chế này được triển khai như thế nào, nó phải có tính linearizable. Không thể để hai node cùng giành được lease tại một thời điểm.

Các dịch vụ coordination như Apache ZooKeeper [20] và etcd thường được sử dụng để triển khai các distributed leases và leader election. Chúng sử dụng các thuật toán consensus để triển khai các thao tác linearizable theo cách fault-tolerant (chúng ta sẽ thảo luận về các thuật toán như vậy ở phần sau của chương này). Có nhiều chi tiết tinh vi liên quan đến việc triển khai leases và leader election một cách chính xác (ví dụ: vấn đề fencing trong mục “Distributed Locks and Leases”), và các thư viện như Apache Curator giúp ích bằng cách cung cấp các recipe ở cấp cao hơn trên nền ZooKeeper. Tuy nhiên, một dịch vụ lưu trữ linearizable là nền tảng cơ bản cho các tác vụ coordination này.

LƯU Ý

Nói một cách chính xác, ZooKeeper cung cấp các thao tác ghi linearizable, nhưng các thao tác đọc có thể bị stale, vì không có sự đảm bảo rằng chúng được phục vụ từ leader hiện tại [20]. etcd kể từ phiên bản 3 cung cấp các thao tác đọc linearizable theo mặc định.

Distributed locking cũng được sử dụng ở mức độ chi tiết hơn nhiều trong một số cơ sở dữ liệu phân tán, chẳng hạn như Oracle Real Application Clusters (RAC) [21]. RAC sử dụng một lock cho mỗi disk page, với nhiều node cùng chia sẻ quyền truy cập vào cùng một hệ thống lưu trữ đĩa. Vì các lock linearizable này nằm trên đường dẫn quan trọng (critical path) của việc thực thi transaction, các deployment của RAC thường có một mạng cluster interconnect chuyên dụng để giao tiếp giữa các database node.

Các ràng buộc và đảm bảo tính duy nhất

Các ràng buộc về tính duy nhất (uniqueness constraints) rất phổ biến trong các cơ sở dữ liệu—ví dụ, một tên người dùng hoặc địa chỉ email phải định danh duy nhất một người dùng, và trong một dịch vụ lưu trữ tệp, không thể có hai tệp có cùng đường dẫn và tên tệp. Nếu bạn muốn thực thi ràng buộc này ngay khi dữ liệu được ghi (sao cho nếu hai người cùng cố gắng tạo một người dùng hoặc một tệp có cùng tên một cách đồng thời, một trong hai người sẽ nhận được lỗi), bạn cần đến linearizability.

Tình huống này tương tự như một lock; khi một người dùng đăng ký dịch vụ của bạn, bạn có thể coi như họ đang giành được một lock cho tên người dùng đã chọn. Thao tác này cũng rất giống với một lệnh CAS nguyên tử (atomic CAS), thiết lập tên người dùng thành ID của người dùng đã chiếm giữ nó, với điều kiện là tên người dùng đó chưa bị chiếm trước đó.

Các vấn đề tương tự cũng phát sinh nếu bạn muốn đảm bảo số dư tài khoản ngân hàng không bao giờ bị âm, hoặc bạn không bán nhiều mặt hàng hơn số lượng hiện có trong kho, hoặc hai người không cùng đặt một chỗ ngồi trên chuyến bay hoặc trong rạp hát một cách đồng thời. Tất cả các ràng buộc này đều yêu cầu một giá trị duy nhất và cập nhật nhất (số dư tài khoản, mức tồn kho, tình trạng ghế ngồi) mà tất cả các node đều thống nhất.

Trong các ứng dụng thực tế, đôi khi việc xử lý các ràng buộc như vậy một cách lỏng lẻo là có thể chấp nhận được—ví dụ, nếu một chuyến bay bị đặt quá chỗ (overbooked), bạn có thể chuyển khách hàng sang một chuyến bay khác và bồi thường cho họ vì sự bất tiện này. Trong những trường hợp như vậy, có thể không cần đến linearizability (chúng ta sẽ thảo luận về các ràng buộc được diễn giải lỏng lẻo như vậy trong mục “Timeliness and Integrity”).

Tuy nhiên, một ràng buộc về tính duy nhất nghiêm ngặt, chẳng hạn như loại bạn thường thấy trong các cơ sở dữ liệu quan hệ, đòi hỏi linearizability. Các loại ràng buộc khác, chẳng hạn như ràng buộc khóa ngoại (foreign-key) hoặc ràng buộc thuộc tính, có thể được thực hiện mà không cần linearizability [22].

Sự phụ thuộc về thời gian giữa các kênh

Có một chi tiết quan trọng cần lưu ý trong Hình 10-1: nếu Aaliyah không hô to tỉ số, Bryce đã không biết rằng kết quả truy vấn của mình là dữ liệu cũ (stale). Anh ấy sẽ chỉ tải lại trang một vài giây sau đó và cuối cùng sẽ thấy tỉ số cuối cùng. Sự vi phạm linearizability chỉ bị phát hiện vì có thêm một kênh truyền thông bổ sung trong hệ thống (tiếng của Aaliyah truyền đến tai Bryce).

Các tình huống tương tự có thể phát sinh trong các hệ thống máy tính. Ví dụ, giả sử trang web của bạn cho phép người dùng tải lên một video, và một tiến trình chạy ngầm sẽ thực hiện transcoding video xuống chất lượng thấp hơn để có thể streaming trên các kết nối internet chậm. Kiến trúc và dataflow của hệ thống này được minh

họa trong Hình 10-5. Bộ transcoder video cần được chỉ thị rõ ràng để thực hiện công việc transcoding, và chỉ thị này được gửi từ web server đến transcoder thông qua một message queue (xem Chương 12). Web server không đưa toàn bộ video vào queue, vì hầu hết các message broker được thiết kế cho các tin nhắn nhỏ, trong khi một video có thể nặng tới nhiều megabyte. Thay vào đó, video trước tiên được ghi vào một dịch vụ lưu trữ tệp, và sau khi việc ghi hoàn tất, chỉ thị gửi tới transcoder mới được đưa vào queue.

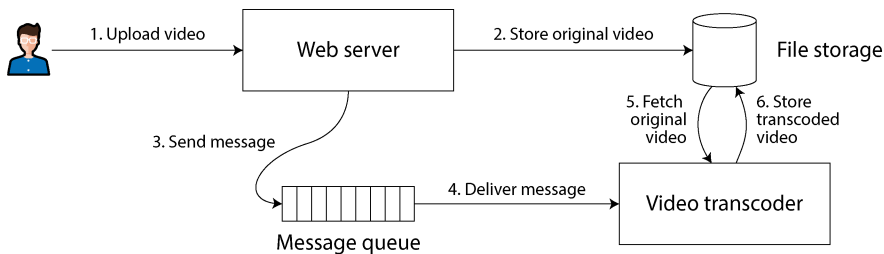


Figure 10-5. Web server và video transcoder giao tiếp thông qua cả storage (lưu trữ) và một message queue, mở ra khả năng xảy ra race condition.

Nếu dịch vụ lưu trữ tệp có tính linearizability, hệ thống này sẽ hoạt động tốt. Nếu nó không có tính linearizability, sẽ có nguy cơ xảy ra race condition: message queue (các bước 3 và 4 trong Hình 10-5) có thể chạy nhanh hơn quá trình replication nội bộ bên trong dịch vụ lưu trữ. Trong trường hợp này, khi bộ transcoder truy xuất video gốc (bước 5), nó có thể thấy một phiên bản cũ của tệp hoặc thậm chí không thấy gì cả. Nếu nó xử lý một phiên bản cũ của video, các video gốc và video đã được transcoded trong lưu trữ tệp sẽ trở nên không nhất quán với nhau một cách vĩnh viễn.

Vấn đề này phát sinh vì có hai kênh truyền thông giữa web server và transcoder: lưu trữ tệp và message queue. Nếu không có sự đảm bảo về tính mới (recency) của linearizability, các race condition giữa hai kênh này hoàn toàn có thể xảy ra. Tình huống này tương tự như trong Hình 10-1, nơi cũng có một race condition giữa hai kênh truyền thông: replication của cơ sở dữ liệu và kênh âm thanh ngoài đời thực giữa miệng của Aaliyah và tai của Bryce.

Một race condition tương tự xảy ra nếu bạn có một ứng dụng di động có thể nhận push notification, và ứng dụng đó truy xuất một số dữ liệu từ máy chủ khi nhận được thông báo. Nếu việc truy xuất dữ liệu có thể trở tới một replica đang bị trễ (lagging), có thể xảy ra tình trạng push notification được truyền đi rất nhanh, nhưng việc truy xuất ngay sau đó lại không thấy được dữ liệu mà thông báo đó đề cập.

Linearizability không phải là cách duy nhất để tránh race condition này, nhưng nó là cách đơn giản nhất để hiểu. Nếu bạn kiểm soát được kênh truyền thông bổ sung (như trong trường hợp của message queue, nhưng không phải trong trường hợp của Aaliyah và Bryce), bạn có thể sử dụng các phương pháp thay thế tương tự như

những gì chúng ta đã thảo luận trong mục “Reading your own writes”, với cái giá phải trả là sự phức tạp tăng thêm.

Triển khai các hệ thống linearizable

Bây giờ chúng ta đã xem xét một vài ví dụ mà linearizability tỏ ra hữu ích, hãy cùng suy nghĩ về cách chúng ta có thể triển khai một hệ thống cung cấp các ngữ nghĩa linearizable.

Vì linearizability về cơ bản có nghĩa là “hành xử như thể chỉ có một bản sao duy nhất của dữ liệu, và tất cả các thao tác trên đó đều là atomic,” nên câu trả lời đơn giản nhất sẽ là thực sự chỉ sử dụng một bản sao duy nhất của dữ liệu. Tuy nhiên, cách tiếp cận đó sẽ không thể chịu lỗi (fault tolerance): nếu node nắm giữ bản sao duy nhất đó gặp lỗi, dữ liệu sẽ bị mất, hoặc ít nhất là không thể truy cập được cho đến khi node đó được khởi động lại.

Hãy xem lại các phương pháp replication từ Chương 6 và xem liệu chúng có thể được làm cho linearizable hay không:

Single-leader replication (có khả năng linearizable)

Trong một hệ thống với single-leader replication, leader nắm giữ bản sao chính của dữ liệu được sử dụng cho các thao tác ghi, và các follower duy trì các bản sao dự phòng của dữ liệu trên các node khác. Miễn là bạn thực hiện tất cả các thao tác đọc và ghi trên leader, chúng có khả năng sẽ đạt được tính linearizability. Tuy nhiên, điều này giả định rằng bạn biết chắc chắn ai là leader. Như đã thảo luận trong mục “Distributed Locks and Leases”, hoàn toàn có khả năng một node nghĩ rằng nó là leader trong khi thực tế không phải vậy—và nếu leader làm tương đó tiếp tục phục vụ các yêu cầu, nó có khả năng vi phạm tính linearizability [23]. Với asynchronous replication, failover thậm chí có thể dẫn đến việc các thao tác ghi đã được commit bị mất, điều này vi phạm cả tính durability lẫn tính linearizability.

Việc sharding một cơ sở dữ liệu single-leader, với mỗi shard có một leader riêng biệt, không ảnh hưởng đến tính linearizability, vì đây chỉ là sự đảm bảo cho một đối tượng duy nhất (single-object guarantee). Các giao dịch xuyên shard (cross-shard transactions) lại là một vấn đề khác (xem mục “Distributed Transactions”).

Các thuật toán consensus (có khả năng linearizable)

Một số thuật toán consensus về bản chất là single-leader replication với cơ chế bầu chọn leader và failover tự động. Chúng được thiết kế cẩn thận để ngăn chặn split brain, cho phép chúng triển khai lưu trữ linearizable một cách an toàn. Ví dụ, ZooKeeper sử dụng thuật toán consensus Zab [24], và etcd sử dụng Raft [25]. Tuy nhiên, chỉ vì một hệ thống sử dụng consensus không đảm bảo rằng tất cả các thao tác trên đó đều là linearizable. Nếu nó cho phép thực hiện các thao tác đọc trên một node mà không kiểm tra xem node đó có còn là leader hay không, kết quả của lần đọc có thể bị cũ nếu một leader mới vừa được bầu chọn.

Multi-leader replication (không linearizable)

Các hệ thống với multi-leader replication thường không linearizable, bởi vì chúng xử lý các thao tác ghi đồng thời trên nhiều node và thực hiện replication chúng một cách bất đồng bộ tới các node khác. Vì lý do này, chúng có thể tạo ra các thao tác ghi xung đột cần phải được giải quyết (xem mục “Dealing with Conflicting Writes”).

Leaderless replication (có lẽ không linearizable)

Đối với các hệ thống sử dụng leaderless replication (kiểu Dynamo; xem mục “Leaderless Replication”), mọi người đôi khi khẳng định rằng bạn có thể đạt được “strong consistency” bằng cách yêu cầu quorum reads và writes ($w + r > n$). Tùy thuộc vào thuật toán chính xác và cách bạn định nghĩa strong consistency, điều này không hoàn toàn đúng.

Các phương pháp giải quyết xung đột LWW dựa trên đồng hồ thời gian thực (ví dụ: trong Cassandra và ScyllaDB) gần như chắc chắn là nonlinearizable, bởi vì các timestamp của đồng hồ không thể được đảm bảo là nhất quán với thứ tự sự kiện thực tế do clock skew (xem mục “Relying on Synchronized Clocks”). Ngay cả với quorum, hành vi nonlinearizable vẫn có thể xảy ra, như được trình bày trong mục tiếp theo.

Về mặt trực giác, có vẻ như quorum reads và writes nên là linearizable trong một mô hình kiểu Dynamo. Tuy nhiên, khi chúng ta có độ trễ mạng biến đổi, các race condition có thể xảy ra, như được minh họa trong Hình 10-6.

Trong Hình 10-6, giá trị ban đầu của x là 0, và một writer client đang cập nhật x thành 1 bằng cách gửi thao tác ghi tới cả ba replica ($n = 3, w = 3$). Đồng thời, client A thực hiện đọc từ một quorum gồm hai node ($r = 2$) và thấy giá trị mới là 1 trên một node và giá trị cũ là 0 trên node còn lại. Cũng đồng thời với thao tác ghi, client B thực hiện đọc từ một quorum khác gồm hai node và nhận lại giá trị cũ là 0 từ cả hai.

Điều kiện quorum đã được đáp ứng ($w + r > n$), nhưng việc thực thi này dù sao vẫn không linearizable. Yêu cầu của B bắt đầu sau khi yêu cầu của A hoàn tất, nhưng B lại trả về giá trị cũ trong khi A trả về giá trị mới. (Đây lại là tình huống Aaliyah và Bryce từ Hình 10-1.)

Có thể làm cho các quorum kiểu Dynamo trở nên linearizable, nhưng phải đánh đổi (trade-off) bằng việc giảm hiệu năng. Một reader phải thực hiện read repair một cách đồng bộ (xem mục “Catching up on missed writes”) trước khi trả kết quả về cho ứng dụng [26]. Ngoài ra, trước khi ghi, một writer phải đọc trạng thái mới nhất của một quorum các node để lấy timestamp mới nhất của bất kỳ thao tác ghi trước đó và đảm bảo rằng thao tác ghi mới có timestamp lớn hơn [27, 28]. Tuy nhiên, Riak không thực hiện read repair đồng bộ vì tổn thất về hiệu năng. Cassandra có đợi read repair hoàn tất đối với các quorum reads [29], nhưng nó mất tính linearizability do việc sử dụng đồng hồ thời gian thực cho các timestamp.

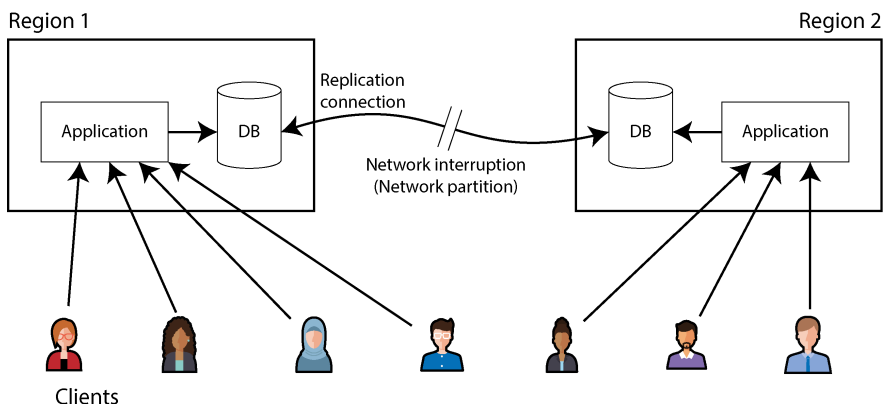


Figure 10-7. Hình 10.1: Một sự gián đoạn mạng buộc phải lựa chọn giữa *linearizability* và *availability*

Với một cơ sở dữ liệu multi-leader, mỗi vùng có thể tiếp tục hoạt động bình thường. Vì các thao tác ghi từ một vùng được replication bất đồng bộ sang vùng còn lại, các thao tác ghi này đơn giản là được xếp hàng và trao đổi khi kết nối mạng được khôi phục.

Mặt khác, nếu sử dụng single-leader replication, leader phải nằm ở một trong các vùng. Mọi thao tác ghi và mọi thao tác đọc linearizable phải được gửi đến leader. Do đó, đối với bất kỳ client nào kết nối với một vùng follower, các yêu cầu đọc và ghi đó phải được gửi đồng bộ qua mạng đến vùng leader.

Nếu kết nối mạng giữa các vùng bị gián đoạn trong thiết lập single-leader, các client kết nối với các vùng follower không thể liên lạc với leader, vì vậy chúng không thể thực hiện bất kỳ thao tác ghi nào vào cơ sở dữ liệu cũng như không thể thực hiện các thao tác đọc linearizable. Chúng vẫn có thể thực hiện các thao tác đọc từ follower, nhưng dữ liệu có thể bị cũ (nonlinearizable). Nếu ứng dụng yêu cầu các thao tác đọc và ghi linearizable, việc gián đoạn mạng sẽ khiến ứng dụng trở nên không khả dụng tại các vùng không thể liên lạc với leader.

Nếu các client có thể kết nối trực tiếp với vùng leader, đây không phải là vấn đề, vì ứng dụng vẫn tiếp tục hoạt động bình thường tại đó. Nhưng các client chỉ có thể tiếp cận vùng follower sẽ gặp phải tình trạng gián đoạn cho đến khi liên kết mạng được sửa chữa.

CAP theorem

Vấn đề này không chỉ là hệ quả của single-leader và multi-leader replication. Bất kỳ cơ sở dữ liệu linearizable nào cũng gặp vấn đề này, bất kể nó được triển khai như thế nào. Vấn đề này cũng không đặc thù cho các deployment đa vùng, mà có thể xảy ra

trên bất kỳ mạng không tin cậy nào, ngay cả trong cùng một vùng. Sự đánh đổi (trade-off) như sau:

- Nếu ứng dụng của bạn *yêu cầu* tính linearizability, và một số replica bị ngắt kết nối với các replica khác do vấn đề mạng, những replica đó sẽ tạm thời không thể xử lý các yêu cầu: chúng phải chờ cho đến khi vấn đề mạng được khắc phục hoặc trả về một lỗi (dù theo cách nào, chúng cũng trở nên *không khả dụng*). Lựa chọn này đôi khi được gọi là *CP* (*consistent under network partitions* - nhất quán khi có phân mảnh mạng).
- Nếu ứng dụng của bạn *không yêu cầu* tính linearizability, nó có thể được viết theo cách mà mỗi replica có thể xử lý các yêu cầu một cách độc lập, ngay cả khi nó bị ngắt kết nối với các replica khác (ví dụ: multi-leader). Trong trường hợp này, ứng dụng có thể duy trì tính *khả dụng* khi đối mặt với vấn đề mạng, nhưng hành vi của nó không phải là linearizable. Lựa chọn này được gọi là *AP* (*available under network partitions* - khả dụng khi có phân mảnh mạng).

Do đó, các ứng dụng không yêu cầu tính linearizability có thể chịu đựng các vấn đề mạng tốt hơn. Kiến thức này được biết đến rộng rãi dưới tên gọi *CAP theorem* [31, 32, 33, 34], được đặt tên bởi Eric Brewer vào năm 2000, mặc dù sự đánh đổi (trade-off) này đã được các nhà thiết kế cơ sở dữ liệu phân tán biết đến từ những năm 1970 [35, 36, 37].

CAP ban đầu được đề xuất như một quy tắc kinh nghiệm (rule of thumb) mà không có các định nghĩa chính xác, với mục tiêu bắt đầu một cuộc thảo luận về các sự đánh đổi (trade-off) trong cơ sở dữ liệu. Vào thời điểm đó, nhiều cơ sở dữ liệu phân tán tập trung vào việc cung cấp các ngữ nghĩa linearizable trên một cluster các máy chủ với bộ nhớ lưu trữ dùng chung (shared storage) [21], và CAP đã khuyến khích các kỹ sư cơ sở dữ liệu khám phá một không gian thiết kế rộng hơn của các hệ thống distributed shared-nothing, vốn phù hợp hơn để triển khai các dịch vụ web quy mô lớn [38]. CAP xứng đáng được ghi nhận vì sự chuyển dịch văn hóa này—nó đã giúp kích hoạt phong trào NoSQL, một sự bùng nổ của các công nghệ cơ sở dữ liệu mới vào khoảng giữa những năm 2000.

CAP theorem khi được định nghĩa một cách chính thức [32] có phạm vi rất hẹp. Nó chỉ xem xét một mô hình nhất quán duy nhất (cụ thể là linearizability) và một loại lỗi duy nhất (phân mảnh mạng - network partitions, mà theo dữ liệu từ Google là nguyên nhân gây ra chưa đến 8% các sự cố [39]). Nó không nói gì về độ trễ mạng, các node bị chết, hoặc các sự đánh đổi (trade-off) khác. Do đó, mặc dù CAP có ảnh hưởng lớn về mặt lịch sử, nó có rất ít giá trị thực tế trong việc thiết kế các hệ thống [4, 45].

Đã có những nỗ lực nhằm tổng quát hóa CAP. Ví dụ, *nguyên lý PACELC* quan sát thấy rằng các nhà thiết kế hệ thống đôi khi cũng có thể chọn làm yếu đi consistency nhằm giảm latency khi mạng vẫn đang hoạt động tốt [40, 46, 47]. Do đó, trong quá

trình xảy ra network partition (P), chúng ta cần chọn giữa availability (A) và consistency (C); nếu không (E), tức là khi không có partition, chúng ta có thể chọn giữa latency thấp (L) và consistency (C). Tuy nhiên, định nghĩa này kế thừa một vài vấn đề của CAP, chẳng hạn như các định nghĩa phản trực giác về consistency và availability.

Có nhiều kết quả về tính bất khả thi thú vị hơn nữa trong các distributed systems [41], và CAP hiện đã bị thay thế bởi các kết quả chính xác hơn [42, 43], vì vậy ngày nay nó chủ yếu chỉ còn mang ý nghĩa lịch sử.

Định lý CAP không hữu ích

CAP đôi khi được trình bày dưới dạng *consistency, availability, partition tolerance: chọn hai trong ba*. Thật không may, cách trình bày này gây hiểu lầm [34]. Bởi vì network partition là một loại fault, chúng không phải là thứ bạn chọn mà là thứ sẽ xảy ra dù bạn muốn hay không. Cách duy nhất để bạn đảm bảo không có network partition là không có mạng—tức là chỉ có một replica duy nhất—nhưng khi đó bạn cũng không có high availability.

Tại những thời điểm khi mạng hoạt động chính xác, một hệ thống có thể cung cấp cả consistency (linearizability) và availability. Khi một network fault xảy ra, bạn phải chọn một trong hai. Do đó, một cách diễn đạt tốt hơn về CAP sẽ là *hoặc là consistent hoặc là available khi bị partition* [44]. Một mạng tin cậy hơn cần phải đưa ra lựa chọn này ít thường xuyên hơn, nhưng tại một thời điểm nào đó, sự lựa chọn là không thể tránh khỏi.

Sơ đồ phân loại CP/AP có một vài thiếu sót khác [4]. *Consistency* được hình thức hóa thành linearizability (định lý không nói gì về các mô hình consistency yếu hơn), và việc hình thức hóa *availability* [32] không khớp với ý nghĩa thông thường của thuật ngữ này [45]. Nhiều hệ thống highly available (fault-tolerant) thực tế không đáp ứng định nghĩa đặc thù về availability của CAP. Hơn nữa, một số nhà thiết kế hệ thống chọn (vì lý do chính đáng) để không cung cấp cả linearizability lẫn dạng availability mà định lý CAP giả định, vì vậy những hệ thống đó không phải là CP cũng không phải là AP [46, 47].

Tóm lại, có rất nhiều sự hiểu lầm và nhầm lẫn xung quanh CAP, và nó không giúp chúng ta hiểu các hệ thống tốt hơn, vì vậy tốt nhất là không nên sa đà vào nó.

Linearizability và độ trễ mạng

Mặc dù linearizability là một sự đảm bảo hữu ích, nhưng đáng ngạc nhiên là rất ít hệ thống có tính linearizable trong thực tế. Ví dụ, ngay cả RAM trên một CPU multi-

core hiện đại cũng không có tính linearizable [48]. Nếu một thread chạy trên một CPU core ghi vào một địa chỉ bộ nhớ, và một thread trên một CPU core khác đọc cùng địa chỉ đó ngay sau đó, thì không có gì đảm bảo rằng nó sẽ đọc được giá trị đã được ghi bởi thread đầu tiên (trừ khi sử dụng một *memory barrier* hoặc *fence* [49]).

Lý do cho hành vi này là vì mỗi CPU core đều có memory cache và store buffer riêng. Các thao tác đọc được phục vụ từ cache theo mặc định, và bất kỳ thay đổi nào cũng được ghi không đồng bộ ra main memory. Vì việc truy cập dữ liệu trong cache nhanh hơn nhiều so với việc truy cập main memory [50], tính năng này là thiết yếu để đạt được hiệu năng tốt trên các CPU hiện đại. Tuy nhiên, điều đó có nghĩa là hiện có nhiều bản sao của dữ liệu (một bản trong main memory, và có thể thêm vài bản nữa trong các cache khác nhau), và các bản sao này được cập nhật không đồng bộ, do đó tính linearizability bị mất đi.

Tại sao lại thực hiện sự đánh đổi (trade-off) này? Việc sử dụng định lý CAP để biện minh cho mô hình memory consistency của multi-core là không hợp lý. Trong phạm vi một máy tính, chúng ta thường giả định sự giao tiếp là tin cậy, và chúng ta không kỳ vọng một CPU core có thể tiếp tục hoạt động bình thường nếu nó bị ngắt kết nối với phần còn lại của máy tính. Lý do của việc từ bỏ linearizability là vì *hiệu năng*, chứ không phải vì fault tolerance [46].

Điều tương tự cũng đúng với nhiều cơ sở dữ liệu phân tán chọn không cung cấp các đảm bảo về linearizability: chúng làm vậy chủ yếu để tăng hiệu năng, chứ không hẳn vì fault tolerance [40]. Các hệ thống linearizable có xu hướng có latency cao hơn—và điều này luôn đúng, không chỉ trong quá trình xảy ra lỗi mạng.

Liệu chúng ta có thể tìm thấy một cách triển khai hiệu quả hơn cho lưu trữ linearizable không? Có vẻ như câu trả lời là không. Attiya và Welch [51] đã chứng minh rằng nếu bạn muốn linearizability, thời gian phản hồi của các yêu cầu read và write sẽ ít nhất tỉ lệ thuận với sự bất định của các độ trễ trong mạng. Trong một mạng có độ trễ biến đổi cao, giống như hầu hết các mạng máy tính (xem mục “Timeouts and Unbounded Delays”), thời gian phản hồi của các thao tác read và write linearizable chắc chắn sẽ cao. Không tồn tại một thuật toán nhanh hơn cho linearizability, nhưng các mô hình consistency yếu hơn có thể nhanh hơn nhiều, vì vậy sự đánh đổi (trade-off) này rất quan trọng đối với các hệ thống nhạy cảm với latency. Trong Chương 13, chúng ta sẽ thảo luận về một số phương pháp để tránh linearizability mà không làm mất đi tính đúng đắn.

ID Generators và Logical Clocks

Trong nhiều ứng dụng, bạn cần gán một loại ID duy nhất nào đó cho các bản ghi cơ sở dữ liệu khi chúng được tạo, điều này cung cấp cho bạn một primary key để tham chiếu đến các bản ghi đó. Trong các cơ sở dữ liệu single-node, việc sử dụng một số nguyên tự tăng (autoincrementing integer) là phổ biến, với ưu điểm là nó chỉ cần 64

bit để lưu trữ (hoặc thậm chí 32 bit, nếu bạn chắc chắn rằng mình sẽ không bao giờ có quá 4 tỷ bản ghi, nhưng điều đó khá rủi ro).

Một ưu điểm khác của các ID tự tăng là thứ tự của các ID cho bạn biết thứ tự mà các bản ghi được tạo ra. Ví dụ, Hình 10-8 hiển thị một ứng dụng chat gán các ID tự tăng cho các tin nhắn chat khi chúng được đăng. Sau đó, bạn có thể hiển thị các tin nhắn theo thứ tự ID tăng dần, và các luồng chat kết quả sẽ hợp lý: Aaliyah đăng một câu hỏi được gán ID 1, và câu trả lời của Bryce cho câu hỏi đó được gán một ID lớn hơn — cụ thể là 3.

Bộ tạo ID single-node này là một ví dụ khác về một hệ thống linearizable. Mỗi yêu cầu lấy ID là một thao tác tăng một biến đếm một cách nguyên tử (atomically) và trả về giá trị cũ của biến đếm (một thao tác *fetch-and-add*); tính linearizability đảm bảo rằng nếu việc đăng tin nhắn của Aaliyah hoàn tất trước khi việc đăng tin của Bryce bắt đầu, thì ID của tin nhắn của Bryce phải lớn hơn ID của Aaliyah. Các tin nhắn của Aaliyah và Caleb trong Hình 10-8 là concurrent, vì vậy tính linearizability không chỉ định cách các ID của chúng phải được sắp xếp, miễn là chúng là duy nhất.

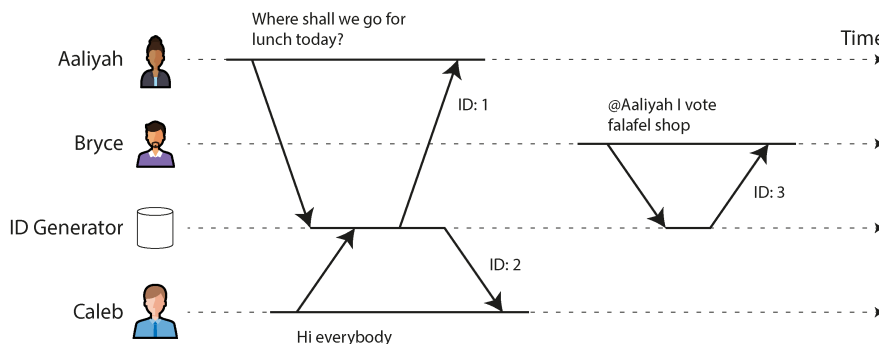


Figure 10-8. Một bộ tạo ID thực hiện gán các ID số nguyên tự tăng cho các tin nhắn trong một ứng dụng chat

Việc triển khai một bộ tạo ID đơn node trong bộ nhớ (in-memory) là khá dễ dàng. Bạn có thể sử dụng chỉ thị tăng nguyên tử (atomic increment) được cung cấp bởi CPU, cho phép nhiều thread tăng cùng một bộ đếm một cách an toàn. Sẽ cần thêm một chút công sức để làm cho bộ đếm có tính bền vững (persistent), sao cho node có thể gặp sự cố và khởi động lại mà không làm reset giá trị bộ đếm, điều vốn sẽ dẫn đến việc tạo ra các ID trùng lặp. Tuy nhiên, các vấn đề thực sự nằm ở sau đây:

- Một bộ tạo ID đơn node không có khả năng chịu lỗi (fault-tolerant) vì node đó là một điểm lỗi duy nhất (single point of failure).
- Nó sẽ chậm nếu bạn muốn tạo một bản ghi ở một vùng (region) khác, vì bạn có khả năng phải thực hiện một vòng truyền tin (round trip) tới nửa kia của hành tinh chỉ để lấy một ID.

- Node duy nhất đó có thể trở thành một nút thắt cổ chai (bottleneck) nếu bạn có throughput ghi cao.

Bạn có thể xem xét các lựa chọn thay thế khác nhau cho bộ tạo ID:

Phân mảnh (sharding) việc cấp phát ID

Bạn có thể có nhiều node thực hiện cấp phát ID—ví dụ, một node chỉ tạo ra các số chẵn và một node chỉ tạo ra các số lẻ. Nhìn chung, bạn có thể dành riêng một số bit trong ID để chứa một số shard. Những ID này vẫn gọn nhẹ, nhưng bạn sẽ mất đi tính chất thứ tự—ví dụ, nếu bạn có các tin nhắn chat với ID là 16 và 17, bạn không biết liệu tin nhắn 16 có thực sự được gửi trước hay không, vì các ID được cấp phát bởi các node khác nhau, và một node có thể đã chạy nhanh hơn node kia.

Các khối ID được cấp phát trước (preallocated)

Thay vì các ID riêng lẻ, bộ tạo ID đơn node có thể cấp phát theo các khối ID. Ví dụ, node A có thể nhận khối ID từ 1 đến 1.000, và node B có thể nhận khối từ 1.001 đến 2.000. Sau đó, mỗi node có thể độc lập cấp phát các ID từ khối của nó, và yêu cầu một khối mới từ bộ tạo ID khi nguồn cung số thứ tự của nó bắt đầu cạn kiệt. Tuy nhiên, cơ chế này cũng không đảm bảo thứ tự chính xác. Có thể xảy ra trường hợp một tin nhắn được cấp một ID trong phạm vi từ 1.001 đến 2.000 và một tin nhắn gửi sau đó lại được cấp một ID trong phạm vi từ 1 đến 1.000 nếu ID đó được cấp bởi một node khác.

UUID ngẫu nhiên

Bạn có thể sử dụng các *mã định danh duy nhất toàn cầu* (universally unique identifiers - UUIDs), còn được gọi là *mã định danh duy nhất toàn cầu* (globally unique identifiers - GUIDs). Chúng có ưu điểm lớn là có thể được tạo cục bộ trên bất kỳ node nào mà không cần giao tiếp, nhưng chúng yêu cầu nhiều không gian lưu trữ hơn (128 bit). UUID có vài phiên bản; đơn giản nhất là phiên bản 4, về cơ bản là một số ngẫu nhiên dài đến mức rất khó có khả năng hai node lại chọn cùng một số. Thật không may, thứ tự của các ID như vậy cũng là ngẫu nhiên, vì vậy việc so sánh hai ID sẽ không cho bạn biết ID nào là mới hơn.

Dùng timestamp của đồng hồ hệ thống để tạo tính duy nhất

Nếu đồng hồ thời gian trong ngày của các node được giữ chính xác tương đối bằng cách sử dụng NTP, bạn có thể tạo ID bằng cách đặt một timestamp từ đồng hồ này vào các bit có ý nghĩa cao nhất (most significant bits) và lấp đầy các bit còn lại bằng thông tin bổ sung để đảm bảo ID là duy nhất ngay cả khi timestamp không duy nhất—ví dụ, một số shard và một số thứ tự tăng dần theo từng shard, hoặc một giá trị ngẫu nhiên dài. Cách tiếp cận này được sử dụng trong UUID phiên bản 7 [52], Snowflake của X [53], ULIDs [54], bộ tạo Flake ID của Hazelcast, MongoDB ObjectIDs, và nhiều cơ chế tương tự khác [52]. Bạn có thể

triển khai các bộ tạo ID này trong mã ứng dụng hoặc bên trong một cơ sở dữ liệu [55].

Tất cả các cơ chế này đều tạo ra các ID duy nhất (ít nhất là với xác suất đủ cao để các va chạm là cực kỳ hiếm), nhưng chúng có các đảm bảo về thứ tự cho ID yếu hơn nhiều so với cơ chế tự động tăng (autoincrementing) đơn node.

Như đã thảo luận trong mục “Timestamps để sắp xếp các sự kiện”, các timestamp của đồng hồ hệ thống (wall-clock timestamps) cùng lắm chỉ có thể cung cấp một thứ tự xấp xỉ. Nếu một thao tác ghi sớm hơn nhận được timestamp từ một đồng hồ chạy nhanh hơn một chút và timestamp của một thao tác ghi muộn hơn lại đến từ một đồng hồ chạy chậm hơn một chút, thứ tự timestamp có thể không nhất quán với thứ tự mà các sự kiện thực sự xảy ra. Với các bước nhảy thời gian do sử dụng đồng hồ không đơn điệu (nonmonotonic clock), ngay cả các timestamp được tạo bởi một node duy nhất cũng có thể bị sắp xếp sai. Do đó, các bộ tạo ID dựa trên thời gian thực của đồng hồ hệ thống khó có khả năng đạt được linearizability (linearizable).

Bạn có thể giảm thiểu sự không nhất quán về thứ tự như vậy bằng cách dựa vào việc đồng bộ hóa đồng hồ với độ chính xác cao, sử dụng đồng hồ nguyên tử hoặc bộ thu GPS. Nhưng cũng sẽ thật tốt nếu chúng ta có thể tạo ra các ID duy nhất và được sắp xếp đúng thứ tự mà không cần dựa vào phần cứng đặc biệt. Tiếp theo, chúng ta sẽ tìm hiểu về một loại đồng hồ cho phép thực hiện điều đó.

Logical Clocks

Trong mục “Unreliable Clocks”, chúng ta đã thảo luận về đồng hồ thời gian thực (time-of-day clocks) và đồng hồ đơn điệu (monotonic clocks). Cả hai đều là *physical clocks*: các thiết bị phần cứng đo lường sự trôi qua của thời gian (giây, mili giây, micro giây, v.v.).

Trong các distributed systems, việc sử dụng một loại đồng hồ khác gọi là *logical clock* (đồng hồ logic) cũng rất phổ biến. Trái ngược với một physical clock, một logical clock là một thuật toán đếm các sự kiện đã xảy ra. Do đó, một timestamp từ một logical clock không cho bạn biết bây giờ là mấy giờ, nhưng bạn *có thể* so sánh hai timestamp từ một logical clock để biết cái nào xảy ra trước và cái nào xảy ra sau.

Các yêu cầu chung đối với một logical clock như sau:

- Các timestamp của nó có kích thước nhỏ gọn (chỉ vài byte) và duy nhất.
- Bạn có thể so sánh bất kỳ hai timestamp nào và xác định cái nào xảy ra trước (tức là, chúng được *totally ordered*).
- Thứ tự của các timestamp *consistent with causality* (nhất quán với causality). Nghĩa là, nếu thao tác A xảy ra trước thao tác B, thì timestamp của A sẽ nhỏ hơn timestamp của B. (Chúng ta đã thảo luận về causality trước đó trong mục “The happens-before relation and concurrency”).

Một bộ tạo ID trên một node duy nhất đáp ứng được các yêu cầu này, nhưng các bộ tạo ID phân tán mà chúng ta vừa thảo luận thì không đáp ứng được yêu cầu về thứ tự nhân quả.

Lamport timestamps

May mắn thay, một phương pháp đơn giản để tạo ra các logical timestamps có tính nhất quán với causality, và bạn có thể sử dụng nó như một bộ tạo ID phân tán. Nó được gọi là *Lamport clock*, được đề xuất vào năm 1978 bởi Leslie Lamport [56], trong một tài liệu hiện là một trong những bài báo được trích dẫn nhiều nhất trong lĩnh vực distributed systems.

Mặc dù Lamport clocks cung cấp một total ordering, nhưng chúng *không* cung cấp linearizability—nghĩa là, chúng không phải là một cách để đảm bảo rằng một giá trị luôn là mới nhất. Chúng chỉ đơn thuần là một cách gán ID cho các sự kiện sao cho nếu sự kiện A xảy ra trước sự kiện B, thì ID của A sẽ nhỏ hơn ID của B.

Hình 10-9 cho thấy cách một Lamport clock sẽ hoạt động trong ví dụ về chat ở Hình 10-8. Mỗi node có một định danh duy nhất, trong Hình 10-9 là tên Aaliyah, Bryce, hoặc Caleb, nhưng trong thực tế có thể là một UUID ngẫu nhiên hoặc thứ gì đó tương tự. Mỗi node cũng lưu giữ một bộ đếm về số lượng các thao tác mà nó đã xử lý. Một Lamport timestamp khi đó đơn giản là một cặp (*counter*, *node ID*). Hai node đôi khi có thể có cùng giá trị counter, nhưng bằng cách đưa node ID vào timestamp, mỗi timestamp sẽ trở nên duy nhất.

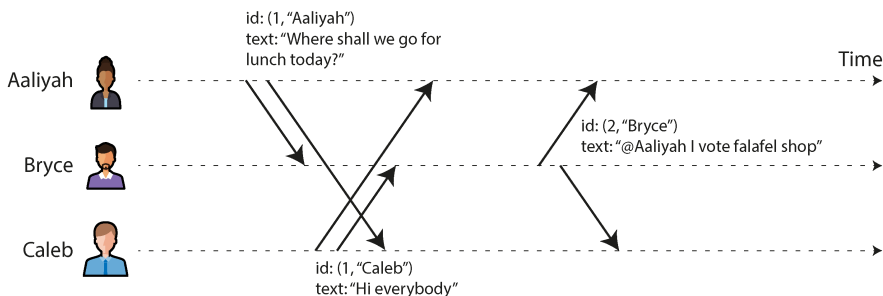


Figure 10-9. *Lamport timestamps cung cấp một thứ tự toàn phần (total ordering) nhất quán với causality.*

Mỗi khi một node tạo ra một timestamp, nó sẽ tăng giá trị counter của mình lên và sử dụng giá trị mới đó. Mỗi khi một node thấy một timestamp từ một node khác, nếu giá trị counter trong timestamp đó lớn hơn giá trị counter cục bộ của nó, nó sẽ tăng counter cục bộ của mình lên để khớp với giá trị trong timestamp đó.

Trong Hình 10-9, Aaliyah vẫn chưa thấy tin nhắn của Caleb khi cô ấy đăng tin nhắn của chính mình, và ngược lại. Giả sử cả hai người dùng đều bắt đầu với giá trị

counter ban đầu là 0, do đó cả hai đều tăng counter cục bộ của mình lên và đính kèm giá trị counter mới là 1 vào tin nhắn. Khi Bryce nhận được những tin nhắn đó, anh ấy tăng giá trị counter cục bộ của mình lên 1. Cuối cùng, Bryce gửi một phản hồi cho tin nhắn của Aaliyah, tăng counter cục bộ của mình và đính kèm giá trị mới là 2 vào tin nhắn.

Để so sánh hai Lamport timestamp, trước tiên ta so sánh giá trị counter của chúng—ví dụ, (2, "Bryce") lớn hơn (1, "Aaliyah") và cũng lớn hơn (1, "Caleb"). Nếu hai timestamp có cùng giá trị counter, chúng ta sau đó sẽ so sánh node ID của chúng, sử dụng phép so sánh chuỗi lexicographic thông thường. Do đó, thứ tự timestamp trong ví dụ này là $(1, \text{"Aaliyah"}) < (1, \text{"Caleb"}) < (2, \text{"Bryce"})$.

Hybrid logical clocks

Lamport timestamp rất tốt trong việc ghi lại thứ tự mà các sự kiện đã xảy ra, nhưng chúng có một số hạn chế:

- Vì chúng không có mối liên hệ trực tiếp với thời gian vật lý, bạn không thể sử dụng chúng để tìm, chẳng hạn như, tất cả các tin nhắn đã được đăng vào một ngày cụ thể; bạn sẽ cần phải lưu trữ thời gian vật lý một cách riêng biệt.
- Nếu hai node không bao giờ giao tiếp với nhau, việc tăng counter của một node sẽ không bao giờ được phản ánh trong counter của node kia. Kết quả là, các sự kiện được tạo ra vào cùng một thời điểm trên các node khác nhau có thể có các giá trị counter khác biệt rất lớn.

Một *hybrid logical clock* kết hợp các ưu điểm của đồng hồ thời gian vật lý (physical time-of-day clocks) với các đảm bảo về thứ tự của Lamport clock [57]. Giống như một đồng hồ vật lý, nó đếm giây hoặc micro giây. Giống như một Lamport clock, khi một node thấy một timestamp từ một node khác lớn hơn giá trị clock cục bộ của nó, nó sẽ đẩy giá trị cục bộ của chính mình tiến về phía trước để khớp với timestamp của node kia. Kết quả là, nếu clock của một node chạy nhanh, các node khác cũng sẽ tương tự đẩy clock của chúng tiến về phía trước khi chúng giao tiếp.

Mỗi khi một timestamp từ một hybrid logical clock được tạo ra, nó cũng được tăng lên, điều này đảm bảo rằng clock tiến về phía trước một cách monotonic ngay cả khi clock vật lý cơ sở nhảy ngược lại—ví dụ, do các điều chỉnh của NTP. Do đó, hybrid logical clock có thể chạy nhanh hơn một chút so với clock vật lý cơ sở. Các chi tiết của thuật toán đảm bảo rằng sự sai lệch này được giữ ở mức nhỏ nhất có thể.

Kết quả là, bạn có thể coi một timestamp từ một hybrid logical clock gần giống như một timestamp từ một đồng hồ thời gian vật lý thông thường, với đặc tính bổ sung là thứ tự của nó nhất quán với quan hệ happens-before. Nó không phụ thuộc vào bất kỳ phần cứng đặc biệt nào và chỉ yêu cầu các clock được đồng bộ hóa tương đối. Ví dụ, CockroachDB sử dụng hybrid logical clocks.

Lamport/hybrid logical clocks so với vector clocks

Trong mục "Multiversion concurrency control", chúng ta đã thảo luận về cách snapshot isolation thường được triển khai: về cơ bản, bằng cách cấp cho mỗi transaction một transaction ID, và cho phép mỗi transaction thấy các write được thực hiện bởi các transaction có ID thấp hơn nhưng làm cho các write bởi các transaction có ID cao hơn trở nên không thể nhìn thấy. Lamport clock và hybrid logical clock là một cách tốt để tạo ra các transaction ID này vì chúng đảm bảo rằng snapshot nhất quán với causality [58].

Khi nhiều timestamp được tạo ra đồng thời, các thuật toán này sắp xếp chúng một cách tùy ý. Điều này có nghĩa là khi bạn nhìn vào hai timestamp, bạn thường không thể biết liệu chúng được tạo ra đồng thời hay một cái xảy ra trước cái kia. (Trong Hình 10-9, bạn thực sự có thể biết rằng tin nhắn của Aaliyah và Caleb chắc chắn đã diễn ra đồng thời, vì chúng có cùng giá trị counter; tuy nhiên, khi các giá trị counter khác nhau, bạn không thể biết liệu chúng có diễn ra đồng thời hay không.)

Nếu bạn muốn có khả năng xác định thời điểm các bản ghi được tạo ra đồng thời, bạn cần một thuật toán khác, chẳng hạn như *vector clock*. Các vector clock duy trì một bộ đếm cho mỗi node và lưu trữ tất cả các giá trị bộ đếm đó cùng với mỗi lần ghi. Nếu lần ghi A có giá trị bộ đếm cao hơn B đối với một node, và lần ghi B có giá trị bộ đếm cao hơn A đối với một node khác, thì A và B chắc chắn là đồng thời (xem mục "Detecting Concurrent Writes"). Nhược điểm là các timestamp từ một vector clock chiếm nhiều không gian hơn nhiều so với các timestamp khác mà chúng ta đã thảo luận—có khả năng là một số nguyên cho mỗi node trong hệ thống.

Các bộ tạo ID có tính linearizability

Mặc dù Lamport clocks và hybrid logical clocks cung cấp các đảm bảo về thứ tự hữu ích, nhưng thứ tự đó vẫn yếu hơn so với bộ tạo ID single-node có tính linearizability mà chúng ta đã thảo luận trước đó. Hãy nhớ rằng linearizability yêu cầu rằng nếu request A hoàn tất trước khi request B bắt đầu, thì B phải có ID cao hơn, ngay cả khi A và B chưa bao giờ giao tiếp với nhau. Ngược lại, Lamport clocks chỉ có thể đảm bảo rằng một node tạo ra các timestamp lớn hơn bất kỳ timestamp nào khác mà node đó đã thấy; không có đảm bảo nào như vậy có thể được đưa ra đối với các timestamp mà nó chưa thấy.

Hình 10-10 cho thấy một ID generator không có tính linearizable có thể gây ra các vấn đề như thế nào. Hãy tưởng tượng trên một trang web mạng xã hội, người dùng A muốn chia sẻ riêng tư một bức ảnh gây xấu hổ với bạn bè của mình. Tài khoản của người dùng A ban đầu ở chế độ công khai, nhưng bằng cách sử dụng laptop, người dùng này thay đổi cài đặt tài khoản sang chế độ riêng tư. Sau đó, họ sử dụng điện thoại để tải ảnh lên. Vì người dùng A đã thực hiện các cập nhật này theo trình tự, họ

có thể kỳ vọng một cách hợp lý rằng việc tải ảnh lên sẽ tuân theo các quyền hạn tài khoản mới đã được giới hạn. Tuy nhiên, như hình vẽ cho thấy, điều này không nhất thiết là như vậy.

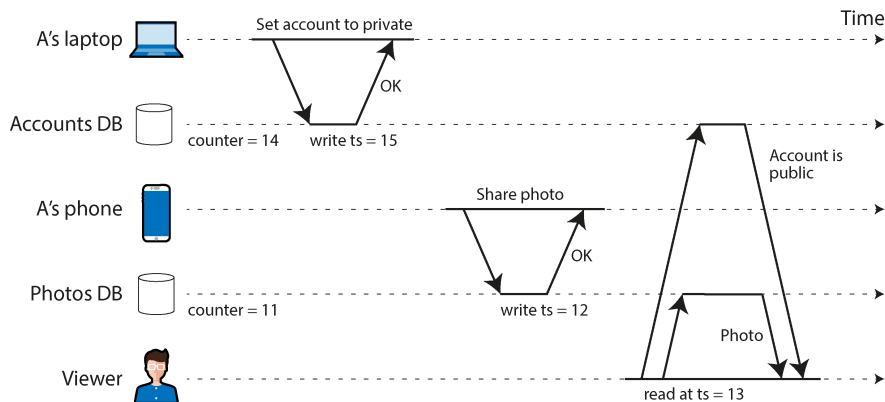


Figure 10-10. Người dùng A đầu tiên thiết lập tài khoản thành riêng tư, sau đó chia sẻ một bức ảnh. Với một bộ tạo ID không có tính linearizable, một người xem không được phép có thể nhìn thấy bức ảnh đó.

Quyền hạn tài khoản và ảnh được lưu trữ trong hai cơ sở dữ liệu riêng biệt (hoặc các shard riêng biệt của cùng một cơ sở dữ liệu), và hãy giả sử rằng chúng sử dụng Lamport clock hoặc hybrid logical clock để gán một timestamp cho mỗi thao tác ghi. Vì cơ sở dữ liệu ảnh không đọc từ cơ sở dữ liệu tài khoản, nên có khả năng bộ đếm cục bộ trong cơ sở dữ liệu ảnh đang chậm hơn một chút, và do đó việc tải ảnh lên được gán một timestamp thấp hơn so với việc cập nhật cài đặt tài khoản.

Bây giờ, giả sử rằng một người xem (người không phải là bạn của A) đang xem hồ sơ của A, và thao tác đọc của họ sử dụng một implementation MVCC của snapshot isolation. Có thể xảy ra trường hợp thao tác đọc của người xem có một timestamp lớn hơn timestamp của việc tải ảnh lên, nhưng lại nhỏ hơn timestamp của việc cập nhật cài đặt tài khoản. Kết quả là, hệ thống sẽ xác định rằng tài khoản vẫn ở chế độ công khai tại thời điểm đọc, và do đó hiển thị cho người xem bức ảnh gây xấu hổ mà lẽ ra họ không nên thấy.

Bạn có thể hình dung một vài cách khả thi để khắc phục vấn đề này. Có lẽ cơ sở dữ liệu ảnh nên đọc trạng thái tài khoản của người dùng trước khi thực hiện thao tác ghi, nhưng việc quên một kiểm tra như vậy là rất dễ xảy ra. Nếu các hành động của A được thực hiện trên cùng một thiết bị, có lẽ ứng dụng trên thiết bị đó có thể theo dõi timestamp mới nhất của các thao tác ghi của người dùng đó—nhưng nếu người dùng sử dụng cả laptop và điện thoại, như trong ví dụ này, thì điều đó không hề dễ dàng. Giải pháp đơn giản nhất trong trường hợp này là sử dụng một bộ tạo ID có tính linearizable, điều này sẽ đảm bảo rằng việc tải ảnh lên được gán một ID lớn hơn so với việc thay đổi quyền hạn tài khoản.

Triển khai một bộ tạo ID có tính linearizable

Cách đơn giản nhất để đảm bảo việc gán ID có tính linearizable là thực sự sử dụng một node duy nhất cho mục đích này. Node đó chỉ cần thực hiện ba việc: tăng một bộ đếm một cách nguyên tử (atomically) và trả về giá trị của nó khi được yêu cầu, lưu trữ bền vững (persist) giá trị bộ đếm (để nó không tạo ra các ID trùng lặp nếu node bị crash và khởi động lại), và thực hiện replication để đảm bảo fault tolerance (sử dụng single-leader replication). Cách tiếp cận này được sử dụng trong thực tế—ví dụ, TiDB/TiKV gọi nó là một *timestamp oracle*, được lấy cảm hứng từ Percolator của Google [59].

Để tối ưu hóa, bạn có thể tránh thực hiện ghi đĩa và replication cho mỗi yêu cầu đơn lẻ. Thay vào đó, bộ tạo ID có thể ghi một bản ghi mô tả một batch các ID; một khi bản ghi đó đã được persist và replicate, node có thể bắt đầu cấp các ID đó cho các client theo trình tự. Trước khi hết các ID trong batch đó, nó có thể persist và replicate bản ghi cho batch tiếp theo. Bằng cách đó, một số ID sẽ bị bỏ qua nếu node bị crash và khởi động lại hoặc nếu bạn thực hiện failover sang một follower, nhưng bạn sẽ không cấp bất kỳ ID nào bị trùng lặp hoặc sai thứ tự.

Bạn không thể dễ dàng sharding bộ tạo ID, vì nếu bạn có nhiều shard độc lập cùng cấp các ID, bạn không còn có thể đảm bảo rằng thứ tự của chúng là linearizable. Bạn cũng không thể dễ dàng phân phối bộ tạo ID qua nhiều vùng (region); do đó, trong một cơ sở dữ liệu phân tán về mặt địa lý, tất cả các yêu cầu cho ID sẽ phải gửi đến một node trong một region duy nhất. Về mặt tích cực, công việc của bộ tạo ID rất đơn giản, vì vậy một node duy nhất có thể xử lý throughput yêu cầu lớn.

Nếu bạn không muốn sử dụng bộ tạo ID một node duy nhất, bạn có thể làm giống như Spanner của Google, như đã thảo luận trong mục “Synchronized clocks for global snapshots”. Nó dựa vào một đồng hồ vật lý không chỉ trả về một timestamp duy nhất, mà là một phạm vi các timestamp cho biết sự không chắc chắn (uncertainty) trong việc đọc đồng hồ. Sau đó, Spanner sẽ đợi cho đến khi khoảng thời gian không chắc chắn đó trôi qua trước khi trả về kết quả.

Giả sử rằng khoảng bất định (uncertainty interval) là chính xác (tức là thời gian vật lý hiện tại thực sự luôn nằm trong khoảng đó), quy trình này cũng đảm bảo rằng nếu một yêu cầu hoàn tất trước khi một yêu cầu khác bắt đầu, thì yêu cầu sau sẽ có timestamp lớn hơn. Cách tiếp cận này đảm bảo việc gán ID có tính linearizability mà không cần bất kỳ sự giao tiếp nào; ngay cả các yêu cầu ở các vùng khác nhau cũng sẽ được sắp xếp đúng thứ tự mà không cần chờ đợi các yêu cầu xuyên vùng. Nhược điểm là bạn cần sự hỗ trợ của cả phần cứng và phần mềm để các đồng hồ được đồng bộ hóa chặt chẽ và tính toán được khoảng bất định cần thiết.

Thực thi các ràng buộc bằng cách sử dụng logical clock

Trong mục “Ràng buộc và đảm bảo tính duy nhất”, chúng ta đã thấy rằng một thao tác CAS có tính linearizability có thể được sử dụng để triển khai các lock, ràng buộc duy nhất và các cấu trúc tương tự trong một hệ thống phân tán. Điều này đặt ra câu hỏi: liệu một logical clock hoặc một bộ tạo ID có tính linearizability cũng đủ để triển khai những thứ này không?

Câu trả lời là: không hẳn. Khi bạn có nhiều node đều đang cố gắng giành lấy cùng một lock hoặc đăng ký cùng một tên người dùng, bạn có thể sử dụng một logical clock để gán timestamp cho các yêu cầu đó và chọn yêu cầu có timestamp thấp nhất làm người chiến thắng. Nếu clock có tính linearizability, bạn biết rằng bất kỳ yêu cầu nào trong tương lai cũng sẽ luôn tạo ra các timestamp lớn hơn, và do đó bạn có thể chắc chắn rằng không có yêu cầu nào trong tương lai nhận được timestamp thấp hơn người chiến thắng.

Thật không may, một phần của vấn đề vẫn chưa được giải quyết: làm thế nào một node biết được liệu timestamp của chính nó có phải là thấp nhất hay không? Để chắc chắn, nó cần nghe được phản hồi từ *mọi* node khác mà có thể đã tạo ra một timestamp [56]. Nếu một trong các node khác đã bị lỗi trong thời gian đó, hoặc không thể kết nối được do vấn đề mạng, hệ thống này sẽ bị đình trệ vì chúng ta không thể chắc chắn rằng timestamp của node đó không thấp hơn. Đây không phải là loại hệ thống fault-tolerant mà chúng ta cần.

Để triển khai các lock, lease và các cấu trúc tương tự một cách fault-tolerant, chúng ta cần thứ gì đó mạnh mẽ hơn logical clock hoặc các bộ tạo ID. Chúng ta cần consensus.

Consensus

Trong chương này, chúng ta đã thấy một vài ví dụ về những thứ vốn dễ dàng khi bạn chỉ có một node duy nhất nhưng sẽ trở nên khó khăn hơn nhiều nếu bạn muốn có fault tolerance:

- Một cơ sở dữ liệu có thể có tính linearizability nếu bạn chỉ có một leader duy nhất và bạn thực hiện tất cả các thao tác đọc và ghi trên leader đó. Nhưng làm thế nào để bạn thực hiện failover nếu leader đó bị lỗi, trong khi vẫn tránh được split brain? Làm thế nào để bạn đảm bảo rằng một node vốn tin rằng mình là leader thực chất không bị bỏ phiếu loại bỏ trong khi nó tạm thời bị tạm dừng?
- Một bộ tạo ID có tính linearizability trên một node duy nhất chỉ là một bộ đếm với một lệnh atomic fetch-and-add—chuyện gì sẽ xảy ra nếu nó bị crash?
- Một thao tác CAS nguyên tử rất hữu ích để quyết định ai nhận được lock hoặc lease khi nhiều tiến trình đang chạy đua để giành lấy nó, ví dụ, hoặc để đảm bảo tính duy nhất của một tệp hoặc người dùng với một tên cho trước. Trên một

node duy nhất, CAS có thể đơn giản chỉ là một lệnh CPU, nhưng làm thế nào để bạn khiến nó trở nên fault-tolerant?

Hóa ra tất cả những điều này đều là các trường hợp của cùng một vấn đề hệ thống phân tán cơ bản: *consensus*. Công thức chuẩn của consensus liên quan đến việc khiến nhiều node đồng ý về một giá trị duy nhất. Đây là một trong những vấn đề quan trọng và cơ bản nhất trong tính toán phân tán; nó cũng nổi tiếng là cực kỳ khó để thực hiện đúng [60, 61], và nhiều hệ thống đã thực hiện sai trong quá khứ. Giờ đây, sau khi chúng ta đã thảo luận về replication (Chương 6), transactions (Chương 8), các mô hình hệ thống (Chương 9), và linearizability (chương này), cuối cùng chúng ta đã sẵn sàng để giải quyết vấn đề consensus.

Các thuật toán consensus nổi tiếng nhất là Viewstamped Replication [62, 63], Paxos [60, 64, 65, 66], Raft [25, 67, 68], và Zab [20, 24, 69]. Những thuật toán này có khá nhiều điểm tương đồng, nhưng chúng không giống nhau [70, 71]. Tất cả chúng đều hoạt động trong một mô hình hệ thống non-Byzantine—tức là, việc truyền thông mạng có thể bị trì hoãn hoặc bị mất một cách tùy ý, và các node có thể bị crash, khởi động lại, hoặc bị mất kết nối, nhưng các thuật toán giả định rằng các node khác tuân thủ đúng protocol và không hành xử một cách ác ý.

Cũng có những thuật toán consensus có thể chịu lỗi một số Byzantine node (tức là, các node không tuân thủ đúng protocol—ví dụ, bằng cách gửi các thông điệp mâu thuẫn đến các node khác). Một giả định phổ biến là có ít hơn một phần ba số node bị Byzantine-faulty [28, 72]. Các thuật toán như vậy được sử dụng trong các blockchain, ví dụ [73]. Tuy nhiên, như đã giải thích trong mục “Byzantine Faults”, các thuật toán Byzantine fault-tolerant nằm ngoài phạm vi của cuốn sách này.

Sự bất khả thi của consensus

Bạn có thể đã nghe nói về *kết quả FLP* [74]—được đặt theo tên các tác giả Fischer, Lynch, và Paterson—thứ chứng minh rằng không có thuật toán nào luôn có thể đạt được consensus nếu có rủi ro một node có thể bị crash. Trong một hệ thống phân tán, chúng ta phải giả định rằng các node có thể bị crash, vì vậy consensus tin cậy là điều bất khả thi. Thế nhưng, chúng ta lại đang ở đây, thảo luận về các thuật toán để đạt được consensus. Chuyện gì đang xảy ra ở đây vậy?

Đầu tiên, FLP không nói rằng chúng ta không bao giờ có thể đạt được consensus; nó chỉ nói rằng chúng ta không thể đảm bảo rằng một thuật toán consensus sẽ *luôn luôn* kết thúc. Hơn nữa, kết quả FLP được chứng minh dựa trên giả định về một thuật toán deterministic trong mô hình hệ thống asynchronous (xem mục “System Model and Reality”), nghĩa là thuật toán không thể sử dụng bất kỳ đồng hồ hay timeout nào. Nếu nó có thể sử dụng timeout để nghỉ ngơi rằng một node khác có thể đã bị crash (ngay cả khi sự nghỉ ngơi đôi khi là sai), thì consensus trở nên có thể giải quyết được [75]. Thậm chí chỉ cần cho phép thuật toán sử dụng các số ngẫu nhiên là đã đủ [76].

Do đó, mặc dù kết quả FLP về sự bất khả thi của consensus có tầm quan trọng lớn về mặt lý thuyết, các hệ thống phân tán thường có thể đạt được consensus trong thực tế.

Nhiều khía cạnh của consensus

Consensus có thể được thể hiện theo nhiều cách. Ví dụ:

- *Single-value consensus* rất giống với một thao tác atomic CAS. Nó có thể được sử dụng để triển khai các lock, lease, và các ràng buộc về tính duy nhất.
- Việc xây dựng một *append-only log* cũng yêu cầu consensus, thứ thường được chính thức hóa dưới dạng *total order broadcast*. Với một log, bạn có thể triển khai state machine replication, leader-based replication, event sourcing, và các pattern hữu ích khác.
- Một thao tác atomic *fetch-and-add* (hoặc atomic increment) cũng hóa ra tương đương với consensus.
- *Atomic commitment* của một transaction đa database hoặc đa shard yêu cầu tất cả các bên tham gia phải đồng ý về việc commit hay abort transaction đó.

Trên thực tế, tất cả các vấn đề này đều tương đương nhau. Nếu bạn có một thuật toán giải quyết được một trong những vấn đề này, bạn có thể chuyển đổi nó thành

giải pháp cho bất kỳ vấn đề nào khác. Đây là một hiểu biết khá sâu sắc và có lẽ gây ngạc nhiên. Đó cũng là lý do tại sao chúng ta có thể gộp tất cả những thứ này lại dưới cái tên “consensus”, mặc dù về bề mặt chúng trông khá khác biệt. Hãy cùng xem xét kỹ hơn từng vấn đề để hiểu tại sao lại như vậy.

Single-value consensus

Khả năng khiến nhiều node đồng ý về một giá trị duy nhất là rất hữu ích. Ví dụ:

- Khi một database với single-leader replication bắt đầu khởi động, hoặc khi leader hiện tại gặp lỗi, nhiều node có thể đồng thời cố gắng trở thành leader. Tương tự, nhiều node có thể tranh giành để giành được một lock hoặc lease. Consensus cho phép chúng quyết định xem ai là người chiến thắng.
- Nếu nhiều người đồng thời cố gắng đặt chiếc ghế cuối cùng trên một máy bay hoặc cùng một ghế trong rạp hát, hoặc cố gắng đăng ký một tài khoản với cùng một username, thì một thuật toán consensus có thể xác định xem ai sẽ thành công nếu không rõ ai là người đã đến trước.

Tổng quát hơn, một hoặc nhiều node có thể *đề xuất* (*propose*) các giá trị, và thuật toán consensus sẽ *quyết định* (*decide*) chọn một trong số các giá trị đó. Trong các ví dụ được đưa ra ở đây, mỗi node có thể đề xuất ID của chính nó, và thuật toán sẽ quyết định ID của node nào sẽ trở thành leader mới, người nắm giữ lease, hoặc người mua ghế máy bay/ghế rạp hát. Trong hình thức luận này, một thuật toán consensus phải thỏa mãn các thuộc tính sau [28]:

Uniform agreement (Sự đồng thuận thống nhất)

Không có hai node nào quyết định khác nhau.

Integrity (Tính toàn vẹn)

Sau khi một node đã quyết định một giá trị, nó không thể thay đổi ý định bằng cách quyết định một giá trị khác.

Validity (Tính hợp lệ)

Nếu một node quyết định giá trị v , thì v được đề xuất bởi một node.

Termination (Tính kết thúc)

Mọi node không bị crash cuối cùng đều quyết định một giá trị.

Nếu bạn muốn quyết định nhiều giá trị, bạn có thể chạy một instance riêng biệt của thuật toán consensus cho mỗi giá trị. Ví dụ, bạn có thể có một lượt chạy consensus riêng biệt cho mỗi ghế có thể đặt trong rạp hát để bạn nhận được một quyết định (một người mua) cho mỗi ghế.

Các thuộc tính uniform agreement và integrity định nghĩa ý tưởng cốt lõi của consensus: mọi người đều quyết định cùng một kết quả, và sau khi bạn đã quyết định, bạn không thể thay đổi ý định. Thuộc tính validity loại bỏ các giải pháp tầm

thường—ví dụ, bạn có thể có một thuật toán luôn quyết định null, bất kể thứ gì được đề xuất; thuật toán này sẽ thỏa mãn các thuộc tính agreement và integrity, nhưng không thỏa mãn thuộc tính validity.

Nếu bạn không quan tâm đến fault tolerance, việc thỏa mãn ba thuộc tính đầu tiên là rất dễ dàng. Bạn chỉ cần hardcode một node làm "kẻ độc tài", và để node đó đưa ra mọi quyết định. Tuy nhiên, nếu node đó bị lỗi, hệ thống không còn khả năng đưa ra bất kỳ quyết định nào—giống như single-leader replication mà không có failover. Mọi khó khăn đều nảy sinh từ nhu cầu về fault tolerance.

Thuộc tính termination chính thức hóa ý tưởng về fault tolerance. Về cơ bản, nó nói rằng một thuật toán consensus không thể chỉ ngồi yên và không làm gì mãi mãi—nói cách khác, nó phải tiến triển. Ngay cả khi một số node bị lỗi, các node khác vẫn phải đạt được một quyết định. (Termination là một thuộc tính liveness, trong khi ba thuộc tính còn lại là các thuộc tính safety—xem mục “Phân biệt giữa safety và liveness”).

Nếu một node bị crash có thể phục hồi, bạn có thể chỉ cần đợi nó quay trở lại. Tuy nhiên, một thuật toán consensus phải đảm bảo rằng nó đưa ra được quyết định ngay cả khi một node bị crash đột ngột biến mất và không bao giờ quay lại. (Thay vì một lỗi crash phần mềm, hãy tưởng tượng một trận động đất khiến datacenter chứa node của bạn bị phá hủy bởi một vụ sạt lở đất. Bạn phải giả định rằng node của mình đã bị chôn vùi dưới 30 feet bùn và sẽ không bao giờ online trở lại.)

Tất nhiên, nếu *tất cả* các node đều crash và không có node nào đang chạy, thì không một thuật toán nào có thể quyết định được bất cứ điều gì. Có một giới hạn về số lượng lỗi mà một thuật toán có thể chịu đựng. Trên thực tế, có thể chứng minh rằng bất kỳ thuật toán consensus nào cũng yêu cầu ít nhất một đa số các node hoạt động chính xác để đảm bảo termination [75]. Đa số đó có thể tạo thành một quorum một cách an toàn (xem “Sử dụng quorum để đọc và ghi”).

Do đó, thuộc tính termination phụ thuộc vào giả định rằng số lượng các node không thể kết nối được ít hơn một nửa. Tuy nhiên, hầu hết các thuật toán consensus đều đảm bảo rằng các thuộc tính safety—agreement, integrity, và validity—luôn được đáp ứng, ngay cả khi đa số các node bị lỗi hoặc xảy ra vấn đề mạng nghiêm trọng [77]. Vì vậy, một sự cố gián đoạn quy mô lớn có thể khiến hệ thống không thể xử lý các yêu cầu, nhưng nó không thể làm hỏng hệ thống consensus bằng cách khiến nó đưa ra các quyết định không nhất quán.

Compare-and-set là consensus

Một thao tác CAS kiểm tra xem giá trị hiện tại của một object có bằng với giá trị mong đợi hay không. Nếu có, nó cập nhật object đó sang một giá trị mới một cách nguyên tử; nếu không, nó giữ nguyên object và trả về một lỗi.

Nếu bạn có một thao tác CAS có tính fault-tolerant và linearizable, việc giải quyết bài toán consensus sẽ trở nên dễ dàng. Ban đầu, hãy thiết lập đối tượng về giá trị null, sau đó yêu cầu mỗi node muốn đề xuất một giá trị thực hiện một thao tác CAS, với giá trị kỳ vọng là null và giá trị mới là giá trị mà nó muốn đề xuất (giả định rằng giá trị đó không phải là null). Giá trị được quyết định sau đó sẽ là bất kỳ giá trị nào mà đối tượng được thiết lập thành.

Tương tự, nếu bạn có một giải pháp cho consensus, bạn có thể triển khai CAS. Bất cứ khi nào một hoặc nhiều node muốn thực hiện một thao tác CAS với cùng một giá trị kỳ vọng, bạn sử dụng giao thức consensus để đề xuất các giá trị mới trong lời gọi CAS và sau đó thiết lập đối tượng thành bất kỳ giá trị nào đã được quyết định bởi consensus. Bất kỳ lời gọi CAS nào có giá trị đề xuất không được quyết định đều trả về một lỗi. Các lời gọi CAS với các giá trị kỳ vọng khác nhau sẽ sử dụng các lượt chạy riêng biệt của giao thức consensus.

Điều này cho thấy CAS và consensus là tương đương nhau [30, 75]. Một lần nữa, cả hai đều đơn giản trên một node duy nhất nhưng đầy thách thức để làm cho chúng có tính fault-tolerant. Như một ví dụ về CAS trong môi trường phân tán, chúng ta đã thấy các thao tác ghi có điều kiện cho các object store trong mục “Databases Backed by Object Storage”, cho phép một thao tác ghi chỉ xảy ra nếu một đối tượng có cùng tên chưa được tạo hoặc sửa đổi bởi một client khác kể từ lần cuối client hiện tại đọc nó.

Shared log đóng vai trò là consensus

Chúng ta đã thấy một vài ví dụ về log, chẳng hạn như replication log, transaction log và write-ahead log. Một log lưu trữ một chuỗi các *log entries*, và bất kỳ ai đọc nó đều thấy các entry giống nhau theo cùng một thứ tự. Đôi khi một log có một writer duy nhất được phép append các entry mới, nhưng một *shared log* là loại mà nhiều node có thể yêu cầu append các entry. Một ví dụ là single-leader replication: bất kỳ client nào cũng có thể yêu cầu leader thực hiện một thao tác ghi, việc mà leader sẽ append vào replication log, và sau đó tất cả các follower sẽ áp dụng các thao tác ghi theo cùng một thứ tự như leader.

Nói một cách trang trọng hơn, một shared log hỗ trợ hai thao tác: bạn có thể yêu cầu thêm một giá trị vào log, và bạn có thể đọc các entry trong log. Nó phải thỏa mãn các thuộc tính sau:

Eventual append

Nếu một node yêu cầu thêm một giá trị vào log, và node đó không bị crash, thì cuối cùng node đó phải đọc được giá trị đó trong một log entry.

Reliable delivery

Không có log entry nào bị mất—nếu một node đọc một log entry, thì cuối cùng mọi node không bị crash cũng phải đọc được log entry đó.

Append-only

Sau khi một node đã đọc một log entry, nó là bất biến, và các log entry mới chỉ có thể được thêm vào sau nó, chứ không phải trước nó. Nếu node đọc lại log, nó sẽ thấy các log entries giống hệt nhau theo cùng một thứ tự như lần đầu nó đọc (ngay cả khi node bị crash và khởi động lại).

Agreement

Nếu hai node cùng đọc một log entry e , thì trước e , chúng phải đã đọc chính xác cùng một chuỗi các log entries theo cùng một thứ tự.

Validity

Nếu một node đọc một log entry chứa một giá trị, thì trước đó đã có một node yêu cầu thêm giá trị đó vào log.

LƯU Ý

Một shared log có thể được triển khai bằng cách sử dụng giao thức *total order broadcast*, còn được gọi là giao thức *atomic broadcast* hoặc *total order multicast* [28, 78, 79]. Để thêm một giá trị vào log, chúng ta "broadcast" nó bằng cách sử dụng giao thức, và khi giao thức "deliver" nó, giá trị đó sẽ trở thành một phần của một log entry có thể đọc được.

Nếu bạn có một bản triển khai của shared log, việc giải quyết bài toán consensus sẽ trở nên dễ dàng. Mọi node muốn đề xuất một giá trị đều yêu cầu giá trị đó được thêm vào log, và bất kỳ giá trị nào được đọc lại trong log entry đầu tiên chính là giá trị đã được quyết định. Vì tất cả các node đều đọc các log entries theo cùng một thứ tự, chúng được đảm bảo sẽ đồng thuận về việc giá trị nào được deliver trước [30].

Ngược lại, nếu bạn có một giải pháp cho consensus, bạn có thể triển khai một shared log. Các chi tiết sẽ phức tạp hơn một chút, nhưng ý tưởng cơ bản là thế này [75]:

1. Bạn có một slot trong log cho mỗi log entry trong tương lai, và bạn chạy một instance riêng biệt của thuật toán consensus cho mỗi slot như vậy để quyết định giá trị nào nên được đưa vào entry đó.
2. Khi một node muốn thêm một giá trị vào log, nó đề xuất giá trị đó cho một trong các slot chưa được quyết định.
3. Khi thuật toán consensus quyết định cho một trong các slot, và tất cả các slot trước đó đã được quyết định, thì giá trị đã quyết định sẽ được nối thêm vào như một log entry mới, và bất kỳ slot liên tiếp nào đã được quyết định cũng sẽ có giá trị đã quyết định của chúng được nối thêm vào log.
4. Nếu một giá trị được đề xuất không được chọn cho một slot, node muốn thêm giá trị đó sẽ thử lại bằng cách đề xuất nó cho một slot muộn hơn.

Điều này cho thấy consensus tương đương với total order broadcast và shared logs. Single-leader replication mà không có failover không đáp ứng được các yêu cầu về liveness vì nó sẽ ngừng truyền tin nếu leader bị crash. Như thường lệ, thách thức nằm ở việc thực hiện failover một cách an toàn và tự động.

Fetch-and-add như là consensus

Bộ tạo ID linearizable mà chúng ta đã thấy trong mục “Linearizable ID Generators” đã tiến gần đến việc giải quyết consensus, nhưng vẫn còn thiếu một chút. Chúng ta có thể triển khai một bộ tạo ID như vậy bằng cách sử dụng một thao tác *fetch-and-add*, giúp tăng một counter một cách nguyên tử và trả về giá trị counter cũ.

Nếu bạn có một thao tác CAS, việc triển khai fetch-and-add rất dễ dàng. Đầu tiên, hãy đọc giá trị counter, sau đó thực hiện một thao tác CAS với giá trị kỳ vọng là giá trị bạn vừa đọc, và giá trị mới là giá trị đó cộng thêm 1. Nếu CAS thất bại, bạn thử lại toàn bộ quá trình cho đến khi CAS thành công. Cách này kém hiệu quả hơn so với một thao tác fetch-and-add nguyên bản khi có sự tranh chấp, nhưng về mặt chức năng thì tương đương. Vì bạn có thể triển khai CAS bằng cách sử dụng consensus, bạn cũng có thể triển khai fetch-and-add bằng cách sử dụng consensus.

Ngược lại, nếu bạn có một thao tác fetch-and-add có khả năng chịu lỗi (fault-tolerant), liệu bạn có thể giải quyết vấn đề consensus không? Hãy giả sử chúng ta khởi tạo counter bằng 0, và mỗi node muốn đề xuất một giá trị sẽ gọi thao tác fetch-and-add để tăng counter. Vì thao tác fetch-and-add là nguyên tử, một node sẽ đọc được giá trị khởi tạo là 0, và tất cả các node khác sẽ đọc được một giá trị đã được tăng lên ít nhất một lần.

Bây giờ, hãy giả sử rằng node đọc được 0 là người chiến thắng, và giá trị của nó được quyết định. Điều này hoạt động đối với node đã đọc 0, nhưng các node khác gặp một vấn đề: chúng biết rằng mình không phải là người chiến thắng, nhưng chúng không biết node nào trong số các node khác đã thắng. Người chiến thắng có thể gửi một tin nhắn đến các node khác để thông báo rằng nó đã thắng, nhưng chuyện gì sẽ xảy ra nếu người chiến thắng bị crash trước khi có cơ hội gửi tin nhắn này? Trong trường hợp đó, các node khác sẽ bị treo, không thể quyết định bất kỳ giá trị nào, và do đó consensus không kết thúc. Và các node khác cũng không thể chuyển sang một node khác vì node đã đọc 0 có thể sẽ quay trở lại và quyết định đúng giá trị mà nó đã đề xuất.

Một ngoại lệ xảy ra nếu chúng ta biết chắc chắn rằng không có nhiều hơn hai node sẽ đề xuất một giá trị. Trong trường hợp đó, các node có thể gửi cho nhau các giá trị mà chúng muốn đề xuất và sau đó mỗi node thực hiện thao tác fetch-and-add. Node đọc được 0 sẽ quyết định giá trị của chính nó, và node đọc được 1 sẽ quyết định giá trị của node kia. Điều này giải quyết vấn đề consensus cho hai node, đó là lý do tại sao chúng ta có thể nói rằng fetch-and-add có một *consensus number* là 2 [30]. Ngược lại,

CAS và shared logs giải quyết consensus cho bất kỳ số lượng node nào có thể đề xuất giá trị, vì vậy chúng có consensus number là ∞ (vô cực).

Atomic commitment như là consensus

Trong mục “Distributed Transactions”, chúng ta đã thấy vấn đề *atomic commitment*, đó là đảm bảo rằng các cơ sở dữ liệu hoặc các shard tham gia vào một distributed transaction đều thực hiện commit hoặc abort một transaction. Chúng ta cũng đã thấy thuật toán *two-phase commit*, vốn dựa vào một coordinator đóng vai trò là một điểm lỗi duy nhất (single point of failure).

Mối quan hệ giữa consensus và atomic commitment là gì? Thoạt nhìn, chúng có vẻ rất giống nhau—cả hai đều yêu cầu các node đi đến một hình thức thỏa thuận nào đó. Tuy nhiên, có một sự khác biệt quan trọng: với consensus, việc quyết định bất kỳ giá trị nào đã được đề xuất đều được chấp nhận, trong khi với atomic commitment, thuật toán *phải* abort nếu *bất kỳ* bên tham gia nào bỏ phiếu abort. Chính xác hơn, atomic commitment yêu cầu các thuộc tính sau [80]:

Uniform agreement (Sự đồng thuận thống nhất)

Không thể có chuyện một node commit trong khi một node khác abort.

Integrity (Tính toàn vẹn)

Một khi một node đã commit, nó không thể thay đổi ý định để abort, và ngược lại.

Validity (Tính hợp lệ)

Nếu một node commit, tất cả các node phải đã bỏ phiếu commit trước đó. Nếu bất kỳ node nào bỏ phiếu abort, tất cả các node phải abort.

Nontriviality

Nếu tất cả các node đều bỏ phiếu commit, và không có tình trạng timeout truyền thông nào xảy ra, thì tất cả các node phải commit.

Termination (Tính kết thúc)

Mọi node không bị crash cuối cùng đều sẽ commit hoặc abort.

Thuộc tính validity đảm bảo rằng một transaction chỉ có thể commit nếu tất cả các node đồng ý, và thuộc tính nontriviality đảm bảo rằng thuật toán không thể chỉ đơn giản là luôn luôn abort (nhưng nó cho phép abort nếu bất kỳ quá trình truyền thông giữa các node nào bị timeout). Ba thuộc tính còn lại về cơ bản giống với consensus.

Nếu bạn có một giải pháp cho consensus, bạn có thể giải quyết atomic commitment theo nhiều cách [80, 81]. Một cách hoạt động như sau: khi bạn muốn commit một transaction, mỗi node gửi phiếu bầu commit hoặc abort của mình tới mọi node khác. Các node nhận được phiếu bầu commit từ chính nó và từ mọi node khác sẽ đề xuất “commit” thông qua thuật toán consensus; các node nhận được phiếu bầu abort, hoặc gặp tình trạng timeout, sẽ đề xuất “abort” thông qua thuật toán consensus. Khi

một node biết được thuật toán consensus đã quyết định điều gì, nó sẽ commit hoặc abort tương ứng.

Trong thuật toán này, "commit" sẽ chỉ được đề xuất nếu tất cả các node đều bỏ phiếu commit. Nếu bất kỳ node nào bỏ phiếu abort, tất cả các đề xuất trong thuật toán consensus sẽ là "abort". Có thể xảy ra trường hợp một số node đề xuất "abort" trong khi những node khác đề xuất "commit" nếu tất cả các node đã bỏ phiếu commit nhưng một số quá trình truyền thông bị timeout; trong trường hợp này, việc các node commit hay abort không quan trọng, miễn là tất cả chúng đều thực hiện cùng một việc.

Nếu bạn có một giao thức atomic commitment có khả năng chịu lỗi (fault-tolerant), bạn cũng có thể giải quyết consensus. Mỗi node muốn đề xuất một giá trị sẽ bắt đầu một transaction trên một quorum các node, và tại mỗi node, nó thực hiện một thao tác CAS (compare-and-set) đơn node để thiết lập một register với giá trị được đề xuất nếu giá trị của nó chưa bị thiết lập bởi một transaction khác. Nếu CAS thành công, node đó bỏ phiếu commit, nếu không nó sẽ bỏ phiếu abort. Nếu giao thức atomic commit thực hiện commit một transaction, giá trị của nó sẽ được quyết định cho consensus; nếu atomic commit abort, node đề xuất sẽ thử lại với một transaction mới.

Điều này cho thấy atomic commit và consensus cũng tương đương với nhau.

Consensus trong thực tế

Chúng ta đã thấy rằng consensus giá trị đơn (single-value consensus), CAS, shared logs, và atomic commitment đều tương đương nhau: bạn có thể chuyển đổi một giải pháp cho một trong các vấn đề này thành giải pháp cho bất kỳ vấn đề nào khác. Đó là một hiểu biết lý thuyết giá trị, nhưng nó không trả lời câu hỏi này: trong số nhiều cách diễn đạt về consensus này, cách nào là hữu ích nhất trong thực tế?

Câu trả lời là hầu hết các hệ thống consensus đều cung cấp shared logs (một sự trừu tượng tương đương với total order broadcast). Raft, Viewstamped Replication, và Zab cung cấp shared logs dùng được ngay. Paxos cung cấp consensus giá trị đơn, nhưng trong thực tế, hầu hết các hệ thống sử dụng Paxos thực chất đều sử dụng phần mở rộng gọi là Multi-Paxos, thứ cũng cung cấp một shared log.

Sử dụng shared logs

Một shared log là lựa chọn phù hợp cho việc replication cơ sở dữ liệu. Nếu mỗi mục trong log đại diện cho một thao tác ghi vào cơ sở dữ liệu, và mỗi replica xử lý các thao tác ghi đó theo cùng một thứ tự bằng cách sử dụng logic xác định (deterministic), thì tất cả các replica cuối cùng sẽ đạt được một trạng thái nhất quán. Ý tưởng này được gọi là *state machine replication* [82], và nó là nguyên lý đằng sau event sourcing, thứ

mà chúng ta đã thấy trong mục “Event Sourcing and CQRS”. Shared log cũng hữu ích cho stream processing, như chúng ta sẽ thấy trong Chương 12.

Tương tự, một shared log có thể được sử dụng để triển khai các transaction có tính serializable. Như đã thảo luận trong mục “Actual Serial Execution”, nếu mỗi mục trong log đại diện cho một transaction xác định cần được thực thi dưới dạng một stored procedure, và nếu mọi node đều thực thi các transaction đó theo cùng một thứ tự, thì các transaction sẽ có tính serializable [83, 84].

LƯU Ý

Các cơ sở dữ liệu được sharding với mô hình strong consistency thường duy trì một log riêng biệt cho mỗi shard, điều này cải thiện khả năng mở rộng nhưng lại giới hạn các đảm bảo về tính nhất quán (ví dụ: các snapshot nhất quán, các tham chiếu foreign-key) mà chúng có thể cung cấp giữa các shard. Các transaction có tính serializable xuyên suốt các shard là khả thi nhưng yêu cầu thêm sự điều phối [85].

Một shared log cũng rất mạnh mẽ vì nó có thể dễ dàng được điều chỉnh để phù hợp với các dạng consensus khác:

- Trước đó chúng ta đã thấy cách sử dụng nó để triển khai consensus cho giá trị đơn (single-value consensus) và CAS; chỉ đơn giản là quyết định giá trị xuất hiện đầu tiên trong log.
- Nếu bạn muốn có nhiều thực thể của single-value consensus—chẳng hạn như một thực thể cho mỗi ghế trong rạp hát nơi nhiều người đang cố gắng đặt chỗ—hãy đưa số ghế vào các mục trong log và quyết định dựa trên mục log đầu tiên chứa số ghế đó.
- Nếu bạn muốn một thao tác atomic fetch-and-add, hãy đặt số cần cộng vào bộ đếm trong một mục log, và để giá trị bộ đếm hiện tại là tổng của tất cả các mục log cho đến thời điểm đó. Một bộ đếm đơn giản trên các mục log có thể được sử dụng để tạo ra các fencing token (xem mục “Fencing off zombies and delayed requests”); ví dụ, trong ZooKeeper, số thứ tự này được gọi là zxid [20].

Từ single-leader replication đến consensus

Trước đó chúng ta đã thấy rằng single-value consensus rất dễ dàng nếu bạn có một node “độc tài” duy nhất đưa ra quyết định, và tương tự, một shared log cũng dễ dàng nếu chỉ có một leader duy nhất được phép append các mục log. Câu hỏi đặt ra là làm thế nào để cung cấp khả năng chịu lỗi (fault tolerance) nếu node đó gặp sự cố.

Theo truyền thống, các cơ sở dữ liệu với single-leader replication không giải quyết vấn đề này: chúng để việc leader failover là một hành động mà một quản trị viên con người phải thực hiện thủ công. Thật không may, điều này đồng nghĩa với một khoảng thời gian downtime đáng kể, vì có giới hạn về tốc độ phản ứng của con người, và nó không thỏa mãn tính chất termination của consensus. Đối với

consensus, chúng ta yêu cầu thuật toán có thể tự động chọn một leader mới. (Không phải tất cả các thuật toán consensus đều có leader, nhưng các thuật toán thường dùng thì có [86, 87].)

Điều này không hề đơn giản. Trước đó chúng ta đã thảo luận về vấn đề split brain, và chúng ta đã xác lập rằng tất cả các node cần đồng thuận về việc ai là leader—if không, hai node có thể mỗi node đều tin rằng mình là leader và đưa ra các quyết định không nhất quán. Do đó, có vẻ như chúng ta cần consensus để bầu chọn một leader, và chúng ta cần một leader để giải quyết consensus. Làm thế nào để chúng ta thoát khỏi tình thế tiến thoái lưỡng nan này?

Trên thực tế, các thuật toán consensus không yêu cầu chỉ có duy nhất một leader tại bất kỳ thời điểm nào. Thay vào đó, chúng đưa ra một đảm bảo yếu hơn: chúng định nghĩa một *epoch number* (được gọi là *ballot number* trong Paxos, *view number* trong Viewstamped Replication, và *term number* trong Raft) và đảm bảo rằng trong mỗi epoch, leader là duy nhất.

Khi một node tin rằng leader hiện tại đã chết vì nó không nhận được tin tức gì từ leader sau một khoảng timeout, nó có thể bắt đầu một cuộc bỏ phiếu để bầu chọn leader mới. Cuộc bầu chọn này được gán một epoch number mới lớn hơn bất kỳ epoch number nào trước đó. Nếu một xung đột xảy ra giữa hai leader trong hai epoch (có lẽ vì leader trước đó hóa ra vẫn chưa chết), thì leader có epoch number cao hơn sẽ chiếm ưu thế.

Trước khi một leader được phép append entry tiếp theo vào shared log, nó phải kiểm tra xem liệu có leader nào khác với số epoch cao hơn có thể sẽ append một entry khác hay không. Nó có thể thực hiện việc này bằng cách thu thập phiếu bầu từ một quorum các node—thông thường, nhưng không phải luôn luôn, là đa số các node [88]. Một node chỉ bỏ phiếu yes nếu nó không biết về bất kỳ leader nào khác có epoch cao hơn.

Do đó, chúng ta có hai vòng bỏ phiếu: một lần để chọn leader, và lần thứ hai để bỏ phiếu cho đề xuất của leader về entry tiếp theo cần append vào log. Các quorum cho hai lần bỏ phiếu này phải chồng lấp lên nhau: nếu một cuộc bỏ phiếu cho đề xuất thành công, ít nhất một trong số các node đã bỏ phiếu cho nó cũng phải tham gia vào cuộc bầu chọn leader thành công gần nhất [88]. Nếu cuộc bỏ phiếu cho một đề xuất thông qua mà không tiết lộ bất kỳ epoch nào có số lớn hơn, leader hiện tại có thể kết luận rằng không có leader nào với số epoch cao hơn được bầu, và do đó nó có thể an toàn append entry đã đề xuất vào log [28, 89].

Hai vòng bỏ phiếu này nhìn bề ngoài có vẻ giống với 2PC (xem mục “Two-Phase Commit”), nhưng chúng là các protocol rất khác nhau. Trong các thuật toán consensus, bất kỳ node nào cũng có thể bắt đầu một cuộc bầu chọn, và nó chỉ yêu cầu một quorum các node phản hồi; trong 2PC, chỉ có coordinator mới có thể yêu

cầu bỏ phiếu, và nó yêu cầu phiếu bầu yes từ *mọi* participant trước khi có thể commit.

Những điểm tinh tế của consensus

Cấu trúc cơ bản này là điểm chung của Raft, Multi-Paxos, Viewstamped Replication và Zab: một cuộc bỏ phiếu bởi một quorum các node sẽ bầu ra một leader, và sau đó cần một cuộc bỏ phiếu quorum khác cho mỗi entry mà leader muốn append vào log [70, 71]. Mỗi log entry mới đều được replication đồng bộ tới một quorum các node trước khi nó được xác nhận với client đã yêu cầu lệnh write. Điều này đảm bảo rằng log entry sẽ không bị mất nếu leader hiện tại gặp lỗi.

Tuy nhiên, chi tiết mới là nơi nảy sinh vấn đề, và đó cũng là nơi các thuật toán này thực hiện các cách tiếp cận khác nhau. Ví dụ, khi leader cũ gặp lỗi và một leader mới được bầu, thuật toán cần đảm bảo rằng leader mới tôn trọng bất kỳ log entry nào đã được append bởi leader cũ trước khi nó gặp lỗi. Raft thực hiện điều này bằng cách chỉ cho phép một node trở thành leader mới nếu log của nó ít nhất là cập nhật (up-to-date) bằng với log của đa số các follower của nó [71]. Ngược lại, Paxos cho phép bất kỳ node nào trở thành leader mới, nhưng yêu cầu nó phải cập nhật log của mình cho đồng bộ với các node khác trước khi nó có thể bắt đầu append các entry mới của riêng mình.

Consistency đối lập với Availability trong bầu chọn leader

Nếu bạn muốn thuật toán consensus đảm bảo nghiêm ngặt các thuộc tính được nêu trong mục “Shared logs as consensus”, điều thiết yếu là leader mới phải cập nhật đầy đủ các log entry đã được xác nhận trước khi nó có thể xử lý bất kỳ lệnh write hoặc các lệnh read có tính linearizable. Nếu một node với dữ liệu cũ (stale data) trở thành leader mới, nó có thể ghi các giá trị mới vào các log entry vốn đã được ghi bởi leader cũ, vi phạm thuộc tính append-only của shared log.

Trong một số trường hợp, bạn có thể chọn làm yếu đi các thuộc tính consensus để phục hồi nhanh hơn sau lỗi của leader hoặc để có thể phục hồi được. Ví dụ, Kafka cung cấp tùy chọn cho phép kích hoạt *unclean leader election*, cho phép bất kỳ replica nào cũng có thể trở thành leader, ngay cả khi nó không cập nhật đầy đủ. Ngoài ra, trong các cơ sở dữ liệu sử dụng asynchronous replication, bạn không thể đảm bảo rằng bất kỳ follower nào cũng được cập nhật đầy đủ khi leader gặp lỗi.

Nếu bạn từ bỏ yêu cầu leader mới phải cập nhật đầy đủ, bạn có thể cải thiện hiệu năng và tính availability, nhưng bạn đang bước đi trên băng mỏng, vì lý thuyết về consensus không còn áp dụng nữa. Mặc dù mọi thứ sẽ hoạt động ổn thỏa chừng nào không có lỗi, nhưng các vấn đề được thảo luận trong Chương 9 có thể dễ dàng gây ra mất mát hoặc hư hỏng dữ liệu.

Một điểm tinh tế khác nằm ở cách các thuật toán xử lý các log entry đã được đề xuất bởi leader cũ trước khi nó gặp lỗi, nhưng cuộc bỏ phiếu để append vào log vẫn chưa hoàn tất. Bạn có thể tìm thấy các thảo luận về những chi tiết này trong phần tài liệu tham khảo của chương này [25, 71, 89].

Đối với các cơ sở dữ liệu sử dụng thuật toán consensus để replication, việc chuyển đổi các thao tác ghi thành các log entry và replication chúng tới một quorum không phải là tất cả những gì cần thiết. Nếu bạn muốn đảm bảo các thao tác đọc có tính linearizable, chúng cũng phải thông qua một cuộc bỏ phiếu quorum, tương tự như một thao tác ghi, để xác nhận rằng node mà nó tin rằng mình là leader thực sự vẫn đang cập nhật dữ liệu mới nhất. Ví dụ, các thao tác đọc linearizable trong etcd hoạt động như thế này.

Ở dạng tiêu chuẩn, hầu hết các thuật toán consensus giả định một tập hợp các node cố định—nghĩa là, các node có thể bị down và sau đó hoạt động trở lại, nhưng tập hợp các node được phép bỏ phiếu là cố định khi cluster được tạo ra. Trong thực tế, việc thêm các node mới hoặc loại bỏ các node cũ trong một cấu hình hệ thống thường là cần thiết. Các thuật toán consensus đã được mở rộng với các tính năng *reconfiguration* để giúp việc này trở nên khả thi. Điều này đặc biệt hữu ích khi thêm

các vùng mới vào một hệ thống, hoặc khi di chuyển từ vị trí này sang vị trí khác (bằng cách đầu tiên thêm các node mới và sau đó loại bỏ các node cũ).

Ưu và nhược điểm của consensus

Mặc dù phức tạp và tinh vi, các thuật toán consensus là một bước đột phá lớn cho các distributed systems. Về bản chất, consensus là "single-leader replication được thực hiện đúng cách", với khả năng failover tự động khi leader gặp lỗi, đảm bảo rằng không có dữ liệu đã được commit nào bị mất và không thể xảy ra split brain, ngay cả khi đối mặt với tất cả các vấn đề mà chúng ta đã thảo luận trong Chương 9.

Bất kỳ hệ thống nào cung cấp khả năng failover tự động nhưng không sử dụng một thuật toán consensus đã được kiểm chứng đều có khả năng không an toàn [90]. Việc sử dụng một thuật toán consensus đã được kiểm chứng không phải là sự đảm bảo cho tính đúng đắn của toàn bộ hệ thống—vẫn còn rất nhiều nơi khác mà bug có thể ẩn nấp—nhưng đó là một điểm khởi đầu tốt.

Tuy nhiên, consensus không được sử dụng ở mọi nơi vì những lợi ích mang lại đi kèm với một sự đánh đổi (trade-off). Các hệ thống consensus luôn yêu cầu một đa số tuyệt đối để hoạt động—ba node để chịu được một lỗi, hoặc năm node để chịu được hai lỗi. Mọi thao tác bạn thực hiện đều yêu cầu giao tiếp với một quorum, vì vậy bạn không thể tăng throughput bằng cách thêm nhiều node hơn (thực tế, mỗi node bạn thêm vào đều làm cho thuật toán chạy chậm hơn). Nếu một network partition cắt đứt một số node khỏi phần còn lại, chỉ có phần đa số của mạng mới có thể tiếp tục tiến triển, còn các node khác sẽ bị chặn.

Các hệ thống consensus thường dựa vào timeout để phát hiện các node bị lỗi. Trong các môi trường có độ trễ mạng biến động cao, đặc biệt là các hệ thống phân tán trên nhiều vùng địa lý, việc tinh chỉnh các timeout này có thể rất khó khăn. Nếu chúng quá lớn, việc phục hồi sau một lỗi sẽ mất nhiều thời gian; nếu chúng quá nhỏ, rất nhiều cuộc bầu chọn leader không cần thiết có thể xảy ra, dẫn đến hiệu năng tồi tệ vì hệ thống có thể kết thúc bằng việc dành nhiều thời gian để chọn leader hơn là thực hiện các công việc hữu ích.

Đôi khi các thuật toán consensus đặc biệt nhạy cảm với các vấn đề mạng. Ví dụ, Raft đã được chứng minh là có những trường hợp biên (edge cases) không mong muốn [91, 92]. Nếu toàn bộ mạng đang hoạt động bình thường ngoại trừ một liên kết mạng cụ thể liên tục không tin cậy, Raft có thể rơi vào các tình huống mà quyền lãnh đạo liên tục nhảy qua lại giữa hai node, hoặc leader hiện tại liên tục bị buộc phải từ chức, khiến hệ thống thực tế không bao giờ tiến triển được. Thuật toán Raft nguyên bản đã được mở rộng với một giai đoạn pre-vote để giải quyết vấn đề này [67]. Paxos cũng phụ thuộc vào các leader, điều này có thể gây ra các vấn đề hiệu năng tương tự. Egalitarian Paxos (EPaxos) và các biến thể của nó sử dụng một giao thức leaderless, giúp nó mạnh mẽ hơn trước các node hoặc các kết nối mạng có hiệu năng kém [86].

Coordination Services

Các thuật toán consensus rất hữu ích trong bất kỳ distributed database nào muốn cung cấp các thao tác linearizable, và nhiều distributed database hiện đại sử dụng chúng để replication. Nhưng có một nhóm hệ thống là người dùng nổi bật đặc biệt của consensus: các *coordination services* như ZooKeeper, etcd, và Consul. Mặc dù các hệ thống này nhìn bề ngoài giống như bất kỳ key-value store nào khác, chúng không được thiết kế cho khối lượng ghi lớn hoặc lưu trữ dữ liệu đa mục đích như hầu hết các database.

Thay vào đó, chúng được thiết kế để điều phối giữa các node của một hệ thống phân tán khác. Ví dụ, Kubernetes dựa vào etcd, trong khi Spark và Flink ở chế độ high availability dựa vào ZooKeeper chạy ngầm. Các coordination service được thiết kế để giữ một lượng nhỏ dữ liệu có thể nằm trọn trong bộ nhớ (mặc dù chúng vẫn ghi vào đĩa để đảm bảo độ tin cậy), và dữ liệu này được replication qua nhiều node thông qua một thuật toán consensus có khả năng chịu lỗi.

Các coordination service được mô phỏng theo dịch vụ khóa Chubby của Google [19, 60]. Chúng kết hợp một thuật toán consensus với một vài tính năng khác mà hóa ra lại đặc biệt hữu ích khi xây dựng các hệ thống phân tán:

Locks và leases

Trước đó chúng ta đã thấy các hệ thống consensus có thể thực hiện một thao tác CAS nguyên tử và có khả năng chịu lỗi như thế nào. Các coordination service dựa vào cách tiếp cận này để thực hiện các lock và lease. Nếu nhiều node cùng lúc cố gắng giành lấy cùng một lease, chỉ một trong số chúng thành công.

Hỗ trợ fencing

Như đã thảo luận trong mục “Distributed Locks and Leases”, khi một tài nguyên được bảo vệ bởi một lease, bạn cần *fencing* (chốt chặn) để ngăn các client can thiệp lẫn nhau trong trường hợp một tiến trình bị tạm dừng hoặc xảy ra độ trễ mạng lớn. Các hệ thống consensus có thể tạo ra các fencing token bằng cách gán cho mỗi log entry một ID tăng dần (*zxid* và *cversion* trong ZooKeeper, hoặc revision number trong etcd).

Phát hiện lỗi

Các client duy trì một session lâu dài với coordination service và định kỳ trao đổi heartbeat để kiểm tra xem node còn lại có còn hoạt động hay không. Ngay cả khi kết nối bị gián đoạn tạm thời hoặc một server gặp lỗi, bất kỳ lease nào mà client đang nắm giữ vẫn sẽ duy trì trạng thái active. Tuy nhiên, nếu không có heartbeat nào trong khoảng thời gian dài hơn timeout của lease, coordination service sẽ giả định rằng client đã chết và giải phóng lease đó (ZooKeeper gọi đây là các *ephemeral nodes*).

Thông báo thay đổi

Một client có thể yêu cầu coordination service gửi cho nó một thông báo bất cứ khi nào các key nhất định thay đổi. Điều này cho phép một client biết được khi nào một client khác gia nhập cluster (dựa trên giá trị mà nó ghi vào coordination service), hoặc ví dụ như khi một client khác gặp lỗi (vì session của nó hết hạn và các ephemeral nodes của nó biến mất). Những thông báo này giúp client không phải thường xuyên poll service để tìm hiểu về các thay đổi.

Phát hiện lỗi và thông báo thay đổi không yêu cầu consensus, nhưng chúng hữu ích cho việc điều phối phân tán cùng với các thao tác nguyên tử và hỗ trợ fencing vốn yêu cầu consensus.

Quản lý Configuration với Coordination Services

Các ứng dụng và hạ tầng thường có các tham số cấu hình như timeout, kích thước thread pool, v.v. Coordination services đôi khi được sử dụng để lưu trữ các dữ liệu cấu hình như vậy, được biểu diễn dưới dạng các cặp key-value. Các tiến trình tải các thiết lập mới nhất khi khởi động và đăng ký nhận thông báo về bất kỳ thay đổi nào. Khi một cấu hình thay đổi, tiến trình có thể bắt đầu sử dụng thiết lập mới ngay lập tức hoặc tự khởi động lại để tải các thay đổi mới nhất.

Quản lý cấu hình không cần đến khía cạnh consensus của một coordination service, nhưng sẽ rất tiện lợi nếu sử dụng một coordination service và dựa vào tính năng thông báo của nó nếu bạn vốn đã đang chạy service đó rồi. Ngoài ra, một tiến trình có thể định kỳ poll để lấy các cập nhật cấu hình từ một tệp hoặc URL, điều này giúp tránh việc phải sử dụng một service chuyên dụng.

Phân bổ công việc cho các node

Một coordination service sẽ hữu ích nếu bạn có nhiều instance của một tiến trình hoặc service, và một trong số chúng cần được chọn làm leader hoặc primary. Nếu leader gặp lỗi, một trong các node khác sẽ tiếp quản. Điều này là cần thiết đối với các cơ sở dữ liệu single-leader, nhưng cũng phù hợp với các bộ lập lịch công việc (job schedulers) và các hệ thống stateful tương tự.

Một trường hợp sử dụng khác là khi bạn có một tài nguyên được sharding (phân mảnh dữ liệu) (như cơ sở dữ liệu, message streams, lưu trữ tệp, hệ thống distributed actor, v.v.) và cần quyết định xem nên gán shard nào cho node nào. Khi các node mới gia nhập cluster, một số shard cần được di chuyển từ các node hiện có sang các node mới để rebalance tải trọng. Khi các node bị loại bỏ hoặc gặp lỗi, các node khác cần tiếp quản công việc của các node đã lỗi đó.

Những loại tác vụ này có thể đạt được bằng cách sử dụng khôn ngoan các thao tác nguyên tử (atomic operations), ephemeral nodes, và các thông báo (notifications) trong một coordination service. Nếu được thực hiện đúng cách, phương pháp này cho phép ứng dụng tự động phục hồi sau các lỗi mà không cần sự can thiệp của con người. Điều này không hề dễ dàng, mặc dù đã có sẵn các thư viện như Apache Curator mọc lên để cung cấp các công cụ cấp cao hơn dựa trên ZooKeeper client API—nhưng nó vẫn tốt hơn nhiều so với việc cố gắng tự triển khai các thuật toán consensus cần thiết từ đầu, vốn rất dễ phát sinh lỗi.

Một coordination service chuyên dụng cũng có lợi thế là nó có thể chạy trên một tập hợp các node cố định (thường là ba hoặc năm), bất kể có bao nhiêu node trong hệ thống phân tán đang dựa vào nó để điều phối. Ví dụ, trong một hệ thống lưu trữ với hàng nghìn shard, việc chạy một thuật toán consensus trên hàng nghìn node sẽ cực kỳ kém hiệu quả; tốt hơn nhiều là "thuê ngoài" việc consensus cho một số lượng nhỏ các node đang chạy một coordination service.

Thông thường, loại dữ liệu được quản lý bởi một coordination service thay đổi khá chậm. Dữ liệu này đại diện cho các thông tin như "node đang chạy trên địa chỉ IP 10.1.1.23 là leader cho shard 7," và những sự gán này thường thay đổi theo thang thời gian tính bằng phút hoặc giờ. Coordination services không được thiết kế để lưu trữ dữ liệu có thể thay đổi hàng nghìn lần mỗi giây. Đối với việc đó, tốt hơn là sử dụng một cơ sở dữ liệu truyền thống; hoặc thay vào đó, có thể sử dụng các công cụ như Apache BookKeeper [93, 94] để nhân bản trạng thái nội bộ thay đổi nhanh của một dịch vụ.

Service discovery

ZooKeeper, etcd, và Consul cũng thường được sử dụng cho *service discovery* (khám phá dịch vụ)—tức là để tìm ra địa chỉ IP nào bạn cần kết nối để tiếp cận một dịch vụ cụ thể (xem mục “Load balancers, service discovery, and service meshes”). Trong môi trường cloud, nơi các máy ảo thường xuyên đến và đi, bạn thường không biết trước địa chỉ IP của các dịch vụ của mình. Thay vào đó, bạn có thể cấu hình các dịch vụ sao cho khi chúng khởi động, chúng sẽ đăng ký các network endpoint của mình vào một service registry, nơi mà các dịch vụ khác có thể tìm thấy chúng.

Sử dụng một coordination service để service discovery có thể rất tiện lợi, vì các tính năng phát hiện lỗi và thông báo thay đổi của nó giúp các client dễ dàng theo dõi các instance của dịch vụ khi chúng đến và đi. Và nếu bạn đã đang sử dụng một coordination service cho các lease, locking, hoặc leader election, thì việc sử dụng nó cho service discovery cũng là điều hợp lý, vì nó đã biết node nào nên nhận các yêu cầu cho dịch vụ của bạn.

Tuy nhiên, việc sử dụng consensus cho service discovery thường là quá mức cần thiết (overkill). Trường hợp sử dụng này thường không yêu cầu linearizability, và điều quan trọng hơn là service discovery phải có độ

sẵn sàng cao và nhanh chóng, vì nếu không có nó, mọi thứ sẽ bị đình trệ. Do đó, thông thường việc cache thông tin `service discovery` sẽ được ưu tiên hơn. Các `client` không thể kết nối với một dịch vụ có thể bỏ qua cache, thử lại với giá trị mới nhất, và cập nhật cache nếu cần thiết. Các cache cũng có thể được làm mới định kỳ bằng cách sử dụng cấu hình `time-to-live (TTL)`. Ví dụ, `service discovery` dựa trên DNS sử dụng nhiều lớp caching để đạt được hiệu năng và độ sẵn sàng tốt.

Để hỗ trợ trường hợp sử dụng này, ZooKeeper hỗ trợ các *observer*. Các replica này nhận log và duy trì một bản sao của dữ liệu được lưu trữ trong ZooKeeper, nhưng không tham gia vào quá trình bỏ phiếu của thuật toán consensus. Các thao tác đọc từ một observer không có tính linearizable vì chúng có thể bị cũ, nhưng chúng vẫn đảm bảo tính sẵn sàng ngay cả khi mạng bị gián đoạn, đồng thời giúp tăng throughput đọc mà hệ thống có thể hỗ trợ thông qua việc caching.

Tóm tắt

Trong chương này, chúng ta đã xem xét chủ đề về strong consistency trong các hệ thống fault-tolerant; nó là gì và làm thế nào để đạt được nó. Chúng ta đã nghiên cứu sâu về linearizability, một cách hình thức hóa phổ biến của strong consistency giúp đảm bảo dữ liệu được replication trông như thể chỉ có một bản sao duy nhất, với tất cả các thao tác thực hiện trên đó một cách atomically. Chúng ta thấy rằng linearizability rất hữu ích nếu bạn cần một số dữ liệu phải luôn mới nhất khi đọc, hoặc nếu bạn cần giải quyết một race condition (ví dụ: nếu nhiều node đang đồng thời cố gắng thực hiện cùng một việc, chẳng hạn như tạo các tệp có cùng tên).

Mặc dù linearizability rất hấp dẫn vì nó dễ hiểu—nó khiến một cơ sở dữ liệu hoạt động giống như một biến trong một chương trình đơn luồng—nhưng nó có nhược điểm là chậm, đặc biệt là trong các môi trường có độ trễ mạng lớn. Nhiều thuật toán replication không đảm bảo tính linearizability, mặc dù về mặt bề ngoài, chúng có vẻ như cung cấp strong consistency.

Tiếp theo, chúng ta áp dụng khái niệm linearizability trong bối cảnh các bộ tạo ID. Một bộ đếm tự tăng (autoincrementing counter) trên một node duy nhất là linearizable nhưng không có tính fault-tolerant. Nhiều cơ chế tạo ID phân tán không đảm bảo rằng các ID được sắp xếp nhất quán với thứ tự mà các sự kiện thực tế đã xảy ra. Các logical clock như Lamport clock và hybrid logical clock cung cấp thứ tự nhất quán với causality nhưng không đảm bảo tính linearizability.

Điều này dẫn chúng ta đến các thuật toán consensus, thứ giúp việc triển khai replication có tính fault-tolerant và linearizable trở nên khả thi. Linearizability có nghĩa là hệ thống phải hoạt động như thể chỉ có một bản sao duy nhất của dữ liệu, và tất cả các thao tác đều diễn ra lần lượt trên bản sao duy nhất đó theo một thứ tự được xác định rõ ràng. Consensus cung cấp điều này bằng cách khiến một nhóm các

node đồng thuận về một trình tự thao tác duy nhất, ngay cả khi các thông điệp bị trễ hoặc một số node bị lỗi. Trình tự thao tác đó khiến một hệ thống phân tán hoạt động như thể chỉ có một node duy nhất đang xử lý các thao tác theo thứ tự, mặc dù một nhóm các node đang cùng nhau làm việc.

Công thức kinh điển của consensus bao gồm việc quyết định một giá trị duy nhất sao cho tất cả các node đều đồng thuận về những gì đã được quyết định, và sao cho chúng không thể thay đổi ý định. Một loạt các vấn đề thực tế có thể quy về consensus và tương đương với nhau (tức là, nếu bạn có giải pháp cho một vấn đề trong số đó, bạn có thể chuyển đổi nó thành giải pháp cho tất cả các vấn đề còn lại). Các vấn đề tương đương như vậy bao gồm các mục sau:

Các thao tác CAS linearizable

Register cần phải *decide* (quyết định) một cách atomically liệu có thiết lập giá trị của nó hay không, dựa trên việc liệu giá trị hiện tại của nó có bằng với tham số được đưa ra trong thao tác hay không.

Locks và leases

Khi nhiều client đang đồng thời cố gắng giành lấy một lock hoặc lease, lock sẽ *decide* xem client nào đã giành được nó thành công.

Các ràng buộc về tính duy nhất (Uniqueness constraints)

Khi nhiều transaction đồng thời cố gắng tạo các bản ghi xung đột với cùng một key, ràng buộc phải *decide* cho phép bản ghi nào và bản ghi nào sẽ thất bại do vi phạm ràng buộc.

Shared logs

Khi nhiều node đồng thời muốn append các mục vào một log, log sẽ *decide* thứ tự mà chúng được append. Shared logs được triển khai bằng cách sử dụng một giao thức total order broadcast.

Atomic transaction commit

Các node cơ sở dữ liệu tham gia vào một distributed transaction đều phải *decide* theo cùng một cách liệu có commit hay abort transaction đó.

Các thao tác fetch-and-add linearizable

Loại thao tác này có thể được sử dụng để triển khai một bộ tạo ID. Nhiều node có thể gọi thao tác này một cách đồng thời, và nó *quyết định* thứ tự mà chúng tăng biến đếm. Trường hợp này thực tế chỉ giải quyết consensus giữa hai node, trong khi các trường hợp khác hoạt động cho bất kỳ số lượng node nào.

Tất cả những điều này đều đơn giản nếu bạn chỉ có một node duy nhất hoặc nếu bạn sẵn lòng giao khả năng ra quyết định cho một node duy nhất. Đây là những gì xảy ra trong một cơ sở dữ liệu *single-leader*: toàn bộ quyền ra quyết định được trao cho leader, đó là lý do tại sao các cơ sở dữ liệu như vậy có thể cung cấp các

thao tác `linearizable`, các ràng buộc về tính duy nhất, một `replication log`, và nhiều thứ khác.

Tuy nhiên, nếu leader duy nhất đó gặp lỗi, hoặc nếu một sự cố gián đoạn mạng khiến leader không thể kết nối được, hệ thống như vậy sẽ không thể tiến triển cho đến khi một con người thực hiện `failover` thủ công. Các thuật toán consensus được sử dụng rộng rãi như Raft và Paxos về bản chất là `single-leader replication` với cơ chế tự động bầu chọn leader và `failover` được tích hợp sẵn nếu leader hiện tại gặp lỗi.

Các thuật toán consensus được thiết kế cẩn thận để đảm bảo rằng không có `write` nào đã được `commit` bị mất trong quá trình `failover` và hệ thống không rơi vào trạng thái `split-brain`, nơi mà nhiều node cùng chấp nhận các `write`. Điều này đòi hỏi rằng mọi `write`, và mọi `linearizable read`, đều phải được xác nhận bởi một `quorum` (thường là đa số) các node. Việc này có thể tốn kém, đặc biệt là giữa các vùng địa lý khác nhau, nhưng nó là không thể tránh khỏi nếu bạn muốn có `strong consistency` và `fault tolerance` mà consensus cung cấp.

Các dịch vụ `coordination` như ZooKeeper và etcd cũng được xây dựng dựa trên các thuật toán consensus. Chúng cung cấp các tính năng như `lock`, `lease`, phát hiện lỗi và thông báo thay đổi, vốn rất hữu ích để quản lý trạng thái của các ứng dụng phân tán. Nếu bạn thấy mình muốn thực hiện một trong những việc có thể quy về consensus, và bạn muốn nó có khả năng `fault-tolerant`, lời khuyên là hãy sử dụng một dịch vụ `coordination`. Nó sẽ không đảm bảo rằng bạn sẽ làm đúng hoàn toàn, nhưng nó có lẽ sẽ giúp ích.

Các thuật toán consensus rất phức tạp và tinh vi, nhưng chúng được hỗ trợ bởi một hệ thống lý thuyết phong phú đã được phát triển từ những năm 1980. Lý thuyết này giúp việc xây dựng các hệ thống có thể chịu được tất cả các lỗi mà chúng ta đã thảo luận trong Chương 9 và vẫn đảm bảo rằng dữ liệu của bạn không bị hỏng. Đây là một thành tựu đáng kinh ngạc, và các tài liệu tham khảo ở cuối chương này sẽ trình bày một số điểm nổi bật của công trình này.

Tuy nhiên, consensus không phải lúc nào cũng là công cụ phù hợp. Trong một số hệ thống, các đặc tính `strong consistency` mà nó cung cấp là không cần thiết, và sẽ tốt hơn nếu có `consistency` yếu hơn với `availability` cao hơn và hiệu năng tốt hơn. Trong những trường hợp này, việc sử dụng `leaderless` hoặc `multi-leader replication`, thứ mà chúng ta đã thảo luận trong Chương 6, là rất phổ biến. Các `logical clock` mà chúng ta thảo luận trong chương này sẽ rất hữu ích trong bối cảnh đó.

Footnotes

References

- [1] Maurice P. Herlihy and Jeannette M. Wing. “Linearizability: A Correctness Condition for Concurrent Objects.” *ACM Transactions on Programming Languages and Systems* (TOPLAS), volume 12, issue 3, pages 463–492, July 1990. [doi:10.1145/78969.78972](#)
- [2] Leslie Lamport. “On Interprocess Communication.” *Distributed Computing*, volume 1, issue 2, pages 77–101, June 1986. [doi:10.1007/BF01786228](#)
- [3] David K. Gifford. “Information Storage in a Decentralized Computer System.” Xerox Palo Alto Research Centers, CSL-81-8, June 1981. Archived at [perma.cc/2XXP-3JPB](#)
- [4] Martin Kleppmann. “Please Stop Calling Databases CP or AP.” *martin.kleppmann.com*, May 2015. Archived at [perma.cc/MJ5G-75GL](#)
- [5] Kyle Kingsbury. “Jepsen: MongoDB Stale Reads.” *aphyr.com*, April 2015. Archived at [perma.cc/DXB4-J4JC](#)
- [6] Kyle Kingsbury. “Computational Techniques in Knossos.” *aphyr.com*, May 2014. Archived at [perma.cc/2X5M-EHTU](#)
- [7] Kyle Kingsbury and Peter Alvaro. “Elle: Inferring Isolation Anomalies from Experimental Observations.” *Proceedings of the VLDB Endowment*, volume 14, issue 3, pages 268–280, November 2020. [doi:10.14778/3430915.3430918](#)
- [8] Paolo Viotti and Marko Vukolić. “Consistency in Non-Transactional Distributed Storage Systems.” *ACM Computing Surveys* (CSUR), volume 49, issue 1, article no. 19, June 2016. [doi:10.1145/2926965](#)
- [9] Peter Bailis. “Linearizability Versus Serializability.” *bailis.org*, September 2014. Archived at [perma.cc/386B-KAC3](#)
- [10] Daniel Abadi. “Correctness Anomalies Under Serializable Isolation.” *dbmsmusings.blogspot.com*, June 2019. Archived at [perma.cc/JGS7-BZFY](#)
- [11] Peter Bailis, Aaron Davidson, Alan Fekete, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. “Highly Available Transactions: Virtues and Limitations.” *Proceedings of the VLDB Endowment*, volume 7, issue 3, pages 181–192, November 2013. [doi:10.14778/2732232.2732237](#), extended version published as [arXiv:1302.0309](#)
- [12] Philip A. Bernstein, Vassos Hadzilacos, and Nathan Goodman. *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987. ISBN: 9780201107159. Available online at [microsoft.com](#).
- [13] Andrei Matei. “CockroachDB’s Consistency Model.” *cockroachlabs.com*, February 2021. Archived at [perma.cc/MR38-883B](#)
- [14] Murat Demirbas. “Strict-Serializability, but at What Cost, for What Purpose?” *muratbuffalo.blogspot.com*, August 2022. Archived at [perma.cc/T84Y-N3U9](#)
- [15] Doug Judd. “Spanner Under the Hood: Understanding Strict Serializability and External Consistency.” *cloud.google.com*, April 2023. Archived at [perma.cc/KJ9F-BJ5T](#)
- [16] FoundationDB project authors. “Developer Guide.” *apple.github.io*. Archived at [perma.cc/F53L-TM9P](#)
- [17] Ben Darnell. “How to Talk About Consistency and Isolation in Distributed DBs.” *cockroachlabs.com*, February 2022. Archived at [perma.cc/53SV-JBGK](#)
- [18] Daniel Abadi. “An Explanation of the Difference Between Isolation Levels vs. Consistency Levels.” *dbmsmusings.blogspot.com*, August 2019. Archived at [perma.cc/QSF2-CD4P](#)

- [19] Mike Burrows. “The Chubby Lock Service for Loosely-Coupled Distributed Systems.” At *7th USENIX Symposium on Operating System Design and Implementation* (OSDI), November 2006.
- [20] Flavio P. Junqueira and Benjamin Reed. *ZooKeeper: Distributed Process Coordination*. O’Reilly Media, 2013. ISBN: 9781449361303
- [21] Murali Vallath. *Oracle 10g RAC Grid, Services & Clustering*. Elsevier Digital Press, 2006. ISBN: 9781555583217
- [22] Peter Bailis, Alan Fekete, Michael J. Franklin, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. “Coordination Avoidance in Database Systems.” *Proceedings of the VLDB Endowment*, volume 8, issue 3, pages 185–196, November 2014. doi:10.14778/2735508.2735509, extended version published as *arXiv:1402.2237*
- [23] Kyle Kingsbury. “Jepsen: etcd and Consul.” *aphyr.com*, June 2014. Archived at *perma.cc/XL7U-378K*
- [24] Flavio P. Junqueira, Benjamin C. Reed, and Marco Serafini. “Zab: High-Performance Broadcast for Primary-Backup Systems.” At *41st IEEE International Conference on Dependable Systems and Networks* (DSN), June 2011. doi:10.1109/DSN.2011.5958223
- [25] Diego Ongaro and John K. Ousterhout. “In Search of an Understandable Consensus Algorithm.” At *USENIX Annual Technical Conference* (ATC), June 2014.
- [26] Hagit Attiya, Amotz Bar-Noy, and Danny Dolev. “Sharing Memory Robustly in Message-Passing Systems.” *Journal of the ACM*, volume 42, issue 1, pages 124–142, January 1995. doi:10.1145/200836.200869
- [27] Nancy Lynch and Alex Shvartsman. “Robust Emulation of Shared Memory Using Dynamic Quorum-Acknowledged Broadcasts.” At *27th Annual International Symposium on Fault-Tolerant Computing* (FTCS), June 1997. doi:10.1109/FTCS.1997.614100
- [28] Christian Cachin, Rachid Guerraoui, and Luís Rodrigues. *Introduction to Reliable and Secure Distributed Programming*, 2nd edition. Springer, 2011. ISBN: 9783642152597, doi:10.1007/978-3-642-15260-3
- [29] Niklas Ekström, Mikhail Panchenko, and Jonathan Ellis. “Possible Issue with Read Repair?” Email thread on *cassandra-dev* mailing list, October 2012. Archived at *perma.cc/49GF-QMWA*
- [30] Maurice P. Herlihy. “Wait-Free Synchronization.” *ACM Transactions on Programming Languages and Systems* (TOPLAS), volume 13, issue 1, pages 124–149, January 1991. doi:10.1145/114005.102808
- [31] Armando Fox and Eric A. Brewer. “Harvest, Yield, and Scalable Tolerant Systems.” At *7th Workshop on Hot Topics in Operating Systems* (HotOS), March 1999. doi:10.1109/HOTOS.1999.798396
- [32] Seth Gilbert and Nancy Lynch. “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services.” *ACM SIGACT News*, volume 33, issue 2, pages 51–59, June 2002. doi:10.1145/564585.564601
- [33] Seth Gilbert and Nancy Lynch. “Perspectives on the CAP Theorem.” *IEEE Computer Magazine*, volume 45, issue 2, pages 30–36, February 2012. doi:10.1109/MC.2011.389
- [34] Eric A. Brewer. “CAP Twelve Years Later: How the ‘Rules’ Have Changed.” *IEEE Computer Magazine*, volume 45, issue 2, pages 23–29, February 2012. doi:10.1109/MC.2012.37
- [35] Susan B. Davidson, Hector Garcia-Molina, and Dale Skeen. “Consistency in Partitioned Networks.” *ACM Computing Surveys*, volume 17, issue 3, pages 341–370, September 1985. doi:10.1145/5505.5508
- [36] Paul R. Johnson and Robert H. Thomas. “RFC 677: The Maintenance of Duplicate Databases.” Network Working Group, January 1975.

- [37] Michael J. Fischer and Alan Michael. “Sacrificing Serializability to Attain High Availability of Data in an Unreliable Network.” At *1st ACM Symposium on Principles of Database Systems* (PODS), March 1982. doi:10.1145/588111.588124
- [38] Eric A. Brewer. “NoSQL: Past, Present, Future.” At *QCon San Francisco*, November 2012.
- [39] Eric Brewer. “Spanner, TrueTime & The CAP Theorem.” *research.google.com*, February 2017. Archived at perma.cc/59UW-RH7N
- [40] Daniel J. Abadi. “Consistency Tradeoffs in Modern Distributed Database System Design.” *IEEE Computer Magazine*, volume 45, issue 2, pages 37–42, February 2012. doi:10.1109/MC.2012.33
- [41] Nancy A. Lynch. “A Hundred Impossibility Proofs for Distributed Computing.” At *8th ACM Symposium on Principles of Distributed Computing* (PODC), August 1989. doi:10.1145/72981.72982
- [42] Prince Mahajan, Lorenzo Alvisi, and Mike Dahlin. “Consistency, Availability, and Convergence.” University of Texas at Austin, Department of Computer Science, Tech Report UTCS TR-11-22, May 2011. Archived at perma.cc/SAV8-9JAJ
- [43] Hagit Attiya, Faith Ellen, and Adam Morrison. “Limitations of Highly-Available Eventually-Consistent Data Stores.” At *ACM Symposium on Principles of Distributed Computing* (PODC), July 2015. doi:10.1145/2767386.2767419
- [44] Adrian Cockcroft. “Migrating to Microservices.” At *QCon London*, March 2014.
- [45] Martin Kleppmann. “A Critique of the CAP Theorem.” *arXiv:1509.05393*, September 2015.
- [46] Daniel Abadi. “Problems with CAP, and Yahoo’s Little Known NoSQL System.” *dbmsmusings.blogspot.com*, April 2010. Archived at perma.cc/4NTZ-CLM9
- [47] Daniel Abadi. “Hazelcast and the Mythical PA/EC System.” *dbmsmusings.blogspot.com*, October 2017. Archived at perma.cc/J5XM-USC2
- [48] Peter Sewell, Susmit Sarkar, Scott Owens, Francesco Zappa Nardelli, and Magnus O. Myreen. “x86-TSO: A Rigorous and Usable Programmer’s Model for x86 Multiprocessors.” *Communications of the ACM*, volume 53, issue 7, pages 89–97, July 2010. doi:10.1145/1785414.1785443
- [49] Martin Thompson. “Memory Barriers/Fences.” *mechanical-sympathy.blogspot.co.uk*, July 2011. Archived at perma.cc/7NXM-GC5U
- [50] Ulrich Drepper. “What Every Programmer Should Know About Memory.” *akkadia.org*, November 2007. Archived at perma.cc/NU6Q-DRXZ
- [51] Hagit Attiya and Jennifer L. Welch. “Sequential Consistency Versus Linearizability.” *ACM Transactions on Computer Systems* (TOCS), volume 12, issue 2, pages 91–122, May 1994. doi:10.1145/176575.176576
- [52] Kyzer R. Davis, Brad G. Peabody, and Paul J. Leach. “Universally Unique IDentifiers (UUIDs).” RFC 9562, IETF, May 2024.
- [53] Ryan King. “Announcing Snowflake.” *blog.x.com*, June 2010. Archived at archive.org
- [54] Alizain Feerasta. “Universally Unique Lexicographically Sortable Identifier.” *github.com*, 2016. Archived at perma.cc/NV2Y-ZP8U
- [55] Rob Conery. “A Better ID Generator for PostgreSQL.” *bigmachine.io*, May 2014. Archived at perma.cc/K7QV-3KFC
- [56] Leslie Lamport. “Time, Clocks, and the Ordering of Events in a Distributed System.” *Communications of the ACM*, volume 21, issue 7, pages 558–565, July 1978. doi:10.1145/359545.359563

- [57] Sandeep S. Kulkarni, Murat Demirbas, Deepak Madeppa, Bharadwaj Avva, and Marcelo Leone. “Logical Physical Clocks.” *18th International Conference on Principles of Distributed Systems* (OPODIS), December 2014. [doi:10.1007/978-3-319-14472-6_2](#)
- [58] Manuel Bravo, Nuno Diegues, Jingna Zeng, Paolo Romano, and Luís Rodrigues. “On the Use of Clocks to Enforce Consistency in the Cloud.” *IEEE Data Engineering Bulletin*, volume 38, issue 1, pages 18–31, March 2015. Archived at [perma.cc/68ZU-4SSH](#)
- [59] Daniel Peng and Frank Dabek. “Large-Scale Incremental Processing Using Distributed Transactions and Notifications.” At *9th USENIX Conference on Operating Systems Design and Implementation* (OSDI), October 2010.
- [60] Tushar Deepak Chandra, Robert Griesemer, and Joshua Redstone. “Paxos Made Live—An Engineering Perspective.” At *26th ACM Symposium on Principles of Distributed Computing* (PODC), June 2007. [doi:10.1145/1281100.1281103](#)
- [61] Will Portnoy. “Lessons Learned from Implementing Paxos.” *blog.willportnoy.com*, June 2012. Archived at [perma.cc/QHD9-FDD2](#)
- [62] Brian M. Oki and Barbara H. Liskov. “Viewstamped Replication: A New Primary Copy Method to Support Highly-Available Distributed Systems.” At *7th ACM Symposium on Principles of Distributed Computing* (PODC), August 1988. [doi:10.1145/62546.62549](#)
- [63] Barbara H. Liskov and James Cowling. “Viewstamped Replication Revisited.” Massachusetts Institute of Technology, Tech Report MIT-CSAIL-TR-2012-021, July 2012. Archived at [perma.cc/56SJ-WENQ](#)
- [64] Leslie Lamport. “The Part-Time Parliament.” *ACM Transactions on Computer Systems*, volume 16, issue 2, pages 133–169, May 1998. [doi:10.1145/279227.279229](#)
- [65] Leslie Lamport. “Paxos Made Simple.” *ACM SIGACT News*, volume 32, issue 4, pages 51–58, December 2001. Archived at [perma.cc/82HP-MNKE](#)
- [66] Robbert van Renesse and Deniz Altinbuken. “Paxos Made Moderately Complex.” *ACM Computing Surveys* (CSUR), volume 47, issue 3, article no. 42, February 2015. [doi:10.1145/2673577](#)
- [67] Diego Ongaro. “Consensus: Bridging Theory and Practice.” PhD thesis, Stanford University, August 2014. Archived at [perma.cc/5VTZ-2ADH](#)
- [68] Heidi Howard, Malte Schwarzkopf, Anil Madhavapeddy, and Jon Crowcroft. “Raft Refloated: Do We Have Consensus?” *ACM SIGOPS Operating Systems Review*, volume 49, issue 1, pages 12–21, January 2015. [doi:10.1145/2723872.2723876](#)
- [69] André Medeiros. “ZooKeeper’s Atomic Broadcast Protocol: Theory and Practice.” Aalto University School of Science, March 2012. Archived at [perma.cc/FVL4-JMVA](#)
- [70] Robbert van Renesse, Nicolas Schiper, and Fred B. Schneider. “Vive la Différence: Paxos vs. Viewstamped Replication vs. Zab.” *IEEE Transactions on Dependable and Secure Computing*, volume 12, issue 4, pages 472–484, September 2014. [doi:10.1109/TDSC.2014.2355848](#)
- [71] Heidi Howard and Richard Mortier. “Paxos vs Raft: Have We Reached Consensus on Distributed Consensus?” At *7th Workshop on Principles and Practice of Consistency for Distributed Data* (PaPoC), April 2020. [doi:10.1145/3380787.3393681](#)
- [72] Miguel Castro and Barbara H. Liskov. “Practical Byzantine Fault Tolerance and Proactive Recovery.” *ACM Transactions on Computer Systems*, volume 20, issue 4, pages 396–461, November 2002. [doi:10.1145/571637.571640](#)

- [73] Shehar Bano, Alberto Sonnino, Mustafa Al-Bassam, Sarah Azouvi, Patrick McCorry, Sarah Meiklejohn, and George Danezis. “SoK: Consensus in the Age of Blockchains.” At *1st ACM Conference on Advances in Financial Technologies (AFT)*, October 2019. doi:10.1145/3318041.3355458
- [74] Michael J. Fischer, Nancy Lynch, and Michael S. Paterson. “Impossibility of Distributed Consensus with One Faulty Process.” *Journal of the ACM*, volume 32, issue 2, pages 374–382, April 1985. doi:10.1145/3149.214121
- [75] Tushar Deepak Chandra and Sam Toueg. “Unreliable Failure Detectors for Reliable Distributed Systems.” *Journal of the ACM*, volume 43, issue 2, pages 225–267, March 1996. doi:10.1145/226643.226647
- [76] Michael Ben-Or. “Another Advantage of Free Choice: Completely Asynchronous Agreement Protocols.” At *2nd ACM Symposium on Principles of Distributed Computing (PODC)*, August 1983. doi:10.1145/800221.806707
- [77] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. “Consensus in the Presence of Partial Synchrony.” *Journal of the ACM*, volume 35, issue 2, pages 288–323, April 1988. doi:10.1145/42282.42283
- [78] Xavier Défago, André Schiper, and Péter Urbán. “Total Order Broadcast and Multicast Algorithms: Taxonomy and Survey.” *ACM Computing Surveys*, volume 36, issue 4, pages 372–421, December 2004. doi:10.1145/1041680.1041682
- [79] Hagit Attiya and Jennifer Welch. *Distributed Computing: Fundamentals, Simulations and Advanced Topics*, 2nd edition. John Wiley & Sons, 2004. ISBN: 9780471453246, doi:10.1002/0471478210
- [80] Rachid Guerraoui. “Revisiting the Relationship Between Non-Blocking Atomic Commitment and Consensus.” At *9th International Workshop on Distributed Algorithms (WDAG)*, September 1995. doi:10.1007/BFb0022140
- [81] Jim N. Gray and Leslie Lamport. “Consensus on Transaction Commit.” *ACM Transactions on Database Systems (TODS)*, volume 31, issue 1, pages 133–160, March 2006. doi:10.1145/1132863.1132867
- [82] Fred B. Schneider. “Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial.” *ACM Computing Surveys*, volume 22, issue 4, pages 299–319, December 1990. doi:10.1145/98163.98167
- [83] Alexander Thomson, Thaddeus Diamond, Shu-Chun Weng, Kun Ren, Philip Shao, and Daniel J. Abadi. “Calvin: Fast Distributed Transactions for Partitioned Database Systems.” At *ACM International Conference on Management of Data (SIGMOD)*, May 2012. doi:10.1145/2213836.2213838
- [84] Mahesh Balakrishnan, Dahlia Malkhi, Ted Wobber, Ming Wu, Vijayan Prabhakaran, Michael Wei, John D. Davis, Sriram Rao, Tao Zou, and Aviad Zuck. “Tango: Distributed Data Structures over a Shared Log.” At *24th ACM Symposium on Operating Systems Principles (SOSP)*, November 2013. doi:10.1145/2517349.2522732
- [85] Mahesh Balakrishnan, Dahlia Malkhi, Vijayan Prabhakaran, Ted Wobber, Michael Wei, and John D. Davis. “CORFU: A Shared Log Design for Flash Clusters.” At *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, April 2012.
- [86] Iulian Moraru, David G. Andersen, and Michael Kaminsky. “There Is More Consensus in Egalitarian Parliaments.” At *24th ACM Symposium on Operating Systems Principles (SOSP)*, November 2013. doi:10.1145/2517349.2517350

- [87] Vasilis Gavrielatos, Antonios Katsarakis, and Vijay Nagarajan. “Odyssey: the Impact of Modern Hardware on Strongly-Consistent Replication Protocols.” At *16th European Conference on Computer Systems* (EuroSys), April 2021. [doi:10.1145/3447786.3456240](#)
- [88] Heidi Howard, Dahlia Malkhi, and Alexander Spiegelman. “Flexible Paxos: Quorum Intersection Revisited.” At *20th International Conference on Principles of Distributed Systems* (OPODIS), December 2016. [doi:10.4230/LIPIcs.OPODIS.2016.25](#)
- [89] Martin Kleppmann. “Distributed Systems.” Lecture Notes. *University of Cambridge*, October 2024. Archived at [perma.cc/SS3Q-FNS5](#)
- [90] Kyle Kingsbury. “Jepsen: Elasticsearch 1.5.0.” *aphyr.com*, April 2015. Archived at [perma.cc/37MZ-JT7H](#)
- [91] Heidi Howard and Jon Crowcroft. “Coracle: Evaluating Consensus at the Internet Edge.” At *Annual Conference of the ACM Special Interest Group on Data Communication* (SIGCOMM), August 2015. [doi:10.1145/2829988.2790010](#)
- [92] Tom Lianza and Chris Snook. “A Byzantine failure in the Real World.” *blog.cloudflare.com*, November 2020. Archived at [perma.cc/83EZ-ALCY](#)
- [93] Ivan Kelly. “BookKeeper Tutorial.” *github.com*, October 2014. Archived at [perma.cc/37Y6-VZWU](#)
- [94] Jack Vanlighty. “Apache BookKeeper Insights Part 1—External Consensus and Dynamic Membership.” *medium.com*, November 2021. Archived at [perma.cc/3MDB-8GFB](#)

Batch Processing

Một hệ thống không thể thành công nếu nó bị ảnh hưởng quá mạnh mẽ bởi một cá nhân duy nhất. Một khi thiết kế ban đầu đã hoàn tất và tương đối vững chắc, thử thách thực sự mới bắt đầu khi những người với nhiều quan điểm khác nhau tiến hành các thử nghiệm của riêng họ.

Donald Knuth, “The Errors of TeX” (1989)

Phần lớn cuốn sách này cho đến nay đã thảo luận về các *request* (yêu cầu) và *query* (truy vấn) cùng với các *response* (phản hồi) hoặc *result* (kết quả) tương ứng. Kiểu xử lý dữ liệu này được giả định trong nhiều hệ thống dữ liệu hiện đại: bạn yêu cầu một thứ gì đó, hoặc bạn gửi một chỉ thị, và hệ thống cố gắng đưa ra câu trả lời cho bạn nhanh nhất có thể.

Một trình duyệt web yêu cầu một trang, một service gọi một API từ xa, các cơ sở dữ liệu, cache, search index và nhiều hệ thống khác hoạt động theo cách này. Chúng ta gọi đây là các *online systems*. Response time thường là thước đo hiệu năng chính của chúng, và chúng thường yêu cầu fault tolerance để đảm bảo tính sẵn sàng cao.

Tuy nhiên, đôi khi bạn cần thực hiện một phép tính lớn hơn hoặc xử lý lượng dữ liệu lớn hơn mức bạn có thể thực hiện trong một request tương tác. Có lẽ bạn cần huấn luyện một mô hình AI, hoặc biến đổi lượng lớn dữ liệu từ dạng này sang dạng khác, hoặc tính toán phân tích trên một dataset cực lớn. Chúng ta gọi các tác vụ này là các công việc *batch processing*, và các hệ thống xử lý chúng đôi khi được gọi là *offline systems*.

Một công việc batch processing nhận dữ liệu đầu vào (vốn chỉ cho phép đọc) và tạo ra dữ liệu đầu ra (vốn được tạo mới hoàn toàn mỗi khi công việc chạy). Nó thường không làm thay đổi dữ liệu theo cách mà một transaction đọc/ghi sẽ thực hiện. Do đó, đầu ra được *derived* (phái sinh) từ đầu vào (như đã thảo luận trong mục “Systems of Record and Derived Data”). Nếu bạn không thích đầu ra, bạn có thể xóa nó, điều chỉnh logic của công việc và chạy lại công việc đó.

Bằng cách coi các đầu vào là bất biến và tránh các side effect (chẳng hạn như việc ghi vào các cơ sở dữ liệu bên ngoài), các batch job đạt được hiệu năng tốt cũng như các lợi ích khác:

- Nếu bạn đưa một bug vào mã nguồn và đầu ra bị sai hoặc bị lỗi, bạn chỉ đơn giản là có thể roll back về một phiên bản mã nguồn trước đó và chạy lại công việc, khi đó đầu ra sẽ chính xác trở lại. Hoặc, thậm chí đơn giản hơn, bạn có thể giữ đầu ra

cũ trong một thư mục khác và chuyển lại về nó. Hầu hết các object store và các định dạng bảng mở (xem mục “Cloud Data Warehouses”) đều hỗ trợ tính năng này, được gọi là *time travel*. Hầu hết các cơ sở dữ liệu với các transaction đọc/ghi đều không có đặc tính này: nếu bạn deploy mã nguồn bị lỗi khiến nó ghi dữ liệu sai vào cơ sở dữ liệu, việc roll back mã nguồn sẽ không giúp gì để sửa dữ liệu đó. Ý tưởng về việc có thể khôi phục từ mã nguồn bị lỗi được gọi là *human fault tolerance* [1].

- Hệ quả của sự dễ dàng khi roll back này là việc phát triển feature có thể tiến hành nhanh chóng hơn so với trong một môi trường mà các sai lầm có thể dẫn đến những thiệt hại không thể đảo ngược. Nguyên tắc *minimizing irreversibility* (giảm thiểu tính không thể đảo ngược) này rất có lợi cho phát triển phần mềm Agile [2].
- Cùng một tập hợp các tệp có thể được sử dụng làm đầu vào cho nhiều loại công việc khác nhau, bao gồm cả các công việc monitoring để tính toán các metric và đánh giá xem đầu ra của một công việc có các feature như mong đợi hay không (ví dụ, bằng cách so sánh nó với đầu ra từ lần chạy trước và đo lường các sai lệch).
- Các framework batch processing sử dụng hiệu quả các tài nguyên tính toán. Mặc dù có thể thực hiện batch-process dữ liệu thông qua các hệ thống dữ liệu online như các cơ sở dữ liệu OLTP và các application server, nhưng làm như vậy có thể tốn kém hơn nhiều về mặt tài nguyên yêu cầu.

Batch processing đã chứng minh được sự hữu ích trong một phạm vi rộng lớn các trường hợp sử dụng, điều mà chúng ta sẽ xem xét lại trong mục “Batch Use Cases”. Tuy nhiên, nó cũng đặt ra những thách thức. Với hầu hết các framework, đầu ra chỉ có thể được xử lý bởi các công việc khác sau khi toàn bộ công việc kết thúc. Batch processing cũng có thể không hiệu quả; bất kỳ thay đổi nào đối với dữ liệu đầu vào—ngay cả một byte duy nhất—cũng yêu cầu batch job phải xử lý lại toàn bộ dataset đầu vào.

Một batch job có thể mất nhiều thời gian để chạy: vài phút, vài giờ, hoặc thậm chí vài ngày. Các job có thể được lập lịch để chạy định kỳ (ví dụ: mỗi ngày một lần). Thước đo hiệu năng chính thường là throughput: lượng dữ liệu mà job có thể xử lý trong một đơn vị thời gian. Một số hệ thống batch xử lý lỗi bằng cách chỉ đơn giản là hủy bỏ và khởi động lại toàn bộ job, trong khi những hệ thống khác có fault tolerance để một job có thể hoàn thành thành công bất chấp việc một số node của nó bị crash.

Ranh giới giữa các hệ thống online và batch processing không phải lúc nào cũng rõ ràng; một truy vấn cơ sở dữ liệu chạy lâu (long-running) trông khá giống với một batch process. Nhưng batch processing cũng có những đặc điểm riêng biệt khiến nó trở thành một thành phần xây dựng (building block) hữu ích để tạo ra các ứng dụng có độ tin cậy, khả năng mở rộng và khả năng bảo trì cao. Ví dụ, nó thường đóng vai trò trong *tích hợp dữ liệu (data integration)*—kết hợp nhiều hệ thống dữ liệu để đạt

được những điều mà một hệ thống đơn lẻ không thể làm được. ETL, như đã thảo luận trong mục “Data Warehousing”, là một ví dụ về điều này.

LƯU Ý

Một giải pháp thay thế cho batch processing là *stream processing*, trong đó job không kết thúc chạy khi đã xử lý xong đầu vào, mà thay vào đó tiếp tục theo dõi đầu vào và xử lý các thay đổi trong đầu vào ngay sau khi chúng xảy ra. Chúng ta sẽ chuyển sang stream processing trong Chương 12.

Batch processing hiện đại chịu ảnh hưởng mạnh mẽ bởi MapReduce, một thuật toán batch processing được Google công bố vào năm 2004 [3] và sau đó được triển khai trong nhiều hệ thống dữ liệu mã nguồn mở khác nhau, bao gồm Hadoop, CouchDB và MongoDB. MapReduce là một mô hình lập trình khá ở mức thấp (low-level), ít tinh vi hơn các engine thực thi truy vấn song song thường thấy, ví dụ, trong các data warehouse [4, 5]. Khi mới ra đời, MapReduce là một bước tiến về quy mô xử lý có thể đạt được trên commodity hardware, nhưng hiện nay nó phần lớn đã lỗi thời và không còn được sử dụng tại Google [6, 7].

Batch processing ngày nay thường được thực hiện bằng cách sử dụng các framework như Spark hoặc Flink, hoặc các engine truy vấn data warehouse. Giống như MapReduce, chúng phụ thuộc nhiều vào sharding (xem Chương 7) và thực thi song song, nhưng chúng có các chiến lược caching và thực thi tinh vi hơn nhiều. Khi các hệ thống này đã trưởng thành, các vấn đề về vận hành đã phần lớn được giải quyết, vì vậy trọng tâm đã chuyển sang khả năng sử dụng. Các mô hình xử lý mới như các API dataflow, ngôn ngữ truy vấn và DataFrame API hiện đang được hỗ trợ rộng rãi. Việc orchestration job và workflow cũng đã trưởng thành. Các bộ lập lịch workflow tập trung vào Hadoop như Oozie và Azkaban đã được thay thế bằng các giải pháp tổng quát hơn như Airflow, Dagster và Prefect, vốn hỗ trợ một loạt các framework batch processing và cloud data warehouse.

Điện toán đám mây đã trở nên phổ biến ở khắp mọi nơi. Các lớp lưu trữ batch đang chuyển dịch từ các hệ thống tệp phân tán (DFSs) như HDFS (Hadoop Distributed File System), GlusterFS và CephFS sang các hệ thống object storage như S3. Các cloud data warehouse có khả năng mở rộng như BigQuery và Snowflake đang làm mờ đi ranh giới giữa data warehouse và batch processing.

Để xây dựng trực giác về bản chất của batch processing, chúng ta sẽ bắt đầu chương này với một ví dụ sử dụng các công cụ Unix tiêu chuẩn trên một máy đơn lẻ. Sau đó, chúng ta sẽ nghiên cứu cách mở rộng việc xử lý dữ liệu lên nhiều máy trong một hệ thống phân tán. Chúng ta sẽ thấy rằng, giống như một hệ điều hành, các framework distributed batch processing có một scheduler và một filesystem. Sau đó, chúng ta sẽ khám phá các mô hình xử lý khác nhau mà chúng ta sử dụng để viết các batch job. Cuối cùng, chúng ta sẽ thảo luận về các trường hợp sử dụng batch processing phổ biến.

Batch Processing với các công cụ Unix

Giả sử bạn có một web server thực hiện thêm một dòng vào một file log mỗi khi nó phục vụ một request. Ví dụ, sử dụng định dạng access log mặc định của NGINX, một dòng của log có thể trông như thế này:

```
216.58.210.78 - - [27/Jun/2025:17:55:11 +0000] "GET /css/
typography.css HTTP/1.1"
200 3377 "https://martin.kleppmann.com/" "Mozilla/5.0
(Macintosh; Intel Mac OS X
10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/
137.0.0.0 Safari/537.36"
```

(Đó thực chất là một dòng; nó được chia thành nhiều dòng ở đây để dễ đọc.) Có rất nhiều thông tin trong dòng đó. Để giải mã nó, bạn cần xem định nghĩa của định dạng log, cụ thể như sau:

```
$remote_addr - $remote_user [$time_local] "$request"
$status $body_bytes_sent "$http_referer" "$http_user_agent"
```

Vì vậy, dòng log này cho biết vào ngày 27 tháng 6 năm 2025, lúc 17:55:11 UTC, máy chủ đã nhận được một yêu cầu cho tệp `/css/typography.css` từ địa chỉ IP client 216.58.210.78. Người dùng không được xác thực, vì vậy `$remote_user` được đặt thành một dấu gạch ngang (-). Trạng thái phản hồi là 200 (tức là yêu cầu đã thành công), và phản hồi có kích thước 3.377 bytes. Trình duyệt web là Chrome 137, và nó đã tải tệp vì tệp được tham chiếu trong trang tại URL **https://martin.kleppmann.com/**.

Mặc dù việc phân tích log (log parsing) có vẻ như được dàn dựng, nhưng nó là một phần quan trọng trong hoạt động của nhiều công ty công nghệ hiện đại và được sử dụng cho mọi thứ, từ các pipeline quảng cáo đến xử lý thanh toán. Thật vậy, nó chính là động lực thúc đẩy việc áp dụng nhanh chóng MapReduce và phong trào "big data".

Phân tích log Đơn giản

Nhiều công cụ khác nhau có thể lấy các tệp log này và tạo ra các báo cáo đẹp mắt về lưu lượng truy cập website của bạn, nhưng để phục vụ mục đích thực hành, hãy tự xây dựng cái của riêng mình bằng cách sử dụng các công cụ Unix cơ bản. Ví dụ, giả sử bạn muốn tìm năm trang phổ biến nhất trên website của mình. Bạn có thể thực hiện việc này trong một Unix shell như sau:

```
cat /var/log/nginx/access.log | .
awk '{print $7}' | .
sort | .
uniq -c | .
sort -r -n | .
head -n 5 .
```

- Đọc tệp log. (Nói một cách chính xác, `cat` là không cần thiết ở đây, vì tệp đầu vào có thể được đưa trực tiếp làm đối số cho `awk`. Tuy nhiên, linear pipeline sẽ rõ ràng hơn khi được viết như thế này.)
- Chia mỗi dòng thành các trường (fields) bằng khoảng trắng, và chỉ xuất ra trường thứ bảy từ mỗi dòng, vốn chính là URL được yêu cầu. Trong dòng ví dụ của chúng ta, URL này là `/css/typography.css`.
- Sắp xếp `sort` danh sách các URL được yêu cầu theo thứ tự bảng chữ cái. Lý do của việc sắp xếp là để đảm bảo rằng, nếu một URL đã được yêu cầu n lần, tệp đã sắp xếp sẽ chứa cùng một URL đó lặp lại n lần liên tiếp.
- Lệnh `uniq` lọc bỏ các dòng lặp lại trong đầu vào bằng cách kiểm tra xem hai dòng liền kề có giống nhau hay không. Tùy chọn `-c` yêu cầu nó cũng xuất ra một bộ đếm: đối với mỗi URL riêng biệt, nó sẽ báo cáo xem URL đó đã xuất hiện bao nhiêu lần trong đầu vào.
- Lệnh `sort` thứ hai sắp xếp theo số $(-n)$ ở đầu mỗi dòng, chính là số lần URL đó được yêu cầu. Sau đó, nó trả về kết quả theo thứ tự ngược $(-r)$ —với số lớn nhất nằm ở đầu.
- Cuối cùng, `head` chỉ xuất ra năm dòng đầu tiên $(-n\ 5)$ của đầu vào và loại bỏ phần còn lại.

Kết quả của chuỗi lệnh đó trông giống như thế này:

```
4189 /favicon.ico
3631 /2016/02/08/how-to-do-distributed-locking.html
2124 /2020/11/18/distributed-systems-and-elliptic-curves.html
1369 /
915 /css/typography.css
```

Mặc dù dòng lệnh phía trước có vẻ hơi khó hiểu nếu bạn không quen thuộc với các công cụ Unix, nhưng nó cực kỳ mạnh mẽ. Nó sẽ xử lý hàng gigabytes tệp log chỉ trong vài giây, và bạn có thể dễ dàng sửa đổi việc phân tích để phù hợp với nhu cầu của mình. Ví dụ, nếu bạn muốn loại bỏ các tệp CSS khỏi báo cáo, bạn có thể thay đổi đối số `awk` thành `$7 !~ /\.css$/ {print $7}`, và nếu bạn muốn đếm các địa chỉ IP client hàng đầu thay vì các trang hàng đầu, bạn có thể thay đổi đối số `awk` thành `{print $1}`, và cứ tiếp tục như vậy.

Chúng ta không có đủ không gian trong cuốn sách này để khám phá chi tiết các công cụ Unix, nhưng chúng rất đáng để tìm hiểu. Nhiều phân tích dữ liệu có thể được

thực hiện trong vài phút bằng cách kết hợp `awk`, `sed`, `grep`, `sort`, `uniq`, và `xargs`, và chúng hoạt động hiệu quả một cách đáng ngạc nhiên [8].

Chuỗi lệnh So với Chương trình Tùy chỉnh

Thay vì sử dụng chuỗi lệnh Unix, bạn có thể viết một chương trình đơn giản để thực hiện cùng một việc. Ví dụ, trong Python, nó có thể trông giống như thế này:

```
from collections import defaultdict

counts = defaultdict(int)

with open('/var/log/nginx/access.log', 'r') as file:
    for line in file:
        url = line.split()[6]
        counts[url] += 1

top5 = sorted(((count, url) for url, count in counts.items()),
              reverse=True)[:5]

for count, url in top5:
    print(f"{count} {url}")
```

- Khởi tạo `counts` là một hash table dùng để lưu trữ bộ đếm cho số lần chúng ta thấy mỗi URL. Giá trị ban đầu của mỗi bộ đếm là 0.
- Lấy URL được yêu cầu, vốn là trường thứ bảy được phân tách bằng khoảng trắng, từ mỗi dòng của log (chỉ số mảng là 6 vì mảng trong Python được đánh chỉ số từ 0).
- Tăng bộ đếm cho URL trong dòng hiện tại của log.
- Sắp xếp nội dung hash table theo giá trị bộ đếm (giảm dần), và lấy ra năm mục đầu tiên.
- In ra năm mục đầu tiên đó.

Chương trình này không súc tích như chuỗi lệnh Unix, nhưng nó khá dễ đọc, và việc bạn thích cái nào hơn một phần là do sở thích cá nhân. Tuy nhiên, bên cạnh những khác biệt cú pháp bề ngoài giữa hai cách, có một sự khác biệt lớn trong luồng thực thi, điều này sẽ trở nên rõ ràng nếu bạn chạy phân tích này trên một tệp lớn.

Sắp xếp so với Tổng hợp trong bộ nhớ (In-Memory Aggregation)

Script Python duy trì một hash table trong bộ nhớ chứa các URL, trong đó mỗi URL được ánh xạ tới số lần nó đã xuất hiện. Ví dụ về Unix pipeline không có hash table như vậy, mà thay vào đó dựa vào việc sắp xếp một danh sách các URL, trong đó nhiều lần xuất hiện của cùng một URL đơn giản là được lặp lại.

Cách tiếp cận nào tốt hơn? Điều đó phụ thuộc vào số lượng URL mà bạn có. Đối với hầu hết các website quy mô nhỏ đến trung bình, bạn có thể nạp vừa tất cả các URL riêng biệt, cùng với một bộ đếm cho mỗi URL, vào (chẳng hạn) 1 GB bộ nhớ. Trong ví dụ này, *working set* của công việc (lượng bộ nhớ mà công việc cần truy cập ngẫu nhiên) chỉ phụ thuộc vào số lượng các URL riêng biệt. Nếu có một triệu mục nhật ký cho một URL duy nhất, không gian yêu cầu trong hash table vẫn chỉ là một URL cộng với kích thước của bộ đếm. Nếu *working set* này đủ nhỏ, một hash table trong bộ nhớ sẽ hoạt động tốt—ngay cả trên một máy tính xách tay.

Mặt khác, nếu *working set* của công việc lớn hơn bộ nhớ hiện có, cách tiếp cận sắp xếp có lợi thế là nó có thể sử dụng đĩa một cách hiệu quả. Đó chính là nguyên lý mà chúng ta đã thảo luận trong mục “Log-Structured Storage”: các khối dữ liệu có thể được sắp xếp trong bộ nhớ và ghi ra đĩa dưới dạng các segment file, sau đó nhiều segment đã sắp xếp có thể được hợp nhất thành một tệp đã sắp xếp lớn hơn. Mergesort có các mẫu truy cập tuần tự giúp hoạt động tốt trên đĩa (xem mục “Sequential Versus Random Writes on SSDs”).

Tiện ích *sort* trong GNU Coreutils (Linux) tự động xử lý các dataset lớn hơn bộ nhớ bằng cách tràn dữ liệu xuống đĩa và tự động song song hóa việc sắp xếp trên nhiều lõi CPU [9]. Điều này có nghĩa là chuỗi lệnh Unix đơn giản mà chúng ta đã thấy trước đó có thể dễ dàng mở rộng cho các dataset lớn mà không bị hết bộ nhớ. Điểm nghẽn (bottleneck) có khả năng sẽ nằm ở tốc độ mà tệp đầu vào có thể được đọc từ đĩa.

Một hạn chế của các công cụ Unix là chúng chạy trên một máy duy nhất. Các dataset quá lớn để có thể nằm gọn trong bộ nhớ hoặc trên đĩa cục bộ sẽ gây ra vấn đề—và đó là lúc các framework xử lý batch phân tán xuất hiện.

Xử lý Batch trong các Hệ thống Phân tán

Máy chạy ví dụ về các công cụ Unix của chúng ta có một số thành phần cùng phối hợp để xử lý dữ liệu nhật ký:

- Các thiết bị lưu trữ được truy cập thông qua giao diện filesystem của hệ điều hành

- Một bộ lập lịch (scheduler) quyết định khi nào các tiến trình được chạy và cách phân bổ tài nguyên CPU cho chúng
- Một chuỗi các chương trình Unix có đầu vào tiêu chuẩn (standard input) và đầu ra tiêu chuẩn (standard output) (`stdin` và `stdout`) được kết nối với nhau bằng các pipe

Chính những thành phần này cũng tồn tại trong các framework xử lý dữ liệu phân tán. Trên thực tế, bạn có thể coi các framework này như là các hệ điều hành phân tán; chúng có filesystem, bộ lập lịch công việc (job schedulers), và các chương trình gửi dữ liệu cho nhau thông qua filesystem hoặc các kênh giao tiếp khác.

Filesystem Phân tán

Filesystem được cung cấp bởi hệ điều hành của bạn bao gồm nhiều lớp:

- Ở cấp thấp nhất, các trình điều khiển thiết bị khối (block device drivers) giao tiếp trực tiếp với đĩa và cho phép các lớp phía trên đọc và ghi các block thô.
- Phía trên lớp block là một page cache giúp giữ các block vừa được truy cập trong bộ nhớ để truy cập nhanh hơn.
- block API được bao bọc trong một lớp filesystem, lớp này chia các tệp lớn thành các block và theo dõi metadata của tệp như inodes, thư mục và các tệp. Ví dụ, hai triển khai phổ biến trên Linux là ext4 và XFS.
- Cuối cùng, hệ điều hành cung cấp các filesystem khác nhau cho các ứng dụng thông qua một API chung được gọi là *virtual filesystem* (VFS). VFS là thứ cho phép các ứng dụng đọc và ghi theo một cách tiêu chuẩn bất kể filesystem nền tảng là gì.

Các distributed filesystem hoạt động theo cách gần như tương tự. Các tệp được chia thành các block, vốn được phân phối trên nhiều máy. Các block của DFS thường lớn hơn nhiều so với các block cục bộ. HDFS mặc định là 128 MB, trong khi JuiceFS và nhiều object store sử dụng các block 4 MB—lớn hơn nhiều so với 4.096 bytes của ext4. Các block lớn hơn đồng nghĩa với việc có ít metadata hơn cần phải theo dõi, điều này tạo ra sự khác biệt lớn đối với các dataset quy mô petabyte. Các block lớn hơn cũng làm giảm chi phí overhead khi tìm kiếm (seeking) một block so với việc đọc nó.

Hầu hết các thiết bị lưu trữ vật lý không thể ghi các block không đầy đủ, vì vậy các hệ điều hành yêu cầu các thao tác ghi phải sử dụng toàn bộ một block ngay cả khi dữ liệu không chiếm hết block đó. Vì các distributed filesystem có các block lớn hơn và thường được triển khai trên nền tảng các filesystem của hệ điều hành, chúng không gặp phải yêu cầu này. Ví dụ, một tệp 900 MB được lưu trữ với các block 128 MB sẽ có bảy block sử dụng 128 MB và một block sử dụng 4 MB.

Các block của DFS được đọc bằng cách thực hiện các yêu cầu mạng tới một máy trong cluster đang lưu trữ block đó. Mỗi máy chạy một daemon, cung cấp một API cho phép các tiến trình từ xa có thể đọc và ghi các block dưới dạng các tệp trên filesystem cục bộ của nó. HDFS gọi các daemon này là DataNodes, trong khi GlusterFS gọi chúng là các tiến trình glusterfsd. Chúng ta sẽ gọi chúng là *data nodes* trong cuốn sách này.

Các distributed filesystem cũng triển khai một cơ chế tương đương với page cache của hệ thống phân tán. Vì các block của DFS được lưu trữ dưới dạng các tệp trên các data nodes, các thao tác đọc và ghi sẽ đi qua hệ điều hành của từng data node, vốn bao gồm một page cache trong bộ nhớ. Điều này giúp giữ các block dữ liệu thường xuyên được đọc trong bộ nhớ trên các data nodes. Một số distributed filesystem cũng triển khai thêm các tầng caching khác, chẳng hạn như client-side caching và local disk caching được tìm thấy trong JuiceFS.

Các filesystem như ext4 và XFS theo dõi metadata lưu trữ bao gồm không gian trống, vị trí các block của tệp, cấu trúc thư mục, thiết lập quyền hạn, và nhiều thứ khác. Các distributed filesystem cũng cần một cách để theo dõi vị trí các tệp nằm rải rác trên các máy, thiết lập quyền hạn, v.v. Hadoop có một dịch vụ gọi là NameNode để duy trì metadata cho cluster. 3FS của DeepSeek có một dịch vụ metadata giúp lưu trữ dữ liệu của nó vào một key-value store như FoundationDB.

Nằm phía trên filesystem là VFS. Một sự tương đồng gần nhất trong batch processing là protocol của một distributed filesystem. Các distributed filesystem phải cung cấp một protocol hoặc interface để các hệ thống batch processing có thể đọc và ghi các tệp. Protocol này đóng vai trò như một interface có thể thay thế (pluggable interface); bất kỳ DFS nào cũng có thể được sử dụng miễn là nó triển khai đúng protocol đó. Ví dụ, API của Amazon S3 đã được áp dụng rộng rãi bởi các hệ thống lưu trữ như MinIO, R2 của Cloudflare, Tigris, B2 của Backblaze, và nhiều hệ thống khác. Các hệ thống batch processing có hỗ trợ S3 có thể sử dụng bất kỳ hệ thống lưu trữ nào trong số này.

Một số DFS triển khai các filesystem tuân thủ POSIX, vốn hiển thị với VFS của hệ điều hành giống như bất kỳ filesystem nào khác. Filesystem in Userspace (FUSE) hoặc protocol Network File System (NFS) thường được sử dụng để tích hợp vào VFS. NFS có lẽ là protocol distributed filesystem nổi tiếng nhất. Protocol này ban đầu được phát triển để cho phép nhiều client có thể đọc và ghi dữ liệu trên một máy chủ duy nhất. Gần đây hơn, các filesystem như Amazon Elastic File System (EFS) và Archil cung cấp các triển khai distributed filesystem tương thích với NFS với khả năng mở rộng cao hơn nhiều. Các client NFS vẫn kết nối tới một endpoint, nhưng bên trong (underneath), các hệ thống này giao tiếp với các dịch vụ metadata phân tán và các data nodes để đọc và ghi dữ liệu.

Distributed Filesystems và Network Storage

Các distributed filesystem dựa trên nguyên tắc *shared-nothing* (xem “Kiến trúc Shared-Memory, Shared-Disk, và Shared-Nothing”), trái ngược với cách tiếp cận shared-disk của các kiến trúc network attached storage (NAS) và storage area network (SAN). Lưu trữ shared-disk được triển khai bởi một thiết bị lưu trữ tập trung, thường sử dụng phần cứng tùy chỉnh và hạ tầng mạng đặc biệt như Fibre Channel. Mặt khác, cách tiếp cận shared-nothing không yêu cầu phần cứng đặc biệt, chỉ cần các máy tính được kết nối bởi một mạng datacenter thông thường.

Nhiều distributed filesystem được xây dựng trên commodity hardware, loại phần cứng này ít tốn kém hơn nhưng có tỷ lệ lỗi cao hơn so với phần cứng cấp doanh nghiệp. Để chịu được các lỗi máy tính và lỗi đĩa, các block dữ liệu được replication trên nhiều máy tính. Điều này cũng cho phép các scheduler phân phối workload đồng đều hơn vì chúng có thể thực thi một tác vụ trên bất kỳ node nào chứa một replica của dữ liệu đầu vào của tác vụ đó.

Replication có thể đơn giản là việc giữ nhiều bản sao của cùng một dữ liệu trên nhiều máy tính, như đã mô tả trong Chương 6, hoặc sử dụng một lược đồ *erasure coding* (mã hóa xóa) như mã Reed–Solomon, cho phép khôi phục dữ liệu bị mất với chi phí lưu trữ thấp hơn so với full replication [10, 11, 12]. Các kỹ thuật này tương tự như RAID, vốn cung cấp tính dư thừa trên nhiều đĩa được gắn vào cùng một máy; điểm khác biệt là trong một distributed filesystem, việc truy cập và replication được thực hiện qua một mạng datacenter thông thường mà không cần phần cứng đặc biệt.

Object Stores

Các dịch vụ object storage như Amazon S3, Google Cloud Storage, Azure Blob Storage và OpenStack Swift đã trở thành một lựa chọn thay thế phổ biến cho distributed filesystem đối với các công việc batch processing. Trên thực tế, ranh giới giữa hai loại này có phần mờ nhạt. Như chúng ta đã thấy trong mục trước và “Databases Backed by Object Storage”, các driver FUSE cho phép người dùng đối xử với các object store như S3 như một filesystem. Một số triển khai DFS, chẳng hạn như JuiceFS và Ceph, cung cấp cả object storage và các API filesystem. Tuy nhiên, các API, hiệu năng và các đảm bảo về tính nhất quán của chúng rất khác nhau. Cần phải cẩn trọng khi áp dụng các hệ thống như vậy để đảm bảo chúng hoạt động như mong đợi, ngay cả khi chúng có vẻ như đang triển khai các API cần thiết.

Mỗi object trong một object store có một URL chẳng hạn như `s3://my-photo-bucket/2025/04/01/birthday.png`. Phần host của URL (*my-photo-bucket*) mô tả bucket nơi các object được lưu trữ, và phần theo sau là *key* của object đó (`/2025/04/01/`

birthday.png trong ví dụ của chúng ta). Một bucket có tên duy nhất trên phạm vi toàn cầu, và key của mỗi object phải là duy nhất trong bucket của nó.

Các object được đọc bằng một lệnh gọi `get` và được ghi bằng một lệnh gọi `put`. Không giống như các tệp trên một filesystem, các object là immutable một khi đã được ghi. Để cập nhật một object, nó phải được ghi lại hoàn toàn bằng một lệnh gọi `put`, tương tự như một key-value store. Azure Blob Storage và S3 Express One Zone hỗ trợ appends, nhưng hầu hết các store khác thì không. Không có các API file handle với các hàm như `fopen` và `fseek`.

Các object có vẻ như được tổ chức thành các thư mục, điều này có chút gây nhầm lẫn vì các object store không có khái niệm về thư mục. Cấu trúc đường dẫn đơn giản chỉ là một quy ước, và các dấu gạch chéo là một phần của key của object. Quy ước này cho phép bạn thực hiện một thao tác tương tự như directory listing bằng cách yêu cầu một danh sách các object với một prefix cụ thể. Tuy nhiên, việc listing các object theo prefix khác với directory listing của một filesystem ở hai điểm:

- Một thao tác prefix `List` hoạt động giống như một lệnh gọi `ls -R` đệ quy trên hệ thống Unix. Nó trả về tất cả các object bắt đầu bằng prefix đó, và các object trong các subpaths cũng được bao gồm.
- Các thư mục trống là không thể tồn tại. Nếu bạn xóa tất cả các object bên dưới `s3://my-photo-bucket/2025/04/01`, thì `01` sẽ không còn xuất hiện khi bạn gọi `List` trên `s3://my-photo-bucket/2025/04`. Một thực hành tốt (best practice) phổ biến là tạo một object có kích thước zero-byte như một cách để đại diện cho một thư mục trống (ví dụ: tạo một file `s3://my-photo-bucket/2025/04/01` trống để giữ nó hiện diện khi tất cả các object con bị xóa).

Các triển khai DFS thường hỗ trợ các thao tác filesystem phổ biến như hard links, symbolic links, file locking và atomic renames. Những tính năng như vậy không có trong các object store. Việc linking và locks thường không được hỗ trợ, trong khi renames thì không có tính nguyên tử (nonatomic); chúng được thực hiện bằng cách sao chép object sang key mới rồi sau đó xóa object cũ. Nếu bạn muốn đổi tên một thư mục, bạn phải đổi tên từng object bên trong nó, vì tên thư mục là một phần của key.

Các key-value store mà chúng ta đã thảo luận trong Chương 4 được tối ưu hóa cho các giá trị nhỏ (thường là kilobytes) và các thao tác đọc/ghi thường xuyên với độ trễ (latency) thấp. Ngược lại, các distributed filesystems và object stores thường được tối ưu hóa cho các object lớn (từ megabytes đến gigabytes) và các thao tác đọc ít thường xuyên hơn nhưng với kích thước lớn hơn. Tuy nhiên, gần đây, các object stores đã bắt đầu bổ sung hỗ trợ cho các thao tác đọc/ghi thường xuyên và nhỏ hơn. Ví dụ, S3 Express One Zone hiện cung cấp độ trễ chỉ trong một mili giây và một mô hình định giá tương tự hơn với các key-value stores.

Một sự khác biệt khác giữa distributed filesystems và object stores là các DFS như HDFS cho phép các tác vụ tính toán được chạy trên chính máy chủ lưu trữ một bản sao của một file cụ thể. Điều này cho phép tác vụ đọc file đó mà không cần phải gửi nó qua mạng, giúp tiết kiệm băng thông nếu mã thực thi của tác vụ nhỏ hơn file mà nó cần đọc. Mặt khác, các object stores thường tách biệt lưu trữ và tính toán. Làm như vậy có thể sử dụng nhiều băng thông hơn, nhưng các mạng datacenter hiện đại rất nhanh, nên điều này thường có thể chấp nhận được. Kiến trúc này cũng cho phép các tài nguyên máy chủ CPU và bộ nhớ có thể được scale độc lập với lưu trữ vì cả hai đã được decouple.

Điều phối công việc phân tán (Distributed Job Orchestration)

Phép ẩn dụ về hệ điều hành của chúng ta cũng áp dụng cho việc điều phối công việc (job orchestration). Khi bạn thực thi một Unix batch job, cần có thứ gì đó thực sự chạy các tiến trình `awk`, `sort`, `uniq`, và `head`. Dữ liệu cần được chuyển từ đầu ra của tiến trình này sang đầu vào của tiến trình khác, bộ nhớ phải được cấp phát cho mỗi tiến trình, các chỉ thị từ mỗi tiến trình phải được lập lịch một cách công bằng và thực thi trên CPU, các ranh giới bộ nhớ và I/O phải được thực thi, và vân vân. Trên một máy đơn lẻ, kernel của hệ điều hành chịu trách nhiệm cho những công việc như vậy. Trong một môi trường phân tán, đây là vai trò của một job orchestrator.

Các framework batch processing gửi một yêu cầu đến scheduler của orchestrator để chạy một job. Các yêu cầu bắt đầu một job chứa các metadata như sau:

- Số lượng tác vụ cần thực thi
- Lượng bộ nhớ, CPU và đĩa cứng cần thiết cho mỗi tác vụ
- Một định danh job
- Thông tin xác thực (access credentials)
- Các tham số job như dữ liệu đầu vào và đầu ra
- Các chi tiết phần cứng yêu cầu như GPU hoặc các loại đĩa
- Vị trí của mã thực thi của job

Các orchestrators như Kubernetes và Hadoop YARN (Yet Another Resource Negotiator) [13] kết hợp thông tin này với metadata của cluster để thực thi job bằng cách sử dụng các thành phần sau:

Các task executors

Một executor daemon như *NodeManager* của YARN hoặc *kubelet* của Kubernetes chạy trên mỗi node trong cluster. Các executor chịu trách nhiệm chạy các tác vụ của job, gửi heartbeats để báo hiệu trạng thái sống (liveness), và theo dõi trạng thái tác vụ cũng như việc cấp phát tài nguyên trên node. Khi một yêu cầu bắt đầu tác vụ được gửi tới một executor, nó sẽ lấy mã thực thi của job và chạy

một lệnh để bắt đầu tác vụ. Sau đó, executor giám sát tiến trình cho đến khi nó hoàn thành hoặc thất bại, tại thời điểm đó nó sẽ cập nhật metadata trạng thái của tác vụ một cách tương ứng.

Nhiều executor cũng làm việc với hệ điều hành để cung cấp cả khả năng cách ly bảo mật và hiệu năng. Ví dụ, YARN và Kubernetes sử dụng *cgroups* của Linux. Điều này ngăn chặn các task truy cập dữ liệu mà không có quyền hoặc gây ảnh hưởng tiêu cực đến hiệu năng của các task khác trên node bằng cách sử dụng quá mức tài nguyên.

Resource manager

Resource manager của một orchestrator lưu trữ metadata về mỗi node, bao gồm phần cứng hiện có (CPU, GPU, bộ nhớ, đĩa cứng, v.v.), trạng thái của task, vị trí mạng, trạng thái của node và các thông tin liên quan khác. Do đó, manager cung cấp một cái nhìn tổng thể về trạng thái hiện tại của cluster. Bản chất tập trung của resource manager có thể dẫn đến các điểm nghẽn về cả khả năng mở rộng và độ tin cậy. YARN sử dụng ZooKeeper, và Kubernetes sử dụng etcd, để lưu trữ trạng thái của cluster (xem mục “Coordination Services”).

Scheduler

Các orchestrator thường có một subsystem scheduler tập trung, nơi tiếp nhận các yêu cầu để bắt đầu, dừng hoặc kiểm tra trạng thái của một job. Ví dụ, một scheduler có thể nhận được yêu cầu bắt đầu một job với 10 task sử dụng một Docker image cụ thể trên các node có một loại GPU cụ thể. Scheduler sử dụng thông tin từ yêu cầu và trạng thái của resource manager để quyết định chạy các task nào trên những node nào. Sau đó, các task executor sẽ được thông báo về công việc được giao và bắt đầu thực thi.

Mặc dù mỗi orchestrator sử dụng các thuật ngữ khác nhau, bạn sẽ tìm thấy các thành phần này trong hầu hết các hệ thống orchestration.

LƯU Ý

Các quyết định scheduling đôi khi yêu cầu các scheduler đặc thù cho ứng dụng để có thể tính đến các yêu cầu cụ thể, chẳng hạn như autoscaling các read replica khi đạt đến một ngưỡng truy vấn nhất định. Scheduler tập trung và các scheduler đặc thù cho ứng dụng làm việc cùng nhau để xác định cách thực thi các task tốt nhất. YARN gọi các subscheduler của nó là *ApplicationMasters*, trong khi Kubernetes gọi chúng là *operators*.

Resource allocation

Các scheduler đóng một vai trò đặc biệt thách thức trong việc orchestration job. Chúng phải tìm cách phân bổ tối ưu các tài nguyên hữu hạn của cluster cho các job có các nhu cầu cạnh tranh nhau. Về cơ bản, các quyết định này phải cân bằng giữa tính công bằng và hiệu năng.

Hãy tưởng tượng một cluster nhỏ với năm node có tổng cộng 160 lõi CPU khả dụng. Scheduler của cluster nhận được hai yêu cầu job, mỗi yêu cầu muốn có 100 lõi để hoàn thành công việc của mình. Cách tốt nhất để schedule workload này là gì?

- Scheduler có thể quyết định chạy 80 task cho mỗi job, sau đó bắt đầu 20 task còn lại của mỗi job khi các task trước đó hoàn thành.
- Scheduler có thể chạy tất cả các task của một job, sau đó chỉ bắt đầu chạy các task của job thứ hai khi có sẵn 100 lõi (một chiến lược được gọi là *gang scheduling*).
- Nếu yêu cầu job thứ hai đến muộn hơn nhiều so với job đầu tiên, scheduler sẽ có thông tin không đầy đủ. Nó phải quyết định xem nên phân bổ toàn bộ 100 lõi cho job đầu tiên, hay giữ lại một số lõi để chờ đợi một job trong tương lai có thể sẽ đến hoặc không.

Đây là một ví dụ rất đơn giản, nhưng chúng ta đã thấy nhiều sự đánh đổi (trade-off) khó khăn. Ví dụ, trong kịch bản gang scheduling, nếu scheduler giữ dự trữ các lõi CPU cho đến khi tất cả 100 lõi sẵn sàng cùng một lúc, các node sẽ ở trạng thái rảnh rỗi. Việc sử dụng tài nguyên của cluster sẽ giảm xuống, và một deadlock có thể xảy ra nếu các job khác cũng cố gắng dự trữ các lõi CPU.

Mặt khác, nếu scheduler chỉ đơn giản chờ đợi cho đến khi 100 lõi sẵn sàng, các job khác có thể chiếm lấy các lõi đó trong thời gian chờ đợi. Cluster có thể không có sẵn 100 lõi trong một thời gian rất dài, dẫn đến tình trạng *starvation*. Ngoài ra, scheduler có thể quyết định *preempt* một số task của job đầu tiên, dùng chúng để nhường chỗ cho job thứ hai. Tuy nhiên, việc task preemption cũng làm giảm hiệu năng của cluster, vì các task bị dừng sẽ cần phải được khởi động lại sau đó.

Bây giờ hãy tưởng tượng một scheduler phải đưa ra các quyết định phân bổ cho hàng trăm hoặc thậm chí hàng triệu yêu cầu job như vậy. Việc tìm ra một giải pháp tối ưu có vẻ là bất khả thi. Trên thực tế, vấn đề này là *NP-hard*, nghĩa là việc tính toán một giải pháp tối ưu cho tất cả các ví dụ, ngoại trừ những ví dụ nhỏ nhất, sẽ chậm đến mức không thể thực hiện được [14, 15].

Trong thực tế, do đó các scheduler sử dụng các heuristic (phương pháp kinh nghiệm) để đưa ra các quyết định không tối ưu nhưng hợp lý. Một số thuật toán thường được sử dụng, bao gồm first-in first-out (FIFO), dominant resource fairness (DRF), priority queues, lập lịch dựa trên capacity hoặc quota, và các thuật toán bin-packing khác nhau. Chi tiết về các thuật toán này nằm ngoài phạm vi của cuốn sách này, nhưng chúng là một lĩnh vực nghiên cứu đầy thú vị.

Scheduling workflows

Ví dụ về các công cụ Unix trong mục “Simple Log Analysis” bao gồm một chuỗi gồm nhiều lệnh, được kết nối bởi các Unix pipe. Mô hình tương tự cũng xuất hiện trong các batch process phân tán: thông thường đầu ra từ một job cần trở thành đầu

vào cho một hoặc nhiều job khác, và mỗi job có thể có nhiều đầu vào được tạo ra bởi các job khác. Điều này được gọi là một *workflow* hoặc *directed acyclic graph* (DAG) của các job.

LƯU Ý

Trong mục “Durable Execution and Workflows”, chúng ta đã thấy các workflow engine cung cấp khả năng thực thi bền vững (durable execution) của một chuỗi các bước, thường là thực hiện các RPC. Trong bối cảnh batch processing, “workflow” có một ý nghĩa khác: nó là một chuỗi các batch process, mỗi bước nhận dữ liệu đầu vào và tạo ra dữ liệu đầu ra, nhưng thông thường không thực hiện các RPC tới các dịch vụ bên ngoài. Các durable execution engine thường xử lý ít dữ liệu hơn trên mỗi yêu cầu so với các đối trọng batch processing của chúng, mặc dù ranh giới giữa chúng có phần mờ nhạt.

Một workflow gồm nhiều job có thể cần thiết vì một vài lý do sau:

- Nếu đầu ra của một job cần trở thành đầu vào cho nhiều job khác mà các job này được duy trì bởi các nhóm khác nhau, tốt nhất là job đầu tiên nên ghi đầu ra của nó vào một vị trí mà tất cả các job khác đều có thể đọc được. Các job tiêu thụ (consuming jobs) sau đó có thể được lập lịch để chạy mỗi khi dữ liệu đó được cập nhật hoặc theo một lịch trình khác.
- Bạn có thể muốn chuyển dữ liệu từ công cụ xử lý này sang công cụ xử lý khác. Ví dụ, một Spark job có thể xuất dữ liệu của nó ra HDFS, sau đó một script Python có thể kích hoạt một truy vấn Trino SQL (xem “Cloud Data Warehouses”) để thực hiện xử lý thêm trên các tệp HDFS và xuất kết quả ra S3.
- Một số data pipeline yêu cầu nhiều giai đoạn xử lý nội bộ. Ví dụ, nếu một giai đoạn cần sharding dữ liệu theo một key, và giai đoạn tiếp theo cần sharding theo một key khác, thì giai đoạn đầu tiên có thể xuất dữ liệu đã được sharding theo cách mà giai đoạn thứ hai yêu cầu.

Trong ví dụ về các công cụ Unix, pipe kết nối đầu ra của một lệnh với đầu vào của một lệnh khác chỉ sử dụng một bộ đệm (buffer) nhỏ trong bộ nhớ và không ghi dữ liệu vào một tệp. Nếu bộ đệm đó bị đầy, process tạo ra (producing process) cần phải đợi cho đến khi process tiêu thụ (consuming process) đã đọc một phần dữ liệu từ bộ đệm trước khi nó có thể xuất thêm—đây là một dạng của backpressure. Spark, Flink và các batch execution engine khác hỗ trợ một mô hình tương tự, nơi đầu ra của một task được chuyển trực tiếp tới một task khác (qua mạng nếu các task đang chạy trên các máy khác nhau).

Tuy nhiên, thông thường một job trong một workflow sẽ ghi đầu ra của nó vào một distributed filesystem hoặc object store và job tiếp theo sẽ đọc nó từ đó. Điều này giúp tách rời (decouple) các job với nhau, cho phép chúng chạy vào các thời điểm khác nhau. Nếu một job có nhiều đầu vào, một workflow scheduler thường sẽ đợi

cho đến khi tất cả các job tạo ra các đầu vào đó hoàn thành thành công trước khi chạy job tiêu thụ các đầu vào đó.

Các scheduler được tìm thấy trong các orchestration framework như ResourceManager của YARN hoặc scheduler tích hợp sẵn của Spark không quản lý toàn bộ workflow; chúng thực hiện scheduling trên cơ sở từng job riêng lẻ. Để xử lý các phụ thuộc này giữa các lần thực thi job, nhiều workflow scheduler khác nhau đã được phát triển, bao gồm Airflow, Dagster và Prefect. Các workflow scheduler có các tính năng quản lý hữu ích khi bảo trì một tập hợp lớn các batch job. Các workflow bao gồm từ 50 đến 100 job là phổ biến trong nhiều data pipeline, và trong một tổ chức lớn, nhiều nhóm có thể đang chạy các job hoặc workflow mà đọc output của nhau qua nhiều hệ thống. Sự hỗ trợ từ công cụ là rất quan trọng để quản lý các dataflow phức tạp như vậy.

Xử lý lỗi

Các batch job thường chạy trong khoảng thời gian dài. Các job chạy lâu với nhiều task song song có khả năng gặp ít nhất một lỗi task trong quá trình thực hiện. Như đã thảo luận trong các mục “Hardware and Software Faults” và “Unreliable Networks”, điều này có thể xảy ra vì nhiều lý do, bao gồm lỗi hardware (đặc biệt là trên commodity hardware) hoặc gián đoạn mạng.

Một lý do khác khiến một task có thể không hoàn thành việc chạy là scheduler có thể chủ động preempt (ngắt) nó. Preemption đặc biệt hữu ích nếu bạn có nhiều mức độ ưu tiên, chẳng hạn như các task ưu tiên thấp có chi phí chạy rẻ hơn và các task ưu tiên cao có chi phí đắt hơn. Các task ưu tiên thấp có thể chạy bất cứ khi nào có tài nguyên tính toán dư thừa, nhưng chúng đối mặt với rủi ro bị preempt vào bất kỳ lúc nào nếu một task ưu tiên cao hơn xuất hiện. Các máy ảo ưu tiên thấp và rẻ hơn như vậy được gọi là *spot instances* trên Amazon EC2, *spot virtual machines* trên Azure, và *preemptible instances* trên Google Cloud [16].

Vì batch processing thường được sử dụng cho các job không nhạy cảm về thời gian, nó rất phù hợp để sử dụng các task ưu tiên thấp và spot instances nhằm giảm chi phí chạy job. Về cơ bản, các job đó có thể sử dụng các tài nguyên tính toán dư thừa mà nếu không chúng sẽ ở trạng thái nhàn rỗi, từ đó tăng mức độ sử dụng của cluster. Tuy nhiên, điều này cũng có nghĩa là các task đó có nhiều khả năng bị scheduler ngắt hơn, vì các vụ preempt xảy ra thường xuyên hơn so với lỗi hardware [17].

Vì các batch job tái tạo output của chúng từ đầu mỗi khi được chạy, các lỗi task để xử lý hơn so với trong các hệ thống online. Hệ thống có thể xóa output một phần từ lần thực thi bị lỗi và lập lịch cho task chạy lại trên một máy khác. Tuy nhiên, sẽ rất lãng phí nếu phải chạy lại toàn bộ job chỉ vì một lỗi task duy nhất. Do đó, MapReduce và các thể hệ kế cận giữ cho việc thực thi các task song song độc lập với nhau, để chúng có thể thử lại công việc ở mức độ chi tiết của từng task riêng lẻ [3].

Fault tolerance trở nên phức tạp hơn khi output của một task trở thành input của một task khác như một phần của workflow. MapReduce giải quyết vấn đề này bằng cách luôn ghi các dữ liệu trung gian như vậy trở lại distributed filesystem và chờ task ghi hoàn tất thành công trước khi cho phép các task khác đọc dữ liệu. Cách này hoạt động tốt, ngay cả trong một môi trường mà preemption là phổ biến, nhưng nó đồng nghĩa với việc phải thực hiện rất nhiều lượt ghi vào DFS, điều này có thể không hiệu quả.

Spark giữ dữ liệu trung gian trong memory (và "spilling" nó ra local disk nếu không đủ chỗ), và chỉ ghi kết quả cuối cùng vào DFS. Nó cũng theo dõi cách dữ liệu trung gian được tính toán, cho phép Spark tính toán lại trong trường hợp dữ liệu bị mất [18]. Flink sử dụng một cách tiếp cận khác dựa trên việc định kỳ checkpointing một snapshot của các task [19]. Chúng ta sẽ quay lại chủ đề này trong mục "Dataflow Engines".

Các mô hình Batch Processing

Chúng ta đã thấy cách các batch job được lập lịch trong một môi trường phân tán. Bây giờ hãy chuyển sự chú ý sang cách các framework batch processing xử lý dữ liệu. Hai mô hình phổ biến nhất là MapReduce và dataflow engines. Mặc dù các dataflow engines phần lớn đã thay thế MapReduce trong thực tế, nhưng việc hiểu cách MapReduce hoạt động là rất hữu ích vì nó đã ảnh hưởng đến nhiều framework batch processing hiện đại.

MapReduce và các dataflow engine đã phát triển để hỗ trợ nhiều mô hình lập trình khác nhau, bao gồm các API lập trình cấp thấp, các ngôn ngữ truy vấn quan hệ và các API DataFrame. Nhiều lựa chọn khác nhau cho phép các kỹ sư ứng dụng, kỹ sư phân tích, nhà phân tích kinh doanh và thậm chí cả những nhân viên không chuyên về kỹ thuật có thể xử lý dữ liệu công ty cho nhiều trường hợp sử dụng khác nhau, điều mà chúng ta sẽ thảo luận trong mục "Các trường hợp sử dụng Batch (Batch Use Cases)".

MapReduce

Mô hình xử lý dữ liệu trong MapReduce rất giống với ví dụ phân tích log web server trong mục "Phân tích log đơn giản (Simple Log Analysis)":

1. Đọc một tập hợp các tệp đầu vào và chia chúng thành các *record*. Trong ví dụ log web server, mỗi record là một dòng trong log (tức là, \n là ký tự phân tách record). Trong MapReduce của Hadoop, tệp đầu vào được lưu trữ trong một hệ thống tệp phân tán như HDFS hoặc một object store như S3. Nhiều định dạng tệp khác nhau được sử dụng, chẳng hạn như Apache Parquet (một định dạng dạng cột, xem mục "Lưu trữ hướng cột (Column-Oriented Storage)") hoặc Apache Avro (một định dạng dựa trên dòng, xem mục "Avro").

2. Gọi hàm mapper để trích xuất một key và value từ mỗi record đầu vào. Trong ví dụ về các công cụ Unix, hàm mapper là `awk '{print $7}'`, hàm này trích xuất URL (\$7) làm key và để trống value.
3. Sắp xếp tất cả các cặp key-value theo key. Trong ví dụ về log, việc này được thực hiện bởi lệnh `sort` đầu tiên.
4. Gọi hàm reducer để lặp qua các cặp key-value đã được sắp xếp. Nếu có nhiều lần xuất hiện của cùng một key, việc sắp xếp đã làm cho chúng nằm cạnh nhau trong danh sách, vì vậy việc kết hợp các giá trị đó rất dễ dàng mà không cần phải giữ nhiều state trong bộ nhớ. Trong ví dụ về các công cụ Unix, reducer được triển khai bởi lệnh `uniq -c`, lệnh này đếm số lượng các record liên tiếp có cùng một key.

Bốn bước đó có thể được thực hiện bởi một MapReduce job. Bước 2 (map) và bước 4 (reduce) là nơi bạn viết mã xử lý dữ liệu tùy chỉnh của mình. Bước 1 (chia tệp thành các record) được xử lý bởi input format parser. Bước 3, bước `sort`, là bước ngầm định trong MapReduce—bạn không cần phải viết nó, bởi vì đầu ra từ mapper luôn được sắp xếp trước khi được đưa cho reducer. Bước sắp xếp này là một thuật toán `batch processing` nền tảng, mà chúng ta sẽ xem lại trong mục “Xáo trộn dữ liệu (Shuffling Data)”.

Để tạo một MapReduce job, bạn cần triển khai hai callback function là mapper và reducer, chúng hoạt động như sau:

Mapper

Mapper được gọi một lần cho mỗi record đầu vào, và nhiệm vụ của nó là trích xuất key và value từ record đó. Đối với mỗi đầu vào, nó có thể tạo ra bất kỳ số lượng cặp key-value nào (bao gồm cả việc không tạo ra cặp nào). Nó không giữ bất kỳ state nào từ record đầu vào này sang record tiếp theo, vì vậy mỗi record được xử lý độc lập. Có thể có nhiều mapper chạy song song trên các phần khác nhau của đầu vào.

Reducer

Framework MapReduce lấy các cặp key-value được tạo ra bởi các mapper, thu thập tất cả các value thuộc về cùng một key, và gọi reducer với một iterator trên tập hợp các value đó. Reducer có thể tạo ra các record đầu ra (chẳng hạn như số lần xuất hiện của cùng một URL). Các reducer cho các key khác nhau cũng có thể được chạy song song.

Trong ví dụ log web server, chúng ta đã có lệnh `sort` thứ hai ở bước 5, lệnh này xếp hạng các URL theo số lượng request. Trong MapReduce, nếu bạn cần một giai đoạn sắp xếp thứ hai, bạn có thể triển khai nó bằng cách viết một MapReduce job thứ hai và sử dụng đầu ra của job thứ nhất làm đầu vào cho job thứ hai. Nhìn theo cách này,

vai trò của mapper là chuẩn bị dữ liệu bằng cách đưa nó vào một dạng phù hợp để sắp xếp, và vai trò của reducer là xử lý dữ liệu đã được sắp xếp.

MapReduce và Lập trình hàm (Functional Programming)

Mặc dù MapReduce được sử dụng để batch processing, nhưng mô hình lập trình này bắt nguồn từ lập trình hàm. Lisp đã giới thiệu map và reduce (hoặc fold) như là các higher-order functions trên các list, và chúng đã len lỏi vào các ngôn ngữ phổ biến như Python, Rust và Java.

Nhiều thao tác xử lý dữ liệu phổ biến, bao gồm cả những thao tác được cung cấp bởi SQL, đều có thể được triển khai dựa trên MapReduce. Nguyên tắc lập trình hàm (functional programming) về việc tránh trạng thái có thể thay đổi (mutable state) giúp cho phép thực thi song song. Vì mọi lời gọi đến mapper và reducer chỉ phụ thuộc vào dữ liệu mà framework MapReduce truyền một cách rõ ràng đến các hàm đó, nên framework có thể tự do chạy các lời gọi độc lập song song trên các node khác nhau. Và nếu một tác vụ thất bại, framework có thể tự do gọi lại mapper và reducer với cùng một đầu vào trên một node khác.

Việc triển khai một công việc xử lý phức tạp bằng cách sử dụng các API MapReduce thô thực sự khá tốn công sức—ví dụ, bất kỳ thuật toán join nào được sử dụng bởi công việc đó đều cần phải được triển khai từ đầu [20]. MapReduce cũng khá chậm so với các bộ xử lý batch hiện đại hơn. Một lý do là I/O dựa trên tệp của nó ngăn cản việc pipelining (tức là xử lý dữ liệu đầu ra trong một công việc hạ nguồn trước khi công việc thượng nguồn hoàn tất).

Dataflow Engines

Để khắc phục một số vấn đề của MapReduce, một số engine thực thi mới cho các tính toán batch phân tán đã được phát triển, trong đó nổi tiếng nhất là Spark [18, 21] và Flink [19]. Chúng được thiết kế khác nhau, nhưng có một điểm chung: chúng xử lý toàn bộ workflow như một công việc duy nhất, thay vì chia nhỏ nó thành các subjob độc lập.

Vì chúng mô hình hóa một cách rõ ràng luồng dữ liệu qua nhiều giai đoạn xử lý, các hệ thống này được gọi là *dataflow engines*. Giống như MapReduce, chúng hỗ trợ một API cấp thấp gọi lặp đi lặp lại một hàm do người dùng định nghĩa để xử lý từng bản ghi một, nhưng chúng cũng cung cấp các toán tử cấp cao hơn như *join* và *group by*. Chúng song song hóa công việc bằng cách sharding đầu vào, và chúng sao chép đầu ra của một tác vụ qua mạng để trở thành đầu vào cho một tác vụ khác. Không giống như trong MapReduce, các toán tử không cần phải đảm nhận các vai trò

nghiêm ngặt là luân phiên giữa map và reduce, mà thay vào đó có thể được lắp ghép theo những cách linh hoạt hơn.

Các API dataflow này thường sử dụng các khối xây dựng kiểu quan hệ để biểu diễn một tính toán: join các dataset dựa trên giá trị của một field; nhóm các tuple theo key; lọc theo một điều kiện; và tổng hợp các tuple bằng cách đếm, cộng, hoặc các hàm khác. Về mặt nội bộ, các thao tác này được triển khai bằng cách sử dụng các thuật toán shuffle mà chúng ta sẽ thảo luận trong mục tiếp theo.

Kiểu engine xử lý này dựa trên các hệ thống nghiên cứu như Dryad [22] và Nephelê [23], và nó mang lại một số lợi thế so với mô hình MapReduce:

- Các công việc tốn kém như sắp xếp chỉ cần được thực hiện ở những nơi thực sự yêu cầu, thay vì luôn luôn diễn ra theo mặc định giữa mỗi giai đoạn map và reduce.
- Khi có nhiều toán tử liên tiếp không làm thay đổi việc sharding của dataset (chẳng hạn như map hoặc filter), chúng có thể được kết hợp thành một tác vụ duy nhất, giúp giảm chi phí sao chép dữ liệu.
- Vì tất cả các join và sự phụ thuộc dữ liệu trong một workflow đều được khai báo rõ ràng, bộ lập lịch (scheduler) có cái nhìn tổng thể về việc dữ liệu nào được yêu cầu ở đâu, nhờ đó nó có thể thực hiện các tối ưu hóa về tính cục bộ (locality). Ví dụ, nó có thể cố gắng đặt tác vụ tiêu thụ dữ liệu trên cùng một máy chủ với tác vụ tạo ra dữ liệu đó, để dữ liệu có thể được trao đổi thông qua một bộ đệm bộ nhớ dùng chung thay vì phải sao chép qua mạng.
- Thông thường, chỉ cần giữ trạng thái trung gian giữa các toán tử trong bộ nhớ hoặc ghi vào đĩa cục bộ là đủ, điều này yêu cầu ít I/O hơn so với việc ghi nó vào một hệ thống tệp phân tán hoặc object store (nơi nó phải được replication tới nhiều máy và ghi vào đĩa trên mỗi replica). MapReduce đã sử dụng tối ưu hóa này cho đầu ra của mapper, nhưng các dataflow engines đã tổng quát hóa ý tưởng này cho tất cả các trạng thái trung gian.
- Các toán tử có thể bắt đầu thực thi ngay khi đầu vào của chúng đã sẵn sàng; không cần phải đợi toàn bộ giai đoạn trước đó kết thúc trước khi giai đoạn tiếp theo bắt đầu.
- Các tiến trình hiện có có thể được tái sử dụng để chạy các toán tử mới, giúp giảm chi phí khởi động so với MapReduce (vốn khởi chạy một JVM mới cho mỗi tác vụ).

Bạn có thể sử dụng các dataflow engine để thực hiện các tính toán tương tự như các workflow MapReduce, và chúng thường thực thi nhanh hơn đáng kể nhờ vào các tối ưu hóa được mô tả ở đây.

Shuffling dữ liệu

Như chúng ta đã thấy, cả ví dụ về các công cụ Unix ở đầu chương và MapReduce đều dựa trên việc sắp xếp. Các batch processor cần có khả năng sắp xếp các dataset có kích thước lên tới petabyte, vốn quá lớn để có thể chứa vừa trên một máy đơn lẻ. Do đó, chúng yêu cầu một thuật toán sắp xếp phân tán, nơi cả đầu vào và đầu ra đều được sharding. Một thuật toán như vậy được gọi là *shuffle*.

Shuffle không phải là ngẫu nhiên

Thuật ngữ *shuffle* có khả năng gây nhầm lẫn. Khi bạn tráo (shuffle) một bộ bài, bạn sẽ nhận được một thứ tự ngẫu nhiên. Ngược lại, quá trình shuffle mà chúng ta đang thảo luận tạo ra một thứ tự đã được sắp xếp, không có tính ngẫu nhiên.

Shuffling là một thuật toán nền tảng cho các batch processor, nơi nó được sử dụng cho các phép join và aggregation. MapReduce, Spark, Flink, Daft, Dataflow và BigQuery [24] đều triển khai các thuật toán shuffle có khả năng mở rộng và hiệu năng cao để xử lý các dataset lớn. Chúng ta sẽ sử dụng shuffle trong Hadoop MapReduce [25] để minh họa, nhưng các khái niệm trong mục này cũng có thể áp dụng cho các hệ thống khác.

Hình 11-1 cho thấy dataflow trong một công việc MapReduce. Chúng ta giả định rằng đầu vào của công việc đã được sharding, và các shard được gán nhãn m_1 , m_2 , và m_3 . Ví dụ, mỗi shard có thể là một tệp riêng biệt trên HDFS hoặc một object riêng biệt trong một object store, và tất cả các shard thuộc cùng một dataset được nhóm vào cùng một thư mục HDFS hoặc có cùng một key prefix trong một bucket của object store.

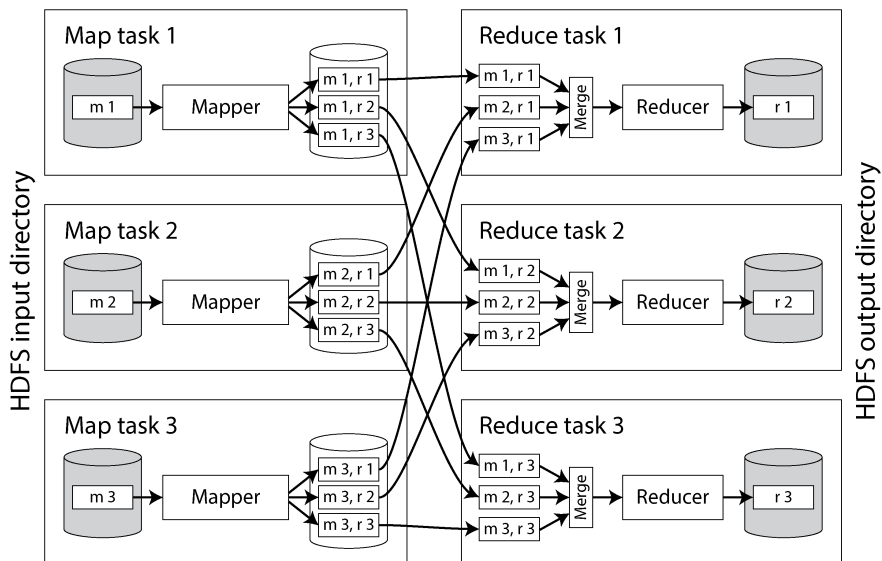


Figure 11-1. Một job MapReduce với ba mapper và ba reducer

Framework bắt đầu một map task riêng biệt cho mỗi input shard. Một task sẽ đọc tệp được chỉ định của nó, truyền từng bản ghi một tới mapper callback. Phía reduce của quá trình tính toán cũng được sharding. Trong khi số lượng map task được quyết định bởi số lượng input shards, thì số lượng reduce task được cấu hình bởi tác giả của job (nó có thể khác với số lượng map tasks).

Output của mapper bao gồm các cặp key-value, và framework cần đảm bảo rằng nếu hai mapper cùng xuất ra một key giống nhau, các cặp key-value đó cuối cùng sẽ được xử lý bởi cùng một reducer task. Để đạt được điều này, mỗi mapper tạo một tệp output riêng biệt trên đĩa cục bộ của nó cho mỗi reducer (ví dụ, tệp *m 1, r 2* trong Hình 11-1 là tệp được tạo bởi mapper 1 chứa dữ liệu dành cho reducer 2). Khi mapper xuất ra một cặp key-value, một mã hash của key thường sẽ quyết định nó được ghi vào tệp reducer nào (tương tự như quy trình được mô tả trong mục “Sharding by Hash of Key”).

Trong khi mapper đang ghi các tệp này, nó cũng sắp xếp các cặp key-value bên trong mỗi tệp. Điều này có thể được thực hiện bằng các kỹ thuật mà chúng ta đã thấy trong mục “Log-Structured Storage”: các batch key-value trước tiên được thu thập trong một cấu trúc dữ liệu đã sắp xếp trong bộ nhớ, sau đó được ghi ra dưới dạng các segment files đã sắp xếp, và các segment files nhỏ hơn sẽ dần dần được merge thành các tệp lớn hơn.

Sau khi mỗi mapper hoàn tất, các reducer kết nối với nó và sao chép tệp chứa các cặp key-value đã sắp xếp phù hợp về đĩa cục bộ của chúng. Một khi reduce task đã có đủ phần output từ tất cả các mapper, nó sẽ merge các tệp này lại với nhau, giữ nguyên

thứ tự sắp xếp theo kiểu mergesort. Các cặp key-value có cùng key giờ đây sẽ nằm liên tiếp nhau, ngay cả khi chúng đến từ các mapper khác nhau. Hàm reducer sau đó được gọi một lần cho mỗi key, mỗi lần với một iterator trả về tất cả các giá trị cho key đó.

Bất kỳ bản ghi nào được xuất bởi hàm reducer sẽ được ghi tuần tự vào một tệp, với mỗi reduce task một tệp riêng. Các tệp này (r_1 , r_2 , và r_3 trong Hình 11-1) trở thành các shards của output dataset của job, và chúng được ghi ngược lại vào distributed filesystem hoặc object store.

Mặc dù MapReduce thực thi bước shuffle giữa các bước map và reduce của nó, các dataflow engine và cloud data warehouse hiện đại còn tinh vi hơn. Các hệ thống như BigQuery đã tối ưu hóa các thuật toán shuffle của chúng để giữ dữ liệu trong bộ nhớ và ghi dữ liệu vào các dịch vụ sorting bên ngoài [24]. Các dịch vụ như vậy giúp tăng tốc việc shuffling và nhân bản dữ liệu đã được shuffle để cung cấp khả năng phục hồi.

Joins và Grouping

Hãy cùng xem cách dữ liệu đã sắp xếp đơn giản hóa các distributed joins và aggregations. Chúng ta sẽ tiếp tục sử dụng MapReduce để minh họa, mặc dù các khái niệm này áp dụng cho hầu hết các hệ thống batch processing.

Một ví dụ điển hình về một join trong một batch job được minh họa trong Hình 11-2. Bên trái là một log các sự kiện mô tả những việc mà người dùng đã đăng nhập thực hiện trên một website (được gọi là *activity events* hoặc *clickstream data*), và bên phải là một database người dùng. Bạn có thể coi ví dụ này là một phần của star schema (xem “Stars and Snowflakes: Schemas for Analytics”); log các sự kiện là fact table, và database người dùng là một trong các dimensions.

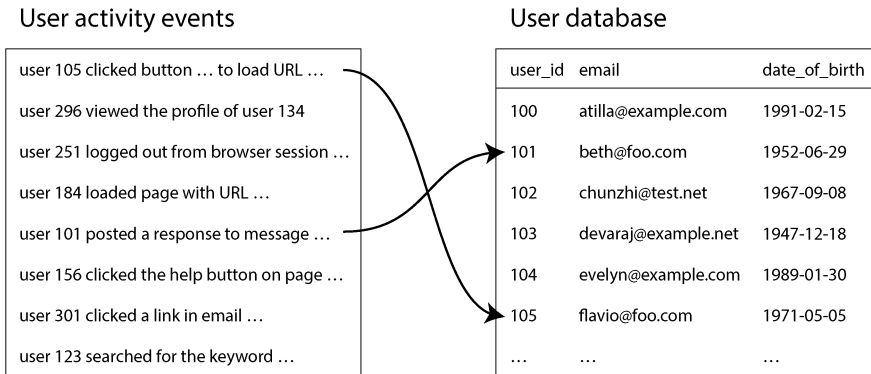


Figure 11-2. Một phép join giữa log các sự kiện hoạt động của người dùng và một cơ sở dữ liệu chứa các profile người dùng

Nếu bạn muốn thực hiện phân tích các sự kiện hoạt động có tính đến thông tin từ cơ sở dữ liệu người dùng (ví dụ: tìm hiểu xem liệu một số trang nhất định có phổ biến hơn với người dùng trẻ tuổi hay lớn tuổi hay không, bằng cách sử dụng trường ngày sinh trong hồ sơ người dùng), bạn cần tính toán một phép join giữa hai bảng này. Bạn sẽ tính toán phép join đó như thế nào, giả sử cả hai bảng đều lớn đến mức chúng phải được sharding?

Bạn có thể tận dụng thực tế rằng trong MapReduce, quá trình shuffle sẽ tập hợp tất cả các cặp key-value có cùng key về cùng một reducer, bất kể ban đầu chúng nằm ở shard nào. Ở đây, user ID có thể đóng vai trò là key. Do đó, bạn có thể viết một mapper duyệt qua các sự kiện hoạt động của người dùng và phát ra (emit) các URL xem trang được gắn key bởi user ID, như được minh họa trong Hình 11-3. Một mapper khác sẽ duyệt qua cơ sở dữ liệu người dùng theo từng dòng, trích xuất user ID làm key và ngày sinh của người dùng làm value.

Sau đó, quá trình shuffle đảm bảo rằng một hàm reducer có thể truy cập ngày sinh của một người dùng cụ thể và tất cả các sự kiện xem trang của người dùng đó cùng một lúc. Công việc MapReduce thậm chí có thể sắp xếp các bản ghi sao cho các reducer luôn thấy bản ghi từ cơ sở dữ liệu người dùng trước, sau đó mới đến các sự kiện hoạt động theo thứ tự timestamp. Kỹ thuật này được gọi là *secondary sort* [25].

Các reducer giờ đây có thể thực hiện logic join thực tế một cách dễ dàng. Giá trị đầu tiên được kỳ vọng là ngày sinh, thứ mà reducer sẽ lưu vào một biến cục bộ. Sau đó, nó lặp qua các sự kiện hoạt động có cùng user ID, xuất ra mỗi URL đã xem cùng với ngày sinh của người xem. Vì một reducer xử lý tất cả các bản ghi cho một user ID cụ thể trong một lần, nó chỉ cần giữ duy nhất một bản ghi người dùng trong bộ nhớ tại bất kỳ thời điểm nào, và nó không bao giờ cần thực hiện bất kỳ yêu cầu nào qua mạng. Thuật toán này được gọi là *sort-merge join*, vì đầu ra của mapper được sắp xếp theo key và sau đó các reducer sẽ hợp nhất các danh sách bản ghi đã được sắp xếp từ cả hai phía của phép join.

Công việc MapReduce tiếp theo trong workflow sau đó có thể tính toán sự phân bố độ tuổi của người xem cho mỗi URL. Để làm được điều này, công việc trước tiên sẽ shuffle dữ liệu bằng cách sử dụng URL làm key. Sau khi đã được sắp xếp, các reducer lặp qua tất cả các lượt xem trang (kèm theo ngày sinh của người xem) cho một URL duy nhất, duy trì một bộ đếm cho số lượt xem của mỗi nhóm tuổi và tăng bộ đếm tương ứng cho mỗi lượt xem trang. Bằng cách này, bạn có thể triển khai một thao tác group by và aggregation.

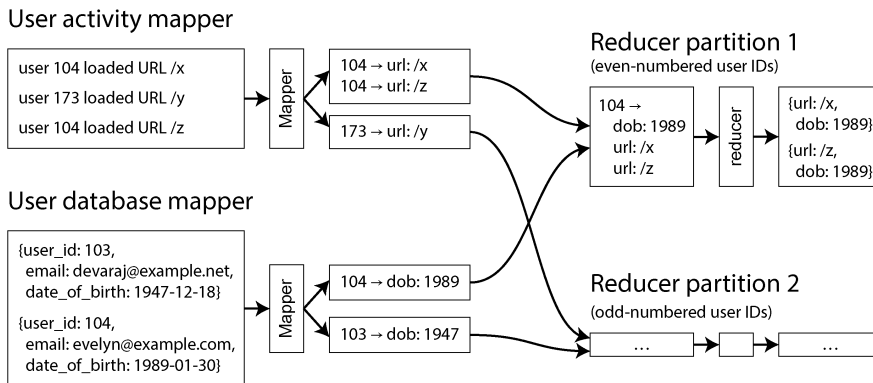


Figure 11-3. Một sort-merge join dựa trên user ID; nếu các dataset đầu vào được sharding thành nhiều tệp, mỗi tệp có thể được xử lý bằng nhiều mapper song song

Ngôn ngữ truy vấn

Qua nhiều năm, các execution engine cho batch processing phân tán đã trở nên trưởng thành. Hạ tầng hiện nay đã đủ mạnh mẽ để lưu trữ và xử lý hàng nghìn petabyte dữ liệu trên các cluster gồm hơn 10.000 máy. Khi vấn đề vận hành vật lý các tiến trình batch ở quy mô lớn như vậy coi như đã được giải quyết, sự chú ý đã chuyển sang việc cải thiện mô hình lập trình.

MapReduce, các dataflow engine và các cloud data warehouse đều đã đón nhận SQL như một ngôn ngữ chung (lingua franca) cho batch processing. Đây là một sự kết hợp tự nhiên, bởi vì các data warehouse truyền thống đều sử dụng SQL, các công cụ phân tích dữ liệu và ETL cũng đã hỗ trợ nó, và tất cả các lập trình viên cũng như nhà phân tích đều biết sử dụng nó.

Bên cạnh việc yêu cầu ít mã nguồn hơn so với các job MapReduce được viết tay, các giao diện ngôn ngữ truy vấn này còn cho phép sử dụng tương tác, trong đó bạn viết các truy vấn phân tích và chạy chúng từ một terminal hoặc GUI. Phong cách truy vấn tương tác này là một cách hiệu quả và tự nhiên để các nhà phân tích kinh doanh, quản lý sản phẩm, đội ngũ bán hàng và tài chính, cùng những người khác khám phá dữ liệu trong một môi trường batch processing. Việc hỗ trợ SQL cũng giúp các hệ thống batch processing phân tán trở nên phù hợp cho các truy vấn khám phá.

Các ngôn ngữ truy vấn cấp cao không chỉ giúp con người sử dụng hệ thống làm việc hiệu quả hơn; chúng còn cải thiện hiệu năng thực thi job ở cấp độ máy móc. Như chúng ta đã thấy trong mục “Cloud Data Warehouses”, các query engine chịu trách nhiệm chuyển đổi các truy vấn SQL thành các batch job để thực thi trong một cluster. Bước chuyển đổi từ truy vấn sang syntax tree rồi đến các toán tử vật lý này cho phép engine tối ưu hóa các truy vấn. Các query engine như Hive, Trino, Spark và Flink có các bộ tối ưu hóa truy vấn dựa trên chi phí (cost-based query optimizers) có thể phân tích các đặc tính của đầu vào join và tự động quyết định thuật toán nào sẽ phù hợp nhất cho tác vụ đang thực hiện. Các bộ tối ưu hóa thậm chí có thể thay đổi thứ tự join để giảm thiểu lượng trạng thái trung gian [19, 26, 27, 28].

Mặc dù SQL là ngôn ngữ truy vấn batch processing đa mục đích phổ biến nhất, các ngôn ngữ khác vẫn được sử dụng cho các nhu cầu chuyên biệt. Ví dụ, Apache Pig là một ngôn ngữ dựa trên các toán tử quan hệ cho phép các pipeline dữ liệu được chỉ định từng bước một, thay vì là một truy vấn SQL lớn duy nhất. DataFrames (được thảo luận trong mục tiếp theo) có các đặc tính tương tự, và Morel là một ngôn ngữ hiện đại hơn chịu ảnh hưởng từ Pig. Những người dùng khác đã áp dụng các ngôn ngữ truy vấn JSON như jq, JMESPath, hoặc JSONPath.

Trong mục “Graph-Like Data Models”, chúng ta đã thảo luận về việc sử dụng đồ thị để mô hình hóa dữ liệu và sử dụng các ngôn ngữ truy vấn đồ thị để duyệt qua các cạnh (edges) và đỉnh (vertices) trong một đồ thị. Nhiều framework xử lý đồ thị cũng hỗ trợ tính toán batch thông qua các ngôn ngữ truy vấn như Gremlin của Apache TinkerPop. Chúng ta sẽ xem xét chi tiết hơn các kịch bản xử lý đồ thị trong mục “Batch Use Cases”.

Sự hội tụ giữa Batch Processing và Cloud Data Warehouses

Về mặt lịch sử, các data warehouse chạy trên các thiết bị phần cứng chuyên dụng và hỗ trợ các truy vấn phân tích dựa trên SQL trên dữ liệu quan hệ. Các framework batch processing như MapReduce được thiết lập để cung cấp khả năng mở rộng và tính linh hoạt lớn hơn bằng cách hỗ trợ logic xử lý được viết bằng một ngôn ngữ lập trình đa mục đích, cho phép đọc và ghi các định dạng dữ liệu tùy ý.

Theo thời gian, hai khái niệm này đã trở nên giống nhau hơn nhiều. Các framework batch processing hiện đại hiện đã hỗ trợ SQL như một ngôn ngữ để viết các batch job, và chúng đạt được hiệu năng tốt trên các truy vấn quan hệ bằng cách sử dụng các định dạng lưu trữ dạng cột (columnar storage) như Parquet và các engine thực thi truy vấn được tối ưu hóa (xem mục “Query Execution: Compilation and Vectorization”). Trong khi đó, các data warehouse đã trở nên có khả năng mở rộng hơn bằng cách chuyển lên cloud (xem mục “Cloud Data Warehouses”) và triển khai nhiều kỹ thuật lập lịch, fault tolerance và shuffling tương tự như các framework batch phân tán. Nhiều hệ thống cũng sử dụng các hệ thống tệp phân tán.

Giống như các hệ thống batch processing đã áp dụng SQL làm mô hình xử lý, các cloud warehouse cũng đã áp dụng các mô hình xử lý thay thế khác. Ví dụ, BigQuery cung cấp một thư viện DataFrames, và thư viện Snowpark của Snowflake tích hợp với Pandas. Các bộ điều phối workflow (orchestrator) cho batch processing như Airflow, Prefect, và Dagster cũng tích hợp với các cloud warehouse.

Tuy nhiên, không phải tất cả các batch job đều dễ dàng biểu diễn bằng SQL, bao gồm các thuật toán đồ thị lặp (iterative graph algorithms) như PageRank, các tác vụ ML phức tạp, và nhiều workflow khác. Việc xử lý dữ liệu AI, bao gồm dữ liệu phi quan hệ và multimodal như hình ảnh, video và âm thanh, cũng có thể khó biểu diễn bằng SQL.

Các cloud data warehouse cũng gặp khó khăn với một số khối lượng công việc nhất định. Tính toán theo từng dòng (row-by-row computation) sẽ kém hiệu quả hơn khi sử dụng các định dạng lưu trữ column-oriented; trong những trường hợp như vậy, các API warehouse thay thế hoặc một hệ thống batch processing sẽ được ưu tiên hơn. Cloud data warehouses cũng có xu hướng đắt đỏ hơn so với các hệ thống batch processing khác. Thay vào đó, việc chạy các job lớn trong các hệ thống batch processing như Spark hoặc Flink có thể mang lại hiệu quả về chi phí hơn.

Cuối cùng, quyết định giữa việc xử lý dữ liệu trong các hệ thống batch hay trong các data warehouse thường phụ thuộc vào các yếu tố như chi phí, sự tiện

lợi, độ dễ triển khai và tính sẵn có. Hầu hết các doanh nghiệp lớn đều có nhiều hệ thống xử lý dữ liệu, điều này mang lại cho họ sự linh hoạt trong quyết định này. Các công ty nhỏ hơn thường chỉ cần dùng một hệ thống duy nhất.

DataFrames

Các nhà khoa học dữ liệu và nhà thống kê thường quen với việc làm việc với mô hình dữ liệu DataFrame được tìm thấy trong R và Pandas (xem “DataFrames, Matrices, and Arrays”). Một DataFrame tương tự như một bảng trong một cơ sở dữ liệu quan hệ: nó là một tập hợp các dòng, và tất cả các giá trị trong cùng một cột đều có cùng một kiểu dữ liệu. Thay vì viết một truy vấn SQL lớn, người dùng gọi các hàm tương ứng với các toán tử quan hệ để thực hiện các thao tác filter, join, sắp xếp, aggregation và các thao tác khác.

Ban đầu, việc thao tác DataFrame thường diễn ra cục bộ, trong bộ nhớ (in-memory). Do đó, các DataFrame bị giới hạn trong các dataset vừa vặn trên một máy đơn lẻ. Các nhà khoa học dữ liệu muốn tương tác với các dataset lớn trong các môi trường batch processing bằng cách sử dụng các API DataFrame mà họ đã quen thuộc, vì SQL và MapReduce không phù hợp với nhu cầu của họ. Các framework xử lý dữ liệu phân tán như Spark, Flink, và Daft đã áp dụng các API DataFrame để đáp ứng nhu cầu này. Tuy nhiên, cách triển khai của chúng hoạt động hơi khác một chút; các DataFrame cục bộ thường được đánh chỉ mục (indexed) và được sắp xếp, trong khi các DataFrame phân tán thì thường không [29]. Điều này có thể dẫn đến những bất ngờ về hiệu năng khi chuyển đổi sang các framework batch.

Các API DataFrame có vẻ tương tự như các API dataflow, nhưng cách triển khai thì khác nhau. Trong khi Pandas thực thi các thao tác ngay lập tức khi các phương thức DataFrame được gọi, Spark trước tiên sẽ dịch tất cả các lệnh gọi API DataFrame thành một query plan và chạy tối ưu hóa truy vấn trước khi thực thi workflow trên nền tảng engine dataflow phân tán của nó.

Các framework như Daft thậm chí còn hỗ trợ tính toán ở cả phía client và phía server. Các thao tác in-memory nhỏ hơn được thực thi trên client, và các dataset lớn hơn được xử lý trên server. Các định dạng lưu trữ columnar như Apache Arrow cung cấp một mô hình dữ liệu thống nhất mà cả engine thực thi phía client và phía server đều có thể chia sẻ.

Các trường hợp sử dụng Batch

Bây giờ chúng ta đã thấy cách batch processing hoạt động, hãy cùng xem cách nó được áp dụng vào một loạt các ứng dụng. Các batch job rất tuyệt vời để xử lý các dataset lớn một cách hàng loạt, nhưng chúng không tốt cho các trường hợp sử dụng yêu cầu độ trễ (latency) thấp. Do đó, bạn sẽ tìm thấy các batch job ở bất cứ nơi nào có

nhiều dữ liệu và độ tươi (freshness) của dữ liệu không quan trọng. Điều này nghe có vẻ hạn chế, nhưng thực tế là một lượng lớn các tác vụ xử lý dữ liệu phù hợp với mô hình này. Ví dụ:

- Việc đối soát kế toán và kiểm kê hàng tồn kho, nơi các công ty xác minh rằng các giao dịch khớp với tài khoản ngân hàng và hàng tồn kho của họ, thường được thực hiện dưới dạng các batch job [30].
- Trong sản xuất, việc dự báo nhu cầu thường chạy dưới dạng một batch job định kỳ [31].
- Các công ty thương mại điện tử, truyền thông và mạng xã hội huấn luyện các mô hình recommendation của họ bằng các batch job [32, 33].
- Nhiều hệ thống tài chính dựa trên batch; ví dụ, mạng lưới ngân hàng Hoa Kỳ hoạt động gần như hoàn toàn dựa trên các batch job [34].

Trong các mục sau đây, chúng ta sẽ thảo luận về một số trường hợp sử dụng batch processing mà bạn sẽ tìm thấy trong hầu hết mọi ngành công nghiệp.

Extract-Transform-Load

Cuốn “Data Warehousing” đã giới thiệu ETL và ELT, nơi một pipeline xử lý dữ liệu thực hiện extract dữ liệu từ một database production, transform nó, và load kết quả vào một hệ thống downstream (chúng ta sẽ sử dụng “ETL” trong mục này để đại diện cho cả khối lượng công việc ETL và ELT). Các batch job thường được sử dụng cho các khối lượng công việc như vậy, đặc biệt là khi hệ thống downstream là một data warehouse.

Bản chất song song của các batch job khiến chúng rất phù hợp cho việc transform dữ liệu, mà phần lớn trong đó liên quan đến các khối lượng công việc “embarrassingly parallel” (song song cực kỳ dễ dàng). Việc filtering dữ liệu, projecting các field, và nhiều thao tác transform data warehouse phổ biến khác đều có thể được thực hiện song song.

Các môi trường batch processing cũng đi kèm với các workflow scheduler mạnh mẽ, giúp việc lập lịch, orchestration, và debug các job của ETL data pipeline trở nên dễ dàng. Khi có lỗi xảy ra, các scheduler thường thử lại các job để giảm thiểu các vấn đề tạm thời có thể phát sinh. Một job thất bại lặp đi lặp lại sẽ được đánh dấu là thất bại, điều này giúp các lập trình viên dễ dàng thấy job nào trong data pipeline của họ đã ngừng hoạt động. Các scheduler như Airflow thậm chí còn đi kèm với các operator source, sink, và query được tích hợp sẵn cho MySQL, PostgreSQL, Snowflake, Spark, Flink, và hàng chục hệ thống phổ biến khác. Sự tích hợp chặt chẽ giữa các scheduler và các hệ thống xử lý dữ liệu giúp đơn giản hóa việc tích hợp dữ liệu.

Chúng ta cũng thấy rằng các batch job rất dễ để troubleshoot và sửa lỗi khi có vấn đề xảy ra. Tính năng này vô cùng giá trị khi debug các data pipeline. Các tệp bị lỗi có thể được kiểm tra dễ dàng để xem điều gì đã sai, và các ETL batch job có thể được sửa và chạy lại. Ví dụ, nếu một tệp đầu vào không chứa một field mà một transformation batch job dự định sử dụng, các kỹ sư dữ liệu có thể dễ dàng phát hiện ra field đó bị thiếu và cập nhật logic transformation hoặc cập nhật job đã tạo ra đầu vào đó.

Các data pipeline từng được quản lý bởi một đội ngũ data engineering duy nhất, vì việc yêu cầu các đội ngũ khác đang làm việc trên các feature sản phẩm phải viết và quản lý các data pipeline batch phức tạp được coi là không công bằng. Gần đây, những cải tiến trong các mô hình batch processing và quản lý metadata đã giúp các kỹ sư trong toàn bộ tổ chức đóng góp và quản lý các data pipeline của riêng họ dễ dàng hơn nhiều. Các thực hành như *data mesh* [35, 36], *data contract* [37], và *data fabric* [38] cung cấp các tiêu chuẩn và công cụ để giúp các đội ngũ công bố dữ liệu của họ một cách an toàn để bất kỳ ai trong tổ chức cũng có thể sử dụng.

Các data pipeline và các truy vấn phân tích đã bắt đầu chia sẻ không chỉ các mô hình xử lý, mà cả các execution engine nữa. Nhiều ETL batch job hiện nay chạy trên cùng các hệ thống với các truy vấn phân tích dùng để đọc đầu ra của chúng. Không hiếm khi thấy cả các thao tác transform của data pipeline và các truy vấn phân tích đều chạy dưới dạng các truy vấn SparkSQL, Trino, hoặc DuckDB. Một kiến trúc như vậy càng làm mờ đi ranh giới giữa application engineering, data engineering, analytics engineering, và business analysis.

Analytics

Trong mục “Operational Versus Analytical Systems”, chúng ta đã thấy rằng các truy vấn phân tích (OLAP) thường quét qua một lượng lớn các bản ghi, thực hiện các thao tác grouping và aggregation. Có thể chạy các khối lượng công việc như vậy trong một hệ thống batch processing, cùng với các khối lượng công việc batch processing khác. Các nhà phân tích viết các truy vấn SQL thực thi trên một query engine, thứ sẽ đọc và ghi vào một distributed filesystem hoặc object store. Metadata của bảng như các ánh xạ table-to-file, tên, và kiểu dữ liệu được quản lý bằng các table format như Apache Iceberg và các catalog như Unity (xem mục “Cloud Data Warehouses”). Kiến trúc này được gọi là một *data lakehouse* [39].

Tương tự như ETL, những cải tiến trong giao diện truy vấn SQL đồng nghĩa với việc nhiều tổ chức hiện đang sử dụng các framework xử lý batch như Spark để phân tích. Các mẫu truy vấn như vậy có hai kiểu:

Truy vấn pre-aggregation (tổng hợp trước)

Dữ liệu được roll up vào các OLAP cube hoặc data mart để tăng tốc truy vấn (xem “Materialized Views and Data Cubes”). Dữ liệu đã được pre-aggregated sẽ được truy vấn trong warehouse hoặc được đẩy tới các hệ thống OLAP thời gian

thực được xây dựng chuyên dụng như Apache Druid hoặc Apache Pinot. Việc pre-aggregation thường diễn ra theo một khoảng thời gian được lên lịch. Các workflow scheduler được thảo luận trong mục “Scheduling workflows” được sử dụng để quản lý các workload này.

Truy vấn ad hoc

Người dùng chạy các truy vấn này để trả lời các câu hỏi kinh doanh cụ thể, điều tra hành vi người dùng, debug các vấn đề vận hành và nhiều việc khác nữa. Response time là rất quan trọng đối với trường hợp sử dụng này. Các nhà phân tích chạy truy vấn một cách lặp đi lặp lại khi họ nhận được phản hồi và tìm hiểu thêm về dữ liệu mà họ đang điều tra. Các framework batch processing với khả năng thực thi truy vấn nhanh giúp giảm thời gian chờ đợi cho các nhà phân tích.

Sự hỗ trợ SQL cho phép các framework batch processing tích hợp với các bảng tính và công cụ trực quan hóa dữ liệu như Tableau, Power BI, Looker và Apache Superset. Ví dụ, Tableau cung cấp các bộ kết nối SparkSQL và Presto, trong khi Apache Superset hỗ trợ Trino, Hive, Spark SQL, Presto và nhiều hệ thống khác mà cuối cùng sẽ thực thi các batch job để truy vấn dữ liệu.

Machine Learning

Machine learning (ML) sử dụng thường xuyên việc batch processing. Các nhà khoa học dữ liệu, ML engineer và AI engineer sử dụng các framework batch processing để điều tra các mẫu dữ liệu, biến đổi dữ liệu và huấn luyện các mô hình ML. Các mục đích sử dụng phổ biến bao gồm những điều sau:

Feature engineering

Dữ liệu thô được lọc và biến đổi thành dữ liệu mà các mô hình có thể dùng để huấn luyện. Các mô hình dự đoán thường cần dữ liệu số, vì vậy các engineer phải biến đổi các dạng dữ liệu khác (chẳng hạn như văn bản hoặc các giá trị rời rạc) thành định dạng yêu cầu.

Huấn luyện mô hình

Dữ liệu huấn luyện là đầu vào của quá trình batch, và các weight của mô hình đã được huấn luyện là đầu ra.

Batch inference

Một mô hình đã được huấn luyện có thể được sử dụng để thực hiện các dự đoán hàng loạt nếu các dataset lớn và không yêu cầu kết quả thời gian thực. Điều này bao gồm cả việc đánh giá các dự đoán của mô hình trên một dataset kiểm định.

Các framework batch processing cung cấp các công cụ dành riêng cho các trường hợp sử dụng này. Ví dụ, MLLib của Apache Spark và FlinkML của Apache Flink đi kèm với rất nhiều công cụ feature engineering, các hàm thống kê và các bộ phân loại (classifiers).

Các ứng dụng ML như recommendation engines và ranking systems cũng sử dụng mạnh mẽ việc xử lý đồ thị (xem “Graph-Like Data Models”). Nhiều thuật toán đồ thị được thể hiện bằng cách duyệt qua từng cạnh một, kết hợp một đỉnh với một đỉnh liền kề để lan truyền một số thông tin nào đó, và lặp lại cho đến khi một điều kiện nhất định được đáp ứng—ví dụ, cho đến khi không còn cạnh nào để đi tiếp, hoặc cho đến khi một chỉ số hội tụ.

Mô hình tính toán *bulk synchronous parallel* (BSP) [40] đã trở nên phổ biến cho việc xử lý batch các đồ thị; nó được triển khai bởi Apache Giraph [20], API GraphX của Spark, và API Gelly của Flink [41], cùng với những cái tên khác. Nó còn được gọi là mô hình *Pregel*, vì bài báo Pregel của Google đã làm phổ biến cách tiếp cận này để xử lý đồ thị [42].

Batch processing cũng là một phần không thể thiếu trong việc chuẩn bị dữ liệu và huấn luyện LLM. Dữ liệu đầu vào là văn bản thô như nội dung của các trang web thường nằm trong một DFS hoặc object store. Dữ liệu này phải được tiền xử lý để phù hợp cho việc huấn luyện. Các bước tiền xử lý phù hợp với các framework batch processing bao gồm những điều sau:

- Trích xuất văn bản thuần túy từ HTML và sửa các văn bản bị lỗi định dạng.
- Phát hiện và loại bỏ các tài liệu chất lượng thấp, không liên quan và bị trùng lặp.
- Tokenizing văn bản (chia nhỏ thành các từ) và chuyển đổi nó thành các embedding, hoặc các biểu diễn số của mỗi từ.

Các framework batch processing như Kubeflow, Flyte và Ray được xây dựng chuyên dụng cho các khối lượng công việc như vậy. Ví dụ, OpenAI sử dụng Ray như một phần trong quá trình huấn luyện ChatGPT của họ [43]. Các framework này có tích hợp sẵn cho các thư viện LLM và AI như PyTorch, TensorFlow, XGBoost và nhiều thư viện khác. Chúng cũng cung cấp hỗ trợ tích hợp cho feature engineering, huấn luyện mô hình, batch inference và fine-tuning (điều chỉnh một foundation model cho các trường hợp sử dụng cụ thể).

Cuối cùng, các nhà khoa học dữ liệu thường thử nghiệm với dữ liệu trong các notebook tương tác như Jupyter hoặc Hex. Notebook được cấu thành từ các *cell*, vốn là những đoạn nhỏ Markdown, Python hoặc SQL. Các cell được thực thi tuần tự để tạo ra các bảng tính, đồ thị hoặc dữ liệu. Nhiều notebook sử dụng batch processing thông qua các DataFrame API hoặc truy vấn các hệ thống như vậy bằng SQL.

Serving Dữ liệu phái sinh

Các batch job thường được sử dụng để xây dựng các dataset đã được tính toán trước hoặc dữ liệu phái sinh như gợi ý sản phẩm, báo cáo dành cho người dùng và các feature cho các mô hình ML. Các dataset này thường được phục vụ từ một database

production, key-value store hoặc search engine. Bất kể hệ thống nào được sử dụng, dữ liệu đã tính toán trước cần phải đi từ distributed filesystem hoặc object store của bộ xử lý batch quay trở lại database đang phục vụ lưu lượng truy cập trực tiếp (live traffic).

Bạn có thể sẽ bị cám dỗ bởi việc sử dụng trực tiếp client library cho database yêu thích của mình ngay trong một batch job và ghi trực tiếp vào database server, từng bản ghi một. Cách này sẽ hoạt động (giả sử các quy tắc firewall của bạn cho phép truy cập trực tiếp từ môi trường batch processing đến các database production), nhưng đó là một ý tưởng tồi vì vài lý do sau:

- Việc thực hiện một network request cho mỗi bản ghi đơn lẻ sẽ chậm hơn nhiều lần so với throughput bình thường của một tác vụ batch. Ngay cả khi client library hỗ trợ batching, hiệu năng có khả năng sẽ rất kém.
- Các framework batch processing thường chạy nhiều tác vụ song song. Nếu tất cả các tác vụ cùng lúc ghi vào cùng một output database với tốc độ kỳ vọng của một quy trình batch, database đó có thể dễ dàng bị quá tải, và hiệu năng truy vấn của nó có khả năng sẽ bị ảnh hưởng. Điều này có thể dẫn đến các vấn đề vận hành ở các phần khác của hệ thống [44].
- Thông thường, các batch job cung cấp một sự đảm bảo all-or-nothing (tất cả hoặc không có gì) sạch sẽ cho đầu ra của công việc. Nếu một job thành công, kết quả là đầu ra của việc thực thi mọi tác vụ đúng một lần, ngay cả khi một số tác vụ đã thất bại và phải được thử lại trong quá trình thực hiện; nếu toàn bộ job thất bại, sẽ không có đầu ra nào được tạo ra. Tuy nhiên, việc ghi vào một hệ thống bên ngoài từ bên trong một job sẽ tạo ra các side effects có thể quan sát được từ bên ngoài mà không thể che giấu theo cách này. Do đó, bạn phải lo lắng về việc kết quả từ các job hoàn thành một phần có thể hiển thị với các hệ thống khác. Nếu một tác vụ thất bại và được khởi động lại, nó có thể tạo ra đầu ra trùng lặp từ lần thực thi thất bại trước đó.

Một giải pháp tốt hơn là để các batch job đẩy các dataset đã tính toán trước vào các stream như các Kafka topic, điều mà chúng ta sẽ thảo luận kỹ hơn trong Chương 12. Các search engine như Elasticsearch, các hệ thống OLAP thời gian thực như Apache Pinot và Apache Druid, các datastores phái sinh như Venice [45], và các cloud data warehouse như ClickHouse đều có khả năng tích hợp sẵn để nạp dữ liệu từ Kafka vào hệ thống của chúng. Việc đẩy dữ liệu qua một streaming system sẽ giải quyết được một vài vấn đề đã nêu ở trên:

- Các streaming system được tối ưu hóa cho các thao tác ghi tuần tự, điều này giúp chúng phù hợp hơn với khối lượng công việc ghi hàng loạt (bulk write) của một batch job.

- Các streaming system có thể đóng vai trò như một buffer giữa batch job và các database production. Các hệ thống downstream có thể throttle tốc độ đọc của chúng để đảm bảo chúng có thể tiếp tục phục vụ lưu lượng production một cách thoải mái.
- Đầu ra của một batch job duy nhất có thể được tiêu thụ bởi nhiều hệ thống downstream.
- Các streaming system có thể đóng vai trò như một ranh giới bảo mật giữa các môi trường batch processing và các mạng production. Chúng có thể được triển khai trong một mạng được gọi là *demilitarized zone* (DMZ) nằm giữa mạng batch processing và mạng production.

Một vấn đề mà streaming không tự giải quyết được là đảm bảo tính chất "tất cả hoặc không có gì" (all-or-nothing). Để cơ chế này hoạt động, khi hoàn tất, các batch job phải gửi một thông báo tới các hệ thống downstream rằng công việc của chúng đã xong và dữ liệu hiện đã có thể được phục vụ. Các consumer của stream cần có khả năng giữ cho dữ liệu mà chúng nhận được không hiển thị đối với các truy vấn, giống như một uncommitted transaction với read-committed isolation (xem mục "Read Committed"), cho đến khi chúng nhận được thông báo rằng công việc đã hoàn tất.

Một pattern khác phổ biến hơn khi bootstrapping các cơ sở dữ liệu là xây dựng một cơ sở dữ liệu hoàn toàn mới *bên trong* batch job và bulk-load các tệp đó trực tiếp vào cơ sở dữ liệu từ một distributed filesystem, object store, hoặc local filesystem. Nhiều hệ thống dữ liệu cung cấp các công cụ bulk import, chẳng hạn như Lightning của TiDB và các Hadoop import jobs của Apache Pinot. RocksDB cũng cung cấp một API để bulk-import các tệp Sorted String Table (SST) từ các batch job.

Việc xây dựng các cơ sở dữ liệu theo batch và bulk-import dữ liệu diễn ra rất nhanh, đồng thời giúp các hệ thống dễ dàng chuyển đổi nguyên tử (atomically) giữa các phiên bản dataset. Mặt khác, việc cập nhật dần (incrementally) các dataset từ các batch job xây dựng cơ sở dữ liệu hoàn toàn mới có thể là một thách thức. Việc áp dụng một cách tiếp cận hybrid khi cần cả bootstrapping lẫn incremental loads là điều phổ biến. Ví dụ, Venice hỗ trợ các hybrid stores cho phép thực hiện các batch row-based updates và thay thế toàn bộ dataset (full dataset swaps).

Tóm tắt

Trong chương này, chúng ta đã khám phá việc thiết kế và triển khai các hệ thống batch processing. Chúng ta bắt đầu với bộ công cụ Unix kinh điển (`awk`, `sort`, `uniq`, v.v.) để minh họa các primitives cơ bản của batch processing như sorting và counting.

Sau đó, chúng ta đã scale up lên các hệ thống distributed batch processing. Các batch framework xử lý các input dataset bất biến (immutable) và có giới hạn (bounded) để

tạo ra dữ liệu đầu ra, cho phép chạy lại (reruns) và debugging mà không gây ra side effects. Quá trình xử lý này bao gồm ba thành phần chính: một orchestration layer để quyết định nơi và khi nào các job chạy, một storage layer để lưu trữ dữ liệu, và một computation layer để xử lý dữ liệu thực tế.

Chúng ta đã xem xét cách các distributed filesystems và object stores quản lý các tệp lớn thông qua block-based replication, caching, và các dịch vụ metadata, cũng như cách các batch framework hiện đại tương tác với các hệ thống này thông qua các pluggable APIs. Chúng ta cũng thảo luận về cách các job orchestrators lập lịch cho các task, phân bổ tài nguyên, và xử lý lỗi trong các cluster lớn, đồng thời so sánh chúng với các workflow orchestrators quản lý vòng đời của một tập hợp các job chạy trong một dependency graph.

Chúng ta đã khảo sát các mô hình batch processing, bắt đầu với MapReduce cùng các hàm map và reduce kinh điển của nó. Tiếp theo, chúng ta chuyển sang các dataflow engines như Spark và Flink, những công cụ cung cấp các dataflow APIs để sử dụng hơn và hiệu năng tốt hơn. Để hiểu cách các batch job scale, chúng ta đã tìm hiểu về thuật toán shuffle, một thao tác nền tảng cho phép thực hiện grouping, joining, và aggregation.

Chúng ta thấy rằng khi các hệ thống batch trở nên hoàn thiện, trọng tâm đã chuyển sang khả năng sử dụng (usability). Các hỗ trợ đã được thêm vào cho các ngôn ngữ truy vấn cấp cao như SQL và DataFrame APIs, giúp các batch job trở nên dễ tiếp cận hơn và dễ tối ưu hóa hơn. Batch framework sẽ tiếp nhận các job được viết bằng các ngôn ngữ này và tự động quyết định cách thực thi chúng một cách hiệu quả trên một cluster các máy chủ.

Chúng ta kết thúc chương bằng việc khảo sát các trường hợp sử dụng batch processing phổ biến, bao gồm các mục sau:

- Các ETL pipeline, thực hiện trích xuất (extract), biến đổi (transform), và nạp (load) dữ liệu giữa các hệ thống bằng cách sử dụng các workflow được lập lịch
- Analytics, nơi các batch job hỗ trợ cả các truy vấn pre-aggregated và ad hoc
- Machine learning, nơi các batch job được sử dụng để chuẩn bị và xử lý các training dataset lớn
- Đổ dữ liệu vào các hệ thống hướng tới production từ các đầu ra batch, thường thông qua các stream hoặc các công cụ bulk-loading, nhằm phục vụ dữ liệu phái sinh (derived data) cho người dùng

Trong chương tiếp theo, chúng ta sẽ chuyển sang stream processing (xử lý luồng), trong đó đầu vào là *unbounded* (không giới hạn)—nghĩa là đầu vào của một công việc là các luồng dữ liệu kéo dài vô tận. Điều này có nghĩa là các công việc không bao giờ kết thúc vì các tác vụ mới có thể đến bất cứ lúc nào. Chúng ta sẽ thấy rằng stream processing và batch processing có một số điểm tương đồng,

nhưng giả định về các luồng không giới hạn cũng có tác động đáng kể đến cách chúng ta xây dựng các hệ thống.

Footnotes

References

- [1] Nathan Marz. “How to Beat the CAP Theorem.” *nathanmarz.com*, October 2011. Archived at perma.cc/4BS9-R9A4
- [2] Molly Bartlett Dishman and Martin Fowler. “Agile Architecture.” At *O’Reilly Software Architecture Conference*, March 2015.
- [3] Jeffrey Dean and Sanjay Ghemawat. “MapReduce: Simplified Data Processing on Large Clusters.” At *6th USENIX Symposium on Operating System Design and Implementation (OSDI)*, December 2004.
- [4] Shivnath Babu and Herodotos Herodotou. “Massively Parallel Databases and MapReduce Systems.” *Foundations and Trends in Databases*, volume 5, issue 1, pages 1–104, November 2013. [doi:10.1561/19000000036](https://doi.org/10.1561/19000000036)
- [5] David J. DeWitt and Michael Stonebraker. “MapReduce: A Major Step Backwards.” Originally published at *databasecolumn.vertica.com*, January 2008. Archived at perma.cc/U8PA-K48V
- [6] Henry Robinson. “The Elephant Was a Trojan Horse: On the Death of Map-Reduce at Google.” *the-paper-trail.org*, June 2014. Archived at perma.cc/9FEM-X787
- [7] Urs Hölzle. “R.I.P. MapReduce. After having served us well since 2003, today we removed the remaining internal codebase for good.” *x.com*, September 2019. Archived at perma.cc/B34T-LLY7
- [8] Adam Drake. “Command-Line Tools Can Be 235x Faster than Your Hadoop Cluster.” *aadrake.com*, January 2014. Archived at perma.cc/87SP-ZMCY
- [9] “**sort: Sort Text Files.**” GNU Coreutils 9.7 Documentation, Free Software Foundation, Inc., 2025. Archived at perma.cc/68KN-E8TL
- [10] Michael Ovsianikov, Silvius Rus, Damian Reeves, Paul Sutter, Sriram Rao, and Jim Kelly. “The Quantcast File System.” *Proceedings of the VLDB Endowment*, volume 6, issue 11, pages 1092–1101, August 2013. [doi:10.14778/2536222.2536234](https://doi.org/10.14778/2536222.2536234)
- [11] Andrew Wang, Zhe Zhang, Kai Zheng, Uma Maheswara G., and Vinayakumar B. “Introduction to HDFS Erasure Coding in Apache Hadoop.” *blog.cloudera.com*, September 2015. Archived at archive.org
- [12] Andy Warfield. “Building and Operating a Pretty Big Storage System Called S3.” *allthingsdistributed.com*, July 2023. Archived at perma.cc/7LPK-TP7V
- [13] Vinod Kumar Vavilapalli, Arun C. Murthy, Chris Douglas, Sharad Agarwal, Mahadev Konar, Robert Evans, Thomas Graves, Jason Lowe, Hitesh Shah, Siddharth Seth, Bikas Saha, Carlo Curino, Owen O’Malley, Sanjay Radia, Benjamin Reed, and Eric Baldeschwieler. “Apache Hadoop YARN: Yet Another Resource Negotiator.” At *4th Annual Symposium on Cloud Computing (SoCC)*, October 2013. [doi:10.1145/2523616.2523633](https://doi.org/10.1145/2523616.2523633)
- [14] Richard M. Karp. “Reducibility Among Combinatorial Problems.” *Complexity of Computer Computations. The IBM Research Symposia Series*. Springer, 1972. [doi:10.1007/978-1-4684-2001-2_9](https://doi.org/10.1007/978-1-4684-2001-2_9)
- [15] J. D. Ullman. “NP-Complete Scheduling Problems.” *Journal of Computer and System Sciences*, volume 10, issue 3, pages 384–393, June 1975. [doi:10.1016/S0022-0000\(75\)80008-0](https://doi.org/10.1016/S0022-0000(75)80008-0)
- [16] Gilad David Maayan. “The Complete Guide to Spot Instances on AWS, Azure and GCP.” *datacenterdynamics.com*, March 2021. Archived at archive.org

- [17] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. “Large-Scale Cluster Management at Google with Borg.” At *10th European Conference on Computer Systems* (EuroSys), April 2015. doi:10.1145/2741948.2741964
- [18] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. “Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing.” At *9th USENIX Symposium on Networked Systems Design and Implementation* (NSDI), April 2012.
- [19] Paris Carbone, Stephan Ewen, Seif Haridi, Asterios Katsifodimos, Volker Markl, and Kostas Tzoumas. “Apache Flink: Stream and Batch Processing in a Single Engine.” *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, volume 38, issue 4, pages 28–38, December 2015. Archived at perma.cc/G3N3-BKX5
- [20] Mark Grover, Ted Malaska, Jonathan Seidman, and Gwen Shapira. *Hadoop Application Architectures*. O’Reilly Media, 2015. ISBN: 9781491900048
- [21] Jules S. Damji, Brooke Wenig, Tathagata Das, and Denny Lee. *Learning Spark*, 2nd edition. O’Reilly Media, 2020. ISBN: 9781492050049
- [22] Michael Isard, Mihai Budiu, Yuan Yu, Andrew Birrell, and Dennis Fetterly. “Dryad: Distributed Data-Parallel Programs from Sequential Building Blocks.” At *2nd European Conference on Computer Systems* (EuroSys), March 2007. doi:10.1145/1272996.1273005
- [23] Daniel Warneke and Odej Kao. “Nephele: Efficient Parallel Data Processing in the Cloud.” At *2nd Workshop on Many-Task Computing on Grids and Supercomputers* (MTAGS), November 2009. doi:10.1145/1646468.1646476
- [24] Hossein Ahmadi. “In-Memory Query Execution in Google BigQuery.” *cloud.google.com*, August 2016. Archived at perma.cc/DGG2-FL9W
- [25] Tom White. *Hadoop: The Definitive Guide*, 4th edition. O’Reilly Media, 2015. ISBN: 9781491901632
- [26] Fabian Hüske. “Peeking into Apache Flink’s Engine Room.” *flink.apache.org*, March 2015. Archived at perma.cc/44BW-ALJX
- [27] Mostafa Mokhtar. “Hive 0.14 Cost Based Optimizer (CBO) Technical Overview.” *hortonworks.com*, March 2015. Archived at archive.org
- [28] Michael Armbrust, Reynold S. Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K. Bradley, Xiangrui Meng, Tomer Kaftan, Michael J. Franklin, Ali Ghodsi, and Matei Zaharia. “Spark SQL: Relational Data Processing in Spark.” At *ACM International Conference on Management of Data* (SIGMOD), June 2015. doi:10.1145/2723372.2742797
- [29] Kaya Kupferschmidt. “Spark vs. Pandas, Part 2—Spark.” *towardsdatascience.com*, October 2020. Archived at perma.cc/5BRK-G4N5
- [30] Ammar Chalifah. “Tracking Payments at Scale.” *bolt.eu.com*, June 2025. Archived at perma.cc/Q4KX-8K3J
- [31] Nafi Ahmet Turgut, Hamza Akyıldız, Hasan Burak Yel, Mehmet İkbal Özmen, Mutlu Polatcan, Pinar Baki, and Esra Kayabali. “Demand Forecasting at Getir Built with Amazon Forecast.” *aws.amazon.com*, May 2023. Archived at perma.cc/H3H6-GNL7
- [32] Jason (Siyu) Zhu. “Enhancing Homepage Feed Relevance by Harnessing the Power of Large Corpus Sparse ID Embeddings.” *linkedin.com*, August 2023. Archived at archive.org
- [33] Avery Ching, Sital Kedia, and Shuojie Wang. “Apache Spark @Scale: A 60 TB+ Production Use Case.” *engineering.fb.com*, August 2016. Archived at perma.cc/F7R5-YFAV

- [34] Edward Kim. “How ACH Works: A Developer Perspective—Part 1.” *engineering.gusto.com*, April 2014. Archived at perma.cc/F67P-VBLK
- [35] Zhamak Dehghani. “How to Move Beyond a Monolithic Data Lake to a Distributed Data Mesh.” *martinfowler.com*, May 2019. Archived at perma.cc/LN2L-L4VC
- [36] Chris Riccomini. “What the Heck Is a Data Mesh?!” *cnr.sb*, June 2021. Archived at perma.cc/NEJ2-BAX3
- [37] Chad Sanderson, Mark Freeman, and B. E. Schmidt. *Data Contracts*. O’Reilly Media, 2025. ISBN: 9781098157623
- [38] Daniel Abadi. “Data Fabric vs. Data Mesh: What’s the Difference?” *starburst.io*, November 2021. Archived at perma.cc/RSK3-HXDK
- [39] Michael Armbrust, Ali Ghodsi, Reynold Xin, and Matei Zaharia. “Lakehouse: A New Generation of Open Platforms That Unify Data Warehousing and Advanced Analytics.” At *11th Annual Conference on Innovative Data Systems Research (CIDR)*, January 2021. Archived at perma.cc/7C6D-T9NR
- [40] Leslie G. Valiant. “A Bridging Model for Parallel Computation.” *Communications of the ACM*, volume 33, issue 8, pages 103–111, August 1990. [doi:10.1145/79173.79181](https://doi.org/10.1145/79173.79181)
- [41] Stephan Ewen, Kostas Tzoumas, Moritz Kaufmann, and Volker Markl. “Spinning Fast Iterative Data Flows.” *Proceedings of the VLDB Endowment*, volume 5, issue 11, pages 1268–1279, July 2012. [doi:10.14778/2350229.2350245](https://doi.org/10.14778/2350229.2350245)
- [42] Grzegorz Malewicz, Matthew H. Austern, Aart J. C. Bik, James C. Dehnert, Ilan Horn, Naty Leiser, and Grzegorz Czajkowski. “Pregel: A System for Large-Scale Graph Processing.” At *ACM International Conference on Management of Data (SIGMOD)*, June 2010. [doi:10.1145/1807167.1807184](https://doi.org/10.1145/1807167.1807184)
- [43] Richard MacManus. “OpenAI Chats About Scaling LLMs at Anyscale’s Ray Summit.” *thenewstack.io*, September 2023. Archived at perma.cc/YJD6-KUXU
- [44] Jay Kreps. “Why Local State Is a Fundamental Primitive in Stream Processing.” *oreilly.com*, July 2014. Archived at perma.cc/P8HU-R5LA
- [45] Félix GV. “Open Sourcing Venice—LinkedIn’s Derived Data Platform.” *linkedin.com*, September 2022. Archived at archive.org

Stream Processing

Một hệ thống phức tạp hoạt động luôn được tìm thấy là đã tiến hóa từ một hệ thống đơn giản hoạt động. Mệnh đề ngược lại cũng có vẻ đúng: Một hệ thống phức tạp được thiết kế từ con số không sẽ không bao giờ hoạt động và không thể làm cho nó hoạt động được.

John Gall, *Systemantics* (1975)

Trong Chương 11, chúng ta đã thảo luận về batch processing—các kỹ thuật đọc một tập hợp các tệp làm đầu vào và tạo ra một tập hợp các tệp đầu ra mới. Đầu ra là một dạng *derived data* (dữ liệu phái sinh); tức là, một dataset có thể được tạo lại bằng cách chạy lại quá trình batch nếu cần thiết. Chúng ta đã thấy ý tưởng đơn giản nhưng mạnh mẽ này có thể được sử dụng để tạo ra các search index, hệ thống gợi ý, analytics, và nhiều thứ khác.

Tuy nhiên, một giả định lớn vẫn tồn tại xuyên suốt Chương 11: cụ thể là, đầu vào là hữu hạn—có kích thước đã biết và xác định—vì vậy quá trình batch biết khi nào nó đã đọc xong đầu vào của mình. Ví dụ, thao tác sắp xếp vốn là trung tâm của MapReduce phải đọc toàn bộ đầu vào trước khi nó có thể bắt đầu tạo ra đầu ra, bởi vì bản ghi đầu vào cuối cùng có thể chính là bản ghi có key thấp nhất và do đó cần phải là bản ghi đầu ra đầu tiên, vì vậy việc bắt đầu tạo đầu ra sớm là điều không thể.

Trong thực tế, rất nhiều dữ liệu là không giới hạn (unbounded) vì chúng đến dần dần theo thời gian. Người dùng của bạn tạo ra dữ liệu vào ngày hôm qua và hôm nay, và họ sẽ tiếp tục tạo ra thêm dữ liệu vào ngày mai. Trừ khi bạn phá sản, quá trình này không bao giờ kết thúc, vì vậy dataset không bao giờ "hoàn chỉnh" theo bất kỳ cách có ý nghĩa nào [1]. Do đó, các bộ xử lý batch phải chia dữ liệu một cách nhân tạo thành các chunk có thời lượng cố định—ví dụ, xử lý lượng dữ liệu của một ngày vào cuối mỗi ngày, hoặc xử lý lượng dữ liệu của một giờ vào cuối mỗi giờ.

Vấn đề với các quá trình batch hàng ngày là những thay đổi trong đầu vào sẽ được phản ánh trong đầu ra muộn hơn một ngày, điều này là quá chậm đối với nhiều người dùng thiếu kiên nhẫn. Để giảm độ trễ, chúng ta có thể chạy quá trình xử lý thường xuyên hơn—chẳng hạn, xử lý lượng dữ liệu của một giây vào cuối mỗi giây—hoặc thậm chí là liên tục, từ bỏ hoàn toàn các lát cắt thời gian cố định và chỉ đơn giản là xử lý mọi event khi nó xảy ra. Đó chính là ý tưởng đằng sau *stream processing*.

Nói chung, một *stream* đề cập đến dữ liệu được cung cấp dần dần theo thời gian. Khái niệm này xuất hiện ở nhiều nơi: trong `stdin` và `stdout` của Unix, các ngôn ngữ lập trình (lazy lists) [2], các filesystem API (như `FileInputStream` của Java), các kết nối TCP, âm thanh và video được truyền qua internet, v.v.

Trong chương này, chúng ta sẽ xem xét các *event streams* như một cơ chế quản lý dữ liệu: đối trọng không giới hạn và được xử lý dần dần so với dữ liệu batch mà chúng ta đã thấy ở chương trước. Trước tiên, chúng ta sẽ thảo luận về cách các stream được biểu diễn, lưu trữ và truyền tải qua mạng, sau đó nghiên cứu mối quan hệ giữa các stream và các database. Cuối cùng, trong phần “Processing Streams”, chúng ta sẽ khám phá các phương pháp và công cụ để xử lý các stream đó một cách liên tục và các cách mà chúng có thể được sử dụng để xây dựng các ứng dụng.

Truyền tải Event Streams

Trong thế giới batch processing, đầu vào và đầu ra của một công việc là các tệp (có lẽ nằm trên một distributed filesystem). Vậy thứ tương đương với streaming sẽ trông như thế nào?

Khi đầu vào là một tệp (một chuỗi các byte), bước xử lý đầu tiên thường là phân tích nó thành một chuỗi các bản ghi. Trong ngữ cảnh stream processing, một bản ghi thường được gọi là một *event*, nhưng về bản chất nó cũng giống như vậy: một đối tượng nhỏ, độc lập và immutable chứa các chi tiết về một điều gì đó đã xảy ra tại một thời điểm. Một event thường chứa một timestamp cho biết nó đã xảy ra khi nào theo đồng hồ thời gian trong ngày (xem “Monotonic Versus Time-of-Day Clocks”).

Ví dụ, điều vừa xảy ra có thể là một hành động mà người dùng đã thực hiện, chẳng hạn như xem một trang hoặc thực hiện một giao dịch mua hàng. Nó cũng có thể bắt nguồn từ một máy móc, chẳng hạn như một phép đo định kỳ từ cảm biến nhiệt độ hoặc một metric về CPU utilization. Trong ví dụ ở phần “Batch Processing with Unix Tools”, mỗi dòng trong web server log là một event.

Một event có thể được mã hóa dưới dạng một chuỗi văn bản, hoặc JSON, hoặc có lẽ dưới dạng nhị phân, như đã thảo luận trong Chương 5. Việc mã hóa này cho phép bạn lưu trữ một event—ví dụ, bằng cách nối nó vào một tệp, chèn nó vào một bảng quan hệ, hoặc ghi nó vào một document database. Việc mã hóa cũng cho phép bạn gửi event qua mạng tới một node khác để xử lý nó.

Trong batch processing, một tệp được ghi một lần và sau đó có khả năng được đọc bởi nhiều job. Tương tự, trong thuật ngữ streaming, một event được tạo ra một lần bởi một *producer* (còn được gọi là *publisher* hoặc *sender*) và sau đó có khả năng được xử lý bởi nhiều *consumer* (*subscribers* hoặc *recipients*) [3]. Trong một filesystem, một tên tệp xác định một tập hợp các bản ghi liên quan; trong một streaming system, các event liên quan thường được nhóm lại với nhau thành một *topic* hoặc *stream*.

Về nguyên tắc, một tệp hoặc database là đủ để kết nối các producer và consumer. Một producer ghi mọi event mà nó tạo ra vào datastore, và mỗi consumer định kỳ poll datastore để kiểm tra các event đã xuất hiện kể từ lần chạy cuối cùng của nó. Đây về

cơ bản là những gì một batch process thực hiện khi nó xử lý lượng dữ liệu của một ngày vào cuối mỗi ngày.

Tuy nhiên, khi chuyển sang hướng xử lý liên tục với độ trễ thấp, việc polling trở nên tốn kém nếu datastore không được thiết kế cho kiểu sử dụng này. Bạn càng poll thường xuyên, tỷ lệ các yêu cầu trả về event mới càng thấp, và do đó chi phí overhead càng cao. Thay vào đó, sẽ tốt hơn nếu các consumer được thông báo khi có các event mới xuất hiện.

Các database theo truyền thống không hỗ trợ cơ chế thông báo kiểu này một cách tốt. Các relational database thường có các *trigger*, thứ có thể phản ứng với một sự thay đổi (ví dụ, một hàng được chèn vào một bảng), nhưng chúng rất hạn chế về những gì có thể làm và đã phần nào là một ý tưởng bổ sung sau (afterthought) trong thiết kế database [4]. Thay vào đó, các công cụ chuyên dụng đã được phát triển cho mục đích truyền tải các thông báo event.

Messaging Systems

Một cách tiếp cận phổ biến để thông báo cho các consumer về các event mới là sử dụng một *messaging system*: một producer gửi một message chứa event, sau đó message này được đẩy tới các consumer. Chúng ta đã chạm tới các hệ thống này trước đó trong phần “Event-Driven Architectures” và bây giờ chúng ta sẽ đi sâu vào chi tiết hơn.

Một kênh giao tiếp trực tiếp như Unix pipe hoặc kết nối TCP giữa producer và consumer sẽ là một cách đơn giản để triển khai một messaging system. Tuy nhiên, hầu hết các messaging system đều mở rộng dựa trên mô hình cơ bản này. Cụ thể, Unix pipes và TCP kết nối chính xác một sender với một recipient, trong khi một messaging system cho phép nhiều producer node gửi message tới cùng một topic và cho phép nhiều consumer node nhận message trong một topic.

Trong mô hình *publish/subscribe* này, các hệ thống khác nhau thực hiện một loạt các cách tiếp cận đa dạng, và không có một câu trả lời duy nhất đúng cho mọi mục đích. Để phân biệt các hệ thống, việc đặt ra hai câu hỏi sau đây là đặc biệt hữu ích:

Điều gì xảy ra nếu các producer gửi message nhanh hơn mức các consumer có thể xử lý?

Nói một cách rộng quát, hệ thống có ba lựa chọn: loại bỏ (drop) các message, đệm (buffer) các message trong một queue, hoặc áp dụng *backpressure* (áp lực ngược) (còn được gọi là *flow control*, thứ ngăn chặn producer gửi thêm message). Ví dụ, Unix pipes và TCP sử dụng backpressure; chúng có một buffer kích thước cố định nhỏ, và nếu nó đầy, sender sẽ bị chặn cho đến khi recipient lấy dữ liệu ra khỏi buffer (xem “Network congestion and queueing”).

Nếu các message được đệm trong một queue, điều quan trọng là phải hiểu điều gì xảy ra khi queue đó lớn dần. Hệ thống có bị crash nếu queue không còn nằm vừa trong bộ nhớ, hay nó sẽ ghi các message vào đĩa? Trong trường hợp sau, việc truy cập đĩa ảnh hưởng như thế nào đến hiệu năng của messaging system [5], và điều gì xảy ra khi đĩa bị đầy [6]?

Điều gì xảy ra nếu các node bị crash hoặc tạm thời ngoại tuyến—liệu có bất kỳ message nào bị mất không?

Tương tự như với các cơ sở dữ liệu, durability có thể yêu cầu sự kết hợp giữa việc ghi vào đĩa và/hoặc replication (xem thanh bên “Replication and Durability”), điều này sẽ đi kèm với một chi phí. Nếu bạn có thể chấp nhận việc đôi khi mất các message, bạn có thể đạt được throughput cao hơn và latency thấp hơn trên cùng một hạ tầng phần cứng.

Việc mất message có thể chấp nhận được hay không phụ thuộc rất nhiều vào ứng dụng. Ví dụ, với các dữ liệu đọc từ cảm biến và các metrics được truyền đi định kỳ, một điểm dữ liệu bị thiếu thỉnh thoảng có lẽ không quan trọng, vì một giá trị cập nhật sẽ được gửi đi ngay sau đó thôi. Tuy nhiên, hãy lưu ý rằng nếu một số lượng lớn message bị loại bỏ, có thể sẽ không thấy ngay được rằng các metrics đang bị sai [7]. Nếu bạn đang đếm các sự kiện, việc chúng được chuyển đến một cách tin cậy là quan trọng hơn, vì mỗi message bị mất đồng nghĩa với việc các bộ đếm bị sai.

Một đặc tính hay của các hệ thống batch processing mà chúng ta đã tìm hiểu ở Chương 11 là chúng cung cấp một sự đảm bảo về độ tin cậy mạnh mẽ. Các task thất bại sẽ tự động được thử lại, và kết quả đầu ra một phần từ các task thất bại sẽ tự động bị loại bỏ. Điều này có nghĩa là kết quả đầu ra sẽ giống như thể không có lỗi nào xảy ra, giúp đơn giản hóa mô hình lập trình. Sau đây trong chương này, chúng ta sẽ xem xét cách chúng ta có thể cung cấp các đảm bảo tương tự trong ngữ cảnh streaming.

Truyền tin trực tiếp từ producer đến consumer

Một số hệ thống messaging sử dụng giao tiếp mạng trực tiếp giữa các producer và consumer mà không cần dùng đến các node trung gian:

- UDP multicast được sử dụng rộng rãi trong ngành tài chính cho các luồng như dữ liệu thị trường chứng khoán, nơi mà latency thấp là yếu tố quan trọng [8]. Mặc dù bản thân UDP không đáng tin cậy, các giao thức ở tầng ứng dụng có thể khôi phục các packet bị mất (producer phải ghi nhớ các packet mà nó đã gửi để có thể truyền lại khi có yêu cầu).
- Các thư viện messaging không dùng broker như ZeroMQ và nanomsg cũng áp dụng cách tiếp cận tương tự, thực hiện messaging theo mô hình publish/subscribe qua TCP hoặc IP multicast.
- Một số agent thu thập metrics, chẳng hạn như StatsD [9], sử dụng UDP messaging không đáng tin cậy để thu thập metrics từ tất cả các máy trên mạng và

giám sát chúng. (Trong giao thức StatsD, các counter metrics chỉ chính xác nếu tất cả các message đều được nhận; việc sử dụng UDP khiến các metrics cùng lắm chỉ là xấp xỉ [10]. Xem thêm “TCP Versus UDP”.)

- Nếu consumer cung cấp một service trên mạng, các producer có thể thực hiện một yêu cầu HTTP hoặc RPC trực tiếp (xem “Dataflow Through Services: REST and RPC”) để đẩy các message đến consumer. Đây chính là ý tưởng đằng sau webhooks [11], một pattern mà trong đó một callback URL của một service được đăng ký với một service khác, và nó sẽ thực hiện một yêu cầu đến URL đó bất cứ khi nào một sự kiện xảy ra.

Mặc dù các hệ thống direct messaging này hoạt động tốt trong các tình huống mà chúng được thiết kế, chúng thường yêu cầu mã nguồn ứng dụng phải nhận thức được khả năng mất message. Các lỗi mà chúng có thể chịu đựng được là khá hạn chế. Ngay cả khi các giao thức phát hiện và truyền lại các packet bị mất trong mạng, chúng thường giả định rằng các producer và consumer luôn ở trạng thái online.

Nếu một consumer đang offline, nó có thể bỏ lỡ các message đã được gửi trong khi nó không thể kết nối được. Một số giao thức cho phép producer thử lại việc chuyển các message bị lỗi, nhưng cách tiếp cận này có thể thất bại nếu producer bị crash, làm mất bộ đệm các message mà nó đáng lẽ phải thử lại.

Message brokers

Một giải pháp thay thế được sử dụng rộng rãi là gửi các message thông qua một *message broker* (còn được gọi là *message queue*), về cơ bản nó là một loại cơ sở dữ liệu được tối ưu hóa để xử lý các luồng message [12]. Nó chạy như một server, với các producer và consumer kết nối tới nó với tư cách là các client. Các producer ghi các message vào broker, và broker chuyển chúng đến các consumer.

Bằng cách tập trung dữ liệu vào broker, các hệ thống này có thể dễ dàng chịu lỗi hơn đối với các client thường xuyên đến và đi (kết nối, ngắt kết nối và gặp sự cố), và vấn đề về durability được chuyển sang cho broker xử lý. Một số message broker chỉ giữ các message trong bộ nhớ, trong khi những cái khác (tùy thuộc vào cấu hình) sẽ ghi chúng vào đĩa để chúng không bị mất trong trường hợp broker gặp sự cố. Khi đối mặt với các consumer chậm, chúng thường cho phép queueing không giới hạn (thay vì loại bỏ message hoặc áp dụng backpressure), mặc dù lựa chọn này cũng có thể phụ thuộc vào cấu hình.

Một hệ quả của queueing là các consumer thường mang tính *asynchronous*. Khi một producer gửi một message, nó thường chỉ đợi broker xác nhận rằng message đã được lưu vào bộ đệm (buffered) chứ không đợi message đó được xử lý bởi các consumer. Việc chuyển giao tới các consumer sẽ diễn ra tại một thời điểm không xác định trong tương lai—thường là trong một phần nhỏ của giây, nhưng đôi khi muộn hơn đáng kể nếu có sự tồn đọng trong queue.

So sánh message broker với cơ sở dữ liệu

Một số message broker thậm chí có thể tham gia vào các giao thức two-phase commit bằng cách sử dụng XA hoặc JTA (xem mục “Distributed Transactions Across Different Systems”). Tính năng này khiến chúng có bản chất khá giống với các cơ sở dữ liệu, mặc dù message broker và cơ sở dữ liệu vẫn có những khác biệt thực tế quan trọng:

- Cơ sở dữ liệu thường giữ dữ liệu cho đến khi nó bị xóa một cách rõ ràng, trong khi một số message broker tự động xóa message sau khi nó đã được chuyển giao thành công tới các consumer. Những message broker như vậy không phù hợp để lưu trữ dữ liệu dài hạn.
- Vì chúng xóa message nhanh chóng, hầu hết các message broker đều giả định rằng working set của chúng khá nhỏ—nghĩa là các queue rất ngắn. Nếu broker cần đệm (buffer) nhiều message vì các consumer chậm (có lẽ là đẩy message ra đĩa nếu chúng không còn đủ chỗ trong bộ nhớ), mỗi message riêng lẻ sẽ mất nhiều thời gian hơn để xử lý, và throughput tổng thể có thể bị giảm sút [5].
- Cơ sở dữ liệu thường hỗ trợ secondary index và nhiều cách khác nhau để tìm kiếm dữ liệu bằng ngôn ngữ truy vấn, trong khi message broker thường hỗ trợ một số cách để đăng ký (subscribe) vào một tập hợp con các topic khớp với một pattern. Cả hai về bản chất đều là những cách để một client chọn phần dữ liệu mà nó muốn biết, nhưng các cơ sở dữ liệu thường cung cấp chức năng truy vấn tiên tiến hơn nhiều.
- Khi truy vấn một cơ sở dữ liệu, kết quả thường dựa trên một snapshot tại một thời điểm của dữ liệu. Nếu một client khác sau đó ghi một thứ gì đó vào cơ sở dữ liệu làm thay đổi kết quả truy vấn, client đầu tiên sẽ không biết rằng kết quả trước đó của nó hiện đã lỗi thời (trừ khi nó lặp lại truy vấn hoặc thăm dò (poll) các thay đổi). Ngược lại, các message broker không hỗ trợ các truy vấn tùy ý và không cho phép cập nhật message sau khi chúng đã được gửi, nhưng chúng có thông báo cho các client khi dữ liệu thay đổi (tức là khi các message mới có sẵn).

Đây là quan điểm truyền thống về message broker, vốn được đóng gói trong các tiêu chuẩn như JMS [13] và AMQP [14] và được triển khai trong các phần mềm như RabbitMQ, ActiveMQ, HornetQ, Qpid, TIBCO Enterprise Message Service, IBM MQ, Azure Service Bus, và Google Cloud Pub/Sub [15]. Mặc dù có thể sử dụng cơ sở dữ liệu làm queue, nhưng việc tinh chỉnh chúng để đạt được hiệu năng tốt không hề đơn giản [16].

Nhiều consumer

Khi nhiều consumer cùng đọc các message trong cùng một topic, hai pattern nhận tin chính được sử dụng, như được minh họa trong Hình 12-1:

Load balancing

Mỗi message được chuyển giao tới *một* trong số các consumer, vì vậy các consumer có thể chia sẻ công việc xử lý các message trong topic. Broker có thể gán các message cho các consumer một cách ngẫu nhiên. Pattern này hữu ích khi các message tốn kém để xử lý, vì vậy bạn muốn có thể thêm các consumer để song song hóa việc xử lý. (Trong AMQP, bạn có thể triển khai load balancing bằng cách có nhiều client tiêu thụ từ cùng một queue, và trong JMS nó được gọi là một *shared subscription*.)

Fan-out

Mỗi message được chuyển đến *tất cả* các consumer. Cơ chế fan-out cho phép nhiều consumer độc lập cùng "tune in" vào cùng một luồng broadcast các message mà không ảnh hưởng đến nhau—tương đương với việc có nhiều batch job cùng đọc một file đầu vào trong streaming. (Tính năng này được cung cấp bởi topic subscriptions trong JMS và exchange bindings trong AMQP.)

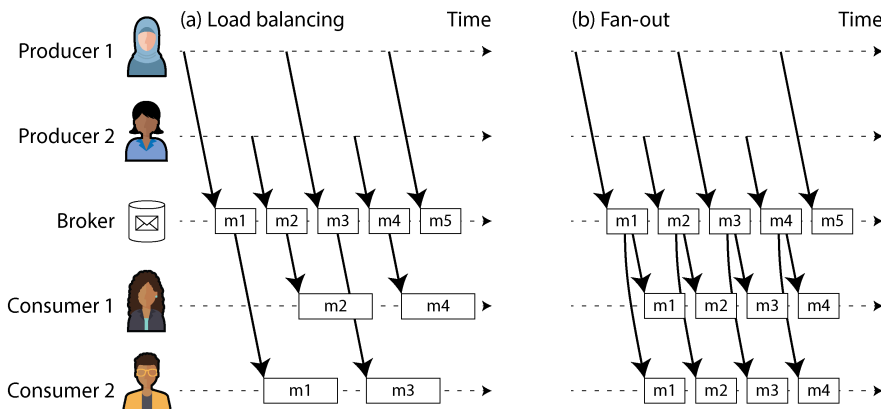


Figure 12-1. (a) *Load balancing* chia sẻ công việc tiêu thụ một topic giữa các consumer; (b) với *fan-out*, mỗi message được chuyển đến nhiều consumer.

Hai pattern này có thể được kết hợp—ví dụ, bằng cách sử dụng tính năng *consumer groups* của Kafka. Khi một consumer group đăng ký vào một topic, mỗi message trong topic đó sẽ được gửi đến một trong các consumer trong group (giúp cân bằng tải giữa các consumer trong group). Nếu hai consumer group riêng biệt cùng đăng ký vào một topic, mỗi message sẽ được gửi đến một consumer trong mỗi group (cung cấp khả năng fan-out giữa các consumer group).

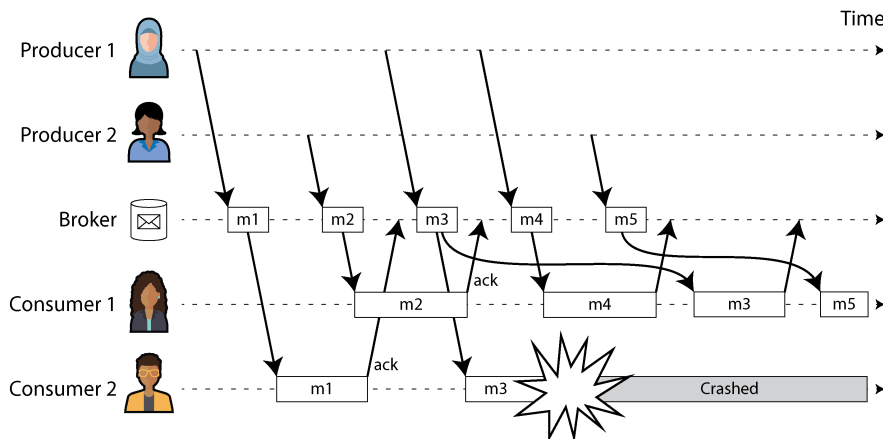
Acknowledgments và redelivery

Các consumer có thể bị crash bất cứ lúc nào. Do đó, một broker có thể gửi một message đến một consumer nhưng consumer đó không bao giờ xử lý nó, hoặc chỉ xử lý một phần trước khi bị crash. Để đảm bảo message không bị mất, các message

broker sử dụng *acknowledgments*: một client phải thông báo rõ ràng cho broker khi nó đã hoàn tất việc xử lý một message để broker có thể xóa nó khỏi queue.

Nếu kết nối tới một client bị đóng hoặc hết thời gian (timeout) mà broker không nhận được acknowledgment, nó sẽ giả định rằng message đó chưa được xử lý, và do đó nó sẽ gửi lại message đó cho một consumer khác. (Lưu ý rằng có thể xảy ra trường hợp message thực tế *đã* được xử lý hoàn tất, nhưng acknowledgment đã bị thất lạc trên mạng. Việc xử lý trường hợp này đòi hỏi một giao thức atomic commit, như đã thảo luận trong mục “Exactly-once message processing”, trừ khi thao tác đó là idempotent hoặc không yêu cầu semantics exactly-once.)

Khi kết hợp với load balancing, hành vi redelivery này có một hiệu ứng thú vị đối với thứ tự của các message. Trong Hình 12-2, các consumer thường xử lý các message theo thứ tự mà chúng được gửi bởi các producer. Tuy nhiên, consumer 2 bị crash trong khi đang xử lý message *m3*, cùng lúc với việc consumer 1 đang xử lý message *m4*. Message *m3* chưa được acknowledgment sau đó được redelivered tới consumer 1, dẫn đến kết quả là consumer 1 xử lý các message theo thứ tự *m4*, *m3*, *m5*. Do đó, *m3* và *m4* không được gửi theo cùng thứ tự như khi chúng được gửi bởi producer 1.



*Figure 12-2. Consumer 2 bị crash trong khi đang xử lý *m3*, vì vậy nó được gửi lại cho consumer 1 vào một thời điểm sau đó.*

Ngay cả khi message broker cố gắng bảo toàn thứ tự của các message (như yêu cầu của cả hai tiêu chuẩn JMS và AMQP), sự kết hợp giữa load balancing với redelivery chắc chắn sẽ dẫn đến việc các message bị thay đổi thứ tự. Để tránh vấn đề này, bạn có thể sử dụng một queue riêng biệt cho mỗi consumer (tức là không sử dụng tính năng load balancing). Việc thay đổi thứ tự message không phải là vấn đề nếu các message hoàn toàn độc lập với nhau, nhưng nó có thể quan trọng nếu có các phụ thuộc nhân quả (causal dependencies) giữa các message, như chúng ta sẽ thấy sau trong chương này.

Redelivery cũng có thể dẫn đến lãng phí tài nguyên, tình trạng đói tài nguyên (resource starvation), hoặc gây tắc nghẽn vĩnh viễn trong một stream. Một kịch bản phổ biến là một producer thực hiện serialization một message không đúng cách—ví dụ, bằng cách bỏ sót một key bắt buộc trong một đối tượng được mã hóa JSON. Nếu message thiếu key đó khiến một consumer bị crash và khởi động lại, nó sẽ không gửi acknowledgment cho message, vì vậy broker sẽ gửi lại nó, điều này sẽ khiến một consumer khác thất bại. Vòng lặp này lặp lại vô tận. Nếu broker đảm bảo strong ordering, sẽ không có tiến triển nào có thể thực hiện được thêm nữa. Các broker cho phép thay đổi thứ tự message có thể tiếp tục thực hiện tiến triển, nhưng chúng sẽ lãng phí tài nguyên vào những message sẽ không bao giờ được acknowledgment.

Dead letter queues (DLQs) được sử dụng để xử lý vấn đề này. Thay vì giữ message trong queue hiện tại và thử lại mãi mãi, message sẽ được chuyển sang một queue khác để giải phóng các consumer [17, 18]. Việc giám sát (monitoring) thường được thiết lập trên các DLQ—bất kỳ message nào trong queue cũng đều là một lỗi. Khi một message mới được phát hiện, một operator có thể quyết định loại bỏ nó vĩnh viễn, sửa đổi và tái tạo message một cách thủ công, hoặc sửa code của consumer để xử lý message đó một cách phù hợp. DLQ rất phổ biến trong hầu hết các hệ thống queuing, nhưng các hệ thống messaging dựa trên log như Apache Pulsar và các hệ thống stream processing như Kafka Streams hiện cũng đã hỗ trợ chúng [19].

Các Message Broker dựa trên log

Gửi một packet qua mạng hoặc thực hiện một request tới một dịch vụ mạng thường là một thao tác tạm thời (transient operation) không để lại dấu vết vĩnh viễn. Mặc dù có thể ghi lại một thao tác như vậy một cách vĩnh viễn (sử dụng packet capture và logging), nhưng chúng ta thường không nghĩ theo cách đó. Các message broker kiểu AMQP/JMS đã kế thừa tư duy messaging tạm thời này. Ngay cả khi chúng có thể ghi các message vào đĩa, chúng cũng nhanh chóng xóa các message đó đi sau khi chúng đã được chuyển đến các consumer.

Các cơ sở dữ liệu và hệ thống tệp tin (filesystems) thực hiện cách tiếp cận ngược lại: mọi thứ được ghi vào một cơ sở dữ liệu hoặc tệp tin thường được kỳ vọng là sẽ được ghi lại vĩnh viễn, ít nhất là cho đến khi có ai đó chủ động chọn xóa nó đi.

Sự khác biệt trong tư duy này có tác động lớn đến cách dữ liệu phái sinh (derived data) được tạo ra. Một feature chính của các batch process, như đã thảo luận trong Chương 11, là bạn có thể chạy chúng lặp đi lặp lại, thử nghiệm với các bước xử lý mà không lo ngại việc làm hỏng dữ liệu đầu vào (vì dữ liệu đầu vào là chỉ đọc). Điều này không đúng với messaging kiểu AMQP/JMS: việc nhận một message mang tính phá hủy nếu acknowledgment khiến nó bị xóa khỏi broker, vì vậy bạn không thể chạy cùng một consumer một lần nữa và mong đợi nhận được cùng một kết quả.

Nếu bạn thêm một consumer mới vào một hệ thống messaging, nó thường chỉ bắt đầu nhận các message được gửi sau thời điểm nó được đăng ký; bất kỳ message nào trước đó đều đã mất và không thể khôi phục. Hãy đối chiếu điều này với các tệp tin và cơ sở dữ liệu, nơi bạn có thể thêm một client mới vào bất kỳ lúc nào, và nó có thể đọc dữ liệu đã được ghi từ rất lâu trong quá khứ (miễn là dữ liệu đó chưa bị ứng dụng ghi đè hoặc xóa một cách rõ ràng).

Tại sao chúng ta không thể có một mô hình hybrid, kết hợp cách tiếp cận lưu trữ bền vững của cơ sở dữ liệu với các tiện ích thông báo có độ trễ thấp (low-latency) của messaging? Đây chính là ý tưởng đằng sau các *log-based message brokers*, thứ đã trở nên rất phổ biến trong những năm gần đây.

Sử dụng log để lưu trữ message

Một log đơn giản là một chuỗi các bản ghi chỉ cho phép ghi thêm (append-only) trên đĩa. Trước đó, chúng ta đã thảo luận về log trong ngữ cảnh của các log-structured storage engine và write-ahead log ở Chương 4, trong ngữ cảnh của replication ở Chương 6, và như một hình thức của consensus ở Chương 10.

Cấu trúc tương tự có thể được sử dụng để triển khai một message broker. Một producer gửi một message bằng cách ghi thêm nó vào cuối log, và một consumer nhận các message bằng cách đọc log một cách tuần tự. Nếu một consumer đi đến cuối log, nó sẽ đợi một thông báo rằng một message mới đã được ghi thêm. Công cụ Unix `tail` với tùy chọn `-f`, dùng để theo dõi một tệp khi có dữ liệu được ghi thêm, về cơ bản hoạt động giống như thế này.

Để mở rộng lên throughput cao hơn mức mà một đĩa đơn có thể cung cấp, log có thể được *sharding* (theo nghĩa đã nói ở Chương 7). Các shard khác nhau sau đó có thể được lưu trữ trên các máy khác nhau, khiến mỗi shard trở thành một log riêng biệt có thể được đọc và ghi độc lập với các shard khác, và một topic có thể được định nghĩa là một nhóm các shard mà tất cả đều mang các message cùng loại. Cách tiếp cận này được minh họa trong Hình 12-3.

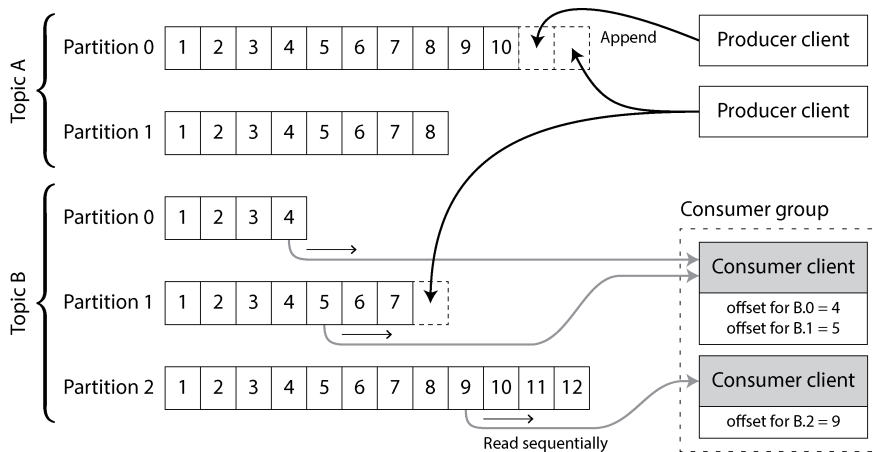


Figure 12-3. Các producer gửi message bằng cách append chúng vào một file partition của topic, và các consumer đọc các file này theo trình tự.

Trong mỗi shard, thứ mà Kafka gọi là một *partition*, broker gán một số thứ tự tăng dần (monotonically increasing sequence number), hay còn gọi là *offset*, cho mỗi message (trong Hình 12-3, các con số trong hộp là các message offset). Một số thứ tự như vậy là hợp lý vì một partition (shard) là dạng append-only, do đó các message bên trong một partition được sắp xếp theo thứ tự hoàn toàn (totally ordered). Không có sự đảm bảo về thứ tự giữa các partition khác nhau.

Apache Kafka [20] và Amazon Kinesis Streams là các message broker dựa trên log (log-based) hoạt động theo cách này. Google Cloud Pub/Sub có kiến trúc tương tự nhưng cung cấp một API kiểu JMS thay vì một sự trừu tượng hóa dạng log [15]. Mặc dù các message broker này ghi tất cả message vào đĩa, chúng vẫn có thể đạt được throughput hàng triệu message mỗi giây bằng cách sharding trên nhiều máy và đạt được khả năng chịu lỗi (fault tolerance) bằng cách replication các message [21, 22].

So sánh log với messaging truyền thống

Cách tiếp cận dựa trên log hỗ trợ một cách dễ dàng việc fan-out messaging, bởi vì nhiều consumer có thể đọc log một cách độc lập mà không ảnh hưởng đến nhau; việc đọc một message không làm xóa nó khỏi log. Để đạt được load balancing giữa một nhóm các consumer, broker có thể gán toàn bộ các shard cho các node trong consumer group thay vì gán từng message riêng lẻ cho các consumer client.

Sau đó, mỗi client sẽ tiêu thụ *tất cả* các message trong các shard mà nó được gán. Thông thường, khi một consumer được gán một log shard, nó sẽ đọc các message trong shard đó một cách tuần tự, theo cách đơn luồng (single-threaded) đơn giản.

Cách tiếp cận load balancing ở mức hạt thô (coarse-grained) này có những nhược điểm:

- Số lượng các node chia sẻ công việc tiêu thụ một topic có thể tối đa bằng số lượng log shards trong topic đó, bởi vì các message trong cùng một shard sẽ được chuyển đến cùng một node. (Có thể tạo ra một cơ chế load balancing mà trong đó hai consumer chia sẻ công việc xử lý một shard bằng cách cả hai cùng đọc toàn bộ tập hợp message nhưng một trong hai chỉ xét các message có offset là số chẵn trong khi cái còn lại xử lý các offset là số lẻ. Ngoài ra, bạn có thể phân bổ việc xử lý message qua một thread pool, nhưng cách tiếp cận đó làm phức tạp việc quản lý consumer offset. Nhìn chung, xử lý một shard theo kiểu đơn luồng là ưu tiên hơn, và tính song song có thể được tăng lên bằng cách sử dụng nhiều shard hơn.)
- Nếu một message duy nhất xử lý chậm, nó sẽ làm trì trệ việc xử lý các message tiếp theo trong shard đó (một dạng head-of-line blocking; xem mục “Describing Performance”).

Do đó, khi các message có thể tốn kém để xử lý và bạn muốn song song hóa việc xử lý trên từng message một, đồng thời thứ tự message không quá quan trọng, thì kiểu message broker theo phong cách JMS/AMQP sẽ được ưu tiên hơn. Mặt khác, trong các tình huống có throughput message cao, nơi mỗi message được xử lý nhanh và thứ tự message là quan trọng, cách tiếp cận dựa trên log hoạt động rất tốt [23, 24]. Tuy nhiên, sự phân biệt giữa hai kiến trúc này đang dần mờ nhạt vì các hệ thống messaging dựa trên log như Kafka hiện đã hỗ trợ các consumer group kiểu JMS/AMQP, cho phép nhiều consumer nhận message từ cùng một partition [25, 26].

Vì các log đã được sharding thường chỉ bảo toàn thứ tự message trong phạm vi một shard duy nhất, nên tất cả các message cần được sắp xếp thứ tự một cách nhất quán đều phải được định tuyến đến cùng một shard. Ví dụ, một ứng dụng có thể yêu cầu các event liên quan đến một người dùng cụ thể phải xuất hiện theo một thứ tự cố định. Điều này có thể đạt được bằng cách chọn shard cho một event dựa trên user ID của event đó (nói cách khác, biến user ID thành *partition key*).

Consumer offsets

Việc tiêu thụ một shard một cách tuần tự giúp dễ dàng xác định những message nào đã được xử lý. Tất cả các message có offset nhỏ hơn offset hiện tại của consumer đều đã được xử lý, và tất cả các message có offset lớn hơn thì vẫn chưa được thấy. Do đó, broker không cần phải theo dõi các acknowledgment cho từng message riêng lẻ mà chỉ cần định kỳ ghi lại các consumer offset. Việc giảm bớt chi phí quản lý (bookkeeping overhead) cùng với các cơ hội để thực hiện batching và pipelining trong cách tiếp cận này giúp tăng throughput của các hệ thống dựa trên log. Tuy nhiên, nếu một consumer gặp lỗi, nó sẽ tiếp tục từ offset được ghi lại gần nhất thay vì offset gần nhất mà nó đã thấy. Điều này có thể khiến consumer thấy một số message hai lần.

Thực tế, offset rất giống với *log sequence number* thường thấy trong replication cơ sở dữ liệu single-leader mà chúng ta đã thảo luận trong mục “Setting Up New Followers”. Trong replication cơ sở dữ liệu, log sequence number cho phép một follower kết nối lại với leader sau khi bị mất kết nối và tiếp tục quá trình replication mà không bỏ lỡ bất kỳ thao tác write nào. Nguyên lý chính xác tương tự cũng được sử dụng ở đây: message broker hoạt động giống như một leader database và consumer hoạt động giống như một follower.

Nếu một consumer node gặp lỗi, một node khác trong consumer group sẽ được chỉ định để nhận các shard của consumer bị lỗi đó, và nó sẽ bắt đầu tiêu thụ các message tại offset được ghi lại gần nhất. Nếu consumer đã xử lý các message tiếp theo những chưa kịp ghi lại offset của chúng, những message đó sẽ bị xử lý lại lần thứ hai khi khởi động lại. Chúng ta sẽ thảo luận về các cách xử lý vấn đề này ở phần sau của chương.

Sử dụng không gian đĩa

Nếu bạn chỉ thực hiện append vào log, cuối cùng bạn sẽ hết dung lượng đĩa. Để thu hồi dung lượng đĩa, log được chia thành các segment, và theo thời gian, các segment cũ sẽ bị xóa hoặc chuyển sang archive storage. (Chúng ta sẽ thảo luận về một cách tinh vi hơn để giải phóng dung lượng đĩa trong mục “Log compaction”).

Điều này có nghĩa là nếu một consumer chậm không thể theo kịp tốc độ của các message, và nó bị tụt lại quá xa đến mức consumer offset của nó trở vào một segment đã bị xóa, nó sẽ bỏ lỡ một số message. Về cơ bản, log thực thi một buffer có kích thước giới hạn, tự loại bỏ các message cũ khi nó đầy, còn được gọi là *circular buffer* hoặc *ring buffer*. Tuy nhiên, vì buffer đó nằm trên đĩa, nó có thể rất lớn.

Hãy thực hiện một phép tính nhanh (back-of-the-envelope calculation). Tại thời điểm viết tài liệu này, một ổ cứng lớn điển hình có dung lượng 20 TB và throughput ghi tuần tự là 250 MB/s. Nếu bạn đang ghi các message với tốc độ nhanh nhất có thể, sẽ mất khoảng 22 giờ cho đến khi ổ đĩa đầy và bạn cần bắt đầu xóa các message cũ nhất. Điều đó có nghĩa là một log dựa trên đĩa luôn có thể buffer ít nhất lượng message tương đương 22 giờ, ngay cả khi bạn có nhiều đĩa với nhiều máy (việc có nhiều đĩa hơn sẽ làm tăng cả dung lượng khả dụng và tổng băng thông ghi). Trong thực tế, các deployment hiếm khi sử dụng hết băng thông ghi của đĩa, vì vậy log thường có thể duy trì một buffer chứa lượng message của vài ngày hoặc thậm chí vài tuần.

Nhiều message broker dựa trên log hiện nay lưu trữ các message trong object storage để tăng khả năng lưu trữ, tương tự như các cơ sở dữ liệu mà chúng ta đã thấy trong mục “Databases Backed by Object Storage”. Các message broker như Apache Kafka và Redpanda phục vụ các message cũ từ object storage như một phần của tiered storage của chúng. Những hệ thống khác, chẳng hạn như WarpStream, Confluent Freight và Bufstream, lưu trữ tất cả dữ liệu của chúng trong object store. Ngoài hiệu quả về chi phí, kiến trúc này giúp việc tích hợp dữ liệu trở nên dễ dàng hơn: các

message trong object storage được lưu trữ dưới dạng các Iceberg table, cho phép thực thi các công việc batch và data warehouse trực tiếp trên dữ liệu mà không cần phải sao chép chúng sang một hệ thống khác.

Khi consumer không thể theo kịp producer

Ở phần đầu của “Messaging Systems”, chúng ta đã thảo luận về ba lựa chọn cho việc cần làm nếu một consumer không thể theo kịp tốc độ gửi tin nhắn của producer: loại bỏ tin nhắn, buffering (đệm), hoặc áp dụng backpressure (áp lực ngược). Trong phân loại này, cách tiếp cận dựa trên log là một dạng buffering với một bộ đệm có kích thước lớn nhưng cố định (bị giới hạn bởi dung lượng đĩa trống hiện có).

Nếu một consumer bị tụt lại quá xa đến mức các tin nhắn mà nó yêu cầu đã cũ hơn những tin nhắn được lưu trữ trên đĩa, nó sẽ không thể đọc được những tin nhắn đó—vì vậy broker sẽ loại bỏ một cách hiệu quả các tin nhắn cũ đã vượt quá khả năng chứa của bộ đệm. Bạn có thể giám sát xem một consumer đang tụt lại bao xa so với head của log và đưa ra cảnh báo nếu nó bị tụt lại đáng kể. Vì bộ đệm lớn, sẽ có đủ thời gian để một người vận hành có thể khắc phục consumer đang bị chậm và cho phép nó bắt kịp trước khi nó bắt đầu bị mất tin nhắn.

Ngay cả khi một consumer bị tụt lại quá xa và bắt đầu mất tin nhắn, thì chỉ có consumer đó bị ảnh hưởng; nó không làm gián đoạn dịch vụ cho các consumer khác. Thực tế này là một lợi thế lớn về mặt vận hành. Bạn có thể tiêu thụ một log production để phục vụ mục đích phát triển, kiểm thử hoặc debugging một cách thực nghiệm mà không phải lo lắng quá nhiều về việc làm gián đoạn các dịch vụ production. Khi một consumer bị tắt hoặc bị crash, nó sẽ ngừng tiêu thụ tài nguyên—thứ duy nhất còn lại là consumer offset của nó.

Hành vi này tương phản với các message broker truyền thống, nơi bạn cần phải cẩn thận xóa bất kỳ queue nào mà các consumer của chúng đã bị tắt để tránh việc chúng tích tụ tin nhắn không cần thiết và chiếm dụng bộ nhớ của các consumer đang hoạt động.

Replaying các tin nhắn cũ

Trước đó chúng ta đã lưu ý rằng với các message broker kiểu AMQP và JMS, việc xử lý và acknowledging tin nhắn là một thao tác mang tính hủy diệt, vì nó khiến các tin nhắn bị xóa trên broker. Ngược lại, trong một message broker dựa trên log, việc tiêu thụ tin nhắn giống như việc đọc từ một tệp hơn: đó là một thao tác chỉ đọc (read-only) và không làm thay đổi log.

Tác dụng phụ duy nhất của việc xử lý, bên cạnh bất kỳ đầu ra nào của consumer, là consumer offset sẽ tiến về phía trước. Nhưng offset nằm dưới sự kiểm soát của consumer, vì vậy nó có thể dễ dàng được thao tác nếu cần thiết—ví dụ, bạn có thể khởi chạy một bản sao của consumer với offset của ngày hôm qua và ghi đầu ra vào

một vị trí khác để xử lý lại lượng tin nhắn của ngày hôm trước. Bạn có thể lặp lại việc này bao nhiêu lần tùy ý với các mã xử lý khác nhau.

Khía cạnh này khiến việc truyền tin dựa trên log giống với các batch process đã thảo luận ở chương trước, nơi dữ liệu phái sinh (derived data) được tách biệt rõ ràng khỏi dữ liệu đầu vào thông qua một quá trình biến đổi có thể lặp lại. Nó cho phép thử nghiệm nhiều hơn và phục hồi dễ dàng hơn sau các lỗi và bug, khiến nó trở thành một công cụ tốt để tích hợp các dataflow trong một tổ chức [27].

Databases và Streams

Chúng ta đã đưa ra một số so sánh giữa message broker và database. Mặc dù chúng theo truyền thống được coi là các danh mục công cụ riêng biệt, chúng ta đã thấy rằng các message broker dựa trên log đã thành công trong việc tiếp nhận các ý tưởng từ database và áp dụng chúng vào việc truyền tin. Chúng ta cũng có thể làm điều ngược lại, lấy các ý tưởng từ việc truyền tin và các stream để áp dụng cho database.

Một cách tiếp cận là sử dụng một event stream làm system of record để lưu trữ dữ liệu (xem “Systems of Record and Derived Data”). Đây là những gì xảy ra trong event sourcing, điều mà chúng ta đã thảo luận trong “Event Sourcing and CQRS”. Thay vì lưu trữ dữ liệu trong một data model bị biến đổi bằng cách cập nhật và xóa, bạn có thể mô hình hóa mọi sự thay đổi trạng thái thành một event bất biến và ghi nó vào một append-only log. Bất kỳ materialized view nào được tối ưu hóa cho việc đọc đều được phái sinh từ các event này. Các message broker dựa trên log (được cấu hình để không bao giờ xóa các event cũ) rất phù hợp cho event sourcing vì chúng sử dụng lưu trữ append-only, và chúng có thể thông báo cho các consumer về các event mới với latency thấp.

Nhưng bạn không nhất thiết phải đi xa đến mức áp dụng event sourcing; ngay cả với các mô hình dữ liệu có thể thay đổi (mutable data models), các event stream vẫn rất hữu ích cho cơ sở dữ liệu. Trên thực tế, mọi thao tác ghi vào cơ sở dữ liệu đều là một event có thể được thu thập, lưu trữ và xử lý. Mối liên kết giữa cơ sở dữ liệu và các stream còn sâu sắc hơn cả việc chỉ lưu trữ vật lý các log trên đĩa—nó mang tính nền tảng.

Ví dụ, một replication log (xem “Implementation of Replication Logs”) là một stream gồm các event ghi của cơ sở dữ liệu, được tạo ra bởi leader khi nó xử lý các transaction. Các follower áp dụng stream ghi đó vào bản sao cơ sở dữ liệu của riêng chúng và do đó có được một bản sao chính xác của cùng một dữ liệu. Các event trong replication log mô tả những thay đổi dữ liệu đã xảy ra.

Chúng ta cũng đã gặp nguyên lý state machine replication trong mục “Using shared logs”, trong đó nêu rõ: nếu mọi event đều đại diện cho một thao tác ghi vào cơ sở dữ liệu, và mọi replica đều xử lý các event giống nhau theo cùng một thứ tự, thì tất cả

các replica sẽ đều dẫn đến cùng một trạng thái cuối cùng. (Việc xử lý một event được giả định là một thao tác deterministic.) Đó chỉ là một trường hợp khác của event streams!

Trong mục này, trước tiên chúng ta sẽ xem xét một vấn đề phát sinh trong các hệ thống dữ liệu không đồng nhất (heterogeneous data systems), sau đó khám phá cách chúng ta có thể giải quyết nó bằng cách đưa các ý tưởng từ event streams vào cơ sở dữ liệu.

Giữ các hệ thống đồng bộ

Như chúng ta đã thấy xuyên suốt cuốn sách này, không một hệ thống đơn lẻ nào có thể thỏa mãn tất cả các nhu cầu về lưu trữ, truy vấn và xử lý dữ liệu. Trong thực tế, hầu hết các ứng dụng không tầm thường đều cần kết hợp nhiều công nghệ khác nhau để đáp ứng các yêu cầu của chúng—ví dụ, sử dụng một cơ sở dữ liệu OLTP để phục vụ các yêu cầu của người dùng, một cache để tăng tốc các yêu cầu phổ biến, một full-text index để xử lý các truy vấn tìm kiếm, và một data warehouse để phân tích. Mỗi thành phần này đều có bản sao dữ liệu riêng, được lưu trữ dưới dạng biểu diễn riêng nhằm tối ưu hóa cho các mục đích riêng của chúng.

Vì cùng một dữ liệu hoặc các dữ liệu liên quan xuất hiện ở nhiều nơi, chúng cần được giữ đồng bộ với nhau. Nếu một mục được cập nhật trong cơ sở dữ liệu, nó cũng cần được cập nhật trong cache, các search index và data warehouse. Với các data warehouse, việc đồng bộ hóa này thường được thực hiện bởi các quy trình ETL (xem “Data Warehousing”), thường bằng cách lấy một bản sao đầy đủ của cơ sở dữ liệu, biến đổi nó và nạp hàng loạt (bulk-loading) vào data warehouse—nói cách khác, đó là một quy trình batch. Tương tự, chúng ta đã thấy trong mục “Batch Use Cases” cách các search index, hệ thống gợi ý và các hệ thống dữ liệu phái sinh (derived data) khác có thể được tạo ra bằng các quy trình batch.

Nếu việc dump toàn bộ cơ sở dữ liệu định kỳ quá chậm, một giải pháp thay thế đôi khi được sử dụng là *dual writes*, trong đó mã nguồn ứng dụng ghi một cách rõ ràng vào từng hệ thống khi dữ liệu thay đổi—ví dụ, đầu tiên ghi vào cơ sở dữ liệu, sau đó cập nhật search index, rồi làm mất hiệu lực (invalidating) các mục trong cache (hoặc thậm chí thực hiện các thao tác ghi đó một cách đồng thời).

Tuy nhiên, dual writes gặp phải những vấn đề nghiêm trọng, một trong số đó là race condition được minh họa trong Hình 12-4. Trong ví dụ này, hai client muốn cập nhật một mục X một cách đồng thời. Client 1 muốn đặt giá trị thành A , và client 2 muốn đặt giá trị thành B . Cả hai client đầu tiên đều ghi giá trị mới vào cơ sở dữ liệu, sau đó ghi vào search index. Do sự trùng hợp ngẫu nhiên về thời điểm, các yêu cầu bị xen kẽ nhau. Cơ sở dữ liệu trước tiên thấy thao tác ghi từ client 1 đặt giá trị thành A , sau đó là thao tác ghi từ client 2 đặt giá trị thành B , vì vậy giá trị cuối cùng trong cơ sở dữ liệu là B . Search index trước tiên thấy thao tác ghi từ client 2, sau đó là client 1,

nên giá trị cuối cùng trong search index là A . Hai hệ thống hiện đang bị mất nhất quán vĩnh viễn với nhau, mặc dù không có lỗi nào xảy ra.

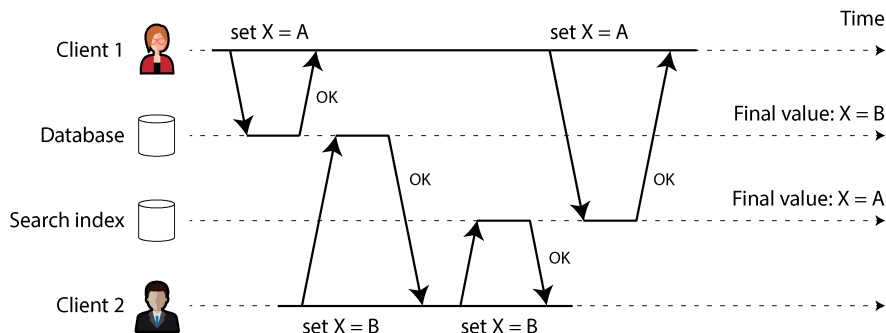


Figure 12-4. Trong cơ sở dữ liệu, X đầu tiên được thiết lập thành A và sau đó là B , trong khi tại search index, các thao tác ghi lại đến theo thứ tự ngược lại.

Trừ khi bạn có thêm một cơ chế phát hiện concurrency (đồng thời) bổ sung, chẳng hạn như version vectors mà chúng ta đã thảo luận trong mục “Detecting Concurrent Writes”, bạn thậm chí sẽ không nhận ra rằng các thao tác ghi đồng thời đã xảy ra. Một giá trị sẽ đơn giản là âm thầm ghi đè lên một giá trị khác.

Một vấn đề khác với dual writes là một trong hai thao tác ghi có thể thất bại trong khi thao tác kia lại thành công. Đây là một vấn đề về fault tolerance (khả năng chịu lỗi) hơn là một vấn đề về concurrency, nhưng nó cũng dẫn đến hệ quả là hai hệ thống trở nên không nhất quán với nhau. Việc đảm bảo rằng chúng hoặc là cả hai cùng thành công hoặc cả hai cùng thất bại là một trường hợp của bài toán atomic commit, vốn rất tốn kém để giải quyết (xem “Two-Phase Commit”).

Nếu bạn chỉ có một cơ sở dữ liệu được replication với một single leader, thì leader đó sẽ quyết định thứ tự của các thao tác ghi, do đó phương pháp state machine replication sẽ hoạt động hiệu quả giữa các replica của cơ sở dữ liệu. Tuy nhiên, trong Hình 12-4 không có một single leader. Cơ sở dữ liệu có thể có một leader và search index có thể có một leader, nhưng không bên nào tuân theo bên nào, vì vậy các xung đột có thể xảy ra (xem “Multi-Leader Replication”).

Tình huống sẽ tốt hơn nếu thực sự chỉ có một leader—ví dụ như cơ sở dữ liệu—và nếu chúng ta có thể biến search index thành một follower của cơ sở dữ liệu. Nhưng liệu điều này có khả thi trong thực tế không?

Change Data Capture

Vấn đề với replication logs của hầu hết các cơ sở dữ liệu là chúng từ lâu đã được coi là một chi tiết triển khai nội bộ của cơ sở dữ liệu, chứ không phải là một API công khai. Các client được cho là sẽ truy vấn cơ sở dữ liệu thông qua data model và ngôn

ngữ truy vấn của nó, chứ không phải phân tích các replication logs và cố gắng trích xuất dữ liệu từ chúng.

Trong nhiều thập kỷ, nhiều cơ sở dữ liệu đơn giản là không có một cách thức được tài liệu hóa để lấy được log của các thay đổi đã được ghi vào chúng. Điều này khiến việc lấy tất cả các thay đổi được thực hiện trong một cơ sở dữ liệu và replication chúng sang một công nghệ lưu trữ khác, chẳng hạn như search index, cache, hoặc data warehouse, trở nên khó khăn.

Gần đây, sự quan tâm đến *change data capture* (CDC) đang ngày càng tăng, đây là quá trình quan sát tất cả các thay đổi dữ liệu được ghi vào một cơ sở dữ liệu và trích xuất chúng dưới một dạng mà chúng có thể được replication sang các hệ thống khác [28]. CDC đặc biệt thú vị nếu các thay đổi được cung cấp dưới dạng một stream, ngay lập tức khi chúng được ghi vào.

Ví dụ, bạn có thể capture các thay đổi trong một cơ sở dữ liệu và liên tục áp dụng các thay đổi tương tự đó vào một search index. Nếu log của các thay đổi được áp dụng theo cùng một thứ tự, bạn có thể kỳ vọng dữ liệu trong search index sẽ khớp với dữ liệu trong cơ sở dữ liệu. Search index và bất kỳ hệ thống derived data nào khác chỉ đơn giản là các consumer của change stream.

Hình 12-5 cho thấy vấn đề concurrency (đồng thời) ở Hình 12-4 được giải quyết như thế nào bằng CDC. Ngay cả khi hai yêu cầu thiết lập X lần lượt thành A và B đến cơ sở dữ liệu cùng một lúc, cơ sở dữ liệu sẽ quyết định thứ tự thực thi chúng và ghi chúng vào replication log theo thứ tự đó. Search index sẽ tiếp nhận và áp dụng chúng theo cùng một thứ tự. Nếu bạn cần dữ liệu trong một hệ thống khác, chẳng hạn như một data warehouse, bạn chỉ đơn giản là thêm nó như một consumer khác của CDC event stream.

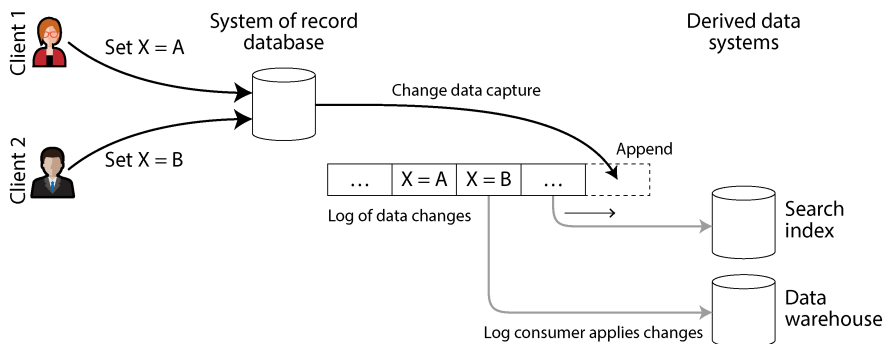


Figure 12-5. Đưa các thay đổi đã được commit vào một cơ sở dữ liệu và lan truyền chúng đến các hệ thống downstream theo cùng một thứ tự

Triển khai CDC

Chúng ta có thể gọi các consumer của log là *hệ thống dữ liệu phái sinh (derived data systems)*, như đã thảo luận trong mục “Systems of Record and Derived Data”. Dữ liệu được lưu trữ trong search index và data warehouse chỉ là một view khác của dữ liệu trong system of record. CDC là một cơ chế để đảm bảo rằng tất cả các thay đổi được thực hiện đối với system of record cũng được phản ánh trong các hệ thống dữ liệu phái sinh, sao cho các hệ thống phái sinh có một bản sao chính xác của dữ liệu.

Về cơ bản, CDC biến một database thành leader (nơi các thay đổi được capture) và biến các database khác thành follower. Một message broker dựa trên log rất phù hợp để vận chuyển các change event từ database nguồn đến các hệ thống phái sinh, vì nó bảo toàn thứ tự của các message (tránh được vấn đề reordering trong Hình 12-2).

Logical replication log có thể được sử dụng để triển khai CDC (xem mục “Logical (row-based) log replication”), mặc dù nó đi kèm với những thách thức, chẳng hạn như xử lý các thay đổi schema và mô hình hóa các update một cách chính xác. Dự án mã nguồn mở Debezium giải quyết các thách thức này. Dự án chứa các *source connector* cho MySQL, PostgreSQL, Oracle, SQL Server, Db2, Cassandra và nhiều database khác. Các connector này gắn vào database replication log và hiển thị các thay đổi theo một event schema tiêu chuẩn. Sau đó, các message có thể được biến đổi và ghi vào các database downstream. Framework Kafka Connect cũng cung cấp các CDC connector cho nhiều database khác nhau. Maxwell thực hiện điều tương tự cho MySQL bằng cách parse binlog [29], GoldenGate cung cấp các tiện ích tương tự cho Oracle, và pgcapture thực hiện điều tương tự cho PostgreSQL.

Giống như các message broker, CDC thường là bất đồng bộ (asynchronous): database system-of-record không đợi một thay đổi được áp dụng cho các consumer trước khi commit nó. Thiết kế này có lợi thế về vận hành là việc thêm một consumer chậm không ảnh hưởng quá nhiều đến system of record, nhưng nó có nhược điểm là

tất cả các vấn đề về replication lag đều xảy ra (xem mục “Problems with Replication Lag”).

Initial snapshot

Nếu bạn có log của tất cả các thay đổi từng được thực hiện đối với một database, bạn có thể tái cấu trúc toàn bộ trạng thái của database bằng cách replay log đó. Tuy nhiên, trong nhiều trường hợp, việc lưu giữ tất cả các thay đổi mãi mãi sẽ đòi hỏi quá nhiều không gian đĩa, và việc replay log sẽ mất quá nhiều thời gian, vì vậy log cần phải được truncate.

Ví dụ, việc xây dựng một full-text index mới yêu cầu một bản sao đầy đủ của toàn bộ database. Việc chỉ áp dụng một log của các thay đổi gần đây sẽ không đủ, vì nó sẽ thiếu các mục không được cập nhật gần đây. Do đó, nếu bạn không có toàn bộ lịch sử log, bạn cần bắt đầu với một snapshot nhất quán, như đã thảo luận trong mục “Setting Up New Followers”.

Snapshot của database phải tương ứng với một vị trí hoặc offset đã biết trong change log, để bạn biết tại thời điểm nào cần bắt đầu áp dụng các thay đổi sau khi snapshot đã được xử lý. Một số công cụ CDC tích hợp tính năng snapshot này, trong khi những công cụ khác để nó là một thao tác thủ công. Debezium sử dụng thuật toán DBLog watermarking của Netflix để cung cấp các incremental snapshots [30, 31].

Log compaction

Nếu bạn chỉ có thể lưu giữ một lượng lịch sử log hạn chế, bạn sẽ cần phải thực hiện quy trình snapshot mỗi khi muốn thêm một hệ thống dữ liệu phái sinh mới. Tuy nhiên, *log compaction* cung cấp một giải pháp thay thế tốt.

Chúng ta đã thảo luận về log compaction trước đó trong mục “Log-Structured Storage”, trong ngữ cảnh của các log-structured storage engine (xem Hình 4-3 để biết ví dụ). Nguyên tắc rất đơn giản: storage engine định kỳ tìm kiếm các log record có cùng key, loại bỏ bất kỳ bản trùng lặp nào và chỉ giữ lại bản update gần nhất cho mỗi key. Điều này có thể làm cho các log segment nhỏ hơn nhiều, vì vậy các segment cũng có thể được merge như một phần của quy trình compaction, như được hiển thị trong Hình 12-6. Quy trình này chạy ngầm.

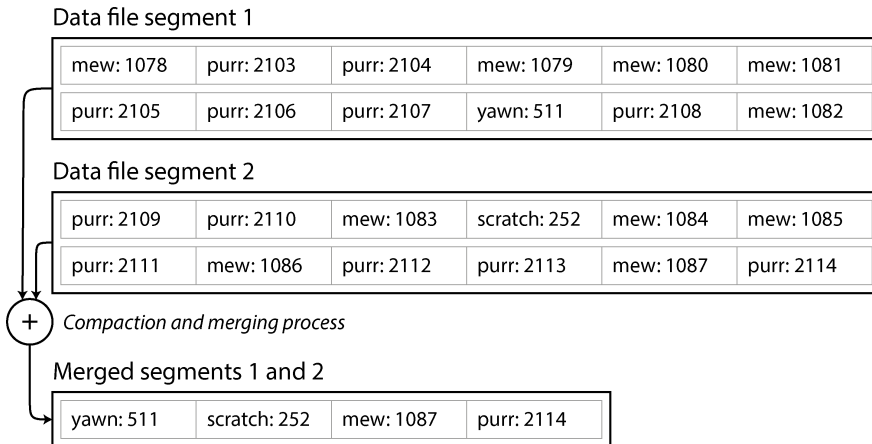


Figure 12-6. Trong log chứa các cặp key-value này, key là ID của một video mèo (mew, purr, scratch, hoặc yawn) và value là số lần video đó đã được phát; log compaction chỉ giữ lại giá trị gần nhất cho mỗi key.

Trong một storage engine cấu trúc dạng log (log-structured), một bản cập nhật với một giá trị null đặc biệt (một *tombstone*) cho biết rằng một key đã bị xóa và khiến nó bị loại bỏ trong quá trình log compaction. Nhưng chừng nào một key chưa bị ghi đè hoặc bị xóa, nó sẽ tồn tại trong log mãi mãi. Không gian đĩa yêu cầu cho một log đã được compaction như vậy chỉ phụ thuộc vào nội dung hiện tại của cơ sở dữ liệu, chứ không phụ thuộc vào số lượng các lần ghi đã từng xảy ra trong cơ sở dữ liệu. Nếu cùng một key thường xuyên bị ghi đè, các giá trị trước đó cuối cùng sẽ được garbage-collected, và chỉ có giá trị mới nhất được giữ lại.

Ý tưởng tương tự cũng hoạt động trong bối cảnh các message broker dựa trên log và CDC. Nếu hệ thống CDC được thiết lập sao cho mọi thay đổi đều có một primary key, và mọi bản cập nhật cho một key đều thay thế giá trị trước đó của key đó, thì chỉ cần giữ lại bản ghi mới nhất cho một key cụ thể là đủ.

Bây giờ, bất cứ khi nào bạn muốn xây dựng lại một hệ thống dữ liệu phái sinh (derived data) như một search index, bạn có thể bắt đầu một consumer mới từ offset 0 của log-compacted topic và quét tuần tự qua tất cả các message trong log. Log được đảm bảo sẽ chứa giá trị mới nhất cho mọi key trong cơ sở dữ liệu (và có thể là một số giá trị cũ hơn). Nói cách khác, bạn có thể sử dụng nó để lấy một bản sao đầy đủ nội dung của cơ sở dữ liệu mà không cần phải thực hiện một snapshot khác của cơ sở dữ liệu nguồn CDC.

Tính năng log compaction này được hỗ trợ bởi Apache Kafka. Như chúng ta sẽ thấy ở phần sau của chương này, nó cho phép message broker được sử dụng để lưu trữ bền vững (durable storage), chứ không chỉ dành cho việc truyền tin tạm thời (transient messaging).

Hỗ trợ API cho change streams

Hầu hết các cơ sở dữ liệu phổ biến hiện nay đều cung cấp change streams như một interface hạng nhất (first-class interface), thay vì là những nỗ lực CDC được lắp ghép thêm và reverse-engineered như trong quá khứ. Các cơ sở dữ liệu quan hệ như MySQL và PostgreSQL thường gửi các thay đổi thông qua chính replication log mà chúng sử dụng cho các replica của chính mình. Hầu hết các nhà cung cấp cloud cũng cung cấp các giải pháp CDC cho sản phẩm của họ—ví dụ, Datastream cung cấp khả năng truy cập dữ liệu streaming cho các cơ sở dữ liệu quan hệ và data warehouse của Google Cloud.

Ngay cả các cơ sở dữ liệu dựa trên quorum và có tính nhất quán cuối cùng (eventually consistent) như Cassandra hiện cũng hỗ trợ CDC. Như chúng ta đã thấy trong mục “Implementing Linearizable Systems”, các client phải lưu các bản ghi (writes) vào đa số các node trước khi chúng được coi là có thể nhìn thấy (visible). Việc hỗ trợ CDC cho quorum writes là một thách thức vì không có một single source of truth duy nhất để đăng ký (subscribe). Việc dữ liệu có hiển thị hay không phụ thuộc vào sở thích về tính nhất quán của mỗi reader. Cassandra né tránh vấn đề này bằng cách cung cấp các phân đoạn log thô (raw log segments) cho mỗi node thay vì cung cấp một stream các mutation duy nhất. Các hệ thống muốn tiêu thụ dữ liệu phải đọc các phân đoạn log thô của từng node và quyết định cách tốt nhất để hợp nhất chúng thành một stream duy nhất (giống như cách một quorum reader thực hiện) [32].

Kafka Connect [33] tích hợp các công cụ CDC cho nhiều hệ thống cơ sở dữ liệu khác nhau với Kafka. Một khi stream các sự kiện thay đổi đã nằm trong Kafka, nó có thể được sử dụng để cập nhật các hệ thống dữ liệu phái sinh như search indexes cũng như cung cấp dữ liệu cho các hệ thống stream processing, như sẽ được thảo luận ở phần sau của chương này.

CDC so với event sourcing

CDC so với event sourcing thì như thế nào? Tương tự như CDC, event sourcing bao gồm việc lưu trữ tất cả các thay đổi đối với trạng thái ứng dụng dưới dạng một log của các sự kiện thay đổi. Sự khác biệt lớn nhất là event sourcing áp dụng ý tưởng này ở một mức độ trừu tượng khác:

- Trong CDC, ứng dụng sử dụng cơ sở dữ liệu theo cách có thể thay đổi (mutable), cập nhật và xóa các bản ghi tùy ý. Log của các thay đổi được trích xuất từ cơ sở dữ liệu ở mức thấp (ví dụ, bằng cách phân tích replication log), điều này đảm bảo rằng thứ tự các bản ghi được trích xuất từ cơ sở dữ liệu khớp với thứ tự mà chúng thực sự được ghi vào, giúp tránh race condition trong Hình 12-4.
- Trong event sourcing, logic ứng dụng được xây dựng một cách rõ ràng dựa trên các event bất biến được ghi vào một event log. Trong trường hợp này, event store

là append-only (chỉ cho phép ghi thêm), và việc cập nhật hoặc xóa các event được khuyến cáo không nên thực hiện hoặc bị cấm hoàn toàn. Các event được thiết kế để phản ánh những điều đã xảy ra ở cấp độ ứng dụng thay vì các thay đổi trạng thái ở cấp độ thấp.

Lựa chọn nào tốt hơn tùy thuộc vào tình huống của bạn. Áp dụng event sourcing là một thay đổi lớn đối với một ứng dụng vốn chưa thực hiện điều đó; nó có một số ưu và nhược điểm mà chúng ta đã thảo luận trong mục “Event Sourcing and CQRS”. Ngược lại, CDC có thể được thêm vào một cơ sở dữ liệu hiện có với những thay đổi tối thiểu; ứng dụng đang ghi vào cơ sở dữ liệu thậm chí có thể không biết rằng CDC đang diễn ra.

Change Data Capture và Database Schemas

Mặc dù CDC có vẻ dễ áp dụng hơn event sourcing, nhưng nó cũng đi kèm với những thách thức riêng. Trong một kiến trúc microservices, một cơ sở dữ liệu thường chỉ được truy cập từ một service duy nhất. Các service khác tương tác với nó thông qua public API của service đó, nhưng chúng thường không truy cập trực tiếp vào cơ sở dữ liệu. Điều này biến cơ sở dữ liệu thành một chi tiết triển khai nội bộ của service, cho phép các lập trình viên thay đổi schema của nó mà không ảnh hưởng đến public API.

Tuy nhiên, các hệ thống CDC thường sử dụng schema của cơ sở dữ liệu thượng nguồn khi nhân bản dữ liệu của nó, điều này biến các schema này thành các public API phải được quản lý giống như public API của service. Việc xóa một cột trong bảng cơ sở dữ liệu sẽ làm hỏng các consumer hạ nguồn vốn phụ thuộc vào field đó. Những thách thức như vậy luôn tồn tại với các pipeline dữ liệu, nhưng chúng thường chỉ ảnh hưởng đến ETL của data warehouse. Vì CDC thường được triển khai như một data stream, các service production khác có thể là các consumer. Việc làm hỏng các consumer như vậy có thể gây ra sự cố gián đoạn đối với khách hàng [34]. Data contracts thường được sử dụng để ngăn chặn những sự cố đứt gãy này.

Một cách phổ biến để decouple schema nội bộ khỏi schema bên ngoài là sử dụng *outbox pattern*. Outbox là các bảng với schema riêng, chúng được phơi bày ra cho hệ thống CDC thay vì domain model nội bộ trong cơ sở dữ liệu [35, 36]. Sau đó, các lập trình viên có thể sửa đổi schema nội bộ theo ý muốn trong khi vẫn giữ nguyên các bảng outbox. Điều này trông có vẻ giống như một thao tác dual write—thực tế đúng là như vậy. Tuy nhiên, outbox tránh được các thách thức mà chúng ta đã thảo luận trong mục “Keeping Systems in Sync” bằng cách giữ cả hai thao tác ghi trong cùng một hệ thống (cơ sở dữ liệu). Thiết kế này cho phép cả hai thao tác ghi xuất hiện trong cùng một transaction.

Tuy nhiên, outbox cũng đưa ra một vài đánh đổi (trade-off). Các lập trình viên vẫn phải duy trì việc chuyển đổi giữa schema nội bộ và schema outbox, điều này có thể đầy thách thức. Một outbox cũng làm tăng lượng dữ liệu mà cơ sở dữ liệu phải ghi vào storage nền tảng của nó, điều này có thể gây ra các vấn đề về hiệu năng.

Tương tự như CDC, việc phát lại (replaying) event log cho phép bạn tái cấu trúc trạng thái hiện tại của hệ thống. Tuy nhiên, log compaction cần được xử lý theo cách khác:

- Một event CDC cho việc cập nhật một bản ghi thường chứa toàn bộ phiên bản mới của bản ghi đó, vì vậy giá trị hiện tại cho một primary key được xác định hoàn toàn bởi event gần nhất của primary key đó, và log compaction có thể loại bỏ các event trước đó của cùng một key.
- Mặt khác, với event sourcing, các event được mô hình hóa ở cấp độ cao hơn. Một event thường thể hiện ý định của một hành động từ người dùng, chứ không phải cơ chế của việc cập nhật trạng thái xảy ra do kết quả của hành động đó. Trong trường hợp này, các event sau thường không ghi đè lên các event trước, vì vậy bạn cần toàn bộ lịch sử các event để tái cấu trúc trạng thái cuối cùng. Log compaction không thể thực hiện theo cách tương tự.

Các ứng dụng sử dụng event sourcing thường có một cơ chế để lưu trữ các snapshot của trạng thái hiện tại được phái sinh từ log của các event, nhờ đó chúng không cần phải xử lý lại toàn bộ log một cách lặp đi lặp lại. Tuy nhiên, đây chỉ là một sự tối ưu hóa hiệu năng để tăng tốc việc đọc và phục hồi sau khi gặp sự cố; mục đích cốt lõi là hệ thống có khả năng lưu trữ tất cả các event thô mãi mãi và xử lý lại toàn bộ event log bất cứ khi nào cần thiết. Chúng ta sẽ thảo luận về giả định này trong mục “Limitations of immutability”.

Trạng thái, Streams và Immutability

Trong Chương 11, chúng ta đã thấy rằng batch processing được hưởng lợi từ tính immutability của các tệp dữ liệu đầu vào, vì vậy bạn có thể chạy các công việc xử lý thử nghiệm trên các tệp dữ liệu đầu vào hiện có mà không lo làm hỏng chúng. Nguyên tắc immutability này cũng chính là điều khiến event sourcing và CDC trở nên mạnh mẽ đến vậy.

Chúng ta thường nghĩ về các cơ sở dữ liệu như là nơi lưu trữ trạng thái hiện tại của ứng dụng. Cách biểu diễn này được tối ưu hóa cho việc đọc, và thường là cách thuận tiện nhất để phục vụ các truy vấn. Bản chất của trạng thái là nó thay đổi, vì vậy các cơ sở dữ liệu hỗ trợ việc cập nhật và xóa dữ liệu cũng như chèn dữ liệu mới. Điều này khớp với tính immutability như thế nào?

Bất cứ khi nào bạn có một trạng thái thay đổi, trạng thái đó chính là kết quả của các event đã làm biến đổi nó theo thời gian. Ví dụ, danh sách các ghế hiện đang trống là kết quả của các yêu cầu đặt chỗ mà bạn đã xử lý, số dư tài khoản hiện tại là kết quả của các khoản ghi có và ghi nợ trong tài khoản, và biểu đồ response time của web server là sự tổng hợp từ các response time riêng lẻ của tất cả các web request đã xảy ra.

Bất kể trạng thái thay đổi như thế nào, luôn luôn có một chuỗi các event đã gây ra những thay đổi đó. Ngay cả khi các việc được thực hiện rồi lại được hoàn tác, sự thật vẫn là những event đó đã xảy ra. Ý tưởng then chốt là mutable state và một append-only log của các immutable events không hề mâu thuẫn với nhau; chúng là hai mặt của cùng một đồng xu. Log của tất cả các thay đổi, hay còn gọi là *changelog*, đại diện cho sự tiến hóa của trạng thái theo thời gian.

Nếu bạn có thiên hướng về toán học, bạn có thể nói rằng trạng thái ứng dụng là những gì bạn nhận được khi bạn tích tích phân của một event stream theo thời gian, và một change stream là những gì bạn nhận được khi bạn tính đạo hàm của trạng thái theo thời gian, như được trình bày trong Hình 12-7 [37, 38]. Sự tương đồng này có những hạn chế (ví dụ, đạo hàm bậc hai của trạng thái dường như không có ý nghĩa gì), nhưng nó là một điểm khởi đầu hữu ích để tư duy về dữ liệu.

$$state(now) = \int_{t=0}^{now} stream(t) dt \qquad stream(t) = \frac{d\ state(t)}{dt}$$

Figure 12-7. *Mối quan hệ giữa trạng thái hiện tại của ứng dụng và một event stream*

Nếu bạn lưu trữ changelog một cách bền vững, điều đó đơn giản mang lại hiệu quả là làm cho trạng thái có thể tái lập được. Nếu bạn coi log của các sự kiện là system of record (hệ thống bản ghi gốc) của mình và bất kỳ trạng thái có thể thay đổi nào đều là derived data (dữ liệu phái sinh) từ đó, việc suy luận về luồng dữ liệu đi qua một hệ thống sẽ trở nên dễ dàng hơn. Như Jim Gray và Andreas Reuter đã phát biểu vào năm 1992 [39]:

Không có nhu cầu cơ bản nào về việc phải duy trì một cơ sở dữ liệu; log đã chứa đựng tất cả thông tin hiện có. Lý do duy nhất để lưu trữ cơ sở dữ liệu (tức là phần cuối cùng hiện tại của log) là để phục vụ hiệu năng của các thao tác truy vấn.

Log compaction là một cách để thu hẹp sự khác biệt giữa trạng thái của log và trạng thái của cơ sở dữ liệu. Việc compaction chỉ giữ lại phiên bản mới nhất của mỗi bản ghi và loại bỏ các phiên bản đã bị ghi đè.

Lợi ích của các immutable event

Tính bất biến (immutability) trong cơ sở dữ liệu là một ý tưởng cũ. Ví dụ, các kế toán viên đã sử dụng tính bất biến trong nhiều thế kỷ để ghi chép sổ sách tài chính. Khi một transaction xảy ra, nó được ghi lại trong một *ledger* (sổ cái) chỉ cho phép ghi thêm (append-only), về cơ bản là một log của các sự kiện mô tả tiền bạc, hàng hóa hoặc dịch vụ đã được chuyển giao. Các tài khoản, chẳng hạn như báo cáo lãi lỗ hoặc

bảng cân đối kế toán, được phái sinh từ các transaction trong sổ cái bằng cách cộng chúng lại [40].

Nếu có sai sót xảy ra, các kế toán viên không xóa hoặc thay đổi transaction sai trong sổ cái. Thay vào đó, họ thêm một transaction khác để bù đắp cho sai sót đó—ví dụ, hoàn lại một khoản phí bị tính sai. Transaction sai vẫn tồn tại trong sổ cái mãi mãi, vì nó có thể quan trọng cho các lý do kiểm toán. Nếu các con số sai được phái sinh từ sổ cái sai đã được công bố, thì các con số cho kỳ kế toán tiếp theo sẽ bao gồm một khoản điều chỉnh. Quy trình này hoàn toàn bình thường trong kế toán [41].

Mặc dù khả năng kiểm toán như vậy đặc biệt quan trọng trong các hệ thống tài chính, nó cũng có lợi cho nhiều hệ thống khác không phải chịu sự điều tiết nghiêm ngặt như vậy. Nếu bạn vô tình deploy một đoạn code lỗi ghi dữ liệu xấu vào cơ sở dữ liệu, việc khôi phục sẽ khó khăn hơn nhiều nếu đoạn code đó có khả năng ghi đè dữ liệu một cách hủy diệt. Với một append-only log chứa các immutable event, việc chẩn đoán điều gì đã xảy ra và khôi phục từ sự cố sẽ dễ dàng hơn nhiều. Tương tự, bộ phận chăm sóc khách hàng có thể sử dụng một audit log để chẩn đoán các yêu cầu và khiếu nại của khách hàng.

Các immutable event cũng ghi lại nhiều thông tin hơn là chỉ trạng thái hiện tại. Ví dụ, trên một trang web mua sắm, một khách hàng có thể thêm một mặt hàng vào giỏ hàng và sau đó lại xóa nó đi. Mặc dù sự kiện thứ hai sẽ triệt tiêu sự kiện thứ nhất dưới góc độ hoàn tất đơn hàng, nhưng việc biết được rằng khách hàng đã cân nhắc một mặt hàng cụ thể nhưng sau đó lại quyết định không mua có thể hữu ích cho mục đích phân tích. Có lẽ họ sẽ chọn mua nó trong tương lai, hoặc có lẽ họ đã tìm thấy một sản phẩm thay thế. Thông tin này được ghi lại trong một event log, nhưng nó sẽ bị mất trong một cơ sở dữ liệu vốn xóa các mặt hàng khi chúng bị loại bỏ khỏi giỏ hàng.

Phái sinh nhiều view từ cùng một event log

Bằng cách tách biệt trạng thái có thể thay đổi khỏi immutable event log, bạn có thể phái sinh nhiều biểu diễn hướng đọc (read-oriented) từ cùng một log của các sự kiện. Điều này hoạt động giống như việc có nhiều consumer của một stream (Hình 12-5)—ví dụ, cơ sở dữ liệu phân tích Druid nạp dữ liệu trực tiếp từ Kafka bằng cách sử dụng cách tiếp cận này, và các Kafka Connect sink có thể xuất dữ liệu từ Kafka sang các cơ sở dữ liệu và index khác nhau [33].

Việc có một bước chuyển đổi (translation) rõ ràng từ event log sang cơ sở dữ liệu giúp việc phát triển ứng dụng của bạn theo thời gian trở nên dễ dàng hơn. Nếu bạn muốn giới thiệu một feature mới hiển thị dữ liệu hiện có theo một cách mới, bạn có thể sử dụng event log để xây dựng một view riêng được tối ưu hóa cho việc đọc cho feature mới đó và chạy nó song song với các hệ thống hiện có mà không cần phải sửa đổi chúng. Chạy song song hệ thống cũ và mới thường dễ dàng hơn so với việc thực hiện một cuộc di chuyển schema (schema migration) phức tạp trong một hệ thống

hiện có. Một khi các bên đọc đã chuyển sang hệ thống mới và hệ thống cũ không còn cần thiết nữa, bạn chỉ đơn giản là tắt nó đi và thu hồi tài nguyên của nó [42, 43].

Chúng ta đã gặp ý tưởng về việc ghi dữ liệu dưới một dạng tối ưu hóa việc ghi (write-optimized), sau đó dịch nó sang các biểu diễn khác nhau được tối ưu hóa việc đọc (read-optimized) khi cần thiết trong phần “Event Sourcing và CQRS”. Quá trình này không nhất thiết đòi hỏi event sourcing; bạn cũng có thể xây dựng nhiều materialized view từ một luồng các sự kiện CDC [44].

Cách tiếp cận truyền thống đối với thiết kế cơ sở dữ liệu và schema dựa trên một quan niệm sai lầm rằng dữ liệu phải được ghi dưới cùng một dạng với dạng mà nó sẽ được truy vấn. Các cuộc tranh luận về normalization và denormalization (xem “Normalization, Denormalization, và Joins”) trở nên phần lớn không còn quan trọng nếu bạn có thể dịch dữ liệu từ một event log tối ưu hóa việc ghi sang trạng thái ứng dụng tối ưu hóa việc đọc. Việc denormalize dữ liệu trong các view tối ưu hóa việc đọc là hoàn toàn hợp lý, vì quá trình dịch cho bạn một cơ chế để giữ cho nó nhất quán với event log.

Trong “Case Study: Social Network Home Timelines”, chúng ta đã thảo luận về home timelines của một mạng xã hội, một cache chứa các bài đăng gần đây của những người mà một người dùng cụ thể đang theo dõi (giống như một hộp thư). Đây là một ví dụ khác về trạng thái tối ưu hóa việc đọc: các home timelines được denormalize rất nhiều, vì các bài đăng của bạn được nhân bản trong tất cả các timeline của những người đang theo dõi bạn. Tuy nhiên, dịch vụ fan-out giữ cho trạng thái được nhân bản này đồng bộ với các bài đăng mới và các mối quan hệ theo dõi mới, giúp việc nhân bản này có thể kiểm soát được.

Kiểm soát concurrency

Nhược điểm lớn nhất của CQRS là các consumer của event log thường là bất đồng bộ, vì vậy một người dùng có thể thực hiện một lệnh ghi vào log, sau đó đọc từ một view phái sinh và thấy rằng lệnh ghi của họ vẫn chưa được phản ánh trong view đó. Chúng ta đã thảo luận về vấn đề này và các giải pháp tiềm năng trong phần “Reading your own writes”.

Một giải pháp là thực hiện các cập nhật của read view một cách đồng bộ với việc append sự kiện vào log. Điều này đòi hỏi hoặc là một distributed transaction giữa event log và view phái sinh, hoặc một cách nào đó để chờ cho đến khi một sự kiện được phản ánh trong view. Cả hai cách tiếp cận này thường không khả thi trong thực tế, vì vậy các view thường được cập nhật một cách bất đồng bộ.

Mặt khác, việc phái sinh trạng thái hiện tại từ một event log cũng đơn giản hóa một số khía cạnh của kiểm soát concurrency. Phần lớn nhu cầu về multi-object transactions (xem “Single-Object và Multi-Object Operations”) bắt nguồn từ việc một hành động duy nhất của người dùng yêu cầu dữ liệu phải được thay đổi ở nhiều

nơi. Với event sourcing, bạn có thể thiết kế một sự kiện sao cho nó là một mô tả độc lập về một hành động của người dùng. Hành động của người dùng khi đó chỉ yêu cầu một lệnh ghi duy nhất tại một nơi—cụ thể là append sự kiện vào log—điều này rất dễ để thực hiện tính atomic.

Nếu event log và trạng thái ứng dụng được sharding theo cùng một cách (ví dụ: xử lý một sự kiện cho một khách hàng trong shard 3 chỉ yêu cầu cập nhật shard 3 của trạng thái ứng dụng), thì một log consumer đơn luồng (single-threaded) trực tiếp sẽ không cần kiểm soát concurrency cho các lệnh ghi. Theo cấu trúc, nó chỉ xử lý một sự kiện duy nhất tại một thời điểm (xem “Actual Serial Execution”). Log loại bỏ tính không xác định (nondeterminism) của concurrency bằng cách xác định một thứ tự tuần tự của các sự kiện trong một shard [27]. Nếu một sự kiện tác động đến nhiều state shards, sẽ cần thêm một chút công việc, điều mà chúng ta sẽ thảo luận trong Chương 13.

Nhiều hệ thống không sử dụng mô hình event-sourced nhưng vẫn dựa vào tính bất biến (immutability) để kiểm soát concurrency. Các cơ sở dữ liệu khác nhau sử dụng các cấu trúc dữ liệu bất biến hoặc dữ liệu đa phiên bản (multiversion data) ở bên trong để hỗ trợ các snapshot tại một thời điểm (xem “Indexes và snapshot isolation”). Các hệ thống quản lý phiên bản như Git, Mercurial và Fossil cũng dựa vào dữ liệu bất biến để bảo tồn lịch sử phiên bản của các tệp.

Hạn chế của tính bất biến

Việc duy trì một lịch sử bất biến (immutable) cho tất cả các thay đổi mãi mãi là khả thi đến mức nào? Câu trả lời phụ thuộc vào mức độ biến động (churn) trong dataset. Một số khối lượng công việc chủ yếu là thêm dữ liệu và hiếm khi cập nhật hoặc xóa; chúng rất dễ để thực hiện tính bất biến. Các khối lượng công việc khác lại có tỷ lệ cập nhật và xóa cao trên một dataset tương đối nhỏ; trong những trường hợp này, lịch sử bất biến có thể tăng lên quá lớn, vấn đề phân mảnh (fragmentation) có thể phát sinh, và hiệu năng của compaction cũng như garbage collection trở nên cực kỳ quan trọng đối với độ tin cậy trong vận hành [45, 46].

Bên cạnh các lý do về hiệu năng, bạn cũng có thể cần xóa dữ liệu vì các lý do hành chính hoặc pháp lý, bất chấp tất cả các tính chất bất biến. Ví dụ, các quy định về quyền riêng tư như GDPR yêu cầu thông tin cá nhân của người dùng phải được xóa và các thông tin sai lệch phải được loại bỏ theo yêu cầu, hoặc một sự rò rỉ thông tin nhạy cảm do vô ý có thể cần được ngăn chặn.

Trong những trường hợp này, chỉ việc append thêm một event khác vào log để chỉ ra rằng dữ liệu trước đó nên được coi là đã xóa là không đủ—thực tế là bạn muốn viết lại lịch sử và giả vờ như dữ liệu đó chưa từng được ghi ngay từ đầu. Ví dụ, Datomic gọi tính năng này là *excision* [47], và hệ thống quản lý phiên bản Fossil có một khái niệm tương tự gọi là *shunning* [48].

Việc xóa dữ liệu một cách thực sự khó khăn một cách đáng ngạc nhiên [49], vì các bản sao có thể tồn tại ở nhiều nơi. Các storage engine, filesystem và SSD thường ghi vào một vị trí mới thay vì ghi đè dữ liệu tại chỗ [41], và các bản backup thường được cố ý để ở trạng thái bất biến nhằm ngăn chặn việc xóa hoặc làm hỏng dữ liệu do vô ý.

Một cách để cho phép xóa dữ liệu bất biến là *crypto-shredding* [50]. Dữ liệu mà bạn có thể muốn xóa trong tương lai sẽ được lưu trữ dưới dạng mã hóa, và khi bạn muốn loại bỏ nó, bạn chỉ cần quên đi khóa mã hóa. Dữ liệu đã mã hóa sau đó vẫn nằm đó, nhưng không ai có thể sử dụng được nó.

Theo một nghĩa nào đó, điều này chỉ là chuyển vấn đề từ chỗ này sang chỗ khác; dữ liệu thực tế vẫn là bất biến, nhưng việc lưu trữ khóa của bạn là có thể thay đổi (mutable). Hơn nữa, bạn phải quyết định ngay từ đầu dữ liệu nào sẽ được mã hóa bằng cùng một khóa, và khi nào bạn sẽ sử dụng các khóa khác nhau—đây là một quyết định quan trọng, vì sau đó bạn có thể thực hiện crypto-shred toàn bộ hoặc không có dữ liệu nào được mã hóa bằng một khóa cụ thể, chứ không thể chỉ xóa một phần. Việc lưu trữ một khóa riêng biệt cho mỗi mục dữ liệu duy nhất sẽ trở nên quá cồng kềnh, vì việc lưu trữ khóa sẽ lớn tương đương với việc lưu trữ dữ liệu chính. Các cơ chế tinh vi hơn, chẳng hạn như puncturable encryption [51], cho phép thu hồi có chọn lọc khả năng giải mã của một khóa, nhưng chúng vẫn chưa được sử dụng rộng rãi.

Nhìn chung, việc xóa dữ liệu thiên về vấn đề "làm cho việc truy xuất dữ liệu trở nên khó khăn hơn" hơn là thực sự "làm cho việc truy xuất dữ liệu trở nên bất khả thi". Tuy nhiên, đôi khi bạn vẫn phải cố gắng thực hiện, như chúng ta sẽ thấy trong mục "Legislation and Self-Regulation".

Xử lý các stream

Cho đến nay trong chương này, chúng ta đã thảo luận về việc các stream đến từ đâu (các event hoạt động của người dùng, cảm biến, và các lệnh ghi vào database) và các stream được vận chuyển như thế nào (thông qua nhắn tin trực tiếp, thông qua các message broker, và trong các event log).

Điều còn lại là thảo luận về những gì bạn có thể làm với stream một khi đã có nó—cụ thể là, bạn có thể *process* (xử lý) nó. Nhìn chung, bạn có ba lựa chọn:

1. Bạn có thể lấy dữ liệu trong các event và ghi nó vào một database, cache, search index, hoặc hệ thống lưu trữ tương tự, từ đó nó có thể được truy vấn bởi các client khác. Như được trình bày trong Hình 12-5, đây là một cách tốt để giữ cho một database đồng bộ với các thay đổi đang diễn ra ở các phần khác của hệ thống—đặc biệt nếu stream consumer là client duy nhất ghi vào database. Ghi vào một hệ thống lưu trữ là sự tương đương trong streaming của những gì chúng ta đã thảo luận trong mục "Batch Use Cases".

2. Bạn có thể đẩy các event tới người dùng theo một cách nào đó—ví dụ, bằng cách gửi cảnh báo qua email hoặc thông báo đẩy (push notification), hoặc bằng cách stream các event tới một dashboard thời gian thực nơi chúng được trực quan hóa. Trong trường hợp này, con người là consumer cuối cùng của stream.
3. Bạn có thể xử lý một hoặc nhiều stream đầu vào để tạo ra một hoặc nhiều stream đầu ra. Các stream có thể đi qua một pipeline bao gồm nhiều giai đoạn xử lý như vậy trước khi chúng cuối cùng kết thúc tại một đầu ra (lựa chọn 1 hoặc 2).

Trong phần còn lại của chương này, chúng ta sẽ thảo luận về lựa chọn 3: xử lý các stream để tạo ra các derived data (dữ liệu phái sinh) khác. Một đoạn mã xử lý các stream như thế này được gọi là một *operator* hoặc một *job*. Nó liên quan chặt chẽ đến các Unix processes và các MapReduce jobs mà chúng ta đã thảo luận trong Chương 11, và pattern của dataflow cũng tương tự: một stream processor tiêu thụ các stream đầu vào theo cách chỉ đọc (read-only) và ghi đầu ra của nó vào một vị trí khác theo cách chỉ chèn thêm (append-only).

Các pattern cho sharding và parallelization trong các stream processor cũng rất giống với các pattern trong MapReduce và các dataflow engines mà chúng ta đã thấy trong Chương 11, vì vậy chúng ta sẽ không lặp lại các chủ đề đó ở đây. Các thao tác mapping cơ bản như biến đổi và lọc các bản ghi cũng hoạt động tương tự.

Điểm khác biệt quan trọng duy nhất so với các batch job là một stream không bao giờ kết thúc. Sự khác biệt này dẫn đến nhiều hệ quả. Như đã thảo luận ở đầu chương này, việc sắp xếp không có ý nghĩa với một dataset không giới hạn (unbounded), vì vậy các sort-merge joins (xem mục “Joins and Grouping”) không thể được sử dụng. Các cơ chế fault-tolerance cũng phải thay đổi. Với một batch job đã chạy trong vài phút, một task bị lỗi có thể đơn giản là được khởi động lại từ đầu, nhưng với một stream job đã chạy trong vài năm, việc khởi động lại từ đầu sau khi gặp sự cố có thể không phải là một lựa chọn khả thi.

Các ứng dụng của Stream Processing

Stream processing từ lâu đã được sử dụng cho mục đích giám sát, nơi một tổ chức muốn được cảnh báo nếu một số sự việc nhất định xảy ra. Ví dụ:

- Các hệ thống phát hiện gian lận cần xác định xem các pattern sử dụng của một thẻ tín dụng có thay đổi bất thường hay không và chặn thẻ nếu có khả năng nó đã bị đánh cắp.
- Các hệ thống giao dịch cần kiểm tra các thay đổi về giá trên một thị trường tài chính và thực thi các giao dịch theo các quy tắc đã định sẵn.
- Các hệ thống sản xuất cần giám sát trạng thái của các máy móc trong một nhà máy và nhanh chóng xác định vấn đề nếu có sự cố xảy ra.

- Các hệ thống quân sự và tình báo cần theo dõi các hoạt động của một kẻ xâm lược tiềm năng và báo động tại bất kỳ dấu hiệu tấn công nào.

Các loại ứng dụng này yêu cầu việc khớp pattern và tương quan khá phức tạp. Tuy nhiên, các ứng dụng khác của stream processing cũng đã xuất hiện theo thời gian. Trong mục này, chúng ta sẽ so sánh và đối chiếu ngắn gọn một số ứng dụng này.

Complex event processing

Complex event processing (CEP) là một phương pháp được phát triển vào những năm 1990 để phân tích các event streams, đặc biệt hướng tới loại ứng dụng yêu cầu tìm kiếm các pattern sự kiện nhất định [52]. Tương tự như cách một biểu thức chính quy (regular expression) cho phép bạn tìm kiếm các pattern ký tự nhất định trong một chuỗi, CEP cho phép bạn chỉ định các quy tắc để tìm kiếm các pattern sự kiện nhất định trong một stream.

Các hệ thống CEP thường sử dụng một ngôn ngữ truy vấn khai báo (declarative query language) cấp cao như SQL, hoặc một GUI, để mô tả các pattern của các sự kiện cần được phát hiện. Các truy vấn này được gửi đến một processing engine, nơi tiêu thụ các stream đầu vào và duy trì một state machine bên trong để thực hiện việc khớp yêu cầu. Khi tìm thấy một sự khớp, engine sẽ phát ra một *complex event* (vì vậy mới có tên gọi này) với các chi tiết về pattern sự kiện đã được phát hiện [53].

Trong các hệ thống này, mối quan hệ giữa các truy vấn và dữ liệu bị đảo ngược so với các cơ sở dữ liệu thông thường. Thông thường, một cơ sở dữ liệu lưu trữ dữ liệu một cách bền vững và coi các truy vấn là tạm thời. Khi một truy vấn đến, cơ sở dữ liệu tìm kiếm dữ liệu khớp với truy vấn đó, và nó sẽ quên truy vấn khi đã hoàn tất. Các engine CEP đảo ngược các vai trò này. Các truy vấn được lưu trữ dài hạn; khi mỗi sự kiện đến, engine sẽ kiểm tra xem liệu nó đã thấy một pattern sự kiện khớp với bất kỳ truy vấn đang chờ (standing queries) nào của nó hay chưa [54].

Các bản triển khai của CEP bao gồm Esper, Apama, và TIBCO StreamBase. Các bộ xử lý stream phân tán như Flink và Spark Streaming cũng có hỗ trợ SQL cho các truy vấn khai báo (declarative queries) trên các stream.

Stream analytics

Stream processing cũng được sử dụng để thực hiện *analytics* trên các stream. Ranh giới giữa CEP và stream analytics khá mong manh, nhưng theo quy tắc chung, stream analytics ít tập trung vào việc phát hiện các chuỗi sự kiện cụ thể mà hướng tới việc tổng hợp (aggregations) và các chỉ số thống kê trên một khối lượng lớn các sự kiện. Các ứng dụng ví dụ bao gồm sau đây:

- Đo lường tốc độ của một loại sự kiện nhất định (tần suất xảy ra trong mỗi khoảng thời gian)

- Tính toán giá trị trung bình trượt (rolling average) của một giá trị trong một khoảng thời gian
- So sánh các thống kê hiện tại với các khoảng thời gian trước đó (ví dụ: để phát hiện các xu hướng hoặc để cảnh báo về các chỉ số cao hoặc thấp bất thường so với cùng thời điểm này vào tuần trước)

Các thống kê như vậy thường được tính toán trên các khoảng thời gian cố định—ví dụ, bạn có thể muốn biết số lượng truy vấn trung bình mỗi giây tới một service trong năm phút qua và thời gian phản hồi percentile thứ 99 của chúng trong giai đoạn đó. Việc tính trung bình trong vài phút giúp làm mượt các biến động không liên quan từ giây này sang giây khác, trong khi vẫn cung cấp cho bạn một bức tranh kịp thời về bất kỳ thay đổi nào trong các mô hình lưu lượng (traffic patterns). Khoảng thời gian mà bạn thực hiện tổng hợp được gọi là một *window*; chúng ta sẽ tìm hiểu kỹ hơn về windowing trong mục “Reasoning About Time”.

Các hệ thống stream analytics đôi khi sử dụng các thuật toán xấp xỉ, chẳng hạn như Bloom filters (mà chúng ta đã gặp trong mục “Bloom filters”) để kiểm tra thành phần tập hợp, HyperLogLog [55] để ước tính cardinality, và các thuật toán ước tính percentile khác nhau (xem mục “Computing Percentiles”). Các thuật toán xấp xỉ tạo ra kết quả xấp xỉ nhưng có ưu điểm là yêu cầu ít bộ nhớ hơn đáng kể trong bộ xử lý stream so với các thuật toán chính xác. Việc sử dụng các thuật toán xấp xỉ này đôi khi khiến mọi người tin rằng các hệ thống stream processing luôn bị mất mát (lossy) và không chính xác, nhưng điều đó là sai. Không có gì mang tính xấp xỉ vốn có trong stream processing, và việc sử dụng các thuật toán xấp xỉ chỉ đơn thuần là một sự tối ưu hóa [56].

Nhiều framework xử lý stream phân tán mã nguồn mở, chẳng hạn như Apache Storm, Spark Streaming, Flink, Samza, Apache Beam, và Kafka Streams, được thiết kế hướng tới mục tiêu analytics [57]. Các dịch vụ được cung cấp (hosted services) bao gồm Google Cloud Dataflow và Azure Stream Analytics.

Duy trì materialized views

Chúng ta đã thấy rằng một stream các thay đổi đối với một cơ sở dữ liệu có thể được sử dụng để giữ cho các hệ thống dữ liệu phái sinh (derived data), chẳng hạn như các cache, search index, và data warehouse, luôn cập nhật so với cơ sở dữ liệu nguồn. Đây là những ví dụ về việc duy trì materialized views: tạo ra một view thay thế trên một dataset để bạn có thể truy vấn nó một cách hiệu quả, và cập nhật view đó bất cứ khi nào dữ liệu nền thay đổi [37].

Tương tự, trong event sourcing, trạng thái ứng dụng được duy trì bằng cách áp dụng một log các sự kiện; ở đây, trạng thái ứng dụng cũng là một dạng materialized view. Không giống như các kịch bản stream analytics, việc chỉ xem xét các sự kiện trong một window thời gian nhất định thường là không đủ. Việc xây dựng materialized

view có khả năng yêu cầu *tất cả* các sự kiện trong một khoảng thời gian tùy ý, ngoại trừ bất kỳ sự kiện lỗi thời nào có thể bị loại bỏ bởi log compaction. Trên thực tế, bạn cần một window kéo dài tận về thời điểm bắt đầu.

Về nguyên tắc, bất kỳ bộ xử lý stream nào cũng có thể được sử dụng để duy trì materialized view, mặc dù nhu cầu duy trì các sự kiện mãi mãi đi ngược lại với các giả định của một số framework hướng tới analytics vốn chủ yếu hoạt động trên các window có thời lượng giới hạn. Kafka Streams và ksqlDB của Confluent hỗ trợ kiểu sử dụng này, dựa trên sự hỗ trợ của Kafka đối với log compaction [58].

Duy trì view tăng dần

Các cơ sở dữ liệu có vẻ rất phù hợp để duy trì materialized view; suy cho cùng, chúng được thiết kế để lưu giữ các bản sao đầy đủ của một dataset. Nhiều cơ sở dữ liệu cũng hỗ trợ materialized views. Chúng ta đã thấy trong mục “Materialized Views and Data Cubes” rằng các truy vấn phân tích điển hình của một data warehouse có thể được materialized thành các OLAP cube.

Thật không may, các cơ sở dữ liệu thường làm mới các bảng materialized view bằng cách sử dụng các batch job định kỳ hoặc các yêu cầu on-demand như `REFRESH MATERIALIZED VIEW` của PostgreSQL, chứ không phải sau mỗi lần cập nhật dữ liệu nguồn. Cách tiếp cận này có hai nhược điểm đáng kể khiến nó không phù hợp để duy trì view trong stream processing:

Hiệu năng kém

Tất cả dữ liệu đều được xử lý lại mỗi khi view được cập nhật, mặc dù hầu hết dữ liệu có khả năng cao là không thay đổi.

Độ tươi của dữ liệu

Những thay đổi trong dữ liệu nguồn không được phản ánh trong một materialized view cho đến khi truy vấn của nó được chạy lại trong lần cập nhật định kỳ tiếp theo.

Có thể viết các database trigger để cập nhật các materialized view một cách hiệu quả khi dữ liệu dễ dàng được partitioning và việc tính toán mang tính chất incremental một cách tự nhiên. Ví dụ, nếu một materialized view duy trì tổng doanh thu bán hàng mỗi ngày, thì dòng cho ngày tương ứng có thể được cập nhật mỗi khi có một giao dịch bán hàng mới xảy ra. Các giải pháp bespoke (được thiết kế riêng) hoạt động được trong một vài trường hợp, nhưng nhiều truy vấn SQL không thể được chuyển đổi sang tính toán incremental một cách dễ dàng hoặc hiệu quả.

Incremental view maintenance (IVM - duy trì view tăng dần) là một giải pháp tổng quát hơn cho các vấn đề vừa nêu. Các kỹ thuật IVM chuyển đổi các truy vấn được viết bằng SQL hoặc các ngôn ngữ khác thành các toán tử có khả năng thực hiện các tính toán incremental. Thay vì xử lý toàn bộ dataset, các thuật toán IVM chỉ tính toán lại và cập nhật những dữ liệu đã thay đổi [38, 59, 60]. Việc tính toán view khi đó trở nên hiệu quả hơn nhiều; điều này có nghĩa là các cập nhật có thể được chạy thường xuyên hơn nhiều, giúp tăng đáng kể độ tươi của dữ liệu.

Các cơ sở dữ liệu như Materialize [61], RisingWave, ClickHouse, và Feldera đều sử dụng các kỹ thuật IVM để cung cấp các materialized views tăng dần hiệu quả. Các cơ sở dữ liệu này nạp các stream của các event để hiển thị các materialized views trong thời gian thực. Các event gần đây được đệm trong bộ

nhớ và được sử dụng định kỳ để cập nhật các materialized views trên đĩa. Các thao tác đọc kết hợp các event gần đây và dữ liệu đã được materialized để cung cấp một view thời gian thực duy nhất. Vì các thao tác đọc thường được thể hiện bằng SQL và các materialized views thường được lưu trữ theo định dạng kiểu OLAP, các hệ thống này cũng hỗ trợ các truy vấn kiểu data warehouse quy mô lớn như những truy vấn đã được thảo luận trong Chương 11.

Tìm kiếm trên các stream

Bên cạnh CEP, cho phép tìm kiếm các pattern bao gồm nhiều event, đôi khi còn có nhu cầu tìm kiếm các event riêng lẻ dựa trên các tiêu chí phức tạp, chẳng hạn như các truy vấn full-text search. Ví dụ, các dịch vụ giám sát truyền thông đăng ký nhận các nguồn cấp dữ liệu bài báo và chương trình phát sóng từ các cơ quan truyền thông và tìm kiếm bất kỳ tin tức nào đề cập đến các công ty, sản phẩm hoặc chủ đề được quan tâm. Điều này được thực hiện bằng cách lập trước một truy vấn tìm kiếm, sau đó liên tục khớp stream các mục tin tức với truy vấn này. Các tính năng tương tự cũng tồn tại trên một số trang web—ví dụ, người dùng các trang bất động sản có thể yêu cầu được thông báo khi một bất động sản mới khớp với tiêu chí tìm kiếm của họ xuất hiện trên thị trường. Tính năng percolator của Elasticsearch [62] là một lựa chọn để triển khai loại tìm kiếm stream này.

Các công cụ tìm kiếm truyền thống trước tiên sẽ index các tài liệu và sau đó chạy các truy vấn trên index đó. Ngược lại, việc tìm kiếm một stream sẽ đảo ngược quá trình xử lý. Các truy vấn được lưu trữ, và các tài liệu được đánh giá dựa trên chúng, giống như trong CEP. Trong trường hợp đơn giản nhất, bạn có thể kiểm tra mọi tài liệu với mọi truy vấn, mặc dù việc này có thể chậm nếu bạn có một số lượng lớn các truy vấn. Để tối ưu hóa quá trình này, có thể index cả các truy vấn cũng như các tài liệu, và do đó thu hẹp tập hợp các truy vấn có thể khớp [63].

Kiến trúc event-driven và RPC

Trong mục “Event-Driven Architectures”, chúng ta đã thảo luận về các hệ thống message-passing như một giải pháp thay thế cho RPC. Cơ chế này để các service giao tiếp với nhau được sử dụng, ví dụ, trong actor model.

Mặc dù các hệ thống này cũng dựa trên các message và event, chúng ta thường không coi chúng là các stream processor vì một vài lý do sau đây:

- Các actor framework chủ yếu là một cơ chế để quản lý concurrency và thực thi phân tán của các module giao tiếp với nhau, trong khi stream processing chủ yếu là một kỹ thuật quản lý dữ liệu.
- Sự giao tiếp giữa các actor thường mang tính tạm thời và là một-đối-một, trong khi các event log có tính bền vững và hỗ trợ nhiều subscriber.

- Các actor có thể giao tiếp theo những cách tùy ý (bao gồm cả các pattern request/response vòng lặp), nhưng các stream processor thường được thiết lập trong các pipeline không có chu trình, nơi mỗi stream là đầu ra của một job cụ thể và được dẫn xuất từ một tập hợp các input stream đã được xác định rõ ràng.

Nói như vậy, có một sự giao thoa nhất định giữa các hệ thống giống như RPC và stream processing. Ví dụ, Apache Storm có một tính năng gọi là *distributed RPC*, cho phép các truy vấn của người dùng được phân bổ tới một tập hợp các node cũng thực hiện xử lý các event stream; các truy vấn này sau đó được đan xen với các event từ các input stream, và kết quả có thể được tổng hợp rồi gửi lại cho người dùng.

Việc xử lý các stream bằng cách sử dụng các actor framework cũng là khả thi. Tuy nhiên, nhiều framework như vậy không đảm bảo việc chuyển phát message trong trường hợp xảy ra crash, vì vậy quá trình xử lý sẽ không có fault tolerance trừ khi bạn triển khai thêm logic retry.

Suy luận về Thời gian

Các stream processor thường cần phải xử lý thời gian, đặc biệt là khi chạy các tác vụ analytics, vốn thường xuyên sử dụng các time window như "giá trị trung bình trong năm phút qua". Ý nghĩa của "năm phút qua" có vẻ không gây tranh cãi và rõ ràng, nhưng đáng tiếc là khái niệm này lại cực kỳ phức tạp.

Trong một batch process, các tác vụ xử lý nhanh chóng tính toán qua một tập hợp lớn các event lịch sử. Nếu cần thực hiện một kiểu phân loại theo thời gian, batch process cần xem xét timestamp được nhúng trong mỗi event. Việc xem xét đồng hồ hệ thống của máy đang chạy process là vô nghĩa, bởi vì thời điểm nó được chạy không liên quan gì đến thời điểm mà các event thực sự xảy ra.

Một batch process có thể đọc lượng event lịch sử của cả một năm chỉ trong vài phút; trong hầu hết các trường hợp, dòng thời gian cần quan tâm là một năm lịch sử đó, chứ không phải là vài phút xử lý. Hơn nữa, việc sử dụng các timestamp trong các event cho phép quá trình xử lý có tính deterministic: chạy lại cùng một process với cùng một input sẽ cho ra cùng một kết quả.

Mặt khác, nhiều framework stream processing sử dụng đồng hồ hệ thống cục bộ trên máy xử lý (gọi là *processing time*) để xác định windowing [64]. Cách tiếp cận này có ưu điểm là đơn giản, và nó hợp lý nếu độ trễ giữa việc tạo event và việc xử lý event là cực kỳ ngắn. Tuy nhiên, nó sẽ gặp vấn đề nếu có bất kỳ sự trễ xử lý (processing lag) đáng kể nào (tức là, nếu việc xử lý diễn ra muộn hơn đáng kể so với thời điểm event xảy ra).

Event time so với processing time

Việc xử lý có thể bị trì hoãn vì nhiều lý do, bao gồm việc xếp hàng (queueing), lỗi network, một vấn đề về hiệu năng dẫn đến tranh chấp (contention) trong message

broker hoặc processor, việc khởi động lại stream consumer, hoặc việc xử lý lại các event trong quá khứ khi đang phục hồi sau một lỗi hoặc sau khi sửa một bug trong code.

Sự chậm trễ của message cũng có thể dẫn đến thứ tự các message không thể dự đoán được. Ví dụ, giả sử một người dùng thực hiện một web request đầu tiên (được xử lý bởi web server A), và sau đó là request thứ hai (được xử lý bởi server B). A và B phát ra các event mô tả các request mà chúng đã xử lý, nhưng event của B đến message broker trước event của A. Khi đó, các stream processor sẽ thấy event B trước rồi mới đến event A, mặc dù chúng đã xảy ra theo thứ tự ngược lại.

Nếu cần một sự ví von, hãy xét các bộ phim *Star Wars*. Tập IV được phát hành vào năm 1977, Tập V vào năm 1980, và Tập VI vào năm 1983, tiếp theo là các Tập I, II, và III lần lượt vào các năm 1999, 2002, và 2005, và các Tập VII, VIII, và IX vào các năm 2015, 2017, và 2019 [65]. Nếu bạn xem các bộ phim theo thứ tự chúng được ra mắt, thứ tự mà bạn xử lý chúng sẽ không nhất quán với thứ tự của cốt truyện. (Số tập giống như timestamp của sự kiện, và ngày bạn xem phim chính là thời gian xử lý.) Là con người, chúng ta có thể đối phó với những sự gián đoạn như vậy, nhưng các thuật toán stream processing cần được viết một cách đặc biệt để thích ứng với các vấn đề về thời gian và thứ tự này.

Việc nhầm lẫn giữa event time và processing time dẫn đến dữ liệu sai lệch. Ví dụ, giả sử bạn có một stream processor đo lường tốc độ của các request (đếm số lượng request mỗi giây). Nếu bạn redeploy stream processor, nó có thể bị tắt trong một phút và sau đó xử lý lượng backlog các event khi nó hoạt động trở lại. Nếu bạn đo lường tốc độ dựa trên processing time, nó sẽ trông như thể có một sự gia tăng đột ngột và bất thường của các request trong khi đang xử lý backlog, trong khi thực tế tốc độ request thực sự vẫn ổn định (Hình 12-8).

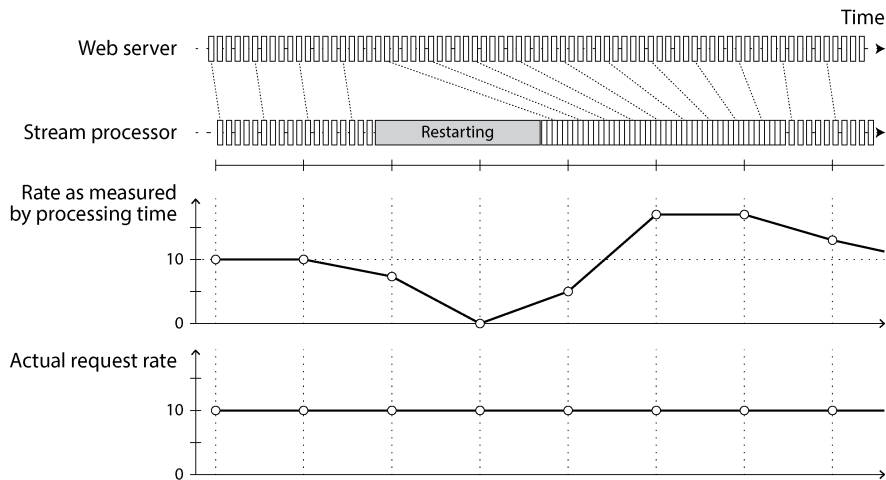


Figure 12-8. Việc thực hiện windowing theo processing time sẽ tạo ra các artifact do sự biến động của tốc độ xử lý.

Xử lý các sự kiện straggler

Một vấn đề hóc búa khi định nghĩa các window dựa trên event time là bạn không bao giờ có thể chắc chắn liệu mình đã nhận được tất cả các sự kiện cho một window cụ thể hay vẫn còn một số sự kiện khác sắp đến.

Ví dụ, giả sử bạn đang nhóm các sự kiện vào các window 1 phút để có thể đếm số lượng request mỗi phút. Bạn đã đếm được một số lượng sự kiện với timestamp rơi vào phút thứ 37 của giờ, và thời gian đã trôi qua; giờ đây hầu hết các sự kiện đang đến đều rơi vào phút thứ 38 và 39 của giờ. Khi nào bạn tuyên bố rằng mình đã hoàn tất window cho phút thứ 37 và xuất giá trị counter của nó?

Bạn có thể thực hiện timeout và tuyên bố một window đã sẵn sàng sau khi bạn không thấy bất kỳ sự kiện mới nào cho window đó trong một khoảng thời gian. Tuy nhiên, một số sự kiện có thể đang được buffer trên một máy khác ở đâu đó, bị trì hoãn do gián đoạn mạng. Bạn cần có khả năng xử lý các sự kiện *straggler* (sự kiện đến muộn) như vậy, những sự kiện đến sau khi window đã được tuyên bố là hoàn tất. Nhìn chung, bạn có hai lựa chọn [1]:

- Bỏ qua các sự kiện straggler, vì trong điều kiện bình thường, chúng có lẽ chỉ chiếm một tỷ lệ nhỏ các sự kiện. Bạn có thể theo dõi số lượng các sự kiện bị dropped như một metric và đưa ra cảnh báo nếu bạn bắt đầu làm mất một lượng dữ liệu đáng kể.
- Xuất bản một bản *correction* (sửa đổi), một giá trị đã được cập nhật cho window bao gồm cả các straggler. Bạn cũng có thể cần phải thu hồi (retract) kết quả đầu ra trước đó.

Trong một số trường hợp, có thể sử dụng một thông điệp đặc biệt để chỉ ra rằng “kể từ bây giờ, sẽ không còn thông điệp nào có timestamp sớm hơn t nữa”, thông điệp này có thể được các consumer sử dụng để kích hoạt các window [66]. Tuy nhiên, nếu nhiều producer trên các máy khác nhau đang tạo ra các sự kiện, mỗi producer đều có ngưỡng timestamp tối thiểu riêng, thì các consumer cần phải theo dõi từng producer một cách riêng biệt. Việc thêm và xóa các producer trong trường hợp này sẽ phức tạp hơn.

Dù sao thì, bạn đang sử dụng đồng hồ của ai?

Việc gán timestamp cho các sự kiện thậm chí còn khó khăn hơn khi các sự kiện có thể được buffer tại nhiều điểm trong hệ thống. Ví dụ, hãy xét một ứng dụng di động báo cáo các sự kiện về các metric sử dụng tới một máy chủ. Ứng dụng có thể được sử dụng khi thiết bị đang ngoại tuyến, trong trường hợp đó nó sẽ buffer các sự kiện cục bộ trên thiết bị và gửi chúng tới máy chủ khi có kết nối internet tiếp theo (có thể là vài giờ hoặc thậm chí vài ngày sau). Đối với bất kỳ consumer nào của stream này, các sự kiện sẽ xuất hiện như những straggler bị trễ cực độ.

Trong bối cảnh này, timestamp trên các sự kiện thực sự nên là thời điểm mà tương tác của người dùng xảy ra, dựa theo đồng hồ cục bộ của thiết bị di động. Tuy nhiên, đồng hồ trên một thiết bị do người dùng kiểm soát thường không thể tin cậy được, vì nó có thể bị đặt sai thời gian một cách vô ý hoặc cố ý (xem “Clock Synchronization and Accuracy”). Thời gian mà sự kiện được máy chủ nhận (theo đồng hồ của máy chủ) có khả năng chính xác cao hơn, vì máy chủ nằm dưới sự kiểm soát của bạn, nhưng lại ít có ý nghĩa hơn trong việc mô tả tương tác của người dùng.

Để điều chỉnh cho các đồng hồ thiết bị không chính xác, một cách tiếp cận là ghi lại ba timestamp [67]:

- Thời điểm sự kiện xảy ra, theo đồng hồ của thiết bị
- Thời điểm sự kiện được gửi tới máy chủ, theo đồng hồ của thiết bị
- Thời điểm sự kiện được máy chủ nhận, theo đồng hồ của máy chủ

Bằng cách lấy timestamp thứ hai trừ đi timestamp thứ ba, bạn có thể ước tính độ lệch (offset) giữa đồng hồ thiết bị và đồng hồ máy chủ (giả sử độ trễ mạng là không đáng kể so với độ chính xác timestamp yêu cầu). Sau đó, bạn có thể áp dụng độ lệch đó vào timestamp của sự kiện, và từ đó ước tính thời gian thực sự mà sự kiện đã xảy ra (giả sử độ lệch đồng hồ thiết bị không thay đổi giữa thời điểm sự kiện xảy ra và thời điểm nó được gửi tới máy chủ).

Vấn đề này không chỉ dành riêng cho stream processing; batch processing cũng gặp phải chính xác những vấn đề tương tự khi suy luận về thời gian. Nó chỉ để nhận thấy hơn trong bối cảnh streaming, nơi chúng ta nhận thức rõ hơn về sự trôi qua của thời gian.

Các loại window

Một khi bạn đã biết cách xác định timestamp của một event, bước tiếp theo là quyết định cách định nghĩa các window (cửa sổ) theo các khoảng thời gian. Sau đó, window có thể được sử dụng để thực hiện các phép aggregation—ví dụ, để đếm các event hoặc tính giá trị trung bình của các giá trị trong window. Có một số loại window thường được sử dụng [64, 68]:

Tumbling windows

Một tumbling window có độ dài cố định, và mỗi event thuộc về chính xác một window. Ví dụ, nếu bạn có một tumbling window một phút, tất cả các event có timestamp từ 10:03:00 đến 10:03:59 sẽ được nhóm vào một window, các event từ 10:04:00 đến 10:04:59 vào window tiếp theo, và cứ tiếp tục như vậy. Bạn có thể triển khai một tumbling window một phút bằng cách lấy timestamp của mỗi event và làm tròn xuống phút gần nhất để xác định window mà nó thuộc về.

Hopping windows

Một hopping window cũng có độ dài cố định, nhưng có sự chồng lấp giữa các window liên tiếp để cung cấp khả năng làm mượt. Ví dụ, một window năm phút với hop size là một phút sẽ chứa các event từ 10:03:00 đến 10:07:59, sau đó window tiếp theo sẽ bao phủ các event từ 10:04:00 đến 10:08:59, và cứ tiếp tục như vậy. Bạn có thể triển khai điều này bằng cách trước tiên tính toán các tumbling window một phút, sau đó thực hiện aggregation trên nhiều window liền kề.

Sliding windows

Một sliding window chứa tất cả các event xảy ra trong một khoảng thời gian nhất định với nhau. Ví dụ, một sliding window năm phút sẽ bao phủ các event tại 10:03:39 và 10:08:12, vì chúng cách nhau chưa đầy năm phút (lưu ý rằng các tumbling window và hopping window năm phút sẽ không đưa hai event này vào cùng một window, vì chúng sử dụng các ranh giới cố định). Một sliding window có thể được triển khai bằng cách duy trì một buffer các event được sắp xếp theo thời gian và loại bỏ các event cũ khi chúng hết hạn trong window.

Session windows

Khác với các loại window khác, một session window không có thời lượng cố định. Thay vào đó, nó được định nghĩa bằng cách nhóm tất cả các event của cùng một người dùng xảy ra gần nhau về mặt thời gian, và window kết thúc khi người dùng không hoạt động trong một khoảng thời gian (ví dụ, nếu không có event nào xảy ra trong 30 phút). Sessionization là một yêu cầu phổ biến đối với phân tích website.

Các thao tác window thường duy trì state tạm thời. Trong một số trường hợp, state có kích thước cố định, bất kể window lớn như thế nào hay có bao nhiêu event xảy ra—ví dụ, một thao tác đếm sẽ chỉ có một bộ đếm duy nhất bất kể kích thước window

hay số lượng event. Mặt khác, các sliding window hoặc stream joins, mà chúng ta sẽ thảo luận trong mục tiếp theo, yêu cầu các event phải được buffer cho đến khi window kết thúc. Do đó, kích thước window lớn hoặc các stream có throughput cao có thể khiến các stream processor phải duy trì rất nhiều state tạm thời. Khi đó, bạn phải lưu ý đảm bảo rằng các máy chủ chạy các tác vụ stream processing có đủ khả năng để duy trì state này, dù là trong memory hay trên disk.

Stream Joins

Trong mục “Joins and Grouping”, chúng ta đã thảo luận về cách các batch job có thể join các dataset theo key và cách các phép join đó tạo thành một phần quan trọng của các data pipeline. Vì stream processing tổng quát hóa các data pipeline thành việc xử lý tăng dần (incremental processing) các dataset không giới hạn, nên nhu cầu thực hiện các phép join trên các stream là hoàn toàn tương tự.

Tuy nhiên, thực tế là các event mới có thể xuất hiện bất cứ lúc nào trên một stream khiến cho việc join trên các stream trở nên thách thức hơn so với trong các batch job. Để hiểu rõ hơn tình huống này, hãy phân biệt giữa ba loại join: *stream-stream* join, *stream-table* join, và *table-table* join. Trong các mục tiếp theo, chúng ta sẽ minh họa từng loại bằng ví dụ.

Stream-stream join (window join)

Giả sử bạn có một tính năng tìm kiếm trên website của mình và muốn phát hiện các xu hướng gần đây trong các URL được tìm kiếm. Mỗi khi có ai đó nhập một truy vấn tìm kiếm, bạn ghi lại một event chứa truy vấn đó và các kết quả trả về. Mỗi khi có ai đó nhấp vào một trong các kết quả tìm kiếm, bạn ghi lại một event khác để ghi nhận lượt nhấp. Để tính tỷ lệ nhấp (click-through rate) cho mỗi URL trong kết quả tìm kiếm, bạn cần kết hợp các event của hành động tìm kiếm và hành động nhấp chuột, vốn được kết nối với nhau bằng cách có cùng một session ID. Các phân tích tương tự cũng cần thiết trong các hệ thống quảng cáo [69].

Lượt nhấp có thể không bao giờ xảy ra nếu người dùng từ bỏ việc tìm kiếm, và ngay cả khi nó xảy ra, khoảng thời gian giữa lúc tìm kiếm và lúc nhấp chuột có thể thay đổi rất lớn. Trong nhiều trường hợp, nó có thể chỉ là vài giây, nhưng cũng có thể kéo dài tới vài ngày hoặc vài tuần (nếu người dùng thực hiện tìm kiếm, quên mất tab trình duyệt đó, rồi sau đó quay lại tab và nhấp vào một kết quả vào một thời điểm nào đó sau đó). Do độ trễ mạng thay đổi, event nhấp chuột thậm chí có thể đến trước event tìm kiếm. Bạn có thể chọn một window phù hợp cho việc join—ví dụ, bạn có thể chọn join một lượt nhấp với một lượt tìm kiếm nếu chúng xảy ra cách nhau tối đa một giờ.

Lưu ý rằng việc embedding các chi tiết của tìm kiếm vào event nhấp chuột không tương đương với việc join các event. Làm như vậy sẽ chỉ cho bạn biết về các trường hợp người dùng đã nhấp vào một kết quả tìm kiếm, chứ không cho biết về các lượt

tìm kiếm mà người dùng không nhấp vào bất kỳ kết quả nào. Để đo lường chất lượng tìm kiếm, bạn cần tỷ lệ nhấp chính xác, điều mà đòi hỏi bạn phải có cả event tìm kiếm và event nhấp chuột.

Để triển khai loại join này, một stream processor cần duy trì state—ví dụ, tất cả các event đã xảy ra trong giờ qua, được lập chỉ mục theo session ID. Bất cứ khi nào một event tìm kiếm hoặc event nhấp chuột xảy ra, nó sẽ được thêm vào index phù hợp, và stream processor cũng kiểm tra index còn lại để xem liệu một event khác cho cùng session ID đã đến hay chưa. Nếu có một event khớp, bạn emit một event cho biết kết quả tìm kiếm nào đã được nhấp. Nếu event tìm kiếm hết hạn mà bạn không thấy event nhấp chuột nào khớp, bạn emit một event cho biết những kết quả tìm kiếm nào đã không được nhấp.

Stream-table join (làm giàu stream)

Trong mục “Joins and Grouping” (Hình 11-2), chúng ta đã thấy một ví dụ về một batch job thực hiện join hai dataset: một tập hợp các event hoạt động của người dùng và một database chứa hồ sơ người dùng. Việc coi các event hoạt động của người dùng là một stream và thực hiện cùng một phép join đó một cách liên tục trong một stream processor là điều tự nhiên. Đầu vào là một stream các event hoạt động chứa một user ID, và đầu ra là một stream các event hoạt động mà trong đó user ID đã được bổ sung thêm thông tin hồ sơ về người dùng. Quá trình này đôi khi được gọi là *enriching* (làm giàu) các event hoạt động bằng thông tin từ database.

Để thực hiện phép join này, stream process cần lấy từng event hoạt động một, tra cứu user ID của event đó trong database, và thêm thông tin hồ sơ vào event hoạt động. Việc tra cứu database có thể được triển khai bằng cách truy vấn một database từ xa; tuy nhiên, như đã thảo luận trong mục “Joins and Grouping”, các truy vấn từ xa như vậy có khả năng sẽ chậm và có nguy cơ làm quá tải database [58].

Một cách tiếp cận khác là tải một bản sao của database vào stream processor để nó có thể được truy vấn cục bộ mà không cần thực hiện một vòng truyền tải qua mạng (network round trip). Kỹ thuật này được gọi là *hash join* vì bản sao cục bộ của database có thể là một hash table trong bộ nhớ nếu nó đủ nhỏ, hoặc là một index trên đĩa cục bộ.

Điểm khác biệt so với các batch job là một batch job sử dụng một snapshot tại một thời điểm của cơ sở dữ liệu làm đầu vào, trong khi một stream processor là một tiến trình chạy lâu dài (long-running), và nội dung của cơ sở dữ liệu có khả năng thay đổi theo thời gian, vì vậy bản sao cục bộ của cơ sở dữ liệu trong stream processor cần được cập nhật liên tục. Vấn đề này có thể được giải quyết bằng CDC. Stream processor có thể đăng ký (subscribe) một changelog của cơ sở dữ liệu profile người dùng cũng như luồng các sự kiện hoạt động. Khi một profile được tạo hoặc sửa đổi, stream processor sẽ cập nhật bản sao cục bộ của nó. Do đó, chúng ta có được một phép join giữa hai luồng: các sự kiện hoạt động và các bản cập nhật profile.

Một phép stream–table join rất giống với một phép stream–stream join. Sự khác biệt lớn nhất là đối với luồng changelog của table, phép join sử dụng một window kéo dài về tận "thời điểm bắt đầu" (một window vô hạn về mặt khái niệm), với các phiên bản mới hơn của các bản ghi sẽ ghi đè lên các phiên bản cũ hơn. Đối với đầu vào là stream, phép join có thể hoàn toàn không cần duy trì một window nào.

Table–table join (duy trì materialized view)

Hãy xét ví dụ về dòng thời gian (timeline) của mạng xã hội mà chúng ta đã thảo luận trong mục “Case Study: Social Network Home Timelines”. Chúng ta đã nói rằng khi một người dùng muốn xem dòng thời gian chính của họ, việc lập qua tất cả những người mà người dùng đó đang theo dõi, tìm các bài đăng gần đây của họ và hợp nhất chúng là quá tốn kém.

Thay vào đó, chúng ta muốn có một timeline cache, một dạng “hộp thư đến” (inbox) cho mỗi người dùng mà các bài đăng sẽ được ghi vào ngay khi chúng được gửi đi, để việc đọc dòng thời gian chỉ là một lần tra cứu duy nhất. Việc tạo (materializing) và duy trì cache này yêu cầu quá trình xử lý sự kiện như sau:

- Khi người dùng u gửi một bài đăng mới, nó sẽ được thêm vào dòng thời gian của mọi người dùng đang theo dõi u .
- Khi một người dùng xóa một bài đăng, hoặc xóa toàn bộ tài khoản của họ, bài đăng đó sẽ bị xóa khỏi dòng thời gian của tất cả người dùng.
- Khi người dùng $u1$ bắt đầu theo dõi người dùng $u2$, các bài đăng gần đây của $u2$ sẽ được thêm vào dòng thời gian của $u1$.
- Khi người dùng $u1$ ngừng theo dõi người dùng $u2$, các bài đăng của $u2$ sẽ bị xóa khỏi dòng thời gian của $u1$.

Để triển khai việc duy trì cache này trong một stream processor, bạn cần các luồng sự kiện cho các bài đăng (gửi và xóa) và cho các quan hệ theo dõi (theo dõi và ngừng theo dõi). Tiến trình stream cần duy trì một cơ sở dữ liệu chứa tập hợp những người theo dõi của mỗi người dùng để nó biết những dòng thời gian nào cần được cập nhật khi một bài đăng mới đến.

Một cách nhìn khác về tiến trình stream này là nó duy trì một materialized view cho một truy vấn thực hiện join hai table (posts và follows)—đại loại như sau:

```
SELECT follows.follower_id AS timeline_id,  
       array_agg(posts.* ORDER BY posts.timestamp DESC)  
FROM posts  
JOIN follows ON follows.followee_id = posts.sender_id  
GROUP BY follows.follower_id
```

Phép join của các stream tương ứng trực tiếp với phép join của các table trong truy vấn này. Các dòng thời gian về cơ bản là một cache của kết quả truy vấn, được cập nhật mỗi khi các table cơ sở thay đổi.

LƯU Ý

Nếu bạn coi một stream là dữ liệu phái sinh (derivative) của một table, như trong Hình 12-7, và coi một phép join là tích của hai table $u \cdot v$, một điều thú vị sẽ xảy ra: luồng các thay đổi đối với materialized join tuân theo quy tắc đạo hàm $(u \cdot v)' = u'v + uv'$. Bất kỳ thay đổi nào của posts đều được join với các người theo dõi hiện tại, và bất kỳ thay đổi nào của follows đều được join với các bài đăng hiện tại [37].

Sự phụ thuộc vào thời gian của các phép join

Ba loại join được mô tả ở đây (stream-stream, stream-table, và table-table) có nhiều điểm chung. Tất cả chúng đều yêu cầu stream processor phải duy trì một state (như các sự kiện tìm kiếm và nhấp chuột, profile người dùng, hoặc danh sách người theo dõi) được phái sinh từ một đầu vào join và truy vấn state đó khi xử lý các bản ghi từ đầu vào còn lại.

Thứ tự của các sự kiện duy trì state là rất quan trọng—ví dụ, việc bạn theo dõi một người dùng trước rồi sau đó mới ngừng theo dõi, hay ngược lại, là điều quan trọng. Trong một event log được sharding như Kafka, thứ tự của các sự kiện trong cùng một shard (partition) được bảo toàn, nhưng thường không có sự đảm bảo về thứ tự giữa các stream hoặc các shard khác nhau.

Điều này đặt ra một câu hỏi: nếu các sự kiện trên các stream khác nhau xảy ra vào khoảng thời gian tương đương nhau, chúng sẽ được xử lý theo thứ tự nào? Ví dụ, trong ví dụ về stream-table join, nếu một người dùng cập nhật hồ sơ của họ, những sự kiện hoạt động nào sẽ được join với hồ sơ cũ (được xử lý trước khi cập nhật hồ sơ), và những sự kiện nào sẽ được join với hồ sơ mới (được xử lý sau khi cập nhật hồ sơ)? Nói cách khác: nếu trạng thái thay đổi theo thời gian, và bạn thực hiện join với một trạng thái, bạn sẽ sử dụng thời điểm nào để thực hiện join?

Sự phụ thuộc vào thời gian như vậy có thể xảy ra ở nhiều nơi. Ví dụ, nếu bạn bán hàng, bạn cần áp dụng đúng mức thuế cho các hóa đơn, điều này phụ thuộc vào quốc gia hoặc bang, loại sản phẩm và ngày bán (vì mức thuế thay đổi theo thời gian). Khi join các giao dịch bán hàng với một bảng chứa các mức thuế, bạn có lẽ muốn join với mức thuế tại thời điểm bán hàng, mức thuế này có thể khác với mức thuế hiện tại nếu bạn đang xử lý lại dữ liệu lịch sử.

Nếu thứ tự các sự kiện giữa các stream không được xác định, phép join sẽ trở nên nondeterministic [70], nghĩa là bạn không thể chạy lại cùng một công việc trên cùng một đầu vào mà chắc chắn nhận được cùng một kết quả. Các sự kiện trên các input stream có thể bị xen kẽ theo một cách khác khi bạn chạy lại công việc đó.

Trong các data warehouse, vấn đề này được gọi là *slowly changing dimension* (SCD - chiều thay đổi chậm), và nó thường được giải quyết bằng cách sử dụng một định danh duy nhất cho một phiên bản cụ thể của bản ghi được join—ví dụ, mỗi khi mức thuế thay đổi, nó sẽ được cấp một định danh mới, và hóa đơn sẽ bao gồm định danh của mức thuế tại thời điểm bán hàng [71, 72]. Sự thay đổi này giúp phép join trở nên deterministic, nhưng nó dẫn đến hệ quả là không thể thực hiện log compaction, vì tất cả các phiên bản của các bản ghi trong bảng đều cần được giữ lại. Ngoài ra, bạn có thể denormalize dữ liệu và đưa mức thuế áp dụng trực tiếp vào mỗi sự kiện bán hàng.

Khả năng chịu lỗi

Trong mục cuối cùng của chương này, hãy cùng xem xét cách các stream processor có thể chịu lỗi (fault tolerance). Chúng ta đã thấy trong Chương 11 rằng các framework batch processing có thể chịu lỗi khá dễ dàng: nếu một task thất bại, nó đơn giản là có thể được bắt đầu lại trên một máy khác, và đầu ra của task thất bại đó sẽ bị loại bỏ. Việc thử lại (retry) một cách minh bạch này khả thi vì các tệp đầu vào là bất biến, mỗi task ghi đầu ra của nó vào một tệp riêng biệt, và đầu ra chỉ trở nên hiển thị khi một task hoàn thành thành công.

Cụ thể, cách tiếp cận batch đối với fault tolerance đảm bảo rằng đầu ra của công việc batch giống như thể không có gì sai sót xảy ra, ngay cả khi một số task đã thất bại. Nó tạo cảm giác như thể mọi bản ghi đầu vào đã được xử lý đúng một lần—không có bản ghi nào bị bỏ qua, và không có bản ghi nào bị xử lý hai lần. Mặc dù việc khởi động lại các task có nghĩa là các bản ghi có thể bị xử lý nhiều lần, nhưng hiệu ứng hiển thị trong đầu ra giống như thể chúng chỉ được xử lý một lần duy nhất. Nguyên tắc này được gọi là *exactly-once semantics* (ngữ nghĩa đúng một lần), mặc dù *effectively-once* (thực tế là một lần) sẽ là một thuật ngữ mô tả chính xác hơn [73].

Vấn đề tương tự về fault tolerance cũng nảy sinh trong stream processing, nhưng nó ít trực tiếp hơn để xử lý. Việc chờ cho đến khi một task kết thúc trước khi làm cho đầu ra của nó hiển thị không phải là một lựa chọn, bởi vì một stream là vô hạn, vì vậy bạn không bao giờ có thể kết thúc việc xử lý nó.

Microbatching và checkpointing

Một giải pháp là chia nhỏ stream thành các khối nhỏ và xử lý mỗi khối giống như một quy trình batch thu nhỏ. Cách tiếp cận này được gọi là *microbatching*, và nó được sử dụng trong Spark Streaming [74]. Kích thước batch thường vào khoảng một giây, đây là kết quả của một sự đánh đổi (trade-off) về hiệu năng: các batch nhỏ hơn sẽ gây ra chi phí lập lịch và điều phối lớn hơn, trong khi các batch lớn hơn đồng nghĩa với việc trì hoãn lâu hơn trước khi kết quả của stream processor trở nên hiển thị.

Microbatching cũng cung cấp một cách ngầm định một tumbling window bằng với kích thước batch (được phân cửa sổ theo thời gian xử lý, không phải theo event

timestamp); bất kỳ công việc nào yêu cầu các window lớn hơn đều cần phải mang theo state một cách rõ ràng từ microbatch này sang microbatch tiếp theo.

Một cách tiếp cận biến thể, được sử dụng trong Apache Flink, là định kỳ tạo ra các rolling checkpoint của state và ghi chúng vào bộ lưu trữ bền vững [75, 76]. Nếu một stream operator bị crash, nó có thể khởi động lại từ checkpoint gần nhất và loại bỏ bất kỳ output nào được tạo ra giữa checkpoint cuối cùng và lần crash đó. Các checkpoint được kích hoạt bởi các barrier trong message stream, tương tự như ranh giới giữa các microbatch, nhưng không ép buộc một kích thước window cụ thể.

Trong phạm vi của framework stream processing, các cách tiếp cận microbatching và checkpointing cung cấp cùng một semantics exactly-once như batch processing. Tuy nhiên, ngay khi output rời khỏi stream processor (ví dụ, khi nó ghi vào một database, publish các message tới một message broker bên ngoài, hoặc kích hoạt việc gửi email), framework không còn khả năng loại bỏ output của một microbatch đã thất bại. Trong trường hợp này, việc khởi động lại một task thất bại sẽ khiến side effect bên ngoài xảy ra hai lần, và chỉ riêng microbatching hoặc checkpointing là không đủ để ngăn chặn vấn đề này.

Xem xét lại atomic commit

Để tạo ra vẻ ngoài của việc xử lý exactly-once khi có lỗi xảy ra, chúng ta cần đảm bảo rằng tất cả các output và side effect của việc xử lý một event đều được duy trì *nếu và chỉ nếu* việc xử lý đó thành công. Điều đó bao gồm bất kỳ message nào được gửi tới các downstream operator hoặc các hệ thống messaging bên ngoài (bao gồm cả email hoặc thông báo push), các lệnh ghi vào database, các thay đổi đối với state của operator, và các xác nhận (acknowledgments) của các input message (bao gồm cả việc di chuyển consumer offset về phía trước trong một log-based message broker).

Tất cả các hành động này phải xảy ra một cách atomic: hoặc tất cả chúng cùng xảy ra, hoặc không có hành động nào xảy ra cả. Nếu cách tiếp cận này nghe có vẻ quen thuộc, đó là vì chúng ta đã thảo luận về nó trong mục “Exactly-once message processing” trong bối cảnh các distributed transaction và two-phase commit.

Chúng ta đã xem xét các vấn đề với các triển khai truyền thống của distributed transaction, chẳng hạn như XA, trong Chương 8. Tuy nhiên, trong các môi trường hạn chế hơn, ta có thể triển khai cơ chế atomic commit như vậy một cách hiệu quả. Cách tiếp cận này được sử dụng trong Google Cloud Dataflow [66, 75], VoltDB [77], và Apache Kafka [78, 79]. Không giống như XA, các triển khai này không cố gắng cung cấp các transaction xuyên suốt các công nghệ không đồng nhất, mà thay vào đó giữ các transaction ở nội bộ bằng cách quản lý cả thay đổi state và messaging bên trong framework stream processing. Chi phí overhead của giao thức transaction có thể được giảm bớt (amortized) bằng cách xử lý nhiều input message trong cùng một transaction.

Idempotence

Mục tiêu của chúng ta là loại bỏ output một phần của bất kỳ task thất bại nào để chúng có thể được thử lại một cách an toàn. Distributed transaction là một cách để đạt được mục tiêu đó; một cách khác là dựa vào *idempotence* (tính lũy đẳng), như chúng ta đã thấy trong mục “Durable Execution and Workflows” [80].

Một operation có tính idempotent là thao tác mà bạn có thể thực hiện nhiều lần, và nó mang lại hiệu ứng tương tự như khi bạn chỉ thực hiện nó một lần duy nhất. Ví dụ, việc xóa một key trong một key-value store là idempotent (việc xóa lại giá trị đó không gây thêm hiệu ứng nào), trong khi việc tăng một counter thì không phải là idempotent (thực hiện việc tăng thêm một lần nữa đồng nghĩa với việc giá trị bị tăng lên hai lần).

Ngay cả khi một operation không có tính idempotent tự nhiên, nó thường có thể được làm cho trở nên idempotent với một chút metadata bổ sung. Ví dụ, khi tiêu thụ các message từ Kafka, mỗi message đều có một offset tăng dần (monotonically increasing) và bền vững. Khi ghi một giá trị vào một database bên ngoài, bạn có thể đính kèm offset của message đã kích hoạt lần ghi cuối cùng cùng với giá trị đó. Do đó, bạn có thể biết liệu một bản cập nhật đã được áp dụng hay chưa và tránh thực hiện lại cùng một bản cập nhật đó. Việc xử lý state trong Storm’s Trident cũng dựa trên một ý tưởng tương tự.

Dựa vào idempotence (tính lũy đẳng) đòi hỏi một vài giả định: việc khởi động lại một tác vụ bị lỗi phải phát lại (replay) các message theo đúng thứ tự (một message broker dựa trên log sẽ thực hiện việc này), quá trình xử lý phải mang tính deterministic (định mệnh), và không có node nào khác có thể cập nhật cùng một giá trị một cách đồng thời [81, 82]. Khi thực hiện failover từ node xử lý này sang node khác, có thể cần đến fencing (chốt chặn) (xem “Distributed Locks and Leases”) để ngăn chặn sự can thiệp từ một node vốn được cho là đã chết nhưng thực tế vẫn đang hoạt động. Bất chấp tất cả những lưu ý đó, các thao tác idempotent có thể là một cách hiệu quả để đạt được ngữ nghĩa exactly-once với chi phí overhead nhỏ.

Xây dựng lại state sau khi gặp lỗi

Bất kỳ stream process nào yêu cầu state—ví dụ như các phép tổng hợp theo window (như bộ đếm, giá trị trung bình và histogram) và bất kỳ table hay index nào được sử dụng để join—đều phải đảm bảo rằng state này có thể được khôi phục sau khi gặp lỗi.

Một lựa chọn là lưu trữ state trong một datastore từ xa và thực hiện replication, mặc dù việc phải truy vấn một database từ xa cho mỗi message riêng lẻ có thể sẽ chậm. Một lựa chọn thay thế là giữ state cục bộ tại stream processor và thực hiện replication định kỳ. Khi đó, khi stream processor đang khôi phục sau một lỗi, tác vụ mới có thể đọc state đã được replication và tiếp tục xử lý mà không làm mất dữ liệu.

Ví dụ, Flink định kỳ chụp các snapshot của operator state và ghi chúng vào storage bền vững như một distributed filesystem [75, 76], và Kafka Streams thực hiện replication các thay đổi state bằng cách gửi chúng tới một Kafka topic chuyên dụng với log compaction, tương tự như CDC [83]. VoltDB thực hiện replication state bằng cách xử lý dư thừa mỗi input message trên nhiều node (xem “Actual Serial Execution”).

Trong một số trường hợp, việc replication state thậm chí có thể không cần thiết, vì nó có thể được xây dựng lại từ các input stream. Ví dụ, nếu state bao gồm các phép tổng hợp trên một window khá ngắn, việc chỉ đơn giản phát lại (replay) các input event tương ứng với window đó có thể đủ nhanh. Nếu state là một bản sao cục bộ của một database được duy trì bởi CDC, database đó cũng có thể được xây dựng lại từ change stream đã được log compaction.

Tất cả những điều này phụ thuộc vào các đặc tính hiệu năng của hạ tầng bên dưới. Trong một số hệ thống, độ trễ mạng có thể thấp hơn độ trễ truy cập đĩa, và băng thông mạng có thể tương đương với băng thông đĩa. Không có giải pháp nào là lý tưởng tuyệt đối cho mọi tình huống, và lợi ích của việc dùng state cục bộ so với state từ xa cũng có thể thay đổi khi các công nghệ storage và networking tiến hóa.

Tóm tắt

Trong chương này, chúng ta đã thảo luận về các event stream, mục đích sử dụng của chúng và cách xử lý chúng. Theo một cách nào đó, stream processing rất giống với batch processing mà chúng ta đã thảo luận ở Chương 11, nhưng được thực hiện liên tục trên các stream không giới hạn (unbounded - không bao giờ kết thúc) thay vì trên một đầu vào có kích thước cố định [84]. Từ góc độ này, các message broker và event log đóng vai trò tương đương với một filesystem trong mô hình streaming.

Chúng ta đã dành thời gian để so sánh hai loại message broker:

Message broker kiểu AMQP/JMS

Broker gán các message riêng lẻ cho các consumer, và các consumer xác nhận (acknowledge) các message riêng lẻ khi chúng đã được xử lý thành công. Các message sẽ bị xóa khỏi broker sau khi chúng đã được xác nhận. Cách tiếp cận này phù hợp như một dạng RPC bất đồng bộ (xem thêm “Event-Driven Architectures”)—ví dụ, trong một task queue, nơi thứ tự chính xác của việc xử lý message không quan trọng và không cần phải quay lại đọc lại các message cũ sau khi chúng đã được xử lý.

Message broker dựa trên log

Broker gán tất cả các message trong một shard cho cùng một consumer node và luôn phân phối các message theo cùng một thứ tự. Tính song song được đạt được thông qua sharding, và các consumer theo dõi tiến trình của chúng bằng cách

checkpoint offset của message cuối cùng mà chúng đã xử lý. Broker lưu giữ các message trên đĩa, vì vậy có thể nhảy ngược lại và đọc lại các message cũ nếu cần thiết.

Cách tiếp cận dựa trên log có những điểm tương đồng với các replication log được tìm thấy trong các cơ sở dữ liệu (xem Chương 6) và các storage engine cấu trúc theo log (xem Chương 4). Nó cũng là một dạng consensus, như chúng ta đã thấy ở Chương 10. Chúng ta đã thấy rằng cách tiếp cận này đặc biệt phù hợp cho các hệ thống stream processing tiêu thụ các input stream và tạo ra các derived state hoặc các output stream phái sinh.

Về nguồn gốc của các stream, chúng ta đã thảo luận về một vài khả năng; các sự kiện hoạt động của người dùng, các cảm biến cung cấp các giá trị đọc định kỳ, và các nguồn dữ liệu (ví dụ: dữ liệu thị trường trong tài chính) được biểu diễn một cách tự nhiên dưới dạng các stream. Chúng ta cũng thấy rằng việc coi các thao tác ghi vào một cơ sở dữ liệu như một stream cũng có thể hữu ích. Chúng ta có thể nắm bắt changelog—lịch sử của tất cả các thay đổi được thực hiện đối với một cơ sở dữ liệu—một cách ngầm định thông qua CDC hoặc một cách tường minh thông qua event sourcing. Log compaction cho phép stream duy trì một bản sao đầy đủ nội dung của một cơ sở dữ liệu.

Việc biểu diễn các cơ sở dữ liệu dưới dạng các stream mở ra những cơ hội mạnh mẽ để tích hợp các hệ thống. Bạn có thể giữ cho các hệ thống dữ liệu phái sinh như search indexes, caches, và các hệ thống phân tích luôn được cập nhật liên tục bằng cách tiêu thụ log của các thay đổi và áp dụng chúng vào hệ thống phái sinh. Bạn thậm chí có thể xây dựng các view mới trên dữ liệu hiện có bằng cách bắt đầu từ đầu và tiêu thụ log của các thay đổi từ lúc bắt đầu cho đến hiện tại.

Các tiện ích để duy trì state dưới dạng các stream và replay các message cũng là cơ sở cho các kỹ thuật cho phép thực hiện stream joins và fault tolerance trong các framework stream processing khác nhau. Chúng ta đã thảo luận về một vài mục đích của stream processing, bao gồm tìm kiếm các mẫu sự kiện (complex event processing), tính toán các aggregations theo window (stream analytics), và giữ cho các hệ thống dữ liệu phái sinh luôn được cập nhật (materialized views).

Sau đó, chúng ta đã thảo luận về những khó khăn khi suy luận về thời gian trong một stream processor, bao gồm sự phân biệt giữa processing time và event timestamps, cùng vấn đề xử lý các straggler events (các sự kiện đến trễ) xuất hiện sau khi bạn nghĩ rằng window của mình đã hoàn tất.

Chúng ta đã phân biệt ba loại joins có thể xuất hiện trong các stream processes:

Stream–stream joins

Cả hai input stream đều bao gồm các sự kiện hoạt động, và toán tử join sẽ tìm kiếm các sự kiện liên quan xảy ra trong một window thời gian. Ví dụ, toán tử join có thể khớp hai hành động được thực hiện bởi cùng một người dùng trong vòng

30 phút với nhau. Hai đầu vào của join thực tế có thể là cùng một stream (một *self join*) nếu bạn muốn tìm các sự kiện liên quan trong chính stream đó.

Stream—table joins

Một input stream bao gồm các sự kiện hoạt động, trong khi stream còn lại là một changelog của cơ sở dữ liệu. Changelog này giữ một bản sao cục bộ của cơ sở dữ liệu luôn được cập nhật. Đối với mỗi sự kiện hoạt động, toán tử join sẽ truy vấn cơ sở dữ liệu và xuất ra một sự kiện hoạt động đã được làm giàu (enriched activity event).

Table—table joins

Cả hai input stream đều là các changelog của cơ sở dữ liệu. Trong trường hợp này, mọi thay đổi ở một bên sẽ được join với state mới nhất của bên còn lại. Kết quả là một stream các thay đổi đối với materialized view của phép join giữa hai bảng.

Cuối cùng, chúng ta đã thảo luận về các kỹ thuật để đạt được fault tolerance và semantics exactly-once trong một stream processor. Tương tự như batch processing, chúng ta cần loại bỏ đầu ra một phần của bất kỳ task nào bị lỗi. Tuy nhiên, vì một stream process là một tác vụ long-running và tạo ra đầu ra liên tục, chúng ta không thể đơn giản là loại bỏ toàn bộ đầu ra. Thay vào đó, một cơ chế phục hồi ở mức độ chi tiết hơn có thể được sử dụng, dựa trên microbatching, checkpointing, transactions, hoặc các thao tác ghi idempotent.

Footnotes

References

- [1] Tyler Akidau, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael J. Fernández-Moctezuma, Reuven Lax, Sam McVeety, Daniel Mills, Frances Perry, Eric Schmidt, and Sam Whittle. “The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing.” *Proceedings of the VLDB Endowment*, volume 8, issue 12, pages 1792–1803, August 2015. [doi:10.14778/2824032.2824076](https://doi.org/10.14778/2824032.2824076)
- [2] Harold Abelson, Gerald Jay Sussman, and Julie Sussman. *Structure and Interpretation of Computer Programs*, 2nd edition. MIT Press, 1996. ISBN: 9780262510875. Archived at archive.org
- [3] Patrick Th. Eugster, Pascal A. Felber, Rachid Guerraoui, and Anne-Marie Kermarrec. “The Many Faces of Publish/Subscribe.” *ACM Computing Surveys*, volume 35, issue 2, pages 114–131, June 2003. [doi:10.1145/857076.857078](https://doi.org/10.1145/857076.857078)
- [4] Don Carney, Uğur Çetintemel, Mitch Cherniack, Christian Convey, Sangdon Lee, Greg Seidman, Michael Stonebraker, Nesime Tatbul, and Stan Zdonik. “Monitoring Streams—A New Class of Data Management Applications.” At *28th International Conference on Very Large Data Bases (VLDB)*, August 2002. [doi:10.1016/B978-155860869-6/50027-5](https://doi.org/10.1016/B978-155860869-6/50027-5)
- [5] Matthew Sackman. “Pushing Back.” *wellquite.org*, May 2016. Archived at perma.cc/3KCZ-RUFY
- [6] Thomas Figg (tef). “How (Not) to Write a Pipeline.” *cobost.org*, June 2023. Archived at perma.cc/A3V8-NYCM
- [7] Vicent Martí. “Brubeck, a statsd-Compatible Metrics Aggregator.” *github.blog*, June 2015. Archived at perma.cc/TP3Q-DJYM

- [8] Seth Lowenberger. “MoldUDP64 Protocol Specification V 1.00.” *nasdaqtrader.com*, July 2009. Archived at perma.cc/7CRQ-QBD7
- [9] Ian Malpass. “Measure Anything, Measure Everything.” *codeascraft.com*, February 2011. Archived at archive.org
- [10] Dieter Plaetinck. “25 Graphite, Grafana and statsd Gotchas.” *grafana.com*, March 2016. Archived at perma.cc/3NP3-67U7
- [11] Jeff Lindsay. “Web Hooks to Revolutionize the Web.” *progrum.com*, May 2007. Archived at perma.cc/BF9U-XNX4
- [12] Jim N. Gray. “Queues Are Databases.” Microsoft Research Technical Report MSR-TR-95-56, December 1995. Archived at arxiv.org
- [13] Mark Hapner, Rich Burrige, Rahul Sharma, Joseph Fialli, Kate Stout, and Nigel Deakin. “JSR-343 Java Message Service (JMS) 2.0 Specification.” *jms-spec.java.net*, March 2013. Archived at perma.cc/E4YG-46TA
- [14] Sanjay Aiyagari, Matthew Arrott, Mark Atwell, Jason Brome, Alan Conway, Robert Godfrey, Robert Greig, Pieter Hintjens, John O’Hara, Matthias Radestock, Alexis Richardson, Martin Ritchie, Shahrokh Sadjadi, Rafael Schloming, Steven Shaw, Martin Sustrik, Carl Trieloff, Kim van der Riet, and Steve Vinoski. “AMQP: Advanced Message Queuing Protocol Specification.” Version 0-9-1, November 2008. Archived at perma.cc/6YJf-GM9X
- [15] “Architectural Overview of Pub/Sub.” *cloud.google.com*, 2025. Archived at perma.cc/VWF5-ABP4
- [16] Aris Tzoumas. “Lessons from Scaling PostgreSQL Queues to 100k Events Per Second.” *rudderstack.com*, July 2025. Archived at perma.cc/QD8C-VA4Y
- [17] Robin Moffatt. “Kafka Connect Deep Dive—Error Handling and Dead Letter Queues.” *confluent.io*, March 2019. Archived at perma.cc/KQ5A-AB28
- [18] Dunith Danushka. “Message Reprocessing: How to Implement the Dead Letter Queue.” *redpanda.com*. Archived at perma.cc/R7UB-WEWF
- [19] Damien Gasparina, Loic Greffier, and Sebastien Viale. “KIP-1034: Dead Letter Queue in Kafka Streams.” *wiki.apache.org*, April 2024. Archived at perma.cc/3VXV-QXAN
- [20] Jay Kreps, Neha Narkhede, and Jun Rao. “Kafka: A Distributed Messaging System for Log Processing.” At *6th International Workshop on Networking Meets Databases (NetDB)*, June 2011. Archived at perma.cc/CSW7-TCQ5
- [21] Jay Kreps. “Benchmarking Apache Kafka: 2 Million Writes Per Second (On Three Cheap Machines).” *engineering.linkedin.com*, April 2014. Archived at archive.org
- [22] Kartik Paramasivam. “How We’re Improving and Advancing Kafka at LinkedIn.” *engineering.linkedin.com*, September 2015. Archived at perma.cc/3S3V-JCYJ
- [23] Philippe Dobbelaere and Kyumars Sheykh Esmaili. “Kafka Versus RabbitMQ: A Comparative Study of Two Industry Reference Publish/Subscribe Implementations.” At *11th ACM International Conference on Distributed and Event-Based Systems (DEBS)*, June 2017. *doi:10.1145/3093742.3093908*
- [24] Kate Holterhoff. “Why Message Queues Endure: A History.” *redmonk.com*, December 2024. Archived at perma.cc/6DX8-XX4W
- [25] Andrew Schofield. “KIP-932: Queues for Kafka.” *wiki.apache.org*, May 2023. Archived at perma.cc/LBE4-BEMK
- [26] Jack Vanlightly. “The Advantages of Queues on Logs.” *jack-vanlightly.com*, October 2023. Archived at perma.cc/WJ7V-287K

- [27] Jay Kreps. “The Log: What Every Software Engineer Should Know About Real-Time Data’s Unifying Abstraction.” *engineering.linkedin.com*, December 2013. Archived at perma.cc/2JHR-FR64
- [28] Andy Hattermer. “Change Data Capture Is Having a Moment. Why?” *materialize.com*, September 2021. Archived at perma.cc/AL37-P53C
- [29] Prem Santosh Udaya Shankar. “Streaming MySQL Tables in Real-Time to Kafka.” *engineeringblog.yelp.com*, August 2016. Archived at perma.cc/5ZR3-2GVV
- [30] Andreas Andreakis, Ioannis Papapanagiotou. “DBLog: A Watermark Based Change-Data-Capture Framework.” Archived at [arXiv:2010.12597](https://arxiv.org/abs/2010.12597), October 2020.
- [31] Jiri Pechanec. “Percolator.” *debezium.io*, October 2021. Archived at perma.cc/EQ8E-W6KQ
- [32] Debezium maintainers. “Debezium Connector for Cassandra.” *debezium.io*. Archived at perma.cc/WR6K-EKMD
- [33] Neha Narkhede. “Announcing Kafka Connect: Building Large-Scale Low-Latency Data Pipelines.” *confluent.io*, February 2016. Archived at perma.cc/8WXJ-L6GF
- [34] Chris Riccomini. “Kafka Change Data Capture Breaks Database Encapsulation.” *cnr.sh*, November 2018. Archived at perma.cc/P572-9MKF
- [35] Gunnar Morling. “‘Change Data Capture Breaks Encapsulation’. Does It, Though?” *decodable.co*, November 2023. Archived at perma.cc/YX2P-WNWR
- [36] Gunnar Morling. “Revisiting the Outbox Pattern.” *decodable.co*, October 2024. Archived at perma.cc/M5ZL-RPS9
- [37] Ashish Gupta and Inderpal Singh Mumick. “Maintenance of Materialized Views: Problems, Techniques, and Applications.” *IEEE Data Engineering Bulletin*, volume 18, issue 2, pages 3–18, June 1995. Archived at archive.org
- [38] Mihai Budiu, Tej Chajed, Frank McSherry, Leonid Ryzhyk, and Val Tannen. “DBSP: Incremental Computation on Streams and Its Applications to Databases.” *SIGMOD Record*, volume 53, issue 1, pages 87–95, March 2024. [doi:10.1145/3665252.3665271](https://doi.org/10.1145/3665252.3665271)
- [39] Jim Gray and Andreas Reuter. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1992. ISBN: 9781558601901
- [40] Martin Kleppmann. “Accounting for Computer Scientists.” *martin.kleppmann.com*, March 2011. Archived at perma.cc/9EGX-P38N
- [41] Pat Helland. “Immutability Changes Everything.” At *7th Biennial Conference on Innovative Data Systems Research (CIDR)*, January 2015. Archived at perma.cc/33WX-3669
- [42] Martin Kleppmann. *Making Sense of Stream Processing*. Report, O’Reilly Media, May 2016. Archived at perma.cc/RAY4-JDVX
- [43] Kartik Paramasivam. “Stream Processing Hard Problems—Part 1: Killing Lambda.” *engineering.linkedin.com*, June 2016. Archived at archive.org
- [44] Stéphane Derosiaux. “CQRS: What? Why? How?” *sderosiaux.medium.com*, September 2019. Archived at perma.cc/FZ3U-HVJ4
- [45] Baron Schwartz. “Immutability, MVCC, and Garbage Collection.” *xaprb.com*, December 2013. Archived at archive.org
- [46] Daniel Eloff, Slava Akhmechet, Jay Kreps, et al. “Re: Turning the Database Inside-out with Apache Samza.” Hacker News discussion, *news.ycombinator.com*, March 2015. Archived at perma.cc/ML9E-JC83

- [47] Cognitect, Inc. “Datomic Documentation: Excision.” *docs.datomic.com*. Archived at [perma.cc/J5QQ-SH32](#)
- [48] “Fossil Documentation: Deleting Content from Fossil.” *fossil-scm.org*, 2025. Archived at [perma.cc/DS23-GTNG](#)
- [49] Jay Kreps. “The irony of distributed systems is that data loss is really easy but deleting data is surprisingly hard.” *x.com*, March 2015. Archived at [perma.cc/7RRZ-V7B7](#)
- [50] Brent Robinson. “Crypto Shredding: How It Can Solve Modern Data Retention Challenges.” *medium.com*, January 2019. Archived at [perma.cc/4LFK-S6XE](#)
- [51] Matthew D. Green and Ian Miers. “Forward Secure Asynchronous Messaging from Puncturable Encryption.” At *IEEE Symposium on Security and Privacy*, May 2015. *doi:10.1109/SP.2015.26*
- [52] David C. Luckham. “What’s the Difference Between ESP and CEP?” *complexevents.com*, June 2019. Archived at [perma.cc/E7PZ-FDEF](#)
- [53] Arvind Arasu, Shivnath Babu, and Jennifer Widom. “The CQL Continuous Query Language: Semantic Foundations and Query Execution.” *The VLDB Journal*, volume 15, issue 2, pages 121–142, June 2006. *doi:10.1007/s00778-004-0147-z*
- [54] Julian Hyde. “Data in Flight: How Streaming SQL Technology Can Help Solve the Web 2.0 Data Crunch.” *ACM Queue*, volume 7, issue 11, December 2009. *doi:10.1145/1661785.1667562*
- [55] Philippe Flajolet, Éric Fusy, Olivier Gandouet, and Frédéric Meunier. “HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm.” At *Conference on Analysis of Algorithms (AofA)*, June 2007. *doi:10.46298/dmtcs.3545*
- [56] Jay Kreps. “Questioning the Lambda Architecture.” *oreilly.com*, July 2014. Archived at [perma.cc/2WY5-HC8Y](#)
- [57] Ian Reppel. “An Overview of Apache Streaming Technologies.” *ianreppel.org*, March 2016. Archived at [perma.cc/BB3E-QJLW](#)
- [58] Jay Kreps. “Why Local State Is a Fundamental Primitive in Stream Processing.” *oreilly.com*, July 2014. Archived at [perma.cc/P8HU-R5LA](#)
- [59] RisingWave Labs. “Deep Dive into the RisingWave Stream Processing Engine—Part 2: Computational Model.” *risingwave.com*, November 2023. Archived at [perma.cc/LM74-XDEL](#)
- [60] Frank McSherry, Derek G. Murray, Rebecca Isaacs, and Michael Isard. “Differential Dataflow.” At *6th Biennial Conference on Innovative Data Systems Research (CIDR)*, January 2013. Archived at [perma.cc/T83W-ZBR2](#)
- [61] Andy Hattemer. “Incremental Computation in the Database.” *materialize.com*, March 2020. Archived at [perma.cc/AL94-YVRN](#)
- [62] Shay Banon. “Percolator.” *elastic.co*, February 2011. Archived at [perma.cc/LS5R-4FQX](#)
- [63] Alan Woodward and Martin Kleppmann. “Real-Time Full-Text Search with Luwak and Samza.” *martin.kleppmann.com*, April 2015. Archived at [perma.cc/2U92-Q7R4](#)
- [64] Tyler Akidau. “The World Beyond Batch: Streaming 102.” *oreilly.com*, January 2016. Archived at [perma.cc/4XF9-8M2K](#)
- [65] Stephan Ewen. “Streaming Analytics with Apache Flink.” At *Kafka Summit*, April 2016. Archived at [perma.cc/QBQ4-F9MR](#)
- [66] Tyler Akidau, Alex Balikov, Kaya Bekiroğlu, Slava Chernyak, Josh Haberman, Reuven Lax, Sam McVeety, Daniel Mills, Paul Nordstrom, and Sam Whittle. “MillWheel: Fault-Tolerant Stream

- Processing at Internet Scale.” *Proceedings of the VLDB Endowment*, volume 6, issue 11, pages 1033–1044, August 2013. doi:10.14778/2536222.2536229
- [67] Alex Dean. “Improving Snowplow’s Understanding of Time.” *snowplow.io*, September 2015. Archived at perma.cc/6CT9-Z3Q2
- [68] “Azure Stream Analytics: Windowing Functions.” Microsoft Azure Reference, *learn.microsoft.com*, July 2025. Archived at archive.org
- [69] Rajagopal Ananthanarayanan, Venkatesh Basker, Sumit Das, Ashish Gupta, Haifeng Jiang, Tianhao Qiu, Alexey Reznichenko, Deomid Ryabkov, Manpreet Singh, and Shivakumar Venkataraman. “Photon: Fault-Tolerant and Scalable Joining of Continuous Data Streams.” At *ACM International Conference on Management of Data (SIGMOD)*, June 2013. doi:10.1145/2463676.2465272
- [70] Ben Kirwin. “Doing the Impossible: Exactly-Once Messaging Patterns in Kafka.” *ben.kirw.in*, November 2014. Archived at perma.cc/A5QL-QRX7
- [71] Pat Helland. “Data on the Outside Versus Data on the Inside.” At *2nd Biennial Conference on Innovative Data Systems Research (CIDR)*, January 2005. Archived at perma.cc/K9AH-LQPS
- [72] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd edition. John Wiley & Sons, 2013. ISBN: 9781118530801
- [73] Viktor Klang. “I’m coining the phrase ‘effectively-once’ for message processing with at-least-once + idempotent operations.” *x.com*, October 2016. Archived at perma.cc/7DT9-TDG2
- [74] Matei Zaharia, Tathagata Das, Haoyuan Li, Scott Shenker, and Ion Stoica. “Discretized Streams: An Efficient and Fault-Tolerant Model for Stream Processing on Large Clusters.” At *4th USENIX Conference in Hot Topics in Cloud Computing (HotCloud)*, June 2012.
- [75] Kostas Tzoumas, Stephan Ewen, and Robert Metzger. “High-Throughput, Low-Latency, and Exactly-Once Stream Processing with Apache Flink.” *ververica.com*, August 2015. Archived at archive.org
- [76] Paris Carbone, Gyula Fóra, Stephan Ewen, Seif Haridi, and Kostas Tzoumas. “Lightweight Asynchronous Snapshots for Distributed Dataflows.” *arXiv:1506.08603*, June 2015.
- [77] Ryan Betts and John Hugg. *Fast Data: Smart and at Scale*. Report, O’Reilly Media, October 2015. Archived at perma.cc/VQ6S-XQQY
- [78] Neha Narkhede and Guozhang Wang. “Exactly-Once Semantics Are Possible: Here’s How Kafka Does It.” *confluent.io*, June 2019. Archived at perma.cc/Q2AU-Q2ED
- [79] Jason Gustafson, Flavio Junqueira, Apurva Mehta, Sriram Subramanian, and Guozhang Wang. “KIP-98—Exactly Once Delivery and Transactional Messaging.” *cwiki.apache.org*, November 2016. Archived at perma.cc/95PT-RCTG
- [80] Pat Helland. “Idempotence Is Not a Medical Condition.” *Communications of the ACM*, volume 55, issue 5, pages 56–65, May 2012. doi:10.1145/2160718.2160734
- [81] Jay Kreps. “Re: Trying to Achieve Deterministic Behavior on Recovery/Rewind.” Email to *samza-dev* mailing list, September 2014. Archived at perma.cc/7DPD-GJNL
- [82] E. N. (Mootaz) Elnozahy, Lorenzo Alvisi, Yi-Min Wang, and David B. Johnson. “A Survey of Rollback-Recovery Protocols in Message-Passing Systems.” *ACM Computing Surveys*, volume 34, issue 3, pages 375–408, September 2002. doi:10.1145/568522.568525
- [83] Adam Warski. “Kafka Streams—How Does It Fit the Stream Processing Landscape?” *softwaremill.com*, June 2016. Archived at perma.cc/WQ5Q-H2J2

[84] Stephan Ewen, Fabian Hueske, and Xiaowei Jiang. “Batch as a Special Case of Streaming and Alibaba’s contribution of Blink.” *flink.apache.org*, February 2019. Archived at perma.cc/A529-SKA9

Triết lý về các Streaming Systems

Nếu một vật được định sẵn cho một vật khác như là mục đích của nó, thì mục đích cuối cùng của nó không thể chỉ là việc bảo tồn sự tồn tại của chính nó. Do đó, một thuyền trưởng không coi việc bảo tồn con tàu được giao phó là mục đích cuối cùng, vì con tàu được định sẵn cho một mục đích khác, cụ thể là để hàng hải.

(Thường được trích dẫn là: Nếu mục tiêu cao nhất của một thuyền trưởng là bảo tồn con tàu của mình, ông ta sẽ giữ nó ở cảng mãi mãi.)

St. Thomas Aquinas, *Summa Theologica* (1265–1274)

Trong Chương 2, chúng ta đã thảo luận về mục tiêu tạo ra các ứng dụng và hệ thống có tính *tin cậy*, *khả năng mở rộng* và *để bảo trì*. Những chủ đề này đã xuyên suốt tất cả các chương—ví dụ, chúng ta đã thảo luận về nhiều thuật toán fault-tolerance giúp cải thiện độ tin cậy, sharding để cải thiện khả năng mở rộng, và các cơ chế cho sự tiến hóa và trừu tượng hóa giúp cải thiện khả năng bảo trì.

Trong chương này, chúng ta sẽ kết hợp tất cả các ý tưởng này lại và xây dựng dựa trên các ý tưởng về kiến trúc streaming/event-driven từ Chương 12 nói riêng để phát triển một triết lý phát triển ứng dụng đáp ứng được các mục tiêu đó. Chương này mang tính quan điểm cá nhân nhiều hơn các chương trước, trình bày một sự đi sâu vào một triết lý cụ thể thay vì so sánh nhiều cách tiếp cận khác nhau.

Tích hợp dữ liệu

Một chủ đề lặp đi lặp lại trong cuốn sách này là đối với bất kỳ vấn đề nào, luôn có nhiều giải pháp, mỗi giải pháp đều có ưu điểm, nhược điểm và đánh đổi (trade-off) riêng. Ví dụ, khi thảo luận về các storage engine trong Chương 4, chúng ta đã xem xét log-structured storage, B-trees và column-oriented storage. Khi thảo luận về replication trong Chương 6, chúng ta đã xem xét các cách tiếp cận single-leader, multi-leader và leaderless.

Nếu bạn gặp một vấn đề như “Tôi muốn lưu trữ một số dữ liệu và tra cứu lại chúng sau đó,” thì không có một giải pháp duy nhất đúng, mà chỉ có nhiều cách tiếp cận mà mỗi cách đều phù hợp trong các hoàn cảnh khác nhau. Một bản triển khai phần mềm thường phải chọn một cách tiếp cận cụ thể. Việc làm cho một đường dẫn mã nguồn (code path) hoạt động mạnh mẽ và đạt hiệu năng tốt đã đủ khó; việc cố gắng đáp ứng quá nhiều trường hợp sử dụng với nhiều tính năng có khả năng dẫn đến việc triển khai kém hiệu quả các tính năng đó so với các công cụ chuyên dụng.

Do đó, lựa chọn công cụ phần mềm phù hợp nhất cũng phụ thuộc vào hoàn cảnh. Mọi phần mềm—ngay cả một cơ sở dữ liệu được gọi là “đa mục đích” (general-purpose)—đều được thiết kế cho một mẫu sử dụng (usage pattern) cụ thể.

Đối mặt với sự phong phú của các lựa chọn thay thế này, thách thức đầu tiên là tìm ra sự tương ứng giữa các sản phẩm phần mềm và các hoàn cảnh phù hợp với chúng. Các nhà cung cấp thường ngần ngại cho bạn biết về những loại workload mà phần mềm của họ không phù hợp, nhưng hy vọng rằng các chương trước đã trang bị cho bạn những câu hỏi cần thiết để giúp bạn hiểu được những ẩn ý đằng sau và nắm bắt tốt hơn các đánh đổi (trade-off).

Tuy nhiên, ngay cả khi bạn hiểu hoàn hảo sự tương ứng giữa các công cụ và hoàn cảnh sử dụng, vẫn còn một thách thức khác. Trong các ứng dụng phức tạp, dữ liệu thường được sử dụng theo nhiều cách khác nhau, và một phần mềm khó có khả năng phù hợp với *tất cả* các cách đó. Do đó, bạn chắc chắn sẽ phải chấp vá nhiều phần mềm lại với nhau để cung cấp đầy đủ chức năng cho ứng dụng của mình.

Kết hợp các công cụ chuyên dụng bằng cách tạo dữ liệu phái sinh (derived data)

Để đưa ra một ví dụ, việc cần tích hợp một cơ sở dữ liệu OLTP với một full-text search index để xử lý các truy vấn cho các từ khóa bất kỳ là điều phổ biến. Mặc dù một số cơ sở dữ liệu (như PostgreSQL) có bao gồm tính năng full-text indexing, điều này có thể đủ cho các ứng dụng đơn giản [1], nhưng các tiện ích tìm kiếm tinh vi hơn đòi hỏi các công cụ truy hồi thông tin chuyên dụng. Ngược lại, các search index thường không phù hợp để làm một system of record bền vững. Do đó, nhiều ứng dụng cần kết hợp hai công cụ để đáp ứng tất cả các yêu cầu của chúng.

Chúng ta đã đề cập đến vấn đề tích hợp các hệ thống dữ liệu trong mục “Keeping Systems in Sync”. Khi số lượng các dạng biểu diễn của dữ liệu tăng lên, vấn đề tích hợp sẽ trở nên khó khăn hơn. Bên cạnh database và search index, có lẽ bạn cần phải lưu giữ các bản sao dữ liệu trong các hệ thống phân tích (data warehouse, hoặc các hệ thống batch processing và stream processing); duy trì các cache hoặc các phiên bản denormalized của các đối tượng được phái sinh từ dữ liệu gốc; truyền dữ liệu qua các hệ thống machine learning, classification, ranking, hoặc recommendation; hoặc gửi thông báo dựa trên những thay đổi của dữ liệu.

Suy luận về dataflows

Khi các bản sao của cùng một dữ liệu cần được duy trì trong nhiều hệ thống lưu trữ để đáp ứng nhiều pattern truy cập khác nhau, bạn cần phải xác định thật rõ ràng về các đầu vào và đầu ra. Dữ liệu được ghi vào đâu đầu tiên, và những dạng biểu diễn nào được phái sinh từ những nguồn nào? Làm thế nào để bạn đưa dữ liệu vào tất cả các nơi phù hợp với đúng định dạng?

Ví dụ, bạn có thể sắp xếp để dữ liệu trước tiên được ghi vào một system-of-record database, sau đó các thay đổi được thực hiện trên database đó sẽ được capture (xem mục “Change Data Capture”) và áp dụng vào search index theo cùng một thứ tự. Nếu CDC là cách duy nhất để cập nhật index, bạn có thể tin tưởng rằng index hoàn toàn được phái sinh từ system of record và do đó nhất quán với nó (ngoại trừ các lỗi phần mềm). Ghi vào database là cách duy nhất để cung cấp đầu vào mới vào hệ thống này.

Việc cho phép ứng dụng ghi trực tiếp vào cả search index và database sẽ dẫn đến vấn đề được trình bày trong Hình 12-4, trong đó hai client gửi các lệnh ghi xung đột một cách đồng thời, và hai hệ thống lưu trữ xử lý chúng theo các thứ tự khác nhau. Trong trường hợp này, cả database lẫn search index đều không “nắm quyền” quyết định thứ tự các lệnh ghi, vì vậy chúng có thể đưa ra các quyết định mâu thuẫn và trở nên mất nhất quán vĩnh viễn với nhau.

Nếu bạn có thể điều hướng tất cả đầu vào của người dùng qua một hệ thống duy nhất để quyết định thứ tự cho tất cả các lệnh ghi, việc phái sinh các dạng biểu diễn khác của dữ liệu bằng cách xử lý các lệnh ghi theo cùng một thứ tự sẽ trở nên dễ dàng hơn nhiều. Đây là một ứng dụng của phương pháp state machine replication mà chúng ta đã thấy trong mục “Consensus in Practice”. Việc bạn sử dụng CDC hay một event sourcing log không quan trọng bằng nguyên tắc quyết định một total order.

Việc cập nhật một hệ thống derived data dựa trên một event log thường có thể thực hiện được một cách deterministic và idempotent (xem mục “Idempotence”), giúp việc khôi phục sau các lỗi trở nên khá dễ dàng.

Derived data so với distributed transactions

Cách tiếp cận kinh điển để giữ các hệ thống dữ liệu nhất quán với nhau bao gồm các distributed transactions, như đã thảo luận trong mục “Two-Phase Commit”. Phương pháp sử dụng các hệ thống derived data có hiệu quả như thế nào khi so sánh với distributed transactions?

Ở cấp độ trừu tượng, chúng đạt được mục tiêu tương tự bằng các phương tiện khác nhau. Distributed transactions sử dụng một giao thức atomic commit để đảm bảo rằng các thay đổi được áp dụng một cách nguyên tử, trong khi các hệ thống dựa trên log đạt được tính đúng đắn thông qua việc retry một cách deterministic và tính idempotent.

Sự khác biệt lớn nhất là các hệ thống transaction thường đảm bảo rằng sau khi một giá trị được ghi, bạn có thể đọc ngay lập tức giá trị mới nhất (xem mục “Reading your own writes”). Ngược lại, các hệ thống derived data thường được cập nhật một cách bất đồng bộ, vì vậy theo mặc định, chúng không đảm bảo rằng các lệnh đọc là dữ liệu mới nhất.

Distributed transactions đã được sử dụng thành công trong các môi trường sẵn sàng chấp nhận các chi phí về hiệu năng và vận hành của chúng. Tuy nhiên, XA có đặc tính về hiệu năng và fault tolerance kém (xem mục “Distributed Transactions Across Different Systems”), điều này hạn chế nghiêm trọng tính hữu dụng của nó. Có thể tạo ra một giao thức tốt hơn cho distributed transactions, nhưng việc khiến một giao thức như vậy được áp dụng rộng rãi và tích hợp với các công cụ hiện có sẽ là một thách thức, và điều đó khó có thể xảy ra sớm.

Trong trường hợp thiếu sự hỗ trợ rộng rãi cho một giao thức distributed transaction tốt, dữ liệu phái sinh (derived data) dựa trên log là cách tiếp cận đầy hứa hẹn nhất để tích hợp các hệ thống dữ liệu khác nhau. Tuy nhiên, các đảm bảo như việc đọc được chính những gì mình vừa ghi (reading your own writes) là rất hữu ích, và sẽ không mang lại hiệu quả nếu chúng ta chỉ nói với mọi người rằng “eventual consistency là không thể tránh khỏi—hãy chấp nhận nó và học cách đối phó” (ít nhất là nếu không có sự hướng dẫn tốt về *cách* để đối phó).

Sau đây trong chương này, chúng ta sẽ thảo luận về một số cách tiếp cận để triển khai các đảm bảo mạnh mẽ hơn trên nền tảng các hệ thống phái sinh bất đồng bộ, và hướng tới một giải pháp trung hòa giữa distributed transactions và các hệ thống dựa trên log bất đồng bộ.

Giới hạn của total ordering

Với các hệ thống đủ nhỏ, việc xây dựng một event log có total ordering là hoàn toàn khả thi (như đã được chứng minh bởi sự phổ biến của các cơ sở dữ liệu với single-leader replication, vốn xây dựng chính xác một log như vậy). Tuy nhiên, khi các hệ thống được scale hướng tới các khối lượng công việc lớn hơn và phức tạp hơn, các hạn chế bắt đầu xuất hiện:

- Trong hầu hết các trường hợp, việc xây dựng một log có total ordering yêu cầu tất cả các event phải đi qua một *single leader node* để quyết định thứ tự. Nếu throughput của các event lớn hơn khả năng xử lý của một máy đơn lẻ, bạn cần phải shard log trên nhiều máy. Khi đó, thứ tự của các event trong hai shard sẽ trở nên mơ hồ.
- Nếu các máy chủ được trải rộng trên nhiều vùng *phân tán về mặt địa lý*—ví dụ, để có thể chịu lỗi khi toàn bộ một datacenter bị ngoại tuyến—bạn thường sẽ có một leader riêng biệt trong mỗi datacenter, bởi vì độ trễ mạng khiến việc điều phối xuyên datacenter một cách đồng bộ trở nên kém hiệu quả. Điều này dẫn đến một thứ tự không xác định cho các event bắt nguồn từ hai datacenter khác nhau.
- Khi các ứng dụng được triển khai dưới dạng *microservices*, một lựa chọn thiết kế phổ biến là triển khai mỗi service và trạng thái bền vững (durable state) của nó như một đơn vị độc lập, không có trạng thái bền vững nào được chia sẻ giữa các

service. Khi hai event bắt nguồn từ các service khác nhau, các event đó không có thứ tự xác định.

- Một số ứng dụng duy trì trạng thái ở phía client, nơi trạng thái được cập nhật ngay lập tức dựa trên đầu vào của người dùng (mà không cần chờ xác nhận từ máy chủ), và thậm chí tiếp tục hoạt động khi ngoại tuyến. Với các ứng dụng như vậy, client và server rất có khả năng sẽ thấy các event theo các thứ tự khác nhau.

Về mặt hình thức, việc quyết định một total order cho các event được gọi là *total order broadcast*, thứ mà như chúng ta đã thấy trong mục "Nhiều khía cạnh của consensus", tương đương với consensus. Hầu hết các thuật toán consensus được thiết kế cho các tình huống mà throughput của một node đơn lẻ là đủ để xử lý toàn bộ luồng event, và các thuật toán này không cung cấp cơ chế để nhiều node có thể chia sẻ công việc sắp xếp thứ tự các event.

Sắp xếp thứ tự các event để nắm bắt causality

Nếu không có mối liên kết nhân quả (causal link) nào tồn tại giữa các event, việc thiếu một total order không phải là vấn đề lớn, vì các event đồng thời có thể được sắp xếp thứ tự một cách tùy ý. Một số trường hợp khác thì dễ xử lý—ví dụ, nhiều lần cập nhật cùng một đối tượng có thể được sắp xếp total order bằng cách định tuyến tất cả các lần cập nhật cho một ID đối tượng cụ thể đến cùng một log shard. Tuy nhiên, các phụ thuộc nhân quả đôi khi phát sinh theo những cách tinh vi hơn.

Ví dụ, hãy xét một dịch vụ mạng xã hội và hai người dùng từng trong một mối quan hệ nhưng vừa mới chia tay. Một người dùng xóa người kia khỏi danh sách bạn bè, sau đó gửi một tin nhắn cho những người bạn còn lại để phàn nàn về người cũ của họ. Ý định của người dùng này là người cũ không nên nhìn thấy tin nhắn khiếm nhã đó, vì tin nhắn được gửi sau khi trạng thái bạn bè của cá nhân kia đã bị thu hồi.

Tuy nhiên, trong một hệ thống lưu trữ trạng thái bạn bè ở một nơi và tin nhắn ở một nơi khác, sự phụ thuộc thứ tự giữa event *unfriend* và event *message-send* có thể bị mất. Nếu sự phụ thuộc nhân quả không được nắm bắt, một service gửi thông báo về các tin nhắn mới có thể xử lý event *message-send* trước event *unfriend*, và do đó gửi thông báo sai cho người cũ.

Trong ví dụ này, các thông báo thực chất là một phép join giữa các tin nhắn và danh sách bạn bè, khiến nó liên quan đến các vấn đề về thời điểm của các phép join mà chúng ta đã thảo luận trước đó (xem "Time dependence of joins"). Thật không may, vấn đề này dường như không có một câu trả lời đơn giản [2, 3]. Các điểm khởi đầu bao gồm những điều sau:

- Các logical timestamp có thể cung cấp total ordering mà không cần coordination (xem "ID Generators and Logical Clocks"), vì vậy chúng có thể giúp ích khi total order broadcast không khả thi. Tuy nhiên, chúng vẫn

yêu cầu các bên nhận phải xử lý các event được chuyển đến không theo thứ tự, và chúng yêu cầu các metadata bổ sung phải được truyền đi.

- Nếu bạn có thể log một event để ghi lại trạng thái của hệ thống mà người dùng đã thấy trước khi đưa ra quyết định và gán cho event đó một identifier duy nhất, thì bất kỳ event nào xảy ra sau đó đều có thể tham chiếu đến identifier của event đó để ghi lại sự phụ thuộc nhân quả (causal dependency) [4].
- Các thuật toán conflict resolution (xem “Automatic conflict resolution”) giúp xử lý các event được chuyển đến theo một thứ tự không mong đợi. Chúng hữu ích cho việc duy trì state, nhưng chúng không giúp ích nếu các hành động có các side effects bên ngoài (chẳng hạn như gửi một thông báo tới người dùng).

Có lẽ trong tương lai, các pattern cho phát triển ứng dụng sẽ xuất hiện cho phép capture các sự phụ thuộc nhân quả một cách hiệu quả, và duy trì derived state một cách chính xác, mà không buộc tất cả các event phải đi qua nút thắt cổ chai của total order broadcast.

Batch và Stream Processing

Mục tiêu của tích hợp dữ liệu là đảm bảo rằng dữ liệu cuối cùng sẽ ở đúng định dạng tại tất cả các vị trí phù hợp. Để làm được điều này, cần phải tiêu thụ các đầu vào, biến đổi, join, lọc, tổng hợp, huấn luyện mô hình, đánh giá, và cuối cùng là ghi vào các đầu ra thích hợp. Các bộ xử lý batch và stream là những công cụ để đạt được mục tiêu này. Đầu ra của các quá trình batch và stream là các dataset phái sinh như search index, materialized view, các gợi ý hiển thị cho người dùng, các số liệu tổng hợp, v.v.

Như chúng ta đã thấy trong Chương 11 và 12, batch và stream processing có rất nhiều nguyên tắc chung. Sự khác biệt cơ bản chính là các bộ xử lý stream hoạt động trên các dataset không giới hạn (unbounded), trong khi đầu vào của quá trình batch có kích thước hữu hạn và đã biết trước.

Duy trì derived state

Batch processing mang đậm phong cách functional (ngay cả khi mã nguồn không được viết bằng ngôn ngữ lập trình functional). Nó khuyến khích các hàm thuần túy (pure functions) có tính xác định (deterministic), với đầu ra chỉ phụ thuộc vào đầu vào và không có side effects nào khác ngoài các đầu ra tường minh, coi đầu vào là immutable và đầu ra là append-only. Stream processing cũng tương tự, nhưng nó mở rộng các toán tử để cho phép một state được quản lý và có khả năng chịu lỗi (fault-tolerant).

Nguyên tắc về các hàm xác định với đầu vào và đầu ra được định nghĩa rõ ràng không chỉ tốt cho khả năng chịu lỗi mà còn đơn giản hóa việc suy luận về các dataflow trong một tổ chức [5]. Bất kể dữ liệu phái sinh là một search index, một mô hình thống kê, hay một cache, việc tư duy theo hướng các data pipeline dùng để phái sinh thứ này từ thứ khác, đẩy các thay đổi state trong một hệ thống thông qua mã ứng dụng functional và áp dụng các tác động đó lên các hệ thống phái sinh, là rất hữu ích.

Về nguyên tắc, các hệ thống dữ liệu phái sinh có thể được duy trì một cách đồng bộ (synchronously), giống như cách một cơ sở dữ liệu quan hệ cập nhật các secondary index một cách đồng bộ trong cùng một transaction với các thao tác ghi vào bảng đang được index. Tuy nhiên, tính bất đồng bộ (asynchrony) mới là thứ làm cho các hệ thống dựa trên event log trở nên mạnh mẽ. Nó cho phép một lỗi ở một phần của hệ thống được khoanh vùng cục bộ, trong khi các distributed transaction sẽ bị abort nếu bất kỳ một bên tham gia nào thất bại, do đó chúng có xu hướng khuếch đại lỗi bằng cách lan truyền chúng ra phần còn lại của hệ thống.

Trong mục “Sharding và Secondary Indexes”, chúng ta đã thấy rằng các secondary index thường xuyên vượt qua ranh giới của các shard. Một hệ thống sharded với các secondary index cần phải gửi các lệnh ghi tới nhiều shard (nếu index được phân mảnh theo term) hoặc gửi các lệnh đọc tới tất cả các shard (nếu index được phân mảnh theo document). Việc giao tiếp xuyên shard như vậy cũng sẽ đạt được độ tin cậy và khả năng mở rộng cao nhất nếu index được duy trì một cách bất đồng bộ [6].

Xử lý lại dữ liệu để tiến hóa ứng dụng

Khi duy trì dữ liệu phái sinh (derived data), cả batch processing và stream processing đều hữu ích. Stream processing cho phép các thay đổi trong đầu vào được phản ánh vào các view phái sinh với độ trễ thấp, trong khi batch processing cho phép một lượng lớn dữ liệu lịch sử tích lũy được xử lý lại để tạo ra các view mới trên một dataset hiện có.

Đặc biệt, việc xử lý lại dữ liệu hiện có cung cấp một cơ chế tốt để bảo trì hệ thống, giúp nó tiến hóa để hỗ trợ các tính năng mới và các yêu cầu đã thay đổi. Nếu không có việc xử lý lại, schema evolution sẽ bị giới hạn trong các thay đổi đơn giản như thêm một field tùy chọn mới vào một bản ghi hoặc thêm một loại bản ghi mới. Ngược lại, với việc xử lý lại, ta có thể cấu trúc lại một dataset thành một mô hình hoàn toàn khác để phục vụ tốt hơn các yêu cầu mới.

Schema Migrations trên đường sắt

Các cuộc “schema migrations” quy mô lớn cũng xảy ra trong các hệ thống phi máy tính. Ví dụ, trong những ngày đầu xây dựng đường sắt ở Anh vào thế kỷ 19, đã có nhiều tiêu chuẩn cạnh tranh khác nhau về khổ đường ray (khoảng cách giữa hai thanh ray). Các đoàn tàu được chế tạo cho khổ đường này không thể chạy trên đường ray của khổ đường khác, điều này đã hạn chế khả năng kết nối trong mạng lưới đường sắt [7].

Sau khi một khổ đường tiêu chuẩn duy nhất cuối cùng được quyết định vào năm 1846, các đường ray với các khổ khác đã phải được chuyển đổi—nhưng làm thế nào để thực hiện việc này mà không phải đóng cửa tuyến đường sắt trong nhiều tháng hoặc nhiều năm? Giải pháp là trước tiên chuyển đổi đường ray sang *khổ kép* (*dual gauge*) hoặc *khổ hỗn hợp* (*mixed gauge*) bằng cách thêm thanh ray thứ ba. Việc chuyển đổi này có thể được thực hiện dần dần, và khi hoàn tất, các đoàn tàu của cả hai khổ đường đều có thể chạy trên tuyến này bằng cách sử dụng hai trong số ba thanh ray. Cuối cùng, khi tất cả các đoàn tàu đã được chuyển đổi sang khổ tiêu chuẩn, thanh ray cung cấp khổ không tiêu chuẩn có thể được tháo bỏ.

Việc “xử lý lại” các đường ray hiện có theo cách này, cho phép các phiên bản cũ và mới tồn tại song song, giúp việc thay đổi khổ đường có thể thực hiện dần dần trong nhiều năm. Tuy nhiên, công việc này rất tốn kém, đó là lý do tại sao các khổ đường không tiêu chuẩn vẫn tồn tại cho đến ngày nay. Ví dụ, hệ thống BART ở vùng Vịnh San Francisco sử dụng một khổ đường khác so với phần lớn nước Mỹ.

Các view phái sinh cho phép sự tiến hóa *dần dần*. Nếu bạn muốn cấu trúc lại một dataset, bạn không cần phải thực hiện migration như một sự thay đổi đột ngột. Thay vào đó, bạn có thể duy trì schema cũ và schema mới song song như hai view phái sinh độc lập trên cùng một dữ liệu nền tảng. Sau đó, bạn có thể bắt đầu chuyển một số lượng nhỏ người dùng sang view mới để kiểm tra hiệu năng và tìm lỗi, trong khi hầu hết người dùng vẫn tiếp tục được điều hướng tới view cũ. Dần dần, bạn có thể tăng tỷ lệ người dùng truy cập view mới, và cuối cùng bạn có thể loại bỏ view cũ [8, 9].

Điểm hay của một cuộc migration dần dần như vậy là mọi giai đoạn của quá trình đều có thể đảo ngược một cách dễ dàng nếu có lỗi xảy ra; bạn luôn có một hệ thống đang hoạt động để quay trở lại. Việc giảm thiểu rủi ro gây ra thiệt hại không thể đảo ngược cho phép bạn tự tin hơn khi tiến hành, và do đó di chuyển nhanh hơn để cải thiện hệ thống của mình [10].

Hợp nhất batch và stream processing

Một đề xuất sớm nhằm hợp nhất batch và stream processing là *lambda architecture* [11], vốn gặp phải một số vấn đề [12] và đã không còn được sử dụng. Các hệ thống gần đây hơn cho phép thực hiện các tính toán batch (xử lý lại dữ liệu lịch sử) và tính toán stream (xử lý các event khi chúng đến) trong cùng một hệ thống [13]—một cách tiếp cận đôi khi được gọi là *kappa architecture* [12].

Việc hợp nhất batch và stream processing trong một hệ thống yêu cầu các tính năng sau:

- Khả năng phát lại (replay) các sự kiện lịch sử thông qua cùng một engine xử lý vốn đang xử lý luồng các sự kiện gần đây. Ví dụ, các message broker dựa trên log có khả năng phát lại các message, và một số stream processor có thể đọc đầu vào từ một distributed filesystem hoặc object storage.
- Semantics exactly-once cho các stream processor—nghĩa là, đảm bảo rằng đầu ra giống hệt như khi không có lỗi nào xảy ra, ngay cả khi trên thực tế đã có lỗi xảy ra. Tương tự như batch processing, điều này đòi hỏi phải loại bỏ các đầu ra một phần của bất kỳ task nào bị lỗi.
- Các công cụ để windowing theo event time, thay vì theo processing time, vì processing time trở nên vô nghĩa khi thực hiện xử lý lại các sự kiện lịch sử. Ví dụ, Apache Beam cung cấp một API để biểu diễn các tính toán như vậy, sau đó chúng có thể được chạy bằng Apache Flink hoặc Google Cloud Dataflow.

Unbundling Databases

Ở cấp độ trừu tượng, các database, batch/stream processor và hệ điều hành đều thực hiện các chức năng giống nhau: chúng lưu trữ một số dữ liệu, và cho phép bạn xử lý cũng như truy vấn dữ liệu đó [14, 15]. Một database lưu trữ dữ liệu trong các bản ghi của một data model (các hàng trong bảng, document, các đỉnh trong một graph, v.v.), trong khi filesystem của một hệ điều hành lưu trữ dữ liệu trong các file—nhưng về cốt lõi, cả hai đều là các hệ thống "quản lý thông tin" [16]. Như chúng ta đã thấy trong Chương 11, các batch processor giống như một phiên bản phân tán của Unix.

Trên thực tế, có nhiều khác biệt thực tế. Ví dụ, nhiều filesystem không xử lý tốt một directory chứa 10 triệu file nhỏ, trong khi một database chứa 10 triệu bản ghi nhỏ là điều hoàn toàn bình thường và không có gì đáng chú ý. Tuy nhiên, những điểm tương đồng và khác biệt giữa hệ điều hành và database là rất đáng để khám phá.

Unix và các relational database đã tiếp cận vấn đề quản lý thông tin với những triết lý rất khác nhau. Unix xem mục đích của nó là cung cấp cho các lập trình viên một sự trừu tượng hóa phần cứng ở mức thấp nhưng khá logic, trong khi các relational database muốn cung cấp cho các lập trình viên ứng dụng một sự trừu tượng hóa mức cao nhằm che giấu sự phức tạp của các cấu trúc dữ liệu trên đĩa, concurrency,

crash recovery, v.v. Unix phát triển các pipe và file vốn chỉ là các chuỗi byte, trong khi các database phát triển SQL và các transaction.

Cách tiếp cận nào tốt hơn? Điều đó tùy thuộc vào những gì bạn muốn. Unix "đơn giản hơn" theo nghĩa nó là một lớp bao bọc (wrapper) khá mỏng quanh các tài nguyên phần cứng; các relational database "đơn giản hơn" theo nghĩa một truy vấn declarative ngắn gọn có thể tận dụng rất nhiều hạ tầng mạnh mẽ (tối ưu hóa truy vấn, index, các phương pháp join, kiểm soát concurrency, replication, v.v.) mà không cần người viết truy vấn phải hiểu các chi tiết triển khai.

Sự căng thẳng giữa các triết lý này đã kéo dài nhiều thập kỷ (cả Unix và relational model đều xuất hiện vào đầu những năm 1970) và vẫn chưa được giải quyết. Ví dụ, phong trào NoSQL có thể được hiểu là muốn áp dụng cách tiếp cận kiểu Unix với các sự trừu tượng hóa mức thấp vào lĩnh vực lưu trữ dữ liệu OLTP phân tán.

Mục này cố gắng hòa giải hai cách tiếp cận, với hy vọng rằng chúng ta có thể kết hợp những gì tốt nhất của cả hai thế giới.

Kết hợp các công nghệ lưu trữ dữ liệu

Trong suốt cuốn sách này, chúng ta đã thảo luận về các feature khác nhau được cung cấp bởi các database và cách chúng hoạt động, bao gồm những thứ sau:

- Các secondary index, cho phép bạn tìm kiếm các bản ghi một cách hiệu quả dựa trên giá trị của một field
- Các materialized view, vốn là một dạng cache được tính toán trước của các kết quả truy vấn
- Các replication log, giúp giữ cho các bản sao dữ liệu trên các node khác luôn được cập nhật
- Các full-text search index, cho phép tìm kiếm từ khóa trong văn bản và được tích hợp sẵn trong một số relational database [1]

Trong các Chương 11 và 12, các chủ đề tương tự đã xuất hiện; chúng ta đã nói về việc xây dựng các full-text search index, về việc bảo trì materialized view, và về việc replication các thay đổi từ một database sang các hệ thống dữ liệu phái sinh (derived data) bằng cách sử dụng CDC.

Có vẻ như tồn tại những điểm tương đồng giữa các feature được tích hợp sẵn trong database và các hệ thống dữ liệu phái sinh (derived data systems) mà mọi người đang xây dựng bằng các bộ xử lý batch và stream.

Tạo một index

Hãy suy nghĩ về những gì xảy ra khi bạn chạy `CREATE INDEX` để tạo một index mới trong một relational database. Database phải quét qua một snapshot nhất quán của

một table, chọn ra tất cả các giá trị field đang được đánh index, sắp xếp chúng và ghi ra index. Sau đó, nó phải xử lý các backlog ghi đã được thực hiện kể từ khi snapshot nhất quán được chụp (giả sử table không bị lock trong khi tạo index, để các thao tác ghi có thể tiếp tục). Khi việc đó hoàn tất, database phải tiếp tục duy trì index luôn được cập nhật bất cứ khi nào một transaction ghi vào table.

Quá trình này cực kỳ giống với việc thiết lập một follower replica mới (xem “Setting Up New Followers”), và cũng rất giống với việc bootstrapping CDC trong một streaming system (xem “Initial snapshot”).

Bất cứ khi nào bạn chạy `CREATE INDEX`, database về cơ bản sẽ xử lý lại dataset hiện có và phái sinh index như một view mới trên dữ liệu hiện có. Dữ liệu hiện có có thể là một snapshot của trạng thái thay vì là một log của tất cả các thay đổi đã từng xảy ra, nhưng cả hai có liên quan chặt chẽ với nhau.

Meta-database của mọi thứ

Dưới góc nhìn này, dataflow xuyên suốt toàn bộ một tổ chức bắt đầu trông giống như một database khổng lồ [5]. Bất cứ khi nào một process batch, stream, hoặc ETL vận chuyển dữ liệu từ nơi này và dạng này sang nơi khác và dạng khác, nó đang hoạt động giống như subsystem database giúp duy trì các index hoặc materialized view luôn được cập nhật.

Nếu nhìn theo cách này, các bộ xử lý batch và stream giống như những triển khai phức tạp của các trigger, stored procedure, và các thuật toán duy trì materialized view. Các hệ thống dữ liệu phái sinh mà chúng duy trì giống như các loại index khác nhau. Ví dụ, một relational database có thể hỗ trợ các B-tree index, hash index, spatial index, và các loại index khác. Trong kiến trúc mới nổi của các hệ thống dữ liệu phái sinh, thay vì triển khai các tiện ích đó như là các feature của một sản phẩm database tích hợp duy nhất, chúng được cung cấp bởi nhiều phần mềm khác nhau, chạy trên các máy khác nhau, và được quản trị bởi các team khác nhau.

Những phát triển này sẽ đưa chúng ta đến đâu trong tương lai? Nếu chúng ta bắt đầu từ tiền đề rằng không có một data model hay định dạng storage duy nhất nào phù hợp cho tất cả các access pattern, thì có hai hướng mà các công cụ storage và xử lý khác nhau vẫn có thể được kết hợp thành một hệ thống gắn kết:

Federated databases (hợp nhất các thao tác đọc)

Có thể cung cấp một giao diện truy vấn thống nhất cho nhiều loại storage engine và phương pháp xử lý nền tảng khác nhau—một cách tiếp cận được gọi là *federated database* hoặc *polystore* [17, 18]. Ví dụ, feature *foreign data wrapper* của PostgreSQL khớp với pattern này, cũng như các engine truy vấn federated như Trino, Hoptimator, và Xorq. Các ứng dụng cần một data model hoặc giao diện truy vấn chuyên dụng vẫn có thể truy cập trực tiếp vào các storage engine nền

tảng, trong khi những người dùng muốn kết hợp dữ liệu từ các nơi khác nhau có thể làm điều đó dễ dàng thông qua giao diện federated.

Một giao diện truy vấn federated tuân theo truyền thống relational của một hệ thống tích hợp duy nhất với một ngôn ngữ truy vấn cấp cao và semantics thanh thoát, nhưng có một triển khai phức tạp.

Unbundled databases (hợp nhất các thao tác ghi)

Trong khi federation giải quyết việc truy vấn chỉ đọc (read-only) trên nhiều hệ thống, nó không có câu trả lời tốt cho việc đồng bộ hóa các thao tác ghi trên các hệ thống đó. Chúng ta đã nói rằng trong một database duy nhất, việc tạo một index nhất quán là một feature được tích hợp sẵn. Khi chúng ta kết hợp nhiều hệ thống storage, chúng ta cũng cần đảm bảo rằng tất cả các thay đổi dữ liệu cuối cùng đều nằm ở tất cả các vị trí chính xác, ngay cả khi đối mặt với các lỗi (faults). Việc làm cho việc kết nối các hệ thống storage lại với nhau một cách tin cậy trở nên dễ dàng hơn (ví dụ, thông qua CDC và event logs) giống như việc *unbundling* (tách rời) các feature duy trì index của một database theo cách có thể đồng bộ hóa các thao tác ghi trên các công nghệ khác biệt [5, 19].

Cách tiếp cận unbundling (tách rời) tuân theo truyền thống Unix về các công cụ nhỏ thực hiện tốt một việc duy nhất [20], giao tiếp thông qua một API cấp thấp đồng nhất (pipes), và có thể được kết hợp bằng cách sử dụng một ngôn ngữ cấp cao hơn (the shell) [14].

Làm sao để unbundling hoạt động hiệu quả

Federation và unbundling là hai mặt của một đồng xu: kết hợp các thành phần đa dạng để tạo thành một hệ thống tin cậy, có khả năng mở rộng và dễ bảo trì. Truy vấn chỉ đọc (read-only querying) theo kiểu federation yêu cầu ánh xạ một mô hình dữ liệu này sang một mô hình dữ liệu khác, điều này đòi hỏi một chút tư duy nhưng cuối cùng là một vấn đề khá dễ kiểm soát. Việc giữ cho các thao tác ghi trên nhiều hệ thống lưu trữ đồng bộ với nhau là một vấn đề kỹ thuật khó hơn, vì vậy chúng ta sẽ tập trung vào vấn đề đó ở đây.

Cách tiếp cận truyền thống để đồng bộ hóa các thao tác ghi yêu cầu các transaction phân tán trên các hệ thống lưu trữ không đồng nhất [17], vốn là những vấn đề gây tranh cãi như đã thảo luận trước đó. Các transaction trong một hệ thống lưu trữ hoặc hệ thống stream processing duy nhất là khả thi, nhưng khi dữ liệu vượt qua ranh giới giữa các công nghệ khác nhau, một event log bất đồng bộ với các thao tác ghi idempotent sẽ là một cách tiếp cận mạnh mẽ và thực tế hơn nhiều.

Ví dụ, các transaction phân tán được sử dụng trong một số stream processor để đạt được ngữ nghĩa exactly-once, và điều này có thể hoạt động khá tốt. Tuy nhiên, khi một transaction cần liên quan đến các hệ thống được viết bởi các nhóm người khác nhau (ví dụ, khi dữ liệu được ghi từ một stream processor sang một distributed key-

value store hoặc search index), việc thiếu một giao thức transaction chuẩn hóa sẽ khiến việc tích hợp trở nên khó khăn hơn nhiều. Một event log có thứ tự các sự kiện với các consumer idempotent là một sự trừu tượng đơn giản hơn nhiều và do đó khả thi hơn nhiều để triển khai trên các hệ thống không đồng nhất [5].

Ưu điểm lớn của việc tích hợp dựa trên log là sự *loose coupling* (tách rời lỏng lẻo) giữa các thành phần khác nhau, điều này thể hiện qua hai cách:

- Ở cấp độ hệ thống, các event stream bất đồng bộ giúp toàn bộ hệ thống trở nên mạnh mẽ hơn trước các sự cố ngừng hoạt động hoặc sự suy giảm hiệu năng của các thành phần riêng lẻ. Nếu một consumer chạy chậm hoặc gặp lỗi, event log có thể đệm các message, cho phép producer và bất kỳ consumer nào khác tiếp tục chạy mà không bị ảnh hưởng. Consumer bị lỗi có thể bắt kịp khi nó được sửa xong, vì vậy nó không bỏ lỡ bất kỳ dữ liệu nào, và lỗi được khoanh vùng. Ngược lại, sự tương tác đồng bộ của các transaction phân tán có xu hướng leo thang các lỗi cục bộ thành các thất bại quy mô lớn.
- Ở cấp độ con người, việc unbundling các hệ thống dữ liệu cho phép các thành phần phần mềm và dịch vụ được phát triển, cải tiến và bảo trì độc lập với nhau bởi các nhóm khác nhau. Sự chuyên môn hóa cho phép mỗi nhóm tập trung vào việc thực hiện tốt một việc duy nhất, với các interface được định nghĩa rõ ràng tới hệ thống của các nhóm khác. Event log cung cấp một interface đủ mạnh để nắm bắt các thuộc tính consistency khá mạnh (nhờ vào tính durability và thứ tự của các sự kiện), nhưng cũng đủ tổng quát để có thể áp dụng cho hầu hết mọi loại dữ liệu.

Hệ thống unbundled so với hệ thống tích hợp

Nếu unbundling thực sự trở thành xu hướng của tương lai, nó sẽ không thay thế các database trong hình thức hiện tại của chúng. Chúng vẫn sẽ được cần đến nhiều như từ trước đến nay, để duy trì state trong các stream processor và để phục vụ các truy vấn cho đầu ra của các batch và stream processor. Các query engine chuyên dụng cũng sẽ tiếp tục đóng vai trò quan trọng đối với các workload cụ thể—ví dụ, các query engine trong data warehouse được tối ưu hóa cho các truy vấn phân tích khám phá và xử lý loại workload này rất tốt.

Sự phức tạp khi vận hành nhiều thành phần hạ tầng có thể là một vấn đề. Mỗi thành phần phần mềm đều có learning curve cùng các vấn đề cấu hình và những đặc điểm vận hành riêng biệt, vì vậy việc triển khai càng ít moving parts càng tốt là điều đáng giá. Một sản phẩm phần mềm tích hợp duy nhất cũng có thể đạt được hiệu năng tốt hơn và có thể dự đoán được hơn trên các loại workload mà nó được thiết kế, so với một hệ thống bao gồm nhiều công cụ mà bạn đã kết hợp bằng mã nguồn ứng dụng [21]. Xây dựng để đạt được khả năng mở rộng mà bạn không cần đến là một sự lãng phí công sức và có thể khóa bạn vào một thiết kế thiếu linh hoạt. Về bản chất, đó là một hình thức premature optimization.

Mục tiêu của unbundling (tách rời) không phải là để cạnh tranh với các cơ sở dữ liệu riêng lẻ về hiệu năng cho các workload cụ thể; mục tiêu là cho phép bạn kết hợp nhiều cơ sở dữ liệu nhằm đạt được hiệu năng tốt cho một phạm vi workload rộng hơn nhiều so với những gì một phần mềm duy nhất có thể làm được. Đó là về chiều rộng, không phải chiều sâu.

Do đó, nếu một công nghệ duy nhất đáp ứng được mọi thứ bạn cần, bạn rất có thể sẽ làm tốt nhất nếu chỉ đơn giản là sử dụng sản phẩm đó thay vì cố gắng tự triển khai lại nó từ các thành phần cấp thấp hơn. Những lợi thế của unbundling và composition (kết hợp) chỉ xuất hiện khi không có một phần mềm duy nhất nào thỏa mãn được tất cả các yêu cầu của bạn.

Các công cụ để composition các hệ thống dữ liệu đang ngày càng trở nên tốt hơn. Debezium có thể trích xuất các change streams từ nhiều cơ sở dữ liệu, giao thức của Kafka đang trở thành một tiêu chuẩn de facto cho các event streams, và các engine incremental view maintenance (xem “Incremental View Maintenance”) giúp việc tính toán trước và cập nhật các cache của các truy vấn phức tạp trở nên khả thi.

Thiết kế Ứng dụng xoay quanh Dataflow

Ý tưởng tổng quát về việc cập nhật derived data (dữ liệu phái sinh) khi dữ liệu nền tảng của nó thay đổi không có gì mới. Ví dụ, các bảng tính có khả năng lập trình dataflow mạnh mẽ [22]: bạn có thể đặt một công thức vào một ô (ví dụ: tổng các ô trong một cột khác), và bất cứ khi nào bất kỳ đầu vào nào của công thức thay đổi, kết quả của công thức sẽ tự động được tính toán lại. Đây chính xác là những gì chúng ta muốn ở cấp độ hệ thống dữ liệu. Khi một bản ghi trong cơ sở dữ liệu thay đổi, chúng ta muốn bất kỳ index nào cho bản ghi đó cũng được tự động cập nhật và bất kỳ các view hoặc aggregations nào phụ thuộc vào bản ghi đó cũng được làm mới tự động. Chúng ta không nên phải lo lắng về các chi tiết kỹ thuật về cách quá trình làm mới này diễn ra và nên có thể tin tưởng một cách đơn giản rằng nó hoạt động chính xác.

Vì vậy, hầu hết các hệ thống dữ liệu vẫn còn nhiều điều phải học hỏi từ các tính năng mà VisiCalc đã có vào năm 1979 [23]. Sự khác biệt so với bảng tính là các hệ thống dữ liệu ngày nay cần phải có khả năng fault-tolerant (chịu lỗi), scalable (có khả năng mở rộng), và có khả năng lưu trữ dữ liệu một cách bền vững (durably). Chúng cũng cần có khả năng tích hợp các công nghệ khác biệt được viết bởi các nhóm người khác nhau theo thời gian và tận dụng các thư viện cũng như dịch vụ hiện có. Thật không thực tế khi kỳ vọng tất cả phần mềm đều được phát triển bằng một ngôn ngữ, framework, hoặc công cụ cụ thể nào đó.

Trong mục này, chúng ta sẽ mở rộng các ý tưởng này và khám phá một số cách để xây dựng các ứng dụng xoay quanh các cơ sở dữ liệu đã được unbundled và dataflow.

Mã ứng dụng như một hàm dẫn xuất

Khi một dataset được dẫn xuất từ một dataset khác, nó sẽ trải qua một loại hàm biến đổi nào đó. Ví dụ:

- Một secondary index là một loại dataset dẫn xuất với một hàm biến đổi đơn giản —đối với mỗi hàng hoặc document trong bảng cơ sở, nó chọn ra các giá trị trong các cột hoặc field đang được index và sắp xếp theo các giá trị đó (giả sử là một SSTable hoặc B-tree index, vốn được sắp xếp theo key).
- Một full-text search index được tạo ra bằng cách áp dụng các hàm xử lý ngôn ngữ tự nhiên khác nhau như phát hiện ngôn ngữ, phân đoạn từ (word segmentation), stemming hoặc lemmatization, sửa lỗi chính tả, và nhận diện từ đồng nghĩa, sau đó là xây dựng một cấu trúc dữ liệu để tra cứu hiệu quả (chẳng hạn như một inverted index).
- Trong một ML system, chúng ta có thể coi mô hình được dẫn xuất từ dữ liệu huấn luyện bằng cách áp dụng các hàm trích xuất feature và phân tích thống kê khác nhau. Khi mô hình được áp dụng cho dữ liệu đầu vào mới, đầu ra của nó được dẫn xuất từ đầu vào đó và các parameter đã học được (và do đó, một cách gián tiếp, từ dữ liệu huấn luyện).
- Một cache thường chứa một sự aggregation của dữ liệu dưới dạng mà nó sẽ được hiển thị trong một UI. Do đó, việc lấp đầy cache yêu cầu kiến thức về những field nào được tham chiếu trong UI; những thay đổi trong UI có thể yêu cầu cập nhật định nghĩa về cách cache được lấp đầy và xây dựng lại cache.

Hàm phái sinh cho một secondary index được yêu cầu phổ biến đến mức nó được tích hợp sẵn vào nhiều cơ sở dữ liệu như một tính năng cốt lõi, và bạn có thể gọi nó chỉ bằng cách chạy CREATE INDEX. Đối với full-text indexing, các feature ngôn ngữ cơ bản cho các ngôn ngữ phổ biến có thể được tích hợp sẵn vào cơ sở dữ liệu, nhưng các tính năng tinh vi hơn thường đòi hỏi việc tinh chỉnh đặc thù cho từng domain. Trong machine learning, feature engineering nổi tiếng là phụ thuộc vào từng ứng dụng cụ thể và thường phải kết hợp kiến thức chi tiết về sự tương tác của người dùng với ứng dụng cũng như việc deployment ứng dụng đó [24].

Khi hàm tạo ra một dataset phái sinh không phải là một hàm khuôn mẫu (cookie-cutter) tiêu chuẩn như hàm tạo secondary index, ta cần mã tùy chỉnh để xử lý các khía cạnh đặc thù của ứng dụng. Chính mã tùy chỉnh này là nơi nhiều cơ sở dữ liệu gặp khó khăn. Mặc dù các cơ sở dữ liệu quan hệ thường hỗ trợ triggers, stored procedures và user-defined functions—những thứ có thể được dùng để thực thi mã ứng dụng bên trong cơ sở dữ liệu—nhưng chúng thường chỉ được xem như một yếu tố bổ sung sau cùng trong thiết kế cơ sở dữ liệu.

Tách rời mã ứng dụng và state

Về lý thuyết, các cơ sở dữ liệu có thể đóng vai trò là môi trường deployment cho các mã ứng dụng tùy ý, giống như một hệ điều hành. Tuy nhiên, trên thực tế, chúng tỏ ra không phù hợp với mục đích này. Chúng không đáp ứng tốt các yêu cầu của phát triển ứng dụng hiện đại, chẳng hạn như quản lý dependency và package, version control, rolling upgrades, khả năng tiến hóa, monitoring, metrics, các lời gọi đến network services, và việc tích hợp với các hệ thống bên ngoài.

Mặt khác, các công cụ deployment và quản lý cluster như Kubernetes, Docker, Mesos, YARN và các công cụ khác được thiết kế đặc biệt cho mục đích chạy mã ứng dụng. Bằng cách tập trung làm tốt một việc duy nhất, chúng có thể thực hiện việc đó tốt hơn nhiều so với một cơ sở dữ liệu vốn chỉ cung cấp việc thực thi các user-defined functions như một trong nhiều tính năng của nó.

Hầu hết các ứng dụng web ngày nay được triển khai dưới dạng các stateless services, trong đó bất kỳ user request nào cũng có thể được định tuyến đến bất kỳ application server nào, và server sẽ quên mọi thứ về request đó sau khi đã gửi response. Với phong cách deployment này, các server có thể được thêm vào hoặc gỡ bỏ tùy ý, điều này rất thuận tiện—nhưng state phải được lưu trữ ở đâu đó (thường là một cơ sở dữ liệu). Xu hướng hiện nay là giữ cho logic ứng dụng stateless tách rời khỏi việc quản lý state (các cơ sở dữ liệu): không đưa logic ứng dụng vào cơ sở dữ liệu và không đưa persistent state vào ứng dụng [25]. Như những người trong cộng đồng lập trình hàm (functional programming) hay nói đùa: "Chúng ta tin vào sự tách rời giữa Giáo hội và Nhà nước" [26].

LƯU Ý

Giải thích một câu đùa thường làm nó mất hay, nhưng dù sao đây cũng là một lời giải thích để không ai cảm thấy bị bỏ lại phía sau. *Church* là một tham chiếu đến nhà toán học Alonzo Church, người đã tạo ra lambda calculus, một dạng tính toán sơ khai là nền tảng cho hầu hết các ngôn ngữ lập trình hàm. lambda calculus không có mutable state (tức là không có các biến có thể bị ghi đè), vì vậy có thể nói rằng mutable state tách rời khỏi công trình của Church.

Trong mô hình ứng dụng web điển hình này, cơ sở dữ liệu đóng vai trò như một loại biến dùng chung có thể thay đổi (mutable shared variable) mà có thể được truy cập một cách đồng bộ (synchronously) qua mạng. Ứng dụng có thể đọc và cập nhật biến này, và cơ sở dữ liệu sẽ đảm nhận việc làm cho nó trở nên durable, đồng thời cung cấp khả năng kiểm soát đồng thời (concurrency control) và fault tolerance.

Tuy nhiên, trong hầu hết các ngôn ngữ lập trình, bạn không thể đăng ký nhận thông báo (subscribe) về những thay đổi trong một biến mutable—you chỉ có thể đọc

nó theo định kỳ. Không giống như trong một bảng tính, những người đọc biến sẽ không được thông báo nếu giá trị của biến thay đổi. (Bạn có thể triển khai các thông báo như vậy trong mã của riêng mình—điều này được gọi là *observer pattern*—nhưng hầu hết các ngôn ngữ không có *pattern* này như một tính năng tích hợp sẵn.)

Các cơ sở dữ liệu đã kế thừa cách tiếp cận thụ động này đối với dữ liệu có thể thay đổi (mutable data). Nếu bạn muốn tìm hiểu xem nội dung của cơ sở dữ liệu có thay đổi hay không, thông thường lựa chọn duy nhất của bạn là poll (tức là lặp lại truy vấn theo chu kỳ). Việc đăng ký nhận các thay đổi (subscribing to changes) chỉ mới bắt đầu xuất hiện như một tính năng.

Dataflow: Sự tương tác giữa các thay đổi trạng thái và mã nguồn ứng dụng

Suy nghĩ về các ứng dụng dưới góc độ dataflow đồng nghĩa với việc phải thương lượng lại mối quan hệ giữa mã nguồn ứng dụng và quản lý trạng thái (state management). Thay vì coi cơ sở dữ liệu như một biến thụ động bị thao tác bởi ứng dụng, chúng ta suy nghĩ nhiều hơn về sự tương tác và cộng tác giữa trạng thái, các thay đổi trạng thái và mã nguồn xử lý chúng. Mã nguồn ứng dụng phản hồi các thay đổi trạng thái ở một nơi bằng cách kích hoạt các thay đổi trạng thái ở một nơi khác.

Chúng ta đã thấy ý tưởng này trong CDC, trong actor model, trong triggers và trong incremental view maintenance. Việc unbundling cơ sở dữ liệu có nghĩa là áp dụng nó vào việc tạo ra các dataset phái sinh bên ngoài cơ sở dữ liệu chính: các cache, index tìm kiếm toàn văn (full-text search), machine learning, hoặc các hệ thống phân tích. Chúng ta có thể sử dụng stream processing và các hệ thống messaging cho mục đích này.

Duy trì dữ liệu phái sinh yêu cầu các thuộc tính sau, những thứ mà các message broker dựa trên log có thể cung cấp:

- Khi duy trì dữ liệu phái sinh, thứ tự của các thay đổi trạng thái thường rất quan trọng (nếu nhiều view được phái sinh từ một event log, chúng cần xử lý các event theo cùng một thứ tự để duy trì tính nhất quán với nhau).
- Fault tolerance là yếu tố thiết yếu—chỉ cần mất một message duy nhất cũng khiến dataset phái sinh bị mất đồng bộ vĩnh viễn với nguồn dữ liệu của nó. Cả việc truyền message và cập nhật trạng thái phái sinh đều phải đảm bảo độ tin cậy.

Việc đảm bảo thứ tự message ổn định và xử lý message có khả năng chịu lỗi (fault-tolerant) là những yêu cầu khá khắt khe, nhưng chúng ít tốn kém hơn và có tính ổn định vận hành cao hơn so với các distributed transactions. Các stream processor hiện đại có thể cung cấp các đảm bảo về thứ tự và độ tin cậy này ở quy mô lớn, đồng thời cho phép mã nguồn ứng dụng được chạy dưới dạng các stream operator.

Mã nguồn ứng dụng này có thể thực hiện các xử lý tùy ý mà các hàm phái sinh tích hợp sẵn trong cơ sở dữ liệu thường không cung cấp. Giống như các công cụ Unix được kết nối bằng các pipe, các stream operator có thể được kết hợp để xây dựng các hệ thống lớn xoay quanh dataflow. Mỗi operator nhận các stream của các thay đổi trạng thái làm đầu vào và tạo ra các stream khác của các thay đổi trạng thái làm đầu ra.

Stream processors và các dịch vụ

Phong cách phát triển ứng dụng đang chiếm ưu thế hiện nay chia nhỏ chức năng thành một tập hợp các *service* giao tiếp thông qua các yêu cầu mạng đồng bộ như REST API. Ưu điểm của kiến trúc hướng dịch vụ (service-oriented architecture) như vậy so với một ứng dụng monolithic duy nhất chủ yếu là khả năng mở rộng về mặt tổ chức thông qua việc tách rời (loose coupling). Các nhóm khác nhau có thể làm việc trên các service khác nhau, giúp giảm nỗ lực điều phối giữa các nhóm (miễn là các service có thể được triển khai và cập nhật độc lập).

Việc kết hợp các stream operator thành các hệ thống dataflow có nhiều đặc điểm tương đồng với cách tiếp cận microservices [27, 28]. Tuy nhiên, cơ chế giao tiếp bên dưới rất khác biệt: đó là các luồng message một chiều, bất đồng bộ thay vì các tương tác request/response đồng bộ.

Bên cạnh những ưu điểm đã được liệt kê trong mục “Event-Driven Architectures”, chẳng hạn như khả năng chịu lỗi tốt hơn, các hệ thống dataflow cũng có thể đạt được hiệu năng tốt hơn so với các REST API hoặc RPC truyền thống. Ví dụ, giả sử một khách hàng đang mua một mặt hàng được định giá bằng một loại tiền tệ nhưng lại thanh toán bằng một loại tiền tệ khác. Để thực hiện việc chuyển đổi tiền tệ, bạn cần biết tỷ giá hối đoái hiện tại. Thao tác này có thể được triển khai theo hai cách [27, 29]:

- Trong cách tiếp cận microservices, mã nguồn xử lý việc mua hàng có lẽ sẽ truy vấn một service hoặc cơ sở dữ liệu tỷ giá hối đoái để lấy tỷ giá hiện tại cho một loại tiền tệ cụ thể.
- Trong cách tiếp cận dataflow, mã nguồn xử lý các giao dịch mua hàng sẽ đăng ký (subscribe) tới một stream các bản cập nhật tỷ giá hối đoái từ trước và ghi lại tỷ giá hiện tại vào một cơ sở dữ liệu cục bộ mỗi khi nó thay đổi. Khi tiến hành xử lý giao dịch mua hàng, mã nguồn xử lý có thể truy vấn trực tiếp cơ sở dữ liệu cục bộ này.

Cách tiếp cận thứ hai thay thế một yêu cầu mạng đồng bộ tới một dịch vụ khác bằng một truy vấn tới cơ sở dữ liệu cục bộ (có thể nằm trên cùng một máy, thậm chí trong cùng một process). Trong cách tiếp cận microservices, bạn có thể tránh được yêu cầu mạng đồng bộ bằng cách lưu cache tỷ giá hối đoái cục bộ ngay tại service xử lý giao dịch mua hàng. Tuy nhiên, để giữ cho cache đó luôn mới, bạn sẽ cần định kỳ poll để

lấy các tỷ giá hối đoái đã được cập nhật hoặc đăng ký tới một stream các thay đổi—đây chính xác là những gì xảy ra trong cách tiếp cận dataflow.

Cách tiếp cận dataflow không chỉ nhanh hơn mà còn mạnh mẽ hơn trước sự cố của một dịch vụ khác. Yêu cầu mạng nhanh nhất và đáng tin cậy nhất chính là không có yêu cầu mạng nào cả! Thay vì dùng RPC, giờ đây chúng ta có một stream join giữa các event mua hàng và các event cập nhật tỷ giá hối đoái.

Phép join này phụ thuộc vào thời gian: nếu các event mua hàng được xử lý lại tại một thời điểm muộn hơn, tỷ giá hối đoái khi đó đã thay đổi. Nếu bạn muốn tái tạo lại kết quả đầu ra ban đầu, bạn sẽ cần lấy được tỷ giá hối đoái lịch sử tại đúng thời điểm mua hàng ban đầu. Bất kể bạn truy vấn một service hay đăng ký tới một stream các bản cập nhật tỷ giá hối đoái, bạn đều sẽ cần xử lý sự phụ thuộc vào thời gian này (xem mục “Time dependence of joins”).

Việc đăng ký tới một stream các thay đổi, thay vì truy vấn trạng thái hiện tại khi cần thiết, đưa chúng ta đến gần hơn với một mô hình tính toán kiểu bảng tính (spreadsheet). Khi một phần dữ liệu thay đổi, bất kỳ dữ liệu phái sinh (derived data) nào phụ thuộc vào nó đều có thể được cập nhật nhanh chóng. Nhiều câu hỏi mở vẫn còn đó—ví dụ, xung quanh các vấn đề như các phép join phụ thuộc vào thời gian—nhưng xây dựng các ứng dụng xoay quanh các ý tưởng dataflow là một hướng đi đầy hứa hẹn để khám phá.

Quan sát Trạng thái Phái sinh (Derived State)

Ở mức độ trừu tượng, các hệ thống dataflow được thảo luận trong mục trước cung cấp cho bạn một quy trình để tạo ra các dataset phái sinh (chẳng hạn như search index, materialized view và các mô hình dự báo) và giữ cho chúng luôn được cập nhật. Hãy gọi quy trình đó là *write path*. Bất cứ khi nào một mẫu thông tin được ghi vào hệ thống, nó có thể đi qua nhiều giai đoạn batch processing và stream processing, và cuối cùng mọi dataset phái sinh đều được cập nhật để bao hàm dữ liệu đã được ghi. Hình 13-1 cho thấy một ví dụ về việc cập nhật một search index.

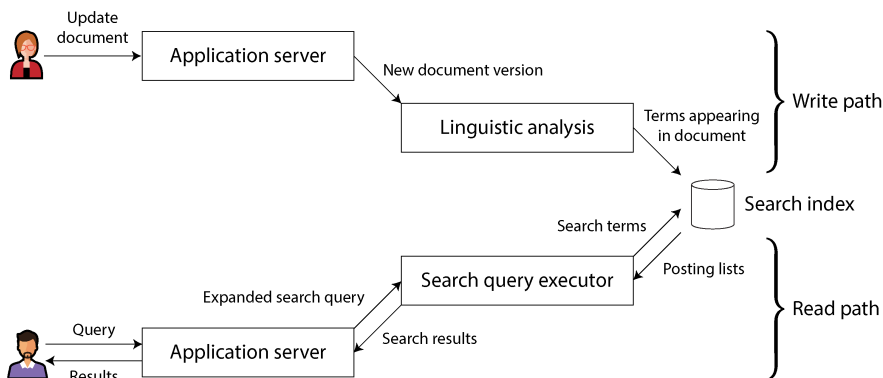


Figure 13-1. Trong một search index, các thao tác ghi (cập nhật document) gặp các thao tác đọc (queries).

Nhưng tại sao ngay từ đầu bạn lại tạo ra tập dữ liệu phái sinh (derived dataset)? Rất có thể là vì bạn muốn truy vấn lại nó vào một thời điểm nào đó sau này. Đây chính là *read path* (luồng đọc): khi phục vụ một yêu cầu của người dùng, bạn đọc từ tập dữ liệu phái sinh, có thể thực hiện thêm các bước xử lý trên kết quả, và xây dựng phản hồi gửi tới người dùng.

Khi kết hợp lại, write path và read path bao quát toàn bộ hành trình của dữ liệu, từ thời điểm nó được thu thập cho đến thời điểm nó được tiêu thụ (có lẽ bởi một con người khác). Write path là phần của hành trình được tính toán trước; nó được thực hiện một cách chủ động (eagerly) ngay khi dữ liệu vừa được đưa vào, bất kể có ai yêu cầu xem nó hay không. Read path là phần của hành trình chỉ xảy ra khi có ai đó yêu cầu. Nếu bạn quen thuộc với các ngôn ngữ lập trình hàm, bạn có thể nhận thấy rằng write path tương tự như eager evaluation (đánh giá chủ động) và read path tương tự như lazy evaluation (đánh giá lười biếng).

Tập dữ liệu phái sinh là nơi mà write path và read path gặp nhau, như được minh họa trong Hình 13-1. Nó đại diện cho một sự đánh đổi (trade-off) giữa lượng công việc cần thực hiện tại thời điểm ghi và lượng công việc cần thực hiện tại thời điểm đọc.

Materialized views và caching

Một full-text search index (index tìm kiếm toàn văn) là một ví dụ điển hình: write path cập nhật index, và read path tìm kiếm các từ khóa trong index đó. Cả việc đọc và ghi đều cần thực hiện một số công việc. Việc ghi cần cập nhật các mục trong index cho tất cả các thuật ngữ xuất hiện trong tài liệu. Việc đọc cần tìm kiếm từng từ trong truy vấn và áp dụng logic Boolean để tìm các tài liệu chứa *tất cả* các từ trong truy vấn (toán tử AND) hoặc *bất kỳ* từ đồng nghĩa nào của mỗi từ (toán tử OR).

Nếu bạn không có một index, một truy vấn tìm kiếm sẽ phải quét qua tất cả các tài liệu (giống như `grep`), điều này sẽ trở nên rất tốn kém nếu bạn có một số lượng lớn tài liệu. Không có index đồng nghĩa với việc ít công việc hơn ở write path (không có index để cập nhật) nhưng lại tốn nhiều công việc hơn rất nhiều ở read path.

Mặt khác, bạn có thể tưởng tượng việc tính toán trước các kết quả tìm kiếm cho tất cả các truy vấn có thể xảy ra. Trong trường hợp đó, bạn sẽ có ít công việc hơn để làm ở read path: không cần logic Boolean, chỉ cần tìm các kết quả cho truy vấn của bạn và trả về chúng. Tuy nhiên, write path sẽ tốn kém hơn nhiều. Tập hợp các truy vấn tìm kiếm có thể được hỏi là vô hạn (hoặc ít nhất là theo hàm mũ dựa trên số lượng thuật ngữ trong corpus), và do đó việc tính toán trước tất cả các kết quả tìm kiếm có thể là điều không khả thi.

Một lựa chọn khác là tính toán trước kết quả tìm kiếm cho chỉ một tập hợp cố định các truy vấn phổ biến nhất, để chúng có thể được phục vụ nhanh chóng mà không cần phải truy cập vào index. Các truy vấn không phổ biến vẫn có thể được phục vụ từ index. Điều này thường được gọi là một *cache* (bộ nhớ đệm) của các truy vấn phổ biến, mặc dù chúng ta cũng có thể gọi nó là một *materialized view*, vì nó sẽ cần được cập nhật khi có các tài liệu mới xuất hiện mà đáng lẽ phải được bao gồm trong kết quả của một trong các truy vấn phổ biến đó.

Từ ví dụ này, chúng ta có thể thấy rằng index không phải là ranh giới duy nhất khả thi giữa write path và read path. Việc caching các kết quả tìm kiếm phổ biến là khả thi, và việc quét kiểu `grep` mà không cần index cũng khả thi trên một số lượng nhỏ tài liệu. Nhìn theo cách này, vai trò của các cache, index và *materialized views* rất đơn giản: chúng dịch chuyển ranh giới giữa read path và write path. Chúng cho phép chúng ta thực hiện nhiều công việc hơn ở write path, bằng cách tính toán trước các kết quả, nhằm tiết kiệm công sức ở read path.

Việc dịch chuyển ranh giới giữa công việc thực hiện ở write path và read path thực chất chính là chủ đề của ví dụ về mạng xã hội trong mục “Case Study: Social Network Home Timelines”. Trong ví dụ đó, chúng ta cũng đã thấy ranh giới giữa write path và read path có thể được vẽ khác nhau đối với những người nổi tiếng so với người dùng thông thường. Sau 500 trang, chúng ta đã quay trở lại điểm xuất phát!

Các client stateful và có khả năng hoạt động offline

Ý tưởng về một ranh giới giữa write path (luồng ghi) và read path (luồng đọc) rất thú vị vì chúng ta có thể thảo luận về việc dịch chuyển ranh giới đó và khám phá xem sự dịch chuyển đó có ý nghĩa gì về mặt thực tế. Hãy xét nó trong một ngữ cảnh khác.

Trong quá khứ, các trình duyệt web là những client stateless (không lưu trạng thái) mà chỉ có thể thực hiện các tác vụ hữu ích khi bạn có kết nối internet (gần như điều duy nhất bạn có thể làm khi ngoại tuyến là cuộn lên hoặc xuống trong một trang mà

bạn đã tải trước đó khi đang trực tuyến). Tuy nhiên, các ứng dụng web JavaScript single-page hiện nay đã có rất nhiều khả năng stateful (có lưu trạng thái), bao gồm cả tương tác UI ở phía client và lưu trữ cục bộ bền vững trong trình duyệt web. Các ứng dụng di động cũng có thể lưu trữ nhiều trạng thái trên thiết bị theo cách tương tự và không yêu cầu một vòng lặp round trip tới máy chủ cho hầu hết các tương tác của người dùng.

Trong “Sync Engines and Local-First Software”, chúng ta đã thấy cách trạng thái cục bộ bền vững cho phép một nhóm các ứng dụng mà trong đó người dùng có thể làm việc ngoại tuyến, không cần kết nối internet, và đồng bộ hóa với các máy chủ từ xa trong nền khi có kết nối mạng [30]. Vì các thiết bị di động đôi khi có kết nối internet di động chậm và không đáng tin cậy, nên sẽ là một lợi thế lớn cho người dùng nếu UI không phải chờ đợi các yêu cầu mạng đồng bộ và các ứng dụng chủ yếu hoạt động ngoại tuyến.

Khi chúng ta chuyển dịch từ giả định về các client stateless giao tiếp với một cơ sở dữ liệu trung tâm sang trạng thái được duy trì trên các thiết bị của người dùng cuối, một thế giới với những cơ hội mới sẽ mở ra. Cụ thể, chúng ta có thể coi trạng thái trên thiết bị như là một *cache của trạng thái trên máy chủ*. Các pixel trên màn hình là một materialized view của các đối tượng mô hình trong ứng dụng client; các đối tượng mô hình là một replica cục bộ của trạng thái trong một datacenter từ xa [31].

Đẩy các thay đổi trạng thái tới các client

Nếu bạn tải một trang web điển hình trong trình duyệt web và dữ liệu sau đó thay đổi trên máy chủ, trình duyệt sẽ không biết về sự thay đổi đó cho đến khi bạn tải lại trang. Trình duyệt chỉ đọc dữ liệu tại một thời điểm duy nhất, giả định rằng nó là tĩnh; trình duyệt không đăng ký (subscribe) các cập nhật từ máy chủ. Do đó, trạng thái trong trình duyệt là một stale cache không được cập nhật trừ khi bạn chủ động thực hiện polling để tìm kiếm các thay đổi. (Các giao thức đăng ký nguồn cấp dữ liệu dựa trên HTTP như RSS thực chất chỉ là một dạng polling cơ bản.)

Các giao thức gần đây hơn đã vượt xa mô hình request/response cơ bản của HTTP. Server-sent events (API EventSource) và WebSockets cung cấp các kênh giao tiếp mà qua đó một trình duyệt web có thể duy trì một kết nối TCP mở tới máy chủ và máy chủ có thể chủ động đẩy các thông điệp tới trình duyệt chừng nào nó còn được kết nối. Điều này mang lại cơ hội cho máy chủ chủ động thông báo cho client người dùng cuối về bất kỳ thay đổi nào đối với trạng thái mà nó đã lưu trữ cục bộ, giúp giảm bớt sự lạc hậu của trạng thái phía client.

Xét về mô hình write path và read path của chúng ta, việc chủ động đẩy các thay đổi trạng thái tới tận các thiết bị client có nghĩa là mở rộng write path tới tận người dùng cuối. Khi một client được khởi tạo lần đầu, nó vẫn sẽ cần sử dụng read path để lấy trạng thái ban đầu, nhưng sau đó nó có thể dựa vào một luồng các thay đổi trạng thái được gửi bởi máy chủ. Do đó, các ý tưởng chúng ta đã thảo luận xoay quanh stream

processing và messaging không bị giới hạn trong việc chạy tại một datacenter; chúng ta có thể tiến xa hơn và mở rộng chúng tới tận các thiết bị của người dùng cuối [32].

Các thiết bị sẽ ngoại tuyến trong một khoảng thời gian, và chúng sẽ không thể nhận bất kỳ thông báo nào về thay đổi trạng thái từ máy chủ trong thời gian đó. Nhưng chúng ta đã giải quyết vấn đề đó; trong “Consumer offsets”, chúng ta đã thảo luận cách một consumer của một message broker dựa trên log có thể kết nối lại sau khi thất bại hoặc bị ngắt kết nối và đảm bảo rằng nó không bỏ lỡ bất kỳ thông điệp nào đã đến trong khi nó bị ngắt kết nối. Kỹ thuật tương tự cũng hoạt động cho từng người dùng riêng lẻ, nơi mỗi thiết bị là một subscriber nhỏ cho một luồng các event nhỏ.

Các luồng event end-to-end

Các công cụ để phát triển các client và UI có trạng thái (stateful), chẳng hạn như React và Elm [33], đã có khả năng cập nhật UI được hiển thị để phản hồi các thay đổi trong trạng thái cơ sở. Sẽ rất tự nhiên nếu mở rộng mô hình lập trình này để cho phép một server đẩy các sự kiện thay đổi trạng thái vào pipeline sự kiện ở phía client này.

Khi đó, các thay đổi trạng thái có thể chảy qua một đường ghi (write path) đầu-cuối: từ tương tác trên một thiết bị gây ra sự thay đổi, đi qua các event log và các hệ thống dữ liệu phái sinh (derived data) cũng như các stream processor khác nhau, cho đến tận giao diện người dùng trên một thiết bị khác. Những thay đổi trạng thái này có thể được lan truyền với độ trễ khá thấp—ví dụ, dưới một giây cho toàn bộ quá trình đầu-cuối.

Một số ứng dụng, chẳng hạn như nhắn tin tức thời và trò chơi trực tuyến, đã có kiến trúc “thời gian thực” như vậy (theo nghĩa là các tương tác có độ trễ thấp, chứ không phải theo nghĩa đảm bảo về response time). Tại sao chúng ta không xây dựng tất cả các ứng dụng theo cách này?

Thách thức nằm ở chỗ giả định về các client stateless và các tương tác request/response đã ăn sâu vào các database, thư viện, framework và protocol của chúng ta. Nhiều datastore hỗ trợ các thao tác đọc và ghi mà trong đó một request trả về một response duy nhất, nhưng ít hơn nhiều các hệ thống hỗ trợ các thao tác mà một request trả về một stream các response theo thời gian (tức là khả năng subscribe vào các thay đổi).

Để mở rộng đường ghi đến tận người dùng cuối, chúng ta sẽ cần phải tư duy lại một cách căn bản về cách xây dựng nhiều hệ thống này, chuyển dịch từ tương tác request/response sang dataflow publish/subscribe [31]. Điều này sẽ đòi hỏi nỗ lực, nhưng nó có lợi thế là giúp các UI phản hồi nhanh hơn và cung cấp khả năng hỗ trợ offline tốt hơn.

Các thao tác đọc cũng là các sự kiện

Chúng ta đã thảo luận rằng khi một stream processor ghi dữ liệu phái sinh vào một store (database, cache, hoặc index) và store đó được truy vấn, thì store đóng vai trò là ranh giới giữa đường ghi và đường đọc. Nó cho phép các truy vấn đọc truy cập ngẫu nhiên vào dữ liệu, điều mà nếu không có nó sẽ đòi hỏi phải quét toàn bộ event log.

Trong nhiều trường hợp, datastore tách biệt với hệ thống streaming. Tuy nhiên, hãy nhớ rằng các stream processor cũng cần duy trì trạng thái để thực hiện các phép aggregation và join. Trạng thái này thường được ẩn bên trong stream processor, nhưng một số framework cho phép nó được truy vấn bởi các client bên ngoài [34], biến chính stream processor thành một loại database đơn giản.

Hãy đẩy ý tưởng đó đi xa hơn. Trong mô hình mà chúng ta đã thảo luận từ trước đến nay, các thao tác ghi vào store đi qua một event log, trong khi các thao tác đọc là các request mạng tạm thời đi trực tiếp đến các node lưu trữ dữ liệu đang được truy vấn. Đây là một thiết kế hợp lý, nhưng không phải là thiết kế duy nhất khả thi. Việc biểu diễn các request đọc dưới dạng các stream sự kiện và gửi cả các sự kiện đọc lẫn các sự kiện ghi qua một stream processor cũng là điều khả thi. Processor sẽ phản hồi các sự kiện đọc bằng cách phát kết quả của việc đọc tới một output stream [35].

Khi cả thao tác ghi và đọc đều được biểu diễn dưới dạng các sự kiện và được định tuyến đến cùng một stream operator, thực tế chúng ta đang thực hiện một phép stream-table join giữa stream của các truy vấn đọc và database. Mỗi sự kiện đọc cần được gửi đến database shard đang giữ dữ liệu liên quan, giống như cách các batch và stream processor thực hiện copartition các đầu vào trên cùng một key khi thực hiện các phép join.

Sự tương ứng này giữa việc phục vụ các request và thực hiện các phép join là rất căn bản [36]. Một request đọc dùng một lần sẽ đi qua toán tử join, sau đó ngay lập tức quên đi request đó; một request subscribe là một phép join bền vững với các sự kiện trong quá khứ và tương lai ở phía bên kia của phép join.

Việc ghi lại một log của các sự kiện đọc cũng có thể mang lại lợi ích trong việc theo dõi các phụ thuộc nhân quả (causal dependencies) và nguồn gốc dữ liệu (data provenance) trên toàn hệ thống. Log này sẽ cho phép bạn tái cấu trúc những gì người dùng đã thấy trước khi họ đưa ra một quyết định cụ thể. Ví dụ, trong một cửa hàng trực tuyến, ngày giao hàng dự kiến và trạng thái kho hàng hiển thị cho khách hàng có khả năng ảnh hưởng đến việc họ có chọn mua một mặt hàng hay không [4]. Để phân tích mối liên kết này, bạn cần ghi lại kết quả truy vấn của người dùng về trạng thái giao hàng và kho hàng.

Do đó, việc ghi các yêu cầu đọc vào bộ lưu trữ bền vững (durable storage) cho phép theo dõi tốt hơn các mối quan hệ nhân quả, nhưng nó làm phát sinh thêm chi phí lưu trữ và I/O. Việc tối ưu hóa các hệ thống như vậy để giảm thiểu chi phí overhead vẫn là một bài toán nghiên cứu mở [2], nhưng nếu bạn đã ghi log các yêu cầu đọc

cho các mục đích vận hành như một tác dụng phụ của quá trình xử lý yêu cầu, thì việc biến log đó thành nguồn của các yêu cầu thay thế cũng không phải là một thay đổi lớn.

Xử lý dữ liệu multishard

Đối với các truy vấn chỉ chạm đến một shard duy nhất, nỗ lực gửi chúng qua một stream và thu thập một stream các phản hồi có lẽ là quá mức cần thiết. Tuy nhiên, ý tưởng này mở ra khả năng thực thi phân tán các truy vấn phức tạp cần kết hợp dữ liệu từ nhiều shard, tận dụng hạ tầng cho việc định tuyến tin nhắn (message routing), sharding và join vốn đã được cung cấp bởi các stream processor.

Tính năng RPC phân tán của Storm hỗ trợ mô hình sử dụng này. Ví dụ, nó đã được sử dụng để tính toán số lượng người đã xem một URL trên một mạng xã hội—tức là hợp (union) của các tập follower của tất cả những người đã đăng URL đó [37]. Vì tập hợp người dùng được sharding, việc tính toán này đòi hỏi phải kết hợp kết quả từ nhiều shard.

Một ví dụ khác về mô hình này xảy ra trong việc ngăn chặn gian lận. Để đánh giá rủi ro liệu một sự kiện mua hàng cụ thể có phải là gian lận hay không, bạn có thể kiểm tra điểm uy tín của địa chỉ IP, địa chỉ email, địa chỉ thanh toán, địa chỉ giao hàng của người dùng, v.v. Mỗi cơ sở dữ liệu uy tín này đều được sharding, vì vậy việc thu thập các điểm số cho một sự kiện mua hàng cụ thể đòi hỏi một chuỗi các thao tác join với các dataset được sharding khác nhau [38].

Các đồ thị thực thi truy vấn nội bộ của các engine truy vấn data warehouse cũng có các đặc điểm tương tự. Nếu bạn cần thực hiện loại join đa shard này, có lẽ sẽ đơn giản hơn nếu sử dụng một cơ sở dữ liệu cung cấp tính năng này thay vì triển khai nó bằng một stream processor. Tuy nhiên, việc coi các truy vấn như là các stream cung cấp một lựa chọn để triển khai các ứng dụng quy mô lớn chạy sát giới hạn của các giải pháp dùng được ngay (out-of-the-box) thông thường.

Hướng tới tính chính xác

Với các stateless service chỉ đọc dữ liệu, sẽ không có vấn đề gì lớn nếu có lỗi xảy ra; bạn có thể sửa lỗi và khởi động lại service, và mọi thứ sẽ trở lại bình thường. Các hệ thống stateful như cơ sở dữ liệu thì không đơn giản như vậy. Chúng được thiết kế để ghi nhớ mọi thứ mãi mãi (ít nhiều là vậy), vì vậy nếu có lỗi xảy ra, các tác động cũng có khả năng kéo dài mãi mãi—điều này có nghĩa là chúng đòi hỏi sự cân nhắc kỹ lưỡng hơn [39].

Chúng ta muốn xây dựng các ứng dụng có độ tin cậy và *chính xác* (tức là các chương trình có ngữ nghĩa được định nghĩa và hiểu rõ, ngay cả khi đối mặt với các lỗi khác nhau). Trong khoảng bốn thập kỷ qua, các thuộc tính transaction gồm atomicity, isolation và durability đã là những công cụ được lựa chọn để xây dựng các ứng dụng

chính xác. Tuy nhiên, những nền tảng đó yếu hơn so với vẻ ngoài của chúng: ví dụ, hãy chứng kiến sự nhầm lẫn của các isolation level yếu (xem mục “Weak Isolation Levels”).

Trong một số lĩnh vực, transaction đã bị từ bỏ hoàn toàn và được thay thế bằng các mô hình cung cấp hiệu năng và khả năng mở rộng tốt hơn nhưng có ngữ nghĩa hỗn loạn hơn. *Consistency* thường được thảo luận nhưng lại được định nghĩa một cách kém cỏi. Một số người khẳng định rằng chúng ta nên “chấp nhận eventual consistency” để có được khả năng sẵn sàng (availability) tốt hơn, trong khi lại thiếu một ý tưởng rõ ràng về việc điều đó có nghĩa là gì trong thực tế.

Đối với một chủ đề quan trọng như vậy, sự hiểu biết và các phương pháp kỹ thuật của chúng ta lại mong manh một cách đáng ngạc nhiên. Ví dụ, rất khó để xác định liệu việc chạy một ứng dụng cụ thể bằng một mức isolation level (cô lập) hoặc cấu hình replication (nhân bản) nhất định có an toàn hay không [40, 41]. Thông thường, các giải pháp đơn giản có vẻ hoạt động chính xác khi tính concurrency (đồng thời) thấp và không có lỗi, nhưng lại lộ ra nhiều bug tinh vi trong các điều kiện khắt khe hơn.

Ví dụ, các thí nghiệm Jepsen của Kyle Kingsbury [42] đã làm nổi bật sự khác biệt rõ rệt giữa các cam kết an toàn mà một số sản phẩm tuyên bố với hành vi thực tế của chúng khi có sự xuất hiện của các vấn đề mạng và sự cố crash. Ngay cả khi các sản phẩm hạ tầng như database không gặp vấn đề, mã nguồn ứng dụng vẫn cần phải sử dụng chính xác các feature mà chúng cung cấp, điều này rất dễ gây lỗi nếu cấu hình khó hiểu (như trường hợp của các mức isolation yếu, cấu hình quorum, v.v.).

Nếu ứng dụng của bạn có thể chấp nhận việc thỉnh thoảng làm hỏng hoặc mất dữ liệu theo những cách không thể dự đoán, cuộc sống sẽ đơn giản hơn nhiều, và bạn có thể chỉ cần cầu may và hy vọng điều tốt đẹp nhất sẽ đến. Nếu bạn cần những đảm bảo mạnh mẽ hơn về tính chính xác, serializability và atomic commit là những phương pháp đã được thiết lập, nhưng chúng đi kèm với một sự đánh đổi (trade-off). Chúng thường chỉ hoạt động trong một datacenter duy nhất (loại bỏ các kiến trúc phân tán về mặt địa lý), và chúng giới hạn khả năng mở rộng cũng như các đặc tính fault-tolerance mà bạn có thể đạt được.

Mặc dù phương pháp transaction truyền thống sẽ không biến mất, nhưng nó không phải là giải pháp cuối cùng để làm cho các ứng dụng trở nên chính xác và có khả năng chống chịu lỗi. Trong mục này, chúng ta sẽ khám phá những cách tư duy khác về tính chính xác trong bối cảnh của các kiến trúc dataflow.

Lập luận End-to-End cho Database

Chỉ vì một ứng dụng sử dụng một hệ thống dữ liệu cung cấp các thuộc tính an toàn tương đối mạnh, chẳng hạn như các transaction có tính serializable, không

có nghĩa là ứng dụng đó được đảm bảo sẽ không bị mất hoặc hỏng dữ liệu. Ví dụ, nếu một ứng dụng có lỗi khiến nó ghi dữ liệu sai hoặc xóa dữ liệu khỏi một database, các transaction có tính serializable sẽ không cứu được bạn. Đây là một lập luận ủng hộ dữ liệu immutable (bất biến) và append-only (chỉ cho phép ghi thêm), bởi vì sẽ dễ dàng khôi phục từ những sai lầm như vậy hơn nếu bạn loại bỏ khả năng mã nguồn bị lỗi phá hủy dữ liệu tốt.

Mặc dù tính bất biến rất hữu ích, nhưng bản thân nó không phải là liều thuốc chữa bách bệnh. Hãy cùng xem xét một ví dụ tinh vi hơn về cách mà sự hỏng dữ liệu có thể xảy ra.

Thực thi một thao tác chính xác một lần (exactly-once)

Trong phần “Distributed Transactions Across Different Systems”, chúng ta đã giới thiệu ý tưởng về ngữ nghĩa *exactly-once* (hoặc *effectively-once*) trong bối cảnh xử lý message. Ý tưởng là thế này: nếu có điều gì đó trực trặc trong khi xử lý một message, bạn có thể từ bỏ (bằng cách bỏ qua message đó và chấp nhận mất dữ liệu) hoặc thử lại. Nếu bạn thử lại, sẽ có rủi ro là việc xử lý thực tế đã thành công ngay từ lần đầu và bạn chỉ không nhận được xác nhận, dẫn đến việc message đó cuối cùng bị xử lý hai lần.

Xử lý hai lần là một dạng hỏng dữ liệu: thật không mong muốn khi tính phí khách hàng hai lần cho cùng một dịch vụ (thu quá nhiều tiền) hoặc tăng một bộ đếm hai lần (khai báo sai một metric). Trong bối cảnh này, *exactly once* có nghĩa là sắp xếp việc tính toán sao cho kết quả cuối cùng giống như thể không có lỗi nào xảy ra, ngay cả khi thao tác đó đã được thử lại do có lỗi. Chúng ta đã thảo luận một vài phương pháp để đạt được mục tiêu này.

Một trong những phương pháp hiệu quả nhất là làm cho thao tác đó trở nên *idempotent* (độc nhất)—nghĩa là, đảm bảo rằng nó có cùng một hiệu ứng, bất kể nó được thực thi một lần hay nhiều lần. Tuy nhiên, thực hiện điều này cho một thao tác vốn không có tính idempotent tự nhiên đòi hỏi sự nỗ lực và cẩn trọng. Bạn có thể cần duy trì thêm metadata bổ sung (chẳng hạn như tập hợp các operation ID đã cập nhật một giá trị) và đảm bảo fencing khi thực hiện failover từ node này sang node khác (xem “Distributed Locks and Leases”).

Ngăn chặn trùng lặp

Mô hình cần loại bỏ các bản sao (duplicates) tương tự cũng xảy ra ở nhiều nơi khác ngoài stream processing. Ví dụ, TCP sử dụng số thứ tự (sequence numbers) trên các gói tin để sắp xếp chúng đúng thứ tự tại bên nhận và để xác định xem có gói tin nào bị mất hoặc bị trùng lặp trên mạng hay không. Bất kỳ gói tin nào bị mất đều được truyền lại và bất kỳ bản sao nào cũng được loại bỏ bởi TCP stack trước khi nó chuyển dữ liệu cho một ứng dụng.

Tuy nhiên, việc loại bỏ bản sao này chỉ hoạt động trong phạm vi của một kết nối TCP duy nhất. Hãy tưởng tượng kết nối TCP là kết nối của một client tới một cơ sở dữ liệu, và nó hiện đang thực thi transaction trong Ví dụ 13-1. Trong nhiều cơ sở dữ liệu, một transaction gắn liền với một kết nối client (nếu client gửi nhiều truy vấn, cơ sở dữ liệu sẽ biết rằng chúng thuộc về cùng một transaction vì chúng được gửi trên cùng một kết nối TCP). Nếu client gặp sự cố gián đoạn mạng và timeout kết nối sau khi đã gửi COMMIT nhưng trước khi nhận được phản hồi từ máy chủ cơ sở dữ liệu, nó sẽ không biết liệu transaction đã được commit hay bị abort (chúng ta đã thấy tình huống này trong Hình 9-1).

Example 13-1. Việc chuyển tiền không có tính idempotent từ tài khoản này sang tài khoản khác

```
BEGIN TRANSACTION;  
UPDATE accounts SET balance = balance + 11.00 WHERE account_id =  
1234;  
UPDATE accounts SET balance = balance - 11.00 WHERE account_id =  
4321;  
COMMIT;
```

Client có thể kết nối lại với cơ sở dữ liệu và thử lại transaction, nhưng giờ đây nó đã nằm ngoài phạm vi loại bỏ bản sao của TCP. Vì transaction trong Ví dụ 13-1 không có tính idempotent, \$22 có thể bị chuyển đi thay vì số tiền \$11 như mong muốn. Do đó, mặc dù mã nguồn như thế này là một ví dụ tiêu chuẩn cho tính atomicity của transaction, nhưng nó không chính xác, và các ngân hàng thực tế không hoạt động như thế này [3].

Các giao thức 2PC (xem “Two-Phase Commit”) phá vỡ sự ánh xạ một-đối-một giữa một kết nối TCP và một transaction, vì chúng phải cho phép một transaction coordinator kết nối lại với cơ sở dữ liệu sau một lỗi mạng và cho cơ sở dữ liệu biết nên commit hay abort một transaction đang trong trạng thái không xác định (in-doubt transaction). Liệu điều này có đủ để đảm bảo rằng transaction sẽ chỉ được thực thi một lần duy nhất không? Rất tiếc là không.

Ngay cả khi chúng ta có thể loại bỏ các transaction trùng lặp giữa client và máy chủ cơ sở dữ liệu, chúng ta vẫn cần lo lắng về mạng giữa thiết bị của người dùng cuối và máy chủ ứng dụng. Ví dụ, nếu client của người dùng cuối là một trình duyệt web, nó có thể sử dụng một yêu cầu HTTP POST để gửi một chỉ thị tới máy chủ. Có lẽ người dùng có kết nối dữ liệu di động yếu, và họ gửi thành công yêu cầu POST, nhưng họ bị mất tín hiệu trước khi kịp nhận được phản hồi từ máy chủ.

Trong trường hợp này, người dùng có thể sẽ thấy một thông báo lỗi, và họ có thể thử lại một cách thủ công. Các trình duyệt web sẽ cảnh báo: “Bạn có chắc chắn muốn gửi lại biểu mẫu này không?”—và người dùng nói có, vì họ muốn thao tác đó được thực hiện. (Mô hình Post/Redirect/Get [43] tránh được thông báo cảnh báo này trong hoạt động bình thường, nhưng nó không giúp ích nếu yêu cầu POST bị timeout.)

Từ góc độ của máy chủ web, việc thử lại là một yêu cầu riêng biệt, và từ góc độ của cơ sở dữ liệu, nó là một transaction riêng biệt. Các cơ chế deduplication thông thường không giúp ích được gì.

Định danh duy nhất cho các yêu cầu

Để làm cho một yêu cầu có tính idempotent thông qua nhiều bước truyền thông mạng, việc chỉ dựa vào cơ chế transaction do cơ sở dữ liệu cung cấp là không đủ. Bạn cần xem xét luồng *end-to-end* của yêu cầu.

Ví dụ, bạn có thể tạo một định danh duy nhất cho mỗi yêu cầu (chẳng hạn như một UUID) và đưa nó vào như một trường ẩn (hidden form field) trong ứng dụng client, hoặc tính toán một mã hash của tất cả các trường biểu mẫu liên quan để rút ra request ID [3]. Nếu trình duyệt web gửi một yêu cầu POST hai lần, hai yêu cầu đó sẽ có cùng một request ID. Sau đó, bạn có thể truyền request ID đó xuyên suốt cho đến tận cơ sở dữ liệu và kiểm tra rằng bạn luôn chỉ thực thi duy nhất một yêu cầu với một ID đã cho, như được trình bày trong Ví dụ 13-2.

Example 13-2. Loại bỏ các yêu cầu trùng lặp bằng cách sử dụng một ID duy nhất

```
ALTER TABLE requests ADD UNIQUE (request_id);

BEGIN TRANSACTION;

INSERT INTO requests
  (request_id, from_account, to_account, amount)
VALUES ('0286FDB8-D7E1-423F-B40B-792B3608036C', 4321, 1234, 11.00);

UPDATE accounts SET balance = balance + 11.00 WHERE account_id =
1234;
UPDATE accounts SET balance = balance - 11.00 WHERE account_id =
4321;

COMMIT;
```

Đoạn mã này dựa trên một ràng buộc duy nhất (uniqueness constraint) trên cột `request_id`. Nếu một transaction cố gắng chèn một ID đã tồn tại, lệnh `INSERT` sẽ thất bại và transaction bị hủy bỏ, ngăn chặn việc nó có hiệu lực hai lần. Các cơ sở dữ liệu quan hệ nhìn chung có thể duy trì ràng buộc duy nhất một cách chính xác, ngay cả ở các mức độ *isolation* yếu (trong khi một bước kiểm tra-rời-chèn ở cấp độ ứng dụng có thể thất bại dưới mức *isolation* không phải là *serializable*, như đã thảo luận trong mục “Write Skew and Phantoms”).

Bên cạnh việc ngăn chặn các yêu cầu trùng lặp, bảng `requests` trong Ví dụ 13-2 đóng vai trò như một dạng event log, thứ có thể hữu ích cho event sourcing hoặc CDC. Việc cập nhật số dư tài khoản không nhất thiết phải diễn ra trong cùng một transaction với việc chèn event, vì chúng là dư thừa và có thể được phái sinh từ request event ở một consumer hạ nguồn—miễn là event được xử lý

exactly-once, điều này một lần nữa có thể được thực thi bằng cách sử dụng request ID.

Lập luận end-to-end

Kịch bản ngăn chặn các transaction trùng lặp này chỉ là một ví dụ về một nguyên tắc tổng quát hơn được gọi là *lập luận end-to-end* (end-to-end argument), vốn đã được Saltzer, Reed và Clark trình bày vào năm 1984 [44]:

Chức năng đang được xét đến chỉ có thể được triển khai hoàn chỉnh và chính xác với kiến thức và sự trợ giúp của ứng dụng nằm tại các điểm cuối (endpoints) của hệ thống truyền thông. Do đó, việc cung cấp chức năng đang được xét đến đó như một tính năng của chính hệ thống truyền thông là điều không thể. (Đôi khi một phiên bản không đầy đủ của chức năng do hệ thống truyền thông cung cấp có thể hữu ích như một sự cải thiện về hiệu năng.)

Trong ví dụ của chúng ta, *chức năng đang được xét đến* là ngăn chặn trùng lặp. Chúng ta đã thấy rằng TCP ngăn chặn các gói tin trùng lặp ở cấp độ kết nối TCP, và một số stream processor cung cấp cái gọi là exactly-once semantics ở cấp độ xử lý message, nhưng điều đó là không đủ để ngăn người dùng gửi một yêu cầu trùng lặp nếu yêu cầu đầu tiên bị hết thời gian chờ (timeout). Bản thân TCP, các database transaction và các stream processor không thể loại bỏ hoàn toàn các sự trùng lặp này. Giải quyết vấn đề đòi hỏi một giải pháp end-to-end: một định danh transaction được truyền xuyên suốt từ client của người dùng cuối đến cơ sở dữ liệu.

Lập luận end-to-end cũng áp dụng cho việc kiểm tra tính toàn vẹn của dữ liệu. Các checksum được tích hợp trong Ethernet, TCP và TLS có thể phát hiện sự hư hỏng của các gói tin trong mạng, nhưng chúng không thể phát hiện sự hư hỏng do lỗi trong phần mềm ở các đầu gửi và nhận của kết nối mạng, hoặc sự hư hỏng trên các đĩa nơi dữ liệu được lưu trữ. Nếu bạn muốn bắt được tất cả các nguồn gây hư hỏng dữ liệu có thể xảy ra, bạn cũng cần các checksum end-to-end.

Một lập luận tương tự cũng áp dụng cho việc mã hóa [44]. Mật khẩu trên mạng WiFi tại nhà của bạn bảo vệ chống lại những người nghe lén lưu lượng WiFi, nhưng không chống lại những kẻ tấn công ở nơi khác trên internet; TLS/SSL giữa client và server của bạn bảo vệ chống lại các kẻ tấn công mạng, nhưng không chống lại việc server bị xâm nhập. Chỉ có mã hóa và xác thực end-to-end mới có thể bảo vệ chống lại tất cả những điều này.

Mặc dù các tính năng cấp thấp (ngăn chặn trùng lặp của TCP, checksum của Ethernet, mã hóa WiFi) không thể tự mình cung cấp các tính năng end-to-end mong muốn, chúng vẫn hữu ích vì chúng làm giảm xác suất xảy ra vấn đề ở các cấp cao hơn. Ví dụ, các yêu cầu HTTP thường sẽ bị xáo trộn nếu chúng ta không có TCP sắp xếp

lại các gói tin theo đúng thứ tự. Chúng ta chỉ cần nhớ rằng các tính năng độ tin cậy cấp thấp tự thân chúng không đủ để đảm bảo tính chính xác end-to-end.

Áp dụng tư duy end-to-end trong các hệ thống dữ liệu

Điều này đưa chúng ta trở lại luận điểm ban đầu: chỉ vì một ứng dụng sử dụng một hệ thống dữ liệu cung cấp các thuộc tính an toàn tương đối mạnh, chẳng hạn như các transaction có tính serializable, không có nghĩa là ứng dụng đó được đảm bảo sẽ không bị mất hoặc hư hỏng dữ liệu. Bản thân ứng dụng cũng cần thực hiện các biện pháp end-to-end, chẳng hạn như ngăn chặn trùng lặp.

Thật đáng tiếc, bởi vì các cơ chế fault-tolerance (chịu lỗi) rất khó để thực hiện chính xác. Các cơ chế độ tin cậy ở cấp thấp, chẳng hạn như trong TCP, hoạt động khá tốt, vì vậy các lỗi ở cấp cao hơn còn lại xảy ra khá hiếm. Sẽ thật tuyệt nếu chúng ta có thể gói gọn các bộ máy fault-tolerance cấp cao vào một abstraction (sự trừu tượng hóa) để mã nguồn ứng dụng không cần phải bận tâm về nó—nhưng có vẻ như chúng ta vẫn chưa tìm ra được abstraction phù hợp.

Từ lâu, các transaction đã được xem là một abstraction hữu ích. Như đã thảo luận trong Chương 8, chúng tiếp nhận một loạt các vấn đề có thể xảy ra (ghi đồng thời, vi phạm ràng buộc, crash, gián đoạn mạng, lỗi đĩa) và thu gọn chúng thành hai kết quả có thể xảy ra: commit hoặc abort. Đó là một sự đơn giản hóa khổng lồ của mô hình lập trình, nhưng bấy nhiêu vẫn là chưa đủ.

Các transaction rất tốn kém, đặc biệt là khi chúng liên quan đến các công nghệ lưu trữ không đồng nhất (xem mục “Distributed Transactions Across Different Systems”). Khi chúng ta từ chối sử dụng distributed transactions vì chúng quá tốn kém, chúng ta sẽ kết thúc bằng việc phải tự triển khai lại các cơ chế fault-tolerance trong mã nguồn ứng dụng. Như nhiều ví dụ xuyên suốt cuốn sách này đã chỉ ra, việc suy luận về concurrency và partial failure là rất khó khăn và trái với trực giác, do đó hầu hết các cơ chế ở cấp ứng dụng đều không hoạt động chính xác. Hệ quả là dữ liệu bị mất hoặc bị hỏng.

Vì những lý do này, việc khám phá các abstraction về fault-tolerance nhằm giúp việc cung cấp các tính chất đúng đắn end-to-end đặc thù cho ứng dụng trở nên dễ dàng, đồng thời vẫn duy trì được hiệu năng tốt và các đặc tính vận hành trong một môi trường phân tán quy mô lớn, là điều rất đáng để thực hiện.

Enforcing Constraints

Hãy cùng suy nghĩ về tính đúng đắn trong bối cảnh của các ý tưởng xoay quanh việc unbundling các cơ sở dữ liệu. Chúng ta đã thấy rằng việc ngăn chặn trùng lặp end-to-end có thể đạt được bằng một request ID được truyền xuyên suốt từ client đến cơ sở dữ liệu nơi ghi dữ liệu. Vậy còn các loại ràng buộc khác thì sao?

Cụ thể, hãy tập trung vào các ràng buộc về tính duy nhất (uniqueness constraints), chẳng hạn như ràng buộc mà chúng ta đã đưa vào trong Ví dụ 13-2. Trong mục “Constraints and uniqueness guarantees”, chúng ta đã thấy một vài ví dụ khác về các feature của ứng dụng cần thực thi tính duy nhất: một username hoặc địa chỉ email phải định danh duy nhất một người dùng, một dịch vụ lưu trữ tệp không thể có nhiều hơn một tệp cùng tên, và hai người không thể đặt cùng một chỗ ngồi trên một chuyến bay hoặc trong một rạp hát.

Các loại ràng buộc khác cũng rất tương tự—ví dụ, đảm bảo rằng số dư tài khoản không bao giờ bị âm, bạn không bán nhiều mặt hàng hơn số lượng hiện có trong kho, hoặc một phòng họp không có các lịch đặt trùng nhau. Các kỹ thuật thực thi tính duy nhất thường cũng có thể được sử dụng cho các loại ràng buộc này.

Uniqueness constraints yêu cầu consensus

Trong Chương 10, chúng ta đã thấy rằng trong một thiết lập phân tán, việc thực thi một uniqueness constraint đòi hỏi consensus. Nếu nhiều request đồng thời có cùng một giá trị, hệ thống bằng cách nào đó cần phải quyết định xem thao tác xung đột nào được chấp nhận và từ chối các thao tác còn lại vì vi phạm ràng buộc.

Cách phổ biến nhất để đạt được consensus này là biến một node duy nhất thành leader và giao cho nó chịu trách nhiệm đưa ra mọi quyết định. Điều này hoạt động tốt chừng nào bạn không ngại việc dồn tất cả các request qua một node duy nhất (ngay cả khi client ở phía bên kia thế giới), và chừng nào node đó không bị lỗi. Các thuật toán consensus như Raft giải quyết vấn đề bầu chọn một leader mới một cách an toàn nếu leader hiện tại đã lỗi (hoặc được cho là đã lỗi do vấn đề mạng) và ngăn chặn tình trạng split brain.

Việc kiểm tra tính duy nhất có thể được scale out bằng cách sharding dựa trên giá trị cần phải duy nhất. Ví dụ, nếu bạn cần đảm bảo tính duy nhất theo request ID, như trong Ví dụ 13-2, bạn có thể đảm bảo rằng tất cả các request có cùng request ID đều được routed đến cùng một shard. Nếu bạn cần các username phải là duy nhất, bạn có thể shard theo hash của username.

Tuy nhiên, việc replication multi-leader bất đồng bộ bị loại trừ, bởi vì các leader khác nhau có thể đồng thời chấp nhận các write xung đột, và do đó các giá trị không còn là duy nhất nữa. Nếu bạn muốn có thể từ chối ngay lập tức bất kỳ write nào vi phạm ràng buộc, thì việc điều phối đồng bộ là không thể tránh khỏi [45].

Tính duy nhất trong messaging dựa trên log

Một shared log đảm bảo rằng tất cả các consumer đều thấy các message theo cùng một thứ tự (đảm bảo total order broadcast mà chúng ta đã thiết lập trong mục “The Many Faces of Consensus”, tương đương với consensus). Trong cách tiếp cận cơ sở dữ liệu unbundled với messaging dựa trên log, chúng ta có thể sử dụng một phương pháp rất tương tự để thực thi các ràng buộc về tính duy nhất.

Một stream processor tiêu thụ tất cả các message trong một log shard một cách tuần tự trên một single thread. Do đó, nếu log được sharding dựa trên giá trị cần phải duy nhất, một stream processor có thể quyết định một cách rõ ràng và xác định xem thao tác nào trong số nhiều thao tác xung đột đã đến trước trong log. Ví dụ, trong trường hợp nhiều người dùng cố gắng yêu cầu cùng một username [46]:

1. Mỗi yêu cầu cho một username được mã hóa thành một message và được append vào một shard được xác định bởi hash của username đó.
2. Một stream processor đọc tuần tự các yêu cầu trong log, sử dụng một cơ sở dữ liệu cục bộ để theo dõi xem những username nào đã được sử dụng. Đối với mỗi yêu cầu cho một username còn trống, nó ghi lại tên đó là đã được sử dụng và phát ra một success message tới một output stream. Đối với mỗi yêu cầu cho một username đã được sử dụng, nó phát ra một rejection message tới một output stream.
3. Client đã yêu cầu username sẽ theo dõi output stream và chờ đợi một success message hoặc rejection message tương ứng với yêu cầu của mình.

Thuật toán này giống với cấu trúc để đạt được consensus bằng cách sử dụng một shared log mà chúng ta đã thấy trong Chương 10. Nó có khả năng mở rộng dễ dàng tới throughput yêu cầu lớn bằng cách tăng số lượng shard, vì mỗi shard có thể được xử lý độc lập.

Cách tiếp cận này không chỉ hoạt động cho các ràng buộc về tính duy nhất mà còn cho nhiều loại ràng buộc khác. Nguyên tắc cơ bản của nó là bất kỳ write nào có thể xung đột đều được điều hướng tới cùng một shard và được xử lý tuần tự. Định nghĩa về một sự xung đột có thể phụ thuộc vào ứng dụng, nhưng stream processor có thể sử dụng logic tùy ý để kiểm định một yêu cầu.

Xử lý yêu cầu đa shard (multishard)

Việc đảm bảo một thao tác được thực thi một cách atomic, trong khi vẫn thỏa mãn các ràng buộc, trở nên thú vị hơn khi có sự tham gia của nhiều shard. Trong Ví dụ 13-2, có khả năng có ba shard: một shard chứa request ID, một shard chứa tài khoản người thụ hưởng, và một shard chứa tài khoản người thanh toán. Không có lý do gì ba thứ đó phải nằm trong cùng một shard, vì chúng đều độc lập với nhau.

Trong cách tiếp cận cơ sở dữ liệu truyền thống, việc thực thi transaction này sẽ yêu cầu một atomic commit trên cả ba shard, điều này về cơ bản buộc nó phải tuân theo một total order đối với tất cả các transaction khác trên bất kỳ shard nào trong số đó. Vì hiện tại đã có sự điều phối cross-shard, các shard khác nhau không còn có thể được xử lý độc lập, do đó throughput có khả năng sẽ bị ảnh hưởng.

Tuy nhiên, độ chính xác tương đương có thể đạt được mà không cần các transaction cross-shard bằng cách sử dụng các sharded log và stream processor. Hình 13-2 cho

thấy một ví dụ về một giao dịch thanh toán cần kiểm tra xem tài khoản nguồn có đủ tiền hay không và, nếu có, sẽ chuyển một số tiền tới tài khoản đích một cách atomic đồng thời khấu trừ phí.

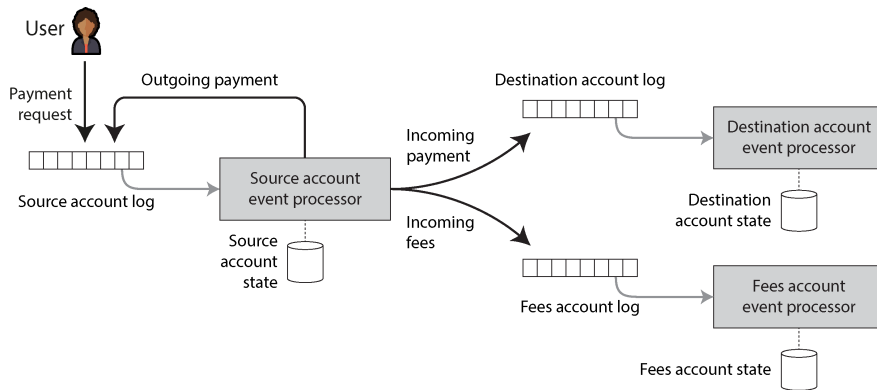


Figure 13-2. Kiểm tra xem một tài khoản nguồn có đủ tiền hay không và thực hiện chuyển tiền một cách nguyên tử (atomically) tới một tài khoản đích và một tài khoản phí, bằng cách sử dụng event logs và các stream processor

Quá trình này hoạt động như sau [47]:

1. Yêu cầu chuyển tiền từ tài khoản nguồn đến tài khoản đích được client của người dùng cấp một request ID duy nhất và được thêm vào một log shard dựa trên ID của tài khoản nguồn.
2. Một stream processor đọc log của các yêu cầu và duy trì một database chứa trạng thái của tài khoản nguồn cùng với các ID của những yêu cầu mà nó đã xử lý. Nội dung của database này hoàn toàn được derived (phái sinh) từ log. Khi stream processor gặp một yêu cầu có ID mà nó chưa từng thấy trước đó, nó sẽ kiểm tra trong database cục bộ xem tài khoản nguồn có đủ tiền để thực hiện việc chuyển khoản hay không.

Nếu có, nó cập nhật database cục bộ để giữ lại số tiền thanh toán trên tài khoản nguồn và phát các event tới nhiều log khác: một outgoing payment event tới log shard của tài khoản nguồn (log đầu vào của chính nó), một incoming payment event tới log shard của tài khoản đích, và một incoming payment event tới log shard của tài khoản phí. Request ID gốc được bao gồm trong các event được phát ra đó.

3. Cuối cùng, outgoing payment event được chuyển trả về cho bộ xử lý tài khoản nguồn (bộ xử lý này có thể đã nhận được các event không liên quan trong thời gian chờ). Stream processor nhận diện dựa trên request ID rằng đây là khoản thanh toán mà nó đã giữ lại trước đó và thực thi việc thanh toán, đồng thời cập

nhật lại trạng thái cục bộ cho tài khoản nguồn. Nó bỏ qua các bản trùng lặp dựa trên request ID.

1. Các log shard cho tài khoản đích và tài khoản phí được tiêu thụ bởi các tác vụ stream processing độc lập. Khi chúng nhận được một incoming payment event, chúng cập nhật trạng thái cục bộ để phản ánh khoản thanh toán, và chúng deduplicate các event dựa trên request ID.

Hình 13-2 cho thấy ba tài khoản nằm trong ba shard riêng biệt, nhưng chúng cũng có thể nằm trong cùng một shard—điều này không quan trọng. Các yêu cầu duy nhất là các sự kiện của bất kỳ tài khoản nào cũng phải được xử lý nghiêm ngặt theo thứ tự log với ngữ nghĩa at-least-once và các stream processor phải có tính deterministic.

Ví dụ, hãy xét điều gì sẽ xảy ra nếu bộ xử lý tài khoản nguồn (source account processor) bị crash trong khi đang xử lý một yêu cầu thanh toán. Các thông điệp đầu ra có thể đã hoặc chưa được phát ra trước khi sự cố crash xảy ra. Sau khi khôi phục từ crash, bộ xử lý sẽ xử lý lại chính yêu cầu đó (do ngữ nghĩa at-least-once), và nó sẽ đưa ra cùng một quyết định về việc có cho phép thanh toán hay không (vì nó có tính deterministic). Do đó, nó sẽ phát ra các thông điệp đầu ra giống hệt nhau với cùng một request ID tới các shard tài khoản outgoing, incoming và fees. Nếu các thông điệp là trùng lặp, các consumer ở hạ nguồn sẽ bỏ qua chúng dựa trên request ID.

Tính atomicity trong hệ thống này không đến từ các transaction, mà đến từ thực tế là việc ghi event yêu cầu ban đầu vào log tài khoản nguồn là một hành động atomic. Một khi event duy nhất đó đã nằm trong log, tất cả các event ở hạ nguồn cuối cùng cũng sẽ được ghi lại—có thể là sau khi các stream processor đã khôi phục từ crash, và có thể kèm theo các bản trùng lặp, nhưng cuối cùng chúng cũng sẽ xuất hiện.

Với ngữ nghĩa exactly-once, ví dụ này trở nên dễ triển khai hơn, vì ngữ nghĩa này đảm bảo rằng trạng thái cục bộ (local state) của stream processor nhất quán với tập hợp các thông điệp mà nó đã xử lý. Do đó, nếu nó bị crash và xử lý lại một số thông điệp, trạng thái cục bộ của nó cũng được thiết lập lại về trạng thái trước khi các thông điệp đó được xử lý.

Nếu người dùng trong Hình 13-2 muốn biết liệu lệnh chuyển tiền của họ có được phê duyệt hay không, họ có thể subscribe vào shard log tài khoản nguồn và chờ đợi event thanh toán outgoing. Để thông báo rõ ràng cho người dùng nếu số dư không đủ, stream processor có thể phát một event "declined payment" tới shard log đó.

Bằng cách chia nhỏ transaction đa shard thành nhiều giai đoạn được sharding khác nhau và sử dụng request ID xuyên suốt (end-to-end), chúng ta đạt được cùng một tính chất đúng đắn (mọi yêu cầu được áp dụng chính xác một lần cho cả tài khoản người trả và người nhận), ngay cả khi có lỗi xảy ra, mà không cần sử dụng giao thức atomic commit.

Tính kịp thời và tính toàn vẹn

Một tính chất thuận tiện của nhiều hệ thống transactional là ngay khi một transaction commit, các thao tác ghi của nó sẽ hiển thị ngay lập tức với các transaction khác. Tính chất này được chính thức hóa là *strict serializability* (được thảo luận trong mục “Linearizability Versus Serializability”).

Điều này không đúng khi thực hiện unbundling một thao tác qua nhiều giai đoạn của các stream processor. Các consumer của một log được thiết kế theo cơ chế bất đồng bộ, vì vậy một producer không đợi cho đến khi thông điệp của nó được xử lý bởi các consumer. Tuy nhiên, một client có thể đợi một thông điệp xuất hiện trên một output stream, giống như người dùng đợi event thanh toán outgoing hoặc event thanh toán bị từ chối trong Hình 13-2, điều này phụ thuộc vào việc liệu có đủ tiền trong tài khoản nguồn hay không.

Trong ví dụ này, tính đúng đắn của việc kiểm tra số dư tài khoản nguồn không phụ thuộc vào việc người dùng thực hiện yêu cầu có đợi kết quả hay không. Việc chờ đợi chỉ nhằm mục đích thông báo đồng bộ cho người dùng liệu việc thanh toán có thành công hay không; thông báo này được tách rời khỏi các tác động của việc xử lý yêu cầu.

Nói một cách tổng quát hơn, thuật ngữ *consistency* gộp hai yêu cầu mà chúng ta nên xem xét riêng biệt:

Tính kịp thời

Tính kịp thời nghĩa là đảm bảo người dùng quan sát hệ thống ở một trạng thái cập nhật nhất. Chúng ta đã thấy trước đó rằng nếu một người dùng đọc từ một bản sao dữ liệu cũ (stale copy), họ có thể quan sát thấy nó ở một trạng thái không nhất quán (xem mục “Problems with Replication Lag”). Tuy nhiên, sự không nhất quán đó chỉ là tạm thời, và cuối cùng nó sẽ được giải quyết chỉ bằng cách chờ đợi và thử lại.

Định lý CAP sử dụng “consistency” theo nghĩa của linearizability, vốn là một cách mạnh mẽ để đạt được tính kịp thời. Các tính chất kịp thời yếu hơn, như read-after-write consistency, cũng có thể hữu ích.

Integrity (Tính toàn vẹn)

Integrity (tính toàn vẹn) nghĩa là không có sự hư hỏng—không mất dữ liệu, và không có dữ liệu mâu thuẫn hoặc sai lệch. Cụ thể, nếu một *derived dataset* (tập dữ liệu phái sinh) được duy trì như một *view* đối với dữ liệu cơ sở, thì việc phái sinh đó phải chính xác. Ví dụ, một *database index* phải phản ánh chính xác nội dung của *database*—một *index* bị thiếu các bản ghi sẽ không có nhiều giá trị.

Nếu integrity bị vi phạm, sự không nhất quán sẽ là vĩnh viễn; trong hầu hết các trường hợp, việc chờ đợi và thử lại sẽ không thể khắc phục được tình trạng hư

hồng database. Thay vào đó, cần phải thực hiện kiểm tra và sửa chữa một cách rõ ràng. Trong ngữ cảnh của các ACID transaction, "consistency" thường được hiểu là một loại khái niệm về integrity đặc thù cho từng ứng dụng. Atomicity và durability là những công cụ quan trọng để bảo vệ integrity.

Dưới dạng khẩu hiệu: các vi phạm về tính kịp thời (timeliness) được cho phép trong eventual consistency, trong khi các vi phạm về integrity dẫn đến sự không nhất quán vĩnh viễn.

Trong hầu hết các ứng dụng, integrity quan trọng hơn nhiều so với tính kịp thời. Các vi phạm về tính kịp thời có thể gây khó chịu và bối rối, nhưng các vi phạm về integrity có thể gây ra thảm họa.

Ví dụ, trên sao kê thẻ tín dụng của bạn, sẽ không có gì đáng ngạc nhiên nếu một transaction mà bạn thực hiện trong vòng 24 giờ qua vẫn chưa xuất hiện. Việc các hệ thống này có một độ trễ nhất định là điều bình thường. Chúng ta biết rằng các ngân hàng thực hiện đối soát và quyết toán các transaction một cách bất đồng bộ, và tính kịp thời không quá quan trọng ở đây [3]. Tuy nhiên, sẽ rất tồi tệ nếu số dư sao kê không bằng tổng các transaction cộng với số dư sao kê trước đó (lỗi trong việc tính tổng), hoặc nếu một transaction bị tính phí cho bạn nhưng không được thanh toán cho người bán (mất tiền). Những vấn đề như vậy sẽ là các vi phạm đối với integrity của hệ thống.

Tính chính xác của các hệ thống dataflow

Các ACID transaction thường cung cấp cả sự đảm bảo về tính kịp thời (ví dụ: linearizability) và integrity (ví dụ: atomic commit). Do đó, nếu bạn tiếp cận tính chính xác của ứng dụng từ góc độ của các ACID transaction, thì sự phân biệt giữa tính kịp thời và integrity là khá không đáng kể.

Mặt khác, một đặc tính thú vị của các hệ thống dataflow dựa trên event mà chúng ta đã thảo luận trong chương này là chúng tách rời (decouple) tính kịp thời và integrity. Khi xử lý các event stream một cách bất đồng bộ, không có sự đảm bảo nào về tính kịp thời, trừ khi bạn xây dựng các consumer một cách rõ ràng để chờ cho đến khi một message đến trước khi trả về kết quả. Ví dụ, một người dùng có thể yêu cầu thanh toán và sau đó đọc trạng thái tài khoản của họ trước khi stream processor thực thi yêu cầu đó; người dùng sẽ không thấy khoản thanh toán mà họ vừa yêu cầu.

Tuy nhiên, thực tế integrity là trọng tâm của các hệ thống streaming. Như chúng ta đã thấy, semantics exactly-once hoặc effectively-once là một cơ chế để bảo vệ integrity. Nếu một event bị mất hoặc có hiệu lực hai lần, integrity của một hệ thống dữ liệu có thể bị vi phạm. Do đó, việc truyền tin fault-tolerant và ngăn chặn trùng lặp (ví dụ: các thao tác idempotent) là quan trọng để duy trì integrity của một hệ thống dữ liệu trước các fault.

Như chúng ta đã thấy ở mục trước, các hệ thống stream processing đáng tin cậy có thể bảo vệ integrity mà không yêu cầu các distributed transaction và một giao thức atomic commit, điều này có nghĩa là chúng có khả năng đạt được độ chính xác tương đương với hiệu năng và tính mạnh mẽ trong vận hành tốt hơn nhiều. Chúng ta đạt được integrity này thông qua sự kết hợp của các cơ chế:

- Biểu diễn nội dung của thao tác write dưới dạng một message duy nhất, thứ có thể được ghi một cách atomic một cách dễ dàng—một cách tiếp cận rất phù hợp với event sourcing
- Phái sinh tất cả các cập nhật trạng thái khác từ message duy nhất đó thông qua các hàm phái sinh xác định (deterministic), tương tự như các stored procedure
- Truyền một request ID do client tạo qua tất cả các tầng xử lý này, cho phép ngăn chặn trùng lặp và đảm bảo tính idempotent từ đầu đến cuối (end-to-end)
- Làm cho các message trở nên immutable và cho phép derived data được xử lý lại theo thời gian, giúp việc khôi phục từ các lỗi (bugs) trở nên dễ dàng hơn

Các ràng buộc được diễn giải lỏng lẻo

Như đã thảo luận trước đó, việc thực thi một ràng buộc tính duy nhất (uniqueness constraint) đòi hỏi sự consensus, thường được triển khai bằng cách dẫn dắt tất cả các event trong một shard cụ thể đi qua một node duy nhất. Hạn chế này là không thể tránh khỏi nếu chúng ta muốn có dạng ràng buộc tính duy nhất truyền thống, và stream processing không thể vượt qua được nó.

Tuy nhiên, nhiều ứng dụng thực tế có yêu cầu nghiệp vụ cho phép vi phạm những gì mà bạn có thể coi là các ràng buộc cứng (hard constraints):

- Nếu khách hàng đặt mua nhiều mặt hàng hơn số lượng bạn có trong kho, bạn có thể đặt thêm hàng, xin lỗi vì sự chậm trễ và tặng họ một mã giảm giá. Điều này cũng giống như những gì bạn phải làm nếu, chẳng hạn, một chiếc xe nâng cán qua một số mặt hàng trong kho của bạn, khiến số lượng hàng tồn kho ít hơn so với những gì bạn nghĩ là mình đang có [3]. Do đó, workflow xin lỗi dù sao cũng cần phải là một phần trong các quy trình nghiệp vụ của bạn để xử lý các sự cố như vậy, và một ràng buộc cứng về số lượng mặt hàng trong kho có thể là không cần thiết.
- Tương tự, nhiều hãng hàng không đặt vé quá số ghế (overbook) với kỳ vọng rằng một số hành khách sẽ lỡ chuyến bay, và nhiều khách sạn đặt phòng quá số lượng hiện có, kỳ vọng rằng một số khách sẽ hủy phòng. Trong những trường hợp này, ràng buộc "mỗi ghế một người" bị vi phạm một cách có chủ đích vì lý do kinh doanh, và các quy trình bồi thường (hoàn tiền, nâng hạng, cung cấp phòng miễn phí tại một khách sạn lân cận) được thiết lập để xử lý nhu cầu vượt quá nguồn

cung. Ngay cả khi không có việc đặt quá chỗ nào xảy ra, các quy trình xin lỗi và bồi thường vẫn sẽ cần thiết để xử lý các sự kiện như chuyến bay bị hủy do thời tiết xấu hoặc nhân viên đình công—việc khắc phục các vấn đề như vậy chỉ là một phần bình thường của kinh doanh [3].

- Nếu ai đó rút nhiều tiền hơn số tiền họ có trong tài khoản, ngân hàng có thể tính phí thấu chi và yêu cầu họ trả lại số tiền còn nợ. Bằng cách giới hạn tổng số tiền rút mỗi ngày, rủi ro đối với ngân hàng sẽ được kiểm soát trong phạm vi nhất định.
- Trong các hệ thống tích hợp dữ liệu giữa các tổ chức, sự không nhất quán chắc chắn sẽ phát sinh, và các cơ chế sửa lỗi là cần thiết để xử lý chúng. Như đã lưu ý trong mục "Các trường hợp sử dụng Batch", việc quyết toán thanh toán giữa các ngân hàng là một ví dụ về điều này.

Trong nhiều bối cảnh kinh doanh, việc tạm thời vi phạm một ràng buộc và sửa chữa nó sau đó bằng cách xin lỗi là điều có thể chấp nhận được. Loại thay đổi này để sửa chữa một sai sót được gọi là một *compensating transaction* (giao dịch bù đắp) [48, 49]. Chi phí cho việc xin lỗi (xét về mặt tiền bạc hoặc uy tín) khác nhau, nhưng nó thường khá thấp; bạn không thể thu hồi một email đã gửi, nhưng bạn có thể gửi một email tiếp theo để đính chính. Nếu bạn vô tình tính phí thẻ tín dụng hai lần, bạn có thể hoàn lại một trong hai lần tính phí, và chi phí đối với bạn chỉ là phí xử lý và có lẽ là một lời phàn nàn từ khách hàng. Một khi tiền đã được rút ra từ máy ATM, bạn không thể lấy lại trực tiếp, mặc dù về nguyên tắc bạn có thể cử người thu hồi nợ để lấy lại tiền nếu tài khoản bị thấu chi và khách hàng không chịu hoàn trả.

Việc chi phí xin lỗi có thể chấp nhận được hay không là một quyết định kinh doanh. Nếu nó có thể chấp nhận được, mô hình truyền thống là kiểm tra tất cả các ràng buộc trước khi thực hiện ghi dữ liệu là sự hạn chế không cần thiết. Việc tiến hành một thao tác ghi một cách lạc quan (optimistically) và kiểm tra ràng buộc sau đó hoàn toàn có thể là một hướng đi hợp lý. Bạn vẫn có thể đảm bảo rằng việc validation diễn ra trước khi thực hiện các hành động mà việc khắc phục sẽ gây tổn kém, nhưng điều đó không có nghĩa là bạn phải validate trước khi thực hiện ghi dữ liệu.

Những ứng dụng này *thực sự* yêu cầu tính toàn vẹn (integrity). Bạn sẽ không muốn mất một lượt đặt chỗ hoặc để tiền biến mất do sự không khớp giữa các khoản ghi có và ghi nợ. Nhưng chúng *không* yêu cầu tính kịp thời trong việc thực thi ràng buộc. Nếu bạn đã bán nhiều mặt hàng hơn số lượng có trong kho, bạn có thể khắc phục vấn đề sau đó. Làm như vậy tương tự như các phương pháp giải quyết xung đột mà chúng ta đã thảo luận trong mục "Giải quyết các thao tác ghi xung đột (Dealing with Conflicting Writes)".

Các hệ thống dữ liệu tránh điều phối

Bây giờ chúng ta đã có hai quan sát thú vị:

- Các hệ thống dataflow có thể duy trì các đảm bảo về tính toàn vẹn trên dữ liệu phái sinh (derived data) mà không cần atomic commit, linearizability, hay sự điều phối cross-shard đồng bộ.
- Mặc dù các ràng buộc về tính duy nhất nghiêm ngặt đòi hỏi tính kịp thời và sự điều phối, nhiều ứng dụng vẫn chấp nhận các ràng buộc lỏng lẻo—những thứ có thể bị vi phạm tạm thời và được sửa chữa sau đó—miễn là tính toàn vẹn được bảo toàn xuyên suốt.

Tổng hợp lại, những quan sát này có nghĩa là các hệ thống dataflow có thể cung cấp các dịch vụ quản lý dữ liệu cho nhiều ứng dụng mà không yêu cầu sự điều phối, trong khi vẫn đưa ra được các đảm bảo về tính toàn vẹn mạnh mẽ. Các hệ thống *tránh điều phối* (*coordination-avoiding*) như vậy có sức hấp dẫn lớn; chúng có thể đạt được hiệu năng và khả năng chịu lỗi tốt hơn so với các hệ thống cần thực hiện điều phối đồng bộ [45].

Ví dụ, một hệ thống như vậy có thể hoạt động phân tán trên nhiều trung tâm dữ liệu trong cấu hình multi-leader, thực hiện replication không đồng bộ giữa các vùng. Bất kỳ trung tâm dữ liệu nào cũng có thể tiếp tục hoạt động độc lập với các trung tâm khác, vì không yêu cầu sự điều phối cross-region đồng bộ. Một hệ thống như vậy sẽ có các đảm bảo về tính kịp thời yếu—nó không thể đạt được tính linearizable nếu không đưa vào sự điều phối—nhưng nó vẫn có thể có các đảm bảo về tính toàn vẹn mạnh mẽ.

Trong bối cảnh này, các transaction có tính serializable vẫn hữu ích như một phần của việc duy trì trạng thái phái sinh, nhưng chúng có thể được chạy ở một phạm vi nhỏ nơi chúng hoạt động hiệu quả [6]. Các distributed transaction không đồng nhất như XA transactions là không cần thiết. Sự điều phối đồng bộ vẫn có thể được đưa vào ở những nơi cần thiết (ví dụ, để thực thi các ràng buộc nghiêm ngặt trước một thao tác mà sau đó không thể phục hồi), nhưng không cần thiết mọi thứ đều phải trả giá cho sự điều phối nếu chỉ một phần nhỏ của ứng dụng cần đến nó [32].

Một cách khác để nhìn nhận về sự điều phối và các ràng buộc là chúng làm giảm số lượng lời xin lỗi mà bạn phải đưa ra do sự không nhất quán, nhưng cũng có khả năng làm giảm hiệu năng và tính khả dụng của hệ thống, và do đó có thể làm tăng số lượng lời xin lỗi mà bạn phải đưa ra do các sự cố ngừng hoạt động (outages). Bạn không thể giảm số lượng lời xin lỗi xuống bằng không, nhưng bạn có thể hướng tới việc tìm ra mức đánh đổi (trade-off) tốt nhất cho nhu cầu của mình—điểm cân bằng (sweet spot) mà ở đó không có quá nhiều sự không nhất quán cũng như không có quá nhiều vấn đề về tính khả dụng.

Tin tưởng, nhưng cần Kiểm chứng

Tất cả những thảo luận của chúng ta về tính chính xác, tính toàn vẹn và khả năng chịu lỗi đều dựa trên giả định rằng một số thứ có thể xảy ra sai sót, nhưng những thứ khác thì không. Chúng ta gọi những giả định như vậy là *mô hình hệ thống (system model)* (xem “System Model and Reality”). Ví dụ, chúng ta nên giả định rằng các tiến trình có thể bị crash, máy chủ có thể đột ngột mất điện, và mạng có thể làm trễ hoặc làm mất các message một cách tùy ý. Chúng ta cũng có thể giả định rằng dữ liệu được ghi vào đĩa không bị mất sau fsync, dữ liệu trong bộ nhớ không bị hỏng, và lệnh nhân của CPU luôn trả về kết quả chính xác.

Những giả định này khá hợp lý, vì chúng đúng trong hầu hết thời gian, và sẽ rất khó để thực hiện bất cứ việc gì nếu chúng ta phải liên tục lo lắng về việc máy tính của mình mắc lỗi. Theo truyền thống, các mô hình hệ thống áp dụng cách tiếp cận nhị phân đối với các lỗi: chúng ta giả định rằng một số thứ có thể xảy ra và những thứ khác thì không bao giờ xảy ra. Trong thực tế, đó là vấn đề về xác suất: một số thứ có khả năng xảy ra cao hơn, những thứ khác thì thấp hơn. Câu hỏi là liệu các vi phạm đối với giả định của chúng ta có xảy ra đủ thường xuyên để chúng ta có thể gặp phải chúng trong thực tế hay không.

Chúng ta đã thấy rằng dữ liệu có thể bị hỏng trong bộ nhớ (xem “Hardware and Software Faults”), trên đĩa (xem “Replication and Durability”), và trên mạng (xem “Weak forms of lying”). Có lẽ đây là điều mà chúng ta nên chú ý nhiều hơn? Nếu bạn đang vận hành ở quy mô đủ lớn, ngay cả những điều rất khó xảy ra cũng vẫn sẽ xảy ra.

Duy trì tính toàn vẹn trước các lỗi phần mềm

Bên cạnh các vấn đề phần cứng như vậy, luôn tồn tại rủi ro về lỗi phần mềm (software bugs), những lỗi vốn không thể bị phát hiện bởi các checksum ở tầng thấp hơn của mạng, bộ nhớ hoặc hệ thống tệp. Ngay cả các phần mềm cơ sở dữ liệu được sử dụng rộng rãi cũng có lỗi—ví dụ, các phiên bản trước đây của MySQL đã không duy trì chính xác các ràng buộc tính duy nhất (uniqueness constraints) [50], và mức isolation serializable của PostgreSQL đã từng xuất hiện các bất thường về write skew trong quá khứ [51], mặc dù MySQL và PostgreSQL là những cơ sở dữ liệu mạnh mẽ và đáng tin cậy, đã được nhiều người kiểm chứng qua nhiều năm. Trong các phần mềm ít trưởng thành hơn, tình hình có khả năng sẽ tồi tệ hơn nhiều.

Bất chấp những nỗ lực đáng kể trong việc thiết kế, kiểm thử và xem xét kỹ lưỡng, các lỗi vẫn có thể len lỏi vào. Mặc dù chúng hiếm khi xảy ra, và cuối cùng sẽ được tìm thấy và sửa chữa, nhưng vẫn có một khoảng thời gian mà các lỗi như vậy có thể làm hỏng dữ liệu.

Khi nói đến mã nguồn ứng dụng, chúng ta phải giả định có nhiều lỗi hơn, vì hầu hết các ứng dụng không nhận được sự xem xét và kiểm thử nhiều như mã nguồn cơ sở

dữ liệu. Nhiều ứng dụng thậm chí không sử dụng chính xác các feature mà cơ sở dữ liệu cung cấp để bảo vệ tính toàn vẹn, chẳng hạn như các ràng buộc khóa ngoại (foreign-key) hoặc ràng buộc tính duy nhất (uniqueness constraints) [25].

Tính nhất quán (consistency) theo nghĩa của ACID dựa trên ý tưởng rằng cơ sở dữ liệu bắt đầu ở một trạng thái nhất quán, và một transaction sẽ chuyển đổi nó từ trạng thái nhất quán này sang một trạng thái nhất quán khác. Do đó, chúng ta kỳ vọng cơ sở dữ liệu luôn ở trong trạng thái nhất quán. Tuy nhiên, khái niệm này chỉ có ý nghĩa nếu chúng ta giả định rằng transaction không có lỗi. Nếu ứng dụng sử dụng cơ sở dữ liệu không đúng cách theo một cách nào đó—ví dụ, sử dụng một mức isolation yếu một cách không an toàn—thì tính toàn vẹn của cơ sở dữ liệu không thể được đảm bảo.

Đừng chỉ tin tưởng mù quáng vào những gì chúng hứa hẹn

Với việc cả phần cứng và phần mềm không phải lúc nào cũng đáp ứng được các lý tưởng của chúng ta, việc hỏng dữ liệu dường như là điều không thể tránh khỏi sớm hay muộn. Do đó, ít nhất chúng ta nên có một cách để tìm ra liệu dữ liệu có bị hỏng hay không để có thể sửa chữa và cố gắng truy tìm nguồn gốc của lỗi. Việc kiểm tra tính toàn vẹn của dữ liệu được gọi là *auditing* (kiểm toán).

Như đã thảo luận trong mục “Lợi ích của các sự kiện bất biến (immutable events)”, auditing không chỉ dành cho các ứng dụng tài chính. Tuy nhiên, khả năng kiểm toán (auditability) rất quan trọng trong tài chính chính vì mọi người đều biết rằng sai sót luôn xảy ra, và tất cả chúng ta đều nhận thấy nhu cầu có thể phát hiện và sửa chữa các vấn đề.

Các hệ thống trưởng thành cũng có xu hướng xem xét khả năng xảy ra những điều khó tin và quản lý rủi ro đó. Ví dụ, các hệ thống lưu trữ quy mô lớn như HDFS và Amazon S3 không hoàn toàn tin tưởng vào các đĩa cứng. Các hệ thống này chạy các tiến trình chạy ngầm để liên tục đọc lại các tệp, so sánh chúng với các replica khác, và di chuyển các tệp từ đĩa này sang đĩa khác nhằm giảm thiểu rủi ro về lỗi hỏng dữ liệu âm thầm (silent corruption) [52, 53].

Nếu bạn muốn chắc chắn rằng dữ liệu của mình vẫn còn đó, bạn phải đọc và kiểm tra nó. Hầu hết thời gian nó vẫn sẽ ở đó, nhưng nếu không, bạn sẽ muốn tìm ra sớm thay vì muộn. Với cùng lập luận đó, việc thỉnh thoảng thử khôi phục từ các bản sao lưu (backups) là rất quan trọng—nếu không, bạn có thể nhận ra bản sao lưu của mình đã bị hỏng khi mọi chuyện đã quá muộn và bạn đã mất dữ liệu. Đừng chỉ tin tưởng mù quáng rằng mọi thứ đang hoạt động tốt.

Các hệ thống như HDFS và S3 vẫn phải giả định rằng các đĩa cứng hoạt động chính xác trong hầu hết thời gian—đây là một giả định hợp lý, nhưng không giống với việc giả định rằng chúng *luôn luôn* hoạt động chính xác. Tuy nhiên, hiện tại không có nhiều hệ thống áp dụng cách tiếp cận kiểu “tin tưởng, nhưng phải xác minh” bằng

cách liên tục tự kiểm toán chính mình. Nhiều hệ thống giả định rằng các đảm bảo về tính chính xác là tuyệt đối và không dự phòng cho khả năng xảy ra lỗi hỏng dữ liệu hiếm gặp. Trong tương lai, chúng ta có thể thấy nhiều hệ thống *self-validating* (tự xác thực) hoặc *self-auditing* (tự kiểm toán) hơn, những hệ thống liên tục kiểm tra tính toàn vẹn của chính chúng thay vì dựa vào sự tin tưởng mù quáng [54].

Thiết kế cho khả năng kiểm toán

Nếu một transaction làm thay đổi nhiều đối tượng trong một cơ sở dữ liệu, nguyên nhân cốt lõi có thể rất khó để xác định sau khi sự việc đã xảy ra. Ngay cả khi bạn ghi lại các transaction log, các thao tác chèn, cập nhật và xóa trong các bảng khác nhau cũng không nhất thiết đưa ra một bức tranh rõ ràng về việc *tại sao* những thay đổi đó lại được thực hiện. Việc gọi logic ứng dụng để quyết định các thay đổi đó là nhất thời và không thể tái lập.

Ngược lại, các hệ thống dựa trên event có thể cung cấp khả năng kiểm chứng (auditability) tốt hơn. Trong cách tiếp cận event sourcing, đầu vào của người dùng cho hệ thống được đại diện dưới dạng một event duy nhất không thể thay đổi, và bất kỳ cập nhật trạng thái nào sau đó đều được phái sinh từ event đó. Quá trình phái sinh này có thể được thực hiện một cách xác định (deterministic) và có thể lặp lại, sao cho việc chạy cùng một log các event qua cùng một phiên bản mã nguồn phái sinh sẽ dẫn đến các cập nhật trạng thái giống hệt nhau.

Việc thể hiện rõ ràng dataflow giúp nguồn gốc (provenance) của dữ liệu trở nên minh bạch hơn nhiều, điều này giúp việc kiểm tra tính toàn vẹn trở nên khả thi hơn nhiều. Đối với event log, chúng ta có thể sử dụng các mã băm (hash) để kiểm tra xem việc lưu trữ event có bị hư hỏng hay không. Đối với bất kỳ trạng thái phái sinh nào, chúng ta có thể chạy lại các bộ xử lý batch và stream đã phái sinh nó từ event log để kiểm tra xem liệu chúng ta có nhận được cùng một kết quả hay không, hoặc thậm chí chạy song song một quá trình phái sinh dự phòng.

Một dataflow xác định và được định nghĩa rõ ràng cũng giúp việc debug và truy vết quá trình thực thi của một hệ thống trở nên dễ dàng hơn để xác định tại sao nó lại thực hiện một hành động nào đó [4, 55]. Nếu có điều gì đó bất ngờ xảy ra, việc có khả năng chẩn đoán để tái lập chính xác các hoàn cảnh dẫn đến sự kiện bất ngờ đó là rất giá trị—một dạng khả năng debug kiểu du hành thời gian.

Lập luận end-to-end một lần nữa

Nếu chúng ta không thể hoàn toàn tin tưởng rằng mọi thành phần riêng lẻ của hệ thống sẽ không bị hư hỏng—rằng mọi phần cứng đều không có lỗi và mọi phần mềm đều không có bug—thì ít nhất chúng ta phải định kỳ kiểm tra tính toàn vẹn của dữ liệu. Nếu không kiểm tra, chúng ta sẽ không biết về sự hư hỏng cho đến khi quá muộn và nó đã gây ra một số thiệt hại ở hạ nguồn, tại thời điểm đó việc truy tìm vấn đề sẽ khó khăn và tốn kém hơn nhiều.

Việc kiểm tra tính toàn vẹn của các hệ thống dữ liệu tốt nhất nên được thực hiện theo phương thức end-to-end. Chúng ta càng đưa được nhiều hệ thống vào một đợt kiểm tra tính toàn vẹn, thì càng ít có cơ hội để sự hư hỏng không bị phát hiện ở một giai đoạn nào đó của quy trình. Nếu chúng ta có thể kiểm tra rằng toàn bộ một pipeline dữ liệu phải sinh là chính xác từ đầu đến cuối, thì bất kỳ ổ đĩa, mạng, dịch vụ và thuật toán nào dọc theo đường đi cũng được bao gồm một cách ngầm định trong đợt kiểm tra đó.

Việc thực hiện các đợt kiểm tra tính toàn vẹn end-to-end liên tục giúp bạn tăng cường sự tin tưởng vào tính chính xác của các hệ thống, từ đó cho phép bạn tiến hành nhanh hơn [56]. Giống như kiểm thử tự động, việc kiểm chứng (auditing) làm tăng khả năng các bug được tìm thấy nhanh chóng, và do đó giảm thiểu rủi ro rằng một thay đổi đối với hệ thống hoặc một công nghệ lưu trữ mới sẽ gây ra thiệt hại. Nếu bạn không ngại việc thực hiện các thay đổi, bạn có thể phát triển một ứng dụng tốt hơn nhiều để đáp ứng các yêu cầu đang thay đổi.

Các công cụ cho các hệ thống dữ liệu có khả năng kiểm chứng

Hiện tại, không có nhiều hệ thống dữ liệu coi khả năng kiểm chứng là một mối quan tâm hàng đầu. Một số ứng dụng tự triển khai các cơ chế kiểm chứng của riêng chúng—ví dụ, bằng cách ghi log tất cả các thay đổi vào một bảng kiểm chứng riêng biệt—nhưng việc đảm bảo tính toàn vẹn của audit log và trạng thái cơ sở dữ liệu vẫn còn khó khăn. Một transaction log có thể được làm cho chống giả mạo (tamper-proof) bằng cách định kỳ ký nó bằng một module bảo mật phần cứng, nhưng điều đó không đảm bảo rằng ngay từ đầu các transaction đúng đã được đưa vào log.

Các blockchain như Bitcoin và Ethereum là những append-only log dùng chung với các kiểm tra tính nhất quán bằng mã hóa; các transaction mà chúng lưu trữ là các sự kiện, và các smart contract về cơ bản là các stream processor. Các consensus protocol mà chúng sử dụng đảm bảo rằng tất cả các node đều đồng thuận về cùng một trình tự các sự kiện. Điểm khác biệt so với các consensus protocol ở Chương 10 là các blockchain có khả năng chịu Byzantine fault (Byzantine fault-tolerant)—tức là, chúng vẫn hoạt động nếu một số node tham gia có dữ liệu bị lỗi vì các replica liên tục kiểm tra tính toàn vẹn của nhau.

Đối với hầu hết các ứng dụng, blockchain có chi phí overhead quá cao để có thể hữu dụng. Tuy nhiên, một số công cụ mã hóa của chúng cũng có thể được sử dụng trong các ngữ cảnh nhẹ nhàng hơn. Ví dụ, *Merkle trees* [57] là các cây chứa các hash có thể được sử dụng để chứng minh một cách hiệu quả rằng một bản ghi xuất hiện trong một dataset (và một vài thứ khác). *Certificate transparency* sử dụng các append-only log được xác thực bằng mã hóa và các Merkle trees để kiểm tra tính hợp lệ của các chứng chỉ TLS/SSL [58, 59]; nó tránh việc cần đến một consensus protocol bằng cách duy trì một single leader cho mỗi log.

Các thuật toán kiểm tra tính toàn vẹn và kiểm toán, như các thuật toán của certificate transparency và các distributed ledgers, có thể được sử dụng rộng rãi hơn trong các hệ thống dữ liệu nói chung trong tương lai. Sẽ cần một số công việc để làm cho chúng có khả năng mở rộng như các hệ thống không có kiểm toán mã hóa và để giữ cho mức giảm hiệu năng thấp nhất có thể, nhưng chúng vẫn rất thú vị.

Tóm tắt

Trong chương này, chúng ta đã thảo luận về các cách tiếp cận mới để thiết kế các hệ thống dữ liệu dựa trên các ý tưởng từ stream processing. Chúng ta bắt đầu với quan sát rằng không có một công cụ đơn lẻ nào có thể phục vụ hiệu quả tất cả các trường hợp sử dụng có thể xảy ra, vì vậy các ứng dụng phải kết hợp nhiều thành phần phần mềm để đạt được mục tiêu của chúng. Chúng ta đã thảo luận cách giải quyết vấn đề *tích hợp dữ liệu (data integration)* này bằng cách sử dụng batch processing và các event stream để cho phép các thay đổi dữ liệu chảy giữa các hệ thống.

Trong cách tiếp cận này, một số hệ thống nhất định được chỉ định là các system of record, và các dữ liệu khác được phái sinh từ chúng thông qua các phép biến đổi. Bằng cách này, chúng ta có thể duy trì các index, materialized views, các mô hình machine learning, các tóm tắt thống kê, và nhiều thứ khác. Việc thực hiện các phép phái sinh và biến đổi này một cách bất đồng bộ và tách rời (loosely coupled) giúp ngăn chặn một vấn đề ở một khu vực lan sang các khu vực không liên quan, làm tăng tính mạnh mẽ và khả năng chịu lỗi của toàn bộ hệ thống.

Việc biểu diễn các dataflow dưới dạng các phép biến đổi từ dataset này sang dataset khác cũng giúp phát triển các ứng dụng. Nếu bạn muốn thay đổi một trong các bước xử lý—ví dụ, để thay đổi cấu trúc của một index hoặc cache—you chỉ cần chạy lại mã biến đổi mới trên toàn bộ input dataset để phái sinh lại đầu ra. Tương tự, nếu có lỗi xảy ra, bạn có thể sửa mã và xử lý lại dữ liệu để khôi phục.

Các quy trình này khá giống với những gì các cơ sở dữ liệu đã thực hiện bên trong, vì vậy chúng ta định nghĩa lại ý tưởng về các ứng dụng dataflow là việc *unbundling* (tách rời) các thành phần của một cơ sở dữ liệu và xây dựng một ứng dụng bằng cách kết hợp các thành phần loosely coupled này.

Trạng thái phái sinh có thể được cập nhật bằng cách quan sát các thay đổi trong dữ liệu cơ sở. Trạng thái đó cũng có thể được quan sát bởi các consumer ở phía hạ nguồn (downstream). Chúng ta thậm chí có thể đưa dataflow này đi xuyên suốt đến tận thiết bị của người dùng cuối hiển thị dữ liệu, và từ đó xây dựng các UI có khả năng cập nhật động để phản ánh các thay đổi dữ liệu và tiếp tục hoạt động offline.

Tiếp theo, chúng ta đã thảo luận về cách đảm bảo rằng tất cả quá trình xử lý này vẫn chính xác khi có sự hiện diện của các lỗi. Chúng ta thấy rằng các đảm bảo tính toàn vẹn mạnh có thể được triển khai một cách có khả năng mở rộng với lý sự kiện bất

đồng bộ, bằng cách sử dụng các định danh yêu cầu end-to-end để làm cho các thao tác có tính idempotent, và bằng cách kiểm tra các ràng buộc một cách bất đồng bộ. Client có thể đợi cho đến khi việc kiểm tra hoàn tất, hoặc tiếp tục mà không cần đợi nhưng phải đối mặt với rủi ro phải xin lỗi về việc vi phạm một ràng buộc. Cách tiếp cận này có khả năng mở rộng và mạnh mẽ hơn nhiều so với cách tiếp cận truyền thống là sử dụng các distributed transactions và phù hợp với cách mà nhiều quy trình kinh doanh hoạt động trong thực tế.

Bằng cách cấu trúc các ứng dụng xoay quanh dataflow và kiểm tra các ràng buộc một cách bất đồng bộ, chúng ta có thể tránh được hầu hết các vấn đề về coordination và tạo ra các hệ thống vừa duy trì được tính toàn vẹn, vừa đạt hiệu năng tốt, ngay cả trong các kịch bản phân tán về mặt địa lý và khi có sự xuất hiện của các fault. Để kết luận, chúng ta đã thảo luận đôi chút về việc sử dụng các audit để xác minh tính toàn vẹn của dữ liệu và phát hiện sự hư hỏng, đồng thời nhận thấy rằng các kỹ thuật được sử dụng bởi blockchain cũng có sự tương đồng với các hệ thống dựa trên event.

Footnotes

References

- [1] Rachid Belaid. “Postgres Full-Text Search Is Good Enough!” *rachbelaid.com*, July 2015. Archived at perma.cc/ZVP9-YDCB
- [2] Philippe Ajoux, Nathan Bronson, Sanjeev Kumar, Wyatt Lloyd, and Kaushik Veeraraghavan. “Challenges to Adopting Stronger Consistency at Scale.” At *15th USENIX Workshop on Hot Topics in Operating Systems (HotOS)*, May 2015.
- [3] Pat Helland and Dave Campbell. “Building on Quicksand.” At *4th Biennial Conference on Innovative Data Systems Research (CIDR)*, January 2009. Archived at arxiv.org
- [4] Jessica Kerr. “Provenance and Causality in Distributed Systems.” *jessitron.com*, September 2016. Archived at perma.cc/DTD2-F8ZM
- [5] Jay Kreps. “The Log: What Every Software Engineer Should Know About Real-Time Data’s Unifying Abstraction.” *engineering.linkedin.com*, December 2013. Archived at perma.cc/2JHR-FR64
- [6] Pat Helland. “Life Beyond Distributed Transactions: An Apostate’s Opinion.” At *3rd Biennial Conference on Innovative Data Systems Research (CIDR)*, January 2007. Archived at perma.cc/2GZG-UZ65
- [7] Lionel A. Smith. “The Broad Gauge Story.” *Journal of the Monmouthshire Railway Society*, Summer 1985. Archived at perma.cc/DDK9-JA6X
- [8] Jacqueline Xu. “Online Migrations at Scale.” *stripe.com*, February 2017. Archived at perma.cc/ZQY2-EAU2
- [9] Flavio Santos and Robert Stephenson. “Changing the Wheels on a Moving Bus—Spotify’s Event Delivery Migration.” *engineering.atspotify.com*, October 2021. Archived at perma.cc/5C4V-G8EV
- [10] Molly Bartlett Dishman and Martin Fowler. “Agile Architecture.” At *O’Reilly Software Architecture Conference*, March 2015.
- [11] Nathan Marz and James Warren. *Big Data: Principles and Best Practices of Scalable Real-Time Data Systems*. Manning, 2015. ISBN: 9781617290343

- [12] Jay Kreps. “Questioning the Lambda Architecture.” *oreilly.com*, July 2014. Archived at perma.cc/PGH6-XUCH
- [13] Raul Castro Fernandez, Peter Pietzuch, Jay Kreps, Neha Narkhede, Jun Rao, Joel Koshy, Dong Lin, Chris Riccomini, and Guozhang Wang. “Liquid: Unifying Nearline and Offline Big Data Integration.” At *7th Biennial Conference on Innovative Data Systems Research* (CIDR), January 2015. Archived at perma.cc/QMA9-8PKL
- [14] Dennis M. Ritchie and Ken Thompson. “The UNIX Time-Sharing System.” *Communications of the ACM*, volume 17, issue 7, pages 365–375, July 1974. doi:10.1145/361011.361061
- [15] Wes McKinney. “The Road to Composable Data Systems: Thoughts on the Last 15 Years and the Future.” *wesmckinney.com*, September 2023. Archived at perma.cc/J9SJ-886N
- [16] Eric A. Brewer and Joseph M. Hellerstein. “CS262a: Advanced Topics in Computer Systems.” Lecture notes, University of California, Berkeley, *cs.berkeley.edu*, August 2011. Archived at perma.cc/TE79-LGWU
- [17] Michael Stonebraker. “The Case for Polystores.” *wp.sigmod.org*, July 2015. Archived at perma.cc/G7J2-KR45
- [18] Jennie Duggan, Aaron J. Elmore, Michael Stonebraker, Magda Balazinska, Bill Howe, Jeremy Kepner, Sam Madden, David Maier, Tim Mattson, and Stan Zdonik. “The BigDAWG Polystore System.” *ACM SIGMOD Record*, volume 44, issue 2, pages 11–16, June 2015. doi:10.1145/2814710.2814713
- [19] David B. Lomet, Alan Fekete, Gerhard Weikum, and Mike Zwilling. “Unbundling Transaction Services in the Cloud.” At *4th Biennial Conference on Innovative Data Systems Research* (CIDR), January 2009. Archived at arxiv.org
- [20] Martin Kleppmann and Jay Kreps. “Kafka, Samza and the Unix Philosophy of Distributed Data.” *IEEE Data Engineering Bulletin*, volume 38, issue 4, pages 4–14, December 2015. Archived at perma.cc/BJMS-TJ4Z
- [21] John Hugg. “Winning Now and in the Future: Where Volt Active Data Shines.” *voltactivedata.com*, March 2016. Archived at perma.cc/44MP-3MWM
- [22] Felienn Hermans. “Spreadsheets Are Code.” At *Code Mesh*, November 2015.
- [23] Dan Bricklin and Bob Frankston. “VisiCalc: Information from Its Creators.” *danbricklin.com*. Archived at archive.org
- [24] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, and Michael Young. “Machine Learning: The High-Interest Credit Card of Technical Debt.” At *NIPS Workshop on Software Engineering for Machine Learning* (SE4ML), December 2014. Archived at perma.cc/M3MD-U7WL
- [25] Peter Bailis, Alan Fekete, Michael J. Franklin, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. “Feral Concurrency Control: An Empirical Investigation of Modern Application Integrity.” At *ACM International Conference on Management of Data* (SIGMOD), June 2015. doi:10.1145/2723372.2737784
- [26] Guy Steele. “Re: Need for Macros (Was Re: Icon).” Email to *ll1-discuss* mailing list, *people.csail.mit.edu*, December 2001. Archived at perma.cc/K9X8-CJ65
- [27] Ben Stopford. “Microservices in a Streaming World.” At *QCon London*, March 2016.
- [28] Adam Bellemare. *Building Event-Driven Microservices*, 2nd edition. O’Reilly Media, 2025. ISBN: 9798341622180

- [29] Christian Posta. “Why Microservices Should Be Event Driven: Autonomy vs Authority.” *blog.christianposta.com*, May 2016. Archived at perma.cc/E6N9-3X92
- [30] Alex Feyerke. “Designing Offline-First Web Apps.” *alistapart.com*, December 2013. Archived at perma.cc/WH7R-S2DS
- [31] Martin Kleppmann. “Turning the Database Inside-out with Apache Samza.” At *Strange Loop*, September 2014. Archived at perma.cc/U6E8-A9MT
- [32] Sebastian Burckhardt, Daan Leijen, Jonathan Protzenko, and Manuel Fähndrich. “Global Sequence Protocol: A Robust Abstraction for Replicated Shared State.” At *29th European Conference on Object-Oriented Programming (ECOOP)*, July 2015. [doi:10.4230/LIPIcs.ECOOP.2015.568](https://doi.org/10.4230/LIPIcs.ECOOP.2015.568)
- [33] Evan Czaplicki and Stephen Chong. “Asynchronous Functional Reactive Programming for GUIs.” At *34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, June 2013. [doi:10.1145/2491956.2462161](https://doi.org/10.1145/2491956.2462161)
- [34] Eno Thereska, Damian Guy, Michael Noll, and Neha Narkhede. “Unifying Stream Processing and Interactive Queries in Apache Kafka.” *confluent.io*, October 2016. Archived at perma.cc/W8JG-EAZF
- [35] Frank McSherry. “Dataflow as Database.” *github.com*, July 2016. Archived at perma.cc/384D-DUFH
- [36] Peter Alvaro. “I See What You Mean.” At *Strange Loop*, September 2015.
- [37] Nathan Marz. “Trident: A High-Level Abstraction for Realtime Computation.” *blog.x.com*, August 2012. Archived at archive.org
- [38] Edi Bice. “Low Latency Web Scale Fraud Prevention with Apache Samza, Kafka and Friends.” At *Merchant Risk Council MRC Vegas Conference*, March 2016. Archived at perma.cc/T3H5-QN3R
- [39] Charity Majors. “The Accidental DBA.” *charity.wtf*, October 2016. Archived at perma.cc/6ANP-ARB6
- [40] Arthur J. Bernstein, Philip M. Lewis, and Shiyong Lu. “Semantic Conditions for Correctness at Different Isolation Levels.” At *16th International Conference on Data Engineering (ICDE)*, February 2000. [doi:10.1109/ICDE.2000.839387](https://doi.org/10.1109/ICDE.2000.839387)
- [41] Sudhir Jorwekar, Alan Fekete, Krithi Ramamritham, and S. Sudarshan. “Automating the Detection of Snapshot Isolation Anomalies.” At *33rd International Conference on Very Large Data Bases (VLDB)*, September 2007.
- [42] Kyle Kingsbury. “Distributed Systems Safety Research.” *jepsen.io*.
- [43] Michael Jouravlev. “Redirect After Post.” *theserverside.com*, August 2004. Archived at archive.org
- [44] Jerome H. Saltzer, David P. Reed, and David D. Clark. “End-to-End Arguments in System Design.” *ACM Transactions on Computer Systems*, volume 2, issue 4, pages 277–288, November 1984. [doi:10.1145/357401.357402](https://doi.org/10.1145/357401.357402)
- [45] Peter Bailis, Alan Fekete, Michael J. Franklin, Ali Ghodsi, Joseph M. Hellerstein, and Ion Stoica. “Coordination Avoidance in Database Systems.” *Proceedings of the VLDB Endowment*, volume 8, issue 3, pages 185–196, November 2014. [doi:10.14778/2735508.2735509](https://doi.org/10.14778/2735508.2735509), extended version published as [arXiv:1402.2237](https://arxiv.org/abs/1402.2237)
- [46] Alex Yarmula. “Strong Consistency in Manhattan.” *blog.x.com*, March 2016. Archived at archive.org
- [47] Martin Kleppmann, Alastair R. Beresford, and Boerge Svingen. “Online Event Processing: Achieving Consistency Where Distributed Transactions Have Failed.” *Communications of the ACM*, volume 62, issue 5, pages 43–49, May 2019. [doi:10.1145/3312527](https://doi.org/10.1145/3312527)

- [48] Jim Gray. “The Transaction Concept: Virtues and Limitations.” At *7th International Conference on Very Large Data Bases* (VLDB), September 1981. Archived at perma.cc/8VPT-N5H6
- [49] Hector Garcia-Molina and Kenneth Salem. “Sagas.” At *ACM International Conference on Management of Data* (SIGMOD), May 1987. [doi:10.1145/38713.38742](https://doi.org/10.1145/38713.38742)
- [50] Annamalai Gurusami and Daniel Price. “Bug #73170: Duplicates in Unique Secondary Index Because of Fix of Bug#68021.” *bugs.mysql.com*, July 2014. Archived at perma.cc/P6BV-W7JJ
- [51] Gary Fredericks. “Postgres Serializability Bug.” *github.com*, September 2015. Archived at perma.cc/N8UP-2822
- [52] Xiao Chen. “HDFS DataNode Scanners and Disk Checker Explained.” *blog.cloudera.com*, December 2016. Archived at perma.cc/6S36-X98L
- [53] Daniel Persson. “How Does Ceph Scrubbing Work?” *youtube.com*, March 2022.
- [54] Jay Kreps. “Getting Real About Distributed System Reliability.” *blog.empathybox.com*, March 2012. Archived at perma.cc/9B5Q-AEBW
- [55] Martin Fowler. “The LMAX Architecture.” *martinfowler.com*, July 2011. Archived at perma.cc/5AV4-N6RJ
- [56] Sam Stokes. “Move Fast with Confidence.” *five-eights.com*, July 2016. Archived at perma.cc/J8C6-DHXB
- [57] Ralph C. Merkle. “A Digital Signature Based on a Conventional Encryption Function.” At *CRYPTO ’87*, August 1987. [doi:10.1007/3-540-48184-2_32](https://doi.org/10.1007/3-540-48184-2_32)
- [58] Ben Laurie. “Certificate Transparency.” *ACM Queue*, volume 12, issue 8, pages 10–19, August 2014. [doi:10.1145/2668152.2668154](https://doi.org/10.1145/2668152.2668154)
- [59] Mark D. Ryan. “Enhanced Certificate Transparency and End-to-End Encrypted Mail.” At *Network and Distributed System Security Symposium* (NDSS), February 2014. [doi:10.14722/ndss.2014.23379](https://doi.org/10.14722/ndss.2014.23379)

Làm điều đúng đắn

Việc cung cấp cho các hệ thống AI sự tươi đẹp, xấu xí và tàn bạo của thế giới, nhưng lại kỳ vọng nó chỉ phản chiếu sự tươi đẹp, là một sự ảo tưởng.

Vinay Uday Prabhu và Abeba Birhane, “Large Datasets: A Pyrrhic Win for Computer Vision?” (2020)

Trong chương cuối cùng của cuốn sách này, hãy lùi lại một bước. Xuyên suốt cuốn sách, chúng ta đã xem xét một loạt các kiến trúc cho các hệ thống dữ liệu, đánh giá các ưu và nhược điểm của chúng, và khám phá các kỹ thuật để xây dựng các ứng dụng tin cậy, có khả năng mở rộng và dễ bảo trì. Tuy nhiên, chúng ta đã bỏ qua một phần cơ bản của cuộc thảo luận, điều mà giờ đây chúng ta nên bổ sung vào.

Mọi hệ thống đều được xây dựng vì một mục đích; mọi hành động chúng ta thực hiện đều có cả những hệ quả dự tính và không dự tính. Mục đích có thể đơn giản như là kiếm tiền, nhưng các hệ quả có thể rất sâu rộng. Chúng ta, những kỹ sư xây dựng các hệ thống này, có trách nhiệm phải xem xét kỹ lưỡng những hệ quả đó và đảm bảo rằng các quyết định của mình không gây ra tác hại.

Chúng ta nói về dữ liệu như một thứ trừu tượng, nhưng hãy nhớ rằng nhiều *dataset* liên quan đến con người: hành vi, sở thích và danh tính của họ. Chúng ta phải đối xử với dữ liệu như vậy bằng tính nhân văn và sự tôn trọng. Người dùng cũng là con người, và phẩm giá con người là tối thượng [1].

Phát triển phần mềm ngày càng liên quan đến việc đưa ra các lựa chọn đạo đức quan trọng. Có các hướng dẫn để giúp các kỹ sư phần mềm điều hướng những vấn đề này, chẳng hạn như ACM Code of Ethics and Professional Conduct [2], nhưng chúng hiếm khi được thảo luận, áp dụng và thực thi trong thực tế. Kết quả là, các kỹ sư và quản lý sản phẩm đôi khi có thái độ hời hợt đối với quyền riêng tư và các hệ quả tiêu cực tiềm ẩn từ sản phẩm của họ [3, 4].

Một công nghệ tự thân nó không tốt hay xấu—điều quan trọng là cách nó được sử dụng và cách nó ảnh hưởng đến con người. Điều này đúng với một hệ thống phần mềm như công cụ tìm kiếm cũng giống như đối với một vũ khí như khẩu súng. Trách nhiệm đạo đức là của chúng ta; sẽ không đủ nếu các kỹ sư phần mềm chỉ tập trung duy nhất vào công nghệ mà phớt lờ các hệ quả của nó.

Tuy nhiên, trái ngược với phần lớn lĩnh vực điện toán, các khái niệm cốt lõi của đạo đức không cố định hay xác định chính xác về ý nghĩa; chúng đòi hỏi sự diễn giải, điều có thể mang tính chủ quan [5]. Điều làm cho một thứ gì đó trở nên “tốt” hoặc “xấu”

không được định nghĩa rõ ràng, và các cuộc thảo luận nghiêm túc về chủ đề này trong giới chuyên gia điện toán vẫn còn thiếu vắng [6]. Suy luận về đạo đức là một việc khó khăn, nhưng nó quá quan trọng để có thể bỏ qua. Điều này bao gồm những gì? Đạo đức không phải là việc đi qua một danh sách kiểm tra để xác nhận bạn tuân thủ; đó là một quá trình tham gia và lặp lại của sự phản tư (reflection), thông qua đối thoại với những người liên quan, cùng với trách nhiệm giải trình đối với các kết quả [7].

Phân tích dự báo

Phân tích dự báo là một phần quan trọng lý giải tại sao mọi người lại hào hứng với big data và AI. Đây cũng là một lĩnh vực đầy rẫy những tình huống tiến thoái lưỡng nan về đạo đức. Sử dụng phân tích dữ liệu để dự báo thời tiết, hoặc sự lây lan của dịch bệnh, là một chuyện [8]; nhưng dự báo liệu một tội phạm có khả năng tái phạm, liệu một người nộp đơn vay vốn có khả năng vỡ nợ, hay liệu một khách hàng bảo hiểm có khả năng đưa ra các yêu cầu bồi thường đắt đỏ hay không lại là một chuyện khác [9]. Những trường hợp sau có ảnh hưởng trực tiếp đến cuộc sống của từng cá nhân.

Lẽ tự nhiên, các mạng lưới thanh toán muốn ngăn chặn các giao dịch gian lận, các ngân hàng muốn tránh các khoản vay xấu, các hãng hàng không muốn tránh các vụ không tặc, và các công ty muốn tránh việc thuê những người làm việc không hiệu quả hoặc không đáng tin cậy. Từ góc độ của họ, chi phí cho một cơ hội kinh doanh bị bỏ lỡ là thấp, nhưng chi phí cho một khoản vay xấu hoặc một nhân viên gây rắc rối là cao hơn nhiều, vì vậy việc các tổ chức muốn thận trọng là điều dễ hiểu. Nếu có nghi ngờ, tốt hơn hết họ nên nói không.

Tuy nhiên, khi việc ra quyết định bằng thuật toán trở nên phổ biến hơn, một người bị thuật toán gắn nhãn (một cách chính xác hoặc sai lệch) là rủi ro có thể phải chịu một lượng lớn những quyết định "no" đó. Việc bị loại trừ một cách có hệ thống khỏi các công việc, chuyển bay, bảo hiểm, thuê bất động sản, dịch vụ tài chính và các khía cạnh then chốt khác của xã hội là một sự hạn chế lớn đối với tự do của một cá nhân, đến mức nó đã được gọi là "nhà tù thuật toán" [10]. Ở những quốc gia tôn trọng nhân quyền, hệ thống tư pháp hình sự giả định một người vô tội cho đến khi được chứng minh là có tội; mặt khác, các hệ thống tự động có thể loại trừ một người khỏi việc tham gia vào xã hội một cách có hệ thống và tùy tiện mà không cần bất kỳ bằng chứng tội lỗi nào, và với rất ít cơ hội để kháng cáo.

Định kiến và Phân biệt đối xử

Các quyết định được đưa ra bởi một thuật toán không nhất thiết là tốt hơn hay tệ hơn so với những quyết định do con người đưa ra. Mỗi người đều có khả năng có những định kiến, ngay cả khi họ tích cực cố gắng chống lại chúng, và các hành vi

phân biệt đối xử có thể trở thành một phần được thể chế hóa về mặt văn hóa. Có hy vọng rằng việc đưa ra quyết định dựa trên dữ liệu, thay vì dựa trên những đánh giá chủ quan và theo bản năng của con người, có thể công bằng hơn và mang lại cơ hội tốt hơn cho những người thường bị bỏ qua hoặc chịu thiệt thòi trong hệ thống truyền thống [11].

Khi chúng ta phát triển các hệ thống phân tích dự đoán và AI, chúng ta không chỉ đơn thuần là tự động hóa quyết định của con người bằng cách sử dụng phần mềm để chỉ định các quy tắc cho việc khi nào nên nói có hoặc không; chúng ta đang để chính các quy tắc đó được suy luận từ dữ liệu. Tuy nhiên, các pattern mà các hệ thống này học được lại rất mờ nhạt: ngay cả khi dữ liệu chỉ ra một sự tương quan, chúng ta cũng có thể không biết tại sao. Nếu đầu vào của một thuật toán mang theo một định kiến có hệ thống, hệ thống đó rất có thể sẽ học và khuếch đại định kiến đó trong đầu ra của nó [12].

Ở nhiều quốc gia, luật chống phân biệt đối xử nghiêm cấm việc đối xử khác biệt với mọi người dựa trên các đặc điểm được bảo vệ như sắc tộc, tuổi tác, giới tính, xu hướng tính dục, khuyết tật hoặc niềm tin. Các feature khác trong dữ liệu của một người có thể được phân tích, nhưng điều gì sẽ xảy ra nếu chúng tương quan với các đặc điểm được bảo vệ? Ví dụ, trong các khu dân cư bị phân tách về sắc tộc, mã bưu điện hoặc thậm chí là địa chỉ IP của một người là một yếu tố dự báo mạnh mẽ về chủng tộc. Nói như vậy, thật nực cười khi tin rằng một thuật toán có thể bằng cách nào đó lấy dữ liệu bị định kiến làm đầu vào và tạo ra đầu ra công bằng và không thiên vị [13, 14]. Tuy nhiên, niềm tin này thường dường như được ngầm định bởi những người ủng hộ việc ra quyết định dựa trên dữ liệu—một thái độ đã bị châm biếm là "machine learning giống như rửa tiền cho định kiến" [15].

Các hệ thống phân tích dự đoán chỉ đơn thuần là ngoại suy từ quá khứ; nếu quá khứ mang tính phân biệt đối xử, chúng sẽ mã hóa và khuếch đại sự phân biệt đối xử đó [16]. Nếu chúng ta muốn tương lai tốt đẹp hơn quá khứ, cần phải có trí tưởng tượng đạo đức, và đó là điều mà chỉ con người mới có thể cung cấp [17]. Dữ liệu và các mô hình nên là công cụ của chúng ta, chứ không phải là chủ nhân của chúng ta.

Trách nhiệm và Trách nhiệm giải trình

Việc ra quyết định tự động mở ra câu hỏi về trách nhiệm và trách nhiệm giải trình [17]. Nếu một con người mắc lỗi, họ có thể phải chịu trách nhiệm, và người bị ảnh hưởng bởi quyết định đó có thể kháng cáo. Thuật toán cũng mắc lỗi, nhưng ai là người chịu trách nhiệm giải trình nếu chúng sai sót [18]? Khi một chiếc xe tự lái gây ra tai nạn, ai là người chịu trách nhiệm? Nếu một thuật toán chấm điểm tín dụng tự động phân biệt đối xử một cách có hệ thống đối với những người thuộc một chủng tộc hoặc tôn giáo nhất định, liệu có bất kỳ biện pháp cứu trợ nào không? Nếu một quyết định từ hệ thống ML của bạn bị đưa ra xem xét tại tòa án, liệu bạn có thể giải thích cho thẩm phán cách mà thuật toán đã đưa ra quyết định đó không? Con người

không nên có khả năng trốn tránh trách nhiệm của mình bằng cách đổ lỗi cho một thuật toán.

Các cơ quan xếp hạng tín dụng là một ví dụ điển hình về việc thu thập dữ liệu để đưa ra quyết định về con người. Một điểm tín dụng thấp khiến cuộc sống trở nên khó khăn, nhưng ít nhất điểm tín dụng thường dựa trên các sự thật liên quan đến lịch sử vay vốn thực tế của một người, và bất kỳ lỗi nào trong hồ sơ đều có thể được sửa đổi (mặc dù các cơ quan này thường không làm điều đó một cách dễ dàng). Tuy nhiên, các thuật toán chấm điểm dựa trên machine learning thường sử dụng phạm vi đầu vào rộng hơn nhiều và mờ ám hơn nhiều, khiến việc hiểu cách một quyết định cụ thể được đưa ra cũng như liệu ai đó có đang bị đối xử một cách bất công hoặc phân biệt đối xử hay không trở nên khó khăn hơn [19].

Một điểm tín dụng tóm tắt câu hỏi "Bạn đã hành xử như thế nào trong quá khứ?", trong khi phân tích dự đoán (predictive analytics) thường hoạt động trên cơ sở "Ai giống bạn, và những người giống bạn đã hành xử như thế nào trong quá khứ?". Việc rút ra những điểm tương đồng với hành vi của người khác đồng nghĩa với việc áp đặt định kiến lên con người—ví dụ, dựa trên nơi họ sinh sống (một biến đại diện gần đúng cho chủng tộc và tầng lớp kinh tế xã hội). Điều gì sẽ xảy ra với những người bị xếp vào sai nhóm? Hơn nữa, nếu một quyết định là sai lệch do dữ liệu bị lỗi, việc tìm kiếm sự cứu xét gần như là không thể [17].

Nhiều dữ liệu có bản chất thống kê, điều này có nghĩa là ngay cả khi phân phối xác suất trên tổng thể là chính xác, các trường hợp cá biệt hoàn toàn có thể sai. Ví dụ, nếu tuổi thọ trung bình ở quốc gia của bạn là 80 tuổi, điều đó không có nghĩa là bạn được dự đoán sẽ qua đời vào đúng ngày sinh nhật lần thứ 80 của mình. Từ giá trị trung bình và phân phối xác suất, bạn không thể nói nhiều về độ tuổi mà một người cụ thể sẽ sống đến. Tương tự, đầu ra của một hệ thống dự đoán mang tính xác suất và hoàn toàn có thể sai trong các trường hợp cá biệt.

Một niềm tin mù quáng vào sự tối thượng của dữ liệu trong việc đưa ra quyết định không chỉ là ảo tưởng mà còn cực kỳ nguy hiểm. Khi việc đưa ra quyết định dựa trên dữ liệu (data-driven decision making) trở nên phổ biến hơn, chúng ta sẽ cần tìm cách tránh việc củng cố các định kiến hiện có, cách làm cho các thuật toán có trách nhiệm giải trình và minh bạch, cũng như cách khắc phục chúng khi chúng không tránh khỏi việc mắc lỗi.

Chúng ta cũng sẽ cần tìm cách hiện thực hóa tiềm năng tích cực của dữ liệu và ngăn chặn việc nó bị sử dụng để gây hại cho con người. Ví dụ, phân tích có thể tiết lộ các đặc điểm tài chính và xã hội trong cuộc sống của con người. Một mặt, sức mạnh này có thể được sử dụng để tập trung viện trợ và hỗ trợ cho những người cần nó nhất. Mặt khác, đôi khi nó lại được sử dụng bởi các doanh nghiệp trục lợi nhằm xác định những người dễ bị tổn thương và bán cho họ các sản phẩm rủi ro như các khoản vay lãi suất cao hoặc các bằng đại học vô giá trị [17, 20].

Vòng lặp phản hồi (Feedback Loops)

Ngay cả với các ứng dụng dự đoán có ít tác động trực tiếp và sâu rộng hơn đến con người, chẳng hạn như các hệ thống gợi ý (recommendation systems), vẫn có những vấn đề khó khăn mà chúng ta phải đối mặt. Khi các dịch vụ trở nên giỏi trong việc dự đoán nội dung mà người dùng muốn xem, chúng có thể kết thúc bằng việc chỉ hiển thị cho mọi người những ý kiến mà họ đã đồng ý, dẫn đến các "phòng vang" (echo chambers)—nơi các định kiến, thông tin sai lệch và sự phân cực có thể nảy sinh. Chúng ta đã và đang thấy tác động mà các phòng vang trên mạng xã hội có thể gây ra đối với các chiến dịch bầu cử.

Khi phân tích dự đoán ảnh hưởng đến cuộc sống của con người, các vấn đề đặc biệt nghiêm trọng sẽ nảy sinh do các vòng lặp phản hồi (feedback loops) tự củng cố. Ví dụ, hãy xét trường hợp các nhà tuyển dụng sử dụng điểm tín dụng để đánh giá các ứng viên tiềm năng. Bạn có thể là một nhân viên tốt với điểm tín dụng tốt, nhưng đột nhiên thấy mình rơi vào khó khăn tài chính do một rủi ro nằm ngoài tầm kiểm soát của bạn. Khi bạn lỡ các kỳ thanh toán hóa đơn, điểm tín dụng của bạn bị giảm sút, và bạn sẽ ít có khả năng tìm được việc làm hơn. Tình trạng thất nghiệp đẩy bạn vào cảnh nghèo đói, điều này lại làm điểm số của bạn tệ hơn nữa, khiến việc tìm kiếm việc làm thậm chí còn khó khăn hơn [17]. Đó là một vòng xoáy đi xuống do những giả định độc hại, được che đậy dưới lớp ngụy trang của sự chặt chẽ về toán học và dữ liệu.

Như một ví dụ khác về feedback loop, các nhà kinh tế học đã phát hiện ra rằng khi các trạm xăng ở Đức áp dụng giá cả theo thuật toán, sự cạnh tranh bị giảm bớt và giá cả đối với người tiêu dùng tăng lên vì các thuật toán đã học được cách thông đồng với nhau [21].

Chúng ta không thể luôn dự đoán được khi nào các vòng lặp phản hồi (feedback loops) như vậy có thể xảy ra. Tuy nhiên, nhiều hệ quả có thể được dự đoán bằng cách suy nghĩ về toàn bộ hệ thống (không chỉ các thành phần máy tính hóa, mà còn cả con người tương tác với nó)—một cách tiếp cận được gọi là *tư duy hệ thống* (systems thinking) [22]. Chúng ta có thể cố gắng tìm hiểu cách một hệ thống phân tích dữ liệu phản ứng với các hành vi, cấu trúc hoặc đặc điểm khác nhau. Liệu hệ thống đó sẽ củng cố và khuếch đại những sự khác biệt hiện có giữa con người (ví dụ: làm cho người giàu giàu hơn hoặc người nghèo nghèo đi), hay nó cố gắng chống lại sự bất công? Ngay cả với những ý định tốt nhất, chúng ta cũng phải cảnh giác trước khả năng xảy ra các hệ quả không mong muốn.

Quyền riêng tư và Theo dõi

Bên cạnh các vấn đề của phân tích dự đoán—tức là sử dụng dữ liệu để đưa ra các quyết định tự động về con người—còn có các vấn đề đạo đức liên quan đến chính

việc thu thập dữ liệu. Mối quan hệ giữa các tổ chức thu thập dữ liệu và những người có dữ liệu đang bị thu thập là gì?

Khi một hệ thống chỉ lưu trữ dữ liệu mà người dùng đã nhập một cách rõ ràng, vì họ muốn hệ thống lưu trữ và xử lý dữ liệu đó theo một cách nhất định, thì hệ thống đang thực hiện một dịch vụ cho người dùng; người dùng chính là khách hàng. Nhưng khi hoạt động của một người dùng bị theo dõi và ghi lại (log) như một tác dụng phụ của các hoạt động khác mà họ đang thực hiện, mối quan hệ này trở nên ít rõ ràng hơn. Dịch vụ không còn chỉ làm những gì người dùng yêu cầu; nó tự đảm nhận những lợi ích riêng, vốn có thể xung đột với lợi ích của người dùng.

Theo dõi dữ liệu hành vi đã trở nên ngày càng quan trọng đối với các feature hướng tới người dùng của nhiều dịch vụ trực tuyến. Theo dõi xem kết quả tìm kiếm nào được nhấp vào giúp cải thiện thứ hạng của các kết quả tìm kiếm; cung cấp các đề xuất (“những người thích X cũng thích Y ”) giúp người dùng khám phá những thứ thú vị và hữu ích; các bài kiểm tra A/B và phân tích luồng người dùng có thể giúp chỉ ra cách cải thiện một UI. Những feature đó đòi hỏi một lượng theo dõi nhất định về hành vi người dùng, và người dùng được hưởng lợi từ chúng.

Tuy nhiên, tùy thuộc vào mô hình kinh doanh của một công ty, việc theo dõi thường không dừng lại ở đó. Nếu dịch vụ được tài trợ thông qua quảng cáo, các nhà quảng cáo mới là khách hàng thực sự, và lợi ích của người dùng sẽ đứng hàng thứ hai. Việc theo dõi dữ liệu trở nên chi tiết hơn, các phân tích trở nên sâu rộng hơn, và dữ liệu được lưu giữ trong một thời gian dài nhằm xây dựng các hồ sơ chi tiết về mỗi người cho mục đích tiếp thị.

Giờ đây, mối quan hệ giữa công ty và người dùng có dữ liệu đang bị thu thập bắt đầu trông khá khác biệt. Người dùng được cung cấp một dịch vụ miễn phí và bị dụ dỗ để tương tác với nó càng nhiều càng tốt. Việc theo dõi người dùng chủ yếu phục vụ không phải cho cá nhân đó mà là cho nhu cầu của các nhà quảng cáo, những người đang tài trợ cho dịch vụ. Mối quan hệ này có thể được mô tả một cách phù hợp bằng một từ mang hàm ý đen tối hơn: *giám sát (surveillance)*.

Giám sát

Như một thí nghiệm tư duy, hãy thử thay thế từ *dữ liệu* bằng *giám sát*, và quan sát xem liệu các cụm từ thông dụng có còn nghe có vẻ tốt đẹp như vậy không [23]. Thử xem nhé: “Trong tổ chức vận hành bởi giám sát của chúng tôi, chúng tôi thu thập các luồng giám sát thời gian thực và lưu trữ chúng trong data warehouse giám sát của mình. Các nhà khoa supervised learning của chúng tôi sử dụng các phân tích nâng cao và xử lý giám sát để rút ra những hiểu biết mới.”

Thí nghiệm tư duy này có vẻ gây tranh cãi một cách bất thường đối với cuốn sách này, *Designing Surveillance-Intensive Applications*, nhưng những từ ngữ mạnh mẽ là cần thiết để nhấn mạnh điểm này. Trong những nỗ lực nhằm làm cho phần mềm

"thâu tóm thế giới" [24], chúng ta đã xây dựng nên hạ tầng giám sát đại chúng lớn nhất từng thấy. Chúng ta đang nhanh chóng tiến tới một thế giới mà mọi không gian có người ở đều chứa ít nhất một microphone kết nối internet, dưới dạng điện thoại thông minh, smart TV, các thiết bị trợ lý điều khiển bằng giọng nói, máy giám sát trẻ em, và thậm chí cả đồ chơi trẻ em sử dụng nhận dạng giọng nói dựa trên cloud. Nhiều thiết bị trong số này có hồ sơ bảo mật rất tồi tệ [25].

Điều mới mẻ so với trước đây là quá trình số hóa đã giúp việc thu thập lượng lớn dữ liệu về con người trở nên dễ dàng. Việc giám sát vị trí và sự di chuyển, các mối quan hệ xã hội và giao tiếp, các giao dịch mua sắm và thanh toán, cũng như dữ liệu sức khỏe của chúng ta đã trở nên gần như không thể tránh khỏi. Một tổ chức giám sát có thể cuối cùng sẽ biết về một người nhiều hơn chính người đó biết về bản thân mình—ví dụ, nhận diện được các bệnh tật hoặc các vấn đề kinh tế trước khi cá nhân đó kịp nhận ra.

Ngay cả những chế độ độc tài và đàn áp nhất trong quá khứ cũng chỉ có thể mơ về việc đặt một chiếc micro vào mọi căn phòng và buộc mọi người phải liên tục mang theo một thiết bị có khả năng theo dõi vị trí và sự di chuyển của họ. Tuy nhiên, những lợi ích mà chúng ta nhận được từ công nghệ kỹ thuật số lớn đến mức hiện nay chúng ta tự nguyện chấp nhận trạng thái bị giám sát toàn diện này. Sự khác biệt chỉ là dữ liệu đang được thu thập bởi các tập đoàn để cung cấp dịch vụ cho chúng ta, thay vì bởi các cơ quan chính phủ nhằm tìm kiếm sự kiểm soát [26].

Không phải tất cả việc thu thập dữ liệu đều nhất thiết được coi là giám sát, nhưng việc xem xét nó dưới góc độ đó có thể giúp chúng ta hiểu được mối quan hệ của mình với bên thu thập dữ liệu. Tại sao chúng ta dường như lại vui vẻ chấp nhận sự giám sát từ các tập đoàn? Có lẽ bạn cảm thấy mình không có gì phải che giấu—nói cách khác, bạn hoàn toàn phù hợp với các cấu trúc quyền lực hiện có, bạn không phải là một nhóm thiểu số bị gạt ra ngoài lề, và bạn không cần phải sợ bị đàn áp [27]. Không phải ai cũng may mắn như vậy. Hoặc có lẽ vì mục đích của nó có vẻ lành tính—đó không phải là sự cưỡng ép và ép buộc lộ liễu, mà chỉ đơn thuần là các đề xuất tốt hơn và tiếp thị cá nhân hóa hơn. Tuy nhiên, khi kết hợp với phần thảo luận về phân tích dự đoán (predictive analytics) ở mục trước, sự phân biệt này dường như trở nên ít rõ ràng hơn.

Chúng ta đã bắt đầu thấy dữ liệu hành vi về việc lái xe, được theo dõi bởi ô tô mà không có sự đồng ý của người lái, gây ảnh hưởng đến phí bảo hiểm của họ [28], và phạm vi bảo hiểm y tế phụ thuộc vào việc mọi người đeo thiết bị theo dõi sức khỏe. Khi sự giám sát được sử dụng để đưa ra các quyết định có tầm ảnh hưởng đến các khía cạnh quan trọng của cuộc sống, chẳng hạn như phạm vi bảo hiểm hoặc việc làm, nó bắt đầu tỏ ra ít lành tính hơn. Phân tích dữ liệu cũng có thể tiết lộ những điều xâm phạm một cách đáng ngạc nhiên—ví dụ, cảm biến chuyển động trong một chiếc smartwatch hoặc thiết bị theo dõi sức khỏe có thể được sử dụng để tính toán xem

bạn đang gõ gì (ví dụ: mật khẩu) với độ chính xác khá cao [29]. Độ chính xác của cảm biến và các thuật toán phân tích sẽ chỉ ngày càng trở nên tốt hơn.

Sự đồng ý và Quyền tự do lựa chọn

Chúng ta có thể khẳng định rằng người dùng tự nguyện chọn sử dụng các dịch vụ theo dõi hoạt động của họ, đồng ý với các điều khoản dịch vụ và chính sách quyền riêng tư, đồng thời chấp thuận việc thu thập dữ liệu. Chúng ta thậm chí có thể tuyên bố rằng người dùng đang nhận được một dịch vụ có giá trị để đổi lấy dữ liệu mà họ cung cấp, và việc theo dõi là cần thiết để cung cấp dịch vụ đó. Chắc chắn rằng các mạng xã hội, công cụ tìm kiếm và nhiều dịch vụ trực tuyến miễn phí khác là hữu ích đối với người dùng—nhưng lập luận này có những vấn đề.

Đầu tiên, chúng ta nên hỏi tại sao việc theo dõi lại cần thiết. Một số hình thức theo dõi trực tiếp góp phần cải thiện các feature cho người dùng—ví dụ, theo dõi tỷ lệ nhấp (click-through rate) trên các kết quả tìm kiếm có thể giúp cải thiện thứ hạng và mức độ liên quan của kết quả tìm kiếm, và theo dõi việc khách hàng có xu hướng mua những sản phẩm nào cùng nhau có thể giúp một cửa hàng trực tuyến gợi ý các sản phẩm liên quan. Tuy nhiên, khi theo dõi tương tác của người dùng để đề xuất nội dung, hoặc để xây dựng hồ sơ người dùng cho mục đích quảng cáo, thì việc liệu điều này có thực sự vì lợi ích của người dùng hay không vẫn chưa rõ ràng. Có phải nó chỉ cần thiết vì các quảng cáo chi trả cho dịch vụ đó không?

Thứ hai, hầu hết người dùng có rất ít kiến thức về việc họ đang đưa loại dữ liệu nào vào các cơ sở dữ liệu của chúng ta, hoặc cách dữ liệu đó được lưu giữ và xử lý—và hầu hết các chính sách quyền riêng tư đều làm mờ mịt vấn đề nhiều hơn là làm sáng tỏ nó. Nếu không hiểu điều gì xảy ra với dữ liệu của mình, người dùng không thể đưa ra sự đồng ý có ý nghĩa. Thông thường, dữ liệu từ một người dùng cũng tiết lộ những điều về những người khác vốn không phải là người dùng của dịch vụ và chưa đồng ý với bất kỳ điều khoản nào. Các tập dữ liệu phái sinh (derived datasets) mà chúng ta đã thảo luận trong vài chương trước—nơi mà dữ liệu từ toàn bộ cơ sở người dùng có thể đã được kết hợp với việc theo dõi hành vi và các nguồn dữ liệu bên ngoài—chính là loại dữ liệu mà người dùng không thể hiểu một cách có ý nghĩa.

Hơn nữa, dữ liệu được trích xuất từ người dùng thông qua một quá trình một chiều, chứ không phải là một mối quan hệ có sự tương hỗ thực sự hay trao đổi giá trị công bằng. Không có sự đối thoại, không có tùy chọn để người dùng thương lượng về lượng dữ liệu họ cung cấp và dịch vụ họ nhận lại. Mối quan hệ giữa dịch vụ và người dùng là bất đối xứng và phiến diện; các điều khoản được thiết lập bởi dịch vụ, chứ không phải bởi người dùng [30, 31].

Tại Liên minh Châu Âu, Quy định Bảo vệ Dữ liệu Chung (GDPR) yêu cầu rằng sự đồng ý phải được "đưa ra một cách tự nguyện, cụ thể, có đầy đủ thông tin và không gây nhầm lẫn", và người dùng phải có thể "từ chối hoặc rút lại sự đồng ý mà không

phải chịu tổn hại”—nếu không, nó sẽ không được coi là “được đưa ra một cách tự nguyện”. Bất kỳ yêu cầu đồng ý nào cũng phải được viết “dưới hình thức dễ hiểu và dễ tiếp cận, sử dụng ngôn ngữ rõ ràng và bình dị,” và “sự im lặng, các ô đã được tích sẵn hoặc sự không hoạt động [không] cấu thành sự đồng ý” [32].

Sự đồng ý không phải là cơ sở duy nhất để xử lý dữ liệu cá nhân một cách hợp pháp theo GDPR. Còn có một số cơ sở khác, bao gồm để tuân thủ các luật lệ khác hoặc để bảo vệ tính mạng của ai đó. Ngoài ra, cơ sở lợi ích hợp pháp cho phép một số cách sử dụng dữ liệu nhất định (ví dụ: để ngăn chặn gian lận) [33] (điều mà những kẻ gian lận có lẽ sẽ không đồng ý). Tuy nhiên, sự đồng ý là cơ sở được sử dụng thường xuyên nhất để xử lý dữ liệu cá nhân trong các dịch vụ internet.

Bạn có thể lập luận rằng một người dùng không đồng ý với việc giám sát có thể đơn giản là chọn không sử dụng một dịch vụ. Nhưng lựa chọn này cũng không hề tự do. Nếu một dịch vụ phổ biến đến mức nó được “hầu hết mọi người coi là thiết yếu để tham gia các hoạt động xã hội cơ bản” [30], thì việc kỳ vọng mọi người từ chối sử dụng nó là không hợp lý—việc sử dụng nó thực tế là bắt buộc. Ví dụ, trong hầu hết các cộng đồng xã hội phương Tây, việc mang theo điện thoại thông minh, sử dụng mạng xã hội để giao lưu và sử dụng Google để tìm kiếm thông tin đã trở thành chuẩn mực. Đặc biệt khi một dịch vụ có hiệu ứng mạng (network effects), sẽ có một chi phí xã hội đối với những người chọn *không* sử dụng nó.

Việc từ chối sử dụng một dịch vụ vì các chính sách theo dõi người dùng của nó nói thì dễ hơn làm. Các nền tảng này được thiết kế đặc biệt để thu hút người dùng. Nhiều nền tảng sử dụng các cơ chế trò chơi và các chiến thuật phổ biến trong cờ bạc để giữ chân người dùng quay trở lại [34]. Ngay cả khi một người dùng vượt qua được điều này, việc từ chối tham gia cũng chỉ là tùy chọn đối với một số ít những người đủ đặc quyền để có thời gian và kiến thức nhằm hiểu được chính sách quyền riêng tư của nó, và những người có khả năng chấp nhận việc có thể bỏ lỡ các cơ hội tham gia xã hội hoặc cơ hội nghề nghiệp có thể phát sinh nếu họ tham gia vào dịch vụ. Đối với những người ở vị thế ít đặc quyền hơn, không có sự tự do lựa chọn có ý nghĩa nào cả; việc giám sát trở nên không thể tránh khỏi.

Quyền riêng tư và việc Sử dụng Dữ liệu

Đôi khi mọi người tuyên bố rằng “quyền riêng tư đã chết” với lý do là một số người dùng sẵn lòng dâng đủ mọi thứ về cuộc sống của họ lên mạng xã hội, đôi khi là những điều tầm thường và đôi khi là những điều cực kỳ riêng tư. Tuy nhiên, tuyên bố này là sai lầm và dựa trên sự hiểu lầm về từ *privacy* (quyền riêng tư).

Có quyền riêng tư không có nghĩa là giữ bí mật mọi thứ; nó có nghĩa là có quyền tự do lựa chọn việc tiết lộ điều gì cho ai, điều gì cần công khai và điều gì cần giữ kín. Quyền riêng tư là một quyền quyết định: nó cho phép mỗi người quyết định họ muốn đứng ở đâu trên phổ giữa sự bí mật và tính minh bạch trong mỗi tình huống

[30]. Đây là một khía cạnh quan trọng trong sự tự do và quyền tự chủ của một con người.

Ví dụ, một người mắc phải một tình trạng y tế hiếm gặp có thể rất sẵn lòng cung cấp dữ liệu y tế riêng tư của họ cho các nhà nghiên cứu nếu điều đó có thể giúp phát triển các phương pháp điều trị cho tình trạng của họ. Tuy nhiên, người này phải có quyền lựa chọn ai có thể truy cập dữ liệu này và vì mục đích gì. Nếu thông tin về tình trạng của họ có thể cản trở việc tiếp cận bảo hiểm y tế hoặc việc làm, chẳng hạn, người này có lẽ sẽ thận trọng hơn nhiều trong việc chia sẻ dữ liệu của mình.

Khi dữ liệu được trích xuất từ con người thông qua hạ tầng giám sát, các quyền riêng tư không nhất thiết bị xói mòn mà thay vào đó được chuyển giao cho bên thu thập dữ liệu. Các công ty thu thập dữ liệu về cơ bản đang nói rằng: "Hãy tin tưởng chúng tôi sẽ làm điều đúng đắn với dữ liệu của bạn", điều này có nghĩa là quyền quyết định việc tiết lộ điều gì và giữ kín điều gì đã được chuyển từ cá nhân sang công ty.

Ngược lại, các công ty chọn giữ bí mật phần lớn kết quả của việc giám sát này, bởi vì việc tiết lộ nó sẽ bị coi là gây cảm giác đáng sợ và sẽ gây hại cho mô hình kinh doanh của họ (vốn dựa trên việc biết nhiều về con người hơn các công ty khác). Thông tin thân mật về người dùng chỉ được tiết lộ một cách gián tiếp—ví dụ, dưới dạng các công cụ để nhắm mục tiêu quảng cáo đến các nhóm người cụ thể (chẳng hạn như những người đang chịu đựng một căn bệnh cụ thể).

Ngay cả khi những người dùng cụ thể không thể bị tái định danh cá nhân từ nhóm người bị nhắm mục tiêu bởi một quảng cáo nhất định, họ đã mất đi quyền tự quyết đối với việc tiết lộ một số thông tin thân mật. Không phải người dùng quyết định điều gì được tiết lộ cho ai dựa trên sở thích cá nhân của họ—mà chính là công ty thực hiện quyền riêng tư với mục tiêu tối đa hóa lợi nhuận của nó.

Nhiều công ty muốn tránh bị *coi là* đáng sợ, tránh câu hỏi về việc thu thập dữ liệu của họ thực sự xâm phạm đến mức nào và thay vào đó tập trung vào việc quản lý nhận thức của người dùng. Và ngay cả những nhận thức này thường cũng được quản lý kém—ví dụ, một điều gì đó có thể đúng về mặt thực tế, nhưng nếu nó gợi lại những ký ức đau buồn, người dùng có thể không muốn bị nhắc lại về nó [35]. Với bất kỳ loại dữ liệu nào, chúng ta nên lường trước khả năng rằng nó có thể sai, không mong muốn hoặc không phù hợp theo một cách nào đó, và chúng ta cần xây dựng các cơ chế để xử lý những thất bại đó. Việc một thứ gì đó là "không mong muốn" hay "không phù hợp" tất nhiên phụ thuộc vào sự phán đoán của con người; các thuật toán không hề hay biết về những khái niệm như vậy trừ khi chúng ta lập trình chúng một cách rõ ràng để tôn trọng các nhu cầu của con người. Với tư cách là những kỹ sư của các hệ thống này, chúng ta phải khiêm tốn, chấp nhận và lập kế hoạch cho những sai sót như vậy.

Các thiết lập quyền riêng tư cho phép người dùng của một dịch vụ trực tuyến kiểm soát những khía cạnh nào trong dữ liệu của họ mà những người dùng khác có thể

thấy là điểm khởi đầu để trao lại một phần quyền kiểm soát cho người dùng. Tuy nhiên, bất kể thiết lập như thế nào, bản thân dịch vụ vẫn có quyền truy cập không hạn chế vào dữ liệu và được tự do sử dụng nó theo bất kỳ cách nào mà chính sách quyền riêng tư cho phép. Ngay cả khi dịch vụ hứa không bán dữ liệu cho bên thứ ba, nó thường tự cấp cho mình các quyền không hạn chế để xử lý và phân tích dữ liệu nội bộ, thường tiến xa hơn nhiều so với những gì người dùng có thể thấy rõ ràng.

Việc chuyển giao quyền riêng tư quy mô lớn từ cá nhân sang các tập đoàn như thế này là điều chưa từng có trong lịch sử [30]. Sự giám sát luôn tồn tại, nhưng trước đây nó thường tốn kém và được thực hiện thủ công, chứ không có khả năng mở rộng và tự động hóa. Các mối quan hệ tin cậy luôn tồn tại—ví dụ, giữa bệnh nhân và bác sĩ, hoặc giữa bị cáo và luật sư—nhưng trong những trường hợp này, việc sử dụng dữ liệu luôn được kiểm soát chặt chẽ bởi các ràng buộc về đạo đức, pháp lý và quy định. Các dịch vụ Internet đã khiến việc tích lũy lượng lớn thông tin nhạy cảm mà không có sự đồng ý có ý nghĩa trở nên dễ dàng hơn nhiều, cũng như việc sử dụng chúng ở quy mô khổng lồ mà người dùng không hiểu điều gì đang xảy ra với dữ liệu riêng tư của mình.

Dữ liệu dưới dạng Tài sản và Quyền lực

Vì dữ liệu hành vi là một sản phẩm phụ từ việc người dùng tương tác với một dịch vụ, nên đôi khi nó được gọi là "data exhaust" (khí thải dữ liệu)—ngụ ý rằng dữ liệu đó là vật liệu phế thải vô giá trị. Nếu nhìn theo cách này, phân tích hành vi và phân tích dự đoán có thể được xem như một hình thức tái chế nhằm trích xuất giá trị từ dữ liệu mà nếu không làm vậy sẽ bị vứt bỏ.

Sẽ chính xác hơn nếu chúng ta nhìn nhận theo chiều ngược lại. Từ góc độ kinh tế, nếu quảng cáo mục tiêu là thứ chi trả cho một dịch vụ, thì hoạt động của người dùng tạo ra dữ liệu hành vi có thể được coi là một hình thức lao động [36]. Ta thậm chí có thể tiến xa hơn và lập luận rằng ứng dụng mà người dùng tương tác chỉ đơn thuần là một phương tiện để dụ dỗ người dùng nạp ngày càng nhiều thông tin cá nhân vào hạ tầng giám sát [30]. Sự sáng tạo tuyệt vời của con người và các mối quan hệ xã hội thường được thể hiện trong các dịch vụ trực tuyến đang bị khai thác một cách đầy hoài nghi bởi cỗ máy trích xuất dữ liệu.

Dữ liệu cá nhân là một tài sản quý giá, được minh chứng bởi sự tồn tại của các bên môi giới dữ liệu (data brokers) hoạt động trong bí mật, thực hiện mua, tổng hợp, phân tích và bán lại dữ liệu cá nhân của mọi người, chủ yếu cho mục đích marketing [20]. Các startup được định giá dựa trên số lượng người dùng, hay còn gọi là "eyeballs" (lượt xem)—tức là dựa trên khả năng giám sát của chúng.

Vì dữ liệu có giá trị, nên nhiều người muốn có nó. Tất nhiên, các công ty muốn nó—đó là lý do tại sao ngay từ đầu họ đã thu thập nó. Nhưng các chính phủ cũng muốn nó, và họ có thể tìm cách giành lấy thông qua các thỏa thuận bí mật, cưỡng ép, bắt

buộc về pháp lý, hoặc đơn giản là trộm cắp [37]. Khi một công ty phá sản, dữ liệu cá nhân mà nó đã thu thập là một trong những tài sản bị đem bán. Và vì dữ liệu rất khó để bảo mật, các vụ vi phạm xảy ra với tần suất đáng lo ngại.

Những quan sát này đã khiến các nhà phê bình nói rằng dữ liệu không chỉ là một tài sản, mà còn là một "toxic asset" (tài sản độc hại) [37], hoặc ít nhất là "hazardous material" (vật liệu nguy hiểm) [38]. Có lẽ dữ liệu không phải là vàng mới, hay dầu mỏ mới, mà đúng hơn là uranium mới [39]. Ngay cả khi chúng ta nghĩ rằng mình có khả năng ngăn chặn việc lạm dụng dữ liệu, thì bất cứ khi nào chúng ta thu thập nó, chúng ta cần phải cân bằng giữa lợi ích và rủi ro khi nó rơi vào tay kẻ xấu. Các hệ thống máy tính có thể bị xâm nhập bởi tội phạm hoặc các cơ quan tình báo nước ngoài thù địch, dữ liệu có thể bị rò rỉ bởi những người bên trong, công ty có thể rơi vào tay những nhà quản lý vô đạo đức không chia sẻ các giá trị của chúng ta, hoặc quốc gia có thể bị tiếp quản bởi một chế độ không ngần ngại ép buộc chúng ta phải bàn giao dữ liệu.

Như quan sát đó gợi ý, khi thu thập dữ liệu, chúng ta cần xem xét không chỉ môi trường chính trị hiện tại, mà cả tất cả các chính phủ có thể có trong tương lai. Không có gì đảm bảo rằng mọi chính phủ được bầu trong tương lai sẽ tôn trọng nhân quyền và các quyền tự do dân sự, và như Bruce Schneier đã quan sát: "Thật là một sự thiếu vệ sinh dân sự khi cài đặt các công nghệ mà một ngày nào đó có thể tạo điều kiện cho một nhà nước cảnh sát" [40].

"Kiến thức là sức mạnh," như câu châm ngôn cổ đã nói. Và hơn thế nữa, "Xem xét kỹ lưỡng người khác trong khi bản thân lại tránh bị xem xét là một trong những hình thức quyền lực quan trọng nhất" [41]. Đây là lý do tại sao các chính phủ độc tài muốn giám sát: nó mang lại cho họ quyền lực để kiểm soát dân cư. Mặc dù các công ty công nghệ ngày nay không công khai tìm kiếm quyền lực chính trị, nhưng dữ liệu và kiến thức mà họ đã tích lũy được—phần lớn là một cách lén lút, nằm ngoài sự giám sát của công chúng—vẫn mang lại cho họ rất nhiều quyền lực đối với cuộc sống của chúng ta [42].

Nhớ về Cách mạng Công nghiệp

Dữ liệu là đặc điểm định nghĩa của kỷ nguyên thông tin. Internet, việc lưu trữ và xử lý dữ liệu, cùng với tự động hóa dựa trên phần mềm đang có tác động lớn đến nền kinh tế toàn cầu và xã hội loài người. Khi cuộc sống hàng ngày và tổ chức xã hội của chúng ta đã bị thay đổi bởi công nghệ thông tin, và có lẽ sẽ tiếp tục thay đổi mạnh mẽ trong những thập kỷ tới, những sự so sánh với Cách mạng Công nghiệp hiện lên trong tâm trí [17, 26].

Cách mạng Công nghiệp diễn ra thông qua những tiến bộ lớn về công nghệ và nông nghiệp, mang lại sự tăng trưởng kinh tế bền vững và cải thiện đáng kể mức sống về lâu dài—tuy nhiên nó cũng đi kèm với những vấn đề lớn. Ô nhiễm không khí (do

khỏi và các quá trình hóa học) và nguồn nước (từ chất thải công nghiệp và con người) thật kinh khủng. Các chủ nhà máy sống trong sự xa hoa, trong khi công nhân thành thị thường phải sống trong những khu nhà chật chội, mất vệ sinh và làm việc nhiều giờ trong điều kiện khắc nghiệt. Lao động trẻ em rất phổ biến, bao gồm cả những công việc nguy hiểm và lương thấp trong các mỏ khai thác.

Phải mất một thời gian dài trước khi các biện pháp bảo vệ được thiết lập, chẳng hạn như các quy định bảo vệ môi trường, các giao thức an toàn cho nơi làm việc, luật cấm lao động trẻ em và kiểm tra vệ sinh thực phẩm. Chắc chắn rằng chi phí kinh doanh đã tăng lên khi các nhà máy không còn được phép đổ chất thải xuống sông, bán thực phẩm nhiễm bẩn, hoặc bóc lột công nhân. Nhưng toàn xã hội đã được hưởng lợi rất lớn từ những quy định này, và ít ai trong chúng ta muốn quay trở lại thời kỳ trước đó [17].

Giống như Cách mạng Công nghiệp có một mặt tối cần được quản lý, quá trình chuyển đổi của chúng ta sang kỷ nguyên thông tin cũng có những vấn đề lớn mà chúng ta cần đối mặt và giải quyết [43, 44]. Việc thu thập và sử dụng dữ liệu là một trong những vấn đề đó. Theo lời của Bruce Schneier [26]:

Dữ liệu là vấn đề ô nhiễm của kỷ nguyên thông tin, và bảo vệ quyền riêng tư là thách thức về môi trường. Hầu hết tất cả các máy tính đều tạo ra thông tin. Nó tồn tại dai dẳng, tích tụ dần. Cách chúng ta đối phó với nó—cách chúng ta kiểm soát và cách chúng ta xử lý nó—là yếu tố then chốt đối với sức khỏe của nền kinh tế thông tin của chúng ta. Giống như việc chúng ta nhìn lại những thập kỷ đầu của thời đại công nghiệp ngày nay và tự hỏi làm thế nào tổ tiên chúng ta có thể phớt lờ ô nhiễm trong cơn lốc xây dựng thế giới công nghiệp, các cháu của chúng ta sẽ nhìn lại chúng ta trong những thập kỷ đầu của kỷ nguyên thông tin này và đánh giá chúng ta về cách chúng ta giải quyết thách thức thu thập và lạm dụng dữ liệu.

Chúng ta nên cố gắng để khiến chúng ta tự hào.

Luật pháp và Tự điều tiết

Các luật bảo vệ dữ liệu có thể giúp bảo vệ quyền của cá nhân. Ví dụ, GDPR quy định rằng dữ liệu cá nhân phải được “thu thập cho các mục đích cụ thể, rõ ràng và hợp pháp và không được xử lý thêm theo cách không tương thích với các mục đích đó” và phải “đầy đủ, có liên quan và giới hạn ở những gì cần thiết liên quan đến các mục đích mà [nó] được xử lý” [32].

Tuy nhiên, nguyên tắc *giảm thiểu dữ liệu* (*data minimization*) này đi ngược lại trực tiếp với triết lý của big data, vốn là tối đa hóa việc thu thập dữ liệu, kết hợp dữ liệu đã thu thập với các dataset khác, cũng như thử nghiệm và khám phá để tạo ra những hiểu biết mới. Khám phá có nghĩa là sử dụng dữ liệu cho các mục đích không lường

trước được, điều mà GDPR tuyên bố là trái ngược với các mục đích "được chỉ định và rõ ràng" mà dữ liệu phải được thu thập. Mặc dù quy định này đã có một số tác động đến ngành quảng cáo trực tuyến [45], nhưng việc thực thi còn yếu [46] và dường như chưa dẫn đến nhiều thay đổi về văn hóa cũng như thực hành trong toàn bộ ngành công nghệ rộng lớn hơn.

Các công ty thu thập nhiều dữ liệu về con người nhìn chung phản đối các quy định vì coi đó là gánh nặng và là rào cản đối với sự đổi mới. Ở một mức độ nào đó, sự phản đối này là có cơ sở. Ví dụ, việc chia sẻ dữ liệu y tế tạo ra những rủi ro rõ ràng về quyền riêng tư nhưng cũng mang lại những cơ hội tiềm năng: bao nhiêu ca tử vong có thể được ngăn chặn nếu việc phân tích dữ liệu có thể giúp chúng ta đạt được khả năng chẩn đoán tốt hơn hoặc tìm ra các phương pháp điều trị hiệu quả hơn [47]? Việc quản lý quá mức có thể ngăn cản những đột phá như vậy. Thật khó để cân bằng giữa các cơ hội tiềm năng với các rủi ro [41].

Về cơ bản, chúng ta cần một sự thay đổi về văn hóa trong ngành công nghệ đối với dữ liệu cá nhân. Chúng ta nên ngừng coi người dùng là các chỉ số cần được tối ưu hóa, và hãy nhớ rằng họ là con người, những người xứng đáng được tôn trọng, có phẩm giá và có agency (quyền tự quyết). Chúng ta nên tự điều tiết các thực hành thu thập và xử lý dữ liệu của mình nhằm thiết lập và duy trì sự tin cậy của những người đang phụ thuộc vào phần mềm của chúng ta [48]. Và chúng ta nên tự giác giáo dục người dùng cuối về cách dữ liệu của họ được sử dụng thay vì để họ rơi vào tình trạng không biết gì.

Chúng ta nên cho phép mỗi cá nhân duy trì quyền riêng tư của họ (tức là quyền kiểm soát dữ liệu của chính họ) và không đánh cắp quyền kiểm soát đó thông qua việc giám sát. Quyền kiểm soát dữ liệu cá nhân của chúng ta giống như môi trường tự nhiên của một vườn quốc gia: nếu chúng ta không bảo vệ và chăm sóc nó một cách rõ ràng, nó sẽ bị hủy hoại. Đó sẽ là bi kịch của tài nguyên chung (tragedy of the commons), và tất cả chúng ta sẽ trở nên tồi tệ hơn vì điều đó. Việc giám sát tràn lan không phải là điều không thể tránh khỏi. Chúng ta vẫn có thể ngăn chặn nó.

Là bước đầu tiên, chúng ta không nên lưu giữ dữ liệu mãi mãi, mà nên loại bỏ nó ngay khi không còn cần thiết, và giảm thiểu những gì chúng ta thu thập ngay từ đầu [48, 49]. Dữ liệu mà bạn không có là dữ liệu không thể bị rò rỉ, bị đánh cắp, hoặc bị chính phủ ép buộc phải bàn giao. Nhìn chung, những thay đổi về văn hóa và thái độ là cần thiết. Với tư cách là những người làm việc trong lĩnh vực công nghệ, nếu chúng ta không xem xét tác động xã hội từ công việc của mình, chúng ta đang không làm tròn trách nhiệm [50].

Tóm tắt

Điều này đưa chúng ta đến phần kết của cuốn sách. Chúng ta đã đi qua rất nhiều nội dung:

- Trong Chương 1, chúng ta đã đối chiếu các hệ thống analytical và operational, so sánh cloud với self-hosting, cân nhắc giữa các hệ thống distributed và single-node, và thảo luận về việc cân bằng giữa nhu cầu kinh doanh với nhu cầu của người dùng.
- Trong Chương 2, chúng ta đã tìm hiểu cách xác định một số yêu cầu nonfunctional, chẳng hạn như hiệu năng, độ tin cậy, khả năng mở rộng và khả năng bảo trì.
- Trong Chương 3, chúng ta đã khám phá một loạt các mô hình dữ liệu, bao gồm các mô hình relational, document và graph, event sourcing, và DataFrames. Chúng ta cũng đã xem xét các ví dụ về các ngôn ngữ truy vấn khác nhau, bao gồm SQL, Cypher, SPARQL, Datalog và GraphQL.
- Trong Chương 4, chúng ta đã thảo luận về các storage engine cho OLTP (LSM-trees và B-trees) và analytics (column-oriented storage), cũng như các index để truy hồi thông tin (full-text và vector search).
- Trong Chương 5, chúng ta đã xem xét các cách khác nhau để encoding các đối tượng dữ liệu thành các byte và cách hỗ trợ sự tiến hóa khi các yêu cầu thay đổi. Chúng ta cũng đã so sánh một số cách mà dữ liệu chảy giữa các process: thông qua các database, các service calls, workflow engines và các kiến trúc event-driven.
- Trong Chương 6, chúng ta đã nghiên cứu các trade-off giữa các cơ chế replication single-leader, multi-leader và leaderless. Chúng ta cũng đã xem xét các mô hình consistency như read-after-write consistency và các sync engine cho phép client hoạt động offline.
- Trong Chương 7, chúng ta đã xem xét sharding, bao gồm các chiến lược để rebalancing, request routing và secondary index.
- Trong Chương 8, chúng ta đã đề cập đến các transaction, xem xét tính durability, cách đạt được các isolation level khác nhau (read committed, snapshot isolation, và serializable), cũng như cách đảm bảo tính atomicity trong các distributed transactions.
- Trong Chương 9, chúng ta đã khảo sát các vấn đề cơ bản xảy ra trong các distributed systems (network faults và độ trễ, lỗi clock, process pauses, crashes) và thấy rằng chúng khiến việc triển khai chính xác ngay cả một thứ tưởng chừng đơn giản như một lock trở nên khó khăn.

- Trong Chương 10, chúng ta đã đi sâu tìm hiểu các dạng consensus khác nhau và mô hình consistency (linearizability) mà nó cho phép.
- Trong Chương 11, chúng ta đã nghiên cứu kỹ về batch processing, xây dựng từ các chuỗi công cụ Unix đơn giản đến các bộ xử lý batch phân tán quy mô lớn sử dụng distributed filesystems hoặc object stores.
- Trong Chương 12, chúng ta đã tổng quát hóa batch processing thành stream processing và thảo luận về các message broker nền tảng, CDC, fault tolerance, cùng các processing patterns như streaming joins.
- Trong Chương 13, chúng ta đã khám phá triết lý về các streaming systems cho phép tích hợp các hệ thống dữ liệu khác biệt, giúp các hệ thống có thể tiến hóa và các ứng dụng có thể scale dễ dàng hơn.

Cuối cùng, trong chương cuối này, chúng ta lùi lại một bước để xem xét một số khía cạnh đạo đức khi xây dựng các data-intensive applications. Chúng ta thấy rằng mặc dù dữ liệu có thể được dùng để làm điều tốt, nó cũng có thể gây ra tác hại đáng kể: đưa ra các quyết định ảnh hưởng nghiêm trọng đến cuộc sống con người và khó có thể kháng cáo, dẫn đến sự phân biệt đối xử và bóc lột, bình thường hóa việc giám sát, và làm lộ lọt thông tin riêng tư. Chúng ta cũng đối mặt với rủi ro bị data breaches, và có thể thấy rằng việc sử dụng dữ liệu với ý định tốt vẫn có thể dẫn đến những hậu quả không mong muốn.

Với tầm ảnh hưởng lớn lao mà phần mềm và dữ liệu mang lại cho thế giới, chúng ta với tư cách là các engineer phải ghi nhớ rằng mình mang trách nhiệm hướng tới kiểu thế giới mà chúng ta muốn sống: một thế giới đối xử với con người bằng sự nhân văn và tôn trọng. Hãy cùng nhau làm việc hướng tới mục tiêu đó.

Footnotes

References

- [1] David Schmudde. “What If Data Is a Bad Idea?” *schmud.de*, August 2024. Archived at perma.cc/ZXU5-XMCT
- [2] Association for Computing Machinery. “ACM Code of Ethics and Professional Conduct.” *acm.org*, 2018. Archived at perma.cc/SEA8-CMB8
- [3] Igor Perisic. “Making Hard Choices: The Quest for Ethics in Machine Learning.” *linkedin.com*, November 2016. Archived at perma.cc/DGF8-KNT7
- [4] John Naughton. “Algorithm Writers Need a Code of Conduct.” *theguardian.com*, December 2015. Archived at perma.cc/TBG2-3NG6
- [5] Deborah G. Johnson and Mario Verdicchio. “Ethical AI Is Not About AI.” *Communications of the ACM*, volume 66, issue 2, pages 32–34, January 2023. [doi:10.1145/3576932](https://doi.org/10.1145/3576932)
- [6] Ben Green. “‘Good’ Isn’t Good Enough.” At *NeurIPS Joint Workshop on AI for Social Good*, December 2019. Archived at perma.cc/H4LN-7VY3

- [7] Marc Steen. “Ethics as a Participatory and Iterative Process.” *Communications of the ACM*, volume 66, issue 5, pages 27–29, April 2023. [doi:10.1145/3550069](#)
- [8] Logan Kugler. “What Happens When Big Data Blunders?” *Communications of the ACM*, volume 59, issue 6, pages 15–16, June 2016. [doi:10.1145/2911975](#)
- [9] Miri Zilka. “Algorithms and the Criminal Justice System: Promises and Challenges in Deployment and Research.” At *University of Cambridge Security Seminar Series*, March 2023. Archived at [archive.org](#)
- [10] Bill Davidow. “Welcome to Algorithmic Prison.” *theatlantic.com*, February 2014. Archived at [archive.org](#)
- [11] Don Peck. “They’re Watching You at Work.” *theatlantic.com*, December 2013. Archived at [perma.cc/YR9T-6M38](#)
- [12] Leigh Alexander. “Is an Algorithm Any Less Racist Than a Human?” *theguardian.com*, August 2016. Archived at [perma.cc/XP93-DSVX](#)
- [13] Jesse Emspak. “How a Machine Learns Prejudice.” *scientificamerican.com*, December 2016. [perma.cc/R3L5-55E6](#)
- [14] Rohit Chopra, Kristen Clarke, Charlotte A. Burrows, and Lina M. Khan. “Joint Statement on Enforcement Efforts Against Discrimination and Bias in Automated Systems.” *ftc.gov*, April 2023. Archived at [perma.cc/YY4Y-RCCA](#)
- [15] Maciej Ceglowski. “The Moral Economy of Tech.” *idlewords.com*, June 2016. Archived at [perma.cc/L8XV-BKTD](#)
- [16] Greg Nichols. “Artificial Intelligence in Healthcare Is Racist.” *zdnet.com*, November 2020. Archived at [perma.cc/3MKW-YKRS](#)
- [17] Cathy O’Neil. *Weapons of Math Destruction: How Big Data Increases Inequality and Threatens Democracy*. Crown Publishing, 2016. ISBN: 9780553418811
- [18] Julia Angwin. “Make Algorithms Accountable.” *nytimes.com*, August 2016. Archived at [archive.org](#)
- [19] Bryce Goodman and Seth Flaxman. “European Union Regulations on Algorithmic Decision-Making and a ‘Right to Explanation.’” At *ICML Workshop on Human Interpretability in Machine Learning*, June 2016. Archived at [arxiv.org](#)
- [20] United States Senate Committee on Commerce, Science, and Transportation, Office of Oversight and Investigations, Majority Staff. “A Review of the Data Broker Industry: Collection, Use, and Sale of Consumer Data for Marketing Purposes.” Staff Report, *commerce.senate.gov*, December 2013. Archived at [perma.cc/32NV-YWLQ](#)
- [21] Stephanie Assad, Robert Clark, Daniel Ershov, and Lei Xu. “Algorithmic Pricing and Competition: Empirical Evidence from the German Retail Gasoline Market.” *Journal of Political Economy*, volume 132, issue 3, pages 723–771, March 2024. [doi:10.1086/726906](#)
- [22] Donella H. Meadows and Diana Wright. *Thinking in Systems: A Primer*. Chelsea Green Publishing, 2008. ISBN: 9781603580557
- [23] Daniel J. Bernstein. “Listening to a ‘big data’/‘data science’ talk. Mentally translating ‘data’ to ‘surveillance’: ‘...everything starts with surveillance...’” *x.com*, May 2015. Archived at [perma.cc/EY3D-WBBJ](#)
- [24] Marc Andreessen. “Why Software Is Eating the World.” *a16z.com*, August 2011. Archived at [perma.cc/3DCC-W3G6](#)
- [25] J. M. Porup. “‘Internet of Things’ Security Is Hilariously Broken and Getting Worse.” *arstechnica.com*, January 2016. Archived at [archive.org](#)

- [26] Bruce Schneier. *Data and Goliath: The Hidden Battles to Collect Your Data and Control Your World*. W. W. Norton, 2015. ISBN: 9780393352177
- [27] The Grugq. “Nothing to Hide.” *grugq.tumblr.com*, April 2016. Archived at perma.cc/BL95-8W5M
- [28] Federal Trade Commission. “FTC Takes Action Against General Motors for Sharing Drivers’ Precise Location and Driving Behavior Data Without Consent.” *ftc.gov*, January 2025. Archived at perma.cc/3XGV-3HRD
- [29] Tony Beltramelli. “Deep-Spying: Spying Using Smartwatch and Deep Learning.” Masters thesis, IT University of Copenhagen, December 2015. Archived at arxiv.org
- [30] Shoshana Zuboff. “Big Other: Surveillance Capitalism and the Prospects of an Information Civilization.” *Journal of Information Technology*, volume 30, issue 1, pages 75–89, April 2015. doi: [10.1057/jit.2015.5](https://doi.org/10.1057/jit.2015.5)
- [31] Michiel Rhoen. “Beyond Consent: Improving Data Protection Through Consumer Protection Law.” *Internet Policy Review*, volume 5, issue 1, March 2016. doi: [10.14763/2016.1.404](https://doi.org/10.14763/2016.1.404)
- [32] “Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016.” *Official Journal of the European Union*, L 119/1, May 2016.
- [33] UK Information Commissioner’s Office. “What Is the ‘Legitimate Interests’ Basis?” *ico.org.uk*. Archived at perma.cc/W8XR-F7ML
- [34] Tristan Harris. “How a Handful of Tech Companies Control Billions of Minds Every Day.” At *TED2017*, April 2017. Archived at archive.org
- [35] Carina C. Zona. “Consequences of an Insightful Algorithm.” At *GOTO Berlin*, November 2016.
- [36] Imanol Arrieta Ibarra, Leonard Goff, Diego Jiménez Hernández, Jaron Lanier, and E. Glen Weyl. “Should We Treat Data as Labor? Moving Beyond ‘Free.’” *American Economic Association Papers Proceedings*, volume 108, pages 38–42, May 2018. doi: [10.1257/pandp.20181003](https://doi.org/10.1257/pandp.20181003)
- [37] Bruce Schneier. “Data Is a Toxic Asset, So Why Not Throw It Out?” *schneier.com*, March 2016. Archived at perma.cc/4GZH-WR3D
- [38] Cory Scott. “Data is not toxic—which implies no benefit—but rather hazardous material, where we must balance need vs. want.” *x.com*, March 2016. Archived at perma.cc/CLV7-JF2E
- [39] Mark Pesce. “Data Is The New Uranium—Incredibly Powerful And Amazingly Dangerous.” *theregister.com*, November 2024. Archived at perma.cc/NV8B-GYGV
- [40] Bruce Schneier. “Mission Creep: When Everything Is Terrorism.” *schneier.com*, July 2013. Archived at perma.cc/QB2C-5RCE
- [41] Lena Ulbricht and Maximilian von Grafenstein. “Big Data: Big Power Shifts?” *Internet Policy Review*, volume 5, issue 1, March 2016. doi: [10.14763/2016.1.406](https://doi.org/10.14763/2016.1.406)
- [42] Ellen P. Goodman and Julia Powles. “Facebook and Google: Most Powerful and Secretive Empires We’ve Ever Known.” *theguardian.com*, September 2016. Archived at perma.cc/8UJA-43G6
- [43] Judy Estrin and Sam Gill. “The World Is Choking on Digital Pollution.” *washingtonmonthbly.com*, January 2019. Archived at perma.cc/3VHF-C6UC
- [44] A. Michael Froomkin. “Regulating Mass Surveillance as Privacy Pollution: Learning from Environmental Impact Statements.” *University of Illinois Law Review*, volume 2015, issue 5, August 2015. Archived at perma.cc/24ZL-VK2T

- [45] Pengyuan Wang, Li Jiang, and Jian Yang. “The Early Impact of GDPR Compliance on Display Advertising: The Case of an Ad Publisher.” *Journal of Marketing Research*, volume 61, issue 1, April 2023. doi:10.1177/00222437231171848
- [46] Johnny Ryan. “Don’t Be Fooled by Meta’s Fine for Data Breaches.” *The Economist*, May 2023. Archived at perma.cc/VCR6-55HR
- [47] Jessica Leber. “Your Data Footprint Is Affecting Your Life in Ways You Can’t Even Imagine.” *fastcompany.com*, March 2016. Archived at archive.org
- [48] Maciej Cegłowski. “Haunted by Data.” *idlewords.com*, October 2015. Archived at archive.org
- [49] Sam Thielman. “You Are Not What You Read: Librarians Purge User Data to Protect Privacy.” *theguardian.com*, January 2016. Archived at archive.org
- [50] Jez Humble. “It’s a cliché that people get into tech to ‘change the world.’ So then, you have to actually consider what the impact of your work is on the world. The idea that you can or should exclude societal and political discussions in tech is idiotic. It means you’re not doing your job.” *x.com*, April 2021. Archived at perma.cc/3NYS-MHLC

Glossary

NOTE

Các định nghĩa trong glossary này ngắn gọn và đơn giản, nhằm truyền tải ý tưởng cốt lõi chứ không bao hàm toàn bộ những sắc thái tinh tế của một thuật ngữ. Để biết thêm chi tiết, hãy tham khảo các phần dẫn chiếu trong văn bản chính.

asynchronous

Không chờ đợi một thứ gì đó hoàn tất (ví dụ: gửi dữ liệu qua mạng tới một node khác), và không đưa ra bất kỳ giả định nào về việc nó sẽ mất bao lâu. Xem “Synchronous Versus Asynchronous Replication”, “Synchronous Versus Asynchronous Networks”, và “System Model and Reality”.

atomic

1. Trong ngữ cảnh của concurrency, mô tả một operation có vẻ như có hiệu lực tại một thời điểm duy nhất, sao cho một tiến trình concurrent khác không bao giờ gặp phải operation đó trong trạng thái “half-finished” (chưa hoàn tất). Xem thêm *isolation*.
2. Trong ngữ cảnh của transaction, mô tả việc nhóm một tập hợp các lệnh write lại với nhau sao cho chúng phải được commit tất cả hoặc được rollback tất cả, ngay cả khi có lỗi xảy ra. Xem “Atomicity” và “Two-Phase Commit”.

backpressure

Buộc bên gửi dữ liệu phải chậm lại khi bên nhận không thể theo kịp. Còn được gọi là *flow control* (kiểm soát luồng). Xem “When an Overloaded System Won’t Recover”.

batch process

Một phép tính lấy một tập hợp dữ liệu cố định (và thường là lớn) làm đầu vào và tạo ra dữ liệu khác làm đầu ra mà không làm thay đổi dữ liệu đầu vào. Xem Chương 11.

bounded

Có một giới hạn trên hoặc kích thước đã biết. Được sử dụng, ví dụ, trong ngữ cảnh độ trễ mạng (xem “Timeouts and Unbounded Delays”) và các dataset (xem phần giới thiệu của Chương 12).

Byzantine fault

Một fault đặc trưng bởi việc một node hoạt động không chính xác theo một cách tùy ý (ví dụ: bằng cách gửi các thông điệp mâu thuẫn hoặc độc hại tới các node khác). Xem “Byzantine Faults”.

cache

Một thành phần ghi nhớ các dữ liệu được sử dụng gần đây để tăng tốc các lần đọc sau này đối với cùng dữ liệu đó. Thông thường nó không đầy đủ; bất kỳ dữ liệu nào thiếu trong cache đều phải được lấy từ một hệ thống lưu trữ dữ liệu bên dưới, chậm hơn, vốn có bản sao đầy đủ của dữ liệu đó.

CAP theorem

Một kết quả lý thuyết bị hiểu lầm rộng rãi và không hữu ích trong thực tế. Xem “The CAP theorem”.

causality

Sự phụ thuộc giữa các sự kiện phát sinh khi một thứ “happens before” (xảy ra trước) một thứ khác trong một hệ thống—ví dụ, một sự kiện xảy ra sau đó nhằm phản hồi, xây dựng dựa trên, hoặc cần được hiểu dưới góc độ của một sự kiện xảy ra trước đó. Xem “The happens-before relation and concurrency”.

consensus

Một vấn đề cơ bản trong tính toán phân tán liên quan đến việc khiến nhiều node đồng thuận về một điều gì đó (ví dụ: node nào nên là leader cho một database cluster). Vấn đề này khó hơn nhiều so với vẻ ngoài của nó. Xem “Consensus”.

data warehouse

Một database mà trong đó dữ liệu từ nhiều hệ thống OLTP đã được kết hợp và chuẩn bị để sử dụng cho các mục đích phân tích. Xem “Data Warehousing”.

declarative

Mô tả các thuộc tính mà một thứ nên có, nhưng không phải là các bước chính xác để đạt được nó. Trong ngữ cảnh của các truy vấn database, một query optimizer sẽ nhận một truy vấn declarative và quyết định cách tốt nhất để thực thi nó. Xem “Terminology: Declarative Query Languages”.

denormalize

Đưa một lượng dư thừa hoặc trùng lặp nhất định vào một dataset đã được *normalized* (chuẩn hóa), thường dưới dạng một *cache* hoặc *index*, nhằm tăng tốc việc đọc. Một giá trị denormalized là một loại kết quả truy vấn được tính toán trước, tương tự như một materialized view. Xem “Normalization, Denormalization, and Joins”.

derived data

Một dataset được tạo ra từ các dữ liệu khác thông qua một quy trình có thể lặp lại, mà bạn có thể chạy lại nếu cần thiết. Thông thường, derived data được cần đến để tăng tốc một loại truy cập đọc cụ thể vào dữ liệu. Indexes, caches, và materialized views là các ví dụ về derived data. Xem “Systems of Record and Derived Data”.

deterministic

Mô tả một hàm luôn tạo ra cùng một đầu ra nếu bạn cung cấp cho nó cùng một đầu vào. Điều này có nghĩa là nó không thể phụ thuộc vào các số ngẫu nhiên, thời gian trong ngày, giao tiếp mạng, hoặc các thứ không thể dự đoán khác. Xem “The Power of Determinism”.

distributed

Chạy trên nhiều node được kết nối bởi một mạng. Đặc trưng bởi các *partial failures* (lỗi cục bộ): một phần của hệ thống có thể bị hỏng trong khi các phần khác vẫn đang hoạt động, và thường thì phần mềm không thể biết chính xác cái gì đang bị hỏng. Xem “Faults and Partial Failures”.

durable

Lưu trữ dữ liệu theo cách mà bạn tin rằng nó sẽ không bị mất, ngay cả khi có nhiều lỗi khác nhau xảy ra. Xem “Durability”.

ETL

Extract–transform–load (trích xuất–biến đổi– nạp). Quy trình trích xuất dữ liệu từ một cơ sở dữ liệu nguồn, biến đổi nó thành một dạng phù hợp hơn cho các truy vấn phân tích, và nạp nó vào một data warehouse hoặc hệ thống batch processing. Xem “Data Warehousing”.

failover

Trong các hệ thống có một single leader, failover là quy trình chuyển vai trò lãnh đạo từ một node này sang một node khác. Xem “Handling Node Outages”.

fault-tolerant

Có khả năng tự động phục hồi nếu có lỗi xảy ra (ví dụ: nếu một máy bị crash hoặc một liên kết mạng bị lỗi). Xem “Reliability and Fault Tolerance”.

flow control

Xem *backpressure*.

follower

Một replica không trực tiếp chấp nhận bất kỳ lệnh ghi nào từ client, mà chỉ xử lý các thay đổi dữ liệu mà nó nhận được từ một leader. Còn được gọi là *secondary*, *read replica*, hoặc *hot standby*. Xem “Single-Leader Replication”.

full-text search

Tìm kiếm văn bản bằng các từ khóa tùy ý, thường đi kèm với các tính năng bổ sung như khớp các từ có cách viết tương tự hoặc các từ đồng nghĩa. Một full-text index là một loại *secondary index* hỗ trợ các truy vấn như vậy. Xem “Full-Text Search”.

graph

Một cấu trúc dữ liệu bao gồm các *vertices* (những thứ mà bạn có thể tham chiếu tới, còn được gọi là *nodes* hoặc *entities*) và các *edges* (các kết nối từ một vertex này

sang một vertex khác, còn được gọi là *relationships* hoặc *arcs*). Xem “Graph-Like Data Models”.

hash

Một hàm biến đổi đầu vào thành một con số trông có vẻ ngẫu nhiên. Cùng một đầu vào sẽ luôn trả về cùng một con số làm đầu ra. Hai đầu vào khác nhau có khả năng sẽ có hai con số đầu ra khác nhau, mặc dù hai đầu vào khác nhau vẫn có thể tạo ra cùng một đầu ra (điều này được gọi là *collision*). Xem “Sharding by Hash of Key”.

idempotent

Mô tả một thao tác có thể được thử lại một cách an toàn; nếu nó được thực thi nhiều hơn một lần, nó có tác dụng tương tự như khi chỉ được thực thi một lần. Xem “Idempotence”.

index

Một cấu trúc dữ liệu cho phép bạn tìm kiếm hiệu quả tất cả các bản ghi có một giá trị cụ thể trong một field cụ thể. Xem “Storage and Indexing for OLTP”.

isolation

Trong ngữ cảnh của các transaction, mô tả mức độ mà các transaction đang thực thi đồng thời có thể gây nhiễu lẫn nhau. Cô lập *Serializable* cung cấp các đảm bảo mạnh mẽ nhất, nhưng các isolation levels yếu hơn cũng được sử dụng. Xem “Isolation”.

join

Gom các bản ghi có điểm chung lại với nhau. Thường được sử dụng nhất khi một bản ghi có một tham chiếu tới một bản ghi khác (một foreign key, một tham chiếu document, hoặc một edge trong một graph) và một truy vấn cần lấy bản ghi mà tham chiếu đó trỏ tới. Xem “Normalization, Denormalization, and Joins” và “Joins and Grouping”.

leader

Khi dữ liệu hoặc một service được replication trên nhiều node, leader là replica được chỉ định để được phép thực hiện các thay đổi. Một leader có thể được bầu thông qua một protocol hoặc được chọn thủ công bởi một administrator. Còn được gọi là *primary* hoặc *source*. Xem “Single-Leader Replication”.

linearizable

Hoạt động như thể chỉ có một bản sao duy nhất của dữ liệu trong hệ thống, bản sao này được cập nhật bởi các thao tác atomic. Xem “Linearizability”.

locality

Một sự tối ưu hóa hiệu năng: đặt nhiều phần dữ liệu vào cùng một nơi nếu chúng thường xuyên được cần đến cùng một lúc. Xem “Data locality for reads and writes”.

lock

Một cơ chế để đảm bảo rằng chỉ có một thread, node, hoặc transaction có thể truy cập vào một thứ gì đó, và bất kỳ ai khác muốn truy cập vào cùng thứ đó đều phải chờ cho đến khi lock được giải phóng. Xem “Two-Phase Locking” và “Distributed Locks and Leases”.

log

Một tệp chỉ cho phép append để lưu trữ dữ liệu. Một *write-ahead log* được sử dụng để giúp một storage engine có khả năng chống chịu trước các lỗi crash (xem “Making B-trees reliable”), một storage engine *log-structured* sử dụng các log làm định dạng lưu trữ chính (xem “Log-Structured Storage”), một *replication log* được sử dụng để sao chép các lệnh ghi từ một leader sang các follower (xem “Single-Leader Replication”), và một *event log* có thể đại diện cho một data stream (xem “Log-Based Message Brokers”).

materialize

Thực hiện một phép tính một cách eager (ngay lập tức) và ghi ra kết quả của nó, trái ngược với việc tính toán nó theo yêu cầu (on demand) khi được yêu cầu. Xem “Event Sourcing and CQRS”.

node

Một instance của phần mềm chạy trên một máy tính, thực hiện giao tiếp với các node khác thông qua một mạng để hoàn thành một tác vụ.

normalized

Được cấu trúc theo cách không có sự dư thừa hay trùng lặp. Trong một cơ sở dữ liệu đã được normalization, khi một phần dữ liệu thay đổi, bạn chỉ cần thay đổi nó ở duy nhất một nơi, thay vì phải thay đổi nhiều bản sao ở nhiều nơi khác nhau. Xem “Normalization, Denormalization, and Joins”.

OLAP

Xử lý phân tích trực tuyến (Online analytical processing). Một pattern truy cập đặc trưng bởi việc aggregation (ví dụ: count, sum, average) trên một số lượng lớn các bản ghi. Xem “Operational Versus Analytical Systems”.

OLTP

Xử lý giao dịch trực tuyến (Online transaction processing). Một pattern truy cập đặc trưng bởi các truy vấn nhanh thực hiện đọc hoặc ghi một số lượng nhỏ các bản ghi, thường được index bằng key. Xem “Operational Versus Analytical Systems”.

percentile

Một cách để đo lường sự phân phối của các giá trị bằng cách đếm xem có bao nhiêu giá trị nằm trên hoặc dưới một ngưỡng nhất định. Ví dụ, thời gian phản hồi ở percentile thứ 95 trong một khoảng thời gian nào đó là thời gian t sao cho 95%

các yêu cầu trong khoảng thời gian đó hoàn thành trong ít hơn t , và 5% mất nhiều thời gian hơn t . Xem “Describing Performance”.

primary key

Một giá trị (thường là một số hoặc một chuỗi) dùng để định danh duy nhất một bản ghi. Trong nhiều ứng dụng, primary key được hệ thống tạo ra khi một bản ghi được khởi tạo (ví dụ: theo thứ tự hoặc ngẫu nhiên); chúng thường không do người dùng thiết lập. Xem thêm *secondary index*.

quorum

Số lượng node tối thiểu cần phải bỏ phiếu cho một thao tác trước khi nó có thể được coi là thành công. Xem “Using quorums for reading and writing”.

rebalance

Di chuyển dữ liệu hoặc các service từ node này sang node khác để phân phối tải một cách công bằng. Xem “Sharding of Key-Value Data”.

replication

Giữ một bản sao của cùng một dữ liệu trên nhiều node (*replica*) để dữ liệu đó vẫn có thể truy cập được nếu một node không thể kết nối tới. Xem Chương 6.

schema

Một mô tả về cấu trúc của dữ liệu, bao gồm các field và datatypes của nó. Việc dữ liệu có tuân thủ một schema hay không có thể được kiểm tra tại các thời điểm khác nhau trong vòng đời của dữ liệu (xem “Schema flexibility in the document model”), và một schema có thể thay đổi theo thời gian (xem Chương 5).

secondary index

Một cấu trúc dữ liệu bổ sung được duy trì song song với bộ lưu trữ dữ liệu chính, cho phép bạn tìm kiếm hiệu quả các bản ghi khớp với một loại điều kiện nhất định. Xem “Multicolumn and Secondary Indexes” và “Sharding and Secondary Indexes”.

serializable

Một sự đảm bảo về *isolation* rằng nếu nhiều transaction thực thi đồng thời, chúng sẽ hoạt động giống như thể chúng đã được thực thi lần lượt theo một thứ tự tuần tự. Xem “Serializability”.

sharding

Chia nhỏ một dataset hoặc một phép tính lớn (vốn quá lớn đối với một máy đơn lẻ) thành các phần nhỏ hơn và phân tán chúng trên nhiều máy. Còn được gọi là *partitioning*. Xem Chương 7.

shared-nothing

Một kiến trúc trong đó các node độc lập—mỗi node có CPU, memory và disk riêng—được kết nối thông qua một mạng thông thường, trái ngược với các kiến

trúc shared-memory hoặc shared-disk. Xem “Shared-Memory, Shared-Disk, and Shared-Nothing Architectures”.

skew

1. Tải không cân bằng giữa các shard, khiến một số shard có rất nhiều yêu cầu hoặc dữ liệu trong khi các shard khác có ít hơn nhiều. Còn được gọi là *hot spots*. Xem “Skewed Workloads and Relieving Hot Spots”.
2. Một sự bất thường về thời gian khiến các sự kiện xuất hiện theo một thứ tự không mong muốn và không tuần tự. Xem các thảo luận về *read skew* trong “Snapshot Isolation and Repeatable Read”, *write skew* trong “Write Skew and Phantoms”, và *clock skew* trong “Timestamps for ordering events”.

split brain

Một kịch bản trong đó hai node đồng thời tin rằng chúng là leader, điều này có thể khiến các đảm bảo của hệ thống bị vi phạm. Xem “Handling Node Outages” và “The Majority Rules”.

stored procedure

Một cách để mã hóa logic của một transaction sao cho nó có thể được thực thi hoàn toàn trên một database server mà không cần giao tiếp qua lại với client trong suốt quá trình transaction. Xem “Actual Serial Execution”.

stream process

Một phép tính chạy liên tục, tiêu thụ một stream các sự kiện không bao giờ kết thúc làm đầu vào và tạo ra đầu ra từ đó. Xem Chương 12.

synchronous

Trái ngược với *asynchronous*.

system of record

Một hệ thống lưu giữ phiên bản dữ liệu chính thống và có thẩm quyền, còn được gọi là *source of truth* (nguồn sự thật). Các thay đổi trước tiên sẽ được ghi vào đây, và các dataset khác có thể được phái sinh từ system of record. Xem “Systems of Record and Derived Data”.

timeout

Một trong những cách đơn giản nhất để phát hiện một fault—cụ thể là bằng cách quan sát việc thiếu phản hồi trong một khoảng thời gian nhất định. Tuy nhiên, không thể biết được liệu một timeout có nguyên nhân từ vấn đề với remote node hay do sự cố trong network. Xem “Timeouts and Unbounded Delays”.

total order

Một cách để so sánh hai thứ (ví dụ: timestamps) cho phép bạn luôn có thể xác định cái nào lớn hơn và cái nào nhỏ hơn. Một thứ tự mà trong đó một số thứ không thể so sánh được (bạn không thể nói cái nào lớn hơn hoặc nhỏ hơn) được gọi là *partial order*.

transaction

Gom nhóm nhiều thao tác read và write lại thành một đơn vị logic nhằm đơn giản hóa việc xử lý lỗi và các vấn đề về concurrency. Xem Chương 8.

two-phase commit (2PC)

Một thuật toán để đảm bảo rằng nhiều database node hoặc là tất cả cùng commit một cách *atomically* (nguyên tử) hoặc tất cả cùng hủy bỏ một transaction. Xem “Two-Phase Commit”.

two-phase locking (2PL)

Một thuật toán để đạt được *serializable isolation* (cơ lập tuần tự hóa) hoạt động bằng cách một transaction giành được một lock trên tất cả dữ liệu mà nó read hoặc write và giữ lock đó cho đến khi kết thúc transaction. Xem “Two-Phase Locking”.

unbounded

Không có bất kỳ giới hạn trên hoặc kích thước xác định nào. Trái ngược với *bounded*.